



ПРОЕКТИРОВАНИЕ ЦИФРОВЫХ УСТРОЙСТВ



А. Ж. Уэйкерли

I

Джон Ф. Уэйкерли

Проектирование цифровых устройств

том I

Перевод с английского Е.В. Воронова, А.Л. Ларина

ПОСТМАРКЕТ
МОСКВА
2002

Оглавление

Содержание книги	14
Программные средства Xilinx Foundation	16
WWW.DDPP.COM	17
Для преподавателей	17
О том, как готовилась эта книга	18
Ошибки	18
Благодарности	18
Глава 1	
Введение	21
1.1. О цифровом проектировании	21
1.2. Соотношение между аналоговым и цифровым	23
1.3. Цифровые устройства	27
1.4. Электронные аспекты цифрового проектирования	28
1.5. Роль программирования в проектировании цифровых устройств	30
1.6. Интегральные схемы	32
1.7. Программируемые логические устройства	37
1.8. Специализированные интегральные схемы	38
1.9. Печатные платы	40
1.10. Уровни проектирования цифровых устройств	41
1.11. Самое главное	46
1.12. Напутствие	47
Упражнения	48
Глава 2	
Числовые системы и коды	49
2.1. Позиционные системы счисления	49
2.2. Восьмеричные и шестнадцатеричные числа	51
2.3. Общие преобразования позиционных систем счисления	53
2.4. Сложение и вычитание десятичных чисел	56
2.5. Представление отрицательных чисел	59
2.5.1. Представление чисел в прямом коде со знаком	59
2.5.2. Системы представления чисел в форме дополнения	60
2.5.3. Дополнительный код	60
2.5.4. Представление двоичных чисел в двоичном дополнительном коде	62
2.5.5. Представление в форме поразрядного дополнения	63
2.5.6. Представление двоичных чисел в обратном коде	63

2.5.7. Представление чисел с избытком	64
2.6. Сложение и вычитание двоичных чисел в дополнительном коде	64
2.6.1. Правила сложения	64
2.6.2. Графическая интерпретация	65
2.6.3. Перенос	66
2.6.4. Правила вычитания	67
2.6.5. Дополнительный код и двоичные числа без знака	68
2.7. Сложение и вычитание двоичных чисел в обратном коде	69
2.8. Двоичное умножение	71
2.9. Двоичное деление	73
2.10. Двоичные коды десятичных чисел	74
2.11. Код Грея	77
2.12. Коды символов	79
2.13. Коды действий, условий и состояний	79
2.14. n-мерные кубы и расстояния	83
2.15. Коды, обнаруживающие и исправляющие ошибки	84
2.15.1. Коды, обнаруживающие ошибки	85
2.15.2. Коды, исправляющие ошибки и обнаруживающие многократные ошибки	87
2.15.3. Коды Хэмминга	88
2.15.4. Циклические коды	92
2.15.5. Двумерные коды	93
2.15.6. Коды с контрольной суммой	95
2.15.7. Коды «m из n»	96
2.16. Коды для последовательной передачи и хранения данных	96
2.16.1. Параллельное и последовательное представление данных	96
2.16.2. Сигнальные коды для последовательной передачи	97
Обзор литературы	101
Упражнения	102
Задачи	104

Глава 3

Цифровые схемы	107
3.1. Логические сигналы и вентили	108
3.2. Семейства логических схем	113
3.3. КМОП-логика	114
3.3.1. Логические уровни КМОП-схем	114
3.3.2. МОП-транзисторы	115
3.3.3. Базовая схема КМОП-инвертора	116
3.3.4. КМОП-схемы И-НЕ и ИЛИ-НЕ	119
3.3.5. Коэффициент объединения по входу	120

3.3.6. Неинвертирующие вентили	122
3.3.7. КМОП-схемы И-ИЛИ-НЕ и ИЛИ-И-НЕ	123
3.4. Электрические свойства КМОП-схем	125
3.4.1. Общий обзор	125
3.4.2. Справочные данные и спецификация	126
3.5. Электрические характеристики	
КМОП-схем в установившемся режиме	129
3.5.1. Логические уровни и помехоустойчивость	129
3.5.2. Поведение схем с активными нагрузками	131
3.5.3. Поведение схем с неидеальными входными сигналами	138
3.5.4. Коэффициент разветвления по выходу	140
3.5.5. Влияние нагрузки	141
3.5.6. Неиспользуемые входы	141
3.5.7. Броски тока и развязывающие конденсаторы	142
3.5.8. Как испортить КМОП-схему	143
3.6. Динамические свойства КМОП-схем	144
3.6.1. Длительность переходного процесса	145
3.6.2. Задержка распространения	151
3.6.3. Потребляемая мощность	153
3.7. Другие варианты входных и выходных цепей КМОП-схем	155
3.7.1. Логические ключи	155
3.7.2. Триггер Шмитта	156
3.7.3. Схемы с тремя состояниями	157
3.7.4. Схемы с открытым стоком	160
3.7.5. Подключение светодиодов	162
3.7.6. Шины с несколькими источниками сигналов	164
3.7.7. Монтажная логика	164
3.7.8. Резисторы, соединяющие выходы схем с шиной питания	165
3.8. Семейства схем КМОП-логики	169
3.8.1. Семейства схем НС и НСТ	169
3.8.2. Семейства схем ВНС и ВНСТ	170
3.8.3. Электрические характеристики схем семейств НС, НСТ, ВНС и ВНСТ	170
3.8.4. Схемы семейств FCT и FCT-T	176
3.8.5. Электрические характеристики схем семейства FCT-T	177
3.9. Логические схемы на биполярных транзисторах	179
3.9.1. Диоды	180
3.9.2. Диодная логика	183
3.9.3. Биполярные транзисторы	185
3.9.4. Транзисторный инвертор	188
3.9.5. Транзисторы Шоттки	189

3.10. Транзисторно-транзисторная логика	191
3.10.1. Базовый ТТЛ-вентиль И-НЕ	191
3.10.2. Логические уровни и задержка схем устойчивости	195
3.10.3. Коэффициент разветвления по выходу	196
3.10.4. Неиспользуемые входы	199
3.10.5. ТТЛ-схемы других типов	201
3.11. Семейства ТТЛ-схем	203
3.11.1. Первые семейства ТТЛ-схем	203
3.11.2. ТТЛ-схемы с транзисторами Шоттки	204
3.11.3. Характеристики ТТЛ-схем	204
3.11.4. Справочные данные для ТТЛ-схем	205
3.12. Сопряжение КМОП- и ТТЛ-схем	208
3.13. Схемы низковольтной КМОП-логики	
и их сопряжение с другими схемами	209
3.13.1. LVTTTL- и LVCMOS-логика с напряжением питания 3.3 В	210
3.13.2. Входы, допускающие напряжение 5 В	211
3.13.3. Выходы, допускающие напряжение 5 В	213
3.13.4. Сопряжение TTL-схем и схем с уровнями LVTTTL:	
сводка результатов	214
3.13.5. Логические схемы с напряжениями питания 2.5 В и 1.8 В	214
3.14. Эмиттерно-связанная логика	215
3.14.1. Базовая схема ЭСЛ	216
3.14.2. Семейства ЭСЛ-схем 10K/10H	219
3.14.3. Семейство ЭСЛ-схем 100K	222
3.14.4. ЭСЛ-схемы с положительным напряжением питания	222
Обзор литературы	223
Упражнения	225
Задачи	230

Глава 4

Принципы проектирования

комбинационных логических схем

4.1. Алгебра преключений	238
4.1.1. Аксиомы	239
4.1.2. Теоремы о функциях одной переменной	242
4.1.3. Теоремы о функциях двух и трех переменных	242
4.1.4. Теоремы о функциях n переменных	244
4.1.5. Двойственность	247
4.1.6. Стандартные представления логических функций	250
4.2. Анализ комбинационных схем	254
4.3. Синтез комбинационных схем	260
4.3.1. Описание и составление схем	260

4.3.2. Преобразование схем	262
4.3.3. Минимизация комбинационных схем	266
4.3.4. Карты Карио	267
4.3.5. Минимизация сумм произведений	269
4.3.6. Упрощение произведений сумм	277
4.3.7. «Безразличные» комбинации переменных	279
4.3.8. Минимизация схем со многими выходами	280
4.4. Программные методы минимизации	283
4.4.1. Представление термов-произведений	284
4.4.2. Нахождение простых импликант путем объединения термов-произведений	287
4.4.3. Нахождение минимального покрытия по таблице простых импликант	289
4.4.4. Другие методы минимизации	291
4.5. Паразитные импульсы на выходе логических схем	292
4.5.1. Статические источники опасности	293
4.5.2. Нахождение статических источников опасности по картам Карио	294
4.5.3. Динамические источники опасности	296
4.5.4. Проектирование схем без источников опасности	296
4.6. Язык описания схем ABEL	297
4.6.1. Структура программ на языке ABEL	298
4.6.2. Работа компилятора языка ABEL	301
4.6.3. Операторы WHEN и блоки равенств	303
4.6.4. Таблицы истинности	304
4.6.5. Диапазоны, наборы и отношения	307
4.6.6. Безразличные комбинации входных сигналов	309
4.6.7. Проверочные векторы	312
4.7. Язык описания схем VHDL	314
4.7.1. Ход выполнения проекта	315
4.7.2. Структура программы	319
4.7.3. Типы и константы	323
4.7.4. Функции и процедуры	329
4.7.5. Библиотеки и пакеты	333
4.7.6. Элементы структурного проектирования	336
4.7.7. Элементы потокового проектирования	341
4.7.8. Элементы иерархического проектирования	344
4.7.9. Отсчет времени и моделирование	351
4.7.10. Синтез	354
Обзор литературы	355
Упражнения	359
Задачи	361

Глава 5

Практическая разработка

схем комбинационной логики 369

5.1. Стандарты документации 370

5.1.1. Блок-схемы 372

5.1.2. Условные обозначения логических схем 374

5.1.3. Имена сигналов и активные уровни 375

5.1.4. Активные уровни на выводах схем 377

5.1.5. Метод проектирования «инверсия к инверсии» 379

5.1.6. Расположение элементов на схеме 383

5.1.7. Шины 386

5.1.8. Дополнительная информация о схемах 386

5.2. Временные соотношения в схеме 389

5.2.1. Временные диаграммы 390

5.2.2. Задержка распространения 392

5.2.3. Временные параметры 392

5.2.4. Временной анализ 396

5.2.5. Программные средства временного анализа 397

5.3. Комбинационные программируемые логические устройства 397

5.3.1. Программируемые логические матрицы 397

5.3.2. Программируемые матричные логические устройства 401

5.3.3. Универсальные матричные логические устройства 405

5.3.4. Схемы бинолярных ПЛУ 407

5.3.5. Схемы ПЛУ на основе КМОП-логики 408

5.3.6. Программирование и тестирование микросхем 411

5.4. Дешифраторы 413

5.4.1. Полные дешифраторы 414

5.4.2. Условные обозначения крупных логических элементов 416

5.4.3. Сдвоенный дешифратор 2x4 типа 74x139 417

5.4.4. Дешифратор 3x8 типа 74x138 420

5.4.5. Расширение полных дешифраторов 422

5.4.6. Описание дешифраторов на языке ABEL

и их реализация в ПЛУ 424

5.4.7. Описание дешифраторов на языке VHDL 431

5.4.8. Дешифраторы для сегментных индикаторов 436

5.5. Шифраторы 440

5.5.1. Приоритетные шифраторы 440

5.5.2. Приоритетный шифратор 74x148 442

5.5.3. Описание шифраторов на языке ABEL

и их реализация в ПЛУ 445

5.5.4. Описание шифраторов на языке VHDL 448

5.6. Устройства с тремя состояниями	449
5.6.1. Буферы с тремя состояниями	449
5.6.2. Стандартные буферы с тремя состояниями в виде ИС малой и средней степени интеграции	452
5.6.3. Описание схем с тремя состояниями на языке ABEL и их реализация в ПЛУ	456
5.6.4. Описание выходов с тремя состояниями на языке VHDL	460
5.7. Мультиплексоры	464
5.7.1. Стандартные мультиплексоры в интегральном исполнении	465
5.7.2. Расширение мультиплексоров	469
5.7.3. Мультиплексоры, демультиплексоры и шины	472
5.7.4. Описание мультиплексоров на языке ABEL и их реализация в ПЛУ	473
5.7.5. Описание мультиплексоров на языке VHDL	477
5.8. Логические элементы ИСКЛЮЧАЮЩЕЕ ИЛИ и проверка на четность	479
5.8.1. Вентили ИСКЛЮЧАЮЩЕЕ ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ	479
5.8.2. Схемы проверки на четность	481
5.8.3. 9-разрядная микросхема проверки на четность 74х280	481
5.8.4. Применение схем проверки на четность	483
5.8.5. Описание схем ИСКЛЮЧАЮЩЕЕ ИЛИ и схем проверки на четность на языке ABEL и их реализация в ПЛУ	485
5.8.6. Описание схем ИСКЛЮЧАЮЩЕЕ ИЛИ и схем проверки на четность на языке VHDL	486
5.9. Компараторы	488
5.9.1. Структура компаратора	489
5.9.2. Итерационные схемы	490
5.9.3. Итерационная схема компаратора	491
5.9.4. Стандартные компараторы в интегральном исполнении	492
5.9.5. Описание компараторов на языке ABEL и их реализация в ПЛУ	496
5.9.6. Описание компараторов на языке VHDL	497
5.10. Сумматоры, вычитающие устройства и АЛУ	500
5.10.1. Полусумматоры и полные сумматоры	501
5.10.2. Сумматоры со сквозным переносом	501
5.10.3. Вычитающие устройства	503
5.10.4. Сумматоры с укороченным переносом	504

5.10.5. Сумматоры, выполненные в виде ИС средней степени интеграции	506
5.10.6. Арифметическо-логические устройства, выполненные в виде ИС средней степени интеграции	509
5.10.7. Ускоренный групповой перенос	512
5.10.8. Описание сумматоров на языке ABEL и их реализация в ПЛУ	515
5.10.9. Описание сумматоров на языке VHDL	516
5.11. Комбинационные умножители	518
5.11.1. Структура комбинационных умножителей	518
5.11.2. Описание процедуры умножения на языке ABEL и ее реализация в ПЛУ	521
5.11.3. Описание процедуры умножения на языке VHDL	522
Обзор литературы	528
Упражнения	529
Задачи	533

ПРЕДИСЛОВИЕ

На протяжении последних трех десятилетий остается справедливым закон Мура (Moore's Law), согласно которому полупроводниковая техника развивается экспоненциально. Эксперты предсказывают, что это будет продолжаться, по крайней мере, в течение еще одного десятилетия. Когда только появились интегральные схемы, в одном корпусе было порядка дюжины транзисторов. Сегодня в результате экспоненциального роста плотности упаковки микропроцессоры преодолели отметку в 10 миллионов транзисторов на один кристалл. Менее чем через 10 лет это число достигнет 100 миллионов.

Чтобы соответствовать закону Мура, радикально изменились методы проектирования. Когда-то разводка логических схем вручную была нормой. Сегодня схема возникает в результате описания ее проектировщиком на языке высокого уровня. Соединения, выполнявшиеся ранее на плате с помощью печатного монтажа, теперь оказались перенесенными внутрь кристалла. Применение принципов программируемой логики позволяет модифицировать логические функции, реализуемые данным кристаллом, и соединения в нем, не вынимая его из схемы, в которой он используется.

Как должна реагировать система образования на требования закона Мура? Что нужно сделать, чтобы студенты могли воспользоваться приобретенными навыками сегодня и имели возможность применить их завтра к устройствам будущих поколений? Именно с этой проблемой столкнулся Джон Уэйкерли, когда приступал к работе.

Его подход многогранен. Он основан на исходных принципах цифровой электроники, не меняющихся с развитием технологии, состоящих в рассмотрении комбинационных и последовательностных логических схем и конечных автоматов. Уэйкерли совмещает эти принципы со средствами и практическими приемами проектирования современных устройств. Его подход включает применение таких языков проектирования, как ABEL и VHDL, представление проекта в виде схемы, состоящей из больших структурных блоков, и реализацию проекта средствами программируемой логики. Успех проектирования в значительной степени определяется применением этих методов.

Самая трудная задача состоит в том, чтобы помочь учащимся адаптироваться к неизбежным предстоящим изменениям. Уэйкерли решает эту задачу, раскрывая то, что происходит на уровнях, лежащих ниже уровня логики. Так, например, он приводит транзисторные схемы вентиля и применяет их при рассмотрении вопросов, относящихся к временным задержкам и шумам. Вентили могут становиться более быстрыми и компактными, работать с другими управляющими напряжениями, но как гарантировать их правильное и надежное функционирование — это как раз то, о чем пойдет речь. Изучая характеристики, ограничения и условия, приводящие к сбою, мы оказываемся способными учесть их на стадии проектирования. Рассматривая в качестве примера различные варианты решения тех или иных задач, мы узнаем, чего стоят компромиссы, и как можно судить о качестве проекта. Благодаря этому наши навыки проектирования останутся пригодными и с появлением новых технологий.

Подход Уэйкери замечателен также формой подачи материала, которая редко встречается в университетских учебниках. Читатель быстро оценит эффективность графического представления, забавный стиль изложения и поучительный характер упражнений.

Закон Мура обрекает учебники на короткую жизнь. А вот учебник Уэйкери является классическим.

Гарольд С. Стоун (Harold S. Stone)
Принстон, Нью Джерси

ВСТУПЛЕНИЕ

Эта книга предназначена для тех, кто хочет проектировать и создавать реальные цифровые устройства. В ее основе лежит следующая основная мысль: чтобы достичь желаемой цели, необходимо овладеть принципами, но в то же время нужно иметь представление о том, как эти принципы реализуются на практике. Таким образом, «принципы и практика» являются предметом нашего рассмотрения.

Материал этой книги пригоден в качестве учебника по *вводному курсу (introductory course)* цифровой электроники для студентов, специализирующихся в области электроники, вычислительной техники и информатики (computer science). Те, кто не знаком с *основными понятиями электроники (electronics concepts)* и не интересуется поведением цифровых устройств с точки зрения протекающих в них электрических процессов (например, студенты, специализирующиеся в области информатики), могут при желании пропустить главу 3; остальной материал изложен в книге независимо от содержания этой главы в той мере, в какой это было возможно. С другой стороны, тот, кто владеет основами электроники и хочет быстро освоить цифровую технику, может сделать это, прочтя главу 3. Студенты, у которых нет начальных сведений по электронике, могут ознакомиться с ними по бесплатно распространяемому 20-страничному учебному пособию Флейшера *Электрические цепи в кратком изложении* (M. Fleisher. *Electrical Circuits Review*), имеющемуся на Web-сайте данной книги по адресу: www.ddpp.com.

Хотя уровень изложения в этой книге ориентирован на начальное изучение, содержащийся в ней материал выходит далеко за рамки того, что может быть включено в типичный вводный курс. Когда я приступил к этой работе, оказалось, что есть много важных вещей, о которых необходимо сказать и которые не укладываются в односеместровый курс в Станфордском университете или в учебное пособие объемом в 400 страниц. Поэтому я последовал своему обычному правилу включения *всего*, что – по крайней мере, на мой взгляд – является сравнительно важным, оставляя за преподавателем или учащимся право решать, что именно является самым важным, в зависимости от обстоятельств. Все же, чтобы облегчить такое решение, заголовки *необязательных разделов (optional sections)* помечены звездочкой. В общем случае эти разделы можно опустить без нарушения целостности изложения в пределах основного текста в дальнейшем.

Несомненно, кто-то воспользуется этой книгой в рамках *продвинутого курса (advanced course)* или в *лабораторном практикуме (laboratory course)*. Подготовленные студенты захотят пропустить начало и сразу поискать самое интересное. Если вы знакомы с основными идеями цифровой электроники, то для вас самыми важными и интересными в этой книге будут разделы, посвященные языкам описания схем (или: языкам описания аппаратуры; hardware description languages, HDLs) ABEL и VHDL, знакомясь с которыми вы обнаружите, что пройденные вами ранее курсы программирования в большой степени подготовили вас к проектированию цифровой аппаратуры.

Специалист, уже работающий в области проектирования цифровых устройств (working digital designer), может воспользоваться этой книгой в качестве

справочника для самообразования, причем сами такие специалисты бывают двух категорий: *новички и профессионалы старой закалки*.

Если вы только что приступили к практической работе по разработке и созданию цифровых устройств и прошли в университете «очень теоретический» курс цифрового проектирования, то вам следует сосредоточить внимание на главах 3, 5, 6 и 8–11, чтобы подготовиться к встрече с реальным миром.

Если у вас уже есть определенный опыт, вам, возможно, не нужны все «упражнения на применение», содержащиеся в этой книге, но принципы, изложенные в главах 2, 4 и 7, могут направить ваши мысли в нужном направлении, а содержащиеся там рассуждения о том, что важно, а что — нет, быть может, избавят от чувства вины за то, что вы не пользовались картами Карно на протяжении 10 лет. Примеры в главах 6, 8 и 9 дадут вам новое представление о многообразии методов проектирования и возможность судить об их достоинствах и недостатках. Наконец, описание на языках ABEL и VHDL и примеры, которыми усеяны главы с 4 по 9, могут послужить первым систематическим введением в технику проектирования на основе языков описания схем.

Всем читателям стоит обратить особое внимание на исчерпывающий предметный указатель и на элементы текста, выделенные курсивом; с помощью этого последнего приема я хотел бы привлечь внимание читателя к определениям и к тем случаям, когда предмет обсуждения является важным.

Содержание книги

Ниже кратко изложено содержание одиннадцати глав этой книги. Это может напомнить вам тот раздел руководства по программному обеспечению, который обычно носит название: «Для тех, кто терпеть не может читать инструкции и наставления». Возможно, что после того, как вы ознакомитесь с этим перечнем, вам не нужно будет читать все остальное в этой книге.

- В главе 1 приводятся несколько исходных определений и формулируются основные правила, согласно которым в этой книге принимаются решения о том, что считается важным, а что — нет.
- Глава 2 является введением в двоичную систему чисел и двоичные коды. Читателю, уже знакомому с двоичной системой чисел по курсам программирования, все же стоит прочесть параграфы 2.10–2.13, чтобы получить представление о том, какую роль двоичные коды играют в аппаратуре. Для подготовленных студентов хорошим введением в коды, исправляющие ошибки, будут параграфы 2.14 и 2.15. Всем следует прочесть раздел 2.16.1; этот материал используется в примерах проектирования цифровых устройств в главе 8.
- В главе 3 — по принципу «Вот все, что вы хотели бы знать об этом» — рассматривается работа цифровых схем, причем главный упор делается на внешние электрические характеристики логических элементов. Исходными являются такие основные понятия электроники, как напряжение, ток и закон Ома; те, кому этот материал не знаком, могут при желании обратиться к упомянутому выше учебному пособию *Электрические цепи в кратком изложении*. Если вам не интересно, как имеют функционируют реальные схемы, или вы мо-

жете позволить себе рюкзак поручить грязную работу кому-то другому, то эту главу можно пропустить.

- Глава 4 посвящена принципам проектирования комбинационных логических схем и содержит алгебру переключений, а также анализ, синтез и минимизацию комбинационных схем. В конце этой главы появляются начальные сведения о языках ABEL и VHDL.
- Глава 5 начинается с обсуждения стандартов для документации на цифровые системы, что, возможно, является самым важным для честолюбивых проектировщиков, когда они только приступают к практической работе. Затем в этой главе читатель знакомится с программируемыми логическими устройствами (ПЛУ; programmable logic devices, PLDs), причем здесь его внимание оказывается сосредоточенным на возможности реализации с помощью таких устройств комбинационных логических функций. Остальная часть этой главы посвящена часто используемым комбинационным логическим функциям и их применениям. Для каждой из этих функций описаны стандартные интегральные схемы (ИС) средней степени интеграции, программы на языке ABEL, позволяющие реализовать эти функции с помощью ПЛУ, и модели на языке VHDL.
- Глава 6 представляет собой коллекцию примеров конструирования более сложных комбинационных устройств. В каждом из примеров показано, как можно реализовать данное устройство на основе ИС средней степени интеграции (в тех случаях, когда это имеет смысл), описать его на языке ABEL, имея в виду реализацию с помощью ПЛУ, или на языке VHDL, когда наметено применение кристаллов CPLD или FPGA.
- Глава 7 знакомит с принципами проектирования последовательностных логических схем, начиная с защелок и триггеров. В этой главе упор сделан на анализ и расчет тактируемых синхронных конечных автоматов. Для смелых и отважных эта глава содержит введение в теорию схем классического образца (fundamental mode circuits), а также анализ и расчет последовательностных схем с обратной связью. Заканчивается глава параграфами, в которых речь идет о возможностях, предоставляемых языками ABEL и VHDL при проектировании последовательностных схем.
- Глава 8 целиком посвящена практическим аспектам проектирования последовательностных схем. Как и ранее в главе 5, внимание читателя в этой главе сосредоточено на часто используемых функциях и приведены примеры применения ИС средней степени интеграции, языка ABEL при реализации в ПЛУ и языка VHDL. В параграфах 8.8 и 8.9 обсуждаются неизбежные препятствия на пути создания идеального, абсолютно синхронного устройства, и здесь читатель лицом к лицу сталкивается с важной проблемой: как жить синхронно в асинхронном мире?
- Глава 9 является собранием примеров проектирования конечных автоматов и более сложных последовательностных схем. В каждом примере показано, как решить задачу, составляя программу на языке ABEL при реализации в ПЛУ или на языке VHDL, когда имеется в виду использование кристаллов CPLD или FPGA.
- Глава 10 служит введением в запоминающие устройства и программируемые матричные интегральные схемы типа CPLD и FPGA. Материал, посвященный

устройствам электронной памяти, охватывает ПЗУ и статические и динамические ОЗУ, включая их внутреннюю структуру и функциональное описание. Последние два параграфа знакомят с архитектурой кристаллов CPLD и FPGA.

- В главе 11 рассмотрено несколько вопросов смешанного характера, относящихся к практической реализации цифровых проектов и представляющих интерес для разработчиков. Когда я начинал писать эту книгу и думал, что ее объем составит 300 страниц, эта глава была включена в план для того, чтобы «раздуть» основной материал до более впечатляющих размеров. Теперь очевидно, что книга и без того получилась довольно объемной, но то, что вошло в эту главу, все равно полезно.

В большинстве глав имеются ссылки на литературу, выражения и задачи. Выражения, как правило, предусматривают быстрый ответ, который можно непосредственно найти в изложенном материале; часто вопрос просто бывает задан другими словами. Задачи могут потребовать чуть большего усилия мысли. Особенно много упражнений в главе 3, и их назначение состоит в том, чтобы облегчить учащемуся, не специализирующемуся в области электроники, постепенно освоить этот материал.

Программные средства Xilinx Foundation

Фирма Xilinx, Inc. (Сан-Хосе, шт. Калифорния; San Jose, CA 95124) любезно разрешила нам поместить в конце этой книги два компакт-диска, содержащих программный продукт этой фирмы Foundation Express, предназначенный для проектирования цифровых устройств. Это всеобъемлющий программный продукт, включающий компилятор языка ABEL, трансляторы языков VHDL и Verilog, графический редактор схем и средства моделирования. В значительной степени это программное обеспечение базируется на популярных пакетах Active-CAD™ и Active-HDL™ фирмы Aldec, Inc. Предлагаемый к книге пакет программ включает также программный продукт FPGA Express™ фирмы Synopsys, позволяющий реализовать проекты цифровых устройств на кристаллах CPLD и FPGA с использованием языков ABEL, VHDL и Verilog; помещенная здесь версия поддерживает кристаллы, производимые фирмой Xilinx.

Программные средства Foundation были очень полезны мне как автору. Используя их, я смог написать и протестировать все примеры программ в этом учебнике. Я верю, что эти средства окажутся еще более полезными студентам, которые станут учиться по этой книге. Эти средства позволят учащимся составлять и испытывать собственные проекты и в лабораторных условиях загружать их в кристаллы семейств CPLD и FPGA фирмы Xilinx. Учебный пакет позволяет создавать проекты, содержащие до 20000 вентилей и размещаемые в одном кристалле. За небольшую плату вы можете получить доступ в режиме online к исчерпывающему набору учебников фирмы Xilinx и к двум ее отличным лабораторным руководствам по адресу: www.prenhall.com/xilinx. Эти и другие ресурсы на данном сайте помогут вам воспользоваться программными средствами Foundation.

Даже в том случае, если вы не готовы создавать собственные оригинальные проекты, вы можете воспользоваться программными средствами Foundation, чтобы проверить или видоизменить любой из примеров в данном учебнике, поскольку

ку исходный текст всех программ имеется на Web-сайте этой книги, о чем сказано ниже.

WWW.DDPP.COM

На специальном Web-сайте этой книги www.ddpp.com размещено много вспомогательного материала. Учащихся прежде всего будут интересовать листинги программ всех примеров, приведенных в этой книге, написанных на языках C, ABEL и VHDL. Кроме того, имеются заархивированные (архиватором ZIP) директории проектов, созданных в программной среде Foundation, включающие не только исходные тексты на языках ABEL и VHDL, но также схемы, созданные средствами графического редактора схем, которыми можно воспользоваться для моделирования в ряде примеров.

Готовя к печати это издание, я удивился и порадовался тому, как много в Интернете справочного материала, относящегося к проектированию цифровых устройств; особенно много сведений сообщают производители интегральных микросхем. На Web-сайте DDPP имеется раздел «живых» ссылок, включающий указатели на многие полезные сайты, которые вы можете использовать в качестве отправной точки в собственном независимом приобретении знаний.

На Web-сайт перенесены два приложения из предыдущих изданий — *Электрические цепи в кратком изложении* Брюса М. Флейшера и *Стандартные обозначения IEEE (IEEE Standard Symbols)*. При выполнении студентами лабораторного практикума для них может оказаться полезным компактный четырехстраничный справочник по цоколевке интегральных микросхем, который в предыдущих изданиях размещался с внутренней стороны обложки.

На Web-сайте DDPP имеется еще одна вещь, которая может понравиться или не понравиться студентам, — это новые задачи, постепенно собираемые мною в процессе преподавания цифровой электроники в Станфордском университете при содействии всех заинтересованных.

Для преподавателей

На Web-сайте DDPP имеются дополнительные материалы, предназначенные только для преподавателей. Эта часть сайта защищена паролем. Если вы собираетесь ею воспользоваться, то, пожалуйста, учтите, что вам понадобится неделя на получение регистрационного имени и пароля путем выполнения указанной на сайте процедуры.

В предназначенной для преподавателей части сайта содержатся файлы со всеми рисунками и таблицами этой книги. Вы можете непосредственно воспользоваться этими файлами для изготовления демонстрационных слайдов или включить отобранный материал в ваши собственные иллюстрации.

Более половины задач этой книги снабжены ответами, помещенными на сайте; объем этого материала эквивалентен 200 печатным страницам. Имеются также примеры экзаменационных заданий с решениями.

Другим важным источником сведений, полезных для преподавателей, является университетская программа фирмы Xilinx (www.xilinx.com/programs/univ). Этот сайт содержит обширную информацию об изделиях фирмы Xilinx,

предлагаемых ею учебных средствах и о скидках на кристаллы и инструментальные модули, которые вы могли бы использовать в своих лабораторных курсах по цифровой электронике.

О том, как готовилась эта книга

Для третьего издания текст был преобразован издательской системой Adobe FrameMaker® из исходной версии второго издания, реализованной на TEXe. Рисунки в предыдущих изданиях были выполнены с помощью Cricket Draw и для данного издания преобразованы в EPS-файлы пакета Adobe Illustrator®.

Сам процесс написания, редактирования, рисования и расчета схем осуществлялся на персональном компьютере с операционной системой Windows 95/98 и оперативной памятью 384 Мбайта, который, к сожалению, иногда все же зависал, когда одновременно было открыто слишком много файлов или запущено слишком много программ. Использование стандартных программ и средств при подготовке этого издания позволило мне обеспечить читателей и преподавателей большим по объему полезным материалом, собранным на Web-сайте этой книги, о чем сказано выше.

Ошибки

Предупреждение. В этой книге могут быть ошибки, и автор не несет никакой ответственности, если эти ошибки вдруг нанесут ущерб вашему карману, введут вас в заблуждение и т.п. Так что юристы и адвокаты могут не беспокоиться.

Но позвольте вас заверить, что при подготовке этой рукописи я самым тщательным образом постарался сделать ее настолько свободной от ошибок, насколько это было возможно, чтобы вы не пострадали. Мне не терпится узнать об остающихся ошибках, которые могут обнаружиться в будущих изданиях и публикациях или в чем-то другом, что будет результатом использования сведений, почерпнутых из этой книги. Поэтому я заплачу 5 долларов за каждую ранее не обнаруженную ошибку тому, кто найдет ее первым, даже в том случае, если это будет техническая или типографская ошибка или какая-либо еще. Пожалуйста, присылайте ваши замечания по электронной почте, используя соответствующий указатель на Web-сайте, или обращайтесь непосредственно ко мне по адресу: john@wakerly.com.

Любой читатель может получить последний по времени обновления список обнаруженных ошибок по ссылке, имеющейся на Web-сайте этой книги. Я надеюсь, что это будет совсем короткий файл.

Благодарности

Появление этой книги стало возможным благодаря помощи многих людей. Большинство из них помогли мне при подготовке первого и второго изданий, и там я поблагодарил их. Подготовка третьего издания оказалась задачей, которую мне пришлось решать в большей степени самостоятельно, но помощь моих коллег Марио Маццола (Mario Mazzola) и Према Джейна (Prem Jain) из компании Cisco Systems облегчила решение этой задачи. Они и фирма в целом позволили мне

сократить мои трудовые обязательства по отношению к Cisco более чем наполовину на протяжении тех восьми месяцев, которые ушли на подготовку этого издания.

За идеи, нашедшие отражение в той части книги, где речь идет о «принципах», я в долгу перед моим учителем, научным руководителем и другом Эдом МакКласки (Ed McCluskey). За часть книги, относящуюся к «применениям», я благодарен тем, кого поместил в мой собственный «Зал Славы Цифрового Проектирования», а именно (в хронологическом порядке): Эду Дэвидсону (Ed Davidson), Джиму МакКлюру (Jim McClure), Куртенею Хитеру (Courtenay Heater), Сэму Вуду (Sam Wood), Курту Уиддоусу (Curt Widdoes), Прему Джейну, Теду Трейси (Ted Traey), Дейву Раауму (Dave Raam), Акилу Дуггалу (Akhil Duggal), Дезу Янгу (Des Young) и Тому Эдсалу (Tom Edsall).

Мысль начать писать эту и многие другие книги заронил в мою душу в начале 70-х годов Гарольд Стоун (Harold Stone) в Станфордском университете. Он предложил мне просматривать его книги и составлять к ним указатели, и его книги по устройству компьютеров вдохновили меня написать мою первую книгу по программному обеспечению. Теперь я хотел бы выразить Гарольду свою запоздалую благодарность за этот начальный толчок, и отдельно сказать ему спасибо за подсказки, которыми я воспользовался при подготовке этого издания!

Летом 1997 года, на ранней стадии размышлений об этом издании, мой друг и коллега Жан-Пьер Стеже (Jean-Pierre Steger) из Бургдорфской технической школы (Burgdorf School of Engineering) близ Берна в Швейцарии взял годичный отпуск, чтобы помочь мне в начальном освоении языка VHDL и программного пакета Foundation фирмы Xilinx, а также в других вопросах. Появлению этого издания на свет способствовали и другие люди, сделавшие ряд критических замечаний и внесшие свои предложения, в том числе Джон Беркнер (John Birkner), Ребекка Фарлей (Rebecca Farley), Дон Гаубац (Don Gaubatz), Джон Гилл (John Gill), Линли Гуэннэп (Linley Gwennep), Джессн Дженкинс (Jesse Jenkins) и Джефф Пернелл (Jeff Purnell).

Чести быть упомянутой здесь, естественно, заслуживает фирма Xilinx, Inc., чей программный пакет Foundation является важным дополнением к этому изданию. Что касается конкретных лиц, то очень полезной была помощь первого руководителя их университетской программы Джейсона Фейнсмита (Jason Feinsmith); с энтузиазмом поддержал наши усилия недавно назначенный руководитель этой программы Патрик Кейн (Patrick Kane).

Со времени выхода в свет второго издания я получил много полезных замечаний от читателей. Помимо предложений об улучшениях и замечаний другого характера, читатели заметили дюжину типографских и технических ошибок, которые были учтены в этом третьем издании.

Большой благодарности заслуживает шефствовавший надо мной редактор издательства Prentice Hall Том Роббинс (Tom Robbins) за проявленное терпение. Он был вторым редактором, кто внес свою лепту в то, что издательство Prentice Hall соблазнилось (сомнительной) перспективой совместной работы со мной; правда, когда он появился, работа над проектом была близка к завершению. Но я знаком с Томом с начала 80-х годов, когда он впервые попытался свести меня с другим издателем, и с тех пор мы стараемся находить возможность

работать вместе. Ныче нашей дружбе пошел уже третий десяток лет. Вклад Тома больше, чем терпение: средн прочих вещей, за которые вы должны сказать ему спасибо, — бесплатное программное обеспечение фирмы Xilinx, приложенное к этой книге.

Спасибо также руководителю производственного отдела издательства Ирвину Зукеру (Irwine Zucker), в максимальной степени сгладившему шероховатости в моих взаимоотношениях с полиграфическим производством и потратившему массу времени, помогая мне на решающей стадии завершения проекта. Если бы не он, я не мог бы отправиться в давно намеченное трехнедельное путешествие с семьей. (Я сказал, что если не закончу работу в срок, то вместо меня по моему билету в Европу полетит наш 90-фунтовый пес!).

Особые слова благодарности — художнику Роберту Мак-Фаддену (Robert McFadden), чей набросок обложки книги висит у меня дома на стене вместе с несколькими другими его замечательными работами. Его рисунок, который я заказал ему и который он выполнил год назад, подталкивал меня, как это ни странно, в работе над тем, что должно было оказаться *внутри* этой книги.

Похоже, что всякий раз, когда я заканчиваю работу над очередным проектом, случается какое-нибудь бедствие. Когда выходило первое издание, произошло землетрясение 1989 года. За четыре дня до завершения работы над вторым изданием мне вырезали аппендикс. На этот раз ничего плохого не происходило вплоть до выхода первого тиража, когда я узнал о кончине Роберта Мак-Фаддена. Он умер совсем молодым, но живы его картины и они по-прежнему вдохновляют воображение многих из нас.

Как всегда, я должен поблагодарить мою жену Кейт (Kate), смилившуюся с тем, что приходится поздно ложиться спать и терпеть мою подавленность, раздражительность, поглощенность работой и телефонные звонки каких-то таинственных людей в те периоды времени, когда я бываю занят очередным сочинением наподобие этого. Мы с Кейт надеемся, что вы приступаете к чтению этой книги с не меньшим удовольствием, чем то, с каким мы заканчиваем ее.

Джон Ф. Уэйкерт
Маунтин-Вью, Калифорния



глава

ВВЕДЕНИЕ

Добро пожаловать в мир цифровой электроники. Возможно, вы являетесь студентом, специализирующимся в области информатики, знаете все о программировании и о программном обеспечении компьютеров и решили разобраться в том, как может функционировать все это фантастическое аппаратное обеспечение. Но возможно, что вы студент, специализирующийся в области электротехники, который уже знает кое-что об аналоговых схемах и хотел бы узнать побольше о том, что его интересует. Неважно! Начиная с довольно примитивного уровня, мы покажем в этой книге, как проектировать цифровые схемы и устройства.

Мы познакомим вас с основными идеями, необходимыми для того, чтобы разобраться в этих вещах, и приведем массу примеров. Вместе с принципами мы постараемся передать дух реального проектирования, рассматривая — там, где это оказывается возможным, — практическое воплощение современными средствами. И я, автор, в дальнейшем часто говорю «мы» в надежде, что вы втянетесь и почувствуете, что ваше обучение происходит в результате действий, совершаемых нами совместно.

1.1. О цифровом проектировании

Некоторые называют его «логическим проектированием». Пожалуйста! Так или иначе, конечная цель любого проекта — создание системы. Чтобы достичь желаемого, вам предстоит с помощью этого учебника овладеть большим, нежели простое логическое уравнение и теоремы.

Как следует из названия этой книги, речь пойдет о принципах и применениях. Большинство из рассматриваемых нами принципов останутся важными на протяжении еще многих лет, хотя возможно, что способы применения некоторых из них могут оказаться такими, какие сегодня еще не известны. Что касается практической реализации, то ко времени начала вашей работы в этой области она может чуть отличаться от того, как она представлена здесь, и наверняка будет продолжать изменяться на протяжении вашей трудовой деятельности. Поэтому материал этой книги, относящийся к «применениям», следует воспринимать как возможность лучше усвоить «принципы» и как способ научиться методам проектирования на примерах.

Одна из целей этой книги состоит в таком представлении основных принципов, которого было бы достаточно для понимания вами, что происходит, когда вы применяете программные средства для выполнения простейших операций. Те же самые основные принципы помогут вам усвоить существо проблемы в случае, если вы столкнетесь с необходимостью усмирить эти программные средства.

В тексте, заключенном в рамку и озаглавленном «Важные соображения...», перечислены несколько ключевых моментов, которые вам предстоит усвоить в процессе обучения. Большинство из них, возможно, покажутся вам сейчас лишними смысла, но позднее вы вернетесь назад и взгляните на них снова.

Проектирование цифровых устройств — это инженерное искусство, и как таковое оно означает «решение проблем». Мой опыт показывает, что только на 5–10 процентов проектирование цифровых устройств является «интересным занятием», когда работа носит творческий характер, вас посещает вспышка прозрения или удастся придумать новый подход. Все остальное, по большей части, — это механическая работа. Уверяю вас, что сегодня выполнять эту рутинную работу много легче, чем 20 или даже 10 лет назад, но вы пока еще не можете тратить 100% или даже 50% времени на интересные занятия.

ВАЖНЫЕ СООБРАЖЕНИЯ ПРИ ЦИФРОВОМ ПРОЕКТИРОВАНИИ

- Хорошие средства не гарантируют хорошего результата, но они здорово помогают, избавляя вас от головной боли при осуществлении необходимых действий.
- Цифровые схемы обладают аналоговыми характеристиками.
- Нужно знать, когда следует волноваться по поводу аналоговых аспектов при проектировании цифрового устройства, а когда не надо.
- Всегда сопровождайте свои проекты надлежащей документацией, чтобы сделать их понятными и вам и другим.
- Применяйте имена сигналов, содержащие указание на их активные уровни, и используйте принцип «инверсия к инверсии» (bubble-to-bubble).
- Разберитесь со стандартными составными блоками и используйте их.
- Проектируйте систему в целом так, чтобы минимизировать ее стоимость, включая ваши собственные инженерные усилия как часть затрат.
- Расчет конечных автоматов подобен программированию; подходите к этому именно с такой точки зрения.
- Применяйте программируемую логику для упрощения конструкции, уменьшения стоимости и обеспечения возможности впоследствии изменить ее в последнюю минуту.
- Избегайте проектирования асинхронных схем. Применяйте синхронные методы проектирования до тех пор, пока не появятся лучшие.
- Возможно точнее учитывайте неизбежно асинхронный характер взаимодействия между различными подсистемами и их взаимодействия с внешним миром, а также предусматривайте наличие надежных синхронизирующих устройств.
- Нужно вовремя отлавливать короткие паразитные импульсы (глюки).

Помимо творческой работы и работы, выполняемой механически, существует много других видов деятельности, в которых специалист по цифровой электронике должен быть компетентным, чтобы работать успешно, в том числе он должен владеть следующими:

- *Отладка.* Практически невозможно быть хорошим разработчиком, не владея техникой выявления неисправностей. Успешная отладка предполагает планирование, систематический подход, терпение и логичность действий: если вы не в состоянии разобраться со случаем, когда проблема *есть*, то вам не догадаться, когда проблемы *нет*!
- *Требования бизнеса и практическая реализация.* На работу специалиста, разрабатывающего цифровые устройства, влияет множество факторов инженерного характера, включая стандарты на документацию, доступность требуемых компонентов, определение характеристик, технические задания, календарные планы, политика фирмы и хождение на ручки к поставщикам.
- *Готовность пойти на риск.* Приступая к созданию того или иного устройства, вы должны тщательно взвесить, чем вы рискуете в отношении того, что может быть достигнуто, и в отношении того, что может быть следствием; причем диапазон рассмотрения должен простирается от выбора новых компонентов (окажутся ли они доступными в момент, когда я буду готов собрать первый образец?) до обязательств по срокам (не потеряю ли я работу, если опоздаю?).
- *Контакты.* В конечном счете, вам предстоит передать успешно завершённый проект другим инженерам, в другое подразделение или покупателям. Без хороших навыков в установлении контактов, вам никогда не осуществить этот шаг успешно. Имейте в виду, что при взаимодействии вы не только отдаете, но и получаете, так что учитесь быть хорошим слушателем!

Дальше в этой главе и повсюду в тексте я буду продолжать высказывать различные утверждения о том, что важно и что не важно. Я думаю, что у меня есть право на это, как у специалиста со сравнительно большим опытом успешной и практической деятельности в области цифрового проектирования. Но я также приглашаю всех поделиться со мной вашим собственным мнением и опытом (пишите мне по электронной почте по адресу john@wakerly.com).

1.2. Соотношение между аналоговым и цифровым

В аналоговых (*analog*) устройствах и системах происходит преобразование таких меняющихся во времени сигналов, которые могут принимать любые значения из непрерывного интервала величин; эти величины могут быть напряжением или током или иметь другую размерность. То же самое происходит в цифровых (*digital*) схемах и системах; отличие состоит только в том, что мы можем притворяться, будто это не так. Цифровой сигнал – это модель, согласно которой в любой момент времени сигнал может принимать только одно из двух дискретных значений, которые мы называем «нулем» (0) и «единицей» (1) (или «низким» и «высоким» уровнями, «ложью» и «истиной», отрицанием и утверждением, Сэмом и Фредом или как-то еще).

Цифровые компьютеры появились в 40-х годах и начали широко применяться на практике в 60-х. Но только в последние 10 или 20 лет «цифровая революция» распространилась на многие другие стороны жизни. Можно привести следующие примеры систем, которые раньше были исключительно аналоговыми и теперь «переходят» в разряд цифровых:

- *Фотография.* До сих пор в большинстве фотоаппаратов для регистрации изображения используются галопные соединения серебра. Однако увеличение объема цифровой памяти в одном кристалле привело к появлению цифровых камер, в которых изображение фиксируется в виде массива точек (пикселей) размером 640х480 или больше, где в каждом пикселе записывается интенсивность красной, зеленой и синей составляющих, причем на каждую из них отводится по 8 битов. Этот большой массив данных можно преобразовать и сжать; в частности, в формате, называемом JPEG, размер записываемого массива данных может составлять только 5% от исходного объема в зависимости от количества деталей в изображении. Таким образом, принцип действия цифровых камер основан на применении цифровой памяти и на цифровой обработке данных.
- *Видеозапись.* На универсальном цифровом диске (digital versatile disc, DVD) видеозображение записывается в цифровом формате с большой степенью сжатия, называемом MPEG-2. Согласно этому стандарту осуществляется кодирование малой доли кадров видеозображения в формате подобном JPEG, а информация об остальных кадрах представляется в виде данных о различии между текущим кадром и предыдущим. Емкость одностороннего DVD-диска в зависимости от одного слоя составляет 35 миллиардов битов, и этого достаточно для записи примерно двухчасового фильма с хорошим качеством, а емкость двухслойного двустороннего диска в четыре раза больше.
- *Запись звука.* Если раньше все сводилось к записи аналоговых колебаний в виде отпечатка на виниловой пластинке или на магнитной ленте, то в настоящее время для записи звука применяют цифровые компакт-диски (compact disks, CDs). Музыка записывается на компакт-диск в виде последовательности 16-разрядных двоичных чисел, соответствующих выборкам, которые берутся из исходного аналогового колебания с интервалом 22,7 микросекунды в каждом из стереоканалов. Запись на целиком заполненном компакт-диске (73 минуты) содержит свыше 6 миллиардов битов информации.
- *Автомобильные карбюраторы.* Если раньше управление в автомобильном двигателе осуществлялось исключительно за счет механических связей (включая хитроумные «аналоговые» механические устройства, чувствительные к температуре, давлению и т.д.), то теперь работу двигателей контролируют встроенные микропроцессоры. Различные электронные и электромеханические датчики преобразуют информацию, характеризующую состояние двигателя, в числа, обрабатывая которые микропроцессор может управлять подачей в двигатель топлива и кислорода. На выходе микропроцессора появляется меняющаяся во времени последовательность чисел, которая воздействует на электромеханические приводы, которые в свою очередь, осуществляют управление двигателем.

- *Телефон.* В момент своего появления сто лет назад телефон состоял из аналоговых микрофона и воспроизводящего устройства, соединенных парой медных проводов (или это была веревка?). Даже сегодня у вас дома вероятнее всего стоит аналоговый телефонный аппарат, с которого на центральную телефонную станцию (ЦС) передаются аналоговые сигналы. Однако на большинстве ЦС эти аналоговые сигналы преобразуются в цифровой вид, прежде чем они отправляются по назначению, независимо от того, является абонентом той же самой ЦС или находится где-то еще. В частных телефонных сетях (private branch exchanges, PBXs), используемых в бизнесе, сигнал уже много лет сохраняет цифровой вид на всем пути до места приема. Сегодня многие коммерческие структуры, ЦС и поставщики традиционных телефонных услуг преобразуются в системы интегрированной связи, в которых оцифрованный звук и поток данных объединяются для передачи по одной IP-сети (IP – Internet Protocol).
- *Светофор.* Для переключения светофора обычно применяются электромеханические таймеры, с помощью которых зеленый свет включается в каждом из направлений на заранее установленное время. Позднее стали использовать контроллеры, позволяющие удерживать светофор в нужном состоянии в течение времени, которое зависит от интенсивности транспортного потока, определяемой с помощью датчиков, установленных в дорожное покрытие. В современных контроллерах применяют микропроцессоры, и это дает возможность так управлять сигналами светофора, чтобы максимизировать пропускную способность данного перекрестка или, как это сделано в некоторых городах в Калифорнии, для того чтобы самым изобретательным образом приводить водителей в замешательство.
- *Кинематографические трюки.* Раньше специальные эффекты реализовывались, как правило, с помощью миниатюрных глиняных моделей, путем фотография – кадр за кадром – последовательных фаз их движения, а также с помощью трюковых фотографий и многократного наложения изображения на пленке. Сегодня космические корабли, страшлища, елпы из других миров и даже дети неистодней (в художественном мультипликационном фильме «Игрушечная история» студии Pixar) сплетзпуются псключительно с помощью цифровых компьютеров. Может быть, в одни прекрасный день вообще отпадет нужда в каскадерах?

Революция в электронике длится уже довольно долго, и переход к «твердотельной» электронике начался с аналоговых элементов и устройств на их основе типа транзисторных приемников. Так почему же теперь происходит *цифровая* революция? На самом деле имеется много причин, подталкивающих нас к тому, чтобы отдать предпочтение цифровым схемам по сравнению с аналоговыми:

- *Воспроизводимость результатов.* При одном и том же наборе входных сигналов (при том же их числе и при той же их зависимости от времени) надлежащим образом спроектированная цифровая схема дает точно те же результаты. Выходные сигналы аналоговой схемы зависят от температуры, напряжения питания и других факторов, а также изменяются при старении компонентов.

- *Удобство проектирования.* Проектирование цифровых устройств, часто называемое «логическим проектированием», представляет собой логическую задачу. Не требуется никаких специальных математических знаний, и поведение небольшой логической схемы можно представить себе мысленно без какого-либо специального учета того, как действуют конденсаторы, транзисторы и другие элементы, для моделирования которых понадобились бы вычисления.
- *Гибкость и функциональность.* Как только задача сведена к дискретному виду, ее можно решить, выполнив последовательность логических шагов в пространстве и во времени. Например, вы можете сконструировать устройство, которое будет преобразовывать ваш речевой сигнал (скремблировать) таким образом, что он будет абсолютно не поддающимся дешифрованию кем-либо, у кого нет вашего «ключа» или пароля, но тот, у кого они есть, сможет услышать вашу речь без искажений.
- *Программируемость.* Возможно, что вы уже хорошо знакомы с цифровыми компьютерами и с легкостью составляете, пишете и отлаживаете программы для них. Угадываете, к чему я? Большая часть работы по проектированию цифровых устройств выполняется сегодня путем написания программ, да-да, и так называемых *языках описания схем (hardware description languages, HDLs)*. Эти языки позволяют задать как структуру цифровой схемы, так и выполняемую ею функцию, или *смоделировать* их. В типичном случае компилятор языка описания схем сопровождается программами моделирования и синтеза. Эти программные средства используются для тестирования поведения модели устройства до его реального воплощения, а затем для синтеза, то есть для преобразования модели в схему согласно технологии выбранных компонентов.
- *Быстродействие.* Сегодняшние цифровые устройства могут работать очень быстро. Отдельные транзисторы в самых быстрых интегральных микросхемах могут переключаться менее чем за 10 пикосекунд, а в законченном сложном устройстве, построенном на таких транзисторах, опрос его входов и формирование выходного сигнала происходят менее чем за 2 наносекунды. Это означает, что такое устройство способно производить 500 миллионов действий в секунду или больше.

КОРОТКИЕ ИНТЕРВАЛЫ ВРЕМЕНИ

Микросекунда (мкс, μs) равна 10^{-6} секунды. *Наносекунда (нс, ns)* — это 10^{-9} секунды, а *пикосекунда (пс, ps)* — 10^{-12} секунды. За одну наносекунду свет проходит в вакууме примерно фут (30 см), а дюйм (2.5 см) проходит за 85 пикосекунд. Поскольку отдельные транзисторы в самых быстрых интегральных микросхемах переключаются сегодня менее чем за 10 пикосекунд, задержка сигнала, распространяющегося со скоростью света, при прохождении от одного транзистора к другому в кремниевом кристалле площадью 0.5 квадратного дюйма (1.5×1.5 см) становится ограничивающим фактором при расчете схем.

- *Экономичность.* Цифровые схемы позволяют реализовать уйму функциональных возможностей в малом объеме. При массовом производстве широко используемые схемы можно «интегрировать» в один «кристалл» очень малой стоимости; благодаря этому появились дешевые изделия, такие как калькуляторы, цифровые часы и звуковые поздравительные открытки. (Вы можете спросить: «Так ли уж это хорошо?» Важно не это!)
- *Устойчиво развивающаяся технология.* Проектируя цифровую систему, вы почти всегда осознаете, что через несколько лет появится более быстрая, более дешевая или еще в каком-нибудь смысле лучшая элементная база. Мудрые проектировщики могут предусмотреть в исходной конструкции системы ожидаемое продвижение вперед, предвосхищая устаревание системы и делая ее в глазах потребителя более ценной. Например, настольные компьютеры часто снабжают «разъемами расширения» для включения более быстрых процессоров и памяти большего объема нежели те, какие были в наличии на момент выпуска этих компьютеров.

Вот так, пожалуй, обстоит дело с цифровым проектированием. Из оставшейся части этой главы вы узнаете чуть больше о технической стороне дела, и это подготовит вас к тому, о чем дальше пойдет речь в этой книге.

1.3. Цифровые устройства

Самые элементарные цифровые устройства называются *вентильми* (*gates*). Нет, нет! Не в честь основателя большой фирмы, специализирующейся на выпуске программного обеспечения. [Игра слов: по-английски термин «вентили» (во мн. числе) пишется и звучит точно так же, как имя Билла Гейтса (Bill Gates), основателя знаменитой компании Microsoft. — *Прим. перев.*] Первоначально вентили получили свое название по выполняемой ими функции, состоящей в том, чтобы пропускать или задерживать поток цифровой информации (служить по отношению к нему «воротами»). В общем случае вентиль имеет один или большее число входов и вырабатывает выходной сигнал, который зависит от значения сигнала на входе (или значений сигналов на входах) в данный момент времени. Хотя входные и выходные сигналы могут быть аналоговыми величинами, такими как напряжение, ток или даже давление жидкости, согласно модели считается, что они принимают только два дискретных значения: 0 и 1.

На рис. 1.1 приведены обозначения трех самых важных типов вентиляей. *Вентиль И* (или *логическая схема И*, *AND gate*) с двумя входами, показанный на рис. (а), вырабатывает 1 на выходе, если на обоих его входах действуют единичные сигналы; в противном случае выходной сигнал равен 0. На рисунке один и тот же вентиль повторен четыре раза с четырьмя возможными комбинациями сигналов на его входах и с соответствующими значениями выходного сигнала. Вентиль называется *комбинационной схемой* (*combinational circuit*), так как сигнал на его выходе определяется только комбинацией входных сигналов в данный момент времени.

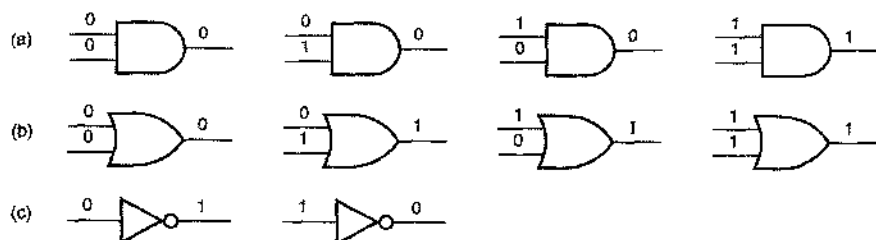


Рис. 1.1. Логические схемы: (а) схема И; (б) схема ИЛИ; (с) схема НЕ или инвертор

Показанная на рис. (б) 2-входовая логическая схема **ИЛИ** (*OR gate*) вырабатывает 1 на выходе, если на одном или на обоих ее входах присутствует 1; выходной сигнал этой схемы равен 0 только в том случае, когда оба входных сигнала равны 0. На рисунке снова приведены четыре возможные комбинации сигналов на входах с соответствующими значениями выходного сигнала.

На выходе логической схемы **НЕ** (*NOT gate*), чаще называемой *инвертором* (*inverter*), вырабатывается сигнал, значение которого противоположно значению входного сигнала, как это показало на рис. (с).

Мы назвали эти три логические схемы самыми важными по убедительной причине. Любую логическую функцию можно реализовать, пользуясь только этими тремя типами логических схем. В главе 3 будет показано, как выглядят логические схемы на транзисторах. Нужно, однако, помнить, что логические схемы строились или могли быть построены и на другой основе, например, на реле или на электронных лампах, а также в виде гидравлической конструкции или молекулярной решетки.

Устройство, способное хранить 0 или 1, называется *триггером* (*flip-flop*). *Состояние* (*state*) триггера — это значение, которое в нем хранится в настоящее время. Сохраняемое значение можно изменить только в определенные моменты времени, задаваемые «тактовым» входным сигналом, и то, что будет храниться в дальнейшем, зависит от предшествующего состояния триггера и от значений сигналов на его «управляющих» входах. Триггер можно построить в виде комбинации клапанов, включенных некоторым хитрым способом, как будет показано в параграфе 7.2.

Цифровая схема, содержащая триггеры, называется *последовательностной схемой* (*sequential circuit*), поскольку значение сигнала на ее выходе в какой-то момент времени зависит не только от сигналов, имеющих место на входах схемы в этот момент времени, но также и от предшествовавшей последовательности значений сигналов, которые были на ее входах ранее. Другими словами, последовательностная схема обладает *памятью* (*memory*) по отношению к событиям, происходившим ранее.

1.4. Электронные аспекты цифрового проектирования

Цифровые схемы не являются в точности двоичной версией «супа из букв»; при всем почтении к рис. 1.1, маленькие нули и единицы не плавают около входов и

выходов изображенных на нем схем. Как мы увидим в главе 3, цифровые схемы имеют дело с аналоговыми напряжениями и токами и строятся из аналоговых компонентов. «Цифровая абстракция» позволяет в большинстве случаев игнорировать аналоговое поведение, так что схемы можно представить в виде моделей, как если бы они на самом деле обрабатывали пули и единицы.

Важный аспект цифровой абстракции состоит в том, чтобы увязать *интервал* аналоговых значений с каждой из логических величин 0 и 1. Как показано на рис. 1.2, для типичного вентиля не гарантируется, что логическому 0 на выходе соответствует точно указываемый уровень напряжения. Скорее, напряжение на выходе вентиля окажется в пределах некоторого интервала, который является *подмножеством* множества значений, которые гарантированно воспринимаются как 0 входами других вентилях. Интервал между границами диапазонов называют *запасом помехоустойчивости (noise margin)*: сигнал на выходе вентиля может быть сильно искажен помехами, но все же он будет правильно интерпретироваться на входах других логических схем.

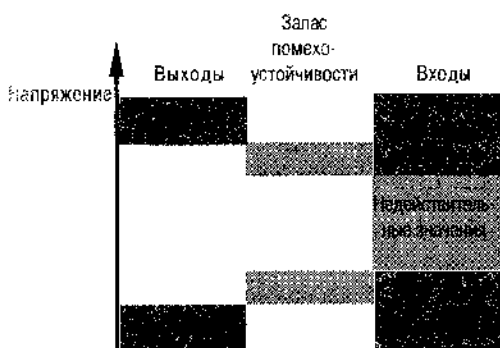


Рис. 1.2. Логические значения и запас помехоустойчивости

Аналогично обстоит дело с логической 1 на выходе. Обратите внимание, что между диапазонами, соответствующими логическому 0 и логической 1, имеется область «недействительных» значений. Хотя у любой конкретной цифровой схемы, работающей при заданном напряжении питания и при фиксированной температуре, граница между двумя диапазонами имеет вполне определенное значение, у разных логических схем их границы лежат *где-то* в области «недействительных» значений. Благодаря этому любой сигнал, принадлежащий диапазонам логического 0 и логической 1, будет одинаково интерпретироваться различными схемами. Это свойство существенно с точки зрения воспроизводимости результатов.

О том, чтобы вентили вырабатывали и распознавали логические сигналы, значения которых лежат внутри соответствующих интервалов, должен позаботиться разработчик *электронной* схемы. Это проблема расчета аналоговой схемы; некоторые аспекты этой проблемы мы затронем в главе 3. Нельзя построить схему, которая вела бы себя желаемым образом при произвольных напряжениях питания, температуре, нагрузке и других факторах. Поэтому разработчик электронной схемы или производитель тех или иных устройств снабжают их *техническими условиями (specification)*, при соблюдении которых правильная работа схемы или устройства гарантируется.

Однако проектировщику *цифрового* устройства нет необходимости вдаваться в детали аналогового поведения его конструкции, чтобы обеспечить правильность ее работы. Скорее всего, вам нужно будет только проконтролировать внешние условия, в которых предстоит функционировать устройству, и убедиться, что они не выходят за рамки известных наперед технических требований. Некоторые аналоговые знания необходимы, чтобы выполнить такую проверку, но далеко не столь глубокие, какие могли бы понадобиться, если начинать проектирование цифрового устройства с нуля. В главе 3 мы дадим вам как раз то, что нужно.

1.5. Роль программирования в проектировании цифровых устройств

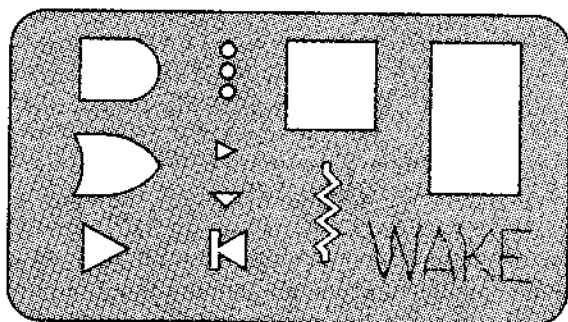
Цифровое проектирование не обязательно требует каких-то программных средств. Например, на рис. 1.3 представлен простейший инструмент разработки цифровых схем «старой школы» — пластиковый трафарет для вычерчивания логических элементов на принципиальной схеме вручную [а не паяльником выжжено имя владельца (WAKE — начальные буквы фамилии автора книги. — Прим. перев.)].

Однако сегодня программные средства являются существенной составной частью цифрового проектирования. Действительно, в последние несколько лет доступность и практичность языков описания схем в сочетании со средствами моделирования схем и загрузки программируемых кристаллов полностью изменили сам характер цифрового проектирования. В этой книге мы будем повсюду широко использовать языки описания схем.

Различные программные средства, применяемые при *автоматизированном проектировании* (*computer-aided design, CAD*), повышают производительность труда разработчика и позволяют выполнить проект более правильно и улучшить его качество. В мире, где властвует дух конкуренции, когда поджимают сроки, без использования программных средств нельзя получить высококачественные результаты. Вот важные примеры программных средств для цифрового проектирования:

- *Графический редактор схем.* Это схемотехнический эквивалент текстового редактора. Он позволяет рисовать схемы на экране, а не карандашом на бумаге. Более совершенные графические редакторы проверяют также, нет ли в схеме простых, легко обнаруживаемых ошибок, таких как короткое замыкание выходов, наличие пикуда не поданных сигналов и т.д. Такие программы подробнее рассматриваются в разделе 11.1.2.

Рис. 1.3. Трафарет разработчика цифровых схем



- *Языки описания схем.* Первоначально эти языки предназначались для моделирования схем, но сегодня все в большей степени они применяются для проектирования аппаратуры. Эти языки можно использовать при разработке чего угодно: от отдельных функциональных модулей до больших цифровых систем и многих кристаллах. В конце главы 4 мы познакомимся с двумя языками описания схем: ABEL и VHDL, а в последующих главах будут приводиться примеры на обоих языках.
- *Компиляторы языков описания схем, средства моделирования и синтеза.* Типичный программный пакет языка описания схем состоит из нескольких компонентов. Находясь в такой среде, проектировщик пишет текстовую «программу», и компилятор языка анализирует ее на наличие синтаксических ошибок. Если процесс компиляции завершается успешно, то проектировщик получает результат для передачи его программе синтеза, которая создает соответствующую конфигурацию схемы на заданной элементной базе. Чаще всего перед синтезом проектировщик использует результат компиляции в качестве входных данных моделирующей программы, чтобы проконтролировать поведение разрабатываемого устройства.
- *Моделирующие программы.* Цикл проектирования заказной однокристальной цифровой интегральной микросхемы долг и дорог. Когда кристалл уже изготовлен, очень трудно, а часто невозможно устранить ошибки, проверяя внутренние соединения (размеры которых, действительно, очень малы), или произвести изменения в логических схемах и соединениях между ними. Как правило, изменения необходимо внести в исходные данные проекта, и для реализации требуемых изменений должен быть изготовлен новый кристалл. Поскольку на выполнение этой процедуры могут уйти месяцы, у разработчиков кристаллов есть стимул сразу (или почти сразу) «получить то, что надо» с первой же попытки. Моделирующие программы помогают проектировщикам предсказать электрические характеристики и функциональное поведение кристалла без его фактического изготовления, позволяя обнаружить если не все, то подавляющее большинство недостатков до того, как кристалл будет сделан.
- Моделирующие программы используются также при разработке «программируемых логических устройств», с которыми мы познакомимся позднее, а также при проектировании больших схем, в которые входит много отдельных компонентов. В этом случае моделирование не столь существенно, поскольку разработчику легче провести замену компонентов или изменить разводку на печатной плате. Однако даже небольшое моделирование позволяет сэкономить время за счет обнаружения простых и глупых ошибок.
- *Программные средства тестирования (test benches).* Разработчики цифровых устройств научились формализовать моделирование и отладку схем в программных средах, получивших название программных или инструментальных средств тестирования. Идея состоит в том, чтобы собрать вокруг разрабатываемого проекта совокупность программ для автоматического контроля выполняемых схемой функций, а также для наблюдения за поведением схемы и определением ее временных характеристик. Это особенно полезно, когда в проекте производятся небольшие изменения; тогда можно загрузить программу тест-

тирования, чтобы убедиться в том, что обнаруженные ранее дефекты устранены, а «улучшения», произведенные в одном месте, ничего не нарушили ни в каком другом месте. Программы тестирования могут быть написаны на том же самом языке описания схем, на котором составлен проект, а также на C или C++ или на комбинации языков, включая языки описания сценариев типа языка PERL.

- *Средства анализа и контроля временных соотношений.* Описание временных параметров является важной составной частью проектирования цифровых устройств. Любой цифровой схеме необходимо время, чтобы выработать новое значение выходного сигнала в ответ на изменение на входе, и проектировщик тратит много усилий, чтобы убедиться, что соответствующие изменения на выходе происходят достаточно быстро (или, в некоторых случаях, не слишком быстро). Специализированные программы позволяют автоматизировать скучную процедуру рисования временных диаграмм, а также определять и контролировать временные соотношения между различными сигналами в сложной системе.
- *Текстовые редакторы.* Наконец, давайте не забудем об обычных текстовых редакторах и о текстовых процессорах. Очевидно, что эти средства нужны для написания исходной программы проекта на языке описания схем, но не менее важной для каждого проекта является та роль, которую они играют при создании документации!

Помимо использования перечисленных программных средств, разработчики иногда пишут специализированные программы на языках высокого уровня, таких как C или C++, или составляют сценарии на языках типа PERL для решения частных проблем, относящихся к проектируемой системе. Например, в разделе 10.1.6 приведена пара программ на языке C, генерирующих «таблицы истинности» для комбинационных схем, реализующих сложные функции.

Хотя средства автоматизации проектирования и важны, умения ими пользоваться еще не достаточно, чтобы быть хорошим проектировщиком цифровых устройств. Вы же не считаете себя великим писателем только потому, что быстро нажимаете на клавиши и ловко управляетесь с текстовым редактором. Обучаясь проектированию цифровых устройств, постарайтесь овладеть всеми средствами, какие только будут вам доступны: графическими редакторами схем, средствами моделирования, компиляторами языков описания схем. Но помните, что само по себе изучение этих средств не гарантирует достижения заведомо хороших результатов. Пожалуйста, будьте внимательны по отношению к тому, что получается в результате использования этих средств.

1.6. Интегральные схемы

Один или большее число вентилях, сформированных на одном кристалле кремния, называются *интегральной схемой* (ИС; *integrated circuit, IC*). Большие интегральные схемы с десятками миллионов транзисторов могут представлять собой квадратную пластину со стороной порядка половины дюйма или больше, тогда как у малых ИС этот размер может быть меньше одной десятой дюйма.

ПРОГРАММИРУЕМЫЕ ЛОГИЧЕСКИЕ УСТРОЙСТВА И МОДЕЛИРОВАНИЕ

Позднее вы узнаете из этой книги, что программируемые логические устройства (ПЛУ) и состоящие из *вентилей решетки, программируемые в процессе эксплуатации (field-programmable gate arrays, FPGAs)* позволяют сконструировать схему или подсистему путем написания своего рода программ. Сегодня имеются ПЛУ и устройства типа FPGA, содержащие до миллиона вентилей, и возможности, предоставляемые кристаллами, построенными по этой технологии, все время возрастают. Если конструкция, созданная вами на основе ПЛУ или устройства типа FPGA, не заработает с первого раза, то часто имеется возможность произвести исправление путем изменений в программе и физического перепрограммирования устройства, не заменяя компоненты и не внося изменений в соединения между ними на уровне системы. Легкость создания опытных образцов и модификации систем на основе ПЛУ и устройств типа FPGA может привести к исключению необходимости моделирования.

Самый распространенный взгляд на тенденции развития промышленности свидетельствует о том, что – по мере совершенствования технологии изготовления кристаллов – проектирование все в большей степени будет осуществляться на уровне кристалла, а не на уровне платы. Поэтому возможность полиого и подробного моделирования станет исключительно важной в типичном случае проектирования цифрового устройства.

Однако возможен и другой подход. Экстраполируя тенденции возможностей, предоставляемых ПЛУ и устройствами типа FPGA, следует ожидать, что в предстоящие десять лет мы станем свидетелями появления микросхем, содержащих в качестве составных элементов не только вентили и триггеры, но также и функциональные блоки более высокого уровня, такие как процессоры, память и контроллеры ввода/вывода. С этой точки зрения большинство проектировщиков цифровых устройств будут иметь дело со сложными компонентами внутри кристалла и такими соединениями, основные функции которых уже будут протестированы производителем микросхем. Имея в виду такое развитие, все же следует допустить возможность неправильного использования программируемых функций высокого уровня, но и в этом случае исправлять ошибки можно простым внесением изменений в программу; в этих условиях детальное моделирование проектируемого устройства до его «пробной реализации» может оказаться напрасной тратой времени. Другой взгляд на эту проблему с той же самой точки зрения состоит в том, что каждое ПЛУ или каждый кристалл FPGA – это просто-напросто устройство, моделирующее работу программы с максимальным быстродействием, и это все, что находится внутри интегральной схемы.

Обоснован ли этот крайний взгляд на вещи? Чтобы узнать ответ, задайте себе вопрос: многих ли вы знаете программистов, отлаживающих новую программу путем «моделирования» ее работы, а не ироесто пытаясь запустить ее?

В любом случае, современные цифровые системы слишком сложны, и у проектировщика нет никаких шансов перебрать все возможные комбинации входных воздействий независимо от того, применяется моделирование или нет. Как и в отношении программного обеспечения, правильность работы цифровой системы лучше всего проверяется на практике, которая и должна засвидетельствовать, что система «правильно спроектирована». Цель этой книги состоит в том, чтобы поощрить такую практику.

Независимо от размеров первоначально отдельные ИС являются элементами значительно большей по величине круглой *полупроводниковой пластины (wafer)* диаметром до 10 дюймов, содержащей от нескольких дюжин до нескольких сотен точных копий одной и той же ИС. Все ИС на этой пластине формируются одновременно, подобно ниццу, которая, в конце концов, иродается иорезанной на куски, за исключением того, что в нашем случае каждый кусочек (одна ИС) называется *кристаллом (die)*. После того, как иластинна изготовлена, осуществляется тестирование всех кристаллов, пока они еще остаются элементами этой иластинны, и неисиранные кристаллы помечаются. Затем иластинна разрезается на отдельные кристаллы, и отмеченные кристаллы при этом отбрасываются. (Сравните с тем, что делает тот, кто выпекает ниццу и продает все куски, даже недостаточно ионирченные!) Каждый непоиеченный кристалл укрепляется в корпусе, контактные площадки кристалла соединяются с выводами корпуса, помещенная в корпус ИС подвергается окончательному тестированию и отиравляется заказчику.

Некоторые уиотребляют термин «ИС» ио отношению к кремниевому кристаллу. Другие ту же самую вещь называют «микросхемой» («чиим»). А есть еще такие, кто словами «ИС» и «микросхема» обозначает комбниацию кремниевго кристалла и ее корпуса. Разработчики цифровых устройств используют эти термны попеременно, не обращая особого внимания на то, кто что говорит. Им не требуется точного определения, поскольку их интересует только фуиunctionальное назначение этих изделий и их электрические свойства. Дальше в этой книге мы будем под ИС ионимать кристалл, заключенный в корпус.

С самого иачала, когда только иоявились ИС, их стали классифицировать ио размерам – малые, средние и большие – в зависимости от того, сколько в них содержится вентиляей. Простейшие микросхемы, до сих пор выпускаемые серийно и содержащие от 1 до 20 вентиляей в эквивалентном выражении, называются ИС *малой степени иинтеграции (small-scale integration, SSI)* или малыми интегральными схемами (МИС). Обычно такая ИС содержит небольшое число вентиляей или триггеров, являющихся элементарными составными элементами при проектировании цифровых устройств.

МИС, которые скорее всего встретятся вам в учебной лаборатории, выиускаются в *корпусе с двухрядным расположением выводов [dual in-line-pin (DIP) package]*, с числом выводов, равным 14. Как иоказано на рис. 1.4(а), расстояние между выводами в ряду составляет 0.1 дюйма, а расстояние между рядами равно 0.3 дюйма. Если ИС для выиолнения ее функции требуется большее число выводов, то кристалл помещают в корпус DIP большего размера [см. рис. (b) и (c)]. *Схема расположения выводов (pin diagram)* указывает распределение имеющихся в схеме сигналов по выводам корпуса, или *цоколевку (pinout)*. На рис. 1.5 представлены схемы расположения выводов для нескольких распространенных МИС. Цоколевку используют только в том случае, когда проектировщику нужно определить иомера выводов в той или иной ИС. В иринципальных схемах цифровых устройств распределение выводов не указывается. Вместо этого различные вентиляи объединяются в группы ио функциональному признаку, и мы расскажем об этом в параграфе 5.1.

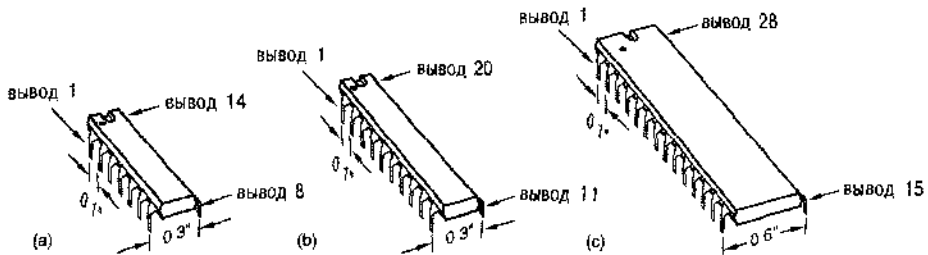


Рис. 1.4. Корпуса с двухрядным расположением выводов: (а) корпус с 14 выводами; (б) корпус с 20 выводами; (с) корпус с 28 выводами

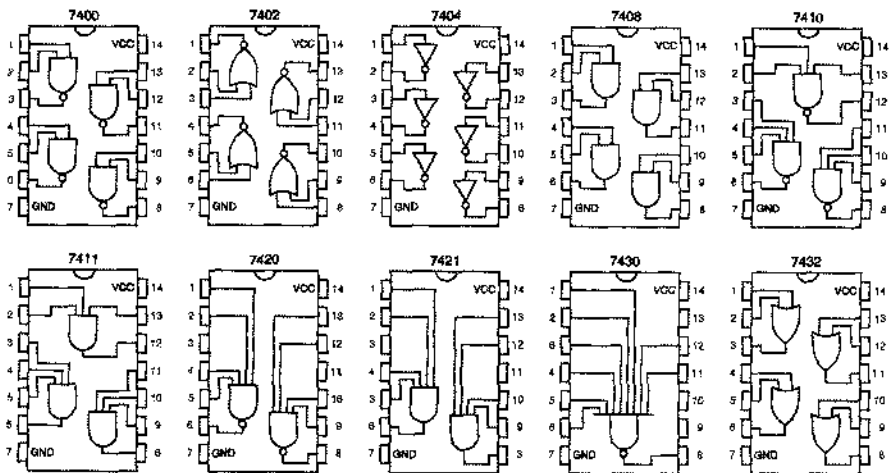


Рис. 1.5. Схемы расположения выводов для нескольких ИС малой степени интеграции серии 7400

Хотя МИС иногда еще применяются в качестве «связующих элементов» при объединении в сложные системы элементов большей степени интеграции, они, как правило, все же вытесняются программируемыми логическими устройствами (ПЛУ), с которыми мы познакомимся в параграфах 5.3 и 8.3.

Следующие по количеству вентилей ИС, выпускаемые серийно, называются интегральными схемами *средней степени интеграции* (*medium-scale integration, MSI*) или средними интегральными схемами (СИС); их содержимое примерно эквивалентно числу вентилей от 20 до 200. В типичном случае СИС представляет собой функциональный блок, такой как дешифратор, регистр или счетчик. В главах 5 и 8 будет сделан особый упор на функциональные блоки такого вида. Хотя применение самих СИС сокращается, эквивалентные им функциональные блоки широко используются при создании больших ИС.

Еще большими ИС являются интегральные схемы *высокой степени интеграции* (*large-scale integration, LSI*) или большие ИС (БИС), содержащие от 200 до

200000 вентилях в эквивалентном выражении или больше. Большими ИС являются запоминающие устройства малого объема, микропроцессоры, программируемые логические устройства и заказные ИС.

ОЧЕНЬ МАЛАЯ СТЕПЕНЬ ИНТЕГРАЦИИ

В предстоящие годы, возможно, основным местом, где по-прежнему будут применять ИС малой и средней степени интеграции, особенно в корпусах DIP, будут учебные лаборатории. Эти схемы предоставляют студентам благоприятную возможность приобрести опыт черновой работы по макетированию простых схем точно таким же образом, как это делали их преподаватели много лет назад.

Однако к моему огромному удивлению и восторгу, в промышленности, производящей ИС, в последние несколько лет возникло направление по выпуску ИС с меньшим уровнем интеграции, чем у МИС. Идея состоит в том, чтобы выпустить на рынок отдельные логические схемы в очень маленьких корпусах. Такие устройства реализуют простые функции, которые иногда бывает нужно выполнить для сопряжения компонентов большей степени интеграции при решении какой-то частной задачи. В некоторых случаях их применяют для того, чтобы преодолеть ошибки в компонентах большей степени интеграции или в их интерфейсах.

Примером такой ИС является схема 74VHC1G00 фирмы Motorola. Этот кристалл представляет собой двухвходовую логическую схему И-НЕ, помещенную в корпус с 5 выводами (питание, земля, два входа и один выход). Размеры всего корпуса, включая выводы, составляют лишь 0.08 дюйма в плоскости (сторона квадрата) и 0.04 дюйма в высоту! Это то, что я назвал бы «очень малой степенью интеграции»!

СТАНДАРТНЫЕ ЛОГИЧЕСКИЕ ФУНКЦИИ

При проектировании цифровых устройств все чаще возникает нужда в стандартных функциях «высокого уровня», которые выполнялись бы готовыми структурными блоками. В свое время эти функции первоначально были реализованы в МИС. Затем последовательно они становились компонентами «макро» библиотек при разработке специализированных ИС, «стандартными ячейками» при проектировании СБИС, «заготовками» функций для языков программирования ПЛУ и библиотечными функциями в языках описания схем типа VHDL.

В этой книге стандартные логические функции появляются в главах 5 и 8 в виде МИС серии 74, а также в виде текста на языке описания схем. Содержащиеся там обсуждения и примеры должны помочь вам понять, о чем идет речь, и послужить основой для использования этих функций в той или иной форме.

Граница между БИС и *сверхбольшими интегральными схемами (СБИС; very large-scale integration, VLSI)* расплывчата и часто ее выражают числом транзисторов, а не числом вентилей. Любая ИС с числом транзисторов более 1 000 000 принадлежит семейству СБИС; сверхбольшими ИС являются в наши дни большинство микропроцессоров и знаменитых устройств, а также большие программируемые логические устройства и заказные ИС. В 1999 году разрабатывались СБИС с числом транзисторов, доходившим до 50 миллионов.

1.7. Программируемые логические устройства

Существует множество ИС, у которых выполняемая ими логическая функция может быть «запрограммирована» уже после их изготовления. В большинстве случаев такие ИС бывают созданы на основе технологий, допускающей *перепрограммирование* выполняемой схемой функции, а это означает, что при обнаружении вами ошибки в разрабатываемом проекте можно устранить ее без физической замены ИС и без изменения схемы соединений. В этой книге мы часто будем ссылаться на предоставляемые возможности с точки зрения проектирования цифровых устройств и на то, как следует воспользоваться этими возможностями.

Исторически первыми программируемыми логическими устройствами были *программируемые логические матрицы (ПЛМ; programmable logic arrays, PLAs)*. ПЛМ представляли собой двухуровневую структуру, состоящую из вентилей И и ИЛИ, с программируемыми пользователем соединениями. Используя эту структуру, проектировщик мог приспособить такую ИС к выполнению любой логической функции, не превосходящей определенного уровня сложности, на основе известной теории логического синтеза и минимизации логических схем, которая будет представлена в главе 4.

Конструкция ПЛМ была усовершенствована, а цена на такие ИС снижена с появлением *программируемых логических устройств решетчатой структуры [programmable array logic (PAL) devices]*. Сегодня общее название таких ИС – *программируемые логические устройства (ПЛУ; programmable logic devices, PLDs)* и в иерархии изготавливаемых промышленностью программируемых устройств они являются схемами «средней степени интеграции». Об архитектуре и технологии ПЛУ будет многое сказано в параграфах 5.3 и 8.3.

Постоянно растущая емкость интегральных схем предоставляла производителям возможность разрабатывать крупные ПЛУ для построения более сложных цифровых устройств. Однако по техническим причинам, которые мы обсудим в параграфе 10.5, основную двухуровневую структуру И–ИЛИ в ПЛУ нельзя было расширить до больших размеров. Вместо этого в промышленности, производящей ИС, произошел переход к архитектуре *составных ПЛУ (complex PLDs, CPLDs)*, допускающей расширение до требуемого объема. Типичная ИС типа CPLD представляет собой совокупность нескольких ПЛУ со структурой соединений между ними, размещенной на том же самом кристалле. Помимо самих ПЛУ, структура соединений в микросхеме также является программируемой, чем обеспечиваются богатые возможности при проектировании. ИС типа CPLD мож-

ио расширять до больших размеров путем увеличения числа отдельных ПЛУ и за счет развития структуры соединений между ними, размещаемой на кристалле.

Примерно в то же время, когда были созданы ИС типа CPLD, другими производителями ИС был предпринят отличающийся от идеи CPLD подход к проблеме увеличения объема программируемых логических схем. По сравнению с ИС типа CPLD *решетки вентилей, программируемые в процессе эксплуатации (field-programmable gate arrays, FPGAs)* содержат много большее число меньших по размерам отдельных логических блоков и имеют развитую распределенную структуру внутренних соединений, которая занимает почти весь кристалл. Рис. 1.6 демонстрирует различие между этими двумя конструкциями микросхем.

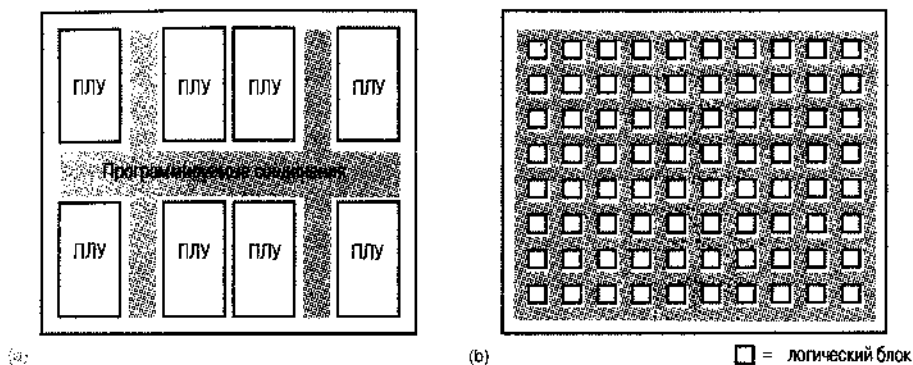


Рис. 1.6. Два подхода к увеличению объема программируемых логических устройств: (а) микросхема типа CPLD; (б) микросхема типа FPGA

Сторонники того и другого подходов обычно приводят доводы «религиозного» характера относительно того, какой из принципов лучше, но самый крупный производитель больших программируемых логических устройств фирма Xilinx, Inc., считает, что у каждого из этих подходов есть свое место, и производит ИС обоих типов. Более важным, нежели архитектура микросхем, является то обстоятельство, что оба подхода поддерживают такой стиль проектирования, который обеспечивает продвижение изделия от первоначальной идеи до опытного образца и серийного производства за очень короткое время.

Существенным с точки зрения быстрого появления на рынке изделий на основе ПЛУ всех типов является использование на стадии проектирования языков описания схем. Такие языки как ABEL и VHDL и сопровождающие их программные средства позволяют реализовать проект, осуществить синтез и произвести загрузку в ПЛУ и микросхему типа CPLD или FPGA буквально за считанные минуты. Вся мощь структурированных иерархических языков высокого уровня типа VHDL особенно ярко проявляется в том, что с их помощью можно создавать конструкции из сотен тысяч или миллионов вентилей, предоставляемых самими большими кристаллами типа CPLD и FPGA.

1.8. Специализированные интегральные схемы

С точки зрения многих разработчиков цифровых устройств самым интересным достижением в технологии ИС является, по-видимому, не постоянный рост объе-

ма кристаллов, а все время расширяющиеся возможности сконструировать свою собственную микросхему. Микросхемы, предназначенные для применений частного характера или для использования лишь в изделиях определенного типа, называются *полузаказными ИС (semicustom ICs)* или *специализированными ИС (application-specific ICs, ASICs)*. В общем случае применение ИС типа ASIC сокращает совокупную стоимость используемых компонентов и изготовления изделия за счет уменьшения числа микросхем, размеров и потребляемой мощности; часто, используя ИС типа ASIC, удается получить лучшие характеристики.

Единовременные затраты на проектирование [nonrecurring engineering (NRE) cost] одной ИС типа ASIC могут превышать стоимость разработки на основе отдельных серийных ИС на 5...250 тысяч долларов. Эти средства должны быть уплачены производителю ИС или тем людям, которые возьмут на себя ответственность за проектирование внутренней структуры микросхемы, создание оснастки типа металлизированных фотошаблонов, используемых при выращивании кристалла, разработку средств тестирования готовой продукции и фактическое изготовление нескольких первых образцов.

Единовременные затраты на проектирование типичной ИС типа ASIC средней сложности объемом около 100 000 вентилей составляют от 30 до 50 тысяч долларов. В обычных условиях проектирование заказной ИС имеет смысл только в том случае, когда указанные затраты можно будет покрыть за счет экономии на себестоимости каждого изделия при ожидаемом объеме продаж.

Если речь идет о проектировании *заказной БИС (custom LSI)*, то есть микросхемы, функции которой, внутренняя архитектура и структура кристалла на уровне транзисторов определяются индивидуальным заказом потребителя, то единовременные затраты на проектирование оказываются очень большими и могут составить 250 тысяч долларов и более. Поэтому проектирование специализированной БИС с нуля осуществляется только в том случае, когда ее применение имеет хорошую коммерческую перспективу или большой объем продаж изделий с ее использованием (например, цифровые наручные часы, сетевой интерфейс или схема сопряжения с системной шиной персонального компьютера).

Чтобы сократить начальные затраты на проектирование, производителями ИС были составлены библиотеки *стандартных элементов (standard cells)*, которые включали широко используемые блоки средней степени интеграции типа дешифраторов, регистров и счетчиков, а также блоки, реализующие наиболее употребительные функции БИС, такие как запоминающие устройства, программируемые логические матрицы и микропроцессоры. При *проектировании с использованием стандартных элементов (standard-cell design)* разработчик соединяет библиотечные блоки между собой точно так же, как это осуществлялось бы при проектировании с использованием отдельных СИС и БИС. Специализированные элементы создаются только в случае крайней необходимости (за дополнительную плату, конечно). Затем все элементы размещаются в кристалле, причем схема их расположения оптимизируется с целью сокращения задержек распространения сигналов и размеров микросхемы. Минимизация размеров кристалла позволяет сократить стоимость одного изделия, так как при этом увеличивается число кристаллов, которые можно изготовить на одной полупроводниковой пластине. В типичном случае затраты на проектирование с использованием стандартных элементов составляют порядка 150 тысяч долларов.

Но 150 тысяч долларов – это все еще большие деньги для многих людей. Поэтому производители ИС сделали еще один шаг по продвижению идеи проектирования специализированных микросхем в массы. Были предложены *вентильные матрицы* (*gate arrays*), то есть ИС, внутренняя структура которых представляет собой решетку вентиляей, соединения между которыми первоначально не заданы. Разработчик логического устройства определяет типы вентиляей и устанавливает соединения между ними. В результате проектирование кристалла осуществляется на таком очень низком уровне, но разработчик остается при этом на высоком уровне, оперируя «макроэлементами», реализующими те же функции, что и отдельные СИС и БИС или стандартные библиотечные элементы; для переноса с высокого уровня проектирования на низкий используется соответствующее программное обеспечение.

Главное отличие проектирования с использованием стандартных элементов от проектирования на основе вентильных матриц заключается в том, что в последнем случае макроэлементы и размещение в кристалле не оптимизируются в той степени, в какой это осуществляется в изделии, созданном на основе стандартных элементов. В результате размеры кристалла могут увеличиться (на 25% и более) и поэтому может возрасти его стоимость. Кроме того, при проектировании на основе вентильных матриц нет возможности создавать собственные элементы. Но с другой стороны, такой проект можно выполнить быстрее, а начальные затраты на его осуществление могут оказаться меньшими и лежать в пределах от 5 тысяч долларов (о которых было сказано в самом начале) до 75 тысяч долларов. (Когда все будет сделано, вы обнаружите, что потрачено именно столько.)

Основные методы проектирования цифровых устройств, которые вы изучите с помощью этой книги, вполне подходят для функционального конструирования специализированных ИС. Однако при проектировании таких ИС обычно необходимо принять во внимание дополнительные возможности или ограничения, зависящие, как правило, от конкретного заказчика и условий осуществления проекта.

1.9. Печатные платы

Обычно ИС устанавливается на *печатной плате* [*printed-circuit board (PCB)* или *printed-wiring board (PWB)*], где соединяется с другими ИС, образуя систему. В типичных цифровых системах используются многослойные печатные платы; на тонкие слои из стекловолокна наносятся медные полупроводники, и затем несколько таких слоев прессуются в одну плату толщиной порядка 1/16 дюйма.

Отдельные проводящие соединения, или *дорожки на печатной плате* (*PCB traces*), как правило, очень узки, и для типичной печатной платы их ширина составляет от 10 до 25 миллов. [1 мил (*mil*) равен 1/1000 дюйма.] У плат, изготовленных по специальной технологии, позволяющей наносить *тонкие линии* (*fine lines*), дорожки крайне узки: их ширина всего лишь 4 мила, и расстояние между соседними дорожками также равно 4 милам. Таким образом, в одном слое печатной платы в полосе шириной 1 дюйм можно провести до 125 соединений. Когда нужно, чтобы плотность соединений была больше, используют большее число слоев.

Большинство применяемых в настоящее время компонентов рассчитаны на *технологии поверхностного монтажа* (*surface-mount technology, SMT*). Вместо

длинных выводов корпусов DIP, проходящих сквозь плату и припаяваемых с обратной стороны, выводы корпусов SMT изогнуты так, чтобы обеспечить плотный контакт с верхней поверхностью печатной платы. Перед тем, как такие компоненты устанавливаются на печатную плату, на контактные площадки на плате наносится специальная «паяльная паста»; для этого используется трафарет с отверстиями, располагающимися против нужных контактных площадок. Далее компоненты в корпусах SMT устанавливаются на соответствующие контактные площадки (вручную или с помощью автомата), где они удерживаются паяльной пастой (иногда приклеиваются). Наконец, полностью собранную плату пропускают через печь, где паяльная паста плавится, а затем по мере остывания затвердевает.

Технология поверхностного монтажа компонентов и нанесение на печатную плату тонких линий дают возможность очень плотно разместить интегральные схемы и другие детали на плате. Такое плотное размещение позволяет не только сэкономить на объеме. В схемах с большим быстродействием плотность упаковки играет важную роль в отношении минимизации неблагоприятных аналоговых явлений, в том числе эффектов, возникающих в цепях с распределенными параметрами, влияния задержек из-за распространения сигнала с конечной скоростью.

Самым строгим требованиям по скорости и плотности удовлетворяют *многокристальные модули (multichip modules, MCM)*. Согласно этой технологии кристаллы ИС не помещаются в отдельные пластиковые или керамические корпуса. Вместо этого, кристаллы ИС, предназначенные для быстродействующей подсистемы (скажем, процессор и его кэш-память) закрепляются непосредственно на подложке с требуемыми соединениями, размещенными в нескольких слоях. Многокристальный модуль герметизируется и снабжается необходимыми внешними выводами для питания, земли и тех сигналов, которые требуются системе.

1.10. Уровни проектирования цифровых устройств

Проектирование цифровых устройств может выполняться на нескольких различных уровнях представления и абстракции. Хотя вы можете выучиться проектированию и работать на каком-то определенном уровне, время от времени у вас будет возникать необходимость перейти на один-два уровня вверх или вниз, чтобы завершить работу. Кроме того, рост плотности схем и увеличение их функциональной сложности постоянно подталкивают промышленность и проектировщиков к тому, чтобы переходить на более высокие уровни абстракции.

Самый низкий уровень представления при проектировании цифровых устройств — это физика того, что происходит в логической схеме, и процессы, протекающие при изготовлении ИС. Заслугой в первую очередь этого уровня является поразительный прогресс в отношении быстродействия и плотности ИС за последние десятилетия. Это стремительное движение отражает *закон Мура (Moore's Law)*, впервые сформулированный основателем фирмы Intel Гордоном Муром (Gordon Moore) в 1965 году, состоявший в том, что число транзисторов, входящих на квадратный дюйм в ИС, каждый год удваивается. В последние годы темп этого движения вперед замедлился: удвоение происходит теперь каждые 18

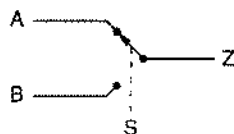
месяцев; но важно отметить, что одновременно с удвоением плотности также вдвое увеличивается быстроедействие схем.

В нашей книге мы не будем касаться этого уровня, но необходимо сознавать его важность. Проектируя системы и планируя производство, важно иметь представление о вероятном прогрессе в области технологии и о других изменениях. Например, недавно уменьшение геометрии в кристалле привело к переходу на более низкие напряжения питания логических схем, вызвав значительные изменения в том, как разработчики проектируют модульные системы, онисыывают и модериизируют их.

Мы совершим скачок в представлениях и абстракции сразу на уровень транзисторов и пройдем весь путь до уровня логического проектирования с использованием языков описания схем. Лпшь коротко мы остановимся на следующем уровне, который включает проектирование компьютеров и систем в целом. «Центральным» в нашем рассмотрении будет уровень функциональных структурных блоков.

Чтобы составить предварительное представление об уровнях проектирования цифровых устройств, которые встретятся на нашем пути, рассмотрим простой пример. Предположим, что нужно построить «мультиплексор» с двумя входами двоичных данных A и B , с двоичным управляющим входом S и двоичным выходом Z . В зависимости от того, какое значение имеет S — 0 или 1, — схема передает на выход Z значения A или B . Принцип действия такого устройства иллюстрирует «модель в виде переключателя» на рис. 1.7. Давайте разберем процесс проектирования такого устройства на нескольких различных уровнях.

Рис. 1.7. Модель мультиплексора в виде переключателя



Хотя логическое проектирование выполняется, как правило, на более высоком уровне, в отношении некоторых устройств полезно оптимизировать их, спустившись до уровня транзисторов. Мультиплексор как раз является таким устройством. На рис. 1.8 показано, как может выглядеть схема мультиплексора, созданная на основе КМОП-технологии с использованием специальных транзисторных структур, называемых «логическими ключами», которые будут рассмотрены в разделе 3.7.1. При таком подходе мультиплексор можно построить всего лишь на шести транзисторах, тогда как при любых других подходах, о которых пойдет речь, для этого требуется, по меньшей мере, 14 транзисторов.

По традиционной теории логического проектирования нам следовало бы воспользоваться «таблицей истинности» для описания логической функции, реализуемой мультиплексором. В таблице истинности перечисляются все возможные комбинации значений на входах и соответствующие данной функции значения выходного сигнала. Поскольку у мультиплексора три входа, имеется 2^3 , то есть 8 возможных комбинаций входных величин, как показано в табл. 1.1.

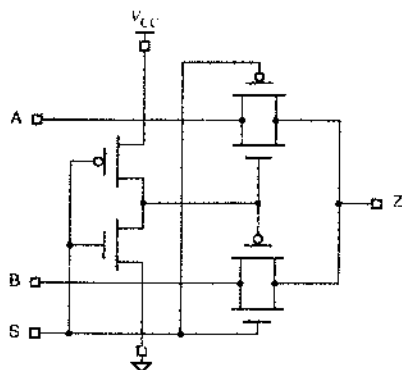


Рис. 1.8. Схема мультиплексора на логических ключах, выполненная по КМОП-технологии

S	A	B	Z
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	0
1	1	1	1

Табл. 1.1. Таблица истинности для мультиплексора

После того как таблица истинности составлена, согласно традиционным методам логического проектирования, которые будут рассмотрены в параграфе 4.3, мы должны воспользоваться булевой алгеброй и хорошо освоенными алгоритмами минимизации, в результате чего из таблицы истинности получим «оптимальное» двухуровневое соотношение в терминах И–ИЛИ. В случае мультиплексора у нас получилось бы следующее выражение:

$$Z = S' \cdot A + S \cdot B.$$

Это соотношение читается так: «Z равно не S и A или S и B». Продвигаясь еще на один шаг вперед, можно превратить данное соотношение в схему, состоящую из соответствующих вентилях, которая реализует заданную логическую функцию, как показано на рис. 1.9. Если иметь в виду КМОП-технологии, то для четырех вентилях в этой схеме потребуется 14 транзисторов.

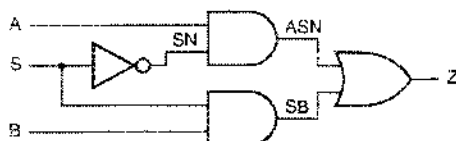


Рис. 1.9. Логическая схема мультиплексора на уровне вентилях

Мультиплексоры применяются очень широко, поэтому в большинстве систем логических элементов имеются готовые структурные блоки, реализующие функцию мультиплексора. Примером может служить микросхема средней степени интеграции 74х157, которая осуществляет переключение входных сигналов, поступающих по двум 4-разрядным шинам. На рис. 1.10 приведено условное обозначение такого устройства; как видно, достаточно изменения одного управляющего бита, чтобы решить проблему одновременного переключения всех сигнальных линий 4-разрядных шин. Рядом с выводами указаны их номера для корпуса DIP с 16 выводами, в котором находится мультиплексор.

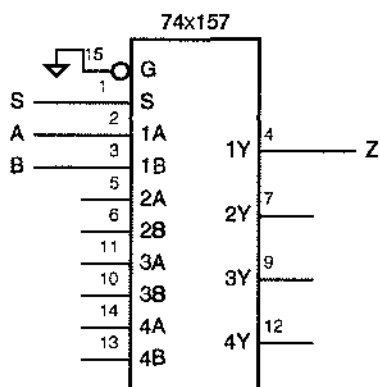


Рис. 1.10. Условное обозначение мультиплексора, выполненного в виде ИС средней степени интеграции

Можно реализовать функцию мультиплексора на части программируемого логического устройства. Языки типа ABEL позволяют с помощью булевых соотношений, подобных приведенному выше, определять выходные сигналы, однако, как правило, для этого удобнее воспользоваться элементами языка более высокого уровня. В табл. 1.2 в качестве примера приведена программа для мультиплексора на языке ABEL. Первыми тремя строками задается имя программного модуля и тип ПЛУ, на котором должна быть реализована данная функция. В следующих двух строках входам и выходу присваиваются определенные номера выводов ИС. Оператор "WHEN" служит выражением фактически реализуемой логической функции, которое совсем нетрудно понять, несмотря на то, что язык ABEL вам еще не знаком.

Табл. 1.2. Программа для мультиплексора на языке ABEL

```
module chapmux
title 'Two-input multiplexer example'
CHAPMUX device 'P16V8'

A, B, S      pin 1, 2, 3;
Z            pin 13 istype 'com';

equations

WHEN S == 0 THEN Z = A; ELSE Z = B;

end chapmux
```


На языке VHDL, который является языком более высокого уровня, функцию мультиплексора можно задать более гибко и по иерархическому принципу. Пример программы для мультиплексора на языке VHDL приведен в табл. 1.3. В первых двух строках указаны стандартная библиотека и набор определений, которые используются в проекте. В следующих четырех строках только определяются входы и выходы проектируемого устройства, а здесь намеренно ничего не говорится о каких-либо деталях того, как именно требуемая функция будет реализована внутри ПЛУ. Функциональное поведение устройства конкретизируется в части программы, начинающейся с заголовка "architecture". Синтаксис языка VHDL требует небольшого навыка, но оператор "when" в данной программе выражает собой, по существу, то же самое, что и в варианте программы на языке ABEL. Средства, ответственные за синтез схем, в программном пакете языка VHDL могут начинать с этого функционального описания, и результатом их работы будет нужная схема, сформированная в ПЛУ, выполненном по заданной технологии.

```
library IEEE;
use IEEE.std_logic_1164.all;

entity Vchaplumux is
    port ( A, B, S: in  STD_LOGIC;
          Z:   out STD_LOGIC );
end Vchaplumux;

architecture Vchaplumux_arch of Vchaplumux is
begin
    Z <= A when S = '0' else B;
end Vchaplumux_arch;
```

Табл. 1.3. Программа для мультиплексора на языке VHDL

Принудительное разделение определений входов/выходов ("entity") и внутренней реализации ("architecture") в языке VHDL позволяет разработчику легко задавать альтернативные реализации функций, не внося изменений нигде более в иерархии проекта. Например, в качестве альтернативы можно было бы задать структурную архитектуру мультиплексора, как это сделано в табл. 1.4. Эта архитектура фактически представляет собой текстовый эквивалент логической схемы, приведенной на рис. 1.9.

```
architecture Vchaplumux_gate_arch of Vchaplumux is
signal SN, ASN, SB: STD_LOGIC;
begin
    U1: INV (S, SN);
    U2: AND2 (A, SN, ASN);
    U3: AND2 (S, B, SB);
    U4: OR2 (ASN, SB, Z);
end Vchaplumux_gate_arch;
```

Табл. 1.4. «Структурная» VHDL-программа для мультиплексора

Заглядывая чуть дальше, мы могли бы убедиться в том, что язык VHDL является довольно мощным средством, фактически позволяющим так описать работу про-

ектируемого устройства, что его функциональное поведение окажется смоделированным на уровне транзисторов (правда, мы не будем заниматься этим в нашей книге). Таким образом, принципиально возможно написать программу на языке VHDL, реализующую на уровне транзисторов мультиплексор, эквивалентный схеме на рис. 1.8.

1.11. Самое главное

Когда функциональное поведение цифровой системы и требования к ее характеристикам заданы, цель практического цифрового проектирования состоит в минимизации ее стоимости. При проектировании на уровне печатной платы (*board-level design*), то есть в случае, когда проектируемая система будет занимать одну печатную плату, это условие обычно означает, что должно быть минимальным число корпусов ИС. Если число ИС слишком велико, они не поместятся на печатной плате. Вы скажете: «Ну, воспользуемся тогда большей платой». К сожалению, размеры платы, как правило, ограничены такими факторами, как уже существующие стандарты (например, на платы, вставляемые в персональные компьютеры), требованиями к конструктивному оформлению (например, предназначенная для тостера плата должна в него входить) или указаниями сверху (например, при утверждении проекта три месяца назад вы неосторожно сказали руководству, что вся система разместится на печатной плате размером 3×5 дюймов, и теперь должны оправдать ожидания). В каждом из этих случаев использование большей по размерам платы или нескольких плат может оказаться неприемлемым по стоимости.

Чаше всего к минимизации числа ИС стремятся даже в том случае, когда стоимость отдельных ИС изменяется. Например, типичные МИС могут стоить 25 центов, тогда как небольшое ПЛУ может стоить доллар. Какую-нибудь конкретную функцию можно реализовать на трех ИС малой и средней степени интеграции (75 центов) или на одном ПЛУ (один доллар). В большинстве случаев отдадут предпочтение более дорогому ПЛУ, и не потому, что разработчик владеет акциями компании, производящей эти микросхемы, а из-за того, что ПЛУ занимает меньше места на плате и в случае его применения значительно легче будет что-то изменить, если создаваемое устройство не заработает правильно с первого раза.

При создании устройств на основе специализированных ИС (*ASIC design*) суть дела немного в другом, но по-прежнему важным является структурный, функциональный подход к проектированию. Нетрудно потратить массу времени на разработку микросхем и минимизацию общего числа вентилях в специализированной ИС, только это редко бывает разумным. Если снижение стоимости отдельного изделия достигается за счет применения меньшего на 10% кристалла, то этим снижением можно пренебречь за исключением случаев, когда эти изделия пред-

стоит производить в большом количестве. Если планируемый объем выпуска изделий невелик (а это бывает в большинстве случаев), то более важным являются два других фактора: продолжительность проектирования во времени и начальные затраты.

Чем быстрее выполняется проект, тем скорее можно вывести изделие на рынок и тем самым повысить доходы за то время, пока изделие пользуется спросом. Снижение начальных вложений является чуть ли не основным моментом, и в небольших компаниях это может быть единственной возможностью успеть выполнить проект полностью до того, как у вашей фирмы кончатся деньги (поверьте, мне довелось с этим столкнуться!). Если первоначальный результат проектирования оказывается успешным, то вслед за этим всегда бывает возможным и полезным «повылизывать» ваш проект, чтобы уменьшить стоимость одиночного изделия. Необходимость сокращения времени разработки и начальных затрат служит аргументом в пользу структурного подхода при проектировании на основе специализированной ИС в противоположность принципу предельной оптимизации, поэтому следует пользоваться стандартными функциональными блоками, имеющимися в библиотеке производителя специализированных ИС.

Проектирование на основе ПЛУ и ИС типа CPLD и FPGA представляет собой комбинацию приведенных выше соображений. Выбор конкретного типа ПЛУ и размеров устройства обычно осуществляется на самой ранней стадии проекта, и пока ваше устройство «влезает» в выбранный кристалл, нет смысла пытаться оптимизировать число вентилях или площадь печатной платы: все, что нужно, уже сделано. Если, однако, реализация новых функций или ошибок выводят вас за пределы возможностей выбранного кристалла, то это тот случай, когда вы должны хорошо поработать над проектом и видоизменить его так, чтобы он оказался выполненным.

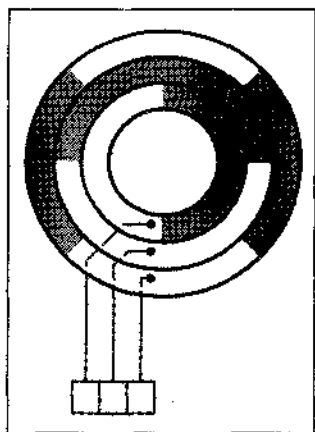
1.12. Напутствие

На этом наше введение подходит к концу. Читая эту книгу, помните о двух вещах. Во-первых, конечная цель цифрового проектирования состоит в том, чтобы создавать системы для решения проблем, стоящих перед людьми. Хотя эта книга и вооружит вас основными инструментами проектирования, держать в уме всю «картину в целом» — это уже ваша забота. Во-вторых, цена является важным фактором при принятии любых решений, причем вы должны учитывать не только стоимость цифровых компонентов, но и затраты на сам процесс проектирования.

Наконец, по мере чтения этого учебника, возможно, встретится что-то такое, что, как вам будет казаться, вы видели раньше, но не помните, где именно. Пожалуйста, обращайтесь в таких случаях к предметному указателю. Я постарался сделать его настолько полезным и полным, насколько это было возможно.

Упражнения

- 1.1. Предложите что-нибудь, что будет лучше выглядеть в качестве заставки к первой главе в следующем издании этой книги.
- 1.2. Дайте три различных определения слова «бит», имея в виду то, как оно было истолковано в этой главе.
- 1.3. Что означают следующие аббревиатуры: ASIC, CAD, CD, CPLD, DIP, DVD, FPGA, HDL, IP, LSI (БИС), MSI (СИС), MCM, NRE, OK, PBX, PCB, PWB, SMT, SSI (МИС), VHDL, VLSI (СБИС).
- 1.4. Исследуйте определения следующих акронимов: ABEL, CMOS (КМОП), JPEG, MPEG, OK, PERL, VHDL. (Действительно ли «OK» является акронимом?)
- 1.5. Назовите три системы, помимо перечисленных в параграфе 1.2, которые раньше были аналоговыми и стали цифровыми уже после того, как вы родились.
- 1.6. Нарисуйте цифровую схему, состоящую из двухвходового вентиля И и трех инверторов, которые подключены ко входам и к выходу вентиля И. Для каждой из четырех возможных комбинаций сигналов на входах этой схемы определите значение сигнала на выходе. Существует ли более простая схема, обеспечивающая то же самое соответствие между входными сигналами и сигналом на выходе?
- 1.7. Когда следует использовать в принципиальной схеме устройства схему расположения выводов, изображенную на рис. 1.5?



ЧИСЛОВЫЕ СИСТЕМЫ И КОДЫ

Цифровые системы строятся на основе схем, в которых происходит обработка двоичных чисел — нулей и единиц. Однако в реальной жизни лишь немногие проблемы можно описать двоичными числами или как-либо-либо числами вообще. Поэтому проектировщик цифровой системы должен установить некоторое соответствие между двоичными числами, обрабатываемыми в цифровых схемах, и числами, событиями и обстоятельствами, относящимися к реальному миру. Цель этой главы состоит в том, чтобы показать, как знакомые числовые величины, а также печатные данные, события и состояния могут быть представлены в цифровой системе и как можно оперировать ими внутри этой системы.

В первых девяти параграфах описываются двоичные числовые системы; будет показано, как в этих системах выполняются сложение, вычитание, умножение и деление. В параграфах 2.10 — 2.13 мы покажем, как с помощью строк, состоящих из двоичных цифр, можно закодировать двоичные числа или символы текста, а также отобразить положение механического объекта или любое другое обстоятельство.

В параграфе 2.14 вводятся «*n*-мерные кубы», которые дают возможность наглядно представить соотношение между различными строками битов. *n*-мерные кубы оказываются особенно полезными при рассмотрении в параграфе 2.15 кодов, обнаруживающих ошибки. Заканчивается эта глава введением в методы кодирования, применяемые при оптимальной передаче данных и при их хранении.

2.1. Позиционные системы счисления

Традиционная система чисел, которой нас научили в школе и которой мы ежедневно пользуемся, является *позиционной системой счисления* (*positional number system*). В такой системе число представляется строкой цифр, в которой каждому разряду принадлежит определенный *вес* (*weight*). Значение числа равно взвешенной сумме его разрядов, например:

$$1734 = 1 \cdot 1000 + 7 \cdot 100 + 3 \cdot 10 + 4 \cdot 1.$$

Каждый вес — это степень числа 10, соответствующая положению цифры в строке. Десятичная точка позволяет использовать как положительные, так и отрицательные степени числа 10:

$$5185.68 = 5 \cdot 1000 + 1 \cdot 100 + 8 \cdot 10 + 5 \cdot 1 + 6 \cdot 0.1 + 8 \cdot 0.01.$$

В общем случае число D вида $d_1 d_0 \cdot d_{-1} d_{-2}$ равно

$$D = d_1 \cdot 10 + d_0 \cdot 10^0 + d_{-1} \cdot 10^{-1} + d_{-2} \cdot 10^{-2}.$$

Число 10 называется *основанием* (*base* или *radix*) числовой системы. В произвольной позиционной числовой системе основанием может быть любое целое число $r \geq 2$, и тогда i -й разряд имеет вес r^i . Общая форма числа в такой системе имеет вид:

$$d_{p-1} \cdots d_0 \cdot d_{-1} \cdots d_{-n},$$

где p цифр находятся слева, а n цифр — справа от точки, отделяющей целую часть числа от дробной (*radix point*). Если эта точка опущена, то предполагается, что ее место — справа от крайней правой цифры. Значение числа равно сумме значений отдельных разрядов, умноженных на соответствующие степени основания:

$$D = \sum_{i=-n}^{p-1} d_i \cdot r^i.$$

За исключением возможных предшествующих и последующих нулей, представление числа в позиционной системе счисления однозначное. (Очевидно, что 0185.63 равно 185.63 и т.д.) Крайний левый разряд в такой записи называется *старшим разрядом* (*high-order* или *most significant digit*), а крайний правый разряд — *младшим разрядом* (*low-order* или *least significant digit*).

Как мы узнаем из главы 3, в нормальных условиях уровень сигнала в цифровой системе бывает низким или высоким, отражая при этом два состояния: разряжено или заряжено, выключено или включено. Считается, что сигналы в таких схемах представляют собой *двоичные числа* (*binary digits*) [или *биты* (*bits*)], которые принимают одно из двух значений: 0 или 1. Таким образом, для представления чисел в цифровой системе обычно в качестве *основания* используется число 2 (*binary radix*). В общем случае двоичное число имеет вид:

$$b_{p-1} b_{p-2} \cdots b_1 b_0 \cdot b_{-1} b_{-2} \cdots b_{-n},$$

и его значение равно

$$B = \sum_{i=-n}^{p-1} b_i \cdot 2^i.$$

Точку в двоичном числе называют *двоичной точкой* (*binary point*). Опираясь с двоичными или другими недесятичными числами, у каждого из таких чисел указывают в виде индекса основание системы счисления, если только оно очевидным образом не следует из контекста. Приведем примеры двоичных чисел и их десятичные эквиваленты:

$$10011_2 = 1 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 19_{10}$$

$$100010_2 = 1 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 34_{10}$$

$$101.001_2 = 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 + 0 \cdot 0.5 + 0 \cdot 0.25 + 1 \cdot 0.125 = 5.125_{10}$$

Крайний левый бит двоичного числа называется *старшим битом* (*high-order* или *most significant bit, MSB*), а самый правый – *младшим битом* (*low-order* или *least significant bit, LSB*).

2.2. Восьмеричные и шестнадцатеричные числа

Основание 10 важно потому, что мы пользуемся им в повседневной жизни, а основание 2 – потому, что непосредственной обработке в цифровых схемах подвергаются двоичные числа. Числами, выраженными в других системах счисления, напрямую оперируют не так часто, но они могут быть важны для документации и для других целей. В частности, для краткой записи многоразрядных двоичных чисел в цифровой системе удобно применять основания 8 и 16.

В *восьмеричной системе счисления* (*octal number system*) в качестве основания используется число 8, а в *шестнадцатеричной системе счисления* (*hexadecimal number system*) – число 16. В табл. 2.1 представлены двоичные целые числа от 0 до 1111 и их восьмеричные, десятичные и шестнадцатеричные эквиваленты. Для восьмеричной системы нужны 8 цифр, поэтому в ней используются цифры 0–7 десятичной системы. Шестнадцатеричной системе нужны 16 цифр, поэтому для нее десятичные цифры 0–9 дополняются буквами A–F (*hexadecimal digit A–F*).

Табл. 2.1. Двоичные, десятичные, восьмеричные и шестнадцатеричные числа

Двоичные	Десятичные	Восьмеричные	Строка из 3 битов	Шестнадцатеричные	Строка из 4 битов
0	0	0	000	0	0000
1	1	1	001	1	0001
10	2	2	010	2	0010
11	3	3	011	3	0011
100	4	4	100	4	0100
101	5	5	101	5	0101
110	6	6	110	6	0110
111	7	7	111	7	0111
1000	8	10	—	8	1000
1001	9	11	—	9	1001
1010	10	12	—	A	1010
1011	11	13	—	B	1011
1100	12	14	—	C	1100
1101	13	15	—	D	1101
1110	14	16	—	E	1110
1111	15	17	—	F	1111

Восьмеричная и шестнадцатеричная системы счисления удобны для представления многоразрядных двоичных чисел потому, что их основания являются степенями числа 2. Поскольку в строке из 3 битов возможны 8 различных комбинаций, из этого следует, что каждую такую строку можно однозначно представить одной восьмеричной цифрой, как это сделано в третьем и четвертом столбцах в табл. 2.1. Точно так же 4-битовую строку можно представить одной шестнадцатеричной цифрой согласно пятому и sixthому столбцам таблицы.

Таким образом, двоичные числа легко преобразовать в восьмеричные (*binary-to-octal conversion*). Начиная с двоичной точки и двигаясь влево, мы просто делим биты на группы по три и каждую группу заменяем соответствующей восьмеричной цифрой:

$$\begin{aligned} 100011001110_2 &= 100\ 011\ 001\ 110_2 = 4316_8 \\ 11101101110101001_2 &= 011\ 101\ 101\ 110\ 101\ 001_2 = 355651_8. \end{aligned}$$

Подобным же образом осуществляется преобразование двоичных чисел в шестнадцатеричные (*binary-to-hexadecimal conversion*), за исключением того, что биты надо разбивать на группы по четыре:

$$\begin{aligned} 100011001110_2 &= 1000\ 1100\ 1110_2 = 8CE_{16} \\ 11101101110101001_2 &= 0001\ 1101\ 1011\ 1010\ 1001_2 = 1DBA9_{16}. \end{aligned}$$

В этих примерах мы были вольны добавлять нули слева до полного числа битов, кратного 3 или 4, по мере необходимости.

Если у двоичного числа есть разряды справа от двоичной точки, то их тоже можно преобразовать в восьмеричные или шестнадцатеричные символы, начиная с двоичной точки и двигаясь вправо. Как слева, так и справа можно добавлять нули до числа битов, кратного трем или четырем, как это показано в следующем примере:

$$\begin{aligned} 10.1011001011_2 &= 010.101\ 100\ 101\ 100_2 = 2.545_8 \\ &= 0010.1011\ 0010\ 1100_2 = 2.B2C_{16}. \end{aligned}$$

«КОГДА МНЕ 64...»

Ставаясь старше, вы обнаруживаете, что шестнадцатеричная система счисления полезна не только применительно к компьютерам. Когда мне исполнилось 40, я сказал друзьям, что мне всего лишь 28_{16} . Причем добавление « $_{16}$ » я произнес, конечно, шепотом. В 50 лет мне будет только 32_{16} .

Все люди с большим воодушевлением отмечают «круглые» юбилеи в 20, 30, 40, 50... лет, но вы, наверно, сможете уверить своих друзей в том, что десятичная система по своей значимости не является более фундаментальной, чем любая другая. Более важные изменения происходят тогда, когда вам исполняется 2 и 4 года, 8 и 16 лет, 32 и 64 года: в этот момент в вашем возрасте появляется новый старший бит. А то почему вы думаете «Битлз» пели: «Когда мне шестьдесят четыре...»?

Осуществить преобразование в обратном направлении, из восьмеричного или шестнадцатеричного вида в двоичный (*octal- или hexadecimal-to-binary conversion*) очень легко. Нужно просто заменить каждую восьмеричную или шестнадцатеричную цифру соответствующей 3- или 4-битовой строкой, как показано ниже:

$$\begin{aligned} 1357_8 &= 001\,011\,101\,111_2 \\ 2046.17_8 &= 010\,000\,100\,110.001\,111_2 \\ \text{BEAD}_{16} &= 1011\,1110\,1010\,1101_2 \\ 9\text{F.46C}_{16} &= 1001\,1111.0100\,0110\,1100_2 \end{aligned}$$

Восьмеричная система счисления была очень популярна 25 лет назад: тогда у ряда миникомпьютеров сигнальные лампочки и переключатели на передней панели были разбиты на группы по три. Однако сегодня восьмеричная система чисел используется не так часто из-за преобладания машин, которые оперируют 8-разрядными байтами (*bytes*). Из восьмеричного представления многобайтовых величин трудно извлечь значения отдельных байтов; например, как выглядят в восьмеричной записи четыре 8-разрядных байта 32-разрядного числа, которое в восьмеричном представлении имеет вид: 12345670123?

В шестнадцатеричной системе 8-разрядный байт представляется двумя цифрами, а 2*n* цифр изображают *n*-байтовое слово: каждая пара цифр образует в точности один байт. Например, 32-разрядное шестнадцатеричное число 5678ABCD₁₆ состоит из четырех байтов, значения которых равны 56₁₆, 78₁₆, AB₁₆ и CD₁₆. В этом контексте состоящее из 4-х битов одноразрядное шестнадцатеричное число называют иногда *полубайтом* (*nibble*); 32-разрядное (4-байтовое) число состоит из восьми полубайтов. Шестнадцатеричные числа часто используют при описании адресного пространства в памяти компьютера. Например, о компьютере с 16-разрядными адресами могут сказать, что его память, предназначенная для чтения и/или записи, располагается по адресам 0–FFFF₁₆, а в отношении части памяти с адресами F000–FFFF₁₆ предусмотрено только чтение из нее. Во многих компьютерных языках программирования используется префикс «0x» (*0x prefix*) для обозначения шестнадцатеричной записи числа, например: 0xBFC0000.

2.3. Общие преобразования позиционных систем счисления

В общем случае преобразование от системы счисления с одним основанием к системе счисления с другим основанием нельзя выполнить простой подстановкой; требуются арифметические операции. В этом параграфе мы покажем, как числа, записанные в числовой системе с произвольным основанием, преобразуются в десятичные числа и наоборот с использованием десятичной арифметики.

В параграфе 2.1 мы видели, что при произвольном основании значение числа задается формулой:

$$D = \sum_{i=-n}^{p-1} d_i \cdot r^i,$$

где r — основание системы счисления и имеются p цифр слева от точки, разделяющей целую и дробную части числа, и n цифр справа от нее. Значение числа можно найти, преобразуя каждый разряд этого числа в его десятичный эквивалент и распространяя правила десятичной арифметики на приведенную формулу. Вот несколько примеров:

$$\begin{aligned} \text{ICE8}_{16} &= 1 \cdot 16^3 + 12 \cdot 16^2 + 14 \cdot 16^1 + 8 \cdot 16^0 = 7400_{10} \\ \text{FIA3}_{16} &= 15 \cdot 16^3 + 1 \cdot 16^2 + 10 \cdot 16^1 + 3 \cdot 16^0 = 61859_{10} \\ 436.5_8 &= 4 \cdot 8^2 + 3 \cdot 8^1 + 6 \cdot 8^0 + 5 \cdot 8^{-1} = 286.625_{10} \\ 132.3_4 &= 1 \cdot 4^2 + 3 \cdot 4^1 + 2 \cdot 4^0 + 3 \cdot 4^{-1} = 30.75_{10} \end{aligned}$$

Для перевода в десятичную форму целых чисел можно получить экономное правило, используя *представление числа в виде поочередных вложений (nested expansion formula)*:

$$D = (((\cdots((d_{p-1} \cdot r + d_{p-2}) \cdot r + \cdots) \cdot r + d_1) \cdot r + d_0$$

Другими словами, приравняв значение суммы равным нулю и начиная с крайнего левого разряда, будем умножать сумму на r и к результату прибавлять следующий разряд до тех пор, пока не будут обработаны все разряды. Можно записать, например:

$$\text{FIAC}_{16} = (((((15) \cdot 16 + 1) \cdot 16 + 10) \cdot 16 + 12$$

Эта формула используется в итеративных алгоритмах преобразования, реализуемых программно (таких, как алгоритм, приведенный в табл. 4.38). Она служит также основой очень удобного метода *преобразования десятичного числа D к представлению в системе счисления с основанием r (decimal-to-radix- r conversion)*. Давайте посмотрим, что получится, если поделить правую часть этой формулы на r . Поскольку слагаемое со скобками делится на r нацело, частное от деления равно

$$Q = (((\cdots((d_{p-1}) \cdot r + d_{p-2}) \cdot r + \cdots) \cdot r + d_1$$

а остаток равен d_0 . Таким образом, d_0 можно вычислить как остаток при делении D на r . Кроме того, частное от деления Q является таким же по форме, как и D в начальной формуле. Поэтому последовательные шаги при делении на r дают очередные значения в разрядах справа налево, до тех пор, пока не будут получены все разряды. Приведем примеры этой процедуры:

$$\begin{aligned} 179 \div 2 &= 89 \text{ остаток } 1 \quad (\text{младший разряд}) \\ \div 2 &= 44 \text{ остаток } 1 \\ \div 2 &= 22 \text{ остаток } 0 \\ \div 2 &= 11 \text{ остаток } 0 \\ \div 2 &= 5 \text{ остаток } 1 \\ \div 2 &= 2 \text{ остаток } 1 \\ \div 2 &= 1 \text{ остаток } 0 \\ \div 2 &= 0 \text{ остаток } 1 \quad (\text{старший разряд}) \end{aligned}$$

$$179_{10} = 10110011_2$$

$$467 \div 8 = 58 \text{ остаток } 3 \text{ (младший разряд)}$$

$$\div 8 = 7 \text{ остаток } 2$$

$$\div 8 = 0 \text{ остаток } 7 \text{ (старший разряд)}$$

$$467_{10} = 723_8$$

$$3417 \div 16 = 213 \text{ остаток } 9 \text{ (младший разряд)}$$

$$\div 16 = 13 \text{ остаток } 5$$

$$\div 16 = 0 \text{ остаток } 13 \text{ (старший разряд)}$$

$$3417_{10} = D59_{16}$$

В табл. 2.2 собраны методы преобразования чисел для часто встречающихся значений основания.

Табл. 2.2. Методы преобразования чисел для часто встречающихся значений основания

Преобразование	Метод	Пример
Двоичного числа в		
Восьмеричное	Подстановка	$10111011001_2 = 10\ 111\ 011\ 001_2 = 2731_8$
Шестнадцатеричное	Подстановка	$10111011001_2 = 101\ 1101\ 1001_2 = 5D9_{16}$
Десятичное	Подстановка	$10111011001_2 = 1 \cdot 1024 + 0 \cdot 512 + 1 \cdot 256 + 1 \cdot 128 + 1 \cdot 64 +$ $+ 0 \cdot 32 + 1 \cdot 16 + 1 \cdot 8 + 0 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 = 1497_{10}$
Восьмеричного числа в		
Двоичное	Подстановка	$1234_8 = 001\ 010\ 011\ 100_2$
Шестнадцатеричное	Подстановка	$1234_8 = 001\ 010\ 011\ 100_2 = 0010\ 1001\ 1100_2 = 29C_{16}$
Десятичное	Подстановка	$1234_8 = 1 \cdot 512 + 2 \cdot 64 + 3 \cdot 8 + 4 \cdot 1 = 668_{10}$
Шестнадцатеричного числа в		
Двоичное	Подстановка	$C0DE_{16} = 1100\ 0000\ 1101\ 1110_2$
Восьмеричное	Подстановка	$C0DE_{16} = 1100\ 0000\ 1101\ 1110_2 =$ $= 1\ 100\ 000\ 011\ 011\ 110_2 = 140336_8$
Десятичное	Подстановка	$C0DE_{16} = 12 \cdot 4096 + 0 \cdot 256 + 13 \cdot 16 + 14 \cdot 1 =$ $= 49374_{10}$
Десятичного числа в		
Двоичное	Деление	$108_{10} \div 2 = 54 \text{ остаток } 0 \text{ (младший разряд)}$ $\div 2 = 27 \text{ остаток } 0$ $\div 2 = 13 \text{ остаток } 1$ $\div 2 = 6 \text{ остаток } 0$ $\div 2 = 3 \text{ остаток } 0$ $\div 2 = 1 \text{ остаток } 1$ $\div 2 = 0 \text{ остаток } 1$ (старший разряд)
		$108_{10} = 1101100_2$

Преобразование	Метод	Пример
Восьмеричное	Деление	$108_{10} \div 8 = 13$ остаток 4 (младший разряд) $\div 8 = 1$ остаток 5 $\div 8 = 0$ остаток 1 (старший разряд) $108_{10} = 154_8$
Шестнадцатеричное	Деление	$108_{10} \div 16 = 6$ остаток 12 (младший разряд) $\div 16 = 0$ остаток 6 (старший разряд) $108_{10} = 6C_{16}$

2.4. Сложение и вычитание недесятичных чисел

При сложении и вычитании недесятичных чисел вручную алгоритм действий тот же самый, которому нас научили в средней школе для десятичных чисел; подвох состоит только в том, что правила сложения и вычитания оказываются другими.

Правила сложения двоичных чисел (*binary addition*) и их вычитания представлены в табл. 2.3. При сложении двух двоичных чисел X и Y мы складываем младшие биты, полагая, что бит переноса в этот разряд (c_{in}) равен 0; в результате получаем бит переноса в старший разряд (c_{out}) и сумму (s) согласно таблице. Затем продолжаем эту процедуру для других разрядов, переходя справа налево и принимая каждый раз во внимание бит переноса, полученный на предыдущем шаге суммирования.

На рис. 2.1 приведены два примера сложения десятичных чисел и соответствующих им двоичных чисел, где стрелками показаны переносы в тех случаях, когда бит переноса равен 1. Те же примеры повторены ниже, где к ним добавлены два других примера, а значения битов переноса представлены двоичной строкой C :

C		10111000		C		001011000
X	190	10111110		X	173	10101101
Y	+141	+ 10001101		Y	+ 44	+ 00101100
$X + Y$	331	101001011		$X + Y$	217	11011001
C		011111110		C		000000000
X	127	01111111		X	170	10101010
Y	+ 63	+ 00111111		Y	+ 85	+ 01010101
$X + Y$	190	10111110		$X + Y$	255	11111111

c_{in} или b_{in}	x	y	c_{out}	s	b_{out}	d
0	0	0	0	0	0	0
0	0	1	0	1	1	1
0	1	0	0	1	0	1
0	1	1	1	0	0	0
1	0	0	0	1	1	1
1	0	1	1	0	1	0
1	1	0	1	0	0	0
1	1	1	1	1	1	1

Табл. 2.3. Правила двоичного сложения и вычитания

$$\begin{array}{r}
 X \quad 190 \quad \begin{array}{cccccccc} & & 1 & 1 & 1 & 1 & & \\ & & 1 & 0 & 1 & 1 & 1 & 1 & 0 \end{array} \\
 Y \quad +141 \quad \begin{array}{cccccccc} & & 1 & 0 & 0 & 0 & 1 & 1 & 0 & 1 \end{array} \\
 \hline
 X+Y \quad 331 \quad \begin{array}{cccccccc} & & 1 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \end{array}
 \end{array}$$

$$\begin{array}{r}
 X \quad 173 \quad \begin{array}{cccccccc} & & 1 & & 1 & 1 & & \\ & & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \end{array} \\
 Y \quad +44 \quad \begin{array}{cccccccc} & & 1 & 0 & 0 & 1 & 0 & 0 & 1 \end{array} \\
 \hline
 X+Y \quad 217 \quad \begin{array}{cccccccc} & & 1 & 1 & 0 & 1 & 1 & 0 & 0 & 1 \end{array}
 \end{array}$$

Рис. 2.1. Примеры сложения десятичных чисел и соответствующих им двоичных чисел

Необходимо взять равный единице бит заема, что приводит к новому вычитанию: $10 - 1 = 1$

После взятия заема на первом шаге в этом разряде возникло новое вычитание $0 - 1$, поэтому мы снова должны произвести заем

Требуется заем проводить несколько раз подряд, прежде чем достигнет такого разряда, где можно получить заем без дальнейшего заимствования, другими словами: $100 = 011$ (вычитаемые биты) $+ 1$ (бит заема)

$$\begin{array}{r}
 \text{уменьшаемое } X \quad 229 \quad \begin{array}{cccccccc} & & 0 & 10 & 1 & 1 & 1 & 10 \end{array} \\
 \text{вычитаемое } Y \quad -46 \quad \begin{array}{cccccccc} & & 0 & 0 & 1 & 0 & 1 & 0 \end{array} \\
 \hline
 \text{разность } X-Y \quad 183 \quad \begin{array}{cccccccc} & & 1 & 0 & 1 & 1 & 0 & 1 \end{array}
 \end{array}$$

$$\begin{array}{r}
 X \quad 210 \quad \begin{array}{cccccccc} & & 0 & 10 & 10 & 0 & 1 & 10 & 0 & 10 \end{array} \\
 Y \quad -109 \quad \begin{array}{cccccccc} & & 1 & 1 & 0 & 1 & 0 & 1 & 1 & 0 & 1 \end{array} \\
 \hline
 X-Y \quad 101 \quad \begin{array}{cccccccc} & & 0 & 1 & 1 & 0 & 0 & 1 & 0 & 1 \end{array}
 \end{array}$$

Рис. 2.2. Примеры вычитания десятичных чисел и соответствующих им двоичных чисел

Подобным образом выполняется и *двоичное вычитание* (*binary subtraction*), только при переходе от одного шага к другому принимаются во внимание не вырабатываются не биты переноса, а биты заема (b_{in} и b_{out}), и результатом вычитания в

каждом разряде ставим разность d . Два примера вычитания десятичных и соответствующих им двоичных чисел приведены на рис. 2.2. На рисунке с помощью стрелок и битов показано, что точно так же, как и при десятичном вычитании, значение двоичного *уменьшаемого* (*minuend*) в каждом разряде меняется при взятии заема из старшего разряда. Те же примеры повторены ниже, к ним добавлены два других примера, а значения битов заема представлены двоичной строкой B : Вычитание очень часто производится в компьютерах при *сравнении чисел* (*comparing numbers*). Если, например, при выполнении операции $X - Y$ возникает требование заема из старшего бита, то это означает, что X меньше Y ; в противном случае X больше или равно Y . В параграфе 5.10 мы рассмотрим соотношение между битами переноса и битами заема в схемах суммирования и вычитания

B		001111100
X	229	11100101
Y	- 46	- 00101110
$X - Y$	183	10110111

B		010101010
X	170	10101010
Y	- 85	- 01010101
$X - Y$	85	01010101

B		011011010
X	210	11010010
Y	-109	- 01101101
$X - Y$	101	01100101

B		000000000
X	221	11011101
Y	- 76	- 01001100
$X - Y$	145	10010001

Можно составить таблицы сложения и вычитания восьмеричных и шестнадцатеричных чисел и чисел, представленных в системе счисления с любым другим основанием. Однако мало кто из специалистов по вычислительной технике берется их запомнить. Если необходимость оперировать недесятичными числами возникает у вас редко, то проще всего в таких случаях преобразовывать числа в десятичный вид, производить с ними требуемые вычисления и затем осуществлять обратное преобразование. С другой стороны, если вам часто приходится оперировать двоичными, восьмеричными и шестнадцатеричными числами, то вам следует попросить у Деда Мороза «ведьмы калькулятор» («*hex calculator*») фирм Texas Instruments или Casio. (Игра слов: англ. «hex» в зависимости от контекста означает «ведьма» или «шестнадцатеричный». — Прим. перев.)

В случае, когда батарейка в вашем калькуляторе разрядится, можно воспользоваться простыми соображениями, облегчающими вычисления с недесятичными числами. Как правило, при сложении (или вычитании) число в каждом разряде можно преобразовать в десятичный вид, выполнить с ним требуемое действие, а затем результат и значение переноса перевести в соответствующую недесятичную систему счисления. (Перенос возникает всякий раз, когда сумма в данном разряде оказывается равной основанию системы счисления или превосходит его.) Выполняя сложение десятичных чисел, мы пользуемся известными нам правилами суммирования в десятичной системе; новым является только то, что нам нужно научиться осуществлять переход от десятичных чисел к недесятичным и в обратную сторону. Последовательность шагов, выполняемых в уме при сложении шестнадцатеричных чисел (*hexadecimal addition*), имеет вид:

C	1 1 0 0	1	1	0	0
X	1 9 B 9 ₁₆	1	9	11	9
Y	+ C 7 E 6 ₁₆	+12	7	14	6
X + Y	E 1 9 F ₁₆	14	17	25	15
		14	16+1	16+9	15
		E	1	9	F

2.5. Представление отрицательных чисел

До сих пор мы имели дело только с положительными числами. Существует много способов представления отрицательных чисел. В повседневной практике мы используем систему представления чисел в прямом коде со знаком. Однако в большинстве компьютеров применяется одна из форм представления чисел в виде дополнения, о чем речь пойдет позднее.

2.5.1 Представление чисел в прямом коде со знаком

В системе представления чисел в прямом коде со знаком (*signed-magnitude system*) число состоит из величины и символа, указывающего на то, какой является эта величина: положительной или отрицательной. Именно так мы обычно интерпретируем десятичные числа +98, -57, +123.5 и -13; мы также считаем, что в случае, если символ знака отсутствует, то знак имеет значение «+». Существует два возможных представления нуля: «+0» и «-0», имеющие одно и то же значение.

Применение системы представления чисел в прямом коде со знаком к двоичным числам сводится к добавлению еще одного двоичного разряда, служащего выражением знака (*знакового бита, sign bit*). По традиции роль знакового бита в двоичной записи играет старший разряд (0 = плюс, 1 = минус), а остальные биты используются для представления величины. Вот несколько 8-разрядных двоичных целых чисел, представленных в прямом коде со знаком, и их десятичные эквиваленты:

$$\begin{array}{ll}
 01010101_2 = +85_{10} & 11010101_2 = -85_{10} \\
 01111111_2 = +127_{10} & 11111111_2 = -127_{10} \\
 00000000_2 = +0_{10} & 10000000_2 = -0_{10}
 \end{array}$$

В рассматриваемой системе число положительных и отрицательных целых чисел одинаково. Величина целого числа, выраженная в битах, лежит в интервале от $-(2^{n-1} - 1)$ до $+(2^{n-1} - 1)$, и существует два возможных представления нуля.

Предположим теперь, что мы хотим собрать цифровую логическую схему сложения чисел в прямом коде со знаком (*signed-magnitude adder*). В схеме должна производиться проверка знаков слагаемых, для того чтобы определить, что делать с их величинами. Если знаки одинаковые, то нужно сложить величины и результату присвоить тот же самый знак. Если знаки различны, то в схеме должно быть осуществлено сравнение величин, из большей величины должна быть вычтена меньшая и результату должен быть присвоен знак большей величины. Все эти «если», «сложить», «вычесть» и «сравнить» приводят к сложной логической схеме. Сумматоры чисел, представленных в форме дополнения, много проще, как

будет показано в дальнейшем. Одно обстоятельство комментирует, пожалуй, сложность оперирования числами, представленными в прямом коде со знаком: коль скоро мы знаем, как собрать сумматор таких чисел, почти тривиальной оказывается задача построения *вычитающего устройства* (*signed-magnitude subtractor*); нужно только изменить знак у *вычитаемого* (*subtrahend*) и подать его вместе с уменьшаемым на входы сумматора.

2.5.2. Системы представления чисел в форме дополнения

Если при представлении чисел в прямом коде со знаком для изменения знака числа меняется значение знакового разряда, то в *системе представления чисел в форме дополнения* (*complement number system*) для этого берется дополнение данного числа по правилам, действующим в пределах этой системы. Взятие дополнения – более сложная процедура, нежели изменение знака, однако два числа, представленные в форме дополнения, можно складывать и вычитать непосредственно, не проверяя их знаков и величин, как это требуется при представлении чисел в прямом коде со знаком. Мы опишем две системы представления чисел в форме дополнения, которые называются «точным дополнением» («дополнительным кодом») и «поразрядным дополнением» («обратным кодом»).

В любой системе представления чисел в форме дополнения обычно имеют дело с фиксированным числом разрядов, равным, скажем, n . (Однако можно увеличить число разрядов за счет «знакового расширения» или уменьшить число разрядов путем отбрасывания старших разрядов, как это делается в задачах 2.23 и 2.24 соответственно.) В дальнейшем будем предполагать, что основание системы счисления равно r , а сами числа имеют вид

$$D = d_{n-1}d_{n-2} \cdots d_1d_0.$$

Точка разделения целой и дробной части пусть находится справа, так что числа являются целыми. Если в результате выполнения той или иной операции требуется большее, чем n , число разрядов, то избыточные старшие разряды отбрасываются. Если дополнение числа D берется дважды, то результат равен D .

2.5.3. Дополнительный код

При *точном дополнении* (*radix-complement system*) дополнение n -разрядного числа получается путем вычитания его из r^n . В случае десятичных чисел точное дополнение носит название *дополнения в десятичной системе* (*10's complement*). В табл. 2.4 приведены несколько примеров 4-разрядных десятичных чисел и результат их вычитания из 10000.

По определению, точное дополнение n -разрядного числа D есть результат вычитания D из r^n . Если D находится между 1 и $r^n - 1$, то в результате вычитания получим другое число, принадлежащее тому же интервалу значений. Если $D = 0$, то результат вычитания равен r^n и имеет вид $100 \cdots 00$, где полное число разрядов равно $n + 1$. Отбросим лишний старший разряд и получим 0. Таким образом, в случае точного дополнения существует одно единственное представление нуля.

Число	Точное дополнение (дополнительный код)	Поразрядное дополнение (обратный код)
1849	8151	8150
2067	7933	7932
100	9900	9899
7	9993	9992
8151	1849	1848
0	10000 (= 0)	9999

Табл. 2.4. Примеры точного и поразрядного дополнений в десятичной системе

Исходя из определения, можно подумать, что для вычисления *точного дополнения* (*computing the radix complement*) числа D нужно производить вычитание. Однако вычитания можно избежать, записывая r^n в виде $(r^n - 1) + 1$, а $r^n - D$ — в виде $((r^n - 1) - D) + 1$. В свою очередь число $r^n - 1$ имеет вид: $mm \dots mm$, где $m = r - 1$ и символ m повторяется n раз. Например, число 10000 равно $9999 + 1$. Если приписать, по определению, что дополнением *одноразрядного* числа d является число $r - 1 - d$, то $(r^n - 1) - D$ получается взятием дополнений в разрядах числа D . Поэтому точное дополнение числа D есть результат взятия дополнений в отдельных разрядах D и прибавления 1. Например, дополнение числа 1849 в десятичной системе равно $8150 + 1$, то есть 8151. Навык применения этого правила стоит закрепить, убедившись в его справедливости на других примерах, приведенных в табл. 2.4. В табл. 2.5 перечислены дополнения *одноразрядных* чисел для двоичной, восьмеричной, десятичной и шестнадцатеричной систем счисления.

Число	Дополнение			
	Двоичное	Восьмеричное	Десятичное	Шестнадцатеричное
0	1	7	9	F
1	0	6	8	E
2	—	5	7	D
3	—	4	6	C
4	—	3	5	B
5	—	2	4	A
6	—	1	3	9
7	—	0	2	8
8	—	—	1	7
9	—	—	0	6
A	—	—	—	5
B	—	—	—	4
C	—	—	—	3
D	—	—	—	2
E	—	—	—	1
F	—	—	—	0

Табл. 2.5. Дополнения *одноразрядных* чисел

2.5.4. Представление двоичных чисел в двоичном дополнительном коде

Точное дополнение для двоичных чисел носит название *двоичного дополнительного кода* (*two's complement*). В этой системе старший разряд числа служит битом знака; число отрицательно тогда и только тогда, когда старший бит равен 1. Десятичный эквивалент двоичного числа, представленного в дополнительном коде, вычисляется так же, как в случае числа без знака, за исключением того, что *вес старшего бита* (*weight of MSB*) равен -2^{n-1} , а не $+2^{n-1}$. Диапазон значений представляемых таким образом чисел простирается от $-(2^{n-1})$ до $+(2^{n-1}-1)$. Вот примеры 8-разрядных двоичных чисел:

$17_{10} = 00010001_2$ $\Downarrow$ 11101110 $+1$ $\hline 11101111_2 = -17_{10}$	побитное дополнение	$-99_{10} = 10011101_2$ $\Downarrow$ 01100010 $+1$ $\hline 01100011_2 = 99_{10}$	побитное дополнение
$119_{10} = 01110111_2$ $\Downarrow$ 10001000_2 $+1$ $\hline 10001001_2 = -119_{10}$	побитное дополнение	$-127_{10} = 10000001_2$ $\Downarrow$ 01111110_2 $+1$ $\hline 01111111_2 = 127_{10}$	побитное дополнение
$0_{10} = 00000000_2$ $\Downarrow$ 11111111 $+1$ $\hline 1\ 00000000_2 = 0_{10}$	побитное дополнение	$-128_{10} = 10000000_2$ $\Downarrow$ 01111111 $+1$ $\hline 10000000_2 = -128_{10}$	побитное дополнение

В левом нижнем примере возникает перенос из старшего разряда. Здесь, как и повсюду при выполнении операций с двоичными числами, представленными в дополнительном коде, этот бит игнорируется и в качестве результата берутся только n младших разрядов.

При представлении двоичных чисел дополнительным кодом роль считается положительным числом, так как его знаковый бит равен 0. Поскольку возможно только одно двоичное представление нуля в дополнительном коде, в нижней части диапазона представляемых чисел имеется *еще одно отрицательное число* (*extra negative number*) -2^{n-1} , у которого нет положительного эквивалента.

Представленное в дополнительном коде n -разрядное двоичное число X можно преобразовать в m -разрядное число, но при этом нужна определенная осторожность. Если $m > n$, то мы должны добавить к числу X слева $m-n$ битов, являющихся копиями знакового бита числа X (см. задачу 2.23). Другими словами, положительное число мы дополняем нулями, а отрицательное — единицами; такое действие называется *знаковым расширением* (*sign extension*). Если $m < n$, то мы отбрасываем $n-m$ битов слева; однако результат будет справедливым только в том случае,

когда все отбрасываемые биты имеют то же значение, что и знаковый бит остающегося числа (см. задачу 2.24).

В большинстве компьютеров и в других цифровых системах используется представление отрицательных чисел в дополнительном коде. Однако, ради полноты, мы рассмотрим также поразрядное дополнение и двоичный обратный код.

*2.5.5. Представление в форме поразрядного дополнения

В системе с поразрядным дополнением (*diminished radix-complement system*) дополнение n -разрядного числа D получается путем его вычитания из $r^n - 1$. Это можно осуществить, беря дополнение в разрядах числа D порознь без добавления 1 в отличие от того, как это делается при взятии точного дополнения. В случае десятичных чисел такое преобразование можно назвать *поразрядным дополнением до девяти* (*9's complement*); примеры таких дополнений приведены в правом столбце табл. 2.4.

*2.5.6. Представление двоичных чисел в обратном коде

В случае двоичных чисел поразрядное дополнение называют *обратным кодом* (*ones' complement*). Как и в дополнительном коде, старший бит является знаковым: он равен 0 у положительных чисел и 1 — у отрицательных. Таким образом, существует два представления нуля: положительный ноль (00...00) и отрицательный ноль (11...11). Положительные числа при их представлении в дополнительном коде и в обратном коде выглядят одинаково. Но представления отрицательных чисел различаются на 1. При вычислении десятичного эквивалента числа, представленного в обратном коде, вес старшего бита равен $-(2^{n-1}-1)$, а не -2^{n-1} . Диапазон чисел, которые могут быть представлены обратным кодом, простирается от $-(2^{n-1}-1)$ до $+(2^{n-1}-1)$. Приведем примеры 8-разрядных двоичных чисел и соответствующий им обратный код:

Главным достоинством представления в обратном коде является его симметрия и легкость реализации. Однако построение сумматора чисел, представленных в обратном коде, требует большей изобретательности, нежели в случае сумматора чи-

$$\begin{array}{rcl}
 17_{10} = 00010001_2 & & -99_{10} = 10011100_2 \\
 \Downarrow & & \Downarrow \\
 11101110_2 = -17_{10} & & 01100011_2 = 99_{10} \\
 \\
 119_{10} = 01110111_2 & & -127_{10} = 10000000_2 \\
 \Downarrow & & \Downarrow \\
 10001000_2 = -119_{10} & & 01111111_2 = 127_{10} \\
 \\
 0_{10} = 00000000_2 \quad (\text{положительный ноль}) & & \\
 \Downarrow & & \\
 11111111_2 = 0_{10} \quad (\text{отрицательный ноль}) & &
 \end{array}$$

*Здесь и далее звездочкой помечены необязательные разделы

сел в дополнительном коде (см. задачу 7.72). Кроме того, схема обнаружения нуля для системы чисел, представленных в обратном коде, должна либо проверять обе возможности, либо всегда преобразовывать $11 \dots 11$ в $00 \dots 00$.

*2.5.7. Представление чисел с избытком

Да, действительно, различных способов представления отрицательных чисел так много, что можно говорить об их избытке, по все же есть еще один, о котором мы расскажем. При *представлении с избытком B (excess- B representation)* целого числа без знака M , значение которого удовлетворяет условию $0 \leq M < 2^m$, строка из m битов служит для выражения целого числа со знаком $M - B$, где B называется *смещением (bias)* данной числовой системы.

Например, в *системе с избытком 2^{m-1} (excess- 2^{m-1} system)* любое число X , принадлежащее интервалу от -2^{m-1} до $+2^{m-1} - 1$, представляется в двоичном виде m битами как $X + 2^{m-1}$ (эта последняя величина всегда неотрицательна и меньше, чем 2^m). Диапазон значений, которые могут быть представлены в такой системе, точно совпадает с множеством возможных значений двоичных m -разрядных чисел в дополнительном коде. В самом деле, вид любого числа в этих двух системах одинаков, за исключением знакового бита, значения которого в них всегда противоположны. (Это справедливо только в том случае, когда смещение равно 2^{m-1} .)

Чаще всего представление с избытком применяется в числовых системах с плавающей точкой (см. Обзор литературы).

2.6. Сложение и вычитание двоичных чисел в дополнительном коде

2.6.1. Правила сложения

Из табл. 2.6, где приведены десятичные числа и их эквиваленты в различных числовых системах, видно, почему представление в дополнительном коде является предпочтительным для выполнения арифметических операций. При счете в прямом направлении, начиная с числа $1000_2 (-8_{10})$, каждое новое число в дополнительном коде вплоть до $0111_2 (+7_{10})$ можно получить из предыдущего, добавляя 1 и игнорируя перепос из 4-го разряда, когда он возникает. Этого нельзя сказать о представлении чисел в прямом коде со знаком и в обратном коде. Поскольку обычное сложение является развитием идеи подсчета, суммирование чисел в *дополнительном коде (two's-complement addition)* можно осуществить путем простого сложения двоичных чисел, пренебрегая переносами из старшего разряда. Результатом всегда будет правильное значение суммы, пока оно не выйдет за допустимые пределы этой числовой системы. Приведем несколько примеров десятичного сложения и сложения двоичных чисел в дополнительном коде, подтверждающих сказанное:

+3	0011	-2	1110
+ +4	+ 0100	+ -6	+ 1010
+7	0111	-8	1 1000
+6	0110	+4	0100
+ -3	+ 1101	+ -7	+ 1001
+3	1 0011	-3	1101

Табл. 2.6. Десятичные и 4-разрядные двоичные числа

Десятичные числа	Двоичные числа			
	в дополнительном коде	в обратном коде	в прямом коде со знаком	в коде с избытком 2^{n-1}
-8	1000	—	—	0000
-7	1001	1000	1111	0001
-6	1010	1001	1110	0010
-5	1011	1010	1101	0011
-4	1100	1011	1100	0100
-3	1101	1100	1011	0101
-2	1110	1101	1010	0110
-1	1111	1110	1001	0111
0	0000	1111 или 0000	1000 или 0000	1000
1	0001	0001	0001	1001
2	0010	0010	0010	1010
3	0011	0011	0011	1011
4	0100	0100	0100	1100
5	0101	0101	0101	1101
6	0110	0110	0110	1110
7	0111	0111	0111	1111

2.6.2. Графическая интерпретация

На рис. 2.3 двоичная система чисел в дополнительном коде изображена иначе с помощью 4-разрядного «счетчика». Здесь числа показаны в циклической форме или в форме представления «по модулю». Этот счетчик действует наподобие реального реверсивного счетчика, который будет рассмотрен в параграфе 8.4. Начиная с произвольного числа, на которое указывает стрелка, мы можем сложить его с $+n$, добавляя n раз по единице, то есть повернув указатель на n позиций по

часовой стрелке. Очевидно также, что можно вычесть n из того или иного числа, отнимая n раз по единице, то есть повернув указатель на n позиций против часовой стрелки. Конечно, результат этих действий будет правильным только в том случае, когда n достаточно мало, так что мы не пересекаем разрыв между -8 и $+7$.

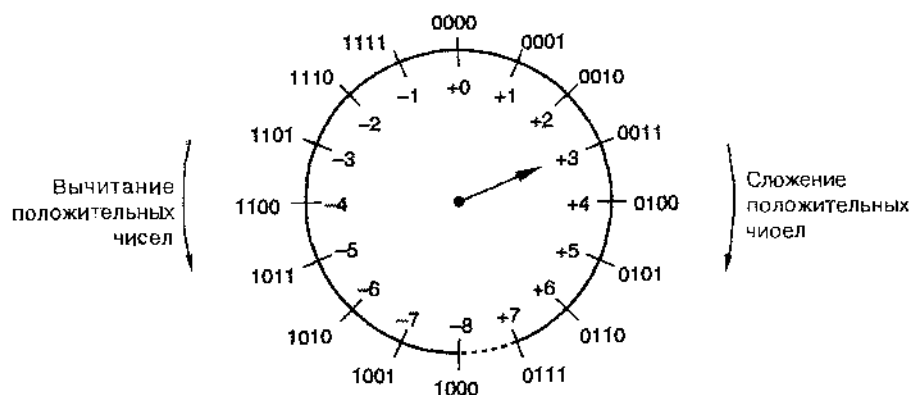


Рис. 2.3. Представление счета «по модулю» для 4-разрядных двоичных чисел в дополнительном коде

Самое интересное здесь заключается в том, что мы можем вычесть n (или прибавить $-n$), повернув указатель на $16 - n$ позиций по часовой стрелке. Обратите внимание: величина $(16 - n)$ — это то, что мы назвали точным дополнением двоичного 4-разрядного числа n , то есть представлением числа $-n$ в дополнительном коде. Эта графическая иллюстрация свидетельствует в пользу утверждения о том, что представление отрицательных чисел в дополнительном коде позволяет складывать их с другими числами по обычным правилам сложения. Прибавление числа эквивалентно повороту указателя на рис. 2.3 на соответствующее число позиций по часовой стрелке.

2.6.3. Переполнение

Если результат выполнения операции сложения выходит за пределы интервала чисел, представимых в данной системе, то говорят, что происходит *переполнение* (*overflow*). В примере на рис. 2.3 переполнение возникает при сложении положительных чисел, когда мы переходим через отметку $+7$. Сложение двух чисел с разными знаками никогда не может привести к переполнению, а сложение двух чисел с одинаковыми знаками может, как это видно из следующих примеров:

$$\begin{array}{rcl}
 \begin{array}{r}
 -3 \quad 1101 \\
 + -6 \quad + 1010 \\
 \hline
 -9 \quad 10111 = +7
 \end{array} & & \begin{array}{r}
 +5 \quad 0101 \\
 + +6 \quad + 0110 \\
 \hline
 +11 \quad 1011 = -5
 \end{array} \\
 \\
 \begin{array}{r}
 -8 \quad 1000 \\
 + -8 \quad + 1000 \\
 \hline
 -16 \quad 10000 = +0
 \end{array} & & \begin{array}{r}
 +7 \quad 0111 \\
 + +7 \quad + 0111 \\
 \hline
 +14 \quad 1110 = -2
 \end{array}
 \end{array}$$

Существует замечательное *правило обнаружения переполнения (overflow rule)* при сложении: сложение приводит к переполнению, если знаки слагаемых одинаковы, а знак суммы отличается от знака слагаемых. Это правило иногда формулируют в терминах переносов, возникающих при выполнении операции сложения: переполнение наступает, если бит переноса в знаковый разряд c_{in} и бит переноса из него c_{out} различны. Внимательное рассмотрение приведенной ранее табл. 2.3 показывает, что эти два правила эквивалентны: имеются только два случая, когда $c_{in} \neq c_{out}$, и это те два случая, в которых $x = y$, а бит суммы имеет другое значение.

2.6.4. Правила вычитания

Числа, представленные в дополнительном коде, могут участвовать в вычитании как обычные двоичные числа без знака, и для них можно сформулировать соответствующие правила обнаружения переполнения. Однако в большинстве случаев схемы, предназначенные для *вычитания чисел в дополнительном коде (two's-complement subtraction)*, не выполняют эту операцию непосредственно. Чаще всего в них осуществляется изменение знака у вычитаемого путем взятия его точного дополнения, а затем оно складывается с уменьшаемым по обычным правилам сложения.

Изменение знака у вычитаемого и сложение с уменьшаемым можно следующим образом выполнить в одной единственной операции сложения: берем поразрядное дополнение вычитаемого и складываем его с уменьшаемым, полагая значение переноса в младший разряд (c_{in}) равным 1, а не 0. Приведем примеры:

$1 \leftarrow c_{in}$			$1 \leftarrow c_{in}$		
+4	0100	0100	+3	0011	0011
- +3	- 0011	+ 1100	- +4	- 0100	+ 1011
+1	-----	10001	-1	-----	1111

$1 \leftarrow c_{in}$			$1 \leftarrow c_{in}$		
+3	0011	0011	-3	1101	1101
- -4	- 1100	+ 0011	- -4	- 1100	+ 0011
+7	-----	0111	+1	-----	10001

Переполнение при вычитании можно обнаружить по тому же правилу, что и при сложении, проверяя знаки уменьшаемого и *обратного кода* вычитаемого. Или, как это видно из приведенных примеров, мы можем сравнить перенос в знаковый разряд с переносом из него: переполнение обнаруживается по тому же правилу, что и при сложении, независимо от знаков исходных чисел, участвующих в операции вычитания, и знака результата.

При попытке изменить знак у «лишнего» отрицательного числа возникает переполнение; согласно сформулированным правилам это произойдет в результате прибавления 1 при взятии точного дополнения:

$$\begin{array}{r}
 -(-8) = -1000 = \quad 0111 \\
 \quad \quad \quad + 0001 \\
 \hline
 \quad \quad \quad 1000 = -8
 \end{array}$$

Однако это число все же может участвовать в сложении и вычитании, пока окончательный результат операции остается внутри диапазона представимых чисел:

$$\begin{array}{rcl}
 +4 & 0100 & \\
 + -8 & + 1000 & \\
 \hline
 -4 & 1100 &
 \end{array}
 \qquad
 \begin{array}{rcl}
 -3 & 1101 & \\
 - -8 & - 1000 & \\
 \hline
 +5 & 0101 &
 \end{array}
 \qquad
 \begin{array}{rcl}
 & 1 & \text{--- } c_m \\
 & 1101 & \\
 + & 0111 & \\
 \hline
 & 10101 &
 \end{array}$$

2.6.5. Дополнительный код и двоичные числа без знака

Поскольку числа, представленные в дополнительном коде, складываются и вычитаются по тем же самым основным алгоритмам двоичного сложения и вычитания, что и числа без знака той же длины, компьютер или какая-либо другая цифровая система могут использовать один и тот же сумматор для обработки чисел обоих типов. Однако результат должен интерпретироваться по-разному в зависимости от того, с чем имеет дело система: с числами со знаком (то есть с числами из интервала от -8 до $+7$; *signed numbers*) или с числами без знака (из интервала от 0 до 15; *unsigned numbers*).

Выше на рис. 2.3 мы представили систему 4-разрядных двоичных чисел в дополнительном коде графически. Подобные построения можно выполнять и для 4-разрядных двоичных чисел без знака, как это сделано на рис. 2.4. Комбинации битов расположены в тех же местах по окружности, а прибавление и отнимание того или иного числа, как и ранее, достигается поворотом указателя на соответствующее число позиций по часовой стрелке или против нее.

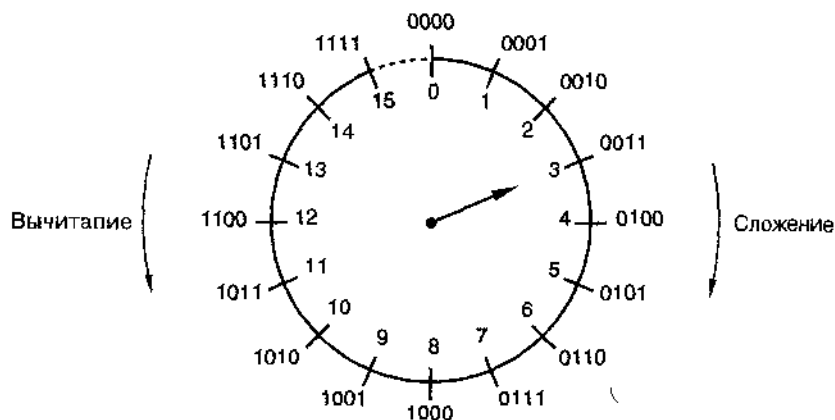


Рис. 2.4. Представление счета «по модулю» для 4-разрядных двоичных чисел без знака

Легко видеть, что выход за пределы диапазона представимых чисел при сложении происходит в том случае, когда указатель, вращаясь по часовой стрелке, проходит через разрыв между 15 и 0. Говорят, что при этом возникает *перенос* (*carry*) из старшего разряда.

Аналогично, при вычитании мы выходим за пределы диапазона представимых чисел, если указатель в результате вращения против часовой стрелки проходит

через этот разрыв. В этом случае говорят, что в старшем разряде возникла потребность заема (*borrow*).

Из рис. 2.4 следует также, что вычитание числа без знака, равного n , можно осуществить, повернув указатель по часовой стрелке на $16 - n$ позиций. Это эквивалентно сложению с точным дополнением числа n . При вычитании заем возникает, если соответствующее сложение с точным дополнением не приводит к переносу.

Таким образом, переполнение или заем, возникающие при сложении чисел без знака, указывают на то, что результат не укладывается в диапазон представимых чисел. При сложении чисел со знаком в дополнительном коде на тот же факт указывает обсуждавшееся нами выше переполнение. Думать о переносе при сложении чисел со знаком излишне в том смысле, что переполнение может наступать или не наступать независимо от того, происходит перенос или нет.

*2.7. Сложение и вычитание двоичных чисел в обратном коде

Взглянем еще раз на табл. 2.6; это поможет нам объяснить правило сложения двоичных чисел, представленных в обратном коде. Если начать с числа $1000_2 (-7_{10})$ и считать в прямом направлении, то каждое следующее число будет получаться из предыдущего в результате добавления 1, за исключением перехода от 1111_2 (отрицательный 0) к $0001_2 (+1_{10})$. При достижении числа 1111_2 мы должны для продолжения правильного счета прибавить 2, а не 1. Из этого следует алгоритм сложения двоичных чисел в обратном коде: следует выполнять стандартное двоичное сложение, но добавлять еще одну 1 при переходе через 1111_2 .

Достижение значения 1111_2 при сложении можно обнаружить, следя за переполнением из знакового разряда. Таким образом, правило сложения двоичных чисел в обратном коде (*ones'-complement addition*) можно сформулировать совсем просто:

- Выполняется стандартное двоичное сложение; если возникает перенос из знакового разряда, то к результату прибавляется 1.

Это правило часто называют *циклическим переносом* (*end-around carry*). Ниже приведены примеры сложения в обратном коде; в последних трех примерах производится циклический перенос:

+3 0011	+4 0100	+5 0101
+ +4 + 0100	+ -7 + 1000	+ -5 + 1010
+7 0111	-3 1100	-0 1111
-2 1101	+6 0110	-0 1111
+ -5 + 1010	+ -3 + 1100	+ -0 + 1111
-7 1011	+3 1001	-0 1110
+ 1	+ 1	+ 1
1000	0011	1111

Если следовать сформулированному алгоритму, осуществляемому в два шага, то в результате сложения числа с его обратным кодом получается отрицательный

0. Действительно, процедура сложения, предусматриваемая этим правилом, никогда не может привести к положительному 0, если только оба слагаемых не равны положительному 0.

Как и в случае с числами, представленными в дополнительном коде, простейший способ реализации *вычитания чисел в обратном коде (ones'-complement subtraction)* состоит в том, чтобы взять дополнение вычитаемого числа и сложить его с уменьшаемым. Правила переполнения для сложения и вычитания в обратном коде — те же самые, что и в случае дополнительного кода.

Все правила преобразования двоичных чисел в числа с обратным знаком, их сложения и вычитания, представленные в этом разделе и ранее, сведены в табл. 2.7.

Табл. 2.7. Сводка правил сложения и вычитания двоичных чисел

Представление чисел	Правила сложения	Правила преобразования чисел в числа с обратным знаком	Правила вычитания
Без знака	Сложить числа. Результат оказывается вне диапазона представимых чисел, если возникает перенос из старшего разряда.	Не применяется.	Отнять вычитаемое от уменьшаемого. Результат оказывается вне диапазона представимых чисел, если в старшем разряде возникает заем.
В прямом коде со знаком	При одинаковых знаках: сложить величины; при появлении переноса из старшего разряда происходит переполнение; результат имеет тот же знак. При разных знаках: вычесть меньшую величину из большей; переполнения быть не может; результат имеет знак большего по величине числа.	Изменить знаковый бит числа.	Изменить знаковый бит вычитаемого и произвести сложение.
В дополнительном коде	Сложить, игнорируя возможный перенос из старшего разряда. Переполнение наступает в том случае, когда переносы в старший разряд и из него различны.	Все биты числа изменить на противоположные; к результату прибавить 1.	Изменить на противоположные все биты вычитаемого и сложить с уменьшаемым, полагая перенос в младший разряд равным 1.
В обратном коде	Сложить; если возникает перенос из старшего разряда, то к результату добавить 1. Переполнение наступает в том случае, когда переносы в старший разряд и из него различны.	Все биты числа изменить на противоположные.	Все биты вычитаемого изменить на противоположные и осуществить сложение.

*2.8. Двоичное умножение

В начальной школе учат *перемножать* путем сдвига множимого, умножения его на значение соответствующего разряда множителя и сложения (*shift-and-add multiplication*) всех полученных таким образом чисел. Тот же принцип можно применить для нахождения *произведения* двух двоичных чисел без знака (*unsigned binary multiplication*). При двоичном умножении образование подлежащих сложению сдвинутых чисел тривиально, так как в каждом из разрядов множителя возможны только два значения: 0 или 1. Приведем примеры:

11	1011	множимое
× 13	× 1101	множитель
— 33	— 1011	
11	0000	сдвинутые
— 143	— 1011	множимые
	— 1011	
	10001111	произведение

В цифровой системе удобнее не формировать вначале все сдвинутые копии множимого и затем их складывать, а при каждом шаге прибавлять очередное сдвинутое множимое к *частичному произведению* (*partial product*). При реализации этого метода в предыдущем примере перемножения 4-разрядных двоичных чисел понадобятся четыре сложения и возникнут четыре частичных произведения:

11	1011	множимое
× 13	× 1101	множитель
—	0000	частичное произведение
	— 1011	сдвинутое множимое
	01011	частичное произведение
	— 0000↓	сдвинутое множимое
	001011	частичное произведение
	— 1011↓↓	сдвинутое множимое
	0110111	частичное произведение
	— 1011↓↓↓	сдвинутое множимое
	10001111	произведение

В общем случае умножения n -разрядного двоичного числа на m -разрядное двоичное число для представления произведения необходимы самое большее $n+m$ битов. В алгоритме сдвига и сложения для получения результата требуется вычислить m частичных произведений и выполнить m сложений, но первое сложение тривиально, поскольку начальное частичное произведение равно нулю. Хотя это первое частичное произведение состоит всего лишь из n значащих битов, после каждого шага сложения длина частичного произведения увеличивается на один значащий бит, так как при каждом сложении может возникнуть перенос. В то же время на каждом шаге еще один бит частичного произведения, начиная с крайнего

правого бита и двигаясь влево, будет оставаться неизменным. В разделе 8.7.2 показано, что алгоритм сдвига и сложения можно реализовать в цифровой схеме, состоящей из регистра сдвига, сумматора и управляющей логики.

Умножение чисел со знаком (*signed multiplication*) можно осуществить с помощью умножения чисел без знака, применяя арифметические правила: выполняется умножение без знака величин и произведению присваивается знак «плюс», если сомножители одного знака, и знак «минус», если их знаки различны. Это очень удобно делать при представлении чисел в прямом коде со знаком, поскольку знак и величина при этом разделены.

При представлении чисел в дополнительном коде выделение величины отрицательного числа и изменение знака у произведения, полученного по правилам перемножения чисел без знака, являются не совсем простыми операциями. Это подталкивает нас к поиску более эффективного способа умножения чисел в дополнительном коде (*two's-complement multiplication*), о чем сейчас и пойдет речь.

В принципе, перемножение чисел без знака сводится к последовательности сложений сдвинутых множимых по правилам сложения чисел без знака; на каждом шаге сдвиг множимого соответствует весу бита множителя. У чисел в дополнительном коде биты имеют те же самые веса, что и у чисел без знака, за исключением старшего бита, вес которого отрицателен (см. раздел 2.5.4). Таким образом, умножение чисел в дополнительном коде можно выполнить как последовательность сложений сдвинутых множимых по правилам сложения чисел в дополнительном коде. Исключение составляет последний шаг, на котором сдвинутое множимое, соответствующее старшему разряду множителя, должно быть взято с обратным знаком, прежде чем оно будет добавлено к частичному произведению. Ниже повторен наш предыдущий пример, только теперь множимое и множитель интерпретируются как числа в дополнительном коде:

-5	1011	множимое
× -3	× 1101	множитель
	00000	частичное произведение
	11011	сдвинутое множимое с обратным знаком
	111011	частичное произведение
	00000↓	сдвинутое множимое
	1111011	частичное произведение
	11011↓↓	сдвинутое множимое
	11100111	частичное произведение
	00101↓↓↓	сдвинутое множимое
	00001111	произведение

Обработка старшего разряда производится несколько хитрее, так как на каждом шаге мы получаем еще один значащий бит и имеем дело с числами со знаком. Поэтому перед сложением каждого сдвинутого множимого с k -разрядным частичным произведением мы увеличиваем число значащих битов в каждом из слагаемых до $k+1$ по правилу знакового расширения. В результате каждая сумма состоит из $k+1$ битов; перенос из старшего разряда $(k+1)$ -разрядной суммы игнорируется.

*2.9. Двоичное деление

Простейший алгоритм двоичного деления основан на методе *деления путем сдвига и вычитания* (*shift-and-subtract division*), которому учат в начальной школе. В табл. 2.8 приведены примеры применения этого метода при делении десятичных и двоичных чисел без знака (*unsigned division*). В обоих случаях мы мысленно сравниваем приведенное делимое с величинами, кратными делителю, для определения того, какое именно кратное сдвинутого делителя надо вычесть. В случае десятичных чисел мы сначала выбираем значение 11 как наибольшее кратное, меньшее 21, а затем берем 99 как наибольшее кратное, меньшее 107. В двоичном случае выбор несколько вроще, поскольку выбрать приходится всего лишь между нулем и самим делителем.

Табл. 2.8. Пример деления столбиком

19	10011	частное
11 $\overline{)217}$	1011 $\overline{)11011001}$	делимое
11	1011	сдвинутый делитель
107	0101	приведенное делимое
99	0000	сдвинутый делитель
8	1010	приведенное делимое
	0000	сдвинутый делитель
	10100	приведенное делимое
	1011	сдвинутый делитель
	10011	приведенное делимое
	1011	сдвинутый делитель
	1000	остаток

Методы деления двоичных чисел представляют собой нечто дополнительное по отношению к методам двоичного умножения. Типичный алгоритм деления имеет дело с $(n+m)$ -разрядным делимым и n -разрядным делителем; результатом его действия становятся m -разрядное частное и n -разрядный остаток. *Переполнение при делении* (*division overflow*) наступает, если делитель равен нулю или в том случае, когда для представления частного потребовалось бы больше, чем m битов. У большинства компьютеров в блоках, выполняющих деление, принято $n = m$.

Деление чисел со знаком (*signed division*) можно выполнить, используя алгоритм деления чисел без знака, по обычным школьным правилам: осуществляем деление величин без знака и полагаем частное положительным, когда операнды имеют одинаковые знаки, и отрицательным, если их знаки различны. Остаток имеет тот же знак, что и делимое. Как и в случае умножения, существуют специальные приемы непосредственного выполнения деления чисел, представленных в дополнительном коде; часто именно такие приемы реализуются в блоках деления в компьютерах (см. Обзор литературы).

2.10. Двоичные коды десятичных чисел

Несмотря на то, что двоичные числа являются наиболее подходящими для вычислений в цифровой системе, большинство людей все еще предпочитают иметь дело с десятичными числами. В результате внешние интерфейсы цифровой системы индицируют и позволяют считывать десятичные числа, а некоторые цифровые устройства и в самом деле оперируют с десятичными числами.

Необходимость представления человеку десятичных чисел не приводит к изменению основного принципа работы цифровых электронных схем: они обрабатывают все же сигналы, принимающие лишь одно из двух значений, которые мы называем 0 и 1. Поэтому десятичное число представляется в цифровой системе строкой битов, различные комбинации значений битов в этой строке служат выражением различных десятичных чисел. Если, например, использовать строку из 4 битов для представления десятичного числа, то комбинацию битов 0000 можно поставить в соответствие десятичной цифре 0, комбинацию 0001 – цифре 1, комбинацию 0010 – цифре 2 и т. д.

Набор строк, состоящих из n битов, в котором различные строки представляют различные числа или другие объекты, называется *кодом* (*code*). Отдельная комбинация из n двоичных цифр носит название *кодového слова* (*code word*). Как мы увидим на примерах десятичных кодов в этом параграфе, между значениями битов в кодовом слове и тем, выражением чего является это слово, может существовать арифметическое соотношение, но может и не существовать. Кроме того, код с n -битовыми строками не обязательно содержит 2^n возможных кодовых слов.

Для представления одной десятичной цифры нужно, по меньшей мере, четыре бита. Можно миллиардами различных способов выбрать десять 4-разрядных кодовых слов, но наиболее употребительными являются десятичные коды, перечисленные в табл. 2.9.

БИНОМИАЛЬНЫЕ КОЭФФИЦИЕНТЫ

Число различных способов выбрать m элементов из n выражается *биномиальным коэффициентом* $\binom{n}{m}$, равным $\frac{n!}{m!(n-m)!}$. Существует $\binom{10}{16}$ различных способов выбрать 10 четырехразрядных кодовых слов из 16 и 10! способов установить соответствие между каждым из выбранных наборов n десяти цифрами. Следовательно, имеется $\frac{16!}{10! 6!} \cdot 10!$ или 29 059 430 400 различных 4-разрядных десятичных кодов.

Возможно, самым «естественным» десятичным кодом являются *двоично-десятичные числа* (*binary-coded decimal, BCD*); в этом коде цифрам от 0 до 9 соответствуют 4-разрядные двоичные эквиваленты без знака от 0000 до 1001. Кодовые слова от 1010 до 1111 не используются. Преобразование десятичных чисел в двоично-десятичные тривиально и состоит в простой подстановке четырех битов на место каждой десятичной цифры; аналогично выполняется и обратное преобразование. В ряде

компьютерных программ один 8-разрядный байт содержит два двоично-десятичных числа; такие двоично-десятичные числа называют упакованными (*packed-BCD representation*). Таким образом, двоично-десятичное число, представленное одним байтом, может иметь значение от 0 до 99 в противоположность обычному 8-разрядному двоичному числу без знака из интервала от 0 до 255. Двоично-десятичные числа с любым количеством цифр можно представить последовательностью байтов из расчета по одному байту на каждые две цифры.

Табл. 2.9. Десятичные коды

Десятичная цифра	Двоично-десятичный код (8421)	Код 2421	Код с избытком 3	Двоично-пятнадцатеричный код	Код «1 из 10»
0	0000	0000	0011	0100001	1000000000
1	0001	0001	0100	0100010	0100000000
2	0010	0010	0101	0100100	0010000000
3	0011	0011	0110	0101000	0001000000
4	0100	0100	0111	0110000	0000100000
5	0101	1011	1000	1000001	0000010000
6	0110	1100	1001	1000010	0000001000
7	0111	1101	1010	1000100	0000000100
8	1000	1110	1011	1001000	0000000010
9	1001	1111	1100	1010000	0000000001
Неиспользуемые кодовые слова					
	1010	0101	0000	0000000	0000000000
	1011	0110	0001	0000001	0000000011
	1100	0111	0010	0000010	0000000101
	1101	1000	1101	0000011	0000000110
	1110	1001	1110	0000101	0000000111
	1111	1010	1111		

Как и в случае двоичных чисел, можно многими способами представлять отрицательные двоично-десятичные числа. У двоично-десятичных чисел со знаком должен быть один лишний разряд для знака. Наиболее распространенными являются представления в прямом коде со знаком и в дополнительном десятичном коде. При представлении двоично-десятичных чисел в прямом коде со знаком последовательность битов для знака произвольна; при представлении в дополнительном коде набором битов 0000 указывается знак «плюс», а набором 1001 – знак «минус».

Сложение двоично-десятичных чисел (*BCD addition*) в одном разряде подобно сложению 4-разрядных двоичных чисел без знака, за исключением необходимости производить коррекцию, когда результат превышает 1001. Коррекция результата осуществляется добавлением 6; приведем примеры.

$$\begin{array}{r}
 5 \quad 0101 \\
 + 9 \quad + 1001 \\
 \hline
 14 \quad 1110 \\
 + 0110 \quad - \text{коррекция} \\
 \hline
 10 + 4 \quad 10100
 \end{array}$$

$$\begin{array}{r}
 8 \quad 1000 \\
 + 8 \quad + 1000 \\
 \hline
 16 \quad 10000 \\
 + 0110 \quad \text{коррекция} \\
 \hline
 10 + 6 \quad 10110
 \end{array}$$

$$\begin{array}{r}
 4 \quad 0100 \\
 + 5 \quad + 0101 \\
 \hline
 9 \quad 1001
 \end{array}$$

$$\begin{array}{r}
 9 \quad 1001 \\
 + 9 \quad + 1001 \\
 \hline
 18 \quad 10010 \\
 + 0110 \quad - \text{коррекция} \\
 \hline
 10 + 8 \quad 11000
 \end{array}$$

Отметим, что при сложении двух цифр в двоично-десятичном коде перенос в следующий разряд происходит в том случае, когда он возникает непосредственно при двоичном сложении, либо при осуществлении коррекции. Во многих компьютерах для выполнения арифметических действий с упакованными двоично-десятичными числами имеются специальные команды, которые автоматически производят коррекцию и учитывают перенос.

Двоично-десятичный код является *взвешенным кодом (weighted code)*, поскольку каждую десятичную цифру можно получить из ее кодового слова, приписывая отдельным битам этого слова определенный вес. Для двоично-десятичных чисел веса битов имеют следующие значения: 8, 4, 2 и 1; по этой причине этот код иногда называют *кодом 8421 (8421 code)*. Другой набор весов дает *код 2421 (2421 code)*, приведенный в табл. 2.9. Достоинство этого кода состоит в том, что он является *самодополняющим кодом (self-complementing code)*; это означает, что кодовое слово любой цифры в обратном десятичном коде можно получить, беря обратные значения отдельных битов в кодовом слове данной цифры.

Другим самодополняющим кодом в табл. 2.9 является *код с избытком 3 (excess-3 code)*. Хотя это не взвешенный код, все же имеется арифметическая связь между ним и двоично-десятичным кодом: в коде с избытком 3 кодовое слово каждой десятичной цифры равно соответствующему кодовому слову двоично-десятичного кода плюс 0011₂. Поскольку кодовые слова в коде с избытком 3 следуют одно за другим в естественной двоичной последовательности, легко строить стандартные двоичные счетчики для счета в этом коде, как это будет продемонстрировано на рис. 8.37.

У десятичных кодов может быть больше четырех битов; например, слова *двоично-пятеричного кода (biquinary code)* состоят из семи битов (см. табл. 2.9). Первые два бита в кодовом слове показывают, какому из двух интервалов принадлежит число: от 0 до 4 или от 5 до 9; последние пять битов указывают, какое именно число из пяти чисел данного интервала представлено этим словом.

Потенциальным достоинством использования в коде большего числа битов, чем это минимально необходимо, является возможность обнаружения ошибок. При случайном изменении любого одного бита в кодовом слове двоично-пятеричного кода получается кодовое слово, не представляющее никакой десятичной цифры, поэтому такое слово можно отметить как ошибочное. Из 128 возмож-

ных 7-разрядных двоичных слов только 10 являются собственно кодовыми словами и распознаются как десятичные цифры; все остальные можно считать ошибочными, если они возникают.

В последнем столбце табл. 2.9 приведен пример самого разреженного кода для десятичных цифр, называемого кодом «1 из 10» (*1-out-of-10 code*): в нем используются 10 из 1024 возможных 10-разрядных двоичных кодовых слов.

2.11. Код Грея

При использовании цифровой электроники в таких электромеханических системах, как механические станки, тормозные системы автомобилей и копировальные устройства, бывают нужны датчики положения в пространстве, вырабатывающие цифровые сигналы. На рис. 2.5 схематически изображен кодирующий диск с набором контактов, позволяющих ему создавать одну из восьми 3-разрядных двоичных комбинаций в зависимости от угла поворота диска. Затемненные участки диска подсоединены к источнику сигнала, соответствующего логической 1, а светлые участки ни к чему не подсоединены, и поэтому сигнал, возникающий при контакте со светлым участком, интерпретируется как логический 0.

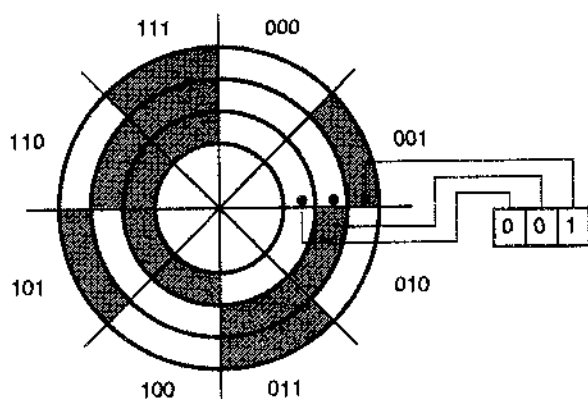


Рис. 2.5. Механический кодирующий диск, вырабатывающий 3-разрядный двоичный код

У кодирующего устройства, изображенного на рис. 2.5, возникают определенные проблемы при снятии сигналов вблизи границ между некоторыми областями. Рассмотрим, например, границу между областями диска 001 и 010, на этой границе изменяются два бита в коде. Какое значение будет вырабатывать кодер, если диск повернуть так, что контакты приходятся точно на теоретическую границу между этими областями? Поскольку мы находимся на границе, оба значения 001 и 010 приемлемы. Однако из-за того, что механический узел не совершенен, два правых контакта могут одновременно касаться областей «1», порождая ошибочное значение кода 011. Точно так же на выходе кодера возможна комбинация 000. В общем случае такого рода проблема может возникнуть на такой границе, где меняется больше, чем один бит. Худший случай наступает тогда, когда изменяются все три бита, как это происходит на стыках 000–111 и 011–100.

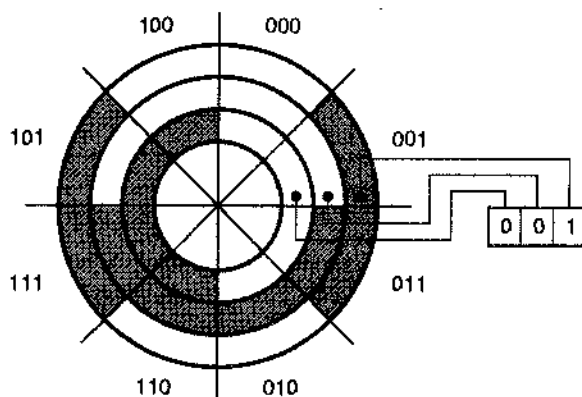
Проблему кодирующего диска можно решить, придумав цифровой код, у которого в каждой паре последовательных кодовых слов изменяется только один бит.

Такой код называется *кодом Грея* (*Gray code*); в табл. 2.10 приведен 3-разрядный код Грея. Сконструируем заново кодирующий диск согласно этому коду, как показано на рис. 2.6. На новом диске только один бит изменяется на каждой границе, так что вырабатываемые датчиком кодовые слова соответствуют значениям по одну или другую сторону границы.

Табл. 2.10. Сравнение 3-разрядного двоичного кода и кода Грея

Десятичное число	Двоичный код	Код Грея
0	000	000
1	001	001
2	010	011
3	011	010
4	100	110
5	101	111
6	110	101
7	111	100

Рис. 2.6. Механический кодирующий диск, вырабатывающий 3-разрядный код Грея



Существует два удобных способа строить код Грея с любым желаемым числом битов. Первый метод основан на том, что код Грея является *рефлексным* (*reflected code*); его можно определить (и построить) рекурсивно по следующим правилам:

1. В однобитовом коде Грея два кодовых слова: 0 и 1.
2. Первые 2^n кодовых слов $(n+1)$ -разрядного кода Грея равны кодовым словам n -разрядного кода Грея, записанным по порядку и дополненным 0 в старшем разряде.
3. Последние 2^n кодовых слов $(n+1)$ -разрядного кода Грея равны кодовым словам n -разрядного кода Грея, но записанным в обратном порядке и дополненным 1 в старшем разряде.

Если провести черту между 3 и 4 строками в табл. 2.10, то можно видеть, что для 3-разрядного кода Грея справедливы правила 2 и 3. Чтобы построить этим методом

n -разрядный код Грея для произвольного значения n , необходимо, конечно, построить также коды Грея для всех длин кодовых слов, меньших n .

Второй метод позволяет получать n -разрядное кодовое слово кода Грея непосредственно из соответствующего кодового слова n -разрядного двоичного кода:

1. Биты в n -разрядном кодовом слове двоичного кода для кода Грея нумеруются справа налево от 0 до $n-1$.
2. i -й бит в кодовом слове кода Грея равен 0, если i -й и $(i+1)$ -й биты в соответствующем слове двоичного кода одинаковы; в противном случае i -й бит равен 1. (Если $i+1=n$, то n -й бит в слове двоичного кода принимается равным 0.)

Обратившись снова к табл. 2.10, мы видим, что для 3-разрядного кода Грея эти правила выполнены.

*2.12. Коды символов

Как следует из предыдущего параграфа, строка битов не обязательно представляет собой число; в самом деле, большая часть информации, обрабатываемой компьютерами, не имеет числовой природы. Наиболее распространенным типом нечисловых данных является *текст (text)*, то есть последовательность символов, выбираемых из некоторого множества. Каждый символ представлен в компьютере строкой битов в соответствии с принятым соглашением.

Чаще всего в качестве кода символов используется код *ASCII (American Standard Code for Information Interchange, Американский стандартный код для обмена информацией; аббревиатура ASCII произносится так: ASS key)*. В коде ASCII каждый символ представлен строкой из 7 битов, что дает в целом 128 различных символов, приведенных в табл. 2.11. Этот код содержит строчные и прописные буквы алфавита, числа, знаки пунктуации и различные не выводимые на печать управляющие символы. Таким образом, строка текста «Yesch!» выглядит как довольно безобидный набор из 7-разрядных двоичных чисел:

1011001 1100101 1100011 1100011 1100011 1101000 0100001.

2.13. Коды действий, условий и состояний

В общем случае коды, рассматривавшиеся до сих пор, применяются для представления чего-то такого, что мы могли бы считать «данными», такими как числа, указатели положения в пространстве и символы. Программистам известно, что в одной компьютерной программе могут присутствовать данные многих типов.

При проектировании цифровых систем мы часто встречаемся с приложениями, когда строка битов не носит характера данных, а должна служить для управления действием, для сигнализации о выполнении условия или для представления текущего состояния аппаратуры. Чаще всего, но-видимому, в таких приложениях применяется простой двоичный код. Если имеется n различных действий, условий или состояний, то их можно представить двоичным кодом с числом битов в слове, равным $b = \lceil \log_2 n \rceil$. [Скобки $\lceil \cdot \rceil$ означают *наименьшее целое, превосходящее величину*, заключенную в скобки, или равное этой величине (*ceiling function*). Таким образом, b — это наименьшее целое, такое что $2^b \geq n$.]

Табл. 2.11. Код ASCII. Стандарт X3.4-1968 Американского национального института стандартов

$b_7b_6b_5b_4$	Строка в 16-ичном коде	$b_3b_2b_1b_0$ (столбец)							
		000	001	010	011	100	101	110	111
		0	1	2	3	4	5	6	7
0000	0	NUL	DLE	SP	0	@	P	'	p
0001	1	SOH	DC1	!	1	A	Q	a	q
0010	2	STX	DC2	"	2	B	R	b	r
0011	3	ETX	DC3	#	3	C	S	c	s
0100	4	EOT	DC4	\$	4	D	T	d	t
0101	5	ENQ	NAK	%	5	E	U	e	u
0110	6	ACK	SYN	&	6	F	V	f	v
0111	7	BEL	ETB	'	7	G	W	g	w
1000	8	BS	CAN	(	8	H	X	h	x
1001	9	HT	EM	)	9	I	Y	i	y
1010	A	LF	SUB	*	:	J	Z	j	z
1011	B	VT	ESC	+	;	K	[	k	{
1100	C	FF	FS	,	<	L	\	l	
1101	D	CR	GS	-	=	M	]	m	}
1110	E	SO	RS	.	>	N	^	n	~
1111	F	SI	US	/	?	O	_	o	DEL

Управляющие коды

NUL	Null	Пустой символ	DLE	Data link escape	Смена канала данных
SOH	Start of heading	Начало заголовка	DC1	Device control 1	Управление устройством 1
STX	Start of text	Начало текста	DC2	Device control 2	Управление устройством 2
ETX	End of text	Конец текста	DC3	Device control 3	Управление устройством 3
EOT	End of transmission	Конец передачи	DC4	Device control 4	Управление устройством 4
ENQ	Enquiry	Запрос	NAK	Negative acknowledge	Отрицательное квитирование
ACK	Acknowledge	Подтверждение	SYN	Synchronize	Синхронизация
BEL	Bell	Звонок	ETB	End transmitted block	Конец передачи блока
BS	Backspace	Возврат	CAN	Cancel	Аннулирование
HT	Horizontal tab	Горизонтальная табуляция	EM	End of medium	Конец носителя
LF	Line feed	Перевод строки	SUB	Substitute	Замена
VT	Vertical tab	Вертикальная табуляция	ESC	Escape	Переход
FF	Form feed	Переход на новую страницу	FS	File separator	Разделитель файлов
CR	Carriage return	Возврат каретки	GS	Group separator	Разделитель групп
SO	Shift out	Сдвиг с потерей данных	RS	Record separator	Разделитель записей
SI	Shift in	Сдвиг с внесением данных	US	Unit separator	Разделитель элементов
SP	Space	Пробел	DEL	Delete or rubout	Уничтожить, стереть

Рассмотрим в качестве примера контроллер простого светофора. Сигналы светофора на пересечении улиц, идущих с севера на юг (N-S) и с востока на запад (E-W), могут образовывать любую из шести возможных комбинаций, перечисленных в табл. 2.12. Эти состояния светофора можно закодировать тремя битами, как показано в последнем столбце таблицы. Используются только шесть из восьми возможных 3-разрядных кодовых слов, и соответствие между шестью выбранными кодовыми словами и состояниями светофора произвольно, так что кодирование можно осуществить многими способами. Опытный разработчик цифровых устройств выбирает способ кодирования так, чтобы минимизировать стоимость схемы или оптимизировать какой-нибудь другой параметр (например, время разработки: нет необходимости перебирать миллиарды возможных вариантов кодирования).

Табл. 2.12. Состояния контроллера светофора

Состояние	Сигналы светофора						Кодовое слово
	N-S	N-S	N-S	E-W	E-W	E-W	
	Зеленый	Желтый	Красный	Зеленый	Желтый	Красный	
N-S проезд разрешен	Вкл.	выкл.	выкл.	выкл.	выкл.	Вкл.	000
N-S приготовиться	выкл.	Вкл.	выкл.	выкл.	выкл.	Вкл.	001
N-S проезд запрещен	выкл.	выкл.	Вкл.	выкл.	выкл.	Вкл.	010
E-W проезд разрешен	выкл.	выкл.	Вкл.	Вкл.	выкл.	выкл.	100
E-W приготовиться	выкл.	выкл.	Вкл.	выкл.	Вкл.	выкл.	101
E-W проезд запрещен	выкл.	выкл.	Вкл.	выкл.	выкл.	Вкл.	110

Другое применение двоичного кода иллюстрирует рис. 2.7(а). В изображенной здесь системе имеется n устройств, каждое из которых может выполнять определенное действие. Характеристики этих устройств таковы, что они могут быть доступны для управления ими только поодиночке. Блок управления вырабатывает слово двоичного кода «выбор устройства», состоящее из $b = \lceil \log_2 n \rceil$ битов, для того чтобы указать, к какому устройству он обращается в тот или иной момент времени. Кодовое слово «выбор устройства» поступает на все устройства, каждое из которых сравнивает его со своим собственным «удостоверением личности», чтобы определить, происходит ли обращение именно к нему. Хотя число битов в кодовых словах двоичного кода минимально, выбор двоичного кода не всегда является лучшим для кодирования действий, условий или состояний. На рис. 2.7(б) показано, как можно управлять n устройствами с помощью n -разрядного кода «1 из n » (*1-out-of- n code*), в словах которого только один бит равен 1, а остальные равны 0. Каждый бит в слове кода «1 из n » непосредственно подается на вход разрешения соответствующего устройства. Это упрощает конструкцию устройств, поскольку им больше не нужно распознавать обращение к себе; все, что нужно устройству, — это одноразрядный вход «разрешения».

Кодовые слова кода «1 из 10» были приведены в табл. 2.9. Иногда слово, состоящее из всех нулей, также может входить в код «1 из 10», чтобы указывать, что никакое устройство не выбирается. Другим употребительным кодом является об-

ратный код «1 из n » (*inverted 1-out-of- n code*): входящие в него кодовые слова имеют один нулевой бит, а все остальные биты равны 1.

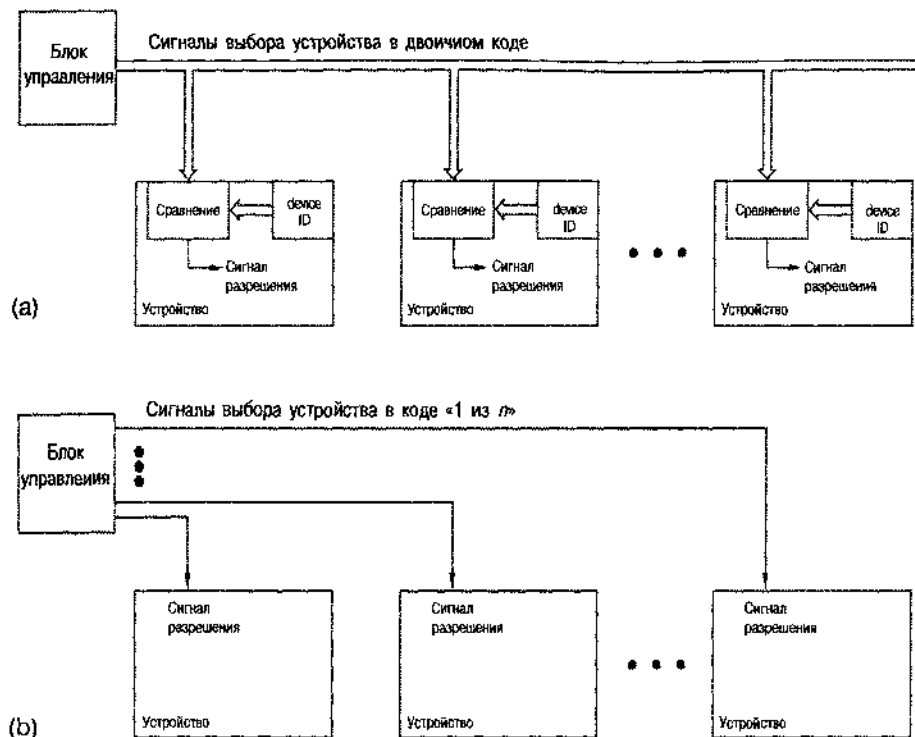


Рис. 2.7. Схема управления для цифровой системы с n устройствами: (a) с использованием двоичного кода (device ID – идентификатор устройства); (b) с использованием кода «1 из n »

В сложных системах может применяться комбинация различных методов кодирования. Рассмотрим, например, систему, подобную приведенной на рис. 2.7(b), в которой каждое из n устройств содержит до s узлов. В этом случае кодовое слово, вырабатываемое блоком управления, может состоять из двух полей: поля выбора устройства, формируемого по принципу «1 из n », и состоящего из $b = \lceil \log_2 s \rceil$ битов поля выбора одного из s узлов внутри данного устройства, формируемого по правилам двоичного кода.

Обобщением кода «1 из n » является код « m из n » (*m -out-of- n code*), в кодовых словах которого m битов равны 1, а все остальные биты равны 0. Слово кода « m из n » можно распознать с помощью логической схемы И с m входами, на выходе которой единичное значение возникает только тогда, когда на все ее входы поданы 1. Сделать это довольно просто и недорого, однако в большинстве случаев значения m таковы, что число кодовых слов в коде « m из n » много больше, чем число слов в коде «1 из n ». Полное число кодовых слов выражается биномиальным коэффициентом $\binom{n}{m}$, равным $\frac{n!}{m!(n-m)!}$. Таким образом, код «2 из 4» состоит из 6 кодовых слов, а код «3 из 10» содержит 120 слов.

Важным частным случаем кода « m из n » является код 8B10B (8B10B code), используемый в гигабитном стандарте Ethernet 802.3z. В этом коде имеется 256 десятиразрядных двоичных кодовых слов, каждое из которых может служить для представления данных, выражаемых 8-ю битами. Большая часть кодовых слов является кодом «5 из 10». Однако, поскольку $\binom{10}{5}$ равно всего лишь 252, используются также некоторые слова кодов «4 из 10» и «6 из 10», дополняющие код до нужного числа кодовых слов, причем осуществлено это весьма интересным способом, о чем подробнее будет рассказано в разделе 2.16.2.

*2.14. n -мерные кубы и расстояние

Строку из n битов можно интерпретировать геометрически как вершину объекта, называемого n -мерным кубом (n -cube). На рис. 2.8 представлены n -мерные кубы для $n = 1, 2, 3, 4$. У n -мерного куба имеется 2^n вершин, каждая из которых отмечена строкой из n битов. Изображенные на рисунке ребра показывают, что у каждой вершины есть n смежных с нею вершин, метки которых отличаются от метки данной вершины только одним битом. Нарисовать n -мерный куб для n больше 4 весьма затруднительно.

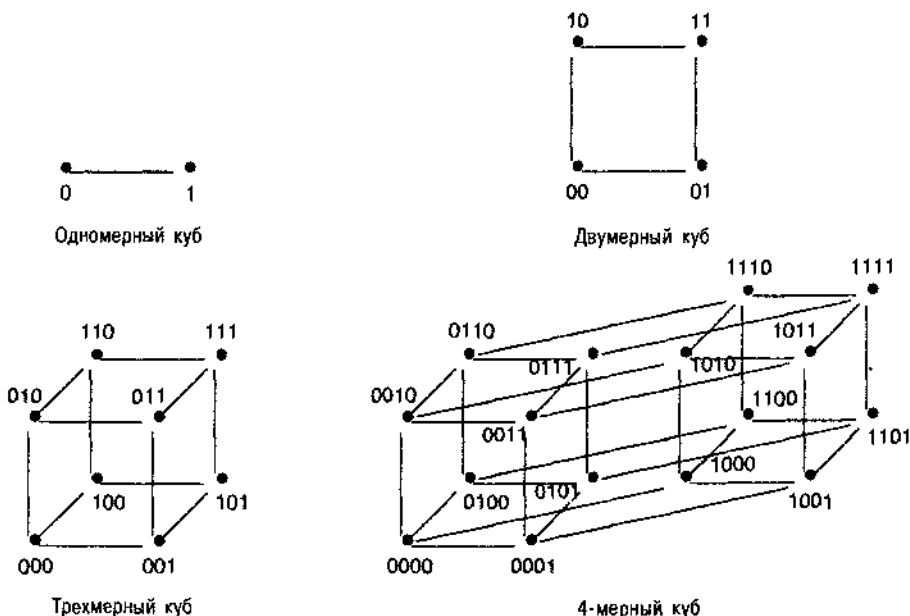


Рис. 2.8. n -мерные кубы для $n = 1, 2, 3$ и 4

Понятие n -мерного куба при разумных значениях n позволяет мысленно представить себе, в чем состоят проблемы кодирования и минимизации логических схем. Например, задача о построении n -разрядного кода Грея эквивалентна нахождению пути по ребрам n -мерного куба, на котором каждая вершина посещается точно один раз. Такие пути для 3- и 4-разрядных кодов Грея показаны на рис. 2.9.

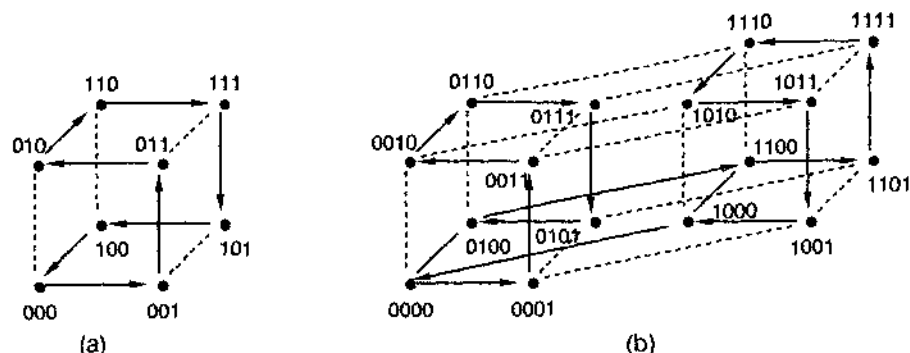


Рис. 2.9. Перемещение по n -мерному кубу в порядке, задаваемом кодом Грея: (а) 3-мерный куб; (б) 4-мерный куб

Кубы дают возможность геометрически интерпретировать понятие *расстояния* (*distance*), называемого также *расстоянием Хэмминга* (*Hamming distance*). Расстояние между двумя n -разрядными строками – это число разрядов, в которых эти строки различаются. В геометрической интерпретации расстояние – это минимальная длина пути между двумя соответствующими вершинами. Расстояние между двумя смежными вершинами равно 1; расстояние между вершинами 3-мерного куба 001 и 100 равно 2. Понятие расстояния является ключевым для понимания и построения кодов, исправляющих ошибки, которым посвящен следующий параграф.

Рассмотрим множество, состоящее из 2^m вершин n -мерного куба, у каждой из которых $n - m$ битов имеют одинаковые значения, а в остальных m разрядах перебираются все возможные 2^m комбинаций; такое множество называется *m -мерным подкубом* (*m -subcube*) n -мерного куба. Например, вершины (000, 010, 100, 110) образуют 2-мерный подкуб 3-мерного куба. Такой подкуб можно записать в виде одной строки: $xx0$, где «х» означает *бит с безразличным значением* (*don't-care*); данному подкубу принадлежат все вершины с одинаковыми битами в разрядах, не обозначенных знаком «х». Как мы увидим в параграфе 4.4, понятие подкубов особенно полезно для мысленного представления того, как действуют алгоритмы минимизации стоимости комбинационных логических схем.

*2.15 Коды, обнаруживающие и исправляющие ошибки

Под *ошибкой* (*error*) в цифровой системе понимают искажение данных, в результате чего их правильные значения теряются. Ошибка бывает вызвана физической *неисправностью* (*failure*). Неисправности могут быть *временными* (*temporary*) или *постоянными* (*permanent*). Например, космические лучи или поток альфа-частиц могут вызвать временный отказ блока памяти, изменив какой-нибудь бит из числа хранящихся в нем. Перегрев схемы или разряд статического электричества могут привести к постоянной неисправности, в результате чего этот узел никогда не будет функционировать правильно.

Влияние неисправностей на данные предсказывают на основе *моделей ошибок* (*error model*). В простейшей модели, которую мы здесь рассмотрим, *ошибки* являются *независимыми* (*independent error model*). Согласно этой модели предполагается, что однократная физическая неисправность может повлиять только на один бит данных; в этом случае говорят, что искаженные данные содержат *одиночную ошибку* (*single error*). Множественные неисправности могут вызвать *многократные ошибки* (*multiple error*), когда ошибка происходит в двух или большем числе битов, но обычно предполагается, что многократные ошибки менее вероятны, чем одиночные.

2.15.1. Коды, обнаруживающие ошибки

Вспомним, что согласно определению, данному в параграфе 2.10, код, состоящий из n -разрядных двоичных строк, не обязательно содержит все возможные 2^n кодовых слов; это как раз тот случай, о котором сейчас пойдет речь. Код, обнаруживающий ошибки (*error-detecting code*), обладает тем свойством, что в результате искажения или повреждения кодового слова вероятнее всего получится двоичная строка, не являющаяся кодовым словом (*noncode word*).

Если в системе используется код, обнаруживающий ошибки, то в ней генерируются, передаются и запоминаются только кодовые слова. Следовательно, правило обнаружения ошибок в двоичной строке совсем простое: если строка является кодовым словом, то предполагается, что она правильная; если строка не является кодовым словом, то она содержит ошибку.

Структуру n -разрядного двоичного кода и предоставляемые им возможности обнаруживать независимые ошибки легко объяснить, воспользовавшись представлением об n -мерном кубе. Код – это просто подмножество вершин n -мерного куба. Для того чтобы код обнаруживал все одиночные ошибки, никакие две вершины, соответствующие кодовым словам, не должны быть смежными.

В качестве примера на рис. 2.10(а) показан 3-разрядный двоичный код, состоящий из пяти кодовых слов. Кодовое слово 111 непосредственно соседствует с кодовыми словами 110, 011 и 101. Так как единичный отказ мог бы преобразовать 111 в 110, 011 или 101, этот код не обнаруживает все одиночные ошибки. Если комбинацию 111 не считать кодовым словом, то получится код, позволяющий обнаруживать одиночные ошибки, как это показано на рис. 2.10(б). Никакая одиночная ошибка не может преобразовать одно кодовое слово в другое.

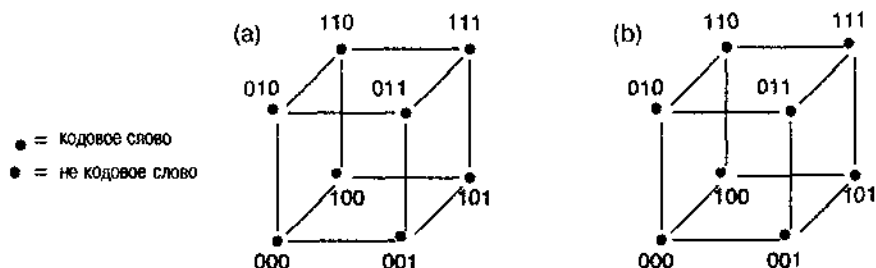


Рис. 2.10. Кодовые слова двух различных 3-разрядных двоичных кодов: (а) минимальное расстояние =1, обнаруживаются не все одиночные ошибки; (б) минимальное расстояние =2, обнаруживаются все одиночные ошибки

Способность кода обнаруживать одиночные ошибки можно сформулировать в терминах расстояний, введенных в предыдущем параграфе:

- Код обнаруживает все одиночные ошибки, если *минимальное расстояние* (*minimum distance*) между кодовыми словами во всех возможных парах равно 2.

В общем случае для построения кода, обнаруживающего одиночные ошибки, с числом кодовых слов, равным 2^n , нам нужно $n+1$ битов. Первые n битов в кодовом слове, называемые *информационными битами* (*information bits*), могут быть любыми из 2^n n -разрядных двоичных строк. Чтобы получить код с минимальным расстоянием 2, мы добавляем еще один бит, называемый *контрольным битом* (*parity bit*), которому присваиваем значение 0, если число единиц среди информационных битов четно, и значение 1 в противном случае. Иллюстрацией сказанного служат первые два столбца в табл. 2.13 для кода с тремя информационными символами. Каждое $(n+1)$ -разрядное двоичное кодовое слово содержит четное число 1 и такой код называется *кодом с проверкой на четность* (*even-parity code*). Можно также построить код, у которого каждое $(n+1)$ -разрядное двоичное кодовое слово содержит нечетное число единиц; такой код представлен в третьем столбце таблицы и носит название *кода с проверкой на нечетность* (*odd-parity code*). Эти коды иногда называют также *кодами с одним контрольным битом* (*1-bit parity code*).

Информационные биты	Код с проверкой на четность	Код с проверкой на нечетность
000	000 0	000 1
001	001 1	001 0
010	010 1	010 0
011	011 0	011 1
100	100 1	100 0
101	101 0	101 1
110	110 0	110 1
111	111 1	111 0

Табл. 2.13. Коды с тремя информационными битами и минимальным расстоянием, равным 2

Коды с одним контрольным символом не обнаруживают ошибки в 2-х битах, поскольку изменение двух битов не нарушает правила четности. Если, например, изменяются три бита в кодовом слове, то в получающейся двоичной строке правило четности нарушается, и эта строка не является кодовым словом. Правда, пользы от этого немного. Если ошибки независимы, то ошибка в 3-х битах существенно менее вероятна, нежели ошибка в 2-х битах, которую обнаружить нельзя. Таким образом, способность кодов с одним контрольным символом обнаруживать ошибки ограничивается практически случаем ошибок в одном бите. Для обнаружения многократных ошибок можно воспользоваться другими кодами, у которых минимальное расстояние больше 2.

2.15.2. Коды, исправляющие ошибки и обнаруживающие многократные ошибки

Воспользовавшись большим числом *проверочных битов* (*check bits*), а не одним контрольным битом, и следуя определенным правилам, можно построить код с минимальным расстоянием больше 2. Прежде чем показать, как это сделать, давайте посмотрим, как можно применять такой код для исправления одиночных ошибок и обнаружения многократных ошибок.

Предположим, что минимальное расстояние кода равно 3. На рис. 2.11 представлен фрагмент n -мерного куба для такого кода. Как видно из рисунка, между каждой парой кодовых слов имеется, по крайней мере, два кодовых слова, не являющихся кодовыми. Предположим теперь, что мы передаем кодовые слова, и пусть сбои приводят к ошибке самое большее в одном бите в каждом принятом слове. Тогда принятое слово, не являющееся кодовым, с ошибкой в одном бите будет ближе к переданному кодовому слову, нежели к какому-либо другому кодовому слову. Поэтому в случае, когда мы принимаем слово, не являющееся кодовым, можно *исправить ошибку* (*error correction*), заменив принятое слово на ближайшее кодовое слово, как это показано стрелками на рисунке. Принятие решения о том, какое слово было передано, на основе принятого слова называется *декодированием* (*decoding*), а устройство, осуществляющее это действие, называется *декодером* (*decoder*), исправляющим ошибки.

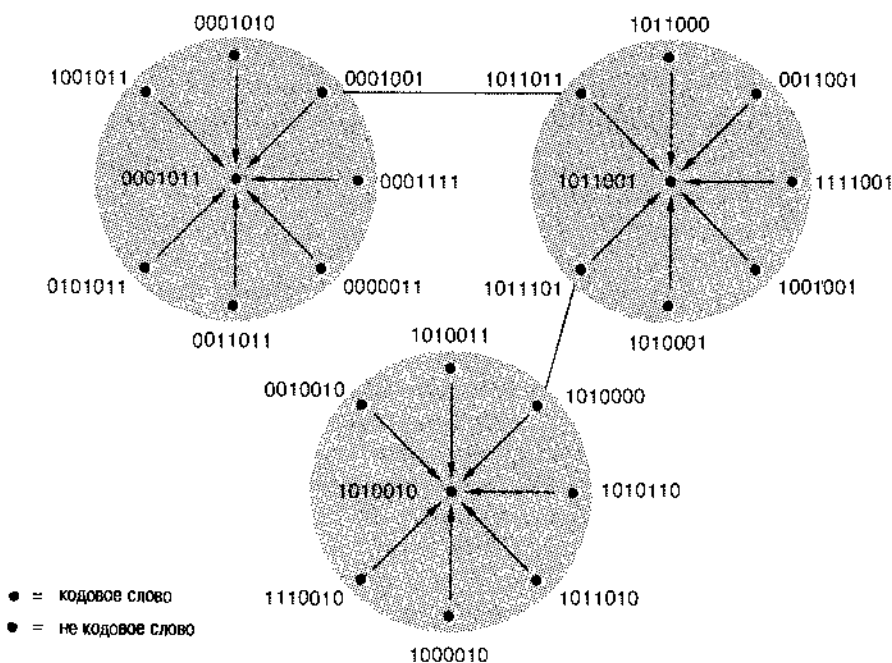


Рис. 2.11. Несколько кодовых слов и слов, не являющихся кодовыми, для 7-разрядного двоичного кода с минимальным расстоянием, равным 3

Код, применяемый для исправления ошибок, называется *корректирующим кодом* (*error-correcting code*). В общем случае справедливо правило: если минимальное расстояние в коде равно $2c+1$, то с его помощью можно исправлять до c ошибок в битах (в предыдущем примере $c=1$). Если минимальное расстояние кода равно $2c+d+1$, то он позволяет исправлять до c ошибок и обнаруживать до d ошибок в других битах.

На рис. 2.12(а) в качестве примера приведен фрагмент n -мерного куба для кода с минимальным расстоянием, равным 4 ($c=1, d=1$). Единичные ошибки в словах 00101010 и 11010011, не являющихся кодовыми, можно исправить. Однако ошибки, приведшие к слову 10100011, исправить нельзя, поскольку это слово не может возникнуть в результате одиночной ошибки, а может лишь стать следствием искажения двух битов в одной из двух возможных пар ошибок. Таким образом, этот код позволяет обнаружить ошибку в двух битах, но не дает возможности исправить ее.

Если принятое слово не является кодовым, то мы не знаем, какое именно кодовое слово в действительности было передано; мы знаем только, какое кодовое слово ближе всего к принятому. Следовательно, как показано на рис. 2.12(б), результат «исправления» тройной ошибки может оказаться неправильным. Подобного рода промахи бывают допустимы, если тройные ошибки происходят с очень малой вероятностью. С другой стороны, в случае, когда тройные ошибки следует принимать во внимание, можно изменить правило декодирования данного кода. Вместо того, чтобы пытаться исправлять ошибки, будем просто воспринимать слова, не являющиеся кодовыми, как содержащие ошибки. Тогда, как показано на рис. 2.12(с), тот же самый код с минимальным расстоянием, равным 4, позволяет обнаруживать до трех ошибок без их исправления ($c=0, d=3$).

2.15.3. Коды Хэмминга

В 1950 году Хэмминг описал общий метод построения кодов с минимальным расстоянием, равным 3, которые теперь называют *кодами Хэмминга* (*Hamming codes*). Для произвольного i его метод дает (2^i-1) -разрядный двоичный код с i проверочными и 2^i-1-i информационными битами. Коды с меньшим числом информационных битов с минимальным расстоянием 3 получаются путем удаления информационных битов из кода Хэмминга с большим числом битов.

Разряды в кодовом слове кода Хэмминга можно пронумеровать от 1 до 2^i-1 . В этом случае, в разряде, номер которого является степенью 2, стоит проверочный бит, а в остальных разрядах помещаются информационные биты. Каждый проверочный бит вместе с некоторым подмножеством информационных битов объединяются в одну группу согласно *проверочной матрице* (*parity-check matrix*). Как показано на рис. 2.13(а), в группу, относящуюся к каждому проверочному биту, попадают такие информационные биты, у которых номер позиции в двоичной записи содержит 1 в том же разряде, что и номер позиции данного проверочного символа. Например, проверочный бит с номером 2 (010) объединяется в одну группу с информационными битами с номерами 3 (011), 6 (110) и 7 (111). При заданной комбинации значений информационных битов значение проверочного бита выбирается по правилу четности; другими словами, полное число единиц в группе с этим проверочным символом должно быть четно.

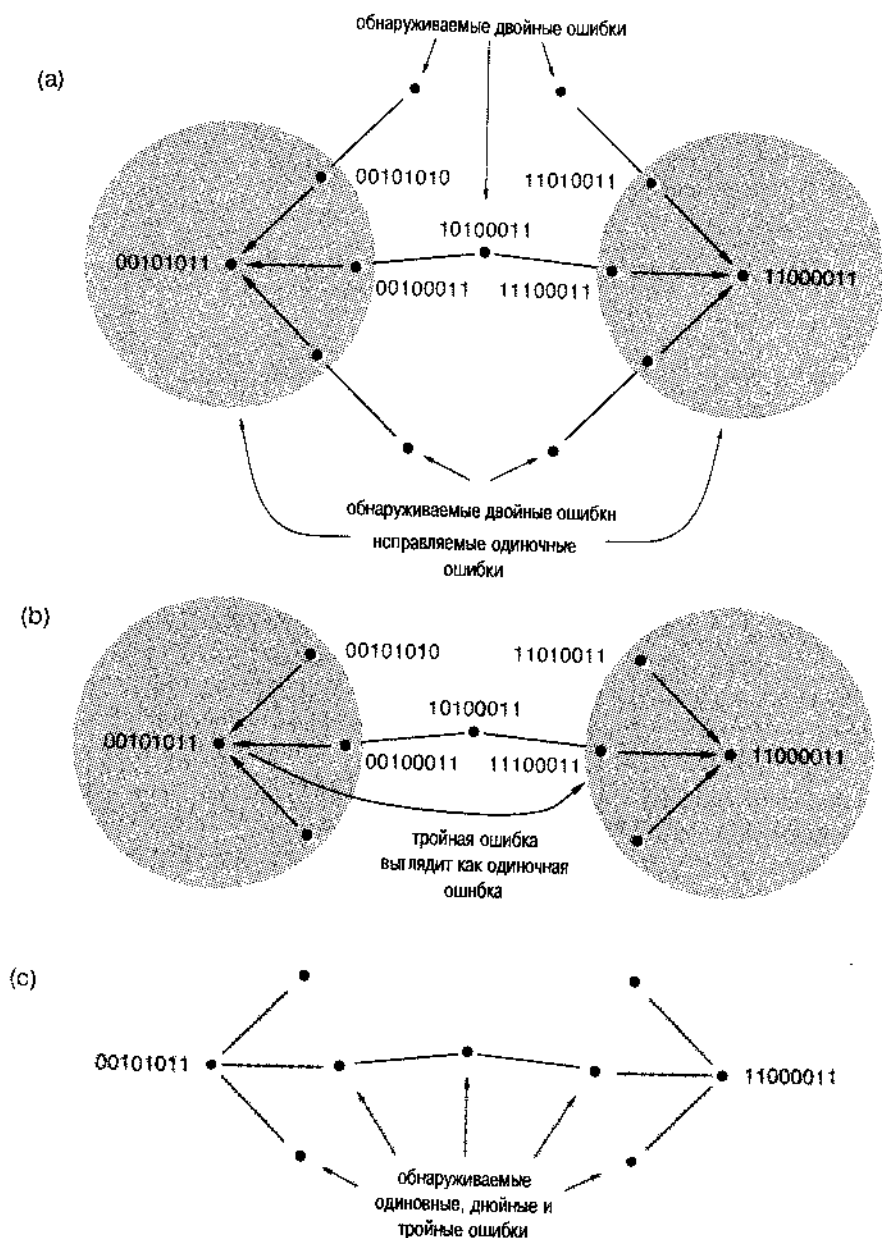


Рис. 2.12. Кодовые слова и слова, не являющиеся кодовыми, для 8-разрядного двоичного кода с минимальным расстоянием, равным 4: (а) исправление одиночных и обнаружение двойных ошибок; (б) ошибочное «исправление» тройной ошибки; (с) отказ от исправления ошибок взамен обнаружения до 3-х ошибок

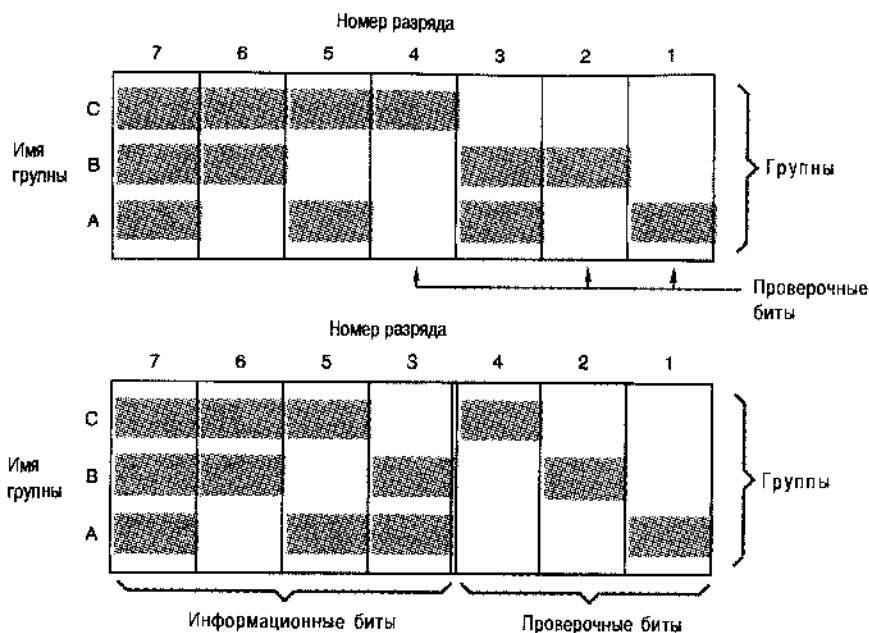


Рис. 2.13. Проверочные матрицы для 7-разрядного кода Хэмминга: (а) с расположением битов в порядке следования их номеров; (б) с разделением проверочных и информационных битов

По традиции разряды в проверочной матрице и в самих кодовых словах бывают переставлены таким образом, чтобы все проверочные биты оказывались справа, как показано на рис. 2.13(б). В первых двух столбцах табл. 2.14 перечислены получающиеся в результате этого кодовые слова.

Можно быть уверенным в том, что минимальное расстояние в коде Хэмминга равно 3, если в любом кодовом слове необходимо изменить, но крайней мере, 3 бита, чтобы получить другое кодовое слово. Другими словами, нужно убедиться в том, что изменение одного бита или двух битов в кодовом слове дает слово, не являющееся кодовым.

Если изменить один бит в j -м разряде кодового слова, то нарушится условие четности в каждой из групп, в которую входит j -й разряд. Поскольку каждый информационный бит содержится, хотя бы в одной группе, как минимум в одной из групп условие четности не будет выполнено и получающееся в результате слово не является кодовым.

Что произойдет, если мы изменим два бита в j -м и k -м разрядах? В тех контрольных группах, которые содержат оба эти разряда, условие четности останется по-прежнему верным, поскольку оно не нарушается при изменении четного числа битов. Поскольку, однако, j и k различны, их выражения в двоичной записи отличаются, по крайней мере, в одном бите, соответствующем одной из контрольных групп. В этой группе изменится только один бит, в результате чего условие четности окажется невыполненным и получившееся слово не будет кодовым.

Табл. 2. 14. Кодовые слова кодов Хэмминга с четырьмя информационными битами с минимальными расстояниями, равными 3 и 4

Код с минимальным расстоянием 3		Код с минимальным расстоянием 4	
Информационные биты	Проверочные биты	Информационные биты	Проверочные биты
0000	000	0000	0000
0001	011	0001	0111
0010	101	0010	1011
0011	110	0011	1100
0100	110	0100	1101
0101	101	0101	1010
0110	011	0110	0110
0111	000	0111	0001
1000	111	1000	1110
1001	100	1001	1001
1010	010	1010	0101
1011	001	1011	0010
1100	001	1100	0011
1101	010	1101	0100
1110	100	1110	1000
1111	111	1111	1111

Если вы понимаете это доказательство, то вам нетрудно увидеть, что правила нумерации разрядов, принятые при построении кода Хэмминга, являются простым следствием того, что содержится в приведенном рассуждении. В первой части доказательства (ошибки в одном бите) мы потребовали, чтобы ни один номер разряда не был равен нулю. А во второй части доказательства (ошибки в двух битах) мы потребовали, чтобы никакие два разряда не имели одинаковых номеров. Таким образом, если номера разрядов записываются i битами, то можно построить код Хэмминга с числом разрядов до $2^i - 1$.

Доказательство указывает также, как можно построить декодер, исправляющий ошибки (*error-correcting decoder*) в принимаемых словах кода Хэмминга. Прежде всего проверяются условия четности во всех контрольных группах; если все условия четности выполнены, то принятое слово считается правильным. Если в одной группе или в большем числе групп условие четности нарушено, то счита-

ется, что произошла одиночная ошибка. Совокупность групп с нечетным числом единиц [пазываемая *синдромом* (*syndrome*)] должна иметь один общий столбец в проверочной матрице; предполагается, что значение соответствующего разряда неверно и оно заменяется на противоположное. Предположим, например, что используется код, указанный на рис. 2.13(б) и принято слово 0101011. Число единиц в группах *B* и *C* нечетно, что соответствует 6-му разряду в проверочной матрице (синдром равен 110, то есть 6). Беря обратное значение бита в 6-м разряде, мы получим правильное слово: 0001011.

Код Хэмминга с минимальным расстоянием 3 легко видоизменить так, чтобы увеличить минимальное расстояние и сделать его равным 4. Просто добавляется еще один проверочный бит, значение которого выбирается таким, чтобы полное число единиц во всех разрядах, включая новый бит, было четным. Как и в коде с одним контрольным символом и проверкой на четность, этот бит гарантирует, что все ошибки, возникающие в нечетном числе разрядов, будут обнаруживаться. В частности, обнаруживаются все тройные ошибки. Мы уже показали, что все одиночные и двойные ошибки обнаруживаются другими проверочными битами, поэтому минимальное расстояние модифицированного кода должно равняться 4.

Коды Хэмминга с минимальным расстоянием, равным 3 и 4, широко применяются для обнаружения и исправления ошибок в запоминающих системах компьютеров, в частности, в больших универсальных вычислительных машинах, где на запоминающие устройства приходится основная масса системных отказов. Эти коды особенно привлекательны, когда длина запоминаемых слов очень велика, поскольку требуемое число проверочных битов растет медленно с увеличением длины слова, как показано в табл. 2.15.

Табл. 2.15. Число разрядов в кодовых словах кодов Хэмминга с минимальными расстояниями, равными 3 и 4

Информационные биты	Код с минимальным расстоянием 3		Код с минимальным расстоянием 4	
	Проверочные биты	Общее число битов	Проверочные биты	Общее число битов
1	2	3	3	4
≤ 4	3	≤ 7	4	≤ 8
≤ 11	4	≤ 15	5	≤ 16
≤ 26	5	≤ 31	6	≤ 32
≤ 57	6	≤ 63	7	≤ 64
≤ 120	7	≤ 127	8	≤ 128

2.15.4. Циклические коды

Помимо кодов Хэмминга придумано много других кодов, обнаруживающих и исправляющих ошибки. Самыми важными являются *циклические коды* [*cyclic-redundancy-check* (*CRC*) *codes*], которые, как это ни странно, включают в себя

коды Хэмминга. Существует развитая теория этих кодов, посвященная не только их способности обнаруживать и исправлять ошибки, но и позволяющая строить для них недорогие кодеры и декодеры (см. Обзор литературы).

Двумя важными приложениями циклических кодов является их использование в дисковых накопителях и в сетях передачи данных. Каждый блок данных на диске (обычно 512 байтов) защищен циклическим кодом, так что ошибки внутри блока могут быть обнаружены, а в некоторых случаях и исправлены. В сетях передачи данных каждый пакет данных заканчивается проверочными битами циклического кода. Именно циклические коды были выбраны для этих двух применений из-за их способности обнаруживать пакеты ошибок. Кроме одиночных ошибок они могут обнаруживать многократные ошибки, которые образуют компактные группы в блоке данных на диске или в пакете. Такие ошибки более вероятны, чем ошибки, равномерно распределенные по разрядам, так как наиболее вероятные причины ошибок в этих двух применениях – это дефекты поверхности дисков и всплески шума в каналах связи.

2.15.5. Двумерные коды

Другой способ получения кода с большим минимальным расстоянием заключается в построении *двумерного кода* (*two-dimensional code*), иллюстрацией чего служит рис. 2.14(а). Из информационных битов образуется двумерная решетка и проверочные биты добавляются как к строкам, так и к столбцам. Для строк используется код C_{row} с минимальным расстоянием d_{row} , а для столбцов – возможно другой код C_{col} с минимальным расстоянием d_{col} . Другими словами, проверочные символы строки выбираются так, чтобы каждая строка была кодовым словом кода C_{row} , а проверочные символы столбца – так, чтобы каждый столбец был кодовым словом кода C_{col} . (Проверочные биты в «углу» решетки можно выбрать согласовано одному из этих кодов.) Минимальное расстояние в двумерном коде равно произведению d_{row} и d_{col} ; в связи с этим двумерный код иногда называют *кодом-произведением* (*product code*).

В двумерном коде, показанном на рис. 2.14(б), для строк и столбцов используются коды с одним контрольным символом проверки на четность, так что минимальное расстояние в двумерном коде равно $2 \cdot 2 = 4$. Мы можем легко убедиться в этом, проверив, что любая комбинация из одной, двух или трех ошибок в битах приведет к нарушению условия четности в строке или в столбце или в том и другом. Чтобы ошибка была необнаруживаемой, необходимо изменить, по меньшей мере, четыре бита, расположенные по углам прямоугольника, как показано на рис. 2.14(с).

Процедура обнаружения и исправления ошибок этим кодом носит совершенно естественный характер. Предположим, что мы считываем информацию по строке за раз. Прочтя каждую строку, мы проверяем ее принадлежность строковому коду. Если в строке обнаруживается ошибка, то только по виду этой строки мы еще не можем сказать, какой бит неверен. Однако в случае, когда испорчена только одна строка, ее можно восстановить, складывая строки поразрядно по правилу ИСКЛЮЧАЮЩЕЕ ИЛИ, опустив при этом поврежденную строку, но принимая во внимание строку контрольных битов по столбцам.

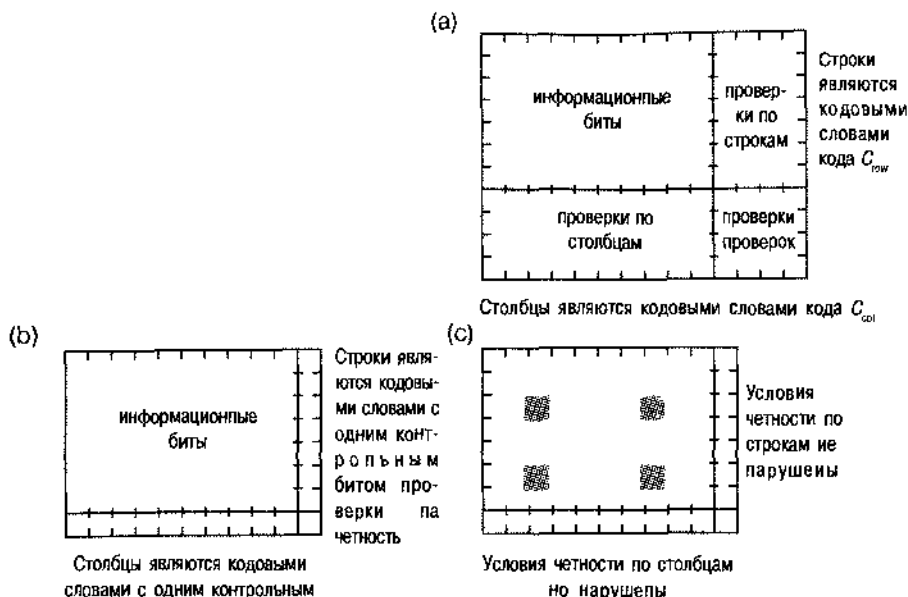


Рис. 2.14. Двумерные коды: (a) общая структура; (b) код с проверкой на четность в строках и столбцах с минимальным расстоянием, равным 4; (c) типичная конфигурация необнаруживаемых ошибок

Чтобы получить еще большее минимальное расстояние, в отношении строк и/или столбцов можно применить код Хэмминга с минимальным расстоянием 3 или 4. Можно также построить код с большим числом измерений и минимальным расстоянием, равным произведению минимальных расстояний в каждом измерении.

Важным применением двумерных кодов являются запоминающие устройства типа *RAID* (Redundant Array of Inexpensive Disks, набор недорогих дисков с избыточностью). В таком устройстве используются $n+1$ дисков для хранения данных, объем которых соответствует емкости n дисков. Например, восемь 8-гигабайтовых дисковых накопителей можно использовать для хранения 64 гигабайтов данных без избыточности, а девятый 8-гигабайтовый диск мог бы служить для записи проверочной информации.

На рис. 2.15 схематически изображен двумерный код для системы RAID, каждый дисковый накопитель считается строкой кода. В каждом накопителе сохраняются m блоков данных, содержащих обычно по 512 байтов. В частности, на 8-гигабайтовом диске можно запомнить около 16 миллионов блоков. Как показано на рисунке, каждый блок включает свои собственные проверочные биты, образуемые по правилу циклического кода, чтобы обнаруживать ошибки внутри блока. В первых n накопителях хранятся данные без избыточности. Каждый блок $(n+1)$ -го накопителя содержит контрольные биты для соответствующих битов первых n накопителей. Это означает, что i -й бит в b -м блоке на $(n+1)$ -м диске выбирается так, чтобы число единиц в i -х разрядах b -х блоков на всех дисках было четным.



Рис. 2.15. Структура кода, исправляющего ошибки, для системы RAID (CRC – проверочные биты циклического кода)

В процессе работы ошибки в информационных битах обнаруживаются циклическим кодом. В любом случае, когда обнаружена ошибка в блоке на одном из дисков, правильное содержимое этого блока можно получить путем простой проверки на четность соответствующих блоков на всех других дисках, включая $(n+1)$ -й. И хотя это требует n дополнительных чтений с дисков, это все же лучше, чем потерять ваши данные! При записи нового информационного блока также требуются дополнительные обращения к дискам для обновления соответствующего проверочного блока (см. задачу 2.46). Поскольку обычно записи происходят значительно реже, чем чтения, эти «накладные» расходы не обременительны.

2.15.6. Коды с контрольной суммой

Операция проверки на четность, о которой говорилось в предыдущих разделах, является по существу сложением битов по модулю 2: сумма набора битов по модулю 2 равна 0, если число единиц в этом наборе четно, и равна 1, если оно нечетно. Правило сложения по модулю можно распространить на другие основания, отличающиеся от 2, и таким образом получать проверочные цифры.

В компьютерах, например, данные хранятся в виде совокупности 8-битовых байтов. Можно считать, что каждый байт имеет десятичное значение от 0 до 255. Поэтому для образования проверки по байтам можно воспользоваться сложением по модулю 256. Одноочный проверочный байт, являющийся суммой по модулю 256 всех информационных байтов, называется *контрольной суммой* (*checksum*). Получающийся в результате *код с контрольной суммой* (*checksum code*) может обнаружить любую однопочную ошибку в *байте*, поскольку такая ошибка приведет к расхождению между вновь вычисленной суммой байтов и контрольной суммой.

Для построения кодов с контрольной суммой при сложении можно использовать и другие модули. Важными, в частности, являются коды с контрольной суммой, которую находят путем сложения по модулю 255, то есть коды с *контрольной суммой, образуемой путем сложения в обратном коде* (*ones'-complement checksum codes*); такие коды обладают особыми свойствами в отношении вычислений и обнаружения ошибок, в связи с чем эти коды применяются для защиты от ошибок в заголовках пакетов в вездесущем протоколе Интернета (Internet Protocol, IP; см. Обзор литературы).

2.15.7. Коды « m из n »

Минимальное расстояние в кодах «1 из n » и « m из n », введенных в параграфе 2.13, равно 2, поскольку изменение только одного бита изменяет число единиц в кодовом слове и приводит, таким образом, к слову, не являющемуся кодовым.

Эти коды обладают другим полезным свойством с точки зрения обнаружения ошибок: они обнаруживают множественные однонаправленные ошибки. Под *однонаправленной ошибкой* (*unidirectional error*) понимают случай, когда изменение во всех ошибочных битах происходит в одну сторону (нули изменяются на единицы или наоборот). Это свойство оказывается очень полезным, если преобладающий механизм ошибок в системе имеет тенденцию изменять все биты в одну и ту же сторону.

2.16. Коды для последовательной передачи и хранения данных

2.16.1. Параллельное и последовательное представление данных

В большинстве компьютеров и в других цифровых системах данные передаются и хранятся в *параллельном формате* (*parallel data*). При параллельной передаче данных для каждого бита в слове данных предоставлена отдельная сигнальная линия. При параллельном хранении данных все биты слова данных можно записать и прочитать одновременно.

В некоторых приложениях применение параллельных форматов не оправдывает затрат. Например, для параллельной передачи байтов данных по телефонной сети потребовалось бы восемь телефонных линий, а для параллельного хранения байтов данных на магнитном диске нужно было бы иметь накопитель с восемью отдельными головками для чтения/записи. *Последовательные форматы* (*serial data*) позволяют передавать данные, а также записывать и считывать их при хранении по одному биту за раз, благодаря чему стоимость системы во многих случаях снижается.

Рис. 2.16 является иллюстрацией основных идей, относящихся к последовательной передаче данных. Периодическим тактовым сигналом CLOCK задается скорость передачи битов: по одному биту за период тактового сигнала. Таким образом, *скорость передачи, измеряемая числом битов в секунду* (бит/с; *bit rate, bps*), равна частоте тактового сигнала, выраженной числом периодов в секунду (герц, Гц).

Величина, обратная скорости передачи (выраженной в бит/с), называется *длительностью бита* (*bit time*); численно она равна периоду тактового сигнала в секундах (с). Это время отводится в линии последовательной передачи данных (названной на рисунке SERDATA) на каждый передаваемый бит. Иногда время, занимаемое каждым битом, называют *битовой ячейкой* (*bit cell*). Фактический вид сигнала, возникающего в линии в пределах каждой ячейки, зависит от *сигнального кода* (*line code*). В случае простейшего сигнального кода, называемого *кодом без возврата к нулю* (*Non-Return-to-Zero, NRZ*), единичный бит передается путем удерживания на линии 1 в течение всего времени, занимаемого битовой ячейкой, а

нулевой бит — путем передачи 0. В следующем разделе обсуждаются более сложные сигнальные коды, реализующие другие правила.

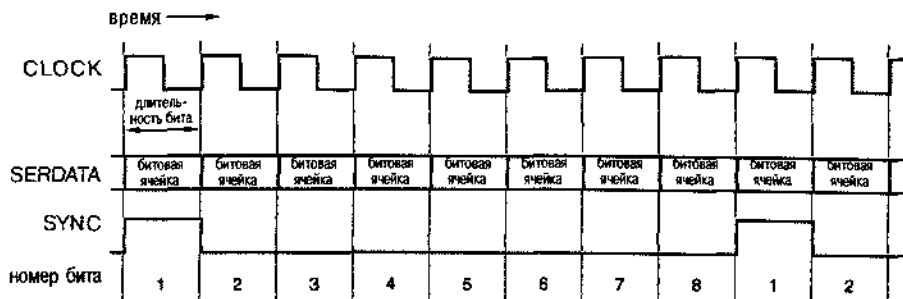


Рис. 2.16. Основные понятия, относящиеся к последовательной передаче данных

Независимо от сигнального кода, в системах с последовательной передачей данных или с последовательным записью и чтением данных при их хранении бывает необходим какой-то способ указания на роль каждого бита в последовательном потоке. Предположим, например, что последовательно передаются 8-битовые байты. Как узнать, какой бит является первым в каждом байте? Необходимую информацию несет сигнал синхронизации (*synchronization signal*), названный SYNC на рис. 2.16. Он равен 1 только на первом бите каждого байта.

Очевидно, что для приема потока последовательно передаваемых данных нам нужны три сигнала: тактовый сигнал для определения границ битовых ячеек, сигнал синхронизации для определения границ между словами и сами данные, передаваемые последовательно. В ряде приложений, в частности, в линиях связи между блоками в компьютере или между узлами в телекоммуникационной системе каждый из этих сигналов передается по отдельному проводу; сокращение числа проводов до 3-х вместо *n* дает большую экономию. Пример системы с 3-проводной последовательной передачей данных будет приведен в разделе 8.5.4.

Во многих случаях стоимость передачи трех отдельных сигналов оказывается все же слишком высокой (например, стоимость трех телефонных линий или трех головок чтения/записи). Тогда все три сигнала объединяются в один последовательный поток данных и применяются сложные аналоговые и цифровые схемы для извлечения из потока данных тактового сигнала и сигнала синхронизации.

*2.16.2. Сигнальные коды для последовательной передачи

Чаще всего для последовательной передачи данных используется один из сигнальных кодов, приведенных на рис. 2.17. В коде NRZ значение каждого бита посылается по линии в течение всего времени, занимаемого битовой ячейкой. Это простейшее и самое надежное правило кодирования для передачи на короткие расстояния. Однако обычно требуется одновременно с данными посылать тактовый сигнал, задающий положение битовых ячеек. В противном случае у приемника не будет

возможности определить, как много нулей или единиц представлено непрерывно удерживаемым 0-ым или 1-ым уровнем. Например, в отсутствие тактового сигнала, задающего границы битовых ячеек, сигнал NRZ, приведенный на рис. 2.17, может быть ошибочно интерпретирован как 01010.

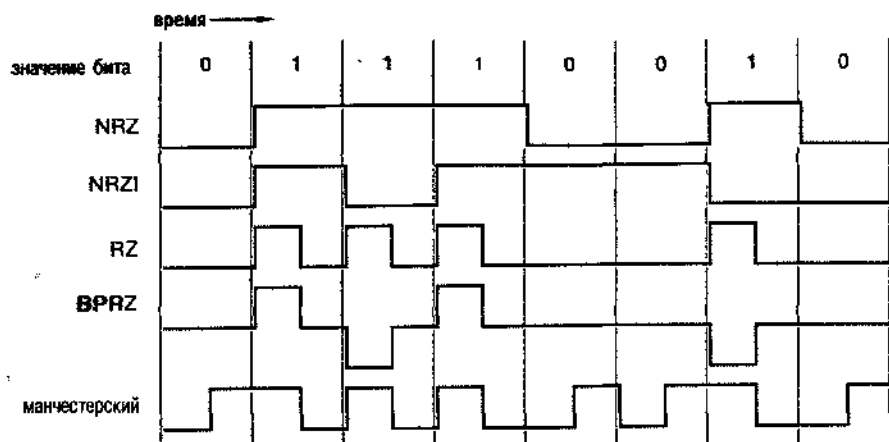


Рис. 2.17. Сигнальные коды, обычно применяемые для последовательной передачи данных

Аналого-цифровой схемой, позволяющей извлечь тактовый сигнал из последовательного потока данных, является *цифровая схема фазовой автоподстройки частоты* (цифровая ФАПЧ; *digital phase-locked loop, PLL*). Схема ФАПЧ будет работать только в том случае, когда в последовательном потоке данных содержится достаточное число переходов от 0 к 1 и от 1 к 0, чтобы схема ФАПЧ могла «догадаться», где происходят подобные переходы в исходном тактовом сигнале. Если данные передаются кодом NRZ, то схема ФАПЧ будет работать только тогда, когда в данных нет длительных интервалов, в течение которых непрерывно следуют единицы или нули.

В некоторых случаях *среда*, по которой последовательно передаются или в которой хранятся данные, *чувствительна только к переходам* (*transition sensitive media*). В такой среде нельзя передавать или хранить абсолютные уровни 0 или 1; передавать или хранить можно только переходы между двумя дискретными уровнями. Например, на магнитном диске или ленте информация запоминается в виде изменения полярности намагниченности материала на участках, соответствующих запоминаемым битам. При воспроизведении информации нельзя определить абсолютную полярность намагниченности на том или ином участке, а можно обнаружить лишь изменение полярности при переходе от одного участка к другому.

Данные, записанные в формате NRZ в среде, чувствительной к переходам, нельзя восстановить однозначно; указавшую на рис. 2.17 последовательность битов можно было бы интерпретировать как 01110010 и 10001101. В *коде без возврата к нулю с инверсией* (*Non-Return-to-Zero Invert-on-1s, NRZI*) этот недостаток преодолевается: единичный бит передается уровнем, противоположным тому, который поддерживался в пределах предыдущей битовой ячейки, а нулевой бит —

посылкой того же уровня. Схема ФАПЧ сможет извлечь тактовый сигнал из данных, передаваемых кодом NRZI, если только данные не будут содержать длинных последовательностей подряд идущих нулей.

Код с возвратом к нулю (*Return-to-Zero, RZ*) подобен коду NRZ за исключением того, что в случае единичного бита уровень 1 удерживается только на части интервала времени, отведенного на передачу бита, обычно в течение 1/2 этого интервала. Если последовательность данных, передаваемых этим кодом, содержит большое число единиц, то она содержит много переходов и это может быть использовано схемой ФАПЧ для извлечения тактовых сигналов. Однако при этом, как и в случае других сигнальных кодов, строка из нулей не содержит переходов, и при наличии длинных последовательностей нулей извлечение тактового сигнала оказывается невозможным.

Другое требование, предъявляемое к сигналам в таких каналах связи, как высокоскоростные волоконно-оптические линии, заключается в том, что последовательный поток данных должен быть *сбалансированным по постоянному току* (*DC balance*). Это значит, что он должен иметь равное число единиц и нулей; наличие постоянной составляющей в потоке (когда единиц много больше, чем нулей, или наоборот) на протяжении сравнительно долгого времени вызывает смещение в приемнике, которое ухудшает его способность надежно различать единицы и нули.

Обычно не гарантируется, что данные, передаваемые кодами NRZ, NRZI или RS, будут сбалансированными по постоянному току; ничто не может предотвратить наличие в потоке данных пользователя длинных строк со словами, в которых единиц больше, чем нулей, или наоборот. Однако сбалансированности по постоянному току все же можно достичь с помощью нескольких лишних битов, кодируя данные пользователя *балансным кодом* (*balanced code*), в каждом слове которого одинаковое число единиц и нулей, и передавая эти кодовые слова в формате NRZ.

В параграфе 2.13, например, мы ввели код 8B10B, который преобразует 8-разрядные двоичные данные пользователя в 10-разрядные двоичные слова, являющиеся по большей части словами кода «5 из 10». Напомним, что в коде «5 из 10» имеется только 252 кодовых слова, но существует еще $\binom{10}{4} = 210$ кодовых слов кода «4 из 10» и столько же кодовых слов кода «6 из 10». Конечно, кодовые слова двух последних кодов не являются точно сбалансированными по постоянному току. В коде 8B10B эта проблема решается путем представления одной 8-разрядной двоичной величины *парой* несбалансированных кодовых слов, беря одно «легкое» слово из кода «4 из 10» и одно «тяжелое» слово из кода «6 из 10». В кодере вырабатывается также *признак рассогласования* (*running disparity*), то есть одиночный бит, указывающий, каким было последнее переданное им несбалансированное кодовое слово: «тяжелым» или «легким». Когда наступает момент передачи очередного несбалансированного кодового слова, кодер выбирает пару слов разного веса. Этот простой прием обеспечивает наличие в коде 8B10B $252 + 210 = 462$ кодовых слов для кодирования 8-разрядных двоичных данных пользователя. Некоторыми «лишними» кодовыми словами удобно кодировать состояния в канале последовательной передачи, не являющиеся данными, такие как IDLE, SYNC и ERROR. Используются не все несбалансированные кодовые слова, и даже не все сбалансированные: вместо таких слов, как 0000011111, предпочтительнее передавать несбалансированные пары с большим числом переходов.

КНЛО-, МЕГА-, ГИГА-, ТЕРА-

Когда речь идет о битах в секунду (бит/с), герцах, омах, ваттах и в большинстве других случаев, относящихся к техническим показателям, приставки к (кило-), М (мега-), Г (гига-) и Т (тера-) означают 10^3 , 10^6 , 10^9 и 10^{12} , соответственно. Однако применительно к объему памяти те же приставки имеют значения 2^{10} , 2^{20} , 2^{30} и 2^{40} . Так сложилось исторически, что они были заимствованы для этой цели, поскольку обычно объем памяти бывает равен степени 2, а значение 2^{10} (1024) очень близко к 1000.

Поэтому теперь, если кто-нибудь предложит за вашу первую работу в качестве инженера 50 килобаксов в год, вам придется договариваться о значении приставки!

Во всех предыдущих кодах сигналы, применяемые для передачи и при чтении/записи, имели только два уровня. В *двуполярном коде с возвратом к нулю (Bipolar Return-to-Zero, BPRZ)* передаются сигналы трех уровней: +1, 0 и -1. Этот код подобен коду RZ, за исключением того, что единичные биты поочередно передаются как +1 и -1; по этой причине такой способ передачи называют также *кодированием с чередованием полярности (Alternate Mark Inversion, AMI)*.

Большим преимуществом кода BPRZ по сравнению с кодом RZ является его сбалансированность по постоянному току. Благодаря этому его можно применять для передачи потока данных по каналу, не допускающему наличия постоянной составляющей, например, по телефонным линиям с трансформаторной связью. Код BPRZ и в самом деле уже не один десяток лет используется в цифровых телефонных линиях T1, где аналоговый речевой сигнал передается по последовательному каналу со скоростью 64 кбит/с в виде потока из 8000 цифровых отсчетов в секунду по 8 битов в каждом в формате BPRZ.

Как и в случае кода RZ, из потока данных в формате BPRZ можно извлечь тактовый сигнал, если только подряд не идет слишком много нулей. Хотя «Телефонная Компания» («The Phone Company», ироническое прозвище корпорации AT&T. — Прим. перев.) не может проконтролировать, что вы говорите (по крайней мере пока), все же у нее есть простой способ ограничения последовательности нулей. Если какой-то из 8-разрядных байтов, возникающих в результате взятия выборок из вашего аналогового речевого сигнала, оказывается состоящим из одних нулей, то происходит замена второго бита со стороны младших разрядов на 1! Это называется *подавлением нулевого кода (zero-code suppression)*. Держу пари, что вы никогда не замечали этого. Вот почему во многих случаях, когда по линиям T1 передаются данные, вы получаете из канала, со скоростью передачи 64 кбит/с только 56 кбит в секунду; младшему разряду каждого байта всегда присваивается единичное значение, чтобы воспрепятствовать изменению других битов из-за подавления нулевого кода.

Последний из кодов, приведенных на рис. 2.17, называется *манчестерским (Manchester) кодом* или *двухфазным (diphase) кодом*. Главное достоинство этого кода состоит в том, что независимо от передаваемой последовательности данных, он обеспечивает по меньшей мере один переход на битовую ячейку, что позволяет с большой легкостью извлекать из него тактовый сигнал. Как показано на рисунке,

ноль кодируется переходом из 0 в 1 в середине битовой ячейки, а единица — переходом из 1 в 0. Но это главное достоинство манчестерского кода является также и его главным недостатком. Поскольку у него большее число переходов приходится на битовую ячейку, чем у других кодов, ему требуется канал с большей шириной полосы, чтобы реализовать заданную скорость передачи. Правда, в случае коаксиального кабеля нет проблемы с шириной полосы, поэтому в первых локальных сетях Ethernet для кодирования данных, передававшихся последовательно со скоростью 10 Мбит/с (мегабит в секунду), был использован манчестерский код.

О «ТЕЛЕФОННОЙ КОМПАНИИ»

Посмотрите фильм 1967 года «Президентский психоаналитик» с Джеймсом Коберпом (James Coburn), где «Телефонная Компания» представлена в забавном свете. Фильм подводит вас к представлению о мире, в котором каждый подключен к телефонной сети. Сегодня, когда все больше распространяются цифровые технологии и дешевые мобильные средства связи, эта идея становится не такой уж надуманной.

Обзор литературы

Содержание первых девяти параграфов этой главы базируется на главе 4 книги Уэйкерли *Архитектура микро-ЭВМ и программирование* (John F. Wakerly. *Microcomputer Architecture and Programming*. Wiley, 1981). Точное, детальное и интересное обсуждение этих вопросов можно найти также в книге Кнута *Получисленные алгоритмы* (Donald E. Knuth. *Seminumerical Algorithms*, third edition. Addison-Wesley, 1997). Читатели с математическими наклонностями оценят блестящий анализ Кнутом свойств и арифметики числовых систем, а другие смогут испытать чувство озарения и порадоваться историческим сведениям, которыми пропизан весь его текст.

В книге Уэйкерли *Архитектура микро-ЭВМ и программирование: Семейство 68000* (John F. Wakerly. *Microcomputer Architecture and Programming: The 68000 Family*. Wiley, 1989) рассказывается об алгоритмах двоичного умножения и деления, а также об арифметических операциях с плавающей точкой. Более подробное рассмотрение арифметических действий и числовых систем с плавающей точкой можно найти в книге Уэйзера и Флинна *Введение в арифметику для разработчиков цифровых систем* (Shlomo Waser and Michael J. Flynn. *Introduction to Arithmetic for Digital Systems Designers*. Holt, Rinehart and Winston, 1982).

Циклические коды основаны на теории конечных полей (*finite fields*), введенных французским математиком Эваристом Галуа (1811–1832) незадолго до того, как он был убит на дуэли политическим оппонентом. Классическим учебником по кодам, обнаруживающим и исправляющим ошибки, является книга Петерсона и Уэлдона *Коды, исправляющие ошибки* (second edition, MIT Press, 1972; рус. пер. — М.: «Мир», 1976); однако эту книгу можно рекомендовать только читателям с повышенной математической подготовкой. Более доступное введение в теорию кодирования можно найти в книге Лин и Костелло *Кодирование с контролем ошибок: принципы и приложения* (S. Lin and D. J. Costello, Jr. *Error Control Coding: Fundamentals and Applications*. Prentice Hall, 1983). Другой подход к теории коди-

рования, ориентированный на связанные проблемы, содержится в книге Майкельсона и Левека *Методы контроля ошибок в цифровой связи* (A. M. Michelson and A. H. Levesque. *Error-Control Techniques for Digital Communication*. Wiley-Interscience, 1985). Вопросам аппаратной реализации кодов в компьютерных системах посвящена книга Уэйкерли *Коды с обнаружением ошибок, схемы с самопроверкой и их приложения* (John F. Wakerly. *Error-Detecting Codes, Self-Checking Circuits, and Applications*. Elsevier/North-Holland, 1978).

В последней из упомянутых работ Уэйкерли показано, что коды с контрольной суммой, вычисляемой в обратном коде, способны обнаруживать длинные пакеты однонаправленных ошибок; это полезное свойство для каналов связи, где имеется тенденция к тому, что все ошибки происходят в одну и ту же сторону. Особые вычислительные свойства этих кодов делают их удобными для эффективного нахождения контрольных сумм программными методами, что важно с точки зрения их применения в протоколе Интернета; см. запросы на комментарии (Requests for Comments, RFCs) RFC-1071 и RFC-1141. В заархивированном виде запросы на комментарии имеются во многих местах в Интернете; поищите по «RFC».

Введение в методы кодирования для последовательной передачи данных, включая математический анализ характеристик некоторых кодов и их требования к полосе пропускания, имеются в книге Гальярди *Введение в технику связи* (R. M. Gagliardi. *Introduction to Communications Engineering*, second edition. Wiley-Interscience, 1988). Прекрасное описание последовательных кодов, применяемых для записи на магнитные диски и ленты, содержит книга Матика *Запоминающие устройства компьютеров и техника записи/чтения* (Richard Matick. *Computer Storage Systems and Technology*. Wiley-Interscience, 1977).

Структура кода 8B10B и его логическое обоснование отлично объяснены в первоначальном патенте Франасека (Peter Franaszek) и Видмера (Albert Widmer): U.S. patent number 4, 486, 739 (1984). Этот и почти все другие патенты США, выданные после 1971 года, можно найти в Интернете на странице www.patents.ibm.com.

Упражнения

2.1. Выполните следующие преобразования из одной системы счисления в другую:

- | | |
|-----------------------------|-------------------------|
| (a) $1101011_2 = ?_{16}$ | (b) $174003_8 = ?_2$ |
| (c) $10110111_2 = ?_{16}$ | (d) $67.24_8 = ?_2$ |
| (e) $10100.1101_2 = ?_{16}$ | (f) $F3F5_{16} = ?_2$ |
| (g) $11011001_2 = ?_8$ | (h) $AB3D_{16} = ?_2$ |
| (i) $101111.0111_2 = ?_8$ | (j) $15C.38_{16} = ?_2$ |

2.2. Преобразуйте следующие восьмеричные числа в двоичные и шестнадцатеричные:

- | | |
|--------------------------------|----------------------------------|
| (a) $1023_8 = ?_2 = ?_{16}$ | (b) $761302_8 = ?_2 = ?_{16}$ |
| (c) $163417_8 = ?_2 = ?_{16}$ | (d) $552273_8 = ?_2 = ?_{16}$ |
| (e) $5436.15_8 = ?_2 = ?_{16}$ | (f) $13705.207_8 = ?_2 = ?_{16}$ |

2.3. Преобразуйте следующие шестнадцатеричные числа в двоичные и восьмеричные:

- (a) $1023_{16} = ?_2 = ?_8$ (b) $7E6A_{16} = ?_2 = ?_8$
 (c) $ABCD_{16} = ?_2 = ?_8$ (d) $C350_{16} = ?_2 = ?_{16}$
 (e) $9E36.7A_{16} = ?_2 = ?_8$ (f) $DEAD.BEEF_{16} = ?_2 = ?_8$

2.4. Каковы восьмеричные значения четырех 8-разрядных байтов в 32-разрядном слове, которое в восьмеричном представлении имеет вид 1234567123₈?

2.5. Преобразуйте следующие числа в десятичные:

- (a) $1101011_2 = ?_{10}$ (b) $174003_8 = ?_{10}$
 (c) $10110111_2 = ?_{10}$ (d) $67.24_8 = ?_{10}$
 (e) $10100.1101_2 = ?_{10}$ (f) $F3A5_{16} = ?_{10}$
 (g) $12010_3 = ?_{10}$ (h) $AB3D_{16} = ?_{10}$
 (i) $7156_8 = ?_{10}$ (j) $15C.38_{16} = ?_{10}$

2.6. Выполните следующие преобразования из одной системы счисления в другую:

- (a) $125_{10} = ?_2$ (b) $3489_{10} = ?_8$
 (c) $209_{10} = ?_2$ (d) $9714_{10} = ?_8$
 (e) $132_{10} = ?_2$ (f) $23851_{10} = ?_{16}$
 (g) $727_{10} = ?_5$ (h) $57190_{10} = ?_{16}$
 (i) $1435_{10} = ?_8$ (j) $65113_{10} = ?_{16}$

2.7. Сложите следующие пары двоичных чисел, указав все переносы:

- (a) $\begin{array}{r} 110101 \\ + 11001 \end{array}$ (b) $\begin{array}{r} 101110 \\ + 100101 \end{array}$ (c) $\begin{array}{r} 11011101 \\ + 11000111 \end{array}$ (d) $\begin{array}{r} 1110010 \\ + 1101101 \end{array}$

2.8. Повторите упражнение 2.7, выполняя вычитание вместо сложения и указывая заемы, а не переносы.

2.9. Сложите следующие пары восьмеричных чисел:

- (a) $\begin{array}{r} 1372 \\ + 4631 \end{array}$ (b) $\begin{array}{r} 47135 \\ + 5125 \end{array}$ (c) $\begin{array}{r} 175214 \\ + 152405 \end{array}$ (d) $\begin{array}{r} 110321 \\ + 56573 \end{array}$

2.10. Сложите следующие пары шестнадцатеричных чисел:

- (a) $\begin{array}{r} 1372 \\ + 4631 \end{array}$ (b) $\begin{array}{r} 4F1A5 \\ + B8D5 \end{array}$ (c) $\begin{array}{r} F35B \\ + 27E6 \end{array}$ (d) $\begin{array}{r} 1B90F \\ + C44E \end{array}$

2.11. Для каждого из десятичных чисел +18, +115, +79, -49, -3 и -100 записать его представление в виде 8-разрядного числа в прямом коде со знаком, в дополнительном коде и в обратном коде.

2.12. Укажите, происходит или не происходит переполнение при сложении следующих 8-разрядных двоичных чисел в дополнительном коде:

- (a) $\begin{array}{r} 11010100 \\ + 10101011 \end{array}$ (b) $\begin{array}{r} 10111001 \\ + 11010110 \end{array}$ (c) $\begin{array}{r} 01011101 \\ + 00100001 \end{array}$ (d) $\begin{array}{r} 00100110 \\ + 01011010 \end{array}$

2.13. Сколько ошибок может обнаружить код с минимальным расстоянием d ?

2.14. Чему равно наименьшее число проверочных битов, необходимых для получения двумерного кода с n информационными битами и с минимальным расстоянием 4?

Задачи

2.15. Вот задача, которая возбudit ваш аппетит. Что является шестнадцатеричным эквивалентом числа 61453_{10} ?

2.16. Каждое из следующих соотношений, содержащих арифметические действия, справедливо, по крайней мере, в одной из систем счисления. Найдите возможные основания соответствующих систем счисления.

(a) $1234 + 5432 = 6666$

(b) $41/3 = 13$

(c) $33/3 = 11$

(d) $23 + 44 + 14 + 32 = 223$

(e) $302/20 = 12.1$

(f) $\sqrt{41} = 5$

2.17. Первая экспедиция на Марс нашла там только развалины цивилизации. По остаткам материальной культуры и по рисункам ученые заключили, что создания, населявшие планету, были четырехлапыми существами с щупальцем, которое разветвлялось на конце на несколько хватких «пальцев». После долгих трудов исследователи смогли расшифровать математику марсиан. Они нашли уравнение

$$5x^2 - 50x + 125 = 0,$$

для которого были указаны решения: $x = 5$ и $x = 8$. Значение $x = 5$ казалось достаточно осмысленным, но другая величина $x = 8$ требовала какого-то объяснения. Тогда ученым пришла на ум обстоятельство, приведшее к развитию системы счисления на Земле, и они нашли свидетельство того, что подобная история была и у марсиан. Как вы думаете: сколько пальцев было у марсиан? (Из журнала *The Bent of Tau Beta Pi*, February 1956.)

2.18. Предположим, что число B , состоящее из $4n$ битов, представлено в виде n -разрядного шестнадцатеричного числа H . Докажите, что точное дополнение к B в двоичной системе счисления является точным дополнением к H в шестнадцатеричной системе. Сформулируйте и докажите аналогичное утверждение для восьмеричного представления.

2.19. Повторите задачу 2.18 для обратного кода числа B в двоичной системе (поразрядного дополнения до 1) и обратного кода числа H в шестнадцатеричной системе (поразрядного дополнения до 15).

2.20. Для заданного x из интервала $-2^{n-1} \leq x \leq 2^{n-1} - 1$ примем, по определению, что $[x]$ является представлением x в дополнительном коде, выраженным положительным числом: $[x] = x$, если $x \geq 0$, и $[x] = 2^n - |x|$, если $x < 0$, где $|x|$ — абсолютное значение x . Пусть y — другое число из того же интервала, что и x . Доказать, что правила сложения в дополнительном коде, приведенные в параграфе 2.6, справедливы, убедившись в том, что следующее соотношение всегда верно:

$$[x + y] = [x] + [y] \bmod 2^n.$$

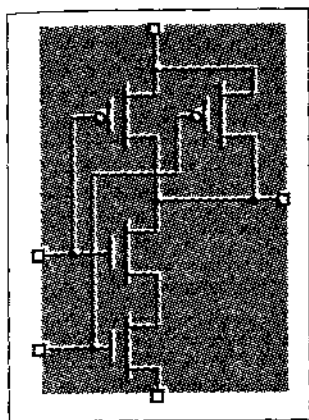
(Указания: Рассмотрите все четыре случая с возможными знаками x и y . Без потери общности можно полагать, что $|x| \geq |y|$.)

2.21. Повторите задачу 2.20, используя подходящие выражения и правила сложения в обратном коде.

2.22. Сформулируйте правило переноса при сложении чисел в дополнительном коде в терминах счета по модулю согласно рис. 2.3.

- 2.23.** Покажите, что число в дополнительном коде можно представить большим числом битов по правилу *знакового расширения*. То есть нужно показать, что для заданного n -разрядного двоичного числа X его представление в дополнительном коде m битами – при $m > n$ – можно получить путем добавления к его n -разрядному представлению слева $m - n$ битов, являющихся копиями знакового бита числа X .
- 2.24.** Покажите, что число в дополнительном коде можно представить меньшим числом битов путем удаления старших разрядов. Другими словами, нужно показать, что при заданном n -разрядном числе X , представленном в дополнительном коде, m -разрядное число Y в дополнительном коде, полученное путем вычеркивания d крайних левых битов в числе X , представляет собой то же самое число, что и X , в том и только в том случае, когда все удаляемые биты равны знаковому биту числа Y .
- 2.25.** Почему в английских терминах «two's complement» («двоичный дополнительный код») и «ones' complement» («двоичный обратный код») пунктуация различна? (См. первые две ссылки в Обзоре литературы.)
- 2.26.** Чтобы найти разность $X - Y$ n -разрядных двоичных чисел без знака X и Y , можно воспользоваться n -разрядным двоичным сумматором, выполняя операцию $X + \bar{Y} + 1$, где $\bar{Y}$ представляет собой поразрядное дополнение числа Y . Убедитесь в этом следующим образом. Во-первых, проверьте, что $(X - Y) = (X + \bar{Y} + 1) - 2^n$. Во-вторых, докажите, что условия возникновения переноса при n -разрядном сложении и заема при n -разрядном вычитании противоположны. То есть нужно доказать, что операция $X - Y$ требует заема из старшего разряда тогда и только тогда, когда в результате операции $X + \bar{Y} + 1$ не возникает переноса из старшего разряда.
- 2.27.** В большинстве случаев для представления произведения двух n -разрядных двоичных чисел в дополнительном коде требуется меньше, чем $2n$ битов. Действительно, имеется только один случай, когда необходимо $2n$ битов. Найдите его.
- 2.28.** Докажите, что число в дополнительном коде можно умножить на 2 путем его сдвига на один разряд влево, полагая при этом, что в младший разряд заносится 0 и игнорируя возможный перенос из старшего разряда, если только при этом не происходит переполнение. Сформулируйте правило обнаружения переполнения.
- 2.29.** Сформулируйте и докажете справедливость утверждения, аналогичного приведенному в задаче 2.28, скорректировав его применительно к умножению на 2 числа в обратном коде.
- 2.30.** Покажите, как осуществляется вычитание двоично-десятичных чисел, сформулировав правила возникновения заемов и необходимости производить коррекцию. Проверьте ваши правила на каждом из следующих примеров вычитания: $9 - 5$, $5 - 7$, $4 - 9$, $1 - 8$.
- 2.31.** Сколько существует различных возможных способов кодирования состояний контроллера светофора (см. табл. 2.12) 3-разрядным двоичным кодом?
- 2.32.** Перечислите все «плохие» границы на механическом кодирующем диске (рис. 2.5), где положение диска может быть отображено неправильно.

- 2.33.** Найдите зависимость от n числа «плохих» границ на механическом кодирующем диске, рассчитанном на n -разрядный двоичный код.
- 2.34.** Показания бортовых радиовысотомеров в гражданской авиации и на частных самолетах кодируются кодом Грея перед тем, как они передаются службам управления воздушным движением. Зачем?
- 2.35.** Каждый раз, когда включается лампочка накаливания, она подвергается перегрузке; поэтому в ряде случаев срок службы лампочки обусловлен не общим временем ее горения, а числом включений/выключений. Воспользуйтесь вашим знанием кодов и предложите для таких случаев способ удвоения срока службы ламп с двумя нитями накаливания.
- 2.36.** Сколько существует различных подкубов n -мерного куба?
- 2.37.** Найдите способ такого изображения 3-мерного куба на листе бумаги (или на другом двумерном объекте), при котором никакие линии не пересекаются, или докажите, что это невозможно.
- 2.38.** Решите задачу 2.36 для 4-мерного куба.
- 2.39.** Найдите выражение для числа m -мерных подкубов n -мерного куба при фиксированном значении m . (Вашим ответом должна быть функция от n и m .)
- 2.40.** Укажите группы проверок на четность для кода Хэмминга с 11 информационными битами и минимальным расстоянием 3.
- 2.41.** Выпишите кодовые слова кода Хэмминга с одним информационным битом.
- 2.42.** Найдите конфигурацию ошибки в 3-х битах, которую не обнаружат двумерные коды, изображенные на рис. 2.14, если «угловые» проверочные биты будут исключены.
- 2.43.** *Скорость кода* – это отношение числа информационных битов к полному числу битов в кодовом слове. Для эффективной передачи информации желательно, чтобы скорость была высокой и стремилась к 1. При различном числе информационных битов вплоть до 100 постройте график, который позволит сравнить скорости кодов с проверкой на четность с минимальным расстоянием 2 и кодов Хэмминга с минимальными расстояниями 3 и 4.
- 2.44.** У какого из кодов с минимальным расстоянием 4 скорость выше: у двумерного кода или у кода Хэмминга? Сопроводите свой ответ таблицей в духе табл. 2.15, включив в нее скорость, а также число проверочных и информационных битов для каждого из кодов с числом информационных битов до 100.
- 2.45.** Постройте код с четырьмя информационными битами и с минимальным расстоянием 6. Приведите список его кодовых слов.
- 2.46.** Опишите действия, которые необходимо выполнить в системе RAID для записи новых данных в информационный блок b на диске d , чтобы данные можно было восстановить в случае ошибки в блоке b на любом из дисков. Требуемое число обращений к дискам нужно минимизировать.
- 2.47.** Изобразите форму сигналов – подобно тому, как это сделано на рис. 2.17, – для набора битов 10101110 при последовательной передаче их с помощью кодов NRZ, NRZI, RZ, BPRZ и манчестерского кода в предположении, что биты передаются слева направо.



3 глава

ЦИФРОВЫЕ СХЕМЫ

Вопреки навязчивой рыночной рекламе мы живем в аналоговом, а не в цифровом мире. Напряжения, ток и другие физические величины в реальных схемах принимают сколь угодно много значений в зависимости от свойств реальных устройств, состоящих из этих схем. Поскольку реальные величины являются непрерывными, мы можем использовать физическую величину, например, напряжение сигнала в схеме, для представления действительного числа (например, 3.14159265358979 вольт представляют математическую постоянную π с точностью 15 десятичных знаков).

К сожалению, в реальных схемах трудно обеспечить стабильность и точность представления физических величин. На параметры схем влияют многие обстоятельства, в том числе технологические допуски, температура, напряжение источника питания, космические лучи и шум, создаваемый другими схемами. Если бы мы использовали аналоговое напряжение для представления π , то обнаружили бы, что величина π , являющаяся абсолютной математической константой, указывается с погрешностью в пределах 10% или больше.

Кроме того, многие математические и логические операции трудно или невозможно выполнить, используя аналоговые величины. При определенном умении можно создать аналоговую схему, выходное напряжение которой представляет собой квадратный корень входного напряжения. Однако никто никогда не создавал аналоговых схем со 100 входами и 100 выходами, у которых выходные напряжения представляли бы собой набор входных напряжений, упорядоченных по величине.

Цель этой главы — дать твердые практические знания электрических свойств цифровых схем, достаточные для понимания и построения реальных схем и систем. В последующих главах мы увидим, что с помощью современных программных средств можно «создавать» схемы абстрактно, используя для этого языки описания схем, и с помощью моделирующих программ проверять их работоспособность. Однако для того, чтобы создавать реальные, высокотехнологичные схемы на уровне плат или на уровне ИС, вам все же необходимо понять большую часть материала этой главы. Если вы хотите начать с разработки и моделирования абстрактных схем, то можно прочитать только первый параграф этой главы, а к остальной части вернуться позже.

3.1. Логические сигналы и вентили

Цифровая логика (digital logic) скрывает подводные камни аналогового мира, отображая бесконечный набор реальных значений физической величины в два подмножества, соответствующих только двум возможным числам или *логическим значениям (logic values)*: 0 и 1. В результате цифровые логические схемы можно анализировать и разрабатывать функционально, используя алгебру переключений, таблицы и другие абстрактные средства, удобные для описания того, как «ведут себя» нули и единицы в схеме.

Логическую величину 0 или 1 часто называют *двоичной цифрой (binary digit)* или *битом (bit)*. Если для решения какой-либо задачи требуется больше двух дискретных значений, можно воспользоваться дополнительными битами; набор из n битов представляет 2^n различных значений.

Примеры физических явлений, обычно используемых для представления битов в современной (и не в очень современной) цифровой технике, приведены в табл. 3.1. У большинства явлений имеется область неопределенности между состояниями 0 и 1 (например: напряжение, равное 1.8 В, слабо светящийся индикатор, лишь частично заряженный конденсатор и т.д.). Эта область неопределенности необходима для того, чтобы состояния 0 и 1 могли быть однозначно определены и надежно обнаружены. Если границы, отделяющие состояния 0 и 1, слишком близки, то шуму легче исказить результаты.

При обсуждении электронных логических схем, выполненных по технологии КМОП и ТТЛ, разработчики часто используют слова «низкий уровень» (LOW) и «высокий уровень» (HIGH) вместо «0» и «1»; им постоянно приходится помнить о том, что они имеют дело с реальными схемами, а не с абстрактными величинами:

низкий уровень — это сигнал в диапазоне численно малых напряжений, который интерпретируется как логический 0;

высокий уровень — это сигнал в диапазоне численно больших напряжений, который интерпретируется как логическая 1.

Заметьте, что присвоение значений 0 и 1 низкому и высокому уровням несколько произвольно. Присвоение значения 0 низкому уровню, а значения 1 высокому уровню выглядит наиболее естественным и называется *положительной логикой (positive logic)*. Противоположное соответствие, то есть 1 — низкий уровень и 0 — высокий уровень, используется не часто и называется *отрицательной логикой (negative logic)*.

Поскольку одному и тому же двоичному значению соответствует широкий диапазон значений физической величины, цифровая логика слабо чувствительна к замене компонентов, вариациям напряжения питания и шуму. Кроме того, для восстановления «ослабленных» значений и преобразования их в «сильные» можно использовать *буферные усилители (buffer amplifiers)*, и тогда цифровые сигналы можно передать на любое расстояние без потери информации. Например, буферный КМОП-усилитель преобразует любое входное напряжение высокого уровня в выходное напряжение, очень близкое к 5.0 В, и любое входное напряжение низкого уровня в выходное напряжение, очень близкое к 0.0 В.

Табл. 3.1. Физические состояния, представляющие биты в компьютерах и системах памяти различного типа

Технология	Состояние, представляющее бит	
	0	1
Пневматическая логика	Жидкость при низком давлении	Жидкость при высоком давлении
Релейная логика	Разомкнутая цепь	Замкнутая цепь
КМОП-логика	0 – 1.5 В	3.5 – 5.0 В
ТТЛ-логика	0 – 0.8 В	2.0 – 5.0 В
Волоконная оптика	Свет выключен	Свет включен
Динамическая память	Конденсатор разряжен	Конденсатор заряжен
Энергонезависимая стираемая память	Электроны захвачены	Электроны освобождены
Бинолярное ПЗУ	Перемычка пережжена	Перемычка цела
Память на ЦМД	Магнитный домен отсутствует	Магнитный домен присутствует
Магнитная лента или диск	Направление магнитного потока «-»	Направление магнитного потока «+»
Полимерная память	Молекула в состоянии А	Молекула в состоянии В
CD-ROM	Отсутствие впадины («кратера»)	Наличие впадины
Перезаписываемый компакт-диск	Краситель в кристаллическом состоянии	Краситель в некристаллическом состоянии

Не вдаваясь в детали, логическую схему можно представить просто как «черный ящик» с некоторым числом входов и выходов. Например, на рис. 3.1 показана логическая схема с тремя входами и одним выходом. Однако такое представление не дает описания того, как реагирует схема на входные сигналы.

Чтобы получить точное описание электрических процессов в электронной схеме, требуется большое количество информации. Однако, поскольку предполагается, что на входы цифровой логической схемы поступают только дискретные сигналы 0 и 1, «логическую» операцию, выполняемую схемой, можно описать с помощью таблицы, в которой игнорируются электрические процессы и перечислены только дискретные значения 0 и 1.

**Рис. 3.1.** «Черный ящик», представляющий логическую схему с тремя входами и одним выходом

Логическая схема, выходные сигналы которой зависят только от значений ее входных сигналов в данный момент времени, называется *комбинационной схемой* (*combinational circuit*). Операция, выполняемая такой схемой, полностью описывается *таблицей истинности* (*truth table*), в которой перечислены все комбинации входных сигналов и соответствующие им значения сигналов на выходе. Табл. 3.2 представляет собой таблицу истинности для логической схемы с тремя входами X , Y и Z и одним выходом F .

Табл. 3.2. Таблица истинности для комбинационной логической схемы

X	Y	Z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Схема с памятью, выходные сигналы которой зависят от текущих значений входных сигналов и от последовательности значений входных сигналов в прошлом, называется *последовательностной схемой* (*sequential circuit*). Поведение такой схемы можно описать *таблицей состояний* (*state table*), которая определяет сигнал на ее выходе и следующее ее состояние в зависимости от текущего состояния и значений сигналов на входах. Последовательностные схемы будут рассмотрены в главе 7.

Как будет показано в параграфе 4.1, для построения любой комбинационной схемы достаточно только трех основных логических схем, реализующих функции И, ИЛИ и НЕ. На рис. 3.2 приведены таблицы истинности и условные обозначения логических «вентилей», выполняющих эти функции. Обозначения и таблицы истинности для схем И и ИЛИ можно расширить на любое число входов. Функции, реализуемые схемами, легко определяются словами:

- Схема И (*AND gate*) вырабатывает 1 на выходе только в том случае, когда на всех ее входах присутствуют 1.
- Схема ИЛИ (*OR gate*) вырабатывает 1 на выходе только в том случае, когда 1 присутствует хотя бы на одном ее входе.
- Схема НЕ (*NOT gate*), обычно называемая *инвертором* (*inverter*), вырабатывает на выходе сигнал, противоположный входному сигналу.

Кружок на выходе инвертора является *символом инверсии* (*inversion bubble*) и используется в этом и других изображениях логических элементов для обозначения операции «инвертирования».

Обратите внимание, что при определении функций И и ИЛИ нам достаточно было задать только условия на входе, при которых на выходе вырабатывается 1,

поскольку в случае, когда выходной сигнал не 1, существует лишь одна возможность: он должен быть равен 0.

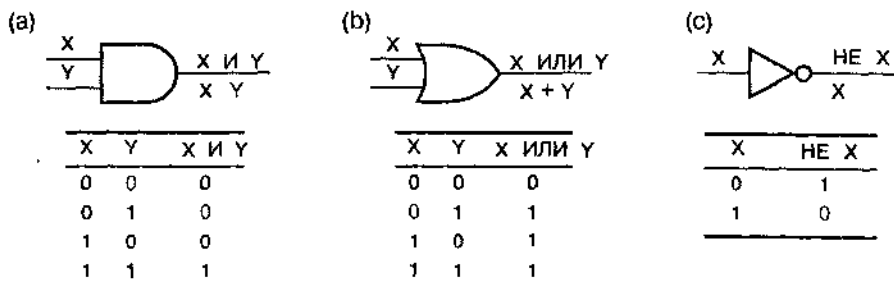


Рис. 3.2. Основные логические элементы: (a) И; (b) ИЛИ; (c) НЕ (инвертор)

Еще две логические функции получены путем объединения функции НЕ с функциями И и ИЛИ в одном вентиле. На рис. 3.3 показаны условные обозначения и таблицы истинности для этих схем; функции, реализуемые этими схемами, также легко описать словами:

- *Схема И-НЕ (NAND gate)* вырабатывает на выходе сигнал, противоположный сигналу на выходе схемы И, то есть 0 только в том случае, когда на всех ее входах присутствуют 1.
- *Схема ИЛИ-НЕ (NOR gate)* вырабатывает на выходе сигнал, противоположный сигналу на выходе схемы ИЛИ, то есть 0 только в том случае, когда хотя бы на одном из ее входов присутствует 1.

Так же, как для схем И и ИЛИ, условные обозначения и таблицы истинности для схем И-НЕ и ИЛИ-НЕ можно расширить на любое число входов.

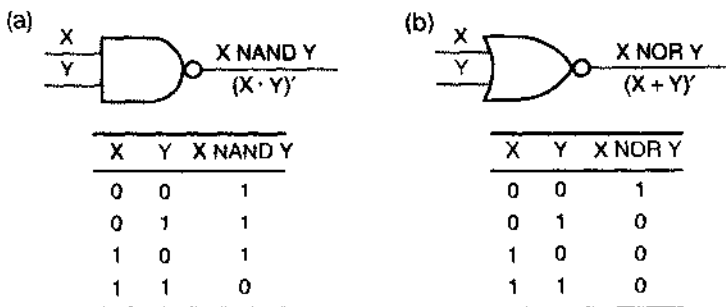


Рис. 3.3. Логические схемы с инверсией: (a) И-НЕ; (b) ИЛИ-НЕ

Рис. 3.4 иллюстрирует применения логических схем И, ИЛИ и НЕ для реализации функции F, соответствующей таблице истинности табл.3.2. В главе 4 вы узнаете, как переходить от таблицы истинности к логической схеме и обратно, а также познакомитесь с системой обозначений алгебры переключений, использованной на рис. 3.2–3.4.

Реальные логические схемы функционируют, кроме того, еще в одном аналоговом измерении — во времени. В качестве примера на рис. 3.5 приведены временные диаграммы, показывающие возможную реакцию схемы, изображенной на

рис. 3.4, на меняющуюся во времени комбинацию входных сигналов. Из временных диаграмм видно, что логические сигналы не переходят с одного уровня на другой мгновенно, а имеется запаздывание между изменением сигналов на входе и соответствующим изменением выходного сигнала. Позже в этой главе вы изучите некоторые причины этих задержек и то, как их определять и учитывать в реальных схемах. Вы также узнаете, как в большинстве последовательностных схем этот аналоговый по времени процесс можно вообще игнорировать и считать, что такая схема совершает переход из одного дискретного состояния в другое в моменты времени, определяемые тактовым сигналом.

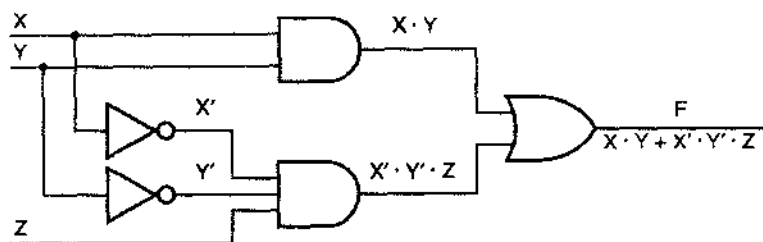


Рис. 3.4. Логическая схема, соответствующая таблице истинности в табл. 3.2

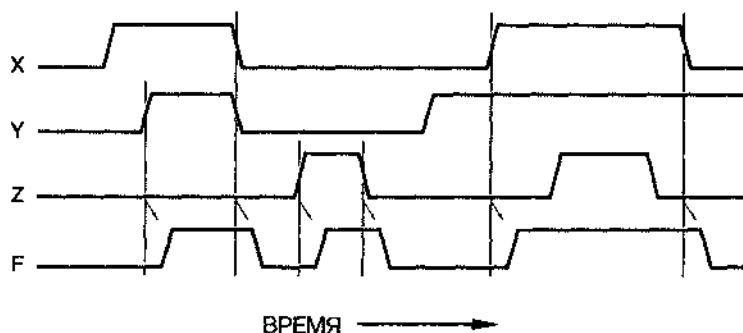


Рис. 3.5. Временные диаграммы для логической схемы, изображенной на рис. 3.4

ДЛЯ НЕ СПЕЦИАЛИСТОВ В ОБЛАСТИ ЭЛЕКТРОНИКИ НАДЕЖДА ОСТАЕТСЯ

Если весь этот электрический «материал» вас беспокоит, не волнуйтесь, по крайней мере пока. Остальная часть книги написана так, — насколько это было возможно, — чтобы ее можно было читать независимо от материала этой главы. Но позже, если вам придется проектировать и создавать реальные цифровые системы, у вас возникнет потребность в этом материале.

Итак, даже если вы ничего не знаете об аналоговой электронике, вам нужно понимать логику работы цифровых схем. Но когда в процессе разработки и отладки приходится учитывать временные соотношения, любой разработчик цифровой

техники должен временно отказаться от «цифровой абстракции» и рассмотреть аналоговые явления, которые ограничивают или нарушают работу цифровых схем. Оставшаяся часть этой главы подготовит вас к этому путем рассмотрения электрических характеристик схем.

3.2. Семейства логических схем

Существует очень много способов проектирования электронных логических схем. В первых электрически управляемых логических схемах, разработанных в 30-е годы фирмой Bell Laboratories, применялись реле. В середине 40-х годов в первой электронной цифровой вычислительной машине «Эниак» (Eniac) использовались логические схемы на вакуумных лампах. В «Эниаке» было около 18000 ламп и примерно столько же логических вентилях, что совсем не много по сегодняшним меркам, когда кристаллы микропроцессоров содержат десятки миллионов транзисторов. Однако «Эниак» мог нанести вам гораздо больше неприятностей, чем чип, если бы вы «урунили» его – он имел 100 футов в длину, 10 футов в высоту, был глубиной 3 фута и потреблял мощность 140 кВт!

Изобретение *полупроводникового диода (semiconductor diode)* и *биполярного плоскостного транзистора (bipolar junction transistor)* привело к появлению в конце 50-х годов более быстрых и мощных компьютеров меньших размеров. В 60-е годы изобретение *интегральной схемы (ИС; integrated circuit, IC)* позволило создавать на одном кристалле большое число диодов, транзисторов и других компонентов, и компьютеры стали еще лучше.

Первые семейства интегральных логических схем также появились в 60-е годы. *Семейство логических схем (logic family)* – это набор различных ИС, имеющих сходные входные, выходные и внутренние характеристики, но выполняющих различные логические функции. Микросхемы одного семейства можно соединять между собой для реализации любой желаемой логической функции. С другой стороны, микросхемы разных семейств могут быть не совместимы из-за различных напряжений питания или из-за других уровней, представляющих логические значения.

Наиболее удачным оказалось *семейство логических схем на биполярных транзисторах (bipolar logic family)* – *транзисторно-транзисторная логика (ТТЛ; transistor-transistor logic, TTL)*. ТТЛ-схемы впервые появились в 60-х годах и сегодня представлены фактически несколькими семействами логических схем, совместимых друг с другом, но отличающихся по быстродействию, потребляемой мощности и стоимости. В разных частях цифровой системы могут быть использованы компоненты нескольких различных ТТЛ-семейств в соответствии с целями и ограничениями проекта. Хотя в 90-е годы ТТЛ-схемы были в значительной степени заменены КМОП-схемами, в учебных лабораториях все же можно встретить ТТЛ-компоненты; поэтому в параграфе 3.10 пойдет речь о семействе ТТЛ.

За десять лет до изобретения биполярного плоскостного транзистора были запатентованы принципы работы транзистора другого типа, названного *полевым транзистором со структурой «металл-окисел-полупроводник» (metal-oxide semiconductor field-effect transistor, MOSFET)*, или просто *МОП-транзистором (MOS transistor)*. Однако первое время, до 60-х годов, МОП-транзисторы было

трудно изготавливать, и лишь благодаря ряду достижений логические схемы и устройства памяти на основе МОП-транзисторов стали реальными. Но все же МОП-схемы значительно отставали от биполярных интегральных схем по быстродействию и были непривлекательны для применения только в отдельных случаях из-за меньшей потребляемой мощности и большей степени интеграции.

Начиная с середины 80-х годов, прогресс в создании МОП-схем, особенно *комплементарных МОП-схем* [КМОП-схем; *complementary MOS (CMOS) circuits*], позволил значительно улучшить их характеристики и такие схемы стали более популярными. В большинстве новых сверхбольших интегральных схем типа микропроцессоров и блоков памяти использована КМОП-технология. Кроме того, в приложениях малого и среднего уровня сложности, для которых когда-то были выбраны логические семейства ТТЛ, теперь, вероятно, будут применяться КМОП-схемы с аналогичными функциональными возможностями, но с большим быстродействием и меньшей потребляемой мощностью. КМОП-схемы теперь составляют подавляющую часть мирового рынка ИС.

КМОП-логика является одновременно наиболее подходящей и самой простой для создания логических схем. Начиная со следующего параграфа, мы опишем базовую структуру логических КМОП-схем и представим самые распространенные серийные семейства КМОП-логики.

Как следствие длительного перехода промышленности от ТТЛ-к КМОП-логике, многие КМОП-семейства были разработаны так, чтобы в какой-то мере быть совместимыми с семействами ТТЛ. В параграфе 3.12 мы покажем, как можно объединять в пределах одной системы ТТЛ-и КМОП-схемы.

3.3. КМОП-логика

Функциональное поведение логической КМОП-схемы понять довольно просто, даже если ваши знания аналоговой электроники не особенно глубоки. Главным элементом в структуре логических КМОП-схем являются описываемые ниже МОП-транзисторы; чаще всего логические КМОП-схемы только из них и состоят. Но до рассмотрения МОП-транзисторов и логических КМОП-схем, мы должны поговорить о логических уровнях.

3.3.1. Логические уровни КМОП-схем

Абстрактные логические элементы оперируют двоичными цифрами 0 и 1. Однако реальные логические схемы имеют дело с электрическими сигналами в виде уровней напряжения. В любой логической схеме имеется диапазон напряжений (или другие состояния схемы), соответствующий логическому 0, и другой, не перекрывающийся с ним диапазон напряжений, соответствующий логической 1.

Типичная логическая КМОП-схема работает от 5-вольтового источника питания. Такая схема может интерпретировать любое напряжение в диапазоне 0–1.5 В как логический 0 и напряжение в диапазоне 3.5–5.0 В – как логическую 1. Таким образом определяются низкий уровень и высокий уровень для 5-вольтовой КМОП-логики (рис. 3.6). Не предполагается, что напряжение окажется в промежуточной области (1.5–3.5 В), кроме интервалов времени, когда сигнал переходит от одного уровня к другому; в противном случае логические значения будут не определены

(то есть схема может интерпретировать их и как 0, и как 1). У КМОП-схем с другими напряжениями питания – например, 3.3 или 2.7 вольт – имеется аналогичное разделение диапазонов напряжений.

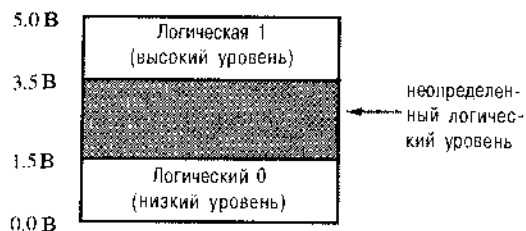


Рис. 3.6. Логические уровни для типичных логических КМОП-схем

3.3.2. МОП-транзисторы

МОП-транзистор можно представить как устройство с 3 выводами, которое действует подобно управляемому напряжением резистору. Как изображено на рис. 3.7, напряжение, приложенное ко входу, изменяет сопротивление между нижним и верхним выводами. В логических схемах МОП-транзистор работает так, что его сопротивление всегда либо очень велико (при этом транзистор «закрыт»), либо очень мало (при этом транзистор «открыт»).

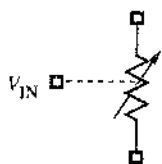


Рис. 3.7. Представление МОП-транзистора в виде резистора, сопротивление которого зависит от управляющего напряжения

Существуют два типа МОП-транзисторов: с *n*-каналом и с *p*-каналом; названия определяются типом полупроводникового материала, использованного в качестве управляемого резистора. Условное обозначение *МОП-транзистора с каналом n-типа* [*n*МОП-транзистор; *n-channel MOS (NMOS) transistor*] приведено на рис. 3.8. Выводы имеют следующие названия: *затвор* (*gate*), *исток* (*source*) и *сток* (*drain*). Глядя на условное обозначение транзистора, можно догадаться, что в нормальных условиях потенциал стока выше потенциала истока.

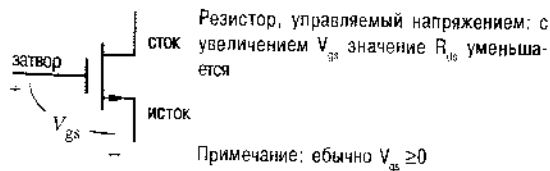


Рис. 3.8. Условное обозначение МОП-транзистора с каналом *n*-типа

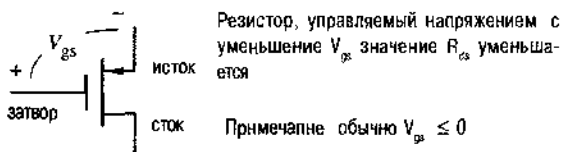
Напряжение между затвором и истоком (V_{gs}) у МОП-транзистора с каналом *n*-типа обычно равно нулю или положительно. Если $V_{gs} = 0$, то сопротивление между стоком и истоком (R_{ds}) очень велико и составляет, по крайней мере, мегаом (10^6 Ом) или больше. По мере увеличения V_{gs} (то есть с увеличением напряжения на затворе) R_{ds} уменьшится до очень малого значения порядка 10 Ом, а у некоторых транзисторов и меньше.

Условное обозначение *МОП-транзистора с каналом p-типа* [*p*МОП-транзистор; *p-channel MOS (PMOS) transistor*] приведено на рис. 3.9. Его функциониро-

вание аналогично работе МОП-транзистора с каналом n -типа, за исключением того, что обычно исток имеет более высокий потенциал, чем сток, а V_{gs} равно нулю или отрицательно. Если V_{gs} равно нулю, то сопротивление между истоком и стоком (R_{ds}) очень велико. С уменьшением V_{gs} (когда напряжение на затворе становится все более отрицательным) R_{ds} уменьшается, принимая в конце концов очень малое значение.

Затвор МОП-транзистора называют изолированным, поскольку он отделен от истока и стока изолирующим материалом, имеющим очень большое сопротивление. Тем не менее, напряжение на затворе создает электрическое поле, которое увеличивает или уменьшает ток, текущий от истока к стоку. Этот «полевой эффект» дал название транзистору — «полевой транзистор».

По этой причине, независимо от напряжения на затворе, никакой ток практически не течет от затвора к истоку или от затвора к стоку. Сопротивления между затвором и другими выводами очень велики, намного больше мегаома. Ток, протекающий по этим сопротивлениям, очень мал, обычно меньше одного микроампера (мкА, 10^{-6} А), и называется *током утечки (leakage current)*.



Резистор, управляемый напряжением с уменьшением V_{gs} значение R_{ds} уменьшается

Примечание: обычно $V_{gs} \leq 0$

Рис. 3.9. Условное обозначение МОП-транзистора с каналом p -типа

Само условное обозначение МОП-транзистора напоминает нам, что между затвором и двумя другими выводами нет никакого соединения. Однако изображение МОП-транзистора наводит на мысль, что затвор имеет емкостную связь с истоком и стоком. В быстродействующих схемах мощность, расходуемая при заряде и разряде этих емкостей при каждом изменении входного сигнала, составляет заметную долю потребляемой схемой мощности.

3.3.3. Базовая схема КМОП-инвертора

Схемы *КМОП-логики (CMOS logic)* образуются в результате совместного использования дополняющих друг друга n МОП- и p МОП-транзисторов. Самой простой КМОП-схемой является логический инвертор, для которого необходимо по одному транзистору каждого типа, соединенных как показано на рис. 3.10(а). Напряжение питания V_{DD} обычно может быть в диапазоне 2–6 В и наиболее часто выбирается равным 5.0 В для совместимости с ТТЛ-схемами.

В идеальном случае поведение схемы КМОП-инвертора можно описать всего лишь двумя строками таблицы, приведенной на рис. 3.10(б):

ИМПЕДАНС И СОПРОТИВЛЕНИЕ

Формально между терминами «импеданс» и «сопротивление» имеется различие, но инженеры-электрики часто используют эти термины как равнозначные. Так же поступаем в этой книге и мы.

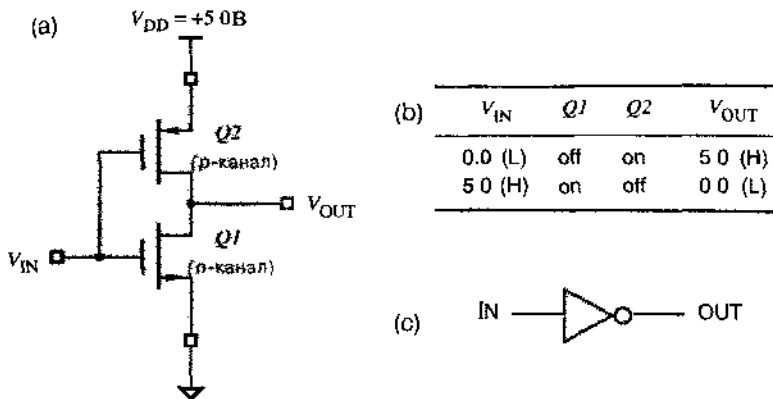


Рис. 3.10. КМОП-инвертор (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, on – открыт, off – закрыт), (с) условное обозначение

1. V_{IN} равно 0.0 В. В этом случае нижний, n -канальный транзистор $Q1$ закрыт, так как у него напряжение V_{gs} равно 0 В, а верхний, p -канальный транзистор $Q2$ открыт, так как у него напряжение V_{gs} имеет большое по величине отрицательное значение (-5.0 В). Поэтому сопротивление транзистора $Q2$, включенного между шиной питания (V_{DD} , + 5.0 В) и выходом (V_{OUT}), мало, и выходное напряжение равно 5.0 В.
2. V_{IN} равно 5.0 В. При этом транзистор $Q1$ открыт, поскольку у него напряжение V_{gs} равно +5.0 В, а транзистор $Q2$ закрыт, так как у него V_{gs} равно 0. Таким образом, транзистор $Q1$ представляет собой малое сопротивление между выходом схемы и землей, и выходное напряжение равно 0 В.

Из сказанного ясно, как ведет себя логический инвертор: при напряжении 0 вольт на входе выходное напряжение равно 5 вольтам, и наоборот.

Другой способ наглядного представления работы КМОП-схемы состоит в изображении транзисторов в виде ключей. Как показано на рис. 3.11(а), n -канальный (нижний) транзистор заменяется ключом с нормально разомкнутым контактом, а p -канальный (верхний) транзистор – нормально замкнутым ключом. При подаче на вход высокого напряжения состояние каждого из ключей изменяется на состояние, противоположное первоначальному, как показано на рис. 3.11(б).

Модель с ключами позволяет нагляднее представить работу КМОП-инвертора. Как показано на рис. 3.12, для p - и n -канальных транзисторов используются различные условные обозначения, чтобы отразить логику их работы. Когда к затвору n -канального транзистора ($Q1$) приложено напряжение высокого уровня, он находится в «открытом» состоянии и ток течет от стока к истоку; это кажется достаточно естественным. Противоположная ситуация наблюдается в отношении p -канального транзистора ($Q2$). Он находится в «открытом» состоянии, когда к его затвору приложено напряжение низкого уровня; кружок на затворе указывает на инвертирование.

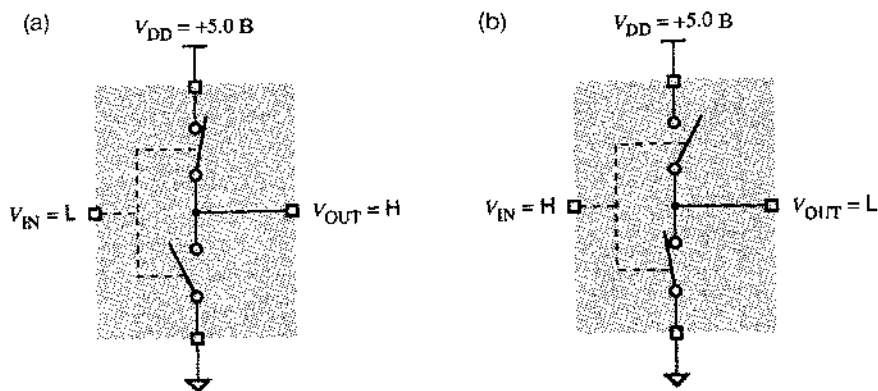


Рис. 3.11. Модель КМОП-инвертора с использованием ключей: (а) низкое входное напряжение; (б) высокое входное напряжение (L – низкий уровень, H – высокий уровень)

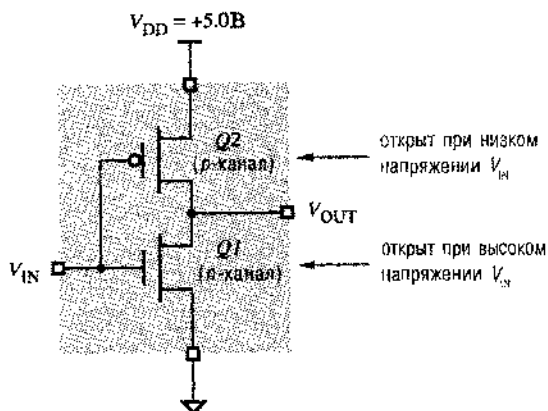


Рис. 3.12. Логика работы КМОП-инвертора

ЧТО ЗАКЛЮЧЕНО В ОБОЗНАЧЕНИЯХ?

Индекс "DD" в обозначении " V_{DD} " относится к выводам *стока* МОП-транзисторов. Это может показаться странным, так как в КМОП-инверторе напряжение источника питания V_{DD} фактически соединено с выводом *истока* pМОП-транзистора. Однако логические КМОП-схемы ведут свое происхождение от логических *н*МОП-схем, где источник питания *был* соединен со стоком pМОП-транзистора через резистор нагрузки, и обозначение " V_{DD} " осталось. Заметьте также, что в КМОП- и *н*МОП-схемах землю иногда обозначают как " V_{SS} ". Некоторые авторы и большинство производителей схем используют обозначение " V_{CC} " для напряжения питания КМОП-схем, так как оно используется в ТТЛ-схемах, которые исторически предшествовали КМОП-схемам. Имея в виду, что можно использовать оба обозначения, начиная с параграфа 3.4 мы будем применять обозначение " V_{CC} ".

3.3.4. КМОП-схемы И-НЕ и ИЛИ-НЕ

Используя n МОП- и p МОП-транзисторы можно создать схемы И-НЕ и ИЛИ-НЕ. Чтобы образовать вентиль с k - входами, необходимо k p -канальных и k n -канальных транзисторов. На рис. 3.13 показана 2-входовая КМОП-схема И-НЕ. Если хотя бы на одном из входов сигнал имеет низкий уровень, то выход Z через малое сопротивление «открытого» p -канального транзистора подключен к шине питания V_{DD} , а цепь на землю разорвана «закрытым» n -канальным транзистором. Если на обоих входах присутствует сигнал высокого уровня, то цепь в сторону шины питания V_{DD} разорвана и выход Z через малое сопротивление подключен к земле. Рис. 3.14 иллюстрирует работу схемы И-НЕ на модели с ключами.

На рис. 3.15 приведена схема ИЛИ-НЕ в КМОП-исполнении. Если сигналы на обоих входах имеют низкий уровень, то выход Z через малые сопротивления «открытых» p -канальных транзисторов подключен к шине питания V_{DD} , а цепь на землю разорвана «закрытыми» n -канальными транзисторами. Если сигнал на любом из входов принимает высокий уровень, то цепь в сторону шины питания V_{DD} разорвана и выход Z через малое сопротивление подключен к земле.

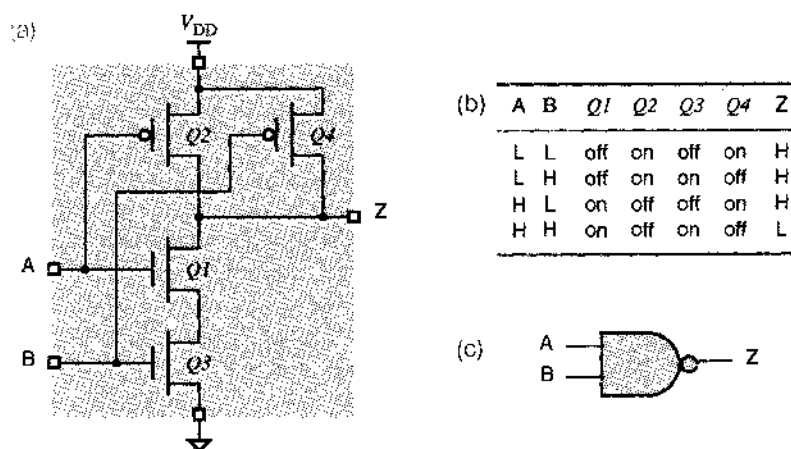


Рис. 3.13. Схема 2-входового КМОП-вентиль И-НЕ: (a) принципиальная схема; (b) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, off – закрыт, on – открыт); (c) условное обозначение

СРАВНЕНИЕ СХЕМ И-НЕ и ИЛИ-НЕ

КМОП-схемы И-НЕ и ИЛИ-НЕ имеют различные характеристики. При одной и той же площади кремневого кристалла, транзистор с каналом n -типа имеет меньшее сопротивление в «открытом» состоянии, чем транзистор с каналом p -типа. Поэтому у последовательно включенных k транзисторов с n -каналом сопротивление в «открытом» состоянии меньше, чем у k транзисторов с p -каналом. В результате быстродействие схемы И-НЕ с k входами обычно выше и предпочтительнее, чем у k -входовой схемы ИЛИ-НЕ, и поэтому схемы И-НЕ предпочтительнее.

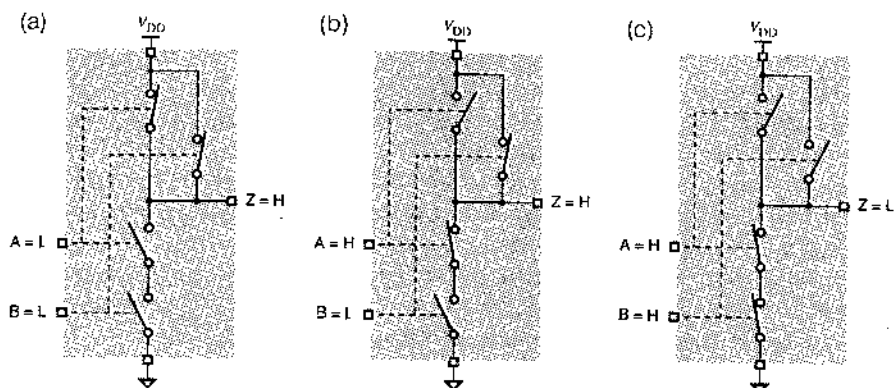


Рис. 3.14. Модель 2-входовой КМОП-схемы И-НЕ на основе ключей: (а) сигналы на обоих входах имеют низкий уровень (L); (б) сигнал на одном входе имеет высокий уровень (H); (с) сигналы на обоих входах имеют высокий уровень

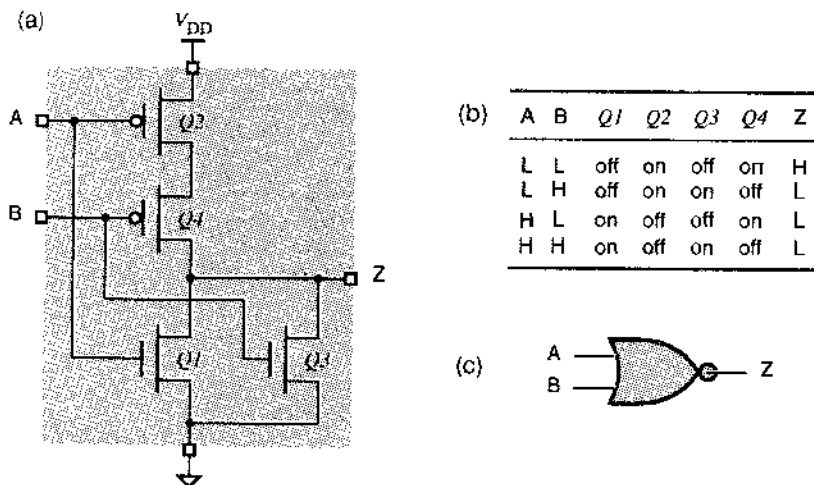


Рис. 3.15. 2-входовая КМОП-схема ИЛИ-НЕ: (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, off – закрыт, on – открыт); (с) условное обозначение

3.3.5. Коэффициент объединения по входу

Число входов, которые может иметь вентиль в конкретном логическом семействе, называется *коэффициентом объединения по входу (fan-in)*. КМОП-схемы с числом входов больше двух можно очевидным способом получить путем последовательно-параллельного расширения схем, представленных на рис. 3.13 и 3.15. Например, на рис. 3.16 показана 3-входовая КМОП-схема И-НЕ.

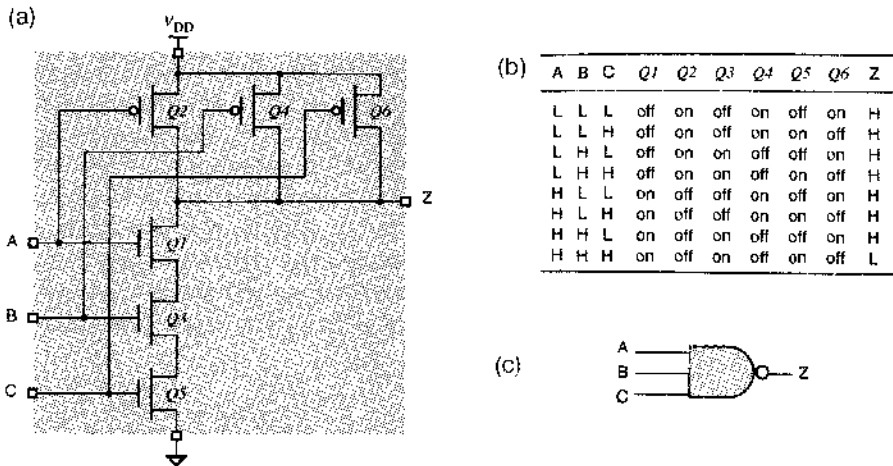


Рис. 3.16. 3-входовая КМОП-схема И-НЕ: (a) принципиальная схема; (b) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, off – закрыт, on – открыт); (c) условное обозначение

В принципе можно создавать КМОП-схемы И-НЕ и ИЛИ-НЕ с очень большим числом входов. Однако на практике сопротивление последовательно включенных «открытых» транзисторов обычно ограничивает коэффициент объединения по входу у КМОП-схем числом 4 для вентилях ИЛИ-НЕ и числом 6 для вентилях И-НЕ.

По мере увеличения числа входов разработчики КМОП-схем могут увеличивать размеры последовательно включенных транзисторов для уменьшения их сопротивления и соответствующей задержки переключения. Однако с некоторого момента этот способ становится неэффективным или непрактичным. Вентиль с большим числом входов можно сделать более быстрым и меньших размеров путем последовательного включения схем с меньшим числом входов. На рис. 3.17 показана логическая структура 8-входового КМОП-элемента И-НЕ. Полная задержка прохождения сигнала через 4-входовую схему И-НЕ, 2-входовую схему ИЛИ-НЕ и инвертор обычно меньше, чем задержка в одноуровневой 8-входовой схеме И-НЕ.

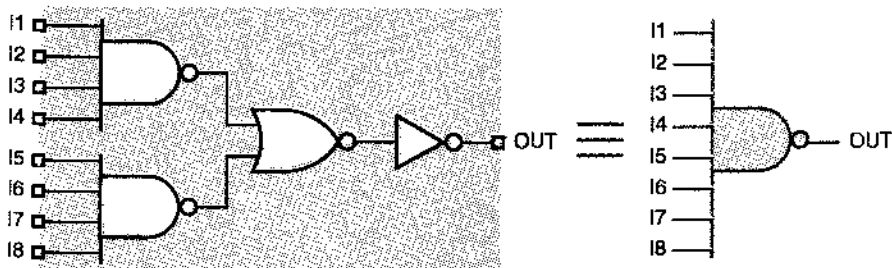


Рис. 3.17. Схемный эквивалент внутренней структуры 8-входового КМОП-элемента И-НЕ

3.3.6. Неинвертирующие вентили

В КМОП-логике и в большинстве других логических семейств простейшими являются схемы инверторов, а следом за ними идут элементы И-НЕ и ИЛИ-НЕ. Инверсия получается «бесплатно» и обычно невозможно создать неинвертирующий вентиль с меньшим числом транзисторов, чем в простом инверторе.

Неинвертирующий КМОП-буфер, а также логические схемы И и ИЛИ получаются в результате подключения инвертора к выходу соответствующего инвертирующего элемента. Реализованные таким образом неинвертирующий буфер и логический элемент И показаны на рис. 3.18 и рис. 3.19 соответственно. Комбинация схемы, приведенной на рис. 3.15(а), с инвертором дает логический элемент ИЛИ.

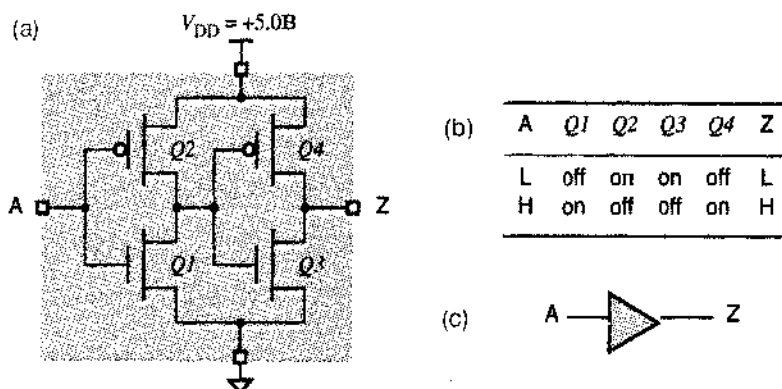


Рис. 3.18. Неинвертирующий КМОП-буфер: (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, off – закрыт, on – открыт); (с) условное обозначение

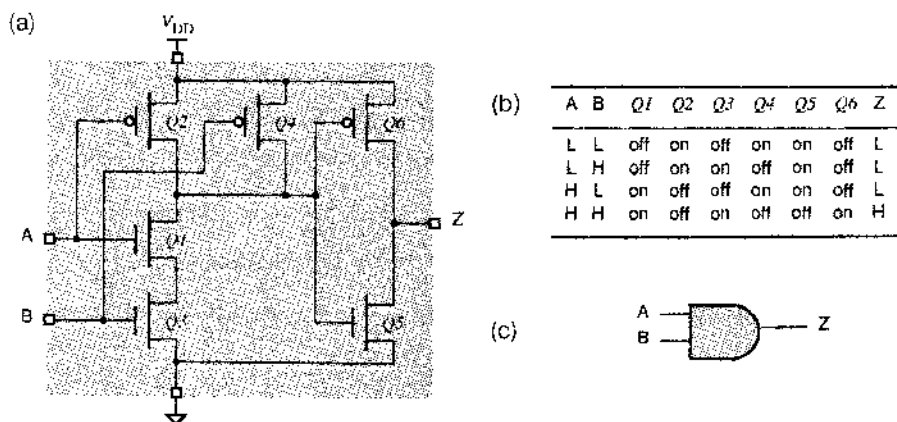


Рис. 3.19. 2-входовая КМОП-схема И: (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, off – закрыт, on – открыт); (с) условное обозначение

3.3.7. КМОП-схемы И–ИЛИ–НЕ и ИЛИ–И–НЕ

По КМОП-технологии всего лишь на одном «слое» транзисторов можно реализовать двухуровневую логику, то есть последовательное выполнение двух логических операций. Например, на рис. 3.20(а) представлена двухвходовая КМОП-схема И–ИЛИ–НЕ (AND-OR-INVERT, AOI gate) с двукратным объединением по ИЛИ. Таблица, описывающая работу этой схемы, приведена на рис. 3.20(б), а схема, реализующая соответствующую логическую функцию с помощью вентилях И и ИЛИ–НЕ, показана на рис. 3.21. Добавляя или удаляя транзисторы в схеме на рис. 3.20(а), можно получить функцию И–ИЛИ–НЕ с другим числом вентилях И и с другим числом входов у этих вентилях.

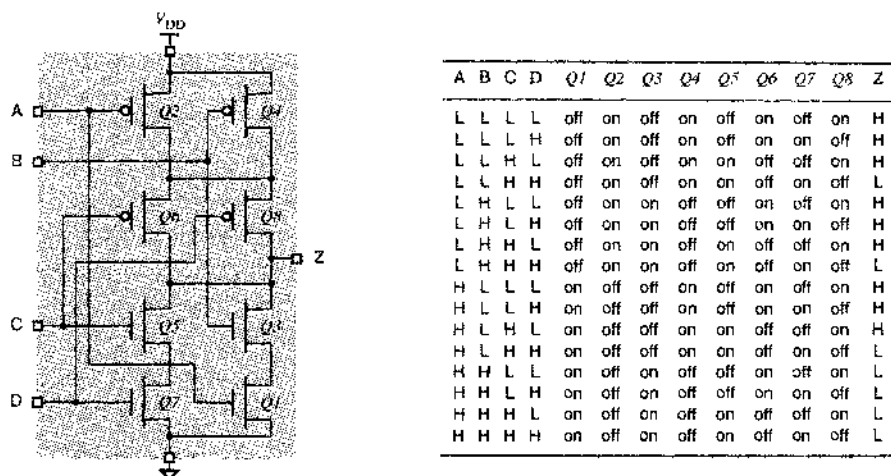


Рис. 3.20. КМОП-схема И–ИЛИ–НЕ: (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, off – закрыт, on – открыт)

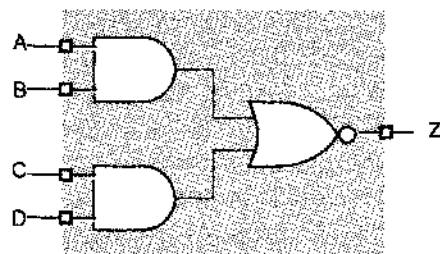


Рис. 3.21. Логическая схема КМОП-вентилей И–ИЛИ–НЕ

Содержание каждого из столбцов Q1–Q8 в таблице на рис. 3.20(б) зависит только от входного сигнала, поданного на затвор соответствующего транзистора. Последний столбец заполняется путем проверки для каждой входной комбинации, оказывается ли выход Z соединенным через «открытый» транзистор с шиной питания V_{DD} или он соединен с землей. Обратите внимание, что при любой комбинации входных сигналов выход никогда не бывает соединен одновременно с шиной питания V_{DD} и с землей; в этом случае напряжение на выходе было бы где-то посередине между

низким и высоким уровнями и не соответствовало бы ни одному из логических уровней, а выходная цепь потребляла бы чрезмерно большую мощность из-за малого сопротивления между шинной нитания V_{DD} и землей.

Можно также разработать схему, реализующую функцию ИЛИ-И-НЕ. Например, на рис. 3.22(а) приведена двухходовая КМОП-схема ИЛИ-И-НЕ (OR-AND-INVERT, OAI gate), а на рис. 3.22(б) – таблица, описывающая работу этой схемы; состояния транзисторов и значения сигналов в каждом столбце определены так же, как это было сделано для КМОП-схемы И-ИЛИ-НЕ. Схема, реализующая соответствующую логическую функцию с помощью вентилях ИЛИ и И-НЕ, показана на рис. 3.23.

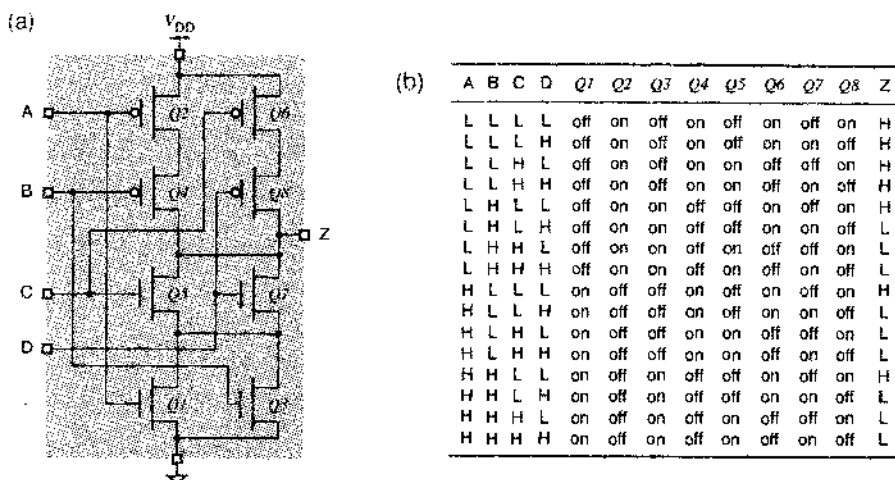


Рис. 3.22. КМОП-схема ИЛИ-И-НЕ: (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень, off – закрыт, on – открыт)

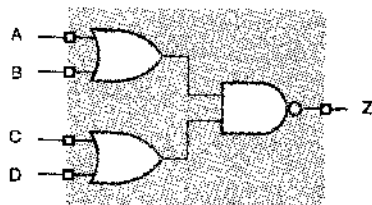


Рис. 3.23. Логическая схема КМОП-вентилей ИЛИ-И-НЕ

Быстродействие и другие электрические характеристики КМОП-схем И-ИЛИ-НЕ и ИЛИ-И-НЕ очень близки к параметрам одиночных КМОП-схем И-НЕ или ИЛИ-НЕ. В результате, эти схемы очень привлекательны, потому что могут выполнять двухуровневую логическую операцию (И-ИЛИ либо ИЛИ-И) с задержкой, соответствующей одному уровню. Большинство конструкторов цифровых устройств не затрудняются применением схем И-ИЛИ-НЕ в своих разработках. Однако в составе СБИС, выполненных по КМОП-технологии, эти схемы используют часто, так как многие языки описания схем могут автоматически преобразовывать схемы логики И/ИЛИ в схемы И-ИЛИ-НЕ, когда это целесообразно.

3.4. Электрические свойства КМОП-схем

В трех следующих параграфах обсуждаются электрические, а не логические аспекты работы КМОП-схем. Если вы проектируете реальные схемы, в которых используются КМОП- или другие логические семейства, то понимание рассматриваемых здесь вопросов является важным. Большая часть материала этого параграфа нацелена на то, чтобы служить основанием для уверенности, что «цифровая абстракция» действительно справедлива применительно к данной схеме. В частности, разработчик схемы или системы должен обеспечить выполнение ряда условий в отношении *предельных технических характеристик (engineering design margins)*, то есть гарантировать, что схема будет работать надлежащим образом даже в самых плохих условиях.

3.4.1. Общий обзор

В параграфах 3.5–3.7 мы рассмотрим следующие вопросы:

- *Логические уровни напряжения.* Гарантируется, что выходные напряжения работающих в нормальных условиях КМОП-устройств будут принадлежать диапазонам низкого и высокого уровней, определенным соответствующим образом. Принадлежность входных напряжений диапазонам низкого и высокого уровней они распознают в несколько более широких пределах. Изготовители КМОП-схем очень тщательно определяют эти диапазоны и условия эксплуатации, чтобы гарантировать совместимость различных элементов одного и того же семейства и обеспечить (при соблюдении определенных предосторожностей) возможность взаимодействия устройств из разных семейств.
- *Запас помехоустойчивости по постоянному току.* Неотрицательный запас помехоустойчивости по постоянному току гарантирует, что наибольшее напряжение низкого уровня на выходе всегда меньше самого высокого напряжения, которое на входе может надежно интерпретироваться как низкий уровень, а наименьшее напряжение высокого уровня на выходе всегда больше, чем самое низкое напряжение, которое на входе может надежно интерпретироваться как высокий уровень. Ясное представление о запасе помехоустойчивости особенно важно в тех случаях, когда используются схемы из нескольких различных семейств.
- *Коэффициент разветвления по выходу.* Этот параметр относится к числу и типу входов, подключаемых к данному выходу. Если к выходу схемы подключено слишком много входов, то ее запас помехоустойчивости по постоянному току может оказаться недостаточным. Кроме того, коэффициент разветвления по выходу может влиять на скорость, с которой выходной сигнал изменяется от одного значения к другому.
- *Быстродействие.* Время, необходимое для изменения выходного сигнала КМОП-схемы от низкого уровня до высокого, или наоборот, зависит как от внутренней структуры устройства, так и от характеристик других устройств, которыми управляет эта схема, в том числе от проводов или проводников печатной платы, подключенных к данному выходу. Мы рассмотрим две характеристики «быстродействия»: время перехода и задержку распространения.
- *Потребляемая мощность.* Мощность, потребляемая КМОП-схемой, зависит от нескольких факторов: от ее внутренней структуры, от значений сигналов,

поступающих на ее входы, от свойств других устройств, которыми управляет данная схема, и от того, как часто ее выходной сигнал изменяется между низкими и высокими уровнями.

- **Шум.** Основной причиной указания предельных технических характеристик является необходимость гарантировать работоспособность схемы в присутствии шума. Шум может создаваться рядом источников; некоторые из них перечислены ниже, начиная с наименее вероятных, вплоть до наиболее вероятных (хотя это и может показаться неожиданным):
 - космические лучи;
 - магнитные поля от близко расположенного оборудования;
 - пульсации напряжения источника питания;
 - процесс переключения в самих логических схемах.
- **Электростатический разряд.** Верите ли вы, что КМОП-схему можно вывести из строя, всего лишь касаясь ее?
- **Выходы с открытым стоком.** У некоторых КМОП-схем отсутствуют p -канальные транзисторы на выходе, нормально подключающие выход к источнику питания. Когда выходной сигнал должен иметь высокий уровень, выход такой схемы ведет себя подобно «отсутствию связи», что в некоторых случаях бывает полезно.
- **Выходы с тремя состояниями.** Некоторые КМОП-схемы имеют дополнительный управляющий вход «разрешение выхода» (*output enable*), который можно использовать для одновременного закрывания p -канального и n -канального выходных транзисторов. Можно образовать шину со многими источниками сигналов, объединяя также выходы нескольких схем, при условии, что какое-то логическое управляющее устройство обеспечит поступление сигнала разрешения выхода не более, чем на одну из этих схем.

3.4.2. Справочные данные и спецификация

Изготовители практических устройств предоставляют *справочные данные* (*data sheets*), в которых сообщается о логических и электрических характеристиках устройств. В табл. 3.3 приведены минимальные сведения об электрических характеристиках простой КМОП-схемы 54/74НС00, представляющей собой счетверенный логический элемент И-НЕ. Разные изготовители обычно указывают дополнительные параметры, и справочные данные могут различаться тем, как определяются даже «стандартные» параметры, приведенные в таблице. Поэтому, как правило, приводятся также схемы тестирования и форма сигналов, используемых для определения значений различных параметров, как показано, например, на рис. 3.24. Заметьте, что этот рисунок содержит информацию о некоторых других параметрах дополнительно к тем, которые приведены для схемы 54/74НС00.

Большинство терминов на этом рисунке, используемых в справочных данных и употребляемых при описании формы сигналов, вероятно, мало что говорит вам сейчас. Однако по прочтении следующих трех параграфов вы узнаете об электрических характеристиках КМОП-схем столько, что сможете понимать основные моменты этой или любой другой справочной информации. Как разработчику логических устройств, вам понадобятся эти знания, чтобы создавать надежные и устойчивые практические схемы и системы.

НЕ БОЙТЕСЬ!

Студентам, изучающим информатику, и другим читателям, не специализирующимся в области электроники, не следует чрезмерно опасаться материала следующих трех параграфов. Требуется понимание только основ электроники на уровне закона Ома.

Табл. 3.3. Справочные данные производителя для типичной КМОП-схемы 54/74НС00 (четыре вентиля 2И-НЕ)

ЭЛЕКТРИЧЕСКИЕ ХАРАКТЕРИСТИКИ ПО ПОСТОЯННОМУ ТОКУ В РАБОЧЕМ ДИАПАЗОНЕ

Выполняются следующие условия, если не оговорены другие:

Коммерческое исполнение: $T_A = -40^\circ\text{C} + +85^\circ\text{C}$, $V_{CC} = 5.0\text{ В} \pm 5\%$;

Военное исполнение: $T_A = -55^\circ\text{C} + +125^\circ\text{C}$, $V_{CC} = 5.0\text{ В} \pm 10\%$.

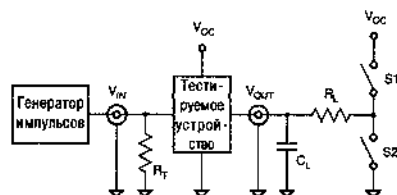
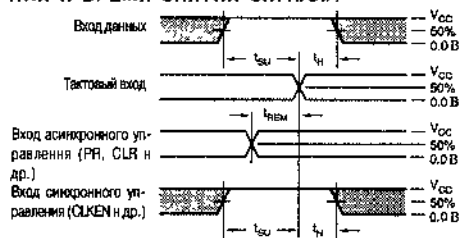
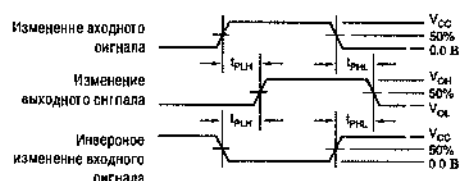
Обозначение	Параметр	Условия тестирования ⁽¹⁾	Мин.	Тип. ⁽²⁾	Макс.	Ед. изм.
V_{IH}	Высокий уровень входного сигнала	Гарантированный высокий логический уровень	3.15	—	—	В
V_{IL}	Низкий уровень входного сигнала	Гарантированный низкий логический уровень	—	—	1.35	В
I_{IH}	Входной ток сигнала высокого уровня	$V_{CC} = \text{Макс.}, V_I = V_{CC}$	—	—	1	мкА
I_{IL}	Входной ток сигнала низкого уровня	$V_{CC} = \text{Макс.}, V_I = 0\text{ В}$	—	—	-1	мкА
V_{IK}	Напряжение на фиксирующем диоде	$V_{CC} = \text{Мин.}, I_N = -18\text{ мА}$	—	-0.7	-1.2	В
I_{OS}	Ток короткого замыкания	$V_{CC} = \text{Макс.}, V_O = 0\text{ В}$	—	—	-35	мА
V_{OH}	Высокий уровень на выходе	$V_{CC} = \text{Мин.}, I_{OH} = -20\text{ мкА}$	4.4	4.499	—	В
		$V_{IN} = V_{IL}, I_{OH} = -4\text{ мА}$	3.84	4.3	—	В
V_{OL}	Низкий уровень на выходе	$V_{CC} = \text{Мин.}, I_{OL} = 20\text{ мкА}$	—	0.001	0.1	В
		$V_{IN} = V_{IH}, I_{OL} = 4\text{ мА}$	—	0.17	0.33	В
I_{CC}	Ток, потребляемый от источника питания, в режиме покоя	$V_{CC} = \text{Мин.}, V_{IN} = 0\text{ В}$ или $V_{CC}, I_O = 0$	—	2	10	мкА

ХАРАКТЕРИСТИКИ ПЕРЕКЛЮЧЕНИЯ В РАБОЧЕМ ДИАПАЗОНЕ, $C_L = 50$ пФ

Обозначение	Параметр ⁽⁴⁾	Условия измерения ⁽¹⁾	Мин.	Тип.	Макс.	Ед. изм.
t_{PD}	Задержка распространения	От входа А или В до выхода Y	—	9	19	нс
C_i	Входная емкость	$V_{IN} = 0$ В	—	3	10	пФ
C_{pd}	Емкость для расчета рассеиваемой мощности, па вентиль	Без нагрузки	—	22	—	пФ

ПРИМЕЧАНИЯ:

1. Для условий, указанных как Макс. или Мин., используйте соответствующее значение согласно электрическим характеристикам.
2. Типичные значения при $V_{CC} = 5.0$ В и температуре $+25^\circ\text{C}$.
3. Одновременно не должно быть короткого замыкания более чем на одном выходе. Продолжительность тестирования в режиме короткого замыкания не должна превышать одной секунды.
4. Этот параметр гарантируется, но не тестируется

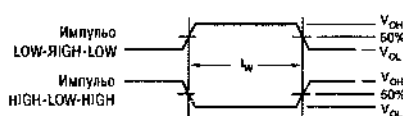
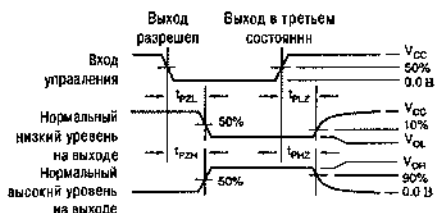
СХЕМА ТЕСТИРОВАНИЯ ВСЕХ ВЫХОДОВ**ВРЕМЯ УСТАНОВЛЕНИЯ, ВРЕМЯ УДЕРЖАНИЯ И ВРЕМЯ СНЯТИЯ СИГНАЛА****ЗАДЕРЖКА РАСПРОСТРАНЕНИЯ****НАГРУЗКА**

Параметр	R_L	C_L	S1	S2
t_{in}	t_{OH}	1 кОм	Разомкнут	Замкнут
t_{in}	t_{OL}	50 пФ или 150 пФ	Замкнут	Разомкнут
t_{in}	t_{OL}	50 пФ или 150 пФ	Разомкнут	Замкнут
t_{in}	t_{OL}	50 пФ или 150 пФ	Замкнут	Разомкнут
t_{in}	—	50 пФ или 150 пФ	Разомкнут	Разомкнут

Определения:

CL = Емкость нагрузки, включая емкость зажима и щупа.

RL = Согласованная нагрузка, равная ZOUT генератора импульсов

ДЛИТЕЛЬНОСТЬ ИМПУЛЬСА**ВРЕМЯ ВЫХОДА ИЗ ТРЕТЬЕГО СОСТОЯНИЯ И ПЕРЕХОДА В ТРЕТЬЕ СОСТОЯНИЕ****Рис. 3.24.** Схема включения и форма сигналов при измерении параметров логических схем серии HC

ЧТО ОЗНАЧАЮТ ЧИСЛА?

В обозначении схем ТТЛ- и КМОП-логики используются два различных префикса «74» и «54», позволяющие различать коммерческий и военный варианты исполнения: 74НС00 – коммерческое исполнение, 54НС00 – военное исполнение.

3.5. Электрические характеристики КМОП-схем в установившемся режиме

В этом параграфе обсуждается установившийся режим КМОП-схем, то есть поведение схем, когда сигналы на входах и выходах не изменяются, а в следующем параграфе рассматривается динамический режим, включая скорость переключения и рассеяние мощности.

3.5.1. Логические уровни и помехоустойчивость

В таблице на рис. 3.10(б) поведение КМОП-инвертора определено только при двух дискретных уровнях напряжения входного сигнала; другие входные напряжения могут привести к другим выходным напряжениям. Полную передаточную характеристику от входа до выхода можно представить в виде графика, изображенного на рис. 3.25. На этом графике показано, что значение входного напряжения (ось X) изменяется от 0 до 5 В; по оси Y отложено выходное напряжение.

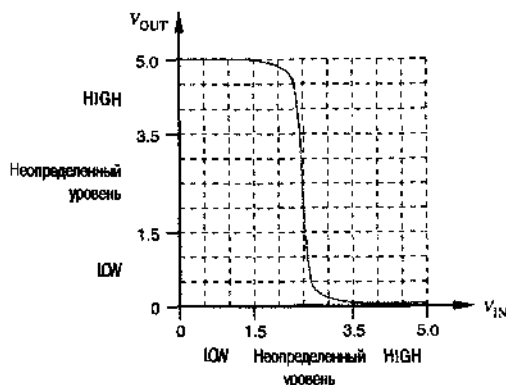


Рис. 3.25. Типичная передаточная характеристика КМОП-инвертора от входа до выхода (LOW – низкий уровень, HIGH – высокий уровень)

Если верить кривой на рис. 3.25, то за низкий уровень входного напряжения КМОП-схемы можно принять любое напряжение ниже 2.4 В, а высокому уровню будет соответствовать все, что выше 2.6 В. Согласно такому определению только при наличии на входе напряжения между 2.4 и 2.6 В выходное напряжение инвертора не соответствует никакому логическому уровню.

К сожалению, типичная передаточная характеристика, показанная на рис. 3.25, всего лишь типична, но не гарантирована. При изменении напряжения источника питания, температуры и нагрузки на выходе передаточная характеристика значи-

тельно изменяется. Она может зависеть даже от того, когда схема была изготовлена. Например, существует легенда, что на одном предприятии в течение нескольких месяцев пытались выяснить, почему схемы, изготовленные в одни дни были хорошими, а в другие — плохими; в конце концов оказалось, что плохие схемы были следствием загрязнения воздуха особенно вредными духами, которыми пользовалась одна из работниц на конвейере!

Инженерная практика заставляет нас пользоваться более осторожным определением низкого уровня и высокого уровня. Такие безопасные уровни для типичного семейства КМОП-логик (серия HC) изображены на рис. 3.26. Эти параметры указываются изготовителями КМОП-схем в справочных данных подобно тому, как это сделано в табл. 3.3, и определяются следующим образом:

- V_{OHmin} — минимальное выходное напряжение высокого уровня;
- V_{IHmin} — минимальное входное напряжение, гарантировано опознаваемое как высокий уровень;
- V_{ILmax} — максимальное входное напряжение, гарантировано опознаваемое как низкий уровень;
- V_{OLmax} — максимальное выходное напряжение низкого уровня.

Входные напряжения определяются, главным образом, порогами переключения двух транзисторов, в то время как выходные напряжения определяются в основном сопротивлениями «открытых» транзисторов.

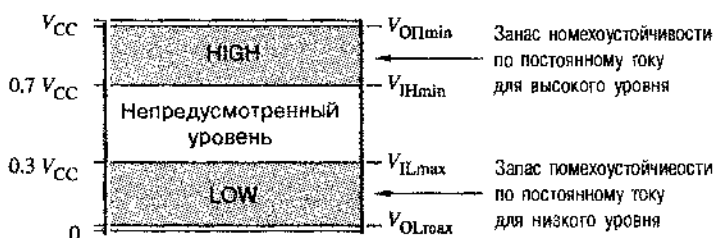


Рис. 3.26. Логические уровни и границы помехоустойчивости для КМОП-семейства серии HC

Значения всех параметров, приведенных на рис. 3.26, гарантируются изготовителями КМОП-схем в некотором диапазоне температур и нагрузок на выходе. Эти значения гарантируются также в некотором диапазоне напряжений источника питания V_{CC} , которое обычно составляет $5.0 \text{ В} \pm 10\%$.

В справочных данных, приведенных в табл. 3.3, указаны значения каждого из этих параметров для КМОП-семейства серии HC. Обратите внимание, что для каждого из параметров V_{OHmin} и V_{OLmax} определены два значения в зависимости от того, велик или мал выходной ток (I_{OH} и I_{OL}). Когда выходы данной схемы соединены только со входами другой КМОП-схемы, выходной ток мал (например, $I_{OL} < 20 \text{ мкА}$) и поэтому падение напряжения на выходных транзисторах очень небольшое. В следующих разделах мы остановимся на этом «чистом» использовании КМОП-схем.

Провод, с помощью которого подводится напряжение питания V_{CC} относительно «земли», часто называют *шиной питания* (*power-supply rail*). Уровни КМОП-схем обычно являются функциями напряжения на шине питания:

$$V_{OHmin} = V_{CC} - 0.1 \text{ В}$$

$$V_{IHmin} = 70\% \text{ от } V_{CC}$$

$$V_{ILmax} = 30\% \text{ от } V_{CC}$$

$$V_{OLmax} = \text{земля} + 0.1 \text{ В}$$

Обратите внимание, что согласно табл. 3.3 напряжение V_{OHmin} равно 4.4 В. Это только на 0.1 В меньше напряжения V_{CC} в худшем случае, так как минимальное значение V_{CC} равно $5.0 \text{ В} - 0.1 \times 5.0 \text{ В} = 4.5 \text{ В}$.

Запас помехоустойчивости по постоянному току (DC noise margin) является мерой того, какой уровень помех в наихудшем случае приводит к такому изменению выходного напряжения, что оно не может быть опознано на входе должным образом. Для КМОП-схем серии НС напряжение V_{ILmax} низкого уровня (1.35 В) превышает напряжение V_{OLmax} (0.1 В) на 1.25 В, так что запас помехоустойчивости по постоянному току для низкого уровня равен 1.25 В. Запас помехоустойчивости по постоянному току для высокого уровня также равен 1.25 В. Вообще, выходы КМОП-схем обеспечивают прекрасную помехоустойчивость по постоянному току при отключении к ним только входов других КМОП-схем.

Независимо от напряжения, подаваемого на вход КМОП-инвертора, он потребляет очень небольшой ток, равный току утечки двух транзисторов в этой схеме. Изготовителем указывается также максимальное значение входного тока:

I_{IH} — максимальный входной ток при высоком уровне на входе.

I_{IL} — максимальный входной ток при низком уровне на входе.

Входной ток, указанный в табл. 3.3 для схем HC00, составляет всего лишь ± 1 мкА. Таким образом, для поддержания входа КМОП-схемы в том или ином состоянии требуется очень небольшая мощность. Этим КМОП-схемы сильно отличаются от логических схем на биполярных транзисторах, таких как TTL- и ECL-схемы, входы которых могут потреблять заметный ток (и мощность) как при одном, так и при другом уровне входного сигнала.

3.5.2. Поведение схемы с активными нагрузками

Как уже сказано, входы КМОП-вентиля имеют очень большое сопротивление и потребляют очень малый ток от схем, которые управляют ими. Но существуют и другие устройства, для которых требуется значительный ток. Когда такое устройство подключено к выходу КМОП-схемы, мы называем его *активной омической нагрузкой* (*resistive load*) или *нагрузкой по постоянному току* (*DC load*). Приведем несколько примеров активных омических нагрузок:

- Дискретные резисторы могут быть включены в качестве согласующих компонентов на конце линии передачи. Эти вопросы обсуждаются в параграфе 11.4.

- Дискретные резисторы реально могут не присутствовать в схеме, но нагрузка в виде одного или большего числа входов ТТЛ-схем или других схем из КМОП-семейства может быть представлена простой резистивной цепью.
- Резисторы могут быть частью потребляющего ток устройства типа светодиода или обмотки реле или имитировать такое устройство.

Когда к выходу КМОП-схемы подключена активная омическая нагрузка, схема ведет себя не так идеально, как было описано раньше. При любом значении сигнала на выходе КМОП-схемы тот из выходных транзисторов, который «открыт», имеет отличное от нуля сопротивление, и наличие нагрузки на выходе вызывает падение напряжения на этом сопротивлении. Таким образом, при низком уровне сигнала выходное напряжение может быть несколько больше, чем 0.1 В, а при высоком уровне сигнала оно может оказаться меньше, чем 4.4 В. Самый простой способ разобраться, как это происходит, состоит в анализе резистивной модели КМОП-схемы и нагрузки.

На рис. 3.27(а) представлена резистивная модель инвертора. Сопротивления p -канального и n -канального транзисторов обозначены как R_p и R_n соответственно. При нормальной работе сопротивление одного из них велико ($> 1 \text{ МОм}$), а сопротивление другого мало (около 100 Ом), в зависимости от того, какому уровню, высокому или низкому, соответствует входное напряжение. Нагрузка в этой схеме состоит из двух резисторов, подключенных к шине питания и к земле; в реальной схеме число резисторов может быть любым. Кроме того, нагрузка может представлять собой даже более сложную резистивную цепь. В любом случае, активную омическую нагрузку, состоящую только из резисторов и источников напряжения, всегда можно представить в виде эквивалентной схемы, применяя теорему Тевенина, как показано на рис. 3.27(б).

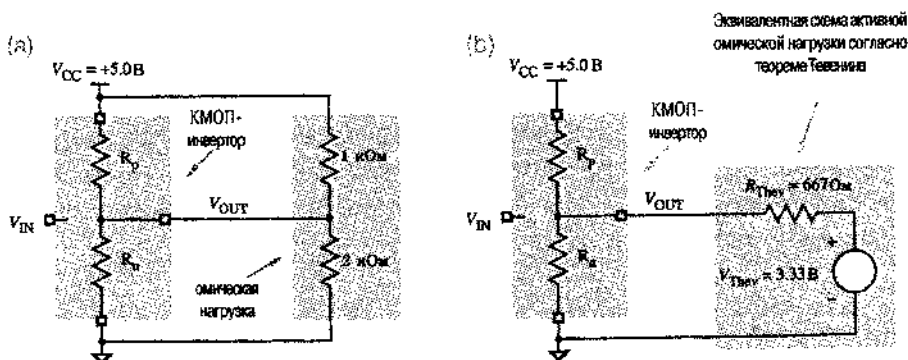


Рис. 3.27. Резистивная модель КМОП-инвертора с активной омической нагрузкой: (а) показана реальная цепь нагрузки; (б) к нагрузке применен теорема Тевенина

ТЕОРЕМА ТЕВЕНИНА

Любой двухполюсник, состоящий только из источников напряжения и резисторов, можно представить в виде эквивалентной схемы (*Thévenin equivalent*), содержащей последовательно включенные один резистор и единственный источник напряжения. Напряжение источника в эквивалентной схеме (напряжение Тевенина; *Thévenin voltage*) равно напряжению холостого хода исходного двухполюсника, а сопротивление в эквивалентной схеме (сопротивление Тевенина; *Thévenin resistance*) находится как напряжение Тевенина, деленное на ток короткого замыкания исходного двухполюсника.

В примере на рис. 3.27 напряжение Тевенина для активной нагрузки, с учетом ее подключения к шине питания V_{CC} , определяется сопротивлениями резисторов 1 кОм и 2 кОм, которые образуют делитель напряжения:

$$V_{Thev} = \frac{2 \text{ кОм}}{2 \text{ кОм} + 1 \text{ кОм}} \cdot 5.0 \text{ В} = 3.33 \text{ В}$$

Ток короткого замыкания равен $(5.0 \text{ В}) / (1 \text{ кОм}) = 5 \text{ мА}$, так что сопротивление Тевенина равняется $(3.33 \text{ В}) / (5 \text{ мА}) = 667 \text{ Ом}$. Сведущий читатель может увидеть, что эта величина равна сопротивлению параллельно включенных резисторов с сопротивлениями 1 кОм и 2 кОм.

Когда на вход КМОП-инвертора подали сигнал высокого уровня, выходной сигнал должен иметь низкий уровень; реальное выходное напряжение можно предсказать, используя резистивную модель, показанную на рис. 3.28. Транзистор с *p*-каналом «закрыт» и его сопротивление столь велико, что им можно пренебречь в последующих вычислениях. Транзистор с *n*-каналом «открыт» и имеет малое сопротивление порядка 100 Ом. (Фактическое сопротивление «открытого» транзистора зависит от конкретного КМОП-семейства и от других характеристик и обстоятельств, например, от рабочей температуры и от того, была ли данная схема изготовлена в «хороший» день.) «Открытый» транзистор и эквивалентный резистор R_{Thev} , показанные на рис. 3.28, образуют простой делитель напряжения. Результирующее выходное напряжение можно найти следующим образом:

$$V_{OUT} = 3.33 \text{ В} \cdot [100 / (100 + 667)] = 0.43 \text{ В}.$$

Точно так же, когда на входе инвертора действует сигнал низкого уровня, выходной сигнал должен иметь высокий уровень, и фактическое выходное напряжение можно определить, воспользовавшись моделью, приведенной на рис. 3.29. Предположим, что сопротивление «открытого» *p*-канального транзистора равно 200 Ом. Снова, «открытый» транзистор и эквивалентный резистор R_{Thev} , указанные на рисунке, образуют простой делитель напряжения; результирующее выходное напряжение можно рассчитать следующим образом:

$$V_{OUT} = 3.33 \text{ В} + (5 \text{ В} - 3.33 \text{ В}) \cdot [667 / (200 + 667)] = 4.61 \text{ В}.$$

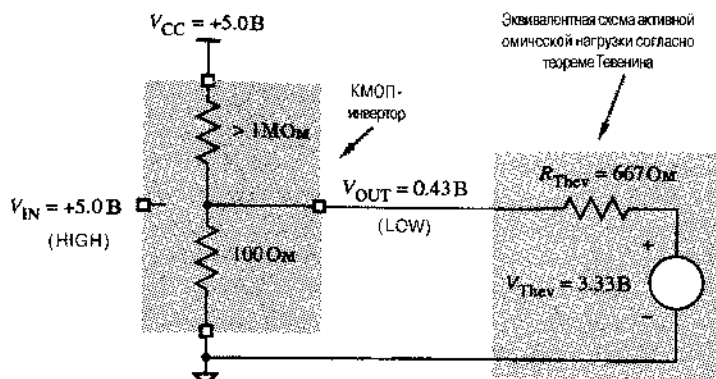


Рис. 3.28. Резистивная модель выходной цепи КМОП-инвертора с активной омической нагрузкой при низком уровне выходного напряжения

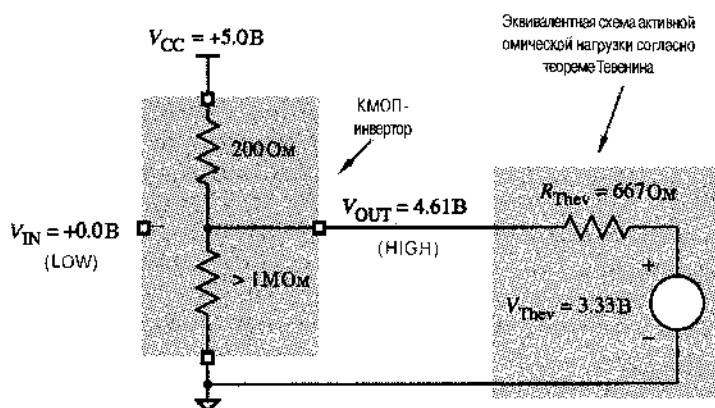


Рис. 3.29. Резистивная модель выходной цепи КМОП-инвертора с активной омической нагрузкой при высоком уровне выходного напряжения

На практике потребность вычислять выходные напряжения так, как это сделано в предыдущих примерах, встречается редко. Действительно, изготовители ИС обычно не указывают эквивалентные сопротивления «открытых» транзисторов, поэтому у вас отсутствовала бы необходимая информация, и желая вы все же провести такой расчет. Вместо этого производители указывают максимальную нагрузку на выходе для каждого из уровней – высокого (HIGH) и низкого (LOW) – и гарантируют определенное выходное напряжение при такой нагрузке в самом плохом случае. Нагрузка определяется на основании известных токов:

- I_{OLmax} – максимальный ток, втекающий в схему со стороны ее выхода при низком уровне и при условии, что выходное напряжение не больше V_{OLmax} .
- I_{OHmax} – максимальный ток, вытекающий из схемы со стороны ее выхода при высоком уровне и при условии, что выходное напряжение не меньше V_{OHmin} .

Иллюстрацией этих определений служит рис. 3.30. Говорят, что через выход схемы течет *втекающий ток* (*sinking current*), когда он протекает от источника питания в нагрузку по нагрузке и по выходной цепи устройства на землю, как показано на рис. 3.30(а). Говорят также, что через выход течет *вытекающий ток* (*sourcing current*) в том случае, когда его путь проходит от источника питания в схеме по ее выходной цепи и по нагрузке на землю, как это изображено на рис. 3.30(б).

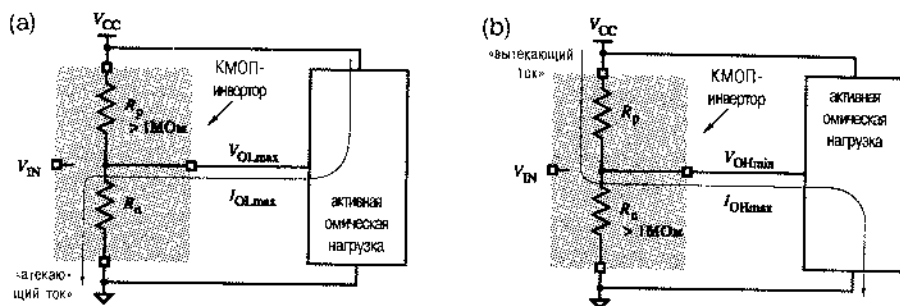


Рис. 3.30. Схемы, иллюстрирующие определение токов: (а) I_{OLmax} ; (б) I_{OHmax}

Большинство КМОП-схем имеет две совокупности нагрузочных характеристик. Одна из них – для «КМОП-нагрузки», когда выход схемы подключен ко входам других КМОП-схем, потребляющих очень небольшой ток. Другой набор характеристик – для «ТТЛ-нагрузки», когда выход соединен с активной омической нагрузкой типа входов ТТЛ-схем или с другими устройствами, потребляющими существенный ток. В качестве примера в табл. 3.3 приведены выходные характеристики КМОП-схем серии НС, и они повторены в табл. 3.4.

Обратите внимание, что выходной ток для высокого уровня выражается отрицательным числом. Обычно принято считать, что *электрический ток* (*current flow*), измеряемый на выходе устройства, положителен, если он *втекает* в устройство; при наличии высокого уровня на выходе вентиля ток *вытекает* из него.

Табл. 3.4. Выходные нагрузочные характеристики для КМОП-схем серии НС при напряжении питания 4.5 В

Параметр	КМОП-нагрузка		ТТЛ-нагрузка	
	Обознач.	Величина	Обознач.	Величина
Максимальный выходной ток низкого уровня (мА)	I_{OLmaxC}	0.02	I_{OLmaxT}	4.0
Максимальное выходное напряжение низкого уровня (В)	V_{OLmaxC}	0.1	V_{OLmaxT}	0.33
Максимальный выходной ток высокого уровня (мА)	I_{OHmaxC}	-0.02	I_{OHmaxT}	-4.0
Минимальное выходное напряжение высокого уровня (В)	V_{OHminC}	4.4	V_{OHminT}	3.84

Как следует из таблицы, при КМОП-нагрузке выходное напряжение КМОП-вентилей поддерживается на уровнях, отличающихся от потенциала земли не от напряжения питания не более чем на 0.1 В. При ТТЛ-нагрузке выходное напряжение может измениться, но совсем немного. Обратите также внимание на то, что при одном и том же выходном токе (± 4 мА) максимальное уменьшение напряжения высокого уровня относительно напряжения питания (0.66 В) вдвое больше, чем максимальное напряжение низкого уровня (0.33 В). Это означает, что p -канальные транзисторы в КМОП-схемах серии HC имеют большее сопротивление в «открытом» состоянии, чем транзисторы с n -каналом. Это естественно, так как в любой КМОП-схеме сопротивление «открытого» p -канального транзистора более чем вдвое превосходит сопротивление «открытого» транзистора с n -каналом при одной и той же площади. Равные падения напряжения для обоих уровней можно получить путем изготовления транзисторов с p -каналом много больших размеров, чем транзисторов с n -каналом, но по различным причинам это не было сделано.

В данной ситуации для определения величины протекающего или вытекающего тока можно воспользоваться законом Ома. На рис. 3.28 «открытый» транзистор с n -каналом представлен 100-омным резистором, падение напряжения на котором составляет 0.43 В; поэтому протекающий ток равен $(0.43 \text{ В}) / (100 \text{ Ом}) = 4.3 \text{ мА}$. Аналогично «открытый» транзистор с p -каналом, изображенный на рис. 3.29 в виде 200-омного резистора, обеспечивает вытекающий ток $(0.39 \text{ В}) / (200 \text{ Ом}) = 1.95 \text{ мА}$.

Фактически сопротивления «открытых» выходных транзисторов КМОП-схем, как правило, в справочных данных не приводятся, поэтому не всегда можно воспользоваться точными моделями, приведенными выше. Однако сопротивления «открытых» транзисторов можно оценить, используя следующие соотношения, опирающиеся на обычно сообщаемые параметры:

$$R_{p(\text{on})} = \frac{V_{CC} - V_{OH(\text{min})T}}{|I_{OH(\text{max})T}|}$$

$$R_{n(\text{on})} = \frac{V_{OL(\text{max})T}}{I_{OL(\text{max})T}}$$

В этих соотношениях используется закон Ома, и сопротивления «открытых» транзисторов определяются как частное от деления напряжения на транзисторе на ток, протекающий через него при активной омической нагрузке в наихудшем случае. Используя численные значения токов и напряжений, приведенные для КМОП-схем серии HC в табл. 3.4, можно подсчитать, что $R_{p(\text{on})} = 175 \text{ Ом}$, а $R_{n(\text{on})} = 82.5 \text{ Ом}$.

Очень хорошие оценки выходного тока в наихудшей ситуации можно получить полагая, что на «открытом» транзисторе нет никакого падения напряжения. Это предположение упрощает анализ и дает приемлемый результат, который почти всегда достаточно хорош для практических целей. Например, на рис. 3.31 показан КМОП-инвертор, выход которого подключен к такой же нагрузке, представленной на теореме Тевенина в виде эквивалентной схемы, какой мы воспользовались

в предыдущих примерах. Резистивная модель выходной цепи не показана, поскольку в этом больше нет необходимости; мы предполагаем, что на «открытых» КМОП-транзисторах нет никакого падения напряжения. При наличии на выходе низкого уровня [рис. 3.31(а)], все напряжение источника в эквивалентной схеме нагрузки, равное 3.33 В, приложено к резистору $R_{Th\text{ев}}$, и протекающий ток примерно равен $(3.33 \text{ В}) / (667 \text{ Ом}) = 5.0 \text{ мА}$. Согласно рис. 3.31(б) при высоком уровне на выходе и в предположении, что напряжение питания равно 5.0 В, падение напряжения на резисторе $R_{Th\text{ев}}$ составляет 1.67 В, а протекающий ток примерно равен $(1.67 \text{ В}) / (667 \text{ Ом}) = 2.5 \text{ мА}$.

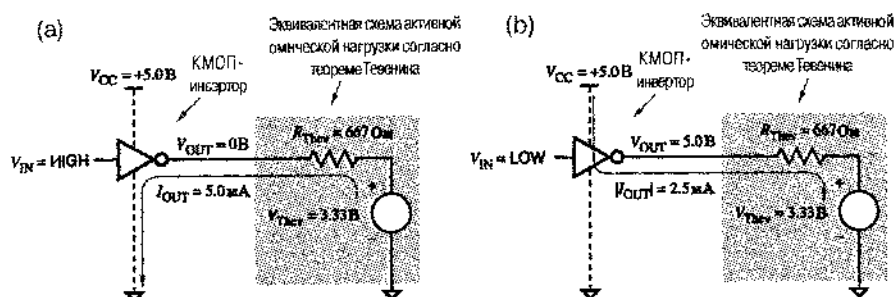


Рис. 3.31. Оценка протекающего и протекающего токов: (а) низкий уровень на выходе; (б) высокий уровень на выходе

Важным свойством КМОП-инвертора (или любой другой КМОП-схемы) является то, что выходная цепь сама по себе потребляет очень небольшой ток при любом из уровней сигнала на выходе — высоком или низком. В каждом из этих случаев один из транзисторов находится в «закрытом» состоянии и его сопротивление велико. Весь ток, о котором мы говорили, течет только тогда, когда к выходу КМОП-схемы подключена активная омическая нагрузка. Если нагрузка отсутствует, то ток не течет, и потребляемая мощность равна нулю. Однако при подключенной нагрузке токи текут и через нагрузку, и через «открытый» транзистор, и в том и в другом рассеивается мощность.

ПРАВДА О ПОТРЕБЛЯЕМОЙ МОЩНОСТИ

Как уже говорилось ранее, сопротивление «закрытого» транзистора превышает один мегаом, но не бесконечно. Поэтому реально через «закрытый» транзистор течет очень маленький ток утечки и выходная цепь КМОП-схемы потребляет соответственно малую, но отличную от нуля мощность. В большинстве случаев эта мощность настолько мала, что ею можно пренебречь. Обычно она бывает существенна только в «дежурном режиме» при питании от аккумуляторов устройств типа ноутбука, на котором первоначально была избрана эта глава.

3.5.3. Поведение схемы с неидеальными входными сигналами

До сих пор мы предполагали, что высокому и низкому уровням на вход КМОП-схем соответствуют идеальные напряжения, очень близкие к напряжению питания и к нулю. Однако поведение реального КМОП-инвертора зависит от входного напряжения не в меньшей степени, чем от свойств нагрузки. Если входное напряжение не близко к напряжению источника питания, то «открытый» транзистор не может быть полностью открыт и его сопротивление может возрасти. Аналогично «закрытый» транзистор не может быть полностью закрыт и его сопротивление может оказаться много меньше одного мегаома. Вместе эти два эффекта приводят к тому, выходное напряжение заметно отличается от напряжения питания или от нуля.

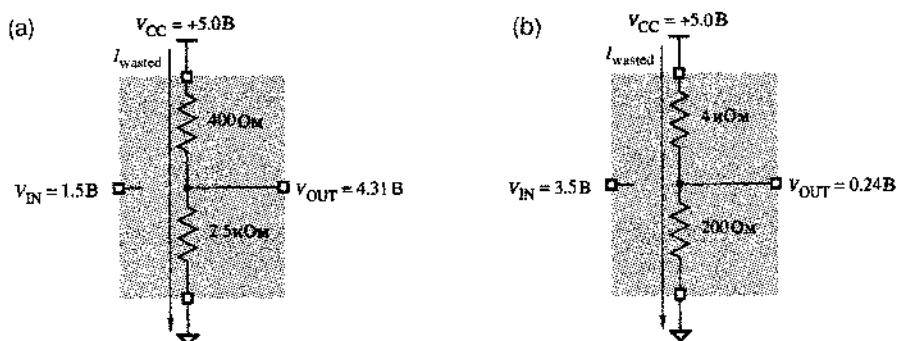


Рис. 3.32. КМОП-инвертор с неидеальными входными напряжениями: (а) эквивалентная схема при входном напряжении 1.5 В; (б) эквивалентная схема при входном напряжении 3.5 В

Например, на рис. 3.32(а) показано возможное поведение КМОП-инвертора при напряжении на входе 1.5 В. Сопротивление транзистора с p -каналом в этой ситуации удваивается, а транзистор с n -каналом начинает открываться. (Указанные значения сопротивлений приняты просто с целью иллюстрации; фактические значения зависят от конкретных характеристик транзисторов.)

Указанное на этом рисунке выходное напряжение, равное 4.31 В, все еще находится в пределах допустимого диапазона для сигнала высокого уровня, но все же отличается от идеального значения 5.0 В. Аналогично, при входном напряжении 3.5 В [рис. 3.32(б)] низкий уровень выходного сигнала составляет 0.24 В, а не 0 В. Небольшое изменение выходного напряжения обычно допустимо; хуже то, что выходная цепь теперь потребляет заметную мощность. Ток, напрасно текущий через инвертор при напряжении на входе 1.5 В, равен

$$I_{\text{wasted}} = 5.0 \text{ В} / (400 \text{ Ом} + 2.5 \text{ кОм}) = 1.72 \text{ мА},$$

а напрасно растрчиваемая мощность составляет

$$P_{\text{wasted}} = 5.0 \text{ В} \cdot I_{\text{wasted}} = 8.62 \text{ мВт}$$

При наличии активной омической нагрузки происходит дополнительное изменение выходного напряжения КМОП-инвертора. Такая нагрузка может иметь место по совершенно разным причинам, рассмотренным ранее. Рис. 3.33 иллюстрирует возможное поведение КМОП-инвертора с активной омической нагрузкой. При 1.5-вольтовом входном напряжении выходное напряжение, равное 3.98 В, все еще находится в пределах допустимого диапазона для сигнала высокого уровня, но оно далеко от идеального значения 5.0 В. Аналогично, при входном напряжении 3.5 В, как показано на рис. 3.34, напряжение сигнала низкого уровня на выходе равно 0.93 В, а не 0 В.

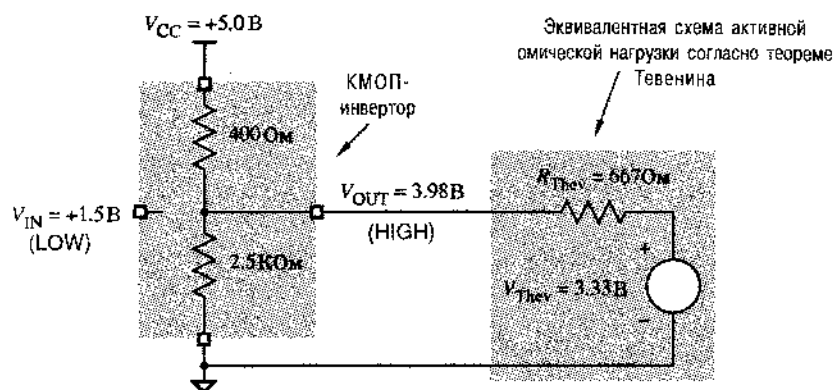


Рис. 3.33. КМОП-инвертор с нагрузкой и идеальным входным напряжением, равным 1.5 В

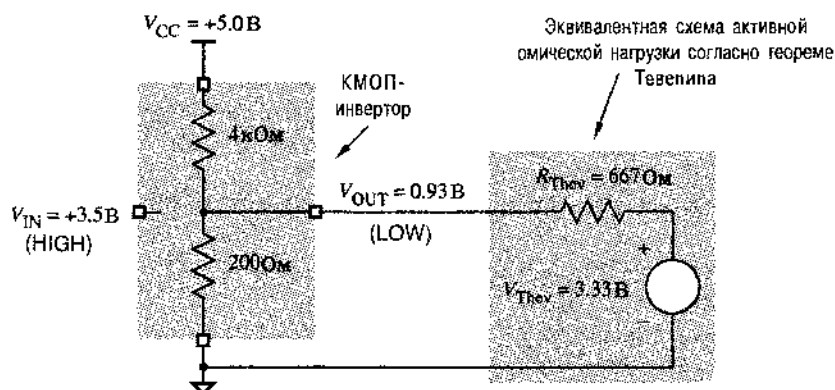


Рис. 3.34. КМОП-инвертор с нагрузкой и идеальным входным напряжением, равным 3.5 В

В «чистых» КМОП-системах все логические схемы принадлежат КМОП-семейству. Поскольку КМОП-схемы имеют очень высокое входное сопротивление, они в очень малой степени нагружают собой выходы КМОП-схем, к которым они подключены. Поэтому все выходные уровни КМОП-схем остаются очень близкими к напряжению источника питания, равному 5 В, или к потенциалу земли, равному

0 В, и совсем нет схем, в выходных цепях которых мощность рассеивается понапрасну. С другой стороны, если входы КМОП-схем соединены с выходами ТТЛ-схем или на входы КМОП-схем поданы неидеальные логические сигналы, то в выходных цепях КМОП-схем рассеивается мощность, как описано в этом разделе. Кроме того, если к выходам КМОП-вентилей подключены входы ТТЛ-схем или другая активная омическая нагрузка, то, как показано в предыдущем разделе, в выходных цепях также рассеивается некоторая мощность.

3.5.4. Коэффициент разветвления по выходу

Коэффициент разветвления по выходу (нагрузочная способность; fanout) логического вентиля — это число входов, которые можно подключить к нему, не нарушая предельных требований по нагрузке. Коэффициент разветвления по выходу зависит не только от характеристик схемы по выходу, но также и от свойств подключаемых к ней входов. Разветвление по выходу необходимо контролировать в обоих возможных случаях, когда сигнал на выходе имеет высокий уровень и низкий уровень.

Например, в табл. 3.4 указано, что максимальный выходной ток I_{OLmaxC} при низком уровне напряжения на выходе КМОП-вентиля серии HC и при его работе на входы КМОП-схем равен 0.02 мА (20 мкА). Раньше мы также установили, что максимальный входной ток I_{lmax} у КМОП-схем серии HC при любом состоянии составляет ± 1 мкА. Поэтому *коэффициент разветвления по выходу низкого уровня (LOW-state fanout)* для вентиля серии HC, нагруженных схемами этой же серии, равен 20. В табл. 3.4 указано также, что максимальный выходной ток I_{OHmaxC} при высоком уровне напряжения на выходе составляет –0.02 мА (–20 мкА). Поэтому, *коэффициент разветвления по выходу высокого уровня (HIGH-state fanout)* для вентиля серии HC, нагруженных схемами этой же серии, также равен 20.

Заметим, что коэффициенты разветвления по выходу низкого уровня и высокого уровня не обязательно равны. Вообще, *полный коэффициент разветвления по выходу (overall fanout)* равен меньшему из коэффициентов низкого уровня и высокого уровня и в предыдущем примере составляет 20.

В только что рассмотренном примере предполагалось, что необходимо сохранить значения выходных напряжений КМОП-вентиля, которые отличаются не более чем на 0.1 В от напряжения питания и от 0 В. Если мы готовы согласиться с несколько худшими условиями на выходе, как у ТТЛ-схем, то для расчета коэффициента разветвления по выходу можно использовать другие значения выходных токов, а именно I_{OLmaxT} и I_{OHmaxT} , равные, как это следует из табл. 3.4, соответственно 4.0 мА и –4.0 мА. Поэтому коэффициент разветвления по выходу вентиля серии HC, нагруженных схемами этой же серии, при уровнях, соответствующих ТТЛ-схемам равен 4000; очевидно, что в этом случае ограничения практически нет.

Все это хорошо, но не совсем. Только что проведенные вычисления дают нам *коэффициент разветвления по выходу для постоянного тока (DC fanout)*, равный по определению, числу входов, которые можно подключить к выходу с фиксированным уровнем сигнала (высоким или низким). Однако даже в том случае, когда требования разветвления по выходу для постоянного тока выполнены, выход КМОП-схемы может не вести себя удовлетворительно при переходах от низкого

уровня к высокому уровню или в обратную сторону, если к нему подключено большое число входов.

Во время переходов выходной ток КМОП-схемы должен заряжать или разряжать паразитную входную емкость схем, подключенных к этому выходу. Если емкость слишком велика, то переход от низкого уровня к высокому уровню (или наоборот) может оказаться слишком медленным, приводя к некорректной работе системы.

Способность выходной цепи заряжать и разряжать паразитную емкость иногда называют *коэффициентом разветвления по выходу для переменного тока (AC fanout)*, хотя он редко вычисляется так же точно, как коэффициент разветвления по выходу для постоянного тока. Как мы увидим в разделе 3.6.1, это больше вопрос о том, на какое снижение быстродействия можно согласиться.

3.5.5. Влияние нагрузки

Нагрузка выхода сверх номинальной величины приводит к следующим последствиям:

- При низком уровне выходное напряжение (V_{OL}) может превысить V_{OLmax} .
- При высоком уровне выходное напряжение (V_{OH}) может упасть ниже V_{OHmin} .
- Задержка распространения сигнала от входа до выхода может превысить значение, указанное в технической документации.
- Время нарастания и время спада сигнала на выходе могут выйти за пределы, указанные в технической документации.
- Может возрасти рабочая температура устройства, что приводит к уменьшению его надежности и, в конечном счете, вызывает отказ в работе.

Первые четыре последствия уменьшают запас помехоустойчивости по постоянному току и ухудшают временные параметры схемы. Таким образом, перегруженная схема может правильно работать в идеальных условиях, но опыт говорит о том, что нас ждет неудача, как только устройство окажется не в столь благоприятных условиях, как в исследовательской лаборатории.

3.5.6. Неиспользуемые входы

Иногда используются не все входы логического вентилля. В реальной конструкции может возникнуть проблема, когда вам необходим вентиль с n входами, а в наличии имеется только вентиль с $n+1$ входами. Если в схеме с $n+1$ входами объединить два входа, то это позволит ей работать в качестве схемы с n входами. Сейчас можно убедиться в этом интуитивно, а в дальнейшем, после изучения параграфа 4.1, вы сможете доказать это, используя алгебру переключений. На рис. 3.35(а) приведена схема И-НЕ с объединенными входами.

Можно также подать на неиспользуемые входы постоянные логические значения. На неиспользуемый вход схем И и И-НЕ следует подать логическую 1, как показано на рис. 3.35(б), а на неиспользуемый вход схем ИЛИ и ИЛИ-НЕ необходимо подать логический 0, как на рис. 3.35(с). В быстродействующей схеме обычно лучше применять способы, указанные на рис. (б) или (с), а не на рис. (а); в последнем случае увеличивается емкостная нагрузка на вентиль, к выходу которого

схема подключена, и это может замедлить его работу. В вариантах (b) и (c) обычно используется резистор с сопротивлением в диапазоне от 1 до 10 кОм; к одному такому резистору можно подключить несколько неиспользуемых входов. Можно также соединять неиспользуемые входы непосредственно с шиной питания и землей соответственно.

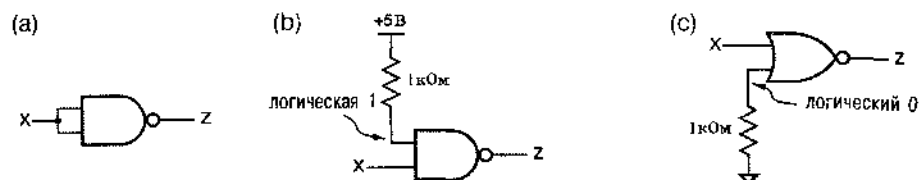


Рис. 3.35. Неиспользуемые входы: (a) объединение с другим входом; (b) схема И-НЕ с одним из входов, подключенным к шине питания; (c) схема ИЛИ-НЕ, один из входов которой заземлен

Неиспользуемые входы КМОП-схем никогда не следует оставлять ни к чему не подключенными. С одной стороны такой *плавающий вход (floating input)* будет вести себя так, как будто к нему приложен низкий уровень сигнала, и обычно осциллограф или вольтметр, подключенные к такому входу, показывают значение 0 В. При этом можно подумать, что неиспользуемый вход схем ИЛИ и ИЛИ-НЕ можно оставлять плавающим, потому что схема будет вести себя так, как будто на этот вход подан логический 0, и он не влияет на выходной сигнал вентиля. Но входное сопротивление КМОП-схем очень велико, поэтому достаточно совсем небольшого шума, чтобы время от времени незаземленный вход вел себя так, как если бы на нем был высокий уровень, создавая тем самым некоторые крайне неприятные неустойчивые состояния схемы.

КОВАРНЫЕ ОШИБКИ

Плавающие входы КМОП-вентилей часто являются причиной таинственного поведения схемы, поскольку из-за шума потенциал неиспользуемого входа хаотически изменяется и эти изменения сказываются в других местах схемы. Когда вы пытаетесь устранить эту проблему и касаетесь щупом осциллографа незаземленного входа, то дополнительной емкости щупа часто бывает достаточно, чтобы ослабить шум и ликвидировать проблему. Если не догадаться, что во всем виноват плавающий вход, то это может сильно сбивать с толку!

3.5.7. Вроски тока и развязывающие конденсаторы

Когда сигнал на выходе КМОП-вентилей переключается между низким и высоким уровнями, ток течет от шины питания V_{CC} к земле через частично открытые *p*- и *n*-канальные транзисторы. Это явление, часто называемое из-за его непродолжительности *бросками тока (current spikes)*, может выглядеть как шум на шине питания КМОП-схем, особенно при одновременном переключении на многих выходах.

По этой причине в системах, построенных на основе КМОП-схем, требуются *развязывающие конденсаторы (decoupling capacitors)*, включаемые между шиной питания V_{CC} и землей. Эти конденсаторы должны быть распределены по всей схеме, но крайней мере, по одному в пределах джойма или у каждой микросхемы, чтобы служить источниками тока во время переходов. *Фильтрующие конденсаторы (filtering capacitors)* большой емкости, обычно находящиеся в самом блоке питания, не удовлетворяют этому требованию, потому что паразитная индуктивность проводов не позволяет току нарастать достаточно быстро, из-за чего возникает необходимость в *физически распределенной* системе развязывающих конденсаторов.

3.5.8. Как испортить КМОП-схему

Ударьте по ней кувалдой. Или просто пройдите по ковру, а затем коснитесь пальцем входного вывода. Поскольку входы КМОП-устройств имеют очень большое сопротивление, их можно разрушить *электростатическим разрядом (electrostatic discharge, ESD)*.

Электростатический разряд наблюдается в том случае, когда при накоплении заряда на одной из поверхностей происходит пробой диэлектрика, разделяющего эти поверхности. В случае входа КМОП-схемы диэлектрик изолирует затвор транзистора от истока и стока. Электростатический разряд может нарушить эту изоляцию, вызывая короткое замыкание между входом и выходом схемы.

Для повышения устойчивости входных цепей к электростатическому разряду в современных КМОП-схемах предпринимаются различные меры, но полностью защитить схему нельзя. Поэтому для предохранения КМОП-схем от электростатического разряда во время транспортировки изготовители обычно упаковывают их в коробки, трубки или вспененную массу из электропроводящих материалов. Чтобы предотвратить повреждение схем электростатическим разрядом при работе с распакованными КМОП-схемами, монтажники и техники обычно надевают на запястье проводящий браслет, который соединен спиральным проводником с заземлением; это обеспечивает стекание статического заряда, который может накопиться на теле человека при его передвижении по производственному помещению или по лаборатории.

Как только КМОП-схема установлена в систему, появляется другой возможный источник повреждения — *защелкивание (latch-up)*. В реальной входной цепи почти любого КМОП-устройства между шиной питания V_{CC} и землей образуются паразитные биполярные транзисторы, представляющие собой «кремниевые управляемые диоды» (тиристоры). При нормальной работе устройства такой «паразитный тиристор» не оказывает никакого влияния. Однако в случае, когда входное напряжение оказывается ниже уровня земли или выше напряжения питания V_{CC} , тиристор может «запуститься», фактически закорачивая цепь между шиной питания и землей. Если тиристор открыт, то единственным способом закрыть его является отключение питания. Прежде, чем у вас появится возможность отключить питание, может рассеяться мощность, достаточная для разрушения схемы (при этом может пойти дым).

НЕДОПУСКАЙТЕ НЕАККУРАТНОГО ОБРАЩЕНИЯ!

В лаборатории ради безопасности следует соблюдать определенные меры предосторожности, связанные с возможностью электростатического разряда, хотя некоторые разработчики и не считают необходимым терпеть возникающие при этом неудобства:

Перед началом работы с КМОП-схемой коснитесь металлического корпуса заземленного прибора или другого места заземления.

Перед транспортировкой КМОП-схемы встаньте ее в электропроводящий материал.

При переноске платы, содержащей КМОП-схемы, возьмите ее за края и, прежде чем будете что-нибудь делать с ней, коснитесь заземления земляным выводом платы.

При передаче КМОП-схемы коллеге, особенно в сухой зимний день, сначала коснитесь его. Он или она только скажут вам за это спасибо.

Одной из возможных причин защелкивания являются «отрицательные выбросы», возникающие при очень быстрых переходах сигнала от высокого уровня к низкому, рассматриваемые в параграфе 11.4. В этом случае входной сигнал перед достижением нормального низкого уровня может опуститься ниже уровня земли на несколько вольт. Правда в современных КМОП-вентиллях имеются специальные цепи, предотвращающие защелкивание при таком кратковременном процессе.

Защелкивание может наступать также в том случае, когда сигналы поступают на входы КМОП-схемы с выходов другой системы или подсистемы с отдельным источником питания. Если на вход КМОП-схемы подан высокий уровень до того, как включен источник питания, то при включении питания вентиль может перейти в «защелкнутое» состояние. Отметим снова, что современные логические КМОП-схемы изготовлены так, чтобы в большинстве случаев предотвратить это явление. Если, однако, выход предыдущей схемы способен выдать большой ток (например, десятки мА), то защелкивание все же возможно. Одно из решений этой проблемы состоит в том, чтобы включать источник питания до присоединения входных кабелей.

3.6. Динамические свойства КМОП-схем

Как быстродействие КМОП-схем, так и потребляемая ею мощность в значительной степени зависят от ее характеристик по переменному току или динамических характеристик, а также от нагрузки, то есть от того, что происходит при изменении уровня сигнала на выходе. Частью разработки внутренней структуры специализированных интегральных КМОП-схем должно быть тщательное исследование влияния нагрузки на функционирование выходных цепей, и схему необходимо переделывать, если нагрузка слишком велика. Даже на уровне разработки плат необходимо учитывать влияние нагрузки на сигналы синхронизации, сигналы, передаваемые по шинам, и на другие сигналы, поступающие с выхода схем с большим коэффициентом разветвления, а также в том случае, когда разводка осуществляется длинными соединениями.

Быстродействие схемы зависит от двух параметров: от времени переходного процесса и от задержки распространения сигнала; этим двум вопросам посвящены следующие два раздела. Рассеяние мощности рассматривается в третьем разделе.

3.6.1. Длительность переходного процесса

Время, в течение которого сигнал на выходе логической схемы изменяется от одного уровня до другого, называется *временем переходного процесса (transition time)*. На рис. 3.36(а) показано, как могло бы выглядеть изменение выходного сигнала при нулевом времени переходного процесса. Однако реальные выходные сигналы не могут изменяться мгновенно, потому что необходимо время для заряда паразитной емкости между проводами и емкости других компонентов, на которые подаются эти сигналы. Более реалистичный вид выходного сигнала изображен на рис. 3.36(б). В течение времени t_r , называемого *временем нарастания (rise time)*, выходной сигнал изменяется от низкого уровня до высокого, а в течение времени t_f , называемого *временем спада (fall time)*, сигнал изменяется от высокого уровня до низкого; время t_f может отличаться от времени t_r .

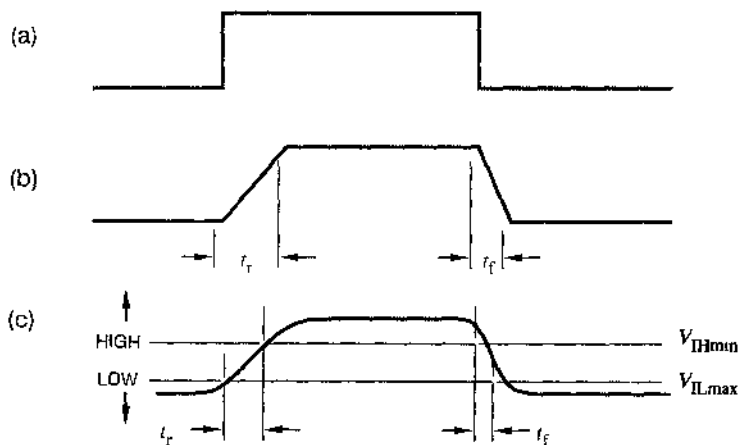


Рис. 3.36. Длительность переходного процесса (а) идеальный случай с нулевым временем переключения, (б) более близкое к действительности приближение, (с) реальная временная диаграмма с нарастанием и спадом сигнала за конечное время (LOW – низкий уровень, HIGH – высокий уровень)

Рис. 3.36(б) не совсем точен, потому что скорость изменения выходного напряжения не устанавливается мгновенно. Реально переходы в начале и в конце бывают плавными, как это изображено на рис. 3.36(с). Чтобы избежать трудностей в определении начала и конца изменения, время нарастания и время спада обычно измеряют по границам логических уровней, как показано на рисунке.

В соответствии с рис. 3.36(с) время нарастания и время спада говорят о том, как долго выходное напряжение преодолевает область «неопределенности» между низким и высоким уровнями. Время изменения сигнала в пределах одного уровня не включается во время нарастания и время спада. Эта часть переходного процес-

са вносит вклад в «задержку распространения», которая рассматривается в следующем разделе.

Время нарастания и время спада сигнала на выходе КМОП-схемы зависят в основном от двух параметров: от сопротивления «открытого» транзистора и от емкости нагрузки. При большей емкости время переходного процесса увеличивается и, поскольку это нежелательно, разработчики логических схем очень редко намеренно подключают конденсатор к выходу логической схемы. Однако *паразитная емкость* (*stray capacitance*) присутствует в любой цепи, что обусловлено, по крайней мере, тремя причинами:

1. Выходные цепи, включая выходные транзисторы вентиля, а также внутренние соединения и корпуса схем типовых логических семейств, в том числе КМОП-схем, имеют некоторую собственную емкость в диапазоне от 2 до 10 пикофарад (пФ).
2. Соединения какого-либо выхода с входами других схем имеют емкость около 1 пФ на дюйм или больше, в зависимости от технологии монтажа.
3. У типовых логических семейств входные цепи схем, включая транзисторы, внутренний монтаж и корпус микросхемы, имеют емкость от 2 до 15 пФ на вход.

Паразитную емкость иногда называют *емкостной нагрузкой* (*capacitive load*) или *нагрузкой по переменному току* (*AC load*).

Воспользовавшись эквивалентной цепью, показанной на рис. 3.37, можно рассчитать время нарастания и время спада сигнала на выходе КМОП-вентиля. Как и в предыдущем параграфе, *p*-канальный и *n*-канальный транзисторы заменены резисторами R_p и R_n соответственно. При нормальной работе сопротивление одного из резисторов велико, а другого — мало в зависимости от уровня сигнала на выходе. Нагрузка на выходе представлена *эквивалентной схемой нагрузки* (*equivalent load circuit*), состоящей из трех компонентов:

- R_L , V_L — два компонента, посредством которых представлена нагрузка по постоянному току; от них зависят напряжения и токи в выходной цепи, когда сигнал на выходе имеет установившееся значение высокого уровня или низкого уровня; при изменении уровня сигнала на выходе нагрузка по постоянному току не оказывает заметного влияния на время перехода;
- C_L — емкость, представляющая собой нагрузку по переменному току; от нее зависят значения напряжений и токов в выходной цепи в процессе изменения выходного сигнала и то, как долго происходит переход от одного уровня до другого.

Если выход КМОП-вентиля нагружен входами только КМОП-схем, то нагрузка по постоянному току незначительна. В оставшейся части этого раздела ради простоты мы рассмотрим только этот случай с $R_L = \infty$ и $V_L = 0$. Наличие некоторой нагрузки по постоянному току повлияет на результат, но не очень сильно (см. упражнение 3.68).

Теперь можно проанализировать время переходного процесса на выходе КМОП-вентиля. Для этого примем $C_L = 100$ пФ, что является умеренной емкостной нагрузкой. Кроме того, будем считать, что, как и в предыдущем разделе, сопротив-

ления «открытых» p -канальных и n -канальных транзисторов равны 200 Ом и 100 Ом соответственно. Время нарастания и время спада сигнала зависят от того, как долго происходит заряд или разряд емкостной нагрузки C_L .

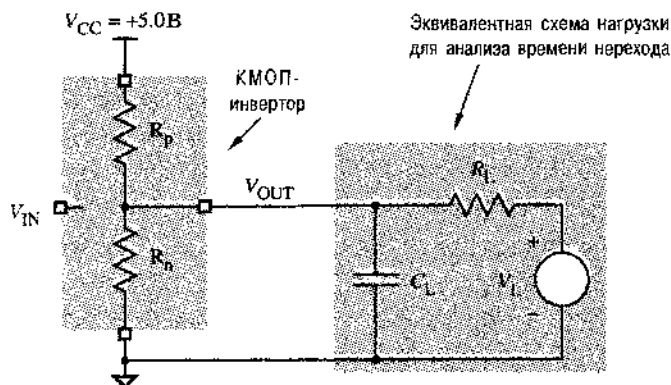


Рис. 3.37. Эквивалентная схема для анализа времени переходного процесса в выходной цепи КМОП-вентилля

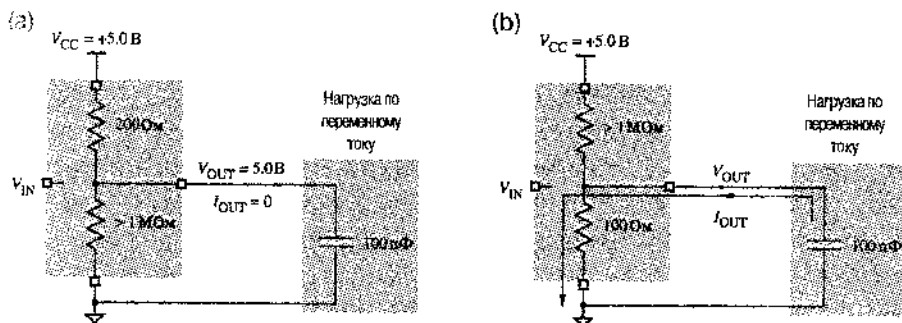


Рис. 3.38. Модель перехода КМОП-вентилля от высокого уровня до низкого: (а) на выходе вентилля высокий уровень; (б) состояние вентилля после того, как p -канальный транзистор оказывается «закрытым», а n -канальный транзистор — «открытым»

Рассмотрим сначала время спада. На рис. 3.38(а) изображено состояние схемы, когда сигнал на выходе имеет установившийся высокий уровень. (R_L и V_L не показаны; они не оказывают никакого влияния, так как мы считаем $R_L = \infty$.) Для нашего анализа предположим, что транзисторы в КМОП-схеме не переходят из «открытого» состояния в «закрытое» и обратно мгновенно. Предположим также, что переход сигнала на выходе КМОП-вентилля к низкому уровню начинается в момент времени $t = 0$, когда схема оказывается в состоянии, указанном на рис. 3.38(б).

В момент времени $t = 0$ напряжение V_{OUT} все еще равно 5.0 В. (Важный принцип электротехники состоит в том, что напряжение на конденсаторе не может изменяться мгновенно.) При $t = \infty$ конденсатор должен полностью разрядиться и

напряжение V_{OUT} будет равно 0. Между указанными значениями напряжение V_{OUT} изменяется по экспоненциальному закону:

$$V_{OUT} = V_{CC} \cdot e^{-t/R_n C_L} = 5,0 \cdot e^{-t/(100 \times 100 \times 10^{-12})} \text{ В} = 5,0 \cdot e^{-t/(10 \times 10^{-9})} \text{ В}.$$

Произведение $R_n C_L$ имеет размерность времени и называется *постоянной времени* (*RC time constant*). Проведенное вычисление говорит о том, что постоянная времени при переходе от высокого уровня к низкому равна 10 наносекундам (нс).

На рис. 3.39 показана зависимость напряжения V_{OUT} от времени. Чтобы определить время спада вспомним, что в качестве границ низкого и высокого уровней входного сигнала КМОП-схемы, подключенной к выходу другой КМОП-схемы, приняты значения 1.5 В и 3.5 В. Чтобы получить время спада, необходимо воспользоваться предыдущим соотношением для $V_{OUT} = 3.5 \text{ В}$ и $V_{OUT} = 1.5 \text{ В}$, откуда следует:

$$t = -R_n C_L \cdot \ln \frac{V_{OUT}}{V_{CC}} = -10 \cdot 10^{-9} \cdot \ln \frac{V_{OUT}}{5,0},$$

$$t_{3,5} = 3.57 \text{ нс},$$

$$t_{1,5} = 12.04 \text{ нс}.$$

Время спада t_f равно разности между этими двумя числами, что составляет около 8.5 нс.

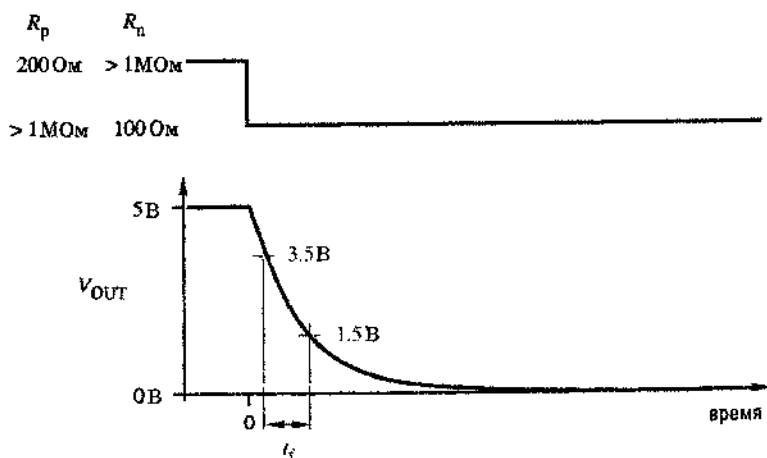


Рис. 3.39. Время спада при переходе напряжения на выходе КМОП-вентили от высокого уровня к низкому уровню

Аналогично можно рассчитать время нарастания сигнала. На рис. 3.40(а) представлено состояние схемы, когда сигнал на выходе имеет установившийся низкий уровень. На рис. 3.40(б) изображен начинающийся в момент времени $t = 0$ переход КМОП-схемы в состояние с высоким уровнем на выходе. Как и прежде, напряжение V_{OUT} не может измениться мгновенно, но при $t = \infty$ конденсатор будет полно-

стью заряжен и напряжение V_{OUT} будет равно 5.0 В. Снова изменение напряжения V_{OUT} описывается экспоненциальным законом:

$$V_{OUT} = V_{CC} \cdot (1 - e^{-t/R_p C_L}) = 5.0 \cdot (1 - e^{-t/(200 \times 100 \times 10^{-12})}) \text{ В} = 5.0 \cdot (1 - e^{-t/(20 \times 10^{-9})}) \text{ В}.$$

Постоянная времени в этом случае равна $R_p C_L = 20$ нс. На рис. 3.41 показана зависимость напряжения V_{OUT} от времени. Чтобы получить время нарастания, снова необходимо воспользоваться имеющимся соотношением для $V_{OUT} = 1.5$ В и $V_{OUT} = 3.5$ В, откуда следует:

$$t = -R_p C_L \cdot \ln \frac{V_{CC} - V_{OUT}}{V_{CC}} = -20 \cdot 10^{-9} \cdot \ln \frac{5.0 - V_{OUT}}{5.0},$$

$$t_{1.5} = 7.13 \text{ нс},$$

$$t_{3.5} = 24.08 \text{ нс}.$$

Время нарастания t_r определяется как разность между полученными двумя числами и приблизительно равно 17 нс.

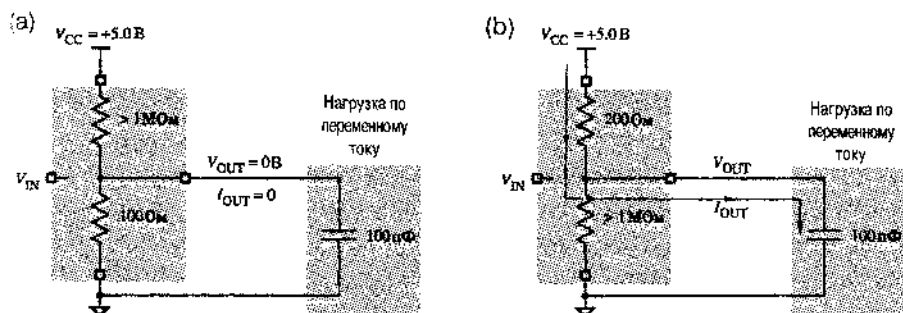


Рис. 3.40. Модель перехода КМОП-веиля от низкого уровня к высокому уровню на выходе: (а) низкий уровень на выходе; (б) состояние схемы после того, как транзистор с n -каналом оказывается «закрытым», а транзистор с p -каналом — «открытым»

В последнем примере предполагается, что p -канальный транзистор имеет вдвое большее сопротивление, чем n -канальный транзистор; в результате время нарастания вдвое больше времени спада. «Слабому» p -канальному транзистору требуется больше времени для того, чтобы подтянуть выходное напряжение до высокого уровня, чем «сильному» n -канальному транзистору для того, чтобы понизить его до низкого уровня; способность веиля неключать выходной сигнал «асимметрична». В быстродействующих КМОП-устройствах p -канальные транзисторы иногда изготавливают большего размера, чтобы уравнивать времена переходов и сделать переходы на выходе от высокого уровня к низкому уровню и обратно более симметричными.

С увеличением емкости нагрузки — независимо от свойств транзисторов — постоянная времени имеет большее значение и длительность переходных процессов

на выходе растет. Таким образом, задача разработчиков быстродействующих схем состоит в минимизации емкости нагрузки, особенно для сигналов, наиболее критичных в отношении времени. Это можно сделать, уменьшая число входов, на которые поступает данный сигнал, путем создания нескольких копий этого сигнала, а также посредством аккуратной разводки схемы.

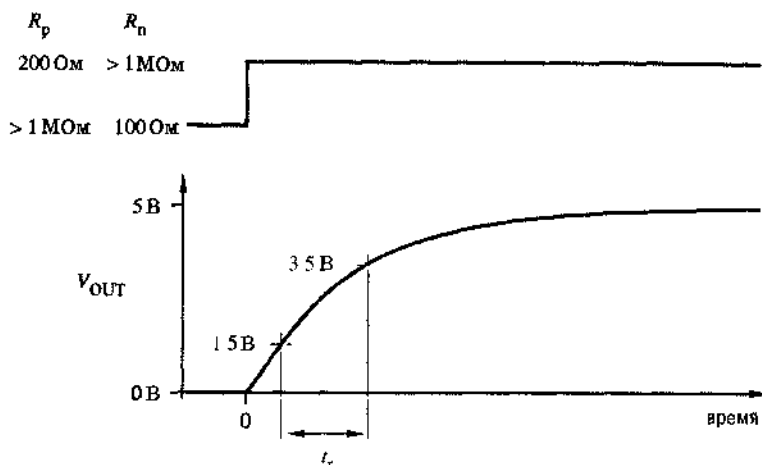


Рис. 3.41. Время нарастания при переходе напряжения на выходе КМОП-вентиля от низкого уровня к высокому

При работе с реальными цифровыми схемами часто желательно оценить время переходных процессов без проведения детального анализа. Полезное эмпирическое правило состоит в том, что время переходного процесса приблизительно равняется постоянной времени RC -цепи заряда или разряда. Например, оценки 10 нс и 20 нс для времени спада и нарастания в рассмотренном примере были бы достаточно приемлемыми, особенно с учетом того, что большинство предположений, прежде всего относительно емкости нагрузки и сопротивлений «открытых» транзисторов, являются приблизительными.

Изготовители коммерческих КМОП-схем в своих справочных данных обычно не указывают сопротивления «открытых» транзисторов. Если тщательно поискать, то эту информацию можно найти в публикуемых изготовителями комментариях. Это сопротивление в любом случае можно оценить как частное от деления напряжения на «открытом» транзисторе на ток, протекающий через активную нагрузку в наихудшем случае, как это было показано в разделе 3.5.2:

$$R_{p(on)} = \frac{V_{CC} - V_{OHminT}}{|I_{OHmaxT}|},$$

$$R_{n(pn)} = \frac{V_{OLmaxT}}{|I_{OLmaxT}|}.$$

НЕ ВСЕ ТАК ПРОСТО!

Расчетная длительность переходного процесса на самом деле сильно зависит от выбора логических уровней. Если в примерах этого раздела в качестве порогов высокого уровня и низкого уровня использовать значения 2.0 В и 3.0 В вместо 1.5 В и 3.5 В, то получим меньшее время для каждого из переходных процессов. С другой стороны, если использовать уровни 0.0 В и 5.0 В, то расчетное время переходных процессов будет бесконечным! Следует также иметь в виду, что у некоторых логических семейств (особенно у семейства ТТЛ), пороги не симметричны относительно средней точки между минимальным и максимальным напряжением. Тем не менее, опыт автора показывает, что оценка по правилу «время переходного процесса равно постоянной времени соответствующей RC-цепи» обычно вполне применима.

3.6.2. Задержка распространения

Время нарастания и время спада сигнала лишь частично описывают поведение логического элемента в динамике; необходимы дополнительные параметры, чтобы связать временные диаграммы входного и выходного сигналов. *Путь прохождения сигнала (signal path)* — это цепочка из электрических звеньев, по которой сигнал от входа проходит к выходу в рассматриваемом логическом элементе. *Задержка распространения (propagation delay)* t_p вдоль пути прохождения сигнала определяется как время, необходимое для того, чтобы изменение входного сигнала привело к изменению выходного сигнала.

Сложный логический элемент с большим числом входов и выходов может иметь различные значения t_p при прохождении сигнала по тому или иному пути. Задержка распространения сигнала по данному пути зависит также от того, в каком направлении изменяется выходной сигнал. На рис. 3.42(а) указаны два различных значения задержки распространения от входа до выхода (без учета времени нарастания и спада) для КМОП-инвертора в зависимости от направления изменения сигнала на выходе:

- t_{pHL} — время между изменением сигнала на входе и соответствующим изменением сигнала на выходе при переходе выходного напряжения с высокого уровня на низкий;
- t_{pLH} — время между изменением сигнала на входе и соответствующим изменением сигнала на выходе при переходе выходного напряжения с низкого уровня на высокий.

Отличная от нуля задержка распространения вызвана несколькими факторами. На скорость изменения состояния транзисторов в КМОП-устройстве оказывают влияние как физические процессы в полупроводнике, так и условия эксплуатации схемы, включая скорость изменения входного сигнала, входную емкость и нагрузку на выходе. В многоуровневых схемах типа неинвертирующего вентиля и в устройствах, реализующих более сложные логические функции, может потребовать-

ся изменение состояний нескольких внутренних транзисторов, прежде чем сможет измениться уровень сигнала на выходе. Но даже тогда, когда напряжение на выходе начинает изменяться, при отличных от нуля времени нарастания и времени спада требуется определенное время, чтобы пересечь область между логическими уровнями, как было объяснено в предыдущем разделе. Все эти факторы вносят свой вклад в задержку распространения сигнала.

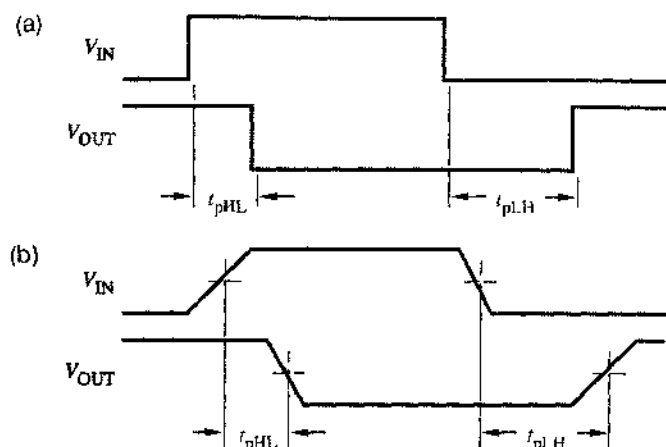


Рис. 3.42. Задержки распространения сигнала для КМОП-инвертора: (а) без учета времени нарастания и спада; (б) определяемые по моментам перехода через средние значения

Чтобы не учитывать влияния времени нарастания и времени спада, производители обычно указывают задержку распространения по моментам перехода входного и выходного сигналов через среднюю точку, как показано на рис. 3.42(б). Однако иногда задержки определяются и по моментам пересечения пороговых значений логических уровней, особенно в тех случаях, когда на работе устройства может неблагоприятно сказаться медленное нарастание и спад входного сигнала. Например, на рис. 3.43 показано, как можно определить минимальную длительность входного импульса для RS-защелки (изучаемой в разделе 7.2.1).

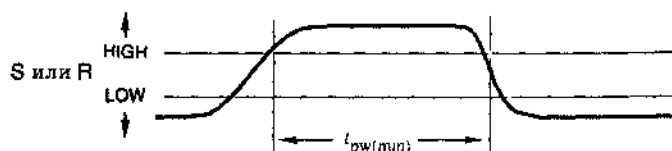


Рис. 3.43. Определение длительности интервала времени по граничным значениям логических уровней в наихудшем случае

Кроме того, производитель может указывать абсолютные максимальные значения времени нарастания и спада входного сигнала, при которых гарантируется надежная работа. Быстродействующие КМОП-схемы при слишком медленных изменениях входного сигнала могут потреблять чрезмерно большой ток или генерировать колебания.

3.6.3. Потребляемая мощность

Мощность, потребляемая КМОП-схемой при неизменном уровне сигнала на выходе, называется *статической рассеиваемой мощностью* (*static power dissipation*) или *мощностью, рассеиваемой в режиме покоя* (*quiescent power dissipation*). (При обсуждении вопроса о том, какая мощность тратится при работе устройства, термины *потребляемая* и *рассеиваемая* мощность означают практически одно и то же.) У большинства КМОП-схем статическая рассеиваемая мощность очень мала. Именно это делает их такими привлекательными для портативных ЭВМ и для других применений, когда требуется, чтобы потребляемая мощность была мала: во время пауз в работе компьютера потребляется очень небольшая мощность. Существенную мощность, называемую *динамической рассеиваемой мощностью* (*dynamic power dissipation*), КМОП-схема потребляет только во время переходных процессов.

Одной из причин рассеивания мощности при переходе из одного состояния в другое является частичное замыкание выходной цепи КМОП-схемы. Если входное напряжение не равно примерно 0 В при напряжении питания V_{CC} , то оба выходных транзистора — *p*-канальный и *n*-канальный — могут быть частично «открыты», при этом их суммарное сопротивление равно 600 Ом или меньше. В этом случае ток течет через транзисторы от шины питания V_{CC} к земле. Таким образом, величина потребляемой мощности зависит, как от значения V_{CC} , так и от скорости переходного процесса на выходе и определяется по формуле

$$P_T = C_{PD} \cdot V_{CC}^2 \cdot f,$$

где:

P_T — мощность рассеиваемая внутри схемы, обусловленная переходными процессами на выходе;

V_{CC} — напряжение питания; как известно всем инженерам-электрикам, мощность рассеиваемая на активной нагрузке (на частично открытых транзисторах) пропорциональна квадрату напряжения;

f — частота переключений (*transition frequency*) в выходном сигнале; ею определяется, сколько раз в секунду на выходе происходят переключения, при которых потребляется мощность (впрочем, обратите внимание, что частота, по определению, равна числу переключений, деленному на 2);

C_{PD} — емкость, которой определяется рассеиваемая мощность (*power-dissipation capacitance*); это константа, обычно указываемая производителем схем, используется для нахождения рассеиваемой мощности по приведенной формуле; величина C_{PD} имеет размерность емкости, но не является реальной емкостью выходной цепи; скорее, она отражает динамику протекания тока через изменяющиеся сопротивления выходных транзисторов в течение одной пары переходов на выходе от высокого уровня к низкому и в обратном направлении; например, C_{PD} для КМОП-клапанов серии HC в типичном случае составляет 20 – 24 пФ, несмотря на то, что реальная емкость выходной цепи много меньше.

Соотношение для P_T справедливо только в том случае, когда изменения сигнала на входе достаточно быстры, что приводит к быстрым переходным процессам на выходе. Если входные сигналы изменяются слишком медленно, то выходные транзисторы остаются частично открытыми длительное время, и потребляемая мощность возрастает. Производители микросхем обычно указывают максимальное время паразитного нарастания и спада входного сигнала, вплоть до которого остается правильным указанный коэффициент C_{PD} .

Второй, и часто более существенной причиной потребления мощности КМОП-схемой является наличие емкостной нагрузки (C_L) на выходе. При переходе от низкого уровня к высокому ток, протекающий через транзистор с p -каналом, заряжает емкость C_L . Аналогично, при переходе от высокого уровня к низкому током, протекающим через транзистор с n -каналом, емкость C_L разряжается. В каждом из этих случаев на сопротивлении «открытого» транзистора рассеивается мощность. Полную мощность, рассеиваемую при заряде и разряде C_L , обозначим P_L .

Величина P_L имеет размерность мощности или энергии, рассеиваемой в единицу времени. Энергию, рассеиваемую за один переход, можно было бы определить, вычисляя ток через заряжающий транзистор как функцию времени (с учетом постоянной времени соответствующей RC -цепи, как это делалось в разделе 3.6.1), возводя этот ток в квадрат, умножая на сопротивление «открытого» транзистора и интегрируя по времени. Ниже описан более простой путь.

За время перехода на емкости нагрузки C_L изменяется на величину $\pm V_{CC}$. Согласно определению емкости, суммарный заряд, который должен протечь, чтобы напряжение на емкости C_L изменилось на величину V_{CC} , равен $C_L \cdot V_{CC}$. Полная энергия, расходуемая за один переход, равна заряду, умноженному на средний перепад напряжения $V_{CC}/2$. Поэтому энергия, потребляемая за все время перехода, равна $C_L \cdot V_{CC}^2/2$. Если за одну секунду происходит $2f$ переходов, то мощность, рассеиваемая из-за наличия емкостной нагрузки, равна

$$P_L = C_L \cdot (V_{CC}^2/2) \cdot 2f = C_L \cdot V_{CC}^2 \cdot f.$$

Суммарная динамическая мощность, рассеиваемая КМОП-схемой, равняется сумме мощностей P_T и P_L :

$$P_D = P_T + P_L = C_{PD} \cdot V_{CC}^2 \cdot f + C_L \cdot V_{CC}^2 \cdot f = (C_{PD} + C_L) \cdot V_{CC}^2 \cdot f.$$

Динамическую рассеиваемую мощность, вычисляемую по этой формуле, часто называют CV^2f -мощностью (CV^2f power). В большинстве случаев при использовании КМОП-схем CV^2f -мощность, безусловно, является главной составляющей полной рассеиваемой мощности. Заметьте, что CV^2f -мощность рассеивается также в биполярных логических схемах, таких как ТТЛ- и ЭСЛ-схемы, но при низких и умеренных частотах она незначительна по сравнению со статической рассеиваемой мощностью (по постоянному току).

3.7. Другие варианты входных и выходных цепей КМОП-схем

В этом параграфе описаны некоторые наиболее часто используемые модификации входных и выходных цепей КМОП-схем, возникшие в процессе приспособления основной конфигурации КМОП-схем к нуждам конкретных приложений.

3.7.1. Логические ключи

Соединенные вместе транзисторы с p - и n -каналами, образуют управляемый логическим сигналом выключатель. Эта КМОП-схема, показанная на рис. 3.44, называется логическим ключом (*стробирующим устройством*; *transmission gate*).

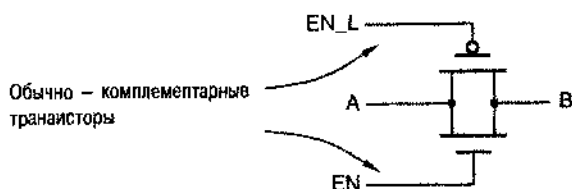


Рис. 3.44. КМОП-ключ

Для нормальной работы ключа необходимо, чтобы его входные сигналы EN и EN_L всегда имели противоположные логические уровни. Когда на вход EN подан высокий уровень, а на вход EN_L — низкий уровень, сопротивление между точками A и B мало (порядка 2–5 Ом). Когда на вход EN подан низкий уровень, а на вход EN_L — высокий уровень, точки A и B разъединены.

Когда ключ замкнут, задержка распространения сигнала от точки A до точки B (или в обратном направлении) очень мала. Благодаря малым задержкам и простой структуре такие ключи часто используются в составе КМОП-схем большей степени интеграции типа мультиплексоров и триггеров. Например, на рис. 3.45 показано применение ключа в 2-входовом мультиплексоре. Если на входе S действует низкий уровень, то с выходом Z соединен вход X; при наличии на входе S высокого уровня с выходом Z соединен вход Y.

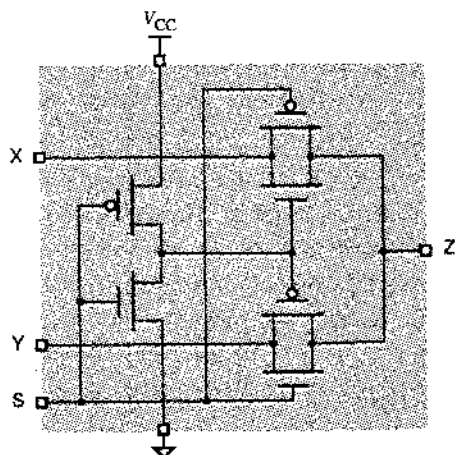


Рис. 3.45. Двухвходовой мультиплексор с КМОП-ключом

По крайней мере одна промышленная фирма (Integrated Device Technology) выпускает ряд логических схем на основе логических ключей. Мультиплексорам этой фирмы требуется несколько наносекунд для образования пути от входа до выхода (от X или от Y до Z) после изменения сигнала «выбор» на входе S (см. рис. 3.45). Однако после того, как путь установлен, задержка распространения сигнала от входа до выхода в большинстве случаев не превосходит 0.25 нс; это самый быстрый дискретный КМОП-мультиплексор из имеющихся в продаже.

3.7.2. Триггер Шмитта

На рис. 3.25 уже была показана передаточная характеристика (зависимость выходного напряжения от входного) типичного КМОП-вентиля. Соответствующая передаточная характеристика инвертора с триггером Шмитта на входе (*Schmitt-trigger input*) показана на рис. 3.46(a). Триггер Шмитта – это специальная схема с внутренней обратной связью, позволяющей смещать порог переключения в зависимости от того, в каком направлении изменяется входной сигнал: от низкого уровня к высокому или от высокого уровня к низкому.

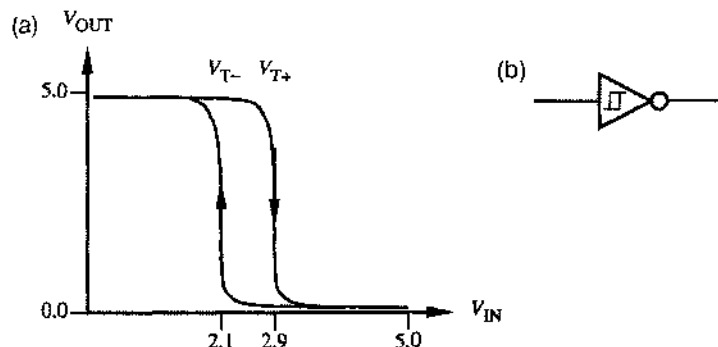


Рис. 3.46. Инвертор с триггером Шмитта: (a) передаточная характеристика; (b) условное обозначение

Предположим, например, что первоначальное напряжение на входе инвертора с триггером Шмитта равно 0 В: сигнал на входе имеет установившийся низкий уровень. Тогда на выходе будет высокий уровень с напряжением около 5.0 В. Когда входное напряжение увеличивается, выходной сигнал не перейдет на низкий уровень до тех пор, пока напряжение на входе не достигнет величины, примерно равной 2.9 В. Однако после перехода выходного напряжения на низкий уровень, оно не возвращается к высокому уровню до тех пор, пока входное напряжение не уменьшится приблизительно до 2.1 В. Таким образом, порог переключения V_{T+} при изменении входного напряжения в сторону увеличения равен примерно 2.9 В, а порог переключения V_{T-} при изменении входного напряжения в сторону уменьшения равен примерно 2.1 В. Различие между этими двумя порогами называется *гистерезисом* (*hysteresis*). У инвертора с триггером Шмитта ширина петли гистерезиса приблизительно равна 0.8 В.

Чтобы продемонстрировать полезность гистерезиса, на рис. 3.47(a) изображен входной сигнал с большим временем нарастания и спада и наложенным на него

шумом порядка 0.5 В. У обычного инвертора без гистерезиса порог переключения в сторону увеличения и уменьшения напряжения одинаков и равен $V_T \approx 2.5$ В. Поэтому обычный инвертор реагирует на шум так, как показано на рис. 3.47(б): на его выходе возникают многократные изменения сигнала, происходящие каждый раз, когда входное напряжение с помехами пересекает порог переключения. Инвертор с триггером Шмитта, как показано на рис. 3.47(с), не реагирует на шум, поскольку у него ширина петли гистерезиса превышает амплитуду помех.

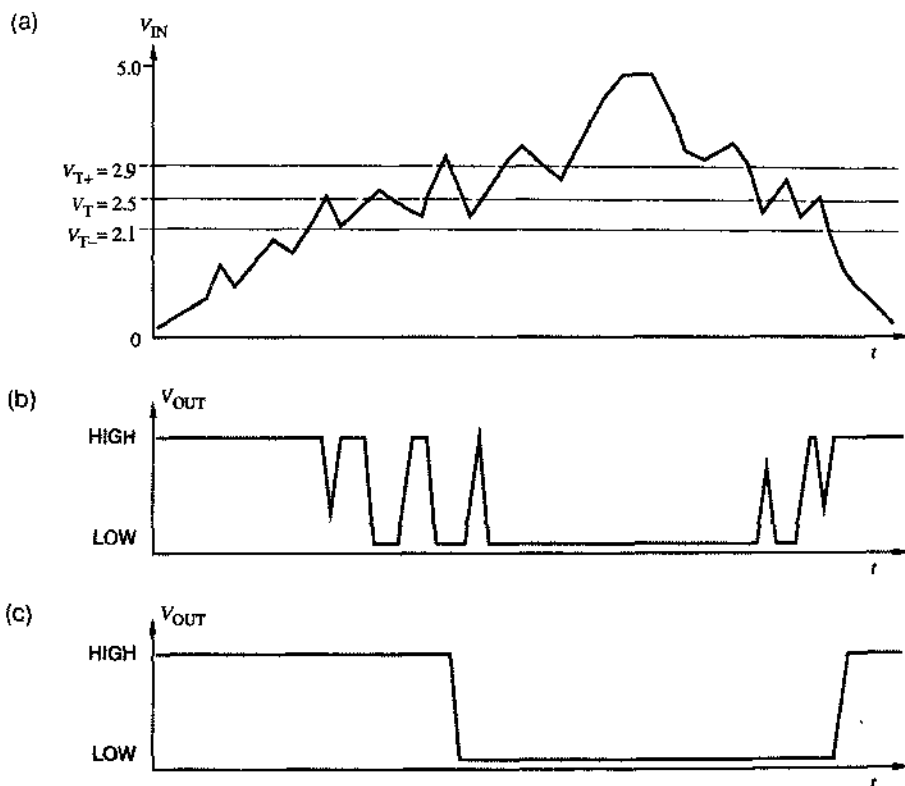


Рис. 3.47. Поведение логических схем при медленном изменении сигнала на входе: (а) медленно изменяющийся входной сигнал с шумом; (б) сигнал на выходе обычного инвертора; (с) сигнал на выходе инвертора с петлей гистерезиса шириной 0.8 В

3.7.3. Схемы с тремя состояниями

У выходов логических схем существует два нормальных состояния – низкий уровень (LOW) и высокий уровень (HIGH), соответствующие логическим значениям 0 и 1. Однако у некоторых выходов имеется третье состояние, называемое *высоким сопротивлением* (*high-impedance state*), *состоянием Hi-Z* (*Hi-Z state*) или *плавающим состоянием* (*floating state*); третьему состоянию не присписывается какое-либо логическое значение. В третьем состоянии выход ведет себя так, как будто он

не соединен со схемой, за исключением малого тока утечки, втекающего или вытекающего через выходной контакт. Таким образом, выход может находиться в одном из трех состояний — в состоянии логического 0, логической 1 и Hi-Z.

ПРИВЕДИТЕ В ПОРЯДОК ПЕРЕДАЧУ ДАННЫХ

По сравнению с обычными логическими схемами вентили с триггером Шмитта на входе менее чувствительны к искажениям сигналов вследствие отражений в линии передачи (этот вопрос обсуждается в параграфе 11.4) и более помехоустойчивы при большой длительности нарастания и спада сигнала. Такие сигналы обычно наблюдаются в физически длинных линиях связи типа шин ввода-вывода и кабелей компьютерных интерфейсов. В этих случаях важна нечувствительность к шумам, так как в длинных сигнальных линиях, скорее всего, будут отражения или на них будет наводиться шум от соседних линий, схем и устройств.

Выход с тремя возможными состояниями называется (какая неожиданность!) *выходом с тремя состояниями* (*three-state output*, *tri-state output*). Схемы с тремя состояниями имеют дополнительный вход, обычно называемый «входом разрешения выхода» или «входом запрещения выхода», для перевода одного или нескольких выходов схемы в высокоомное состояние.

Шина с тремя состояниями (*three-state bus*) образуется путем соединения нескольких выходов с тремя состояниями. Блок управления входами разрешения выхода должен гарантировать, что в любой момент времени не более одного выхода находится в состоянии логического 0 или логической 1 (не в состоянии Hi-Z). Задавать на шину логические уровни (высокий и низкий) может единственное устройство, на которое подан сигнал разрешения выхода. Примеры организации шины с тремя состояниями приведены в параграфе 5.6.

На рис. 3.48(а) показана принципиальная схема КМОП-буфера с тремя состояниями (*three-state buffer*). Ради упрощения схемы входящие в нее элементы И-НЕ, ИЛИ-НЕ и инвертор представлены их условными обозначениями, а не в виде комбинации транзисторов; фактически в этой схеме используется 10 транзисторов (см. задачу 3.80). Из таблицы на рис. 3.48(б) видно, что при низком уровне на входе разрешения (EN) выходные транзисторы закрыты, и выход находится в третьем состоянии. В противном случае на выходе возникает высокий уровень или низкий уровень в зависимости от «данных» на входе А. В условном обозначении буферов и вентилях с тремя состояниями обычно бывает указан вход разрешения, изображаемый сверху, как показано на рис. 3.48(с).

ЮРИДИЧЕСКАЯ СПРАВКА

«TRI-STATE» — торговая марка фирмы National Semiconductor Corporation. Их юрист думает, что вам это следует знать.

Чтобы обеспечить соответствующий динамический режим выходных транзисторов при переходе в третье состояние и при выходе из него, реальная схема управления выходом с тремя состояниями может отличаться от показанной на рисунке. В частности, устройства с тремя состояниями на выходе обычно проектируют так, чтобы задержка при подаче разрешающего значения сигнала (переход от третьего состояния на выходе к низкому уровню или к высокому уровню) была несколько больше, чем задержка при подаче запрещающего значения сигнала (переход от низкого уровня или от высокого уровня на выходе к третьему состоянию). Это позволяет блоку управления одновременно подавать сигнал разрешения выхода на управляющий вход одного устройства и сигнал запрещения выхода – на управляющий вход другого устройства с гарантией, что второе устройство перейдет в третье состояние раньше, чем первое устройство выставит на шине высокий или низкий уровень.

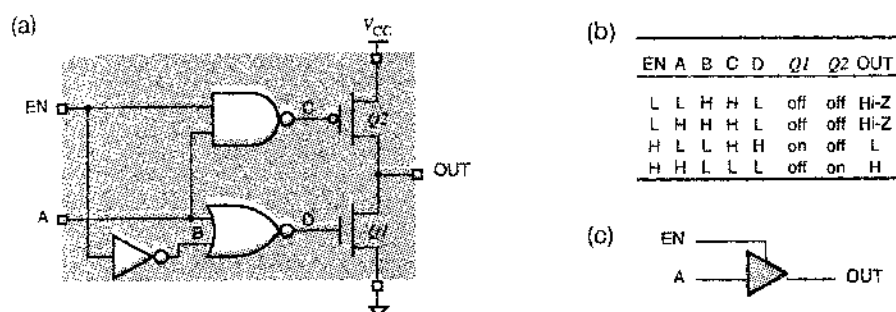


Рис. 3.48. КМОП-буфер с тремя состояниями: (a) принципиальная схема; (b) таблица, описывающая работу схемы; (c) условное обозначение (A – вход данных, EN – управляющий вход, OUT – выход; L – низкий уровень, H – высокий уровень; on – открыт, off – закрыт; Hi-Z – третье состояние)

Если два выхода с тремя состояниями, подключенные к одной шине, в одно и то же время находятся в активном режиме и пытаются установить противоположные уровни, то возникнет ситуация, подобная изображенной на рис. 3.56, где соединены между собой выходы обычных вентилях. В этой ситуации на шине устанавливается напряжение, не соответствующее какому-либо из логических уровней. Если такое состояние имеет место в течение короткого времени, то схемы, вероятнее всего, не будут повреждены, но значительный ток, протекающий через соединенные между собой выходы, может создать импульсную помеху и повлиять на работу каких-то других узлов в системе.

Когда КМОП-схема с тремя состояниями находится в третьем состоянии, на ее выходе течет ток утечки до 10 мкА. Этот ток, а также входные токи схем, подключенных к данному выходу, необходимо принять во внимание при расчете максимального числа устройств, которые можно подключить к шине с тремя состояниями. Другими словами, схема с тремя состояниями, которой разрешено установить на выходе низкий или высокий уровень, должна допускать втекание или вытекание по ее выходу токов утечки всех других выходов с тремя состояниями, подключенных к шине, каждый из которых может достигать до 10 мкА, а также входных токов всех входов, подключенных к шине. Как и в случае обычных КМОП-вентилей, рас-

чет необходимо провести отдельно для низкого уровня и для высокого уровня, чтобы гарантировать выполнение требований, касающихся коэффициента разветвления по выходу для конкретной конфигурации схемы.

*3.7.4. Схемы с открытым стоком

Говорят, что транзисторы с *p*-каналом в выходных цепях КМОП-схем обеспечивают *активное подтягивание* (*active pull-up*) выходного напряжения к потенциалу шины питания при переходе с низкого уровня на высокий уровень. Эти транзисторы отсутствуют в вентилях с *открытым стоком на выходе* (*open-drain outputs*), таких как вентиль И-НЕ, показанный на рис. 3.49(а). Сток верхнего транзистора с *n*-каналом остается внутри схемы не присоединенным ни к чему, поэтому если сигнал на выходе не соответствует низкому уровню, то выход «разомкнут», как указано на рис. 3.49(б). Иногда выход с открытым стоком обозначается подчеркнутым ромбом [рис. 3.49(с)]. Аналогичная структура имеется в семействах ТТЛ-схем; она называется «схемой с открытым коллектором» и описана в разделе 3.10.5.

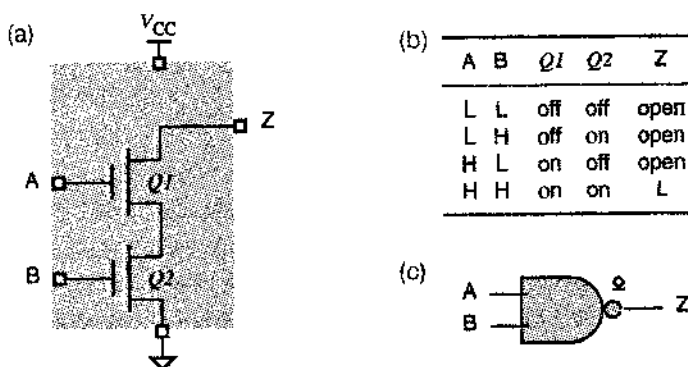


Рис. 3.49. КМОП-схема И-НЕ с открытым стоком: (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень; on – открыт, off – закрыт; open – разомкнут); (с) условное обозначение

Схеме с открытым стоком требуется внешний *подтягивающий резистор* (*pull-up resistor*) для *пассивного подтягивания* (*passive pull-up*) выхода к высокому уровню. Например, на рис. 3.50 показан КМОП-вентиль И-НЕ с открытым стоком с подтягивающим резистором и с подключенной нагрузкой.

Для обеспечения максимального возможного быстродействия, подтягивающий резистор в схеме с открытым стоком должен быть как можно меньше; это минимизирует постоянную времени RC при переходе с низкого уровня на высокий (время нарастания). Однако сопротивление подтягивающего резистора не может быть сколь угодно малым; его минимальное значение определяется максимальным током I_{OLmax} , который может втекать в схему с открытым стоком со стороны ее выхода. Например, у КМОП-схем серий HC и HCT $I_{OLmax} = 4$ мА и сопротивление подтягивающего резистора не может быть меньше, чем $5.0 \text{ В} / 4 \text{ мА}$ или 1.25 кОм . Так как эта величина на порядок больше, чем сопротивление «открытого» транзистора с *p*-каналом в обычном КМОП-вентиле, переключение выходного напряже-

ния с низкого уровня на высокий в случае вентиля с открытым стоком происходит медленнее, чем в случае обычного вентиля с активным подтягиванием.

Предположим, например, что используется КМОП-вентиль с открытым стоком серии HC (рис. 3.50) с сопротивлением подтягивающего резистора 1.5 кОм и емкостью нагрузки 100 пФ . В разделе 3.5.2 было показано, что при наличии на выходе низкого уровня выходное сопротивление КМОП-вентиля серии HC равно примерно 80 Ом . Таким образом, постоянная времени при переходе с высокого уровня на низкий равна приблизительно $80 \text{ Ом} \cdot 100 \text{ пФ} = 8 \text{ нс}$, и время спада выходного сигнала составляет 8 нс . Однако для перехода с низкого уровня на высокий постоянная времени равняется примерно $1.5 \text{ кОм} \cdot 100 \text{ пФ} = 150 \text{ нс}$, и время нарастания составляет 150 нс . Таким образом, нарастание выходного напряжения происходит относительно медленно в противоположность значительно более быстрому спаду напряжения, как показано на рис. 3.51. Один из приятелей автора называет такое медленное нарастание сигнала на выходе *болотом* (*ooze*).

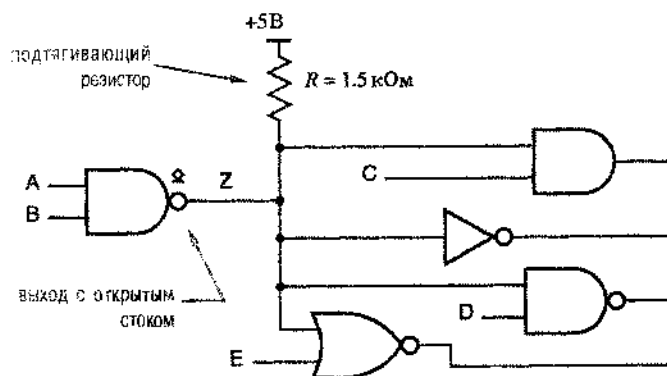


Рис. 3.50. КМОП-вентиль И-НЕ с открытым стоком с подключаемой нагрузкой

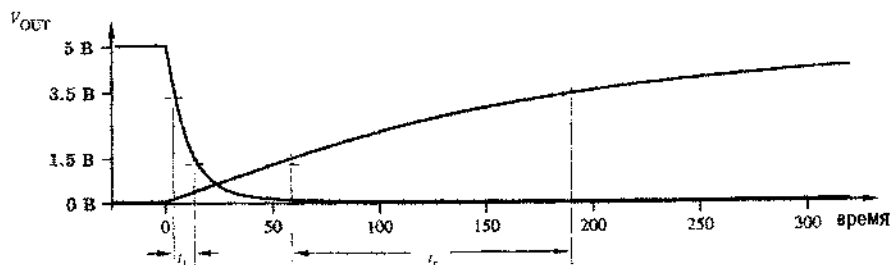


Рис. 3.51. Нарастание и спад напряжения на выходе КМОП-вентиля с открытым стоком

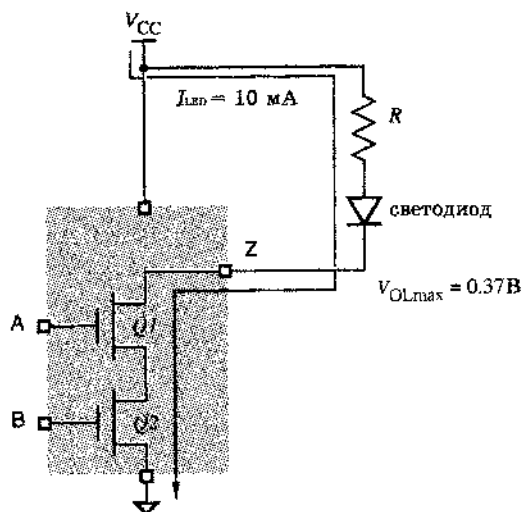
Так почему схемы с открытым стоком все же применяются? Несмотря на большое время нарастания, они могут быть полезны, по крайней мере, в трех случаях: для подключения светодиодов и других устройств, для реализации монтажной логики и для создания шин с многими источниками сигналов.

*3.7.5. Подключение светодиодов

Схему с открытым стоком, можно использовать для включения светодиода, как показано на рис. 3.52. Если сигнал на входе А или В имеет низкий уровень, то соответствующий транзистор с *n*-каналом закрыт и светодиод погашен. Когда на входы А и В одновременно подан высокий уровень, оба транзистора открыты, и на выходе Z сигнал имеет низкий уровень и светодиод светится. Сопротивление подтягивающего резистора *R* выбрано таким, чтобы во включенном состоянии через светодиод протекал необходимый ток.

Обычно для нормальной яркости свечения светодиодов требуется ток порядка 10 мА. У КМОП-схем серии HC и HCT втекающий или вытекающий ток составляет 4 мА и они, как правило, не применяются для включения светодиодов. Однако у более совершенных КМОП-схем типа 74АС и 74АСТ втекающий ток достигает 24 мА и больше, и их можно эффективно использовать для включения светодиодов.

Рис. 3.52. Подключение светодиода к выходу вентиля с открытым стоком



Чтобы найти нужное сопротивление резистора *R*, необходимы следующие данные:

1. Ток светодиода I_{LED} , необходимый для получения желаемой яркости; для типичного светодиода эта величина составляет 10 мА.
2. Падение напряжения на светодиоде в «открытом» состоянии V_{LED} , приблизительно равное 1.6 В в типичном случае.
3. Напряжение на выходе вентиля с открытым стоком V_{OL} , в который втекает ток светодиода. В КМОП-схемах серий 74АС и 74АСТ значение V_{OLmax} равно 0.37 В. Если в выходную цепь вентиля втекает ток I_{LED} и на выходе поддерживается напряжение меньшее, скажем, 0.2 В, то вычисляемое ниже сопротивление резистора оказывается немного меньше требуемого, но обычно это не причиняет вреда. Через светодиод потечет немного больший ток, чем I_{LED} , и он будет светиться чуть ярче, чем ожидалось.

Используя перечисленные данные, можно записать следующее соотношение:

$$V_{OL} + V_{LED} + (I_{LED} \cdot R) = V_{CC}.$$

Приимая $V_{CC} = 5.0$ В и беря приведенные выше типичные значения, можно найти требуемое значение R :

$$R = \frac{V_{CC} - V_{OL} - V_{LED}}{I_{LED}} = (5.0 - 0.37 - 1.6) \text{ В} / 10 \text{ мА} = 303 \text{ Ом}.$$

Заметьте, что для подключения светодиода не обязательно использовать выход с открытым стоком. На рис. 3.53(а) показано, как подключить светодиод к выходу обыкновенного КМОП-вентиля И-НЕ. Если на обоих входах имеется высокий уровень, то нижние (n -канальные) транзисторы обеспечивают на выходе низкий уровень, как и в случае схемы с открытым стоком. Если на любом из входов действует низкий уровень, то выходное напряжение имеет высокий уровень; при этом хотя бы один из верхних (p -канальных) транзисторов открыт и через светодиод ток не течет.

Некоторые семейства КМОП-схем позволяют включать светодиод, когда на выходе имеется высокий уровень, как показано на рис. 3.53(б). Это возможно, если вытекающий ток достаточен для удовлетворения требованиям, предъявляемым светодиодом. Однако последний вариант применяется не так часто как первый, потому что в большинстве случаев КМОП- и ТТЛ-схемы при наличии высокого уровня на выходе не могут давать такой же большой выходной ток, какой может втекать в них со стороны выхода при низком уровне.

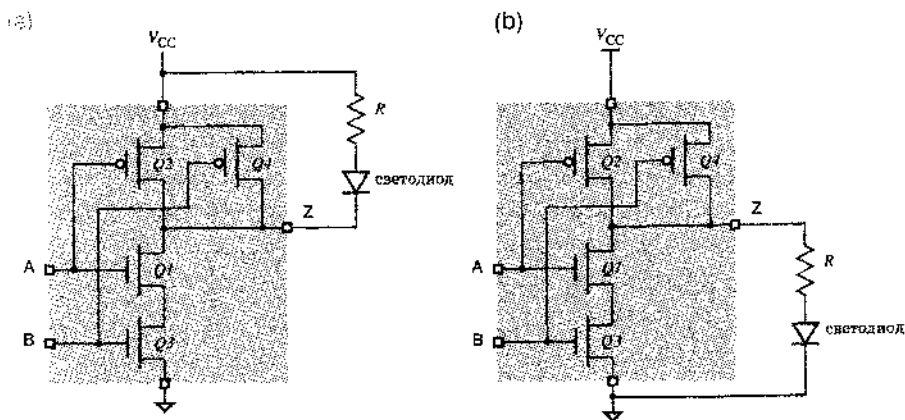


Рис. 3.53. Подключение светодиода к выходам обычных КМОП-схем: (а) светодиод светится, когда по нему течет ток, втекающий в схему при низком уровне на ее выходе; (б) светодиод светится, когда по нему течет ток, вытекающий из схемы при высоком уровне на ее выходе

СОПРОТИВЛЕНИЯ РЕЗИСТОРОВ

В большинстве случаев точное значение сопротивления резистора, включаемого последовательно со светодиодом, несущественно, поскольку для получения одинаково воспринимаемой яркости у многих близких по параметрам светодиодов необходимы примерно одни и те же токи и сопротивления резисторов. В примере данного раздела можно воспользоваться любым из имеющихся в наличии резисторов с номинальными сопротивлениями 270, 300 или 330 Ом.

* 3.7.6. Шины с несколькими источниками сигналов

Выходы с открытым стоком можно соединять вместе для того, чтобы позволить нескольким устройствам выдавать информацию на *общую шину (open-drain bus)*, но только одному из них в данный момент. В любой момент времени все выходы, подключенные к шине, кроме одного, находятся в состоянии, соответствующем высокому уровню (то есть в разомкнутом состоянии). Остающийся выход либо поддерживает высокий уровень на шине, либо создает на шине низкий уровень в зависимости от того, что требуется передать по шине: логическую 1 или логический 0. Блок управления указывает конкретную схему, которой разрешается использовать шину в течение некоторого времени.

Например, на рис. 3.54 к общей шине подключены восемь выходов 2-входовых схем И-НЕ с открытым стоком. На верхний вход каждого элемента И-НЕ поступает бит данных, а нижний вход является управляющим. В любой момент времени не более чем на одном управляющем входе присутствует высокий уровень, позволяя передать соответствующий бит данных по шине. (Фактически на шину попадает инверсное значение бита данных.) Выходы других вентилях находятся в состоянии, соответствующем высокому уровню, то есть «разомкнуты», поэтому значение сигнала на шине определяется сигналом на входе данных того из вентилях, на управляющий вход которого подан сигнал разрешения.

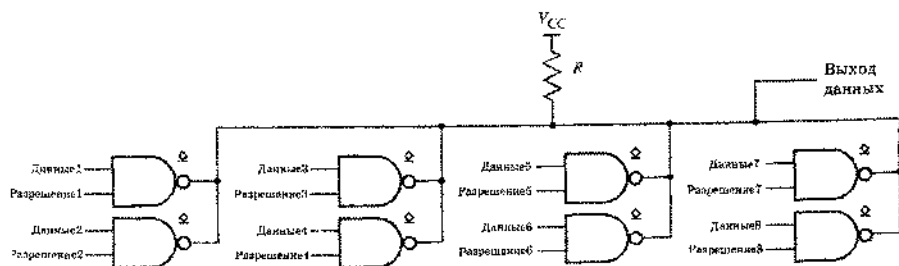


Рис. 3.54. Шина с подключенными к ней восемью схемами с открытым стоком

* 3.7.7. Монтажная логика

Подключая несколько выходов схем с открытым стоком к одному резистору, соединяющему эти выходы с шиной питания, можно реализовать так называемую *монтажную логику (wired logic)*. Мы получаем здесь логическую функцию И, так

как высокий уровень возникает на объединенном выходе (в действительности, выходы всех схем будут при этом разомкнуты) только в том случае, когда все выходы отдельных вентилях падают в состояние, соответствующем высокому уровню; достаточно на любом из выходов появиться низкому уровню, как объединенный выход также перейдет на низкий уровень. Например, на рис. 3.55 показана реализация функции «монтажное И» (*wired AND*) с тремя входами. Если на обоих входах какой-либо 2-входовой схемы И присутствует высокий уровень, то на объединенном выходе будет низкий уровень; в противном случае благодаря резистору R , соединяющему выходы схем с шиной питания, на объединенном выходе будет высокий уровень.

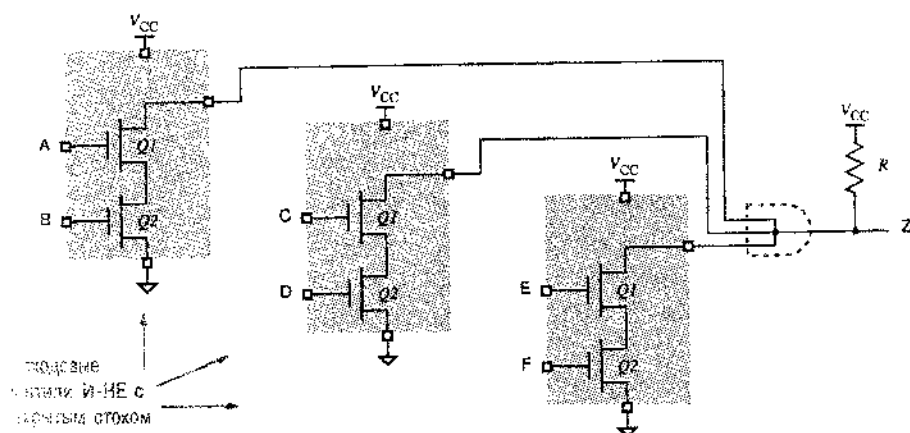


рис. 3.55. Реализация функции «монтажное И» путем объединения выходов трех вентилях И-НЕ с открытым стоком

Заметьте, что невозможно реализовать монтажную логику, используя схемы со стандартным выходом. Если две схемы, выходы которых соединены вместе, пытаются установить на своих выходах противоположные логические значения, то по выходным цепям этих схем потечет очень большой ток и на выходе установится неформальное напряжение. На рис. 3.56 показан такой случай, который иногда называют *борьбой* (*fighting*). Точное значение выходного напряжения зависит от «соотношения сил» борющихся транзисторов, но в случае КМОП-схем с напряжением питания 5 В оно обычно равно 1–2 В, что почти всегда не соответствует никакому из логических уровней. Хуже всего, если борьба между выходами схем продолжается дольше нескольких секунд: микросхемы могут так нагреться, что это приведет к их повреждению и, касаясь их, можно обжечься!

3.7.8. Резисторы, соединяющие выходы схем с шиной питания

При использовании схем с открытым стоком необходимо вычислить сопротивление резистора R , соединяющего их выходы с шиной питания (*pull-up-resistor calculation*). Чтобы установить диапазон возможных значений сопротивления R , выполняются следующие расчеты:

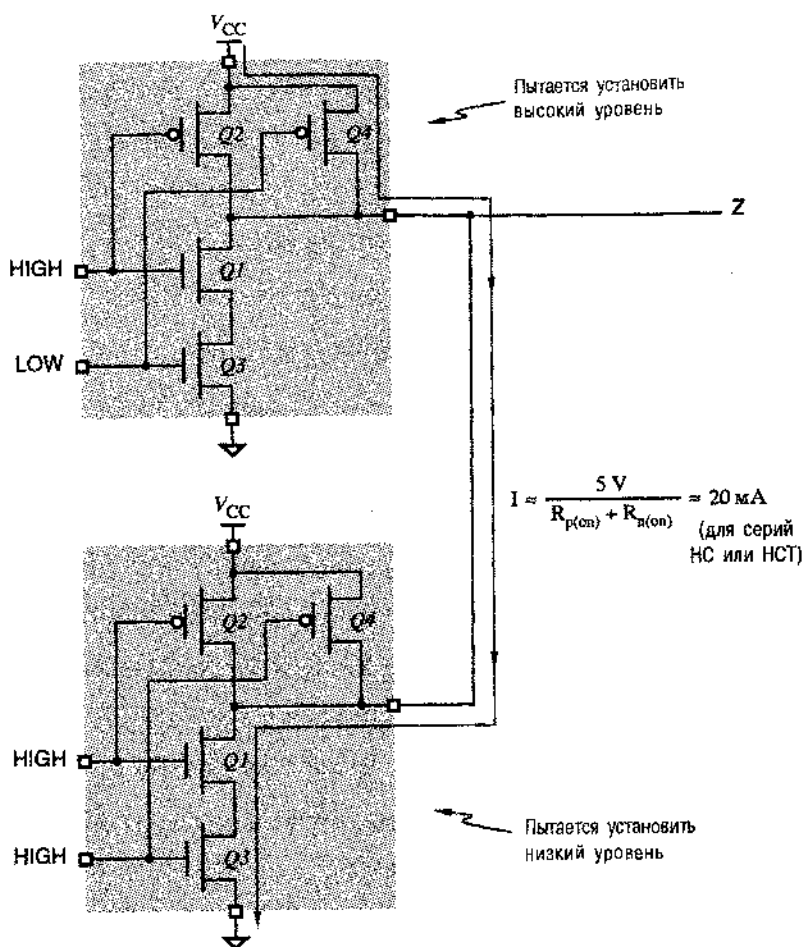


Рис. 3.56. Выходы двух КМОП-схем, пытающихся установить противоположные логические уровни на одной и той же линии

Минимальное значение

Сумма тока, протекающего по резистору R при низком уровне на выходах, объединяемых монтажной логикой, и входных токов схем, входы которых подключены к этим выходам при низком уровне на них, не должна превышать допустимого выходного тока схемы, находящейся в активном режиме, при низком уровне на ее выходе; этот ток равен 4 мА для серий НС и НСТ и 24 мА для серий АС и АСТ.

Максимальное значение

Падение напряжения на резисторе R при высоком уровне на выходе не должно приводить к тому, чтобы выходное напряжение было меньше $V_{\text{Нmin}} = 2.4 \text{ В}$ плюс 400 мВ в качестве запаса надежности. На резисторе будет падать некоторое напряжение из-за токов утечки выходов, объединенных монтажной логикой, при высоком уровне на этих выходах и входных токов схем, подключенных к этим выходам.

Предположим, например, что четыре выхода схем с открытым стоком объединены вместе и нагружены двумя входами ТТЛ-схем серии LS, как показано на рис. 3.57 (см. параграф 3.11). В схему с низким уровнем на выходе должен втекать со стороны ее выхода ток 0.4 мА от каждого входа ТТЛ-схемы серии LS плюс ток, протекающий через резистор R , соединяющий объединенные выходы схем с шиной питания. Для того, чтобы общий ток не превосходил допустимого значения I_{OLmax} равного 4 мА для серий НСТ, ток текущий по резистору R , не должен превышать

$$I_{R(max)} = 4 - (2 \cdot 0.4) = 3.2 \text{ мА}.$$

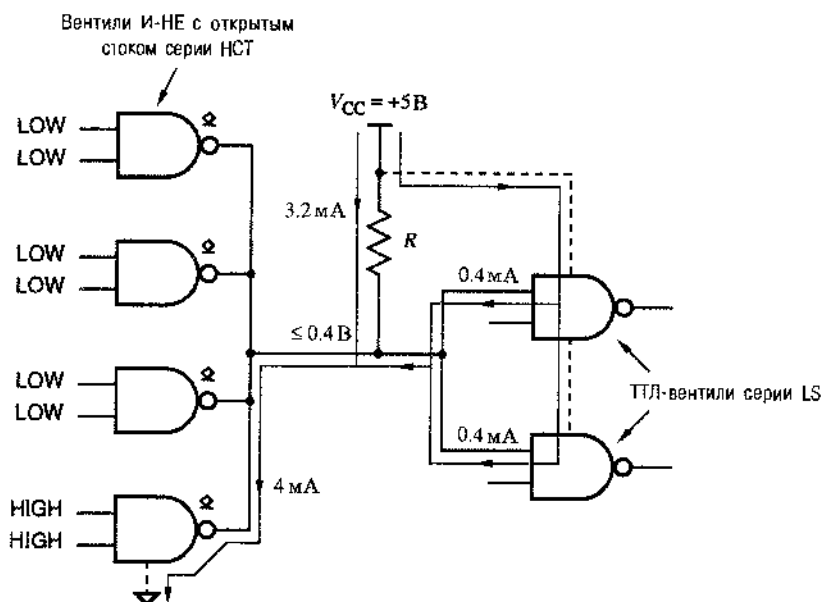


Рис. 3.57. Четыре схемы с открытым стоком, объединенные монтажной логикой и нагруженные двумя входами, при низком уровне на выходе

Полагая, что выходное напряжение V_{OL} низкого уровня схемы с открытым стоком равно 0.0 В , получим минимальное значение R :

$$R_{min} = (5.0 - 0.0) / I_{R(max)} = 1562.5 \text{ Ом}.$$

При высоком уровне на выходе максимальный ток утечки транзисторных схем с открытым стоком равен 5 мкА , а типичный входной ток ТТЛ-схемы серии LS равен 20 мкА . Следовательно, при высоком уровне на выходе, как показано на рис. 3.58, потребуется ток

$$I_{R(leak)} = (4 \cdot 5) + (2 \cdot 20) = 60 \text{ мкА}.$$

Этот ток создаст на резисторе R падение напряжения, которое не должно привести к уменьшению выходного напряжения до значения, меньшего чем $V_{OHmin} = 2.4 \text{ В}$; из этого условия находим максимальное значение R :

$$R_{\max} = (5.0 - 2.4) / I_{R(\text{leak})} = 43.3 \text{ кОм.}$$

Следовательно, можно воспользоваться любым значением R между 1562.5 Ом и 43.3 кОм. При большей величине сопротивления становится меньше потребляемая мощность и улучшается помехоустойчивость при низком уровне, в то время как меньшие значения сопротивления приводят к увеличению потребляемой мощности, но позволяют достичь лучшей помехоустойчивости при высоком уровне и обеспечивают большую скорость перехода от низкого уровня к высокому.

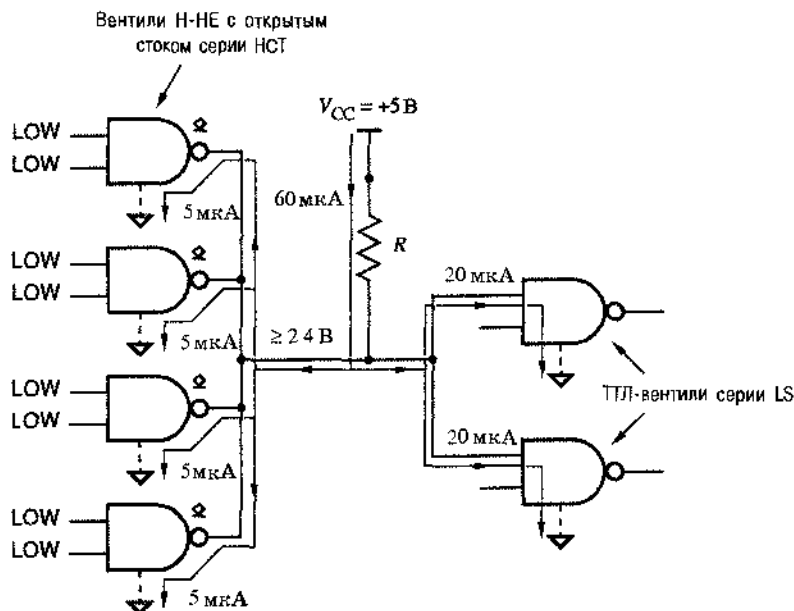


Рис. 3.58. Четыре схемы с открытым стоком, объединенные монтажной логикой и нагруженные двумя входами, при высоком уровне на выходе

ДОПУЩЕНИЕ, КАСАЮЩЕЕСЯ ОТКРЫТОГО СТОКА

Чтобы учесть наихудший случай при вычислении сопротивления резистора для схемы с открытым стоком, мы предполагаем, что выходное напряжение может опускаться до 0.0 В, а не до 0.4 В ($V_{OL\max}$): даже если выход схемы с открытым стоком настолько мощный, что выходное напряжение может понизиться до 0.0 В (тогда как требуется, чтобы оно опустилось только до 0.4 В), мы никогда не допустим, чтобы ток, втекающий в схему со стороны ее выхода, был больше 4 мА, так что перегрузка не произойдет. Некоторые разработчики, проводя такие вычисления, предпочитают использовать величину 0.4 В, полагая, что в случае, когда выходное напряжение может опуститься ниже 0.4 В, небольшое превышение номинального значения втекающего тока (4 мА) не нанесет вреда.

3.8. Семейства схем КМОП-логики

Первым коммерчески успешным КМОП-семейством были *КМОП-схемы серии 4000* (4000-series CMOS). Достоинством схем серии 4000 была малая рассеиваемая мощность, но они были довольно медленными и их было не просто состыковать с наиболее популярным в то время логическим семейством – бинарными ТТЛ-схемами. Поэтому в большинстве приложений серия 4000 была вытеснена более совершенными КМОП-семействами, о которых и будет рассказано.

Все рассматриваемые здесь КМОП-схемы обозначаются как "74FAM $_{nn}$ ", где "FAM" – буквенное обозначение семейства (FAMily), а nn – числовой указатель реализуемой функции. Схемы из различных семейств с одним и тем же значением nn реализуют одну и ту же функцию. Например, любая из схем 74НС30, 74НСТ30, 74АС30, 74АСТ30 и 74АНС30 – это 8-входовой элемент И-НЕ.

Префикс "74" – это просто номер, который использовался одним из первых производителей ТТЛ-схем, популярной компанией Texas Instruments. Префикс "54" используется для аналогичных схем, которые могут работать в более широком диапазоне температур и напряжений питания и применяются в военных разработках. Обычно схемы серии 54 изготавливаются так же, как схемы серии 74, за исключением того, что они по-другому протестированы, экранированы и маркированы. Кроме того, эти схемы сопровождаются многочисленной дополнительной документацией и, конечно, стоят дороже.

3.8.1. Семейства схем НС и НСТ

Первые два КМОП-семейства серии 74 – это семейства *НС* (*быстродействующие КМОП-схемы; High-speed CMOS*) и *НСТ* (*совместимые с ТТЛ быстродействующие КМОП-схемы; High-speed CMOS, TTL compatible*). По сравнению с исходным семейством 4000, схемы семейств НС и НСТ обладают большим быстродействием и имеют лучшие значения втекающего и вытекающего токов. Схемы семейства НСТ работают с напряжением питания $V_{CC} = 5$ В и их можно использовать совместно с ТТЛ-схемами, также работающими с 5-вольтовым напряжением питания.

Семейство НС предназначено для применения в системах, использующих только КМОП-логику; схемы этого семейства могут работать с любым напряжением питания от 2 до 6 В. При более высоком напряжении повышается быстродействие, а при более низком – снижается потребляемая мощность. Понижение питающего напряжения особенно эффективно, так как почти вся мощность, рассеиваемая КМОП-схемами, пропорциональна квадрату напряжения питания (CV^2f -мощность).

Схемы семейства НС не вполне совместимы с ТТЛ-схемами даже в том случае, когда напряжение питания равно 5 В. В частности, схемы семейства НС сконструированы так, чтобы работать с входными уровнями КМОП-схем. На рис. 3.59(а) приведены входные и выходные уровни схем семейства НС при напряжении питания 5 В. Входные уровни ТТЛ-схем не совсем соответствуют диапазону, указанному на рис. 3.59(а); у схем семейства НСТ входные уровни другие [рис. 3.59(б)]. Эти уровни устанавливаются в процессе изготовления, путем создания транзисторов с различными порогами переключения, что приводит к различным передаточным характеристикам, показанным на рис. 3.60.

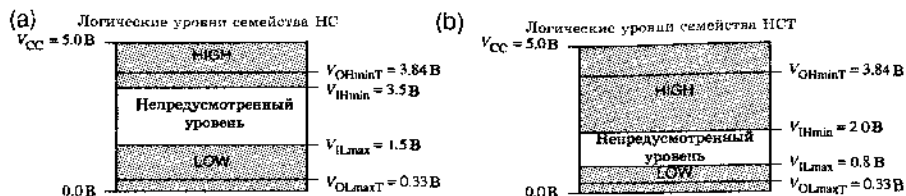
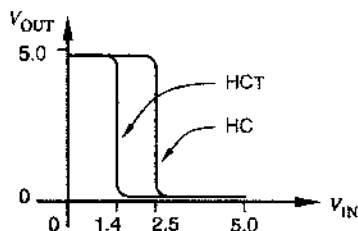


Рис. 3.59. Входные и выходные уровни КМОП-схем при напряжении питания 5 В: (a) семейство HC; (b) семейство HCT

Рис. 3.60. Типичные передаточные характеристики схем семейств HC и HCT



В параграфе 3.12 подробнее рассматривается сопряжение КМОП- и ТТЛ-схем. Пока же полезно просто заметить, что схемы серий HC и HCT одинаковы в отношении выходных характеристик; они отличаются только входными уровнями.

3.8.2. Семейства схем VHC и VHCT

В 80-е и 90-е годы появилось несколько новых семейств КМОП-схем. Двумя самыми последними и, вероятно, наиболее универсальными семействами являются VHC (КМОП-схемы с повышенным быстродействием; *Very High-speed CMOS*) и VHCT (совместимые с ТТЛ КМОП-схемы с повышенным быстродействием; *Very High-speed CMOS, TTL compatible*). Схемы этих семейств работают примерно вдвое быстрее, чем схемы семейств HC/HCT при сохранении совместности со своими предшественниками.

Подобно семействам HC и HCT, семейства VHC и VHCT отличаются одно от другого только допустимыми входными уровнями; выходные характеристики у них совпадают.

Так же, как и в семействах HC/HCT, схемы семейств VHC/VHCT имеют симметричный выходной каскад (*symmetric output drive*), то есть втекающий и вытекающий токи у этих схем равны; выход одинаково «мощный» в обоих состояниях. Другие логические семейства, включая рассматриваемые ниже семейства FCT и TTL, имеют асимметричный выходной каскад (*asymmetric output drive*); у них втекающий ток при низком уровне на выходе намного больше вытекающего тока при высоком уровне на выходе.

3.8.3. Электрические характеристики схем семейств HC, HCT, VHC и VHCT

В этом разделе собраны электрические характеристики схем семейств HC, HCT, VHC и VHCT. Приведенные характеристики относятся к случаю, когда напряжение

питания равно номинальному значению 5 В, хотя работа возможна (но с другими параметрами) при любом напряжении питания в диапазоне от 2 до 5,5 В (для семейств НС/НСТ до 6 В). Более подробно работа этих схем при низком и при смешанном напряжениях питания будет рассмотрена в параграфе 3.13.

Коммерческие схемы (серия 74) предназначены для работы в диапазоне температур от 0°C до 70°C, а военные схемы (серия 54) могут работать от -55°C до 125°C. Параметры схем в табл. 3.5 приведены для рабочей температуры 25°C. Полные справочные данные, предоставляемые изготовителем, дают дополнительную информацию о работе схем во всем диапазоне температур.

Большинство схем в пределах данного логического семейства имеют одинаковые входные и выходные электрические параметры, отличаясь обычно только потребляемой мощностью и задержкой распространения. В табл. 3.5 приведены параметры двухходовой схемы И-НЕ 74х00 и дешифратора 3х8 74х138 семейств НС, НСТ, VНС и VНСТ. Логическая схема '00 (вентиль И-НЕ) представляет собой наименьший стандартный логический блок в каждом семействе, а схема '138 является представителем «среднего уровня», содержание которого эквивалентно примерно 15 вентилям И-НЕ. (Параметры микросхемы '138 включены с целью сравнения в разделе 3.8.4 со свойствами самого быстрого семейства FCT; микросхема '00 в семействе FCT отсутствует.)

Табл. 3.5. Быстродействие и потребляемая мощность КМОП-схем при напряжении питания 5 В

Параметр	Номер ИС	Обозначение	Условия	Семейство			
				НС	НСТ	VНС	VНСТ
Типичная задержка распространения (нс)	'00	t_{PD}		9	10	5.2	5.5
	'138			18	20	7.2	8.1
Потребляемый ток в режиме покоя (мкА)	'00	I_{CC}	$V_{in} = 0 \text{ или } V_{CC}$	2.5	2.5	5.0	5.0
	'138		$V_{in} = 0 \text{ или } V_{CC}$	40	40	40	40
Потребляемая мощность в режиме покоя (мВт)	'00		$V_{in} = 0 \text{ или } V_{CC}$	0.0125	0.0125	0.025	0.025
	'138		$V_{in} = 0 \text{ или } V_{CC}$	0.2	0.2	0.2	0.2
Емкость для расчета потребляемой мощности (пФ)	'00	C_{PD}		22	15	19	17
	'138			55	51	34	49
Динамическая потребляемая мощность (мВт/МГц)	'00			0.55	0.38	0.48	0.43
	'138			1.38	1.28	0.85	1.23
Полная потребляемая мощность (мВт)	'00		$f = 100 \text{ кГц}$	0.068	0.050	0.073	0.068
	'00		$f = 1 \text{ МГц}$	0.56	0.39	0.50	0.45
	'00		$f = 10 \text{ МГц}$	5.5	3.8	4.8	4.3
	'138		$f = 100 \text{ кГц}$	0.338	0.328	0.285	0.323
	'138		$f = 1 \text{ МГц}$	1.58	1.48	1.05	1.43
	'138		$f = 10 \text{ МГц}$	14.0	13.0	8.7	12.5
	'138		$f = 100 \text{ кГц}$	0.61	0.50	0.38	0.37
Произведение быстродействия на мощность (пДж)	'00		$f = 1 \text{ МГц}$	5.1	3.9	2.6	2.5
	'00		$f = 10 \text{ МГц}$	50	38	25	24
	'138		$f = 100 \text{ кГц}$	6.08	6.55	2.05	2.61
	'138		$f = 1 \text{ МГц}$	28.4	29.5	7.56	11.5
	'138		$f = 10 \text{ МГц}$	251	259	63	101
	'138		$f = 100 \text{ кГц}$	0.61	0.50	0.38	0.37

"V" и "A" – почти одно и то же

Схемы логических семейств VHC и VHCT выпускаются несколькими производителями, в том числе фирмами Motorola, Fairchild и Toshiba. Совместимые с ними семейства с похожими, но не точно такими же параметрами выпускаются фирмами Texas Instruments и Philips; эти семейства называются AHC и AHCT, где "A" означает «улучшенный» ("Advanced").

Первая строка в табл. 3.5 характеризует задержку распространения. Как было сказано в разделе 3.6.2, для определения задержки можно воспользоваться двумя величинами: t_{PHL} и t_{PLH} ; число, приведенное в таблице, представляет собой худшее из этих двух значений. Забегая вперед и обращаясь к табл. 3.11, можно видеть, что быстродействие схем семейств HC и HCT примерно такое же, как у TTL-схем LS, а схемы семейств VHC и VHCT по быстродействию близки к TTL-схемам ALS. Задержка распространения у схемы '138 несколько больше, чем у схемы '00, так как внутри схемы '138 сигналы должны проходить сквозь три или четыре слоя вентилей.

Из второй и третьей строк таблицы видно, что рассеиваемая мощность в режиме покоя у этих КМОП-схем практически равна нулю, во всяком случае заметно меньше милливатта (мВт), когда на входах действуют сигналы, соответствующие уровням КМОП-схем: 0 В для низкого уровня и V_{CC} для высокого уровня. (Обратите внимание, что в таблице значения рассеиваемой мощности в режиме покоя даны для одного вентиля схемы '00, в то время как в случае схемы '138 они относятся ко всей микросхеме средней степени интеграции.)

В разделе 3.6.3 было показано, что динамическая мощность, рассеиваемая КМОП-вентилем, зависит от перепада выходного напряжения (обычно равного V_{CC}), частоты переключения (f) и емкости, которая заряжается и разряжается при переключении:

$$P_D = (C_L + C_{PD}) \cdot V_{DD}^2 \cdot f$$

Здесь C_{PD} – емкость вентиля, которой определяется рассеиваемая мощность, а C_L – емкость нагрузки, подключенной к выходу КМОП-схемы в рассматриваемом случае. В таблице приведены значения емкости C_{PD} и соответствующей ей динамической рассеиваемой мощности в милливаттах на мегагерц при условии, что $C_L = 0$. Полная рассеиваемая мощность для разных частот определяется как сумма рассеиваемой динамической мощности на данной частоте и рассеиваемой мощности в режиме покоя.

ОБРАТИТЕ ВНИМАНИЕ НА СИСТЕМУ ОБОЗНАЧЕНИЙ

На место символа "х" в обозначении "74х00" помещается название семейства: HC, HCT, VHC, VHCT, FCT, LS, ALS, AS или F. Кроме того, эту схему из разных семейств можно просто обозначать как "00", отбрасывая "74х".

ПОЛЕЗНЫЕ СООБРАЖЕНИЯ

На входы схем семейств НСТ и VHST можно подавать также сигналы от ТТЛ-схем с нижней границей высокого уровня на выходе 2.4 В. Как было показано в разделе 3.5.3, по выходной цепи КМОП-схемы может протекать дополнительный ток от источника питания, если уровень сигнала на каком либо входе неидеален. В случае НСТ- или VHST-инвертора с входным высоким уровнем равным 2.4 В, нижний выходной транзистор с n -каналом полностью «открыт». Но и верхний транзистор с p -каналом также частично «открыт». Это приводит к тому, что в режиме покоя протекает дополнительный ток, обозначаемый в справочных данных как ΔI_{CC} или I_{CC1} , который при неидеальном уровне входного сигнала у НСТ- и VHST-схем может достигать 2–3 мА.

Следующий параметр, указанный в таблице, называется *произведением скорости действия на мощность (speed-power product)*; он представляет собой произведение задержки распространения и потребляемой мощности типичного вентиля; единицей измерения является пикоджоуль (пДж). Вспомним из физики, что джоуль является единицей измерения энергии, поэтому произведение величины, имеющей размерность времени, на мощность отражает своего рода эффективность: оно показывает, сколько энергии затрачивается логическим вентилем на переключение. В наше время очевидно, что чем меньше потребляется энергии, тем лучше.

Табл. 3.6. Входные параметры КМОП-схем при напряжении питания V_{CC} от 4.5 В до 5.5 В

Параметр	Обозначение	Условие	Семейство			
			НС	НСТ	VHC	VHST
Входной ток утечки (мкА)	I_{Imax}	V_{in} = любой	± 1	± 1	± 1	± 1
Максимальная входная емкость (пФ)	C_{inmax}		10	10	10	10
Напряжение низкого уровня на входе (В)	V_{ILmax}		1.35	0.8	1.35	0.8
Напряжение высокого уровня на входе (В)	V_{IHmin}		3.85	2.0	3.85	2.0

ЭКОНОМИЯ ЭНЕРГИИ

Существуют как практические, так и геологические причины сокращения потребляемой мощности в цифровых системах. Более низкий расход энергии означает уменьшение стоимости источников питания и систем охлаждения. Кроме того, повышение надежности цифровой системы за счет работы при более низкой температуре больше, чем при использовании какой-либо другой стратегии повышения надежности.

В табл. 3.6 приведены входные параметры типичных КМОП-схем каждого семейства. В отношении некоторых параметров предполагается, что напряжение питания, номинально равное 5 В, может изменяться в пределах $\pm 10\%$; то есть значение V_{CC} может быть между 4.5 В и 5.5 В. Эти параметры уже рассматривались в предыдущих разделах, но ради удобства мы вновь кратко укажем, что именно они означают:

- $I_{In\max}$ — максимальный входной ток при любом значении входного напряжения; этот параметр говорит о том, что протекающий или протекающий входной ток КМОП-схемы не превышает 1 мкА, каким бы ни было напряжение на входе; другими словами, входы КМОП-схем не создают почти никакой нагрузки на постоянному току для схем, к выходам которых они подключены.
- $C_{In\max}$ — максимальная входная емкость; этим параметром можно воспользоваться, чтобы оценить нагрузку на переменному току для выхода, к которому подключен данный вход наряду с другими входами; кроме того, большинство производителей в качестве типичного указывает меньшее значение входной емкости — около 5 нФ, и это является хорошей оценкой нагрузки на переменному току, разве что вам не повезет.
- $V_{IL\max}$ — максимальное входное напряжение, которое гарантированно распознается как низкий уровень; заметьте, что значения $V_{IL\max}$ для семейств НС/ВНС отличаются от значений этого параметра для семейств НСТ/ВНСТ; обратите внимание также, что у КМОП-схем, не предназначенных для совместного использования с ТТЛ-схемами, это напряжение равно 1.35 В, что составляет 30% от минимального значения напряжения питания, в то время как у схем, совместных со схемами ТТЛ-семейств, оно равняется 0.8 В.
- $V_{IH\min}$ — минимальное напряжение, которое гарантированно распознается на входе как высокий уровень; у КМОП-схем, не предназначенных для совместного использования с ТТЛ-схемами, это напряжение равно 3.85 В, что составляет 70% от максимального напряжения питания, в то время как у схем, совместных со схемами ТТЛ-семейств, оно равняется 2.0 В. (В отличие от уровней КМОП-схем, входные уровни ТТЛ-схем не расположены симметрично относительно земли и к шине питания.)

Табл. 3.7. Выходные параметры КМОП-схем при напряжении питания V_{CC} от 4.5 В до 5.5 В

Параметр	Обозначение	Условие	Семейство			
			НС	НСТ	ВНС	ВНСТ
Выходной ток при низком уровне (мА)	$I_{OL\max C}$	КМОП-нагрузка	0.02	0.02	0.05	0.05
	$I_{OL\max T}$	ТТЛ-нагрузка	4.0	4.0	8.0	8.0
Выходное напряжение при низком уровне (В)	$V_{OL\max C}$	$I_{out} \leq I_{OL\max C}$	0.1	0.1	0.1	0.1
	$V_{OL\max T}$	$I_{out} \leq I_{OL\max T}$	0.33	0.33	0.44	0.44
Выходной ток при высоком уровне (мА)	$I_{OH\max C}$	КМОП-нагрузка	-0.02	-0.02	-0.05	-0.05
	$I_{OH\max T}$	ТТЛ-нагрузка	-4.0	-4.0	-8.0	-8.0
Выходное напряжение при высоком уровне (В)	$V_{OH\min C}$	$I_{out} \leq I_{OH\max C}$	4.4	4.4	4.4	4.4
	$V_{OH\min T}$	$I_{out} \leq I_{OH\max T}$	3.84	3.84	3.80	3.80

МОЩНОСТЬ, ПОТРЕБЛЯЕМАЯ КМОП- и ТТЛ-СХЕМАМИ

При переключении с большой частотой (f) КМОП-схемы фактически потребляют большую мощность, чем ТТЛ-схемы. Сравните, например, данные для КМОП-схем семейства НСТ, приведенные в табл. 3.5 для частоты переключения $f = 10$ МГц, с данными для ТТЛ-схем семейства LS в табл. 3.11; на указанной частоте КМОП-схема потребляет втрое большую мощность, чем ТТЛ-схема. НСТ- и LS-схемы можно применять в системах с максимальной тактовой частотой порядка 20 МГц, так что можно подумать, что КМОП-схемы не достаточно хороши для высокоскоростных систем. Однако для большинства вентилей в типичных системах частота переключения намного меньше максимальной частоты в системе (см., например, задачу 3.77). Таким образом, в типичных системах на основе КМОП-схем полная рассеиваемая мощность меньше, чем в аналогичных системах, построенных на ТТЛ-схемах.

Совместимые с ТТЛ КМОП-схемы обычно имеют два набора выходных параметров; используется тот или другой набор в зависимости от характера нагрузки. *КМОП-нагрузка (CMOS load)* — это такая нагрузка, при которой в режиме покоя втекающий и вытекающий постоянные токи очень малы: 20 мкА для схем НС/НСТ и 50 мкА для схем ВНС/ВНСТ. Сюда, конечно, относится случай, когда к выходам КМОП-схем подключены только входы КМОП-схем. При КМОП-нагрузке выходное напряжение КМОП-схем отличается от потенциала земли 0 В и от напряжения питания V_{CC} не более, чем на 0.1 В. (В таблице имеется в виду падающий случай, когда $V_{CC} = 4.5$ В; поэтому $V_{OHminC} = 4.4$ В.)

В случае *ТТЛ-нагрузки (TTL load)* может потребоваться намного больший втекающий или вытекающий ток: до 4 мА для схем НС/НСТ и до 8 мА для схем ВНС/ВНСТ. При этом на «открытых» транзисторах в выходной цепи падает большое напряжение, но выходное напряжение все еще остается в пределах нормального диапазона входных уровней ТТЛ-схем.

В табл. 3.7 перечислены выходные параметры КМОП-схем для КМОП- и ТТЛ-нагрузок. Эти параметры означают следующее:

- I_{OLmaxC} — максимальный выходной ток при низком уровне в случае КМОП-нагрузки; так как эта величина положительна, данный ток *втекает* в схему со стороны ее выхода.
- I_{OLmaxT} — максимальный выходной ток при низком уровне в случае ТТЛ-нагрузки.
- V_{OLmaxC} — максимальное напряжение, которое гарантируется на выходе при низком уровне в случае КМОП-нагрузки, то есть пока не превышено значение I_{OLmaxC} .
- V_{OLmaxT} — максимальное напряжение, которое гарантируется на выходе при низком уровне в случае ТТЛ-нагрузки, то есть пока не превышено значение I_{OLmaxT} .
- I_{OHmaxC} — максимальный выходной ток при высоком уровне в случае КМОП-нагрузки; эта величина отрицательна, так как, данный ток *вытекает* из схемы со стороны ее выхода.

- I_{OHmaxT} — максимальный выходной ток при высоком уровне в случае ТТЛ-нагрузки.
- V_{OHminC} — минимальное напряжение, которое гарантируется на выходе при высоком уровне в случае КМОП-нагрузки, то есть пока не превышено значение I_{OHmaxC} .
- V_{OHminT} — минимальное напряжение, которое гарантируется на выходе при высоком уровне в случае ТТЛ-нагрузки, то есть пока не превышено значение I_{OHmaxT} .

Значениям напряжений, приведенным выше, определяется запас помехоустойчивости по постоянному току. Запас помехоустойчивости по постоянному току при низком уровне равен разности между V_{ILmax} и V_{OLmax} . Он зависит как от выходных характеристик схемы, служащей источником сигнала, так и от входных характеристик схем, играющих роль нагрузки. Например, запас помехоустойчивости по постоянному току при низком уровне у схемы семейства НСТ, нагруженной несколькими входами схем этого же семейства (КМОП-нагрузка) составляет $0.8\text{ В} - 0.1\text{ В} = 0.7\text{ В}$. При ТТЛ-нагрузке запас помехоустойчивости для НСТ-выходов падает до значения $0.8\text{ В} - 0.33\text{ В} = 0.47\text{ В}$. Точно так же запас помехоустойчивости по постоянному току при высоком уровне равен разности между V_{OHmin} и V_{IHmin} . Вообще, когда к схемам одного семейства подключаются схемы другого семейства, необходимо сравнить соответствующие значения V_{OLmax} и V_{OHmin} управляющего вентиля со значениями V_{ILmax} и V_{IHmin} всех управляемых схем, чтобы определить запас помехоустойчивости в наихудшем случае.

Параметрами I_{OLmax} и I_{OHmax} , приведенными в таблице, определяется коэффициент разветвления по выходу; они особенно важны в том случае, когда к данному выходу подключены входы схем из одного или большего числа других семейств. Для определения способности выхода обеспечить номинальный коэффициент разветвления по выходу, необходимо выполнить следующие вычисления:

Коэффициент разветвления по выходу при высоком уровне

Суммируются значения I_{IHmax} всех подключенных входов: сумма должна быть меньше величины I_{OHmax} , указанной для выхода схемы, являющейся источником сигнала.

Коэффициент разветвления по выходу при низком уровне

Суммируются значения I_{ILmax} всех подключенных входов: сумма должна быть меньше величины I_{OLmax} , указанной для выхода схемы, являющейся источником сигнала.

Заметьте, что входные и выходные характеристики конкретных ИС могут отличаться от значений, указанных в табл. 3.7, так что при анализе реальной конструкции надо всегда обращаться к справочным данным производителей.

***3.8.4. Схемы семейств FCT и FCT-T**

В начале 90-х годов было начато производство еще одного КМОП-семейства. Главным достоинством семейства *FCT* (*совместимые с ТТЛ сверхбыстродействующие*)

щие КМОП-схемы; *Fast CMOS, TTL compatible*) явилось то, что в схемах этого семейства удалось достичь или превзойти быстродействие и нагрузочную способность схем лучших ТТЛ-семейств при одновременином уменьшении потребляемой мощности и сохранении полной совместимости с ТТЛ-схемами.

У первого семейства FCT был тот недостаток, что КМОП-уровень V_{OH} равнялся напряжению питания 5 В: это приводило к рассеянию очень большой CV^2f -мощности и появлению в высокоскоростных приложениях (с тактовой частотой 25 МГц и выше) помех, обусловленных изменением выходных напряжений от 0 В до почти 5 В. Поэтому в новых устройствах быстро нашло применение модифицированное семейство FCT-T (*совместимые с ТТЛ сверхбыстродействующие КМОП-схемы с ТТЛ-уровнем V_{OH} ; Fast CMOS, TTL compatible with TTL V_{OH}*); пониженное выходное напряжение высокого уровня позволило уменьшить как потребляемую мощность, так и помехи от переключения при сохранении того же быстродействия, как и у первоначального семейства FCT. В названии схем с таким выходом используется приставка "Т": например, 74FCT138Т вместо 74FCT138.

Семейство FCT-T остается очень популярным и сегодня. Схемы FCT-T используются, главным образом, для управления шинами и в других случаях больших нагрузок. По сравнению с другими КМОП-семействами у схем этого семейства могут быть очень большими протекающий или протекающий токи: до 64 мА при низком уровне на выходе.

*3.8.5. Электрические характеристики схем семейства FCT-T

В табл. 3.8 приведены электрические характеристики схемы семейства FCT-T, относящиеся к случаю, когда напряжение питания равно 5 В. Данное семейство было разработано специально для совместной работы с ТТЛ-схемами, поэтому предполагается, что схемы FCT-T работают только при номинальном напряжении питания 5 В, а логические уровни определяются транзисторно-транзисторной логикой. Некоторые производители начинают продавать микросхемы, обладающие подобными свойствами, но уже с напряжением питания 3.3 В, используя при этом маркировку FCT. Однако это другие схемы с другими цифрами в обозначении.

Отдельных логических вентилях в семействе FCT нет. Возможно, самым простым логическим элементом семейства FCT является дешифратор 74FCT138Т с шестью входами и восемью выходами, внутреннее содержимое которого эквивалентно примерно дюжине 4-входовых вентилях. (Функция, реализуемая этим элементом, описана позже, в разделе 5.4.4.) Сравнивая его задержку распространения и потребляемую мощность, приведенные в табл. 3.8, с соответствующими параметрами схем семейств НСТ и VHCT, указанными в табл. 3.5, можно видеть, что семейство FCT-T лучше и по быстродействию, и по рассеиваемой мощности. При сравнении заметьте, что производители схем FCT-T указывают только максимальные, а не типичные значения задержки распространения.

В отличие от других КМОП-семейств, для семейства FCT-T не указывается значение C_{PD} . Вместо этого приводится значение параметра I_{CCD} :

I_{CCD} — ток, потребляемый схемой от источника питания в динамическом режиме; измеряется в мА/МГц; это значение дополнительного тока, потребляемого от источника питания, когда сигнал на одном из входов изменяется с частотой 1 МГц.

Табл. 3.8. Параметры дешифратора 74FCT138T, принадлежащего семейству FCT-T

Описание	Обозначение	Условия	Значение
Максимальная задержка распространения (нс)	t_{PD}		5.8
Ток, потребляемый в режиме покоя (мкА)	I_{CC}	$V_{\text{in}} = 0$ или V_{CC}	200
Мощность, потребляемая в режиме покоя (мВт)		$V_{\text{in}} = 0$ или V_{CC}	1.0
Ток, потребляемый в динамическом режиме (мА/МГц)	I_{CCD}	Выходы не нагружены, сигнал изменяется на одном входе	0.12
Приращение потребляемого тока в режиме покоя при подключении одного ТТЛ-входа (мА)	ΔI_{CC}	$V_{\text{in}} = 3.4 \text{ В}$	2.0
Полная потребляемая мощность (мВт)		$f = 100 \text{ кГц}$	0.60
		$f = 1 \text{ МГц}$	1.06
		$f = 10 \text{ МГц}$	1.6
Произведение задержки на мощность (пДж)		$f = 100 \text{ кГц}$	6.15
		$f = 1 \text{ МГц}$	9.3
		$f = 10 \text{ МГц}$	41
Входной ток утечки (мкА)	I_{Imax}	V_{in} — любое	± 5
Типичная входная емкость (пФ)	C_{intyp}		5
Входное напряжение низкого уровня (В)	V_{ILmax}		0.8
Входное напряжение высокого уровня (В)	V_{IHmin}		2.0
Выходной ток низкого уровня (мА)	I_{OLmax}		64
Выходное напряжение низкого уровня (В)	V_{OLmax}	$I_{\text{out}} = I_{\text{OLmax}}$	0.55
Выходной ток высокого уровня (мА)	I_{OHmax}		-15
Выходное напряжение высокого уровня (В)	V_{OHmax}	$ I_{\text{out}} = I_{\text{OHmax}} $	2.4
	V_{OHtyp}	$ I_{\text{out}} = I_{\text{OHmax}} $	3.3

Параметр I_{CCD} дает ту же самую информацию о схеме, что и C_{PD} , но другим способом. Мощность, рассеиваемая внутри схемы при переключении с заданной частотой, можно рассчитать по формуле:

$$P_T = V_{CC} \cdot I_{CCD} \cdot f$$

Таким образом, величина I_{CCD}/V_{CC} численно равна параметру C_{PD} , указываемому для других КМОП-семейств (см. задачу 3.83). Кроме того, для схем семейства FCT-T задается параметр ΔI_{CC} , характеризующий величину дополнительного тока в режиме покоя при неидеальном значении входного сигнала высокого уровня (см. приведенные выше «Полезные соображения»).

СВЕРХБЫСТРАЯ КОММУТАЦИЯ

Выходные сопротивления схем, принадлежащих семействам FCT и FCT-T, очень малы и, как следствие, крайне мало время переключения этих схем. Фактически, переходные процессы настолько быстры, что часто являются основным источником «апалоговых» проблем, включая помехи от переключения и наводки на земляную шину. Поэтому в случае применения этих и других сверхбыстродействующих компонентов при проектировании реальной печатной платы следует предпринять особые меры предосторожности в отношении апалоговых аспектов работы проектируемого устройства. Чтобы уменьшить влияние отражений в линиях передачи (см. параграф 11.4), — а это другая проблема, возникающая в быстродействующих конструкциях, — в выходные цепи некоторых схем семейства FCT-T последовательно включают резисторы с сопротивлением 25 Ом.

3.9. Логические схемы на биполярных транзисторах

В биполярных логических схемах в качестве основных компонентов используются полупроводниковые диоды и биполярные транзисторы. Простейшие биполярные логические схемы, в которых логические функции реализуются с помощью диодов и резисторов, называются схемами *диодной логики (diode logic)*. Внутри большинства TTL-вентилей используется диодная логика, а увеличение нагрузочной способности достигается с помощью транзисторных схем. В некоторых TTL-схемах для реализации логических функций используют параллельное включение транзисторов. Для достижения очень высокого быстродействия в ЭСЛ-схемах, описанных в параграфе 3.14, транзисторы применяются в качестве токовых переключателей.

Данный параграф посвящен принципам работы биполярных логических схем, собранных из диодов и транзисторов, а в следующем параграфе подробно рассматриваются TTL-схемы. Хотя эти схемы являются самым распространенным семейством биполярных логических схем, к настоящему времени они в значительной степени вытеснены КМОП-семействами, о которых говорилось в предыдущих параграфах.

Тем не менее, полезно изучить принцип работы ТТЛ-схем для решения задач, связанных с сопряжением ТТЛ-схем с КМОП-схемами, которое будет рассмотрено в параграфе 3.12. Кроме того, понимание работы ТТЛ-схем поможет вам осознать, что именно благодаря сходству логических уровней промышленность смогла илавио перейти от ТТЛ-схем к КМОП-схемам с иапряжением питания 5 В, а теперь – к КМОП-схемам с иеишм иапряжением питания, равным 3.3 В, и лучшими характеристиками, о чем пойдет речь в параграфе 3.13. Если вас не интересуют все ужасные подробности ТТЛ-схем, то можно перепрыгнуть к параграфу 3.11 и ограничиться кратким обзором семейств ТТЛ.

3.9.1. Диоды

Полупроводниковый диод (semiconductor diode) состоит из полупроводников двух типов, иазываемых полупроводниками *p*-типа и *n*-типа, которые соединены друг с другом, как показано на рис. 3.61(а). Это, в основном, тот же самый материал, который используется в МОП-транзисторах с каналами *p*-типа и *n*-типа. Контакт между полупроводниками *p*-типа и *n*-типа называется *p-n*-переходом. (Обычно диод представляет собой один монокристалл полупроводника, в который вводят различные примеси, чтобы придать одной иоловине кристалла свойства иолупроводника *n*-типа, а другой половине – свойства полупроводника *p*-типа.)

Физические свойства *p-n*-перехода таковы, что ток может легко протекать от полупроводника *p*-типа к полупроводнику *n*-типа в положительном направлении. Таким образом, если мы составляем цепь, показанную на рис. 3.61(б), то *p-n*-переход представляет собой почти короткое замыкание. Но согласно тем же физическим свойствам протекание тока через *p-n*-переход в противоположном направлении – от полупроводника *n*-типа к полупроводнику *p*-типа – весьма затруднено. Таким образом, в схеме, изображенной на рис. 3.61(с), *p-n*-переход ведет себя почти как разомкнутый участок цепи. Говорят, что *p-n*-переход *действует как диод (diode action)*.

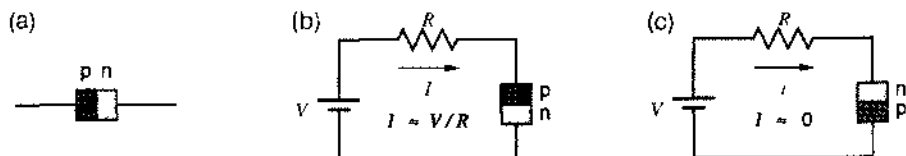


Рис. 3.61. Полупроводниковые диоды: (а) *p-n*-переход; (б) смещенный в прямом направлении переход, допускающий протекание тока, (с) смещенный в обратном направлении переход, препятствующий протеканию тока

Хотя можно создать электровакуумные лампы и другие устройства, ведущие себя как диод, в современных системах применяются полупроводниковые диоды, то есть *p-n*-переходы, которые впредь мы будем иазывать просто *диодами (diode)*. На рис. 3.62(а) показано условное обозначение диода. Как только что было объяснено, в нормальных условиях заметный ток может протекать только в направлении, указанном двумя стрелками: от *анода (anode)* к *катоду (cathode)*. В действительности, диод представляет собой короткое замыкание, пока напряжение на переходе анод–катод неотрицательно. Если напряжение, приложенное между апо-

дом и катодом, отрицательно, то диод ведет себя подобно разомкнутой цепи и ток не течет.

Это правило иллюстрирует вольт-амперная характеристика идеального диода, приведенная на рис. 3.62(б). Если напряжение V между анодом и катодом отрицательно, то говорят, что диод смещен в обратном направлении (*reverse-biased diode*) и ток I через него равен нулю. Если напряжение V не отрицательно, то диод считается смещенным в прямом направлении (*forward-biased diode*) и может протекать сколь угодно большой ток I . Фактически, напряжение V никогда не может стать больше нуля, поскольку идеальный диод, смещенный в прямом направлении, представляет собой короткое замыкание с нулевым сопротивлением.

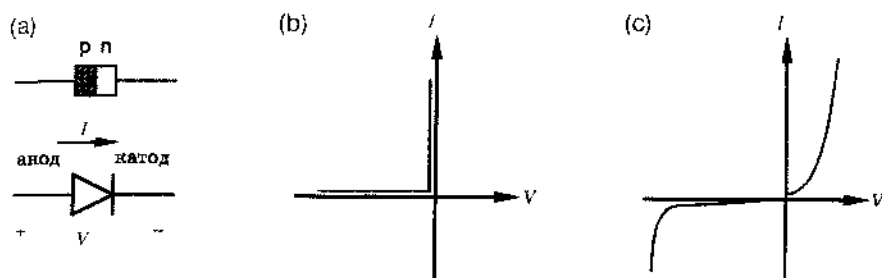


Рис. 3.62. Диоды: (а) условное обозначение; (б) вольт-амперная характеристика идеального диода; (с) вольт-амперная характеристика реального диода

У неидеального, реального диода сопротивление не бесконечно, когда он смещен в обратном направлении, и отлично от нуля при прямом смещении, так что вольт-амперная характеристика выглядит так, как показано на рис. 3.62(с). При прямом смещении диод ведет себя как небольшое нелинейное сопротивление; с увеличением тока на нем падает большее напряжение, но ток и напряжение изменяются не строго пропорционально. Когда диод смещен в обратном направлении, течет небольшой отрицательный ток утечки (*leakage current*). Если отрицательное напряжение становится слишком большим, наступает *пробой диода* (*diode breakdown*), при этом может течь большой отрицательный ток; в большинстве случаев пробой стараются избегать.

СТРЕЛОК, ДЕЙСТВИТЕЛЬНО, ДВЕ

...на рис. 3.62(а). Вторая стрелка входит в условное обозначение диода, напоминая нам о направлении тока. С учетом этого существует много способов запомнить, какой вывод называется анодом, а какой — катодом. Приверженцы высококачественных усилителей на лампах возможно номнят, что электроны движутся от разогретого катода к аноду, и поэтому ток течет от анода к катоду. Те, кто смотрел передачу «Улица Сезам», когда электровакуумные лампы, в основном, уже вышли из моды, возможно предпочитают пользоваться буквами алфавита — электрический ток течет согласно алфавиту от А (anode) к С (cathode). [Спасибо Рикку Кейси (Rick Casey) за подсказку этого мнемонического правила.]

На рис. 3.63(а) и (б) показано, как можно очень просто смоделировать реальный диод. Когда диод смещен в обратном направлении, он подобен разомкнутой цепи; мы пренебрегаем током утечки. При прямом смещении диод ведет себя как резистор с небольшим сопротивлением R_f , включенный последовательно с источником небольшого напряжения V_d . Сопротивление R_f называют *сопротивлением открытого диода* (*forward resistance*), а напряжение V_d — *падением напряжения на открытом диоде* (*diode-drop*).

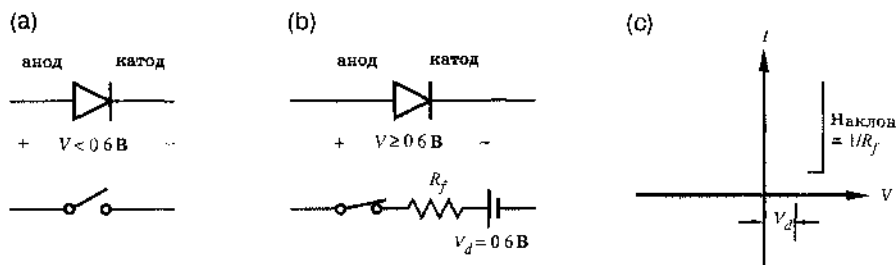


Рис. 3.63. Эквивалентная схема реального диода (а) при обратном смещении, (б) при прямом смещении, (с) вольт-амперная характеристика диода, смещенного в прямом направлении

Правильный выбор значений R_f и V_d позволяет получить приемлемую кусочно-линейную аппроксимацию вольт-амперной характеристики реального диода, показанную на рис. 3.63(с). У типичного маломощного диода типа 1N914 сопротивление R_f равно примерно 25 Ом, а падение напряжения на нем V_d составляет около 0.6 В.

Чтобы «почувствовать» диод, следует помнить, что в действительности реальный диод не содержит источника напряжения 0.6 В, который присутствует в эквивалентной схеме. Верно только то, что из-за нелинейности вольт-амперной характеристики реального диода до тех пор, пока падение напряжения на диоде V не достигнет величины порядка 0.6 В, через него не будет протекать сколько-нибудь значительный ток. Заметьте также, что в типичном случае сопротивление открытого диода, равное 25 Ом, мало по сравнению с другими сопротивлениями в цепи, поэтому после того, как падение напряжения на смещенном в прямом направлении диоде V достигает 0.6 В, дальнейшее увеличение напряжения очень мало. Таким образом, для практических целей можно считать, что падение напряжения на диоде, смещенном в прямом направлении, составляет 0.6 В или близкую к этому значению величину.

СТАБИЛИТРОНЫ

В стабилитронах (*Zener diodes*) используется режим пробоя диодов, в частности, большая крутизна вольт-амперной характеристики в области пробоя. Вместе с резистором, ограничивающим ток пробоя, стабилитрон можно применять для стабилизации напряжения. Для таких применений выпускается множество стабилитронов с различными напряжениями пробоя.

3.9.2. Диодная логика

Свойства диода можно использовать для реализации логических операций. Рассмотрим логическую систему с напряжением питания 5 В и параметрами, указанными в табл. 3.9. В пределах 5-вольтового диапазона напряжения сигнала разбиты на два поддиапазона: нижний (LOW) и верхний (HIGH) с зоной запаса помехоустойчивости между ними шириной 1 В. Напряжение, попадающее в поддиапазон LOW, рассматривается как логический 0, а напряжению, попадающему в поддиапазон HIGH, присписывается значение логической 1.

Табл. 3.9. Логические уровни в простой диодной логической системе

Уровень сигнала	Обозначение	Значение в двоичной системе
0–2 вольт	LOW	0
2–3 вольт	запас помехоустойчивости	не определено
3–5 вольт	HIGH	1

Используя эти определения, можно образовать диодную схему И, как показано на рис. 3.64(а). Предположим, что в этой схеме на оба входа X и Y подан сигнал, имеющий высокий уровень, скажем 4 В, как изображено на рис. 3.64(б). Тогда оба диода смещены в прямом направлении и выходное напряжение V_z больше значения 4 В на величину прямого падения напряжения на диоде, то есть равно приблизительно 4.6 В. От шины питания с напряжением 5 В через эти два диода в источник входного напряжения 4 В текут небольшие токи, величина которых определяется сопротивлением резистора R. Стрелки на рисунке показывают пути, по которым текут эти токи.

Теперь предположим, что напряжение V_x снижается до 1 В, как показано на рис. 3.64(с). В диодной схеме И выходное напряжение равно наименьшему из двух входных напряжений плюс падение напряжения на открытом диоде. Таким образом, напряжение V_z падает до 1.6 В, а диод D2 оказывается смещенным в обратном направлении (напряжение на аноде равно 1.6 В, а на катоде по-прежнему 4 В). Низкий уровень на одном из входов «оннжаст» напряжение на выходе диодной схемы И до низкого уровня. Ясно, что наличие низкого уровня на двух входах также создает низкий уровень на выходе. Сказанное отражено на рис. 3.64(д) в виде таблицы и повторено в терминах двоичных логических значений на рис. 3.64(е); очевидно, что это схема И.

На рис. 3.65 (а) приведена логическая схема с двумя соединенными один за другим вентилями И, а на рис. 3.65(б) – эквивалентная электрическая схема с конкретной комбинацией значений входных сигналов. Этот пример показывает необходимость диодов в схеме И: наличие диода D3 позволяет сигналу на выходе Z первого вентиля И оставаться на высоком уровне, в то время как напряжение на выходе С второго вентиля И снижается до низкого уровня в результате прохождения сигнала со входа В через диод D4.

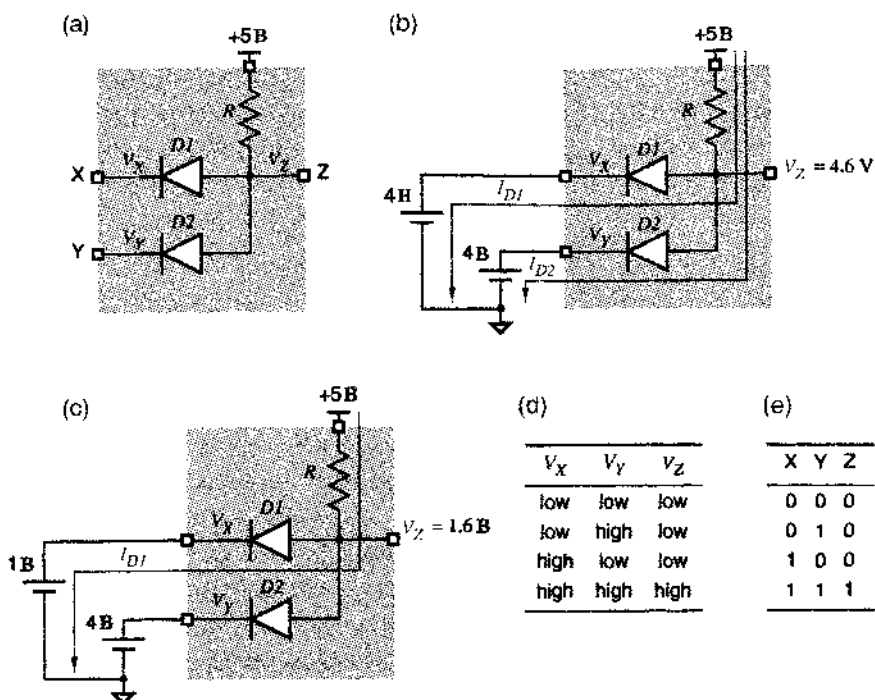


Рис. 3.64. Диодная схема И: (а) электрическая схема; (б) случай, когда сигналы на обоих входах имеют высокий уровень (HIGH); (с) случай, когда на одном из входов имеется высокий уровень (HIGH), а на другом – низкий уровень (LOW); (д) таблица, описывающая работу схемы; (е) таблица истинности

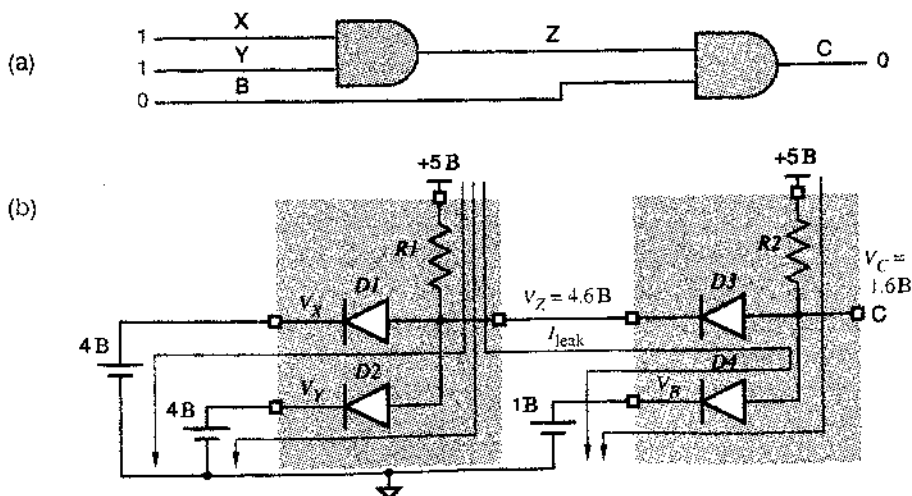


Рис. 3.65. Две схемы И: (а) логическая схема; (б) электрическая схема

Когда диодные вентили соединяются так, как показано на рис. 3.65, уровни логических сигналов отклоняются от напряжения питания и потенциала земли и приближаются к области неопределенности. Таким образом, для нормальной работы за диодным вентилем И должен следовать транзисторный усилитель, восстанавливающий логические уровни; такая схема применяется в ТТЛ-вентиле И-НЕ, рассматриваемых в разделе 3.10.1. Однако иногда для разработчиков логических схем бывает соблазнительным воспользоваться дискретными диодами, чтобы реализовать требуемую логику в том или ином частном случае; см. например, задачу 3.95.

3.9.3. Биполярные транзисторы

Биполярный транзистор (bipolar junction transistor) – это устройство с тремя выводами, которое в большинстве логических схем действует подобно ключу, управляемому током. Если в один из выводов, называемый *базой (base)*, втекает небольшой ток, то ключ «замкнут»: между двумя другими выводами, которые называются *эмиттером (emitter)* и *коллектором (collector)*, может протекать ток. Если в базу ток не втекает, то ключ «разомкнут» и ток между эмиттером и коллектором не течет.

Изучение транзистора мы начнем с рассмотрения двух диодов, соединенных так, как показано на рис. 3.66(a). В этой схеме ток может протекать от точки В к точке С или к точке Е, если соответствующий диод открыт. Однако от точки С к точке Е и в обратном направлении ток протекать не может, так как при любом выборе напряжений в точках В, С и Е один или оба диода будут заперты. На рис. 3.66(b) эти два диода показаны в виде *p-n*-переходов.

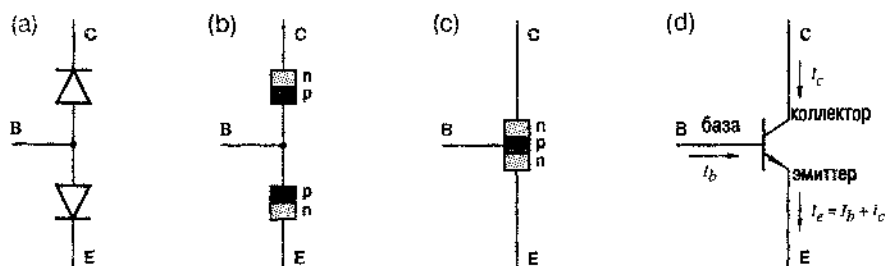


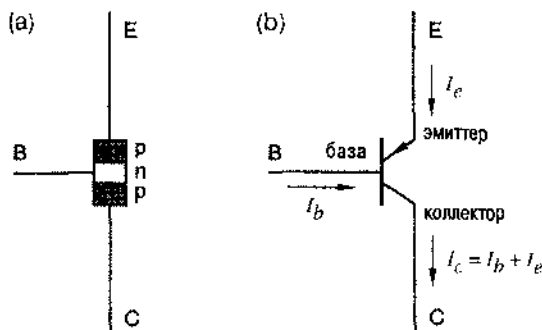
Рис. 3.66. Переход от полупроводниковых диодов к *n-p-n*-транзистору: (a) встречное включение диодов; (b) представление диодов в виде *p-n*-переходов; (c) структура *n-p-n*-транзистора; (d) условное обозначение *n-p-n*-транзистора

Теперь предположим, что диоды, включенные встречно, изготавливаются так, что область полупроводника *p*-типа у них общая, как показано на рис. 3.66(c). Получающаяся структура называется *n-p-n-транзистором (npn transistor)*; она обладает поразительным свойством (но крайней мере, физикам, придумавшим транзисторы еще в 50-е годы, это свойство казалось потрясающим!): если обеспечить протекание тока через *p-n*-переход база-эмиттер, то тогда возможно также протекание тока и через *n-p*-переход коллектор-база (что в нормальных условиях невозможно), так что в результате ток течет от коллектора к эмиттеру.

Условное обозначение $n-p-n$ -транзистора показано на рис. 3.66(d). Обратите внимание, что в условном обозначении транзистора имеется стрелка, указывающая положительное направление тока. Кроме того, это напоминает нам, что переход база-эмиттер является $p-n$ -переходом, таким же, как диод, в условном обозначении которого имеется стрелка, направленная в ту же сторону.

Можно также изготовить $p-n-p$ -транзистор (*pnp transistor*), показанный на рис. 3.67. Однако $p-n-p$ -транзисторы редко используются в цифровых схемах, так что в дальнейшем мы не будем их рассматривать.

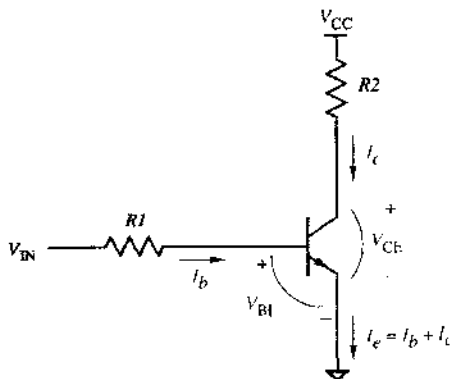
Рис. 3.67. $p-n-p$ -транзистор: (a) структура; (b) условное обозначение



Ток эмиттера I_e , вытекающий из $n-p-n$ -транзистора, равен сумме токов I_b и I_c , втекающих в транзистор со стороны базы и коллектора. Транзистор часто является составной частью *усилителя* (*amplifier*) сигнала, поскольку в пределах некоторого рабочего диапазона (*активная область; active region*) ток коллектора равен произведению тока базы на фиксированную константу ($I_c = \beta \cdot I_b$). Однако в цифровых схемах, как объясняется ниже, транзистор обычно используют в качестве простого ключа, который всегда либо «замкнут», либо «разомкнут».

На рис. 3.68 показано включение $n-p-n$ -транзистора по схеме с *общим эмиттером* (*common-emitter configuration*), которая чаще всего встречается в цифровых схемах. Эта схема помимо одного $n-p-n$ -транзистора содержит два дискретных резистора $R1$ и $R2$. Если напряжение V_{IN} равно 0 или отрицательно, то переход база-эмиттер оказывается запертым, и ток базы (I_b) течь не может. Если скоро отсутствует ток базы, то не может протекать и ток коллектора (I_c); в таком случае говорят, что транзистор находится в состоянии *отсечки* (*cut off*), то есть *закрит* (OFF).

Рис. 3.68. Схема с общим эмиттером на $n-p-n$ -транзисторе



Поскольку переход база-эмиттер является не идеальным, а реальным диодом, прежде чем сможет протечь какой-либо ток базы, напряжение V_{IN} должно достичь, по крайней мере, величины +0.6 В (падение напряжения на открытом диоде). Как только это произойдет, согласно закону Ома получим:

$$I_b = \frac{(V_{IN} - 0.6)}{R_I}.$$

(Обычно пренебрегают малым сопротивлением R_f открытого перехода база-эмиттер по сравнению с сопротивлением базового резистора R_I .) Когда течет ток базы I_b , тогда может течь пропорциональный ему ток коллектора, равный

$$I_c = \beta \cdot I_b$$

Коэффициент пропорциональности β называется *коэффициентом усиления (gain) тока* транзистора; в типичном случае величина β имеет значение порядка 100.

Хотя ток базы I_b определяет ток коллектора I_c , он, помимо этого, косвенно влияет на напряжение V_{CE} между коллектором и эмиттером, так как V_{CE} равно напряжению питания V_{CC} минус падение напряжения на резисторе R_2 :

$$V_{CE} = V_{CC} - I_c \cdot R_2 = V_{CC} - \beta \cdot I_b \cdot R_2 = V_{CC} - \beta \cdot (V_{IN} - 0.6) \cdot R_2 / R_I.$$

Однако в идеальном транзисторе напряжение V_{CE} никогда не может быть меньше нуля (транзистор, конечно, не может быть источником отрицательного потенциала), а в реальном транзисторе V_{CE} никогда не может быть меньше напряжения $V_{CE(sat)}$, являющегося параметром транзистора; величина $V_{CE(sat)}$ обычно бывает порядка 0.2 В.

Если значения V_{IN} , β , R_I и R_2 таковы, что напряжение V_{CE} , вычисленное согласно приведенному выше выражению, меньше чем $V_{CE(sat)}$, то транзистор не может находиться в активной области, и указанное соотношение не применимо. В этом случае транзистор переходит в область насыщения (*saturation region*) или, как говорят, оказывается насыщенным (*saturated*) и полностью открыт (ON). Независимо от того, насколько велик ток I_b , втекающий в базу, напряжение V_{CE} не может стать меньше $V_{CE(sat)}$, поэтому ток коллектора I_c определяется, главным образом, сопротивлением резистора нагрузки R_2 :

$$I_c = (V_{CC} - V_{CE(sat)}) / (R_2 + R_{CE(sat)}),$$

где $R_{CE(sat)}$ — сопротивление транзистора в режиме насыщения (*saturation resistance*). Как правило, сопротивление $R_{CE(sat)}$ не более 50 Ом и пренебрежимо мало по сравнению с R_2 .

Специалистам в области информатики может понравиться представление об n - p - n -транзисторе как об устройстве, которое непрерывно наблюдает за тем, что происходит вокруг, выполняя приведенную в табл. 3.10 программу, моделирующую поведение транзистора (*transistor simulation*).

Табл. 3.10. Программа на языке C, моделирующая поведение n - p - n -транзистора в схеме с общим эмиттером

```

/* Transistor parameters */
#define DIODEDROP 0.6 /* volts */
#define BETA 10;
#define VCE_SAT 0.2 /* volts */
#define RCE_SAT 50 /* ohms */

main()
{
    float Vcc, Vin, R1, R2; /* circuit parameters */
    float Ib, Ic, Vce; /* circuit conditions */

    if (Vin < DIODEDROP) { /* cut off */
        Ib = 0.0;
        Ic = 0.0;
        Vce = Vcc;
    }
    else { /* active or saturated */
        Ib = (Vin - DIODEDROP) / R1;
        if ((Vcc - ((BETA * Ib) * R2)) >= VCE_SAT) { /* active */
            Ic = BETA * Ib;
            Vce = Vcc - (Ic * R2);
        }
        else { /* saturated */
            Vce = VCE_SAT;
            Ic = (Vcc - Vce) / (R2 + RCE_SAT);
        }
    }
}

```

3.9.4. Транзисторный инвертор

На рис. 3.69 показано, что на n - p - n -транзисторе, включенном по схеме с общим эмиттером, можно построить инвертор логических сигналов. Когда входное напряжение имеет низкий уровень, напряжение на выходе имеет высокий уровень и наоборот.

В цифровых схемах воздействие, оказываемое на биполярные транзисторы, часто приводит к тому, что они всегда либо закрыты, либо находятся в насыщении. То есть цифровые схемы типа инвертора, изображенного на рис. 3.69, разработаны так, чтобы транзисторы в них всегда находились (ну, почти всегда) в одном из состояний, изображенных на рис. 3.70. Если входное напряжение V_{IN} соответствует низкому уровню, то оно настолько мало, что ток I_b равен нулю и транзистор закрыт; цепь между эмиттером и коллектором разомкнута. Если напряжение V_{IN} соответствует высокому уровню, то оно настолько велико (а сопротивление $R1$ достаточно мало и коэффициент β достаточно велик), что транзистор попадет в состояние насыщения при любом разумном значении сопротивления $R2$; цепь между коллектором и эмиттером выглядит почти как короткое замыкание. Наличие входных напряжений, попадающих в область неопределенности между низ-

ким и высоким уровнями, не допустимо, за исключением переходных процессов. Эта область неопределенности соответствует запасу помехоустойчивости, о котором шла речь в связи с табл. 3.9.

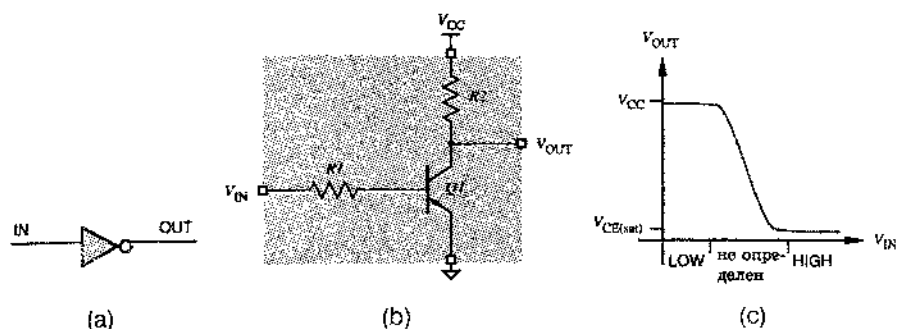


Рис. 3.89. Транзисторный инвертор: (а) условное обозначение (IN – вход, OUT – выход); (б) принципиальная схема; (с) передаточная характеристика (LOW – низкий уровень, HIGH – высокий уровень)

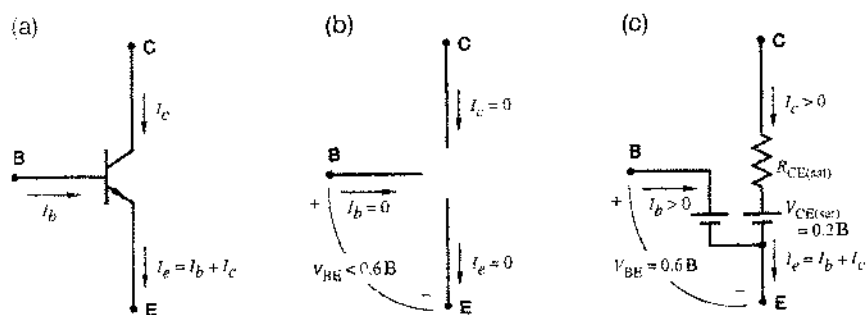


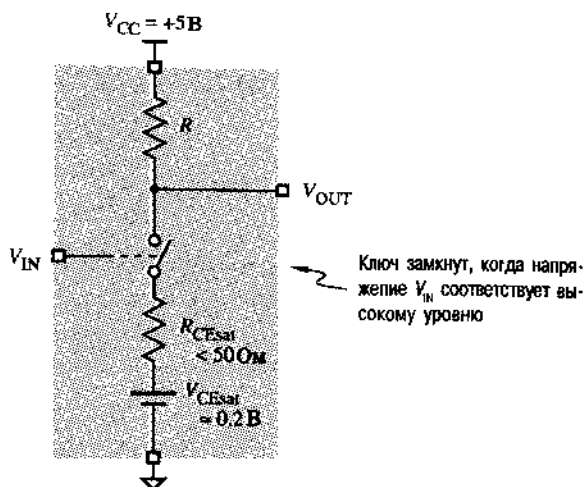
Рис. 3.70. Обычные состояния $n-p-n$ -транзистора в цифровой схеме: (а) условное обозначение транзистора и протекающие в нем токи; (б) эквивалентная схема транзистора в состоянии отсечки (OFF); (с) эквивалентная схема транзистора в состоянии насыщения (ON)

Другой способ, позволяющий наглядно представить работу транзисторного инвертора, показан на рис. 3.71. Если напряжение V_{IN} соответствует высокому уровню, то транзисторный ключ замкнут, выход инвертора соединен с землей и выходное напряжение безусловно соответствует низкому уровню. Если входное напряжение V_{IN} соответствует низкому уровню, то транзисторный ключ разомкнут и выход инвертора через резистор подключен к шине питания +5 В; выходное напряжение соответствует высокому уровню, если выход не слишком сильно нагружен (то есть в том случае, когда он не соединен с землей резистором с малым сопротивлением, что было бы неправильно).

3.9.5. Транзисторы Шоттки

Когда напряжение на входе насыщенного транзистора изменяется, выходное напряжение не начинает изменяться немедленно; для выхода из насыщения требует-

Рис. 3.71. Эквивалентная схема транзисторного инвертора в виде ключа

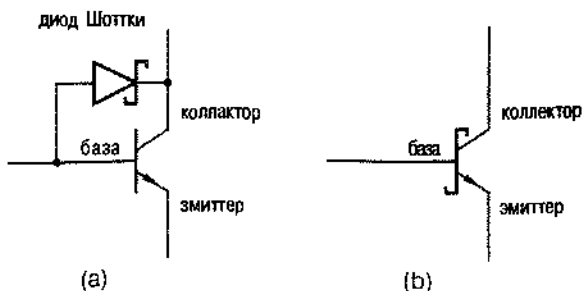


ся дополнительное время, называемое *временем удержания (storage time)*. В первом семействе ТТЛ фактически время удержания составляло значительную долю задержки распространения.

Время удержания можно исключить и тем самым уменьшить задержку распространения, обеспечивая условия, при которых транзисторы в нормальном режиме не попадают в область насыщения. В современных семействах ТТЛ это достигается включением диода Шоттки (Schottky diode) между базой и коллектором каждого транзистора, который может оказаться в насыщении, как показано на рис. 3.72. Получающиеся транзисторы, не попадающие в область насыщения, называются *транзисторами с фиксирующими диодами Шоттки (Schottky-clamped transistors)* или короче *транзисторами Шоттки (Schottky transistors)*.

Падение напряжения на диоде Шоттки при смещении в прямом направлении существенно меньше, чем у обычного диода, и составляет 0.25 В против 0.6 В. У обычного транзистора напряжение между базой и коллектором в режиме насыщения равно 0.4 В, как показано на рис. 3.73(а). В транзисторе Шоттки благодаря применению диода Шоттки ток в базовой цепи разветвляется, и часть этого тока отводится в коллектор прежде, чем транзистор входит в режим насыщения, как это показано на рис. 3.73(б). На рис. 3.74 представлена принципиальная схема простого инвертора с транзистором Шоттки.

Рис. 3.72. Транзистор Шоттки: (а) схема; (б) условное обозначение



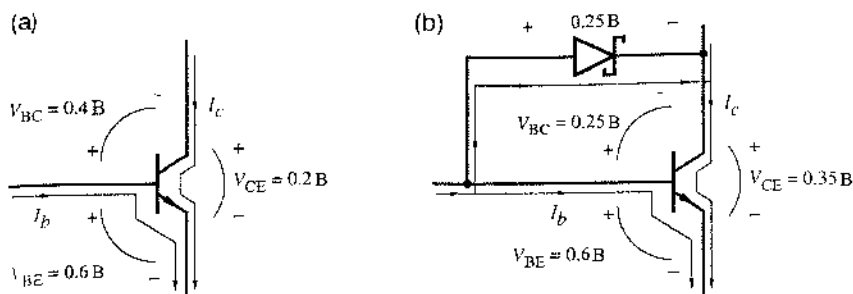


Рис. 3.73. Работа транзистора при большом токе в цепи базы: (а) обычный транзистор в режиме насыщения; (б) транзистор с диодом Шоттки, предотвращающим насыщение

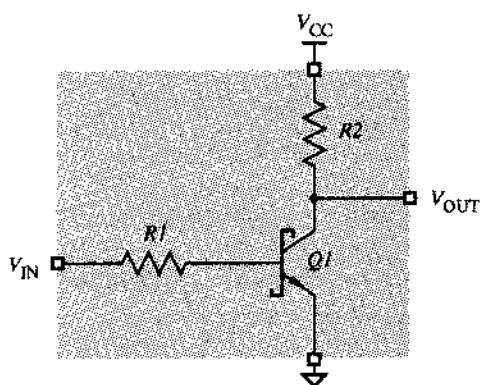


Рис. 3.74. Инвертор с транзистором Шоттки

3.10. Транзисторно-транзисторная логика

Наиболее часто применяемыми биполярными логическими схемами являются схемы транзисторно-транзисторной логики (ТТЛ). В действительности, существует много различных семейств ТТЛ, различающихся быстродействием, потребляемой мощностью и другими характеристиками. В качестве примера в этом параграфе выбран типичный представитель ТТЛ-семейств – схемы с диодами Шоттки и малой потребляемой мощностью серии LS (семейство LS-TTL).

У схем ТТЛ логические уровни, в основном, те же самые, что и у ТТЛ-совместимых КМОП-схем, о которых шла речь в предыдущих параграфах. При рассмотрении поведения ТТЛ-схем мы воспользуемся следующими определениями низкого и высокого уровней:

низкий уровень (LOW)	0 – 0.8 вольт,
высокий уровень (HIGH)	2.0 – 5.0 вольт.

3.10.1. Базовый ТТЛ-вентиль И-НЕ

На рис. 3.75 приведена принципиальная схема двухвходового ТТЛ-вентиль И-НЕ 74LS00 серии LS. Функция И-НЕ реализуется путем объединения диодной схемы И с инвертирующим буферным усилителем. Чтобы лучше понять работу схемы, разделим ее на три части, как показано на рисунке:

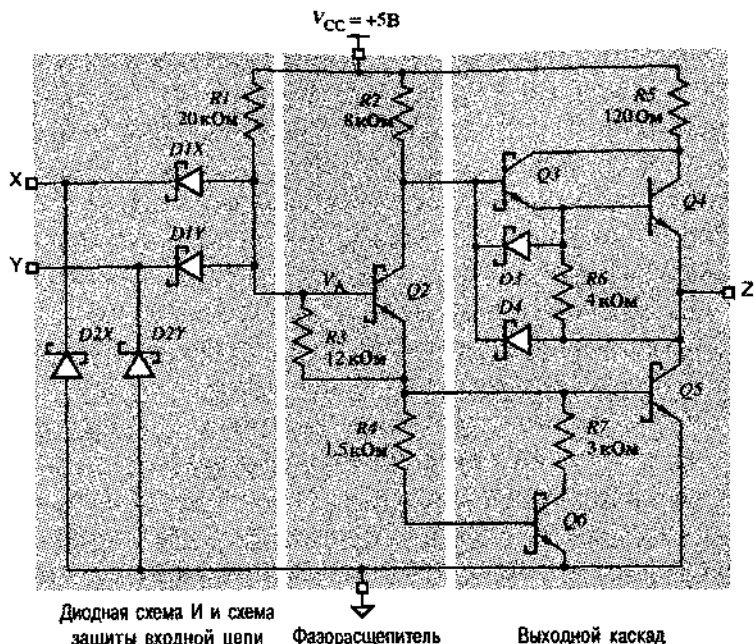


Рис. 3.75. Принципиальная схема двухвходового ТТЛ-вентиля И-НЕ серии LS

- диодная схема И и схема защиты входной цепи,
- фазорасщепитель,
- выходной каскад,

и рассмотрим их работу порознь.

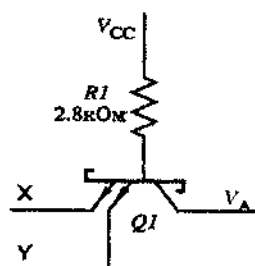
Диоды $D1X$, $D1Y$ и резистор $R1$ на рис. 3.75 образуют точно такую же диодную схему И (diode AND gate), что и в разделе 3.9.2. Фиксирующие диоды (clamp diodes) $D2X$ и $D2Y$ при нормальной работе не оказывают никакого влияния, но ограничивают нежелательные отрицательные выбросы на входах величиной, равной падению напряжения на открытом диоде. Такие отрицательные выбросы могут появляться при изменении входного напряжения от высокого уровня до низкого в результате эффектов, возникающих в линиях передачи, которые рассматриваются в параграфе 11.4.

Транзистор $Q2$ и связанные с ним резисторы образуют фазорасщепитель (phase splitter), сигналы с выходов которого поступают на выходной каскад. В зависимости от того, низкий или высокий уровень напряжения V_A на выходе диодной схемы И, транзистор $Q2$ закрыт или открыт.

Выходной каскад (output stage) содержит два транзистора $Q4$ и $Q5$, один из которых в любой момент времени открыт, а другой закрыт. Выходной каскад ТТЛ-схемы иногда называют двухтактным выходным каскадом (totem-pole output или push-pull output). Подобно транзисторам с p -каналом и n -каналом в КМОП-схемах, транзисторы $Q4$ и $Q5$ осуществляют подтягивание выходного напряжения к напряжению питания или к потенциалу земли, обеспечивая на выходе сигналы высокого и низкого уровня соответственно.

ГДЕЖЕ ТРАНЗИСТОР Q1?

Обратите внимание, что в схеме на рис. 3.75 нет транзистора Q1, а другие транзисторы обозначены традиционным образом; в некоторых ТТЛ-схемах реально присутствует транзистор, обозначаемый Q1. В этих схемах для реализации логики вместо диодов D1X и D1Y применяется многоэмиттерный транзистор Q1. В этом транзисторе на каждый логический вход приходится один эмиттер, как показано на рисунке справа. Достаточно хотя бы на одном из эмиттеров иметь низкий уровень, чтобы транзистор был открыт, и в этом случае напряжение V_A будет соответствовать низкому уровню.



Функционирование ТТЛ-вентилей И-НЕ отражено на рис. 3.76(a). Вентиль действительно реализует функцию И-НЕ. На рис. 3.76(b) и (c) приведены таблица истинности и условное обозначение этой схемы. Путем простого изменения числа диодов в диодной схеме И, можно создать ТТЛ-вентиль И-НЕ с любым желаемым числом входов. Имеющиеся в продаже ТТЛ-вентили И-НЕ имеют до 13 входов. ТТЛ-инвертор, представляет собой 1-входовый элемент И-НЕ, у которого отсутствуют указанные на рис. 3.75 диоды D1Y и D2Y.

(a)	X	Y	V_A	Q2	Q3	Q4	Q5	Q6	V_Z	Z
	L	L	≤ 1.05	off	on	on	off	off	2.7	H
	L	H	≤ 1.05	off	on	on	off	off	2.7	H
	H	L	≤ 1.05	off	on	on	off	off	2.7	H
	H	H	1.2	on	off	off	on	on	≤ 0.35	L

(b)	X	Y	Z
	0	0	1
	0	1	1
	1	0	1
	1	1	0

(c)	X	Y	Z
	0	0	1
	0	1	1
	1	0	1
	1	1	0



Рис. 3.76. Функционирование ТТЛ-вентилей И-НЕ с двумя входами: (a) таблица, описывающая работу схемы [L – низкий уровень (LOW), H – высокий уровень (HIGH); on – открыт, off – закрыт]; (b) таблица истинности; (c) условное обозначение

Поскольку обычно выходные транзисторы Q4 и Q5 находятся в противоположных состояниях и один из них открыт (ON), а другой закрыт (OFF), у вас может возникнуть вопрос о роли резистора R5 с сопротивлением 120 Ом в выходном каскаде. Ведь отсутствие этого резистора позволило бы схеме отдавать больший ток при высоком уровне на выходе. Это, конечно, верно с точки зрения постоянно-

го тока. Однако при изменении сигнала на выходе ТТЛ-схемы от высокого уровня до низкого или наоборот имеется короткий интервал времени, когда оба транзистора могут быть открыты. Резистор $R5$ служит для ограничения тока, протекающего в это время от шины питания V_{CC} к земле. Когда происходит переключение выходного каскада ТТЛ-схемы, то даже при наличии резистора с сопротивлением 120 Ом наблюдаются так называемые броски тока, в пределах которых величина тока превышает нормальные значения. Это явление подобно броскам тока, появляющимся при переключении быстродействующих КМОП-схем.

До сих пор мы считали, что сигналы поступают на вход ТТЛ-вентиля от импульсного источника напряжения. На рис. 3.77 показана ситуация, когда на вход ТТЛ-схемы поступает сигнал низкого уровня с выхода другой ТТЛ-схемы. Транзистор $Q5A$ в левой схеме, являющейся источником сигнала, открыт (ON), и по нему течет ток, вытекающий через диод $D1XB$ из правой схемы, на вход которой подан данный сигнал. Если ток течет в выходную цепь ТТЛ-схемы при низком уровне сигнала на выходе, как это имеет место в нашем случае, то говорят, что *ток втекает (sinking current)* в эту схему со стороны ее выхода.

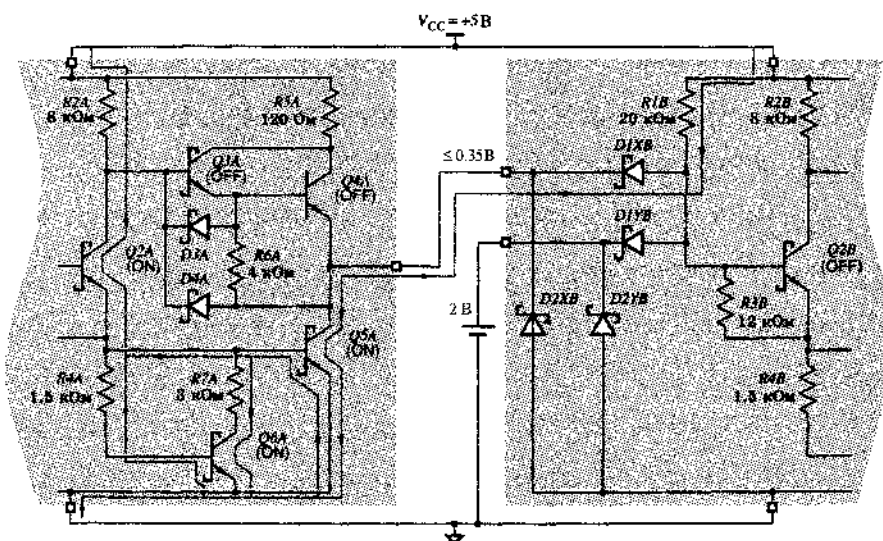


Рис. 3.77. ТТЛ-вентиль с низким уровнем сигнала на выходе, нагруженный входом другого ТТЛ-вентиля (ON – открыт, OFF – закрыт)

СНОВА БРОСКИ ТОКА

Броски тока могут проявляться в ТТЛ- и КМОП-схемах в виде помехи на шине питания и земляной шине, особенно в тех случаях, когда происходит одномоментное переключение на нескольких выходах. По этой причине для надежной работы схемы требуются развязывающие конденсаторы, включаемые между шиной питания V_{CC} и землей. Эти конденсаторы должны быть распределены по всей схеме, по крайней мере, по одному в пределах дюйма или у каждой микросхемы, чтобы служить источниками тока во время переходов.

На рис. 3.78 показан тот же случай с высоким уровнем сигнала, подаваемого с выхода одной схемы на вход другой. Теперь в схеме, служащей источником сигнала, транзистор $Q4A$ открыт и по нему течет небольшой ток утечки смещенных в обратном направлении диодов $D1XB$ и $D2XB$ в схеме, на вход которой подан данный сигнал. Когда ток течет из ТТЛ-схемы со стороны ее выхода при высоком уровне сигнала, этот ток называют *вытекающим* (*sourcing current*).

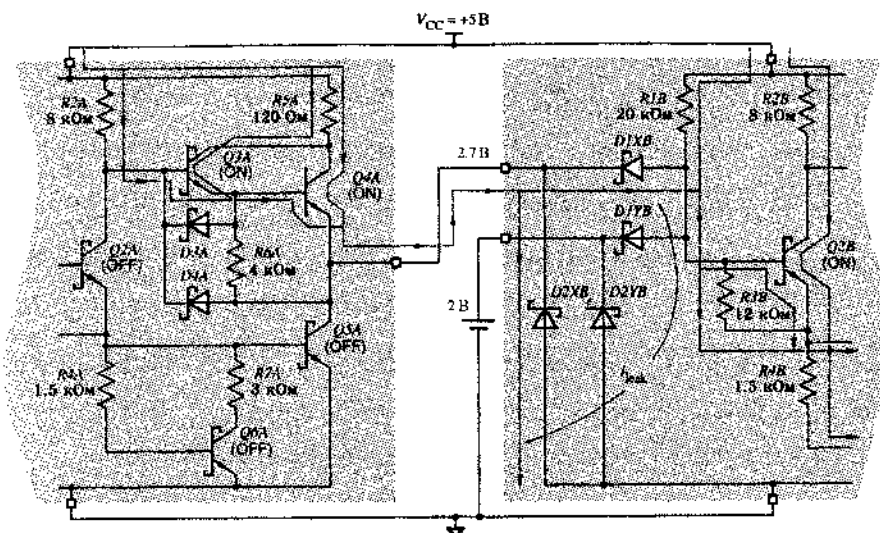


Рис. 3.78. ТТЛ-вентиль с высоким уровнем сигнала на выходе, нагруженный входом другого ТТЛ-вентиля (ON – открыт, OFF – закрыт)

3.10.2. Логические уровни и запас помехоустойчивости

В начале этого параграфа мы условились, что применительно к ТТЛ-схемам речь идет о сигналах низкого уровня при напряжениях между 0 В и 0.8 В и о сигналах высокого уровня при напряжениях между 2.0 В и 5.0 В. В действительности, уровни входных и выходных ТТЛ-сигналов можно задать более точно так же, как это было сделано для сигналов КМОП-схем:

- V_{OHmin} – минимальное выходное напряжение высокого уровня (HIGH), равное 2.7 В для большинства ТТЛ-семейств;
- V_{IHmin} – минимальное входное напряжение, гарантированно воспринимаемое как высокий уровень (HIGH); для всех ТТЛ-семейств оно равно 2.0 В.
- V_{ILmax} – максимальное входное напряжение, гарантированно воспринимаемое как низкий уровень (LOW), равно 0.8 В для большинства ТТЛ-семейств
- V_{OLmax} – максимальное выходное напряжение низкого уровня (LOW), равное 0.5 В для большинства ТТЛ-семейств.

На рис. 3.79 показаны границы уровней и запас помехоустойчивости.

Согласно техническим условиям, у большинства ТТЛ-схем напряжение V_{OHmin} больше напряжения V_{IHmin} на 0.7 В, так что при высоком уровне *запас помехоустойчивости*

стойчивости по постоянному току (*DC noise margin*) ТТЛ-схем составляет 0.7 В. Это означает, что в наихудшем случае выходной высокий уровень не будет надежно восприниматься на входе как высокий уровень, когда помеха, но меньшей мере, равна 0.7 В. Однако при низком уровне напряжение V_{ILmax} превышает напряжение V_{OLmax} только на 0.3 В, так что запас помехоустойчивости по постоянному току в этом случае составляет всего лишь 0.3 В. Таким образом, ТТЛ-схемы и ТТЛ-совместимые схемы более чувствительны к шуму при низком уровне сигнала, чем при высоком.

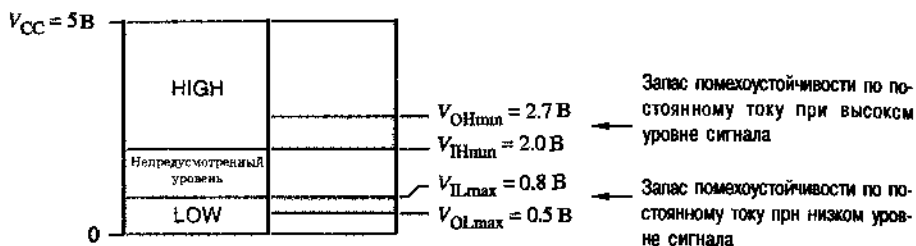


Рис. 3.79. Границы уровней и запас помехоустойчивости популярных ТТЛ-схем (семейств 74LS, 74S, 74ALS, 74AS, 74F)

3.10.3. Коэффициент разветвления по выходу

Как было сказано ранее в разделе 3.5.4, коэффициент разветвления по выходу (*fano*), по определению, равен числу входов логических схем, которые могут быть подключены к одному выходу. Как было показано там же, у КМОП-схем, нагруженных входами КМОП-схем, коэффициент разветвления по выходу по постоянному току по существу не ограничен, потому что входы КМОП-схем практически не потребляют ток как при низком, так и при высоком уровне сигнала. С входами ТТЛ-схем дело обстоит иначе. В результате для ТТЛ- и КМОП-схем имеются предельные значения коэффициентов разветвления по выходу в случае их нагрузки входами ТТЛ-схем, о чем сейчас и пойдет речь.

Как и в случае КМОП-схем, ток, текущий во входной или выходной вывод ТТЛ-схемы считается положительным, если он действительно *втекает* в схему по этому выводу, и отрицательным, если *вытекает* из схемы по этому выводу. В результате, когда выход соединен с одним или большим числом входов, алгебраическая сумма всех входных токов и выходного тока равна 0.

Величина тока, протекающего через ТТЛ-вход, зависит от того, каков уровень входного напряжения — высокий или низкий, — и определяется двумя параметрами:

I_{ILmax} — максимальное значение входного тока, необходимого для получения на входе низкого уровня; вспомним схему на рис. 3.77: ток действительно течет от шины питания V_{CC} по резистору R_{IB} , через диод D_{1XB} , во вход и через выходной транзистор Q_{5A} схемы, служившей источником сигнала, на землю; так как при низком уровне ток вытека-

ет из ТТЛ-входа, величина I_{ILmax} отрицательна; у большинства входов ТТЛ-схем LS-серии $I_{ILmax} = -0.4$ мА; это значение иногда называют *единичной нагрузкой при низком уровне (LOW-state unit load)* ТТЛ-схем серии LS;

I_{IHmax} — максимальное значение тока, необходимого для того, чтобы сигнал на входе был сигналом высокого уровня; как показано на рис. 3.78, ток течет от шины питания V_{CC} по резистору R_{5A} и транзистору Q_{4A} схемы, служащей источником сигнала, и *втекает* во входную цепь схемы, подключенной к данному выходу, где течет на землю через смещенные в обратном направлении диоды D_{1XB} и D_{2XB} ; так как при высоком уровне на входе ток *втекает* по входному выводу ТТЛ-схемы, величина I_{IHmax} положительна; у большинства ТТЛ-схем серии LS $I_{IHmax} = 20$ мкА, и это значение иногда называют *единичной нагрузкой при высоком уровне (HIGH-state unit load)*.

Подобно выходам КМОП-схем, ТТЛ-схема со стороны сс выхода может быть источником тока или приемником тока некоторой величины в зависимости от того, каков уровень сигнала на выходе — высокий или низкий:

I_{OLmax} — максимальное значение тока, который может потреблять выходная цепь схемы при низком уровне выходного напряжения и при условии, что это напряжение не превосходит V_{OLmax} ; поскольку ток втекает в выходную цепь, величина I_{OLmax} положительна и ее значение для большинства ТТЛ-схем серии LS равно 8 мА;

I_{OHmax} — максимальное значение тока, который может выдать выходная цепь схемы при высоком уровне выходного напряжения и при условии, что это напряжение не менее V_{OHmin} ; так как ток вытекает из схемы, величина I_{OHmax} отрицательна и ее значение для большинства ТТЛ-схем серии LS равно -400 мкА.

Обратите внимание, что значение I_{OLmax} для типичных ТТЛ-схем серии LS равно в 20 раз больше абсолютного значения I_{ILmax} . Поэтому говорят, что у ТТЛ-схем серии LS *коэффициент разветвления по выходу при низком уровне* выходного напряжения (*LOW-state fanout*) равен 20, поскольку при низком уровне сигнала выход можно нагрузить 20 входами. Точно так же абсолютное значение I_{OHmax} равно в 20 раз больше значения I_{IHmax} , так что считается, что *коэффициент разветвления по выходу* ТТЛ-схем серии LS *при высоком уровне* выходного напряжения (*HIGH-state fanout*) также равен 20. *Результирующий коэффициент разветвления по выходу (overall fanout)* равен меньшему из коэффициентов разветвления при низком и при высоком уровне выходного напряжения.

Если нагрузка ТТЛ-схемы превышает номинальный коэффициент разветвления по выходу, то возможны те же опасные последствия, какие были отмечены в отношении КМОП-устройств в разделе 3.5.5, а именно может сократиться или вовсе отсутствовать запас надежности по постоянному току, могут увеличиться время переключения процесса и задержка, а схема может перегреться.

АСИММЕТРИЯ ВЫХОДА ТТЛ-СХЕМ

Хотя коэффициент разветвления по выходу у ТТЛ-схем серии LS при высоком и при низком уровне выходного напряжения одинаков, семейство LS-TTL и другие ТТЛ-семейства имеют четко выраженную асимметрию, проявляющуюся в разных значениях втекающего и вытекающего токов: при низком уровне в выходную цепь ТТЛ-схемы серии LS может втекать ток 8 мА, а при высоком уровне из схемы может вытекать только ток, не превышающий 400 мкА.

Из-за этой асимметрии не возникает никаких проблем, когда сигнал с выхода данной ТТЛ-схемы подается на входы других ТТЛ-схем, поскольку асимметричными являются требования в отношении входного тока ТТЛ-схем (ток I_{ILmax} велик, в то время как ток I_{IHmax} мал). Однако эта асимметрия становится ограничением, когда ТТЛ-схемы используются для включения и выключения светодиодов, реле, соленоидов или других устройств, которым требуются большие токи, составляющие часто десятки миллиампер. Схемы, в которых применяются такие устройства следует разрабатывать так, чтобы ток через управляемое устройство протекал (и оно при этом было «включено»), когда напряжение на выходе ТТЛ-схемы имеет низкий уровень, а при высоком уровне ток либо не протекал, либо был мал. Специальные ТТЛ-схемы, предназначенные для того, чтобы выполнять функции буфера или драйвера, устроены так, что при низком уровне напряжения на выходе втекающий ток может достигать 60 мА, но вытекающий ток при высоком уровне напряжения все же довольно мал (2.4 мА).

ОБОЖЖЕННЫЕ ПАЛЬЦЫ

Если втекающий ток ТТЛ- или КМОП-схемы много больше, чем I_{OLmax} , то устройство может быть повреждено, особенно если такой ток течет дольше секунды. Предположим, например, что выход ТТЛ-схемы с низким уровнем напряжения замкнут накоротко с шиной питания 5 В. Сопротивление $R_{CE(sat)}$ открытого и находящегося в режиме насыщения транзистора Q5 в типичном выходном каскаде ТТЛ-схемы не больше 10 Ом. Таким образом, на транзисторе Q5 должна рассеиваться мощность, равная примерно $5^2/10$ или 2.5 ватта. Не пытайтесь убедиться в этом, если вы не готовы к последствиям! Выделяющегося тепла достаточно, чтобы за очень короткое время испортить устройство (и обжечь ваш палец).

В общем случае для того, чтобы убедиться, что схема не перегружена по выходу, необходимо проверить выполнение следующих двух условий:

при высоком уровне напряжения на выходе сумма значений I_{IHmax} для всех входов, подключенных к данному выходу, не должна превышать абсолютного значения тока I_{OHmax} для схемы, являющейся источником сигнала;

при низком уровне напряжения на выходе сумма значений $I_{L\max}$ для всех входов, подключенных к данному выходу, не должна превышать абсолютного значения тока $I_{OL\max}$ для схемы, являющейся источником сигнала.

Предположим, например, что вы спроектировали систему, в которой к выходу какой-то ТТЛ-схемы серии LS подключены десять входов ТТЛ-схем серии LS и три входа ТТЛ-схем серии S. При высоком уровне напряжения на выходе данной схемы потребуется суммарный ток, равный $10 \cdot 20 \text{ мкА} + 3 \cdot 50 \text{ мкА} = 350 \text{ мкА}$. Эта величина находится в пределах нагрузочной способности ТТЛ-схемы серии LS при высоком уровне, поскольку максимальный ток, отдаваемый схемой, равен 400 мкА. Но при низком уровне напряжения на выходе данной схемы требуется суммарный ток $10 \cdot 0.4 \text{ мА} + 3 \cdot 2.0 \text{ мА} = 10.0 \text{ мА}$. Это больше, чем максимальный ток, втекающий в ТТЛ-схему серии LS со стороны ее выхода при низком уровне, равный 8 мА, так что схема перегружена.

3.10.4. Неиспользуемые входы

С неиспользуемыми входами ТТЛ-схем можно поступить так же, как с входами КМОП-схем (см. раздел 3.5.6), а именно: неиспользуемые входы можно соединить с используемым входом, либо подать на них напряжение, соответствующее высокому или низкому уровню, в зависимости от реализуемой логической функции.

Сопротивление резистора, с помощью которого вход соединяется с шиной питания или с землей, у ТТЛ-схем более критично, чем у КМОП-схем, потому что входные токи ТТЛ-схем существенно больше, особенно при низком уровне напряжения на входе. Если сопротивление резистора слишком велико, то падение напряжения на нем может привести к тому, что потенциал входа окажется вне стандартных диапазонов, соответствующих низкому и высокому уровням.

Рассмотрим, например, случай, когда резистор соединяет входы с землей, как показано на рис. 3.80. Через этот резистор от каждого из подключенных к нему неиспользуемых входов ТТЛ-схем серии LS должен протекать ток 0.4 мА. Несмотря на это, падение напряжения на резисторе не должно превышать 0.5 В, чтобы входное напряжение, соответствовало низкому уровню и было не больше напряжения, поступающего с выхода нормальной схемы. Если к резистору подключено n входов ТТЛ-схем серии LS, то должно выполняться неравенство:

$$n \cdot 0.4 \text{ мА} \cdot R_{pd} < 0.5 \text{ В}.$$

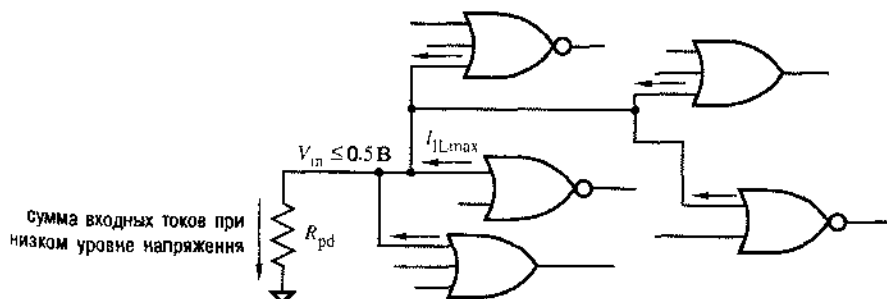


Рис. 3.80. Резистор, соединяющий входы ТТЛ-схем с землей

ПЛАВАЮЩИЕ ВХОДЫ ТТЛ

Рассмотрение входных цепей ТТЛ-схем показывает, что неиспользуемые входы, оставленные не подключенными (или *плавающими*), ведут себя так, как будто на них подано напряжение, соответствующее высокому уровню: потенциал такого входа подтягивается до высокого уровня благодаря базовому резистору R_I (см. рис. 3.75). Однако резистор R_I осуществляет такое подтягивание менее эффективно, чем это происходит в случае, когда данный вход подключен к выходу какой-либо ТТЛ-схемы. В результате небольшой шум в цепи, создаваемый другими схемами при переключении, может оказаться достаточным, чтобы потенциал плавающего входа был ошибочно воспринят как низкий уровень на входе. Поэтому, ради надежности, неиспользуемые входы ТТЛ-схем следует подключать к стабильному источнику напряжения, соответствующего высокому или низкому уровню.

Таким образом, если резистор должен обеспечить низкий уровень для 10 входов ТТЛ-схем серии LS, то его сопротивление должно удовлетворять неравенству: $R_{pd} < 0.5 / (10 \cdot 4 \cdot 10^{-4})$ или $R_{pd} < 125 \text{ Ом}$.

Точно так же оценивается сопротивление резистора, который соединяет входы с шиной питания, как показано на рис. 3.81. Этот резистор должен обеспечить протекание тока 20 мкА в каждом из неиспользуемых входов, причем входное напряжение должно быть не меньше 2.7 В, то есть не меньше напряжения высокого уровня, поступающего с выхода нормальной схемы. Поэтому, падение напряжения на этом резисторе не должно превышать 2.3 В; если к резистору подключено n входов ТТЛ-схем серии LS, то должно выполняться неравенство:

$$n \cdot 20 \text{ мкА} \cdot R_{pu} < 2.3 \text{ В}$$

Таким образом, если к резистору R_{pu} подключены 10 входов ТТЛ-схем серии LS, то $R_{pu} < 2.3 / (10 \cdot 20 \cdot 10^{-6})$ или $R_{pu} < 11.5 \text{ кОм}$.

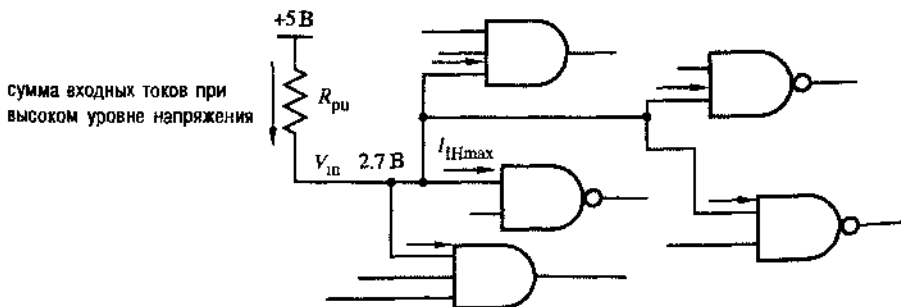


Рис. 3.81. Резистор, соединяющий входы ТТЛ-схем с шиной питания

ПОЧЕМУ ПРИМЕНЯЕТСЯ РЕЗИСТОР?

Вы можете спросить: «Почему для подключения входов к шине питания или к земле применяется резистор, если непосредственное соединение с шиной питания или с землей обеспечивает хороший высокий или низкий уровень?»

Ну, непосредственное подключение к шине питания 5 В с целью поддержания на входе высокого уровня не рекомендуется потому, что возможное повышение напряжения на входе вследствие переходного процесса до значений, превосходящих 5.5 В, может повредить некоторые из ТТЛ-схем, в частности те, у которых во входной цепи включен многоэмиттерный транзистор. В этом случае резистор ограничивает ток и предотвращает повреждение схемы.

Низкий уровень в большинстве случаев крайне достигается путем непосредственного соединения входа с землей, без включения резистора. Повсюду в этой книге вы увидите много примеров такого рода включения. Однако в некоторых случаях резистор все же желателен для того, чтобы можно было преодолеть «постоянный» низкий уровень, обеспечиваемый этим резистором, подавая высокий уровень для целей проверки системы (см. раздел 11.2.2).

3.10.5. ТТЛ-схемы других типов

Вентиль И-НЕ является базовым элементом ТТЛ-семейства, но по той же самой общей схеме можно построить вентили и других типов.

На рис. 3.82 показана принципиальная схема ИЛИ-НЕ ТТЛ-семейства серии LS. Если на какой-либо из входов X или Y подать сигнал высокого уровня, то соответствующий транзистор фазорасщепителя $Q2X$ или $Q2Y$ будет открыт, вследствие чего окажутся закрытыми транзисторы $Q3$ и $Q4$, а транзисторы $Q5$ и $Q6$ — открытыми, так что на выходе установится низкий уровень. Если на обоих входах имеется сигнал низкого уровня, то оба транзистора фазорасщепителя закрыты, что приводит к появлению на выходе высокого уровня. Работа этой схемы отражена в таблице, приведенной на рис. 3.83(а).

Входные цепи, фазорасщепитель и выходной каскад ТТЛ-схемы ИЛИ-НЕ серии LS почти совпадают с аналогичными элементами ТТЛ-схемы И-НЕ этой серии. Отличие состоит в том, что в схеме И-НЕ семейства LS-TTL для реализации функции И применяются диоды, в то время как в схеме ИЛИ-НЕ этого семейства функция ИЛИ реализуется в фазорасщепителе параллельно включенными транзисторами.

Входные и выходные параметры ТТЛ-схемы ИЛИ-НЕ, а также ее быстродействие сравнимы с аналогичными характеристиками ТТЛ-схемы И-НЕ. Однако в n -входовом вентиле ИЛИ-НЕ используется большее число транзисторов и резисторов, чем в n -входовом вентиле И-НЕ, и поэтому они дороже из-за большей площади, занимаемой ими на поверхности кристалла кремния. Кроме того, внутренний ток утечки ограничивает число транзисторов $Q2$, которые можно включить парал-

тельно, поэтому схемы ИЛИ-НЕ имеют меньший коэффициент объединения по входу. (Наибольший коэффициент объединения по входу ТТЛ-микросхем ИЛИ-НЕ равен всего лишь 5, в то время как микросхемы И-НЕ могут иметь 13 входов.) Вследствие этого в устройствах, создаваемых на основе ТТЛ-схем, вентили ИЛИ-НЕ применяются реже, чем вентили И-НЕ.

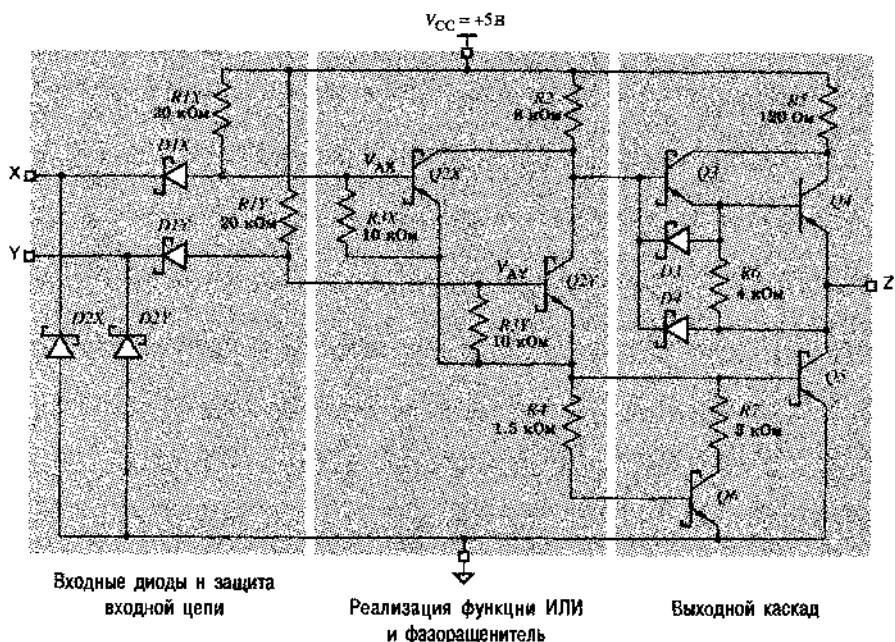


Рис. 3.82. Принципиальная схема двухвходового ТТЛ-вентиля ИЛИ-НЕ серии LS

(a)	X	Y	V_{AX}	Q2X	V_{AY}	Q2Y	Q3	Q4	Q5	Q6	V_Z	Z
	L	L	≤ 1.05	off	≤ 1.05	off	on	on	off	off	≥ 2.7	H
	L	H	≤ 1.05	off	1.2	on	off	off	on	on	≤ 0.35	L
	H	L	1.2	on	≤ 1.05	off	off	off	on	on	≤ 0.35	L
	H	H	1.2	on	1.2	on	off	off	on	on	≤ 0.35	L

(b)	X	Y	Z
	0	0	1
	0	1	0
	1	0	0
	1	1	0

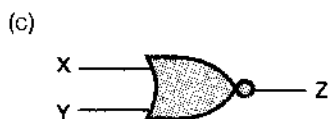


Рис. 3.83. Двухвходовой ТТЛ-вентиль ИЛИ-НЕ серии LS: (a) таблица, описывающая работу схемы [L – низкий уровень (LOW), H – высокий уровень (HIGH); on – открыт, off – закрыт]; (b) таблица истинности; (c) условное обозначение

Для ТТЛ-схем наиболее «естественными» являются инвертирующие вентили, такие как И-НЕ и ИЛИ-НЕ. Неинвертирующие ТТЛ-схемы имеют дополнительный инвертирующий каскад, находящийся, как правило, между входным каскадом и фазоинвертирующим. В результате неинвертирующие ТТЛ-схемы обычно содержат большее число элементов и функционируют медленнее, чем инвертирующие схемы, на базе которых они созданы.

Подобно КМОП-схемам у ТТЛ-схем может быть выход с тремя состояниями. Такие схемы имеют вход «разрешение выхода» или «запрещение выхода»: сигнал, действующий на этом входе, позволяет перевести выход в высокоомное состояние, когда оба выходных транзистора закрыты.

Некоторые ТТЛ-схемы имеют выход с открытым коллектором (*open-collector output*). В таких схемах полностью отсутствует верхняя половина выходного каскада, изображенного на рис. 3.75, так что подтягивание потенциала выхода до высокого уровня обеспечивается только с помощью внешнего резистора. Применения ТТЛ-схем с открытым коллектором и необходимые в этом случае расчеты подобны тем, что были приведены для КМОП-схем с открытым стоком.

3.11. Семейства ТТЛ-схем

Развитие ТТЛ-схем за прошедшие годы явилось реакцией на требования разработчиков цифровых систем улучшить эксплуатационные показатели. В результате появились и исчезли три ТТЛ-семейства и сегодня в распоряжении разработчиков имеется выбор из пяти выживших семейств. Все ТТЛ-семейства совместимы в том смысле, что работают при одном и том же напряжении питания и их логические уровни одинаковы, но каждое семейство имеет свои достоинства в отношении быстродействия, потребляемой мощности и стоимости.

3.11.1. Первые семейства ТТЛ-схем

Первое ТТЛ-семейство логических схем было создано фирмой Sylvania в 1963 году. Оно стало популярным благодаря фирме Texas Instruments, чьи схемы серии 7400 и другие ТТЛ-компоненты быстро стали промышленным стандартом.

Как и в случае КМОП-схем серии 7400, обозначение схем данного ТТЛ-семейства имеет вид: 74FAM n , где FAM – буквенный признак семейства, а n – номер, указывающий реализуемую функцию. Схемы различных семейств с одним и тем же значением n реализуют одну и ту же функцию. У первого ТТЛ-семейства буквенный признак FAM отсутствует, и это семейство называется 74-й серией ТТЛ (*74-series TTL*).

Два других ТТЛ-семейства с улучшенными техническими характеристиками стали результатом изменения сопротивлений резисторов в схемах исходного ТТЛ-семейства. В семействе 74H (*быстродействующие ТТЛ-схемы; High-speed TTL*) применение резисторов с меньшими значениями сопротивлений позволило уменьшить задержку распространения за счет увеличения потребляемой мощности. В семействе 74L (*маломощные ТТЛ-схемы; Low-power TTL*) использованы резисторы с большим сопротивлением, чтобы уменьшить потребляемую мощность за счет увеличения задержки распространения.

Благодаря наличию трех ТТЛ-семейств разработчики цифровых систем в 70-е годы имели возможность в процессе проектирования выбирать между высокой скоростью и малым потреблением мощности. Однако, подобно многим в то время, они хотели «иметь все сразу и сейчас». Это стало возможным с появлением транзисторов Шоттки и ТТЛ-схемы семейств 74, 74Н и 74L вышли из употребления; далее в этом параграфе речь пойдет о характеристиках улучшенных современных ТТЛ-схем.

3.11.2. ТТЛ-схемы с транзисторами Шоттки

Исторически первым семейством, в котором были применены транзисторы Шоттки, было семейство 74S (*ТТЛ-схемы с транзисторами Шоттки; Schottky TTL*). Применение транзисторов Шоттки и малых сопротивлений резисторов обеспечивает схемам этого семейства намного большее быстродействие, но и большую потребляемую мощность, чем у схем исходного семейства 74-й серии.

Вероятно, самым распространенным и, безусловно, самым дешевым является ТТЛ-семейство 74LS (*маломощные ТТЛ-схемы с транзисторами Шоттки; Low-power Schottky TTL*), появившееся вскоре после семейства 74S. Одновременное применение транзисторов Шоттки и резисторов с большими сопротивлениями, позволило получить в схемах 74LS такое же быстродействие, как и у схем семейства 74, но сократить потребляемую мощность примерно в пять раз. Таким образом, при разработке новых устройств на основе ТТЛ-схем применение логического семейства 74LS стало предпочтительным.

Развитие технологии производства ИС и новые идеи в схемотехнике привели к появлению еще двух серий с транзисторами Шоттки. У схем семейства 74AS (*улучшенные ТТЛ-схемы с транзисторами Шоттки, Advanced Schottky TTL*) примерно вдвое большее быстродействие, чем у схем семейства 74S, и приблизительно такая же потребляемая мощность. Меньшую потребляемую мощность и большее быстродействие, чем у схем семейства 74LS, имеют схемы семейства 74ALS (*улучшенные маломощные ТТЛ-схемы с транзисторами Шоттки, Advanced Low-power Schottky TTL*), и они составляют конкуренцию схемам семейства 74LS в популярности с точки зрения универсальности применения в новых разработках на основе ТТЛ-схем. В семействе 74F (*быстрые ТТЛ-схемы; Fast TTL*) реализуется компромисс между быстродействием и потребляемой мощностью; схемы семейства 74F находятся между схемами 74AS и 74ALS и являются, по-видимому, самыми популярными сегодня по применению в разработках высокоскоростных устройств на основе ТТЛ-схем.

3.11.3. Характеристики ТТЛ-схем

Основные характеристики современных ТТЛ-схем приведены в табл. 3.11. В первых двух строках указаны задержка распространения (в наносекундах) и потребляемая мощность (в милливаттах) типичного 2-входового вентиля И-НЕ для каждого из семейств.

Одним из критериев качества логического семейства является произведение задержки на мощность, приведенное в третьей строке таблицы. Как уже говорилось ранее, это просто произведение задержки распространения на потребляемую мощность типичного вентиля. Произведение задержки на мощность является

своего рода показателем эффективности: эта величина показывает, какая энергия расходуется вентилем при переключении сигнала на его выходе.

Табл. 3.11. Характеристики ТТЛ-схем

Описание	Обозначение	Семейство				
		74S	74LS	74AS	74ALS	74F
Максимальная задержка распространения (нс)		3	9	1.7	4	3
Потребляемая мощность на вентиль (мВт)		19	2	8	1.2	4
Произведение задержки на мощность (пДж)		57	18	13.6	4.8	12
Входное напряжение низкого уровня (В)	V_{ILmax}	0.8	0.8	0.8	0.8	0.8
Выходное напряжение низкого уровня (В)	V_{OLmax}	0.5	0.5	0.5	0.5	0.5
Входное напряжение высокого уровня (В)	V_{IHmin}	2.0	2.0	2.0	2.0	2.0
Выходное напряжение высокого уровня (В)	V_{OHmin}	2.7	2.7	2.7	2.7	2.7
Входной ток низкого уровня (мА)	I_{ILmax}	-2.0	-0.4	-0.5	-0.2	-0.6
Выходной ток низкого уровня (мА)	I_{OLmax}	20	8	20	8	20
Входной ток высокого уровня (мкА)	I_{IHmax}	50	20	20	20	20
Выходной ток высокого уровня (мкА)	I_{OHmax}	-1000	-400	-2000	-400	-1000

В остальных строках табл. 3.11 указаны входные и выходные параметры типичных ТТЛ-схем в каждом из семейств. Используя эту информацию, можно анализировать поведение ТТЛ-схем, не вдаваясь в подробности их внутреннего устройства. Эти параметры были определены и рассмотрены в разделах 3.10.2 и 3.10.3. Как всегда, входные и выходные параметры конкретных схем могут отличаться от значений приведенных в табл. 3.11, поэтому при анализе реальных устройств необходимо всегда принимать во внимание справочные данные, сообщаемые производителем.

3.11.4. Справочные данные для ТТЛ-схем

В табл. 3.12 приведена часть типичных справочных данных производителя для микросхемы 74LS00. Электрические характеристики схемы 54LS00 указаны в табли-

це, совпадают с характеристиками схемы 74LS00, за исключением того, что они выдерживаются при работе во всем «военном» диапазоне температур и напряжений и стоимость схемы 54LS00 выше. У большинства TTL-схем есть соответствующий эквивалент в 54-й серии (военный вариант). Приведенные в таблице справочные данные объединены в следующие три блока:

- *Рекомендуемые условия работы (recommended operating conditions)* включают напряжения питания, диапазоны входного напряжения, нагрузочную способность по постоянному току и температурный диапазон, в котором устройство работает нормально.
- *Электрические характеристики (electrical characteristics)* содержат дополнительные сведения о постоянных напряжениях и токах на входах и на выходе устройства при работе в рекомендуемых условиях:
 - I_I — максимальный входной ток для очень большого по величине входного напряжения высокого уровня;
 - I_{OS} — выходной ток при коротком замыкании на землю выхода схемы, на котором в отсутствие короткого замыкания присутствует сигнал высокого уровня;
 - I_{CCH} — ток, потребляемый от источника питания, когда на всех выходах (у четырех вентилях И-НЕ) сигналы имеют высокий уровень (в таблице приведено значение для всей микросхемы, содержащей четыре вентиля И-НЕ, так что ток, потребляемый одним вентиляем, составляет четвертую часть указанной величины);
 - I_{CCL} — ток, потребляемый от источника питания, когда на всех выходах (у четырех вентилях И-НЕ) сигналы имеют низкий уровень.
- *Характеристики переключения (switching characteristics)* представляют собой максимальные и типичные значения задержки распространения для «типичных» условий эксплуатации: $V_{CC} = 5 \text{ В}$ и $T_A = 25^\circ\text{C}$. При других напряжениях питания и температурах осторожный разработчик должен увеличить эти задержки на 5–10%, а при большей нагрузке необходимо внести еще большую поправку.

В справочных данных производителя имеется также четвертый блок сведений:

- *Предельные значения (absolute maximum ratings)*, то есть наилучшие условия, при которых устройство может работать или храниться без повреждения.

Полные справочные данные содержат также схемы тестирования, которые применяются для измерения параметров устройств в процессе их изготовления, и графики, показывающие, как изменяются типичные параметры при изменении таких условий работы, как напряжение питания (V_{CC}), температура окружающей среды (T_A) и нагрузка (R_L , C_L).

Табл. 3.12. Типичные справочные данные производителя для схемы 74LS00

РЕКОМЕНДУЕМЫЕ УСЛОВИЯ РАБОТЫ							
Параметр	Описание	SN54LS00			SN74LS00		
		Мин.	Ном.	Макс.	Мин.	Ном.	Макс.
V_{CC}	Напряжение питания	4.5	5.0	5.5	4.75	5.0	5.25
V_{IH}	Входное напряжение высокого уровня	2.0			2.0		
V_{IL}	Входное напряжение низкого уровня			0.7			0.8
I_{OH}	Выходной ток высокого уровня			-0.4			0.4
I_{OL}	Выходной ток низкого уровня			4			8
T_A	Температура окружающего воздуха	-55		125	0		70
ЭЛЕКТРИЧЕСКИЕ ХАРАКТЕРИСТИКИ В ПРЕДЕЛАХ РЕКОМЕНДУЕМОГО ДИАПАЗОНА ТЕМПЕРАТУР ОКРУЖАЮЩЕГО ВОЗДУХА							
Параметр	Условия тестирования ⁽¹⁾	SN54LS00			SN74LS00		
		Мин.	Тип. ⁽²⁾	Макс.	Мин.	Тип. ⁽²⁾	Макс.
V_{IK}	$V_{CC} = \text{Мин.}, I_K = -18 \text{ мА}$			-1.5			-1.5
V_{OH}	$V_{CC} = \text{Мин.}, V_{IL} = \text{Макс.}, I_{OH} = -0.4 \text{ мА}$	2.5	3.4		2.7	3.4	
V_{OL}	$V_{CC} = \text{Мин.}, V_{IH} = 2.0 \text{ В}, I_{OL} = 4 \text{ мА}$	0.25	0.4		0.25	0.4	
	$V_{CC} = \text{Мин.}, V_{IH} = 2.0 \text{ В}, I_{OL} = 8 \text{ мА}$				0.35	0.5	
I_I	$V_{CC} = \text{Макс.}, V_I = 7.0 \text{ В}$			0.1			0.1
I_{IH}	$V_{CC} = \text{Макс.}, V_I = 2.7 \text{ В}$			20			20
I_{IL}	$V_{CC} = \text{Макс.}, V_I = 0.4 \text{ В}$			-0.4			-0.4
$I_{IOS}^{(3)}$	$V_{CC} = \text{Макс.}$	-20		-100	-20		-100
I_{CSH}	$V_{CC} = \text{Макс.}, V_I = 0 \text{ В}$	0.8	1.6		0.8	1.6	
I_{CSL}	$V_{CC} = \text{Макс.}, V_I = 4.5 \text{ В}$	2.4	4.4		2.4	4.4	
ХАРАКТЕРИСТИКИ ПЕРЕКЛЮЧЕНИЯ ПРИ $V_{CC} = 5.0 \text{ В}, T_A = 25^\circ\text{C}$							
Параметр	От (вход)	К (выход)	Условия тестирования	Мин.	Тип.	Макс.	Ед. изм.
t_{PLH}	А или В	Y	$R_L = 2 \text{ кОм}, C_I = 15 \text{ пФ}$		9	15	нс
t_{PHL}					10	15	нс

ПРИМЕЧАНИЯ:

- 1 Там, где указано «Макс.» или «Мин.», следует использовать соответствующие значения из раздела «Рекомендуемые условия работы»
- 2 Все типичные значения приведены для $V_{CC} = 5.0 \text{ В}, T_A = 25^\circ\text{C}$.
- 3 В каждом испытании не должно быть замкнуто на землю более одного выхода, продолжительность короткого замыкания не должна превышать одной секунды

*3.12. Сопряжение КМОП- и ТТЛ-схем

При выборе логического семейства разработчик цифровой аппаратуры по умолчанию руководствуется общими требованиями в отношении быстродействия, потребляемой мощности, стоимости и так далее. Однако в некоторых случаях конструктор может выбирать в качестве элементной базы и другие семейства из-за их доступности или других специальных требований. (Например, не весь ряд схем семейства 74LS имеется в семействе 74НСТ и наоборот.) Таким образом, для разработчика важно понимать, как соединяются выходы ТТЛ-схем с входами КМОП-схем и наоборот.

При сопряжении ТТЛ- и КМОП-схем следует учесть несколько факторов, и первый из них – запас помехоустойчивости. Запас помехоустойчивости по постоянному току при низком уровне зависит от напряжения V_{OLmax} на выходе схемы, являющейся источником сигнала, а также от напряжения V_{ILmax} для входа, подключаемого к данному выходу, и равняется $V_{OLmax} - V_{ILmax}$. Точно так же запас помехоустойчивости по постоянному току при высоком уровне равняется $V_{OHmin} - V_{IHmin}$. На рис. 3.84 приведены соответствующие значения для КМОП- и ТТЛ-схем.

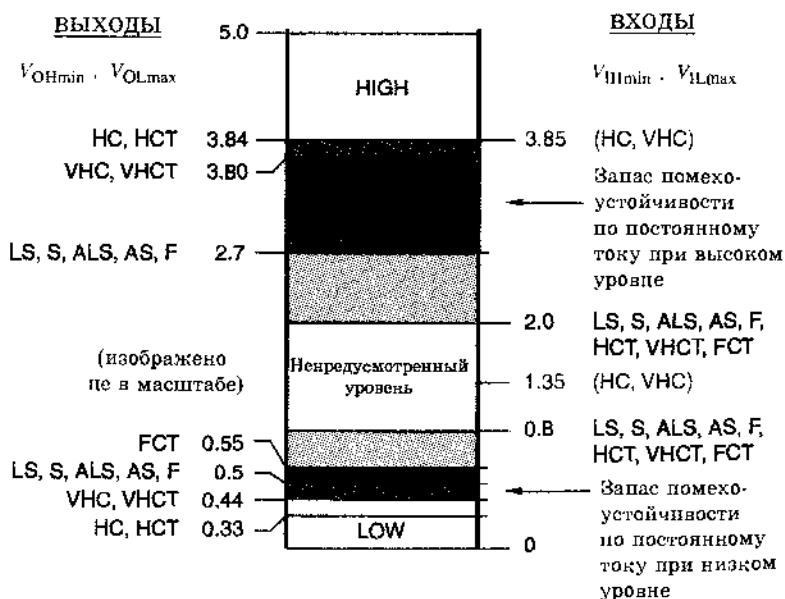


Рис. 3.84. Выходные и входные уровни сигналов, которые необходимо учитывать при сопряжении ТТЛ- и КМОП-схем: HIGH – высокий уровень, LOW – низкий уровень. (Заметьте, что схемы семейств HC и VHC по входу не совместимы со схемами ТТЛ.)

Например, запас помехоустойчивости по постоянному току при низком уровне сигнала на входе ТТЛ-схемы, подключенной к выходу одной из схем, принадлежащих семействам НС или НСТ, равен $0.8 \text{ В} - 0.33 \text{ В} = 0.47 \text{ В}$, а при высоком

уровне сигнала составляет $3.84 \text{ В} - 2.0 \text{ В} = 1.84 \text{ В}$. С другой стороны, запас по устойчивости при высоком уровне сигнала на выходе ТТЛ-схемы, нагруженной входами схем серии НС или VНС, равен $2.7 \text{ В} - 3.85 \text{ В} = -1.15 \text{ В}$. Другими словами, к выходу ТТЛ-схемы нельзя подключать входы НС- или VНС-схем, если только высокий уровень на выходе какой-то ТТЛ-схемы не окажется выше, а порог высокого уровня на входе у некоторой КМОП-схемы ниже по сравнению со значениями в наихудшем случае, и сумма отклонений не составит 1.15 В . Чтобы выходы ТТЛ-схем были правильно согласованы со входами КМОП-схем, КМОП-устройства должны принадлежать семействам НСТ, VНСТ или FСТ и не быть схемами из семейств НС или VНС.

В качестве следующего фактора рассмотрим коэффициент разветвления по выходу. Как и в случае с «чистыми» ТТЛ-схемами (см. раздел 3.10.3), разработчик должен сложить входные токи устройств, подключаемых к выходу, и сравнить результат с возможностями данной схемы по выходу при обоих уровнях выходного сигнала. Если ТТЛ-схема управляет КМОП-схемами, то проблем с коэффициентом разветвления по выходу не возникает, так как входам КМОП-схем при любом уровне сигнала почти не требуется никакого тока. С другой стороны, входам ТТЛ-схем, особенно при низком уровне входного сигнала, требуется значительный ток, по сравнению с возможностями выходных каскадов схем НС и НСТ. Например, к выходу схемы из семейств НС или НСТ можно подключить 10 входов LS-схем или только два входа схем, принадлежащих семейству S-TTL.

Последний фактор — величина емкостной нагрузки. Мы видели, что емкостная нагрузка приводит к увеличению как задержки, так и мощности, рассеиваемой логической схемой. Изменение задержки особенно заметно у схем НС и НСТ, для которых время переходного процесса растет примерно на 1 нс при увеличении емкости нагрузки на каждые 5 пФ. Транзисторы выходных каскадов схем FСТ во «включенном» состоянии имеют очень малое сопротивление, поэтому для данных схем время переходного процесса растет только на 0.1 нс с увеличением емкости нагрузки на каждые 5 пФ.

При заданной емкости нагрузки, напряжении питания и одинаковых условиях эксплуатации, динамическая рассеиваемая мощность у всех КМОП-семейств одна и та же, поскольку значения каждой из переменных, входящих в выражение CV^2f , одни и те же. С другой стороны, динамическая мощность, рассеиваемая в выходных цепях ТТЛ-схем, несколько меньше из-за меньшего перепада напряжения между высоким и низким уровнями у этих схем.

*3.13. Схемы низковольтной КМОП-логики и их сопряжение с другими схемами

Два важных фактора подтолкнули производителей ИС к снижению напряжения питания КМОП-схем:

- В большинстве случаев сигнал на выходе КМОП-схемы изменяется от потенциала земли до напряжения на шине питания, так что величина V в выражении CV^2f равняется напряжению питания. При снижении напряжения питания динамическая рассеиваемая мощность уменьшается еще быстрее.

- По мере продвижения технологии ко все меньшим размерам транзисторов, изоляция в виде окисл кремния между затвором КМОП-транзистора и стоком и истоком становится все более тонкой и поэтому неспособна выдержать разность потенциалов, достигающую до 5 В.

В результате группой промышленных стандартов ИС Объединенного технического совета по электронным приборам (JEDEC) в качестве очередного «стандарта» для логических схем были выбраны следующие напряжения питания: $3.3 \text{ В} \pm 0.3 \text{ В}$, $2.5 \text{ В} \pm 0.2 \text{ В}$, и $1.8 \text{ В} \pm 0.15 \text{ В}$. Стандартами JEDEC определены также входные и выходные напряжения логических уровней устройств, работающих с этими напряжениями питания.

Переход к меньшим напряжениям происходил постепенно и будет продолжаться дальше. В отношении дискретных логических семейств тенденция состояла в том, чтобы выпускать компоненты с меньшим напряжением питания и с меньшими значениями напряжений на выходах, но допускающие, тем не менее, более высокие напряжения на входах. В следующем разделе мы увидим, что этот подход позволяет КМОП-схемам с напряжением питания 3.3 В работать совместно с 5-вольтовыми КМОП- и ТТЛ-схемами.

Подобный подход использован во многих специализированных интегральных схемах и микропроцессорах, но часто применяется также и другой метод. Упомянутые устройства достаточно велики, так что имеет смысл снабдить их двумя источниками питания. Низкое напряжение, скажем 2.5 В, служит питанием для внутренних узлов микросхемы, ее *логического ядра* (*core logic*). Большее напряжение, например 3.3 В, используется для питания внешних цепей ввода и вывода, образующих *интерфейсный блок* (*pad ring*), посредством которого осуществляется сопряжение со схемами старшего поколения, применяемыми в системе. Для быстрого и безошибочного преобразования логических уровней между логическим ядром и интерфейсным блоком применяются специальные буферные схемы.

*3.13.1. LVTTTL- и LVCMOS-логика с напряжением питания 3.3 В

На рис. 3.85 наглядно представлены соотношения между уровнями сигналов для обычных ТТЛ-семейств и низковольтных КМОП-схем, работающих при своих номинальных напряжениях питания; эти соотношения взяты из указаний по применению фирмы Texas Instruments. Исходные симметричные уровни сигнала для чисто 5-вольтовых КМОП-семейств типа НС и VHC показаны на рис. 3.85(а). В КМОП-схемах, совместимых с ТТЛ-схемами, таких как НСТ, VHCT и FCT, уровни напряжения сдвинуты вниз, как показано на рис. 3.85(б).

Первым шагом на пути уменьшения напряжения питания КМОП-схем стало напряжение 3.3 В. Фактически стандарт JEDEC для 3.3-вольтовой логики определяет два набора уровней. Уровни LVCMOS (*низковольтные уровни КМОП-схем; low-voltage CMOS*) относятся к случаю использования только КМОП-схем, когда выходы схем слабо нагружены по постоянному току (меньше 100 мкА), так что напряжения V_{OL} и V_{OH} отличаются от потенциала земли и от напряжения питания не более, чем на 0.2 В. Уровни LVTTTL (*низковольтные уровни схем, совместимых с ТТЛ; low-*

voltage TTL), приведенные на рис. 3.85(с), используются в приложениях, где выходы существенно нагружены по постоянному току, и поэтому напряжение V_{OL} может достигать 0.4 В, а напряжение V_{OH} может опускаться до 2.4 В.

Расположение логических уровней TTL-схем в нижней части 5-вольтового диапазона в действительности было совершенно случайным. Как показано на рис. 3.85(б) и (с), уровни LVTTTL оказались возможным задать так, чтобы они точно совпадали с уровнями TTL-схем. Таким образом, к выходу схемы с уровнями LVTTTL можно без проблем подключать входы TTL-схем до тех пор, пока не нарушаются требования относительно величины выходного тока (I_{OLmax} , I_{OHmax}). Аналогично к выходу TTL-схемы можно подключать вход схемы с уровнями LVTTTL, за исключением тех случаев, когда подаваемый сигнал превышает напряжение питания V_{CC} схем с уровнями LVTTTL, равное 3.3 В, о чем речь пойдет ниже.

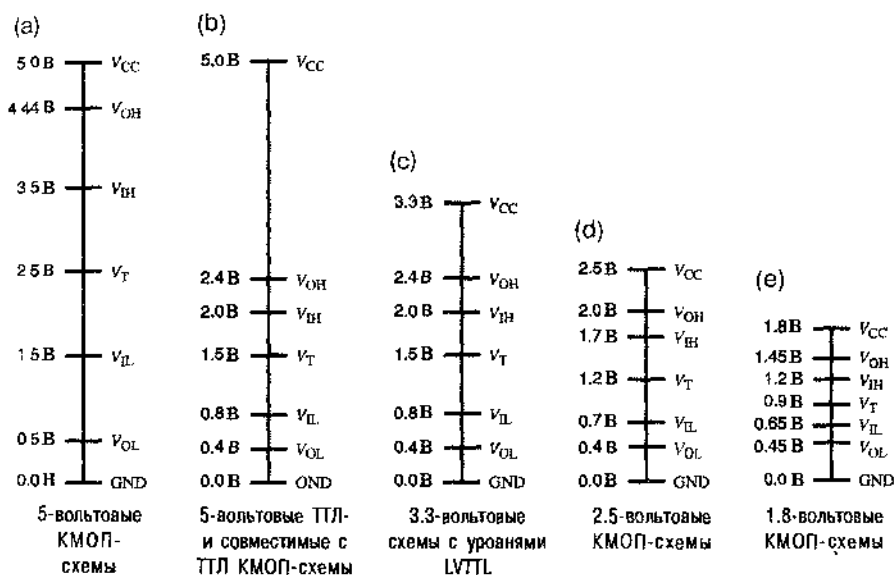


Рис. 3.35. Сравнение логических уровней: (а) 5-вольтовые КМОП-схемы; (б) 5-вольтовые TTL-схемы, а также совместимые с TTL 5-вольтовые КМОП-схемы; (с) 3.3-вольтовые схемы с уровнями LVTTTL; (д) 2.5-вольтовые КМОП-схемы; (е) 1.8-вольтовые КМОП-схемы (GND – земля)

*3.13.2. Входы, допускающие напряжение 5 В

На входы вентиля не всегда можно подавать напряжение, превышающее напряжение питания V_{CC} . Эта проблема возникает в том случае, если в системе применяются схемы как 5-вольтовых, так и 3.3-вольтовых логических семейств. Если, например, 5-вольтовые КМОП-схемы нагружены не сильно, то они вполне могут иметь на выходе 4.9 В, и даже при умеренной нагрузке КМОП- и TTL-схемы обычно дают на выходе 4.0 В. Такие высокие напряжения могут «не понравиться» входам 3.3-вольтовых схем.

Максимальное напряжение V_{Imax} , которое можно подать на вход, указывается в разделе сообщаемых производителем справочных данных, озаглавленном «Пре-

дельные значения». Для схем серии НС величина V_{imax} равняется напряжению питания V_{CC} . Таким образом, если у схемы из этой серии напряжение питания равно 3.3 В, то на ее входы нельзя подавать сигналы с каких бы то ни было выходов 5-вольтовых ТТЛ- или КМОП-схем. С другой стороны, для схем серии VHC напряжение V_{imax} равно 7 В; следовательно, схемы VHC с напряжением питания 3.3 В можно применять для преобразования выходных сигналов 5-вольтовых схем к уровням 3.3-вольтовых устройств для совместного использования с микропроцессорами, блоками памяти и другими устройствами в подсистемах с напряжением питания 3.3 В.

Из рис. 3.86 видно, почему на одни входы можно подавать напряжение 5 В, а на другие нельзя. Как показано на рис. 3.86(а), входная цепь схем НС и НСТ, в действительности, содержит два смещенных в обратном направлении *фиксирующих диода* (*clamp diodes*), которые мы прежде не показывали: один диод включен между каждым из входов и шиной питания V_{CC} , а другой – между входом и землей. Назначение этих диодов состоит в том, чтобы с помощью диода $D1$ шунтировать вход на землю, когда входной сигнал во время переходного процесса становится отрицательным, а с помощью диода $D2$ замыкать вход на шину питания, когда входной сигнал превышает напряжение питания V_{CC} . Такие кратковременные всплески могут происходить в результате отражений в линии передачи (см. параграф 11.4). Шунтирование входа на землю на время отрицательных выбросов напряжения или на шину питания V_{CC} на время положительных выбросов уменьшает амплитуду и длительность отраженных сигналов.

С помощью диода $D2$ нельзя, конечно, отличить кратковременный положительный выброс от превышения входным напряжением напряжения питания V_{CC} в течение длительного времени. Следовательно, если выход 5-вольтовой схемы соединив с одним из таких входов, то у этого входа не будет очень большого сопротивления, которое обычно ассоциируется с входом КМОП-схемы. Вместо этого входное сопротивление схемы будет относительно малым, равным сопротивлению смещенного теперь в прямом направлении диода $D2$, подключенного к шине питания V_{CC} , по которому потечет большой ток.

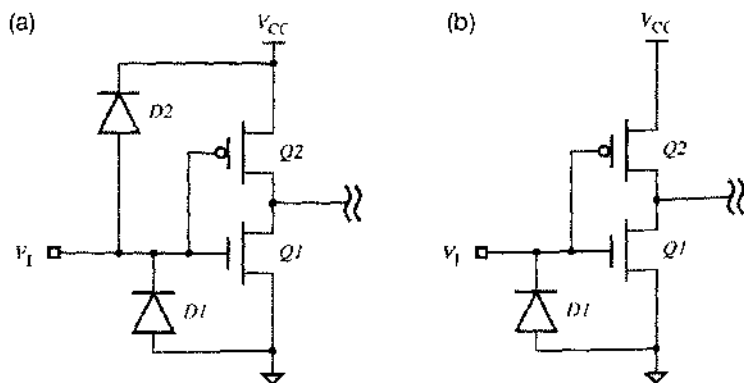


Рис. 3.86. Входные цепи КМОП-схем: (а) семейство НС, не допускающее появления на входе напряжения, равного 5 В; (б) семейство VHC, допускающее появление на входе напряжения, равного 5 В

На рис. 3-86(б) показан фрагмент КМОП-схемы, на вход которой можно подавать напряжение 5 В. Во входной цепи этой схемы просто отсутствует диод $D2$; благодаря диоду $D1$ сохраняется шунтирование входа при отрицательных выбросах напряжения. Такой вид имеет входная цепь в схемах VHC и ANC.

Структура входной цепи должна быть такой, как показано на рис. 3.86(б), но этого не достаточно, чтобы вход допускал подачу на него напряжения 5 В. Процесс изготовления микросхем должен включать создание транзисторов, способных выдерживать больше напряжения, чем V_{CC} . По этой причине в семействе VHC напряжение $V_{I_{max}}$ ограничено величиной 7.0 В. В технологическом процессе изготовления многих специализированных интегральных схем с напряжением питания 3.3 В нет возможности получить входы, допускающие напряжение 5 В, даже если есть желание отказаться от выгод, связанных с диодом $D2$, который шунтирует вход при положительных всплесках в линии передачи.

*3.13.3. Выходы, допускающие напряжение 5 В

В случае, когда выходы схем с тремя состояниями с напряжением питания 3.3 В и 5 В подключаются к одной шине, необходимо проверить способность выходных цепей выдерживать напряжение 5 В. Когда выход 3.3-вольтовой схемы находится в третьем состоянии (в состоянии Hi-Z), схема с напряжением питания 5 В может стать источником сигнала, передаваемого по шине и на выходе 3.3-вольтовой схемы может появиться напряжение 5 В.

Рис. 3.87 поясняет, почему в этой ситуации некоторые выходные цепи допускают напряжение 5 В, а другие не допускают этого. Как показано на рис. 3.87(а), в обычной КМОП-схеме с тремя состояниями на выходе имеются n -канальный транзистор $Q1$ между выходом и землей и p -канальный транзистор $Q2$ между выходом и шиной питания V_{CC} . Когда выход находится в состоянии Hi-Z, с помощью схемы (не показанной на рисунке) напряжение на затворе транзистора $Q1$ поддерживается равным примерно 0 В, а напряжение на затворе транзистора $Q2$ – примерно равным напряжению питания V_{CC} , так что оба транзистора закрыты.

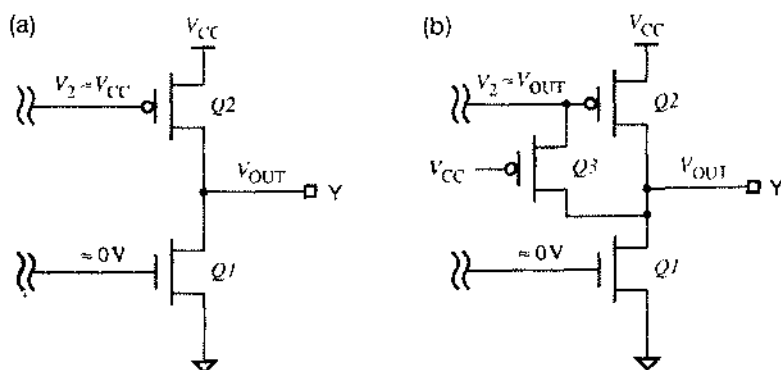


Рис. 3.37. Выходные цепи КМОП-схем с тремя состояниями: (а) выходы схем HC и VHC, не допускающих напряжение 5 В; (б) выходы схем LVC, допускающих напряжение 5 В

Рассмотрим теперь, что произойдет, если $V_{CC} = 3.3$ В, а от другого устройства на выходной контакт Y [рис. 3.87(a)] поступает напряжение 5 В. Тогда на стоке транзистора Q2 (вывод Y) будет 5 В, в то время как напряжение на затворе (V_2) равно всего лишь 3.3 В. Когда потенциал затвора окажется ниже, чем потенциал стока, транзистор Q2 начнет проводить, цепь от точки Y до шины питания V_{CC} будет иметь относительно малое сопротивление и потечет большой ток. Выходные цепи схем с тремя состояниями семейств НС и VНС имеют именно такую структуру, и поэтому напряжение 5 В для них не допустимо.

На рис. 3.87(b) приведена выходная цепь, допускающая напряжение 5 В. Дополнительный p -канальный транзистор Q3 позволяет предотвратить отпирание транзистора Q2, когда этого не должно быть. Если напряжение V_{OUT} больше напряжения V_{CC} , то открывается транзистор Q3. Этим обеспечивается относительно малое сопротивление между точкой Y и затвором транзистора Q2, который в данном случае остается закрытым, потому что напряжение V_2 на его затворе теперь не меньше напряжения на стоке. Такой является выходная цепь схем LVC фирмы Texas Instruments (низковольтные КМОП-схемы; low-voltage CMOS).

*3.13.4. Сопряжение TTL-схем и схем с уровнями LVTTTL: сводка результатов

На основе сведений, приведенных в предыдущих разделах, можно сделать вывод о возможности применения в одной системе TTL-схем (с напряжением питания 5 В) и схем с уровнями LVTTTL (с напряжением питания 3.3 В), только следуя следующим трем правилам:

1. Сигналы с выходов схем с уровнями LVTTTL можно непосредственно подавать на входы TTL-схем при соблюдении обычных ограничений на выходной ток (I_{OLmax} , I_{OHmax}) схем, являющихся источниками сигналов.
2. Сигналы с выходов TTL-схем можно непосредственно подавать на входы схем с уровнями LVTTTL, если последние допускают входные напряжения 5 В.
3. Выходы TTL-схем и схем с уровнями LVTTTL с тремя состояниями можно подключать к одной и той же шине при условии, что выходы схем с уровнями LVTTTL допускают напряжение 5 В.

*3.13.5. Логические схемы с напряжениями питания 2.5 В и 1.8 В

Переход от 3.3-вольтовых схем к 2.5-вольтовым схемам не так прост. Известно, что выходы 3.3-вольтовых схем можно соединять с входами 2.5-вольтовых схем, если эти входы допускают напряжение 3.3 В. Однако даже беглый взгляд на рис. 3.85(c) и (d) показывает, что выходное напряжение V_{OH} 2.5-вольтовой схемы равняется входному напряжению V_{IH} 3.3-вольтовой схемы. Другими словами, запас помехоустойчивости по постоянному току при высоком уровне, когда выход 2.5-вольтовой схемы соединен с входом 3.3-вольтовой схемы, равен нулю, что не желательно.

Эта проблема решается путем применения преобразователя уровня (*level translator*) или схемы сдвига уровня (*level shifter*), то есть устройства, на которое подаются оба напряжения питания и внутри которого происходит подтягивание

более низких логических уровней (соответствующих напряжению питания 2.5 В) до больших значений (соответствующих напряжению питания 3.3 В). Сегодня многие специализированные интегральные схемы и микропроцессоры содержат внутри себя преобразователи уровней, что позволяет логическому ядру работать с напряжением питания 2.5 В или 2.7 В, а интерфейсному блоку – с напряжением питания 3.3 В, о чем мы говорили в начале этого параграфа. Если в будущем станут популярными 2.5-вольтовые дискретные устройства, то можно ожидать, что главные поставщики полупроводниковых приборов начнут производить преобразователи уровней также в виде отдельных компонентов.

Следующим шагом будет переход от 2.5-вольтовой логики к 1.8-вольтовой. Из рис. 3.85(d) и (e) можно видеть, что запас помехоустойчивости по постоянному току при высоком уровне фактически отрицателен, когда сигнал с выхода 1.8-вольтовой схемы поступает на вход 2.5-вольтовой схемы, так что преобразователи уровня будут необходимы также и в этом случае.

*3.14. Эмиттерно-связанная логика

Ключом к уменьшению задержки распространения в биполярных логических схемах является предотвращение насыщения находящихся в них транзисторов. В разделе 3.9.5 уже было объяснено, что диоды Шоттки предотвращают насыщение транзисторов в ТТЛ-схемах. Однако насыщение можно также предотвратить, используя совершенно другой принцип, а именно – с помощью схем, называемых *логическими схемами на переключателях тока (current-mode logic, CML)* или схемами *эмиттерно-связанной логики (ЭСЛ; emitter-coupled logic, ECL)*.

В отличие от других логических схем, рассматриваемых в этой главе, ЭСЛ-схемы не обеспечивают большого различия напряжений между низким и высоким уровнями. Эти схемы дают небольшой перепад напряжения, меньше вольта, и внутри у них происходит переключение тока между двумя возможными путями в зависимости от значения сигнала на выходе.

Первое семейство логических схем ЭСЛ было представлено фирмой General Electric в 1961 году. Идея вскоре получила развитие и фирмой Motorola и другими были созданы популярные до настоящего времени семейства ЭСЛ-схем 10К и 100К. Эти схемы очень быстры; задержка распространения у них менее 1 нс. У последнего семейства ЭСЛ-схем ECLinPS (буквально: пикосекундная ЭСЛ; ECL in picoseconds) максимальные задержки меньше 0.5 нс (500 пс), включая задержку сигнала при включении и выключении ИС. На всем протяжении развития технологии цифровых схем ЭСЛ-схемы того или иного типа всегда оказывались самыми быстродействующими среди логических схем, выпускавшихся в дискретном исполнении.

Однако сегодня ЭСЛ-схемы не столь популярны, как КМОП- и ТТЛ-схемы, главным образом потому, что они потребляют намного большую мощность. Фактически из-за большой потребляемой мощности создание супер-ЭВМ на схемах ЭСЛ типа Cray-1 и Cray-2 представляло собой столь же сложную задачу для техники охлаждения, что и для цифровой электроники. Кроме того, у ЭСЛ-схем велико значение такого параметра, как произведение задержки на потребляемую мощность, для них не удается обеспечить большую степень интеграции, а сигналы

имеют настолько короткие фронты, что в большинстве случаев требуется учитывать эффекты, возникающие в линии передачи; наконец, схемы ЭСЛ непосредственно не совместимы с ТТЛ- и КМОП-схемами. Тем не менее, ЭСЛ-схемы по-прежнему находят свое место в качестве логических и интерфейсных элементов в сверхбыстродействующих устройствах связи, включая волоконно-оптические интерфейсы приемопередатчиков для гигабитной сети Ethernet и сетей с асинхронной передачей данных (ATM).

*3.14.1. Базовая схема ЭСЛ

Основная идея применения токового зеркального переключателя в логической схеме показана на рис. 3.88 на примере схемы инвертора/буфера. У этой схемы два выхода: инвертирующий выход (OUT1) и неинвертирующий выход (OUT2). Два транзистора образуют дифференциальный усилитель (*differential amplifier*) с общим эмиттерным резистором. Постоянные напряжения в этом примере имеют следующие значения: $V_{CC} = 5.0$ В, $V_{BB} = 4.0$ В и $V_{EE} = 0$ В; низкий и высокий уровни на входе, по определению, равны 3.6 В и 4.4 В. В действительности данная схема дает на выходе в качестве низкого и высокого уровней напряжения, которые на 0.6 В выше (4.2 В и 5.0 В соответственно), но в реальных ЭСЛ-схемах это скорректировано.

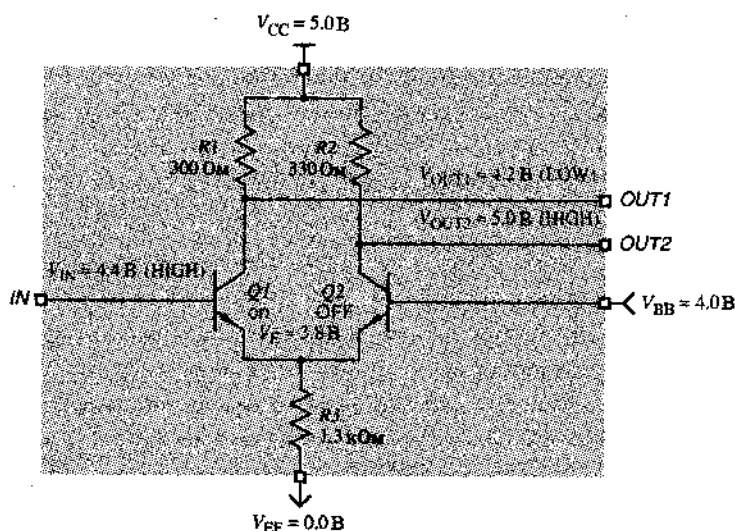


Рис. 3.83. Базовая схема инвертора/буфера ЭСЛ при высоком уровне сигнала на входе (LOW – низкий уровень, HIGH – высокий уровень; оп – открыт, OFF – закрыт)

Пусть напряжение V_{IN} соответствует высокому уровню, как показано на рисунке; тогда транзистор $Q1$ открыт, но не насыщен, а транзистор $Q2$ закрыт. Это достигается путем аккуратного выбора сопротивлений резисторов и уровней напряжения. Благодаря наличию резистора $R2$ напряжение V_{OUT2} поднимается до 5.0 В (высокий уровень). Можно показать, что падение напряжения на резисторе

$R1$ составляет около 0.8 В, так что напряжение V_{OUT1} равно примерно 4.2 В (низкий уровень).

Когда напряжение V_{IN} соответствует низкому уровню, как показано на рис. 3.89, транзистор $Q2$ открыт, но не насыщен, а транзистор $Q1$ закрыт. При этом напряжение V_{OUT1} благодаря наличию резистора $R1$ поднимается до 5.0 В, а напряжение V_{OUT2} , как нетрудно показать, равно примерно 4.2 В.

Выходы этого инвертора называют *дифференциальными выходами (differential outputs)*, потому что они всегда комплементарны, и значение выходного сигнала можно определить по разности между выходными напряжениями ($V_{OUT1} - V_{OUT2}$), а не по их абсолютным значениям. Другими словами, можно считать, что выходной сигнал равен 1, если $(V_{OUT1} - V_{OUT2}) > 0$, и равен 0, если $(V_{OUT1} - V_{OUT2}) < 0$. Входную цепь можно сделать такой, чтобы на каждый логический вход сигнал поступал по двум проводам и его логическое значение определялось по указанному правилу; такие входы называются *дифференциальными входами (differential inputs)*.

Дифференциальные сигналы используются в большинстве устройств на основе ЭСЛ-схем, выполняющих функции «интерфейса» и «размножителя тактовых сигналов» из-за их малой асимметрии и высокой помехоустойчивости. Малая асимметрия обусловлена тем, что время перехода от 0 к 1 или от 1 к 0 слабо зависит от пороговых напряжений, которые могут изменяться с температурой или от схемы к схеме: момент перехода зависит только от того, когда одно напряжение изменит знак относительно другого. Точно так же, «относительность» определения 0 и 1 обеспечивает превосходную помехоустойчивость, так как шум, создаваемый колебаниями напряжения питания или поступающий от внешних источников, как правило, является *синфазным сигналом (common-mode signal)*, который действует на обоих дифференциальных входах одновременно, и при этом значение разности остается неизменным.

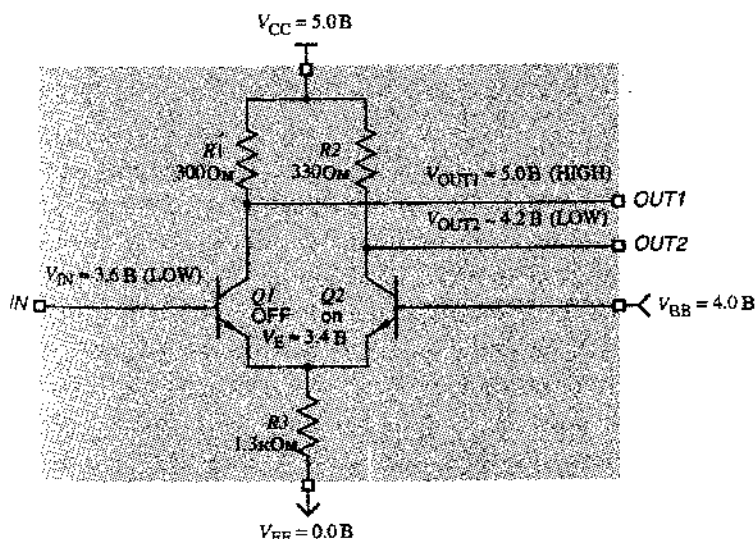


Рис. 3.89. Базовая схема инвертора/буфера ЭСЛ при низком уровне сигнала на входе (LOW – низкий уровень, HIGH – высокий уровень; оп – открыт, OFF – закрыт)

Можно, конечно, определять логическое значение, учитывая абсолютный уровень сигнала на одном входе, который в этом случае называется *несимметричным входом* (*single-ended input*). Чтобы избежать очевидного удвоения числа сигнальных линий, в большинстве «логических» приложений ЭСЛ-схем сигналы передаются по одному проводу на несимметричный вход. Базовая схема ЭСЛ-инвертора, изображенная на рис. 3.89, имеет несимметричный вход. Кроме того, у нее всегда имеются оба «выхода», поэтому фактически схема является инвертирующим или неинвертирующим буфером, в зависимости от того, каким выходом мы пользуемся: OUT1 или OUT2.

Чтобы реализовать ту или иную логику на основе базовой схемы (рис. 3.89), параллельно с транзистором Q1 включаются дополнительные транзисторы, подобно тому, как это делается в ТТЛ-схеме ИЛИ-НЕ. На рис. 3.90 в качестве примера приедена 2-входовая ЭСЛ-схема ИЛИ/ИЛИ-НЕ. Если на каком-либо входе имеется высокий уровень, то соответствующий входной транзистор открыт и напряжение V_{OUT1} соответствует низкому уровню (выход ИЛИ-НЕ). В то же самое время транзистор Q3 закрыт и напряжение V_{OUT2} соответствует высокому уровню (выход ИЛИ).

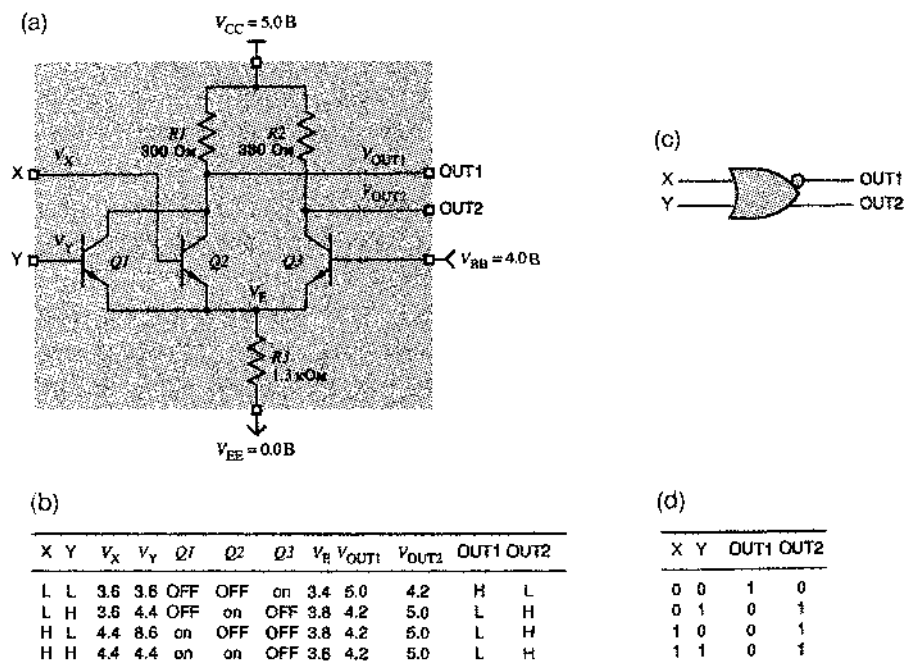


Рис. 3.90. Схема 2-входового ЭСЛ-вентиль ИЛИ/ИЛИ-НЕ: (а) принципиальная схема; (б) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень; on – открыт, OFF – закрыт); (с) условное обозначение; (д) таблица истинности

Вспомним, что напряжения входных уровней для инвертора/буфера, по определению, составляют 3.6 В и 4.4 В, в то время как получаемые на выходе напряжения равны 4.2 В и 5.0 В. Очевидно, что это вызывает затруднения. Для того, чтобы

выходные напряжения соответствовали входным уровням, можно было бы последовательно с каждым выходом включить диоды, понизив тем самым напряжения на 0,6 В, но при этом остается другая проблема – мал коэффициент разветвления по выходу. При высоком уровне на выходе данной схемы в базы входных транзисторов схем, подключенных к этому выходу, течет ток, что приводит к дополнительному падению напряжения на резисторах $R1$ или $R2$ и уменьшению выходного напряжения (и у нас нет достаточного запаса помехоустойчивости, чтобы преодолеть это). Перечисленные проблемы решены в серийных ЭСЛ-схемах, таких как 10К, которые описаны ниже.

*3.14.2. Семейства ЭСЛ-схем 10К/10Н

Микросхемы самого популярного сегодня ЭСЛ-семейства имеют обозначение, состоящее из 5 цифр в виде «10ххх» (например, 10102, 10181, 10209), поэтому его обычно называют *ЭСЛ-семейством 10К (ECL 10К)*. У схем этого семейства имеются некоторые усовершенствования по сравнению с базовой ЭСЛ-схемой, описанной выше:

- * Эмиттерные повторители, включенные на каждом из выходов, сдвигают выходные напряжения настолько, что они соответствуют входным уровням и обеспечивают большую нагрузочную способность по току, до 50 мА на выход.
- * Напряжение смещения $V_{ВВ}$ обеспечивается внутренней цепью, поэтому отдельного, внешнего источника питания не требуется.
- * Семейство разработано так, чтобы оно работало с $V_{СС} = 0$ В (земля) и $V_{ЕЕ} = -5.2$ В. Как правило, при наблюдении сигналов относительно земли уровень шума оказывается меньшим, чем в случае, когда уровни сигналов отсчитываются относительно шины питания. В ЭСЛ-схемах уровни логических сигналов определяются относительно напряжения $V_{СС}$ на шине питания, поэтому разработчики семейства решили сделать это напряжение равным 0 В («чистая» земля) и использовать в качестве $V_{ЕЕ}$ отрицательное напряжение. Помехи, возникающие в цепи питания и появляющиеся на шине $V_{ЕЕ}$, являются «синфазным сигналом», который ослабляется в дифференциальном усилителе благодаря конфигурации его входной цепи.
- * Микросхемы с префиксом 10Н (*ЭСЛ-семейство 10Н; ECL 10Н family*) являются полностью скорректированными по напряжению, поэтому они будут нормально работать при напряжениях питания $V_{ЕЕ}$, отличающихся от -5.2 В; этот режим работы будет рассмотрен в разделе 3.14.4.

На рис. 3.91 показано, как для ЭСЛ-семейства 10К определены низкий и высокий уровни. Обратите внимание: несмотря на то, что напряжение питания отрицательно, названия уровней «низкий» (LOW) и «высокий» (HIGH) приняты для ЭСЛ-схем в соответствии с алгебраически низким и высоким напряжениями соответственно.

Запас помехоустойчивости по постоянному току у схем ЭСЛ-семейства 10К намного меньше, чем у КМОП- и ТТЛ-схем, всего лишь 0.155 В при низком уровне и 0.125 В при высоком уровне. Однако ЭСЛ-схемы не нуждаются в таком же большом запасе помехоустойчивости, как схемы КМОП- и ТТЛ-семейств. В отличие от КМОП- и ТТЛ-схем, ЭСЛ-схемы при переключении создают очень малые помехи

на шине питания и на шине земли; токи, потребляемые ЭСЛ-схемами, остаются неизменными, так как в них просто происходит переключение тока с одного пути на другой. К тому же в ЭСЛ-схемах выходные сопротивления эмиттерных повторителей в любом состоянии очень малы, и внешнему источнику трудно создать помеху в сигнальной линии, подключаемой к такому выходу.

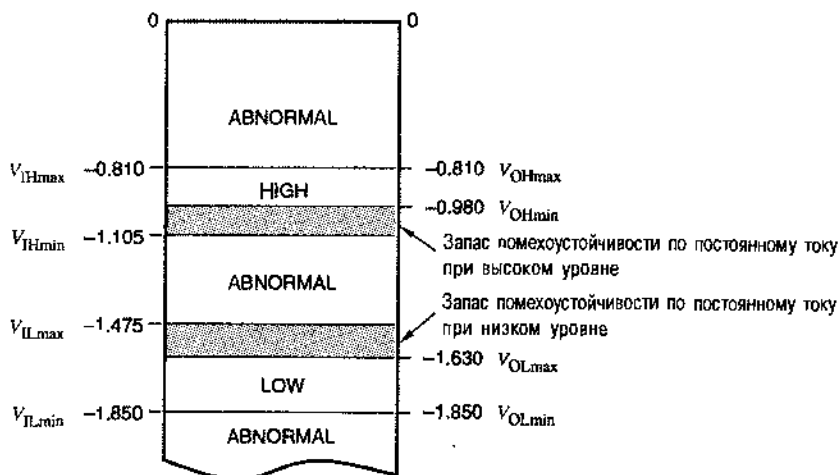
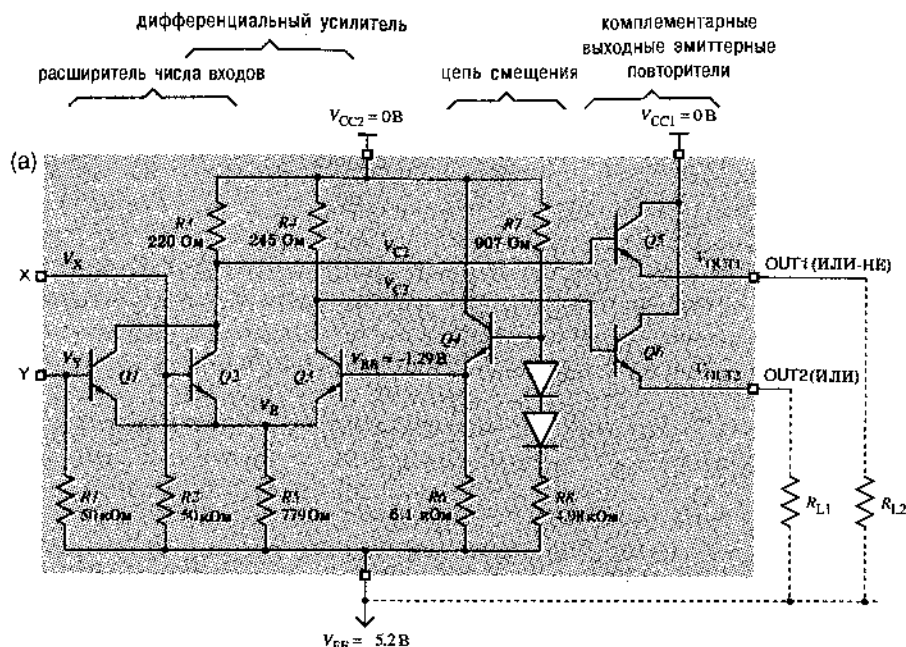


Рис. 3.91. Логические уровни ЭСЛ-семейства 10K (ABNORMAL – непредусмотренный уровень, HIGH – высокий уровень, LOW – низкий уровень)

На рис. 3.92(а) приведена схема одного из четырех вентилях ИЛИ/ИЛИ-НЕ в микросхеме 10102. С помощью резисторов $R1$ и $R2$ на входах обеспечивается присутствие на любом из них низкого уровня, если этот вход остается или к чему не подключенным. Номиналы компонентов в цепи смещения выбраны так, чтобы получить напряжение $V_{BB} = -1.29$ В, необходимое для правильной работы дифференциального усилителя. У выходных транзисторов, включенных по схеме эмиттерного повторителя, потенциал каждого эмиттера ниже потенциала соответствующей базы на величину напряжения на открытом диоде; этим достигается требуемый сдвиг выходного уровня. Таблица на рис. 3.92(б) описывает работу схемы.

Выходы эмиттерных повторителей в схемах ЭСЛ-семейства 10K, требуют подключения внешних резисторов, как показано на рисунке. Применение в данном случае внешних резисторов, а не внутренних имеет серьезные основания. Скорость нарастания и спада сигналов на выходе ЭСЛ-схем при переключении настолько велика (типичное значение времени переключения составляет 2 нс), что любое соединение длиной в несколько дюймов необходимо рассматривать как длинную линию, которая должна иметь на концах согласованные нагрузки (см. параграф 11.4). Дело здесь даже не в мощности, которая бесполезно рассеивалась бы на внутреннем резисторе: применяя ЭСЛ-схемы семейства 10K, разработчик имеет возможность выбрать внешний резистор так, чтобы одновременно обеспечить нагрузку для эмиттерного повторителя и согласовать длину линии на конце. В простейшем случае при коротких соединениях подходящей нагрузкой является резистор с сопротивлением от 270 Ом до 2 кОм, включенный между каждым из выходов и шиной питания V_{EE} .



(b)

X	Y	V_X	V_Y	Q1	Q2	Q3	V_E	V_{C2}	V_{C3}	V_{OUT1}	V_{OUT2}	OUT1	OUT2
L	L	-1.8	-1.8	OFF	OFF	on	-1.9	-0.2	-1.2	-0.9	-1.8	H	L
L	H	-1.8	-0.9	OFF	on	OFF	-1.5	-1.2	-0.2	-1.8	-0.9	L	H
H	L	-0.9	-1.8	on	OFF	OFF	-1.5	-1.2	-0.2	-1.8	-0.9	L	H
H	H	-0.9	-0.9	on	on	OFF	-1.5	-1.2	-0.2	-1.8	-0.9	L	H

(c)

X	Y	OUT1	OUT2
0	0	1	0
0	1	0	1
1	0	0	1
1	1	0	1

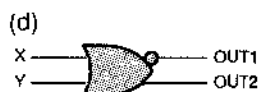


Рис. 3.92. Двухвходовая ЭСЛ-схема ИЛИ/ИЛИ-НЕ серии 10К: (a) принципиальная схема; (b) таблица, описывающая работу схемы (L – низкий уровень, H – высокий уровень; OFF – закрыт, on – открыт); (c) таблица истинности; (d) условное обозначение

У типичного вентиля ЭСЛ-семейства 10К задержка распространения равна 2 нс и сопоставима с задержкой ТТЛ-схем 74АС. Если выходы вентиля семейства 10К оставить ни к чему не подключенными, то он потребляет мощность около 26 мВт, что также сравнимо с мощностью, потребляемой ТТЛ-схемой 74АС, которая составляет примерно 20 мВт. Однако окопечная нагрузка, необходимая для ЭСЛ-схемы 10К, также потребляет мощность от 10 до 150 мВт на каждый выход в зависимости от типа нагрузки, тогда как выход ТТЛ-схемы 74АС может требовать, а может и не требовать оконечной нагрузки, потребляющей мощность, в зависимости от физических характеристик приложения.

*3.14.3. Семейство ЭСЛ-схем 100K

Микросхемы *ЭСЛ-семейства 100K (ECL 100K family)* имеют обозначение, состоящее из 6 цифр в виде «100xxx» (например, 100101, 100117, 100170), но в общем случае реализуемые ими функции отличаются от функций, выполняемых схемами 10K с теми же номерами. Семейство 100K имеет следующие существенные отличия от семейства 10K:

- Меньшее напряжение питания $V_{EE} = -4.5$ В.
- Другие логические уровни, как следствие другого напряжения питания.
- Меньшие значения задержки распространения: типичное значение 0.75 нс
- Меньшие значения времени перехода: типичное значение 0.70 нс.
- Большая потребляемая мощность: типичное значение 40 мВт на вентиль.

*3.14.4. ЭСЛ-схемы с положительным напряжением питания

Мы уже говорили о достоинствах ЭСЛ-схем с отрицательным напряжением питания ($V_{EE} = -5.2$ В или -4.5 В), которые заключаются в их нечувствительности к шумам, но с другой стороны схемам с отрицательным напряжением питания присущ большой недостаток: в самых популярных сегодня КМОП- и ТТЛ-схемах, в специализированных интегральных схемах и в микропроцессорах применяется положительное напряжение питания, обычно равное $+5.0$ В и имеющее тенденцию к понижению до $+3.3$ В. Поэтому для систем, содержащих как ЭСЛ-схемы, так и КМОП/ТТЛ-схемы, требуются два источника питания. Кроме того, сопряжение обычных ЭСЛ-схем 10K или 100K с отрицательными логическими уровнями и КМОП/ТТЛ-схем с положительными уровнями требует наличия специальных схем преобразования уровней, на которые нужно подавать оба напряжения питания.

В ЭСЛ-схемах с положительным напряжением питания (*positive ECL; PECL*, произносится "peckle") используется стандартное напряжение питания $+5.0$ В. Обратите внимание, что в ЭСЛ-схеме 10K, изображенной на рис. 3.92, нет ничего, что требовало бы обязательного заземления шины V_{CC} и соединения с источником питания -5.2 В шины V_{EE} . Схема будет работать точно так же, если заземлить шину V_{EE} , а шину V_{CC} соединить с источником напряжения $+5.2$ В.

Таким образом, ЭСЛ-схемы с положительным напряжением питания представляют собой не что иное, как обычные ЭСЛ-схемы с заземленной шиной V_{EE} и поданым на шину V_{CC} напряжением $+5.0$ В. При этом напряжение между шинами V_{EE} и V_{CC} немного меньше, чем у обычной ЭСЛ-схемы 10K, и больше, чем у обычной ЭСЛ-схемы 100K, но схемы серий 10N и 100K будут хорошо работать с немного большими или немного меньшими напряжениями питания.

Так же, как и у ЭСЛ-схем, логические уровни сигналов ЭСЛ-схем с положительным напряжением питания привязаны к потенциалу шины V_{CC} , так что высокому уровню сигналов в этих схемах соответствует напряжение, примерно равное $V_{CC} - 0.9$ В, а низкому уровню – напряжение, примерно равное $V_{CC} - 1.7$ В, или около 4.1 В и 3.3 В соответственно при номинальном напряжении питания $V_{CC} = 5$ В. Так как эти уровни привязаны к значению V_{CC} , они смещаются вверх и вниз при любых изменениях V_{CC} . Таким образом, при проектировании устройств на ЭСЛ-

схемах с положительным напряжением питания особенно пристального внимания требует разводка питания, чтобы предотвратить появление помех на шине V_{CC} , искажающих логические уровни сигналов, передаваемых и получаемых этими схемами.

Вспомним теперь, что с выходов ЭСЛ-схем можно снимать дифференциальные сигналы, а у самих схем могут быть дифференциальные входы. Дифференциальный вход относительно слабо реагирует на напряжение, одновременно действующее на обоих входах, и чувствителен только к разности напряжений на них. Поэтому при использовании ЭСЛ-схем с положительным напряжением питания для ослабления влияния помех, упомянутых в предыдущем абзаце, довольно эффективно можно применять дифференциальные сигналы.

Вполне естественно обеспечить КМОП-схемы дифференциальными входами и выходами, совместимыми с ЭСЛ-схемами с положительным питанием, позволяя тем самым осуществлять прямое сопряжение между КМОП-схемами и такими устройствами, как волоконно-оптический приемопередатчик, которому требуются уровни сигналов ЭСЛ-схем с положительным или отрицательным напряжением питания. Сегодня, когда КМОП-схемы переходят на 3.3-вольтовое питание, стало возможным создание ЭСЛ-подобных дифференциальных входов и выходов, у которых логические уровни привязаны к напряжению питания 3.3 В, а не к 5 В.

Обзор литературы

Студенты, которым необходимо изучить основы, могут обратиться к *Обзору электрических цепей* Флейшера (Bruce M. Fleischer. *Electrical Circuits Review*). Это 20-страничное учебное пособие содержит все начальные понятия цепей, которые используются в данной главе. Оно доступно как в виде приложения в первом издании этой книги, так и в виде pdf-файла на Web-странице этой книги на сайте www.ddpp.com.

За потрясающими успехами цифровой электроники в течение нескольких последних десятилетий не следует терять из виду, что логические схемы занимали важное место в технологиях, существовавших до появления транзисторов. В главе 5 *Введения в методологию переключающих схем* Клира (George J. Klir. *Introduction to the Methodology of Switching Circuits*. Van Nostrand, 1972) показано, как можно (и как можно было) реализовать логику на основе различных физических устройств, включая ртуть, электронные лампы и пневмосистемы.

Если вы интересуетесь историческими подробностями, то прекрасным введением во все первые семейства биполярных логических схем может служить *Разработка логических устройств на интегральных микросхемах* Ункаса (William E. Wickes. *Logic Design with Integrated Circuits*. Wiley-Interscience, 1968). Классическим введением в электрические характеристики ТТЛ-схем является *Справочник по применению ТТЛ-схем* под редакцией Алфке и Ларсена (The TTL Applications Handbook, edited by Peter Alfke and Ih Larsen. Fairchild Semiconductor, 1973). Первые разработчики логических устройств охотно пользовались также *Руководством по ТТЛ-схемам* Ланкастера (Don Lancaster. *The TTL Cookbook*).

В отношении других материалов по электронике, представленных в этой главе, вы можете обратиться почти к любому современному учебнику по электронике. Многие из них содержат значительно более подробное теоретическое рассмотрение работы цифровых схем; см., например, *Микроэлектронику* Миллмана и Грабела (J. Millman and A. Grabel. *Microelectronics*, second edition. McGraw-Hill, 1987). Другое хорошее введение в ИС и основные логические семейства можно найти в книге *Проектирование систем на основе СБИС* Мугоры (Saburo Muroga. *VLSI System Design*. Wiley, 1982). В частности, имеются две хорошие книги по МОП- и КМОП-схемам: *Введение в системы на основе СБИС* Меда и Конвея (Carver Mead and Lynn Conway. *Introduction to VLSI Systems*. Addison-Wesley, 1980) и *Принципы проектирования КМОП СБИС* Весте и Эшрагьяна (Neil H. E. Weste and Kamran Eshraghian. *Principles of CMOS VLSI Design*. Addison-Wesley, 1993).

Веселое и легко читаемое введение в цифровые схемы можно найти в книге *Ритмы булевого буги-вуги* Максфилда (Clive Maxfield. *Beep to the Boolean Boogie*. LLN Technology Publishing, www.lln-publishing.com). Некоторые думают, что рецепт приготовления морепродукта краба, приведенный в Приложении Н, — это все, что есть стоящего в данной книге! Однако даже без этого рецепта книга является прекрасно иллюстрированным классическим произведением, знакомящим вас с основными принципами цифровой электроники, с ее компонентами и их работой.

Хорошее понимание электрических аспектов работы цифровой схемы является обязательным для успешного конструирования быстродействующих устройств. Бесспорно лучшей книгой на эту тему является *Проектирование быстродействующих цифровых устройств: Справочник черной магии* Джонсона и Грэхема (Howard Johnson and Martin Graham. *High-Speed Digital Design: A Handbook of Black Magic*. Prentice Hall, 1993). В ней фундаментальные принципы электроники объединены с потрясающей интуицией и опытом практического конструирования цифровых систем.

Характеристики сегодняшних семейств логических схем можно найти в справочниках, издаваемых производителями устройств. Исчерпывающие справочники по ТТЛ- и КМОП-схемам издаваемые фирмами Texas Instruments и Motorola, перечислены в табл. 3.13. Оба эти производителя хранят последние версии своих справочников в Интернете на сайтах www.ti.com и www.mot.com. Фирма Motorola предоставляет также хорошее введение в проектирование систем на основе ЭСЛ-схем в виде *Справочника по проектированию систем на основе ЭСЛ-схем* фирмы Motorola (*MECL System Design Handbook*, publ. HB205, rev. 1, 1988; MECL — Motorola Emitter-Coupled Logic).

Стандарты на цифровые логические уровни, утвержденные Объединенным техническим советом по электронным приборам JEDEC (Joint Electronic Device Engineering Council), можно найти на Web-сайте www.jedec.org. Стандарты JEDEC для 3.3-вольтовой, 2.5-вольтовой и 1.8-вольтовой логики были изданы соответственно в 1994, 1995 и 1997 годах.

Табл. 3.13. Справочники производителей логических схем

Производитель	Порядковый номер	Типы микросхем	Название	Год
Texas Instruments	SDLD001	74, 74S, 74LS TTL	<i>TTL Logic Data Book</i>	1988
Texas Instruments	SDAD001C	74AS, 74ALS TTL	<i>ALS/AS Logic Data Book</i>	1995
Texas Instruments	SDFD001B	74F TTL	<i>F Logic Data Book</i>	1994
Texas Instruments	SCLD001D	74HC, 74HCT CMOS	<i>HC/HCT Logic Data Book</i>	1997
Texas Instruments	SCAD001D	74AC, 74ACT CMOS	<i>AC/ACT Logic Data Book</i>	1997
Texas Instruments	SCLD003A	74AHC, 74AHCT CMOS	<i>AHC/AHCT Logic Data Book</i>	1997
Motorola	DL121/D	74F, 74LSTTL	<i>Fast and LSTTL Data</i>	1989
Motorola	DL129/D	74HC, 74HCT	<i>High-Speed CMOS Data</i>	1988
Motorola	DL138/D	74AC, 74ACT	<i>FACT Data</i>	1988
Motorola	DL122/D	10K ECL	<i>MECL Device Data</i>	1989

Упражнения

- 3.1. В некотором логическом семействе низкому уровню сигнала соответствуют напряжения от 0.0 В до 0.8 В, а высокому уровню — напряжения от 2.0 В до 3.3 В. Используя правила положительной логики, укажите уровень сигнала, соответствующий каждому из следующих напряжений:
 (a) 0.0 В (b) 3.0 В (c) 0.8 В (d) 1.9 В
 (e) 2.0 В (f) 5.0 В (g) -0.7 В (h) -3.0 В
- 3.2. Повторите упражнение 3.1, используя правила отрицательной логики.
- 3.3. Подумайте, чем логический буферный усилитель отличается от усилителя звуковых сигналов.
- 3.4. Является ли буферный усилитель эквивалентом 1-входового вентиля И или 1-входового вентиля ИЛИ?
- 3.5. Справедливо или нет следующее утверждение: для заданного набора входных сигналов сигнал на выходе схемы И-НЕ противоположен сигналу на выходе схемы ИЛИ-НЕ?
- 3.7. Какие транзисторы используются в КМОП-схемах?
- 3.8. (Только для увлеченных.) Нарисовать эквивалентную схему КМОП-инвертора, используя однополюсное реле на два положения.
- 3.9. Какая КМОП-схема заданной площади на поверхности кристалла кремния, вероятнее всего, будет более быстродействующей: схема И-НЕ или схема ИЛИ-НЕ?
- 3.10. Дайте определенное «коэффициенту объединения по входу» и «коэффициенту разветвления по выходу». Который из них вы должны вычислять?

- 3.11. Нарисуйте принципиальную схему, составьте таблицу, описывающую работу схемы, и приведите условное обозначение 3-входовой КМОП-схемы ИЛИ-НЕ, как на рис. 3.16.
- 3.12. Нарисуйте модель 2-входовой КМОП-схемы ИЛИ-НЕ на основе ключей, как на рис. 3.14, для всех четырех комбинаций входных сигналов.
- 3.13. Нарисуйте принципиальную схему, составьте таблицу, описывающую работу схемы, и приведите условное обозначение КМОП-схемы ИЛИ, как на рис. 3.19.
- 3.14. Какая из КМОП-схем содержит меньшее число транзисторов: инвертирующий вентиль или неинвертирующий вентиль?
- 3.15. Назовите и нарисуйте условные обозначения четырех различных 4-входовых КМОП-схем, в каждой из которых используется по 8 транзисторов.
- 3.16. На рис. X3.16(a) представлена КМОП-схема И-ИЛИ-НЕ. Составьте таблицу, описывающую работу этой схемы, как на рис. 3.15(b), и нарисуйте соответствующую принципиальную схему, используя вентили И, ИЛИ и инверторы.

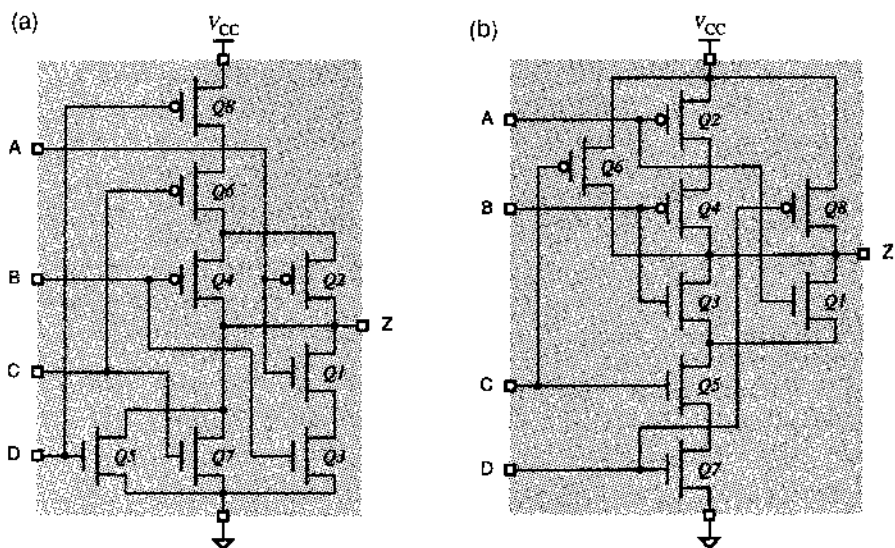


Рис. X3.16.

- 3.17. На рис. X3.16(b) представлена КМОП-схема ИЛИ-И-НЕ. Составьте таблицу, описывающую работу этой схемы, как на рис. 3.15(b), и нарисуйте соответствующую принципиальную схему, используя вентили И, ИЛИ и инверторы.
- 3.18. Чем духи могут навредить разработчикам цифровых схем?
- 3.19. Каков запас помехоустойчивости по постоянному току КМОП-инвертора при высоком уровне сигнала, переходная характеристика которого в наихуд-

шем случае подобна характеристике, изображенной на рис. 3.25? Каков запас помехоустойчивости по постоянному току при низком уровне сигнала? (Воспользуйтесь стандартными значениями порогов для низкого и высокого уровней, равными 1.5 В и 3.5 В.)

- 3.20. Используя справочные данные из табл. 3.3, определите в наихудшем случае запас помехоустойчивости по постоянному току для схемы 74НС00 при низком и высоком уровнях сигнала. Сформулируйте предположения, которые необходимы вам для ответа на этот вопрос.
- 3.21. В параграфе 3.5 даны определения семи различных электрических параметров для КМОП-схем. Используя справочные данные из табл. 3.3, определите наихудшие значения каждого из этих параметров для схемы 74НС00. Сформулируйте предположения, которые необходимы вам для ответа на этот вопрос.
- 3.22. Основываясь на принятых в параграфе 3.4 правилах и определениях, решите, каким является выходной ток: вытекающим или втекающим, если его величина оказалась равной отрицательному числу?
- 3.23. Для каждой из следующих активных нагрузок определите, остаются ли выходные напряжения в пределах уровней, заданных техническими условиями для схемы гражданского применения 74НС00. (Обратитесь к табл. 3.3 и воспользуйтесь значениями $V_{OHmin} = 3.84$ В и $V_{CC} = 5.0$ В.)
 - (a) резистор с сопротивлением 120 Ом подключен к шине питания V_{CC}
 - (b) резистор с сопротивлением 270 Ом подключен к шине питания V_{CC} , а резистор с сопротивлением 330 Ом – к земле
 - (c) резистор с сопротивлением 1 кОм подключен к земле
 - (d) резисторы с сопротивлением 150 Ом подключены к шине питания V_{CC} и к земле
 - (e) резистор с сопротивлением 100 Ом подключен к шине питания V_{CC}
 - (f) резистор с сопротивлением 75 Ом подключен к шине питания V_{CC} , а резистор с сопротивлением 150 Ом – к земле
 - (g) резистор с сопротивлением 75 Ом подключен к шине питания V_{CC}
 - (h) резистор с сопротивлением 270 Ом подключен к шине питания V_{CC} , а резистор с сопротивлением 150 Ом – к земле.
- 3.24. При какой величине входного сигнала высокого уровня из диапазона допустимых значений от 2.0 В до 5.0 В схема 74НС00 будет потреблять наибольшую мощность. (Воспользуйтесь сведениями из табл. 3.3.)
- 3.25. Определите коэффициент разветвления по выходу для постоянного тока при высоком и низком уровнях сигнала на выходе схемы 74НС00, если она нагружена входами схем 74LS00 (Воспользуйтесь сведениями из табл. 3.3 и 3.12.)
- 3.26. Оцените сопротивления «открытых» *p*-канального и *n*-канального выходных транзисторов в схеме 74НС00, используя информацию из табл. 3.3.
- 3.27. При каких обстоятельствах не опасно оставлять неиспользуемые входы КМОП-схем ни к чему не подключенными?

- 3.28. Объясните смысл явления «защелкивания» и назовите обстоятельства, при которых оно происходит.
- 3.29. Объясните, почему неправильно размещать все развязывающие конденсаторы в одном углу печатной платы.
- 3.30. В каком случае важно подать руку другу?
- 3.31. Назовите две составляющие задержки КМОП-вентиля. На какую из них больше всего влияет емкость нагрузки?
- 3.32. Определите постоянную времени для каждой из следующих комбинаций сопротивления резистора и емкости конденсатора:
(a) $R = 100 \text{ Ом}$, $C = 50 \text{ нФ}$ (b) $R = 330 \text{ Ом}$, $C = 150 \text{ пФ}$
(c) $R = 1 \text{ кОм}$, $C = 30 \text{ нФ}$ (d) $R = 4.7 \text{ кОм}$, $C = 100 \text{ пФ}$
- 3.33. На какие параметры КМОП-схемы, кроме задержки, влияет емкость нагрузки?
- 3.34. Объясните, почему коэффициент разветвления по выходу для постоянного тока вообще не зависит от числа входов КМОП-схем, подключаемых к выходу КМОП-вентиля.
- 3.35. КМОП-схемы семейства 74VHC могут работать при напряжении питания 3.3 В. Насколько меньше потребляемая при этом мощность по сравнению со случаем, когда напряжение питания равно 5 В?
- 3.36. Пусть инвертор с триггером Шмитта имеет следующие параметры: $V_{ILmax} = 0.8 \text{ В}$, $V_{IHmin} = 2.0 \text{ В}$, $V_{T+} = 1.6 \text{ В}$ и $V_{T-} = 1.3 \text{ В}$. Какова ширина петли гистерезиса у этого инвертора?
- 3.37. Зачем схемы с тремя состояниями на выходе обычно создаются так, чтобы выключение происходило быстрее, чем включение?
- 3.38. Рассмотрите все «за» и «против» выбора больших или меньших значений сопротивления нагрузочного резистора, включаемого на выходе КМОП-схем с открытым стоком или на выходе TTL-схем с открытым коллектором.
- 3.39. Пусть для нормальной яркости свечения светодиода по нему в «открытом» состоянии должен протекать ток около 5 мА при падении напряжения примерно 2.0 В. Определите необходимое сопротивление резистора, если светодиод включен так, как показано на рис. 3.52.
- 3.40. Как изменится ответ в упражнении 3.39, если светодиод включен так, как показано на рис. 3.53(а)?
- 3.41. «Монтажное И» реализуется простым объединением двух выходов с открытым стоком или с открытым коллектором без прохождения сигнала через еще один уровень транзисторной схемы. В таком случае почему схема «монтажное И» может оказаться медленнее, чем микросхема, реализующая функцию И?

- 3.42. Какое из рассмотренных в этой главе семейств имеет большую нагрузочную способность: КМОП или ТТЛ?
- 3.43. Сформулируйте кратко различие между схемами НС и НСТ. То же самое сделайте для схем АС и АСТ.
- 3.44. Почему в число параметров схем FCT не включены величины, относящиеся к КМОП-нагрузке, например, V_{OLmaxC} , как это сделано для схем НСТ и АСТ?
- 3.45. Почему схемы FCT-Т потребляют меньшую мощность, чем схемы FCT?
- 3.46. Сколько диодов требуется для реализации n -выходовой диодной схемы И?
- 3.47. Справедливо или нет следующее утверждение: в ТТЛ-схеме ИЛИ-НЕ применяется диодная логика?
- 3.48. Какой из выходных токов ТТЛ-вентиля больше: втекающий или вытекающий?
- 3.49. Найдите максимальный коэффициент разветвления по выходу для каждого из следующих случаев нагрузки выхода ТТЛ-схемы несколькими ТТЛ-входами. Укажите также для каждого случая, насколько «избыточной» оказывается нагрузочная способность при низком или высоком уровнях.
- | | |
|---------------------------------|----------------------------------|
| (a) выход 74LS нагружен на 74LS | (b) выход 74LS нагружен на 74S |
| (c) выход 74S нагружен на 74AS | (d) выход 74F нагружен на 74S |
| (e) выход 74AS нагружен на 74AS | (f) выход 74AS нагружен на 74F |
| (g) выход 74ALS нагружен на 74F | (h) выход 74AS нагружен на 74ALS |
- 3.50. На каком резисторе рассеивается большая мощность: на том, который соединяет неиспользуемый вход ТТЛ-схемы ИЛИ-НЕ семейства LS с землей, или на том, который соединяет неиспользуемый вход ТТЛ-схемы И-НЕ семейства LS с шиной питания? В каждом случае используйте максимальное допустимое значение сопротивления резистора.
- 3.51. Какая схема, по вашему мнению, обладает большим быстродействием: ТТЛ-вентиль И или ТТЛ-вентиль И-ИЛИ-НЕ? Почему?
- 3.52. Опишите основное достоинство и основной недостаток транзисторов Шоттки в ТТЛ-схемах.
- 3.53. Используя справочные данные из табл. 3.12, определите запас помехоустойчивости по постоянному току схемы 74LS00 для наилучшего случая при низком и высоком уровнях.
- 3.54. В разделах 3.10.2 и 3.10.3 даны определения восьми различных параметров ТТЛ-схем. Используя справочные данные из табл. 3.12, найдите значение каждого из них для схемы 74LS00 в самом плохом случае.
- 3.55. Для каждой из перечисленных омических нагрузок определите, превышена ли нагрузочная способность схемы 74LS00 гражданского применения. (Обратитесь к табл. 3.12, и воспользуйтесь значениями $V_{OLmax} = 0.5$ В и $V_{CC} = 5.0$ В.)

- (а) резистор с сопротивлением 470 Ом подключен к шине питания V_{CC}
- (б) резистор с сопротивлением 330 Ом подключен к шине питания V_{CC} , а резистор с сопротивлением 470 Ом — к земле
- (с) резистор с сопротивлением 10 кОм подключен к земле
- (д) резисторы с сопротивлением 390 Ом подключены к шине питания V_{CC} и к земле
- (е) резистор с сопротивлением 600 Ом подключен к шине питания V_{CC}
- (ф) резисторы с сопротивлением 510 Ом подключены к шине питания V_{CC} и к земле
- (г) резистор с сопротивлением 4,7 кОм подключен к земле
- (х) резистор с сопротивлением 220 Ом подключен к шине питания V_{CC} , а резистор с сопротивлением 330 Ом к земле

3.56. Подсчитайте запас надежности по постоянному току при низком и высоком уровнях для каждого из следующих случаев, когда к выходу ТТЛ-схемы подключен вход ТТЛ-совместимой КМОП-схемы или наоборот:

- (а) выход 74НСТ нагружен на 74LS
- (б) выход 74VНСТ нагружен на 74AS
- (с) выход 74LS нагружен на 74НСТ
- (д) выход 74S нагружен на 74VНСТ

3.57. Подсчитайте максимальный коэффициент разветвления по выходу для каждого из следующих случаев, когда к выходу ТТЛ-совместимой КМОП-схемы подключены несколько входов ТТЛ-схем. Для каждого случая укажите также, насколько «избыточной» оказывается нагрузочная способность при низком или высоком уровнях.

- (а) выход 74НСТ нагружен на 74LS
- (б) выход 74НСТ нагружен на 74AS
- (с) выход 74VНСТ нагружен на 74AS
- (д) выход 74VНСТ нагружен на 74LS

3.58. Какое из рассмотренных в этой главе логических семейств имеет наименьшую динамическую рассеиваемую мощность при заданных значениях емкостной нагрузки и частоты переключения?

Задачи

3.59. Составьте схему из КМОП-транзисторов, эквивалентную схеме, приведенной на рис. Х3.59. (Указание: Потребуется только шесть транзисторов.)

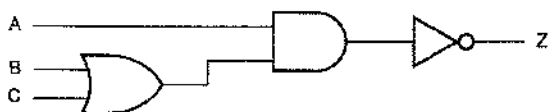


Рис. Х3.59.

3.60. Составьте схему из КМОП-транзисторов, которая будет вести себя так же, как схема на рис. Х3.60. (Указание: Потребуется только шесть транзисторов.)

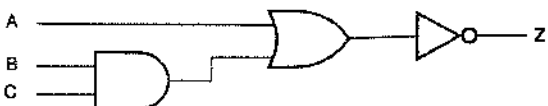


Рис. Х3.60.

- 3.61. Нарисуйте принципиальную схему, составьте таблицу, описывающую работу схемы, и приведите ее условное обозначение так, как это сделано на рис. 3.19, для КМОП-схемы с двумя входами A и B и выходом Z , у которой $Z = 1$, если $A = 0$ и $B = 1$, и $Z = 0$ в остальных случаях. (Указание: потребуется только шесть транзисторов.)
- 3.62. Нарисуйте принципиальную схему, составьте таблицу, описывающую работу схемы, и приведите ее условное обозначение так, как это сделано на рис. 3.19, для КМОП-схемы с двумя входами A и B и выходом Z , у которой $Z = 0$, если $A = 1$ и $B = 0$, и $Z = 1$ в остальных случаях. (Указание: потребуется только шесть транзисторов.)
- 3.63. Нарисуйте структурную схему 8-входовой КМОП-схемы ИЛИ-НЕ, считая, что в распоряжении имеются вентили не более чем с 4 входами. Пользуясь общим представлением о характеристиках КМОП-схем, выберите вариант схемы с минимальной задержкой распространения при заданной площади на поверхности кристалла кремния и объясните свой выбор.
- 3.64. Разработчики КМОП-схем, совместных с ТТЛ-схемами, по-видимому, могли сделать так, чтобы падение напряжения на «открытом» транзисторе — при наличии внешней нагрузки — было таким же малым при высоком уровне, как и при низком, просто увеличив размеры транзистора с p -каналом. Как вы считаете, почему они не потрудились это сделать?
- 3.65. Какой ток и какая мощность «тратятся впустую» в схеме на рис. 3.32(б)?
- 3.66. Проведите детальный расчет значения V_{OUT} в схемах на рис. 3.34 и 3.33. (Указание: Для каждой из схем воспользуйтесь эквивалентным представлением КМОП-инвертора по теореме Тевенна.)
- 3.67. Рассмотрите динамический режим работы выходной цепи КМОП-схемы с заданной емкостной нагрузкой. Если сопротивление цепи заряда вдвое больше сопротивления цепи разряда, то будет ли время нарастания точно вдвое превышать время спада? Если нет, то какие другие факторы влияют на время перехода?
- 3.68. Определите время спада на выходе КМОП-инвертора, изображенного на рис. 3.37, при $R_L = 1 \text{ кОм}$ и $V_L = 2.5 \text{ В}$. Объясните ваш результат и сравните его с результатом, полученным в разделе 3.6.1.
- 3.69. Повторите задачу 3.68 для времени нарастания.
- 3.70. Считая, что транзисторы в КМОП-буфере с тремя состояниями из семейства FCT являются идеальными переключающими устройствами с нулевой задержкой и порогом переключения 1.5 В , определите величину t_{PLZ} , если схема тестирования и форма сигналов такие, как указано на рис. 3.24. (Указание: Вы должны найти это время, воспользовавшись настоящей временной цепью.) Объясните различие между вашим результатом и значениями, приведенными в табл. 3.3.
- 3.71. Повторите задачу 3.70 для времени t_{PHZ} .

- 3.72. Используя данные из табл. 3.7, оцените сопротивления «открытых» p -канального и n -канального транзистора в КМОП-схеме серии 74VHC.
- 3.73. Постройте матрицу $4 \times 4 \times 2 \times 2$ запаса помехоустойчивости по постоянному току в худшем случае при низком и высоком уровнях для различных комбинаций сопряжения КМОП-схем, как показано на рис. X3.73: схемы серий HC, HCT, VHC и VHCT нагружены входами схем серий HC, HCT, VHC и VHCT при наличии КМОП- или ТТЛ-нагрузки. (Указание: Возможны 64 различных комбинации, но во многих случаях получается одинаковый результат; при некоторых комбинациях запас помехоустойчивости оказывается отрицательным.)

		Вход							
		HC		HCT		VHC		VHCT	
Выход	HC	CL	TL	CL	TL	CL	TL	CL	TL
	CH	TH	CH	TH	CH	TH	CH	TH	TH
HCT	CL	TL	CL	TL	CL	TL	CL	TL	TL
	CH	TH	CH	TH	CH	TH	CH	TH	TH
VHC	CL	TL	CL	TL	CL	TL	CL	TL	TL
	CH	TH	CH	TH	CH	TH	CH	TH	TH
VHCT	CL	TL	CL	TL	CL	TL	CL	TL	TL
	CH	TH	CH	TH	CH	TH	CH	TH	TH

Пояснение

CL – КМОП-нагрузка, низкий уровень

CH – КМОП-нагрузка, высокий уровень

TL – ТТЛ-нагрузка, низкий уровень

TH – ТТЛ-нагрузка, высокий уровень

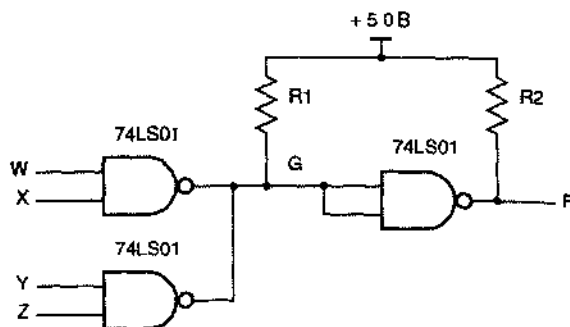
Рис. X3.73.

- 3.74. Воспользуйтесь рис. 3.85 и определите запас помехоустойчивости по постоянному току, когда к выходу 3.3-вольтовой КМОП-схемы, допускающей напряжение 5 В, подключена 5-вольтовая КМОП-схема с ТТЛ-уровнями на входе, и наоборот.
- 3.75. Воспользуйтесь рис. 3.85 и определите запас помехоустойчивости по постоянному току, когда к выходу 2.5-вольтовой КМОП-схемы, допускающей напряжение 3.3 В, подключена 3.3-вольтовая КМОП-схема, и наоборот.
- 3.76. Пусть в примере со светодиодом в разделе 3.7.5 разработчик выбрал сопротивление резистора равным 300 Ом и нашел, что на выходе схемы с открытым стоком при включенном светодиоде устанавливается напряжение 0.1 В. Чему равен ток, протекающий через светодиод, и какая мощность рассеивается на резисторе в этом случае?
- 3.77. Рассмотрите 8-разрядный двоичный КМОП-счетчик (параграф 8.4), на вход которого поступает сигнал с частотой 16 МГц. Что принять за частоту изменения младшего бита и какова частота изменения старшего бита при определении динамической рассеиваемой мощности? Каким значением частоты следует воспользоваться при определении динамической рассеиваемой мощности всем 8-разрядным счетчиком?
- 3.78. Используя только схемы И и ИЛИ-НЕ, нарисуйте схему, реализующую ту же логическую функцию, что и схема на рис. 3.55

- 3.79. Оцените величину выходного напряжения в точке Z на рис. 3.56, считая, что КМОП-схемы принадлежат семейству НСТ.
- 3.80. Перерисуйте принципиальную схему КМОП-буфера с тремя состояниями, приведенную на рис. 3.48, используя вместо условных обозначений схем И-НЕ, ИЛИ-НЕ и инвертора реальные транзисторы. Можете ли вы составить схему с меньшим числом транзисторов для реализации той же самой функции? Если можете, то нарисуйте ее.
- 3.81. Измените схему КМОП-буфера с тремя состояниями, показанную на рис. 3.48, так, чтобы выход находился в высокоомном состоянии, когда на входе разрешения присутствует высокий уровень. Измененная схема должна содержать не больше транзисторов, чем исходная.
- 3.82. Используя информацию из табл. 3.3, оцените, какой ток может протекать по каждому выходному контакту двух различных схем 74НС00, участвующих в «борьбе».
- 3.83. Покажите, что при задании напряжения питания значение тока I_{CCD} для семейства FCT можно получить, зная параметр C_{PD} для семейств НСТ/АСТ, и наоборот.
- 3.84. Может ли величина V_C в схеме на рис. 3.65(b) равняться 5.2 В, если каждая из величин V_Z и V_B равна 4.6 В? Объясните свой ответ.
- 3.85. Измените программу, приведенную в табл. 3.10, так, чтобы учесть ток утечки транзистора в «закрытом» состоянии.
- 3.86. Считая условия «идеальными», найдите минимальное напряжение, воспринимаемое ТТЛ-вентилем И-НЕ, изображенным на рис. 3.75, как высокий уровень, если на одном его входе присутствует низкий уровень, а на другом — высокий.
- 3.87. Считая условия «идеальными», найдите максимальное напряжение, воспринимаемое ТТЛ-вентилем И-НЕ, изображенным на рис. 3.75, как низкий уровень, если на обоих его входах присутствует низкий уровень.
- 3.88. Найдите ТТЛ-схему гражданского применения, которая может быть источником тока 40 мА при высоком уровне на выходе. Где она применяется?
- 3.89. Что произойдет, если вы попытаетесь заземлить катод светодиода, а его анод соединить с выходом обычной ТТЛ-схемы подобно тому, как это показано на рис. 3.53(b) для КМОП-схемы?
- 3.90. Что произойдет, если вы попытаетесь подключить 12-вольтное реле к выходу обычной ТТЛ-схемы?
- 3.91. Предположим, что единственный резистор, подключенный к источнику питания +5 В, обеспечивает логическую 1 на 15 различных входах схем 74LS00. Чему равно максимальное сопротивление этого резистора? Каков в этом случае запас помехоустойчивости по постоянному току при высоком уровне?

- 3.92. В схеме на рис. X3.92 вентили И-НЕ с открытым коллектором применяются для реализации «монтажной логики». Составьте таблицу истинности для выходного сигнала F и, если вы уже прочли параграф 4.2, запишите функцию, выражающую зависимость F от входных сигналов.

Рис. X3.92.



- 3.93. Каково максимальное допустимое сопротивление резистора $R1$ на рис. X3.92? Предположите, что требуемый запас помехоустойчивости при высоком уровне составляет 0.7 В. Параметры схемы 74LS01 возьмите из табл. 3.11, где они приведены в столбце 74LS, за исключением тока утечки $I_{\text{ОНmax}}$, протекающего в выходную цепь при высоком уровне, который примите равным 100 мкА.
- 3.94. Предположим, что выходной сигнал F на рис. X3.92 подан на входы двух инверторов 74S04. Определите минимальное и максимальное допустимое сопротивление резистора $R2$, предполагая, что требуемый запас помехоустойчивости при высоком уровне равен 0.7 В.
- 3.95. Разработчик логического устройства обнаружил ошибку в работе некоторой схемы после того, как она была подготовлена к производству и было изготовлено 1000 экземпляров. Часть этой схемы изображена на рис. X3.95 черным цветом; все вентили И-НЕ представляют собой элементы микросхемы 74LS00. Конструктор устранил неисправность, добавив два диода, указанных на рисунке синим цветом. Что делают эти диоды? Опишите влияние этих диодов на логику работы схемы и объясните, как изменится при этом запас помехоустойчивости.
- 3.96. Буфер 74LS125 является схемой с тремя состояниями. При разрешающем сигнале на управляющем входе в выходную цепь при низком уровне может течь ток 24 мА, а при высоком уровне выход может служить источником тока 2.6 мА. При запрещающем сигнале на управляющем входе, ток утечки выходной цепи составляет ± 20 мкА (знак зависит от напряжения на выходе: плюс соответствует случаю, когда потенциал выхода поддерживается другими устройствами на высоком уровне, а минус — случаю, когда на выходе низкий уровень). Предположим, что разработанная система имеет много модулей, подключенных к одной шине, и каждый модуль содержит один вентиль 74LS125 для формирования сигнала на шине и один вентиль 74LS04

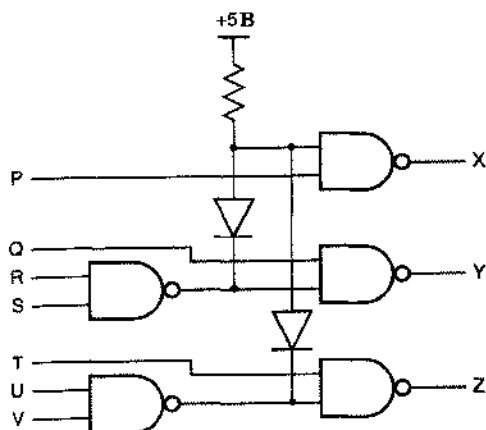


Рис. ХЗ.95.

для приема информации с шины. Какое максимальное число модулей можно подключить к шине, чтобы не превысить предельные значения параметров схемы 74LS125?

- 3.97. Повторите задачу 3.96, считая на этот раз, что между сигнальной шиной и шиной питания +5 В включен резистор для поддержания на сигнальной шине высокого уровня, когда никакое устройство этого не делает. Определите наибольшее возможное сопротивление резистора и максимальное число модулей, которые можно подключить к шине.
- 3.98. Найдите в справочнике по ТТЛ-схемам принципиальную схему реального вентиля с тремя состояниями и объясните, как она работает.
- 3.99. К шине подключены схемы с тремя состояниями или с открытым коллектором; оконечная нагрузка шины имеет вид, показанный на рис. ХЗ.99(а). Идея состоит в том, что соответствующим выбором сопротивлений резисторов $R1$ и $R2$ разработчик может получить любые требуемые значения V и R в эквивалентной схеме нагрузки, приведенной на рис. ХЗ.99(б). величиной V определяется напряжение на шине, когда все устройства находятся в неактивном состоянии, а сопротивление R выбирается равным волновому сопротивлению шины, чтобы избежать отражений в линии передачи (см. параграф 11.4). Для каждой из следующих пар значений V и R определите необходимые сопротивления резисторов $R1$ и $R2$:
- (а) $V = 2.15$ В, $R = 148.5$ Ом (б) $V = 2.7$ В, $R = 180$ Ом
 (с) $V = 3.0$ В, $R = 130$ Ом (д) $V = 2.5$ В, $R = 75$ Ом
- 3.100. Для каждой пары значений $R1$ и $R2$, найденных в задаче 3.99, определите, может ли схема с тремя состояниями, принадлежащая одному из семейств 74LS, 74S и 74FCT-T, обеспечить необходимые логические уровни. Для надежной работы токи I_{OL} и I_{OH} не должны выходить за пределы, указанные в технических характеристиках, при $V_{OL} = V_{OLmax}$ и $V_{OH} = V_{OHmin}$ соответственно.

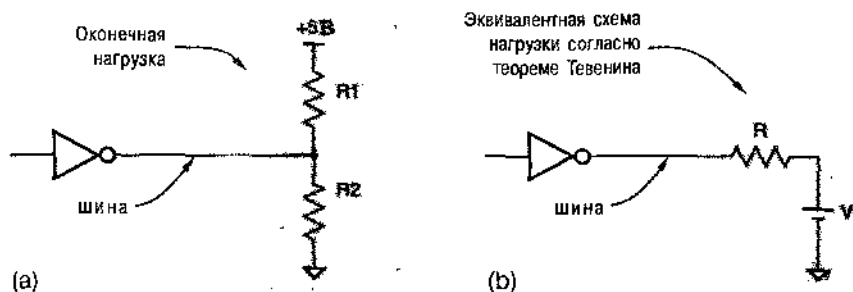
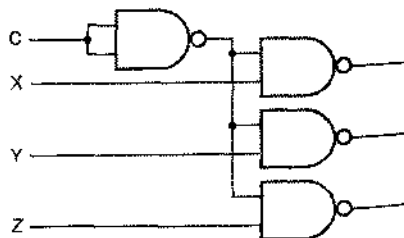
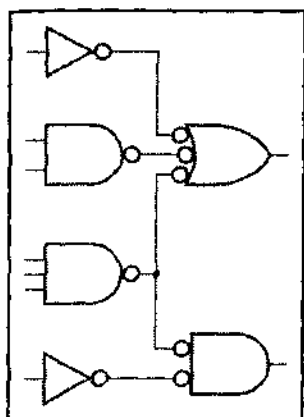


Рис. X3.99.

- 3.101. Используя графики из справочника по ТТЛ-схемам, сформулируйте практические правила определения того, как изменяется максимальная задержка распространения ТТЛ-схем семейства LS при неоптимальных условиях в зависимости от напряжения питания, температуры и нагрузок.
- 3.102. Определите зависимость полной мощности, потребляемой схемой, приведенной на рис. X3.102, от частоты переключения f в двух случаях: (а) используются схемы 74LS; (б) используются схемы 74НС. Предположите, что входная емкость ТТЛ-вспятия равна 3 пФ, входная емкость КМОП-вентиля равна 7 пФ, а для расчета мощности, рассеиваемой внутри схем серии 74LS, следует взять емкость, равную 20 пФ. Кроме того, пусть в схеме имеется паразитная емкость монтажа, равная 20 пФ. Предположите также, что на входах X, Y и Z всегда присутствует сигнал высокого уровня, а на вход С поступают прямоугольные импульсы частоты f с уровнями КМОП-сигналов. Другую информацию, необходимую для решения этой задачи, можно найти в табл. 3.5 и 3.11. Сформулируйте другие предположения, которые помогут вам добиться успеха. Начиная с какой частоты ТТЛ-схема рассеивает меньшую мощность, чем КМОП-схема?

Рис. X3.102.





ПРИНЦИПЫ ПРОЕКТИРОВАНИЯ КОМБИНАЦИОННЫХ ЛОГИЧЕСКИХ СХЕМ

Логические схемы подразделяются на два класса: «комбинационные» и «последовательностные». *Комбинационной* является такая логическая схема, сигналы на выходах которой зависят только от текущих значений входных сигналов. Барабанный переключатель каналов у старых телевизоров подобен комбинационной схеме: номер выбранного канала, служащий его «выходом», определяется только текущим положением ручки переключателя, играющим роль «входа».

Сигналы на выходах *последовательностной* логической схемы определяются не только текущими значениями входных сигналов, но зависят также от последовательности значений входных сигналов в прошлом, причем возможно, что эта зависимость простирается сколь угодно далеко во времени. Переключатель каналов телевизора или видеомagnetofона, управляемый нажатием кнопок «вверх» и «вниз», является последовательностным устройством: выбор канала зависит от последовательности нажатий вверх/вниз, имевших место в прошлом, по крайней мере, с того момента, когда вы включили телевизор 10 часов назад, а, возможно, и с еще более давнего времени, когда ваш аппарат был включен впервые. Последовательностные схемы рассматриваются в главах 7 и 8.

Комбинационная схема может состоять из произвольного числа логических вентилей и инверторов, но в ней нет обратных связей. Под *обратной связью* понимают наличие в схеме пути, по которому сигнал с выхода вентилей может пройти на вход того же вентилей; в общем случае такие петли обратной связи делают схему последовательностной.

Анализ комбинационного устройства мы начнем с логической схемы и перейдем к формальному описанию реализуемой ею функции; таким формальным описанием будут таблица истинности или логическое выражение. При *синтезе* мы будем действовать в обратном порядке: исходя из формального описания, мы получим логическую схему.

У комбинационных схем может быть один или большее число выходов. Большинство методов анализа и синтеза очевидным образом можно перенести со случая однопроводного выхода на схему со многими выходами (например: «Повторить эти шаги для каждого выхода»). Мы укажем также, как некоторые приемы распространяются на случай многих выходов не столь очевидным образом, позволяя достичь большей эффективности.

СИНТЕЗ И ПРОЕКТИРОВАНИЕ

Синтез является составной частью проектирования логических схем, поскольку решение реальной задачи проектирования начинается с неформального (словесного или мысленного) описания схемы. Обычно при проектировании самой трудной частью работы, требующей творческого подхода, является формализация описания схемы, состоящая в указании входных и выходных сигналов схемы и задании ее функционального поведения посредством таблиц истинности или уравнений. После того, как схема формально описана, обычно начинается рутинная процедура синтеза, цель которой состоит в получении логической схемы устройства, обеспечивающей требуемое функциональное поведение. Материал первых четырех параграфов этой главы является основой таких рутинных процедур независимо от того, выполняются они вручную или с помощью компьютера. В последних двух параграфах описываются реальные языки программирования ABEL и VHDL. При разработке проекта с использованием одного из этих языков действия, необходимые для синтеза, может выполнить за нас компьютерная программа. В последующих главах мы встретимся со многими примерами реального проектирования.

Цель этой главы заключается в том, чтобы вооружить вас солидной теоретической базой для анализа и синтеза комбинационных логических схем, причем база эта в дальнейшем окажется важной вдвойне, когда мы перейдем к последовательностным схемам. Большая часть процедур анализа и синтеза, рассматриваемых в этой главе, в настоящее время автоматизирована и является составной частью реализуемых с помощью компьютеров средств проектирования. Однако для того, чтобы воспользоваться этими средствами, необходимо понимать исходные принципы: только тогда вы сможете разобраться в том, что неправильно, если результаты окажутся неожиданными или нежелательными.

Овладев принципами, нетрудно понять, как можно представить и проанализировать комбинационные функции с помощью языков описания схем. Последние два параграфа этой главы знакомят с основными чертами языков ABEL и VHDL, которыми мы будем пользоваться для проектирования всевозможных логических схем повсюду в этой книге.

Прежде чем заняться комбинационными логическими схемами, необходимо познакомиться с алгеброй переключений, основным математическим аппаратом анализа и синтеза логических схем любого типа.

4.1. Алгебра переключений

Формальные методы анализа цифровых схем уходят своими корнями в работу английского математика Джорджа Буля. В 1854 году он ввел двузначную алгебраическую систему, называемую теперь *булевой алгеброй* (*Boolean algebra*) в качестве «средства для выражения... фундаментальных законов умозаключения на символическом языке Исчислений». С помощью этой системы философ, логик или обитатель планеты Вулкан могут высказывать истинные или ложные суждения, составлять из них новые высказывания и определять истинность или лож-

ность этих высказываний. Пусть, например, мы согласны с тем, что «Тот, кто не выучил данный материал, либо тупица, либо не энтузиаст» и «Конструктор компьютерной техпкпк занедомо не тупица»; тогда мы в состоянпп отвечать на вопросы типа: «Если вы фапат копструирования компьютеров, означает ли это, что вы уже изучили этот материал?»

Значительно позднее Буля, в 1938 году Клод Э. Шеннон показал, как приспособить булеву алгебру для описания поведения и анализа схем, составленных из реле, которые в то время были самыми распространеным логическими элементами. В алгебре переключений (*switching algebra*) Шеннопа состояние контакта реле — замкнутое или разомкнутое — представляется переменной X , которая может принимать одно из двух значений: 0 или 1. В современной цифровой технике эти два значения соответствуют широкому спектру физических состояний: высокое или низкое напряжение, включенный или выключенный свет, заряженный или разряженный конденсатор, разрушенная или нетропутая плавкая перемычка и т. д., как это было подробно описано в табл. 3.1.

Теперь мы перейдем непосредственно к алгебре переключений, имея в виду как основные принципы, так и все то, что мы уже знаем о поведении логических элементов (вентилей и инверторов). Дальнейшее историческое и математическое обсуждение этой темы можно найти в литературе.

4.1.1. Аксиомы

В алгебре переключений для представления значения логического сигнала используется символическая переменная, например X . В зависимости от технологии логический сигнал может иметь одно из двух значений: низкий или высокий уровень, «включено» или «выключено» и т. д. Мы говорим, что переменная X равна 0 для одного из этих значений и 1 — для другого.

Например, для КМОИ- и ТТЛ-схем, рассмотренных в главе 3, при *положительной логике* (*positive-logic convention*) значение «0» связывается с низким уровнем напряжения, а значение «1» — с высоким уровнем. При *отрицательной логике* (*negative-logic convention*) соотношение между значениями логической переменной и напряжениями противоположно: 0 — это высокий уровень, 1 — низкий. Однако выбор положительной или отрицательной логики не влияет на то, как мы алгебраически описываем поведение схемы; от этого выбора зависят только детали умозрительного перехода от физики к алгебре, как мы объясним это позднее при обсуждении вопроса о «двойственности». Пока же мы можем игнорировать физическую реализацию логических схем и полагать, что они непосредственно оперируют логическими символами 0 и 1.

Аксиомы (*axioms*) или *постулаты* (*postulates*) математической системы — это набор основных утверждений, про которые мы предполагаем, что они справедливы, и из которых можно вывести все другие свойства системы. Первые две аксиомы алгебры переключений устанавливают «цифровую абстракцию», формально утверждая, что переменная X может принимать только одно из двух значений:

$$(A1) \quad X = 0, \text{ если } X \neq 1; \quad (A1') \quad X = 1, \text{ если } X \neq 0.$$

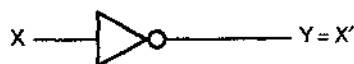
Обратите внимание, что мы приписываем эти аксиомы в паре, различие между аксиомами $A1$ и $A1'$ состоит лишь в перемене местами символов 0 и 1. Этим свойством обладают все аксиомы алгебры переключений, и это обстоятельство служит основанием принципа «двойственности», о котором пойдет речь позднее.

В разделе 3.3.3 мы рассмотрели конструкцию инвертора – логической схемы, у которой уровень сигнала на выходе противоположен [или является *дополнением* (*complement*) по отношению к] уровню сигнала на входе. Для обозначения инверсии мы используем символ «штрих» [*prime*, ($'$)]. Другими словами, в случае, когда переменная X означает сигнал на входе инвертора, X' является значением сигнала на его выходе. Формально эти обозначения сводятся ко второй паре аксиом:

(A2) Если $X=0$, то $X'=1$; (A2') Если $X=1$, то $X'=0$.

Как показано на рис. 4.1, сигнал на выходе инвертора с входным сигналом X можно обозначить как угодно, скажем, Y . Однако по правилам алгебры мы пишем: $Y=X'$, чтобы выразить тот факт, что «сигнал Y всегда имеет значение, противоположное сигналу X ». Штрих ($'$) служит *алгебраическим оператором* (*algebraic operator*), а X' является *выражением* (*expression*), которое можно прочесть так: « X штрих» или «НЕ X » (*operation* НЕ, *NOT operation*). Употребление этих терминов аналогично тому, чему учат в языках программирования, где говорят: если J – целая переменная, то J – это выражение, значение которого равно 0 – J . Хотя это может казаться мелочью, но все же необходимо четко различать имена сигналов (X, Y), выражения (X') и соотношения ($Y=X'$); это очень важно при изучении стандартов на документацию и программных средств логического проектирования. В нашей книге мы подчеркиваем это различие: на логических схемах имена сигналов набраны черным шрифтом, а выражения – красным.

Рис. 4.1. Названия сигналов и алгебраическое обозначение инверсии



В разделе 3.3.6 было показано, как строится 2-входовая КМОП-схема И, то есть схема, у которой сигнал на выходе равен 1 только в том случае, когда оба ее входных сигнала равны 1. Функцию, выполняемую 2-входовой схемой И, иногда называют *логическим умножением* (*logical multiplication*) и изображают алгебраическим знаком умножения – точкой [*multiplication dot*, ($\cdot$)]. Другими словами, значение сигнала на выходе схемы И с входными сигналами X и Y равно $X \cdot Y$, как показано на рис. 4.2(а). Некоторые авторы, особенно математики и логики, обозначают логическое умножение *знаком конъюнкции* (*wedge*): ($X \wedge Y$). Мы следуем обычной инженерной практике и употребляем точку: ($X \cdot Y$). Когда дело дойдет до языков описания схем, нам встретится несколько других символов для обозначения тех же самых вещей.

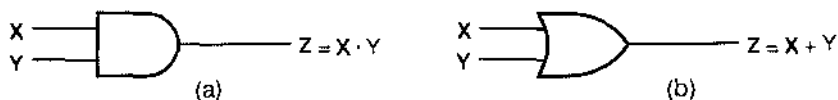


Рис. 4.2. Названия сигналов и алгебраические обозначения:
(а) схема И; (б) схема ИЛИ

ЗАМЕЧАНИЕ ОБ ОБОЗНАЧЕНИЯХ

Некоторые авторы дополнение к X обозначают также $\bar{X}$, $\sim X$ или $\neg X$. Надчеркивание, по-видимому, является самым распространенным и лучше всего выглядит в печатном тексте. Однако мы используем «штрих», чтобы для вас стало привычным записывать логические выражения в виде одной текстовой строки, не выходя за ее границы; это приучит вас также к тому, чтобы заключать в скобки сложные инвертируемые подвыражения: именно это вы должны будете делать, используя языки описания схем и другие средства.

В разделе 3.3.6 мы рассмотрели также структуру 2-входовой КМОП-схемы ИЛИ, то есть схемы, у которой выходной равен 1, если хотя бы на одном из ее входов имеется 1. Функцию, выполняемую 2-входовой схемой ИЛИ, иногда называют *логическим сложением* (*logical addition*) и изображают алгебраическим знаком «плюс» (+). Если на входах схемы ИЛИ действуют сигналы X и Y , то ее выходной сигнал равен $X + Y$, как показано на рис. 4.2(b). Некоторые авторы обозначают логическое сложение *знаком дизъюнкции* (*vee*): $(X \vee Y)$, но мы следуем обычной инженерной практике и используем знаком «или»: $(X + Y)$. Напомним еще раз, что в языках описания схем могут применяться и другие символы. Принято считать, что в логических выражениях, содержащих как умножение, так и сложение, операции умножения имеют *приоритет* (*precedence*) по отношению к операциям сложения точно так же, как это имеет место в целочисленных выражениях в обычных языках программирования. Другими словами, выражение $W \cdot X + Y \cdot Z$ эквивалентно выражению $(W \cdot X) + (Y \cdot Z)$.

Последними тремя парами аксиом даются формальные определения *операций И (AND operation)* и *ИЛИ (OR operation)* путем перечисления значений сигнала на выходе каждой из рассмотренных схем для всех возможных комбинаций входных сигналов:

И ВОТ ЕЩЕ ЧТО ...

В старых книжках логическое умножение обозначалось простой *записью рядом* [*juxtaposition*, (XY)], но мы так делать не будем. Запись рядом не вносит путаницы лишь в том случае, когда названия сигналов могут состоять только из одного символа. В противном случае как различить: является XY логическим произведением или двухбуквенным именем сигнала? В алгебре часто имена переменных состоят из одной буквы, но в задачах реального цифрового проектирования мы предпочитаем давать сигналам названия, состоящие из нескольких букв, которые несли бы смысловую нагрузку. Таким образом, нам необходим разделительный знак между именами, и роль такого знака как раз могла бы играть скорее точка, нежели пробел. В языках описания схем эквивалентом точки в качестве знака умножения служат символы $*$ или $\&$, и такой знак является обязательным при написании логической формулы на любом из этих языков.

$$\begin{aligned}
 (A3) \quad 0 \cdot 0 &= 0, & (A3') \quad 1 + 1 &= 1; \\
 (A4) \quad 1 \cdot 1 &= 1, & (A4') \quad 0 + 0 &= 0; \\
 (A5) \quad 0 \cdot 1 &= 1 \cdot 0 = 0, & (A5') \quad 1 + 0 &= 0 + 1 = 1.
 \end{aligned}$$

Пятью парами аксиом (A1–A5) и (A1'–A5') исчерпывается алгебра иереклосчений. Все другие сведения о системе можно вывести из этих аксиом, используя их в качестве отправной точки.

4.1.2. Теоремы о функциях одной переменной

Осуществляя анализ или синтез логической схемы, мы часто записываем алгебраические выражения, чтобы охарактеризовать фактическое или желаемое поведение схемы. Теоремы (theorems) алгебры иереклосчений представляют собой заведомо верные утверждения, которые позволяют нам преобразовывать алгебраические выражения, чтобы упростить анализ или более эффективно осуществить синтез соответствующей схемы. Например, теорема, утверждающая, что $X + 0 = X$, позволяет повсюду, где в выражении встретится $X + 0$, заменить эту сумму на X .

В табл. 4.1 перечислены все теоремы алгебры иереклосчений, касающиеся функций одной переменной X . Как узнать, что эти теоремы справедливы? Мы можем либо сами доказать их, либо поверить на слово тому, кто сделал это. Ну, ладно: поскольку это учебный курс, давайте поупражняемся в доказательствах.

Большинство теорем алгебры иереклосчений доказывается исключительно просто методом, носящим название *полной индукции* (perfect induction). В этом методе ключевой является аксиома A1: так как переменная в алгебре иереклосчений может принимать только два различных значения — 0 и 1, — теорему, касающуюся единственной переменной X , можно доказать, убедившись в ее справедливости для обоих значений X , то есть для $X = 0$ и для $X = 1$. Например, для доказательства теоремы T1 выполним две подстановки:

$$\begin{aligned}
 [X = 0] \quad 0 + 0 &= 0 & \text{— утверждение справедливо согласно аксиоме } A4'; \\
 [X = 1] \quad 1 + 0 &= 1 & \text{— утверждение справедливо согласно аксиоме } A5'.
 \end{aligned}$$

Все теоремы из табл. 4.1 можно доказать методом полной индукции, что мы и приглашаем вас сделать в упражнениях 4.2 и 4.3.

Табл. 4.1. Теоремы алгебры иереклосчений для функций одной переменной

(T1) $X + 0 = X$	(T1') $X \cdot 1 = X$	(Тождества)
(T2) $X + 1 = 1$	(T2') $X \cdot 0 = 0$	(Поглащающие элементы)
(T3) $X + X = X$	(T3') $X \cdot X = X$	(Идемпотентность)
(T4) $(X')' = X$		(Инволюция)
(T5) $X + X' = 1$	(T5') $X \cdot X' = 0$	(Дополнения)

4.1.3. Теоремы о функциях двух и трех переменных

Теоремы алгебры иереклосчений, относящиеся к функциям двух и трех переменных, перечислены в табл. 4.2. Каждая из теорем легко доказывается полной ин-

дукцией, то есть путем нахождения функции для четырех возможных комбинаций X и Y или для восьми возможных комбинаций X , Y и Z .

В первых двух парах теорем речь идет о коммутативности и ассоциативности логического сложения и логического умножения; эти утверждения идентичны коммутативному и ассоциативному правилам для сложения и умножения целых и действительных чисел. Эти теоремы утверждают, что наличие скобок и порядок членов в логической сумме и в логическом произведении несущественны. Например, со строго алгебраической точки зрения выражение типа $W \cdot X \cdot Y \cdot Z$ допускает неоднозначное толкование; его следует записывать в виде $(W \cdot (X \cdot (Y \cdot Z)))$ или $((W \cdot X) \cdot Y) \cdot Z$ или $(W \cdot X) \cdot (Y \cdot Z)$ (см. задачу 4.34). Но наши теоремы говорят, что неопределенность данного выражения по форме не страшна, поскольку в любом случае мы получаем один и тот же результат.

Табл. 4.2. Теоремы алгебры переключений для функций двух и трех переменных

(T6)	$X + Y = Y + X$	(T6')	$X \cdot Y = Y \cdot X$	(Коммутативность)
(T7)	$(X + Y) + Z = X + (Y + Z)$	(T7')	$(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$	(Ассоциативность)
(T8)	$X \cdot Y + X \cdot Z = X \cdot (Y + Z)$	(T8')	$(X + Y) \cdot (X + Z) = X + Y \cdot Z$	(Дистрибутивность)
(T9)	$X + X \cdot Y = X$	(T9')	$X \cdot (X + Y) = X$	(Поглощение)
(T10)	$X \cdot Y + X \cdot Y' = X$	(T10')	$(X + Y) \cdot (X + Y') = X$	(Объединение)
(T11)	$X \cdot Y + X' \cdot Z + Y \cdot Z = X \cdot Y + X' \cdot Z$			(Согласованность)
(T11')	$(X + Y) \cdot (X' + Z) \cdot (Y + Z) = (X + Y) \cdot (X' + Z)$			

Эти рассуждения кажутся тривиальными, и это действительно так, но они очень важны, так как образуют теоретическую базу для использования логических схем с числом входов больше двух. Мы ввели знаки $\cdot$ и $+$ как *двучленные операторы* (*binary operators*), то есть операторы, связывающие две переменные. Однако на практике мы используем логические схемы И и ИЛИ с тремя, четырьмя и большим числом входов. Из теорем следует, что входы логических схем можно подключать к источникам сигналов в любом порядке; эта возможность используется во многих программах разводки соединений на печатных платах и в группах специализированных ИС. Можно воспользоваться, в принципе, одним n -входовым вентиляем, либо двухвходовыми вентилями в количестве $(n - 1)$ штук, хотя задержка распространения и стоимость в последнем случае будут, вероятнее всего, больше.

Теорема T8 идентична дистрибутивному закону для целых и действительных чисел, то есть логическое умножение распределяется по компонентам логического сложения. Следовательно, можно «разнести множитель по слагаемым» («*multiplying out*») и представить выражение в виде суммы произведений, как это делается в следующем примере:

$$V \cdot (W + X) \cdot (Y + Z) = V \cdot W \cdot Y + V \cdot W \cdot Z + V \cdot X \cdot Y + V \cdot X \cdot Z.$$

Однако в алгебре переключений имеется и незнакомое правило, состоящее в том, что справедливо и обратное: логическое сложение разносится по компонентам логического умножения, о чем свидетельствует теорема T8'. Таким образом, можно также «разнести слагаемое по сомножителям» («*adding out*») и представить выражение в виде произведения сумм:

$$(V \cdot W \cdot X) + (Y \cdot Z) = (V + Y) \cdot (V + Z) \cdot (W + Y) \cdot (W + Z) \cdot (X + Y) \cdot (X + Z).$$

Теоремы Т9 и Т10 часто используются при минимизации логических функций. Если, например, в логическом выражении появляется подвыражение $X + X \cdot Y$, то согласно *теореме поглощения (covering theorem)* Т9 нам нужно оставить в выражении только X ; говорят, что X *поглощает (cover)* Y . Теорема объединения (*combining theorem*) Т10 говорит, что в случае, когда в выражении возникает подвыражение $X \cdot Y + X \cdot Y'$, мы можем заменить его на X . Поскольку Y должно быть либо нулем, либо единицей, рассматриваемое подвыражение равняется 1 в том и только в том случае, если X равняется 1.

Хотя теорему Т9 можно было бы доказать полной индукцией, ее справедливость проявится с большей очевидностью, если мы выведем ее из других теорем, принятых нами на вооружение к этому моменту:

$$\begin{aligned} X + X \cdot Y &= X \cdot 1 + X \cdot Y \quad (\text{согласно теореме Т1}) \\ &= X \cdot (1 + Y) \quad (\text{согласно теореме Т8}) \\ &= X \cdot 1 \quad (\text{согласно теореме Т2}) \\ &= X \quad (\text{согласно теореме Т1}). \end{aligned}$$

Подобным образом можно доказать теорему Т10, используя при этом другие теоремы, причем ключевую роль играет теорема Т8 при переписывании левой части в виде $X \cdot (Y + Y')$.

Теорема Т11 известна как *теорема согласованности (consensus theorem)*. Член $Y \cdot Z$ называют *консенсусом (consensus)* членов $X \cdot Y$ и $X' \cdot Z$. Идея состоит в том, что если $Y \cdot Z$ равняется 1, то либо $X \cdot Y$, либо $X' \cdot Z$ должны равняться 1; так как Y и Z оба равны 1 и либо X , либо X' должно быть единицей. Таким образом, член $Y \cdot Z$ является избыточным и может быть опущен в правой части Т11. У теоремы согласованности есть два важных приложения. Ее можно применить для устранения паразитных импульсов, возникающих в результате гонок в комбинационных логических схемах, как мы увидим это в параграфе 4.5. Кроме того, на этой теореме основан итеративно-консенсусный метод нахождения простых импликант (см. Обзор литературы).

Во всех этих теоремах любую переменную можно заменить произвольным логическим выражением. В простейшем случае на место одной или большего числа переменных можно поставить дополнение к ним:

$$(X + Y') + Z' = X + (Y' + Z') \quad (\text{по теореме Т7}).$$

Но можно выполнить также и более сложные подстановки:

$$(V' + X) \cdot (W \cdot (Y' + Z)) + (V' + X) \cdot (W \cdot (Y' + Z))' = V' + X \quad (\text{по теореме Т10}).$$

4.1.4. Теоремы о функциях n переменных

Несколько важных теорем, приведенных в табл. 4.3, справедливы для произвольного числа переменных n . Большинство этих теорем можно доказать двухшаговым методом, который называется *ограниченной индукцией (finite induction)*: сначала убеждаются в том, что утверждение теоремы справедливо при $n = 2$ [*основной шаг (basis step)*], а затем доказывают, что если утверждение теоремы верно при $n = i$, то оно выполняется также и при $n = i + 1$ [*шаг по индукции*].

(induction step)]. Рассмотрим, например, обобщенную теорему идемпотентности. При $n = 2$ теорема Т12 эквивалентна теореме Т3 и поэтому верна. Если она верна для суммы, в которую X входит i раз, то утверждение теоремы справедливо также для суммы из $i + 1$ величин X согласно следующему рассуждению:

$$\begin{aligned} X + X + X + \dots + X &= X + (X + X + \dots + X) && \text{(слева и справа } X \text{ входит } i + 1 \text{ раз)} \\ &= X + (X) && \text{(если теорема Т12 верна при } n = i) \\ &= X && \text{(согласно теореме Т3).} \end{aligned}$$

Таким образом, утверждение теоремы выполняется при всех конечных значениях n .

Табл. 4.3. Теоремы алгебры переключений для n переменных

(T12)	$X + X + \dots + X = X$	(Обобщенная идемпотентность)
(T12')	$X \cdot X \cdot \dots \cdot X = X$	
(T13)	$(X_1 \cdot X_2 \cdot \dots \cdot X_n)' = X_1' + X_2' + \dots + X_n'$	(Теоремы Де Моргана)
(T13')	$(X_1 + X_2 + \dots + X_n)' = X_1' \cdot X_2' \cdot \dots \cdot X_n'$	
(T14)	$[F(X_1, X_2, \dots, X_n, \dots)]' = F(X_1', X_2', \dots, X_n', \dots)$ (Обобщенная теорема Де Моргана)	
(T15)	$F(X_1, X_2, \dots, X_n) = X_1 \cdot F(1, X_2, \dots, X_n) + X_1' \cdot F(0, X_2, \dots, X_n)$	(Теоремы расширения Шеннона)
(T15')	$F(X_1, X_2, \dots, X_n) = [X_1 + F(0, X_2, \dots, X_n)] \cdot [X_1' + F(1, X_2, \dots, X_n)]$	

По-видимому чаще всего в алгебре переключений применяются теоремы Т13 и Т13', называемые *теоремами Де Моргана (DeMorgan's theorems)*. Теорема Т13 говорит, что в случае, если берется дополнение к сигналу на выходе n -входового вентиля И, то результат эквивалентен прохождению через n -входовой вентиль ИЛИ сигналов, являющихся дополнениями к сигналам на входах вентиль И. Другими словами, схемы на рис. 4.3(а) и (б) эквивалентны.

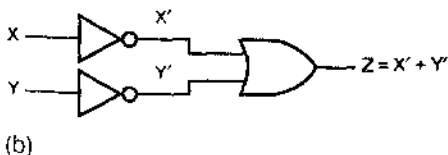
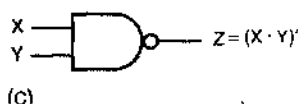
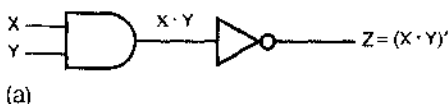


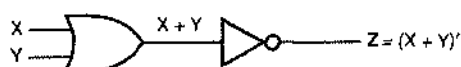
Рис. 4.3. Эквивалентные схемы согласно теореме Де Моргана Т13: (а) И-НЕ; (б) НЕ-ИЛИ; (с) обозначение вентиль И-НЕ; (д) изображение схемы, эквивалентной вентиль И-НЕ

В разделе 3.3.4 было объяснено, как устроена КМОП-схема И-НЕ. При любом наборе входных сигналов сигнал на выходе вентиль И-НЕ является дополнением к выходному сигналу вентиль И с теми же самыми входными сигналами, и поэтому вентиль И-НЕ можно обозначить так, как показано на рис. 4.3(с). Однако по своей конструкции КМОП-схема И-НЕ не представляет собой последовательного включения схемы И и транзисторного инвертора (схемы НЕ); вентиль И-НЕ – это просто

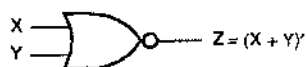
такое удачное объединение транзисторов, которое реализует функцию И-НЕ. Теорема Т13 говорит нам, что обозначение, приведенное на рис. 4.3(d), выражает ту же самую логическую функцию (кружочки на входах вентиля ИЛИ указывают на логическое инвертирование). Другими словами, можно считать, что схема И-НЕ выполняет функцию НЕ-ИЛИ.

Глядя на входы и выход схемы И-НЕ, нельзя решить, как именно эта схема устроена внутри: состоит ли она из последовательно включенных схемы И и инвертора, или из инверторов, за которыми следует схема ИЛИ, или непосредственно реализует требуемую функцию средствами КМОП-логики. Происходит это потому, что любая схема И-НЕ выполняет в точности одну и ту же логическую функцию. Хотя выбор обозначения не имеет никакого отношения к тому, что делает схема, мы покажем в параграфе 5.1, что подходящий выбор значительно облегчает понимание функции, выполняемой данной схемой.

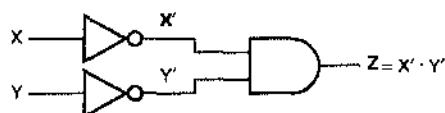
Подобную эквивалентность символических изображений можно вывести из теоремы Т13'. Как показано на рис. 4.4, функцию ИЛИ-НЕ можно реализовать, включая друг за другом вентиль ИЛИ и инвертор, либо пропустив сначала входные сигналы через инверторы и подавая их затем на вентиль И.



(a)



(c)



(b)



(d)

Рис. 4.4. Эквивалентные схемы согласно теореме Де Моргана Т13'. (a) ИЛИ-НЕ, (b) НЕ-И; (c) обозначение вентиля ИЛИ-НЕ, (d) изображение схемы, эквивалентной вентилю ИЛИ-НЕ

Теоремы Т13 и Т13' представляют собой частные случаи обобщенной теоремы Де Моргана (*generalized DeMorgan's theorem*) Т14, которая применима к произвольному логическому выражению F . По определению, дополнением логического выражения (*complement of a logic expression*), обозначаемым $(F)'$, является выражение, значение которого противоположно значению F для любых возможных комбинаций входных сигналов. Теорема Т14 очень важна, так как она дает нам способ преобразовывать и упрощать дополнения выражений.

Теорема Т14 утверждает, что дополнение заданного логического выражения с n переменными можно получить, меняя местами знак $+$ и $\cdot$ и заменяя все переменные их дополнениями. Пусть, например, имеется выражение:

$$\begin{aligned} F(W, X, Y, Z) &= (W' \cdot X) + (X \cdot Y) + [W \cdot (X' + Z')] \\ &= ((W')' \cdot X) + (X \cdot Y) + (W \cdot ((X')' + (Z')')). \end{aligned}$$

Во второй строке мы заключили дополнения переменных в скобки, чтобы напомнить, что «штрих» является оператором, а не частью имени переменной.

По теореме T14 получаем:

$$[F(W, X, Y, Z)]' = ((W')' + X') \cdot (X' + Y') \cdot (W' + ((X')' \cdot (Z')')).$$

Используя теорему T4, последнее выражение можно упростить:

$$[F(W, X, Y, Z)]' = (W + X') \cdot (X' + Y') \cdot (W' + (X \cdot Z)).$$

В самом общем случае теорема T14 позволяет находить дополнение заключенного в скобки выражения путем замены $+$ на $\cdot$ и наоборот, взятия дополнений переменных, не помеченных знаком дополнения ($'$), и опускания этого знака у переменных, дополнения которых фигурируют в исходном выражении.

Обобщенную теорему Де Моргана T14 можно доказать, убедившись в том, что все логические функции можно записать либо в виде суммы, либо в виде произведения подфункций, и применяя затем рекурсивно теоремы T13 и T13'. Однако более информативным и более убедительным является доказательство на основе принципа двойственности, который сейчас будет объяснен.

4.1.5. Двойственность

Все аксиомы алгебры переключений были сформулированы попарно. Каждая аксиома, помеченная штрихом (например, A5') получается из аксиомы, не помеченной штрихом (например, A5), в результате того, что 0 и 1 меняются местами, и то же самое происходит со знаками $\cdot$ и $+$, если таковые имеются. Таким образом, мы приходим к следующей *метатеореме (metatheorem)*, то есть к теореме о теоремах:

Принцип двойственности: Любая теорема или тождество алгебры переключений остаются справедливыми, если повсюду меняются местами 0 и 1 и знаки $\cdot$ и $+$.

Метатеорема верна, поскольку истинными являются все утверждения, двойственные по отношению к аксиомам; следовательно, все утверждения, двойственные по отношению к теоремам алгебры переключений, можно доказать, используя двойственные аксиомы.

В конце концов, что значат имена и символы, если уж на то пошло? Если бы в программном обеспечении, с помощью которого была набрана эта книга, была ошибка, в результате чего повсюду в этой главе 0 и 1 и знаки $+$ и $\cdot$ поменялись бы местами, то вы научились бы той же самой алгебре переключений; разве что терминология окажется при этом немного странной, потому что слова типа «произведение» будут употребляться для описания операций со знаком « $+$ ».

Двойственность важна, потому что она удваивает полезность всего, что вы узнаете об алгебре переключений и о преобразованиях функций, относящихся к переключениям. Практическая ценность этого, со студенческой точки зрения, состоит в том, что учить надо только половину всего необходимого! Например, выучив однажды, как по выражениям вида «сумма произведений» синтезировать двухуровневые логические схемы И–ИЛИ, вы автоматически осваиваете двойственный метод синтеза схем ИЛИ–И по выражениям вида «произведение сумм».

В алгебре переключений существует все же одно принятое условие, когда знаки $\cdot$ и $+$ не считаются равноправными, так что принцип двойственности не обя-

зательно оказывается верным. Можете догадаться, о чем идет речь до того, как прочитаете приведенный ниже ответ на этот вопрос? Рассмотрите утверждение теоремы Т9 и вам станет ясной абсурдность «двойственного утверждения»:

$$X + X \cdot Y = X \quad (\text{теорема Т9}),$$

$$X \cdot X + Y = X \quad (\text{в результате применения принципа двойственности}),$$

$$X + Y = X \quad (\text{согласно теореме Т3}).$$

Очевидно, что последняя строка в этих выкладках не верна. Где мы ноступили неправильно? Проблема заключается в старшинстве операторов. Мы могли занисать левую часть равенства в первой строке без скобок согласно условию, что у оператора $\cdot$ более высокий приоритет. Но коль скоро мы применяем принцип двойственности, нам следовало бы вместо этого назначить больший приоритет оператору $+$ или записать вторую строку в виде: $X \cdot (X + Y) = X$. Лучший способ избегать подобных затруднений состоит в том, чтобы до применения принципа двойственности записывать исходное выражение со скобками.

Давайте дадим формальное определение *двойственному логическому выражению* (*dual of a logic expression*). Если $F(X_1, X_2, \dots, X_n, +, \cdot, ')$ логическое выражение, содержащее переменные $X_1, X_2, \dots, X_n$ и операторы $+$, $\cdot$ и $'$, и такое, что в нем проставлены все необходимые скобки, то выражением, двойственным по отношению к F , которое мы будем обозначать F^D , является то же самое выражение при условии, что знаки $+$ и $\cdot$ поменяны местами:

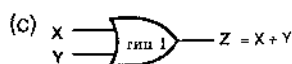
$$F^D(X_1, X_2, \dots, X_n, +, \cdot, ') = F(X_1, X_2, \dots, X_n, \cdot, +, ').$$

Вы уже знаете это, конечно, но мы специально привели данное определение в такой форме, чтобы подчеркнуть близость двойственности и обобщенной теоремы Де Моргана Т14, утверждение которой мы можем теперь представить в виде:

$$[F(X_1, X_2, \dots, X_n)]' = F^D(X_1', X_2', \dots, X_n').$$

Давайте проверим справедливость этого утверждения на примере физической цепи.

На рис. 4.5(а) приведена таблица, описывающая функцию, реализуемую логическим элементом, с точки зрения электрических потенциалов на его входах и выходе; этот логический элемент назовем просто «вентилем 1-го типа».



X	Y	Z
LOW	LOW	LOW
LOW	HIGH	LOW
HIGH	LOW	LOW
HIGH	HIGH	HIGH

X	Y	Z
0	0	0
0	1	0
1	0	0
1	1	1

X	Y	Z
1	1	1
1	0	1
0	1	1
0	0	0

Рис. 4.5. Логический «вентиль 1-го типа»: (а) таблица, описывающая работу схемы с точки зрения уровней напряжения на входах и выходе; (б) таблица с логическими значениями на входах и выходе и условное обозначение в положительной логике; (с) таблица с логическими значениями на входах и выходе и условное обозначение в отрицательной логике

В положительной логике [низкий уровень (LOW) соответствует 0, высокий уровень (HIGH) – 1] – это вентиль И, но в отрицательной логике (LOW = 1, HIGH = 0) – это вентиль ИЛИ, как это видно из рис. (b) и (c). Можно представить себе также некий «вентиль 2-го типа» (рис. 4.6), который в положительной логике является схемой ИЛИ, а в отрицательной логике – схемой И. Подобные таблицы можно составить для вентилях с числом входов больше двух.

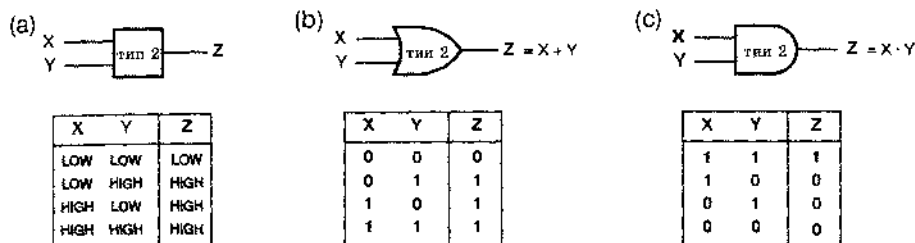


Рис. 4.6. Логический «вентиль 2-го типа»: (a) таблица, описывающая работу схемы с точки зрения уровней напряжения на входах и выходе; (b) таблица с логическими значениями на входах и выходе и условное обозначение в положительной логике; (c) таблица с логическими значениями на входах и выходе и условное обозначение в отрицательной логике

Предположим, что вам задано произвольное логическое выражение $F(X_1, X_2, \dots, X_n)$. Действуя по правилам положительной логики, вы можете построить схему, соответствующую этому выражению, используя инверторы для операций НЕ, вентиля 1-го типа для операций И и вентили 2-го типа для операций ИЛИ, как показано на рис. 4.7. Пусть теперь, не изменяя схемы, мы совершаем переход от положительной логики к отрицательной. Тогда мы должны будем заново нарисовать нашу схему так, как это сделано на рис. 4.8.

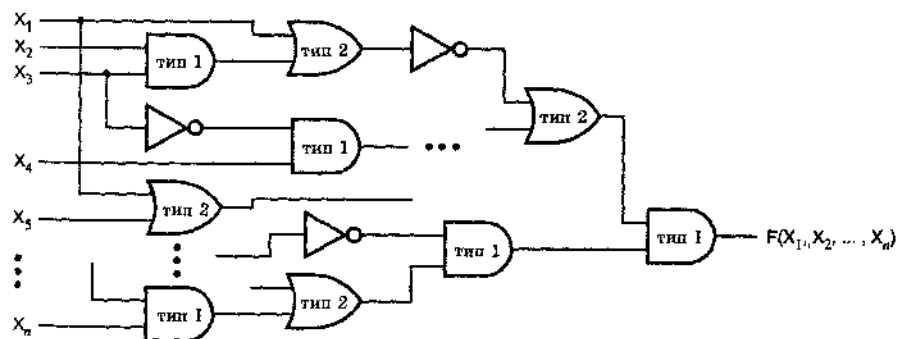


Рис. 4.7. Схема, реализующая некоторую логическую функцию с помощью инверторов и вентилях 1-го и 2-го типа по правилам положительной логики

Ясно, что при любой возможной комбинации входных напряжений (высоких и низких уровней) напряжение на выходе схемы останется тем же самым. Однако, с точки зрения алгебры переключений, значение выходного сигнала – 0 или 1 – во второй схеме противоположно тому, что имело место при положительной логике. Но точно так же противоположным становится и значение каждого вход-

ного сигнала. Поэтому для каждой возможной комбинации сигналов на входах схемы, приведенной на рис. 4.7, сигнал на ее выходе противоположен тому, что получится на выходе схемы, приведенной на рис. 4.8, когда сигналы на ее входах будут представлять собой комбинацию значений, противоположных входным сигналам схемы на рис. 4.7:

$$F(X_1, X_2, \dots, X_n) = [F^D(X_1', X_2', \dots, X_n')]'$$

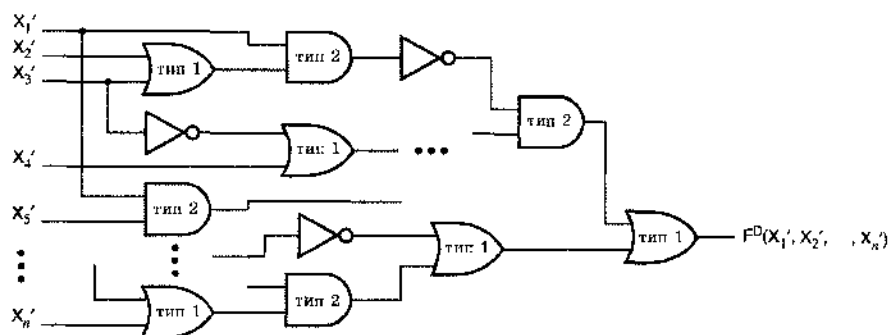


Рис. 4.8. Интерпретация предыдущей схемы в терминах отрицательной логики

Беря дополнение выражений по обе стороны равенства, мы получаем обобщенную теорему Де Моргана:

$$[F(X_1, X_2, \dots, X_n)]' = F^D(X_1', X_2', \dots, X_n').$$

Потрясающе!

ДВОЙСТВЕННОСТЬ – ЭТО ЗДОРОВО НЕ ТОЛЬКО ДЛЯ СТУДЕНТОВ, НО И ДЛЯ АВТОРОВ

Теперь вы видите, что двойственность является основой для обобщенной теоремы Де Моргана. Забегая вперед, скажем, что вы должны будете выучить половину всех методов преобразования и упрощения логических функций. Но это позволяет и мне сократить вдвое тот материал, который я должен здесь изложить.

4.1.6. Стандартные представления логических функций

Прежде чем перейти к анализу и синтезу комбинационных логических схем, мы введем необходимую терминологию и обозначения.

Основной формой представления логической функции является *таблица истинности (truth table)*. Этот совсем уж прямолинейный способ представления по своей идеологии подобен доказательству методом полной индукции и состоит в простом перечислении значений сигнала на выходе схемы для всех возможных комбинаций входных сигналов. По сложившейся традиции комбинации входных сигналов располагаются в виде строк в порядке возрастания эквивалентных им

двоичных чисел, а соответствующие значения выходного сигнала записываются столбиком справа от этих строк. В общем случае таблица истинности для функции 3-х переменных выглядит так, как это указано в табл. 4.4.

Строка	X	Y	Z	F
0	0	0	0	$F(0,0,0)$
1	0	0	1	$F(0,0,1)$
2	0	1	0	$F(0,1,0)$
3	0	1	1	$F(0,1,1)$
4	1	0	0	$F(1,0,0)$
5	1	0	1	$F(1,0,1)$
6	1	1	0	$F(1,1,0)$
7	1	1	1	$F(1,1,1)$

Табл. 4.4. Структура таблицы истинности для произвольной логической функции 3-х переменных $F(X,Y,Z)$

Строки пронумерованы числами от 0 до 7, соответствующими двоичной записи комбинаций входных сигналов, но эта нумерация не является существенной частью таблицы истинности. Примером таблицы истинности для конкретной логической функции 3-х переменных служит табл. 4.5. Разные комбинации нулей и единиц в правом столбце таблицы задают различные логические функции; всего имеется 2^8 таких комбинаций. Таким образом, логическая функция, приведенная в табл. 4.5, — это одна из 2^8 различных логических функций трех переменных

Строка	X	Y	Z	F
0	0	0	0	1
1	0	0	1	0
2	0	1	0	0
3	0	1	1	1
4	1	0	0	1
5	1	0	1	0
6	1	1	0	1
7	1	1	1	1

Табл. 4.5. Таблица истинности для конкретной логической функции 3-х переменных $F(X,Y,Z)$

В таблице истинности для логической функции n переменных имеется 2^n строк. Очевидно, что на практике таблицу истинности можно выписать только для логических функций с небольшим числом переменных: студент, скажем, мог бы составить таблицу для функции с числом переменных, равным 10, но для кого-то другого вполне хватило бы 4–5 переменных.

Содержащуюся в таблице истинности информацию можно выразить также алгебраическими средствами. Чтобы сделать это, нам, прежде всего, нужны некоторые определения:

- *Литерал (literal)* – это переменная или дополнение переменной. Примеры: X , Y, X', Y' .
- *Терм-произведение (product term)* – одиночный литерал или логическое произведение двух или более литералов. Примеры: Z' , $W \cdot X \cdot Y$, $X \cdot Y \cdot Z$, $W' \cdot Y \cdot Z$.
- *Сумма произведений (sum-of-products expression)* – выражение, представляющее собой логическую сумму термов-произведений. Пример: $Z' + W \cdot X \cdot Y + X \cdot Y \cdot Z + W' \cdot Y \cdot Z$.
- *Терм-сумма (sum term)* – одиночный литерал или логическая сумма двух или более литералов. Примеры: Z' , $W + X + Y$, $X + Y + Z$, $W + Y + Z$.
- *Произведение сумм (product-of-sums expression)* – выражение, представляющее собой логическое произведение термов-сумм. Пример: $Z' \cdot (W + X + Y) \cdot (X + Y + Z) \cdot (W + Y + Z)$.
- *Нормальный терм (normal term)* – это терм-произведение или терм-сумма, в которых нет переменных, встречающихся более одного раза. Терм, не являющийся нормальным, всегда можно упростить, сведя его к нормальному терму, воспользовавшись для этого одной из теорем Т3, Т3', Т5 или Т5'. Примеры термов, не являющихся нормальными: $W \cdot X \cdot X \cdot Y$, $W + W + X' + Y$, $X \cdot X' \cdot Y$. Примеры нормальных термов: $W \cdot X \cdot Y$, $W + X' + Y$.
- *Минтерм (minterm)* с n переменными – нормальный терм-произведение с n литералами. Существует 2^n таких термов-произведений. Примеры минтермов с 4-мя переменными: $W \cdot X' \cdot Y' \cdot Z'$, $W \cdot X' \cdot Y' \cdot Z$, $W \cdot X' \cdot Y \cdot Z'$.
- *Макстерм (maxterm)* с n переменными – нормальный терм-сумма с n литералами. Существует 2^n таких термов-сумм. Примеры макстермов с 4-мя переменными: $W' + X' + Y' + Z'$, $W + X' + Y' + Z$, $W' + X' + Y + Z'$.

Таблица истинности тесно связана с минтермами и макстермами. Минтерм можно определить как терм-произведение, равный 1 точно в одной строке таблицы истинности. Табл. 4.6 демонстрирует это соответствие на примере таблицы истинности для случая 3-х переменных.

Табл. 4.6. Минтермы и макстермы для логической функции 3-х переменных $F(X,Y,Z)$

Строка	X	Y	Z	F	Минтерм	Макстерм
0	0	0	0	$F(0,0,0)$	$X' \cdot Y' \cdot Z'$	$X + Y + Z$
1	0	0	1	$F(0,0,1)$	$X' \cdot Y' \cdot Z$	$X + Y + Z'$
2	0	1	0	$F(0,1,0)$	$X' \cdot Y \cdot Z'$	$X + Y' + Z$
3	0	1	1	$F(0,1,1)$	$X' \cdot Y \cdot Z$	$X + Y' + Z'$
4	1	0	0	$F(1,0,0)$	$X \cdot Y' \cdot Z'$	$X' + Y + Z$
5	1	0	1	$F(1,0,1)$	$X \cdot Y' \cdot Z$	$X' + Y + Z'$
6	1	1	0	$F(1,1,0)$	$X \cdot Y \cdot Z'$	$X' + Y' + Z$
7	1	1	1	$F(1,1,1)$	$X \cdot Y \cdot Z$	$X' + Y' + Z'$

Минтерм с n переменными можно представить в виде n -разрядного двоичного целого числа, играющего роль *номера минтерма* (*minterm number*). Мы будем называть минтерм, соответствующий i -й строке таблицы истинности, *минтермом i* (*minterm i*). Та или иная переменная входит в минтерм i в виде ее дополнения, если соответствующий бит в двоичном представлении числа i , равен 0; в противном случае эта переменная фигурирует в минтерме не в форме дополнения. Например, двоичное представление номера 5-й строки имеет вид 101, и поэтому минтерм записывается как $X \cdot Y' \cdot Z$. Как вы, наверное, уже догадались, в случае макстермов имеет место прямо противоположное соответствие: переменная входит в *макстерм i* (*maxterm i*) в форме ее дополнения, если соответствующий бит в двоичном представлении числа i равен 1. Таким образом, макстерм 5 (101) выглядит так: $X' + Y + Z'$. Заметьте, что все это имеет смысл только в том случае, когда нам известно число переменных в таблице истинности; в нашем примере оно равнялось трем.

Опираясь на соответствие между таблицей истинности и минтермами, легко по таблице истинности представить логическую функцию в алгебраической записи. Одна из форм такой записи — *каноническая сумма* (*canonical sum*) логической функции, то есть сумма минтермов, соответствующих тем строкам таблицы истинности (комбинациям входных сигналов), для которых значение функции (выходного сигнала) равно 1. Например, каноническая сумма для логической функции из табл. 4.5 имеет вид:

$$F = \sum_{X,Y,Z}(0, 3, 4, 6, 7) \\ = X' \cdot Y' \cdot Z' + X' \cdot Y \cdot Z + X \cdot Y' \cdot Z' + X \cdot Y \cdot Z' + X \cdot Y \cdot Z.$$

Здесь $\sum_{X,Y,Z}(0, 3, 4, 6, 7)$ — *список минтермов* (*minterm list*), означающий «сумму минтермов 0, 3, 4, 6 и 7 с переменными X, Y и Z ». Список минтермов называют также *множеством включений* (*on-set*) логической функции. Можно мысленно представить себе, что каждый из минтермов, входящих в это множество, «включает» выходной сигнал при одной вполне определенной комбинации входных сигналов. Любую логическую функцию можно записать в виде канонической суммы.

Каноническим произведением (*canonical product*) логической функции называется произведение макстермов, соответствующих тем комбинациям входных сигналов, для которых значение функции равно 0. Например, каноническое произведение логической функции из табл. 4.5 имеет вид

$$F = \prod_{X,Y,Z}(1, 2, 5) \\ = (X + Y + Z') \cdot (X + Y' + Z) \cdot (X' + Y + Z').$$

Здесь $\prod_{X,Y,Z}(1, 2, 5)$ — *список макстермов* (*maxterm list*), означающий «произведение макстермов 1, 2 и 5 с переменными X, Y и Z ». Список макстермов называют также *множеством выключений* (*off-set*) логической функции. Можно мысленно представить себе, что каждый из макстермов, входящих в это множество, «выключает» выходной сигнал при одной вполне определенной комбинации входных сигналов. Любую логическую функцию можно записать в виде канонического произведения.

Список минтермов легко преобразовать в список макстермов, и наоборот. Для функций n переменных возможные номера минтермов и макстермов принадлежат множеству $\{0, 1, \dots, 2^n - 1\}$; список минтермов и список макстермов содержат подмножества этих номеров. Чтобы перейти от одного списка к другому, нужно взять дополнение множества; например,

$$\begin{aligned}\Sigma_{A,B,C}(0, 1, 2, 3) &= \Pi_{A,B,C}(4, 5, 6, 7), \\ \Sigma_{X,Y}(1) &= \Pi_{X,Y}(0, 2, 3), \\ \Sigma_{W,X,Y,Z}(0, 1, 2, 3, 5, 7, 11, 13) &= \Pi_{W,X,Y,Z}(4, 6, 8, 9, 10, 12, 14, 15).\end{aligned}$$

Теперь вам известны пять возможных представлений комбинационной логической функции:

1. Таблица истинности.
2. Алгебраическая сумма минтермов, то есть каноническая сумма.
3. Список минтермов, обозначаемый символом Σ .
4. Алгебраическое произведение макстермов, то есть каноническое произведение.
5. Список макстермов, обозначаемый символом Π .

Каждое из этих представлений несет в себе одну и ту же информацию; когда любое одно из них задано, четыре других можно получить путем простого механического вывода.

4.2. Анализ комбинационных схем

Мы осуществляем анализ комбинационной логической схемы, описывая формально логическую функцию, которую реализует эта схема. Получив описание логической функции, мы можем предпринять ряд других действий:

- Определить реакцию схемы на различные комбинации входных воздействий.
- Преобразовать алгебраическую запись и предложить другую структуру схемы, реализующей эту логическую функцию.
- Преобразовать алгебраическую запись так, чтобы подогнать ее под имеющуюся структуру схемы. Например, сумма произведений непосредственно соответствует структуре схемы, создаваемой в программируемой логической ИС.
- Использовать алгебраическое описание работы схемы при анализе системы большего размера, включающей в себя эту схему.

Если имеется графическое изображение комбинационной схемы, такое, например, как на рис. 4.9, то существует несколько способов получить формальное описание функции, которую реализует эта схема. Самым простым функциональным описанием является таблица истинности.

Используя только основные аксиомы алгебры переключений, мы можем составить таблицу истинности для схемы с n входами, проследив путь от входов к выходам для всех 2^n комбинаций входных сигналов. Для каждой такой комбинации определяются сигналы, возникающие на выходах всех вентилей под действием данных входных сигналов, перенося информацию от входов схемы к ее выходам.

На рис. 4.10 демонстрируется применение этого «исчерпывающего» метода к схеме, рассматриваемой в нашем примере. У каждой сигнальной линии в схеме выписана последовательность из восьми логических значений, которые возникают на этой линии, когда на входы схемы XYZ поочередно подаются сигналы 000, 001, ..., 111. Чтобы записать таблицу истинности, достаточно воспроизвести последовательность на выходе последнего вентиля ИЛИ, как это сделано в табл. 4.7. Составив таблицу истинности для рассматриваемой схемы, мы можем прямо написать логическое выражение в виде канонической суммы или канонического произведения по нашему желанию.

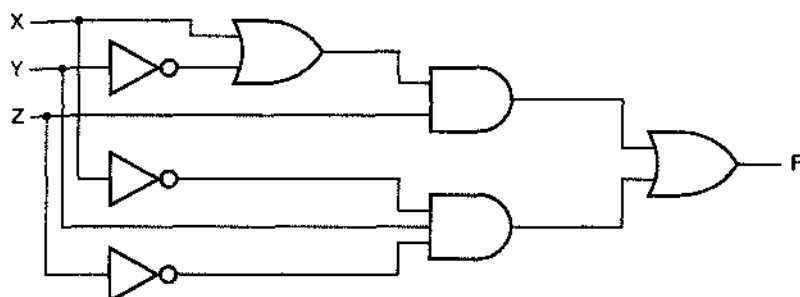


Рис. 4.9. Логическая схема с тремя входами и одним выходом

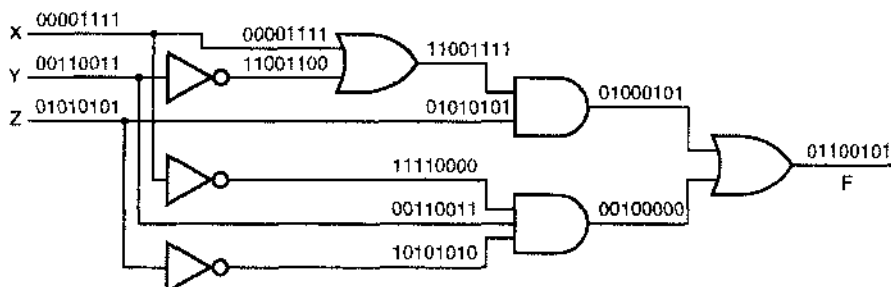


Рис. 4.10. Сигналы, возникающие на выходах вентилях под воздействием всех комбинаций входных сигналов

Строка	X	Y	Z	F
0	0	0	0	0
1	0	0	1	1
2	0	1	0	1
3	0	1	1	0
4	1	0	0	0
5	1	0	1	1
6	1	1	0	0
7	1	1	1	1

Табл. 4.7. Таблица истинности для логической схемы, приведенной на рис. 4.9

МЕНЕЕ УТОМИТЕЛЬНЫЙ СПОСОБ ПРОДВИЖЕНИЯ

Результаты, представленные на рис. 4.10, легко получить с помощью типичных средств логического проектирования, содержащих программу логического моделирования. Сначала вы рисуете схему. Затем на входы X , Y и Z подаете сигналы с выходов 3-разрядного счетчика. (У большинства моделирующих программ имеются такие встроенные счетчики, предназначенные как раз для упражнения подобного толка.) Счетчик многократно перебирает все восемь возможных комбинаций входных сигналов в том порядке, как это показано на рисунке. Моделирующая программа позволяет наблюдать временную диаграмму результирующих сигналов в любой промежуточной точке схемы, а также на ее выходе.

Число комбинаций входных сигналов растет экспоненциально с увеличением числа входов, так что полный перебор может быстро стать утомительным. Вместо этого обычно применяется алгебраический подход, при котором сложность описания всего лишь более или менее пропорциональна размерам схемы. Этот метод прост: составляется логическое выражение, снабженное необходимыми скобками, соответствующее логическим операторам и структуре схемы. Процедура составления логического выражения начинается со входов схемы, и затем поочередно совершается переход к выходам вентилях, встречающихся на пути между входами и выходом. По мере этого движения можно упрощать получающиеся выражения согласно теоремам алгебры переключений, но можно также отложить все алгебраические преобразования до того момента, когда будет получено выражение для сигнала на выходе схемы.

Рис. 4.11 демонстрирует применение алгебраического метода в нашем примере. Функция, выполняемая схемой в целом, определяется логическим выражением, относящимся к выходу последнего вентиля ИЛИ:

$$F = ((X + Y') \cdot Z) + (X' \cdot Y \cdot Z').$$

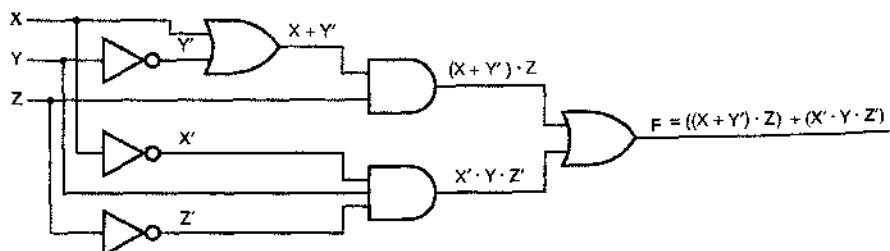


Рис. 4.11. Логические выражения для различных сигнальных линий

При получении этого выражения никакие теоремы алгебры переключений не применялись. Однако нам можно воспользоваться, чтобы преобразовать данное выражение к другому виду. Например, можно получить сумму произведений, «разнося множитель по слагаемым»:

$$F = X \cdot Z + Y' \cdot Z + X' \cdot Y \cdot Z'.$$

Это выражение соответствует другой схеме, реализующей ту же самую логическую функцию; новая схема приведена на рис. 4.12.

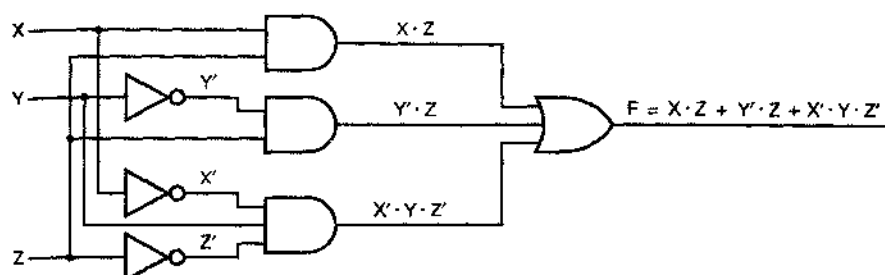


Рис. 4.12. Двухуровневая схема И-ИЛИ

Но с другой стороны, «разнося слагаемые по сомножителям», можно исходное выражение представить в виде произведения сумм:

$$\begin{aligned} F &= ((X + Y') \cdot Z) + (X' \cdot Y \cdot Z') \\ &= (X + Y' + X') \cdot (X + Y' + Y) \cdot (X + Y' + Z') \cdot (Z + X') \cdot (Z + Y) \cdot (Z + Z') \\ &= 1 \cdot 1 \cdot (X + Y' + Z') \cdot (X' + Z) \cdot (Y + Z) \cdot 1 \\ &= (X + Y' + Z') \cdot (X' + Z) \cdot (Y + Z). \end{aligned}$$

Соответствующая логическая схема показана на рис. 4.13.

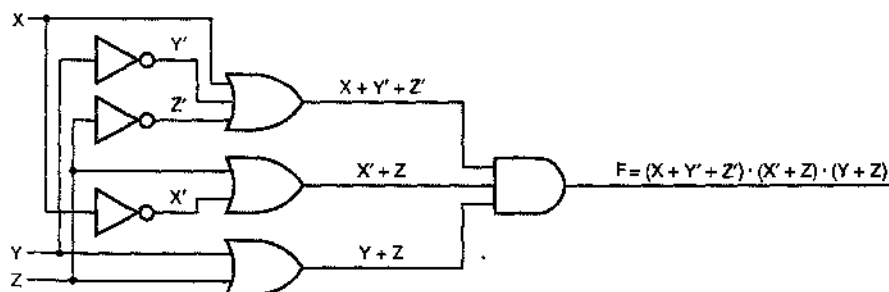


Рис. 4.13. Двухуровневая схема ИЛИ-И

В нашем следующем примере алгебраического анализа рассматривается схема с вентилями И-НЕ и ИЛИ-НЕ, приведенная на рис. 4.14. Этот анализ чуть неприятнее, чем в предыдущем примере, так как каждый из вентилях описывается дополнением подвыражения, а не простой суммой или произведением. Однако результирующее выражение для выходного сигнала можно упростить, многократно применяя обобщенную теорему Де Моргана:

$$\begin{aligned} F &= [((W \cdot X')' \cdot Y')' + (W' + X + Y')' + (W + Z)']' \\ &= ((W' + X)' + Y)' \cdot (W \cdot X' \cdot Y)' \cdot (W' \cdot Z)' \\ &= ((W \cdot X')' \cdot Y) \cdot (W' + X + Y') \cdot (W + Z) \\ &= ((W' + X) \cdot Y) \cdot (W' + X + Y') \cdot (W + Z). \end{aligned}$$

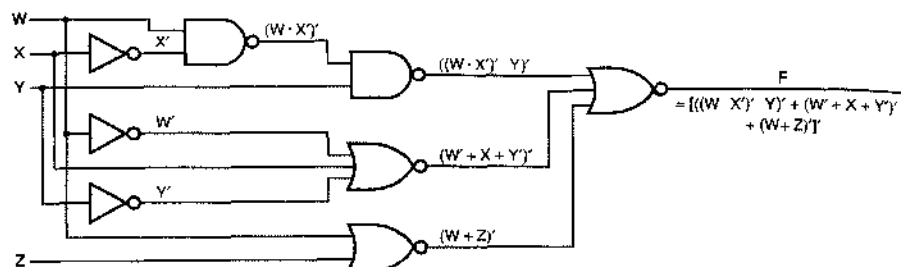


Рис. 4.14. Алгебраический анализ логической схемы с вентилями И-НЕ и ИЛИ-НЕ

Довольно часто теорема Де Моргана применяется для упрощения алгебраического анализа *графически*. Вспомните: у каждого из вентилях И-НЕ и ИЛИ-НЕ имеется по два эквивалентных графических изображения, как это было показано на рис. 4.3 и 4.4. В результате аккуратного перерисовывания схемы, приведенной на рис. 4.14, оказывается возможным взаимно уничтожить часть инверсий согласно теореме Т4 $[(X')' = X]$, как это показано на рис. 4.15. Это преобразование непосредственно приводит нас к более простому выражению для выходного сигнала:

$$F = ((W' + X) \cdot Y) \cdot (W' + X + Y') \cdot (W + Z).$$

На рис. 4.14 и 4.15 представлены два различных способа изображения физически одной и той же логической схемы. Однако, упрощая логическое выражение по правилам алгебры переключений, мы получаем выражение, соответствующее схеме, физически отличающейся от исходной. Например, последнее упрощенное выражение соответствует схеме, приведенной на рис. 4.16, которая физически отличается от каждой из схем на предыдущих двух рисунках. Более того, разнося множители по слагаемым или слагаемые по сомножителям, мы могли бы получить выражения в виде суммы произведений и произведения сумм, соответствующие еще двум схемам, физически отличающимся от уже упомянутых, но реализующим ту же самую логическую функцию.

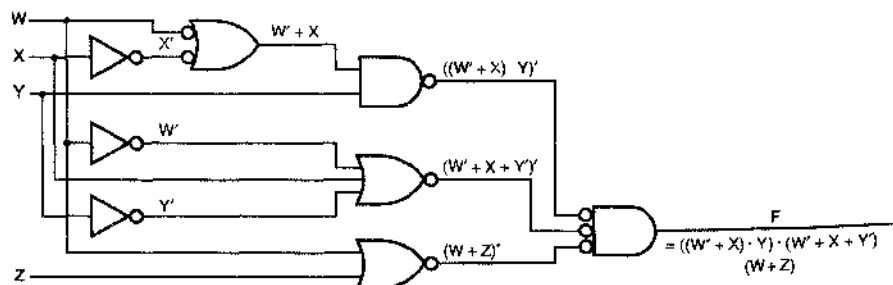


Рис. 4.15. Алгебраический анализ схемы, которая получается из предыдущей схемы заменой условных обозначений некоторых вентилях И-НЕ и ИЛИ-НЕ

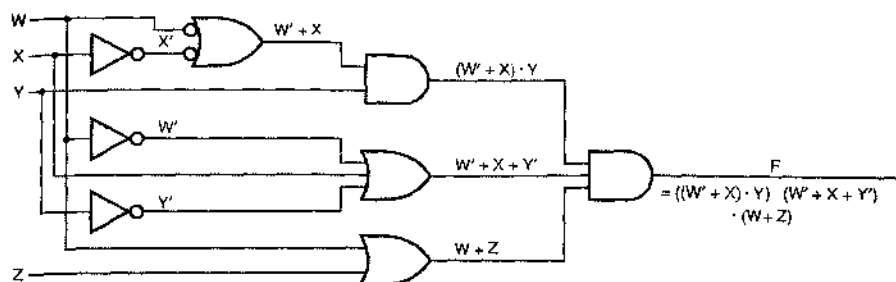


Рис. 4.16. Другая схема, реализующая ту же самую логическую функцию

Хотя выше мы использовали логические выражения для отображения информации о физической структуре схемы, так происходит не всегда. Например, выражением $G(W, X, Y, Z) = W \cdot X \cdot Y + Y \cdot Z$ можно было бы описать любую из схем, изображенных на рис. 4.17. Как правило, единственно верный способ определения структуры того или иного устройства состоит в том, чтобы рассмотреть его принципиальную схему. Однако для определенного ограниченного класса схем информацию об их структуре можно вывести из логических выражений. Например, схему на рис. (а) можно было бы описать, не прибегая к ее графическому изображению, как «двухуровневую схему И–ИЛИ для функций $W \cdot X \cdot Y + Y \cdot Z$ », тогда как о схеме на рис. (б) можно было бы сказать как о «двухуровневой схеме НЕ–И–НЕ–И для функции $W \cdot X \cdot Y + Y \cdot Z$ ».

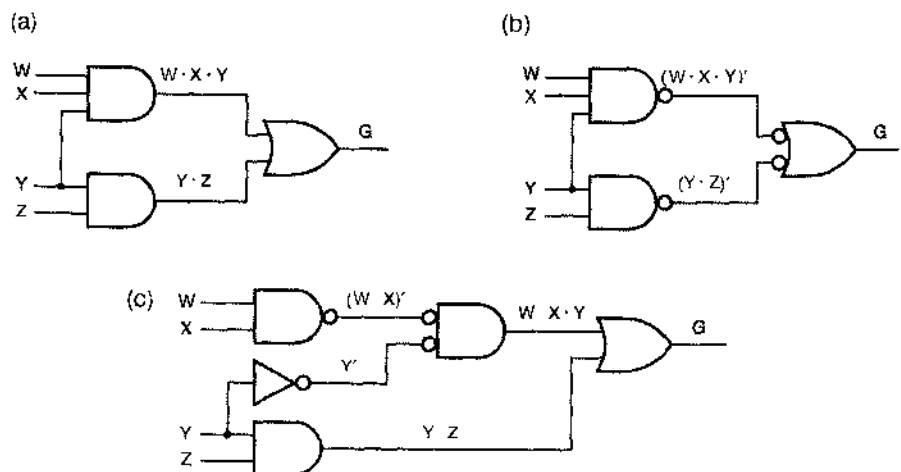


Рис. 4.17. Три схемы для функции $G(W, X, Y, Z) = W \cdot X \cdot Y + Y \cdot Z$. (а) двухуровневая схема И–ИЛИ; (б) двухуровневая схема И–НЕ–И–НЕ; (с) еще один вариант

4.3. Синтез комбинационных схем

4.3.1. Описание и составление схем

Что является отправной точкой при составлении логических схем? Обычно у нас есть словесное описание проблемы, либо мы формулируем ее сами. Иногда описание представляет собой перечень комбинаций входных сигналов, при которых сигнал на выходе должен принимать значения 0 или 1, то есть является словесным выражением таблицы истинности, списка минтермов Σ или списка макстермов Π , о которых говорилось выше. Например, описание 4-разрядного устройства для обнаружения простых чисел могло бы быть таким: «при заданной 4-разрядной двоичной комбинации на входе $N = N_3N_2N_1N_0$ схема, реализующая требуемую функцию, вырабатывает на выходе 1, если $N = 1, 2, 3, 5, 7, 11$ и 13 , и 0 – в противном случае». При таком описании логической функции схему можно составить непосредственно, воспользовавшись канонической суммой или произведением. Для устройства, обнаруживающего простые числа, имеем:

$$\begin{aligned}
 F &= \Sigma_{N_3, N_2, N_1, N_0} (1, 2, 3, 5, 7, 11, 13) \\
 &= N_3' \cdot N_2' \cdot N_1' \cdot N_0 + N_3' \cdot N_2' \cdot N_1 \cdot N_0' + N_3' \cdot N_2' \cdot N_1 \cdot N_0 + N_3' \cdot N_2 \cdot N_1' \cdot N_0 + \\
 &\quad + N_3' \cdot N_2 \cdot N_1 \cdot N_0 + N_3 \cdot N_2' \cdot N_1 \cdot N_0 + N_3 \cdot N_2 \cdot N_1' \cdot N_0.
 \end{aligned}$$

Соответствующая схема показана на рис. 4.18.

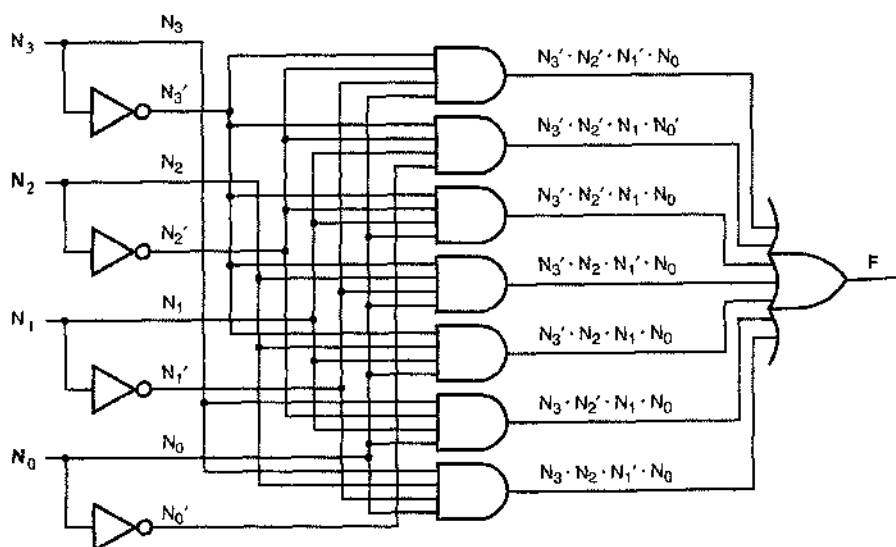


Рис. 4.18. Схема 4-разрядного устройства для обнаружения простых чисел, составленная по логической функции в виде канонической суммы

Чаше мы описываем логическую функцию, используя соединительные слова разговорного языка «и», «или» и «не». Например, вы могли бы описать схему охранной сигнализации вашего дома следующими словами: «Сигнал ALARM («тревога») должен быть равен 1, если равен 1 входной сигнал PANIC («паника»), или в

том случае, когда равен 1 входной сигнал ENABLE («сигнализация включена»), равен 0 сигнал EXITING («мы выходим») и безопасность дома нарушена; дом находится в безопасности, если все входные сигналы WINDOW («окно»), DOOR («дверь») и GARAGE («гараж») равны 1. Такое описание можно напрямую перевести в алгебраическую запись:

$$\begin{aligned} \text{ALARM} &= \text{PANIC} + \text{ENABLE} \cdot \text{EXITING}' \cdot \text{SECURE}' \\ \text{SECURE} &= \text{WINDOW} \cdot \text{DOOR} \cdot \text{GARAGE} \\ \text{ALARM} &= \text{PANIC} + \text{ENABLE} \cdot \text{EXITING}' \cdot (\text{WINDOW} \cdot \text{DOOR} \cdot \text{GARAGE})'. \end{aligned}$$

Заметьте, что мы применили в алгебре нереключений тот же метод, каким пользуются в обычной алгебре, когда нужно сформулировать сложное высказывание: мы ввели вспомогательную переменную SECURE («дом защищен»), чтобы упростить первое равенство, написали выражение для переменной SECURE и осуществили подстановку, которая позволила получить окончательное выражение. Используя вентили И, ИЛИ и НЕ, легко нарисовать схему, которая реализует это окончательное выражение, что и сделано на рис. 4.19. Схема *realizes* [воплощает в «железе» (*realize*, «makes real»)] то или иное выражение, если функция, описывающая выходной сигнал, равна этому выражению; говорят, что схема является *реализацией* (*realization*) данной функции.

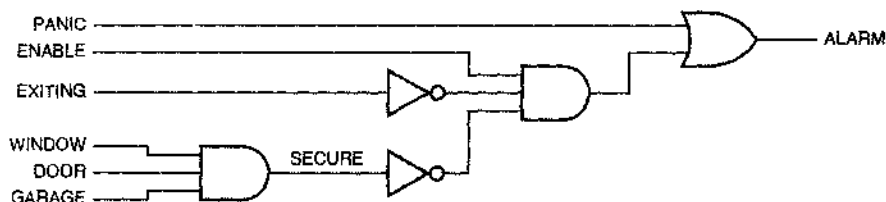


Рис. 4.19. Схема охранной сигнализации, составленная по логическому выражению

Когда имеется какое-либо выражение рассматриваемой логической функции, — любое ее выражение, — можно поступать и иначе, а не только непосредственно составлять схему по этому выражению. В результате преобразования данного выражения можно получать и другие схемы. Например, приведенное выше выражение для сигнала ALARM можно преобразовать в сумму произведений, и тогда соответствующая схема будет такой, как показано на рис. 4.20. Если число переменных не слишком велико, то по выражению можно составить таблицу истинности и воспользоваться любым из методов синтеза, применяемых в таком случае, включая рассмотренное ранее представление в виде канонической суммы или канонического произведения, а также методы минимизации, о которых речь пойдет позднее.

В общем случае, особенно если число переменных велико, легче описать схему словами, используя логические связи, и записать соответствующее логическое выражение, нежели составлять полную таблицу истинности. Однако иногда приходится иметь дело с расплывчатыми словесными описаниями логических функций типа: «Выходной сигнал ERROR («ошибка») должен равняться 1, если входные сигналы GEARUP, GEARDOWN и GEARCHECK несовместимы». [Здесь: gear

– шасси самолета; gearup – «шасси убрано»; geardown – «шасси выпущено»; gearcheck – проверка того, что шасси выпущено и зафиксировано в этом состоянии. – Прим. перев.] В такой ситуации лучше всего составить таблицу истинности, так как это позволит задать необходимые значения выходного сигнала при различных комбинациях входных сигналов, опираясь на наше знание и понимание нривходящих обстоятельств (например, что нельзя осуществить торможение, если шасси не выпущено).

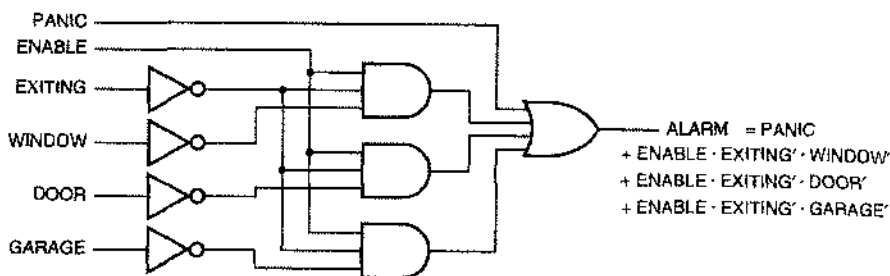


Рис. 4.20. Вариант схемы охранной сигнализации, построенный по сумме произведений

4.3.2. Преобразование схем

Рассматривавшиеся нами до сих пор методы проектирования предполагали использование вентилях И, ИЛИ и НЕ. С тем же успехом мы могли бы воспользоваться вентилями И-НЕ и ИЛИ-НЕ: для большинства технологий эти схемы являются более быстродействующими, нежели схемы И и ИЛИ. Однако люди, как правило, не формулируют логические суждения с использованием связок И-НЕ и ИЛИ-НЕ. Другими словами, вы вряд ли скажете: «Я не назначу вам свидание, если вы не будете опрятны или богаты, и также в том случае, если вы не будете элегантны или дружелюбны». Для вас будет более естественно сказать: «Я назначу вам свидание, если вы будете опрятны и богаты или в том случае, если вы будете элегантны и дружелюбно настроены». Таким образом, при наличии «естественного» логического выражения нам необходимы способы преобразовывать его в другие формы.

Любое логическое выражение можно преобразовать в эквивалентное выражение вида «сумма произведений», просто разнесением содержащихся в нем множителей по слагаемым. Как показано на рис. 4.21(а), такое выражение можно реализовать непосредственно с помощью вентилях И и ИЛИ. Инверторы, необходимые на входах для образования дополнения входных сигналов, на рисунке не показаны.

На рис. 4.21 (b) показано, что между каждым из выходов вентилях И и соответствующими входами вентиля ИЛИ в двухуровневой схеме И–ИЛИ можно вставить пару инверторов. По теореме Т4 эти инверторы не оказывают влияния на функцию, реализуемую этой схемой. Действительно, мы нарисовали второй инвертор в каждой паре с кружком инверсии на его входе, чтобы подчеркнуть графически, что инверсии взаимно уничтожаются. Однако в том случае, когда инвертирование

отнесено к вентилям И и ИЛИ, мы получаем вентили И-НЕ на первом уровне и вентили НЕ-ИЛИ на втором уровне. Но это два различных условных изображения для одного и того же типа вентиля, а именно – для вентиля И-НЕ. Таким образом, двухуровневую схему И-ИЛИ (AND-OR circuit) можно преобразовать в схему И-НЕ-И-НЕ (NAND-NAND circuit) простой заменой вентилях.

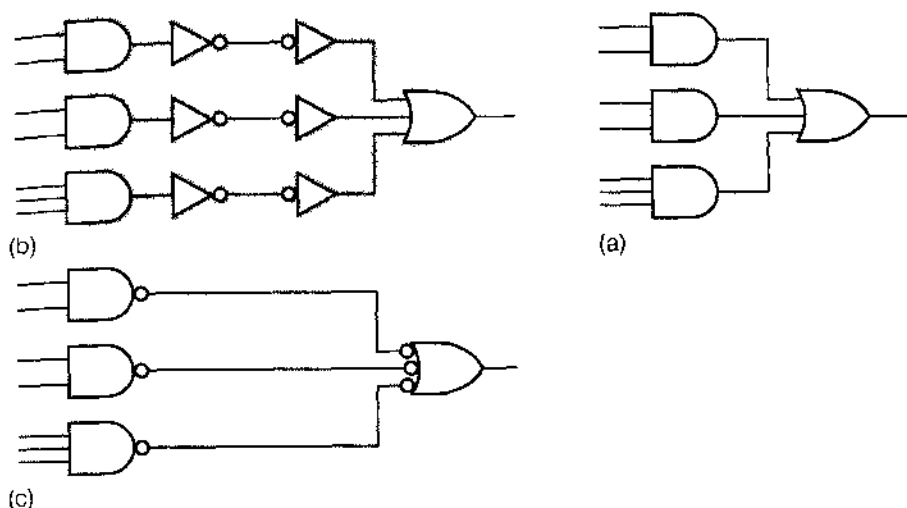


Рис. 4.21. Альтернативные реализации суммы произведений: (а) И-ИЛИ; (б) И-ИЛИ с дополнительными парами инверторов; (с) И-НЕ-И-НЕ

Если все термы-произведения в сумме произведений содержат точно по одному литералу, то в процессе преобразования схемы И-ИЛИ в схему И-НЕ-И-НЕ можно добавлять или опускать инверторы. В примере, приведенном на рис. 4.22, инвертор на входе W больше не нужен, но на входе Z необходимо добавить инвертор.

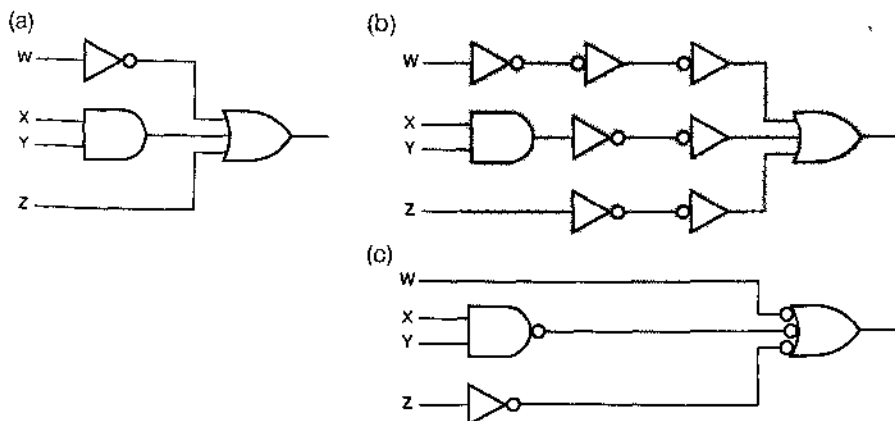
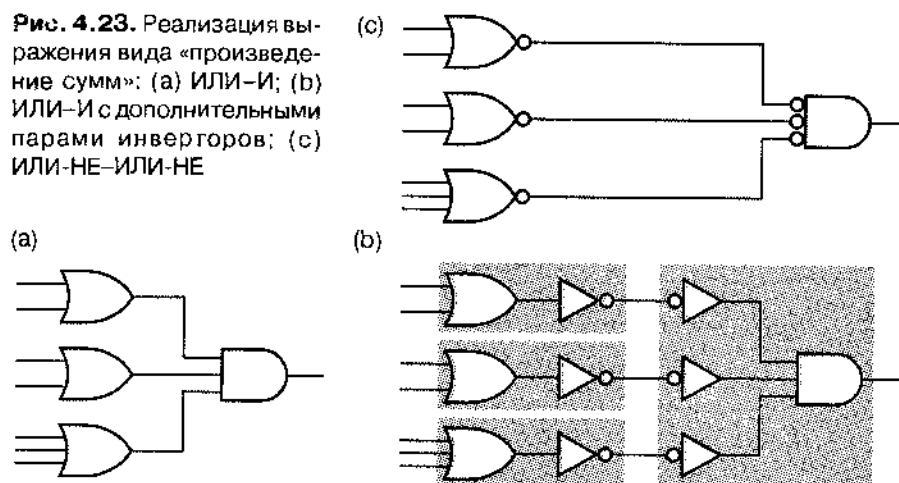


Рис. 4.22. Другой пример двухуровневой схемы, построенной по сумме произведений: (а) И-ИЛИ; (б) И-ИЛИ с дополнительными парами инверторов; (с) И-НЕ-И-НЕ

Итак, мы убедились, что любое выражение вида «сумма произведений» можно реализовать одним из двух способов: как схему И–ИЛИ или как схему И–НЕ–И–НЕ. Справедливо также и двойственное этому утверждение: любое выражение вида «произведение сумм» можно реализовать как схему ИЛИ–И (*OR-AND circuit*) или как схему ИЛИ–НЕ–ИЛИ–НЕ (*NOR-NOR circuit*). На рис. 4.23 приведен пример. Любое логическое выражение можно свести к эквивалентному произведению сумм, разнося содержащиеся в нем слагаемые по сомножителям; следовательно, любое логическое выражение можно реализовать как в виде схемы ИЛИ–И, так и в виде схемы ИЛИ–НЕ–ИЛИ–НЕ.

Рис. 4.23. Реализация выражения вида «произведение сумм»: (а) ИЛИ–И; (б) ИЛИ–И с дополнительными парами инверторов; (с) ИЛИ–НЕ–ИЛИ–НЕ



Подобного рода преобразования применимы к произвольным логическим схемам. На рис. 4.24(а), например, приведена схема, состоящая из вентилях И и ИЛИ. Добавляя пары инверторов, мы получаем схему, изображенную на рис. (б). Однако в этой схеме один из вентилях – 2-входовый вентиль И с одним инвертированным входом – не является вентилем стандартного типа. Тогда можно применить отдельно стоящий инвертор, как показано на рис. (с), чтобы получить схему, в которой используются только вентили стандартного типа: И–НЕ, И и инверторы. На самом деле лучше воспользоваться инвертором так, как это продемонстрировано на рис. (д); тогда путь сигнала – с точки зрения его задержки – сокращается на один уровень, а нижняя схема И преобразуется в схему ИЛИ–НЕ. В большинстве технологий, используемых при производстве логических схем, инвертирующие вентили типа И–НЕ и ИЛИ–НЕ являются более быстродействующими, чем неинвертирующие вентили типа И и ИЛИ.

ЗАЧЕМ НУЖНА МИНИМИЗАЦИЯ

Минимизация является важным этапом как при создании специализированных ИС, так и при проектировании на базе ПЛУ. В специализированной ИС для лишних вентилях и при наличии у вентилях лишних входов необходима дополнительная площадь на поверхности кристалла, а это приводит к увеличению стоимости микросхем. В программируемых ИС число вентилях фиксировано, и можно подумать, что проблемы лишних вентилях нет; это действительно так, но только до тех пор, пока вы не вышли за пределы того, что имеется, и должны перейти на более медленные или более дорогие ИС большего объема. К счастью, у большинства программных средств, используемых при проектировании специализированных ИС и устройств на базе ПЛУ, имеются встроенные программы минимизации. Назначение разделов с 4.3.3 по 4.3.8 состоит в том, чтобы дать вам почувствовать, как именно осуществляется минимизация.

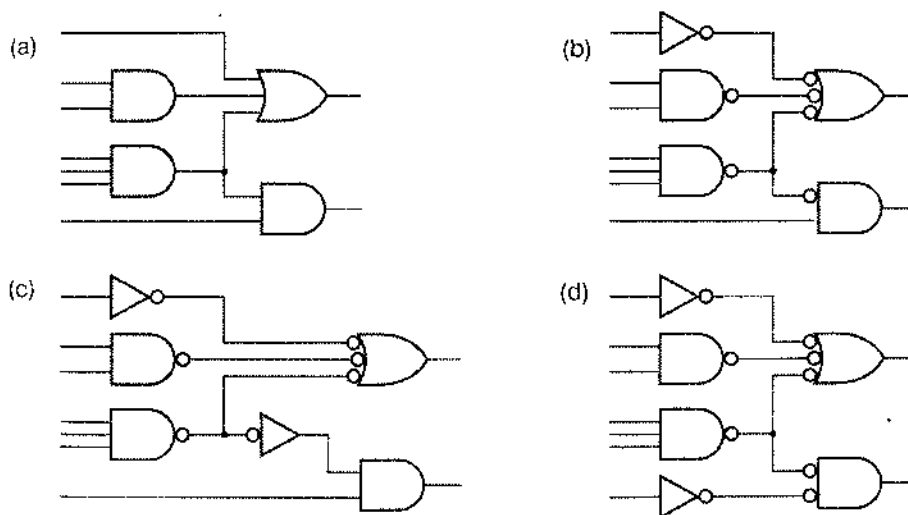


Рис. 4.24. Преобразования на уровне условных обозначений логических схем: (a) исходная схема; (b) результат преобразования с нестандартным вентилям; (c) схема с отдельным инвертором, позволяющим исключить нестандартный вентиль; (d) предпочтительное расположение отдельного инвертора

4.3.3. Минимизация комбинационных схем

Чаще всего не экономично напрямую строить логическую схему по логическому выражению, которое первым пришло вам в голову. Особенно расточительно поступать так, руководствуясь канонической суммой или каноническим произведением, поскольку число возможных минтермов и макстермов (а, значит, и вентилей) растет экспоненциально с увеличением числа переменных. Мы *минимизируем* (*minimize*) комбинационную схему, сокращая число и размер вентилей, необходимых для ее построения.

Отправным моментом в традиционных методах минимизации комбинационных схем, к изучению которых мы приступаем, служат таблицы истинности или, что эквивалентно, списки минтермов и макстермов. Если логическая функция задана не в одной из этих форм, то прежде чем мы воспользуемся этими методами, необходимо преобразовать данную функцию и представить ее в подходящем виде. Если, например, речь идет о произвольном логическом выражении, то для составления таблицы истинности нужно найти значения этого выражения при всех комбинациях значений входных сигналов. Методы минимизации уменьшают стоимость двухуровневых схем И-ИЛИ, ИЛИ-И, И-НЕ-И-НЕ и ИЛИ-НЕ-ИЛИ-НЕ одним из трех способов:

1. Путем минимизации числа вентилей на первом уровне.
2. Путем минимизации числа входов у каждого вентиля первого уровня.
3. Путем минимизации числа входов у вентиля второго уровня. В действительности, последнее является побочным эффектом реализации первого из этих способов.

Однако методы минимизации не учитывают стоимости входных инверторов; при минимизации предполагается, что уже имеются в наличии как сами входные сигналы, так и их дополнения. Это не всегда имеет место при проектировании на уровне вентилей и при разработке специализированных ИС; но при проектировании на основе ПЛУ это требование вполне уместно: в ПЛУ вы имеете все входные сигналы и их дополнения «бесплатно».

Большинство методов минимизации основано на обобщении комбинационных теорем T10 и T10':

$$\begin{aligned} & \text{заданный терм-произведение} \cdot Y + \text{заданный терм-произведение} \cdot Y' = \\ & = \text{заданный терм-произведение}, \end{aligned}$$

$$\begin{aligned} & (\text{заданный терм-сумма} + Y) \cdot (\text{заданный терм-сумма} + Y') = \\ & = \text{заданный терм-сумма}. \end{aligned}$$

Другими словами, в случае, когда два термина, являющихся произведениями или суммами, различаются только тем, что какая-то переменная содержится в одном из них непосредственно, а в другом – в форме дополнения, эти два термина можно объединить в один терм, в котором число переменных на единицу меньше. Таким образом экономится один вентиль, а у остающегося вентиля на один вход будет меньше.

Если этот алгебраический метод многократно применить к устройству для обнаружения простых чисел, изображенному на рис. 4.18, и объединить минтермы 1, 3, 5 и 7, то получим:

$$\begin{aligned}
 F &= \sum_{N_3, N_2, N_1, N_0} (1, 2, 3, 5, 7, 11, 13) \\
 &= N_3' \cdot N_2' \cdot N_1' \cdot N_0 + N_3' \cdot N_2' \cdot N_1 \cdot N_0 + N_3' \cdot N_2 \cdot N_1' \cdot N_0 + \\
 &\quad + N_3' \cdot N_2 \cdot N_1 \cdot N_0 + \dots \\
 &= (N_3' \cdot N_2' \cdot N_1' \cdot N_0 + N_3' \cdot N_2' \cdot N_1 \cdot N_0) + (N_3' \cdot N_2 \cdot N_1' \cdot N_0 + \\
 &\quad + N_3' \cdot N_2 \cdot N_1 \cdot N_0) + \dots \\
 &= N_3' \cdot N_2' \cdot N_0 + N_3' \cdot N_2 \cdot N_0 + \dots \\
 &= N_3' \cdot N_0 + \dots
 \end{aligned}$$

Результирующая схема показана на рис. 4.25; в ней на три вентиля меньше, а у одного из оставшихся вентилях на два входа меньше.

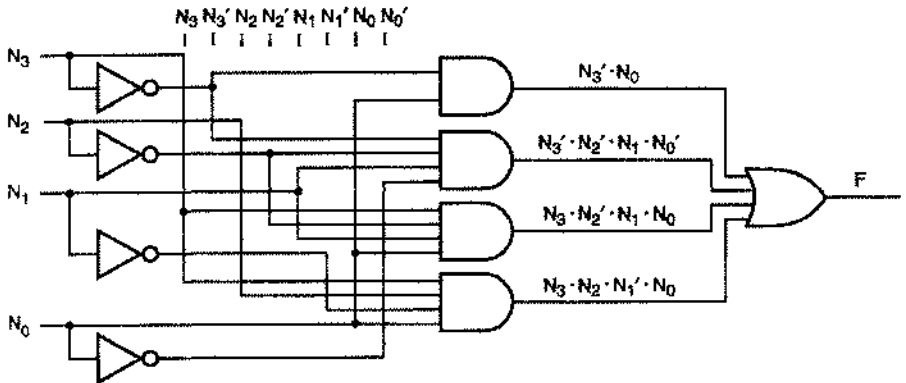


Рис. 4.25. Упрощенная реализация суммы произведений для 4-разрядного устройства распознавания простых чисел

Если еще немного потрудиться над последним выражением, то можно было бы сэкономить пару входов у вентилях первого уровня, но число вентилях уменьшить уже не удастся. В мешанине алгебраических символов бывает трудно найти термы, которые можно объединить. В следующем разделе мы начнем изучать метод минимизации, который более нагляден и удобен для анализа и преобразования логических схем вручную. Начальной точкой в этом рассмотрении будет графический эквивалент таблицы истинности.

4.3.4. Карты Карно

Карта Карно (Karnaugh map) — это графическое представление таблицы истинности, которой задается логическая функция. На рис. 4.26 приведены карты Карно для логических функций 2-х, 3-х и 4-х переменных. Карта для логической функции n переменных представляет собой решетку с 2^n клетками, по одной клетке на каждый минтерм.

Разметка строк и столбцов карты Карно выполняется таким образом, что для любой клетки легко определить соответствующую ей комбинацию переменных по заголовкам строки и столбца, на пересечении которых находится эта клетка. Число в левом верхнем углу каждой клетки соответствует номеру минтерма в таблице истинности в предположении, что переменные в таблице истинности расположены

в алфавитном порядке их имен слева направо (например, X, Y, Z), а строки пронумерованы в порядке нарастания двоичных чисел, подобно тому, как это делается в примерах в данном учебнике. Например, 13-я клетка для случая 4-х переменных соответствует той строке таблицы истинности, в которой $WXYZ = 1101$.

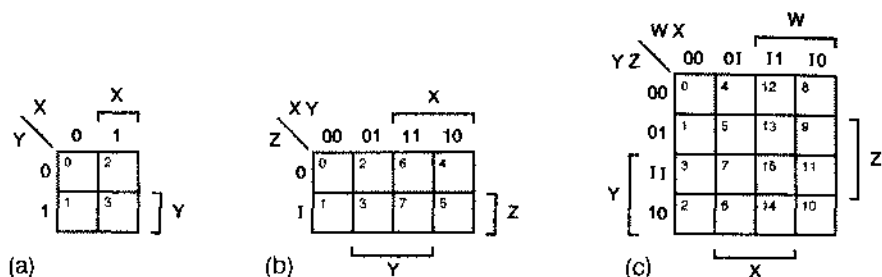


Рис. 4.26. Карты Карно: (а) случай 2-х переменных; (б) случай 3-х переменных; (с) случай 4-х переменных

Когда составляется карта Карно для заданной функции, в каждую клетку помещается информация, извлекаемая из строки таблицы истинности с подходящим номером: 0, если функция равна нулю при этой комбинации переменных, и 1 – в противном случае.

В этой книге мы будем применять двойную разметку строк и столбцов карты, несмотря на ее избыточность. Рассмотрим, например, карту для случая 4-х переменных на рис. 4.26(с). Столбцы помечены четырьмя возможными комбинациями W и X: $WX = 00, 01, 11$ и 10 . Аналогично, строки помечены комбинациями YZ. Эти метки снабжают нас всей необходимой информацией. Однако мы будем также расставлять квадратные скобки, чтобы каждой из четырех переменных поставить в соответствие определенные области на карте. Каждая область, отмеченная скобкой, – это часть карты, в пределах которой указанная переменная равна 1. Очевидно, что скобки несут ту же самую информационную нагрузку, что и метки, которыми помечены строки и столбцы.

Когда карта рисуется от руки, гораздо легче парировать скобки, нежели выписывать все метки. Однако мы сохраним на картах Карно, предназначенных для учебных целей, также и метки в качестве дополнительного средства, облегчающего понимание. В любом случае, вы безусловно должны размечать строки и столбцы в надлежащем порядке, чтобы сохранить соответствие между клетками карты и номерами строк в таблице истинности, указанное на рис. 4.26.

Чтобы представить логическую функцию в виде карты Карно, мы просто переносим единицы и нули из таблицы истинности или ее эквивалента в соответствующие клетки карты. На рис. 4.27(а) и (б) приведены таблица истинности и карта Карно для логической функции, которую мы рассматривали в параграфе 4.2 (продолжаем долбить одно и то же). В дальнейшем, чтобы не рябило в глазах, мы будем заносить в карту только единицы или нули, но не те и другие одновременно.

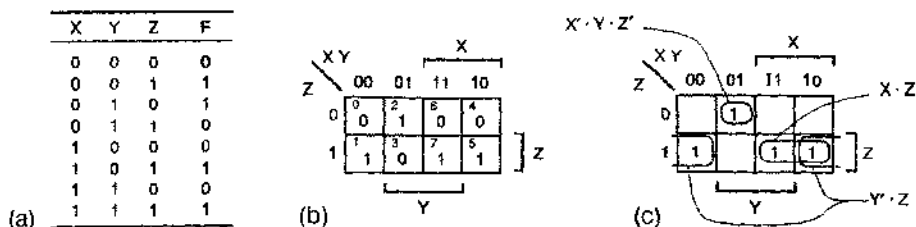


Рис. 4.27. $F = \sum_{x,y,z}(1, 2, 5, 7)$: (a) таблица истинности; (b) карта Карно; (c) объединение соседних клеток, содержащих 1

4.3.5. Минимизация сумм произведений

Вы, наверное, уже успели удивиться «страшному» порядку, в котором следуют строки и столбцы в карте Карно. Для установления этого порядка имеется очень веская причина: каждой клетке соответствует такая комбинация переменных, которая отличается от комбинаций, соответствующих клеткам, падающим в непосредственном соседстве с данной клеткой, лишь значением одной переменной. Например, 5-я и 13-я клетки в карте для случая 4-х переменных различаются только значением W . В случае 3-х и 4-х переменных чуть менее очевидными соседями являются соответствующие клетки в крайнем левом и крайнем правом столбцах и в верхней и нижней строках; например, 12-я и 14-я клетки в случае 4-х переменных являются смежными, поскольку они различаются только значением Y .

Каждая комбинация переменных с единичным значением функции в таблице истинности соответствует минтерму в канонической сумме данной логической функции. Поскольку пара соседних клеток карты Карно, содержащих 1, указывает на наличие минтермов, различающихся значением только одной переменной, эту пару минтермов можно объединить в один терм-произведение на основании обобщения теоремы T10: $\text{term} \cdot Y + \text{term} \cdot Y' = \text{term}$. Таким образом, картой Карно можно воспользоваться для упрощения канонической суммы логической функции.

Рассмотрим, например, клетки 5 и 7 на рис. 4.27(b) и их вклад в каноническую сумму этой функции:

$$\begin{aligned}
 F &= \dots + X \cdot Y' \cdot Z + X \cdot Y \cdot Z \\
 &= \dots + (X \cdot Z) \cdot Y' + (X \cdot Z) \cdot Y \\
 &= \dots + X \cdot Z.
 \end{aligned}$$

Вспоминая о том, что таблицу следует представлять себе «свернутой», мы видим, что клетки 1 и 5 на рис. 4.27(b) также являются смежными и их можно объединить:

$$\begin{aligned}
 F &= X' \cdot Y' \cdot Z + X \cdot Y' \cdot Z + \dots \\
 &= X' \cdot (Y' \cdot Z) + X \cdot (Y' \cdot Z) + \dots \\
 &= Y' \cdot Z + \dots
 \end{aligned}$$

В общем случае логическую функцию можно упростить, объединяя пары соседних клеток, содержащих 1, (пары минтермов) везде, где только это возможно, и выписывая сумму термов-произведений, покрывающую все клетки, содержащие 1. На рис. 4.27(с) представлен результат для логической функции, рассматриваемой в нашем примере. Мы обвели пары единиц, чтобы указать, что соответствующие минтермы объединяются в один терм-произведение. Схема И-ИЛИ, реализующая эту функцию, приведена на рис. 4.28.

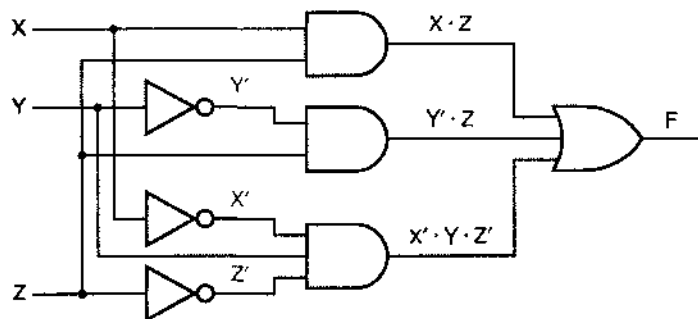


Рис. 4.28. Минимизированная схема И-ИЛИ

В отношении многих логических функций процедуру объединения клеток можно распространить на случай, когда один терм-произведение получается в результате объединения не двух клеток, а большего их числа. Рассмотрим, например, каноническую сумму логической функции $F = \sum_{XYZ} (0, 1, 4, 5, 6)$. Выполняя итеративно те же алгебраические преобразования, что и в предыдущих примерах, можно четыре из пяти минтермов объединить в один:

$$\begin{aligned}
 F &= X' \cdot Y' \cdot Z' + X' \cdot Y' \cdot Z + X \cdot Y' \cdot Z' + X \cdot Y' \cdot Z + X \cdot Y \cdot Z \\
 &= [(X' \cdot Y') \cdot Z' + (X' \cdot Y') \cdot Z] + [(X \cdot Y') \cdot Z' + (X \cdot Y') \cdot Z] + X \cdot Y \cdot Z \\
 &= X' \cdot Y' + X \cdot Y' + X \cdot Y \cdot Z' \\
 &= [X' \cdot (Y') + X \cdot (Y')] + X \cdot Y \cdot Z' \\
 &= Y' + X \cdot Y \cdot Z'.
 \end{aligned}$$

В общем случае объединение 2^i клеток, содержащих 1, приводит к образованию терма-произведения с $n - i$ литералами, где n — число переменных у данной функции.

Вот точное математическое правило для определения того, как именно можно объединять клетки, содержащие 1, образуя соответствующий терм-произведение:

- Набор из 2^i клеток, содержащих 1, можно объединить, если существует i таких переменных рассматриваемой логической функции, что в пределах данного набора перебираются все 2^i возможных комбинаций этих переменных, тогда как остальные $n - i$ переменных во всех клетках набора имеют одни и те же значения. Соответствующий терм-произведение содержит $n - i$ литералов, причем та или иная переменная входит в него в виде дополнения, если она имеет значение 0 во всех объединяемых клетках, и — непосредственно, если ее значение равно 1.

Графически это означает, что можно обводить *прямоугольные наборы единиц* (*rectangular sets of 1s*), содержащихся в 2^n клетках («прямоугольные» как в буквальном, так и в переносном смысле этого слова), распространив определение «прямоугольный» на случаи, когда необходимо принимать во внимание свернутость карты по краям. Литералы, которые войдут в соответствующие термы-произведения, можно непосредственно определить по карте; вопрос о каждой из переменных решается по одному из следующих правил:

- Если обведенная область покрывает только ту часть карты, где переменная равна 0, то эта переменная входит в терм-произведение в виде дополнения.
- Если обведенная область покрывает только ту часть карты, где переменная равна 1, то эта переменная входит в терм-произведение непосредственно.
- Если в обведенную область попадают участки карты, где переменная равна 0, а также участки карты, где переменная равна 1, то эта переменная отсутствует в терм-произведении.

Сумма произведений для пашей функции должна состоять из термов-произведений (обведенных наборов клеток, содержащих 1), которые покрывают все единицы на карте и не содержат нулей.

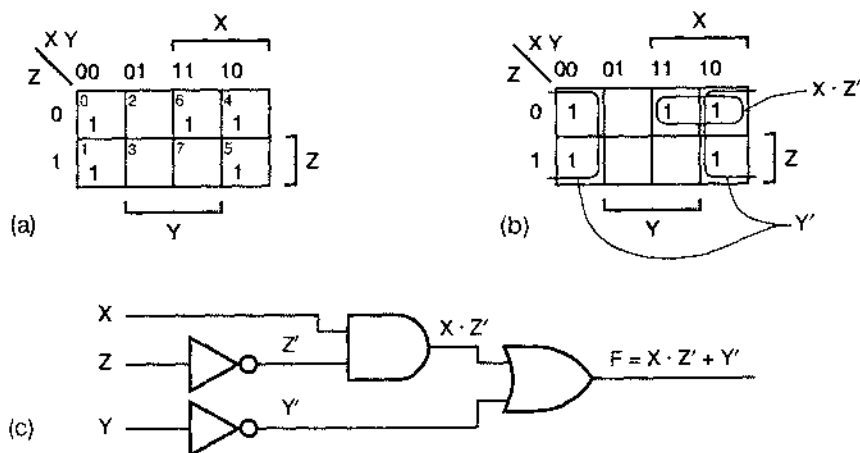


Рис. 4.29. $F = \sum_{x,y,z}(0, 1, 4, 5, 6)$. (a) первоначальная карта Карио; (b) карта Карио с обведенными термами-произведениями; (c) схема И-ИЛИ

В самом последнем нашем примере логическая функция имела вид: $F = \sum_{x,y,z}(0, 1, 4, 5, 6)$; карта Карио для нее представлена на рис. 4.29(a) и (b). Мы обвели два набора единиц: один из них охватывает четыре клетки и соответствует терму-произведению Y , а другой состоит из двух единиц и эквивалентен терму-произведению $X \cdot Z'$. Обратите внимание на то, что во второй терм-произведение входит на один литерал меньше, чем в соответствующий терм-произведение в нашем алгебраическом решении ($X \cdot Y \cdot Z'$). Включив клетку 6 в наибольший возможный набор единиц, мы нашли более экономичную реализацию данной логической функции,

поскольку 2-входовой вентиль И, скорее всего, будет стоить дешевле, чем 3-входовой вентиль. То, что два разных термина-произведения покрывают теперь одну и ту же клетку, содержащую 1 (4-ю клетку), никак не отражается на логической функции, так как при логическом сложении $1 + 1 = 1$, а не 2! Соответствующая двухуровневая схема И-ИЛИ показана на рис. 4.29(с).

Другим примером может служить минимизация схемы устройства для обнаружения простых чисел, приведенная ранее на рис. 4.18; результат минимизации представлен на рис. 4.30.

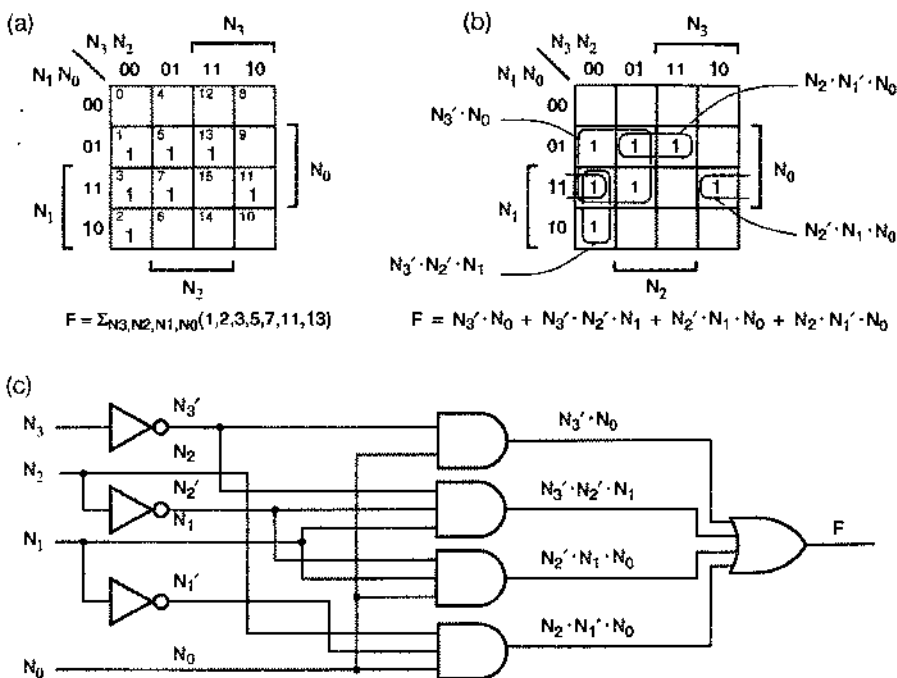


Рис. 4.30. Устройство для обнаружения простых чисел: (а) исходная карта Карно; (б) обведенные наборы единиц и соответствующие им термы-произведения; (с) минимизированная схема

Теперь нам понадобятся еще несколько определений, чтобы внести ясность в то, что мы делаем:

- *Минимальная сумма (minimal sum)* логической функции $F(X_1, \dots, X_n)$ – это такое выражение для F вида «сумма произведений», что не существует представлений F в виде суммы произведений с меньшим числом термов-произведений, а в любое другое выражение вида «сумма произведений» с тем же самым числом термов входит, по меньшей мере, столько же литералов.

Другими словами, минимальная сумма содержит наименьшее возможное число термов-произведений (равное числу вентилей на первом уровне и числу входов вентилей на втором уровне) и, кроме этого, наименьшее возможное число

литералов (равное числу входов вентиля на первом уровне). Таким образом, среди трех вариантов схем для обнаружения простых чисел, только схема, приведенная на рис. 4.30 является реализацией минимальной суммы.

Следующее определение точно устанавливает смысл слов «влечет за собой», когда мы говорим о логических функциях:

- Логическая функция $P(X_1, \dots, X_n)$ *влечет за собой* (*implies*) логическую функцию $F(X_1, \dots, X_n)$, если для каждой комбинации переменных, такой что $P = 1$, имеет место также $F = 1$.

Иначе это можно выразить так: если P влечет за собой F , то F равно 1 для каждой комбинации переменных, для которой $P = 1$, и, может быть, еще при каких-нибудь комбинациях. Такую связь между P и F можно символически записать в виде $P \Rightarrow F$. Об этой связи между P и F можно сказать также, что « F включает в себя (*includes*) P » или что « F покрывает (*covers*) P ».

- *Простая импликанта* (*prime implicant*) логической функции $F(X_1, \dots, X_n)$ — это нормальный терм-произведение $P(X_1, \dots, X_n)$, который влечет за собой F , причем такой, что если любую переменную удалить из P , то получающийся при этом терм-произведение уже не влечет за собой F .

На языке карты Карно простая импликанта функции F представляет собой объединенный набор клеток, содержащих 1, удовлетворяющий нашим правилам объединения, такой, что если мы попытаемся его расширить (покрыть вдвое большее число клеток), то в него обязательно попадет, по крайней мере, один 0.

Теперь пришел черед самого важного; речь идет о теореме, которая устанавливает предел того, сколько труда нужно затратить на поиск минимальной суммы логической функции:

Теорема о простых импликантах.

Минимальная сумма является суммой простых импликант. Это означает, что при поиске минимальной суммы не нужно рассматривать термы-произведения, не являющиеся простыми импликантами. Эта теорема легко доказывается от противного. Предположим, что терм-произведение P в «минимальной» сумме не является простой импликантой. Тогда, по определению простой импликанты, из P можно удалить какой-то литерал, если только P не является единицей, и получить новый терм-произведение P^* , который все еще влечет за собой F . Если мы заменим P на P^* в сумме, относительно которой мы предположили, что она минимальна, то получающаяся в результате этого сумма будет по-прежнему равна F , но будет содержать на один литерал меньше. Поэтому исходная сумма, фигурировавшая в нашем предположении, вовсе не была минимальной.

Рассмотрим в качестве другого примера минимизации функцию 4-х переменных, приведенную на рис. 4.31. В данном случае имеется две простые импликанты, и совершенно очевидно, что их обе необходимо включить в минимальную сумму, чтобы покрыть все клетки таблицы, содержащие 1. Мы не приводим принципиальную схему, поскольку теперь вы уже сами знаете, как это сделать.

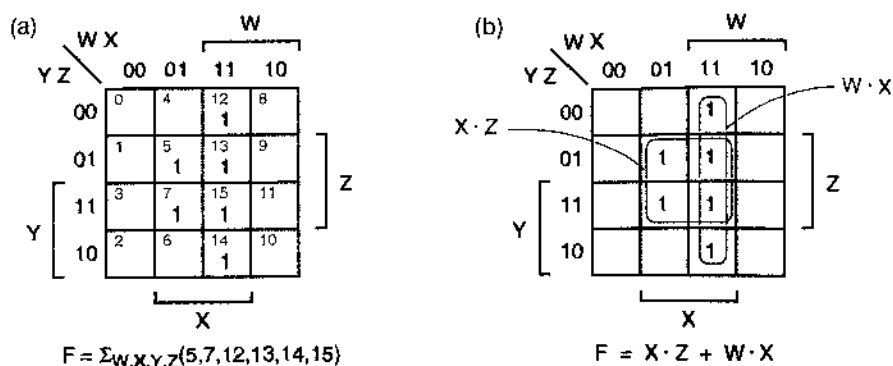


Рис. 4.31. $F = \sum_{w,x,y,z}(5, 7, 12, 13, 14, 15)$: (a) карта Карио; (b) простые импликанты

Сумма всех простых импликант называется *полной суммой* (*complete sum*). Хотя полная сумма всегда указывает допустимый способ реализации логической функции, она не всегда является минимальной. Рассмотрим, например, логическую функцию, представленную на рис. 4.32. У нее пять простых импликант, но минимальная сумма включает только три из них. Спрашивается, как можно систематически определять, какие импликанты следует брать, а какие — отбрасывать? Нужны еще два определения:

- *Особенная клетка, содержащая 1 (distinguished 1-cell)*, — это такая клетка, которая соответствует комбинации переменных, покрываемой только одной простой импликантой.
- *Существенная простая импликанта (essential prime implicant)* логической функции — это простая импликанта, покрывающая одну особенную клетку, содержащую 1, или большее их число.

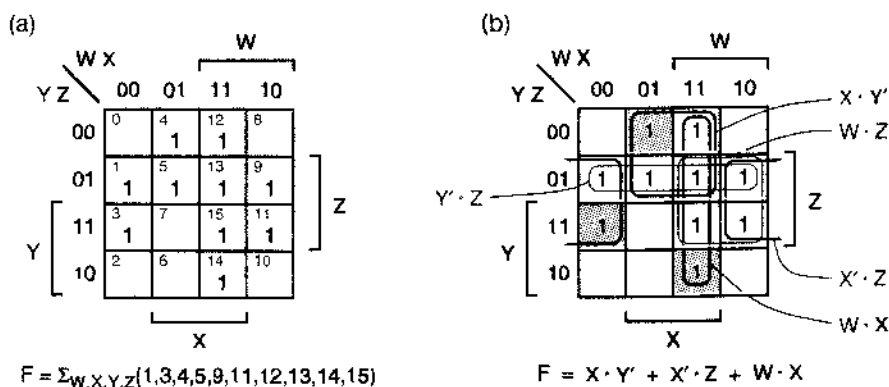


Рис. 4.32. $F = \sum_{w,x,y,z}(1, 3, 4, 5, 9, 11, 12, 13, 14, 15)$: (a) карта Карио; (b) простые импликанты и особенные клетки, содержащие 1

Поскольку существенная простая импликанта является *единственной* простой импликантой, покрывающей одну из клеток, содержащих 1, она должна входить в

любую минимальную сумму данной логической функции. Таким образом, первый шаг в процедуре выбора простых импликант совсем прост: мы устанавливаем, какие из клеток, содержащих 1, являются особенными, берем соответствующие им простые импликанты и включаем их в качестве существенных простых импликант в минимальную сумму. После этого остается лишь определить, как покрыть остающиеся клетки, содержащие 1, — если таковые имеются, — не покрытые существенными простыми импликантами. В примере на рис. 4.32 три особенные клетки, содержащие 1, заштрихованы, а соответствующие им существенные простые импликанты обведены более жирными линиями. В этом примере все клетки, содержащие 1, покрываются существенными простыми импликантами, так что больше делать ничего не нужно. Аналогично обстоит дело в примере на рис. 4.33, где все простые импликанты являются существенными и поэтому все они включаются в минимальную сумму.

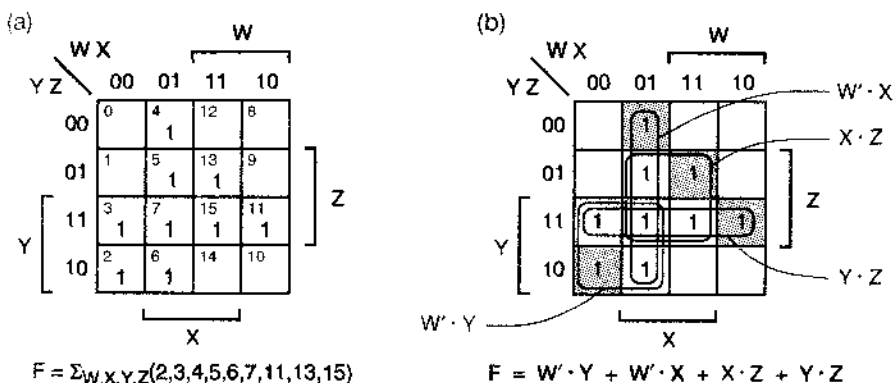


Рис. 4.33. $F = \sum_{W,X,Y,Z}(2, 3, 4, 5, 6, 7, 11, 13, 15)$: (a) карта Карно; (b) простые импликанты и особенные клетки, содержащие 1

Логическая функция, у которой не все клетки, содержащие 1, покрываются существенными простыми импликантами, приведена на рис. 4.34. Исключая существенные простые импликанты и покрываемые ими клетки, содержащие 1, мы получаем редуцированную карту, в которой имеется лишь одна клетка, содержащая 1, которую покрывают две простые импликанты. В данном случае осуществить выбор между ними легко: мы берем терм-произведение $W' \cdot Z$, так как в нем меньше переменных и поэтому соответствующая ему логическая схема с меньшим числом входов дешевле.

В более сложных случаях нам понадобится еще одно определение:

- О двух простых импликантах P и Q , относящихся к редуцированной карте, говорят, что P *перекрывает* (*eclipses*) Q (пишется: $P \dots Q$), если P покрывает по меньшей мере все клетки, содержащие 1, покрываемые Q .

Если стоимость P не больше, чем стоимость Q , и если P перекрывает Q , то исключение Q из дальнейшего рассмотрения не может помешать нам найти минимальную сумму: другими словами, простая импликанта P , по крайней мере, так же хороша, как и простая импликанта Q .

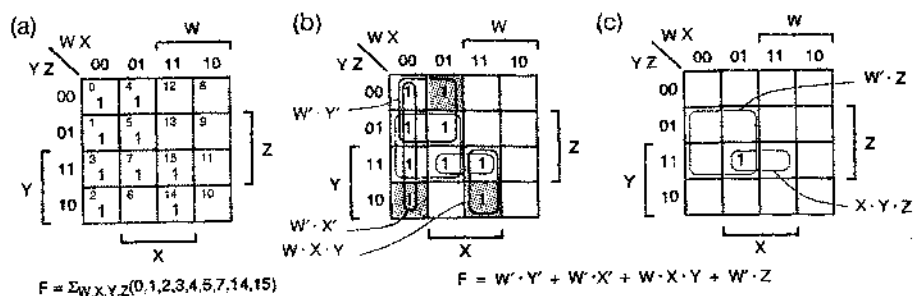


Рис. 4.34. $F = \Sigma_{wxyz}(0, 1, 2, 3, 4, 5, 7, 14, 15)$: (a) карта Карно; (b) простые импликанты и особые клетки, содержащие 1; (c) редуцированная карта после исключения существенных простых импликант и покрываемых ими клеток, содержащих 1

Пример перекрытия представлен на рис. 4.35. После исключения существенных простых импликант у нас остаются две клетки, содержащих 1, каждая из которых покрывается двумя простыми импликантами. Однако простая импликанта $X \cdot Y \cdot Z$ не покрывает две другие простые импликанты, которые, таким образом, можно исключить из рассмотрения. В данном случае две клетки, содержащие 1, покрываются единственной простой импликантой $X \cdot Y \cdot Z$, которая является *существенной простой импликантой второго порядка (secondary essential prime implicant)* и которая должна быть включена в минимальную сумму.

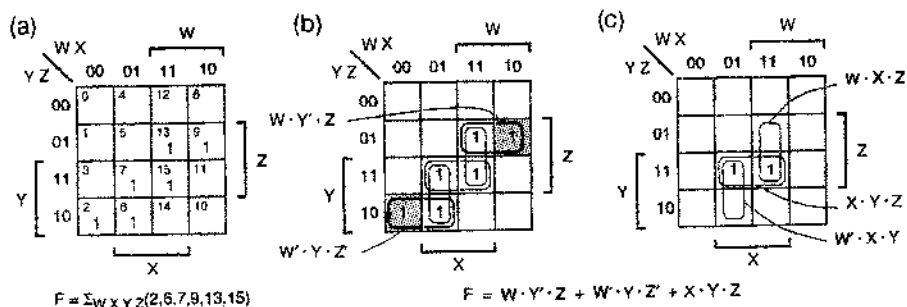


Рис. 4.35. $F = \Sigma_{wxyz}(2, 6, 7, 9, 13, 15)$: (a) карта Карно; (b) простые импликанты и особые клетки, содержащие 1; (c) редуцированная карта после исключения существенных простых импликант и покрываемых ими клеток, содержащих 1

На рис. 4.36 показан более трудный случай: здесь у логической функции нет существенных простых импликант. Для этой функции методом проб и ошибок можно найти две различные минимальные суммы.

Возможен другой систематический подход к данной проблеме, который называется *методом ветвления (branching method)*. Начиная с любой клетки, содержащей 1, мы произвольно выбираем покрывающую ее простую импликанту и включаем ее в наше рассмотрение, как если бы она была существенной. Это упрощает оставшуюся часть задачи, которую можно решить обычным способом нахождения минимальной суммы. Этот процесс повторяется, начиная с

каждой другой простой импликанты, покрывающей начальную клетку, содержащую 1; результатом каждый раз будет новая гипотетическая минимальная сумма. На этом пути можно слоткнуться, и тогда метод ветвления необходимо применять рекурсивно. В конце концов мы сравним все полученные таким образом гипотетические минимальные суммы и выбираем одну из них, которая является минимальной на самом деле.

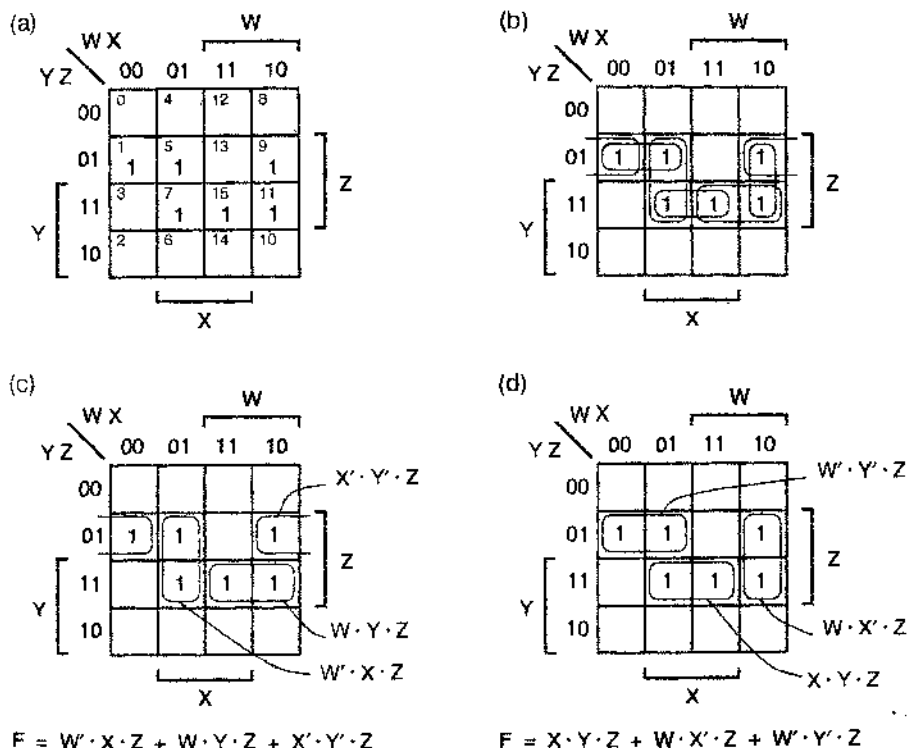


Рис. 4.36. $F = \sum_{w,x,y,z} (1, 5, 7, 9, 11)$: (a) карта Карно; (b) простые импликанты; (c) минимальная сумма; (d) другая минимальная сумма

4.3.6. Упрощение произведений сумм

На основании принципа двойственности можно минимизировать выражения вида «произведение сумм», рассматривая нули на карте Карно. Каждый 0 на карте соответствует макстерму в каноническом произведении логической функции. Все, что было изложено в предыдущем разделе, можно переформулировать в плане двойственности, включая правила составления термов-сумм, соответствующих объединенным наборам нулей, в результате чего находится *минимальное произведение* (*minimal product*).

Если, однако, нам известно, как находить минимальные суммы, то отыскание минимального произведения для данной логической функции, к счастью, оказывается более легким. На первом шаге берется дополнение F' функции F . Если

функция F представлена в виде списка минтермов или в виде таблицы истинности, то найти дополнение совсем легко: просто нули в F заменяются в F' на единицы. Затем мы находим минимальную сумму для F' по правилам, описанным в предыдущем разделе. Наконец, по обобщенной теореме Де Моргана берем дополнение к результату, что и приводит нас к минимальному произведению для $(F')' = F$. (Обратите внимание, что простое «разнесение слагаемых по сомножителям» в минимальной сумме для исходной функции не гарантирует того, что получаемое при этом произведение сумм будет минимальным; см., например, задачу 4.61.)

В общем случае, для того чтобы построить двухуровневую схему, которая была бы самой дешевой реализацией некоторой логической функции, необходимо найти как минимальную сумму, так и минимальное произведение, и сравнить их. Если в минимальной сумме данной логической функции много слагаемых, то минимальное произведение той же самой логической функции может состоять из меньшего числа сомножителей. Вот тривиальный пример такого сопоставления — функция ИЛИ 4-х переменных:

$$\begin{aligned} F &= (W) + (X) + (Y) + (Z) \quad (\text{сумма четырех элементарных термов-произведений}) \\ &= (W + X + Y + Z) \quad (\text{произведение, состоящее из одного термина-суммы}). \end{aligned}$$

В качестве нетривиального примера приглашаем вас найти минимальное произведение для функции на рис. 4.33, которую мы минимизировали выше; в минимальном произведении всего два термина-суммы.

Иногда бывает верно и обратное, как это видно на простейшем примере функции И 4-х переменных:

$$\begin{aligned} F &= (W) \cdot (X) \cdot (Y) \cdot (Z) \quad (\text{произведение четырех элементарных термов-сумм}) \\ &= (W \cdot X \cdot Y \cdot Z) \quad (\text{сумма, состоящая из одного термина-произведения}). \end{aligned}$$

МИНИМИЗАЦИЯ ПРИМЕНительно К ПЛУ

Структура типичного ПЛУ представляет собой решетку И-ИЛИ, соответствующую логическому выражению вида «сумма произведений», поэтому можно подумать, что при проектировании на основе ПЛУ нужна только минимальная сумма произведений. Однако у большинства ПЛУ на выходах решетки И-ИЛИ имеются программируемые инверторы/буферы, с помощью которых любой из выходных сигналов легко инвертировать или пропускать без инверсии. Таким образом, применительно к ПЛУ можно воспользоваться эквивалентом минимальной суммы и на решетке И-ИЛИ реализовать дополнение желаемой функции, а затем осуществить инвертирование, соответственно запрограммировав инвертор/буфер. Большинство программ минимизации логических функций для ПЛУ автоматически находят как минимальную сумму, так и минимальное произведение и выбирают ту из форм, в которой меньше термов.

Нетривиальным примером является функция, приведенная на рис. 4.29, для которой стоимость реализации произведения сумм больше.

У некоторых логических функций стоимость реализации для обеих форм одинакова. Рассмотрим, например, функцию ИСКЛЮЧАЮЩЕЕ ИЛИ 3-х переменных; каждое из минимальных выражений состоит из четырех термов и в каждом терме содержатся три литерала:

$$\begin{aligned} F &= \sum_{XYZ} (1, 2, 4, 7) \\ &= (X' \cdot Y' \cdot Z) + (X' \cdot Y \cdot Z') + (X \cdot Y' \cdot Z') + (X \cdot Y \cdot Z) \\ &= (X + Y + Z) \cdot (X + Y' + Z') \cdot (X' + Y + Z') \cdot (X' + Y' + Z). \end{aligned}$$

Но все же в большинстве случаев либо одна, либо другая форма дает лучшие результаты. Рассмотреть обе формы особенно полезно при проектировании устройств на основе ПЛУ.

*4.3.7. «Безразличные» комбинации переменных

Иногда требования, предъявляемые к комбинационной схеме, таковы, что не имеет значения, какой сигнал будет возникать на ее выходах при определенных комбинациях входных сигналов. Такие комбинации называются *безразличными* (*don't-care*). Это может иметь место по той причине, что значения выходного сигнала при этих комбинациях сигналов на входах и в самом деле не играют никакой роли, либо потому, что при нормальной работе схемы эти комбинации входных сигналов никогда не возникают. Предположим, например, что мы хотим создать устройство для обнаружения простых чисел, 4-разрядное двоичное слово $N = N_3N_2N_1N_0$ на входах которого всегда является двоично-десятичной цифрой; в этом случае минтермы 10–15 никогда не нужно будет принимать во внимание. С учетом этого функцию, реализуемую устройством, предназначенным для обнаружения простых двоично-десятичных чисел, можно записать так:

$$F = \sum_{N_3N_2N_1N_0} (1, 2, 3, 5, 7) + d(10, 11, 12, 13, 14, 15).$$

Здесь $d(\dots)$ – синтаксис безразличных комбинаций переменных для данной функции, образующих так называемое *d-множество* (*d-set*). Согласно этой записи функция F должна принимать значение 1 при комбинациях переменных из множества включений $(1, 2, 3, 5, 7)$, может иметь любые значения при комбинациях переменных из d -множества и должна равняться 0 при всех других комбинациях переменных (принадлежащих 0-множеству).

На рис. 4.37 показано, как найти минимальную реализацию суммы произведений для устройства, обнаруживающего простые двоично-десятичные числа, с учетом безразличных комбинаций переменных. Клетки, соответствующие безразличным комбинациям переменных, помечены на карте буквой d . Процедура обведения наборов единиц (простых импликант) видоизменяется следующим образом:

- При обводе единиц допускается включение клеток, помеченных буквой d , с целью получения наборов возможно большего размера. Это сокращает число переменных в соответствующих простых импликантах. В примере на рис. 4.37 таких простых импликант две: $N_2 \cdot N_0$ и $N_2' \cdot N_1$.

- Не обводятся наборы, состоящие только из клеток, помеченных буквой d. Если бы соответствующие термы-произведения были приняты во внимание, то это привело бы к ненужному увеличению стоимости реализации. В примере на рис. 4.37 таких термов-произведений два: $N_3 \cdot N_2$ и $N_3 \cdot N_1$.
- Как обычно, не обводятся никакие клетки, содержащие 0. (Это просто наивно.)

Остальная часть процедуры – та же, что и раньше. В частности, мы ищем особые клетки, содержащие 1, и не особые клетки, помеченные буквой d, и включаем в составленное логическое выражение только существенные простые импликанты и такие другие импликанты, которые необходимы, чтобы покрыть все единицы на карте. В примере на рис. 4.37 двух существенных простых импликант достаточно, чтобы покрыть все единицы на карте. При этом оказываются покрытыми также две клетки, помеченные буквой d, так что при безразличных комбинациях переменных 10 и 11 функция F будет иметь значение 1, а при других безразличных комбинациях переменных – 0.

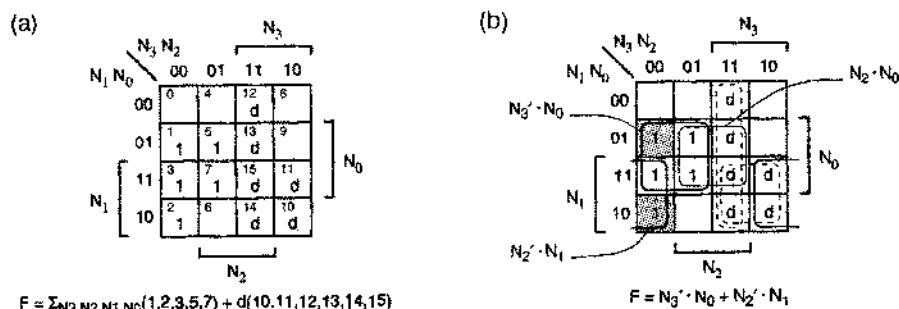


Рис. 4.37. Устройство для обнаружения простых чисел в двоично-десятичном представлении: (a) исходная карта Карно; (b) карта Карно с простыми импликантами и особыми клетками, содержащими 1

Некоторые языки описания схем, в том числе язык ABEL, содержат средства, с помощью которых разработчик указывает безразличные комбинации входных сигналов, и программа логической минимизации учитывает эти комбинации при нахождении минимальной суммы.

*4.3.8. Минимизация схем со многими выходами

На практике у большинства комбинационных логических схем число выходов бывает больше одного. Задачу минимизации схемы с n выходами всегда можно свести к решению n независимых задач с единственным выходом. Однако, поступая так, мы можем пропустить те или иные возможности оптимизации. Рассмотрим, например, следующие две логические функции:

$$F = \sum_{x,y,z} (3, 6, 7)$$

$$G = \sum_{x,y,z} (0, 1, 3).$$

На рис. 4.38 представлена схема с выходами F и G, получающаяся в результате решения двух независимых задач минимизации схем с одним выходом, относя-

щихся к функциям F и G порознь. Однако, как показано на рис. 4.39, можно найти также такую пару выражений вида «сумма произведений», в каждую из которых входит один и тот же терм-произведение, в результате чего получается схема, содержащая на один вентиль меньше, чем в первоначальном варианте.

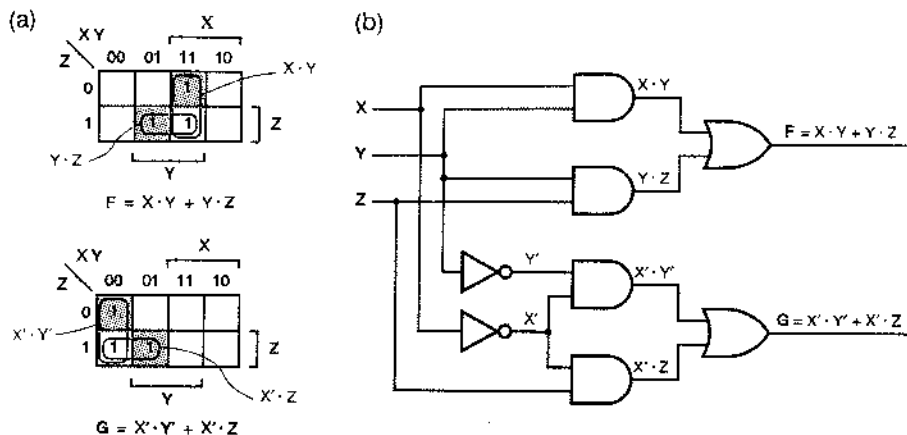


Рис. 4.38. Задача о проектировании схемы с двумя выходами, рассматриваемая как две независимые задачи о построении схем с одним выходом: (a) карты Карно; (b) «минимальная» схема

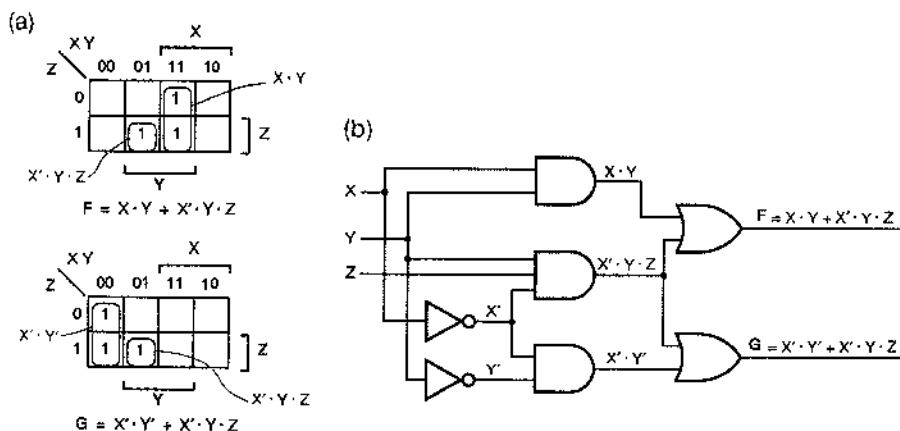


Рис. 4.39. Минимизация схемы с двумя выходами: (a) минимизируемые карты и выделение общего терма; (b) минимальная схема, составленная с учетом двух выходов

При проектировании комбинационных схем со многими выходами на основе отдельных вентилей, как это имеет место в случае специализированных ИС, наличие общих термов-произведений очевидным образом приводит к уменьшению размера схемы и ее стоимости. Кроме того, в ПЛУ структура, реализующая сумму произведений, повторена много раз, и мы знаем теперь, как минимизировать такую структуру отдельно для каждого выхода; в некоторых ПЛУ допускается со-

вместное использование термов-произведений схемами, относящимися к разным выходам. Иден, излагаемые в этом разделе, использованы во многих программах логической минимизации.

Возможно, что в примере, приведенном на рис. 4.39, вы смогли непосредственно «угадать» минимальное решение, рассматривая карты Карно для функций F и G . Однако минимизацию схем большего размера можно осуществить только путем применения формального алгоритма минимизации для случая многих выходов. Сейчас этот алгоритм в общих чертах будет описан; подробнее с ним можно ознакомиться по литературе.

Ключом к успешной минимизации схемы со многими выходами, описываемой набором из n функций, является рассмотрение не только самих исходных n функций, относящихся к отдельным выходам, но также и «функций-произведений». m -произведением (m -product function) для набора из n функций называется функция, являющаяся произведением m функций, где $2 \leq m \leq n$. Существует $2^n - n - 1$ таких функций. В нашем примере, к счастью, $n = 2$ и есть только одна функция-произведение $F \cdot G$, которую надо рассматривать. На рис. 4.40 повторены карты Карно для функций F и G и приведена карта Карно для $F \cdot G$; в общем случае карта для m -произведения получается путем операции И над картами всех m его компонент.

Простая импликанта схемы со многими выходами (*multiple-output prime implicant*) для набора из n функций — это простая импликанта одной из n функций или одной из функций-произведений. Первый шаг минимизации в случае многих выходов состоит в нахождении всех простых импликант схемы со многими выходами. Каждая простая импликанта m -произведения является кандидатом на включение в логические выражения, относящиеся к соответствующим m выходам схемы. Если бы мы попытались минимизировать набор из 8 функций, мы должны были бы найти простые импликанты для $2^8 - 8 - 1 = 247$ функций-произведений, а также для 8 заданных функций. Очевидно, что минимизация схемы со многими выходами — занятие не для слабонервных!

После того, как простые импликанты схемы со многими выходами найдены, мы пытаемся упростить задачу, выбирая существенные среди них. *Особенной клеткой, содержащей 1* (*distinguished 1-cell*), на карте, относящейся к той или иной функции одиночного выхода F , называется содержащая 1 клетка, которая покрывается точно одной простой импликантой функции F или простой импликантой одной из функций-произведений, в которые входит F . На рис. 4.40 особенные клетки, содержащие 1, заштрихованы. *Существенной простой импликантой* (*essential prime implicant*) определенной функции одиночного выхода является простая импликанта, покрывающая особенную клетку, содержащую 1. Как и в случае минимизации с одним выходом, существенные простые импликанты должны быть включены в решение, обеспечивающее минимальную стоимость реализации. В остающейся части алгоритма речь идет только о клетках, содержащих 1, не покрываемых существенными простыми импликантами.

Заключительный этап состоит в выборе минимального набора простых импликант, чтобы покрыть остающиеся клетки, содержащие 1. На этом этапе мы должны рассматривать все n функций одновременно, включая возможность совместного использования одних и тех же компонентов; детали этой процедуры

обсуждаются в Обзоре литературы. В примере на рис. 4.40(с) мы видим, что существует терм-произведение, который покрывает остающиеся клетки, содержащие 1, в F и G одновременно.

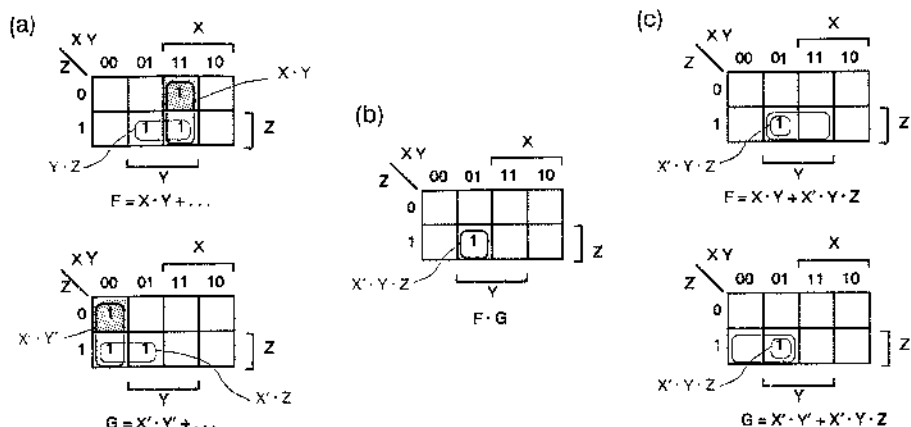


Рис. 4.40. Карты Карно для совокупности, состоящей из двух функций: (а) карты для функций F и G; (б) карта для 2-произведения $F \cdot G$; (с) редуцированные карты для F и G после исключения существенных простых импликант и покрываемых ими единиц

*4.4. Программные методы минимизации

Очевидно, что логическая минимизация может быть очень сложной и запутанной процедурой. При практическом проектировании логических схем вы, вероятнее всего, встретитесь с задачами минимизации двоякого рода, а именно: со случаями функций небольшого числа переменных — с этими случаями вы сможете разобраться сами, пользуясь описанными выше методами, — и с более сложными схемами со многими выходами, в отношении которых вы будете беспомощны, не имея под рукой подходящей программы минимизации.

Вы знаете, что в случае функций небольшого числа переменных минимизацию можно осуществить визуально, используя карты Карно. В этом параграфе мы покажем, что те же самые операции можно производить (по крайней мере, в принципе) в случае функций со сколь угодно большим числом переменных, используя табличный метод, называемый *алгоритмом Куина–Мак-Класки (Quine–McClusky algorithm)*. Подобно другим алгоритмам, алгоритм Куина–Мак-Класки можно представить в виде программы для компьютера. Как и в методе, основанном на картах Карно, алгоритм выполняется в два шага: (а) нахождение всех простых импликант функций и (б) выбор минимального набора простых импликант, который покрывает функцию.

Алгоритм Куина–Мак-Класки часто описывают в терминах заполняемых вручную таблиц и выполняемых вручную проверок. Поскольку, однако, никто никогда не делает этого вручную, для нас более естественно описать данный алгоритм в терминах структур данных и функций на языке программирования высокого уровня. Цель этого параграфа заключается в том, чтобы дать представление о сложности

сти вычислений, производимых при решении большой задачи минимизации. Мы рассматриваем только полностью определенные функции для логических схем с одним выходом; задачи с безразличными комбинациями переменных и со многими выходами решаются на основе алгоритмов, являющихся непосредственной модификацией алгоритма, применяемого в случае схем с одним выходом (см. Обзор литературы).

*4.4.1. Представление термов-произведений

Отправным моментом в алгоритме минимизации Куина–Мак-Класки служит таблица истинности или эквивалентный ей список минтермов для рассматриваемой функции. Если функция задана иначе, то сначала ее необходимо преобразовать в одну из упомянутых форм. Например, в произвольном логическом выражении с n переменными можно разнести множители по слагаемым (возможно, применяя по ходу дела теорему Де Моргана), чтобы прийти к выражению вида «сумма произведений». Когда такое выражение получено, каждый терм-произведение с p переменными порождает 2^{n-p} минтермов в списке минтермов.

В разделе 4.1.6 было объяснено, что минтерм логической функции n переменных можно представить в виде n -разрядного двоичного целого числа (номера минтерма), в котором каждый бит указывает, берется ли дополнение соответствующей переменной, или она входит в минтерм непосредственно. Однако алгоритму минимизации приходится иметь дело также с термами-произведениями, которые не являются минтермами, поскольку какие-то переменные в них отсутствуют вовсе. Таким образом, в общем случае мы должны предусмотреть три возможности для каждой переменной в терме-произведении:

- 1 — переменная входит в терм-произведение непосредственно;
- 0 — переменная входит в терм-произведение в виде дополнения;
- x — переменная отсутствует в терме-произведении.

Строка из n таких символов служит представлением терма-произведения в виде куба (*cube representation*). Если, например, число переменных в терме-произведении может достигать до 8 ($X_7, X_6, \dots, X_1, X_0$), то запись термов-произведений в виде кубов имеет вид:

$$\begin{aligned} X_7' \cdot X_6 \cdot X_5 \cdot X_4' \cdot X_3 \cdot X_2 \cdot X_1 \cdot X_0' &\equiv 01101110, \\ X_3 \cdot X_2 \cdot X_1 \cdot X_0' &\equiv \text{xxxx}1110, \\ X_7 \cdot X_5' \cdot X_4 \cdot X_3 \cdot X_2' \cdot X_1 &\equiv 1x01101x, \\ X_6 &\equiv x1\text{xxxxxx}. \end{aligned}$$

Заметьте, что, ради удобства, мы назвали переменные именно так, как бывают указаны разряды в n -разрядном двоичном целом числе.

Согласно терминологии параграфа 2.14, где были введены n -мерные кубы и m -мерные подкубы, строка $1x01101x$ представляет собой 2-мерный подкуб 8-мерного куба, а строка 01101110 — 0-мерный подкуб 8-мерного куба. Однако в литературе по минимизации обычно подразумевается, что максимальная размерность куба или подкуба равна n , и поэтому m -мерный подкуб просто называют « m -мерным кубом» или, для краткости, «кубом»; в данном параграфе мы будем следовать этой традиции.

Чтобы задать терм-произведение в компьютерной программе, можно воспользоваться структурой данных с n элементами, каждый из которых принимает одно из трех возможных значений. На языке C терм-произведение можно описать следующими объявлениями:

```
typedef enum {complemented, uncomplemented, doesntappear} TRIT;
typedef TRIT[16] CUBE; /* Represents a single product
                        term with up to 16 variables */
```

Однако эти объявления не приводят к сколько-нибудь эффективному внутреннему представлению кубов. Как мы увидим дальше, с кубами легче манипулировать, используя обычные компьютерные операторы, если терм-произведение с n переменными представлен в программе двумя n -разрядными словами, как это предлагается сделать в следующем примере:

```
#define MAX_VARS 16 /* Max # of variables in a product term */
typedef unsigned short WORD; /* Use 16-bit words */
struct cube {
    WORD t; /* Bits 1 for uncomplemented variables. */
    WORD f; /* Bits 1 for complemented variables. */
};
typedef struct cube CUBE;
CUBE P1, P2, P3; /* Allocate three cubes for use by program. */
```

Здесь слово WORD — это 16-разрядное двоичное целое; терм-произведение с 16 переменными представлен двумя словами WORD, как показано на рис. 4.41(а). В первом слове в структуре CUBE битом 1 отмечается каждая переменная, входящая в терм-произведение непосредственно [символ t — от «true» (истина)]; во втором слове битом 1 отмечается каждая переменная, входящая в терм-произведение в виде дополнения [символ f — от «false» (ложь)]. Если в каком-то определенном разряде стоит 0 в обоих словах WORD, то соответствующая переменная отсутствует; наличие 1 в одном и том же разряде в обоих словах WORD не предусматривается. Таким образом, программная переменная P1 на рис. 4.41(б) представляет собой терм-произведение $P1 = X_{15} \cdot X_{12}' \cdot X_{10}' \cdot X_9 \cdot X_4' \cdot X_1 \cdot X_0$. При желании представить логическую функцию F с числом переменных до 16, содержащую не более 100 термов-произведений, мы могли бы объявить массив из 100 структур CUBE:

```
CUBE F[100]; /* Storage for a logic function
              with up to 100 product terms. */
```

Воспользовавшись описанным представлением куба, можно на языке C кратко и эффективно записать функции, оперирующие термами-произведениями обычным образом. В табл. 4.8 приведены несколько таких функций. В соответствии с двумя из этих функций на рис. 4.42 показано, как можно сравнить два куба и объединить их, если это возможно, согласно теореме T10: $\text{term} \cdot X + \text{term} \cdot X' = \text{term}$. Эта теорема говорит о том, что два термина-произведения можно объединить, если они различаются только в одной переменной, которая в одном из термов входит в виде дополнения, а в другой — непосредственно. Объединение двух

m -мерных кубов дает $(m + 1)$ -мерный куб. Используя представление термов-произведений в виде кубов, можно проиллюстрировать применение теоремы об объединении следующими примерами:

$$\begin{aligned} 010 + 000 &= 0x0, \\ 00111001 + 00111000 &= 0011100x, \\ 101xx0x0 + 101xx1x0 &= 101xxx0, \\ x111xx00110x000x + x111xx00010x000x &= x111xx00x10x000x. \end{aligned}$$

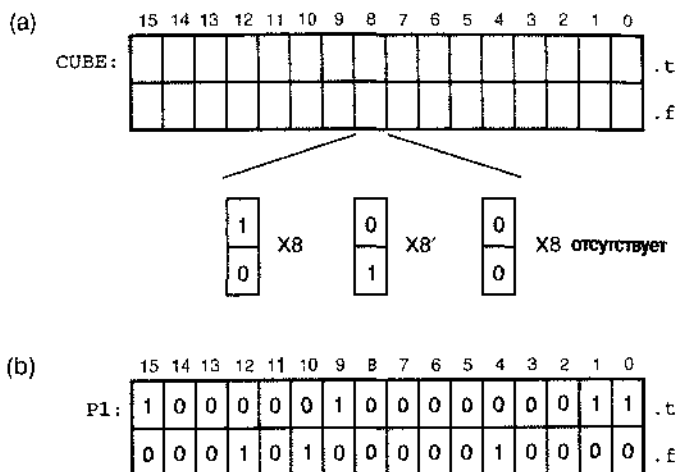


Рис. 4.41. Внутреннее представление термов-произведений с числом переменных до 16 в программе на языке C: (a) формат в общем случае; (b) $P1 = X15 \cdot X12' \cdot X10' \cdot X9 \cdot X4' \cdot X1 \cdot X0$

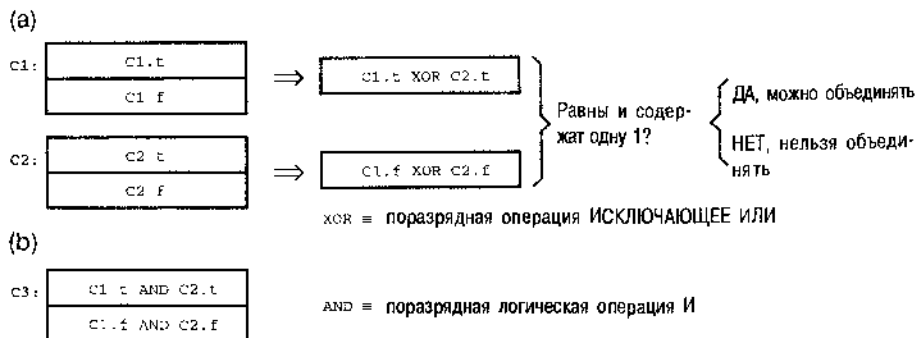


Рис. 4.42. Операции с кубами: (a) определение того, можно ли объединить два куба на основании теоремы T10: $\text{term} \cdot X + \text{term} \cdot X' = \text{term}$; (b) объединение термов по теореме T10

Табл. 4.8. Функции сравнения и объединения кубов, используемые в программе минимизации

```

int EqualCubes(CUBE C1, CUBE C2)      /* Returns true if C1 and C2 are identical. */
{
    return ( (C1.t == C2.t) && (C1.f == C2.f) );
}

int Oneone(WORD w)                    /* Returns true if w has exactly one 1 bit.      */
{                                     /* Optimizing the speed of this routine is critical */
    int ones, b;                      /* and is left as an exercise for the hacker.      */
    ones = 0;
    for (b=0; b<MAX_VARS; b++) {
        if (w & 1) ones++;
        w = w>>1;
    }
    return((ones==1));
}

int Combinable(CUBE C1, CUBE C2)
{
    WORD twordt, twordf;              /* Returns true if C1 and C2 differ in only one variable, */
                                      /* which appears true in one and false in the other.      */

    twordt = C1.t ^ C2.t;
    twordf = C1.f ^ C2.f;
    return( (twordt==twordf) && Oneone(twordt) );
}

void Combine(CUBE C1, CUBE C2, CUBE *C3)
{                                     /* Combines C1 and C2 using theorem T10, and stores the */
    C3->t = C1.t & C2.t;              /* result in C3. Assumes Combinable(C1,C2) is true.      */
    C3->f = C1.f & C2.f;
}

```

*4.4.2. Нахождение простых импликант путем объединения термов-произведений

Первый шаг в алгоритме Куила–Мак–Класки состоит в определении всех простых импликант логической функции. С помощью карт Карно мы делаем это визуально, находя «возможно больший прямоугольный набор единиц». В данном алгоритме это осуществляется систематически путем повторного применения теоремы T10 для объединения имптермов, 1-мерных кубов, 2-мерных кубов и т.д.; в результате получаются кубы возможно большей размерности (наименьшие возможные термы-произведения), покрывающие только такие комбинации переменных, для которых значение функции равно 1.

В табл. 4.9 приведена программа на языке C, реализующая этот алгоритм для функций с числом переменных до 16. Для того чтобы оперировать m -мерными кубами, число которых равно MAX_VARS, используются двумерные массивы `cubes[m][j]` и `covered[m][j]`. 0-мерные кубы (имптермы) вводятся пользователем. Начиная с 0-мерных кубов, программа на каждом уровне проверяет все

возможные пары кубов и объединяет их, когда это возможно, в кубы следующего уровня. Те кубы, которые объединяются в куб следующего уровня, помечаются как «покрытые» (covered); остающиеся не покрытыми кубы являются простыми импликантами.

Табл. 4.9. Программа на языке C, которая находит простые импликанты по алгоритму Куипа-Мак-Класки

```

#define TRUE 1
#define FALSE 0
#define MAX_VARS 50

void main()
{
    CUBE cubes[MAX_VARS+1][MAX_CUBES];
    int covered[MAX_VARS+1][MAX_CUBES];
    int numCubes[MAX_VARS+1];
    int m;          /* Value of m in an m-cube, i.e., "level m." */
    int j, k, p;     /* Indices into the cubes or covered array. */
    CUBE tempCube;
    int found;

    /* Initialize number of m-cubes at each level m. */
    for (m=0; m<MAX_VARS+1; m++) numCubes[m] = 0;

    /* Read a list of minterms (0-cubes) supplied by the user, storing them */
    /* in the cubes[0,j] subarray, setting covered[0,j] to false for each */
    /* minterm, and setting numCubes[0] to the total number of minterms read. */
    ReadMinterms;

    for (m=0; m<MAX_VARS; m++)          /* Do for all levels except the last */
        for (j=0; j<numCubes[m]; j++)    /* Do for all cubes at this level */
            for (k=j+1; k<numCubes[m]; k++) /* Do for other cubes at this level */
                if (Combinable(cubes[m][j], cubes[m][k])) {
                    /* Mark the cubes as covered. */
                    covered[m][j] = TRUE; covered[m][k] = TRUE;
                    /* Combine into an (m+1)-cube, store in tempCube. */
                    Combine(cubes[m][j], cubes[m][k], &tempCube);
                    found = FALSE; /* See if we've generated this one before. */
                    for (p=0; p<numCubes[m+1]; p++)
                        if (EqualCubes(cubes[m+1][p], tempCube)) found = TRUE;
                    if (!found) { /* Add the new cube to the next level. */
                        numCubes[m+1] = numCubes[m+1] + 1;
                        cubes[m+1][numCubes[m+1]-1] = tempCube;
                        covered[m+1][numCubes[m+1]-1] = FALSE;
                    }
                }
    }

    for (m=0; m<MAX_VARS; m++)          /* Do for all levels */
        for (j=0; j<numCubes[m]; j++)    /* Do for all cubes at this level */
            /* Print uncovered cubes -- these are the prime implicants. */
            if (!covered[m][j]) PrintCube(cubes[m][j]);
}

```

Несмотря на то, что программа в табл. 4.9 является короткой, опытный программист может загрустить, разглядывая ее структуру. Глубина вложенных во внутреннем цикле `for` составляет четыре уровня, а число проходов, которые, возможно, необходимо будет выполнить, порядка $\text{MAX_VARS} \cdot \text{MAX_CUBES}^3$. Да-да, это показатель степени, а не сноска! Если выбрать величину MAX_CUBES равной 1000 (это значение в большой степени произвольно, но, в действительности, для многих функций этого может оказаться совсем не достаточно), то внутренний цикл будет выполняться миллиарды и миллиарды раз.

Конечно, максимальное число минтермов у функции n переменных равно 2^n , и поэтому, по всем правилам, следовало бы объявить в программе в табл. 4.9 значение MAX_CUBES равным не меньшей мере 2^{16} , чтобы иметь возможность обработать максимально возможное число 0-мерных кубов. Такое объявление не было бы чрезмерно завышенным. Если у функции n переменных есть терм-произведение, равный одной переменной, то фактически понадобятся 2^{n-1} минтермов, чтобы покрыть этот терм-произведение.

На самом деле, в случае больших кубов ситуация еще хуже. Число возможных m -мерных подкубов n -мерного куба равно $\binom{n}{m} \times 2^{n-m}$, где биномиальный коэффициент $\binom{n}{m}$ — это число способов, какими можно распределить нули и единицы по остающимся переменным. Для функции 16 переменных худший случай имеет место при $m = 5$; существует 8 945 664 возможных 5-мерных подкуба 16-мерного куба. Полное число различных m -мерных подкубов n -мерного куба для всех значений m равно 3^n . Так что в общем случае программе минимизации может потребоваться *гораздо* больше памяти, чем это предусмотрено в программе в табл. 4.9.

Существует несколько приемов, с помощью которых можно оптимизировать необходимый объем памяти и время счета в программе в табл. 4.9 (см. задачи 4.77–4.80), но их применение дает ничтожный результат по сравнению с непреодолимой комбинаторной сложностью задачи. Таким образом, даже при теперешних быстрых компьютерах и громадной памяти прямое применение алгоритма Куина–Мак-Класки для нахождения простых импликант обычно бывает ограничено функциями лишь небольшого числа переменных (менее 15–20).

*4.4.3. Нахождению минимального покрытия по таблице простых импликант

После того как найден список всех простых импликант комбинационной логической функции, наступает второй этап процедуры ее минимизации — выбор минимального подмножества простых импликант, покрывающих все единицы функции. В алгоритме Куина–Мак-Класки для этого используется двумерный массив, называемый *таблицей простых импликант* (*prime-implicant table*). На рис. 4.43(а) приведена небольшая, но показательная таблица простых импликант, возникающая в задаче минимизации логической функции с картой Карно, представленной на рис. 4.35. Каждому минтерму функции соответствует один столбец, а каждой простой импликанте — одна строка. Каждый элемент таблицы представляет

собой бит, равный 1 в том и только в том случае, когда простая импликанта данной строки покрывает минтерм данного столбца (на рисунке эти элементы отмечены галочкой).

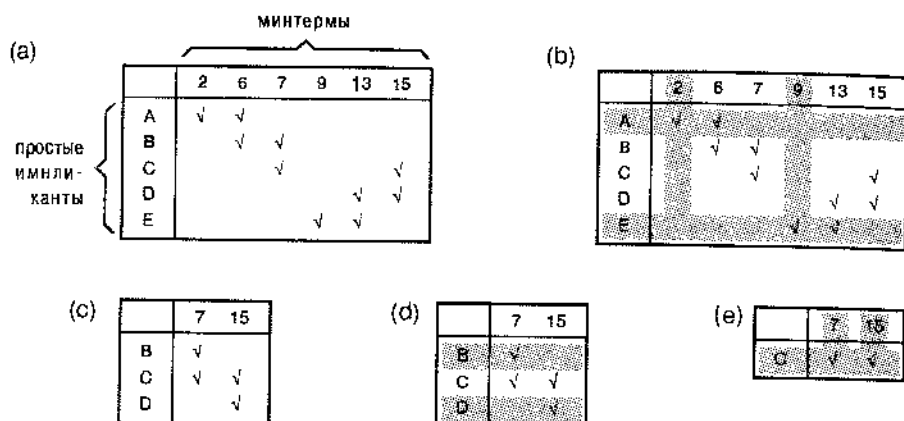


Рис. 4.43. Таблицы простых импликант: (а) исходная таблица; (б) выделение особых элементов, равных 1, и существенных простых импликант; (с) вид таблицы после исключения существенных простых импликант; (д) обнаружение перекрывающихся строк; (е) таблица с существенной простой импликантой второго порядка как результат исключения перекрывающихся строк

Выбор простых импликант по таблице осуществляется путем последовательного выполнения шагов, аналогичных тем, которые мы совершали в разделе 4.3.5, используя карты Карно:

1. Находим особые элементы, равные 1. Их легко найти в таблице, беря столбцы с единственной единицей, как показано на рис. 4.43 (б).
2. Включаем все существенные простые импликанты в минимальную сумму. Существенной простой импликантой соответствует строка, содержащая галочку в одном столбце с особым элементом, равным 1, или в большем числе таких столбцов.
3. Исключаем из рассмотрения существенные простые импликанты и покрываемые ими элементы, равные 1 (минтермы). В таблице это осуществляется вычеркиванием соответствующих строк и столбцов, выделенных цветом на рис. 4.43(б). Если остаются какие-либо строки, в которых нет галочек, то они тоже вычеркиваются; соответствующие *простые импликанты* являются *избыточными* (*redundant prime implicants*), то есть полностью покрываемыми существенными простыми импликантами. Оставшаяся на этом шаге таблица меньших размеров показана на рис. 4.43(с).
4. Исключаем из рассмотрения все простые импликанты, которые «непокрываются» другими простыми импликантами с той же или меньшей стоимостью реализации. В таблице это делается путем вычеркивания тех строк, у которых отмеченные галочками столбцы образуют подмножество множества

столбцов, отмеченных галочками в другой строке; вычеркиваются также все, кроме одной, строки в множестве строк с идентичными наборами столбцов, отмеченных галочками. Эти действия иллюстрируются рисунком (d); в результате этих действий таблица сокращается еще больше и принимает вид, указанный на рис. 4.43(е).

При реализации функции на ПЛУ можно считать, что все ее простые импликанты имеют одинаковую стоимость, поскольку ко входам всех вентилях И можно подвести все входные сигналы. В противном случае необходимо отсортировать и разобрать простые импликанты на группы по числу входов у вентилях И.

5. Находим особенные элементы, равные 1, и включаем в минимальную сумму все существенные простые импликанты второго порядка. Здесь, как и выше, существенной простой импликанте второго порядка соответствует строка, содержащая галочку в одном столбце с особенным элементом, равным 1, или в большем числе таких столбцов.
6. Если все оставшиеся столбцы покрываются существенными простыми импликантами второго порядка, как это имеет место на рис. 4.43(е), то задача решена. Если на предыдущем шаге существенные простые импликанты второго порядка не найдены, то мы возвращаемся к шагу 3 и повторяем наши действия. В противном случае необходимо воспользоваться методом ветвления, рассмотренным в разделе 4.3.5. Согласно этому методу берем строки по одной, предполагаем, что выбранная строка является существенной, и рекурсивно выполняем (чсртыхаясь) шаги 3–6.

Хотя алгоритм отбора простых импликант, основанный на таблице простых импликант, является довольно простым, необходимая структура данных в соответствующей компьютерной программе колоссальна, так как приходится оперировать с числом битов порядка $p \cdot 2^n$, где p — число простых импликант, а n — число битов на входе (предполагается, что заданная функция принимает значение 1 для большинства комбинаций переменных). Хуже всего, что для выполнения шагов, бесечно описанных нами выше в нескольких предложениях, требуются огромные по объему вычисления.

* 4.4.4. Другие методы минимизации

Хотя предыдущие разделы и служат введением в алгоритмы минимизации логических функций, эти методы ни в коем случае не являются самыми новыми и самыми замечательными. Подталкиваемые все возрастающей плотностью кристаллов СБИС, многие исследователи нашли более эффективные способы минимизации комбинационных логических функций. Их результаты можно отнести, грубо говоря, к одной из трех категорий:

1. *Вычислительные усовершенствования.* Обычно в улучшенных алгоритмах используются специально приспособленные структуры данных или в другой очередности выполняются шаги, что позволяет снизить требования к памяти и сократить время счета по классическим алгоритмам.
2. *Эвристические методы.* В некоторых случаях задачи минимизации оказываются слишком большими, чтобы их решать, используя «точный» алгоритм. С

такими задачами можно попытаться справиться путем сокращений и угадываний, основанных на хорошем знании предмета, результатом чего бывает уменьшение во много раз объема памяти и времени счета по сравнению с тем, что требует «точный» алгоритм. В противоположность выводу доказуемо минимального выражения для логической функции, эвристическими методами пытаются найти «почти минимальное» выражение.

Даже в тех случаях, когда задачу можно решить «точным» методом, эвристический подход часто приводит к лучшему решению, которое оказывается в десятки раз более быстрым. Самая удачная эвристическая программа Espresso-II справляется с большинством задач, давая минимальный или почти минимальный результат (сводя функцию к одному или к двум термам-произведениям), включая задачи с несколькими десятками переменных и несколькими сотнями термов-произведений.

3. *Другой взгляд на вещи.* Как упоминалось выше, минимизацию схем с несколькими выходами можно осуществить напрямую методами, которые представляют собой простейшую формальную модификацию методов минимизации схем с одним выходом. Однако, подойдя к задаче минимизации схем со многими выходами как к задаче многозначной (недвоичной) логики, авторам алгоритма Espresso-MV удалось достичь значительно лучших результатов по сравнению с Espresso-II.

Более подробно об этих методах см. ссылки в Обзоре литературы.

*4.5. Паразитные импульсы на выходе логических схем

Методы анализа, о которых шла речь в параграфе 4.2, не учитывают задержку в схеме и предсказывают только *поведение схемы в установившемся режиме (steady-state behavior)*. Другими словами, эти методы позволяют находить сигнал на выходе схемы как функцию входных сигналов в предположении, что входные сигналы сохраняют свои значения в течение относительно длительного времени по сравнению с задержками в электронных цепях. Однако в параграфе 3.6 было объяснено, что в действительности задержка между изменением входного сигнала и соответствующим изменением выходного сигнала у реальной логической схемы не равна нулю и зависит от многих факторов.

Из-за задержек *переходной процесс (transient behavior)* в логической схеме может отличаться от того, что предсказывает анализ ее поведения в установившемся режиме. В частности на выходе схемы может возникать короткий импульс (выброс, всплеск), часто называемый *паразитным импульсом (глюком, glitch)*, тогда как анализ поведения в установившемся режиме предсказывает, что выходной сигнал не должен изменяться. Говорят, что существует *источник опасности (hazard)*, когда в схеме может возникать такой паразитный импульс. Возникает паразитный импульс в действительности или нет – зависит от точных значений задержек и от других электрических характеристик схемы. Поскольку эти параметры трудно контролировать в серийном производстве схем, проектировщик должен побеспокоиться об устранении источников опасности (*возможности возникновения паразитного импульса*) даже в том случае, когда сбой может

наступить лишь при наихудшей комбинации логических значений и электрических параметров.

*4.5.1. Статические источники опасности

Единичный статический источник опасности (static-1 hazard) – это возможность возникновения нулевого паразитного импульса на выходе схемы, когда ожидается – согласно статическому анализу функции, реализуемой этой схемой, – что выходной сигнал будет в точности оставаться постоянным и равным 1. Формальное определение состоит в следующем:

Определение: Единичный статический источник опасности – это две комбинации входных сигналов, такие что: (а) при переходе от одной комбинации к другой меняется только один из входных сигналов, (б) обе комбинации дают 1 на выходе и возможно кратковременное появление на выходе 0 в течение переходного процесса, вызванного изменением входного сигнала.

Рассмотрим, например, логическую схему, приведенную на рис. 4.44(а). Предположим, что оба входных сигнала X и Y равны 1, а Z изменяется от 1 до 0. Тогда временные диаграммы будут такими, как показано на рис. 4.44(б), где мы предположили, что задержка распространения в каждом вентиле и инверторе равна одному и тому же единичному отрезку времени. Несмотря на то, что «статический» анализ предсказывает наличие 1 на выходе при обеих комбинациях входных сигналов $X, Y, Z = 111$ и $X, Y, Z = 110$, временные диаграммы показывают, что F проваливается до 0 в течение единичного отрезка времени при переходе Z от 1 до 0 из-за задержки в инверторе, на выходе которого вырабатывается сигнал Z' .

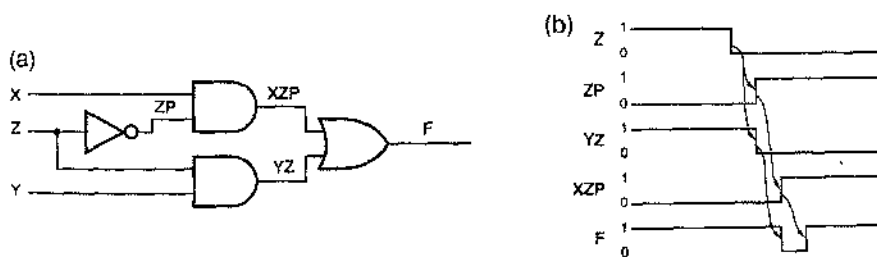


Рис. 4.44. Схема с единичным статическим источником опасности: (а) принципиальная схема; (б) временные диаграммы

Нулевой статический источник опасности (static-0 hazard) – это возможность возникновения единичного паразитного импульса, когда ожидается, что на выходе схемы будет постоянный 0:

Определение: Нулевой статический источник опасности – это две соседние комбинации входных сигналов, такие что: (а) при переходе от одной комбинации к другой меняется только один из входных сигналов, (б) обе комбинации дают 0 на выходе и возможно кратковременное появление на выходе 1 в течение переходного процесса, вызванного изменением входного сигнала.

Поскольку нулевой статический источник опасности является двойственным по отношению к единичному статическому источнику опасности, в схеме ИЛИ–И

двойственной по отношению к схеме, изображенной на рис. 4.44(а), имеется нулевой статический источник опасности.

Схема ИЛИ–И с четырьмя нулевыми источниками опасности представлена на рис. 4.45(а). Один из источников опасности имеет место тогда, когда $W, X, Y = 0$ и сигнал Z изменяется так, как показано на рис. 4.45(б). Вы, по-видимому, сможете сами найти три других источника опасности, а также устранить их все, после того как изучите следующий раздел.

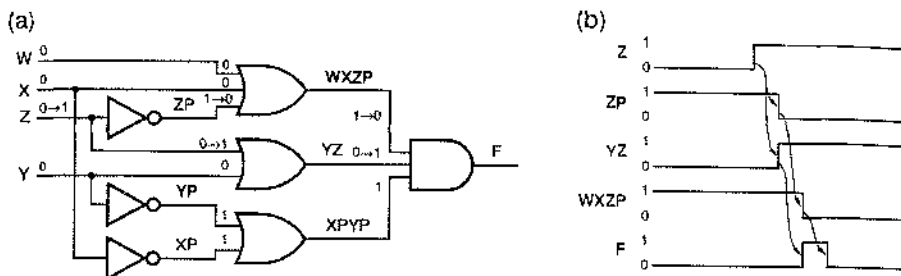


Рис. 4.45. Схема с нулевыми статическими источниками опасности: (а) принципиальная схема; (б) временные диаграммы

*4.5.2. Нахождение статических источников опасности по картам Карно

Карты Карно позволяют обнаружить статические источники опасности в двухуровневой схеме, реализующей сумму произведений или произведение сумм. Наличие или отсутствие источников опасности зависит от способа реализации заданной логической функции.

В правильно построенной двухуровневой схеме И–ИЛИ, реализующей сумму произведений, нет нулевых статических источников опасности. Такой источник опасности существовал бы в схеме указанного вида только в том случае, если бы на входы одного и того же вентиля И были поданы одновременно и сам входной сигнал и его инверсия, но это было бы глупо. Однако в такой схеме могут быть единичные статические источники опасности. Их существование можно предвидеть, разглядывая карту Карно, где обведены термы-произведения, соответствующие в схеме вентилям И.

На рис. 4.46(а) показана карта Карно для схемы, приведенной на рис. 4.44. Из карты следует, что не существует одного терма-произведения, который покрывал бы обе комбинации переменных $X, Y, Z = 111$ и $X, Y, Z = 110$. Таким образом, интуитивно ясно, что выходной сигнал может на короткое время «проваливаться» до 0, если сигнал на выходе вентиля И, покрывающего одну из комбинаций, переходит в 0 до того, как сигнал на выходе вентиля И, покрывающего другую комбинацию переменных, переходит в 1. Способ исключения этого источника опасности очевиден: просто вводим лишний терм-произведение (вентиль И), чтобы покрыть опасную пару комбинаций переменных, как показано на рис. 4.46(б). Оказывается, что лишний терм-произведение является консенсусом (consensus) двух исходных термов; в самом общем случае для исключения источников опасности необходимо добавлять консенсусные термы. Соответствующая схема, в которой нет источников опасности, показана на рис. 4.47.

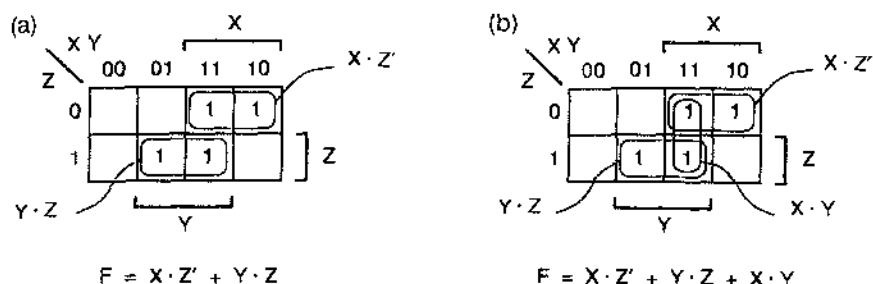


Рис. 4.46. Карта Карио для схемы, изображенной на рис. 4.44: (а) согласно исходной конструкции; (б) вариант, в котором единичный статический источник опасности устранен

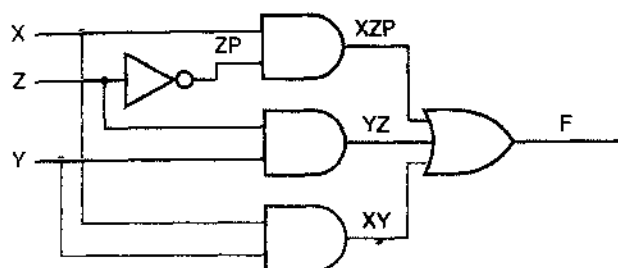


Рис. 4.47. Схема подавлением единичного источника опасности

Другой пример приведен на рис. 4.48. В этом примере для устранения единичных статических источников опасности необходимо добавить три термина-произведения.

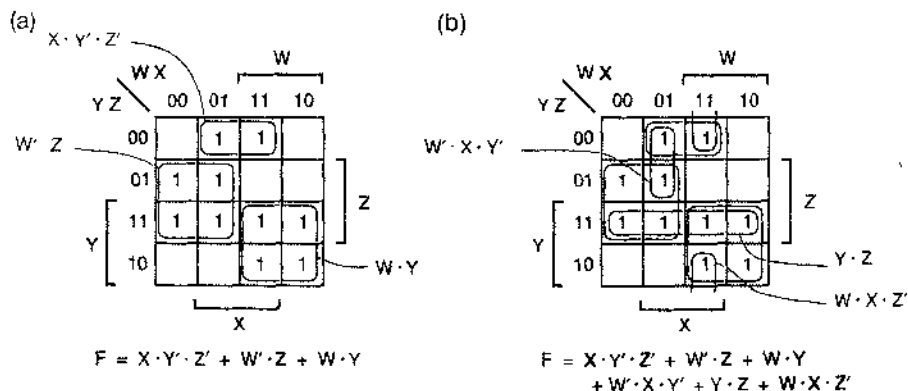


Рис. 4.48. Карта Карио для другой схемы, реализующей сумму произведений: (а) исходный проект; (б) включение дополнительных термов-произведений для подавления единичных статических источников опасности

Правильно составленная двухуровневая схема ИЛИ-И, реализующая произведение сумм, не имеет единичных статических источников опасности. Однако у нее могут быть нулевые статические источники опасности. Эти источники опасности можно обнаружить и устранить, рассматривая соседние нули в карте Карио по принципу, являющемуся двойственным по отношению к тому, что объяснено выше.

*4.5.3. Динамические источники опасности

Динамический источник опасности (dynamic hazard) – это возможность того, что выходной сигнал будет менять свое значение несколько раз в результате однократного изменения комбинации входных сигналов. Многократные изменения выходного сигнала могут происходить в том случае, когда существует несколько путей с различными задержками, по которым изменяющийся сигнал проходит от входа до выхода.

Рассмотрим, например, схему на рис. 4.49; здесь есть три разных пути от входа X до выхода F. Один путь проходит через медленный вентиль ИЛИ, а другой – через еще более медленный вентиль ИЛИ. Если сигналы W, X, Y, Z на входах схемы имеют значения 0, 0, 0, 1, то выходной сигнал равен 1, как показано на рисунке. Теперь предположим, что значение сигнала X изменяется и становится равным 1. Положим, что все другие вентили, за исключением тех двух, которые помечены как «медленный» и «еще более медленный», являются настолько быстродействующими, что переходы, указанные черным цветом, происходят сразу же, и выходной сигнал становится равным 0. Через некоторое время изменяется сигнал на выходе «медленного» вентиля ИЛИ, вызывая переходы, указанные синим прямым шрифтом, так что выходной сигнал принимает значение 1. Наконец, изменяется сигнал на выходе «еще более медленного» вентиля ИЛИ, в результате чего происходят переходы, указанные синим курсивом, а выходной сигнал принимает свое окончательное значение, равное 0.

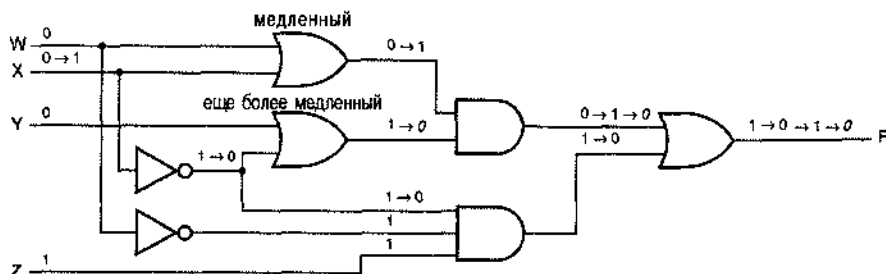


Рис. 4.49. Схема с динамическим источником опасности

В правильно построенных двухуровневых схемах И–ИЛИ и ИЛИ–И динамические источники опасности отсутствуют; это справедливо в том случае, когда никакой входной сигнал и его инверсия не поданы одновременно на входы одного и того же вентиля первого уровня. Для обнаружения динамических источников опасности в многоуровневых схемах используется метод, упоминаемый в Обзоре литературы.

*4.5.4. Проектирование схем без источников опасности

Ситуаций, когда требуются комбинационные схемы, свободные от источников опасности, совсем немного; один из таких случаев – разработка последовательных схем с обратной связью. Методы обнаружения источников опасности в

произвольных схемах, о которых говорится в Обзоре литературы, довольно трудно применять на практике. Поэтому в случае, когда вам необходимо построить схему без источников опасности, лучше всего использовать такую конфигурацию схемы, которую легко проанализировать.

Мы упоминали, в частности, что в правильно спроектированной двухуровневой схеме И-ИЛИ нет нулевых статических и динамических источников опасности. В таких схемах могут существовать единичные статические источники опасности, но их можно найти и исключить с помощью карт, используя описанный выше метод. Если вопрос стоимости схемы не критичен, то свободная от источников опасности реализация получится в результате применения грубого метода, состоящего в использовании полной суммы, то есть суммы всех простых импlicants логической функции (см. задачу 4.34). Аналогично, по принципу двойственности, можно построить свободную от источников опасности двухуровневую схему ИЛИ-И для любой логической функции. Наконец, имейте в виду, что все сказанное о схемах И-ИЛИ естественным образом переносится на случай проектирования соответствующих схем И-НЕ-И-НЕ, а все сказанное о схемах ИЛИ-И применительно к схемам ИЛИ-НЕ-ИЛИ-НЕ.

В БОЛЬШИНСТВЕ СЛУЧАЕВ ИСТОЧНИКИ ОПАСНОСТИ НЕ СТРАШНЫ

Любую комбинационную схему можно проанализировать на наличие в ней источников опасности. Однако структура хорошо спроектированной *синхронной* цифровой системы бывает такова, что для большинства узлов системы нет необходимости проводить анализ на наличие источников опасности. В синхронной системе изменение всех сигналов на входе комбинационной схемы происходит в определенный момент времени, а выходные сигналы «принимаяются во внимание» только спустя время, необходимое для того, чтобы они приняли значения, соответствующие установившемуся режиму. Анализ на наличие источников опасности и их устранение необходимы, как правило, только в случае проектирования асинхронных последовательностных схем, таких как схемы с обратной связью, рассматриваемые в параграфе 7.9. Разработкой подобных схем приходится заниматься не часто, но если уж имеет место такой случай, то понимание природы источников опасности абсолютно необходимо для получения надежного результата.

4.6. Язык описания схем ABEL

Язык описания схем ABEL предназначен для того, чтобы позволить разработчику задавать логические функции, которые должны быть реализованы в ПЛУ. Программа на языке ABEL представляет собой текстовый файл, содержащий несколько элементов:

- Документацию, в том числе имя программы и комментарии.
- Объявления, указывающие входы и выходы логических функций, которые должны быть реализованы.

- Операторы, задающие логические функции, которые должны быть реализованы.
- Как правило, объявление типа ПЛУ или другого устройства, в котором должны быть реализованы указанные логические функции.
- Как правило, «векторы проверок», которыми задаются ожидаемые значения логических функций для некоторых комбинаций переменных.

Язык ABEL поддерживается *процессором языка ABEL (ABEL language processor)*, который мы будем называть также *компилятором языка ABEL (ABEL compiler)*. Назначение компилятора состоит в том, чтобы оттранслировать текстовый файл, написанный на языке ABEL, в «рисунок соединений», который можно было бы загрузить в физическое ПЛУ. Хотя большинство ПЛУ можно программировать только по рисунку, соответствующему выражениям вида «сумма произведений», язык ABEL допускает задание функций, которые предстоит реализовать в ПЛУ, в виде таблиц истинности, либо в виде вложенных операторов “IF”, а также в формате любых алгебраических выражений. Компилятор обрабатывает эти форматы и минимизирует результирующее соотношение так, чтобы подогнать его, если только это возможно, под структуру имеющегося ПЛУ.

О структурах ПЛУ, рисунках соединений и о связывании с этим понятием и представлениях речь пойдет позднее в параграфе 5.3, где будет показано, как в программе на языке ABEL учитываются особенности данного ПЛУ. А пока что мы покажем, как можно воспользоваться языком ABEL для задания комбинационных логических функций без обязательного объявления типа устройства, в котором они должны быть реализованы. Впоследствии, в параграфе 7.11, мы сделаем то же самое для последовательностных логических функций.

4.6.1. Структура программ на языке ABEL

В табл. 4.10 приведена типичная структура программы на языке ABEL, а в табл. 4.11 – реальная программа, демонстрирующая следующие характерные особенности языка:

- *Идентификаторы (identifiers)* должны начинаться с буквы или символа подчеркивания, могут содержать до 31 буквы, цифры и символа подчеркивания и чувствительны к регистру.
- Программный файл начинается с оператора *module*, который связывает идентификатор (*Alarm Circuit*) с программным модулем. В больших программах может быть много модулей; каждый из них имеет свой собственный локальный заголовок, свои объявления и равенства. Заметьте, что ключевые слова, такие как “module”, не чувствительны к регистру.
- Оператором *title* задается строка заголовка, которая будет вставляться компилятором в создаваемые файлы документации.

ЮРИДИЧЕСКАЯ СПРАВКА

ABEL (Advanced Boolean Equation Language) – товарный знак фирмы Data I/O Corporation (Redmond, WA 98073).

```

module module name
title string
deviceID device deviceType;
pin declarations
other declarations
equations
equations
test_vectors
test vectors
end module name

```

Табл. 4.10. Типичная структура программы на языке ABEL

Табл. 4.11. Программа на языке ABEL для схемы охранной сигнализации, приведенной на рис. 4.19

```

module Alarm_Circuit
title 'Alarm Circuit Example
J. Wakerly, Micro Systems Engineering'
ALARMCKT device 'P16V8C';

" Input pins
PANIC, ENABLEA, EXITING      pin 1, 2, 3;
WINDOW, DOOR, GARAGE        pin 4, 5, 6;
" Output pins
ALARM                        pin 11 istype 'com';

" Constant definition
X = .X.;

" Intermediate equation
SECURE = WINDOW & DOOR & GARAGE;

equations
ALARM = PANIC # ENABLEA & !EXITING & !SECURE;

test_vectors
([PANIC,ENABLEA,EXITING,WINDOW,DOOR,GARAGE] -> [ALARM])
[ 1, .X., .X., .X., .X., .X.] -> [ 1];
[ 0, 0, .X., .X., .X., .X.] -> [ 0];
[ 0, 1, 1, .X., .X., .X.] -> [ 0];
[ 0, 1, 0, 0, .X., .X.] -> [ 1];
[ 0, 1, 0, .X., 0, .X.] -> [ 1];
[ 0, 1, 0, .X., .X., 0] -> [ 1];
[ 0, 1, 0, 1, 1, 1] -> [ 0];

end Alarm_Circuit

```

- *Строка (string)* – это последовательность символов, заключенная в одинарные кавычки.

- Необязательное объявление *device* содержит идентификатор устройства (ALARMCKT) и строку, посредством которой указывается тип устройства ('P16V8C' для ИС GAL16V8). Компилятор использует идентификатор в именах генерируемых им файлов документации, а по типу устройства определяет, может ли это устройство реально выполнять логические функции, заданные в программе.
- *Комментарии (comments)* начинаются с двойной кавычки и заканчиваются другой двойной кавычкой или символом конца строки в зависимости от того, что встречается первым.
- *Объявление выводов (pin declaration)* называет компилятору имена, присваиваемые внешним выводам устройства. Если имени сигнала предшествует префикс NOT (!), то сигнал на выводе будет инверсией по отношению к сигналу с данным именем. Объявление выводов может включать в себя номера выводов, а может и не включать; если номера не заданы, то компилятор назначает их в зависимости от возможностей программируемого устройства.
- Ключевое слово *istype* предшествует списку, состоящему из одного или большего числа свойств, разделенных запятыми. Эта запись указывает компилятору тип выходного сигнала. Ключевое слово "com" означает комбинационный выход. Если ключевое слово *istype* отсутствует, то компилятор, как правило, полагает, что сигнал является входным, если только он не появляется в каком-нибудь из равенств слева; в последнем случае компилятор пытается вывести свойства выходного сигнала из контекста. В интересах вашей собственной безопасности лучше всего помещать вслед за ключевым словом *istype* описание всех выходных сигналов!
- *Другие объявления (other declarations)* позволяют разработчику определять константы и выражения, чтобы сделать программу более удобочитаемой и упростить само логическое проектирование.
- Оператор *equations* указывает, что далее следуют логические равенства, которыми определяются выходные сигналы в виде функций от входных сигналов.
- *Равенства (equations)* записываются аналогично операторам присваивания в обычном языке программирования. Каждое равенство заканчивается точкой с запятой. В языке ABEL для логических операций используются следующие символы:

& И (AND),
 # ИЛИ (OR),
 ! НЕ (NOT; применяется в качестве префикса),
 \$ ИСКЛЮЧАЮЩЕЕ ИЛИ (XOR),
 !\$ ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ (XNOR).

Как и в обычных языках программирования, операция И (&) в логических выражениях имеет приоритет по отношению к операции ИЛИ (#). При наличии директивы @ALTERNATE компилятор распознает альтернативный набор символов для указанных логических операций: *, +, /, +: и *: соответственно. В этой книге повсюду используются символы, принятые в языке ABEL по умолчанию.

- Необязательный оператор *test_vectors* указывает на то, что далее следуют векторы проверок.
- *Векторы проверок (test vectors)* ставят в соответствие комбинациям входных сигналов ожидаемые значения выходных сигналов; они используются при моделировании и при тестировании, как объясняется в разделе 4.6.7.
- Компилятор распознает несколько специальных констант, в том числе символ *X* для обозначения одиночного бита с «безразличным» значением.
- Оператором *end* отмечается конец модуля.

Для задания значений сигналов на выходах комбинационных схем используется оператор *неактивируемого присваивания* = (*unlocked assignment operator*). Левая часть равенства обычно содержит имя сигнала. Правая часть равенства является логическим выражением, которое не обязательно должно быть вида «сумма произведений». Имени сигнала в левой части равенства может предшествовать необязательный оператор *!* (НЕ); это эквивалентно инвертированию логического выражения, стоящего справа. Задача компилятора состоит в том, чтобы создать такой рисунок соединений, при котором сигнал, названный в левой части равенства, будет результатом реализации логического выражения, указанного в правой части равенства.

4.6.2. Работа компилятора языка ABEL

Программа, приведенная в табл. 4.11, реализует функцию ALARM, описанную в разделе 4.3.1. Сигналу, называвшемуся ранее ENABLE, в программе соответствует идентификатор ENABLED, поскольку ENABLE является зарезервированным словом языка ABEL.

Обратите внимание, что не все равенства размещены вслед за оператором *equations*. *Промежуточное равенство (intermediate equation)* для идентификатора SECURE появляется раньше. Это равенство просто играет роль определения, которое связывает выражение с идентификатором SECURE. Компилятор языка ABEL подставляет это выражение на место идентификатора SECURE всякий раз, когда SECURE встречается после его определения.

На рис. 4.19 была приведена схема охранной сигнализации, непосредственно реализующая выражения SECURE и ALARM посредством многоуровневой логики. Компилятор языка ABEL так не поступает и не использует выражения для соединений вентилях. Наоборот, он «разгрызает» выражения, стремясь получить минимальный двухуровневый вариант вида «сумма произведений», пригодный для реализации в ПЛУ. Таким образом, в результате компиляции программа, приведенная в табл. 4.11, должна выдать схему, эквивалентную схеме И-ИЛИ, изображенной на рис. 4.20, которая и оказывается минимальной.

Компилятор действительно это делает. В табл. 4.12 приведен файл синтезированных равенств, созданный компилятором языка ABEL. Заметьте, что компилятор составляет равенства только для единственного выходного сигнала ALARM. Сигнал SECURE нигде не появляется.

Компилятор находит минимальные выражения вида «сумма произведений» как для самой функции ALARM, так и для ее дополнения *!ALARM*. Как упоминалось ранее, во многих ПЛУ имеется возможность выбирать инвертированные или не

инвертируемые значения выходных сигналов схем И-ИЛИ. «Равенство обратной полярности» в табл. 4.12 представляет собой реализацию выражения для !ALARM вида «сумма произведений»; если бы было выбрано инверсное значение выходного сигнала, то использовалось бы это равенство.

В данном примере в равенстве обратной полярности на один терм-произведение меньше, чем в равенстве нормальной полярности для сигнала ALARM, так что компилятор выберет это равенство, если программируемое устройство допускает выбор инвертированного или неинвертированного выходного сигнала. Пользователь может также заставить компилятор использовать нормальную или обратную полярность сигнала, включив в список свойств сигнала istype ключевое слово "buffer" или "invert" соответственно. (В некоторых компиляторах языка ABEL для той же цели можно использовать ключевые слова "pos" и "neg", но при этом нужно принять во внимание то, о чем будет сказано в разделе 4.6.6.)

Табл. 4.12. Файл синтезированных равенств, созданный компилятором языка ABEL для программы, приведенной в табл. 4.11

ABEL 6.30

Design alarmckt created Tue Nov 24 1998

Title: Alarm Circuit Example

Title: J. Wakerly, Micro Systems Engineering

P-Terms	Fan-in	Fan-out	Type	Name (attributes)
4/3	6	1	Pin	ALARM

=====

```

4/3          Best P-Term Total: 3
              Total Pins: 7
              Total Nodes: 0
              Average P-Term/Output: 3

```

Equations:

```

ALARM = (ENABLEA & !EXITING & !DOOR
        # ENABLEA & !EXITING & !WINDOW
        # ENABLEA & !EXITING & !GARAGE
        # PANIC);

```

Reverse-Polarity Equations:

```

!ALARM = (!PANIC & WINDOW & DOOR & GARAGE
        # !PANIC & EXITING
        # !PANIC & !ENABLEA);

```

4.6.3. Операторы WHEN и блоки равенств

Кроме равенств, в языке ABEL имеется другой способ задания комбинационных логических функций – оператор *WHEN* (*WHEN statement*), также размещаемый в части программы, содержащей равенства. В табл. 4.13 показана общая структура оператора *WHEN*; она подобна структуре оператора *IF* в обычных языках программирования. Предложение *ELSE* является необязательным. Здесь *LogicExpression* – выражение, принимающее значение «true» (1) или «false» (0). В зависимости от значения *LogicExpression* «исполняется» либо *TrueEquation*, либо *FalseEquation*. Однако в данном случае необходимо уточнить, что именно мы понимаем под словом «исполняется», и об этом говорится ниже.

```

WHEN LogicExpression THEN
    TrueEquation;
ELSE
    FalseEquation;

```

Табл. 4.13. Структура оператора *WHEN* в языке ABEL

В простейшем случае *TrueEquation* и необязательное *FalseEquation* представляют собой операторы присваивания, как это имеет место в первых двух операторах *WHEN* в табл. 4.14 (в отношении *X1* и *X2*). В этом случае выполняется логическая операция И, операндами которой являются *LogicExpression* и правая часть *TrueEquation*, а также логическая операция И, операндами которой являются дополнение к *LogicExpression* и правая часть *FalseEquation*. Таким образом, равенства для *X1A* и *X2A*, в которых нет операторов *WHEN*, дают те же самые результаты, что и соответствующие операторы *WHEN*.

Заметьте, что в первом примере *X1* фигурирует в *TrueEquation*, а *FalseEquation* отсутствует. Спрашивается, что будет с *X1*, если *LogicExpression* (*!A#B*) окажется ложным? Можно подумать, что для таких комбинаций входных сигналов значение *X1* должно быть безразличным, но это не так; сейчас мы это объясним.

Формально оператором неактируемого присваивания (=) задается комбинация входных сигналов, которую следует добавить в множество включений для выходного сигнала, фигурирующего в левой части равенства. Вначале множество включений данного выходного сигнала является пустым; оно пополняется всякий раз, как этот выходной сигнал появляется в левой части равенства. Другими словами, осуществляется логическая операция ИЛИ, операндами которой служат правые части всех равенств для одного и того же неинвертированного выходного сигнала. (Если в левой части равенства указано дополнение данного входного сигнала, то берется дополнение правой части этого равенства, и только после этого результат включается в операцию ИЛИ.) Таким образом, значение *X1* равно 1 только при такой комбинации входных сигналов, когда выражение *LogicExpression* (*!A#B*) истинно и выражение (*C&!D*), стоящее справа в равенстве *TrueEquation*, также истинно.

Во втором примере *X2* фигурирует в левой части двух равенств, так что эквивалентное равенство для *X2A* получается путем выполнения операции ИЛИ над вы-

ражениями, являющимися результатом операции И над правыми частями этих равенств при соответствующих условиях.

Роль *TrueEquation* и необязательного *FalseEquation* в операторе WHEN могут играть любые равенства. Кроме того, операторы WHEN могут быть «вложены» один в другой, путем помещения другого оператора WHEN на место *FalseEquation*. Когда имеются вложенные операторы, все условия, приводящие к «исполняемому» утверждению, объединяются посредством логической операции И. Этот принцип иллюстрируется равенством для ХЗВ в табл. 4.14, которое является не содержащим WHEN эквивалентом оператора для ХЗ.

Другой оператор WHEN может играть роль *TrueEquation*, но только в том случае, если он заключен в фигурные скобки, как показано в таблице в примере для Х4. Это просто один из примеров использования фигурных скобок.

В уже рассмотренных нами примерах во всех частях любого из операторов WHEN происходило присвоение значения одному и тому же выходному сигналу, но это не обязательно всегда должно быть так. Соответствующим примером служит второй от конца оператор WHEN в табл. 4.14.

Часто бывает полезно осуществить несколько присваиваний в *TrueEquation* и/или в *FalseEquation*. С этой целью язык ABEL допускает наличие блоков равенств повсюду, где могут находиться одиночные равенства. Блок равенств (*equation block*) — это заключенная в фигурные скобки последовательность операторов, как показано на примере последнего оператора WHEN в таблице. Отдельные операторы в этой последовательности могут быть простыми операторами присваивания, операторами WHEN или вложенными блоками равенств. После закрывающей фигурной скобки блока точка с запятой не ставится. Шутки ради, в табл. 4.15 приведены равенства, составленные компилятором языка ABEL для всей программы в нашем примере.

4.6.4. Таблицы истинности

Язык ABEL предоставляет еще одну возможность задания комбинационных логических функций — с помощью *таблицы истинности* (*truth table*), формат которой в общем случае имеет вид, указанный в табл. 4.16. Таблица истинности вводится ключевым словом *truth_table*. В *input-list* (*список входных сигналов*) и *output-list* (*список выходных сигналов*) перечисляются имена входных и выходных сигналов, фигурирующих в таблице. Каждый из этих списков состоит либо из одного имени сигнала, либо из набора имен; наборы исчерпывающим образом описаны в разделе 4.6.5. За введением в таблицу истинности следует ряд утверждений, каждое из которых задает значение входного сигнала и требуемое значение выходного сигнала с помощью *неактивируемого оператора таблицы истинности* $\rightarrow$ (*unclocked truth-table operator*). Например, таблица истинности для инвертора имеет вид:

```
truth_table (X -> NOTX)
    0 -> 1;
    1 -> 0;
```

Табл. 4.14. Примеры операторов WHEN

```

module WhenEx
title 'WHEN Statement Examples'

" Input pins
A, B, C, D, E, F                pin;

" Output pins
X1, X1A, X2, X2A, X3, X3A, X4    pin istance 'ccm';
X5, X6, X7, X8, X9, X10         pin istance 'com';

equations

WHEN (!A # B) THEN X1 = C & !D;

X1A = (!A # B) & (C & !D);

WHEN (A & B) THEN X2 = C # D;
ELSE X2 = E # F;

X2A = (A & B) & (C # D)
      # !(A & B) & (E # F);

WHEN (A) THEN X3 = D;
ELSE WHEN (B) THEN X3 = E;
ELSE WHEN (C) THEN X3 = F;

X3A = (A) & (D)
      # !(A) & (B) & (E)
      # !(A) & !(B) & (C) & (F);

WHEN (A) THEN
  {WHEN (B) THEN X4 = D;}
ELSE X4 = E;

WHEN (A & B) THEN X5 = D;
ELSE WHEN (A # !C) THEN X6 = E;
ELSE WHEN (B # C) THEN X7 = F;

WHEN (A) THEN {
  X8 = D & E & F;
  WHEN (B) THEN X8 = 1; ELSE {X9 = D; X10 = E;}
} ELSE {
  X8 = !D # !E;
  WHEN (D) THEN X9 = 1;
  {X10 = C & D;}
}

end WhenEx

```

Табл. 4.15. Файл синтезированных равенств, созданный компилятором языка ABEL для программы, приведенной в табл. 4.14

ABEL 6.30				Equations:	Reverse-Polarity Eqns:
Design whenex created Wed Dec 2 1998				X1 = (C & !D & !A # C & !D & B);	!X1 = (A & !B # D # !C);
Title: WHEN Statement Examples				X1A = (C & !D & !A # C & !D & B);	!X1A = (A & !B # D # !C);
P-Terms	Fan-in	Fan-out	Type Name		
2/3	4	1	Pin X1	X2 = (D & A & B # C & A & B # !B & E # !A & E # !B & F # !A & F);	!X2 = (!C & !D & A & B # !B & !E & !F # !A & !E & !F);
2/3	4	1	Pin X1A		!X2A = (!C & !D & A & B # !B & !E & !F # !A & !E & !F);
6/3	6	1	Pin X2	X2A = (D & A & B # C & A & B # !B & E # !A & E # !B & F # !A & F);	!X3 = (!C & !A & !B # !A & B & !E # !D & A # !A & !B & !F);
6/3	6	1	Pin X2A		!X3A = (!C & !A & !B # !A & B & !R # !D & A # !A & !B & !F);
3/4	6	1	Pin X3	X3 = (C & !A & !B & F # !A & B & E # D & A);	!X4 = (A & !B # !D & A # !A & !R);
3/4	6	1	Pin X3A	X3A = (C & !A & !B & F # !A & B & E # D & A);	!X5 = (!A # !D # !B);
2/3	4	1	Pin X4	X4 = (D & A & B # !A & E);	!X6 = (A & B # C & !A # !E);
1/3	3	1	Pin X5	X5 = (D & A & B);	!X7 = (A # !C # !F);
2/3	4	1	Pin X6	X6 = (A & !B & E # !C & !A & E);	!X8 = (A & !B & !F # D & !A & E # A & !B & !E # !D & A & !B);
1/3	3	1	Pin X7	X7 = (C & !A & F);	!X9 = (!D # A & B);
4/4	5	1	Pin X8	X8 = (D & A & E & F # A & B # !A & !E # !D & !A);	!X10 = (A & B # !D & !A # !C & !A # A & !E);
2/2	3	1	Pin X9	X9 = (D & !A # D & !B);	
2/4	5	1	Pin X10	X10 = (C & D & !A # A & !B & E);	
=====					
36/42	Best P-Term Total: 30				
	Total Pins: 19				
	Total Nodes: 0				
	Average P-Term/Output: 2				

Табл. 4.16. Структура таблицы истинности в языке ABEL

Список значений входных

```

truth_table (input-list -> output-list)
input-value -> output-value;
...
input-value -> output-value;

```

сигналов не обязательно должен быть полным; следует указывать лишь множество включений, если только не предусматривается наличие безразличных комбинаций (см. раздел 4.6.6). В табл. 4.17 показано, как можно на языке ABEL задать

функцию устройства для обнаружения простых чисел. Ради удобства, идентификатор NUM определен как синоним набора из четырех входных битов [N3, N2, N1, N0], благодаря чему значения 4-разрядного двоичного входного сигнала можно записывать в виде десятичных целых чисел.

Табл. 4.17. Программа на языке ABEL для устройства, обнаруживающего простые числа

```

module PrimeDet
title '4-Bit Prime Number Detector'

" Input and output pins
N0, N1, N2, N3                pin;
F                              pin istype 'com';

" Definition
NUM = [N3, N2, N1, N0];

truth_table (NUM -> F)
    1 -> 1;
    2 -> 1;
    3 -> 1;
    5 -> 1;
    7 -> 1;
    11 -> 1;
    13 -> 1;

end PrimeDet

```

В пределах одной и той же программы на языке ABEL можно использовать как таблицы истинности, так и равенства. Последовательность равенств вводится ключевым словом *equations*, а одиночная таблица истинности – ключевым словом *truth_table*.

4.6.5. Диапазоны, наборы и отношения

Большинство цифровых систем содержат шины, а также регистры и другие схемы, в которых происходит обработка двух или большего числа сигналов, идентичных по своему характеру. В языке ABEL предусмотрено несколько экономичных и удобных способов определения таких сигналов и обращения с ними.

Один из таких приемов применяется для задания имен сигналов, подобных друг другу и указываемых по порядку номера. Как показано в табл. 4.18 *диапазон (range)* имен сигналов можно определить, указав первое имя и последнее имя в этом диапазоне, разделенные двумя точками “. .”. Например, запись “N3. . N0” тождественна записи “N3, N2, N1, N0”. Обратите внимание, что, как видно из таблицы, возможно задание диапазона с возрастанием и с убыванием номеров.

Далее, в случае, когда все сигналы из некоторой группы обрабатываются совершенно одинаково, нам нужны средства более компактной записи равенств, чтобы уменьшить вероятность появления ошибок и несовместимостей. *Набор (set)* в

языке ABEL – это заданная совокупность сигналов, обрабатываемых как один сигнал. Когда присваивания и логические операции типа И и ИЛИ употребляются применительно к набору, они выполняются для каждого элемента набора.

Табл. 4.13. Примеры диапазонов, наборов и отношений

```

module SetOps
title 'Set Operation Examples'

" Input and output pins
N3..N0, M3..M0, SEL                pin;
Y1..Y4, Z0..Z3, EQ, GE, GTR, LTH, UNLUCKY  pin istype 'com';

" Definitions
N    = [N3,N2,N1,N0];
M    = [M3,M2,M1,M0];
YOUT = [Y1..Y4];
ZOUT = [Z3..Z0];

COMP = [EQ,GE];
GT    = [ 0, 1];
LT    = [ 0, 0];

equations

YOUT = N & M;
ZOUT = (SEL & N) # (!SEL & M);
EQ = (N == M);
GE = (N >= M);
GTR = (COMP == GT);
LTH = (COMP == LT);
UNLUCKY = (N == 13) # (M == ~hD) # ((N + M) == ~b1101);

end SetOps

```

Каждый набор определяется в начале программы путем связывания имени с заключенным в квадратные скобки списком элементов набора (например, $N = [N3, N2, N1, N0]$ в табл. 4.18). Для списка элементов набора может быть использовано сокращенное обозначение ($YOUT = [Y1..Y4]$), но имена элементов не обязательно должны быть похожими или как-то связанными с именем набора ($COMP = [EQ, GE]$). Элементами набора могут быть также константы ($GT = [0, 1]$). В любом случае, как мы увидим в дальнейшем, существенны число и порядок элементов в наборе.

К наборам применимо большинство операторов языка ABEL. Когда та или иная операция осуществляется с двумя или большим числом наборов, все наборы должны иметь одно и то же число элементов. Операция выполняется по отдельности с элементами наборов, располагающимися на одинаковых позициях, независимо от их имен или номеров. Таким образом, равенство " $YOUT = N \& M$ " эквивалентно следующим четырем равенствам:


```

Y1 = N3 & M3;
Y2 = N2 & M2;
Y3 = N1 & M1;
Y4 = N0 & M0;

```

Если операция включает набор n переменные, не являющиеся элементами наборов, то эти последние участвуют в операции с элементами набора в каждой позиции по отдельности. Таким образом, равенство $ZOUT = (SEL \& N) \# (!SEL \& M)$ эквивалентно четырём равенствам вида $Z_i = (SEL \& N_i) \# (!SEL \& M_i)$ для значений i от 0 до 3.

Другой важной особенностью языка ABEL является возможность преобразования «отношений» в логические выражения. *Отношение (relation)* – это пара операндов, соединённых одним из *операторов отношения (relational operators)*, перечисленных в табл. 4.19. Компилятор преобразует отношение в логическое выражение, принимающее значение 1 тогда и только тогда, когда отношение истинно.

Табл. 4.19. Операторы отношения в языке ABEL

Символ	Отношение
$=$	равно
$\neq$	не равно
$<$	меньше
$\leq$	меньше или равно
$>$	больше
$\geq$	больше или равно

Операнды в отношении считаются целыми числами без знака, но любой из них может быть целым числом или набором. Если операндом является набор, то он воспринимается как целое двоичное число без знака, причем крайняя левая переменная в наборе представляет собой старший разряд этого числа. По умолчанию, числа в программах, написанных на языке ABEL, полагаются десятичными. Шестнадцатеричные и двоичные числа обозначаются приставками “^h” и “^b” соответственно, как это продемонстрировано в последнем равенстве в табл. 4.18.

Наборы и отношения позволяют представить массу функциональных зависимостей на языке ABEL в виде совсем небольшого числа строк в программе. Например, равенства в табл. 4.18 порождают минимизированные равенства с 69 термами-произведениями, как это следует из результатов, приведённых в краткой форме в табл. 4.20.

*4.6.6. Безразличные комбинации входных сигналов

Некоторые версии компилятора языка ABEL обладают ограниченной способностью оперировать с безразличными комбинациями входных сигналов. Как упоминалось выше, равенствами языка ABEL определяются комбинации входных сигналов, принадлежащие множеству включений логической функции; про остающиеся комбинации предполагается, что они принадлежат множеству выключе-

Табл. 4.20. Информация о синтезированных равенствах, созданных компилятором языка ABEL для программы, приведенной в табл. 4.18

P-Terms	Fan-in	Fan-out	Type	Name (attributes)
1/2	2	1	Pin	Y1
1/2	2	1	Pin	Y2
1/2	2	1	Pin	Y3
1/2	2	1	Pin	Y4
2/2	3	1	Pin	Z0
2/2	3	1	Pin	Z1
2/2	3	1	Pin	Z2
2/2	3	1	Pin	Z3
16/8	8	1	Pin	EQ
23/15	8	1	Pin	GE
1/2	2	1	Pin	GTR
1/2	2	1	Pin	LTH
16/19	8	1	Pin	UNLUCKY

69/62	Best P-Term Total: 53			
	Total Pins: 22			
	Total Nodes: 0			
	Average P-Term/Output: 4			

ний. Если вместо этого некоторые комбинации входных сигналов можно отнести к ϕ -множеству, то программа может оказаться способной учесть эти безразличные комбинации входных сигналов и достичь лучших результатов минимизации.

В языке ABEL определены два механизма отнесения комбинаций входных сигналов к ϕ -множеству. Чтобы воспользоваться любым из них, вы должны включить в вашу программу директиву компилятору @DCSET или поместить «dc» в следующий за ключевым словом *istype* список свойств выходных сигналов, относительно которых вы хотите, чтобы их значения не принимались во внимание при некоторых комбинациях входных сигналов.

Один из этих механизмов реализуется оператором *неактивируемого присваивания не принимаемых во внимание значений* *?* = (*don't-care unlocked assignment operator*). Данный оператор используется в равенствах вместо оператора *=*, чтобы указать, что комбинацию входных сигналов, при которой выражение в правой части истинно, следует отнести к ϕ -множеству, а не к множеству включений. Хотя этот оператор задокументирован в компиляторе языка ABEL, которым я пользуюсь, к сожалению, похоже на то, что он не работает, и поэтому я не стану более распространяться на эту тему.

Второй механизм – это таблица истинности. Если безразличные комбинации входных сигналов разрешены, то к ϕ -множеству относится любая комбинация, которая явным образом не включена в таблицу истинности. Таким образом, устройство для обнаружения простых двоично-десятичных чисел можно задать программой на языке ABEL, представленной в табл. 4.21. Подразумевается, что комбинации входных сигналов 10–15 являются безразличными, так как их нет в таблице истинности и в программе присутствует директива @DCSET.

```

module DontCare
title 'Dont Care Examples'
@DCSET

" Input and output pins
N3..NO, A, B          pin;
F, Y                  pin 1stype 'com';

NUM = [N3..NO];
X = .X.;

truth_table (NUM->F)
    0->0;
    1->1;
    2->1;
    3->1;
    4->0;
    5->1;
    6->0;
    7->1;
    8->0;
    9->0;

truth_table ([A,B]->Y)
    [0,0]->0;
    [0,1]->X;
    [1,0]->X;
    [1,1]->1;

end DontCare

```

Табл. 4.21. Программа на языке ABEL, в которой разрешены безразличные комбинации входных сигналов

Можно также явно задавать безразличные комбинации, как это показано во второй таблице истинности. Согласно указанию в самом начале этого параграфа компилятор языка ABEL распознает .X. как специальную однобитовую константу со значением «все равно». В табл. 4.21 идентификатор X приравнен этой константе только для того, чтобы упростить запись безразличных комбинаций в таблице истинности. Минимизированные равенства, полученные для программы в табл. 4.21, приведены в табл. 4.22. Обратите внимание, что два равенства для F не совпадают; компилятор выбрал различные значения выходного сигнала для безразличных комбинаций входных сигналов.

```

Equations:
F = (!N2 & N1
    # !N3 & NO);
Y = (B);

Reverse-Polarity Equations:
!F = (N2 & !NO
    # N3
    # !N1 & !NO);
!Y = (!B);

```

Табл. 4.22. Минимизированные равенства для программы в табл. 4.21

4.6.7. Проверочные векторы

Программа на языке ABEL может содержать необязательные проверочные векторы, как это было продемонстрировано в табл. 4.11. В общем случае формат проверочных векторов очень похож на таблицу истинности, как это видно из табл. 4.23. Ключевое слово *test_vectors* вводит таблицу истинности. В *input-list* и *output-list* перечисляются имена входных и выходных сигналов. Каждый из этих списков содержит либо имя единичного сигнала, либо набор. За этим введением в блок проверочных векторов следует последовательность равенств, каждым из которых с помощью оператора $\rightarrow$ задается значение входного сигнала и ожидаемое значение выходного сигнала.

Табл. 4.23. Структура проверочных векторов в языке ABEL

```
test_vectors (input-list  $\rightarrow$  output-list)
            input-value  $\rightarrow$  output-value;
            ...
            input-value  $\rightarrow$  output-value;
```

Проверочные векторы используются в языке ABEL, главным образом, в двух ситуациях и со следующими целями:

1. После того как компилятор языка ABEL транслирует программу в «конфигурацию соединений» для заданной ИС, он осуществляет моделирование работы результирующего запрограммированного устройства, подавая входные сигналы проверочных векторов на входы программной модели устройства и сравнивая сигналы на ее выходах с соответствующими значениями выходных сигналов в проверочных векторах. Разработчик может задать ряд проверочных векторов, чтобы убедиться в том, что устройство функционирует ожидаемым образом при некоторых или при всех возможных комбинациях входных сигналов.
2. После того как ПЛУ запрограммировано физически, программирующее устройство подает входные сигналы из проверочных векторов на входы микросхемы и сравнивает сигналы на ее выходах с соответствующими выходными сигналами проверочных векторов. Это делается для того, чтобы проверить правильность программирования и функционирования микросхемы.

К сожалению, как мы сейчас объясним, проверочные векторы языка ABEL редко оказываются очень уж эффективными при выполнении любой из этих задач.

Проверочные векторы из табл. 4.11 повторены в табл. 4.24, за исключением того, что, ради удобства представления, в качестве постоянной «все равно» .X. использован идентификатор X и в виде комментария добавлены номера проверочных векторов.

Табл. 4.24, действительно, выглядит, как вполне хороший набор проверочных векторов. С точки зрения разработчика эти векторы полностью описывают ожидаемое функционирование схемы охранной сигнализации, и это описание позволяет вектор за вектором проследить работу схемы:

Табл. 4.24. Проверочные векторы программы для схемы охранной сигнализации из табл. 4.11

test_vectors									
([PANIC, ENABLEA, EXITING, WINDOW, DOOR, GARAGE] -> [ALARM])									
[	1,	X,	X,	X,	X,	X]	-> [	1];	"1
[	0,	0,	X,	X,	X,	X]	-> [	0];	"2
[	0,	1,	1,	X,	X,	X]	-> [	0];	"3
[	0,	1,	0,	0,	X,	X]	-> [	1];	"4
[	0,	1,	0,	X,	0,	X]	-> [	1];	"5
[	0,	1,	0,	X,	X,	0]	-> [	1];	"6
[	0,	1,	0,	1,	1,	1]	-> [	0];	"7

1. Если PANIC имеет единичное значение, то выходной сигнал тревоги F должен быть включен независимо от значений других входных сигналов. Все остальные векторы относятся к случаям, когда PANIC равно 0.
2. Если сигнал тревоги не разрешен, то выходной сигнал должен быть выключен.
3. Если сигнал тревоги разрешен и мы выходим, то сигнал тревоги должен быть выключен.
- 4-6. Если сигнал тревоги разрешен и мы не выходим, то сигнал тревоги должен быть включен, если любой из сигналов с выходов датчиков WINDOW, DOOR и GARAGE равен 0.
7. Если сигнал тревоги разрешен, мы не выходим, и сигналы всех датчиков равны 1, то выходной сигнал должен быть выключен.

Проблема состоит в том, что язык ABEL не воспринимает значения «все равно» у входных сигналов проверочных векторов так, как следовало бы. Например, для выполнения требования, содержащегося в проверочном векторе 1, нужно по всем правилам проверить 32 различных комбинации входных сигналов, соответствующие всем возможным наборам значений ENABLEA, EXITING, WINDOW, DOOR и GARAGE, которые объявлены безразличными. Но компилятор языка ABEL этого не делает. В этой ситуации он интерпретирует значение «все равно» как: «пользователю все равно, какое значение входного сигнала я использую» и присваивает значение 0 всем входным сигналам в проверочном векторе, значения которых объявлены безразличными. В нашем примере можно было бы следующим образом неправильно записать равенство для выходного сигнала: "F = PANIC & !ENABLEA # ENABLEA & ..."; проверки согласно векторам по-прежнему проходили бы, хотя кнопка PANIC срабатывала бы только в том случае, когда сигнал тревоги не разрешен.

Второе применение проверочных векторов заключается в тестировании физического устройства. Большинство физических дефектов в логических устройствах можно обнаружить, используя модель *одиночной неисправности типа залипания* (single stuck-at fault model), согласно которой предполагается, что любой физический дефект эквивалентен залипанию входа или выхода вентилей на логическом значении 0 или 1. Набор проверочных векторов, вроде бы, содер-

жнт все функциональные требования, предъявляемые к схеме, как это сделано в табл. 4.24. Однако даже принимая во внимание весь этот набор нельзя гарантировать, что все одиночные неисправности типа залипания можно будет обнаружить. Проверочные векторы должны быть выбраны так, чтобы любая возможная неисправность типа залипания приводила к возникновению на выходе схемы неправильного значения при какой-нибудь комбинации входных сигналов в проверочном векторе.

В табл. 4.25 приведен правильный набор проверочных векторов для устройства охранной сигнализации в случае, когда оно представляет собой двухуровневую схему, реализующую выражение вида «сумма произведений». Первые четыре вектора обнаруживают неисправности типа залипания на единичном значении в вентиле ИЛИ, а последние три вектора обнаруживают неисправности типа залипания на нулевом значении в вентиле И; этого оказывается достаточно для обнаружения всех одиночных неисправностей типа залипания. Если вам что-то известно о тестировании неисправностей, то для небольшой схемы вы можете вручную составить проверочные векторы (как это было сделано мною в данном примере), но большинство конструкторов при разработке своих проектов на основе ПЛУ используют независимые автоматизированные средства составления эффективных проверочных векторов.

Табл. 4.25. Проверочные векторы, обеспечивающие обнаружение единичной неисправности типа залипания, для схемы охранной сигнализации, реализующей минимальное выражение вида «сумма произведений»

test_vectors									
([PANIC, ENABLEA, EXITING, WINDOW, DOOR, GARAGE] -> [ALARM])									
[	1,	0,	1,	1,	1,	1]	->	[	1]; "1
[	0,	1,	0,	0,	1,	1]	->	[	1]; "2
[	0,	1,	0,	1,	0,	1]	->	[	1]; "3
[	0,	1,	0,	1,	1,	0]	->	[	1]; "4
[	0,	0,	0,	0,	0,	0]	->	[	0]; "5
[	0,	1,	1,	0,	0,	0]	->	[	0]; "6
[	0,	1,	0,	1,	1,	1]	->	[	0]; "7

4.7. Язык описания схем VHDL

В середине 80-х годов Министерство обороны США (U.S. Department of Defense, DoD) и Институт инженеров по электротехнике и электронике (Institute of Electrical and Electronic Engineers, IEEE) поддержали разработку довольно мощного языка описания схем VHDL. С самого начала и по настоящее время отличительными особенностями этого языка является следующее:

- Проектируемые устройства можно иерархически разбивать на составные элементы.
- Каждый элемент устройства имеет ясно очерченный интерфейс (для соединения его с другими элементами) и точное функциональное описание (для его моделирования).

- Функциональное описание может быть основано на алгоритме, либо на реальной конструкции, которыми определяется работа элемента. Например, первоначально можно описать работу элемента посредством алгоритма, и это сделает возможной верификацию элементов более высокого уровня, в которых используется данный элемент; позднее алгоритмическое определение можно заменить структурной схемой.
- Все можно моделировать: параллелизм, временные соотношения и синхронизацию тактовыми сигналами. На языке VHDL можно описать как асинхронные, так и синхронные последовательные структуры.
- Можно моделировать выполняемые устройством в целом логические действия и его временные характеристики.

Таким образом, с самого начала VHDL является языком документации и моделирования, позволяющим точно задавать и имитировать поведение цифровых систем.

Хотя язык VHDL и его среда моделирования сами по себе были важными нововведениями, квантовый скачок полезности и популярности языка VHDL произошел с появлением коммерческих программных средств синтеза на основе VHDL (*VHDL synthesis tools*). Применяя эти средства, можно строить логические схемы непосредственно из описания их работы на языке VHDL. С помощью VHDL разрабатывается, моделируется и синтезируется все, что угодно, от простой комбинационной схемы до законченной микропроцессорной системы в одном кристалле.

В 1987 году Институтом инженеров по электротехнике и электронике был принят стандарт языка VHDL (*VHDL-87*), а в 1993 году этот стандарт был расширен (*VHDL-93*). В этом параграфе речь пойдет о таких правилах, которые действуют в обеих версиях языка. Другие особенности языка VHDL будут рассмотрены в параграфе 7.12 применительно к проектированию последовательностных логических схем.

ЧТО ТАКОЕ VHDL?

VHDL означает “VHSIC Hardware Description Language” («язык описания схем на основе VHSIC»). В свою очередь, VHSIC (Very High Speed Integrated Circuit, интегральная схема с очень высоким быстродействием) было названием программы поддержки Министерством обороны США исследований в области высокоэффективной интегральной электроники.

4.7.1. Ход выполнения проекта

Прежде чем обратиться к самому языку, полезно составить представление об окружающей среде, в которой развивается VHDL-проект. Процесс проектирования на основе языка VHDL, или *ход выполнения проекта (design flow)*, состоит из ряда этапов. Через эти этапы бывает необходимо пройти при разработке устройств на основе любого языка описания схем; в общих чертах они представлены на рис. 4.50.

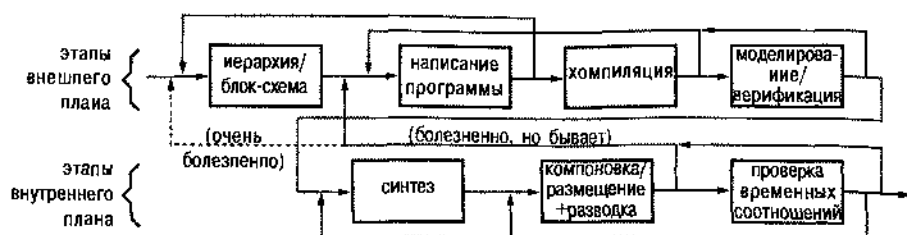


Рис. 4.50. Этапы в ходе выполнения проекта на основе языка VHDL или любого другого языка описания схем

Действия так называемого «внешнего плана» (“front-end”) начинаются с осознания основного подхода и функций отдельных блоков на уровне блок-схемы. Большие логические проекты типа программного обеспечения обычно являются иерархическими, и язык VHDL служит хорошей основой как для определения модулей и их интерфейсов, так и для детализации в дальнейшем.

ЯЗЫКИ VERILOG И VHDL

Примерно в то же время, когда разрабатывался язык VHDL, на сцене появился другой язык описания схем. Язык *Verilog HDL*, или просто *Verilog*, был предложен в 1984 году фирмой Gateway Design Automation в качестве собственного языка описания схем и средства моделирования. Когда в 1988 году появились программные средства синтеза на основе языка Verilog, выпущенные только-только оперившейся фирмой Synopsys, а фирма Gateway была в 1989 году приобретена фирмой Cadence Design Systems, такое сочетание стало решающим фактором, приведшим к повсеместному распространению этого языка.

Сегодня оба языка — VHDL и Verilog — широко применяются и делят рынок логического синтеза примерно поровну. Синтаксис языка Verilog берет свое начало в языке C и, в некотором отношении, ему легче научиться и им легче пользоваться, тогда как язык VHDL больше похож на язык Ada (язык программирования, поддерживаемый Министерством обороны США) и в большей степени пригоден для больших проектов.

Рассматривая вопрос о том, с изучения какого из этих языков следует начинать, и сравнивая с этой точки зрения их относительные достоинства и недостатки, лучше всего, по-видимому, положиться на заключение Дэвида Пеллерина (David Pellerin) и Дугласа Тейлора (Douglas Taylor), сделанное ими в их книге *C VHDL теперь все просто! (VHDL Made Easy*. Prentice Hall, 1997):

Оба языка легко выучить и обоими языками трудно овладеть.

После того как вы изучили один из этих языков, у вас не будет затруднений при переходе к другому.

ЧТО ЗНАЧИТ VERILOG?

Слово “Verilog” не является акронимом, но мне кажется, что оно вполне могло бы быть сокращением от “VERify LOGic” («проверяй логику»).

Следующий шаг состоит в фактическом описании на языке VHDL модулей, их интерфейсов и деталей их внутреннего устройства. В этой части проекта вы можете, в принципе, воспользоваться любым текстовым редактором, поскольку на языке VHDL пишется текст. Однако в большинстве случаев среда, в которой осуществляется проектирование, включает специализированный *текстовый редактор VHDL (VHDL text editor)*, который облегчает работу. Такие редакторы обычно содержат автоматическое выделение ключевых слов языка VHDL, автоматический отступ от начала строки, встроенные шаблоны часто используемых программных структур, встроенную проверку синтаксиса и упрощенный доступ к компилятору.

Написав некоторую программу, вы, безусловно, захотите ее оттранслировать. *Компилятор языка VHDL (VHDL compiler)* анализирует ваш текст на отсутствие в нем синтаксических ошибок и проверит совместимость вашей программы с другими модулями, на которые имеются ссылки. Компилятор подготавливает также внутреннюю информацию, которая понадобится моделирующей программе на следующем этапе вашего проекта. При программировании часто бывает так, что вам хочется приступить к компиляции еще до того, как программа будет написана до конца. Компилирование по частям, поможет предотвратить размножение синтаксических ошибок, появление несовместимых имен и так далее, и, наверняка, позволит вам испытать столь необходимое чувство движения вперед на стадии, когда конца проекта еще не видно!

Самым впечатляющим этапом, по-видимому, является моделирование. *Моделирующая программа VHDL (VHDL simulator)* позволяет задавать входные сигналы и подавать их на входы разрабатываемой конструкции, а также наблюдать выходные сигналы, не собирая схему физически. При выполнении небольших проектов типа домашнего задания по курсу цифровой электроники входные сигналы можно задать вручную и визуально наблюдать выходные сигналы. Но в случае больших проектов язык VHDL дает возможность осуществлять тестирование, создавая *программные средства тестирования* («*испытательные стенды*», “*test benches*”), в которых входные сигналы подаются автоматически, а выходные сигналы сравниваются с ожидаемыми.

На самом деле, моделирование является одной из ступеней более крупного этапа *верификации (verification)*. Наблюдение того, как на выходах моделируемой схемы возникают сигналы, действительно доставляет большое удовлетворение, но цель моделирования шире: она состоит в том, чтобы проверить схему и убедиться, что она работает так, как хочется. В типичном большом проекте существенные усилия затрачиваются как на стадии написания программы, так и после этого, когда бывает необходимо задать достаточно широкий диапазон ус-

ловий тестирования схемы для проверки правильности реализуемых ею логических действий. Обнаружение ошибок в проекте на этой стадии очень ценно; если ошибки обнаружатся позднее, то чаще всего все так называемые этапы «внутреннего плана» (“back-end”) придется повторить.

Заметьте, что существует, по крайней мере, два аспекта верификации. При функциональной верификации (*functional verification*) логика работы схемы изучается независимо от временных соображений; задержки в вентилях и другие временные параметры считаются равными нулю. При проверке временных соотношений (*timing verification*) работа схемы исследуется с учетом предполагаемых задержек; мы проверяем при этом, в частности, удовлетворяются ли требования по времени установления сигналов и их удержания в случае последовательностных устройств типа триггеров. Принято осуществлять функциональную верификацию до перехода к выполнению этапов внутреннего плана. Что касается проведения проверки временных соотношений, то на этой стадии наши возможности ограничены, так как требуемые временные соотношения в очень сильной степени зависят от результатов синтеза и подгонки. Можно выполнить предварительную проверку временных соотношений, чтобы приобрести большую уверенность в правильности самого подхода к проекту в целом, но проведение детальной проверки временных соотношений необходимо отложить до самого конца.

После верификации мы готовы перейти к стадии «внутреннего плана». Характер действий на этом этапе и используемые средства довольно сильно зависят от технологии, по которой выполнен кристалл, выбранный для данного проекта, но существуют три основных этапа. Первый из них — это синтез (*synthesis*), то есть преобразование описания на языке VHDL в набор примитивов или компонентов, которые можно будет образовать в выбранном кристалле. Например, в случае ПЛУ или ИС типа CPLD программа синтеза может выдать равенства, ориентированные на реализацию двухуровневыми схемами выражений вида «сумма произведений». В случае специализированных ИС результатом действия программы синтеза могут быть список вентилей и список соединений (*netlist*), которым определяются необходимые соединения вентилей между собой. Разработчик может «помочь» программе синтеза, задав ограничения (*constraints*), характерные для выбранной технологии, такие как максимальное число логических уровней или нагрузочная способность логических буферов.

На этапе компоновки (*fitting*) программа компоновки (*fitter*) отображает синтезированные примитивы и компоненты на имеющиеся в микросхеме ресурсы. В случае ПЛУ и ИС типа CPLD это может означать приписывание равенств тем или иным элементам И-ИЛИ. В случае специализированных ИС на этом этапе происходит раскладка отдельных вентилей в нужной конфигурации и нахождение путей для их соединения с учетом физических ограничений в кристалле данной ИС; эту процедуру называют размещением и разводкой (*place and route*). На этой стадии разработчик, как правило, имеет возможность ввести дополнительные ограничения, такие как размещение модулей в кристалле или назначение выводов для внешних входов и выходов.

«Последний» этап заключается в проверке временных соотношений в схеме с учетом ее размещения в кристалле. Только на этой стадии можно с разумной

точностью и найти реальные задержки в схеме, обусловленные длиной соединений, величиной нагрузки и другими факторами. Обычно на этом этапе используются те же самые условия тестирования, что и при функциональной верификации, только на данном шаге в эти условия помещается схема, которая в действительности будет построена.

Как и в любом другом творческом процессе может случиться так, что после продвижения на два шага вперед, вам приходится делать шаг назад (если не хуже!). Как отмечено на рисунке, при написании программы могут встретиться проблемы, которые заставят вернуться назад и пересмотреть иерархию, и почти наверняка в процессе компиляции и моделирования обнаружатся ошибки, из-за которых вам придется переписать часть программы.

Самыми болезненными в ходе выполнения проекта бывают те проблемы, с которыми вы сталкиваетесь при выполнении этапов внутреннего плана. Если, например, результат синтеза не влезает в имеющийся кристалл FPGA или не удовлетворяет временным требованиям, может случиться так, что вы должны будете вернуться назад и пересмотреть сам подход к проекту в целом. Стоит напомнить, что прекрасные программные средства не заменяют все же тщательного продумывания проекта в самом начале.

4.7.2. Структура программы

При создании языка VHDL имелось в виду воплотить в нем принципы структурного программирования с заимствованием идей у языков программирования Паскаль и Ада. Ключевая идея состояла в том, чтобы задать интерфейс схемного модуля, а его внутреннее устройство скрыть. Таким образом, *объект (entity)* в языке VHDL — это просто объявление входов и выходов модуля, а *архитектура (architecture)* — подробное описание внутренней структуры модуля или его поведения.

РАБОТАЕТ!?

Как разработчик и системщик с многолетним опытом, я всегда думал, что знаю, что имеется в виду, когда кто-то говорит о своей схеме: «Она работает!». На мой взгляд, это означает, что вы можете пойти в лабораторию, подать питание на макет (и при этом не пойдет дым), нажать кнопку «Пуск» и с помощью осциллографа или логического анализатора наблюдать, как ваш макет последовательно выполняет необходимые действия.

Но с годами значение слов «она работает» изменилось, по крайней мере, для некоторых людей. Когда несколько лет назад я перешел на новую работу, я порадовался, услышав, что несколько основных специализированных ИС для нового важного продукта «работают». Но позднее (спустя совсем немного времени) я понял, что эти ИС работают только при моделировании и что коллективу, занятому в этом проекте, предстоит еще не раз возвращаться назад и понадобится несколько месяцев напряженного труда для синтеза, подгонки и проверки временных соотношений, прежде чем они смогут заказать опытные образцы. Да, верно: «Она работает!». Только это напоминает мне, как мои дети говорят о домашнем задании: «Готово!».

Рис. 4.51(а) иллюстрирует этот принцип. Многие разработчики склонны считать объявление объекта в языке VHDL «оболочкой» архитектуры, скрывающей детали того, что находится внутри, но обеспечивающей «защепки» для других модулей, использующих данный модуль. Эта идея служит основой иерархического подхода к проектированию систем: архитектура верхнего уровня может использовать (или «обрабатывать») другие объекты, оставляя архитектурные детали объектов нижнего уровня скрытыми от объектов более высокого уровня. Как показано на рис. 4.51(б), архитектура более высокого уровня может использовать объекты более низкого уровня многократно, и несколько архитектур верхнего уровня могут использовать один и тот же объект более низкого уровня. На рисунке архитектуры В, Е и F являются самостоятельными; они не используют никакие другие объекты.

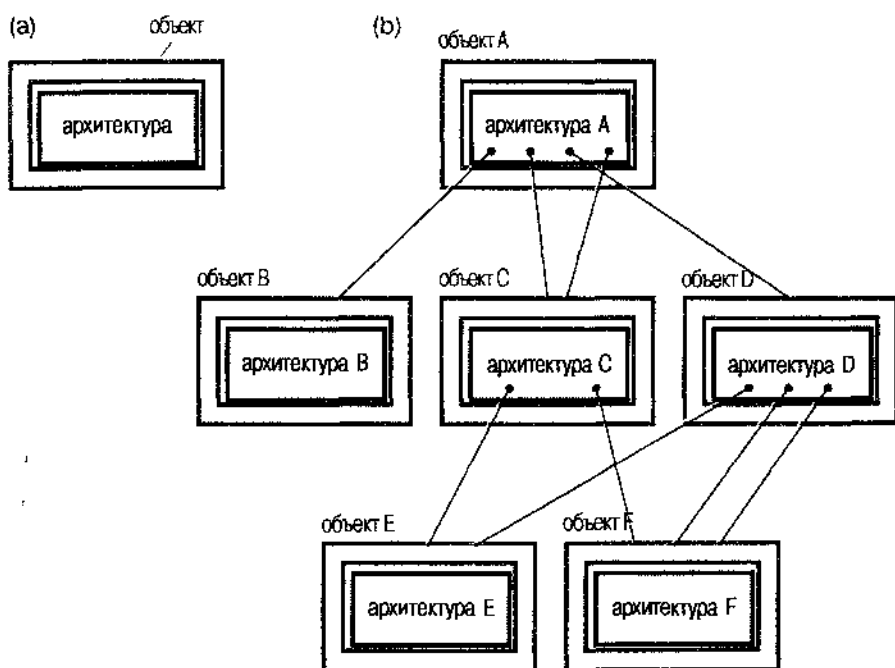


Рис. 4.51. Объекты и архитектуры языка VHDL: (а) идея «оболочки»; (б) иерархическое использование

ПИШИТЕ ВСЕ, ЧТО ХОТИТЕ!

В действительности, язык VHDL позволяет определять несколько архитектур, объединенных в одном объекте, и предоставляет средства управления конфигурацией, посредством которых можно указывать, какая именно архитектура будет использоваться при очередном запуске компилятора или программы синтеза. Это позволяет опробовать различные архитектурные подходы, не выкидывая и не пряча другие варианты. Однако в нашем учебнике мы не воспользуемся этой возможностью и не станем ее обсуждать подробнее.

В текстовом файле на языке VHDL *объявление объекта (entity declaration)* и *определение архитектуры (architecture definition)* разделены, как это сделано на рис. 4.52. В табл. 4.26 в качестве примера приведена очень простая программа на языке VHDL для 2-входового вентиля «запрета». В больших проектах объекты и архитектуры иногда бывают помещены в отдельные файлы, связь между которыми компилятор обнаруживает по их объявленным именам.

текстовый файл (например, mydesign.vhd)

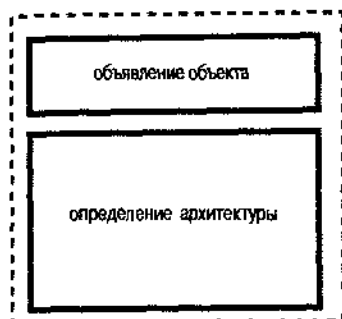


Рис. 4.52. Общий вид файла программы на языке VHDL

Табл. 4.26. Программа на языке VHDL для вентиля «запрета»

```
entity Inhibit is      -- also known as 'BUT-NOT'
  port (X,Y: in BIT;   -- as in 'X but not Y'
        Z:  out BIT); -- (see [Klir, 1972])
end Inhibit;

architecture Inhibit_arch of Inhibit is
begin
  Z <= '1' when X='1' and Y='0' else '0';
end Inhibit_arch;
```

Как и в других языках программирования, в языке VHDL пробелы и переходы с одной строки на другую в общем случае игнорируются, и для удобства чтения их можно вставлять как угодно. *Комментарии (comments)* начинаются с двух дефисов (--) и заканчиваются концом строки.

В языке VHDL определено много специальных строк символов, называемых *зарезервированными словами (reserved words)* или *ключевыми словами (keywords)*. В приведенном примере имеется несколько ключевых слов: entity, port, is, in, out, end, architecture, begin, when, else и not. Определяемые пользователем *идентификаторы (identifiers)* начинаются с буквы и содержат буквы, цифры и подчеркивания. (Символ подчеркивания не может следовать за другим символом подчеркивания и не может быть последним символом идентификатора.) В данном примере идентификаторами являются Inhibit, X, Y, BIT, Z и Inhibit_arch. «BIT» — это встроенный идентификатор предопределенного типа; он не считается зарезервированным словом, так как его можно переопределять. Зарезервированные слова и идентификаторы не чувствительны к регистру.

В табл. 4.27 представлен синтаксис объявления объекта. Целью объявления объекта, помимо присвоения объекту имени, является определение сигналов внешнего интерфейса или *портов* (*ports*) в части объявления объекта, которая называется *объявлением портов* (*port declaration*). Кроме ключевых слов *entity*, *is*, *port* и *end*, объявление объекта содержит следующие элементы:

<i>entity-name</i>	выбираемое пользователем имя объекта;								
<i>signal-names</i>	список выбираемых пользователем имен сигналов внешнего интерфейса, состоящий из одного имени или из большего числа имен, разделенных запятой;								
<i>mode</i>	одно из четырех зарезервированных слов, определяющих направление передачи сигнала: <table data-bbox="301 478 1063 821"> <tr> <td><i>in</i></td><td>сигнал на входе объекта;</td></tr> <tr> <td><i>out</i></td><td>сигнал на выходе объекта; заметьте, что значение такого сигнала нельзя «прочитать» внутри структуры объекта; он доступен только объектам, использующим данный объект;</td></tr> <tr> <td><i>buffer</i></td><td>сигнал на выходе объекта, такой что его значение можно читать также внутри структуры данного объекта;</td></tr> <tr> <td><i>inout</i></td><td>сигнал, который может быть входным или выходным для данного объекта; обычно этот режим используется применительно к входам/выходам ПЛУ с тремя состояниями;</td></tr> </table>	<i>in</i>	сигнал на входе объекта;	<i>out</i>	сигнал на выходе объекта; заметьте, что значение такого сигнала нельзя «прочитать» внутри структуры объекта; он доступен только объектам, использующим данный объект;	<i>buffer</i>	сигнал на выходе объекта, такой что его значение можно читать также внутри структуры данного объекта;	<i>inout</i>	сигнал, который может быть входным или выходным для данного объекта; обычно этот режим используется применительно к входам/выходам ПЛУ с тремя состояниями;
<i>in</i>	сигнал на входе объекта;								
<i>out</i>	сигнал на выходе объекта; заметьте, что значение такого сигнала нельзя «прочитать» внутри структуры объекта; он доступен только объектам, использующим данный объект;								
<i>buffer</i>	сигнал на выходе объекта, такой что его значение можно читать также внутри структуры данного объекта;								
<i>inout</i>	сигнал, который может быть входным или выходным для данного объекта; обычно этот режим используется применительно к входам/выходам ПЛУ с тремя состояниями;								
<i>signal-type</i>	встроенный или определенный пользователем тип сигнала; в следующих разделах мы будем много говорить об этом.								

Обратите внимание, что после заключительного *signal-type* нет точки с запятой; изменение порядка следования закрывающей скобки и точки с запятой после нее — типичная синтаксическая ошибка программиста, начинающего писать на языке VHDL.

Табл. 4.27. Синтаксис объявления объекта на языке VHDL

```
entity entity-name is
  port (signal-names : mode signal-type;
        signal-names : mode signal-type;
        ...
        signal-names : mode signal-type);
end entity-name;
```

Порты объекта, а также направление передачи и типы сигналов — это все, что видят другие модули, использующие данный модуль. Внутренняя работа объекта задается его *определением архитектуры* (*architecture definition*), синтаксис которого в общем случае имеет вид, указанный в табл. 4.28. *Имя объекта* (*entity-name*) в этом определении должно быть таким же, какое раньше было присвоено объекту в объявлении объекта. *Имя архитектуры* (*architecture-name*) — это выбираемый пользователем идентификатор, обычно так или иначе связанный с именем объекта; при желании имя архитектуры может быть тем же самым, что и имя объекта.

Табл. 4.28. Синтаксис определения архитектуры на языке VHDL

```

architecture architecture-name of entity-name is
    type declarations
    signal declarations
    constant declarations
    function definitions
    procedure definitions
    component declarations
begin
    concurrent-statement
    ...
    concurrent-statement
end architecture-name;

```

Сигналы внешнего интерфейса архитектуры (порты) наследуются от той части объявления соответствующего объекта, где объявляются порты. У архитектуры могут быть также сигналы и другие объявления, являющиеся для нее локальными, подобно тому как это имеет место в других языках высокого уровня. В отдельном «пакете», используемом несколькими объектами, можно сделать объявления, общие для этих объектов, о чем будет сказано позднее.

Объявления в табл. 4.28 могут располагаться в произвольном порядке. В свое время мы рассмотрим много различных способов записи объявлений и операторов в определении архитектуры. Начать легче всего с *объявления сигнала* (*signal declaration*), которое сообщает ту же самую информацию о сигнале, какую содержит объявление порта, за исключением того, что вид сигнала не задается:

```
signal signal-names : signal-type;
```

В архитектуре может быть объявлено любое число сигналов, начиная с нуля, и они приблизительно соответствуют помеченным соединениям в принципиальной схеме. Их можно считывать и записывать внутри определения архитектуры и, подобно другим локальным элементам, на них можно ссылаться только в пределах данного определения архитектуры.

Переменные (variables) в языке VHDL похожи на сигналы, за исключением того, что, как правило, они не имеют никакого физического смысла в схеме. Действительно, обратите внимание, что, согласно табл. 4.28, в определении архитектуры не предусмотрено «объявление переменных». Переменные используются в функциях, процедурах и процессах языка VHDL. Каждый из этих элементов программы мы рассмотрим позднее. Вот у них внутри имеются *объявления переменных (variable definitions)*, и эти объявления в точности подобны объявлениям сигналов, за исключением того, что употребляется ключевое слово *variable*:

```
variable variable-names : variable-type;
```

4.7.3. Типы и константы

Каждому сигналу, переменной и константе в программе на языке VHDL необходимо поставить в соответствие *тип (type)*. Типом определяется множество или диа-

пазон значений, которые может принимать данный элемент, и обычно имеется набор операторов (таких как сложение, логическое И и т.д.), связываемых с данным типом.

В языке VHDL есть всего лишь несколько *предопределенных типов* (*predefined types*); они перечислены в табл. 4.29. В дальнейшем в этой книге будут использованы только следующие предопределенные типы: *integer*, *character* и *boolean*. Вы можете подумать, что при цифровом проектировании большую роль должны играть имена "bit" и "bit_vector", но оказывается, что более полезны определяемые пользователем варианты этих типов, как это вскоре будет объяснено.

Табл. 4.29. Предопределенные типы языка VHDL

bit	character	severity_level
bit_vector	integer	string
boolean	real	time

Типом *integer* определяется диапазон значений целых чисел, который, как минимум, простирается от -2147483647 до $+2147483647$ (от $-2^{31} + 1$ до $+2^{31} - 1$); в некоторых реализациях языка VHDL этот диапазон может быть и шире. Типом *boolean* предусматриваются два значения: *true* и *false*. Тип *character* содержит все символы 8-битового набора ISO, из которых первые 128 являются символами стандарта ASCII. Встроенные операторы для типов *integer* и *boolean* приведены в табл. 4.30.

Табл. 4.30. Предопределенные операторы для типов *integer* и *boolean* в языке VHDL

Операторы для типа <i>integer</i>		Операторы для типа <i>boolean</i>	
+	сложение	and	И
-	вычитание	or	ИЛИ
*	умножение	nand	И-НЕ
/	деление	nor	ИЛИ-НЕ
mod	деление по модулю	xor	ИСКЛЮЧАЮЩЕЕ ИЛИ
rem	остаток от деления по модулю	xnor	ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ
abs	абсолютное значение	not	дополнение (инверсия)
**	возведение в степень		

Чаще всего в типичных программах на языке VHDL используются *определяемые пользователем типы* (*user-defined types*), а из них самыми употребительными являются *перечислимые типы* (*enumerated types*), которые определяются путем перечисления их значений. Предопределяемые типы *boolean* и *character* — это *неречислимые типы*. Формат объявления типа в случае перечислимого типа указан в первой строке табл. 4.31. Здесь *value-list* представляет собой список (перечисление) всех возможных значений этого типа, разделяемых запятыми. Значе-

ниями могут быть определяемые пользователем идентификаторы или символы (где под «символом» понимается символ ISO, заключенный в одинарные кавычки). Идентификаторы чаще всего применяются для обозначения альтернатив или состояний конечного автомата, например:

```
type traffic_light_state is (reset, stop, wait, go);
```

Символы используются в очень важном случае стандартного определяемого пользователем логического типа *std_logic* (см. табл. 4.32), являющегося частью стандартного пакета IEEE 1164, рассматриваемого в разделе 4.7.5. Этот тип включает не только «0» и «1», но также и семь других значений, которые оказываются полезными при моделировании логического сигнала (бита) в реальной логической схеме, как это детально объясняется в разделе 5.6.4.

Табл. 4.31. Синтаксис объявления типов и констант в языке VHDL

```
type type-name is (value-list);

subtype subtype-name is type-name start to end;
subtype subtype-name is type-name start downto end;

constant constant-name: type-name := value;
```

Табл. 4.32. Определение типа *std_logic* в языке VHDL (о значении «resolved» см. раздел 5.6.4)

```
type STD_ULOGIC is ( 'U', -- Uninitialized
                    'X', -- Forcing Unknown
                    '0', -- Forcing 0
                    '1', -- Forcing 1
                    'Z', -- High Impedance
                    'W', -- Weak Unknown
                    'L', -- Weak 0
                    'H', -- Weak 1
                    '-' -- Don't care
                );

subtype STD_LOGIC is resolved STD_ULOGIC;
```

Язык VHDL позволяет пользователю создавать также *подтипы* (*subtypes*) согласно синтаксису, указанному в табл. 4.31. Значения подтипа должны быть слитным подмножеством значений, предусмотренных основным типом, начиная со *start* и кончая *end*. Для перечислимого типа «слитность» означает расположение на соседних позициях в исходном списке значений *value-list*. Вот несколько примеров определения подтипов:

```
subtype twoval_logic is std_logic range '0' to '1';
subtype fourval_logic is std_logic range 'X' to 'Z';
subtype negint is integer range -2147483647 to -1;
subtype bitnum is integer range 31 downto 0;
```

СТРОГОЕ СОБЛЮДЕНИЕ ТИПОВ

VHDL, как и C, является языком со строгим контролем типов. Это означает, что компилятор не позволит вам присвоить сигналу или переменной значение, которое в точности не согласуется с объявленным типом этого сигнала или переменной.

Требование строго соблюдать типы — это одновременно и благо, и бедствие. Это делает вашу программу более надежной и ее легче отлаживать, поскольку оказывается трудным делать «глупые ошибки», присваивая значения неправильного типа или не подходящей величины. С другой стороны, иногда это может раздражать. Даже при выполнении простых действий может потребоваться обращение к функциям преобразования типов в явном виде, например, когда необходимо 2-битовый сигнал интерпретировать как целое число, чтобы перейти к одному из возможных случаев в операторе “case”.

Заметьте, что порядок следования значений в указываемом диапазоне может быть в сторону возрастания или в сторону убывания в зависимости от того, какое из ключевых слов *to* или *downto* употреблено. Из-за некоторых особенностей подтипов это различие может быть существенным, но мы не будем использовать эти особенности в нашей книге и поэтому ограничимся уже сказанным.

В языке VHDL есть два предопределенных подтипа *integer*:

```
subtype natural is integer range 0 to highest-integer;
subtype positive is integer range 1 to highest-integer;
```

Константы (constants) способствуют удобству чтения программ, возможности их поддержания и сопровождения, а также переносу на какой-либо другой язык. Синтаксис объявления констант (*constant declaration*) в языке VHDL указан в последней строке в табл. 4.31; его можно проиллюстрировать следующими примерами:

```
constant BUS_SIZE: integer := 32;      -- width of component
constant MSB: integer := BUS_SIZE-1;  -- bit number of MSB
constant Z: character := 'Z';         -- synonym for Hi-Z value
```

ЧТО ЗА СИМВОЛ?

Вы можете удивиться тому, что в типе *std_logic* значения задаются символами, а не однобуквенными идентификаторами. Ясно, что “0”, “X” и т.д. было бы легче набрать, чем “U”, “X” и т.д. Ну, прежде всего, тогда потребовался бы другой идентификатор, не “—”, для безразличных значений, но это еще полбеды. Главная причина употребления символов в одинарных кавычках состоит в том, что нельзя воспользоваться «нулем» и «единицей» — “0” и “1”, — поскольку уже принято, что их следует распознавать как настоящие целые числа. Это возвращает нас снова к строгому следованию типам в языке VHDL; едва ли было целесообразно позволить компилятору осуществлять автоматическое преобразование типов в зависимости от контекста.

НЕНАТУРАЛЬНЫЕ ДЕЙСТВИЯ

В языке VHDL подтип *"natural"* определяется как множество неотрицательных целых чисел, начиная с 0, но большинство математиков считают, что, по определению, натуральные числа начинаются с 1. Действительно в своей ранней истории человечество начинало счет с 1; понятие "0" появилось много позднее. Однако полемика на эту тему продолжается, особенно в наш компьютерный век, когда большинству из нас приходится начинать счет с 0. Последние соображения по данной проблеме можно найти в Интернете, осуществив поиск по словам *"natural numbers"*.

Обратите внимание, что значение константы может быть задано простым выражением. Константы можно использовать повсюду, где встречаются соответствующие значения, и они особенно полезны при определении типов, как будет вскоре показано.

Другую очень важную группу определяемых пользователем типов образуют *типы массивов (array types)*. Как и в других языках, в языке VHDL *массив (array)*, по определению, — это упорядоченный набор элементов одного и того же типа, отдельные компоненты которого выбираются с помощью *индекса массива (array index)*. Возможны несколько вариантов синтаксиса объявления массива в языке VHDL; они представлены в табл. 4.33. В первых двух вариантах *start* и *end* являются целыми числами, которыми задается возможный диапазон изменения индекса массива *n*, следовательно, полное число элементов массива. В последних трех вариантах диапазоном изменения индекса массива являются все значения указанного типа (*range-type*) или подмножество этих значений.

Табл. 4.33. Синтаксис объявления массивов в языке VHDL

```

type type-name is array (start to end) of element-type;
type type-name is array (start downto end) of element-type;
type type-name is array (range-type) of element-type;
type type-name is array (range-type range start to end) of element-type;
type type-name is array (range-type range start downto end) of element-type;

```

В табл. 4.34 приведены примеры объявления массивов. Первые два примера совсем обычны и демонстрируют задание диапазона изменения индекса в сторону возрастания и в сторону убывания. Следующий пример показывает, как можно воспользоваться константой `WORD_LEN` при объявлении массива; отсюда видно также, что границу диапазона можно задать простым выражением. Из третьего примера следует, что сам элемент массива может быть массивом; таким образом создается двумерный массив. Последний пример показывает, что множество возможных значений элементов массива можно задать, указав перечислимый тип (или подтип); в этом примере массив состоит из четырех элементов согласно данному нами чуть раньше определению типа `traffic_light_state`.

Табл. 4.34. Примеры объявления массивов в языке VHDL

```

type monthly_count is array (1 to 12) of integer;
type byte is array (7 downto 0) of STD_LOGIC;

constant WORD_LEN: integer := 32;
type word is array (WORD_LEN-1 downto 0) of STD_LOGIC;

constant NUM_REGS: integer := 8;
type reg_file is array (1 to NUM_REGS) of word;

type statecount is array (traffic_light_state) of integer;

```

Элементы массива считаются упорядоченными слева направо в том же направлении, в каком индекс пробегает свои значения. Таким образом, индексы самых левых элементов массивов типов `monthly_count`, `byte`, `word`, `reg_file` и `statecount` в табл. 4.34 равны 1, 7, 31, 1 и `reset` соответственно.

Обращение к отдельным элементам массивов в операторах программы на языке VHDL осуществляется путем указания имени массива и индекса элемента в круглых скобках. Если, например, `M`, `B`, `W`, `R` и `S` — сигналы или переменные тех пяти типов массивов, которые приведены в табл. 4.34, то любая из записей `M(1)`, `B(5)`, `W(WORD_LEN-5)`, `R(0,0)`, `R(0)` и `S(reset)` является правильным указанием элемента.

Массивы-литералы (array literals) можно задать, перечисляя в скобках значения элементов. Например, переменной `B` типа `byte` можно задать значение, состоящее из одних единиц, оператором

```
B := ('1', '1', '1', '1', '1', '1', '1', '1');
```

В языке VHDL возможно и в более сжатой форме задавать значения, указывая индекс. Например, следующая запись обеспечивает присвоение единичных значений всем элементам переменной `W` типа `word`, за исключением младших разрядов каждого байта, которым присваиваются нулевые значения:

```
W := (0 => '0', 8 => '0', 16 => '0', 24 => '0', others => '1');
```

Рассмотренные правила справедливы при любом *типе элементов (element-type)*, но литерал типа `STD_LOGIC` легче всего записать в виде «строки». *Строкой (string)* в языке VHDL является последовательность символов ISO, заключенная в двойные кавычки, типа `"Hi there"`. Строка — это, конечно, массив символов; поэтому массиву типа `STD_LOGIC` заданной длины можно присвоить значение, выраженное строкой той же длины, если только символы в строке принадлежат набору из девяти символов, которыми, по определению, исчерпываются возможные значения элементов типа `STD_LOGIC`: `'0'`, `'1'`, `'U'` и т.д. Таким образом, предыдущие два примера можно переписать в виде:

```

B = "11111111";
W = "11111110111111101111111011111110";

```

Можно также указывать подмножество непосредственно следующих один за другим элементов массива или, как говорят, *вырезку из массива (array slice)*, задавая начальный и конечный индексы подмножества; например: M(6 to 9), B(3 downto 0), W(15 downto 8), R(0, 7 downto 0), R(1 to 2), S(stop to go). Заметьте, что направление изменения индекса в вырезке должно быть таким же, как у исходного массива.

Наконец, массивы или элементы массивов можно объединять с помощью *оператора конкатенации & (concatenation operator)*, который соединяет массивы и элементы в том порядке, в каком они записаны слева направо. Например, запись '0' & '1' & "12" эквивалентна строке "0112", а выражение B(6 downto 0) & B(7) представляет собой циклический сдвиг 8-разрядного массива B на 1 разряд влево.

Самым важным типом массивов в типичной программе на языке VHDL является определяемый пользователем в соответствии со стандартом IEEE 1164 логический тип `std_logic_vector`, которым задается упорядоченный набор элементов типа `std_logic`. Определение этого типа имеет вид:

```
type STD_LOGIC_VECTOR is array (natural range <>) of STD_LOGIC;
```

Это пример *типа массива без ограничений (unconstrained array type)*: диапазон возможных значений индекса массива не задан, за исключением того, что он должен быть подмножеством определенного типа, в данном случае – типа `natural`. Эта особенность языка VHDL позволяет записывать архитектуры, функции и другие элементы программ в более общем виде, до некоторой степени независимо от размеров массивов и диапазонов возможных значений индексов. Действительный диапазон значений индекса определяется в тот момент, когда сигналу или переменной ставится в соответствие этот тип. В следующем разделе мы увидим примеры этого.

4.7.4. Функции и процедуры

Подобно функциям в любом языке программирования высокого уровня, *функция (function)* в языке VHDL получает ряд *аргументов (arguments)* и возвращает *результат (result)*. При определении функции на языке VHDL и при ее вызове каждый из аргументов и результат имеют предустановленный тип.

Синтаксис *определения функции (function definition)* приведен в табл. 4.35. За присвоением функции определенного имени следует список *формальных параметров (formal parameters)*, который используется в теле функции; число параметров может быть любым, начиная с нуля. При вызове функции формальные параметры в обращении к ней замещаются *действительными параметрами (actual parameters)*. В соответствии со строгим следованием типам в языке VHDL действительные параметры должны быть того же типа или подтипа, что и формальные параметры. Когда функция вызывается из архитектуры, на место ее вызова возвращается значение, тип которого указывается посредством *return-type*.

Табл. 4.35. Синтаксис определения функции в языке VHDL

```

function function-name (
    signal-names : signal-type;
    signal-names : signal-type;
    ...
    signal-names : signal-type
) return return-type is
    type declarations
    constant declarations
    variable declarations
    function definitions
    procedure definitions
begin
    sequential-statement
    ...
    sequential-statement
end function-name;

```

Как видно из таблицы, внутри функции можно определить ее собственные локальные типы, константы, переменные и вложенные функции и процедуры. Между ключевыми словами *begin* и *end* располагается ряд «последовательных операторов», которые исполняются при вызове функции. Последовательные операторы и их синтаксис будут предметом более подробного разбора позднее, но вам должны быть известны приводимые здесь примеры с учетом вашего предыдущего опыта программирования.

Архитектуру «вейтиля запрета» на языке VHDL, приведенную в табл. 4.26, можно видоизменить, используя функцию, как показано в табл. 4.36. В определении функции момент возврата к месту вызова указывается ключевым словом *return*, за которым следует выражение, значение которого и будет возвращено. Тип результата вычисления этого выражения должен быть согласован со значением *return-type* в объявлении функции.

В логическом пакете стандарта IEEE 1164 определено много функций, оперирующих типами *std_logic* и *std_logic_vector*. Помимо того, что предусматривается ряд определяемых пользователем типов, пакет содержит также основные логические операции над сигналами или переменными этих типов, такие как *and* и *or*. Язык VHDL обладает тем достоинством, что в нем возможна *перегрузка операторов* (*operator overloading*). Это позволяет пользователю выбирать активизируемую функцию посредством применения символа встроенной операции (*and*, *or*, *+* и т. д.) к согласованному набору типов операторов. Возможно несколько определений одного и того же символа операции; каждый раз, когда встречается данный оператор, компилятор автоматически выбирает определение, согласованное по типу операндов.

В качестве примера табл. 4.37 содержит код, взятый из пакета IEEE, которым определяется операция «*and*» для операндов типа *std_logic*. Может казаться, что этот отрывок сложен, но нами уже введены все основные элементы языка, употребляемые здесь (за исключением слова «*resolved*», которое будет рассмотрено в разделе 5.6.4 в связи с логическими схемами с тремя состояниями).

```

architecture Inhibit_archf of Inhibit is
function ButNot (A, B: bit) return bit is
begin
    if B = '0' then return A;
    else return '0';
    end if;
end ButNot;

begin
    Z <= ButNot(X,Y);
end Inhibit_archf;

```

Табл. 4.36. Программа для «функции запрета» на языке VHDL

Табл. 4.37. Определения в стандарте IEEE 1164, относящиеся к операции "and" над величинами типа STD_LOGIC

```

SUBTYPE UX01 IS resolved std_ulogic RANGE 'U' TO '1';
-- ('U','X','0','1')
TYPE stdlogic_table IS ARRAY(std_ulogic, std_ulogic) OF std_ulogic;
-- truth table for "and" function
CONSTANT and_table : stdlogic_table := (
--
--      { U   X   0   1   Z   W   L   H   -   }
--
    ( 'U', 'U', '0', 'U', 'U', 'U', '0', 'U', 'U' ), -- { U |
    ( 'U', 'X', '0', 'X', 'X', 'X', '0', 'X', 'X' ), -- { X |
    ( '0', '0', '0', '0', '0', '0', '0', '0', '0' ), -- { 0 |
    ( 'U', 'X', '0', '1', 'X', 'X', '0', '1', 'X' ), -- { 1 |
    ( 'U', 'X', '0', 'X', 'X', 'X', '0', 'X', 'X' ), -- { Z |
    ( 'U', 'X', '0', 'X', 'X', 'X', '0', 'X', 'X' ), -- { W |
    ( '0', '0', '0', '0', '0', '0', '0', '0', '0' ), -- { L |
    ( 'U', 'X', '0', '1', 'X', 'X', '0', '1', 'X' ), -- { H |
    ( 'U', 'X', '0', 'X', 'X', 'X', '0', 'X', 'X' ) -- { - |
);

FUNCTION "and" ( L : std_ulogic; R : std_ulogic ) RETURN UX01 IS
BEGIN
    RETURN (and_table(L, R));
END "and";

```

Переменные данной функции могут быть величинами типа `std_ulogic` или его подтипа `std_logic`. Другой подтип `UX01`, по определению, должен быть использован в качестве типа возвращаемого функцией результата; даже если один из операндов в операции "and" имеет нелогическое значение ('Z', 'W' и т. д.), то результатом обращения к функции будет только одно из четырех возможных значений. Тип `stdlogic_table` задает двумерный массив 9×9, индексам которого является пара величин типа `std_ulogic`. Элементы таблицы `and_table` расположены так, что при значениях любого из индексов '0' или 'L' (слабый '0') элемент равен '0'. Значение

'1' извлекается только тогда, когда оба индекса равны '1' или 'H' (слабая '1'). В противном случае результат операции равен '0' или 'X'.

Двойные кавычки у имени функции в самом определении функции указывают на перегрузку оператора. «Исполняемая» часть функции состоит всего лишь из одного оператора, который возвращает элемент таблицы, выбранный по двум переменным L и R функции "and".

Из-за требований языка VHDL строго следовать типам, часто бывает необходимо преобразовать сигнал одного типа в сигнал другого типа; пакет IEEE 1164 содержит несколько функций преобразования: например, переход от типа BIT к типу STD_LOGIC и наоборот. Величину типа STD_LOGIC_VECTOR обычно нужно преобразовать в соответствующее целое число. В стандарте IEEE 1164 нет такой функции преобразования, поскольку разным разработчикам могут понадобиться различные представления чисел, например, со знаком или без знака. Однако можно самим задать свое собственное преобразование так, как это сделано в табл. 4.38.

Табл. 4.38. Функция преобразования типа STD_LOGIC_VECTOR в тип INTEGER на языке VHDL

```
function CONV_INTEGER (X: STD_LOGIC_VECTOR) return INTEGER is
  variable RESULT: INTEGER;
begin
  RESULT := 0;
  for i in X'range loop
    RESULT := RESULT * 2;
    case X(i) is
      when '0' | 'L' => null;
      when '1' | 'H' => RESULT := RESULT + 1;
      when others    => null;
    end case;
  end loop;
  return RESULT;
end CONV_INTEGER;
```

В функции CONV_INTEGER применен простой итеративный алгоритм, эквивалентный приведенной в параграфе 2.3 формуле представления числа в виде последовательных вложений. Мы отложим описание используемых здесь операторов FOR, CASE и WHEN до раздела 4.7.8, сосредоточив внимание на основной идее. Оператор null (null statement) совсем прост: он означает «ничего не делать». Пределы выполнения цикла FOR определяются параметром "X'range", где одиночная кавычка после имени сигнала означает «атрибут» ("attribute"), а range – встроенный идентификатор атрибута, который применяется только по отношению к массивам и означает «перебрать все значения индекса данного массива слева направо».

Можно осуществить преобразование и в обратном направлении, то есть перейти от целого числа к величине типа STD_LOGIC_VECTOR, как показано в табл. 4.39. В этом случае необходимо задать не только преобразуемое целое (ARG),

но также и желаемое число разрядов (SIZE) в результате преобразования. Обратите внимание, что в данной функции объявлена локальная переменная "result" типа STD_LOGIC_VECTOR с интервалом, в пределах которого изменяется индекс, зависящим от значения SIZE. По этой причине параметр SIZE должен быть константой или какой-то другой величиной, которая известна к моменту компиляции функции CONV_STD_LOGIC_VECTOR. В этой функции реализуется алгоритм последовательного деления, также описанный в параграфе 2.3.

Табл. 4.39. Функция преобразования типа INTEGER в тип STD_LOGIC_VECTOR на языке VHDL

```
function CONV_STD_LOGIC_VECTOR(ARG: INTEGER; SIZE: INTEGER)
    return STD_LOGIC_VECTOR is
    variable result: STD_LOGIC_VECTOR (SIZE-1 downto 0);
    variable temp: integer;
begin
    temp := ARG;
    for i in 0 to SIZE-1 loop
        if (temp mod 2) = 1 then result(i) := '1';
        else result(i) := '0';
        end if;
        temp := temp / 2;
    end loop;
    return result;
end;
```

Процедура (procedure) в языке VHDL похожа на функцию за исключением того, что она не возвращает результат. Если обращение к функции может играть роль выражения, то вызов процедуры можно использовать в качестве оператора. В языке VHDL допускается задание аргументам процедур типа out или типа inout, что фактически и делает возможным «возвращение» процедурой результата. Мы не будем пользоваться процедурами в нашей книге и поэтому не станем рассматривать их подробнее.

4.7.5. Библиотеки и пакеты

VHDL-библиотека (*library*) – это место, где компилятор VHDL хранит информацию об отдельном варианте проекта, включая промежуточные файлы, используемые при анализе, моделировании и синтезе в рамках данной разработки. Место библиотеки в файловой системе компьютера зависит от реализации. Для очередного VHDL-проекта компилятор автоматически создает библиотеку под именем "work" и использует ее.

У законченного VHDL-проекта обычно бывает много файлов, каждый из которых содержит различные компоненты проекта, включая объекты и архитектуры. Анализируя отдельные файлы, компилятор помещает результаты в библиотеку "work", а также ищет в этой библиотеке необходимые определения, например, другие объекты. С учетом этого большой проект можно разбить на несколько файлов; компилятор найдет все, что нужно, по внешним указателям.

Но не вся необходимая информация может находиться в библиотеке “work”. Например, разработчик может опираться на определения и функциональные модули, общие для некоторого семейства различных проектов. У каждого проекта есть своя собственная библиотека “work” (обычно это подкаталог внутри всего каталога, относящегося к данному проекту), но в нем должны быть ссылки на общую библиотеку, содержащую совместно используемые определения. Даже в малых проектах может использоваться библиотека, которая содержит, например, стандартные определения IEEE. Разработчик может присвоить имя такой библиотеке с помощью предложения *library (library clause)* в начале соответствующего файла. Например, библиотеку IEEE можно задать фразой:

```
library ieee;
```

Предложение “library work;” помещается в начале каждого файла VHDL-проекта явно.

Присвоение имен библиотекам проекта обеспечивает доступ к любым ранее проанализированным и запомненным объектам и архитектурам, но не к определениям типов и тому подобному. Эту функцию выполняют «пакеты» и «предложения use».

VHDL-пакет (*package*) – это файл, содержащий определения элементов, которые могут быть использованы другими программами. В пакет можно включить элементы такого рода, как сигнал, тип, константа, функция, процедура и объявления компонентов.

Помещенные в пакет сигналы являются «глобальными» и доступны любому VHDL-объекту, использующему этот пакет. Типы и константы, упомянутые в пакете, известны в любом файле, использующем этот пакет. Аналогично, из файла, использующих данный пакет, можно вызвать перечисленные в нем функции и процедуры, а архитектуры, опирающиеся на этот пакет, могут «обрабатывать» включенные в него компоненты, описываемые в следующем разделе.

Проект может «использовать» тот или иной пакет, если в начале файла, относящегося к данному проекту, помещено предложение *use (use clause)*. Например, чтобы воспользоваться всеми определениями пакета, содержащего стандарт IEEE 1164, нам следует написать:

```
use ieee.std_logic_1164.all;
```

Здесь “ieee” – это имя библиотеки, ранее введенное предложением *library*. В этой библиотеке файл с именем “std_logic_1164” содержит желаемые определения. Приставка “all” велит компилятору использовать все определения этого файла. Вместо “all” можно написать имя какого-то одного элемента, когда необходимо использовать только его определение, например:

```
use ieee.std_logic_1164.std_ulogic;
```

Эта фраза обеспечит доступ только к определению типа *std_ulogic*, приведенному в табл. 4.32, без учета всех других родственных типов и функций. Однако, записывая подряд несколько предложений “use”, можно добавить использование и других определений.

VHDL-СТАНДАРТЫ IEEE

Язык VHDL предоставляет замечательную возможность расширять типы данных и множество функций. Это важное свойство, поскольку встроенные типы `BIT` и `BIT_VECTOR`, в действительности, вовсе не адекватны требованиям моделирования реальных схем, когда обрабатываемые сигналы могут соответствовать третьему состоянию, а также быть неизвестными, безразличными и меняющейся интенсивности.

Поэтому вскоре после формализации языка в стандарте IEEE 1076 коммерческие поставщики начали вводить свои собственные встроенные типы данных, чтобы оперировать логическими величинами, отличающимися от 0 и 1. У разных поставщиков были, конечно, различные определения для этих расширенных типов, из-за чего началось возведение «Вавилонской башни».

Чтобы избежать этого, Институтом инженеров по электротехнике и электронике был разработан стандартный логический пакет 1164 (`std_logic_1164`), девятизначная логическая система которого удовлетворяет большую часть потребностей проектировщиков. За этим пакетом позднее последовал стандарт 1076-3, описываемый в разделе 5.9.6, который содержит несколько пакетов со стандартными типами и операциями для векторов `STD_LOGIC`, интерпретируемых как целые числа со знаком и без знака. Эти пакеты включают `std_logic_arith`, `std_logic_signed` и `std_logic_unsigned`.

Следование стандартам IEEE гарантирует разработчикам высокую степень переносимости их проектов и возможность их взаимодействия. Последнее становится все более важным, так как переход на очень большие специализированные ИС делает необходимой кооперацию не только многих проектировщиков, но также и многих поставщиков, каждый из которых вносит свой вклад в создаваемую по кусочкам «систему в одном кристалле».

```

package package-name is
    type declarations
    signal declarations
    constant declarations
    component declarations
    function declarations
    procedure declarations
end package-name;

package body package-name is
    type declarations
    constant declarations
    function definitions
    procedure definitions
end package-name;

```

Табл. 4.40. Синтаксис определения VHDL-пакета

Стандартными пакетами не исчерпываются все возможности. Любой разработчик может написать свой собственный пакет согласно синтаксису, приведенному в табл. 4.40. Все элементы, объявленные между “package” и первым оператором “end”, видны из любого файла проекта, использующего этот пакет; элементы, следующие за ключевым словом “package body”, являются локальными. Заметьте, в частности, что первая часть содержит «объявления функций», но не определения. В *объявлении функции (function declaration)* перечислены только имя функции, аргументы и тип вплоть до ключевого слова “is” в табл. 4.35, но не включая его. Полное определение функции приводится в теле пакета и пользователи функции его не видят.

4.7.6. Элементы структурного проектирования

Теперь мы, наконец, готовы взглянуть на внутренности VHDL-проекта, на «исполняемую» часть архитектуры. Вспомним, что согласно табл. 4.28 тело архитектуры представляет собой ряд *параллельных операторов (concurrent statements)*. В языке VHDL каждый параллельный оператор выполняется одновременно с другими параллельными операторами в теле данной архитектуры.

Такой режим исполнения заметно отличается от последовательного выполнения операторов в программном обеспечении, написанном на одном из обычных языков программирования. Параллельные операторы необходимы для того, чтобы отобразить поведение схемы, у которой соединенные между собой элементы воздействуют друг на друга непрерывно, а не в отдельные моменты времени, следующие один за другим в определенном порядке. Таким образом, правило моделирования состоит в том, что в случае, когда последним оператором в теле VHDL-архитектуры изменяется сигнал, используемый в первом операторе данной архитектуры, программа должна вернуться назад к первому оператору и скорректировать его результаты, приведя их в соответствие с только что изменившимся сигналом. На самом деле моделирующая программа будет осуществлять множественные изменения и обновления результатов до тех пор, пока моделируемая схема не стабилизируется; подробнее мы рассмотрим этот вопрос в разделе 4.7.9.

В языке VHDL есть несколько параллельных операторов, а также механизм связывания в один узел набора последовательных операторов с тем, чтобы они исполнялись, как один параллельный оператор. Различное использование параллельных операторов привело к возникновению трех стилей проектирования и описания схем, слегка отличающихся один от другого; об этом и пойдет речь в данном разделе и двух следующих.

Самым главным параллельным оператором в языке VHDL является *оператор component (component statement)*, синтаксис которого указан в табл. 4.41. Здесь *component-name* — имя определенного ранее объекта, который должен быть использован или *подвергнут обработке (instantiate)* в теле данной архитектуры. Для каждого оператора component с обращением к названному объекту создается одна копия этого объекта; каждая копия должна иметь уникальную метку *label*.

Табл. 4.41. Синтаксис оператора `component` в языке VHDL

```
label: component-name port map(signal1, signal2, ..., signaln);
```

```
label: component-name port map(port1=>signal1, port2=>signal2, ..., portn=>signaln);
```

Ключевое слово `port map` вводит список, посредством которого портам названного объекта ставятся в соответствие сигналы данной архитектуры. Список может быть представлен одним из двух различных способов. Первый из них является позиционным: как и в обычных языках программирования, сигналы, упоминаемые в списке, связываются с портами объекта в том же самом порядке, в каком порты перечислены в определении объекта. Второй способ записи – явный: каждый порт объекта связывается с сигналом посредством оператора “=>”, и эти соответствия могут следовать в любом порядке.

До того как компонент будет подвергнут обработке внутри архитектуры, он должен быть декларирован *объявлением компонента* (*component declaration*) в определении архитектуры (см. табл. 4.28). Как видно из табл. 4.42, объявление компонента является по существу таким же, что и часть объявления соответствующего объекта, где объявляются порты: приводятся имя, режим и тип каждого порта.

```
component component-name
  port (signal-names : mode signal-type;
        signal-names : mode signal-type;
        ...
        signal-names : mode signal-type);
end component;
```

Табл. 4.42. Синтаксис объявления компонента в языке VHDL

Используемые в архитектуре компоненты могут быть либо ранее определенными элементами данного проекта, либо библиотечными элементами. Табл. 4.43 представляет собой пример VHDL-объекта и его архитектуры, в которой используются компоненты «устройства для обнаружения простых чисел», структурно идентичные отдельным вентилям в схеме на рис. 4.30(с). В объявлении объекта названы входы схемы и ее выход. В части архитектуры, отведенной под объявление, присваиваются имена всем сигналам и компонентам, которые используются внутри данной архитектуры. Этот пример был разработан и скомпилирован в программной среде Xilinx Foundation 1.5 (см. Обзор литературы), где INV, AND2, AND3 и OR4 являются предопределенными компонентами.

Заметьте, что операторы `component` в табл. 4.43 исполняются *параллельно*. Даже в том случае, когда операторы расположены в другом порядке, результатом синтеза будет та же самая схема и моделирование ее работы будет приводить к одному и тому же.

VHDL-архитектуру, в которой используются компоненты, часто называют *структурным описанием* (*structural description*) или *структурной моделью*

(*structural design*), поскольку ею задается реализующая данный объект точная конфигурация соединений, по которым сигналы передаются от одного элемента к другому. В этом отношении ясное структурное описание эквивалентно схеме устройства или списку соединений в нем.

Табл. 4.43. Структурная VHDL-программа для устройства, обнаруживающего простые числа

```
library IEEE;
use IEEE.std_logic_1164.all;

entity prime is
    port ( N: in STD_LOGIC_VECTOR (3 downto 0);
          F: out STD_LOGIC );
end prime;

architecture prime1_arch of prime is
    signal N3_L, N2_L, N1_L: STD_LOGIC;
    signal N3L_N0, N3L_N2L_N1, N2L_N1_N0, N2_N1L_N0: STD_LOGIC;
    component INV port (I: in STD_LOGIC; O: out STD_LOGIC); end component;
    component AND2 port (I0,I1: in STD_LOGIC; O: out STD_LOGIC); end component;
    component AND3 port (I0,I1,I2: in STD_LOGIC; O: out STD_LOGIC); end component;
    component OR4 port (I0,I1,I2,I3: in STD_LOGIC; O: out STD_LOGIC); end component;
begin
    U1: INV port map (N(3), N3_L);
    U2: INV port map (N(2), N2_L);
    U3: INV port map (N(1), N1_L);
    U4: AND2 port map (N3_L, N(0), N3L_N0);
    U5: AND3 port map (N3_L, N2_L, N(1), N3L_N2L_N1);
    U6: AND3 port map (N2_L, N(1), N(0), N2L_N1_N0);
    U7: AND3 port map (N(2), N1_L, N(0), N2_N1L_N0);
    U8: OR4 port map (N3L_N0, N3L_N2L_N1, N2L_N1_N0, N2_N1L_N0, F);
end prime1_arch;
```

В некоторых приложениях бывает необходимо создать несколько копий определенного блока внутри архитектуры. В разделе 5.10.2 мы увидим, например, что *n*-разрядный «сумматор со сквозным переносом» можно образовать каскадным включением *n* «полных сумматоров». В языке VHDL имеется оператор *generate* (*generate statement*), который позволяет создавать такие повторяющиеся блоки посредством своего рода цикла “for” без необходимости выписывать все копии по отдельности.

Синтаксис простого итеративного цикла *generate* показан в табл. 4.44. Идентификатор *identifier* объявляется явно как переменная, тип которой совместим с диапазоном *range*. Параллельный оператор *concurrent statement* исполняется однократно для каждого возможного значения переменной *identifier* в пределах диапазона; переменную *identifier* можно использовать внутри параллельного оператора. В табл. 4.45 показано, как можно построить 8-разрядный инвертор.

Табл. 4.44. Синтаксис цикла *for-generate* на языке VHDL

```
label: for identifier in range generate
    concurrent-statement
end generate;
```

Табл. 4.45. VHDL-объект и его архитектура для 8-разрядного инвертора

```

library IEEE;
use IEEE.std_logic_1164.all;

entity inv8 is
    port ( X: in STD_LOGIC_VECTOR (1 to 8);
          Y: out STD_LOGIC_VECTOR (1 to 8) );
end inv8;

architecture inv8_arch of inv8 is
    component INV port (I: in STD_LOGIC; O: out STD_LOGIC); end component;
begin
    g1: for b in 1 to 8 generate
        U1: INV port map (X(b), Y(b));
    end generate;
end inv8_arch;

```

Значение константы должно быть известно к моменту компиляции программы, написанной на языке VHDL. Во многих приложениях бывает полезно разработать и откомпилировать объект и его архитектуру, оставляя некоторые из его параметров не заданными, например, разрядность шины. Сделать это позволяет имеющийся в языке VHDL инструмент "generic".

С помощью *объявления общности* (*generic declaration*) в объявлении объекта можно определить одну или большее число *настраиваемых констант* (*generic constant*); это необходимо сделать до объявления портов согласно синтаксису, указанному в табл. 4.46. Каждую поименованную константу можно использовать в определении архитектуры данного объекта, а задание ее значения откладывается до того момента, когда этот объект будет подвергаться обработке оператором *component* из другой архитектуры. Значения присваиваются настраиваемым константам в этом операторе *component* с помощью предложения *generic map* таким же способом, какой употреблен в предложении *port map*. В табл. 4.47 приведен пример, в котором одновременно используются инструмент *generic* и оператор *generate* для создания «шинного инвертора» с задаваемой пользователем разрядностью. В программе, представленной в табл. 4.48, обрабатывается несколько копий такого инвертора, каждый со своим числом сигнальных линий.

```

entity entity-name is
    generic (constant-names : constant-type;
            constant-names : constant-type;
            ...
            constant-names : constant-type);
    port (signal-names : mode signal-type;
          signal-names : mode signal-type;
          ...
          signal-names : mode signal-type);
end entity-name;

```

Табл. 4.46. Синтаксис
объявления общности в
объявлении объекта

Табл. 4.47. VHDL-объект и его архитектура для шинного инвертора с произвольной разрядностью

```

library IEEE;
use IEEE.std_logic_1164.all;

entity businv is
    generic (WIDTH: positive);
    port ( X: in STD_LOGIC_VECTOR (WIDTH-1 downto 0);
          Y: out STD_LOGIC_VECTOR (WIDTH-1 downto 0) );
end businv;

architecture businv_arch of businv is
    component INV port (I: in STD_LOGIC; O: out STD_LOGIC); end component;
begin
    g1: for b in WIDTH-1 downto 0 generate
        U1: INV port map (X(b), Y(b));
    end generate;
end businv_arch;

```

Табл. 4.48. VHDL-объект и его архитектура, в которой используется шинный инвертор с произвольной разрядностью

```

library IEEE;
use IEEE.std_logic_1164.all;

entity businv_example is
    port ( IN8: in STD_LOGIC_VECTOR (7 downto 0);
          OUT8: out STD_LOGIC_VECTOR (7 downto 0);
          IN16: in STD_LOGIC_VECTOR (15 downto 0);
          OUT16: out STD_LOGIC_VECTOR (15 downto 0);
          IN32: in STD_LOGIC_VECTOR (31 downto 0);
          OUT32: out STD_LOGIC_VECTOR (31 downto 0) );
end businv_example;

architecture businv_ex_arch of businv_example is
    component businv
        generic (WIDTH: positive);
        port ( X: in STD_LOGIC_VECTOR (WIDTH-1 downto 0);
              Y: out STD_LOGIC_VECTOR (WIDTH-1 downto 0) );
    end component;
begin
    U1: businv generic map (WIDTH=>8) port map (IN8, OUT8);
    U2: businv generic map (WIDTH=>16) port map (IN16, OUT16);
    U3: businv generic map (WIDTH=>32) port map (IN32, OUT32);
end businv_ex_arch;

```

4.7.7. Элементы потокового проектирования

Если бы операторы `component` были единственными параллельными операторами языка VHDL, то он лишь немногим отличался бы от простого иерархического языка описания соединений со строгим соблюдением типов. Несколько дополнительных параллельных операторов языка VHDL позволяют описывать схему в терминах потока данных и выполняемых схемой операций над этими данными. Такой подход носит название *потокового описания (dataflow description)* или *потокового проектирования (dataflow design)*.

В потоковых проектах используются два дополнительных параллельных оператора; они приведены в табл. 4.49. Чаще всего используется первый из них; он называется *параллельным сигнальным оператором присваивания (concurrent signal-assignment statement)*. Его можно прочесть так: «Сигнал с именем *signal-name* принимает значение выражения *expression*». Поскольку в языке VHDL необходимо строго соблюдать типы, тип выражения *expression* должен быть совместим с типом сигнала *signal-name*. В общем случае это означает, что типы должны быть либо тождественно одинаковыми, либо тип *expression* должен являться подтипом типа *signal-name*. В случае массивов, тип элементов и длина должны быть согласованными, однако множество значений и направление изменения индекса могут не совпадать.

Табл. 4.49. Синтаксис параллельных сигнальных операторов присваивания в языке VHDL

```

signal-name <= expression;

signal-name <= expression when boolean-expression else
               expression when boolean-expression else
               ...
               expression when boolean-expression else
               expression;

```

В табл. 4.50 представлена архитектура объекта для устройства, обнаруживающего простые числа (см. табл. 4.43), записанная в потоковой форме. При таком подходе вентили и соединения между ними в явном виде не указываются; вместо этого используются встроенные операторы языка VHDL `and`, `or` и `not`. (На самом деле для сигналов типа `STD_LOGIC` таких встроенных операторов нет, но они определяются и перегружаются пакетом IEEE 1164.) Заметьте, что у оператора `not` самый высокий приоритет, так что для получения нужного результата не требуется заключать в скобки подвыражение типа `"not N(3)"`.

Можно также воспользоваться второй, *условной* формой *параллельного сигнального оператора присваивания (conditional signal-assignment statement)* с ключевыми словами `when` и `else`, как показано в табл. 4.49. В этом случае в выражении *boolean-expression* отдельные булевы термы объединяются посредством встроенных булевых операторов языка VHDL, таких как `and`, `or` и `not`.

Под булевыми термами обычно понимаются булевы переменные или результаты сравнения, выполняемого с помощью *операторов отношений* (*relational operators*) =, /= (не равно), >, >=, < и <=.

Табл. 4.50. Поточковая VHDL-архитектура для устройства, обнаруживающего простые числа

```

architecture prime2_arch of prime is
signal N3L_N0, N3L_N2L_N1, N2L_N1_N0, N2_N1L_N0: STD_LOGIC;
begin
    N3L_N0      <= not N(3)                                and N(0);
    N3L_N2L_N1 <= not N(3) and not N(2) and      N(1)      ;
    N2L_N1_N0  <=              not N(2) and      N(1) and N(0);
    N2_N1L_N0  <=              N(2) and not N(1) and N(0);
    F <= N3L_N0 or N3L_N2L_N1 or N2L_N1_N0 or N2_N1L_N0;
end prime2_arch;

```

Табл. 4.51 содержит пример использования условных параллельных операторов присваивания. Каждый бит переменной типа STD_LOGIC, например, N(3), сравнивается со знаковыми литералами '1' и '0' и результат возвращается в виде значения типа boolean. Результаты сравнения объединяются в булево выражение, помещенное в каждом операторе между ключевыми словами when и else. В общем случае требуются предложения else; совокупный набор условий в каждом из операторов должен покрывать все возможные комбинации входных сигналов.

Табл. 4.51. Архитектура устройства для обнаружения простых чисел, в которой использованы условные присваивания

```

architecture prime3_arch of prime is
signal N3L_N0, N3L_N2L_N1, N2L_N1_N0, N2_N1L_N0: STD_LOGIC;
begin
    N3L_N0      <= '1' when N(3)='0' and N(0)='1' else '0';
    N3L_N2L_N1 <= '1' when N(3)='0' and N(2)='0' and N(1)='1' else '0';
    N2L_N1_N0  <= '1' when N(2)='0' and N(1)='1' and N(0)='1' else '0';
    N2_N1L_N0  <= '1' when N(2)='1' and N(1)='0' and N(0)='1' else '0';
    F <= N3L_N0 or N3L_N2L_N1 or N2L_N1_N0 or N2_N1L_N0;
end prime3_arch;

```

Параллельный оператор присваивания другого рода – это *избирательное присваивание сигналу его значения* (*selected signal-assignment statement*), синтаксис которого указан в табл. 4.52. Этот оператор вычисляет заданное выражение *expression* и присваивает сигналу с именем *signal-name* значение сигнала *signal-value*, соответствующее той из альтернатив *choices*, значение которой равно *expression*. Альтернативой в каждом предложении when может быть одиночное возможное значение *expression* или список значений, разделенных вертикальной чертой (|). Альтернативы *choices* в данном операторе должны быть взаимно исключающими и в совокупности включать все возможные случаи. В последнем предложении when можно воспользоваться ключевым словом *others* в качестве указания на все значения *expression*, которые еще не были упомянуты.

```

with expression select
  signal-name <= signal-value when choices,
                signal-value when choices,
                ...
                signal-value when choices;

```

Табл. 4.52. Синтаксис избирательного сигнального оператора присваивания в языке VHDL

В архитектуре устройства для обнаружения простых чисел, приведенной в табл. 4.53, использован избирательный сигнальный оператор присваивания. Все *альтернативы*, для которых F равно '1' могли бы быть записаны в одном предложении *when*; в нашем примере они разнесены по нескольким предложениям только с учебной целью. Здесь избирательный сигнальный оператор присваивания как бы считывает запись множества включений функции F.

```

architecture prime4_arch of prime is
begin
  with N select
    F <= '1' when "0001",
         '1' when "0010",
         '1' when "0011" | "0101" | "0111",
         '1' when "1011" | "1101",
         '0' when others;
end prime4_arch;

```

Табл. 4.53. Архитектура устройства для обнаружения простых чисел, в которой используется избирательное присваивание сигналу его значения

Ту же самую архитектуру можно слегка видоизменить, чтобы воспользоваться более удобной числовой интерпретацией N в определении функции. Применяя приведенное ранее преобразование CONV_INTEGER, можно записать *альтернативу* в терминах целых чисел, которые, как это можно видеть из табл. 4.54, являются простыми, что и требовалось. О таком варианте представления структуры можно говорить как о «новедическом» описании, поскольку желаемая функция отображена в нем таким образом, что поведение устройства оказывается совершенно очевидным.

ПОЛНЫЙ ПЕРЕБОР

При условии и избирательном присваивании сигналу его значения требуется перечисление всех возможных условий. В условном сигнальном присваивании заключительной фразой *“else expression”* покрываются опущенные условия. При избирательном сигнальном присваивании все остающиеся условия можно подобрать ключевым словом *“others”* в последнем предложении *when*.

Глядя на табл. 4.53, можно подумать, что вместо слова *“others”* мы могли бы записать девять остающихся 4-битовых комбинаций “0000”, “0100” и т.д. Но это не так! Не забывайте, что STD_LOGIC – это девятизначная система, так что у 4-разрядной величины типа STD_LOGIC_VECTOR в действительности имеется 9^4 возможных значений. Поэтому *“others”* в данном примере на самом деле покрывает 6554 случая!

Табл. 4.54.

Описание устройства для обнаружения простых чисел, носящее поведенческий характер

```
architecture prime5_arch of prime is
begin
  with CONV_INTEGER(N) select
    F <= '1' when 1 | 2 | 3 | 5 | 7 | 11 | 13,
        '0' when others;
end prime5_arch;
```

4.7.8. Элементы поведенческого проектирования

Как видно из последнего примера, иногда параллельным оператором можно непосредственно описать требуемое поведение логической схемы. И это очень хорошо, потому что возможность *поведенческого описания* (*behavioral description*) и выполнение *поведенческого проекта* (*behavioral design*) является главным достоинством языков описания схем вообще и языка VHDL, в частности. Однако для большинства поведенческих описаний нужны некоторые дополнительные элементы языка, рассматриваемые в этом разделе.

Ключевым поведенческим элементом языка VHDL является «процесс». *Процесс* (*process*) — это совокупность «последовательных» операторов (они будут описаны чуть ниже), которые выполняются одновременно с другими параллельными операторами и с другими процессами. С помощью процесса можно задать сложное взаимодействие сигналов и событий таким способом, что при моделировании это взаимодействие реализуется практически за нулевое время в модели, а результатом синтеза становится комбинационная или последовательностная схема, которая выполняет моделируемую операцию непосредственно.

Оператор процесса (*process statement*) в языке VHDL можно использовать повсюду, где возможно употребление параллельного оператора. Оператор процесса вводится ключевым словом *process*; синтаксис этого оператора приведен в табл. 4.55. Оператор *process* пишется внутри некоторой объемлющей архитектуры, поэтому ему доступны все типы, сигналы, константы, функции и процедуры, объявленные в этой архитектуре, а также так или иначе видимые из этой архитектуры. Но можно также определять и локальные типы, переменные, константы, функции и процедуры внутри данного процесса.

Табл. 4.55. Синтаксис оператора *process* в языке VHDL

```
process (signal-name, signal-name, ..., signal-name)
  type declarations
  variable declarations
  constant declarations
  function definitions
  procedure definitions
begin
  sequential-statement
  ...
  sequential-statement
end process;
```

Обратите внимание на то, что внутри процесса можно объявлять только «переменные», но не сигналы. *Переменная (variable)* в языке VHDL отслеживает состояние процесса только внутри него и вне процесса ее не видно. В зависимости от того, как используется переменная, ей в конце концов будет или не будет соответствовать определенный сигнал при физической реализации создаваемой схемы. Синтаксис определения переменной внутри процесса подобен синтаксису объявления сигнала в архитектуре, за исключением того, что используется ключевое слово *variable*:

```
variable variable-names : variable-type;
```

VHDL-процесс всегда либо *выполняется (running process)*, либо *приостановлен (suspended process)*. Перечнем сигналов в определении процесса, который называется *списком чувствительности (sensitivity list)*, задаются условия, когда процесс выполняется. Первоначально процесс остановлен; когда изменяется значение любого из сигналов в его списке чувствительности, исполнение процесса возобновляется, начиная с его первого последовательного оператора, и оно продолжается, пока не будет достигнут конец. Если какой-либо сигнал из списка чувствительности изменяет свое значение в результате исполнения процесса, то процесс выполняется снова. Это продолжается до тех пор, пока запуск процесса не перестанет приводить к изменению значения любого из этих сигналов. При моделировании все это происходит за нулевое время в модели.

Если процесс записан надлежащим образом, то, будучи запущен, он исполняется один или несколько раз и останавливается. Однако существует возможность записать процесс неправильно, который никогда не остановится. Рассмотрим, например, процесс всего с одним последовательным оператором “X <= not X” и списком чувствительности “(X)”. Поскольку на каждом проходе значение X изменяется, процесс будет запущен навсегда, хотя и будет занимать нулевое время в модели. Едва ли это полезно! На практике в моделирующих программах имеются средства защиты, которые обычно обнаруживают подобное нежелательное поведение и прерывают исполнение такого процесса после, скажем, тысячи проходов.

Список чувствительности является необязательным; при моделировании исполнение процесса, у которого нет списка чувствительности, начинается в нулевой момент времени. Одно из применений такого процесса – это генерирование входных колебаний при тестировании (см. табл. 4.65 ниже).

В языке VHDL имеются последовательные операторы нескольких видов. Первый из них – это *последовательный сигнальный оператор присваивания (sequential signal-assignment statement)*; у него тот же самый синтаксис, что и у параллельного аналога (*signal-name <= expression*), но последовательный оператор располагается в теле процесса, а не в теле архитектуры. Аналогичный оператор для переменных – это *оператор присваивания значения переменной (variable-assignment statement)*, синтаксис которого имеет вид: “*variable-name := expression*;”. Заметьте, что в случае переменных используется другой оператор присваивания, а именно: “*=*”.

Ради учебных целей, потоковая архитектура устройства для обнаружения простых чисел из табл. 4.50 воспроизведена в табл. 4.56 как процесс. Обратите внимание, что мы все еще продолжаем совершенствовать архитектуру того же самого объекта `prime`, который первоначально был объявлен в табл. 4.43. В этой новой архитектуре (`prime6_arch`) имеется только один параллельный оператор; этим оператором является процесс. Список чувствительности процесса содержит только массив `N`, представляющий собой первичные сигналы на входах устройства, реализующего желаемую комбинационную логическую функцию. Выходы вентилях И необходимо задать как переменные, а не как сигналы, поскольку внутри процесса определения сигналов не разрешены. В противном случае тело процесса было бы очень похоже на тело исходной архитектуры. На самом деле типичные программные средства синтеза, вероятнее всего, построили бы одну и ту же схему по любому из этих описаний.

Табл. 4.56. Потоковая VHDL-архитектура устройства обнаружения простых чисел, основанная на использовании процесса

```
architecture prime6_arch of prime is
begin
  process(N)
    variable N3L_N0, N3L_N2L_N1, N2L_N1_N0, N2_N1L_N0: STD_LOGIC;
  begin
    N3L_N0      := not N(3)                                and N(0);
    N3L_N2L_N1 := not N(3) and not N(2) and N(1)          ;
    N2L_N1_N0  :=              not N(2) and N(1) and N(0);
    N2_N1L_N0  :=              N(2) and not N(1) and N(0);
    F <= N3L_N0 or N3L_N2L_N1 or N2L_N1_N0 or N2_N1L_N0;
  end process;
end prime6_arch;
```

ПРИЧУДЛИВОЕ ПОВЕДЕНИЕ

Помните, что операторы выполняются внутри процесса *последовательно*. Предположим, что по какой-то причине мы записали последний оператор в табл. 4.56 (присвоение значения сигналу `F`) первым. Тогда мы наблюдали бы довольно причудливое поведение процесса.

При первом запуске процесса моделирующая программа пожаловалась бы, что значения переменных считываются до того, как этим переменным присвоены какие-либо значения. При последующих возобновлениях процесса сигналу `F` присваивалось бы значение, основанное на *предыдущих* значениях переменных, которые сохранялись, пока процесс был приостановлен. Затем переменным присваивались бы новые значения, которые запоминались бы до очередного запуска процесса. Таким образом, значение сигнала на выходе схемы всегда отставало бы от значений входных сигналов на один шаг.

Другие последовательные операторы, помимо простого присваивания, дают возможность творчески подойти к описанию поведения схемы. По-видимому, самый знакомый из них – это оператор *if* (*if statement*), синтаксис которого приведен в табл. 4.57. В первой и простейшей форме этого оператора проверяется булево выражение *boolean-expression* и, если оно имеет значение *true*, то выполняется последовательный оператор *sequential-statement*. Во второй форме добавляется предположение “*else*” с другим последовательным оператором *sequential-statement*, который выполняется, если булево выражение имеет значение *false*.

Табл. 4.57. Синтаксис оператора *if* в языке VHDL

```

if boolean-expression then sequential-statement
end if;

if boolean-expression then sequential-statement
else sequential-statement
end if;

if boolean-expression then sequential-statement
elsif boolean-expression then sequential-statement
:::
elsif boolean-expression then sequential-statement
end if;

if boolean-expression then sequential-statement
elsif boolean-expression then sequential-statement
:::
elsif boolean-expression then sequential-statement
else sequential-statement
end if;

```

Для образования вложенных операторов *if-then-else* в языке VHDL используется специальное ключевое слово *elsif*, которое вводит «середину» предложения. Последовательный оператор *sequential-statement* предложения *elsif* выполняется в том случае, когда булево выражение *boolean-expression* в этом предложении истинно, а все предшествующие булевы выражения *boolean-expressions* оказываются ложными. Последовательный оператор *sequential-statement* заключительного необязательного предложения *else* выполняется только тогда, когда все предыдущие выражения *boolean-expressions* имеют значения *false*.

В табл. 4.58 представлен вариант архитектуры устройства для обнаружения простых чисел, в котором используется оператор *if*. Локальная переменная *NI* введена для того, чтобы отобразить преобразованное значение входного воздействия *N* в виде целого числа; это позволяет оперировать целыми числами при сравнениях в операторе *if*.

Булевы выражения в табл. 4.58 не перекрываются, то есть в любой момент времени значение *true* имеет только одно из них. На самом деле в данном при-

ложении не было необходимости использовать в полном объеме возможности вложенных операторов `if`. С помощью средств синтеза можно было бы построить схему, в которой вычисление логических выражений происходило последовательно, но схема при этом работала бы медленнее. Когда нужно выбирать среди нескольких альтернатив на основании значения только одного сигнала или выражения, обычно более читабельным и дающим лучший результат синтеза является оператор `case` (*case statement*).

Табл. 4.58. Архитектура устройства для обнаружения простых чисел, в которой использован оператор `if`

```
architecture prime7_arch of prime is
begin
  process(N)
    variable NI: INTEGER;
  begin
    NI := CONV_INTEGER(N);
    if NI=1 or NI=2 then F <= '1';
    elsif NI=3 or NI=5 or NI=7 or NI=11 or NI=13 then F <= '1';
    else F <= '0';
    end if;
  end process;
end prime7_arch;
```

Синтаксис оператора `case` представлен в табл. 4.59. В этом операторе вычисляется заданное выражение *expression*, но его значению выбирается одна из альтернатив *choices* и исполняются соответствующие последовательные операторы *sequential-statements*. Заметьте, что в каждом из наборов альтернатив *choices* можно записать один или большее число последовательных операторов. Сами альтернативы *choices* могут иметь форму одного значения или нескольких значений, разделенных вертикальной чертой (`|`). Альтернативы *choices* должны быть взаимно исключающими и содержать все возможные значения типа выражения *expression*; в последней альтернативе *choices* можно воспользоваться ключевым словом `others` для указания всех значений, которые еще не были упомянуты ранее.

```
case expression is
  when choices => sequential-statements
  ...
  when choices => sequential-statements
end case;
```

Табл. 4.59. Синтаксис оператора `case` в языке VHDL

В табл. 4.60 приведена еще одна архитектура устройства для обнаружения простых чисел, на этот раз записанная с использованием оператора `case`. Подобно тому, как это было в варианте с параллельным оператором `select`, оператор `case` позволяет в очень наглядной форме задать желаемое функциональное поведение.

Табл. 4.60. Архитектура устройства для обнаружения простых чисел, в которой использован оператор `case`

```

architecture prime8_arch of prime is
begin
  process(N)
  begin
    case CONV_INTEGER(N) is
      when 1 => F <= '1';
      when 2 => F <= '1';
      when 3 | 5 | 7 | 11 | 13 => F <= '1';
      when others => F <= '0';
    end case;
  end process;
end prime8_arch;

```

Другой важный класс последовательных операторов образуют *операторы loop (loop statements)*. Синтаксис простейшего из них указан в табл. 4.61; в этом примере возникает бесконечный цикл. Хотя в обычных языках, на которых пишется программное обеспечение, бесконечные циклы нежелательны, мы увидим в параграфе 7.12, как можно с большой пользой употребить такой цикл при моделировании работы схемы.

```

loop
  sequential-statement
  ...
  sequential-statement
end loop;

```

Табл. 4.61. Синтаксис основного оператора `loop` в языке VHDL

Более знакомым вариантом цикла является уже рассматривавшийся выше *цикл for (for loop)*, синтаксис которого указан в табл. 4.62. Заметьте, что переменная цикла *identifier* объявляется неявно ее умножением в цикле `for`, и она имеет тот же тип, что и диапазон *range*. Эту переменную можно использовать в последовательных операторах внутри цикла, и посредством ее перебираются все значения диапазона *range* слева направо по мере перехода от одного шага итерации к другому.

```

for identifier in range loop
  sequential-statement
  ...
  sequential-statement
end loop;

```

Табл. 4.62. Синтаксис цикла `for` в языке VHDL

Имеются еще два других полезных последовательных оператора, которые могут исполняться внутри цикла; это *операторы exit (exit statement) и next*

(*next statement*). Оператор *exit*, когда он выполняется, передает управление оператору, непосредственно следующему за концом цикла. С другой стороны, оператор *next* вызывает пропуск всех остальных в цикле операторов и переход к началу следующей итерации данного цикла.

Наше старое доброе устройство для обнаружения простых чисел вновь представлено в табл. 4.63, на этот раз – в виде архитектуры, в которой использован цикл *for*. Замечательно то, что в данном примере дается истинно поведенческое описание: здесь язык VHDL на самом деле используется для определения того, является входное воздействие *N* простым числом или нет. Мы увеличили также размерность массива *N* до 16 разрядов, чтобы подчеркнуть тот факт, что мы способны теперь создавать компактные модели схем, не перечисляя в явном виде сотни простых чисел.

Табл. 4.63. Архитектура устройства для обнаружения простых чисел, в которой использован оператор *for*

```
library IEEE;
use IEEE.std_logic_1164.all;

entity prime9 is
    port ( N: in STD_LOGIC_VECTOR (15 downto 0);
          F: out STD_LOGIC );
end prime9;

architecture prime9_arch of prime9 is
begin
    process(N)
        variable NI: INTEGER;
        variable prime: boolean;
    begin
        NI := CONV_INTEGER(N);
        prime := true;
        if NI=1 or NI=2 then null; -- take care of boundary cases
        else for i in 2 to 253 loop
            if NI mod i = 0 then
                prime := false; exit;
            end if;
        end loop;
        end if;
        if prime then F <= '1'; else F <= '0'; end if;
    end process;
end prime9_arch;
```

Оператор *loop* последнего вида – это цикл *while* (*while loop*), синтаксис которого приведен в табл. 4.64. В такой конструкции булево выражение *boolean-expression* вычисляется перед началом каждой итерации, и цикл выполняется только до тех пор, пока значение этого выражения остается равным *true*.

ПЛОХОЙ ПРОЕКТ

Приведенный в табл. 4.63 вариант структуры – это хороший пример применения цикла `for` и плохой пример проектирования схемы. Хотя VHDL и является мощным языком программирования, использование всех его возможностей в полном объеме при описании устройства может оказаться неэффективным, а само проектируемое устройство – несинтезируемым.

Виновник этого в табл. 4.63 – оператор `mod`. Для этой операции требуется деление целых чисел, тогда как большинство программных средств, ориентированных на язык VHDL, не в состоянии синтезировать схемы деления за исключением особых случаев типа деления на степень 2 (реализуемого сдвигом).

Но даже если бы программные средства могли синтезировать делители, мы не захотели бы задавать структуру устройства для обнаружения простых чисел в таком виде. Приведенное в табл. 4.63 описание предполагает создание комбинационной схемы, поэтому программные средства должны были бы образовать 252 комбинационных делителя, по одному на каждое значение `i`, для «развертывания» цикла `for` и реализации схемы!

Процессы можно использовать для описания и поведения как комбинационных, так и последовательностных схем. В главе 5, в разделах, относящихся к языку VHDL, будет дано много больше примеров описания комбинационных схем. Для описания последовательностных схем понадобятся некоторые дополнительные свойства этого языка; они будут рассмотрены в параграфе 7.12, а примеры проектирования последовательностных схем будут приведены в VHDL-разделах главы 8.

4.7.9. Отсчет времени и моделирование

Ни в одном из примеров, с которыми мы имели дело до сих пор, отсчет времени при работе схемы не моделировался: все происходило в модели за нулевое время. Однако в языке VHDL имеются замечательные средства отображать ход времени, и это является еще одной действительно важной стороной данного языка. В нашей книге мы не станем вдаваться в подробности по этому поводу, но несколько идей здесь все же приведем.

Язык VHDL позволяет с помощью ключевого слова `after` задавать временную задержку в любом сигнальном операторе присваивания, в том числе при последовательном, параллельном, условном и избирательном присваивании. Например, архитектуру вентилей запрета из табл. 4.26 можно было бы записать в следующем виде:

```
Z <= '1' after 4 ns when X='1' and Y='0' else '0' after 3 ns;
```

Этим моделируется вентиль запрета с задержкой, равной 4 нс при переходе выходного сигнала из 0 в 1, и с задержкой всего 3 нс при переходе из 1 в 0. В типичной среде проектирования специализированных ИС такими параметрами задержки паделены написанные на языке VHDL модели всех библиотечных компонентов

нижнего уровня. Эти оценки позволяют моделирующей VHDL-программе приближенно предсказывать временное поведение больших по размеру схем, составленных из таких компонентов.

Другой способ включения отсчета времени предоставляет последовательный оператор *wait* (*wait statement*). Этим оператором можно воспользоваться, чтобы приостановить процесс на заданное время. В табл. 4.65 приведена в качестве примера программа, в которой оператор *wait* использован для того, чтобы сформировать в модели входное воздействие для тестирования вентиля запрета, состоящее в переборе четырех различных комбинаций входных сигналов с шагом 10 нс по оси времени.

Табл. 4.64. Синтаксис цикла

while в языке VHDL

```
while boolean-expression loop
    sequential-statement
    ...
    sequential-statement
end loop;
```

Табл. 4.65. Использование оператора *wait* языка VHDL для генерирования входных воздействий при тестировании

```
entity InhibitTestBench is
end InhibitTestBench;

architecture InhibitTB_arch of InhibitTestBench is
    component Inhibit port (X,Y: in BIT; Z: out BIT); end component;
    signal XT, YT, ZT: BIT;
begin
    U1: Inhibit port map (XT, YT, ZT);
    process
    begin
        XT <= '0'; YT <= '0';
        wait for 10 ns;
        XT <= '0'; YT <= '1';
        wait for 10 ns;
        XT <= '1'; YT <= '0';
        wait for 10 ns;
        XT <= '1'; YT <= '1';
        wait; -- this suspends the process indefinitely
    end process;
end InhibitTB_arch;
```

Когда уже имеется VHDL-программа, корректная с точки зрения синтаксиса, для наблюдения за ее работой можно воспользоваться VHDL-средствами моделирования. Мы не будем подробно рассматривать эти вопросы, но составить общее представление о том, как работает моделирующая программа, полезно.

В момент, когда моделирующая программа начинает работать, *время в модели (simulation time)* равно нулю. В этот момент всем сигналам присваиваются их

значения по умолчанию (от которых работа вашей программы не должна зависеть!). Инициализируются также сигналы и переменные, начальные значения которых объявлены явно (мы не говорим, как это делается). Затем моделирующая программа начинает исполнение всех процессов и параллельных операторов в данном проекте.

Конечно, моделирующая программа не может фактически обеспечить одновременное протекание процессов и исполнение параллельных операторов, но она притворяется, будто делает это, с помощью упорядоченного по времени «списка событий» и «матрицы чувствительности к сигналам». Заметьте, что каждый параллельный оператор эквивалентен одному процессу.

В нулевой момент времени в модели моделирующая программа готова начать исполнение всех процессов и выбирает один из них. Выполняются все последовательные операторы этого процесса, включая циклы, если они предусмотрены. Когда исполнение этого процесса заканчивается, выбирается еще один процесс и так далее, до тех пор пока не будут выполнены все процессы. Этим завершается *цикл моделирования (simulation cycle)*.

Во время исполнения процесса могут возникать новые значения сигналов. Немедленного присвоения сигналам этих значений не происходит; вместо этого новые значения помещаются в *список событий (event list)*, и намечается, что они станут эффективными в определенный момент времени. Если присвоение отнесено к явно заданному времени в модели (например, с задержкой, указанием в предложении *after*), то в списке событий предусматривается выполнение данного действия именно в этот момент времени. В противном случае считается, что присвоение должно произойти «немедленно»; однако в действительности реализация этого события откладывается до момента времени, равного текущему времени в модели плюс один «элементарный сдвиг по времени». Под *элементарным сдвигом по времени (delta delay)* понимается бесконечно короткий отрезок времени, такой что текущее время в модели плюс любое число элементарных сдвигов все еще остается равным тому же самому значению. Этот принцип позволяет, когда необходимо, многократно исполнять процесс в модели за нулевое время.

После того как цикл моделирования завершен, просматривается список событий в поисках одного или нескольких сигналов, которые изменяются в ближайший очередной момент времени. Таким моментом может быть более позднее время, отличающееся от текущего лишь на один элементарный сдвиг, либо очередной момент может определяться реальной задержкой в схеме, и тогда время в модели продвигается до этого момента. В любом случае осуществляется запланированное изменение сигнала. Некоторые из процессов могут быть чувствительны к изменяющимся сигналам согласно сведениям, хранящимся в матрице чувствительности к сигналам. В ней для каждого сигнала указано, у каких процессов этот сигнал имеется в списке чувствительности. (У процесса, эквивалентного параллельному оператору, в список чувствительности записываются все его управляющие сигналы и данные.) Все процессы, чувствительные к только что изменившемуся сигналу, намечаются для исполнения в очередном цикле моделирования, к которому моделирующая программа и приступает.

Двухфазная работа моделирующей программы на каждом цикле моделирования, сопровождающаяся просмотром списка событий и составлением очередного расписания присвоения сигналам их новых значений, продолжается неограниченно долго, до тех пор, пока список событий не будет исчерпан. На этом моделирование завершается.

Механизм списка событий позволяет имитировать исполнение параллельных процессов, несмотря на то, что работа моделирующей программы представляет собой один поток операций, выполняемых на единственном компьютере. А механизм элементарных сдвигов по времени обеспечивает правильность работы моделирующей программы, хотя может потребоваться многократное исполнение процесса или набора процессов в моменты времени, разделенные элементарными сдвигами, прежде чем изменяющиеся сигналы приобретут свои установившиеся значения. Этот механизм используется также для обнаружения выходящих из-под контроля процессов (типа $X \leq \text{not } X$); если в результате выполнения тысячи циклов моделирования на тысяче элементарных сдвигов по времени не происходит продвижение времени в модели на сколько-нибудь «реальную» величину, то, вероятнее всего, что-то не так.

4.7.10. Синтез

Как мы упоминали в начале этого параграфа, язык VHDL первоначально предназначался для описания логических схем и моделирования и лишь позднее был приспособлен для синтеза. В этом языке много конструктивов и излишеств, которые не могут быть синтезированы. Однако представленные в этом параграфе сокращенные версии языка и стили написания программ в общем случае обеспечивают синтезирование большинством программных средств.

Но все же от того, как вы напишете программу, сильно зависит качество схемы, которую вы получите в результате синтеза. Приведем несколько примеров.

- Конструкции с «последовательным» управлением типа `if-elsif-elsif-else` могут приводить к цепочке последовательно включенных логических схем, проверяющих соответствующие условия. Если условия являются взаимно исключающими, то лучше использовать операторы `case` или `select`.
- Цикл в процессе обычно «разворачивается» в множество копий той или иной комбинационной логической схемы, чтобы обеспечить исполнение этого оператора. Если вы хотите, чтобы эта комбинационная логическая схема была только в одном экземпляре и использовалась последовательно шаг за шагом, то вы должны спроектировать последовательностную схему; о таких схемах речь пойдет в последующих главах.
- Если вы ошибетесь в условном операторе в процессе, не указав выход для какой-либо комбинации входных сигналов, то это приведет к созданию компилятором «защелки» для удержания старого значения сигнала, который в противном случае мог бы измениться. В общем случае образование таких защелок нежелательно.

Кроме того, в зависимости от реализации программных средств могут оказаться несинтезируемыми и другие элементы и структуры языка. Вам, безусловно, сле-

дует обратиться к документации, чтобы узнать, что запрещено, что разрешено и что рекомендуется в вашем конкретном случае.

В предвидимом будущем проектировщикам, использующим программные средства синтеза, для получения хороших результатов придется обратить довольно пристальное внимание на стиль написания своих программ. А в настоящее время критерий «хорошего стиля программирования» в определенной степени зависит как от средств синтеза, так и от технологии микросхем, в которых нужно разместить результат синтеза. Хотя примеры, которые будут приведены в оставшейся части этой книги, будут синтаксически и семантически корректными, они дадут лишь поверхностное представление о методах написания программ на языках описания схем, которые применяются в больших проектах. Искусство и практика проектирования больших устройств с использованием языков описания схем все еще находятся в состоянии интенсивного развития.

Обзор литературы

Историческое описание того, как Буль развивал «науку Логике», появилось в 1972 году в книге Голдстейна *Компьютер от Паскаля до фон Неймана* (Herman H. Goldstine. *The Computer from Pascal to von Neumann*. Princeton University Press, 1972). Шеннон показал, как можно было бы применить работы Буля к логическим схемам ("A Symbolic Analysis of Relay and Switching Circuits". Trans. AIEE, Vol. 57, 1938, pp. 713–723; рус. пер. в книге: К. Шеннон. *Труды по теории информации и кибернетике*. – М.: ИЛ, 1963).

Хотя основой алгебры переключений стала двузначная булева алгебра, возможна логика и с большим числом значений. Существуют булевы алгебры с 2^n значениями для любого целого n ; см., например, *Дискретные математические структуры и их приложения* Стоуна (Harold S. Stone. *Discrete Mathematical Structures and Their Applications*. SRA, 1973). Формально такие алгебры могут быть определены с помощью так называемых *постулатов Хантингтона*, введенных Хантингтоном (E. V. Huntington) в 1907 году; см., например, *Цифровое проектирование* Маю (M. Morris Mano. *Digital Design*. Prentice Hall, 1984). Математическое развитие булевой алгебры на основе современного набора постулатов появилось в 1970 году в книге Биркгофа и Барти *Современная прикладная алгебра* (G. Birkhoff and T. C. Bartee. *Modern Applied Algebra*. McGraw-Hill, 1970; рус. пер.: Г. Биркгоф, Т. Барти. *Современная прикладная алгебра*. – М.: Мир, 1976). В нашем «прямом» развитии алгебры переключений в инженерном стиле мы следуем книгам Мак-Класки *Введение в теорию переключательных схем* и *Принципы логического проектирования* (Edward J. McCluskey. *Introduction to the Theory of Switching Circuits*. McGraw-Hill, 1965; *Logic Design Principles*. Prentice Hall, 1986).

Теорему о простых импликантах доказал Куин (W. V. Quine. "The Problem of Simplifying Truth Functions". Am. Math. Monthly, Vol. 59, No. 8, 1952, pp. 521–531). На самом деле можно доказать более общую теорему о простых импликантах, согласно которой существует по меньшей мере одна минимальная сумма, то есть сумма простых импликант, даже в случае, когда ограничение на число литералов в определении «минимальный» снято.

Графический метод упрощения булевых функций был предложен Вейчем (E. W. Veitch. "A Chart Method for Simplifying Boolean Functions". Proc. ACM, May 1952, pp. 127–133). Его диаграмма Вейча, приведенная на рис. 4.53, фактически представляет собой заново изобретенную карту, придуманную английским археологом Маркандом (A. Marquand. "On Logical Diagrams for n Terms". Philosophical Magazine XII, 1881, pp. 266–270). В диаграмме Вейча и карте Марканда используется «естественный» порядок двоичной нумерации строк и столбцов, в результате чего некоторые смежные строки и столбцы отличаются более чем в одном значении и термы-произведения не всегда покрывают смежные клетки. Карно показал, как решить эту задачу (M. Karnaugh. "A Map Method for Synthesis of Combinational Logic Circuits". Trans. AIEE, Comm. and Electron., Vol. 72, Part 1, November 1953, pp. 593–599). С другой стороны, Клир в своей книге *Введение в методологию переключательных схем* (George J. Klir. *Methodology of Switching Circuits*. Van Nostrand, 1972) утверждает, что для минимизации логических функций двоичный порядок счета так же хорош, как и порядок в карте Карно, а может быть даже и лучше.

Рис. 4.53. Диаграмма Вейча или карта Марканда для 4-х переменных

Y Z \ W X	00	01	10	11
00	0	4	8	12
01	1	5	9	13
10	2	6	10	14
11	3	7	11	15

На данном этапе аргументы в пользу карт Карно против диаграмм Вейча, конечно, неуместны, поскольку никто больше не вычерчивает никаких карт для минимизации логических схем. Вместо этого мы пользуемся компьютерными программами, реализующими алгоритмы логической минимизации. Первый такой алгоритм описан Куином (W. V. Quine. "A Way to Simplify Truth Functions". Am. Math. Monthly, Vol. 62, No. 9, 1955, pp. 627–631) и усовершенствован Мак-Класки (E. J. McCluskey. "Minimization of Boolean Functions". Bell Sys. Tech. J., Vol. 35, No. 5, November 1956, pp. 1417–1444). Алгоритм Куина–Мак-Класки исчерпывающим образом описан в упомянутых выше книгах Мак-Класки.

В своей книге 1965 года Мак-Класки привел также итеративно-консенсусный алгоритм нахождения простых импликант и доказал, что он работает. Отправным моментом в этом алгоритме является логическое выражение вида «сумма произведений» или, что эквивалентно, список кубов. Термы-произведения не обязательно должны быть минтермами или простыми импликантами, но могут быть чем-то средним между ними. Другими словами, в случае функции n переменных, кубы в списке могут быть любой размерности, или всех размерностей, от 0 до n . Начиная с этого списка кубов, алгоритм генерирует список всех кубов, являющихся простыми импликантами функции, не прибегая к составлению полного списка минтермов.

Итеративно-консенсусный алгоритм впервые был опубликован Моттом (T.H. MOTT, Jr. "Determination of the Irredundant Normal Forms of a Truth Function by

Iterated Consensus of the Prime Implicants". IRE Trans. Electron. Computers, Vol. EC-9, No. 2, 1960, pp. 245–252). Затем Тизоном был предложен обобщенный консенсусный алгоритм (Pierre Tison. "Generalization of Consensus Theory and Application to the Minimization of Boolean Functions". IEEE Trans. Electron. Computers, Vol. EC-16, No. 4, 1967, pp. 446–456). Все эти алгоритмы описаны в книге Даунса *Логическое проектирование на Паскале* (Thomas Downs. *Logic Design with Pascal*. Van Nostrand Reinhold, 1988).

Как мы уже объясняли в разделе 4.4.4, у некоторых логических функций число простых импликант бывает огромным, поэтому найти их все или выбрать минимальное покрытие на регулярной основе оказывается трудно выполнимым или невозможным. Существуют, однако, эффективные эвристические методы решения, дающие результаты, близкие к оптимальным. Один из таких методов – Espresso II – описан в книге Брайтона и др. *Алгоритмы логической минимизации для синтеза СВЧ* (R. K. Brayton, C. McMullen, G. D. Hachtel, and A. Sangiovanni-Vincentelli. *Logic Minimization Algorithms for VLSI Synthesis*. Kluwer Academic Publishers, 1984). Более поздние алгоритмы Espresso-MV и Espresso-EXACT описаны Руделлом и Санджованни-Винченцелли (R. L. Rudell and A. Sangiovanni-Vincentelli. "Multiple-Valued Minimization for PLA Optimization". IEEE Trans. CAD, Vol. CAD-6, No. 5, 1987, pp. 727–750).

В этой главе мы рассмотрели метод нахождения статических источников опасности в двухуровневых схемах И–ИЛИ и ИЛИ–И по картам Карно. Задача поиска источников опасности может возникнуть в отношении любой комбинационной схемы. В своих книгах 1965 года и 1986 года Мак-Класки вводит понятия 0- и 1-совокупностей литералов (0-sets, 1-sets) для конкретной схемы и показывает, как с их помощью можно находить статические источники опасности; там же вводятся P- и S-совокупности (P-sets, S-sets), позволяющие находить динамические источники опасности.

Существует много более глубоких и разнообразных аспектов теории переключений, которые в этой книге опущены, но в деталях разбираются в других книгах и публикациях. Хорошим началом в изучении классической теории переключений может служить книга Кохави *Теория переключаемых схем и конечных автоматов* (Zvi Kohavi. *Switching and Finite Automata Theory*, second edition. McGraw-Hill, 1978), которая включает теорию множеств, симметричные цепи, функциональную декомпозицию, пороговую логику, обнаружение неисправностей и активизацию тракта проверки логической схемы. Другой областью, представляющей большой академический (но незначительный коммерческий) интерес, является недвоичная многозначная логика (multiple-valued logic), в которой каждый сигнал может принимать более двух значений. Хорошее введение в многозначную логику содержится в книге Мак-Класки 1986 года, где приводятся все «за» и «против» и объясняется, почему она с трудом находит коммерческое применение.

Годами я бился в поисках доступного и надежного источника сведений о языке ABEL и, наконец, нашел: приложение А в книге Пеллерина и Холли *Цифровое проектирование на языке ABEL* (David Pellerin and Michael Holley. *Digital Design Using ABEL*. Prentice Hall, 1994). Данный источник следует считать, по-видимому, самым авторитетным: именно Пеллерин и Холли придумали этот язык и напи-

сали исходный код компилятора! В соавторстве с Тейлором (Douglas Taylor) Пеллерин написал также отличное введение в язык VHDL, озаглавленное *C VHDL теперь все просто!* (*VHDL Made Easy!* Prentice Hall, 1997).

VHDL – это “Very Huge Design Language” («гигантский язык проектирования»), и в нашей книге нашла отражение только сокращенная версия этого языка с основными его особенностями и возможностями. В языке VHDL имеется много опций (например, метки операторов), которыми мы не пользуемся, но информация о них содержится в учебниках и справочниках по языку VHDL. Одной из лучших книг является *Справочник проектировщика по языку VHDL* (*The Designer's Guide to VHDL*, Morgan Kaufmann, 1996). В этой книге довольно большого объема (700 страниц) язык представлен в хорошо систематизированной форме; в очень полезном и информативном приложении исчерпывающим образом описан синтаксис языков VHDL-87 и VHDL-93. Мы заимствовали из этой книги подмножество языка, совместимое с обоими стандартами.

Несколько более кратким введением в язык VHDL является книга Сйохолма и Линдха *VHDL для разработчиков* (Stefan Sjöholm and Lemnart Lindh. *VHDL for Designers*, Prentice Hall, 1997). Эту книгу можно рекомендовать читателям, поскольку в ней сделан упор на практические аспекты проектирования, включая синтез и тестирование. Другая практическая книга – это *VHDL для программируемой логики* Скахилла из фирмы Cypress Semiconductor (Kevin Skahill. *VHDL for Programmable Logic*, Addison-Wesley, 1996).

Все примеры на языках ABEL и VHDL, приведенные в данной главе и повсюду в этой книге, были откомпилированы и в большинстве случаев протестированы средствами моделирования с помощью студенческой версии программного обеспечения Foundation 1.5 фирмы Xilinx, Inc. (San Jose, CA 95124, www.xilinx.com). В программном обеспечении Foundation объединены схематический редактор, текстовый редактор языка описания схем, компиляторы языков ABEL, VHDL и Verilog и моделирующая программа фирмы Aldec, Inc. (Henderson, NV 89014, www.aldec.com), а также собственные специализированные средства фирмы Xilinx для проектирования на основе ИС типа CPLD и FPGA и их программирования. Это программное обеспечение содержит также отличное справочное руководство по языкам ABEL и VHDL. Программное обеспечение Foundation прилагается к этой книге на компакт-диске.

VHDL-пакеты стандартов IEEE являются важной составной частью в любой среде проектирования на языке VHDL. В принципе, перечень определений типов и функций имеется в качестве приложения в учебниках по языку VHDL, но если вы проявите любознательность, то найдете исчерпывающие сведения в библиотеке любой VHDL-системы проектирования, в том числе в программном обеспечении Foundation.

При рассмотрении проверочных векторов языка ABEL мы затронули вопрос о тестировании устройств. Существует огромная устоявшаяся литература по тестированию цифровых устройств; хорошей отправной точкой в изучении этих вопросов служит книга Мак-Клаеки 1986 года. Генерирование набора проверочных векторов, которые полностью тестировали бы большую схему типа ПЛУ, является задачей, которую лучше всего поручить программным средствам. По меньшей мере у одной из фирм весь бизнес сосредоточен на выпуске программ, автоматически вырабатывающих проверочные векторы для тестирования ПЛИУ: это – ACUGEN Software, Inc. (Nashua, NH 03063, www.acugen.com).

Упражнения

- 4.1. Составьте из переменных NERD (тупица), DESIGNER (разработчик), FAILURE (терпеть неудачу) и STUDIED (быть обученным) логическое выражение, равное 1 в случае преуспевающего разработчика, который никогда ничему не учился, и тупицы, который все время учится.
- 4.2. Докажите методом полной индукции теоремы T2–T5.
- 4.3. Докажите методом полной индукции теоремы T1'–T3' и T5'.
- 4.4. Докажите методом полной индукции теоремы T6–T9.
- 4.5. Согласно теореме Де Моргана, дополнение логического выражения $X + Y \cdot Z$ равно $X' \cdot Y' + Z'$. Однако оба выражения равны 1 при $XYZ = 110$. Как это может быть, чтобы выражение и его дополнение равнялись 1 при одной и той же комбинации переменных? Что здесь неправильно?
- 4.6. Используя теоремы алгебры переключений, упростите каждую из следующих логических функций:
- $F = W \cdot X \cdot Y \cdot Z + (W \cdot X \cdot Y \cdot Z' + W \cdot X' \cdot Y \cdot Z + W' \cdot X \cdot Y \cdot Z + W \cdot X \cdot Y' \cdot Z)$
 - $F = A \cdot B + A \cdot B \cdot C' \cdot D + A \cdot B \cdot D \cdot E' + A \cdot B \cdot C' \cdot E + C' \cdot D \cdot E$
 - $F = M \cdot N \cdot O + Q' \cdot P' \cdot N' + P \cdot R \cdot M + Q' \cdot O \cdot M \cdot P' + M \cdot R$
- 4.7. Составьте таблицу истинности для каждой из следующих логических функций
- $F = X' \cdot Y + X' \cdot Y' \cdot Z$
 - $F = W' \cdot X + Y' \cdot Z' + X' \cdot Z$
 - $F = W + X' \cdot (Y' + Z)$
 - $F = A \cdot B + B' \cdot C + C' \cdot D + D' \cdot A$
 - $F = V \cdot W + X' \cdot Y' \cdot Z$
 - $F = (A' + B' + C \cdot D) \cdot (B + C' + D' \cdot E')$
 - $F = (W \cdot X)' \cdot (Y' + Z')'$
 - $F = (((F + B)' + C')' + D)'$
 - $F = (A' + B + C) \cdot (A + B' + D') \cdot (B + C' + D') \cdot (A + B + C + D)$
- 4.8. Составьте таблицу истинности для каждой из следующих логических функций
- $F = X' \cdot Y' \cdot Z' + X \cdot Y \cdot Z + X \cdot Y' \cdot Z$
 - $F = M' \cdot N' + M \cdot P + N' \cdot P$
 - $F = A \cdot B + A \cdot B' \cdot C' + A' \cdot B \cdot C$
 - $F = A' \cdot B \cdot (C \cdot B \cdot A' + B \cdot C')$
 - $F = X \cdot Y \cdot (X' \cdot Y \cdot Z + X \cdot Y' \cdot Z + X \cdot Y \cdot Z' + X' \cdot Y' \cdot Z)$
 - $F = M \cdot N + M' \cdot N' \cdot P'$
 - $F = (A + A') \cdot B + B \cdot A \cdot C' + C \cdot (A + B') \cdot (A' + B)$
 - $F = X \cdot Y' + Y \cdot Z + Z' \cdot X$
- 4.9. Запишите каноническую сумму и каноническое произведение для каждой из следующих логических функций:
- $F = \sum_{X,Y} (1, 2)$
 - $F = \prod_{A,B} (0, 1, 2)$
 - $F = \sum_{A,B,C} (2, 4, 6, 7)$
 - $F = \prod_{W,X,Y} (0, 1, 3, 4, 5)$
 - $F = X + Y' \cdot Z'$
 - $F = V' + (W' \cdot X)'$

4.10. Запишите каноническую сумму и каноническое произведение для каждой из следующих логических функций:

$$(a) F = \sum_{X,Y,Z}(0, 3)$$

$$(b) F = \prod_{A,B,C}(1, 2, 4)$$

$$(c) F = \sum_{A,B,C,D}(1, 2, 5, 6)$$

$$(d) F = \prod_{M,N,P}(0, 1, 3, 6, 7)$$

$$(e) F = X' + Y \cdot Z' + Y' \cdot Z$$

$$(f) F = A' \cdot B + B' \cdot C + A$$

4.11. Если каноническая сумма логической функции n переменных является также и минимальной суммой, то сколько литералов содержится в каждом терм-произведении этой суммы? Могут ли существовать в этом случае какие-либо другие минимальные суммы?

4.12. Назовите две причины, по которым стоимость инверторов не учитывается в определении «минимальный» применительно к минимизации логических схем.

4.13. С помощью карт Карно найдите минимальные выражения вида «сумма произведений» для следующих логических функций. На каждой карте отметить особенные клетки, содержащие 1.

$$(a) F = \sum_{X,Y,Z}(1, 3, 5, 6, 7)$$

$$(b) F = \sum_{W,X,Y,Z}(1, 4, 5, 6, 7, 9, 14, 15)$$

$$(c) F = \prod_{W,X,Y}(0, 1, 3, 4, 5)$$

$$(d) F = \sum_{W,X,Y,Z}(0, 2, 5, 7, 8, 10, 13, 15)$$

$$(e) F = \prod_{A,B,C,D}(1, 7, 9, 13, 15)$$

$$(f) F = \sum_{A,B,C,D}(1, 4, 5, 7, 12, 14, 15)$$

4.14. Найдите минимальные выражения вида «произведение сумм» для каждой функции из упражнения 4.13 методом, изложенным в разделе 4.3.6.

4.15. Найдите минимальные выражения вида «произведение сумм» для функций на каждом из следующих рисунков и сравните их стоимость со стоимостью найденных ранее минимальных выражений вида «сумма произведений»: (a) рис. 4.27; (b) рис. 4.29; (c) рис. 4.33.

4.16. С помощью карт Карно найдите минимальные выражения вида «сумма произведений» для следующих логических функций. На каждой карте отметить особенные клетки, содержащие 1.

$$(a) F = \sum_{A,B,C}(0, 1, 2, 4)$$

$$(b) F = \sum_{W,X,Y,Z}(1, 4, 5, 6, 11, 12, 13, 14)$$

$$(c) F = \prod_{A,B,C}(1, 2, 6, 7)$$

$$(d) F = \sum_{W,X,Y,Z}(0, 1, 2, 3, 7, 8, 10, 11, 15)$$

$$(e) F = \sum_{W,X,Y,Z}(1, 2, 4, 7, 8, 11, 13, 14)$$

$$(f) F = \prod_{A,B,C,D}(1, 3, 4, 5, 6, 7, 9, 12, 13, 14)$$

4.17. Найдите минимальные выражения вида «произведение сумм» для каждой функции из упражнения 4.16 методом, изложенным в разделе 4.3.6.

4.18. Найдите полную сумму для логических функций (d) и (e) из упражнения 4.16.

4.19. С помощью карт Карно найдите минимальные выражения вида «сумма произведений» для следующих логических функций. На каждой карте отметить особенные клетки, содержащие 1.

$$(a) F = \sum_{W,X,Y,Z}(0, 1, 3, 5, 14) + d(8, 15)$$

$$(b) F = \sum_{W,X,Y,Z}(0, 1, 2, 8, 11) + d(3, 9, 15)$$

$$(c) F = \sum_{A,B,C,D}(1, 5, 9, 14, 15) + d(11)$$

$$(d) F = \sum_{A,B,C,D}(1, 5, 6, 7, 9, 13) + d(4, 15)$$

- 4.20. Повторите упражнение 4.19 и найдите минимальные выражения вида «произведение сумм» для каждой логической функции.
- 4.21. Для каждой логической функции из двух предыдущих упражнений проверьте равенство минимальных выражений вида «сумма произведений» и вида «произведение сумм». Сравните также стоимость реализации выражений того и другого вида.
- 4.22. Для каждого из следующих логических выражений найдите все статические источники опасности в соответствующей двухуровневой схеме И-ИЛИ или ИЛИ-И и постройте свободную от источников опасности схему, реализующую ту же самую логическую функцию:
- (a) $F = W \cdot X + W \cdot Y'$
 - (b) $F = W \cdot X' \cdot Y' + X \cdot Y' \cdot Z + X \cdot Y$
 - (c) $F = W' \cdot Y + X' \cdot Y' + W \cdot X \cdot Z$
 - (d) $F = W' \cdot X + Y' \cdot Z + W \cdot X \cdot Y \cdot Z + W \cdot X' \cdot Y \cdot Z'$
 - (e) $F = (W + X + Y) \cdot (X' + Z')$
 - (f) $F = (W + Y' + Z') \cdot (W + X' + Z') \cdot (X' + Y + Z)$
 - (g) $F = (W + Y + Z') \cdot (W + X' + Y + Z) \cdot (X' + Y') \cdot (X + Z)$
- 4.23. Запишите набор проверочных векторов на языке ABEL для устройства из табл. 4.17, обнаруживающего простые числа.
- 4.24. Напишите VHDL-программу (объект и структуру) для изображенной на рис. 4.19 схемы охранной сигнализации в стиле структурного проектирования.
- 4.25. Сделайте то же самое, что и в упражнении 4.24, в стиле потокового проектирования.
- 4.26. Сделайте то же самое, что и в упражнении 4.24, в стиле поведенческого проектирования.
- 4.27. Напишите структурную VHDL-программу, соответствующую изображенной на рис. 5.17 логической схеме, построенной на основе вентилях И-НЕ.

Задачи

- 4.28. Придумайте не тривиально выглядящую логическую схему с обратной связью, но такую, у которой выходной сигнал зависит только от текущего значения входного сигнала.
- 4.29. Докажите теорему объединения T10, не используя полной индукции, но предполагая, что теоремы T1–T9 и T1'–T9' верны.
- 4.30. Покажите, что теорема объединения T10 является всего лишь частным случаем согласованности (теорема T11) с учетом поглощения (теорема T9).
- 4.31. Докажите, что $(X + Y') \cdot Y = X \cdot Y$, не используя метод полной индукции, но предполагая, что теоремы T1–T11 и T1'–T11' верны.
- 4.32. Докажите, что $(X + Y) \cdot (X' + Z) = X \cdot Z + X' \cdot Y$, не используя метод полной индукции, но предполагая, что теоремы T1–T11 и T1'–T11' верны.

- 4.33. Покажите, что n -входовой вентиль И можно заменить $n - 1$ двухвходовыми вентилями И. Можно ли утверждать то же самое относительно вентилях И-НЕ? Объясните свой ответ.
- 4.34. Сколько существует физически различных способов реализации логического выражения $V \cdot W \cdot X \cdot Y \cdot Z$ на четырех двухвходовых вентилях И (типа 74х08)? Объясните свой ответ.
- 4.35. Докажите по правилам алгебры переключений, что в результате объединения двух входов $(n + 1)$ -входowego вентиля И или ИЛИ получается функциональный эквивалент n -входowego вентиля.
- 4.36. Докажите по индукции теоремы Де Моргана ($T13$ и $T13'$).
- 4.37. Какое из условных обозначений ТТЛ-вентиля ИЛИ-НЕ – на рис. 4.4(с) или (d) – точнее отражает его внутреннее устройство? Почему?
- 4.38. Используя теоремы алгебры переключений, преобразуйте следующее выражение так, чтобы в нем было возможно меньшее число инверсий (допускается инвертирование того, что заключено в скобки):

$$B' \cdot C + A \cdot C \cdot D' + A' \cdot C + E \cdot B' + E \cdot (A + C) \cdot (A' + D').$$
- 4.39. Докажите справедливость или ошибочность следующих утверждений:
 (а) Пусть A и B – логические переменные. Тогда из равенств $A \cdot B = 0$ и $A + B = 1$ следует, что $A = B'$.
 (б) Пусть X и Y – логические выражения. Тогда из равенств $X \cdot Y = 0$ и $X + Y = 1$ следует, что $X = Y'$.
- 4.40. Докажите теоремы расширения Шеннона. (Указание: Не пугайтесь, это легко.)
- 4.41. Теоремы расширения Шеннона можно обобщить, распространив их на случай не одной, а i переменных; из этого следует, что логическую функцию можно представить в виде суммы или произведения 2^i членов. Сформулируйте обобщенные теоремы расширения Шеннона (*generalized Shannon-expansion theorems*).
- 4.42. Покажите, как с помощью обобщенных теорем расширения Шеннона прийти к представлению функций в виде канонической суммы или канонического произведения.
- 4.43. Вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ (*Exclusive OR gate; XOR*) – это двухвходовая логическая схема, на выходе которой логическая 1 появляется тогда и только тогда, когда точно один из сигналов на ее входах равен 1. Составьте таблицу истинности для логической функции ИСКЛЮЧАЮЩЕЕ ИЛИ, запишите ее представление в виде суммы произведений и нарисуйте соответствующую схему вида И-ИЛИ.
- 4.44. Какую функцию, с точки зрения алгебры переключений, выполняет двухвходовой вентиль XOR, у которого входы соединены вместе? В чем состоит возможное различие между выходными сигналами реального вентиля XOR и идеального элемента, реализующего ту же логическую функцию?
- 4.45. Спроектировав и изготовив цифровую систему на базе ИС малой степени интеграции, разработчик обнаружил, что необходимо добавить еще один инвер-

тор. В системе имеются следующие резервные логические схемы. 3-входовой вентиль ИЛИ, 2-входовой вентиль И и 2-входовой вентиль XOR. Как следует осуществить нивертирование, не добавляя еще одну микросхему?

- 4.46. Множество логических элементов различного типа называют *полным* (*complete set*), если с их помощью можно реализовать любую логическую функцию. Например, двухвходовые вентили И, двухвходовые вентили ИЛИ и инверторы образуют полное множество, поскольку любую логическую функцию можно представить в виде суммы произведений переменных и их дополнений, а функции И и ИЛИ с любым числом переменных можно реализовать на двухвходовых вентилях. Является ли полным множество, состоящее из одного логического элемента И-НЕ? Докажите ваше утверждение.
- 4.47. Является ли полным множество, состоящее из одного логического элемента ИЛИ-НЕ? Докажите ваше утверждение.
- 4.48. Является ли полным множество, состоящее из одного логического элемента XOR?
- 4.49. Дайте определение двухвходового вентиля, отличающегося от схем И-НЕ, ИЛИ-НЕ и XOR, который сам по себе образует полное множество, если допускается наличие постоянных входных сигналов, равных 0 и 1. Приведите соответствующее доказательство.
- 4.50. Некоторые думают, что существует четыре основных логических функции: И, ИЛИ, НЕ и BUT. На рис. X4.50 приведено возможное условное обозначение вентиля BUT (*BUT gate*) с 4-мя входами и 2-мя выходами. Придумайте полезную, нетривиальную функцию, которую мог бы выполнять вентиль BUT. Эта функция должна быть так или иначе связана со смысловым значением названия схемы (англ. "but" означает «но»). Следует иметь в виду, что симметричность условного обозначения предполагает симметрию функции по отношению ко входам А и В в верхней и нижней частях схемы и по отношению к выходам схемы Z1 и Z2. Опишите словами вашу функцию BUT и составьте для нее таблицу истинности.



Рис. X4.50

- 4.51. Запишите логические выражения для выходных сигналов Z1 и Z2 вентиля BUT, разработанного вами в предыдущей задаче, и нарисуйте соответствующую принципиальную схему, состоящую из вентилях И и ИЛИ и инверторов.
- 4.52. Большинство учащихся не испытывают никаких затруднений при преобразовании логического выражения согласно теореме Т8 путем «разнесения множителя по слагаемым», однако многих ставит в тупик преобразование логического выражения согласно теореме Т8' путем «разнесения слагаемого по сомножителям». Как можно воспользоваться двойственностью, чтобы преодолеть такое затруднение?

- 4.53. Сколько существует различных логических функций n переменных?
- 4.54. Сколько существует различных логических функций $F(X, Y)$ двух переменных? Запишите упрощенные логические выражения для каждой из них.
- 4.55. Логической функцией, двойственной по отношению к самой себе (*self-dual logic function*), является такая функция F , что $F = F^D$. Какие из следующих функций являются двойственными по отношению к самим себе? [Символом $\oplus$ обозначена операция ХОИ (ИСКЛЮЧАЮЩЕЕ ИЛИ).]
- | | |
|---|---|
| (a) $F = X$ | (b) $F = \sum_{xyz} (0, 3, 5, 6)$ |
| (c) $F = X \cdot Y' + X' \cdot Y$ | (d) $F = W \cdot (X \oplus Y \oplus Z) + W' \cdot (X \oplus Y \oplus Z)'$ |
| (e) Функция F семи переменных, такая что $F = 1$ в том и только в том случае, когда 4 или большее число переменных равны 1. | (f) Функция F десяти переменных, такая что $F = 1$ в том и только в том случае, когда 5 или большее число переменных равны 1. |
- 4.56. Сколько существует логических функций n переменных, двойственных по отношению к самим себе? (Указание: Рассмотрите структуру таблицы истинности какой-нибудь функции, двойственной по отношению к самой себе.)
- 4.57. Докажите, что любая логическая функция n переменных $F(X_1, \dots, X_n)$, которую можно записать в виде $F = X_1 \cdot G(X_2, \dots, X_n) + X_1' \cdot G^D(X_2, \dots, X_n)$, является двойственной по отношению к самой себе.
- 4.58. Сравните по быстродействию схемы на рис. 4.24(а), (с) и (д), предположив, что задержка распространения инвертирующего вентиля равна 5 нс, а задержка распространения неинвертирующего вентиля равна 8 нс.
- 4.59. Найдите минимальные выражения вида «произведение сумм» для логических функций, представленных на рис. 4.27 и 4.29.
- 4.60. Покажите, пользуясь правилами алгебры переключений, что логические функции, полученные в задаче 4.59, равны функциям И–ИЛИ, приведенным на рис. 4.27 и 4.29.
- 4.61. Проверьте, являются ли минимальными выражения вида «произведение сумм», полученные путем «разнесения слагаемого по сомножителям» из минимальных сумм, приведенных на рис. 4.27 и 4.29.
- 4.62. Докажите справедливость правила объединения на карте Карно 2^n клеток, содержащих 1, исходя из аксиом и теорем алгебры переключений.
- 4.63. *Неприводимая сумма (irredundant sum)* для логической функции F — это такая сумма простых импликант F , что при удалении любой из простых импликант эта сумма уже не равняется F . Согласно этому определению, неприводимая сумма выглядит очень похожей на минимальную сумму, однако неприводимая сумма не обязательно является минимальной. Например, в минимальной сумме функции, приведенной на рис. 4.35, только три термина-произведения, тогда как существует неприводимая сумма с четырьмя терминами-произведениями. Найдите эту неприводимую сумму и нарисуйте карту для данной функции, обведя только простые импликанты, входящие в неприводимую сумму.

- 4.64. Найдите другую логическую функцию в параграфе 4.3, у которой есть одна или большее число неприводимых сумм, не являющихся минимальными, начертите карту этой функции и обведите только те простые импликанты, которые входят в неприводимую сумму.
- 4.65. Начертите карту Карно и присвойте имена переменных входам схемы И-ХОР, изображенной на рис. Х4.65 так, чтобы сигнал на ее выходе равнялся $F = \sum_{w,x,y,z}(6, 7, 12, 13)$. Будьте внимательны: выходной вентиль является двухвходовой логической схемой ИСКЛЮЧАЮЩЕЕ-ИЛИ, а не схемой ИЛИ.

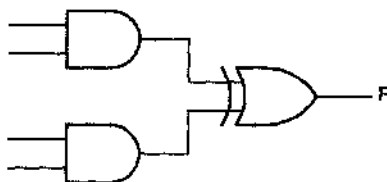


Рис. Х4.65

- 4.66. На входы 3-разрядного «сравнивающего устройства» поступают два 3-разрядных двоичных числа: $P = P_2P_1P_0$ и $Q = Q_2Q_1Q_0$. Составьте схему, реализующую минимальную сумму произведений, согласно которой 1 появляется на выходе тогда и только тогда, когда $P > Q$.
- 4.67. Найдите минимальные выражения вида «сумма произведений» для схемы с тремя выходами, реализующей функции: $F = \sum_{x,y,z}(0, 1, 2)$, $G = \sum_{x,y,z}(1, 4, 6)$ и $H = \sum_{x,y,z}(0, 1, 2, 4, 6)$.
- 4.68. Проверьте, является ли минимальной суммой следующее выражение. Сделайте это простейшим возможным способом (то есть алгебраически, а не с помощью карт).
- $$F = T' \cdot U \cdot V \cdot W \cdot X + T' \cdot U \cdot V' \cdot X \cdot Z + T' \cdot U \cdot W \cdot X \cdot Y' \cdot Z$$
- 4.69. В основном тексте утверждалось, что отправной точкой для традиционных методов минимизации комбинационных схем является таблица истинности или ее эквивалент. Сама по себе карта Карно содержит ту же самую информацию, что и таблица истинности. По заданному выражению вида «сумма произведений» можно расставить на карте единицы, непосредственно соответствующие каждому произведению, не составляя в явном виде таблицы истинности или списка минтермов, а затем осуществить процедуру минимизации с использованием карт. Найдите указанным способом минимальные выражения вида «сумма произведений» для каждой из следующих логических функций:
- $F = X' \cdot Z + X \cdot Y + X \cdot Y' \cdot Z$
 - $F = A' \cdot C' \cdot D + B' \cdot C \cdot D + A \cdot C' \cdot D + B \cdot C \cdot D$
 - $F = W \cdot X \cdot Z' + W \cdot X' \cdot Y \cdot Z + X \cdot Z$
 - $F = (X' + Y') \cdot (W' + X' + Y) \cdot (W' + X + Z)$
 - $F = A \cdot B \cdot C' \cdot D' + A' \cdot B \cdot C' + A \cdot B \cdot D + A' \cdot C \cdot D + B \cdot C \cdot D'$
- 4.70. В условиях задачи 4.69 найдите минимальные выражения вида «произведение сумм» для каждой из указанных там логических функций.

4.71. Выведите минимальное выражение вида «произведение сумм» для функции, реализуемой устройством обнаружения простых двоично-десятичных чисел (см. рис. 4.37). Проверьте, равно ли это выражение в алгебраическом смысле минимальному выражению вида «сумма произведений», и объясните полученный результат.

4.72. Карту Карно для функции 5 переменных (5-variable Karnaugh map) можно представить так, как показано на рис. X4.72. В такой карте клетки, занимающие одинаковое положение в подкартах для $V=0$ и $V=1$, считаются соседними. (В разделах 7.4.4 и 7.4.5 приведено несколько подробно разобранных примеров карт Карно для функций 5 переменных.) Найдите с помощью карт Карно выражения вида «сумма произведений» для каждой из следующих функций 5 переменных:

$$(a) F = \sum_{V,W,X,Y,Z} (5, 7, 13, 15, 16, 20, 25, 27, 29, 31)$$

$$(b) F = \sum_{V,W,X,Y,Z} (0, 7, 8, 9, 12, 13, 15, 16, 22, 23, 30, 31)$$

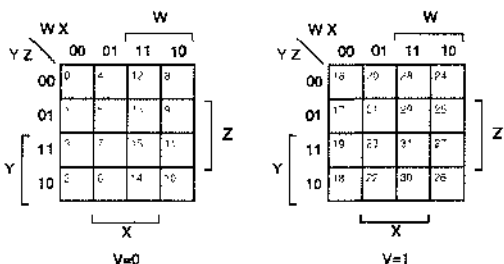
$$(c) F = \sum_{V,W,X,Y,Z} (0, 1, 2, 3, 4, 5, 10, 11, 14, 20, 21, 24, 25, 26, 27, 28, 29, 30)$$

$$(d) F = \sum_{V,W,X,Y,Z} (0, 2, 4, 6, 7, 8, 10, 11, 12, 13, 14, 16, 18, 19, 29, 30)$$

$$(e) F = \prod_{V,W,X,Y,Z} (4, 5, 10, 12, 13, 16, 17, 21, 25, 26, 27, 29)$$

$$(f) F = \sum_{V,W,X,Y,Z} (4, 6, 7, 9, 11, 12, 13, 14, 15, 20, 22, 25, 27, 28, 30) + d(1, 5, 29, 31)$$

Рис. X4.72



4.73. В условиях задачи 4.72 найдите минимальные выражения вида «произведение сумм» для каждой из указанных там логических функций.

4.74. Карту Карно для функции 6 переменных (6-variable Karnaugh map) можно изобразить так, как показано на рис. X4.74. В такой карте клетки, занимающие одинаковое положение в соседних подкартах, считаются соседними. Минимизируйте следующие функции 6 переменных с помощью карт Карно:

$$(a) F = \sum_{U,V,W,X,Y,Z} (1, 5, 9, 13, 21, 23, 29, 31, 37, 45, 53, 61)$$

$$(b) F = \sum_{U,V,W,X,Y,Z} (0, 4, 8, 16, 24, 32, 34, 36, 37, 39, 40, 48, 50, 56)$$

$$(c) F = \sum_{U,V,W,X,Y,Z} (2, 4, 5, 6, 12-21, 28-31, 34, 38, 50, 51, 60-63).$$

4.75. Существует $2n$ m -мерных подкубов n -мерного куба при $m = n - 1$. Представьте их в виде текстовых строк и укажите соответствующие термы-произведения. (По мере необходимости вы можете использовать многоточия, например: 1, 2, ..., n .)

4.76. Существует только один m -мерный подкуб n -мерного куба при $m = n$; его представление в виде текстовой строки имеет вид: $xx \dots xx$. Запишите терм-произведение, соответствующий этому кубу.

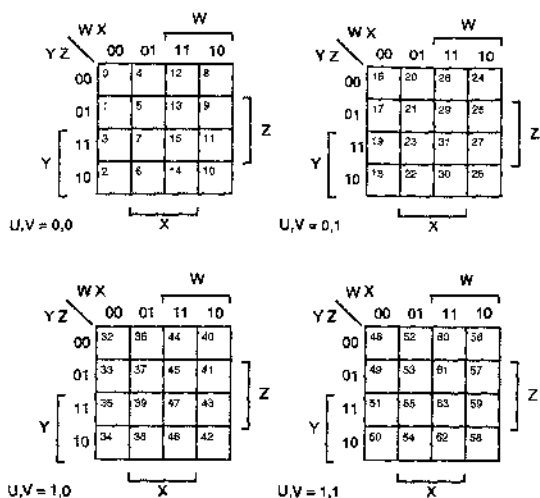


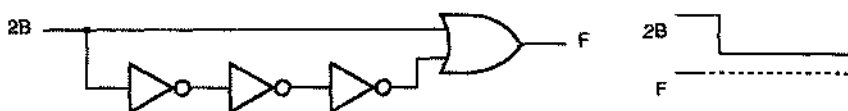
Рис. X4.74

- 4.77. В приведенной в табл. 4.9 программе на языке С память используется неэффективно, поскольку она резервируется для максимального числа кубов на каждом уровне, даже если этот максимум никогда не достигается. Перепишите программу таким образом, чтобы массивы `cubes` и `covered` были одномерными и на каждом уровне число элементов в них было бы только таким, какое необходимо. (Указание: Вы можете разместить кубы последовательно, заимывая начальную точку массива на каждом уровне.)
- 4.78. Сколько раз во внутреннем цикле в программе в табл. 4.9 происходит обращение к каждому отдельному m -мерному кубу только за тем, чтобы «посмотреть на него» и отбросить? Предложите способ исключения таких нерациональных действий.
- 4.79. В третьем цикле `for` в табл. 4.9 производится попытка объединения каждого из m -мерных кубов на данном уровне со всеми другими m -мерными кубами этого уровня. На самом деле, объединять можно только m -мерные кубы с символами 'x' в одинаковых позициях; поэтому есть возможность сократить число итераций в цикле, используя более сложную структуру данных. Предложите структуру данных, в которой кубы данного уровня были бы рассортированы по положению в них символов 'x', и найдите наибольший объем памяти, необходимой для различных элементов в этой структуре данных. Перепишите соответствующим образом программу, приведенную в табл. 4.9.
- 4.80. Проанализируйте, превышает ли экономия, достигаемая за счет уменьшения числа итераций во внутреннем цикле в задаче 4.79, затраты, необходимые для поддержания более сложной структуры данных. Примите разумные предположения о распределении кубов на каждом уровне и оцените зависимость ваших результатов от этих предположений.
- 4.81. Оптимизируйте функцию `Oneone` в программе в табл. 4.8. Очевидный способ оптимизации заключается в том, чтобы раньше выходить из цикла, однако существуют и другие возможности, позволяющие полностью исключить цикл `for`. Одна из них основана на просмотре таблицы, а другая состоит

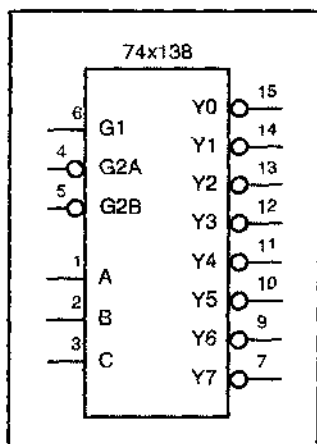
в искусном вычислении, включающем инвертирование, выполнение операции ИСКЛЮЧАЮЩЕЕ ИЛИ и сложение.

- 4.82. Расширьте программу на языке С из табл. 4.9, включив обработку безразличных значений. Введите новую структуру данных `dc[MAX_VARS+1][MAX_CUBES]` для регистрации случаев, когда данный куб содержит только безразличные значения, и обновляйте ее всякий раз при чтении кубов и их создании.
- 4.83. (Схема Гамлета.) Продолжите временную диаграмму и объясните, какую функцию выполняет схема, приведенная на рис. Х4.83. В каком месте схема ведет себя в соответствии с ее названием?

Рис. Х4.83



- 4.84. Докажите, что в двухуровневой схеме И-ИЛИ, реализующей полную сумму логической функции, никогда нет источников опасности.
- 4.85. Найдите логическую функцию четырех переменных, для которой реализация минимальной суммы произведений не является свободной от источников опасности, но существует реализация суммы произведений без источников опасности с меньшим числом термов-произведений, чем в полной сумме.
- 4.86. Беря в качестве отправной точки операторы `WHEN` в программе на языке ABEL в табл. 4.14, составьте логические равенства для переменных `X4–X10`. Объясните, есть ли какие-либо расхождения между вашими результатами и равенствами, приведенными в табл. 4.15.
- 4.87. Нарисуйте принципиальную схему, соответствующую приведенному в табл. 4.12 минимальному двухуровневому выражению логической функции, реализуемой устройством охранной сигнализации. На входах и выходах всех инверторов, вентилях И и вентилях ИЛИ проставьте пару чисел $(t0, t1)$, где $t0$ – номер проверочного вектора из табл. 4.25, обнаруживающего ошибку типа залипания на 0 в этой сигнальной линии, а $t1$ – номер проверочного вектора, обнаруживающего ошибку типа залипания на 1.
- 4.88. Напишите потоковую VHDL-программу (включая объявление объекта и определение архитектуры) для схемы полного сумматора, приведенной на рис. 5.86.
- 4.89. Используя ваш объект из задачи 4.88, напишите структурную VHDL-программу для 4-разрядного сумматора со сквозным переносом, структура которого изображена на рис. 5.87.
- 4.90. Используя ваш объект из задачи 4.88, напишите структурную VHDL-программу для 16-разрядного сумматора со сквозным переносом, построенного по принципу, указанному на рис. 5.87. Используйте оператор `generate` для создания 16 полных сумматоров и соединения их между собой.
- 4.91. Перепишите архитектуру устройства обнаружения простых чисел, приведенную в табл. 4.63, используя оператор `while`.



ПРАКТИЧЕСКАЯ РАЗРАБОТКА СХЕМ КОМБИНАЦИОННОЙ ЛОГИКИ

В предыдущей главе были даны теоретические основы, используемые при разработке комбинационных схем. В данной главе, опираясь на этот фундамент, мы будем создавать многие устройства и структуры, а также описывать методы, применяемые инженерами для решения практических задач, возникающих при разработке цифровых проектов.

Практическая комбинационная схема может иметь множество входов и выходов и потребовать сотен, тысяч, даже миллионов составляющих; поэтому для описания изделия в целом таблица истинности должна содержать *миллиарды и миллиарды* строк. Таким образом, проблемы, связанные с разработкой большинства реальных комбинационных устройств, слишком сложны, чтобы решать их «в лоб» на основе теоретических методов.

ВАЖНОСТЬ СХЕМ 74-Й СЕРИИ

В этой главе мы рассмотрим широко используемые ИС серии 74, в которой логические функции представлены достаточно полно. Эти изделия являются основными кирпичиками в арсенале разработчика цифровых устройств, поскольку уровень их функциональных возможностей часто соответствует уровню мышления конструктора при разбиении большой задачи на меньшие куски.

Понимать функции, реализуемые ИС серии 74, важно даже в том случае, когда вы проектируете устройство на основе ПЛУ, ИС типа FPGA или специализированных ИС. В разработке с применением ПЛУ стандартные функции, реализуемые с помощью ИС, можно использовать в качестве отправной точки при выводе логических выражений более сложных специальных функций. И в ИС типа FPGA, и в специализированных ИС базовые конструктивные блоки (или «типовые ячейки», или «макроячейки»), предоставляемые производителем этих ИС, фактически можно считать ИС средней степени интеграции, реализующими те же функции, что и схемы 74-й серии, вплоть до похожих обозначений.

Но подождите, скажете Вы, как может человек вообще представить себе такую сложную принципиальную схему? Ключом является структурный образ мышления. Сложная схема или система задумывается как совокупность подсистем меньшего размера, каждая из которых описывается намного проще.

В комбинационных логических конструкциях имеется несколько простых структур – декодеры, мультиплексоры, комбинаторы и т.п., – которые систематически используются в качестве строительных блоков в более крупных системах. В настоящей главе представлены самые важные из этих структур. Мы описываем каждую структуру в общем виде, а затем приводим примеры и указываем приложения с использованием компонентов 74-й серии и путем программирования ПЛИУ на языках ABEL и VHDL.

Прежде чем браться за стандартные комбинационные блоки, нам следует обсудить несколько важных тем. Первая из них – это стандарты на документацию, используемые разработчиками цифровых устройств, чтобы гарантировать правильность, технологичность и удобство в эксплуатации их проектов. Затем обсуждается схема синхронизации – узел ответственный за успешную реализацию цифрового проекта в целом. В третьей части описывается внутренняя структура комбинационных ПЛИУ, которые позже мы используем в качестве «универсальных» составных элементов.

5.1. Стандарты документации

Хорошая документация важна для правильного проектирования и эффективного технического обслуживания цифровых систем. Кроме того, будучи точной и полной, документация должна быть до некоторой степени руководством, позволяющим инженеру-испытателю, технику по обслуживанию или даже самому инженеру-разработчику (спустя шесть месяцев после создания схемы) понять, как работает система только на основании сведений, содержащихся в этой документации.

Хотя тип документации зависит от сложности системы, средств ее разработки и возможностей производства, комплект документации, как правило, должен содержать, по крайней мере, следующие шесть составляющих:

1. *Техническими требованиями (circuit specification)* точно определяется, для чего предназначена схема или система, включая описание всех ее входов и выходов («интерфейсов») и выполняемых ею функций. Заметьте, что «спецификация» не должна определять, как система достигает результата, она указывает только, каковы эти результаты. Впрочем, многие компании часто объединяют технические данные с одним или несколькими документами, о которых идет речь ниже, где описывается, как работает система.
2. *Блок-схема (block diagram)* – это неформальное наглядное изображение главных функциональных модулей системы и основных связей между ними.
3. *Принципиальная схема (schematic diagram)* – это формальное детальное изображение электрических компонентов системы, их взаимосвязей и всех подробностей, необходимых для создания системы, включая типы ИС, их обозначения и номера выводов. Ранее мы использовали термин *логическая схема (logic diagram)* для неформального изображения со значительно мень-

шим уровнем детализации. В большинстве программ, с помощью которых рисуются схемы, можно создавать *список компонентов* (*bill of materials, BOM*), входящих в состав схемы; этот список говорит отделу снабжения, какие электрические компоненты необходимо заказать для создания системы.

4. *Временная диаграмма* (*timing diagram*) показывает зависимость различных логических сигналов от времени, а также причинно-следственные связи между наиболее важными из них и задержки.
5. *Структурное описание логического устройства* (*structured logic device description*) отражает внутреннее функционирование программируемого логического устройства (ПЛУ), неперепрограммируемой вентиляционной матрицы (ИС типа FPGA) или специализированной ИС (ASIC). Обычно оно пишется на одном из языков описания схем (HDLs), таких как ABEL или VHDL, но может быть представлено и в виде логических выражений, таблиц состояний или диаграмм состояний. В некоторых случаях для моделирования работы схемы или для наблюдения за ее поведением может использоваться традиционный язык программирования типа языка C.
6. *Описание схемы* (*circuit description*) — это текстовый комментарий, который вместе с остальной документацией объясняет внутреннюю работу схемы. В описании схемы должны быть перечислены все принятые допущения и потенциальные подводные камни, которые могут проявиться в работе схемы, а также должно быть указано использование любых неочевидных «уловок» в данной конструкции. Хорошее описание схемы содержит также определения акронимов и других специальных терминов, а также ссылки на документы, имеющие отношение к данному проекту.

Вероятно, вы уже не раз видели блок-схемы. В следующем разделе мы приведем несколько правил изображения блок-схем, а затем в оставшейся части этого параграфа сконцентрируем внимание на схемных решениях комбинационной логики. В разделе 5.2.1 вводятся временные диаграммы. Структурные логические описания в виде программ на языках ABEL и VHDL представлены в разделах 4.6 и 4.7. В разделе 10.1.6 будет показано, как можно воспользоваться программой на языке C для формирования содержимого постоянного запоминающего устройства.

ДОКУМЕНТАЦИЯ В СЕТИ

В настоящее время профессиональная техническая документация аккуратно поддерживается в корпоративных локальных сетях, поэтому очень полезно включать в спецификацию и описание схемы URL (Uniform Resource Locator; унифицированный указатель информационного ресурса в сети Интернет), чтобы можно было легко найти ссылки. В некоторых компаниях сетевая документация настолько важна и авторитетна, что в подстрочных примечаниях на каждой странице любых технических данных они предупреждают о том, что «печатная версия этого документа является неконтролируемой копией», то есть печатная копия вполне может быть устаревшей.

НЕ ЗАБЫВАЙТЕ ЗАПИСЫВАТЬ!

При разработке новых изделий проектировщикам логических устройств приходится развивать свой собственный язык и вырабатывать подходящие навыки письма, особенно при изложении основных логических идей и описании логической структуры устройства. Самыми удачливыми разработчиками логических устройств (а затем руководителями проектов, главными идеологами и предпринимателями) становятся те, кто доходчиво для других излагает свои идеи, предложения и решения.

Очень важным с практической точки зрения является последний раздел документации — описание схемы. Так же, как опытный программист перед написанием программы в кодах так или иначе описывает ее, опытный разработчик логических устройств начинает не с рисования схемы, а с ее описания. К сожалению, иногда описание схемы создается последним, а иногда вообще не бывает написано. Без описания схемы ее трудно отлаживать, изготавливать, тестировать, эксплуатировать, изменять и улучшать.

5.1.1. Блок-схемы

На блок-схеме (*block diagram*) изображаются входы, выходы, функциональные модули, внутренние пути данных и основные управляющие сигналы системы. Вообще говоря, блок-схема не должна быть настолько подробной, чтобы занимать больше одной страницы, но тем не менее она не должна быть слишком неконкретной. Небольшая блок-схема может иметь от трех до шести блоков, в то время как большая, в зависимости от сложности системы, может содержать от 10 до 15 блоков. В любом случае на блок-схеме должны быть показаны наиболее важные элементы системы и то, как они взаимодействуют. Для большой системы могут потребоваться дополнительные блок-схемы отдельных подсистем, но должна всегда присутствовать блок-схема «верхнего уровня», показывающая систему в целом.

Пример блок-схемы приведен на рис. 5.1. В каждом блоке указана реализуемая им функция, а не отдельные микросхемы, входящие в его состав. В качестве другого примера на рис. 5.2(а) показано условное обозначение 32-разрядного регистра. Если регистр должен быть построен на основе четырех 8-разрядных регистров 74LS377 и эти сведения важны для кого-то, кому предстоит воспользоваться этой блок-схемой (например, из соображений стоимости), то эту информацию можно указать, как это сделано на рис. 5.2(б). Однако неправильно разбивать блок на части, чтобы показать отдельные микросхемы [рис. 5.2(с)].

Шина (bus) — это совокупность двух или большего числа родственных по своему назначению сигнальных линий. На блок-схеме шины изображаются двойной линией или линией большей толщины. Косая черта с числом, если таковая имеется, указывает количество отдельных сигнальных линий в шине. Размер шины может быть также указан в ее названии (например, `INBUS[31..0]` или `INBUS[31:0]`). Активные уровни (определяемые ниже) и кружки инверсии могут быть или не быть указаны на блок-схеме; в большинстве случаев на таком уровне детализации это несущественно. Однако главные управляющие сигналы и шины должны быть названы, как правило, теми же самыми именами, под которыми они появляются в более подробной схеме.

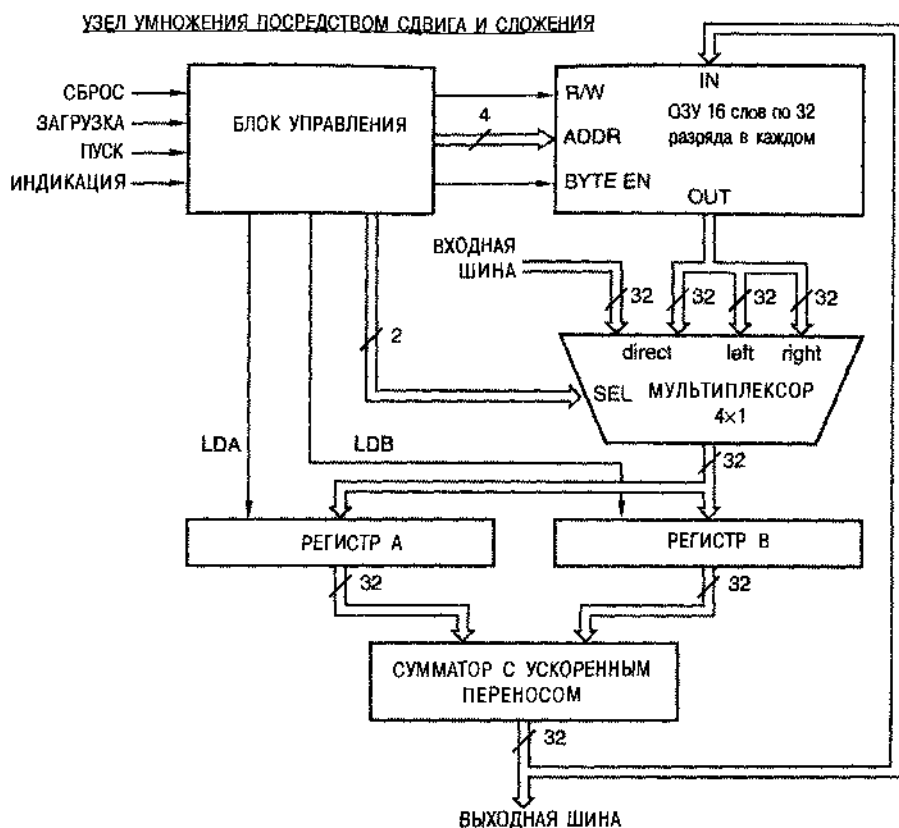


Рис. 5.1. Блок-схема цифрового устройства

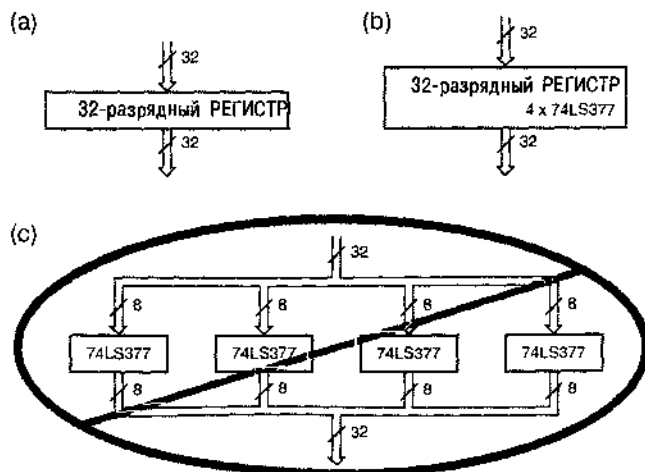


Рис. 5.2. Условное обозначение 32-разрядного регистра: (а) реализация не уточнена; (б) задан тип микросхем; (с) слишком подробно

В блок-схеме поток управляющих сигналов и данных должен быть ясно представлен. Принципиальные схемы обычно рисуются так, чтобы сигналы проходили слева направо, но в блок-схемах достичь этого идеала труднее. Входы и выходы могут располагаться на любой стороне блока, а направление потока сигналов может быть произвольным. Чтобы устранить неопределенность, на шинах и обычных сигнальных линиях рисуют стрелки-указатели.

5.1.2. Условные обозначения логических схем

На рис. 5.3(а) представлены условные обозначения логических схем И и ИЛИ, а также буферов. (Вспомните из главы 3, что буфер – это схема, которая просто преобразует «слабые» логические сигналы в «сильные».) Условное обозначение вентилей И и ИЛИ с большим числом входов приведено на рис. 5.3(б). Небольшая окружность, называемая *кружком инверсии* (*inversion bubble*), обозначает логическую инверсию или взятие доополнения и применяется в условных обозначениях схем И-НЕ, ИЛИ-НЕ и инверторов, как показано на рис. 5.3(с).

В случае схем с инверсией на выходе логические выражения можно преобразовывать, используя обобщенную теорему Де Моргана. Если, например, X и Y – сигналы на входах схемы И-НЕ, а Z – сигнал на ее выходе, то можно записать:

$$Z = (X \cdot Y)' = X' + Y'.$$

Это приводит к двум различным, но одинаково правильным условным обозначениям схемы И-НЕ, что было продемонстрировано на рис. 4.3. Фактически аналогичные преобразования можно применять также к схемам с неинвертируемыми выходами. Рассмотрим, например, следующие соотношения для схемы И:

$$Z = X \cdot Y = ((X \cdot Y)')' = (X' + Y')'.$$

Отсюда следует, что схему И можно изобразить в виде схемы ИЛИ с инверсией на входах и на выходе.

СТАНДАРТ IEEE ДЛЯ ОБОЗНАЧЕНИЯ ЛОГИЧЕСКИХ СХЕМ

Вместе с Американским национальным институтом стандартов (ANSI) Институт инженеров по электротехнике и электронике (IEEE) разработал стандартный набор условных обозначений логических схем. Последняя редакция этого стандарта – ANSI/IEEE Std91–1984. *Стандарт IEEE на графическое условное изображение логических функций* (*IEEE Standard Graphic Symbols for Logic Functions*) допускает изображение логических вентилей как в виде прямоугольников, так и с помощью специальных для каждой из схем графических символов. В этой книге мы пользовались и будем продолжать пользоваться условными обозначениями вентилей в виде различных графических символов, а в Интернете на сайте www.ddpp.com имеется справочник по условным обозначениям вентилей в виде прямоугольников согласно стандарту IEEE.

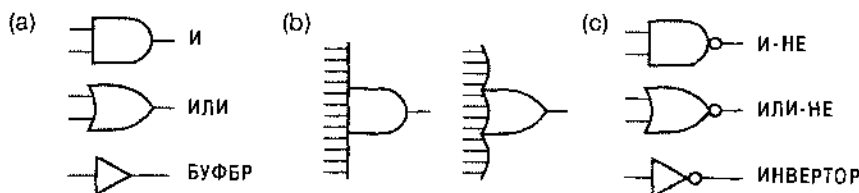


Рис. 5.3. Условные обозначения основных логических вентилей: (а) схема И, схема ИЛИ и буфер, (б) увеличение числа входов, (с) кружки инверсии.

На рис. 5.4 приведены варианты условных обозначений стандартных вентилей, которые можно получить с помощью преобразований. Несмотря на то, что оба условных обозначения в каждой паре представляют одну и ту же логическую функцию, выбор того или другого варианта на принципиальной схеме не произволен, по крайней мере в том случае, когда мы твердо следуем соответствующим стандартам на документацию. Как будет показано в следующих трех разделах, правильный выбор названий сигналов и условных обозначений вентилей может существенно облегчить понимание и использование принципиальных схем.

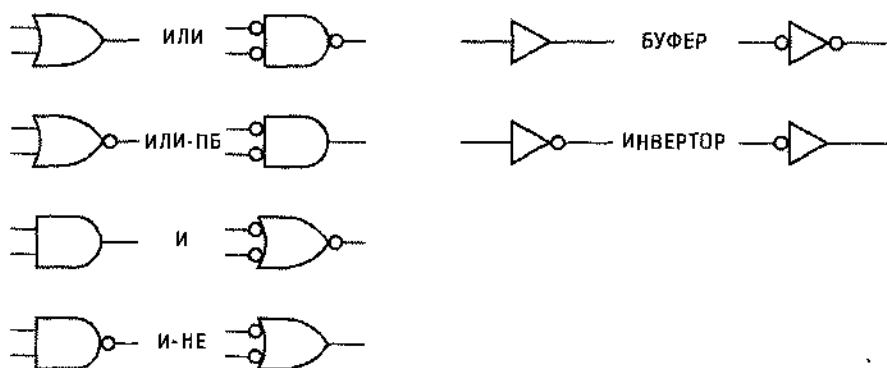


Рис. 5.4. Эквивалентные условные обозначения логических элементов согласно обобщенной теореме Де Моргана

5.1.3. Имена сигналов и активные уровни

В принципиальной схеме каждый сигнальный вход и выход должен иметь описательную буквенно-цифровую метку — имя сигнала. Большинство систем автоматизированного проектирования (CAD) позволяют при рисовании принципиальных схем включать в названия сигналов некоторые специальные символы типа *, _ и !. В примерах анализа и синтеза, приведенных в главе 4, мы называли сигналы, главным образом, одной буквой (X, Y и т.д.), поскольку схемы были простыми. Однако, в реальных системах разумно выбранные названия сигналов сообщают человеку, читающему принципиальную схему, такую же информацию, что и имена пере-

менных в программе. Имя сигнала определяет действие, которым он управляет (GO, PAUSE), условие, которое он обнаруживает (READY, ERROR), или выражаемые им данные (INBUS [31:0]).

С каждым именем сигнала связывают *активный уровень (active level)*. Сигнал является *активным сигналом высокого уровня (active high)*, если он вызывает исполнение названного действия или означает выполнение названного условия при высоком уровне (HIGH), то есть при значении, равном логической 1. (Согласно определению позитивной логики, повсюду используемой в этой книге, понятия «высокий уровень» и «1» эквивалентны.) Сигнал является *активным сигналом низкого уровня (active low)*, если он вызывает исполнение названного действия или означает выполнение названного условия при низком уровне (LOW), то есть при значении, равном логическому 0. Считается, что *сигнал подан или присутствует (asserted)*, когда он имеет активный уровень. Говорят также, что *сигнал не подан или отсутствует (negated или, иногда, deasserted)*, когда он имеет неактивный уровень.

Значение активного уровня сигнала обычно входит в состав его имени согласно принятому соглашению. В табл. 5.1 приведены примеры различных *соглашений об именах сигналов с активным уровнем (active-level naming conventions)*. Иногда выбор того или другого варианта имени сигнала всего лишь дело личных предпочтений, но чаще выбор определяется средствами разработки проекта. Так как значение активного уровня является частью имени сигнала, оно должно быть совместимо с требованиями автоматизированных средств проектирования, обрабатывающих имена сигналов, таких как схемные редакторы, компиляторы языков описания схем и моделирующие программы. В нашей книге мы будем использовать последний из вариантов, приведенных в таблице: к имени сигнала с низким активным уровнем в конце добавляется *суффикс _L (L suffix)*, а имя сигнала с высоким активным уровнем не имеет никакого суффикса. Суффикс _L можно читать как приставку «не».

Табл. 5.1. В каждой строке указаны принятые обозначения активных уровней

Активный низкий уровень	Активный высокий уровень
READY~	READY+
ERROR.L	ERROR.H
ADDR15(L)	ADDR15(H)
RESET*	RESET
ENABLE~	ENABLE
~GO	GO
/RECEIVE	RECEIVE
TRANSMIT_L	TRANSMIT

Крайне важно понять разницу между именами сигналов, выражениями и равенствами. *Имя сигнала* — это только название, буквенно-цифровая метка. *Логическое выражение* объединяет имена сигналов с помощью операторов булевой алгебры типа И, ИЛИ и НЕ, как мы объясняли и делали это в главе 4. *Логическое равенство* приписывает логическому выражению имя сигнала, оно описывает действие одного сигнала в терминах других сигналов.

Различие между именами сигналов и логическими выражениями можно связать с концепцией, принятой в языках программирования: левая часть оператора присваивания содержит *имя переменной*, а правая – *выражение*, значение которого будет дано названной переменной [например, $Z = (X+Y)$ в языке C]. В языке программирования вы не можете поместить выражение в левую часть оператора присваивания. При цифровом проектировании нельзя использовать логическое выражение в качестве имени сигнала.

Логические сигналы могут иметь такие имена как X, READY и GO_L. В GO_L суффикс «_L» как раз является частью имени сигнала, подобно символу подчеркивания в имени переменной в программе на языке C. Не существует сигнала с именем READY', эта запись представляет собой выражение, поскольку символ ' является оператором. Однако могут быть два сигнала с именами READY и READY_L, такими что при нормальной работе схемы $READY_L = READY'$. В этой книге особое внимание обращено на различие между именами сигналов, которые всегда набраны черным шрифтом, и логическими выражениями, которые всегда напечатаны синим цветом, когда они приведены на схеме вблизи соответствующих сигнальных линий.

5.1.4. Активные уровни на выводах схем

Когда мы рисуем условные обозначения вентилей И и ИЛИ или прямоугольник, изображающий большую логическую схему, мы считаем, что *внутри* этого символического изображения находится схема, реализующая данную логическую функцию. На рис. 5.5(а), приведены условные обозначения вентилей И и ИЛИ и большого логического элемента с входом ENABLE. В отношении вентилей И и ИЛИ предполагается, что их входные сигналы имеют высокий активный уровень: требуется наличие логических единиц на входе, чтобы сигнал на выходе принял соответствующее значение. Аналогично для большой схемы: сигнал на входе ENABLE имеет высокий активный уровень: то есть он должен быть равен 1, чтобы дать возможность схеме выполнить свою операцию. На рис. 5.5(б) показаны те же самые логические элементы с низкими активными уровнями сигналов на входах и выходах. *Внутри* символических изображений реализуются те же самые логические функции, но кружки инверсии указывают на то, что теперь для реализации соответствующих логических функций сигналы на входах должны принимать значения 0, и на выходах нули появляются только в том случае, когда схемы надлежащим образом «делают свое дело».

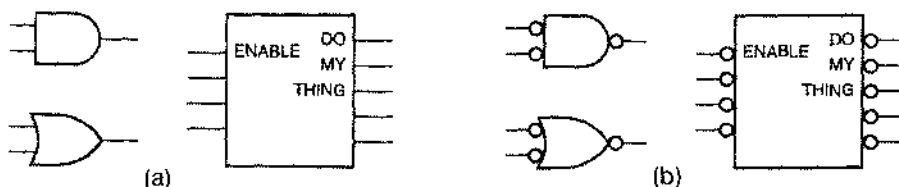


Рис. 5.5. Условные обозначения (а) схемы И и ИЛИ и большого логического элемента, (б) те же самые элементы с низкими активными уровнями сигналов на входах и выходах

Таким образом, в изображении входов и выходов вентилях и больших логических схем может содержаться информация об активном уровне сигнала. Кружок инверсии используется для того, чтобы показать, что сигнал на данном выводе имеет низкий активный уровень, а отсутствие кружка у вывода говорит о том, что активным является высокий уровень сигнала. Например, схема И, приведенная на рис. 5.6(а), реализует логическую функцию И для двух входных сигналов с высоким активным уровнем и формирует на выходе сигнал с высоким активным уровнем: если на обоих входах присутствуют 1, то на выходе тоже появляется 1. Схема И-НЕ, изображенная на рис. 5.6(б), также реализует функцию И, но на ее выходе сигнал имеет низкий активный уровень. Даже схемы ИЛИ-НЕ или ИЛИ можно представить в виде схемы И, используя низкие активные уровни сигналов на входах и выходах, как показано на рис. 5.6(с) и (д). Можно сказать, что все четыре схемы, приведенные на рисунке, реализуют одну и ту же функцию: сигнал присутствует на выходе каждой схемы, если он присутствует на обоих ее входах. На рис. 5.7 то же самое показано для функции ИЛИ: сигнал присутствует на выходе каждой схемы, если он присутствует хотя бы на одном из его входов.

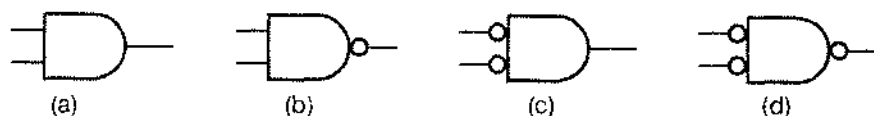


Рис. 5.6. Четыре варианта реализации функции И: (а) схема И (74х08); (б) схема И-НЕ (74х00); (с) схема ИЛИ-НЕ (74х02); (д) схема ИЛИ (74х32)

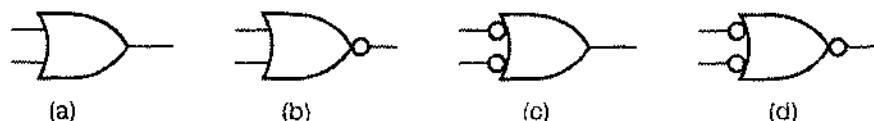


Рис. 5.7. Четыре варианта реализации функции ИЛИ: (а) схема ИЛИ (74х32); (б) схема ИЛИ-НЕ (74х02); (с) схема И-НЕ (74х00); (д) схема И

Иногда для увеличения коэффициента разветвления логического сигнала без изменения его функции используется неинвертирующий буфер. На рис. 5.8 показаны возможные условные обозначения инверторов и неинвертирующих буферов. В терминах активных уровней все схемы, соответствующие приведенным условным обозначениям, реализуют в точности одну и ту же функцию: на выходе каждой схемы сигнал присутствует только в том случае, когда присутствует сигнал на входе.

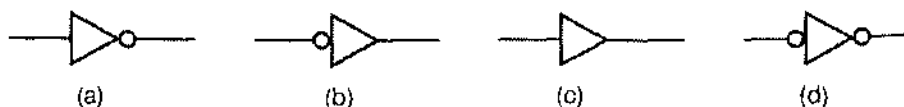


Рис. 5.8. Варианты условных обозначений: (а, б) инверторы; (с, д) неинвертирующие буферы

5.1.5. Метод проектирования «инверсия к инверсии»

Опытные разработчики логических устройств создают свои схемы на языке логических функций, которые должны выполняться *внутри* символически изображенного блока. Проектируете ли вы на основе дискретных схем или с использованием языков описания схем, таких как ABEL и VHDL, проще всего представлять логические сигналы и их взаимодействия, пользуясь именами высоких активных уровней сигналов. Однако, как только схема готова к реализации, вам, вероятно, придется иметь дело с низкими активными уровнями сигналов из-за требований, предъявляемых внешним окружением.

Когда вы проектируете платы или специализированные ИС на уровне отдельных вентилях, часто главным требованием является быстродействие. Как было показано в разделе 3.3.6, инвертирующие вентили обычно быстрее, чем неинвертирующие, поэтому перевод некоторых сигналов в форму с низким активным уровнем часто приводит к существенному улучшению характеристик.

При проектировании с применением крупных узлов многие из них могут содержать в своем составе микросхемы или другие компоненты, у которых на отдельных входах и выходах активным может быть низкий уровень сигнала. Причины, по которым используется низкий активный уровень, могут быть разными: от улучшения характеристик до многолетних традиций, но в любом случае вам все же придется иметь с ним дело.

Логическое проектирование по принципу «инверсия к инверсии» (bubble-to-bubble logic design) представляет собой практику выбора таких условных обозначений и имен сигналов, включая указатели активного уровня, которые делают функцию, реализуемую логической схемой, более понятной. Обычно это означает выбор имени сигнала, типа вентиля и условного обозначения такими, чтобы большинство кружков инверсии «взаимно уничтожались» и логическая блок-схема могла быть проанализирована так, как если бы все сигналы имели высокий активный уровень.

Предположим, например, что нам требуется сформировать сигнал пуска "GO", когда при наличии сигнала готовности "READY" мы получаем сигнал запроса "REQUEST". Из постановки задачи ясно, что требуется схема И; на языке булевой алгебры мы бы записали: $GO = READY \cdot REQUEST$. Однако для реализации функции И мы можем воспользоваться различными схемами в зависимости от необходимого активного уровня для сигнала GO и активных уровней имеющихся входных сигналов.

ИМЯ СИГНАЛА!

Хотя абсолютно необходимо присваивать имена только сигналам, действующим на основных входах и выходах схемы, большинство разработчиков считают полезным давать имена также и внутренним сигналам. В процессе отладки схемы хорошо иметь имя для указания на странно ведущий себя внутренний сигнал. Большинство систем машинного проектирования автоматически генерируют метки для неназванных сигналов, но имя, выбранное пользователем, предпочтительнее имени, создаваемого компьютером, например, такого: XSIG1057.

На рис. 5.9(а) показан простейший случай, где сигнал GO должен быть с высоким активным уровнем, так же как и имеющиеся входные сигналы; здесь мы применяем схему И. С другой стороны, если устройство, на которое поступает сигнал GO, требует, чтобы на его входе был сигнал с низким активным уровнем GO_L, то мы можем использовать схему И-НЕ, как показано на рис. 5.9(б). Если имеющиеся входные сигналы имеют низкий активный уровень, можно воспользоваться схемой ИЛИ-НЕ или ИЛИ, как показано на рис. 5.9(с) и (д).

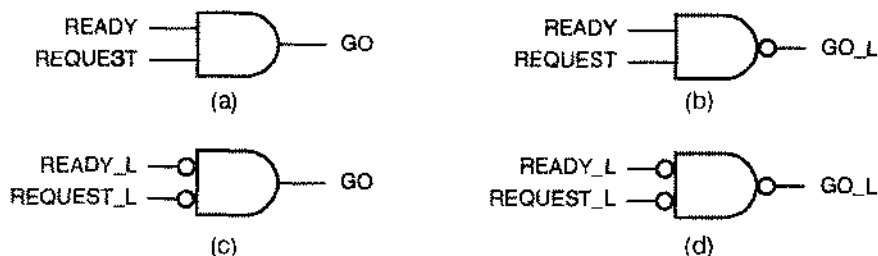


Рис. 5.9. Различные варианты формирования сигнала GO: (а) высокие активные уровни сигналов на входах и на выходе; (б) высокие активные уровни сигналов на входах, низкий активный уровень сигнала на выходе; (с) низкие активные уровни сигналов на входах, высокий активный уровень сигнала на выходе; (д) низкие активные уровни сигналов на входах и на выходе

Активные уровни имеющихся сигналов не всегда соответствуют активным уровням схем, которыми мы располагаем. Предположим, например, что имеются входные сигналы READY_L (низкий активный уровень) и REQUEST (высокий активный уровень). На рис. 5.10 показаны два различных способа формирования сигнала GO с использованием инвертора для образования активного уровня, необходимого для схемы И. Второй вариант, как правило, предпочтительнее, поскольку инвертирующие вентили, подобные схеме ИЛИ-НЕ, обычно быстрее неинвертирующих, таких как схема И. В каждом случае мы изобразили инвертор по-разному, чтобы сделать активный уровень на его выходе соответствующим имени сигнала.

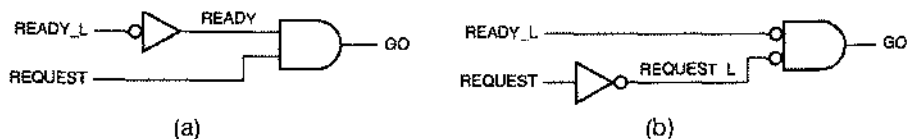


Рис. 5.10. Еще два варианта формирования сигнала GO с разными активными уровнями входных сигналов: (а) вариант со схемой И; (б) вариант со схемой ИЛИ-НЕ

Чтобы понять выгоду от применения метода проектирования по принципу «инверсия к инверсии», рассмотрим схему на рис. 5.11(а). Что она делает? В параграфе 4.2 мы изучили несколько способов анализа такой схемы, и, конечно, могли

бы получить логическое выражение для выходного сигнала DATA, используя эти методы. Однако в случае, когда схема перерисована так, как показано на рис. 5.11(b), выходную функцию можно усмотреть непосредственно из схемы следующим образом: сигнал DATA появляется на выходе, когда активен сигнал ADATA_L или сигнал BDATA_L; если на вход ASEL подан сигнал активного уровня, то сигнал ADATA_L активен только в том случае, когда активен сигнал A, то есть ADATA_L является копией A. Когда сигнал ASEL принимает противоположное значение, активен сигнал BSEL и сигнал BDATA_L является копией сигнала B. Другими словами, сигнал DATA является копией сигнала A, если активен сигнал ASEL, и сигнал DATA является копией сигнала B, если сигнал ASEL не активен. Даже при том, что в схеме имеются пять символов инверсии, мысленно мы должны выводить только одно отрицание, чтобы понять работу схемы, а именно, учесть, что сигнал BSEL активен в отсутствие сигнала ASEL.

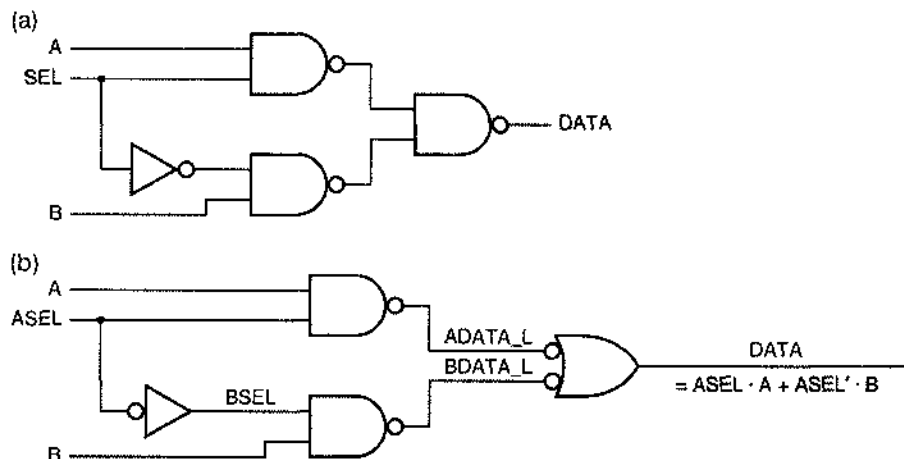


Рис. 5.11. 2-входовой мультиплексор (пока не предполагается, что вы знаете, что это такое): (a) загадочная принципиальная схема; (b) хорошая принципиальная схема, в которой указаны активные уровни сигналов и использованы соответствующие условные обозначения

При желании можно записать алгебраическое выражение для выходного сигнала DATA. Используя методику из параграфа 4.2, мы просто составляем выражения, переходя от схемы к схеме в сторону выхода. При этом можно игнорировать пары символов инверсии, которые сокращаются, и прямо записать выражение, указанное на рисунке синим цветом.

На рис. 5.12 показан другой пример. Глядя непосредственно на схему, мы видим, что сигнал ENABLE_L активен, если активны сигналы READY_L и REQUEST_L или сигнал TEST. Выходной сигнал HALT активен, если оба сигнала READY_L и REQUEST_L не активны или если активен сигнал LOCK_L. Еще раз, в этом примере есть только одно место, где активный уровень на входе вентиля не соответствует уровню входного сигнала, и это отражено в словесном описании схемы.

ПРАВИЛА ЛОГИЧЕСКОГО ПРОЕКТИРОВАНИЯ ПО ПРИНЦИПУ «ИНВЕРСИЯ К ИНВЕРСИИ»

При проектировании логических устройств по принципу «инверсия к инверсии» оказываются полезными следующие правила.

- Имя сигнала на выходе устройства должно содержать указание на тот же самый активный уровень, какой имеет выходной контакт устройства, то есть сигнал должен иметь низкий активный уровень, если выходной контакт в условном обозначении устройства содержит кружок инверсии, и иметь высокий активный уровень, если инверсии нет.
- Если активный уровень входного сигнала совпадает с активным уровнем входного контакта, на который поступает данный сигнал, то логическая функция внутри символически обозначенной схемы активизируется, когда сигнал принимает активный уровень. Это является общепринятым для принципиальных схем.
- Если активный уровень входного сигнала противоположен активному уровню входного контакта, на который подается этот сигнал, то логическая функция внутри символически обозначенной схемы активизируется, когда указанный сигнал инвертирован. Всякий раз, если это возможно, такой ситуации следует избегать, потому что для понимания работы схемы она заставляет нас все время помнить об этом отрицании.

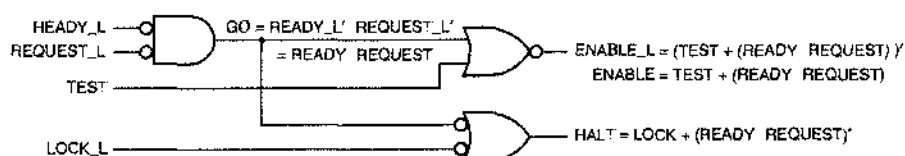


Рис. 5.12. Другой пример правильно нарисованной принципиальной схемы

При желании можно записать алгебраические выражения для сигналов на выходах ENABLE_L и HALT. По мере составления выражения в результате перехода от схемы к схеме в сторону выхода, мы получаем выражения вида $\overline{READY_L' \cdot REQUEST'}$. Однако выражение $\overline{READY_L'}$ можно упростить в соответствии с соглашением об указании активного уровня в именах сигналов. В схеме нет сигнала с именем READY, но если бы он присутствовал, то, согласно правилу присвоения имен, удовлетворял бы соотношению: $READY = \overline{READY_L'}$. Это позволяет нам записать выражения для сигналов ENABLE_L и HALT так, как показано на рис. 5.12. Инвертируя обе части равенства для ENABLE_L, получаем соотношение, которое описывает гипотетический высокий активный уровень выходного сигнала ENABLE в условиях гипотетических высоких активных уровней входных сигналов.

В этой и последующих главах мы увидим еще примеры построения схем по принципу «инверсия к инверсии», в частности тогда, когда начнем использовать более крупные логические элементы.

5.1.6. Расположение элементов на схеме

Блок-схемы и принципиальные схемы следует рисовать так, чтобы входящие в их состав вентили были «нормально» ориентированы, то есть входы были слева, а выходы справа. В условных обозначениях больших логических элементов входы обычно также расположены слева, а выходы справа.

На листе с иловой схемой входы системы должны размещаться слева, а выходы справа, и основное направление передачи сигналов должно быть слева направо. Если вход или выход находятся в середине страницы, то они должны быть продлены до левого или правого края соответственно. В таком случае читающий схему сможет найти все входы и выходы, глядя только на края страницы. Если есть возможность, то все пути сигналов в пределах листа должны быть неразрывными; линии, обозначающие связи, могут быть разорваны в том случае, когда рисунок оказывается перегруженным, но разрывы должны быть помечены в обоих направлениях, как описано ниже.

Иногда, ради большей наглядности, блок-схемы рисуют так, чтобы не было пересечения линий, но в принципиальных схемах это никогда не делается. В таких схемах линии могут пересекаться, но места соединений бывают четко обозначены точками. Правда, в некоторых системах САД (а также у некоторых разработчиков) не иолучается изображать хорошо различные точки в местах соединения. Чтобы отличать простое пересечение линий от соединения, принимается соглашение, согласно которому возможны только соединения типа «Т», как иоказано на рис. 5.13. Это хорошее соглашение, ему стоит следовать в любом случае.

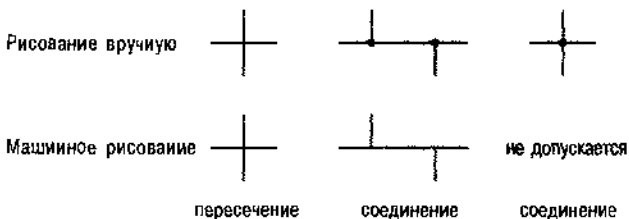


Рис. 5.13. Пересечение линий и места соединения

Легче всего работать со схемами, помещающимися на одном листе. Реально самый большой лист бумаги может быть размером Е (34" × 44"; приблизительно формат А0). Хотя иарисовать на нем можно очень много, но работать с листом бумаги такого размера неудобно. Лучшим компромиссом между илощадью для рисования и удобством пользования является лист размера В (11" × 17"). Его легко сложить для хранения и с него можно снять копию большинством офисных копировальных устройств. Независимо от размера бумаги и схемы лучше всего использовать страницу в альбомной ориентации, то есть так, чтобы прохождение большинства сигналов изображалось в направлении слева направо вдоль длинной стороны листа.

Если схема не помещается на одиом листе, то ее следует разбить на отдельные листы так, чтобы минимизировать связи между листами (и иутаницу). Можно использовать также систему координат, аналогичную применяемой на дорожных

картах и именовать сигналы *флажками (signal flags)* с указанием источников и приемников, размещенных на разных листах. Исходящий сигнал должен иметь флажки со ссылками на все точки назначения этого сигнала, в то время как входящий сигнал должен иметь флажок со ссылкой только на источник. Другими словами, входящий сигнал должен сопровождаться указанием, где он сформирован, а не местом где-нибудь в середине цепочки адресатов, которые используют этот сигнал.

Многостраничные схемы обычно имеют «илоскую» структуру. Каждый лист, как показано на рис. 5.14, выделен из полной схемы и может соединяться с любым другим листом так, как будто все листы расположены на одном большом листе. Однако почти так же, как и программы, схемы могут быть организованы по иерархическому принципу, как изображено на рис. 5.15. При таком подходе схема «верхнего уровня» занимает всего один лист и может быть блок-схемой. Как правило, схема верхнего уровня не содержит никаких вентилях или других логических элементов; на ней бывают изображены только блоки, соответствующие крупным узлам системы, и связь между ними. В свою очередь, состав блоков или крупных узлов системы раскрывается на листах более низкого уровня, которые могут содержать описания на уровне обычных вентилях или сами состоять из блоков, раскрываемых на еще более низких уровнях иерархии. Если более детальное раскрытие на одном из нижних уровней требуется не один раз, то обращение к этому листу (или, в терминах программирования, «вызов») со стороны листов более высокого уровня может происходить многократно.

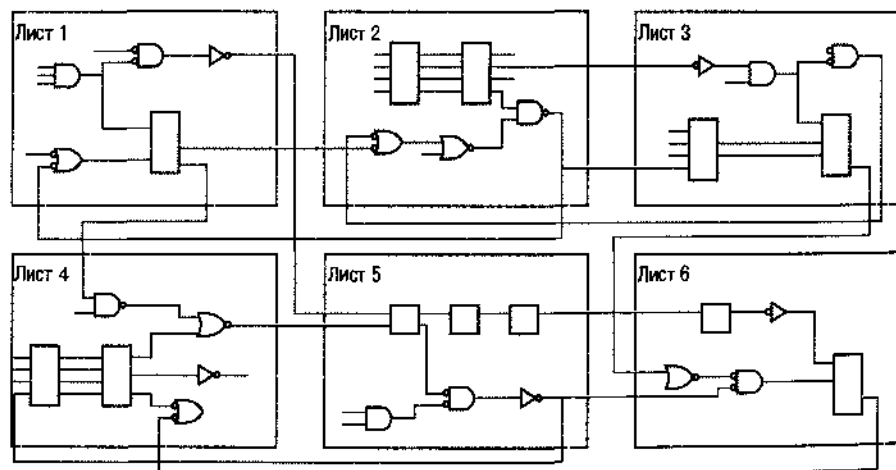


Рис. 5.14. Плоская структура схемы

Большинство автоматизированных систем проектирования поддерживают как плоские, так и иерархические варианты представления схем. В обоих случаях очень важно правильно присвоить имена сигналам, чтобы избежать некоторых распространенных ошибок:

- Подобно любой другой программе, программа, с помощью которой создается схема, делает то, что вы говорите, а не то, что вы имеете в виду. Если вы

используете на разных листах слегка различающиеся имена одного и того же сигнала, то они не будут связаны между собой.

- Если же, наоборот, вы неосторожно используете одно и то же имя для различных сигналов на разных листах плоского варианта представления схемы, то в большинстве случаев программа послушно соединит их вместе, даже если вы не соединили их с помощью флажков на разнесенных листах. (При иерархическом варианте построения схем обычно допускается повторное использование имени в различных местах иерархии, так как программа связывает каждое имя с его местом в уровне иерархии.)
- При иерархическом варианте построения схем необходимо быть внимательным при присвоении имен внешним интерфейсным сигналам на листах нижних уровней иерархии. Эти имена появятся внутри блоков, соответствующих листам нижнего уровня, при использовании данных блоков в верхних уровнях иерархии. Очень легко перепутать имена сигналов или ошибочно указать активный уровень, расплачиваясь за это неправильной работой блока.
- Наконец, существует проблема, как правило, непосредственно не связанная с присвоением имен, но похожая, что у всех программ, с помощью которых рисуются схемы, есть свои капризы, проявляющиеся в том, что сигналы, которые выглядят как связанные один с другим, реально оказываются не связанными. Использование соглашения о T-образном соединении, продемонстрированного на рис. 5.13, помогает свести эту проблему к минимуму.

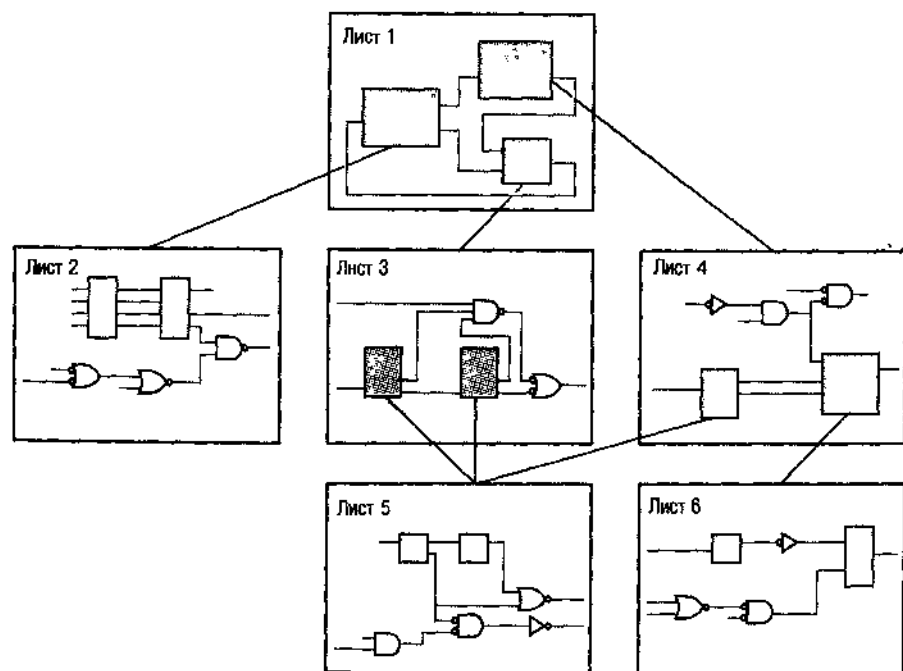


Рис. 5.15. Иерархическая структура схемы

К счастью, у большинства программ, применяемых для создания схем, есть возможность обнаруживать многие из указанных ошибок, производя, например, поиск имен сигналов, не связанных ни с какими входами и выходами, а также имен, многократно присвоенных разным выходным сигналам. Но большинство разработчиков узнает, насколько важна тщательная перепроверка схемы вручную только на собственном горьком опыте создания печатной платы или специализированной ИС по схеме, содержащей какую-нибудь глупую ошибку.

5.1.7. Шины

Согласно введению ранее определенному, шиной называется совокупность из двух или большего числа родственных по своему назначению сигнальных линий. Например, адресная шина микропроцессорной системы может состоять из 16 линий ADDR0–ADDR15, а шина данных — из 8 линий DATA0–DATA7. Имена сигналов в шине не обязательно должны быть близкими или упорядоченными, как в этих первых примерах. Микропроцессорная система, например, может иметь шину управления, состоящую из пяти сигналов: ALE, MIO, RD_L, WR_L, и RDY.

Для уменьшения объема рисунка и улучшения читаемости принципиальной схемы, используют специальную систему обозначения шин. Как показано на рис. 5.16, шина имеет свое содержательное имя, типа ADDR[15:0], DATA[7:0] или CONTROL. Для обозначения размерности шины ее имя может включать скобки и двоеточие. Шины изображаются более жирными линиями, чем обыкновенные сигналы. Включение отдельных сигналов в шину или отвод из нее осуществляется обычными сигнальными линиями, соединяемыми с шиной с указанием имени сигнала. Часто используется также специальный символ подключения, как показано в нашем примере.

Система САД прослеживает путь отдельных сигналов в шине. Когда приходит время на самом деле изготовлять устройство по данной принципиальной схеме, отдельные сигнальные линии в шине рассматриваются так, как если бы все они были нарисованы отдельно.

Символы справа на рис. 5.16 являются сигнальными флажками, указывающими связи между листами. Они показывают, что шина LA уходит на лист с номером 2, шина DB является двунаправленной и соединяется с листом 2, шина CONTROL также двунаправленная и соединяется с листами 2 и 3.

5.1.8. Дополнительная информация о схеме

В полных принципиальных схемах указываются типы ИС, порядковые номера микросхем и их цоколевка, как показано на рис. 5.17. Тип ИС (IC type) — это имя, идентифицирующее интегральную схему, которая выполняет данную логическую функцию. Например, 2-входовой вентиль И-НЕ можно отождествить с микросхемами 74HC00 или 74LS00. Кроме логической функции, тип ИС указывает логическое семейство, к которому относится эта микросхема, и ее быстродействие.

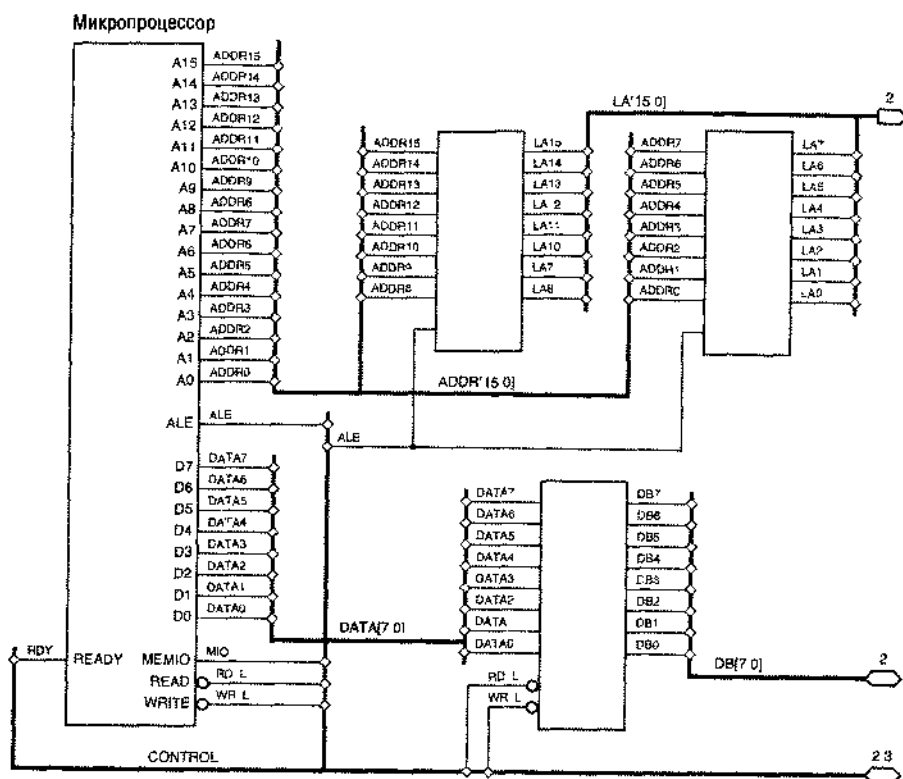


Рис. 5.15. Примеры шин

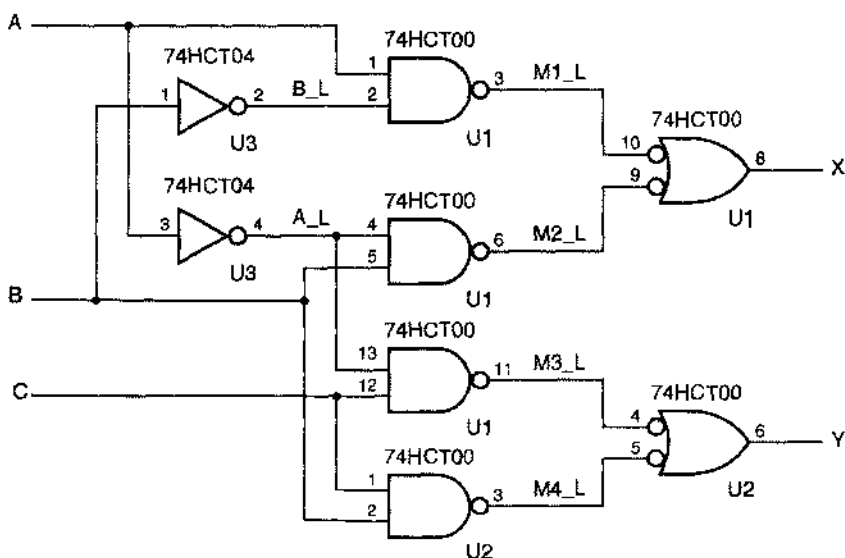


Рис. 5.17. Принципиальная схема блока на микросхемах 74HC00

Порядковый номер (*reference designator*) — это номер, под которым данная ИС фигурирует в системе. Вместе с документацией, относящейся к механической конструкции системы, порядковый номер позволяет размещать ИС при монтаже на определенных местах, а также находить их при тестировании и обслуживании системы. По традиции порядковому номеру ИС предшествует символ U (от слова "unit").

Цоколевка (*pin numbers*) каждой ИС используется для разводки отдельных логических сигналов по ее выводам. Номера выводов написаны около соответствующих входов и выходов стандартного условного обозначения логического элемента, как показано на рис. 5.17.

Чтобы доставить вам удовольствие от правильно нарисованных схем, мы будем в дальнейшем указывать порядковые номера и цоколевку во всех примерах логических схем, где применяются МИС и СИС.

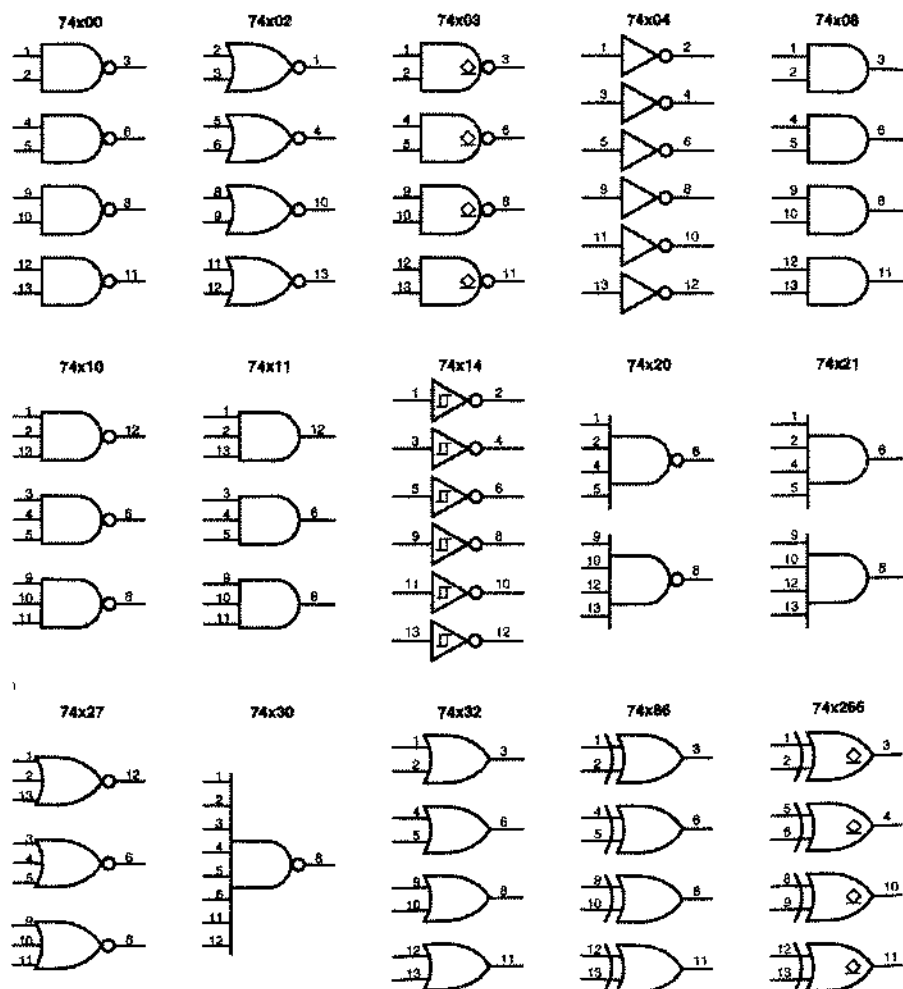


Рис. 5.18. Цоколевка ИС малой степени интеграции в стандартных корпусах DIP

На рис. 5.18 приведена цоколевка довольно многих ИС малой степени интеграции, которые встречаются в примерах повсюду в этой книге. В условных обозначениях ряда вентилях использованы специальные графические элементы:

- В условном обозначении инвертора с триггером Шмитта (микросхема 74х14) специальным символом указывается наличие гистерезиса.
- В условном обозначении счетверенного вентиля И-НЕ (микросхема 74х03) и счетверенной схемы ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ (микросхема 74х266) имеется специальный символ, говорящий о том, выход выполнен с открытым коллектором или с открытым стоком.

Когда вы готовите принципиальную схему при разработке печатной платы с помощью программы рисования схем, эта программа автоматически воспроизводит номера выводов устройств, которые вы выбираете из библиотеки компонентов. Обратите внимание, что цоколевки ИС могут отличаться в зависимости от типа корпуса, поэтому следует быть внимательным, выбирая подходящую модификацию компонента из библиотеки. На рис. 5.18 показана цоколевка микросхем, выпускаемых в корпусах DIP (dual in-line-pin; с двухрядным расположением выводов). Микросхемы в таких корпусах вы можете встретить в лаборатории по курсу цифрового проектирования или в промышленных печатных платах с низкой плотностью монтажа микросхем «через отверстия».

5.2. Временные соотношения в схеме

«Время решает все» — это справедливо для инвестиций, в комедии, да и в цифровом проектировании. Как мы узнали в параграфе 3.6, в реальных схемах требуется время для возникновения на выходе реакции на входное воздействие, а многие современные схемы и системы настолько быстры, что существенной становится даже задержка сигнала, распространяющегося «со скоростью света» от выхода одного логического элемента до входа другого в пределах платы или корпуса микросхемы.

Большинство цифровых систем представляют собой последовательно включенные схемы, которые работают в пошаговом режиме, управляемые периодическим синхронизирующим сигналом, и частота этого сигнала ограничена максимальным временем, которое требуется для завершения всех операций на одном шаге. Таким образом, чтобы создавать быстродействующие схемы, которые работали бы правильно при всех условиях, разработчики цифровой аппаратуры должны четко представлять себе временные соотношения.

Последние несколько лет продемонстрировали существенные количественные и качественные достижения средств автоматизированного проектирования схем в отношении анализа временных диаграмм. Тем не менее, довольно часто наиболее сложной проблемой при разработке печатных плат и особенно специализированных ИС является достижение требуемых временных характеристик. В этом параграфе мы начинаем с основ, так что вы поймете, что делают программные средства, когда вы пользуетесь ими, а также разберетесь, как исправлять свои схемы, когда временные соотношения оказываются не совсем удачными.

5.2.1. Временные диаграммы

Временная диаграмма (timing diagram) иллюстрирует характер изменения сигналов в цифровой схеме в зависимости от времени. Временные диаграммы составляют важную часть документации любой цифровой системы. Ими можно воспользоваться как для объяснения временных соотношений между сигналами в пределах системы, так и для того, чтобы сформулировать временные требования к внешним сигналам, которые поступают в систему.

На рис. 5.19(а) изображена блок-схема простого комбинационного устройства с двумя входами и двумя выходами. На рис. 5.19(б) показана задержка выходных сигналов относительно входного сигнала GO в предположении, что входной сигнал ENB зафиксирован на постоянном уровне. В каждой осциллограмме верхняя линия соответствует логической 1, а нижняя линия – логическому 0. Переходы сигналов изображены в виде наклонных линий, чтобы напоминать нам о том, что в реальных схемах изменения сигналов не происходят мгновенно. (Кроме того, наклонные линии лучше смотрятся, чем вертикальные.)

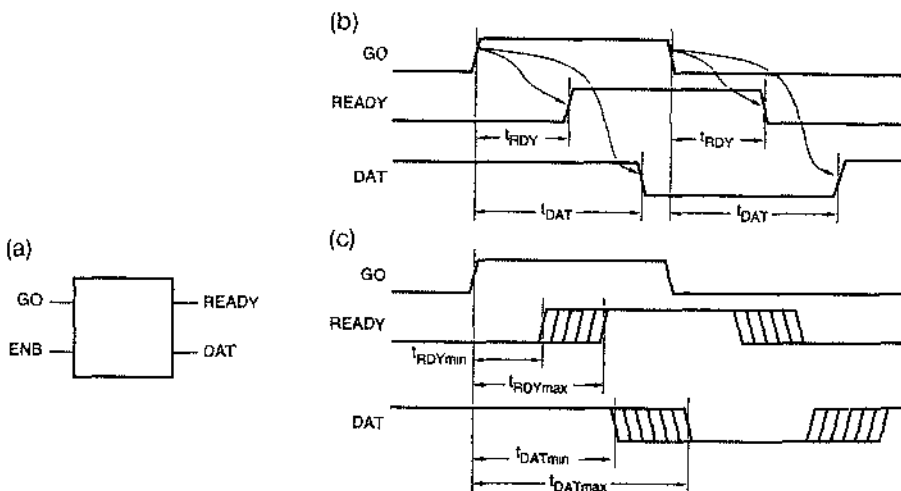


Рис. 5.19. Временные диаграммы для комбинационной схемы: (а) блок-схема устройства; (б) причинная обусловленность и задержка распространения; (с) минимальные и максимальные задержки.

Иногда, особенно в случае сложных временных диаграмм, стрелками отражают *причинную обусловленность связи (causality)*, то есть информацию о том, изменение какого входного сигнала вызывает изменение того или иного выходного сигнала. В любом случае наиболее важной информацией, которую дают временные диаграммы, является величина *задержки (delay)* между переходами в сигналах.

Различные пути прохождения сигнала в схеме могут иметь различные задержки. На рис. 5.19(б), например, показано, что задержка между сигналами GO и READY меньше, чем задержка между сигналами GO и DAT. Точно так же задержка между

входным сигналом ENB и выходными сигналами может быть различной, и это можно было бы наблюдать на другой временной диаграмме. Как мы увидим ниже, задержка при прохождении сигнала по некоторому заданному пути может быть различной в зависимости от того, в каком направлении изменяется сигнал на выходе: от низкого уровня до высокого или от высокого уровня до низкого (это явление не отражено на рисунке).

В реальной схеме задержка обычно измеряется между средними точками переходов в сигналах, как показано на наших временных диаграммах. Одна единственная временная диаграмма может содержать много различной информации о задержках. Каждая задержка имеет свое обозначение; на нашем рисунке это t_{RDY} и t_{DAT} . В сложных временных диаграммах задержки обычно нумеруются для облегчения ссылки на них (например: $t_1, t_2, \dots, t_{42}$). В любом случае временная диаграмма обычно сопровождается *таблицей временных параметров (timing table)*, в которой указаны величина каждой задержки и условия, при которых задержка имеет такое значение.

Величина задержки редко определяется одним числом, так как задержки в реальных цифровых компонентах могут сильно зависеть от напряжения питания, температуры и различных производственных факторов. Поэтому в таблице временных параметров может быть указан диапазон значений, определяющий *минимальное, типичное и максимальное* значение для каждой задержки. Наличие некоторого диапазона возможных значений задержки иногда отражается на временной диаграмме, как показано на рис. 5.19(с), где переходы происходят не в строго фиксированные моменты времени.

Для некоторых сигналов нет необходимости изображать на временной диаграмме, как именно меняется значение сигнала в конкретный момент времени: от 1 до 0 или от 0 до 1; достаточно только показать, что переход имеет место. Этим свойством обладает любой сигнал, который несет информацию о бите «данных»: фактическое значение бита данных изменяется в зависимости от приводящих обстоятельств, но, независимо от его значения, бит передается, сохраняется или обрабатывается в определенный момент времени относительно «управляющих» сигналов в системе. Временная диаграмма на рис. 5.20(а) поясняет эту идею. Сигнал «данные» обычно имеет установившееся значение, равное 0 или 1, а переходы происходят только в определенные моменты времени. Понятие о не строго фиксированной величине задержки можно также применить к сигналам «данные», как показано на рисунке для сигнала DATAOUT.

В цифровых системах обработка группы сигналов данных в шине довольно часто производится идентичными схемами. В этом случае все сигналы в шине имеют одни и те же временные параметры и могут быть представлены одной временной диаграммой и соответствующей записью в таблице временных параметров. Если известно, что сигналы в шине в течение определенного времени имеют конкретные постоянные значения, то иногда это отображают на временной диаграмме двоичными, восьмеричными или шестнадцатеричными числами, как показано на рис. 5.20(б).

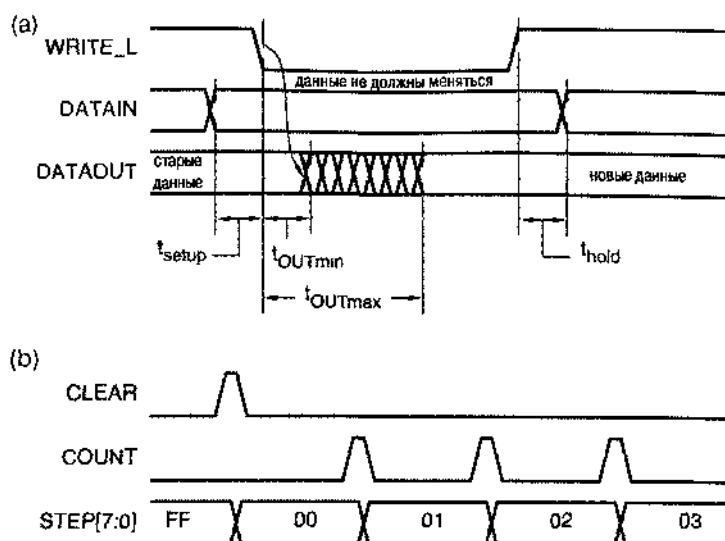


Рис. 5.20. Временные диаграммы сигналов «данные»: (а) фиксированные и строго фиксированные моменты переходов; (б) последовательность значений сигналов на 8-разрядной шине

5.2.2. Задержка распространения

В разделе 3.6.2 мы формально определили *задержку распространения* сигнала как время, затрачиваемое для того, чтобы изменение сигнала во входной цепи привело к изменению сигнала в выходной цепи. В комбинационной схеме с несколькими входами и выходами много различных путей прохождения сигнала, и каждый путь может иметь свое значение задержки распространения. Кроме того, задержка распространения при изменении выходного сигнала от низкого уровня до высокого (t_{PLH}) может отличаться от задержки, когда этот сигнал изменяется от высокого уровня до низкого (t_{PHL}).

Производитель комбинационных ИС обычно указывает все эти различные задержки распространения или, но крайней мере, задержки, которые могут представлять интерес в типичных приложениях. При объединении различных ИС в конструкцию больших размеров, для анализа временных соотношений в схеме в целом используются индивидуальные характеристики отдельных микросхем. Задержка распространения сигнала при прохождении через все логические элементы схемы равна сумме задержек, вносимых отдельными элементами.

5.2.3. Временные параметры

Временные параметры устройства можно охарактеризовать минимальным, типичным и максимальным значениями для каждой задержки распространения и для каждого направления изменения сигнала:

- **Максимальная задержка (maximum delay).** Этот параметр является одним из наиболее часто используемых опытными разработчиками, так как задержка распространения «никогда» не может превышать максимального значения. Однако понятие «никогда» варьируется от семейства к семейству и от изготовителя к изготовителю. Например, «максимальные» задержки распространения ТТЛ-схем 74LS и 74S бывают указаны для напряжения питания $V_{CC} = 5$ В при температуре $T_A = 25^\circ\text{C}$ и при почти отсутствующей емкостной нагрузке. Если напряжение питания или температура изменяются или если емкостная нагрузка больше 15 пФ, то задержка может быть больше. С другой стороны, «максимальная» задержка распространения для микросхем 74АС и 74АСТ относится ко всему рабочему диапазону напряжений питания и температур и к случаю большей емкостной нагрузки, равной 50 пФ.
- **Типичная задержка (typical delay).** Этот один из параметров, чаще всего используемых теми разработчиками, которые не предполагают находиться вблизи своего изделия, когда оно покинет дружественную среду лабораторий и будет отправлено заказчику. «Типичная» задержка – это величина, которая характеризует устройство, изготовленное в хороший день и работающее при почти идеальных условиях.
- **Минимальная задержка (minimum delay).** Это наименьшее значение задержки распространения, которое когда-либо может быть получено. Работа большинства правильно построенных схем не зависит от этого параметра; то есть они будут нормально работать, даже если задержка нулевая. На это следует обращать внимание, потому что производители не указывают минимальную задержку для большинства логических семейств с умеренным быстродействием, включая ТТЛ-семейства 74LS и 74S. Однако у быстродействующих ИС, включая схемы ЭСЛ, а также КМОП-семейства 74АС и 74АСТ, минимальная задержка указывается и она отлична от нуля; это позволяет разработчику гарантированно выполнить временные требования, предъявляемые защелками и триггерами, которые рассматриваются в параграфе 7.2.

Табл. 5.2 содержит типичные и максимальные значения задержки для некоторых КМОП- и ТТЛ-схем серии 74. В табл. 5.3 приведены те же самые параметры для большинства КМОП- и ТТЛ-схем средней степени интеграции, которые появятся в дальнейшем в этой главе.

НАСКОЛЬКО ТИПИЧНО ТИПИЧНОЕ?

Большинство ИС, возможно до 99%, действительно изготовлены в «хорошие» дни и имеют величину задержки, близкую к «типичному» значению. Однако если вы проектируете систему, которая работает только в том случае, когда все 100 используемых в ней ИС имеют «типичные» временные параметры, то, согласно теории вероятностей, система не будет работать с вероятностью $(1 - 0.99^{100}) \cdot 100\% = 63\%$. Впрочем, взгляните на следующее рассуждение, вынесенное за пределы основного текста...

Табл. 5.2. Задержка распространения в наносекундах некоторых 5-вольтовых КМОП- и ТТЛ-схем малой степени интеграции

Номер микросхемы	74НСТ		74АНСТ				74LS			
	Тип.		Тип.		Макс.		Тип.		Макс.	
	$t_{рЛН}$, $t_{рНЛ}$	$t_{рЛН}$, $t_{рНЛ}$	$t_{рЛН}$	$t_{рНЛ}$	$t_{рЛН}$	$t_{рНЛ}$	$t_{рЛН}$	$t_{рНЛ}$	$t_{рЛН}$	$t_{рНЛ}$
'00, '10	11	35	5.5	5.5	9.0	9.0	9	10	15	15
'02	9	29	3.4	4.5	8.5	8.5	10	10	15	15
'04	11	35	5.5	5.5	8.5	8.5	9	10	15	15
'08, '11	11	35	5.5	5.5	9.0	9.0	8	10	15	20
'14	16	48	5.5	5.5	9.0	9.0	15	15	22	22
'20	11	35					9	10	15	15
'21	11	35					8	10	15	20
'27	9	29	5.6	5.6	9.0	9.0	10	10	15	15
'30	11	35					8	13	15	20
'32	9	30	5.3	5.3	8.5	8.5	14	14	22	22
'86 (2 уровня)	13	40	5.5	5.5	10	10	12	10	23	17
'86 (3 уровня)	13	40	5.5	5.5	10	10	20	13	30	22

СЛЕДСТВИЕ ЗАКОНА МЭРФИ

Закон Мэрфи гласит: «Если какая-то неприятность может случиться, то она произойдет». Следствие этого закона выглядит так: «Если вы хотите, чтобы случилась какая-то неприятность, то она не произойдет».

Имея в виду предыдущий пример, вы, возможно, полагаете, что при работе в лабораторных условиях с вероятностью 63% потенциальные проблемы, связанные с временными характеристиками, обнаруживаются. Однако проблемы не распределены равномерно, так как все ИС из данной партии склонны вести себя примерно одинаково. Следствие из закона Мэрфи говорит, что *все опытные образцы* изделия будут собраны из микросхем одной и той же «хорошей» партии. Поэтому все будет прекрасно работать в течение времени, достаточного только для того, чтобы система была принята в массовое производство и все начали радоваться и поздравлять себя с успехом.

Затем, без ведома производственного отдела, от поставщика прибывает «медленная» партия ИС какого-нибудь типа, устанавливаемых в каждой изготавливаемой системе, так что *все перестает работать*. Инженеры-технологи суетятся, пытаются разобраться в проблеме (не простой, потому что разработчик закопался и не потрудился представить описание схемы), а тем временем компания терпит крупные убытки, поскольку не в состоянии отправить свою продукцию.

Табл. 5.3. Задержка распространения в наносекундах некоторых КМОП- и TTL-схем средней степени интеграции (any select – любой вход выбора, any data – любой вход данных, enable – вход разрешения, any input – любой вход, select – вход выбора, any – любой, output – выход)

Номер микросхемы	От	До	74HCT		74AHCT / FCT		74LS			
			Тип.	Макс.	Тип.	Макс.	Тип.	Макс.	Тип.	Макс.
			t_{PLH} / t_{PHL}	t_{PLH} / t_{PHL}	t_{PLH} / t_{PHL}	t_{PLH} / t_{PHL}	t_{PLH}	t_{PHL}	t_{PLH}	t_{PHL}
*138	any select	output (2)	23	45	8.1 / 5	13 / 9	11	18	20	41
	any select	output (3)	23	45	8.1 / 5	13 / 9	21	20	27	39
	G2A, G2B	output	22	42	7.5 / 4	12 / 8	12	20	18	32
	G1	output	22	42	7.1 / 4	11.5 / 8	14	13	26	38
*139	any select	output (2)	14	43	6.5 / 5	10.5 / 9	13	22	20	33
	any select	output (3)	14	43	6.5 / 5	10.5 / 9	18	25	29	38
	enable	output	11	43	5.9 / 5	9.5 / 9	16	21	24	32
*151	any select	Y	17	51	- / 5	- / 9	27	18	43	30
	any select	$\bar{Y}$	18	54	- / 5	- / 9	14	20	23	32
	any data	Y	16	48	- / 4	- / 7	20	16	32	26
	any data	$\bar{Y}$	15	45	- / 4	- / 7	13	12	21	20
	enable	Y	12	36	- / 4	- / 7	26	20	42	32
	enable	$\bar{Y}$	15	45	- / 4	- / 7	15	18	24	30
*153	any select	output	14	43	- / 5	- / 9	19	25	29	38
	any data	output	12	43	- / 4	- / 7	10	17	15	26
	enable	output	11	34	- / 4	- / 7	16	21	24	32
*157	select	output	15	46	6.8 / 7	11.5 / 10.5	15	18	23	27
	any data	output	12	38	5.6 / 4	9.5 / 6	9	9	14	14
	enable	output	12	38	7.1 / 7	12.0 / 10.5	13	14	21	23
*182	any $\bar{G}_1, \bar{P}_1$	C1–3	13	41			4.5	4.5	7	7
	any $\bar{G}_1, \bar{P}_1$	$\bar{G}$	13	41			5	7	7.5	10.5
	any $\bar{P}_1$	$\bar{P}$	11	35			4.5	6.5	6.5	10
	CO	C1–3	17	50			6.5	7	10	10.5
*280	any input	EVEN	18	53	- / 6	- / 10	33	29	50	45
	any input	ODD	19	56	- / 6	- / 10	23	31	35	50
*283	CO	any Si	22	66			16	15	24	24
	any Ai, Bi	any Si	21	61			15	15	24	24
	CO	C4	19	58			11	11	17	22
	any Ai, Bi	C4	20	60			11	12	17	17
*381	CIN	any Fi					18	14	27	21
	any Ai, Bi	$\bar{G}$					20	21	30	33
	any Ai, Bi	$\bar{P}$					21	33	23	33
	any Ai, Bi	any Fi					20	15	30	23
	any select	any Fi					35	34	53	51
	any select	$\bar{G}, \bar{P}$					31	32	47	48
*682	any Pi	$\overline{PEQ\bar{Q}}$	26	69	- / 7	- / 11	13	15	25	25
	any Qi	$\overline{PEQ\bar{Q}}$	26	69	- / 7	- / 11	14	15	25	25
	any Pi	$\overline{PGT\bar{Q}}$	26	69	- / 9	- / 14	20	15	30	30
	any Qi	$\overline{PGT\bar{Q}}$	26	69	- / 9	- / 14	21	19	30	30

У схем малой степени интеграции задержка распространения от любого входа до выхода одна и та же. Заметьте, что в случае TTL-схем задержки, как правило, различны при изменении сигнала от низкого уровня до высокого и от высокого уровня до низкого (t_{PLH} и t_{PHL}), а у КМОП-схем этого обычно нет. КМОП-схемы имеют более симметричные выходные характеристики, поэтому можно не обращать внимания на небольшое различие между этими двумя случаями.

ОЦЕНКА МИНИМАЛЬНЫХ ЗАДЕРЖЕК

Если минимальная задержка ИС не указана, то предусмотрительный разработчик принимает ее равной нулю.

Некоторые схемы не будут работать, если задержка распространения фактически стремится к нулю, но затраты на изменения в схеме, обеспечивающие ее работоспособность при нулевой задержке, могут быть непомерно велики, тем более что такой случай, как предполагается, никогда не произойдет. Чтобы получить изделие, которое при «разумных» условиях всегда будет работать, разработчики часто принимают величину минимальной задержки ИС равной четверти или трети заявленного *типичного* значения задержки.

Задержка между моментом изменения входного сигнала и моментом изменения сигнала на выходе зависит от внутреннего пути сигнала, и в больших схемах этот путь может быть разным для различных входных комбинаций. Например, 2-входовой элемент ИСКЛЮЧАЮЩЕЕ ИЛИ в микросхеме 74LS86 состоит, как показано на рис. 5.71, из четырех вентилях И-НЕ и имеет два пути различной длины от любого входа до выхода. Если сигнал на одном из входов имеет низкий уровень, а на другом изменяется, то изменение проходит через два вентиля И-НЕ и мы получаем первый набор задержек, приведенный в табл. 5.2. Если сигнал на одном из входов имеет высокий уровень, а на другом изменяется, то изменение проходит внутри схемы через *три* вентиля И-НЕ и мы получаем второй набор задержек. Подобное поведение демонстрируют также микросхемы 74LS138 и 74LS139 (см. табл. 5.3). Однако у соответствующих КМОП-схем таких различий нет; точнее, различия достаточно малы и ими можно пренебречь.

5.2.4. Временной анализ

Для точного анализа временных соотношений в устройстве со многими МИС и СИС разработчику, вероятно, придется изучать ее поведение до мельчайших подробностей. Когда, например, инвертирующие ТТЛ-схемы (И-НЕ, ИЛИ-НЕ и т.д.) включаются последовательно, переход сигнала с низкого уровня на высокий на выходе одного из вентилях вызовет изменение сигнала от высокого уровня до низкого на выходе следующего вентиля, так что средняя задержка оказывается между величинами t_{PLH} и t_{PHL} . С другой стороны, при последовательном включении неинвертирующих вентилях (И, ИЛИ и т.д.) переключение вызывает изменение сигналов на всех выходах в одном и том же направлении, так что различие между значениями t_{PLH} и t_{PHL} увеличивается. Читателю предоставляется возможность провести подобного рода анализ в упражнениях 5.8–5.13.

Анализ оказывается более сложным, если задержка определяется устройствами средней степени интеграции, или в том случае, когда для сигнала имеется много путей от данного входа до данного выхода. Таким образом, в больших схемах, анализ задержки прохождения сигнала по всем возможным путям при переходе во всех направлениях может быть очень сложным.

Чтобы иметь возможность упростить анализ в «наихудшем случае», разработчики часто используют единственный параметр – *задержку в наихудшем случае*

(*worst-case delay*), которая равна наибольшему из значений $t_{\text{рЛН}}$ и $t_{\text{рИЛ}}$. Тогда задержка в схеме для наихудшего случая вычисляется как сумма задержек, вносимых отдельными компонентами в наихудшем случае, независимо от направления переключения и других условий работы схемы. Это может дать чрезмерно завышенную оценку полной задержки, вносимой схемой, но сокращает время разработки и гарантирует работоспособность изделия.

5.2.5. Программные средства временного анализа

Еще проще провести временной анализ с помощью современных программных средств, входящих в состав автоматизированных систем логического проектирования. Встроенные библиотеки таких систем обычно содержат не только условные обозначения и функциональные модели различных логических элементов, но также модели их поведения во времени. В режиме моделирования можно задать последовательность входных сигналов и наблюдать, как и когда их действие проявляется в выходных сигналах. Обычно имеется возможность варьировать значения задержек, выбирая минимальные, типичные или максимальные значения или некоторую их комбинацию.

Даже используя моделирование, вы не защищены от ловушек. Обычно разработчик задает входные последовательности сигналов, для которых моделирующая программа должна сформировать выходные сигналы. Таким образом, необходимо иметь хорошее чутье при выборе тестирующих сигналов и условий моделирования, чтобы воспроизвести наихудший случай и наблюдать соответствующие задержки.

Некоторые программы временного анализа позволяют автоматически находить задержки по всем возможным путям сигнала в схеме и рассчитывать упорядоченный список их значений, начиная с наибольшего. Однако эти результаты могут быть слишком оптимистичными, поскольку некоторые пути при нормальной работе схемы фактически не реализуются, и для соответствующей интерпретации результатов разработчику приходится все же напрягать свои умственные способности.

5.3. Комбинационные программируемые логические устройства

5.3.1. Программируемые логические матрицы

Исторически первыми программируемыми логическими устройствами (ПЛУ) были *программируемые логические матрицы* (ПЛМ; *programmable logic arrays, PLA*). ПЛМ представляют собой комбинационное двухуровневое устройство И–ИЛИ, которое можно запрограммировать для реализации любого логического выражения вида «сумма произведений» с учетом ограничений, накладываемых устройством. Такими ограничениями являются:

- число входов (*inputs*) n ,
- число выходов (*outputs*) m и
- число термов-произведений (*product terms*) p .

О таком устройстве можно говорить как о «ПЛМ размера $n \times m$ с p термами-произведениями». В общем случае p гораздо меньше числа минтермов с n переменными (2^n). Таким образом, ПЛМ не может реализовать произвольную логическую функцию с n переменными и m выходными значениями; их возможности ограничены функциями, которые могут быть представлены в виде суммы произведений с p или меньшим числом термов-произведений.

ПЛМ размера $n \times m$ с p термами-произведениями содержит $p \cdot 2n$ -входовых вентилях И и m p -входовых вентилях ИЛИ. На рис. 5.21 приведена небольшая ПЛМ с четырьмя входами, шестью вентилями И, тремя вентилями ИЛИ и тремя выходами. Сигналы поступают на входы буферов, на выходах каждого из которых появляются прямой и инверсный сигналы, используемые внутри матрицы. Возможные соединения в матрице обозначены символом $\times$; программирование устройства состоит в сохранении только тех соединений, которые необходимы. Выбираемые соединения выполнены в виде *плавких перемычек (fuses)*, которые фактически являются пережигаемыми соединениями или энергонезависимыми ячейками памяти в зависимости от технологии; мы рассмотрим это в разделах 5.3.4 и 5.3.5. Таким образом, сигналы на входах каждого вентиля И могут представлять собой любое подмножество первичных входных сигналов и их инверсий. Точно так же сигналы на входах каждого вентиля ИЛИ могут быть любым подмножеством выходных сигналов схем И.

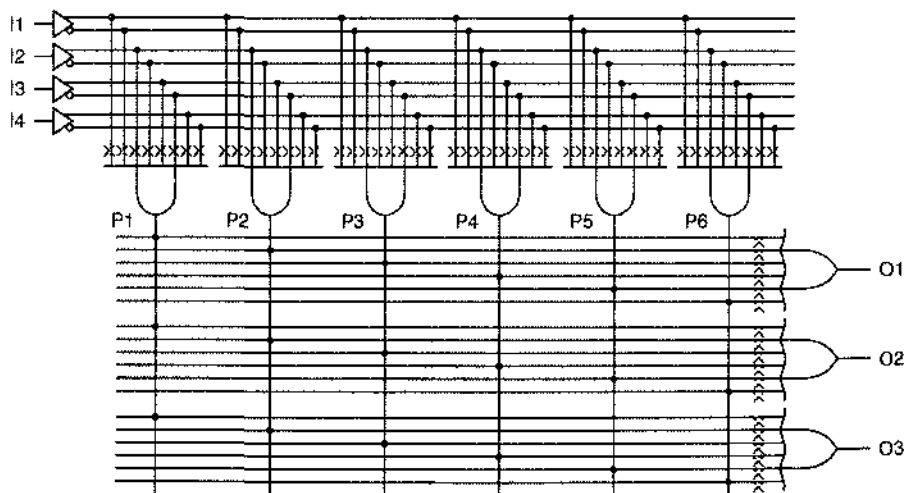


Рис. 5.21. ПЛМ размера 4×3 с шестью термами-произведениями

Возможен более компактный способ представления ПЛМ, указанный на рис. 5.22. Такое изображение условных обозначений вентилях точнее соответствует их фактическому размещению внутри кристалла ПЛМ (см., например, рис. 5.28).

На рис. 5.22 изображена схема ПЛМ, способная реализовать любые три 4-входовые комбинационные логические функции, которые можно записать в виде сумм произведений, состоящих в общей сложности из шести или меньшего числа различных термов-произведений, например:

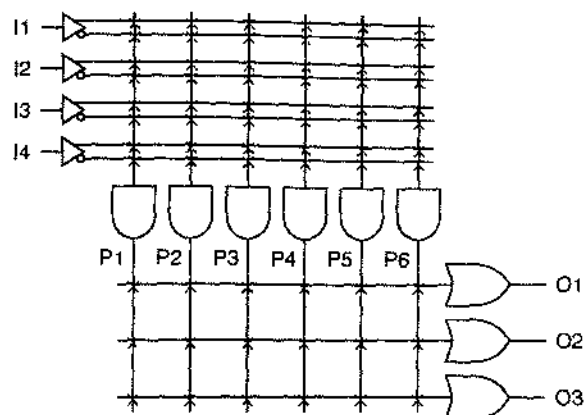


Рис. 5.22. Компактное представление ПЛМ размера 4×3 с шестью термами-произведениями

Всего в этих соотношениях имеется восемь термов-произведений, но первые два слагаемых в выражении для O3 те же самые, что и первые слагаемые в выражениях для O1 и O2. Этим логическим выражениям соответствует конфигурация запрограммированных соединений, приведенная на рис. 5.23.

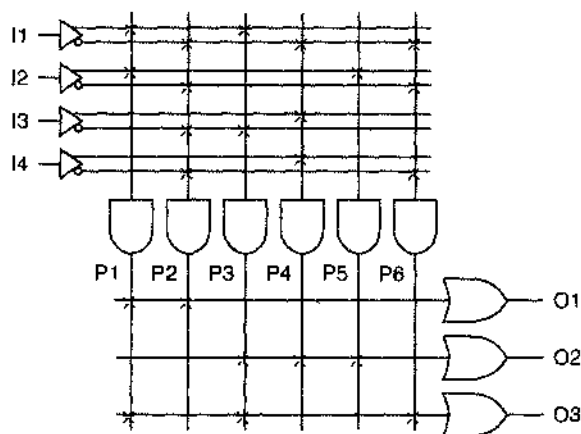


Рис. 5.23. Схема ПЛМ размера 4×3, запрограммированная для реализации трех логических выражений

$$O1 = I1 \cdot I2 + I1' \cdot I2' \cdot I3' \cdot I4'$$

$$O2 = I1 \cdot I3' + I1' \cdot I3 \cdot I4 + I2$$

$$O3 = I1 \cdot I2 + I1 \cdot I3' + I1' \cdot I2' \cdot I4'.$$

Иногда ПЛМ должна быть запрограммирована так, чтобы на выходе постоянно были 1 или 0. Как показано на рис. 5.24, сделать это не сложно. Терм-произведение P1 всегда равен 1, потому что на входы соответствующего вентиля И не подается ни один из входных сигналов и поэтому на его выходе всегда присутствует высокий уровень; этим термом-константой, равным 1, определяется сигнал на выходе O1. На входы вентилей ИЛИ, формирующих сигнал на выходе O2, не подан ни один терм-произведение, поэтому сигнал на этом выходе всегда равен 0. Возмо-

жеи другой метод получения на выходе константы, равной 0, как показано на рисунке в отношении выхода ОЗ. На входы вентиля, формирующего терм-произведение Р2 поданы все входные переменные и их инверсии; поэтому Р2, а вместе с ним и ОЗ, всегда равны 0 ($X \cdot X' = 0$).

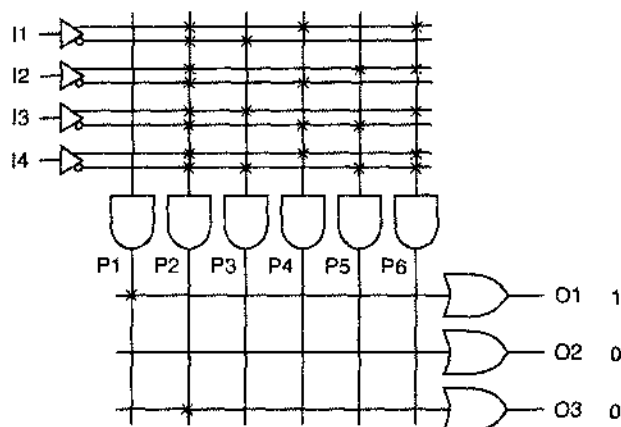


Рис. 5.24. ПЛМ размера 4х3, запрограммированная для получения на выходах констант, равных 0 и 1

В нашем примере ПЛМ имеет слишком мало входов, выходов и вентилей И (термов-произведений) чтобы быть очень полезной. В ПЛМ с n входами, в принципе, можно было бы получить до 2^n термов-произведений для реализации всех возможных минтермов с n переменными. Фактическое число термов-произведений в типичных коммерческих ПЛМ много меньше, примерно от 4 до 16 на выход, независимо от значения n .

Типичным примером ПЛМ является схема 82S100 фирмы Signetics, которая выпускалась в середине 70-х годов. У нее было 16 входов, 48 вентилей И и 8 выходов. Таким образом, в схеме было $2 \times 16 \times 48 = 1536$ плавающих перемычек в матрице вентилей И и $8 \times 48 = 384$ плавающих перемычки в матрице вентилей ИЛИ. Позже эти микросхемы были вытеснены устройствами типа PAL, CPLD и FPGA, однако внутри больших специализированных ИС для реализации сложной комбинационной логики часто создаются структуры типа ПЛМ.

МАЛОВЕРОЯТНЫЙ СБОЙ

Теоретически, когда все входные сигналы в схеме на рис. 5.24 изменяются одновременно, на выходе вентиля, формирующего терм-произведение Р2, может появиться кратковременный паразитный импульс, то есть сигнал вида 0-1-0. В типичных приложениях эта ситуация очень маловероятна, и она невозможна в том случае, когда один из входов оказывается неиспользуемым и на него подан постоянный логический сигнал.

5.3.2. Программируемые матричные логические устройства

Частным случаем ПЛИМ, и на сегодня самым распространенным типом ПЛУ, являются *программируемые матричные логические устройства PAL (programmable array logic devices)*. В отличие от ПЛИМ, в которой программируемыми являются и матрицы вентилях И, и матрицы вентилях ИЛИ, схема PAL имеет *фиксированные* соединения в матрице вентилях ИЛИ.

В первых схемах PAL, появившихся в конце 70-х годов, применялась ТТЛ-совместимая биполярная технология. Основными новшествами в первых схемах PAL, помимо броского имени (англ. pal – товарищ, приятель), были использование фиксированной матрицы вентилях ИЛИ и двунаправленные выводы входов/выходов.

Хорошей иллюстрацией этих идей служит схема PAL16L8, показанная на рис. 5.25 и 5.26; эта схема – одна из наиболее широко применяемых в настоящее время комбинационных ПЛУ. В этой схеме программируемая матрица вентилях И имеет 64 строки и 32 столбца, пронумерованные на рисунке для целей программирования мелким шрифтом, и $64 \times 32 = 2048$ плавких перемычек. Каждый из 64 вентилях И в этой матрице имеет 32 входа, предназначенных для подключения 16 входных сигналов и их инверсий; отсюда число 16 в обозначении схемы «PAL16L8».

С каждым выходным контактом микросхемы PAL16L8 связаны восемь вентилях И. Семь из них формируют входные сигналы для фиксированных 7-входовых вентилях ИЛИ. Выход восьмого вентиля И, который мы называем *вентилем разрешения выходного сигнала (output-enable gate)*, соединен с входом управления третьим состоянием выходного буфера; буфер открыт и пропускает сигнал на выход только тогда, когда на выходе вентиля разрешения выходного сигнала присутствует 1. Таким образом, микросхема PAL16L8 способна реализовать только такие логические функции, которые можно представить в виде суммы, состоящей из семи или меньшего числа термов-произведений. Каждый терм-произведение может быть функцией всех 16 входных сигналов или части из них, но возможны только семь таких термов-произведений.

ДРУЗЬЯ И ВРАГИ

PAL является зарегистрированной торговой маркой фирмы Advanced Micro Devices, Inc. Подобно другим торговым маркам, она должна использоваться только как прилагательное. Использовать аббревиатуру PAL в качестве существительного или без указания, что это торговая марка, вы можете только под свою ответственность (как я узнал об этом из письма юристов фирмы AMD в феврале 1989 года).

Имея в виду это предупреждение, я предлагаю пользоваться описательным именем, которое несет больше информации о внутренней структуре устройства: *элемент с фиксированной матрицей ИЛИ (fixed-OR element, FOE; англ. foe – враг)*.

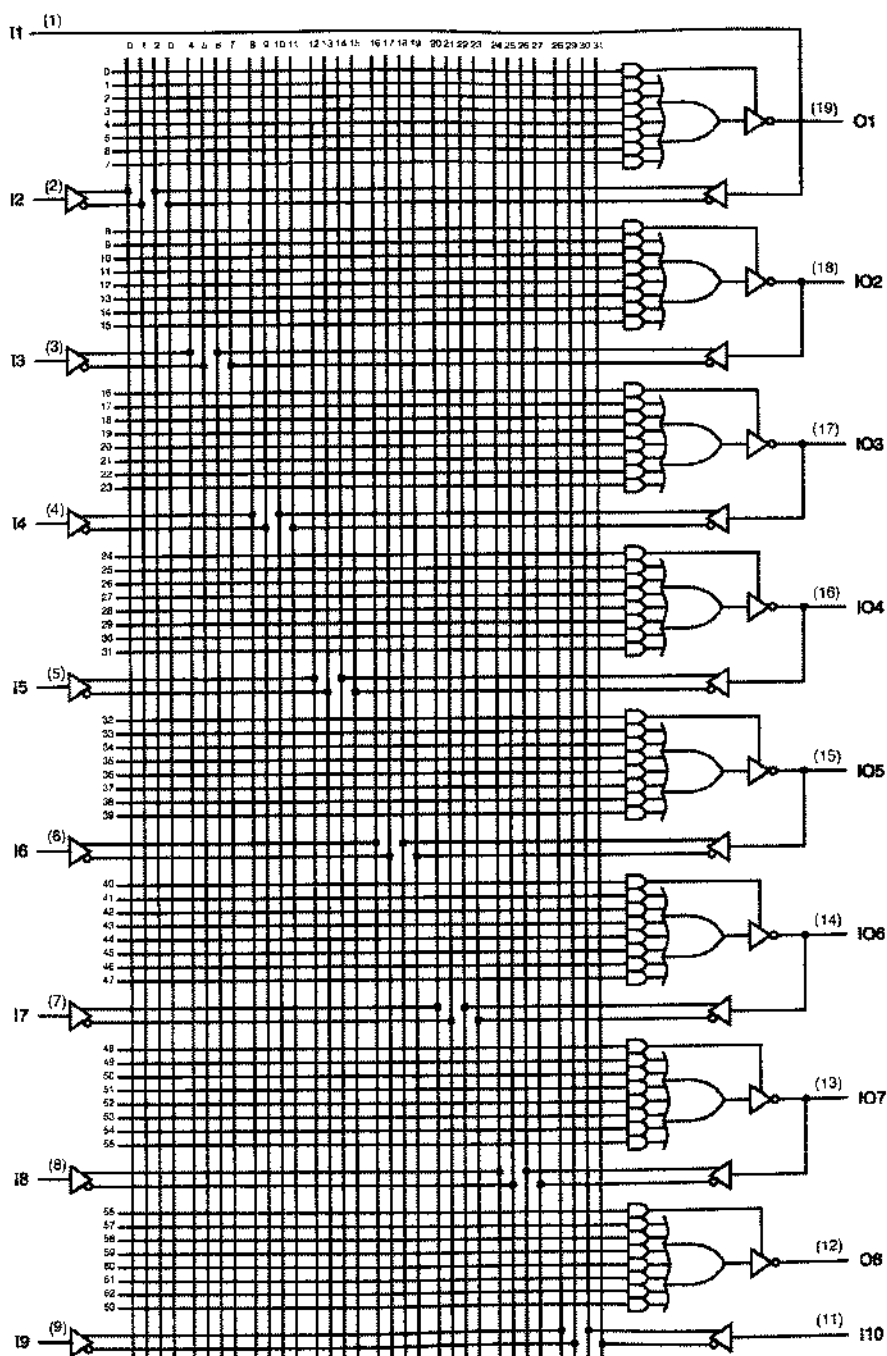


Рис. 5.25. Принципиальная схема ИС PAL16L8

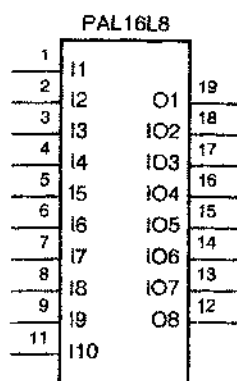


Рис. 5.26. Традиционное условное обозначение микросхемы PAL16L8

Хотя у микросхемы PAL16L8 может быть до 16 входов и до 8 выходов, она размещена в корпусе DIP всего лишь с 20 выводами, включая два вывода для подключения напряжения питания и земли (угловые выводы 10 и 20). Это достигается благодаря наличию шести двунаправленных выводов (13–18), которые можно использовать как входы или выходы, либо как то и другое. Таким образом, различия между PAL16L8 и структурой ПЛИС сводятся к следующему:

- PAL16L8 имеет фиксированную матрицу вентилях ИЛИ с семью вентилями И, постоянно соединенными с каждым из вентилях ИЛИ. Выходы вентилях И нельзя соединить с входами нескольких вентилях ИЛИ; если терм-произведение необходимо двум вентилям ИЛИ, то это необходимо сформировать дважды.
- Каждый выход микросхемы PAL16L8 может находиться в третьем состоянии и управляется индивидуальным сигналом разрешения выхода с тремя состояниями, предназначенным для этого вентилях И (вентилем разрешения выходного сигнала). Следовательно, состояние выходов можно запрограммировать так, чтобы они всегда были активны, всегда были заблокированы или управлялись бы комбинацией входных сигналов, включенных в соответствующий терм-произведение.

ДОСТАТОЧНО ЛИ СЕМИ ТЕРМОВ-ПРОИЗВЕДЕНИЙ?

Для двухуровневой структуры И–ИЛИ наилучшей логической функцией является ИСКЛЮЧАЮЩЕЕ ИЛИ (контроль четности) с n переменными; в этом случае требуется $2^n - 1$ термов-произведений. Однако часто бывает так, что менее своенравную функцию можно реализовать с помощью микросхемы PAL16L8 даже в том случае, когда у нее число термов-произведений больше 7. Для этого ее нужно преобразовать к 4-уровневому виду И–ИЛИ–И–ИЛИ, и тогда данная функция может быть реализована за два прохода сквозь матрицу И–ИЛИ. К сожалению, в результате использования выходных сигналов ПЛУ в качестве термов, образующихся на первом проходе, при этом удваивается задержка, так как входной сигнал должен дважды пройти через ПЛУ, прежде чем он достигнет выхода.

КОМБИНАЦИОННЫЙ, НЕ КОМБИНАТОРНЫЙ!

Шагом назад при популяризации микросхем PAL было введение слова «комбинаторный» для обозначения комбинационных схем. Комбинационные схемы не имеют памяти: в любой момент времени их выходные сигналы определяются текущей комбинацией входных сигналов. У образованного специалиста по компьютерам слово «комбинаторный» ассоциируется с биномиальными коэффициентами, сложностью решения задач и именем информатики Дональдом Кнпутом.

- Между выходом каждого вентиля ИЛИ и внешним выводом микросхемы PAL16L8 включен инвертор.
- Шесть из выходных выводов ИС PAL16L8, названных *I/O-выводами (I/O pins)*, можно использовать также в качестве входов. Благодаря этому возникает много возможностей использования каждого из I/O-выводов в зависимости от того, как запрограммировано устройство:
 - Если вентиль, управляющий I/O-выводом, вырабатывает постоянный сигнал, равный 0, то выходной буфер всегда находится в третьем состоянии и вывод используется строго в качестве входа.
 - Если входной сигнал из I/O-вывода не используется никакими схемами в матрице вентилей И, то вывод можно использовать строго как выход. В зависимости от того, как запрограммирован вентиль разрешения выходного сигнала, выходной буфер может быть активным всегда или только при некоторых входных условиях.
 - Если вентиль, управляющий I/O-выводом, вырабатывает постоянный сигнал, равный 1, то выходной буфер всегда активен, но данный вывод можно все же использовать также и как вход. Таким образом, выходами можно воспользоваться для образования на первом проходе «вспомогательных термов» в случае логических функций, которые не могут быть выполнены за один проход из-за ограничения по числу термов-произведений, доступных на одном выходе. В разделе 5.4.6 будет приведен соответствующий пример.
 - В другом случае, когда для данного I/O-вывода выходной сигнал постоянно разрешен, его можно использовать в качестве входного сигнала вентилей И, результатом действия которых определяется тот же самый выходной сигнал. Другими словами, в микросхеме PAL16L8 можно создать последовательностную схему с обратной связью. Этот случай мы рассмотрим в разделе 8.2.6.

Микросхема PAL20L8 является другим комбинационным ПЛУ, подобным PAL16L8, за исключением того, что корпус этой микросхемы имеет на четыре вывода больше (эти выводы работают только на вход) и каждый из ее вентилей И снабжен еще восьмью входами, позволяющими использовать дополнительные входные сигналы. Выходы у этой микросхемы организованы так же, как у схемы PAL16L8.

5.3.3 Универсальные матричные логические устройства

В параграфе 8.3 будут введены последовательностные ПЛУ – программируемые логические устройства, в которых некоторые или все выходы вентилей ИЛИ снабжены триггерами. Эти устройства можно запрограммировать для реализации ряда полезных функций, выполняемых последовательностными схемами.

Один тип последовательностных ПЛУ, впервые представленных фирмой Lattice Semiconductor и особенно популярных, назван *универсальными матричными логическими устройствами GAL (generic array logic device)*. Единственное устройство GAL типа *GAL16V8* можно сконфигурировать (путем программирования и создания соответствующих соединений) так, чтобы имитировались схема вида И–ИЛИ, триггеры и выходные цепи, встречающиеся во всем многообразии комбинационных и последовательностных устройств PAL, включая уже рассмотренную нами микросхему PAL16L8. Более того, конфигурация GAL может быть электрически стерта и перепрограммирована.

На рис. 5.27 показана принципиальная схема ИС GAL16V8, сконфигурированной как исключительно комбинационное устройство, подобное PAL16L8. Эта конфигурация достигается программированием двух не показанных на рисунке соединений, «управляющих архитектурой». В изображенной конфигурации устройство носит название *GAL16V8C*.

Самое важное, что следует отметить при сравнении ИС GAL16V8C с ИС PAL16L8, состоит в том, что между каждым выходом вентиля ИЛИ и выходным буфером с тремя состояниями включен вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ. Один из входов вентиля ИСКЛЮЧАЮЩЕЕ ИЛИ может быть «подтянут» к уровню логической 1, но плавкой перемычкой соединен с землей (0 В). Если эта перемычка сохранена, то вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ просто пропускает без изменений сигнал, поступающий с выхода схемы ИЛИ; но если перемычку искрежить, то вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ инвертирует сигнал, поступающий с выхода схемы ИЛИ. Говорят, что эта плавкая перемычка управляет *полярностью выходного сигнала (output polarity)* на соответствующем выходном контакте.

БЫСТРОДЕЙСТВИЕ КОМБИНАЦИОННЫХ ПЛУ

Быстродействие комбинационных ПЛУ обычно выражается одним числом t_{PD} , характеризующим задержку прохождения сигнала от любого входа до любого выхода при произвольном направлении исклечения. Выпускаются ПЛУ с различным быстродействием; широко распространены микросхемы с задержкой 10 нс. В 1998 году самыми быстрыми были комбинационные ПЛУ на биполярных транзисторах PAL16L8 с задержкой 5 нс и 3.3-вольтовые ПЛУ на КМОП-транзисторах GAL22LV10 с задержкой 3.5 нс.

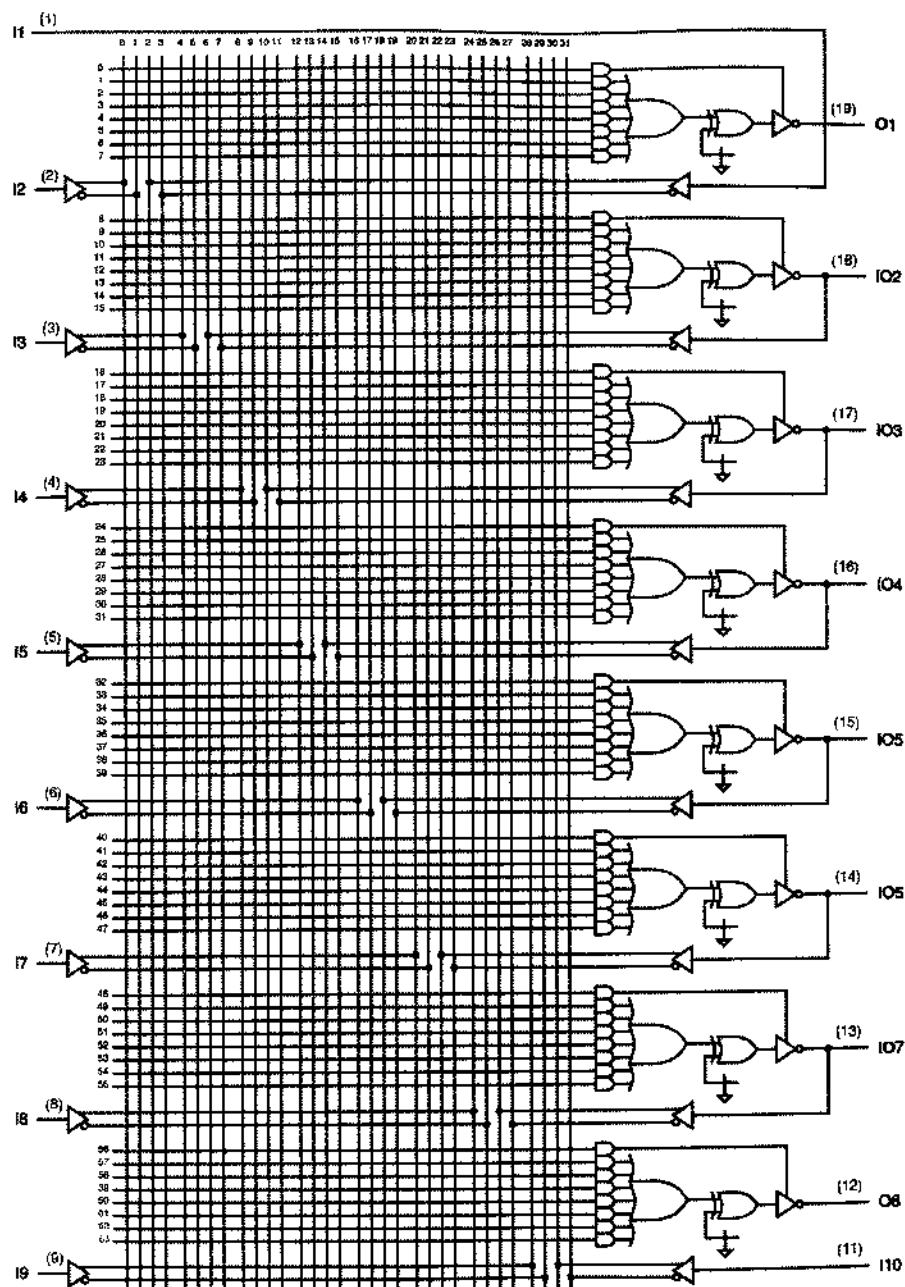


Рис. 5.27. Принципиальная схема ИС GAL16V8C

Возможность управления полярностью выходного сигнала является очень важным свойством современных ПЛТУ, в том числе и микросхемы GAL16V8. Мы видели в разделе 4.6.2, что при минимизации заданной логической функции компля-

тор языка ABEL находит минимальные выражения вида «сумма произведений» как для самой функции, так и для ее инверсии. В случае, когда инверсия имеет меньшее число термов-произведений, этим можно воспользоваться, если переключку, управляющую полярностью соответствующего выходного сигнала в схеме GAL16V8, разрушить. Транслятор автоматически выбирает лучший вариант и соответствующим образом решает, какие плавкие переключки следует перечеркнуть, если эта операция не отменена.

Несколько фирм выпускают микросхему *PALCE16V8*, которая является эквивалентом *GAL16V8*. Имеются также микросхемы *GAL20V8* или *PALCE20V8* в корпусе с 24 выводами, которые можно сконфигурировать так, чтобы они имитировали структуру схемы *PAL20L8* или какой-либо другой схемы из числа последовательностных ПЛУ, рассматриваемых в разделе 8.3.2.

ОФИЦИАЛЬНОЕ ПРЕДУПРЕЖДЕНИЕ

GAL является торговой маркой фирмы Lattice Semiconductor, Hillsboro, OR 97124.

*5.3.4. Схемы биполярных ПЛУ

Для построения и физического программирования ПЛУ применяется несколько различных технологий. В первых коммерческих ПЛМ и устройствах PAL были использованы схемы на биполярных транзисторах. В качестве примера на рис. 5.28 показано, как можно построить приведенную в разделе 5.3.1 ПЛМ размера 4×3 на основе биполярной TTL-подобной технологии. Каждое возможное соединение реализовано в виде последовательно включенных диода и металлического соединения, которое может присутствовать или отсутствовать. Если соединение имеется, то диод подключает соответствующий ему вход к диодной схеме И. Если соединение отсутствует, то соответствующий вход не оказывает никакого влияния на эту схему И.

Диодная схема И предназначена для того, чтобы на вертикальной «линии И» высокий уровень возникал только в том случае, когда на всех до единой горизонтальных «входных линиях», подключенных через диод к данной линии И, имеется высокий уровень. Если какая-то из входных линий имеет низкий уровень, то это приводит к низкому уровню на всех линиях И, с которыми соединена данная входная линия. Эта первая совокупность схемных элементов, реализующих функции И, которая называется *матрицей И (AND plane)*.

За каждой линией И следует инвертирующий буфер, так что в целом реализуется функция И-НЕ. Выходы схем И-НЕ первого уровня объединяются другим набором программируемых диодных схем И, за которыми снова следуют инверторы. В результате мы имеем двухуровневую структуру И-НЕ-И-НЕ, которая является функциональным эквивалентом описанной ранее ПЛМ со структурой И-ИЛИ. Совокупность схемных элементов, реализующих функцию ИЛИ (или образующих второй уровень И-НЕ – в зависимости от того, как на это посмотреть), называется *матрицей ИЛИ (OR plane)*. При изготовлении кристалла биполярного ПЛУ предус-

матрируется наличие в нем всех диодов, включенных последовательно с крошечными пережигаемыми перемычками (*fusible link*) (на рис. 5.28 они изображены небольшими волнистыми линиями). Применяя специальные трафареты, можно выбрать отдельные перемычки, приложить к ним высокое напряжение (10–30 В), и таким образом выбранную перемычку испарить.

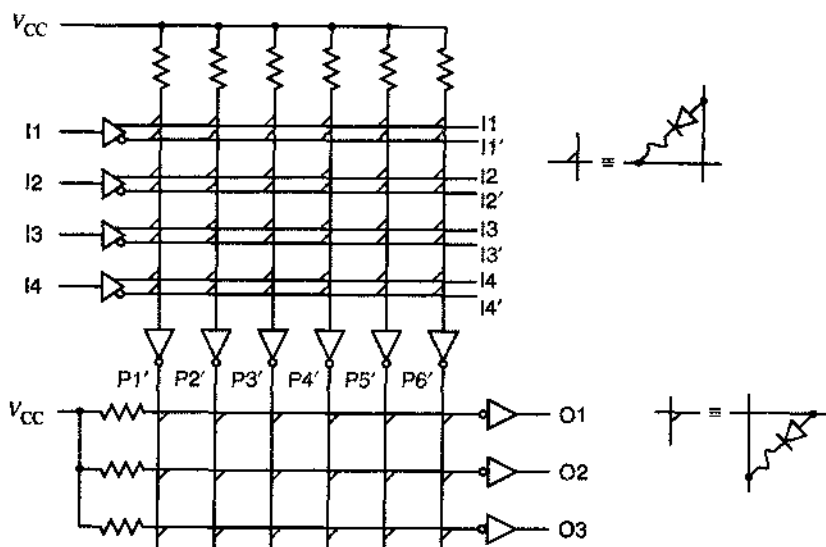


Рис. 5.28. ПЛМ размера 4×3 на основе ТТЛ-подобных схем с открытым коллектором и диодной логики

Первые биполярные ПЛУ были не очень надежны. Иногда зафиксированная конфигурация изменялась из-за не полностью испаренной перемычки, которая могла «снова вырасти», а иногда причиной периодических отказов были плавающие капельки внутри корпуса ИС. Однако в дальнейшем эти проблемы в значительной степени были устранены, и надежная технология с плавкими перемычками применяется в биполярных ПЛУ и сегодня.

*5.3.5 Схемы ПЛУ на основе КМОП-логики

Хотя биполярные ПЛУ все еще остаются доступными, они в значительной степени оказались вытесненными ПЛУ на основе КМОП-логики, которые обладают рядом преимуществ, в том числе меньшей потребляемой мощностью и возможностью перепрограммирования. На рис. 5.29 показан КМОП-вариант ПЛМ размера 4×3 из раздела 5.3.1.

В каждом пересечении входной линии с «линией слова» вместо диода помещен n -канальный транзистор с программируемым подключением. Если сигнал на входе имеет низкий уровень, то транзистор «закрыт», а если входной сигнал имеет высокий уровень, то транзистор «открыт», что приводит к появлению на линии И низкого уровня. В результате получаем схему И с инверсией на входе (то есть схему ИЛИ-НЕ). По своей структуре и выполняемой функции эта часть схемы

подобно обычному КМОП-вентилю ИЛИ-НЕ с k входами, за исключением того, что обычное последовательное соединение k транзисторов с каналом p -типа, обеспечивающих высокий уровень на выходе, заменено резистором (на самом деле высокий уровень на линии И в ПЛМ обеспечивается одним постоянно открытым p -канальным транзистором).

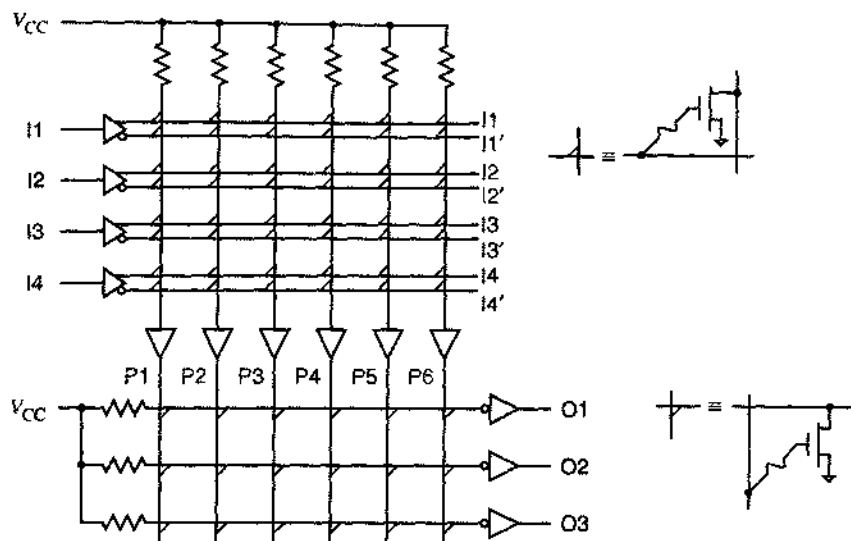


Рис. 5.29. ПЛМ размера 4×3 на основе КМОП-логики

Из рис. 5.29 следует, что — в отличие от схемы на рис. 5.28 — применение вентилей И с инверсией на входе нейтрализуется использованием входных линий с инверсными значениями значений сигналов для каждого входа. Обратите внимание также на то, что соединение между матрицей И и матрицей ИЛИ является неинвертирующим, так что матрица И реализует истинную функцию И.

Выходы линий И первого уровня объединяются в матрице ИЛИ другим набором программируемых соединений, реализующих функцию ИЛИ-НЕ. На выходе каждой линии ИЛИ-НЕ включен инвертор, так что в результате получаем истинную функцию ИЛИ, а в целом ПЛМ реализует функцию И-ИЛИ, что и требовалось.

В ПЛУ, изготавливаемых по КМОП-технологии, программируемые перемычки, показанные на рис. 5.29, первоначально бывают не расплавлены. В устройствах, не программируемых в процессе работы, типа заказных СБИС наличие или отсутствие отдельных перемычек определяется маской металлизации при изготовлении устройства. Однако в КМОП-схемах типа EPLD, рассматриваемых ниже, чаще всего, безусловно, применяется другая технология программирования.

В стираемом программируемом логическом устройстве (*erasable programmable logic device, EPLD*) можно запрограммировать любую желаемую конфигурацию связей, по можно также вернуть устройство в его первоначальное состояние, «стирая» эти связи электронным путем или облучая ультрафиолетовым светом. Нет, стирание не вызывает внезапного появления или исчезновения

связей! Правильнее сказать, что в устройствах EPLD применяется другая технология, называемая «МОП-структура с плавающим затвором».

Как показано на рис. 5.30, в схемах EPLD используются МОП-транзисторы с плавающим затвором (*floating-gate MOS transistor*). Такой транзистор имеет два затвора. «Плавающий» затвор ни к чему не подключен и окружен диэлектриком с очень малой проводимостью. В исходном состоянии в плавающем затворе нет никакого заряда, и он не влияет на работу схемы. В этом состоянии все транзисторы фактически «открыты»; то есть во всех точках пересечения линий в матрицах И и ИЛИ имеется логическая связь.

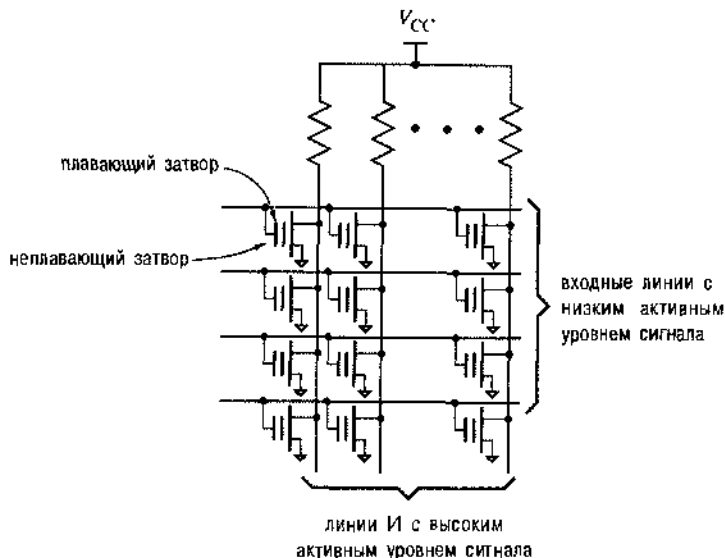


Рис. 5.30. Матрица И в перепрограммируемом логическом устройстве типа EPLD на основе МОП-транзисторов с плавающим затвором

Программирование схем EPLD осуществляется с помощью программатора: к неплавающему затвору в каждом месте, где связь не требуется, прикладывается высокое напряжение. Это вызывает временный пробой в диэлектрике, что позволяет отрицательному заряду накопиться в плавающем затворе. Когда высокое напряжение снимается, в плавающем затворе остается отрицательный заряд. При выполнении последующих операций отрицательный заряд препятствует «открыванию» транзистора, когда на неплавающий затвор поступает сигнал высокого уровня; транзистор оказывается фактически отключенным от схемы.

Производители схем EPLD утверждают, что надлежащим образом запрограммированная ячейка сохраняет 70% своего заряда, по крайней мере, в течение 10 лет, даже в том случае, если микросхема будет храниться при температуре 125°C. Поэтому в большинстве приложений можно считать, что результат программирования сохраняется постоянно. Однако запрограммированную в схеме EPLD конфигурацию можно стереть.

Хотя некоторые первые микросхемы EPLD помещались в корпус с прозрачной крышкой и для стирания использовали свет, наиболее популярные современные

устройства являются электрически стираемыми программируемыми логическими устройствами (*electrically erasable PLD*). Плавающие затворы в электрически стираемом ПЛУ окружены очень тонким слоем диэлектрика, и заряд можно удалить из них путем подачи на неплавящийся затвор напряжения, полярность которого противоположна полярности, необходимой для накопления заряда в плавающем затворе. Таким образом, тот же самый программатор, который обычно используется для программирования ПЛУ, можно применить также для стирания схемы EPLD перед программированием ее заново.

В больших по размерам «сложных» ПЛУ (*"complex" PLD, CPLD*) также применяется метод программирования с плавающим затвором. Даже в еще больших устройствах, часто называемых *перепрограммируемыми вентилями матрицы* (*field-programmable gate arrays, FPGA*), для управления каждым соединением используют ячейки оперативного запоминающего устройства (ОЗУ). Ячейки ОЗУ являются энергозависимыми: они не сохраняют свое состояние при выключении питания. Поэтому при подаче на FPGA напряжения питания в его ОЗУ необходимо занести конфигурацию, хранящуюся отдельно во внешней энергонезависимой памяти. Такой памятью обычно служит программируемое постоянное запоминающее устройство (ППЗУ), подключенное непосредственно к схеме FPGA, или память микропроцессорной подсистемы, загружающей схему FPGA при инициализации системы в целом.

*5.3.6. Программирование и тестирование микросхем

Для разрушения плавких перемычек, накопления заряда в плавающих затворах транзисторов или для осуществления каких-то других действий, необходимых для программирования ПЛУ, применяется специальное оборудование. Эта аппаратура, имеющаяся в настоящее время почти во всех лабораториях, разрабатывающих и создающих цифровые устройства, называется *программаторами ПЛУ (PLD programmer)* или *программаторами ПЗУ (PROM programmer)*. [Программаторы можно применять как для записи в программируемые постоянные запоминающие устройства (ППЗУ), так и для программирования ПЛУ.] Типичный программатор ПЛУ имеет разъем или разъемы, с помощью которых подключаются программируемые устройства, и канал «загрузки» желаемой конфигурации соединений в программатор. Обычно загрузка осуществляется из подключенного к программатору персонального компьютера.

Как правило, при программировании микросхема ПЛУ переводится программатором в специальный режим работы. Например, применительно к описанным в этой главе ПЛУ программирование соединений осуществляется по отношению к восьми плавким перемычкам одновременно, и происходит это следующим образом:

1. Для перевода микросхемы в режим программирования на один из специально предназначенных для этого выводов подается высокое напряжение (порядка 14 В).
2. Путем подачи двоичного «адреса» на определенные входы микросхемы, выбирается группа из восьми плавких перемычек. (Например, в микросхеме 82S100 имеется 1920 перемычек, и поэтому ей требуется 8 входов, чтобы выбрать одну из 240 групп, по 8 перемычек в каждой.)

ИЗМЕНЕНИЕ «ЖЕЛЕЗА» НА ЛЕТУ

В типичном случае нужная конфигурация соединений заносится в ОЗУ, входящее в состав FPGA, из ПЗУ, но существуют такие приложения, где конфигурация соединений фактически считывается с дискеты. Вы только что получили дискету с новой версией программы? Считайте, что вы только что получили также новый вариант аппаратуры!

Эта концепция приводит нас к захватывающей идее, уже использованной в некоторых приложениях, а именно – к созданию «перестраиваемых аппаратных средств», когда аппаратная часть перестраивается «на лету» с целью оптимизировать ее параметры применительно к решаемой в данный момент конкретной задаче.

3. На выходы микросхемы подается 8-разрядное число, задающее желаемый результат программирования для каждой из выбранных перемычек (выходы в режиме программирования используются как входы).
4. На некоторое время (порядка 100 микросекунд) увеличивается напряжение на другом специально предназначенном для этого выводе для программирования выбранных восьми плавких перемычек.
5. Напряжение на втором специальном выводе уменьшается (до 0 В), чтобы программатор мог выполнить считывание и проверить правильность программирования выбранных восьми плавких перемычек.
6. Шаги с 1 по 5 повторяются для каждой группы из восьми плавких перемычек.

Многие ПЛУ – в частности, самые большие схемы CPLD – обладают свойством *программируемости в системе (in-system programmability)*. Это означает, что устройство может быть запрограммировано после того, как оно уже запаяно в систему. В этом случае конфигурация разрушаемых перемычек вводится последовательно с помощью четырех дополнительных сигналов и выводов, называемых *портом JTAG (JTAG port, JTAG – Joint Test Automation Group)*, который определен стандартом IEEE 1149.1. Эти сигналы позволяют составить из различных устройств на данной печатной плате «цепочку последовательного опроса» («daisy chain») для выбора и программирования в процессе изготовления платы через единственный специальный разъем порта JTAG. При этом не требуется никакого специального высоковольтного источника питания; в каждом устройстве для получения высокого напряжения, необходимого при программировании, применяется внутренняя схема накачки заряда.

Как было отмечено выше, на 5-м шаге проверяется правильность программирования выбранных плавких перемычек. Если после первого программирования перемычек обнаружены ошибки, то операцию можно повторить; если ошибки обнаруживаются после нескольких попыток, то микросхема бракуется (часто с большим пристрастием и желанием нанести ей вред).

При проверке конфигурации запрограммированного устройства подтверждение того факта, что плавкие перемычки установлены должным образом, еще не

доказывает, что устройство будет выполнять логическую функцию, соответствующую установленной конфигурации перемычек. Это происходит потому, что устройство может иметь не связанные с программированием внутренние дефекты, такие как отсутствие соединений между плавкими перемычками и элементами решетки И-ИЛИ.

Единственный способ обнаружить все дефекты состоит в том, чтобы поставить устройство в нормальный режим работы, подать на входы набор нормальных логических сигналов и наблюдать сигналы на выходах. Соответствующие наборы входных и выходных сигналов, называемые тестовыми векторами, могут быть заданы разработчиком, как мы видели в разделе 4.6.7, или могут быть образованы автоматически в соответствии со специальной программой генерирования тестовых векторов. Независимо от того, как получены тестовые векторы, у большинства программаторов есть возможность подавать входные тестовые векторы на ПЛУ и сравнивать выходные сигналы с ожидаемыми.

Большинство ПЛУ имеют *защиту конфигурации соединений (security fuse)*, которая, будучи установлена, блокирует возможность чтения конфигурации перемычек в устройстве. Производители могут запрограммировать схему так, чтобы никто не мог считать из ПЛУ конфигурацию перемычек с целью копирования устройства. Тем не менее, даже если установлена защита конфигурации соединений, тестовые векторы все же работают, так что проверка ПЛУ возможна.

5.4. Дешифраторы

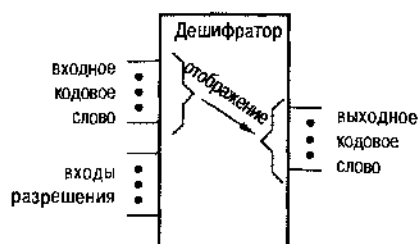
Дешифратор (decoder) — это логическая схема с несколькими входами и несколькими выходами, которая преобразует кодированные входные сигналы в кодированные выходные сигналы, причем входные и выходные коды различны. Входной код обычно имеет меньшее число разрядов, чем выходной код, и между входными и выходными кодовыми словами имеется взаимно-однозначное соответствие. При *взаимно-однозначном соответствии (one-to-one mapping)* каждое входное кодовое слово порождает отличное от других выходное кодовое слово.

Общая структура декодера приведена на рис. 5.31. Для того чтобы дешифратор нормально выполнял функцию отображения, необходимо подать сигналы на входы разрешения, если таковые имеются. Иначе дешифратор отображает все входные кодовые слова кода в единственное «запрещенное» выходное кодовое слово.

В большинстве случаев роль входного кода играет n -разрядный двоичный код, где n -разрядное двоичное слово представляет одну из 2^n различных кодированных величин. Обычно это целые числа от 0 до $2^n - 1$. Иногда, для представления меньшего, чем 2^n , числа величин, применяют усеченный n -разрядный двоичный код. Например, в двоично-десятичном коде 4-разрядные комбинации от 0000 до 1001 представляют десятичные цифры от 0 до 9, а комбинации от 1010 до 1111 не используются.

В большинстве случаев роль выходного кода играет m -разрядный код «1 из m », у которого в любой момент времени отличен от нуля один бит. Таким образом, в коде «1 из 4» с высоким активным уровнем сигнала на выходах кодовые слова имеют вид: 0001, 0010, 0100 и 1000. При низком активном уровне сигнала на выходах кодовые слова имеют вид: 1110, 1101, 1011 и 0111.

Рис. 5.31. Схематическое изображение дешифратора



5.4.1. Полные дешифраторы

Самым распространенным является дешифратор $n \times 2^n$ или *полный дешифратор* (*binary decoder*). На вход такого дешифратора поступает n -разрядное двоичное кодовое слово, а на выходе возникает слово кода «1 из 2^n ». Полный дешифратор применяется в том случае, когда необходимо активизировать точно один из 2^n выходов, определяемый n -разрядным двоичным числом на входе.

На рис. 5.32(а), например, указаны входы и выходы, а в табл. 5.4 приведена таблица истинности дешифратора 2×4 . Входное кодовое слово $I1, I0$ представляет собой целое число из интервала от 0 до 3. Выходные кодовые слова $Y3, Y2, Y1, Y0$ формируются по следующему правилу: Y_i равно 1 только в том случае, когда входное кодовое слово является двоичным представлением числа i и *входной сигнал разрешения* (*enable input*) EN равен 1. Если $EN = 0$, то на всех выходах устанавливается логический 0. На рис. 5.32(б) показана схема дешифратора 2×4 на уровне вентилей. Каждый вентиль И декодирует (*decode*) одну комбинацию входного кодового слова $I1, I0$.

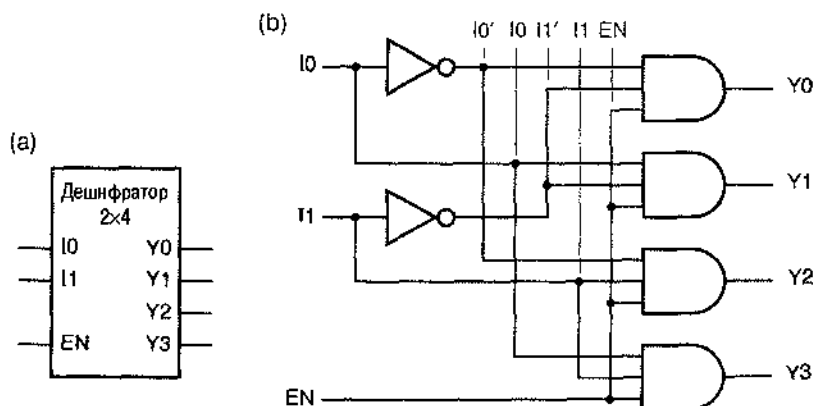


Рис. 5.32. Дешифратор 2×4 (а) входы и выходы, (б) принципиальная схема

В таблице истинности полного дешифратора среди входных комбинаций фигурирует символ «безразличного» значения. Если одно или большее число входных величин не влияют на значения выходных сигналов при какой-то определенной комбинации сигналов на остальных входах, то такие входные сигналы в данной комбинации отмечаются символом «х». Это правило позволяет значительно уменьшить число строк в таблице истинности, а также делает более ясной функцию, выполняемую входными сигналами.

Табл. 5.4. Таблица истинности для полного дешифратора 2×4

Входы			Выходы			
EN	I1	I0	Y3	Y2	Y1	Y0
0	x	x	0	0	0	0
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

n -разрядные входные кодовые слова полного дешифратора не обязательно должны представлять собой целые числа от 0 до $2^n - 1$. В табл. 5.5, например, приводятся выходные сигналы механического кодирующего диска на восемь положений, которые представляют собой 3-разрядный код Грея. Восемь позиций диска можно декодировать 3-разрядным полным дешифратором присвоив сигналам на выходах дешифратора соответствующие значения, как показано на рис. 5.33.

Табл. 5.5. Позиционное кодирование 3-разрядным механическим кодирующим диском

Положение диска	I2	I1	I0	Двоичный выход дешифратора
0°	0	0	0	Y0
45°	0	0	1	Y1
90°	0	1	1	Y3
135°	0	1	0	Y2
180°	1	1	0	Y6
225°	1	1	1	Y7
270°	1	0	1	Y5
315°	1	0	0	Y4

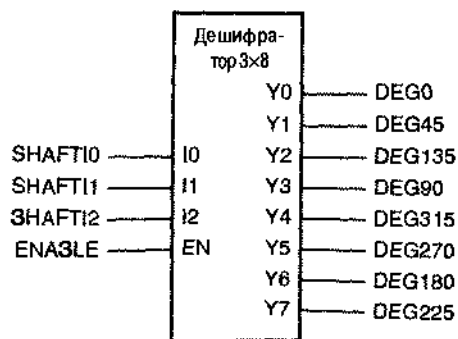


Рис. 5.33. Применение полного дешифратора 3×8 для декодирования кода Грея

Кроме того, нет необходимости использовать все выходы дешифратора, или даже декодировать все возможные входные комбинации. Например, *десятичный дешифратор (decimal decoder)* или *двоично-десятичный дешифратор (BCD decoder)* декодирует только первые десять входных двоичных комбинаций 0000–1001, формируя на выходе сигналы Y0–Y9.

5.4.2. Условные обозначения крупных логических элементов

Перед описанием некоторых серийно выпускаемых дешифраторов 74-й серии, являющихся ИС средней степени интеграции, следует обсудить общие рекомендации по графическому изображению крупных логических элементов.

Самое главное правило состоит в том, что в условном обозначении этих элементов входы рисуются слева, а выходы – справа. К верхней и нижней стороне условного изображения сигналы обычно не подводятся. Однако иногда сверху и снизу в явном виде изображают выводы для подключения напряжения питания и земли, особенно в том случае, когда они выведены в «нестандартных» местах. (У большинства СИС напряжение питания и земля подключаются к выводам, расположенным по углам корпуса; например, у микросхем в корпусе DIP с 16 выводами это выводы 8 и 16.)

Подобно обозначениям вентилях, в условных обозначениях более крупных логических элементов с каждым выводом связывают тот или иной активный уровень сигнала. Что касается активных уровней, то здесь важно использовать согласованные обозначения внутренних сигналов и внешних выводов.

ЛОГИЧЕСКИЕ СЕМЕЙСТВА

Большинство наборов логических вентилях и более крупных схем, многие из которых были описаны в параграфах 3.8 и 3.11, имеются в различных КМОП- и ТТЛ-семействах. В качестве примера можно привести микросхемы 74LS139, 74S139, 74ALS139, 74AS139, 74F139, 74HC139, 74HCT139, 74ACT139, 74AC139, 74FCT139, 74AHC139, 74AHCT139, 74LC139, 74LVC139 и 74VHC139. Все они являются полными дешифраторами 2×4, выполняющими одну и ту же логическую функцию, но реализованы в различных по своим электрическим характеристикам ТТЛ- и КМОП-семействах, причем иногда в разных корпусах. Кроме того, логические «макро»-элементы с теми же самыми именами выводов и теми же функциями, что и 74S139 и другие популярны схемы 74-й серии, в качестве стандартных блоков имеются в большинстве библиотек в среде проектирования схем FPGA и специализированных ИС.

Повсюду в этой книге обозначение «74х» применяется как универсальный префикс. Поэтому иногда мы будем его опускать и писать, например, 74S139. В реальной принципиальной схеме устройства, которое вы собираетесь построить или моделировать, необходимо указывать полное название микросхемы, так как временные параметры, нагрузка и корпус зависят от выбранного семейства.

Как объяснялось в разделе 5.1.4, у больших логических элементов имена сигналов почти всегда выбираются по названиям функций, выполняемых *внутри* этих элементов. На рис. 5.34(а), например, приведено условное обозначение одного из двух дешифраторов 2×4 в микросхеме средней степени интеграции 74х139, которая будет подробно описана в следующем разделе. Когда сигнал на входе G переходит на активный уровень, возникает сигнал на одном из выходов Y0–Y3, номер которого определяется 2-разрядным кодом, поданным на входы A и B. Условное обозначение наглядно говорит о том, что сигналы на входе G и на всех выходных выводах имеют низкий активный уровень.

Когда условное обозначение микросхемы 74х139 фигурирует в принципиальной схеме реального устройства, каждый из сигналов на ее входах и выходах, связанных с другими микросхемами, имеет имя, которое указывает его функцию в этом устройстве. Однако при рассмотрении микросхемы 74х139 отдельно тоже удобно, чтобы каждому внешнему выводу соответствовало какое-то имя сигнала. Рис. 5.34(б) демонстрирует выбранные нами в данном случае имена. Выводам с высоким активным уровнем сигнала дают то же самое имя, какое имеет внутренний сигнал, в то время как выводы с низким активным уровнем сигнала получают имя внутреннего сигнала с добавлением суффикса «_L».

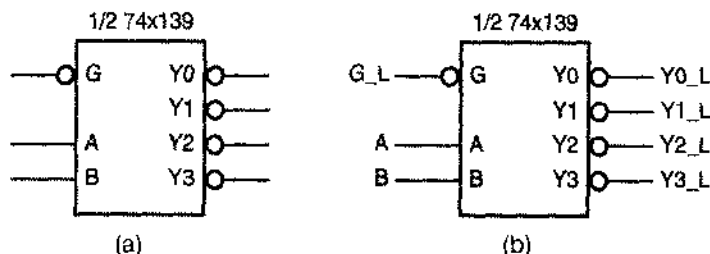


Рис. 5.34. Условное обозначение половины двойного дешифратора 2×4 типа 74х139 : (а) условное обозначение; (б) имена сигналов по умолчанию на внешних выводах

СТАНДАРТ IEEE НА УСЛОВНЫЕ ОБОЗНАЧЕНИЯ

Повсюду в этой книге мы применяем «традиционные» обозначения больших логических элементов. Стандарт IEEE предусматривает несколько различных вариантов обозначения больших логических элементов. Условные обозначения стандарта IEEE, а также все «за» и «против» и сравнение этого стандарта с традиционными обозначениями приведены в руководстве, доступном в Интернете на сайте www.ddpp.com.

5.4.3. Сдвоенный дешифратор 2×4 типа 74х139

В одном корпусе микросхемы средней степени интеграции 74х139 находятся два независимых одинаковых дешифратора 2×4. Принципиальная схема этой ИС на уровне вентилях приведена на рис. 5.35(а). Обратите внимание, что выходы и вход разрешения микросхемы 74х139 имеют низкий активный уровень. Большинство де-

шифраторов средней степени интеграции первоначально были разработаны с низкими активными уровнями выходных сигналов, так как инвертирующие ТТЛ-вентили обычно работают быстрее неинвертирующих. Обратите также внимание на то, что микросхемы 74х139 имеют дополнительные инверторы на входах разрешения. Без этих инверторов каждый такой вход представлял бы собой нагрузку на переменному или по постоянному току, эквивалентную входам четырех вентилях, а не одного, существенно ограничивая число устройств, которые можно было бы подключить к выходу устройства, управляющего работой дешифратора.

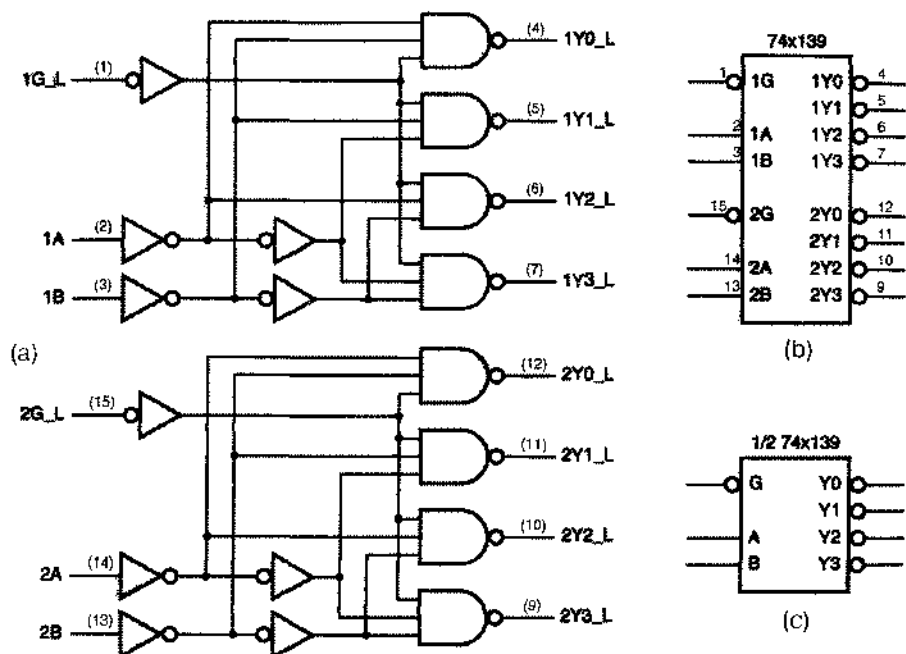


Рис. 5.35. Сдвоенный дешифратор 2×4 типа 74х139 (а) принципиальная схема с цоколевкой стандартного корпуса DIP с 16 выводами, (б) традиционное условное обозначение, (с) условное обозначение одного дешифратора

Табл. 5.6. Таблица истинности для половины сдвоенного дешифратора 2×4 типа 74х139

Входы			Выходы			
G_L	B	A	$Y3_L$	$Y2_L$	$Y1_L$	$Y0_L$
1	x	x	1	1	1	1
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1

На рис. 5.35(b) приведено условное обозначение микросхемы 74х139. Обратите внимание, что все имена сигналов внутри схемы имеют высокий активный уровень (отсутствует суффикс "_L"), а кружки инверсии указывают на низкий активный уровень входных и выходных сигналов. Часто на принципиальной схеме можно использовать условное обозначение только одной половины микросхемы '139, как показано на рис. 5.35(c). В этом случае выбор той или другой половины конкретной микросхемы '139 можно отложить до завершения работы над схемой в целом.

Табл. 5.6 представляет собой таблицу истинности дешифратора типа 74х139. В справочниках некоторых производителей в таблицах истинности для обозначения уровня напряжения входного и выходного сигнала применяют буквы L и H, так что здесь не может быть никакой неоднозначности относительно функции, реализуемой устройством, записавшую таким образом таблицу истинности иногда называют *функциональной таблицей* (*function table*). Однако, поскольку повсюду в этой книге используется положительная логика, мы можем, не опасаясь неоднозначности, использовать 0 и 1. В любом случае таблица истинности дает описание логической функции на основе *внешнего обозначения выводов* устройства. Таблица истинности для функции, реализуемой *внутри* условного обозначения, выглядела бы точно так же, как в табл. 5.4, за исключением того, что входные сигналы имели бы имена G, B, A.

Некоторые разработчики логических устройств изображают микросхему 74х139 и другие логические схемы без кружков инверсии. Вместо этого они используют подчеркивание названия сигнала внутри символа, чтобы указать отрицание, как это сделано на рис. 5.36(a). Эта система обозначений самосогласованна, но несовместима с нашими стандартами изображения принципиальных схем, к которым применим принцип проектирования «инверсия к инверсии». Условное обозначение, представленное на рис. 5.36(b), абсолютно *неверно*: согласно этому обозначению, для работы дешифратора к входу разрешения должна быть приложена логическая 1, а не 0.

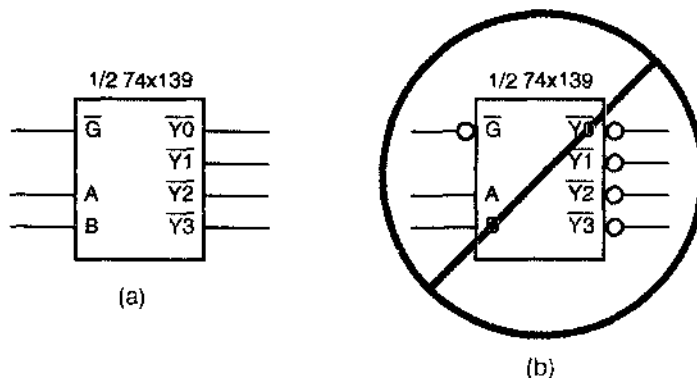


Рис. 5.36. Другие варианты обозначения микросхемы 74х139 (a) правильный, но нежелательный вариант, (b) неправильный вариант из-за наличия двойных отрицаний

ПЛОХИЕ ИМЕНА

В справочниках некоторых производителей имеются противоречия, подобные изображенным на рис. 5.36(b). Например, в справочных данных фирмы Texas Instruments для микросхемы 74AHC139 применяются имена сигналов с низким активным уровнем типа $\overline{1G}$ для входов разрешения, с надчеркиванием для указания низкого активного уровня сигнала на этом выводе, но для всех выходных выводов с низким активным уровнем сигнала используются обозначения типа Y0, как для сигнала с высоким активным уровнем. С другой стороны, в справочных данных фирмы Motorola для микросхемы 74VHC139 надчеркивание в именах как для входов разрешения, так и для выходов используется правильно, но эти надчеркивания едва видны в функциональной таблице устройства из-за проблем с типографией.

У меня есть личный опыт создания печатной платы с большим числом новых микросхем от поставщика, в документации которого было ясно указано, что вход разрешения имеет низкий активный уровень, и только при первом включении выяснилось, что активный уровень на этом входе высокий.

Мораль этой истории состоит в том, что вы должны изучить описание каждого компонента, чтобы знать, как он реально работает. И если это совершенно новое устройство, то либо с помощью поставщика, либо с помощью группы по специализированным ИС вашей собственной компании, следует перепроверить все полярности сигналов и назначение выводов до того, как вы приступите к изготовлению печатной платы. Однако даю полную гарантию, что обозначения сигналов в этой книге *непротиворечивы и правильны*.

5.4.4 Дешифратор 3x8 типа 74x138

Имеющаяся в продаже ИС 74x138, принципиальная схема которой на уровне вентилей и условное обозначение приведены на рис. 5.37, является дешифратором 3x8; таблица истинности этого дешифратора представлена в табл. 5.7. Подобно микросхеме 74x139, ИС 74x138 имеет низкий активный уровень выходных сигналов и три входа разрешения ($G1$, $G2A_L$, $G2B_L$); для того, чтобы сигнал на выбранном выходе имел активный уровень, на все три входа разрешения должны быть поданы сигналы активного уровня.

Логическая функция дешифратора 138 проста: уровень сигнала на некотором выходе становится активным только в том случае, когда на управляющие входы дешифратора поданы разрешающие сигналы и выбран этот выход. Таким образом, легко записать логические соотношения, выражающие внутренние выходные сигналы через внутренние входные сигналы; например, для сигнала Y5 имеем:

$$Y5 = \underbrace{G1 \cdot G2A \cdot G2B}_{\text{разрешение}} \cdot \underbrace{C \cdot B' \cdot A}_{\text{выбор}}$$

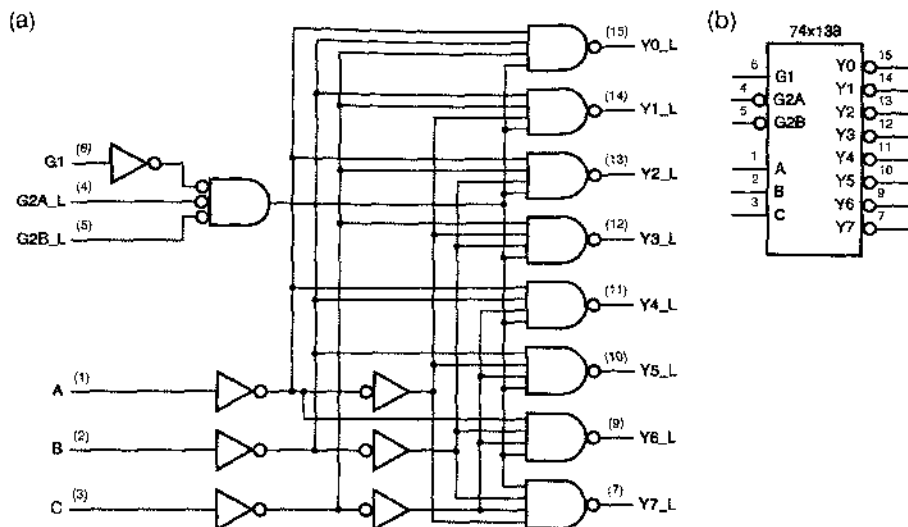


Рис. 5.37. Дешифратор 3х8 типа 74х138: (а) принципиальная схема с цоколевкой для стандартного корпуса DIP с 16 выводами; (б) традиционное условное обозначение

Табл. 5.7. Таблица истинности для дешифратора 3х8 типа 74х138

Входы						Выходы							
G1	G2A_L	G2B_L	C	B	A	Y7_L	Y6_L	Y5_L	Y4_L	Y3_L	Y2_L	Y1_L	Y0_L
0	x	x	x	x	x	1	1	1	1	1	1	1	1
x	1	x	x	x	x	1	1	1	1	1	1	1	1
x	x	1	x	x	x	1	1	1	1	1	1	1	1
1	0	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	0	1	1	1	1	1	1	0	1	1	1
1	0	0	1	0	0	1	1	1	0	1	1	1	1
1	0	0	1	0	1	1	1	0	1	1	1	1	1
1	0	0	1	1	0	1	0	1	1	1	1	1	1
1	0	0	1	1	1	0	1	1	1	1	1	1	1

Однако из-за кружков инверсии соотношения между внутренними и внешними сигналами имеют вид:

$$G2A = G2A_L'$$

$$G2B = G2B_L'$$

$$Y5 = Y5_L'$$

Поэтому при желании можно записать следующее равенство, связывающее внешний выходной сигнал $Y5_L$ с внешними входными сигналами:

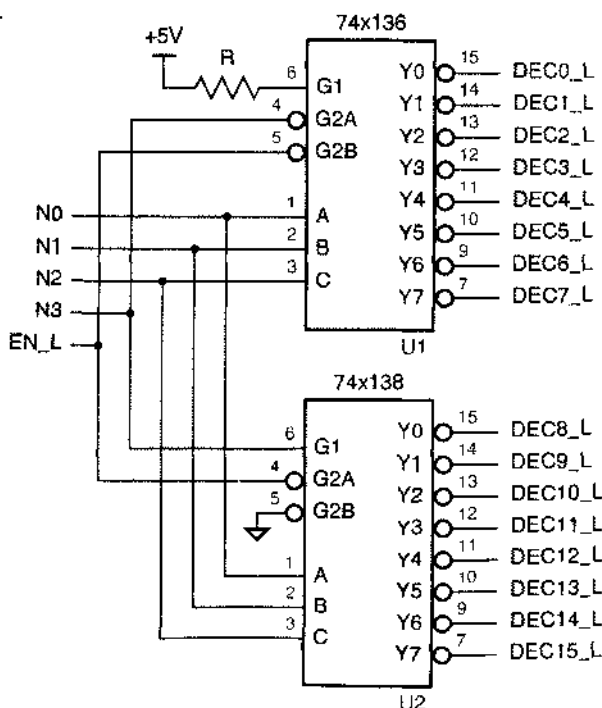
$$\begin{aligned} Y5_L = Y5' &= (G1 \cdot G2A_L' \cdot G2B_L' \cdot C \cdot B' \cdot A)' \\ &= G1' + G2A_L + G2B_L + C' + B + A'. \end{aligned}$$

На первый взгляд не похоже, что это выражение соответствует тому, что вы могли бы ожидать от дешифратора, так как оно представляет скорее логическую сумму, чем произведение. Однако, если вы пользуетесь принципом проектирования «инверсия к инверсии», то вас это не должно волновать; вы только присвойте выходному сигналу имя, указывающее на низкий активный уровень, и помните, что у этого выхода активным является низкий уровень, когда будете подключать его к входам других схем.

5.4.5. Расширение полных дешифраторов

Большие кодовые слова можно декодировать с помощью нескольких полных дешифраторов. На рис. 5.38 показано, как можно объединить два дешифратора 3×8 , чтобы получить дешифратор 4×16 . Благодаря наличию у микросхемы 74х138 входов разрешения как с высоким, так и с низким активным уровнем сигнала можно разрешать работу того или другого дешифратора, непосредственно используя значение старшего бита входного кодового слова. Верхний дешифратор (U1) работает при $N3 = 0$, а нижний (U2) – при $N3 = 1$.

Рис. 5.38. Схема дешифратора 4×16 на основе микросхем 74х138



Чтобы обрабатывать еще большие кодовые слова, можно применять иерархическое каскадное включение нолиных дешифраторов. На рис. 5.39 показано, как можно декодировать два старших бита в 5-разрядном кодовом слове, с помощью половины микросхемы 74x139, активизируя тем самым один из четырех дешифраторов 74x138 для декодирования трех младших битов.

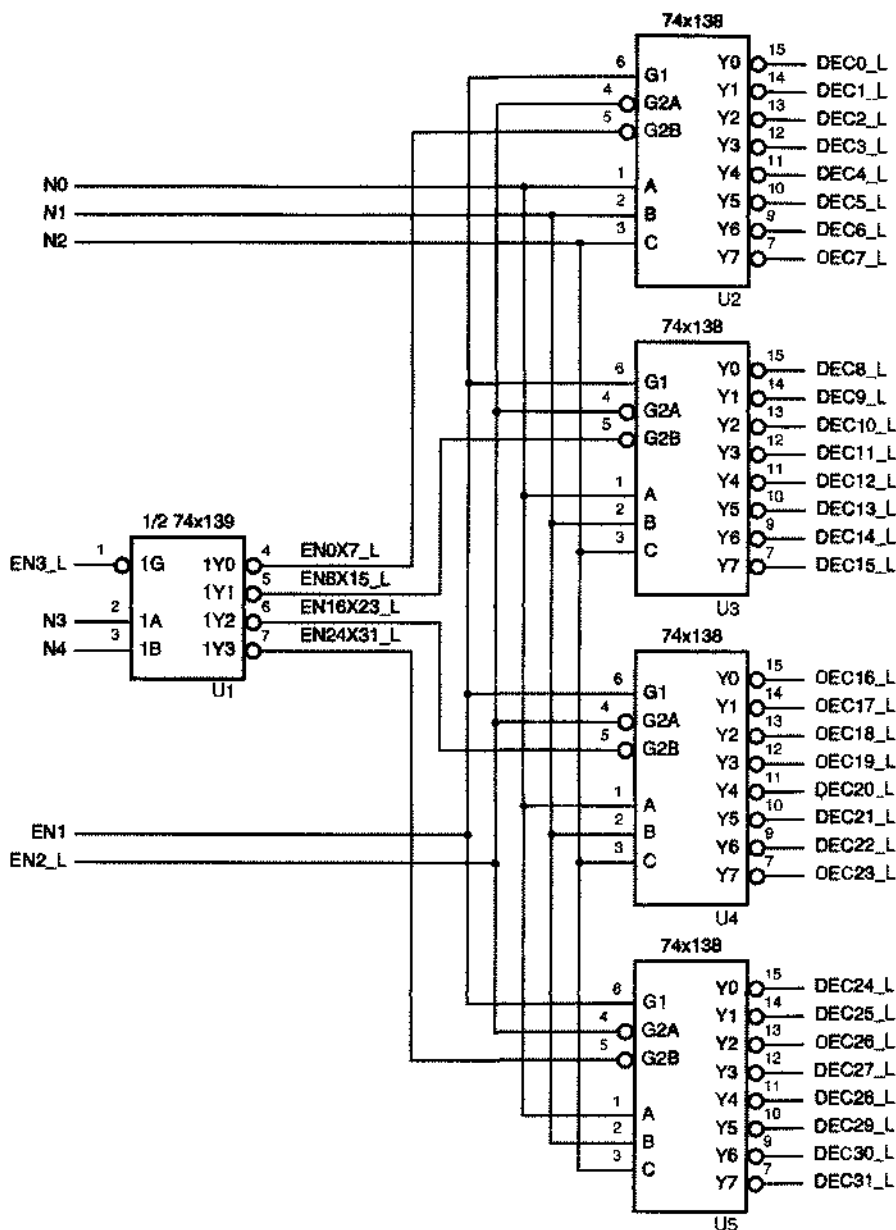


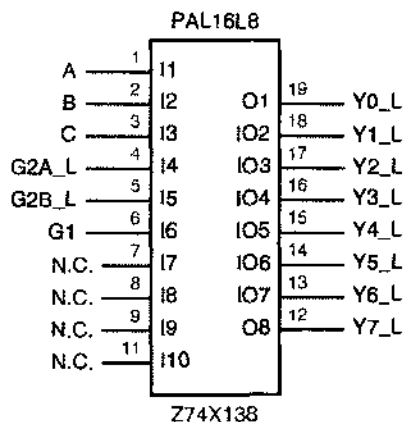
Рис. 5.39. Схема дешифратора 5x32 на основе микросхем 74x138 и 74x139

5.4.6. Описание дешифраторов на языке ABEL и их реализация в ПЛУ

При проектировании логических схем нет ничего проще, чем описать дешифратор в ПЛУ с помощью подходящих соотношений. Так как логическое выражение для каждого выхода обычно представляет собой только один терм-произведение, дешифраторы очень легко реализовать в ПЛУ при малом расходе ресурса, выражаемого возможным числом термов-произведений.

В качестве примера в табл. 5.8 приведена программа на языке ABEL для полного дешифратора 3×8, реализуемого в микросхеме PAL16L8, аналогичного дешифратору 74х138. Заметьте, что некоторые из входных выводов и все выходные выводы имеют имена, соответствующие низкому активному уровню (присутствует суффикс "_L") в соответствии с принципиальной схемой, изображенной на рис. 5.37. Однако в программе введены также для каждого из сигналов имена, соответствующие высокому активному уровню, чтобы все соотношения можно было записать «естественным» образом в терминах сигналов, имеющих высокий активный уровень. Ниже в отступлении, заключенном в рамку, приведен другой способ достижения того же самого результата.

Рис. 5.40. Условное изображение микросхемы PAL16L8, используемой в качестве дешифратора 74х138 (N.C. – не подключен)



Обратите также внимание, что в программе на языке ABEL введено константное выражение для ENB. Здесь ENB не является ни входным, ни выходным сигналом; просто это определяемое пользователем имя. Вскуду, где в разделе equations появляется «ENB», транслятор подставляет выражение (G1 & G2A & G2B). Присвоение константному выражению определяемого пользователем имени делает программу легко читаемой и более удобной для работы с ней.

Если бы вам был нужен только дешифратор типа '138, то лучше применить реальный дешифратор '138, нежели более дорогое ПЛУ. Если же необходимо выполнять нестандартные функции, то с помощью ПЛУ обычно это достигается намного дешевле и легче, чем в результате применения МИС и СИС. Пусть, например, вам нужен дешифратор типа '138, но с высокими активными уровнями

выходных сигналов; тогда в табл. 5.8 необходимо изменить только одну строку в объявлении выводов:

```
Y0, Y1, Y2, Y3, Y4, Y5, Y6, Y7      pin 19..12 istype 'com';
```

Табл. 5.8. Программа на языке ABEL для полного дешифратора 3×8, аналогичного дешифратору 74x138

```
module Z74X138
title '74x138 Decoder PLD
J. Wakerly, Stanford University'
Z74X138 device 'P16L8';

" Input and output pins
A, B, C, G2A_L, G2B_L, G1      pin 1, 2, 3, 4, 5, 5;
Y0_L, Y1_L, Y2_L, Y3_L, Y4_L, Y5_L, Y6_L, Y7_L  pin 19..12 istype 'com';

" Active-high signal names for readability
G2A = !G2A_L;
G2B = !G2B_L;
Y0 = !Y0_L;
Y1 = !Y1_L;
Y2 = !Y2_L;
Y3 = !Y3_L;
Y4 = !Y4_L;
Y5 = !Y5_L;
Y6 = !Y6_L;
Y7 = !Y7_L;

" Constant expression
ENB = G1 & G2A & G2B;

equations
Y0 = ENB & !C & !B & !A;
Y1 = ENB & !C & !B & A;
Y2 = ENB & !C & B & !A;
Y3 = ENB & !C & B & A;
Y4 = ENB & C & !B & !A;
Y5 = ENB & C & !B & A;
Y6 = ENB & C & B & !A;
Y7 = ENB & C & B & A;

end Z74X138
```

(В табл. 5.8 необходимо удалить также независимые определения сигналов Y0–Y7.) Так как для каждого из равенств требовалось одно произведение шести переменных (включая три переменных в выражении для ENB), для каждого инвертированного выражения необходима сумма шести термов-произведений, что меньше семи возможных в PAL16L8. Если программируется микросхема PAL16V8 или какая-либо другая микросхема, в которой можно выбирать полярность выходных сигналов, то трапезатор выберет инвертирующую полярность выходных сигналов, чтобы использовать только по одному терму-произведению на выход.

ОПИСАНИЯ ВЫВОДОВ С НИЗКИМ АКТИВНЫМ УРОВНЕМ СИГНАЛА

Язык ABEL позволяет использовать префикс инверсии (!) в именах сигналов при определении выводов в программе. Если при определении имени вывода употреблен префикс инверсии, то транслятор автоматически добавляет спереди к имени сигнала префикс инверсии, где бы в программе ни появился этот сигнал. Если где-то он уже инвертируется, то это приведет к двойной инверсии.

Данное свойство можно использовать для задания различных, но непротиворечивых обозначений при определении входов и выходов с низкими активными уровнями сигналов, а именно: каждому сигналу с низким активным уровнем дают имя, соответствующее *высокому* активному уровню, но в описании вывода, относящегося к этому сигналу, предваряют его префиксом инверсии. Для дешифратора 3×8, описанного в табл. 5.8, мы заменяем первую часть программы текстом, приведенным в табл. 5.9; раздел *equations* остается в программе неизменным.

Это дело вкуса, какое из обозначений применять, но выбор может зависеть также от возможностей программных средств автоматизированной системы проектирования, используемой для изображения схемы. Многие программные средства позволяют автоматически создавать схематическое изображение логического блока, задаваемого программой на языке ABEL. Если программные средства позволяют помещать кружки инверсии на выбранных входах и выходах условного обозначения элемента, то, согласно табл. 5.9, мы получим имена сигналов, имеющих высокий активный уровень внутри изображаемого блока. Затем можно ввести внешние кружки на нули сигналов с низким активным уровнем, чтобы получить условное обозначение элемента, показанное на рис. 5.41(а), соответствующее требованиям, изложенным в разделе 5.4.2.

С другой стороны, у вас может не быть возможности или желания вводить кружки инверсии в условное обозначение элемента, создаваемое системой CAD. В этом случае следует использовать обозначения из табл. 5.8; это приведет к созданию системой CAD условного обозначения, в котором активный уровень указан в имени сигнала внутри контура функционального обозначения элемента; никакие внешние кружки инверсии в этом случае не нужны [см. рис. 5.41(б)]. Заметьте, что в отличие от рис. 5.36(а) мы используем обозначение, записываемое в виде текстовой строки (L), а не применяем надчеркивания в имени сигнала для указания активного уровня. Правильно выбранные обозначения, записываемые в виде текстовой строки, обеспечивают возможность использования программных средств различных систем CAD.

В дальнейшем в каждом из примеров описания на языке ABEL мы в значительной мере произвольно выбираем то или другое обозначение только ради того, чтобы помочь вам освоиться с обоими подходами.

Табл. 5.9. Альтернативный вариант объявлений в программе полного дешифратора 3×8 типа 74х138

```

" Input and output pins
A, B, C, !G2A, !G2B, G1                pin 1, 2, 3, 4, 5, 6;
!Y0, !Y1, !Y2, !Y3, !Y4, !Y5, !Y6, !Y7  pin 19..12 istype 'com';

" Constant expression
ENB = G1 & G2A & G2B;

```

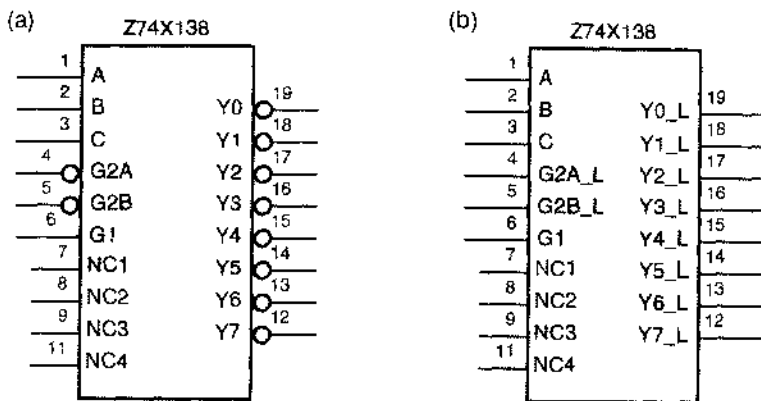


Рис. 5.41. Возможные варианты условных обозначений дешифратора типа 74х138 на основе ПЛУ, создаваемые системой САД: (а) на основе табл. 5.9 после введения кружков инверсии вручную; (б) на основе табл. 5.8 (NC – не подключен)

Другое простое изменение позволяет добавить альтернативные входы разрешения, сигналы на которых объединяются по ИЛИ с сигналами на основных входах разрешения. Чтобы это сделать, необходимо только определить дополнительные выводы и видоизменить определение ENB:

```

EN1,  EN2_L                pin 7, 8;
...
EN2   = !EN2_L;
...
ENB = G1 & G2A & G2B # EN1 # EN2;

```

Это изменение увеличивает число термов-произведений, приходящихся на один выход, до трех, и выходные сигналы принимают следующий вид:

```

Y0 = G1 & G2A & G2B & !C & !B & !A
    # EN1 & !C & !B & !A
    # EN2 & !C & !B & !A;

```

(Вспомните, что в микросхеме PAL16L8 между матрицей И-ИЛИ и выходом ПЛУ помещен встроенный инвертор, а в микросхеме PAL16V8 в этом месте не помещен)

устанавливаем при программировании инвертор, поэтому на реальном выходе будет желаемый низкий активный уровень.)

Если вы вводите дополнительные разрешающие сигналы в вариант программы с высоким активным уровнем сигналов на выходах, то ПЛУ должно реализовать дополнение приведенного выше выражения вида «сумма произведений». Сразу не очевидно, сколько термов-произведений будет иметь это выражение, и можно ли его реализовать в микросхеме PAL16L8, но мы можем воспользоваться транслятором языка ABEL, чтобы получить ответ:

```
!Y0 = C # B # A # !G2B & !EN1 & !EN2
      # !G2A & !EN1 & !EN2
      # !G1 & !EN1 & !EN2;
```

В этом выражении шесть термов-произведений, так что оно реализуемо в микросхеме PAL16L8.

В качестве заключительной уловки можно добавить вход для оперативного управления активным уровнем выходного сигнала, выбирая его высоким или низким, изменив все равенства следующим образом:

```
POL    pin 9;
...
Y0 = POL $ (ENB & !C & !B & !A);
Y1 = POL $ (ENB & !C & !B & A);
...
Y7 = POL $ (ENB & C & B & A);
```

В результате введения операции ИСКЛЮЧАЮЩЕЕ ИЛИ число необходимых термов-произведений на один выход увеличивается до 9 при любой полярности сигнала на выходном контакте. Таким образом, даже в микросхеме PAL16V8 нельзя реализовать эту функцию в том виде, как она записана.

Эту функцию все же можно реализовать, если создать *вспомогательный выход (helper output)* для уменьшения быстро нарастающего числа термов-произведений. Как показано в табл. 5.10, мы назначаем выходной контакт для сигнала ENB (опуская выход Y7_L) и исключаем равенство для ENB в раздел программы equations. Это уменьшает число требуемых термов-произведений до пяти при любой полярности выходных сигналов.

Табл. 5.10. Фрагмент программы на языке ABEL, иллюстрирующий логику с двумя проходами

```
...
" Output pins
Y0_L, Y1_L, Y2_L, Y3_L      pin 19, 18, 17, 16 istype 'com';
Y4_L, Y5_L, Y6_L, ENB      pin 15, 14, 13, 12 istype 'com';

equations
ENB = G1 & G2A & G2B # EN1 # EN2;
Y0 = POL $ (ENB & !C & !B & !A);
...
```


Помимо того, что приходится жертвовать выводом ради вспомогательного выхода, недостатком этого варианта, является меньшее быстродействие. Любые изменения входных сигналов, входящих во вспомогательное выражение, должны пройти через ПЛУ дважды, прежде чем они достигнут конечного выхода. Этот метод реализации называется *логикой с двумя проходами (two-pass logic)*. Многие программные средства синтеза, ориентированные на ПЛУ и схемы FPGA, могут автоматически реализовать логику с двумя или большим числом проходов, если заданное выражение нельзя реализовать за один проход через логическую матрицу.

Дешифраторы могут быть заказными и иметь другой алгоритм работы. Обычно у заказных дешифраторов сигнал на одном из выходов соответствует декодированию нескольких входных комбинаций. Предположим, например, что вам необходимо создать набор разрешающих сигналов в соответствии с табл. 5.11. Для реализации заданной функции можно расширить возможности стандартного дешифратора 74х138, как показано на рис. 5.42. У этого подхода, который потенциально является менее дорогим, чем применение ПЛУ, есть недостатки, состоящие в том, что для формирования заданных выходных сигналов требуются дополнительные компоненты, это вызывает дополнительную задержку и построившую схему не так просто изменить.

Табл. 5.11. Таблица истинности для заказного дешифратора

CS_L	RD_L	A2	A1	A0	Активизируемые выходы
1	x	x	x	x	нет
x	1	x	x	x	нет
0	0	0	0	0	BILL_L, MARY_L
0	0	0	0	1	MARY_L, KATE_L
0	0	0	1	0	JOAN_L
0	0	0	1	1	PAUL_L
0	0	1	0	0	ANNA_L
0	0	1	0	1	FRED_L
0	0	1	1	0	DAVE_L
0	0	1	1	1	KATE_L

Решение той же самой задачи на основе ПЛУ показано в табл. 5.12. Заметьте, что в этой программе для обозначения выводов с низким активным уровнем сигнала применен способ, описанный выше в рамке (вам стоит научиться свободно оперировать с любыми обозначениями). Для реализации каждого из последних шести соотношений требуется один вентиль И в ПЛУ. Транслятор языка ABEL минимизирует также логическое выражение MARY, чтобы реализовать его с помощью только одного вентиля И. Выходные сигналы с высоким активным уровнем можно было бы получить путем изменений только двух строк в разделе объявлений:

```
BILL, MARY, JOAN, PAUL      pin 19, 18, 17, 16 istype 'com';
ANNA, FRED, DAVE, KATE      pin 15, 14, 13, 12 istype 'com';
```

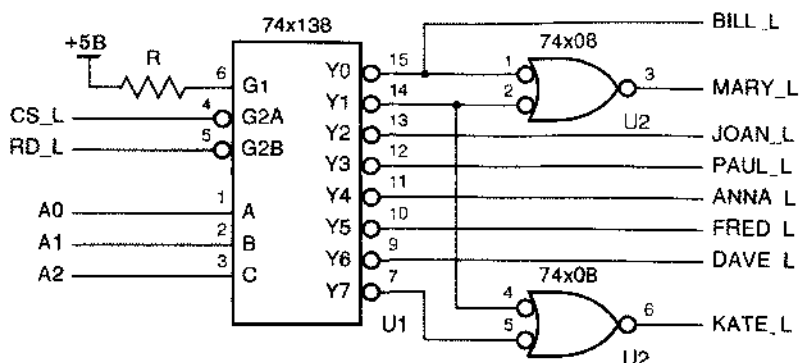


Рис. 5.42. Схема заказного дешифратора

Табл. 5.12. Программа заказного дешифратора на языке ABEL

```

module CUSTMDEC
title 'Customized Decoder PLD
J. Wakerly, Stanford University'
CUSTMDEC device 'P16L8';

" Input pins
!CS, !RD, A0, A1, A2      pin 1, 2, 3, 4, 5;
" Output pins
!BILL, !MARY, !JOAN, !PAUL  pin 19, 18, 17, 16 istype 'com';
!ANNA, !FRED, !DAVE, !KATE  pin 15, 14, 13, 12 istype 'com';

equations
BILL = CS & RD & (!A2 & !A1 & !A0);
MARY = CS & RD & (!A2 & !A1 & !A0 # !A2 & !A1 & A0);
KATE = CS & RD & (!A2 & !A1 & A0 # A2 & A1 & A0);
JOAN = CS & RD & (!A2 & A1 & !A0);
PAUL = CS & RD & (!A2 & A1 & A0);
ANNA = CS & RD & (A2 & !A1 & !A0);
FRED = CS & RD & (A2 & !A1 & A0);
DAVE = CS & RD & (A2 & A1 & !A0);

end CUSTMDEC

```

Другой способ задания соотношений приведен в табл. 5.13. В большинстве прикладных задач этот стиль более понятен, особенно в том случае, когда сигналы на входах выбора одного из выходов имеют числовое значение.

Табл. 5.13. Эквивалентные равенства для заказного дешифратора на языке ABEL

```
ADDR = [A2,A1,A0];
```

```
equations
```

```
BILL = CS & RD & (ADDR == 0);
MARY = CS & RD & ((ADDR == 0) # (ADDR == 1));
KATE = CS & RD & ((ADDR == 1) # (ADDR == 7));
JOAN = CS & RD & (ADDR == 2);
PAUL = CS & RD & (ADDR == 3);
ANNA = CS & RD & (ADDR == 4);
FRED = CS & RD & (ADDR == 5);
DAVE = CS & RD & (ADDR == 6);
```

5.4.7. Описание дешифраторов на языке VHDL

Существует несколько подходов к проектированию дешифраторов на языке VHDL. В простейшем случае следовало бы занисать структурный эквивалент принципиальной схемы дешифратора, как это сделано в табл. 5.14 для полиного дешифратора 2х4, приведенного на рис. 5.32. Предполагается, что компоненты and3 и inv уже существуют в микросхеме, для которой пишется программа. Конечно, такое механическое преобразование существующих устройств в эквивалент в виде списка соединений в первую очередь убивает желание пользоваться языком VHDL.

Табл. 5.14. Структурная программа дешифратора, изображенного на рис. 5.32, на языке VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;

entity V2to4dec is
  port (I0, I1, EN: in STD_LOGIC;
        Y0, Y1, Y2, Y3: out STD_LOGIC );
end V2to4dec;

architecture V2to4dec_s of V2to4dec is
  signal NOTI0, NOTI1: STD_LOGIC;
  component inv port (I: in STD_LOGIC; O: out STD_LOGIC ); end component;
  component and3 port (I0, I1, I2: in STD_LOGIC; O: out STD_LOGIC ); end component;
begin
  U1: inv port map (I0,NOTI0);
  U2: inv port map (I1,NOTI1);
  U3: and3 port map (NOTI0,NOTI1,EN,Y0);
  U4: and3 port map ( I0,NOTI1,EN,Y1);
  U5: and3 port map (NOTI0, I1,EN,Y2);
  U6: and3 port map ( I0, I1,EN,Y3);
end V2to4dec_s;
```

Вместо этого хотелось бы написать программу, в которой язык VHDL был бы использован так, чтобы сделать проектирование дешифратора более понятным и удобным. В табл. 5.15 продемонстрирован один из подходов к написанию программы полного дешифратора 3×8, эквивалентного дешифратору 74х138, на языке VHDL в стиле потокового проектирования. Адресные входы A (2 downto 0) и декодированные выходные сигналы с низким активным уровнем Y_L (0 to 7) ради удобства чтения представлены в виде векторов. В операторе select перечислены восемь случаев декодирования, в каждом из которых 8-разрядному внутреннему сигналу Y_L_i присваивается соответствующая комбинация входных сигналов с низким активным уровнем. Эта комбинация присваивается фактически выходному сигналу схемы Y_L только в том случае, когда сигналы на всех входах разрешения активны.

Табл. 5.15. VHDL-программа полного дешифратора 3×8 типа 74х138 в стиле потокового проектирования

```

library IEEE;
use IEEE.std_logic_1164.all;

entity V74x138 is
    port (G1, G2A_L, G2B_L: in STD_LOGIC;           -- enable inputs
          A: in STD_LOGIC_VECTOR (2 downto 0);      -- select inputs
          Y_L: out STD_LOGIC_VECTOR (0 to 7) );      -- decoded outputs
end V74x138;

architecture V74x138_a of V74x138 is
    signal Y_L_i: STD_LOGIC_VECTOR (0 to 7);
begin
    with A select Y_L_i <=
        "01111111" when "000",
        "10111111" when "001",
        "11011111" when "010",
        "11101111" when "011",
        "11110111" when "100",
        "11111011" when "101",
        "11111101" when "110",
        "11111110" when "111",
        "11111111" when others;
    Y_L <= Y_L_i when (G1 and not G2A_L and not G2B_L)='1' else "11111111";
end V74x138_a;

```

Этот вариант хорош для начала и работает, но в нем есть скрытая ловушка. Корректировки, учитывающие тот факт, что два входных и все выходные сигналы имеют низкий активный уровень, оказываются спрятанными в заключительном операторе присваивания. Это верно, что большинство программ на языке VHDL пишется почти полностью для сигналов с высоким активным уровнем, но если мы разрабатываем устройство с низкими активными уровнями сигналов на внешних выводах, то нам действительно следует привести их к более систематическому и более удобному виду.

Табл. 5.16. VHDL-архитектура, пригодная для работы с различными активными уровнями сигналов

```

architecture V74x138_b of V74x138 is
    signal G2A, G2B: STD_LOGIC;           -- active-high version of inputs
    signal Y: STD_LOGIC_VECTOR (0 to 7);   -- active-high version of outputs
    signal Y_s: STD_LOGIC_VECTOR (0 to 7); -- internal signal
begin
    G2A <= not G2A_L; -- convert inputs
    G2B <= not G2B_L; -- convert inputs
    Y_L <= not Y;     -- convert outputs
    with A select Y_s <=
        "10000000" when "000",
        "01000000" when "001",
        "00100000" when "010",
        "00010000" when "011",
        "00001000" when "100",
        "00000100" when "101",
        "00000010" when "110",
        "00000001" when "111",
        "00000000" when others;
    Y <= Y_s when (G1 and G2A and G2B)='1' else "00000000";
end V74x138_b;

```

Табл. 5.16 демонстрирует такой подход. Изменения не коснулись объявлений в разделе *entity*. Однако в архитектуре V74x138_a для каждого из внешних выводов с низким активным уровнем определена версия сигнала с высоким активным уровнем, и переход от сигналов с высоким активным уровнем к сигналам с низким активным уровнем осуществляется явными операторами присваивания. Функция самого дешифратора определена с использованием только сигналов с высоким активным уровнем, что является, вероятно, самым большим достоинством этого подхода. Другой плюс состоит в том, что если требуется изменить активные внешние уровни, то это легко сделать путем исправлений лишь в определенных местах в программе, число которых невелико.

ПРОГРАММА ВЫПОЛНЯЕТСЯ НЕ ПО ПОРЯДКУ

В табл. 5.16 все три оператора преобразования активного уровня собраны вместе в начале программы, хотя никакое значение не может быть присвоено сигналу *Y_L* до тех, пока не будет присвоено значение сигналу *Y*, что делается в программе позднее. Помните, что это правильно, поскольку операторы присваивания в теле архитектуры выполняются параллельно. Это означает, что присвоение значения любому сигналу вызывает пересчет во всех других операторах, в которых фигурирует этот сигнал, независимо от его расположения в теле архитектуры.

Если место расположения оператора "*Y_L <= not Y*" беспокоит вас, то можно поместить его в конце тела архитектуры, но программа немного более удобна для отладки в представленном виде, когда все преобразования активных уровней собраны вместе.

С активными уровнями можно оперировать даже более систематическим образом. Как показано в табл. 5.17, архитектуру V74x138 можно организовать иерархически, используя компонент V3to8dec, в котором фигурируют сигналы только с высокими активными уровнями; определение самого компонента в стиле потокового проектирования приведено в табл. 5.18. Еще раз: не требуется никаких изменений в определении объекта V74x138 на верхнем уровне. Соотношение между объектами показано на рис. 5.43.

Табл. 5.17. Иерархическое определение дешифратора типа 74x138 с преобразованием активных уровней

```
architecture V74x138_c of V74x138 is
    signal G2A, G2B: STD_LOGIC;           -- active-high version of inputs
    signal Y: STD_LOGIC_VECTOR (0 to 7);   -- active-high version of outputs
    component V3to8dec port (G1, G2, G3: in STD_LOGIC;
                             A: in STD_LOGIC_VECTOR (2 downto 0);
                             Y: out STD_LOGIC_VECTOR (0 to 7) ); end component;
begin
    G2A <= not G2A_L; -- convert inputs
    G2B <= not G2B_L; -- convert inputs
    Y_L <= not Y;     -- convert outputs
    U1: V3to8dec port map (G1, G2A, G2B, A, Y);
end V74x138_c;
```

Табл. 5.18. Определение дешифратора 3x8 с высокими активными уровнями сигналов в потоковом стиле

```
library IEEE;
use IEEE.std_logic_1164.all;

entity V3to8dec is
    port (G1, G2, G3: in STD_LOGIC;
          A: in STD_LOGIC_VECTOR (2 downto 0);
          Y: out STD_LOGIC_VECTOR (0 to 7) );
end V3to8dec;

architecture V3to8dec_a of V3to8dec is
    signal Y_s: STD_LOGIC_VECTOR (0 to 7);
begin
    with A select Y_s <=
        "10000000" when "000",
        "01000000" when "001",
        "00100000" when "010",
        "00010000" when "011",
        "00001000" when "100",
        "00000100" when "101",
        "00000010" when "110",
        "00000001" when "111",
        "00000000" when others;
    Y <= Y_s when (G1 and G2 and G3)='1'
        else "00000000";
end V3to8dec_a;
```

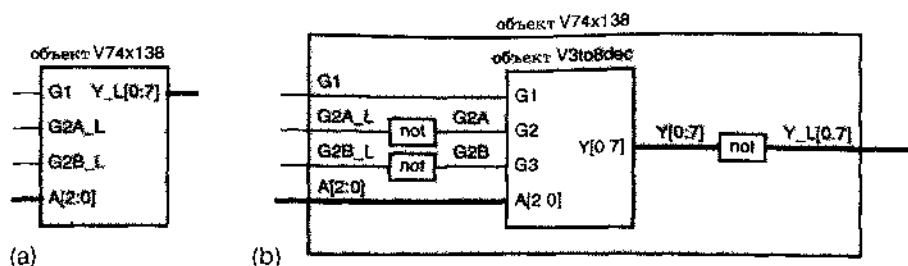


Рис. 5.43. VHDL-объект V74x138. (a) верхний уровень; (b) внутренняя организация с архитектурой V74x138_c

Еще один подход к конструированию дешифратора позволит замснить архитектуру V3to8dec_a в табл. 5.18 архитектурой V3to8dec_b, представленной в табл. 5.19. Вместо параллельных операторов используются процесс и последовательные операторы, в результате чего работа дешифратора оказывается заданной в поведенческом стиле. Однако внимательное сравнение этих двух вариантов показывает, что реально они отличаются только синтаксисом.

Табл. 5.19. Определение архитектуры дешифратора 3x8 в поведенческом стиле

```
architecture V3to8dec_b of V3to8dec is
    signal Y_s: STD_LOGIC_VECTOR (0 to 7);
begin
    process(A, G1, G2, G3, Y_s)
    begin
        case A is
            when "000" => Y_s <= "10000000";
            when "001" => Y_s <= "01000000";
            when "010" => Y_s <= "00100000";
            when "011" => Y_s <= "00010000";
            when "100" => Y_s <= "00001000";
            when "101" => Y_s <= "00000100";
            when "110" => Y_s <= "00000010";
            when "111" => Y_s <= "00000001";
            when others => Y_s <= "00000000";
        end case;
        if (G1 and G2 and G3)='1' then Y <= Y_s;
        else Y <= "00000000";
        end if;
    end process;
end V3to8dec_b;
```

СОГЛАСОВАНИЕ ИМЕН

Имена портов объекта на рис. 5.43 изображены внутри соответствующего блока. Имена сигналов, поступающих к портам при использовании объекта, написаны у сигнальных линий. Обратите внимание, что имена сигналов могут, но не обязаны совпадать. VHDL-компилятор все сохраняет без изменений, сопоставляя с каждым именем область его действия. Ситуация полностью аналогична тому, как используются имена переменных и параметры в структурированных, процедурных языках программирования типа языка С.

Табл. 5.20. Истинно поведенческое определение архитектуры дешифратора 3×8

```
architecture V3to8dec_c of V3to8dec is
begin
  process (G1, G2, G3, A)
    variable i: INTEGER range 0 to 7;
  begin
    Y <= "00000000";
    if (G1 and G2 and G3) = '1' then
      for i in 0 to 7 loop
        if i=CONV_INTEGER(A) then Y(i) <= '1'; end if;
      end loop;
    end if;
  end process;
end V3to8dec_c;
```

В качестве последнего примера в табл. 5.20 приведена архитектура дешифратора 3×8, написанная в стиле, еще более близком к истинно поведенческому программированию. (Напомним, что функция CONV_INTEGER была определена в разделе 4.7.4.) Из всех примеров этот — единственный, в котором функциональное поведение дешифратора описывается по существу без включения таблицы истинности в VHDL-программу. В этом отношении данный вариант более гибок, поскольку его легко видоизменить для получения полного дешифратора любого размера. Но с другой стороны отсутствие таблицы истинности приводит к меньшей гибкости, поскольку таблицу истинности легко заменить при разработке заказных дешифраторов, подобных дешифратору, представленному в табл. 5.11.

***5.4.8. Дешифраторы для семисегментных индикаторов**

Посмотрите на свое запястье и вы, вероятно, увидите *семисегментный индикатор* (*seven-segment display*). Индикатор такого типа на светодиодах или жидких кристаллах применяется в часах, калькуляторах и измерительных приборах для отображения данных в десятичной системе. Цифра образуется в результате свечения определенных линейных сегментов, полное число которых равно семи [рис. 5.44(а)].

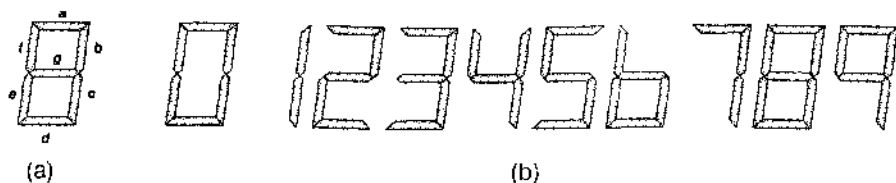


Рис. 5.44. Семисегментный индикатор (а) обозначение сегментов; (б) десятичные цифры

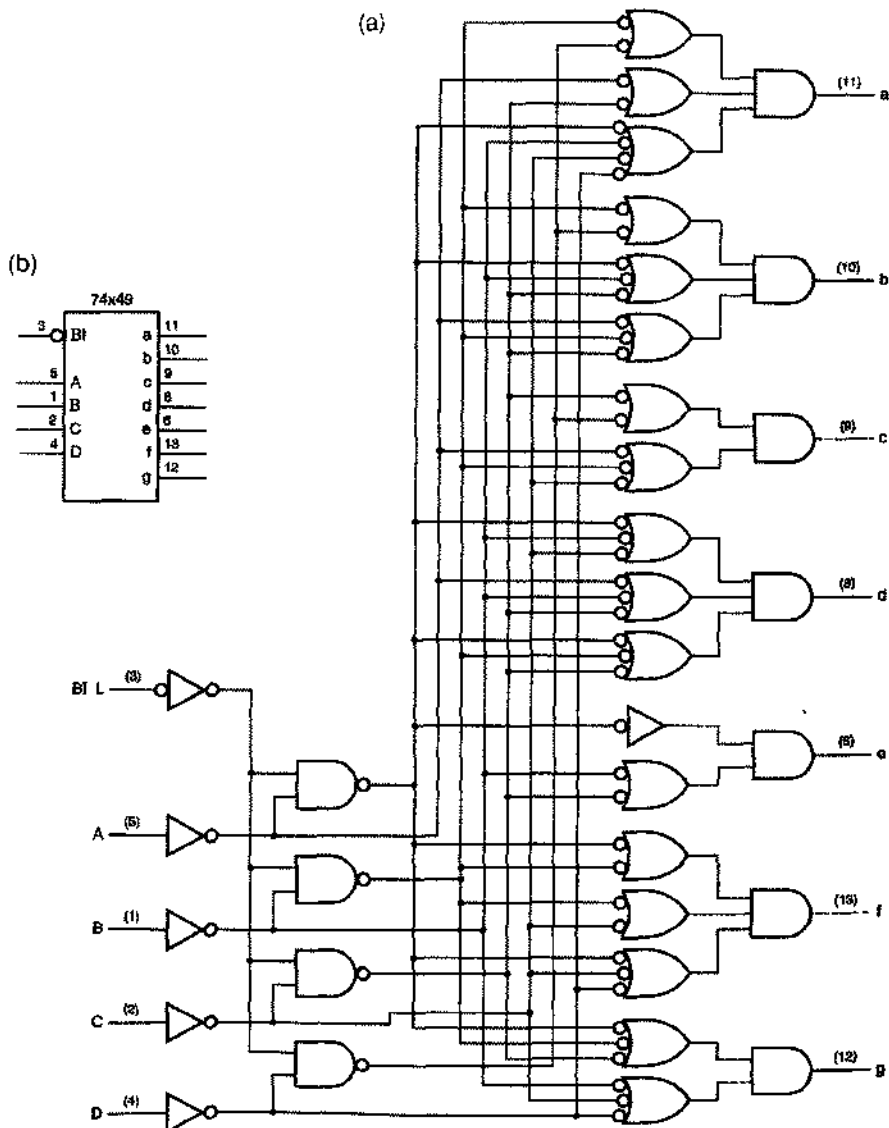


Рис. 5.45. Семисегментный дешифратор 74x49 (а) принципиальная схема с цоколевкой, (б) традиционное условное обозначение

На входы семисегментного дешифратора (*seven-segment decoder*) поступает 4-разрядный двоично-десятичный код, а на его выходах возникает «семисегментный код», графически представленный на рис. 5.44(b). На рис. 5.45 и в табл. 5.21 приведены принципиальная схема и таблица истинности семисегментного дешифратора 74х49. Если не принимать во внимание странный (или хитрый?) «запирающий вход» $\overline{BI_L}$, то каждый выходной сигнал дешифратора 74х49, относящийся к соответствующему сегменту, является минимальным выражением вида «произведение сумм», значения которого при десятичных входных комбинациях «безразличны». Использование на каждом выходе структуры НЕ-ИЛИ-И может показаться немного странным, но, согласно обобщенной теореме Де Моргана, она эквивалентна вентиллю И-ИЛИ-НЕ, который – при реализации его на основе КМОП-или ТТЛ-технологий – является довольно быстрой и компактной структурой.

Табл. 5.21. Таблица истинности семисегментного дешифратора 74х49

Входы					Выходы						
$\overline{BI_L}$	D	C	B	A	a	b	c	d	e	f	g
0	x	x	x	x	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1	1	1	1	0
1	0	0	0	1	0	1	1	0	0	0	0
1	0	0	1	0	1	1	0	1	1	0	1
1	0	0	1	1	1	1	1	1	0	0	1
1	0	1	0	0	0	1	1	0	0	1	1
1	0	1	0	1	1	0	1	1	0	1	1
1	0	1	1	0	0	0	1	1	1	1	1
1	0	1	1	1	1	1	1	0	0	0	0
1	1	0	0	0	1	1	1	1	1	1	1
1	1	0	0	1	1	1	1	0	0	1	1
1	1	0	1	0	0	0	0	1	1	0	1
1	1	0	1	1	0	0	1	1	0	0	1
1	1	1	0	0	0	1	0	0	0	1	1
1	1	1	0	1	1	0	0	1	0	1	1
1	1	1	1	0	0	0	0	1	1	1	1
1	1	1	1	1	0	0	0	0	0	0	0

В большинстве современных семисегментных индикаторов имеются встроенные дешифраторы, чтобы 4-разрядное двоично-десятичное слово можно было непосредственно подать на индикатор. Многие старые семисегментные дешифраторы, выполненные в виде отдельных микросхем и имеющие специальные выходы

для работы с высокими напряжениями или большими токами, хорошо подходят для подключения больших по размерам и потребляющих значительную мощность элементов отображения.

В табл. 5.22 приведена программа семисегментного дешифратора на языке ABEL. Наборы использованы в определении рисунка цифр только ради наглядности.

Табл. 5.22. Программа семисегментного дешифратора типа 74x49 на языке ABEL

```

module Z74X49H
title 'Seven-Segment_Decoder
J. Wakerly, Micro Design Resources, Inc.'
Z74X49H device 'P16L8';

" Input pins
A, B, C, D                pin 1, 2, 3, 4;
BI_L                      pin 5;

" Output pins
SEGA, SEGB, SEGC, SEGD    pin 19, 18, 17, 16 istype 'com';
SEGE, SEGF, SEGJ         pin 15, 14, 13 istype 'com';

" Definitions
BI = 'BI_L';
DIGITIN = [D,C,B,A];
SEGOUT = [SEGA,SEGB,SEGC,SEGD,SEGE,SEGF,SEGJ];

" Segment encodings for digits
DIG0 = [1,1,1,1,1,1,0]; " 0
DIG1 = [0,1,1,0,0,0,0]; " 1
DIG2 = [1,1,0,1,1,0,1]; " 2
DIG3 = [1,1,1,1,0,0,1]; " 3
DIG4 = [0,1,1,0,0,1,1]; " 4
DIG5 = [1,0,1,1,0,1,1]; " 5
DIG6 = [1,0,1,1,1,1,1]; " 6 'tsil' included
DIG7 = [1,1,1,0,0,0,0]; " 7
DIG8 = [1,1,1,1,1,1,1]; " 8
DIG9 = [1,1,1,1,0,1,1]; " 9 'tail' included
DIGA = [1,1,1,0,1,1,1]; " A
DIGB = [0,0,1,1,1,1,1]; " b
DIGC = [1,0,0,1,1,1,0]; " C
DIGD = [0,1,1,1,1,0,1]; " d
DIGE = [1,0,0,1,1,1,1]; " E
DIGF = [1,0,0,0,1,1,1]; " F

equations

SEGOUT = 'BI & ( (DIGITIN == 0) & DIG0 # (DIGITIN == 1) & DIG1
# (DIGITIN == 2) & DIG2 # (DIGITIN == 3) & DIG3
# (DIGITIN == 4) & DIG4 # (DIGITIN == 5) & DIG5
# (DIGITIN == 6) & DIG6 # (DIGITIN == 7) & DIG7
# (DIGITIN == 8) & DIG8 # (DIGITIN == 9) & DIG9
# (DIGITIN == 10) & DIGA # (DIGITIN == 11) & DIGB
# (DIGITIN == 12) & DIGC # (DIGITIN == 13) & DIGD
# (DIGITIN == 14) & DIGE # (DIGITIN == 15) & DIGF );

end Z74X49H

```

5.5. Шифраторы

Как правило, код на выходе дешифратора имеет большее число разрядов, чем код на его входе. Если код на выходе устройства имеет *меньшее число* разрядов, чем код на входе, то это устройство обычно называют *шифратором* (*encoder*). Рассмотрим, например, устройство с восьмью двоичными входными сигналами, представляющими целое число без знака, и двумя выходами, где единичные сигналы означают, что поданное на вход число является простым или делится на 7. Такое устройство можно было бы называть шифратором, обнаруживающим простые числа и приносящим удачу.

Вероятно, простейшим с точки зрения построения является *двоичный шифратор* (*binary encoder*) или шифратор $2^n \times n$. Как показано на рис. 5.46(а), он реализует точно обратную функцию по отношению к полному дешифратору с кодом «1 из 2^n » на входе и n -разрядным двоичным кодом на выходе. Соотношения для шифратора 8×3 с входными сигналами I_0 – I_7 и выходными сигналами Y_0 – Y_2 имеют вид:

$$Y_0 = I_1 + I_3 + I_5 + I_7$$

$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$Y_2 = I_4 + I_5 + I_6 + I_7.$$

Соответствующая принципиальная схема показана на рис. 5.46(б). В общем случае шифратор $2^n \times n$ можно построить, воспользовавшись n вентилями ИЛИ с 2^{n-1} входами каждый. Если j -й разряд в двоичном представлении числа i равен 1, то i -й разряд входного кода поступает на j -й вентиль ИЛИ.

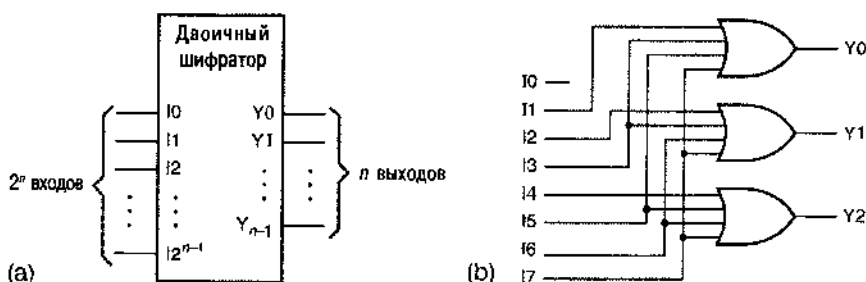


Рис. 5.46. Двоичный шифратор: (а) общая структура; (б) шифратор 8×3

5.5.1. Приоритетные шифраторы

Выходные сигналы n -разрядного полного дешифратора, являющиеся словами кода «1 из 2^n », обычно используются для управления набором из 2^n устройств, когда предполагается, что в любой момент времени активным является самое большее одно устройство. Рассмотрим обратную ситуацию, то есть систему с 2^n входами, сигнал на каждом из которых указывает на наличие требования обслуживания, как показано на рис. 5.47. Такая структура часто применяется в микропроцессорных подсистемах ввода/вывода, где входные сигналы могут быть требованиями прерывания.

В ситуации, когда надо наблюдать за входами и указывать, какое из подключенных к ним устройств требует обслуживания в данный момент времени, естественным может выглядеть применение двоичного шифратора, приведенного на рис. 5.46. Однако этот шифратор работает правильно только в том случае, если в любой момент времени гарантировано активизирован не более чем один вход. Если одновременно может поступать несколько запросов, то шифратор будет давать нежелательные результаты. Предположим, например, что на входах I2 и I4 шифратора 8×3 одновременно присутствуют 1; тогда на выходе появится комбинация 110, соответствующая двоичному представлению числа 6.

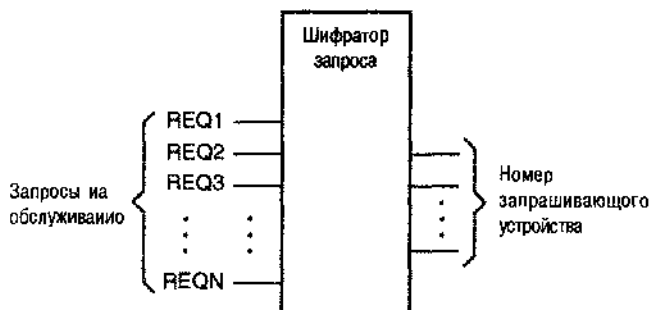


Рис. 5.47. Система с 2^n запрашивающими устройствами и «шифратор запроса», в любой момент время показывающий, какой из сигналов запроса активен

В приведенном примере на выходе должно было бы быть 2 или 4, но не 6. Как может шифратор узнать, какое выбрать число? Решение состоит в назначении входным линиям *приоритетов* (*priority*) так, чтобы при наличии нескольких запросов число на выходе шифратора соответствовало номеру запрашивающего устройства с наивысшим приоритетом. Схема, реализующая такой алгоритм, называется *приоритетным шифратором* (*priority encoder*).

Условное обозначение 8-входового приоритетного шифратора показано на рис. 5.48. Вход I7 имеет высший приоритет. Если хотя бы на одном из входов присутствует сигнал, то на выходах A2–A0 появляется число, соответствующее входному сигналу с наивысшим приоритетом. На выходе IDLE сигнал появляется в том случае, когда ни один из входов не активизирован.

Чтобы записать логические выражения для выходных сигналов приоритетного шифратора, определим сначала восемь промежуточных переменных H0–H7, таких что H_i равно 1 только в том случае, когда i является входом с высшим приоритетом из числа тех, на которые поданы 1:

$$H_7 = I_7$$

$$H_6 = I_6 \cdot I_7'$$

$$H_5 = I_5 \cdot I_6' \cdot I_7'$$

...

$$H_0 = I_0 \cdot I_1' \cdot I_2' \cdot I_3' \cdot I_4' \cdot I_5' \cdot I_6' \cdot I_7'$$

Используя эти сигналы, получаем соотношения для выходов A2–A0, подобные тем, какие были написаны для простого двоичного шифратора:

$$A2 = I4 + I5 + I6 + I7$$

$$A1 = I2 + I3 + I6 + I7$$

$$A0 = I1 + I3 + I5 + I7$$

Сигнал на выходе IDLE равен 1, если ни на одном из входов нет 1:

$$IDLE = (I0 + I1 + I2 + I3 + I4 + I5 + I6 + I7)'$$

$$= I0' \cdot I1' \cdot I2' \cdot I3' \cdot I4' \cdot I5' \cdot I6' \cdot I7'$$

5.5.2. Приоритетный шифратор 74х148

Имеющаяся в продаже микросхема 74х148 является ИС средней степени интеграции и представляет собой 8-входовой приоритетный шифратор. Условное обозначение этого приоритетного шифратора приведено на рис. 5.49, а на рис. 5.50 показана его принципиальная схема. Основное различие между этой ИС и «типичным» приоритетным шифратором, изображенным на рис. 5.48, состоит в том, что входные и выходные сигналы данного шифратора имеют низкий активный уровень. Кроме того, имеется вход разрешения EI_L , который должен быть активизирован для того, чтобы какой-либо из выходов шифратора был активным. Полная таблица истинности представлена в табл. 5.23.

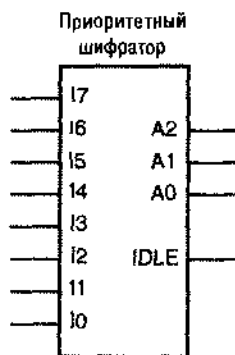


Рис. 5.48. Условное обозначение типичного 8-входового приоритетного шифратора

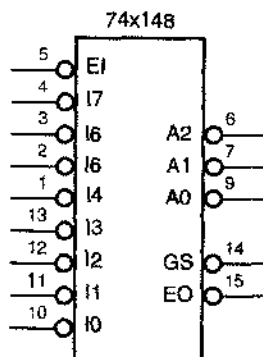


Рис. 5.49. Условное обозначение 8-входового приоритетного шифратора 74х148

Роль выхода IDLE в микросхеме '148 играет выход GS_L , который активен, когда работа устройства разрешена, и сигналы на одном или большем числе входов запроса имеют активный уровень. Производитель называет этот сигнал «Group Select» (сигналом «группового выбора»), но его обозначение легче запомнить как «Got Something» («что-то получено»). Сигнал EO_L — это выходной сигнал разрешения; он предназначен для подачи его на вход EI_L другого шифратора '148, который обрабатывает запросы с более низким приоритетом. Сигнал EO_L имеет активный уровень в том случае, когда на входе EI_L имеется активный уровень, но ни на одном из входов запроса нет сигнала с активным уровнем; именно в этом случае разрешается работа шифратора '148, обрабатывающего запросы с более низким приоритетом.

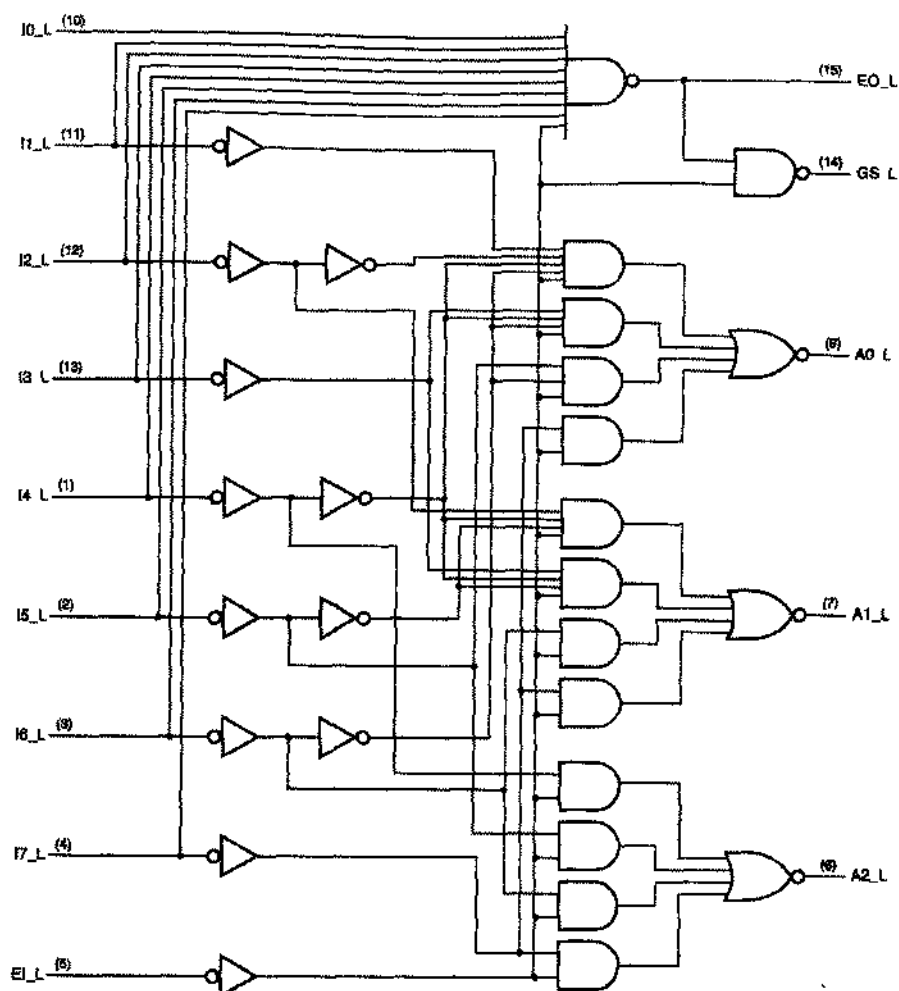


Рис. 5.50. Принципиальная схема 8-входового приоритетного шифратора 74x148 с номерами выводов для стандартного корпуса DIP-16

На рис. 5.51 показано, как, используя этот принцип, можно соединить четыре микросхемы 74x148, чтобы получить 32 входа запроса и 5-разрядный выход RA4–RA0, указывающий устройство с наивысшим приоритетом из числа тех, которые послали запрос. Поскольку в любой момент времени выходы A2–A0 могут быть активными не более чем у одной микросхемы 74x148, для формирования сигналов RA2–RA0 выходы отдельных микросхем 74x148 можно подать на вентили ИЛИ. Аналогично можно получить сигналы RA4 и RA3, объединив в шифраторе 4x2 отдельные выходы GS_L. Сигнал на выходе RGS имеет активный уровень, если присутствует сигнал хотя бы на одном из выходов GS.

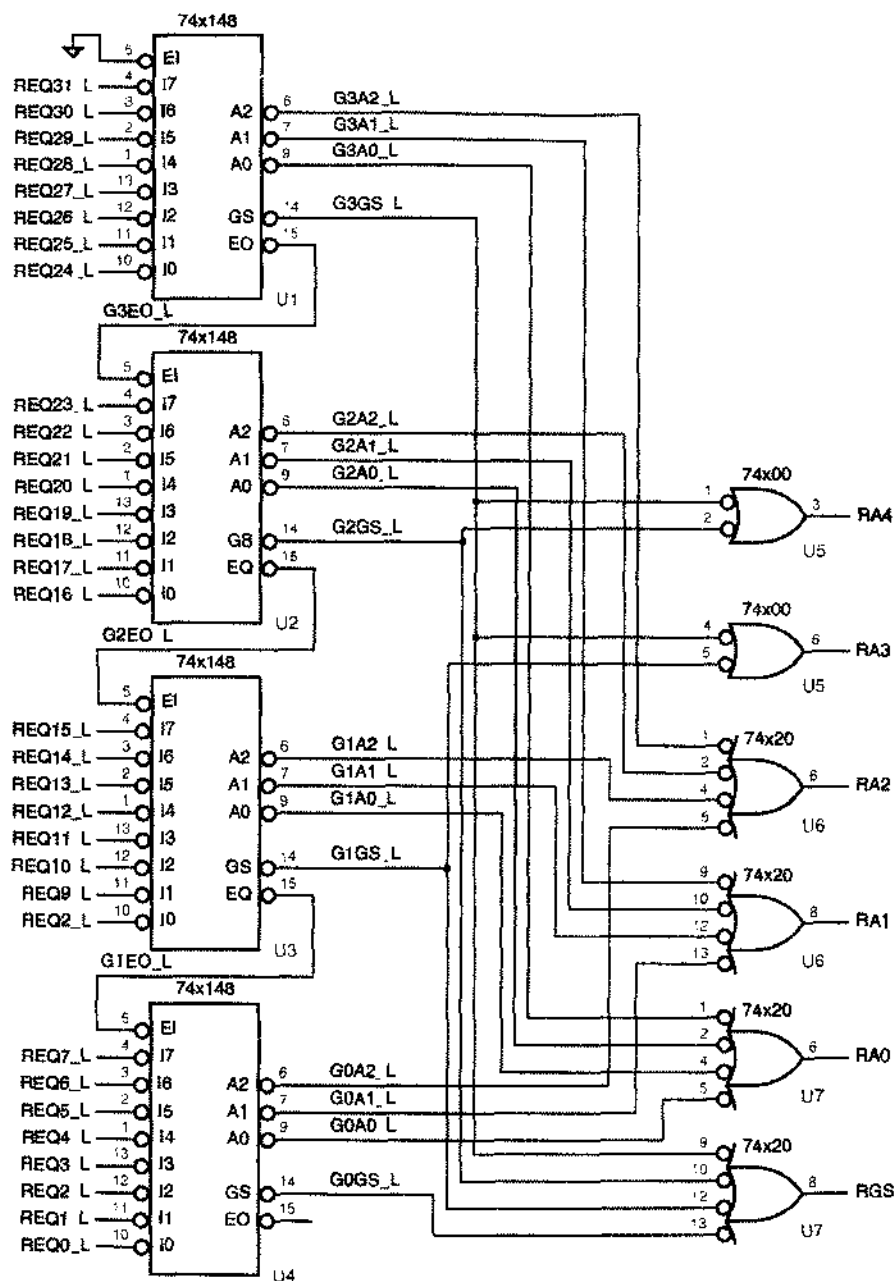


Рис. 5.51. Каскадное включение четырех микросхем 74x148 для обработки 32 запросов

Табл. 5.23. Таблица истинности для 8-входового приоритетного шифратора 74х148

Входы									Выходы				
Е1_L	10_L	11_L	12_L	13_L	14_L	15_L	16_L	17_L	A2_L	A1_L	A0_L	GS_L	EO_L
1	x	x	x	x	x	x	x	x	1	1	1	1	1
0	x	x	x	x	x	x	x	0	0	0	0	0	1
0	x	x	x	x	x	x	0	1	0	0	1	0	1
0	x	x	x	x	x	0	1	1	0	1	0	0	1
0	x	x	x	x	0	1	1	1	0	1	1	0	1
0	x	x	x	0	1	1	1	1	1	0	0	0	1
0	x	x	0	1	1	1	1	1	1	0	1	0	1
0	x	0	1	1	1	1	1	1	1	1	0	0	1
0	0	1	1	1	1	1	1	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0

5.5.8. Описание шифраторов на языке ABEL и их реализация в ПЛУ

Для описания шифраторов на языке ABEL можно воспользоваться явными соотношениями для каждой входной комбинации, как это сделано в табл. 5.8, или таблицами истинности. Однако обычно число входов велико, поэтому число входных комбинаций очень велико, и применение этого метода на практике чаще всего затруднительно.

Как мы описали бы, например, 15-входовый приоритетный шифратор с входами P0–P14? Очевидно, что мы не хотим иметь дело со всеми 2^{15} возможными входными комбинациями! Один из способов состоит в том, чтобы разбить приоритетную функцию на две части. Сначала записываем соотношения для 15 переменных H_i ($0 \leq i \leq 14$), таких что H_i равна 1, если P_i – вход с наибольшим приоритетом из числа тех входов, на которые поданы сигналы активного уровня. Так как, по определению, в любой момент времени самое большее одна переменная H_i равна 1, для получения 4-разрядного номера активного входа с наибольшим приоритетом достаточно подать сигналы H_i на двоичный шифратор.

Программа на языке ABEL, в которой использован такой подход, приведена в табл. 5.24, а на рис. 5.52 показана схема шифратора на основе одной микросхемы PAL20L8 или GAL20V8. На входы P0–P14 поступают сигналы запросов, вход P14 имеет высший приоритет. Если вход разрешения EN_L активизирован, то сигналы на выходах Y3_L–Y0_L (с низким активным уровнем) определяют номер запроса с наибольшим приоритетом, а сигнал на выходе GS имеет активный уровень, когда поступает хотя бы один запрос. Когда сигнал EN_L снят все выходные сигналы Y3_L–Y0_L и GS имеют неактивный уровень. Сигнал на выходе ENOUT_L принимает активный уровень, если активным является уровень сигнала на входе EN_L и запросы отсутствуют.

Табл. 5.24. Программа для 15-входового приоритетного шифратора на языке ABEL

```

module PRIOR15
title '15-Input Priority Encoder '
PRIOR15 device 'P20LS';

* Input and output pins
P0..P14, EN_L, pin 1..11, 13, 14, 23, 16, 17;
Y3_L, Y2_L, Y1_L, Y0_L, GS, ENOUT_L, pin 18..21, 15, 22 istype 'com';

* Active-level translation
EN = !EN_L; ENOUT = !ENOUT_L;
Y3 = !Y3_L; Y2 = !Y2_L; Y1 = !Y1_L; Y0 = !Y0_L;

* Constant expressions
H14 = EN&P14;
H13 = EN&!P14&P13;
H12 = H13&!P14&!P13&P12;
H11 = EN&!P14&!P13&!P12&P11;
H10 = EN&!P14&!P13&!P12&!P11&P10;
H9 = EN&!P14&!P13&!P12&!P11&!P10&P9;
H8 = EN&!P14&!P13&!P12&!P11&!P10&!P9&P8;
H7 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&P7;
H6 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&P6;
H5 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&!P6&P5;
H4 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&!P6&!P5&P4;
H3 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&!P6&!P5&!P4&P3;
H2 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&!P6&!P5&!P4&!P3&P2;
H1 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&!P6&!P5&!P4&!P3&!P2&P1;
H0 = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&!P6&!P5&!P4&!P3&!P2&!P1&P0;

equations

Y3 = H0 # H9 # H10 # H11 # H12 # H13 # H14;
Y2 = H4 # H5 # H6 # H7 # H12 # H13 # H14;
Y1 = H2 # H3 # H6 # H7 # H10 # H11 # H14;
Y0 = H1 # H3 # H5 # H7 # H9 # H11 # H13;

GS = EN&(P14#P13#P12#P11#P10#P9#P8#P7#P6#P5#P4#P3#P2#P1#P0);
ENOUT = EN&!P14&!P13&!P12&!P11&!P10&!P9&!P8&!P7&!P6&!P5&!P4&!P3&!P2&!P1&!P0;

end PRIOR15

```

Обратите внимание, что в программе на языке ABEL выражения для переменных H_i записаны как «иостоянные» до объявления equations. Таким образом, эти сигналы не будут образованы в явном виде. Точнее, они будут включены в последующие выражения для $Y0$ – $Y3$, которые компилятор преобразует с целью получения минимальных выражений вида «сумма произведений». Оказывается, что каждый из выходных сигналов Y_i состоит только из семи термов-произведений, что видно из структуры равенств.

Используя оператор WHEN языка ABEL, можно создать приоритетный шифратор на еще более интуитивном уровне. Как показано в табл. 5.25, последовательность вложенных один в другой операторов WHEN точно выражает логическую функцию приоритетного шифратора. Эта программа выдает точно такой же набор равенств для выходных сигналов, что и предыдущая.

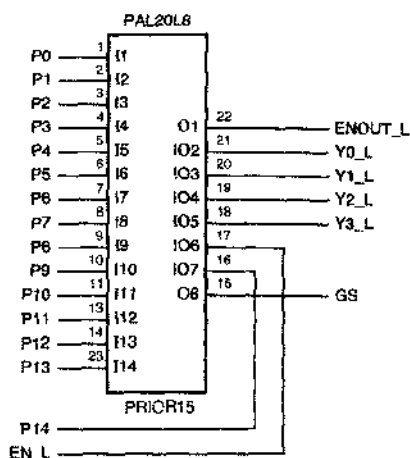


Рис. 5.52. 15-входовый приоритетный шифратор на основе ПЛУ

Табл. 5.25. Альтернативный вариант программы на языке ABEL для 15-входового приоритетного шифратора

```

module PRIOR15W
title '15-Input Priority Encoder'
PRIOR15W device 'P20L8';

" Input and output pins
P0..P14, EN_L          pin 1..11, 13, 14, 23, 16, 17;
Y3_L, Y2_L, Y1_L, Y0_L, GS, ENOUT_L  pin 18..21, 15, 22 istype 'com';

" Active-level translation
EN = !EN_L; ENOUT = !ENOUT_L;

" Set definition
Y_L = {Y3_L, Y2_L, Y1_L, Y0_L};

equations

WHEN !HN THEN !Y_L = 0;          " Note: !Y_L == active-high Y
ELSE WHEN P14 THEN !Y_L = 14;    " Can't define active-high Y set
    ELSE WHEN P13 THEN !Y_L = 13; " due to ABEL set quirks
    ELSE WHEN P12 THEN !Y_L = 12;
    ELSE WHEN P11 THEN !Y_L = 11;
    ELSE WHEN P10 THEN !Y_L = 10;
    ELSE WHEN P9 THEN !Y_L = 9;
    ELSE WHEN P8 THEN !Y_L = 8;
    ELSE WHEN P7 THEN !Y_L = 7;
    ELSE WHEN P6 THEN !Y_L = 6;
    ELSE WHEN P5 THEN !Y_L = 5;
    ELSE WHEN P4 THEN !Y_L = 4;
    ELSE WHEN P3 THEN !Y_L = 3;
    ELSE WHEN P2 THEN !Y_L = 2;
    ELSE WHEN P1 THEN !Y_L = 1;
    ELSE WHEN P0 THEN !Y_L = 0;
    ELSE {!Y_L = 0; ENOUT = 1;};

GS = EN&(P14#P13#P12#P11#P10#P9#P8#P7#P6#P5#P4#P3#P2#P1#P0);

end PRIOR15W

```

5.5.4. Описание шифраторов на языке VHDL

На языке VHDL шифраторы описываются так же, как и на языке ABEL. Мы можем ввести в программу на языке VHDL эквивалент таблицы истинности или явную запись соотношений, но гораздо нагляднее поведенческое описание шифратора. Так как в языке VHDL логическая конструкция IF-THEN-ELSE лучше всего описывает назначение приоритетов и доступна только в пределах процесса, мы применим поведенческий подход, ориентированный на использование процессов.

Табл. 5.25. Поведенческая программа на языке VHDL для 8-входового приоритетного шифратора типа 74x148

```

library IEEE;
use IEEE.std_logic_1164.all;

entity V74x148 is
    port (
        EI_L: in STD_LOGIC;
        I_L: in STD_LOGIC_VECTOR (7 downto 0);
        A_L: out STD_LOGIC_VECTOR (2 downto 0);
        EO_L, GS_L: out STD_LOGIC
    );
end V74x148;

architecture V74x148p of V74x148 is
    signal EI: STD_LOGIC;           -- active-high version of input
    signal I: STD_LOGIC_VECTOR (7 downto 0); -- active-high version of inputs
    signal EO, GS: STD_LOGIC;       -- active-high version of outputs
    signal A: STD_LOGIC_VECTOR (2 downto 0); -- active-high version of outputs
begin
    process (EI_L, I_L, EI, EO, GS, I, A)
        variable j: INTEGER range 7 downto 0;
    begin
        EI <= not EI_L; -- convert input
        I <= not I_L;   -- convert inputs
        EO <= '1'; GS <= '0'; A <= "000";
        if (EI)='0' then EO <= '0';
        else for j in 7 downto 0 loop
            if I(j)='1' then
                GS <= '1'; EO <= '0'; A <= CONV_STD_LOGIC_VECTOR(j,3);
                exit;
            end if;
        end loop;
        EO_L <= not EO; -- convert output
        GS_L <= not GS; -- convert output
        A_L <= not A;   -- convert outputs
    end process;
end V74x148p;

```

Табл. 5.26 представляет собой поведенческую программу на языке VHDL для приоритетного шифратора, эквивалентного шифратору 74x148. В ней используется цикл FOR для поиска активизированного входа, начиная со входа с высшим

приоритетом. Подобно некоторым нашим предыдущим программам, в начале и в конце программы выполняется явное преобразование активного уровня. Кроме того, вспомните, что в разделе 4.7.4 была определена функция `CONV_STD_LOGIC_VECTOR(j, n)` преобразования целого числа j в вектор `STD_LOGIC_VECTOR` заданной длины n . Эту программу легко модифицировать, если нужно задать приоритеты в другом порядке или изменить число входов, а также с целью расширения функциональных возможностей типа обнаружения входа, имеющего «второй по старшинству уровень приоритета», о чем будет сказано в разделе 6.2.3.

5.6. Устройства с тремя состояниями

В разделах 3.7.3 и 3.10.5 были описаны электрические схемы КМОП- и TTL-устройств, выходы которых могут находиться в одном из трех состояний: 0, 1 и Hi-Z. В этом параграфе мы покажем, как применять такие устройства.

5.6.1. Буферы с тремя состояниями

Основным устройством с тремя состояниями является *буфер с тремя состояниями* (*three-state buffer*), который часто называют *драйвером с тремя состояниями* (*three-state driver*). На рис. 5.53 показаны условные обозначения четырех конструктивно различных буферов с тремя состояниями. На рис. (a) и (b) изображены неинвертирующие буферы, а на рис. (c) и (d) – инверторы. У схем с тремя состояниями имеется *вход разрешения* (*three-state enable*), указываемый в условном обозначении сверху. Когда на этот вход подан сигнал разрешения, который может иметь высокий активный уровень [рис. (a) и (c)] или низкий активный уровень [рис. (b) и (d)], устройство ведет себя как обыкновенный буфер или инвертор. Когда сигнал на входе разрешения имеет неактивный уровень, выход устройства становится «плавающим», то есть переходит в высокоимпедансное, разомкнутое состояние (Hi-Z) и функционально ведет себя так, как будто там ничего нет.



Рис. 5.53. Различные буферы с тремя состояниями: (a) неинвертирующий, с высоким активным уровнем сигнала разрешения; (b) неинвертирующий, с низким активным уровнем сигнала разрешения; (c) инвертирующий, с высоким активным уровнем сигнала разрешения; (d) инвертирующий, с низким активным уровнем сигнала разрешения

Устройства с тремя состояниями позволяют нескольким источникам совместно использовать одну «линию коллективного пользования» при условии, что в это время на линии «говорят» только одно устройство. На рис. 5.54 приведен пример, как это можно сделать. Тремя входными битами `SSRC2–SSRC0` выбирается один из восьми источников данных, который может выдавать сигналы на единственную линию `SDATA`. Дешифратор 3×8 типа 74x138 обеспечивает на данном отрезке вре-

мени активизацию сигнала только на одной из восьми линий SEL, разрешая только одному буферу с тремя состояниями выдавать данные на линию SDATA. Но если активный уровень присутствует не на всех входах EN, то все буферы остаются в третьем состоянии. В этом случае логическое значение на линии SDATA не определено.

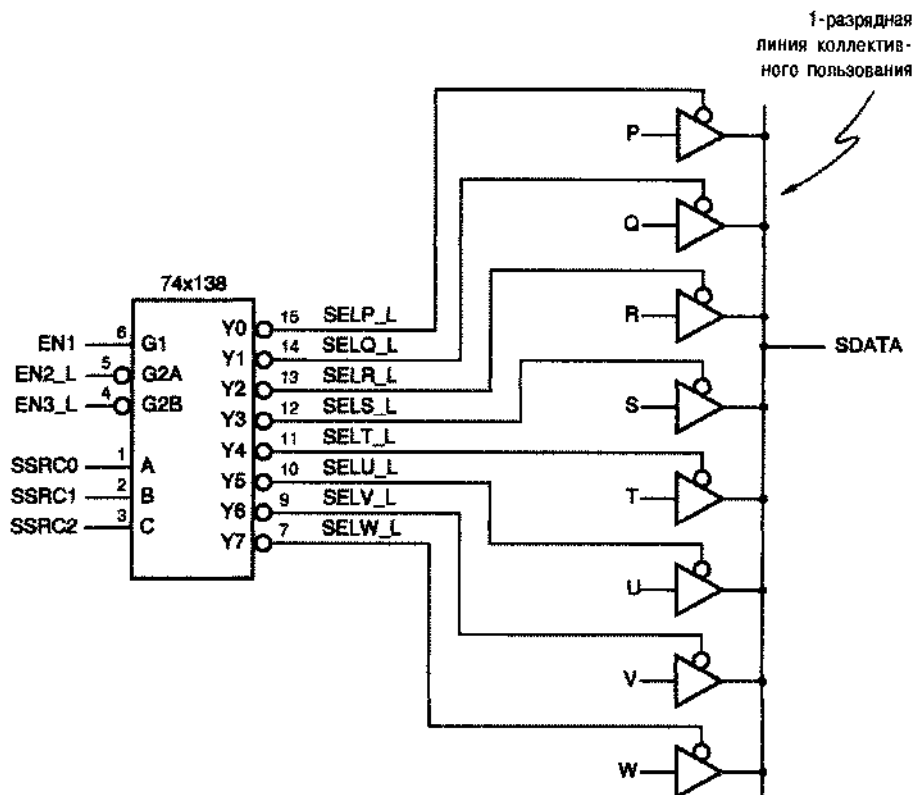


Рис. 5.54. Восемь источников сигналов с тремя состояниями, совместно использующих линию коллективного пользования

Типичные устройства с тремя состояниями создаются так, чтобы переход в состояние Hi-Z происходил быстрее, чем выход из этого состояния. (В обозначениях, принятых в справочных данных, обе величины t_{pLZ} и t_{pHZ} меньше, чем значения t_{pZL} и t_{pZN} ; см. также раздел 3.7.3.) Это означает, что в случае, когда выходы двух устройств с тремя состояниями подключены к одной и той же линии коллективного пользования и мы одновременно переводим в третье состояние одно из устройств и выводим из этого состояния другое устройство, первое устройство отключается от линии коллективного пользования прежде, чем будет подключено второе. Это важный момент. Действительно, если бы оба устройства одновременно были открыты и попытались установить на линии противоположные значения выходных сигналов (0 и 1), то в системе потек бы чрезмерно большой ток, создавая помехи (см. раздел 3.7.7). Когда такая ситуация возникает, ее часто называют *борьбой* (*fighting*).

ОПРЕДЕЛЕНИЕ ПОНЯТИЯ «НЕОПРЕДЕЛЕННЫЙ»

Фактический уровень напряжения плавающего сигнала зависит от конкретных значений сопротивления и емкости нагрузки и может изменяться со временем. Кроме того, реакция других схем на такой уровень сигнала зависит от входных характеристик этих схем, так что лучше не приписывать плавающему сигналу никакого другого значения, кроме «неопределенного». Иногда, для того чтобы напряжение на плавающем выходе было приближено к напряжению высокого уровня и интерпретировалось как логическая 1, линию коллективного пользования, имеющую источники с тремя состояниями, соединяют резистором с шиной питания. Это особенно важно для линий коллективного пользования, управляющих КМОП-устройствами, которые могут потреблять от источника питания слишком большой ток, когда напряжение на их входах находится посредине между логическим 0 и логической 1.

К сожалению, из-за задержек в схеме управления и асимметрии временных характеристик трудно обеспечить «одновременное» изменение сигналов на входах разрешения различных устройств с тремя состояниями. Но даже если это оказалось бы возможным, мы столкнулись бы с той же проблемой в случае подключения к линии коллективного пользования устройств с тремя состояниями из логических семейств с различным быстродействием (или даже микросхем одной серии, изготовленных в разные дни). Время включения (t_{pZL} или t_{pZH}) «быстрого» устройства может быть меньше, чем время выключения (t_{pLZ} и t_{pHZ}) «медленного» устройства, и выходы могут по-прежнему участвовать в борьбе.

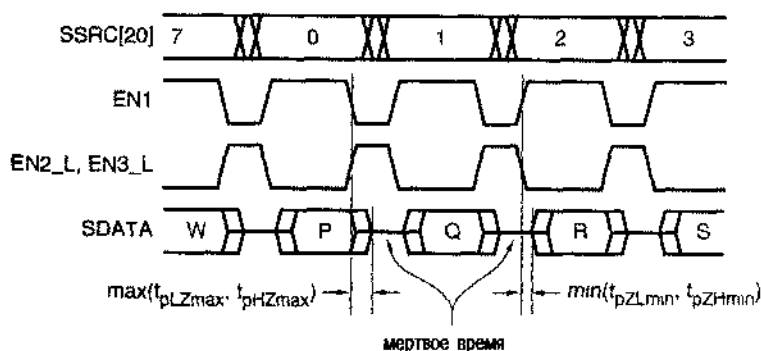


Рис. 5.55. Временные диаграммы сигналов для линии коллективного пользования, изображенной на рис. 5.54

Единственный реально безопасный способ применения устройств с тремя состояниями заключается в применении такой логики управления, которая гарантирует мертвое время (*dead time*) на линии коллективного пользования, в течение которого все устройства находятся в третьем состоянии. Мертвое время должно быть достаточно большим по сравнению с разницей между временами выключения и включения устройств в худшем случае и по сравнению с наибольшей

разницей во времени прихода сигналов, управляющих третьим состоянием. На рис. 5.55 приведены временные диаграммы, иллюстрирующие такой режим работы линии коллективного пользования, изображенной на рис. 5.54. Эти временные диаграммы показывают также принятое обозначение сигналов в схемах с тремя состояниями: когда устройство находится в состоянии Hi-Z, сигнал на его выходе изображается «неопределенным» уровнем посередине между 0 и 1.

5.6.2. Стандартные буферы с тремя состояниями в виде ИС малой и средней степени интеграции

Подобно логическим вентилям, в одной ИС малой степени интеграции могут быть размещены несколько независимых буферов с тремя состояниями. Например, на рис. 5.56 показана цоколевка микросхем 74x125 и 74x126, каждая из которых содержит в корпусе с 14 выводами по четыре независимых неинвертирующих буфера с тремя состояниями. Входы управления третьим состоянием у микросхем 74x125 имеют низкий активный уровень сигнала, а у микросхем 74x126 – высокий активный уровень сигнала.

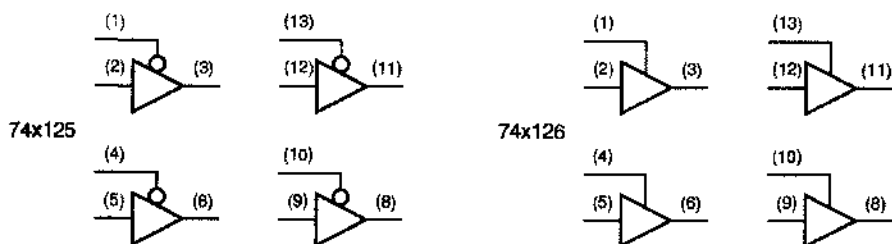


Рис. 5.56. Цоколевка микросхем 74x125 и 74x126, содержащих буферы с тремя состояниями

В большинстве случаев число линий в шине коллективного пользования больше единицы, что позволяет передавать по шине несколько битов данных. Например, в 8-разрядной микропроцессорной системе шина данных состоит из восьми сигнальных линий, периферийные устройства обычно выдают данные на шину в виде восьми битов одновременно. Следовательно, периферийное устройство позволяет восьми буферам с тремя состояниями одновременно выдавать сигналы на шину. В этом случае нет необходимости иметь независимые входы разрешения, как это предусмотрено в микросхемах 74x125 и 74x126.

Поэтому для уменьшения размера корпуса при использовании многоразрядных шин обычно применяют ИС, содержащие много буферов с тремя состояниями и общим входом разрешения. На рис. 5.57, например, показаны принципиальная схема и условное обозначение ИС 74x541, содержащей восемь отдельных неинвертирующих буферов с тремя состояниями. Чтобы разрешить устройству выдавать свои сигналы на шину, сигналы G1_L и G2_L на обоих входах разрешения должны иметь активный уровень. Небольшие прямоугольные символы внутри условного обозначения буферов указывают на наличие у входных характеристик гистерезиса (hysteresis), повышающего помехоустойчивость (см. раздел 3.7.2). Типичное значение ширины петли гистерезиса у микросхемы 74x541 составляет 0,4 вольта.

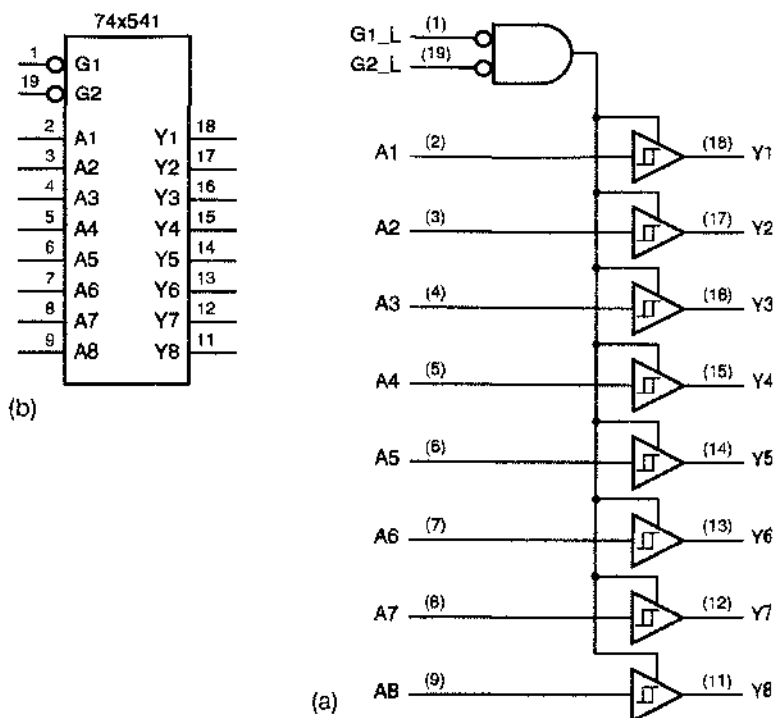


Рис. 5.57. ИС 74х541, содержащая восемь буферов с тремя состояниями: (а) принципиальная схема и цоколевка для стандартного корпуса DIP с 20 выводами, (б) традиционное условное обозначение

На рис. 5.58 приведен фрагмент микропроцессорной системы с 8-разрядной шиной данных DB [0–7], где в качестве входных портов применены ИС 74х541. Микропроцессор выбирает входной порт 1 путем выдачи единичного сигнала INSEL1 и устанавливает режим чтения, вырабатывая единичный сигнал READ. Выбранная ИС 74х541 передает на микропроцессорную шину данные поступающие от пользователя данные. Выдача других единичных сигналов INSEL совместно с единичным сигналом READ позволяет выбирать другие входные порты.

В продаже имеется много других ИС, содержащих по 8 буферов с тремя состояниями. Примером может служить ИС 74х540, идентичная ИС 74х541, за исключением того, что буферы в ней – инвертирующие. Микросхемы 74х240 и 74х241 подобны схемам '540 и '541, за исключением того, что они разбиты на две 4-разрядные группы, у каждой из которых своя, общая для четырех буферов линия разрешения.

Шинный приемопередатчик (bus transceiver) между каждой парой выводов содержит два буфера с тремя состояниями, включенные в противоположных направлениях, так что данные могут передаваться в любую сторону. Например, на рис. 5.59 приведена принципиальная схема и условное обозначение ИС 74х245, содержащей восемь приемопередатчиков с тремя состояниями. Сигналом на вхо-

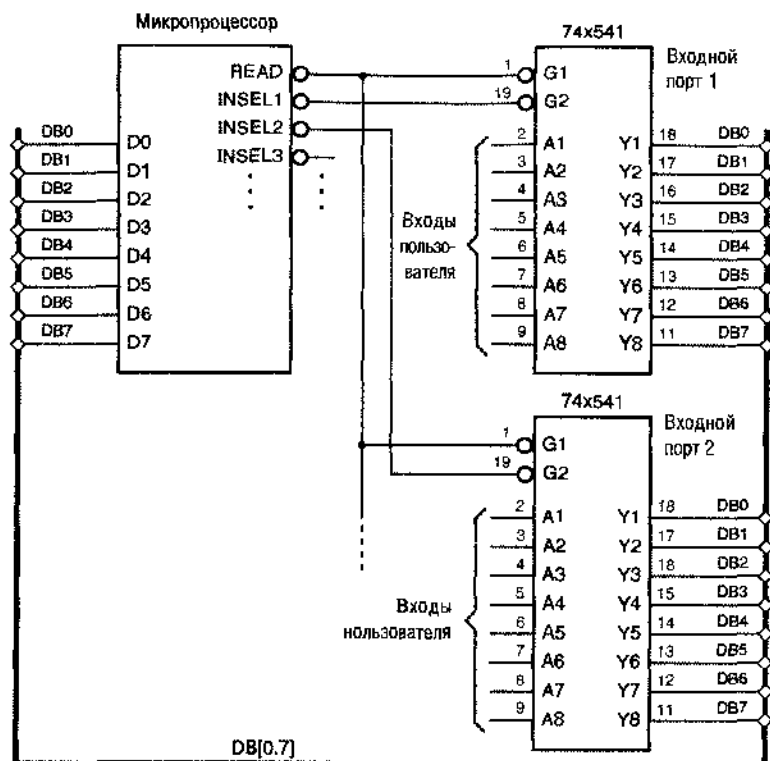


Рис. 5.58. Использование микросхем 74x541 в качестве входных портов микропроцессора

де DIR определяется направление передачи: от шины A к шине B (когда DIR = 1) или в обратном направлении (когда DIR = 0). Буфер с тремя состояниями передает данные в выбранном направлении только при условии, что сигнал G_L имеет активный уровень.

Шинный приемопередатчик обычно включается между двумя двунаправленными шинами (*bidirectional buses*), как изображено на рис. 5.60. Из табл. 5.27 следует, что возможны три различных режима работы в зависимости от значений сигналов G_L и DIR. Как правило, заботой разработчика является обеспечение того, чтобы ни на одну из шин одновременно не поступали сигналы от двух устройств. Правда, в случае, когда приемопередатчик заблокирован, чему соответствует последняя строка таблицы, возможна одновременная независимая передача данных по обеим шинам.

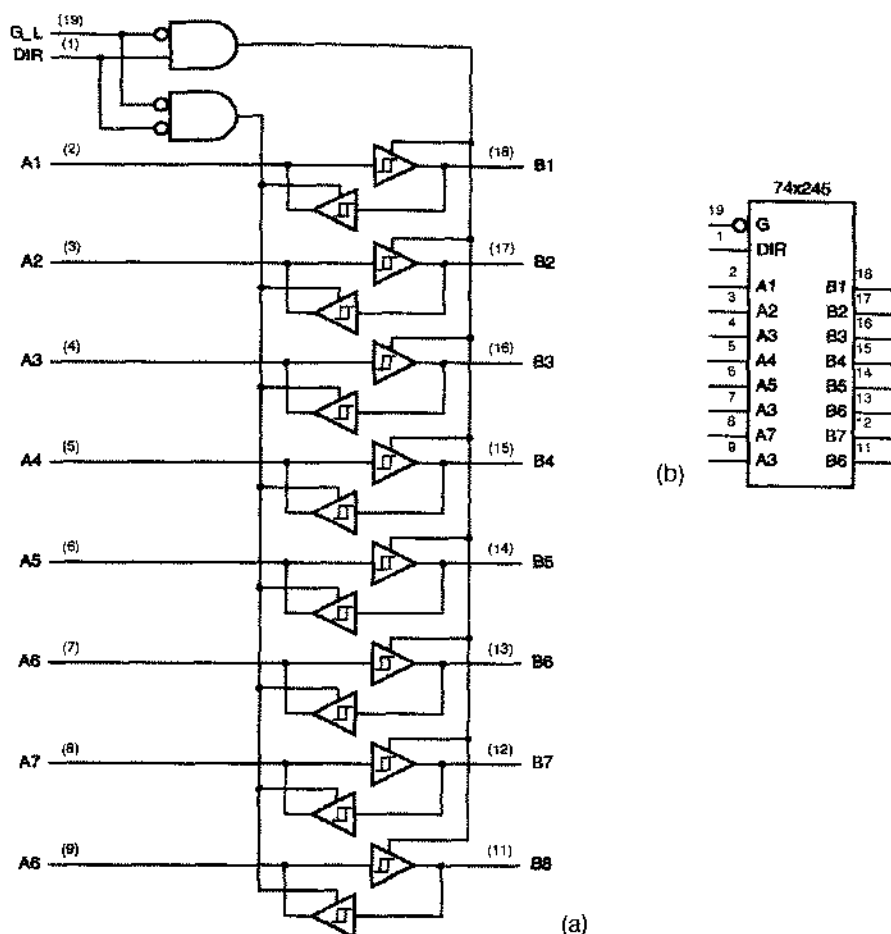


Рис. 5.59. ИС 74x245, содержащая восемь двунаправленных буферов с тремя состояниями (а) принципиальная схема, (б) традиционное условное обозначение

Табл. 5.27. Режимы работы шинного приемопередатчика с парой двунаправленных шин

ENTFR_L	АТОВ	Операция
0	0	Передача данных от источника на шине В адресату на шине А.
0	1	Передача данных от источника на шине А адресату на шине В.
1	x	Независимая передача данных по шинам А и В.

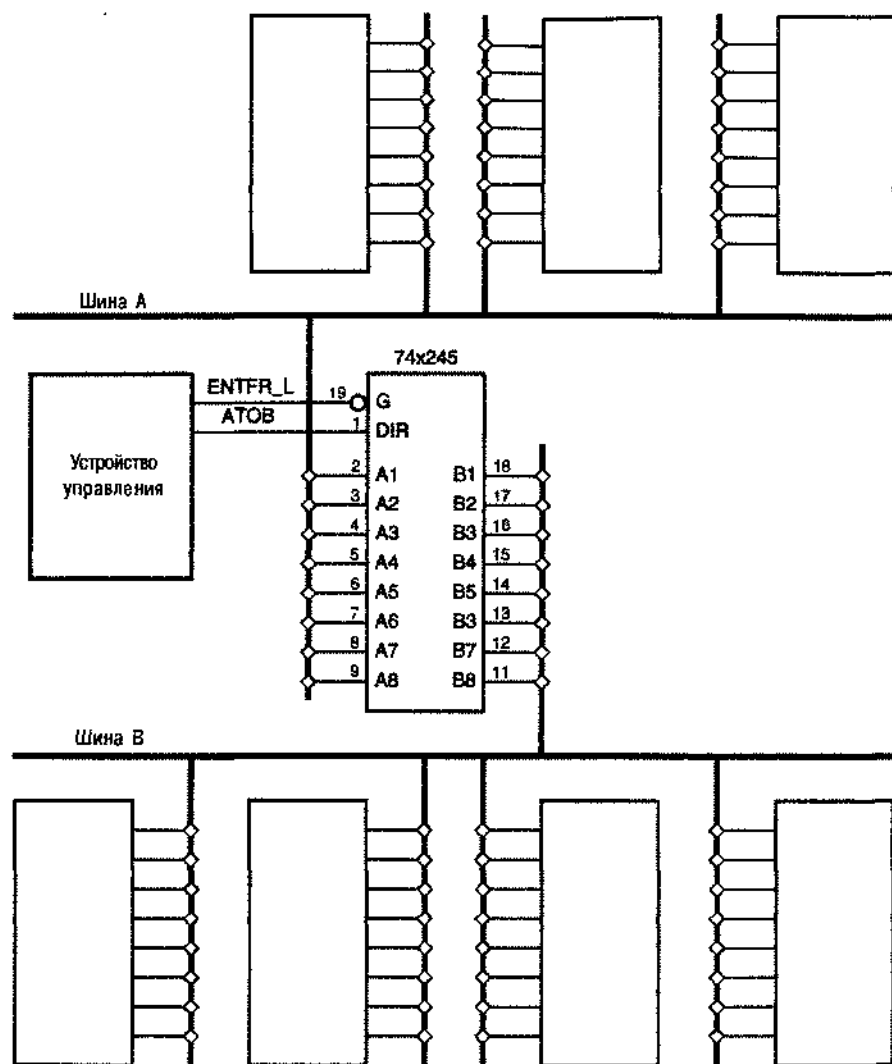


Рис. 5.60. Двусторонние шины и применение приемопередатчика

5.5.3. Описание схем с тремя состояниями на языке ABEL и их реализация в ПЛУ

В рассмотренных ранее примерах применения комбинационных ПЛУ с двусторонними выводами (IO2–IO7 в микросхемах PAL16L8 и GAL16V8) эти выводы использовались в статическом режиме, то есть выход, подключенный к данному выводу постоянно был либо в активном режиме, либо в третьем состоянии. В такой ситуации о соответствующем программировании выходов может позабо-

тятся компилятор: все лавкие перемиычки будут либо нережежены, либо все останутся нетронутыми. В языке ABEL, по умолчанию, выход с тремя состояниями, подключенный к данному выводу, бывает запрограммирован так, чтобы всегда быть в активном режиме, если ния сигнала, соответствующее этому выводу, появляется в левой части какого-либо равенства, или всегда находиться в третьем состоянии в противном случае.

Выходами с тремя состояниями можно также управлять динамически: сигналом на отдельном входе, термом-произведением или используя логику с двумя проходами для более сложных логических выражений. В языке ABEL к имени сигнала в левой части выражения добавляется *суффикс атрибута .OE (attribute suffix .OE)*, чтобы показать, что это соотношение относится к сигналу разрешения буфера, через который проходит сигнал с данным именем. В ИС PAL16L8 и GAL16V8 перевод выхода в активный режим осуществляется единственным вентилем И, поэтому правую часть соотношения, задающего сигнал разрешения, необходимо свести к одному терму-произведению.

В табл. 5.28 приведен фрагмент простой программы для ПЛУ с управлением третьим состоянием. Эта программа получена адаптацией программы для дешифратора типа 74x138 из табл. 5.8 и включает сигнал управления третьим состоянием на выходе OE для всех восьми выходов дешифратора. Обратите внимание, что набор сигналов Y определен так, чтобы единственное соотношение определяло возможность всем восьми выходам находиться в активном состоянии; здесь суффикс .OE относится к каждому элементу набора.

В предыдущем примере выводы Y0–Y7 в течение всего времени являются выходами, находящимися либо в активном режиме, либо в плавающем состоянии, то есть используются строго как «выходные контакты». Но выводы I/O (IO2–IO7 в ИС 16L8 и 16V8) можно применять как «двунаправленные выводы»; то есть их можно использовать динамически как входы или как выходы в зависимости от того, какой сигнал вырабатывает вентиль, управляющий выходным буфером: логический 0 или логическую 1. Ниже в качестве примера использования выводов I/O приведены условия функционирования 2-разрядного шивийного приемонередатчика на четыре направления:

- Приемонередатчик работает на четыре 2-разрядные двунаправленные шины A[1:2], B[1:2], C[1:2] и D[1:2].
- Источник данных, выдающий сигналы на шины, определяется тремя входными сигналами выбора S[2:0] согласно табл. 5.29. Если S2 = 0, то на шинах устанавливаются постоянные значения, в противном случае сигналы на них определяются сигналами одной из других шин. Однако, когда выбранный источник данных сам является шивий-адресатом, на шине источника устанавливается 00.
- Каждая шина имеет свой собственный сигнал разрешения выхода AOE_L, BOE_L, COE_L или DOE_L. Кроме того, имеется «главный» сигнал разрешения выхода MOE_L. Приемонередатчик выдает сигналы на конкретную шину только в том случае, когда сигнал MOE_L и сигнал разрешения выхода на эту шину имеют активный уровень.

Табл. 5.28. Программа на языке ABEL для полного дешифратора 3×8 типа 74х138 с управлением третьим состоянием на выходах

```

module Z74X138T
title '74x138 Decoder with Three-State Output Enable'
Z74X138T device 'P16L8';

" Input pins
A, B, C, !G2A, !G2B, G1, !OE pin 1, 2, 3, 4, 5, 6, 7;
" Output pins
!Y0, !Y1, !Y2, !Y3                pin 19, 18, 17, 16 istype 'com';
!Y4, !Y5, !Y6, !Y7                pin 15, 14, 13, 12 istype 'com';

" Constant expression
ENB = G1 & G2A & G2B;
Y = {Y0..Y7};

equations
Y.OE = OE;
Y0 = ENB & !C & !B & !A;
...
Y7 = ENB & C & B & A;

end Z74X138T

```

Табл. 5.29. Коды выбора шины для шинного приемопередатчика на четыре направления

S2	S1	S0	Выбираемый источник
0	0	0	00
0	0	1	01
0	1	0	10
0	1	1	11
1	0	0	Шина А
1	0	1	Шина В
1	1	0	Шина С
1	1	1	Шина D

В табл. 5.30 представлена программа на языке ABEL, описывающая работу приемопередатчика. Согласно выражениям для сигналов разрешения (.OE), каждая шина может служить источником данных, если сигнал MOE и собственный сигнал OE этой шины имеют активный уровень. Если S2 = 0, то сигналы на каждой шине определяются значениями S1 и S0; если в качестве источника выбрана дру-

гая шина, то сигналы на данной шине определяются сигналами выбранной шины. Если, в качестве писточипка выбрана сама шина-адресат, то результатом вычисления выходного выражения, как и требуется, будет 00.

Табл. 5.30. Программа на языке ABEL для 2-разрядного шинного приемопередатчика на четыре направления

```

module XCVR4X2
title 'Four-way 2-bit Bus Transceiver'
XCVR4X2 device 'P16L8';

" Input pins
A1I, A2I                                pin 1, 11;
!AOE, !BOE, !COE, !DOE, !MOE           pin 2, 3, 4, 5, 6;
S0, S1, S2                               pin 7, 8, 9;
" Output and bidirectional pins
A1O, A2O                                pin 19, 12          istype 'com';
B1, B2, C1, C2, D1, D2                  pin 18, 17, 16, 15, 14, 13 istype 'com';

" Set definitions
ABUS0 = [A1O, A2O];
ABUS1 = [A1I, A2I];
BBUS  = [B1, B2];
CBUS  = [C1, C2];
DBUS  = [D1, D2];
SEL   = [S2, S1, S0];
CONST = [S1, S0];

" Constants
SELA = [1, 0, 0];
SELB = [1, 0, 1];
SELC = [1, 1, 0];
SELD = [1, 1, 1];

equations
ABUS0.OE = AOE & MOE;
BBUS.OE  = BOE & MOE;
CBUS.OE  = COE & MOE;
DBUS.OE  = DOE & MOE;
ABUS0 = !S2&CONST # (SEL==SELB)&BBUS # (SEL==SELC)&CBUS # (SEL==SELD)&DBUS;
BBUS  = !S2&CONST # (SEL==SELA)&ABUS1 # (SEL==SELC)&CBUS # (SEL==SELD)&DBUS;
CBUS  = !S2&CONST # (SEL==SELA)&ABUS1 # (SEL==SELB)&BBUS # (SEL==SELD)&DBUS;
DBUS  = !S2&CONST # (SEL==SELA)&ABUS1 # (SEL==SELB)&BBUS # (SEL==SELC)&CBUS;

end XCVR4X2

```

На рис. 5.61 представлена принципиальная схема устройства на ИС PAL16L8 (или GAL16V8) с требуемыми входами и выходами. Так как ИС имеет только шесть двунаправленных выводов, а согласно техническим требованиям их должно быть восемь, для шины А используется одна пара входов и одна пара выходов. Это отражено в программе путем введения отдельных сигналов и наборов для входа и выхода шины А.

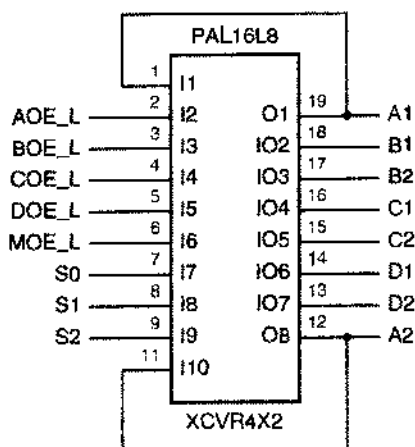


Рис. 5.61. Входы и выходы 2-разрядного шинного приемопередатчика на четыре направления, реализованного в ПЛУ

*5.5.4. Описание выходов с тремя состояниями на языке VHDL

В самом языке VHDL отсутствуют встроенные типы и операторы для выходов с тремя состояниями. Однако в нем имеются примитивы, которые можно использовать для создания соответствующих сигналов и систем, имеющих третье состояние; эти примитивы используются в пакете IEEE 1164. Прежде всего, объявлением типа `STD_LOGIC` в пакете IEEE 1164 'Z' определяется как одно из девяти возможных значений сигнала; это значение используется для указания высокоомного состояния. Вы можете присвоить это значение любому сигналу типа `STD_LOGIC` и определения стандартных логических функций допускают возможность входных сигналов со значением 'Z' (в общем случае сигнал 'Z' на входе вызовет сигнал 'U' на выходе).

Как на языке VHDL описать шины с тремя состояниями, если в нашем распоряжении имеются сигналы с тремя состояниями? У шины с тремя состояниями в общем случае бывает два или большее число источников, хотя рассматриваемые нами алгоритмы работают точно так же, когда имеется всего лишь один источник. В языке VHDL нет явной языковой конструкции для объединения выходов с тремя состояниями в шину. Вместо этого компилятор автоматически объединяет вместе сигналы, которые создаются двумя или большим числом разных процессов, то есть сигналы, имена которых находятся в левой части оператора присваивания в двух или в большем числе процессов. Однако, как объяснено ниже, эти сигналы должны иметь соответствующий тип.

В пакете IEEE 1164 тип `STD_LOGIC` фактически определен как *подтип* (subtype) типа `STD_ULOGIC`. В языке VHDL тип `STD_ULOGIC`, называемый *неразрешенным типом* (unresolved type), используется для любого сигнала, который может

быть образован в двух или в большем числе процессов. (Здесь термин «разрешение» употребляется в том же значении, какое он имеет в выражениях «разрешение конфликта», «разрешающая способность». – Прим. перев.) Определение неразрешенного типа включает в себя функцию разрешения (*resolution function*), которая вызывается каждый раз, когда происходит присвоение значения сигналу этого типа. Как следует из названия функции, она решает, каким должно быть результирующее значение сигнала на линии, к которой подключены выходы нескольких источников. Это позволяет смоделировать условия формирования сигналов, подобные тем, какие имеют место в реальных устройствах.

В табл. 5.31 и 5.32 приведены определения типов `STD_ULOGIC` и `STD_LOGIC` и функция разрешения “resolved” из пакета IEEE 1164. Когда в процессах возникает *n* различных возможных значений сигнала, они поступают во входной вектор *s* и функция “resolved” определяет конечное значение типа `STD_LOGIC` по двумерному массиву `resolution_table`. Если, например, сигнал формируется четырьмя источниками, то VHDL-компилятор автоматически создает 4-элементный вектор, содержащий значения сигналов на выходах источников, и передает этот вектор функции `resolved` всякий раз, когда изменяется любое из этих значений. Результат возвращается моделирующей программе.

Табл. 5.31. Объявления типов `STD_ULOGIC` и `STD_LOGIC` в пакете IEEE 1164

```

PACKAGE std_logic_1164 IS
  -- logic state system (unresolved)
  TYPE std_ulogic IS ( 'U', -- Uninitialized
                       'X', -- Forcing Unknown
                       '0', -- Forcing 0
                       '1', -- Forcing 1
                       'Z', -- High Impedances
                       'W', -- Weak Unknown
                       'L', -- Weak 0
                       'H', -- Weak 1
                       '-' -- Don't care
                     );

  -- unconstrained array of std_ulogic
  TYPE std_ulogic_vector IS ARRAY ( NATURAL RANGE <> ) OF std_ulogic;

  -- resolution function
  FUNCTION resolved ( s : std_ulogic_vector ) RETURN std_ulogic;

  -- *** industry standard logic type ***
  SUBTYPE std_logic IS resolved std_ulogic;
  ...

```

Благодаря строгому упорядочению сигналов по «силе» в массиве `resolution_table` ('U' > 'X' > '0', '1' > 'W' > 'L', 'H' > '-') порядок, в котором значения сигналов появляются в векторе *s*, не влияет на результат, выдаваемый функцией `resolved`. Чем скорее в результате частичного выполнения про-

предыдущего разрешения уже получено какое-то определенное значение сигнала, в дальнейшем уже не может возникнуть «более слабое» значение; конфликты 0/1 и L/H всегда разрешаются в пользу более сильных неопределенных значений 'X' и 'W' соответственно.

Табл. 5.32. Тело пакета IEEE 1164 в части, касающейся типов STD_ULOGIC и STD_LOGIC

```

PACKAGE BODY std_logic_1164 IS
-- local type
    TYPE stdlogic_table IS ARRAY(std_ulogic, std_ulogic) OF std_ulogic;

-- resolution function
    CONSTANT resolution_table : stdlogic_table := (
--
--      | U  X  0  1  Z  W  L  H  -  |
--
--      ( 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', 'U', ) -- | U |
--      ( 'U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X', ) -- | X |
--      ( 'U', 'X', '0', 'X', '0', '0', '0', '0', 'X', ) -- | 0 |
--      ( 'U', 'X', 'X', '1', '1', '1', '1', '1', 'X', ) -- | 1 |
--      ( 'U', 'X', '0', '1', 'Z', 'W', 'L', 'H', 'X', ) -- | Z |
--      ( 'U', 'X', '0', '1', 'W', 'W', 'W', 'W', 'X', ) -- | W |
--      ( 'U', 'X', '0', '1', 'L', 'W', 'L', 'W', 'X', ) -- | L |
--      ( 'U', 'X', '0', '1', 'H', 'W', 'W', 'H', 'X', ) -- | H |
--      ( 'U', 'X', 'X', 'X', 'X', 'X', 'X', 'X', 'X', ) -- | ~ |
--    );

    FUNCTION resolved ( s : std_ulogic_vector ) RETURN std_ulogic IS
        VARIABLE result : std_ulogic := 'Z'; -- weakest state default
    BEGIN
        -- the test for a single driver is essential otherwise the
        -- loop would return 'X' for a single driver of '-' and that
        -- would conflict with the value of a single driver unresolved
        -- signal.
        IF (s'LENOTH = 1) THEN RETURN s(s'LOW);
        ELSE
            FOR i IN s'RANGE LOOP
                result := resolution_table(result, s(i));
            END LOOP;
        END IF;
        RETURN result;
    END resolved;
END std_logic_1164;

```

Итак, пужно ли все это знать для образования в VHDL-программе выходов с тремя состояниями? Ну, обычно ист. Но это может помочь, если результаты вашего моделирования не согласуются с реальностью. Все, что обычно требуется для создания выходов с тремя состояниями в VHDL-программе, это объявить соответствующие сигналы величинами типа STD_LOGIC, и пусть моделирующая программа осуществляет разрешение по мере необходимости.

В табл. 5.33 в качестве примера описана система, в которой четыре 8-разрядных драйвера с тремя состояниями (в четырех процессах) используются для выбора одной из четырех 8-разрядных шин A, B, C и D для выдачи результата на шину X. Напомним, что конструкция "(others => 'Z')" означает вектор, все элементы которого имеют значение 'Z', а длина определяется требованием согласования с левой частью оператора присваивания.

Табл. 5.33. VHDL-программа с четырьмя 8-разрядными драйверами с тремя состояниями

```

library IEEE;
use IEEE.std_logic_1164.all;

entity V3statex is
    port (
        G_L: in STD_LOGIC;           -- Global output enable
        SEL: in STD_LOGIC_VECTOR (1 downto 0); -- Input select 0,1,2,3 ==> A,B,C,D
        A, B, C, D: in STD_LOGIC_VECTOR (1 to 0); -- Input buses
        X: out STD_ULOGIC_VECTOR (1 to 8)         -- Output bus (three-state)
    );
end V3statex;

architecture V3states of V3statex is
begin
    process (G_L, SEL, A)
    begin
        if G_L='0' and SEL = "00" then X <= To_StdULogicVector(A);
        else X <= (others => 'Z');
        end if;
    end process;

    process (G_L, SEL, B)
    begin
        if G_L='0' and SEL = "01" then X <= To_StdULogicVector(B);
        else X <= (others => 'Z');
        end if;
    end process;

    process (G_L, SEL, C)
    begin
        if G_L='0' and SEL = "10" then X <= To_StdULogicVector(C);
        else X <= (others => 'Z');
        end if;
    end process;

    process (G_L, SEL, D)
    begin
        if G_L='0' and SEL = "11" then X <= To_StdULogicVector(D);
        else X <= (others => 'Z');
        end if;
    end process;
end V3states;

```

Язык VHDL достаточно гибок и им можно воспользоваться для определения других типов операций на шине. Можно было бы, например, определить подтип и функцию разрешения для выходов с открытым стоком, чтобы реализовать функцию «монтажное И». Однако эту возможность используют редко, поскольку

определение необходимых типов выходов для ПЛУ, схем FPGA и специализированных ИС, как правило, уже бывает сделано за вас в библиотеках, предоставляемых производителями этих компонентов.

5.7. Мультиплексоры

Мультиплексором (multiplexer) называется цифровой переключатель, который осуществляет передачу на выход данных, поступающих от одного из n источников. На рис. 5.62(а) изображены входы и выходы n -входового b -разрядного мультиплексора. Имеются n источников b -разрядных данных и b -разрядный выход. У типичных, выпускаемых серийно мультиплексоров $n = 1, 2, 4, 8$ или 16 , а $b = 1, 2$ или 4 . Имеются s входов, с помощью которых выбирается один из n источников, поэтому $s = \lceil \log_2 n \rceil$. По сигналу на входе разрешения EN мультиплексор «выполняет свою работу»; когда $EN = 0$, сигналы на всех выходах равны 0. По-английски, для краткости, мультиплексор часто называют *mux*.

На рис. 5.62(б) приведена схема переключения, являющаяся грубым эквивалентом мультиплексора. Но, в отличие от механического переключателя, мультиплексор является однонаправленным устройством: информационные потоки направлены только от входов (расположенных слева) к выходам (расположенным справа).

Для сигналов на выходе мультиплексора можно записать обычное логическое выражение:

$$iY = \sum_{j=0}^{n-1} EN \cdot M_j \cdot iD_j.$$

Символ суммирования означает здесь логическую сумму термов-произведений. Переменная iY — это i -й выходной бит ($1 \leq i \leq b$), а переменная iD_j — i -й входной бит от j -го источника ($0 \leq j \leq n-1$). M_j представляет собой минтерм j , который содержит s входных сигналов выбора. Таким образом, когда на вход мультиплексора подан сигнал разрешения, а число на входах выбора равно j , сигнал на каждом выходе iY принимает значение iD_j соответствующего бита выбранного входа.

Очевидно, что мультиплексоры являются полезными устройствами в любом приложении, где данные от многих источников должны быть переданы адресату. Распространенным является применение мультиплексора в компьютерах между регистрами процессора и его арифметическо-логическим устройством (АЛУ). Рассмотрим, например, 16-разрядный процессор, у которого каждая команда имеет 3-разрядное поле, определяющее один из восьми используемых регистров. Сигналы с этого 3-разрядного поля поступают на входы выбора 8-входового 16-разрядного мультиплексора. Входы данных мультиплексора связаны с восемью регистрами, а данные с его выходов поступают в АЛУ для выполнения команды, использующей выбранный регистр.

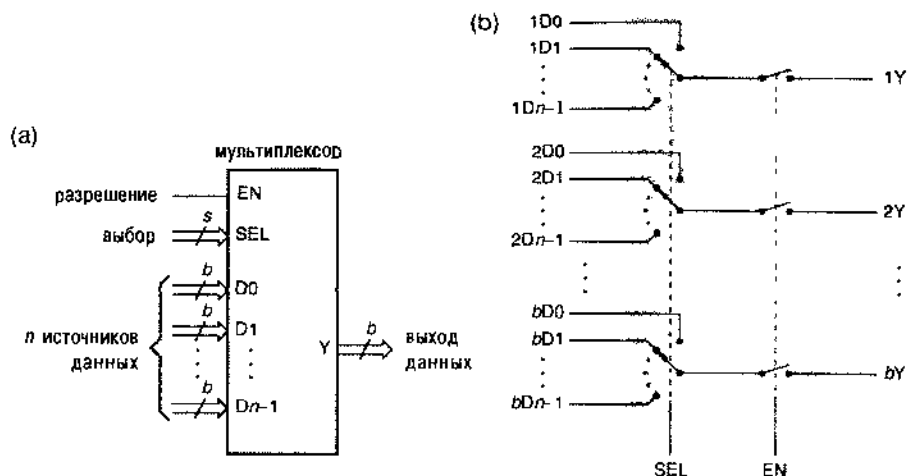


Рис. 5.62. Структура мультиплексора: (а) входы и выходы, (б) функциональный эквивалент

5.7.1. Стандартные мультиплексоры в интегральном исполнении

Возможности мультиплексоров, серийно выпускаемых в виде ИС средней степени интеграции, ограничены числом выводов у дешевых корпусов ИС. Часто используются мультиплексоры, размещенные в корпусах с 16 выводами. На одном краю ряда мультиплексоров стоит показанная на рис. 5.63 ИС 74×151 , в которой осуществляется выбор сигнала на одном из восьми 1-разрядных входов. Входы выбора обозначены буквами C, B и A , где C является старшим разрядом в числовом представлении. Сигнал на входе разрешения EN_L имеет низкий активный уровень; у схемы есть выходы как с высоким активным уровнем сигнала (Y), так и с низким активным уровнем сигнала (Y_L).

Таблица истинности мультиплексора 74×151 приведена в табл. 5.34. Здесь мы снова расширили нашу систему обозначений для таблиц истинности. До сих пор в наших таблицах истинности каждой входной комбинации соответствовал сигнал на выходе, равный 0 или 1. В таблице для мультиплексора 74×151 в графе «Входы» часть входов отсутствует. Сигнал на каждом выходе задан как 0, 1 или логическая функция остальных входов (например, D_0 или D_0'). При такой системе обозначений размеры таблицы сокращаются на восемь столбцов и восемь строк, а логическая функция представляется более наглядно, чем в случае, если бы таблица была полной.

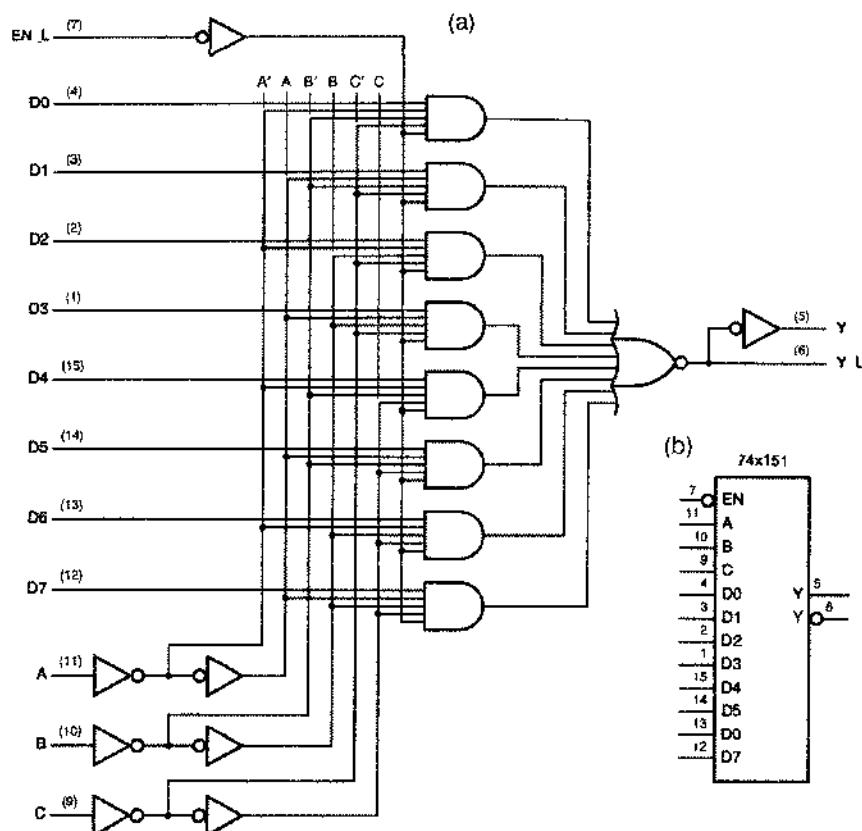


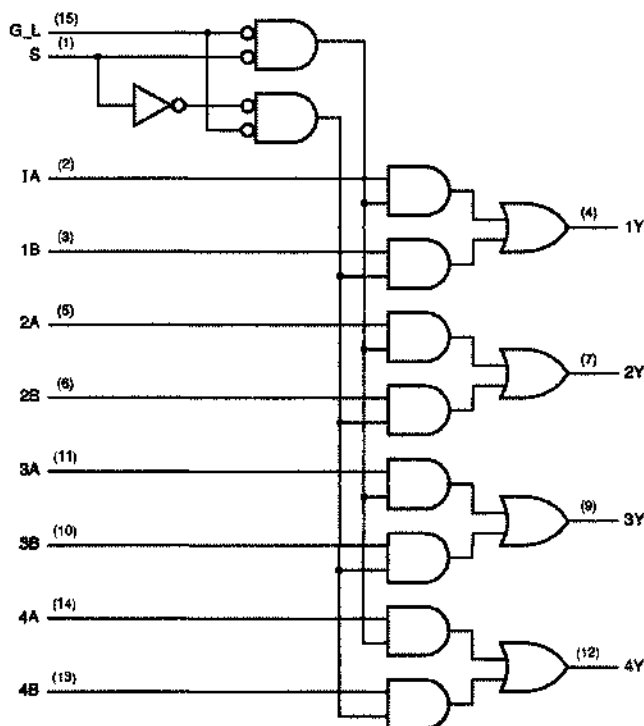
Рис. 5.63. 8-входовый 1-разрядный мультиплексор 74x151: (а) принципиальная схема с цоколевкой для стандартного корпуса DIP с 16 выводами; (б) традиционное условное обозначение

Табл. 5.34. Таблица истинности 8-входового 1-разрядного мультиплексора 74x151

Входы				Выходы	
EN_L	C	B	A	Y	Y_L
1	x	x	x	0	1
0	0	0	0	D0	D0'
0	0	0	1	D1	D1'
0	0	1	0	D2	D2'
0	0	1	1	D3	D3'
0	1	0	0	D4	D4'
0	1	0	1	D5	D5'
0	1	1	0	D6	D6'
0	1	1	1	D7	D7'

Другим крайним представителем мультиплексоров в корпусах с 16 выводами является микросхема 74х157, показанная на рис. 5.64, в которой осуществляется выбор сигналов, поступающих на один из двух 4-разрядных входов. Только для того, чтобы запутать существо дела, производитель дал входу выбора имя S, а входу разрешения с низким активным уровнем сигнала имя G_L. Заметьте также, что источники данных вместо D0 и D1, как в нашем общем примере, названы A и B. Из табл. 5.35 видно, что применение нашей расширенной системы обозначений в таблице истинности делает описание микросхемы 74х157 очень компактным.

(a)



(b)

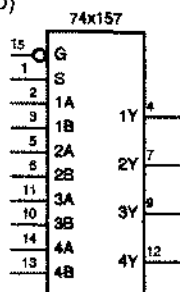


Рис. 5.54. 2-входовой 4-разрядный мультиплексор 74х157: (a) принципиальная схема с цоколевкой для стандартного корпуса DIP с 16 выводами; (b) традиционное условное обозначение

Входы		Выходы			
G_L	S	1Y	2Y	3Y	4Y
1	x	0	0	0	0
0	0	1A	2A	3A	4A
0	1	1B	2B	3B	4B

Табл. 5.35. Таблица истинности 2-входового 4-разрядного мультиплексора 74х157

Промежуточным звеном между мультиплексорами 74х151 и 74х157 является 4-входовой 2-разрядный мультиплексор 74х153. В этом устройстве, условное обозначение которого приведено на рис. 5.65, у каждого разряда есть отдельные входы разрешения (1G, 2G). Как следует из табл. 5.36, выполняемая им функция очень проста.

Рис. 5.65. Традиционное условное обозначение мультиплексора 74х153

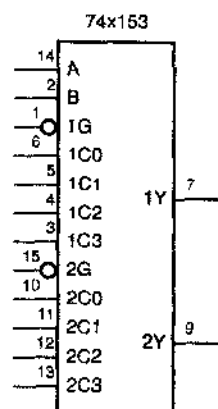


Табл. 5.35. Таблица истинности 4-входового 2-разрядного мультиплексора 74х153

Входы				Выходы	
1G_L	2G_L	B	A	1Y	2Y
0	0	0	0	1C0	2C0
0	0	0	1	1C1	2C1
0	0	1	0	1C2	2C2
0	0	1	1	1C3	2C3
0	1	0	0	1C0	0
0	1	0	1	1C1	0
0	1	1	0	1C2	0
0	1	1	1	1C3	0
1	0	0	0	0	2C0
1	0	0	1	0	2C1
1	0	1	0	0	2C2
1	0	1	1	0	2C3
1	1	x	x	0	0

Некоторые мультиплексоры имеют выходы с тремя состояниями. Вход разрешения такого мультиплексора вместо принудительного удержания выходов в нулевом состоянии, переводит их в состояние Hi-Z. Например, ИС 74х251 идентична

микросхеме '151 в отношении выводов логической логики внутреннего построения за исключением того, что выходы Y и Y_L имеют третье состояние. Когда уровень сигнала на входе EN_L неактивен, вместо появления на выходах низкого уровня на них устанавливается состояние Hi-Z. Точно так же, в отличие от ИС '153 и '157, ИС 74х253 и 74х257 имеют третье состояние на выходах. Особенно полезно наличие выходов с тремя состояниями для образования больших по размеру мультиплексоров путем объединения n -входовых мультиплексоров, как это делается в следующем разделе.

5.7.2. Расширение мультиплексоров

Число входов у мультиплексоров, выполненных в виде ИС средней степени интеграции, редко соответствует требованиям решаемой задачи. Например, ранее мы говорили, что 8-входовой 16-разрядный мультиплексор можно было бы использовать при создании процессора для компьютера. Эту функцию можно реализовать с помощью шестнадцати 8-входовых 1-разрядных мультиплексоров 74х151 или эквивалентных им ячеек в специализированных ИС; каждый такой мультиплексор имеет дело с одним битом всех входов и выхода. 3 разряда поля выбора регистра в процессоре можно было бы соединить с входами A, B и C всех 16 мультиплексоров, так что в любой момент времени все они выбирали бы в качестве источника один и тот же регистр.

В этом примере устройство, реализующее 3-разрядное поле выбора регистра, должно иметь достаточную нагрузочную способность, чтобы работать с 16 подключенными входами. Это обеспечивают ИС серии 74LS, поскольку типичные устройства этого семейства при нагрузке типа LS-TTL имеют коэффициент разветвления по выходу, равный 20.

К счастью, микросхема '151 была разработана так, чтобы каждый из входов A, B и C нагружал схему, вырабатывающую сигналы для этих входов, как один TTL-вентиль типа LS. Теоретически микросхема '151 могла бы не иметь трех инверторов первого ряда на входах выбора, показанных на рис. 5.63, но тогда каждый вход выбора был бы эквивалентен пяти нагрузкам типа LS-TTL, и драйверы в схеме выбора регистра должны были бы иметь коэффициент разветвления по выходу, равный 80.

Другим направлением, в котором можно расширять мультиплексоры, является число источников данных. Предположим, нам необходим 32-входовой 1-разрядный мультиплексор. На рис. 5.66 показан один из вариантов построения такого мультиплексора. Для него требуется 5-разрядный сигнал выбора. Два старших разряда декодируются дешифратором 2х4 (половина микросхемы 74х139), выходные сигналы которого служат сигналами разрешения для четырех 8-входовых мультиплексоров 74х151. Так как на каждом отрезке времени разрешена работа только одного мультиплексора '151, для получения результирующего выходного сигнала можно просто с помощью схемы ИЛИ объединить выходы всех мультиплексоров.

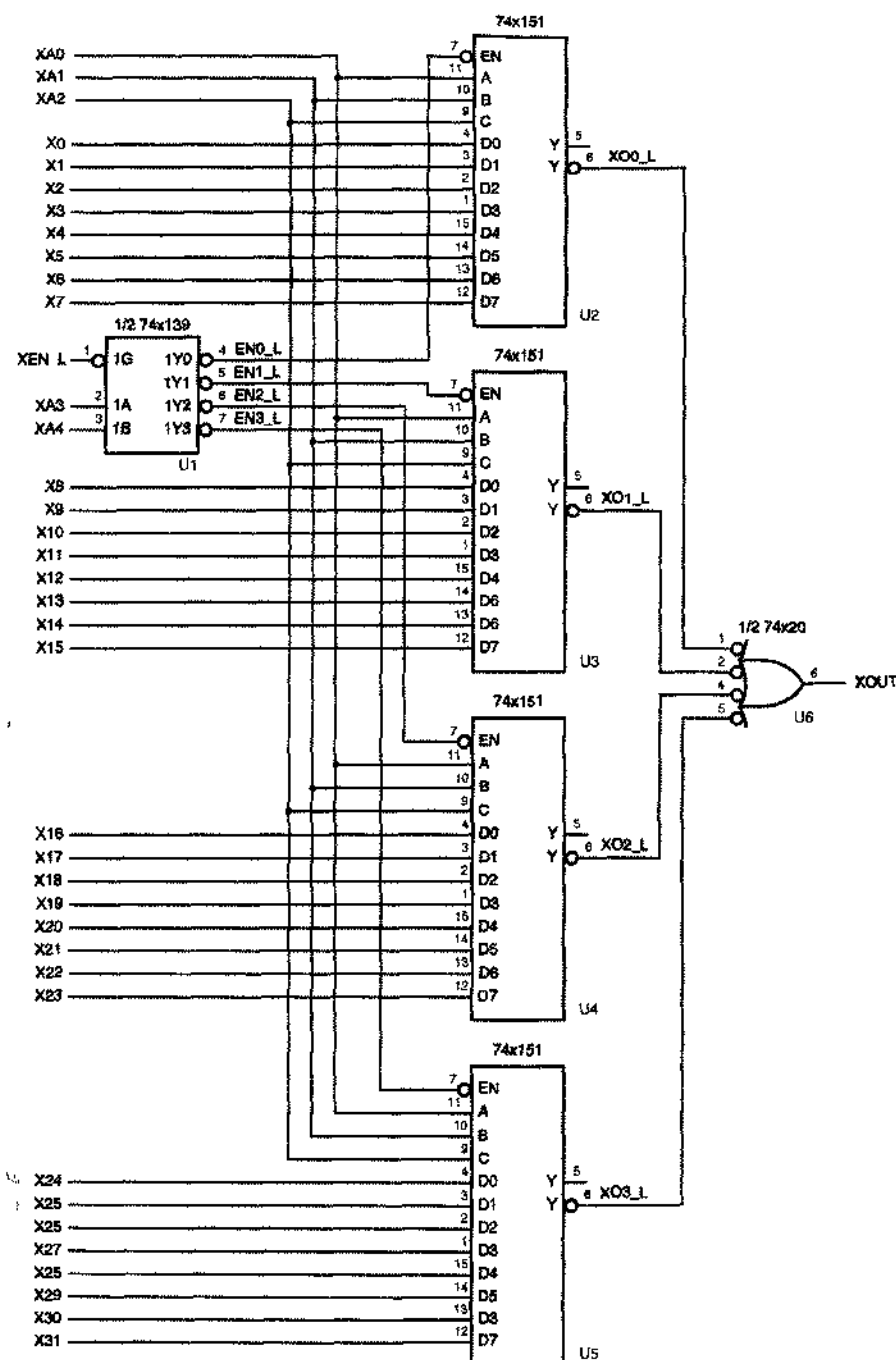


Рис. 5.66. Объединение ИС 74x151 для образования мультиплексора 32×1

НАГРУЗОЧНАЯ СПОСОБНОСТЬ ИСТОЧНИКА УПРАВЛЯЮЩЕГО СИГНАЛА В СПЕЦИАЛИЗИРОВАННЫХ ИС

Именно такими соображениями относительно коэффициента разветвления по выходу, какие приведены выше, довольно часто приходится руководствоваться при разработке специализированных ИС. Когда управляющие сигналы — типа сигналов, поступающих с выхода выбора регистра в нашем примере, — должны осуществлять переключение в большом числе разрядов, требуемый коэффициент разветвления по выходу может быть огромным. В КМОП-схемах это относится к нагрузке не по постоянному току, а к емкостной нагрузке, которая снижает быстродействие. В таком случае разработчик должен аккуратно разделить нагрузку и выбрать точки буферизации управляющих сигналов для уменьшения числа входов, нагружающих каждый выход. Дополнительные буферы нужно вводить осмотрительно, чтобы значительно не увеличить необходимую площадь кристалла и не включить слишком много буферов последовательно, что привело бы к неприемлемой задержке.

Мультиплексор 32×1 можно построить также на основе ИС 74×251. Схема идентична приведенной на рис. 5.66 за исключением того, что отсутствует выходной вентиль И-НЕ. Вместо этого, выходы Y_i (и при желании Y_L) четырех схем '251 просто соединяются вместе. Дешифратор '139 гарантирует, что в любой момент времени активизировано не более одной ИС '251, имеющей выходы с тремя состояниями. Если дешифратор '139 заблокирован (сигнал XEN_L имеет неактивный уровень), то заперты все мультиплексоры '251, и сигналы на выходах $XOUT$ и $XOUT_L$ не определены. Однако при этом можно иметь на выходе высокий уровень, соединив каждый из этих выходов резистором с шиной питания +5 вольт.

ИСПОЛЬЗУЙТЕ ПРИНЦИП «ИНВЕРСИЯ К ИНВЕРСИИ»

Применяемый при логическом проектировании принцип «инверсия к инверсии» должен помочь разобраться в приведенных примерах мультиплексоров. Так как выходы дешифратора и входы разрешения мультиплексора имеют низкий активный уровень, их можно связать напрямую. Рассматривая выполняемую устройством логическую функцию, можно игнорировать кружки инверсии, просто считая, что при возникновении на том или ином выходе дешифратора единичного сигнала включается соответствующий мультиплексор.

Принцип «инверсия к инверсии» дает также два варианта реализации конечной функции ИЛИ (см. рис. 5.66). В качестве наиболее очевидного варианта можно было бы применить 4-входовую схему ИЛИ, соединенную с выходами Y . Однако для повышения быстродействия мы использовали инвертирующий вентиль — 4-входовую схему И-НЕ, подключенную к выходам Y_L . Это исключает задержку, вносимую двумя инверторами: один из них применен внутри микросхемы '151 для получения сигнала Y из Y_L , а вторая инвертирующая схема — это та, которая используется для реализации функции ИЛИ на основе базовой схемы ИЛИ-НЕ в КМОП- или TTL-вентиллях.

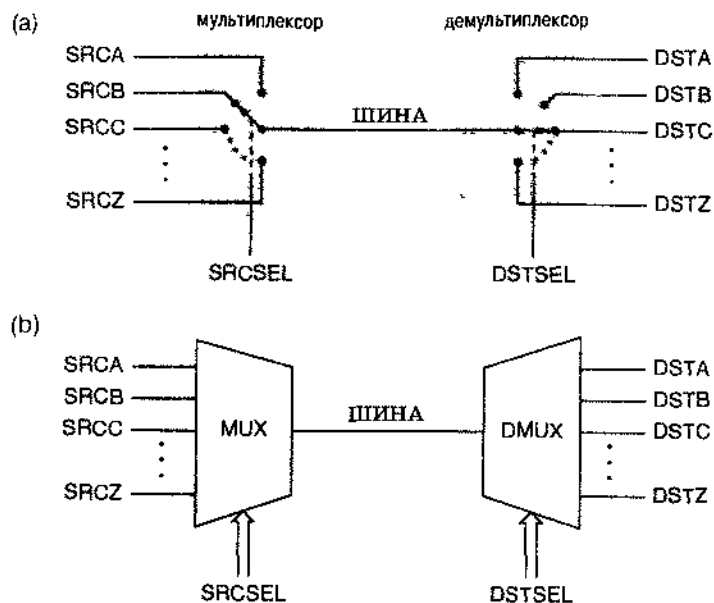


Рис. 5.67. Мультиплексор, работающий на шину, и демультиплексор, получающий сигнал с шины. (а) эквивалентная схема в виде переключателей, (б) блок-схема с условными обозначениями

5.7.3. Мультиплексоры, демультиплексоры и шины

Чтобы выбрать один из n источников данных для передачи их по шине, можно воспользоваться мультиплексором. Чтобы направить данные к одному из m адресатов на приемном конце шины, можно применить демультиплексор (*demultiplexer*). Такая система с 1-разрядной шиной изображена в виде переключателей на рис. 5.67(а). В блок-схемах мультиплексоры и демультиплексоры часто изображают в виде трапеций [рис. 5.67(б)], чтобы наглядно показать, как сигналы одного, выбранного из многих, источника данных поступают на шину и направляются к тому или иному адресату, выбранному из многих адресатов.

Функция, реализуемая демультиплексором, прямо противоположна функции, выполняемой мультиплексором. Например, 1-разрядный демультиплексор с n выходами имеет один вход данных и s входов выбора одного из $n = 2^s$ выходов данных. При нормальной работе сигналы на всех выходах, кроме выбранного, равны 0; данные на выбранном выходе совпадают с данными на входе. Это определение можно обобщить на b -разрядный демультиплексор с n выходами; у такого демультиплексора b входов данных, и в нем с помощью s сигналов на входах выбора выбирается один из $n = 2^s$ наборов с b выходами данных в каждом.

В качестве демультиплексора можно применять полный дешифратор с входом разрешения, как показано на рис. 5.68. Вход разрешения дешифратора подключается к линии данных, а от сигналов на входах выбора зависит, на какой из его выходных линий сигнал будет определяться битом данных. Сигналы на остальных выход-

ных линиях имеют неактивный уровень. Таким образом, микросхему 74х139 можно использовать как 2-разрядный демультиплексор с 4 выходами с низким активным уровнем сигналов на входе данных и на выходах, а дешифратор 74х138 можно применять как 1-разрядный демультиплексор с 8 выходами. Обычно в каталоге производителя эти ИС не на самом деле обозначаются как «дешифраторы/демультиплексоры».

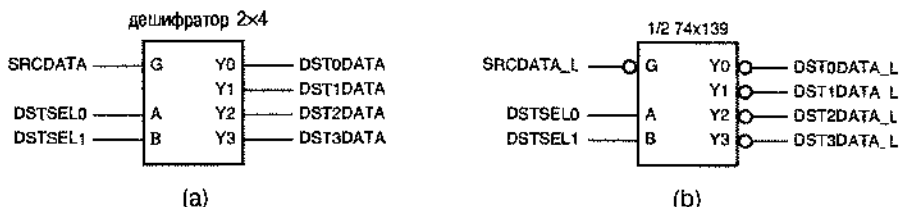


Рис. 5.58. Применение полного дешифратора 2х4 в качестве 1-разрядного демультиплексора с 4 выходами: (а) стандартный дешифратор; (б) демультиплексор 74х139

5.7.4. Описание мультиплексоров на языке ABEL и их реализация в ПЛУ

Мультиплексоры очень просто проектировать, используя язык ABEL и комбинационные ПЛУ. Например, функции 2-разрядного мультиплексора с 4 входами 74х153 можно в точности реализовать с помощью ИС PAL16L8, как показано на рис. 5.69 и в табл. 5.37. Стоит отметить некоторые особенности проектирования на основе ПЛУ и их программирования:

- Имена сигналов в программе на языке ABEL немного изменены по сравнению с именами сигналов, указанных на входах и выходах мультиплексора 74х153 на рис. 5.65, так как в языке ABEL не разрешается использовать цифру в качестве первого символа в имени сигнала.
- Мультиплексор 74х153 имеет двенадцать входов, в то время как у PAL16L8 только десять входов. Поэтому роль двух входов мультиплексора типа 153 играют выводы I/O ИС 16L8, которые теперь не пригодны для использования в качестве выходов.
- Выходами мультиплексора типа 153 (1Y и 2Y) назначены выводы 19 и 12 микросхемы 16L8, которые *только и могут быть выходами*. Этот вариант предпочтительнее по сравнению с использованием в качестве выходов выводов I/O; при имеющихся возможностях лучше в качестве резервных оставить выводы I/O, чем выводы, которые могут служить только выходами.
- Хотя равенства в программе мультиплексора написаны во вполне естественной форме суммы произведений, они прямо не отображаются на структуру ИС 16L8 из-за наличия инвертора между матрицей И-ИЛИ и фактическими выводами выходов. Поэтому транслятор языка ABEL должен инвертировать равенства табл. 5.37 и затем свести результат к виду «сумма произведений». В случае ИС GAL16V8 можно использовать любой вариант равенств.

Рис. 5.69. Схема PAL16L8 в качестве мультиплексора типа 74x153

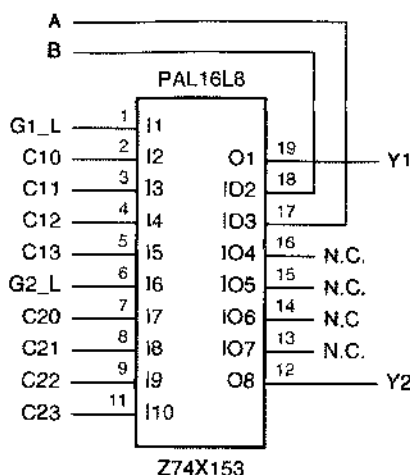


Табл. 5.37. Программа на языке ABEL для 2-разрядного мультиплексора с 4 входами типа 74x153

```

module Z74X153
title '74x153-like multiplexer PLD '
Z74X153 device 'P16L8';

" Input and output pins
A, B, G1_L, G2_L      pin 17, 18, 1, 6;
C10, C11, C12, C13    pin 2, 3, 4, 5;
C20, C21, C22, C23    pin 7, 8, 9, 11;
Y1, Y2                pin 19, 12 istype 'com';

" Active-level conversion
G1 = !G1_L; G2 = !G2_L;

equations
Y1 = G1 & ( !B & !A & C10
           # !B & A & C11
           # B & !A & C12
           # B & A & C13);

Y2 = G2 & ( !B & !A & C20
           # !B & A & C21
           # B & !A & C22
           # B & A & C23);

end Z74X153

```

Еще проще выразить функции мультиплексора, используя наборы и отношения языка ABEL. В табл. 5.39, например, приведена программа на языке ABEL для 4-входового 8-разрядного мультиплексора. В ней отсутствует оператор device, потому что у этой функции слишком много входов и выходов, чтобы ее можно было реализовать в каком-либо ПЛЮ из числа тех, какие были описаны нами до сих пор.

Однако совершенно очевидно, что таким образом всего лишь несколькими строками программы можно задать мультиплексор любых размеров.

```

!Y1 = (!B & !A & !C10
      # !B & A & !C11
      # B & !A & !C12
      # B & A & !C13
      # G1);
!Y2 = (!B & !A & !C20
      # !B & A & !C21
      # B & !A & !C22
      # B & A & !C23
      # G2);

```

Табл. 5.38. Инвертированные выражения для сигналов на выходах 2-разрядного мультиплексора с 4 входами типа 74х153

Табл. 5.39. Программа на языке ABEL для 4-входового 8-разрядного мультиплексора

```

module mux4in8b
title '4-inpnt, 8-bit wide multiplexer PLD'

" Input and output pins
!G                                pin;                                " Output enable for Y bus
S1..S0                            pin;                                " Select inputs, 0-3 ==> A-D
A1..A8, B1..B8, C1..C8, D1..D8 pin; " 8-bit input buses A, B, C, D
Y1..Y8                            pin istance 'com'; " 8-bit three-state output bus

" Sets
SEL = [S1..S0];
A = [A1..A8];
B = [B1..B8];
C = [C1..C8];
D = [D1..D8];
Y = [Y1..Y8];

equations
Y.OE = G;
WHEN (SEL == 0) THEN Y = A;
ELSE WHEN (SEL == 1) THEN Y = B;
ELSE WHEN (SEL == 2) THEN Y = C;
ELSE WHEN (SEL == 3) THEN Y = D;
end mux4in8b

```

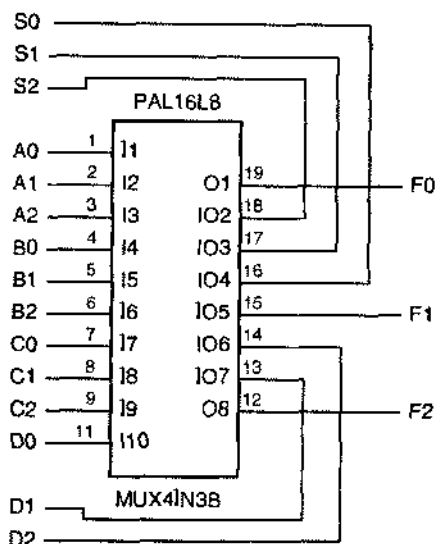
Аналогично, используя язык ABEL, легко построить специализированный мультиплексор согласно техническим требованиям заказчика. Предположим, например, что вам необходима схема, выбирающая с помощью трех управляющих битов одну из четырех 18-разрядных входных шин A, B, C и D для передачи с нее данных на 18-разрядную выходную шину F согласно табл. 5.40. Число комбинаций управляющих битов больше, чем число входов мультиплексора, так что стандарт-

ный 4-входовой мультиплексор не полностью удовлетворяет заданным условиям (см. задачу 5.61). 4-входовой 3-разрядный мультиплексор с требуемым режимом работы можно разработать так, чтобы он вписался в одну микросхему PAL16L8 или GAL16V8, как показано на рис. 5.70 и в табл. 5.41, а для того чтобы построить 18-разрядный мультиплексор, можно воспользоваться шестью экземплярами таких 3-разрядных устройств. В качестве альтернативы можно применить одно ПЛУ больших размеров. В любом случае программа на языке ABEL очень легко модифицируется применительно к различным критериям выбора.

Табл. 5.40. Правила выбора входа специализированного 4-входового 18-разрядного мультиплексора

S2	S1	S0	Выбираемый вход
0	0	0	A
0	0	1	B
0	1	0	A
0	1	1	C
1	0	0	A
1	0	1	D
1	1	0	A
1	1	1	B

Рис. 5.70. Схема PAL16L8 в качестве специализированного 4-входового 3-разрядного мультиплексора



Так как при реализации этой функции используются все имеющиеся выводы ИС PAL16L8, при назначении выводов надо быть очень внимательным. В частности, под выходы надо отвести два вывода, предназначенные только для того, чтобы быть выходами (O1 и O8), с тем чтобы число доступных входных выводов было возможно большим.

САМЫЙ ПРОСТОЙ, НО НЕ САМЫЙ ДЕШЕВЫЙ

Как вы видите, очень просто запрограммировать ПЛУ на выполнение функций мультиплексора и дешифратора. Но если вам нужен обычный дешифратор или мультиплексор, то дешевле, как правило, применить стандартную СИС, чем программировать ИЛУ. Подход, основанный на применении ПЛУ, лучше, когда мультиплексор должен выполнять некоторые нестандартные функции, а также в том случае, если вы опасаетесь, что вам придется изменять его функцию в процессе отладки.

Табл. 5.41. Программа на языке ABEL для специализированного 4-входового 3-разрядного мультиплексора

```

module mux4in3b
title 'Specialized 4-input, 3-bit Multiplexer'
mux4in3b device 'P16L8';

" Input and output pins
S2..S0                pin 16..18;                " Select inputs
A0..A2, B0..B2, C0..C2, D0..D2 pin 1..9, 11, 13, 14; " Bus inputs
F0..F2                pin 19, 15, 12 istype 'com'; " Bus outputs

" Sets
SEL = {S2..S0};
A = {A0..A2};
B = {B0..B2};
C = {C0..C2};
D = {D0..D2};
F = {F0..F2};

equations
WHEN (SEL== 0) # (SEL== 2) # (SEL== 4) # (SEL== 6) THEN F = A;
ELSE WHEN (SEL== 1) # (SEL== 7) THEN F = B;
ELSE WHEN (SEL== 3) THEN F = C;
ELSE WHEN (SEL== 5) THEN F = D;

end mux4in3b

```

5.7.5. Описание мультиплексоров на языке VHDL

Описывать мультиплексоры на языке VHDL очень просто. В архитектуре, написанной в нотации в стиле, оператор SELECT обеспечивает требуемые функциональные возможности, что можно видеть в табл. 5.42, где дано описание 4-входового 8-разрядного мультиплексора на языке VHDL.

В поведенческой архитектуре выбор осуществляется оператором CASE. В табл. 5.43, например, приведена архитектура для того же самого модуля mux4in8b, основанная на использовании процесса.

Табл. 5.42. Поточковая VHDL-программа для 4-входового 8-разрядного мультиплексора

```

library IEEE;
use IEEE.std_logic_1164.all;

entity mux4in8b is
    port (
        S: in STD_LOGIC_VECTOR (1 downto 0);    -- Select inputs, 0-3 ==> A-D
        A, B, C, D: in STD_LOGIC_VECTOR (1 to 8); -- Data bus input
        Y: out STD_LOGIC_VECTOR (1 to 8)         -- Data bus output
    );
end mux4in8b;

architecture mux4in8b of mux4in8b is
begin
    with S select Y <=
        A when "00",
        B when "01",
        C when "10",
        D when "11",
        (others => 'U') when others; -- this creates an 8-bit vector of 'U'
end mux4in8b;

```

Табл. 5.43. Поведенческая архитектура для 4-входового 8-разрядного мультиплексора

```

architecture mux4in8p of mux4in8b is
begin
    process(S, A, B, C, D)
    begin
        case S is
            when "00" => Y <= A;
            when "01" => Y <= B;
            when "10" => Y <= C;
            when "11" => Y <= D;
            when others => Y <= (others => 'U'); -- 8-bit vector of 'U'
        end case;
    end process;
end mux4in8p;

```

Так же, как в языке ABEL, в VHDL-программе мультиплексора очень просто реализовать любые требуемые правила выбора входа. Например, в табл. 5.44 приведена написанная в поведенческом стиле программа для специализированного 4-входового 18-разрядного мультиплексора с правилами выбора, указанными в табл. 5.40.

Если в каждом из этих примеров сигналы на входах выбора не действительны (например, содержат значения 'U' или 'X'), то для того, чтобы облегчить обнаружение ошибок в процессе моделирования, сигналам на выходной шине присваиваются значения 'U'.

Табл. 5.44. Поведенческая VHDL-программа для специализированного 4-входового 18-разрядного мультиплексора

```

library IEEE;
use IEEE.std_logic_1164.all;

entity mux4in3b is
  port (
    S: in STD_LOGIC_VECTOR (2 downto 0);    -- Select inputs, 0-7 ==> ABACADAB
    A, B, C, D: in STD_LOGIC_VECTOR (1 to 18); -- Data bus inputs
    Y: out STD_LOGIC_VECTOR (1 to 18)        -- Data bus output
  );
end mux4in3b;

architecture mux4in3p of mux4in3b is
begin
  process(S, A, B, C, D)
  variable i: INTEGER;
  begin
    case S is
      when "000" | "010" | "100" | "110" => Y <= A;
      when "001" | "111" => Y <= B;
      when "011" => Y <= C;
      when "101" => Y <= D;
      when others => Y <= (others => 'U'); -- 18-bit vector of 'U'
    end case;
  end process;
end mux4in3p;

```

5.8. Логические элементы ИСКЛЮЧАЮЩЕЕ ИЛИ и проверка на четность

5.8.1. Вентили ИСКЛЮЧАЮЩЕЕ ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ

Вентиль *ИСКЛЮЧАЮЩЕЕ ИЛИ* (*Exclusive-OR, XOR*) является 2-входовой схемой, выходной сигнал которой равен 1 лишь в том случае, когда 1 присутствует только на одном из его входов. Существует и другой вариант определения: вентиль *ИСКЛЮЧАЮЩЕЕ ИЛИ* вырабатывает на выходе 1, если сигналы на его входах различны. Сигнал на выходе вентиль *ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ* (*Exclusive NOR, XNOR*) или *ЭКВИВАЛЕНТНОСТЬ* (*Equivalence*) имеет прямо противоположное значение: он равен 1, если сигналы на входах вентиль одинаковы. Таблица истинности для этих функций приведена в табл. 5.45. Операцию *ИСКЛЮЧАЮЩЕЕ ИЛИ* иногда обозначают символом $\oplus$, то есть

$$X \oplus Y = X' \cdot Y + X \cdot Y'$$

Хотя *ИСКЛЮЧАЮЩЕЕ ИЛИ* не является одной из основных функций алгебры неключений, дискретные схемы, реализующие функцию *ИСКЛЮЧАЮЩЕЕ ИЛИ*, довольно распространены на практике. В большинстве технологий, используемых

при создании переключающих схем, функцию ИСКЛЮЧАЮЩЕЕ ИЛИ непосредственно реализовать нельзя; вместо этого применяются многоразрядные конструкции, подобные тем, какие показаны на рис. 5.71.

Табл. 5.45. Таблица истинности для функций ИСКЛЮЧАЮЩЕЕ ИЛИ (XOR) и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ (XNOR)

X	Y	$X \oplus Y$ (XOR)	$(X \oplus Y)'$ (XNOR)
0	0	0	1
0	1	1	0
1	0	1	0
1	1	0	1

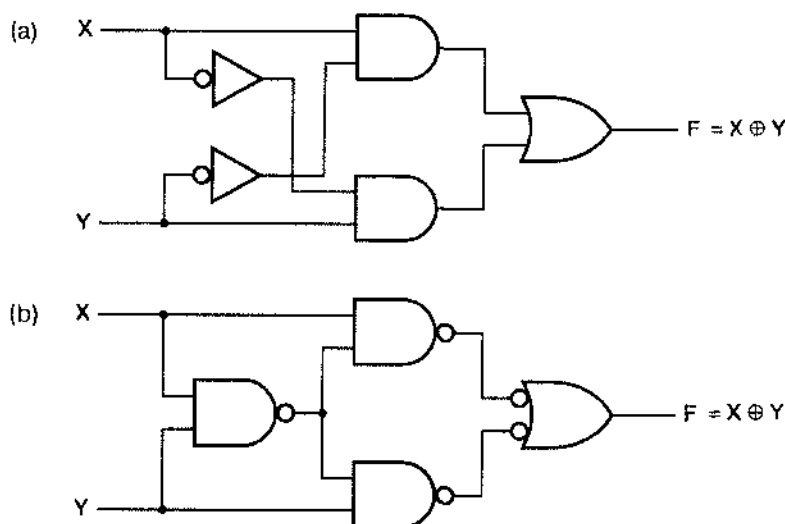


Рис. 5.71. Варианты реализации 2-входовой схемы ИСКЛЮЧАЮЩЕЕ ИЛИ с помощью нескольких вентилях: (а) на основе структур И-ИЛИ; (б) трехуровневый вариант на основе вентилях И-НЕ.

Условные обозначения схем, реализующих функции ИСКЛЮЧАЮЩЕЕ ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ, даны на рис. 5.72. Для каждой схемы имеются четыре варианта условных обозначений. Все эти варианты вытекают из простого правила:

- Инвертирование любых двух сигналов (обоих входных сигналов или одного входного и выходного) у схем ИСКЛЮЧАЮЩЕЕ ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ не изменяет результирующей логической функции.

Используя при проектировании принцип «инверсия к инверсии», мы выбираем такое условное обозначение, которое является наиболее наглядным с точки зрения выполняемой логической функции.

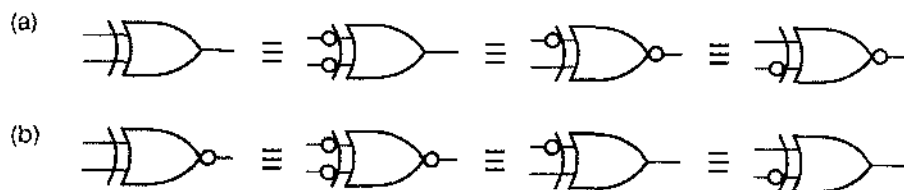


Рис. 5.72. Условные обозначения: (а) схемы ИСКЛЮЧАЮЩЕЕ ИЛИ; (б) схемы ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ

На рис. 5.73 приведена цоколевка МИС 74х86 с 14 выводами, в одном корпусе которой размещаются четыре схемы ИСКЛЮЧАЮЩЕЕ ИЛИ. Новые семейства МИС не содержат схем ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ, хотя их легко реализовать в FPGA; они имеются также в библиотеках специализированных ИС и в виде примитивов в языках описания схем.

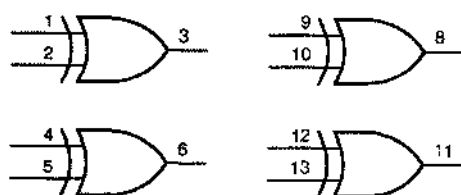


Рис. 5.73. Цоколевка микросхемы 74х86, содержащей четыре 2-входных вентиля ИСКЛЮЧАЮЩЕЕ ИЛИ

5.8.2. Схемы проверки на четность

Если включить последовательно n схем ИСКЛЮЧАЮЩЕЕ ИЛИ так, как показано на рис. 5.74 (а), то получится схема ИСКЛЮЧАЮЩЕЕ ИЛИ с $n + 1$ входами и одним выходом. Такую схему называют *схемой проверки на нечетность (odd-parity circuit)*, потому что сигнал на ее выходе равен 1, если единицы присутствуют на нечетном числе ее входов. Схема, приведенная на рис. 5.74(б), также осуществляет проверку на нечетность, но является более быстрой, поскольку в ней вентили образуют древовидную структуру. Если инвертировать выход любой из представленных схем, то получим *схему проверки на четность (even-parity circuit)*, у которой выходной сигнал равен 1, если на четном числе входов имеются единицы.

5.8.8. 9-разрядная микросхема проверки на четность 74х280

Вместо того, чтобы создавать многоразрядную схему проверки на четность с помощью отдельных схем ИСКЛЮЧАЮЩЕЕ ИЛИ, выгоднее разместить все вентили ИСКЛЮЧАЮЩЕЕ ИЛИ в одном корпусе СИС, у которой с внешними выводами соединены только первичные входы и выход. Таким устройством является изображенная на рис. 5.75 9-разрядная схема проверки на четность 74х280. У нее девять входов и два выхода, которые указывают на наличие четного или нечетного числа единиц на входах.

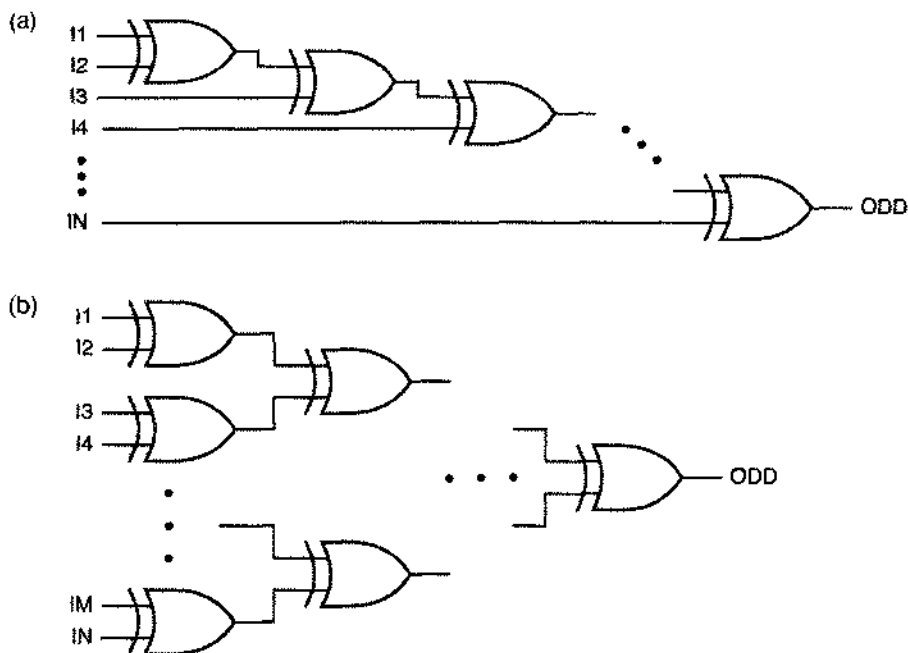


Рис. 5.74. Каскадное включение схем ИСКЛЮЧАЮЩЕЕ ИЛИ: (а) соединение с последовательным опросом, (б) древовидная структура

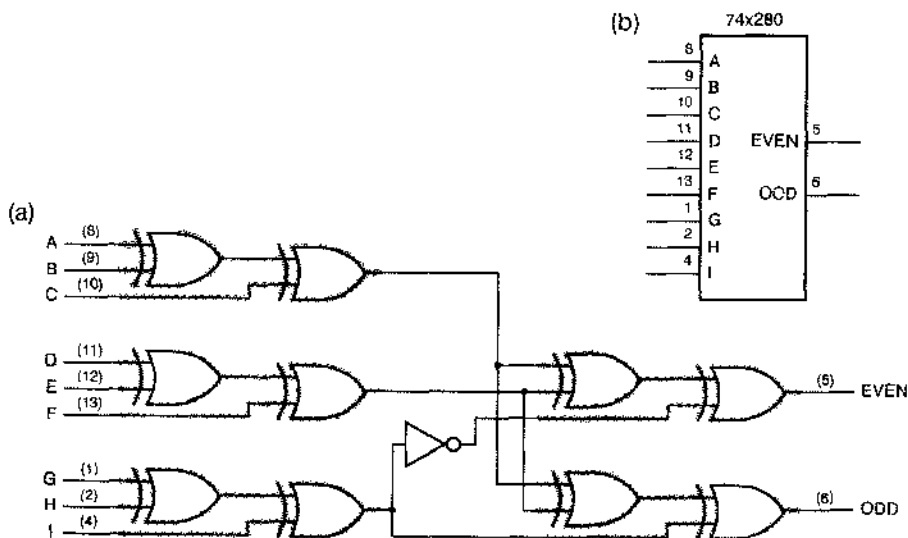


Рис. 5.75. 9-разрядная схема проверки на четность/нечетность 74x280. (а) принципиальная схема и цоколевка для стандартного корпуса DIP с 16 выводами, (б) традиционное условное обозначение (EVEN – четное число единиц на входах, ODD – нечетное число единиц на входах)

ПОВЫШЕНИЕ БЫСТРОДЕЙСТВИЯ ДРЕВОВИДНОЙ СХЕМЫ ИСКЛЮЧАЮЩЕЕ ИЛИ

Если бы каждый из вентилях ИСКЛЮЧАЮЩЕЕ ИЛИ на рис. 5.75 был составлен из отдельных схем И-НЕ, как показано на рис. 5.71(б), то ИС 74х280 была бы значительно медленнее: задержка распространения сигнала в ней была бы эквивалентна $4 \cdot 3 + 1 = 13$ задержкам одного вентиля И-НЕ. Поэтому типичное исполнение ИС 74х280 основано на применении структуры И-ИЛИ-НЕ с 4 выводами. Именно такой структурой реализуется функция, выполняемая каждой парой схем ИСКЛЮЧАЮЩЕЕ ИЛИ, объединенных на рисунке одним затененным прямоугольником; при этом задержка примерно равна задержке одного вентиля И-НЕ. На входах А-1 помещены буферы, состоящие из двух инверторов, для того чтобы каждый из входов представлял собой одиночную нагрузку для схемы, с выхода которой поступает сигнал. Результирующая задержка распространения у выполненной таким образом схемы 74х280 составляет 5, а не 13 задержек одной инвертирующей схемы.

5.8.4. Применение схем проверки на четность

В параграфе 2.15 мы описали коды для обнаружения ошибок, возникающих при передаче и хранении данных, в которых используется дополнительный бит, называемый битом четности. В коде с проверкой на четность бит четности выбирают так, чтобы общее число единичных битов в кодовом слове было четно. Схемы проверки на четность, аналогичные 74х280, применяются как для формирования нужного значения бита четности при хранении или передаче кодового слова, так и для проверки на четность при считывании кодового слова из памяти или при его приеме.

На рис. 5.76 показано, как можно применять схему проверки на четность для обнаружения ошибок в памяти микропроцессорной системы. Память хранит 8-разрядные байты плюс бит четности для каждого байта. Для передачи данных в память и для извлечения данных из нее микропроцессор использует двунаправленную шину D[0:7]. Кроме того, имеются две линии управления RD и WR для задания желаемой операции – чтения или записи – и вырабатывается сигнал ERROR, указывающий на наличие ошибки при проверке на четность при чтении. Другие детали, касающиеся микросхем памяти (адресные входы и др.), на рисунке опущены; подробнее микросхемы памяти описаны в главе 10. При рассмотрении проверки на четность нас интересуют только линии передачи данных в память и из нее.

Для сохранения байта в микросхеме памяти мы задаем адрес (не показанный на рисунке), размещаем на шине D[0-7] запоминаемый байт и подаем сигнал записи WR: на вход PIN при этом поступает сформированный бит четности. Вентиль И, предшествующий входу I микросхемы 74х280, обеспечивает постоянное присутствие на этом входе 0, кроме интервалов времени, в течение которых выполняется операция чтения; благодаря этому во время записи сигнал на выходе микросхемы '280 зависит только от четности данных на шине D. Со входом PIN соединен выход ODD микросхемы '280, поэтому общее число запоминаемых единиц оказывается четным.

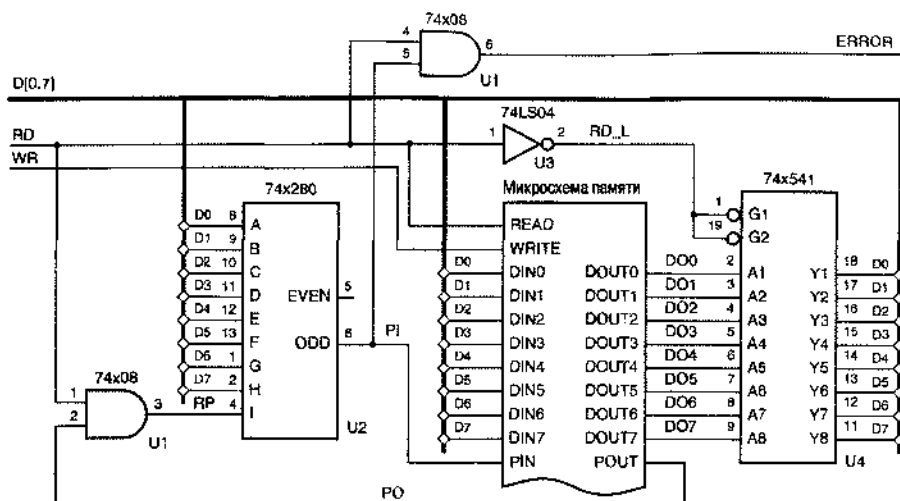


Рис. 5.75. Формирование бита четности и проверка на четность для 8-разрядной системы памяти

Чтобы извлечь байт, мы задаем адрес (не показанный на рисунке) и подаем сигнал чтения RD; значение байта появляется на выходах DOUT[0–7], а бит четности — на выходе POUT. Микросхема 74x541 выполняет роль буфера при передаче байта на шину D, а микросхема 74x280 осуществляет проверку на четность. Если при чтении проверка на четность 9-разрядного слова DOUT[0–7], POUT дает нечетное число единиц, то вырабатывается сигнал ошибки ERROR.

Схемы проверки на четность применяются также при работе с кодами, исправляющими ошибки, такими как коды Хэмминга, описанные в разделе 2.15.3. На рис. 2.13 была приведена матрица проверки на четность для 7-разрядного кода Хэмминга. Ошибки, возникающие в этом коде, можно исправлять так, как показано на рис. 5.77. На шине DU[1–7] присутствует 7-разрядное слово, возможно содержащее ошибки. С помощью трех микросхем 74x280 проверяется четность в трех 4-битовых группах, указываемых проверочной матрицей. На выходах микросхем 74x280 формируется синдром, которым определяется номер ошибочного входного бита, если таковой имеется. Микросхема 74x138 используется для декодирования синдрома. Если синдром равен нулю, то вырабатывается сигнал NOERROR_L (этот сигнал можно было бы назвать также ERROR). В противном случае ошибочный бит исправляется путем его инвертирования. Исправленное кодовое слово появляется на шине DC_L.

Заметим, что из-за низкого активного уровня сигналов на выходах микросхемы 74x138 сигналы на шине DC_L также имеют низкий активный уровень. Если необходима шина DC с высоким активным уровнем сигналов, то на входе или выходе каждой из схем ИСКЛЮЧАЮЩЕЕ ИЛИ можно поместить отдельный инвертор, либо воспользоваться дешифратором с высоким активным уровнем сигналов на выходах, либо использовать схемы ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ.

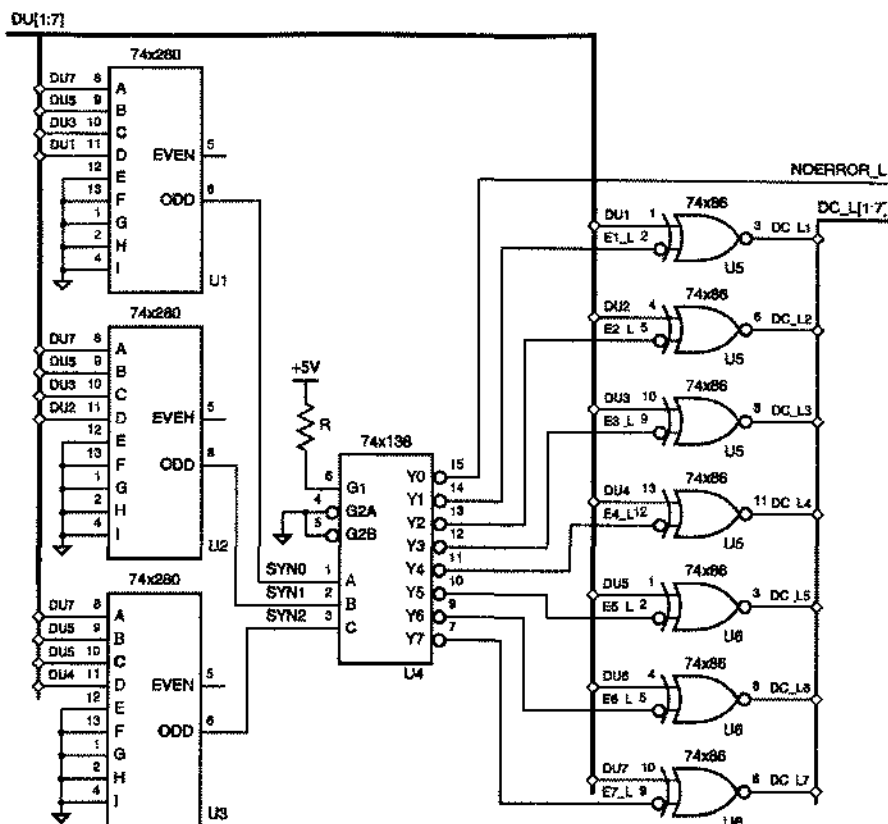


Рис. 5.77. Схема, исправляющая ошибки в 7-разрядном коде Хэмминга

5.8.5. Описание схем ИСКЛЮЧАЮЩЕЕ ИЛИ и схем проверки на четность на языке ABEL и их реализация в ПЛУ

В языке ABEL функцию ИСКЛЮЧАЮЩЕЕ ИЛИ реализует оператор \$, а дополнение этой функции – ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ – обозначается !\$\$. В принципе, употребление этих операторов в выражениях на языке ABEL ничем не ограничено. Вы можете, например, задать сигнал на выходе ПЛУ, эквивалентный сигналу EVEN на выходе ИС 74x280, следующим выражением на языке ABEL:

```
EVEN = !(A$B$C$D$E$F$G$H$I).
```

Однако в большинстве ПЛУ выражения реализуются посредством двухуровневой логики И-ИЛИ и лишь в малой степени могут напрямую реализовать функцию ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ, если такая возможность вообще имеется. К сожалению, карта Карно n -входовой схемы ИСКЛЮЧАЮЩЕЕ ИЛИ выглядит как шахматная доска с 2^{n-1} простыми импlicantsами. Поэтому для реализации приведенного выше

простого соотношения средствами, рассчитанными на выражения вида «сумма произведений», потребовалось бы 256 термов-произведений, что находится за пределами возможностей любого ПЛУ.

В ПЛУ так называемой серии X двухвходовую схему ИСКЛЮЧАЮЩЕЕ ИЛИ можно реализовать непосредственно трехуровневой структурой, в которой с помощью одного вентиля ИСКЛЮЧАЮЩЕЕ ИЛИ объединяются два независимых выражения вида «сумма произведений». Такая структура оказывается полезной при построении счетчиков. Однако для создания больших схем ИСКЛЮЧАЮЩЕЕ ИЛИ при проектировании на уровне платы обычно бывает необходимо воспользоваться специализированным узлом формирования проверочного символа и проверки на четность типа 74х280, а разработчик специализированных ИС должен составлять из отдельных схем ИСКЛЮЧАЮЩЕЕ ИЛИ многоуровневое дерево проверки на четность, подобное тому, которое показано на рис. 5.74(б).

5.8.6. Описание схем ИСКЛЮЧАЮЩЕЕ ИЛИ и схем проверки на четность на языке VHDL

Подобно языку ABEL, в языке VHDL имеются примитивы xor и xnor, реализующие функции ИСКЛЮЧАЮЩЕЕ ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ (оператор xnor имеется только в версии языка VHDL-93). В табл. 5.46 в качестве примера приведена программа в потоковом стиле для 3-входового устройства ИСКЛЮЧАЮЩЕЕ ИЛИ, где использован примитив xor. Можно также создавать схемы ИСКЛЮЧАЮЩЕЕ ИЛИ и схемы проверки на четность, описывая их поведение, как это сделано в табл. 5.47 для 9-входовой схемы проверки на четность, аналогичной ИС 74х280.

Табл. 5.46. Потокковая VHDL-программа для 3-входовой схемы ИСКЛЮЧАЮЩЕЕ ИЛИ

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vxor3 is
    port (
        A, B, C: in STD_LOGIC;
        Y: out STD_LOGIC
    );
end vxor3;

architecture vxor3 of vxor3 is
begin
    Y <= A xor B xor C;
end vxor3;
```

Когда синтезируется схема ИСКЛЮЧАЮЩЕЕ ИЛИ с большим числом входов, описанная на языке VHDL, программные средства наилучшим образом используют технологию, по которой выполнено ПЛУ, для реализации этой функции. И нет ничего удивительного в том, что при попытке указать VHDL-программе, приведенной в табл. 5.47, в качестве ПЛУ ИС 16V8 средства синтеза признают ее непригодной!

Табл. 5.47. Поведенческая VHDL-программа для 9-входовой схемы проверки на четность

```

library IEEE;
use IEEE.std_logic_1164.all;

entity parity9 is
    port (
        I: in STD_LOGIC_VECTOR (1 to 9);
        EVEN, ODD: out STD_LOGIC
    );
end parity9;

architecture parity9p of parity9 is
begin
    process (I)
        variable p : STD_LOGIC;
    begin
        p := I(1);
        for j in 2 to 9 loop
            if I(j) = '1' then p := not p; end if;
        end loop;
        ODD <= p;
        EVEN <= not p;
    end process;
end parity9p;

```

Библиотеки типичных специализированных ИС и ИС типа FPGA содержат в качестве примитивов двух- и трехвходовые схемы ИСКЛЮЧАЮЩЕЕ ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ. В специализированных интегральных КМОП-схемах эти примитивы обычно очень эффективно реализуются на уровне транзисторов с помощью логических ключей (см. задачи 5.56 и 5.73). Применяя эти примитивы, можно сформировать быстрые и компактные древовидные схемы ИСКЛЮЧАЮЩЕЕ ИЛИ. Однако типичные VHDL-средства синтеза недостаточно развиты, чтобы создать эффективную древовидную структуру на основе поведенческой программы типа той, которая приведена в табл. 5.47. Чтобы получить точно то, что мы хотим, следует написать структурную программу.

В табл. 5.48 представлена структурная программа на языке VHDL для 9-входовой схемы ИСКЛЮЧАЮЩЕЕ ИЛИ, являющейся эквивалентом ИС 74х280, изображенной на рис. 5.75(а), как по структуре, так и по реализуемой функции. В качестве основного блока древовидной схемы ИСКЛЮЧАЮЩЕЕ ИЛИ в этом примере использован предварительно определенный компонент `vxor3`. В случае специализированной ИС мы бы заменили этот компонент 3-входовым примитивом ИСКЛЮЧАЮЩЕЕ ИЛИ из библиотеки специализированных ИС. Кроме того, если доступна 3-входовая схема ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ, то можно было бы исключить явную инверсию сигнала `Y3`, а вместо этого сформировать сигнал `U5` с помощью функции `XNOR`, используя неинвертированный сигнал `Y3` в качестве третьей из ее переменных.

Табл. 5.48. Структурная VHDL-программа для устройства проверки на четность, аналогичного ИС 74х280

```

library IEEE;
use IEEE.std_logic_1164.all;

entity V74x280 is
    port (
        I: in STD_LOGIC_VECTOR (1 to 9);
        EVEN, ODD: out STD_LOGIC
    );
end V74x280;

architecture V74x280s of V74x280 is
    component vxor3
        port (A, B, C: in STD_LOGIC; Y: out STD_LOGIC);
    end component;
    signal Y1, Y2, Y3, Y3N: STD_LOGIC;
begin
    U1: vxor3 port map (I(1), I(2), I(3), Y1);
    U2: vxor3 port map (I(4), I(5), I(6), Y2);
    U3: vxor3 port map (I(7), I(8), I(9), Y3);
    Y3N <= not Y3;
    U4: vxor3 port map (Y1, Y2, Y3, ODD);
    U5: vxor3 port map (Y1, Y2, Y3N, EVEN);
end V74x280s;

```

Наш заключительный пример, приведенный в табл. 5.49, является вариантом схемы декодера Хэмминга (рис. 5.77) на языке VHDL. Функция syndrome (DU) вычисляет 3-разрядный синдром 7-разрядного неисправленного вектора входных данных DU. В «главном» процессе исправленному вектору выходных данных DC первоначально присваивается значение DU. Функция CONV_INTEGER (см. раздел 4.7.4) преобразует 3-разрядный синдром в целое число. Если синдром отличен от нуля, то соответствующий бит в DC инвертируется, и таким образом исправляется предполагаемая одиночная ошибка. Равенство синдрома нулю говорит о том, что ошибки либо нет, либо произошли необнаруживаемые ошибки; при этом на выходе NOERROR возникает единичный сигнал активного уровня.

5.9. Компараторы

Сравнение двух двоичных слов с целью обнаружения их равенства — это операция, широко применяемая в компьютерных системах и устройствах сопряжения. На рис. 2.7(а), например, была показана структура системы, в которой каждое из устройств способно сравнивать слово «выбор устройства» с предварительно установленным в нем «идентификатором устройства». Схема, которая сравнивает два двоичных слова и показывает, равны они или нет, называется *компаратором* (*comparator*). Некоторые компараторы интерпретируют входные слова как числа со знаком или без знака, а также выдают арифметическое соотношение между

Табл. 5.49. Посежденческая VHDL-программа для исправления ошибок в коде Хэмминга

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity hamcorr is
  port (
    DU: IN STD_LOGIC_VECTOR (1 to 7);
    DC: OUT STD_LOGIC_VECTOR (1 to 7);
    NOERROR: OUT STD_LOGIC
  );
end hamcorr;

architecture hamcorr of hamcorr is

  function syndrome (D: STD_LOGIC_VECTOR)
    return STD_LOGIC_VECTOR is
    variable SYN: STD_LOGIC_VECTOR (2 downto 0);
  begin
    SYN(0) := D(1) xor D(3) xor D(5) xor D(7);
    SYN(1) := D(2) xor D(3) xor D(6) xor D(7);
    SYN(2) := D(4) xor D(5) xor D(6) xor D(7);
    return(SYN);
  end syndrome;

begin
  process (DU)
    variable i: INTEGER;
  begin
    DC <= DU;
    i := CONV_INTEGER(syndrome(DU));
    if i = 0 then NOERROR <= '1';
    else NOERROR <= '0'; DC(i) <= not DU(i); end if;
  end process;
end hamcorr;

```

словами (больше или меньше). Эти устройства часто называются *компараторами значений* (*magnitude comparators*).

5.9.1. Структура компаратора

Схемы ИСКЛЮЧАЮЩЕЕ ИЛИ и ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ можно считать 1-разрядными компараторами. На рис. 5.78(а) вепталь ИСКЛЮЧАЮЩЕЕ ИЛИ (ИС 74х86) представлен как 1-разрядный компаратор. На выходе DIFF появляется сигнал высокого уровня в том случае, если сигналы на входах различны. Объединяя схемой ИЛИ выходы четырех схем ИСКЛЮЧАЮЩЕЕ ИЛИ, можно получить 4-разряд-

ный компаратор, как показано на рис. 5.78(b). Сигнал на выходе DIFF появляется в том случае, когда хотя бы в одном разряде значения входных сигналов различны. При наличии в достаточном количестве схем ИСКЛЮЧАЮЩЕЕ ИЛИ и вентилях ИЛИ с большим числом входов можно строить компараторы с любым числом входов.

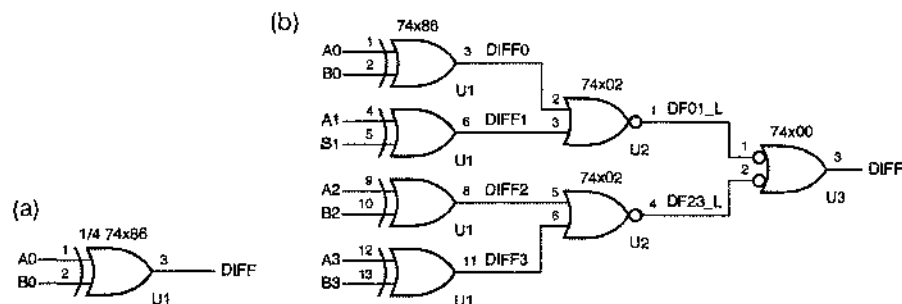


Рис. 5.78. Компараторы на основе ИС 74х86: (a) 1-разрядный компаратор; (b) 4-разрядный компаратор

5.9.2. Итерационные схемы

Итерационная схема (iterative circuit) представляет собой специальный тип комбинационной схемы, структура которой показана на рис. 5.79. Схема состоит из n идентичных модулей, каждый из которых имеет как *первичные входы и выходы (primary inputs and outputs)*, так и *междукаскадные входы и выходы (cascading inputs and outputs)*. В большинстве итерационных схем у крайнего левого модуля имеются отдельные входы, называемые *граничными входами (boundary inputs)* и на них подаются фиксированные логические значения. Специально выделенные выходы крайнего правого модуля называются *граничными выходами (boundary outputs)*, и обычно сигналы на этих выходах несут важную информацию.

ИТЕРАЦИОННЫЙ КОМПАРАТОР

Рассмотренные в разделе 5.9.1 n -разрядные компараторы можно было бы назвать *параллельными компараторами*, потому что в них одновременно анализируется каждая пара входных битов и n -разрядные результаты сравнения параллельно поступают на n -входовую схему ИЛИ или И. Можно также создать «итерационный компаратор», где сравнение производится поочередно в каждом разряде; в таком компараторе на разряд приходится небольшое фиксированное число логических элементов. Прежде чем браться за реализацию итерационного компаратора, следует разобраться в общем понятии «итерационные схемы», о которых идет речь в разделе 5.9.2.

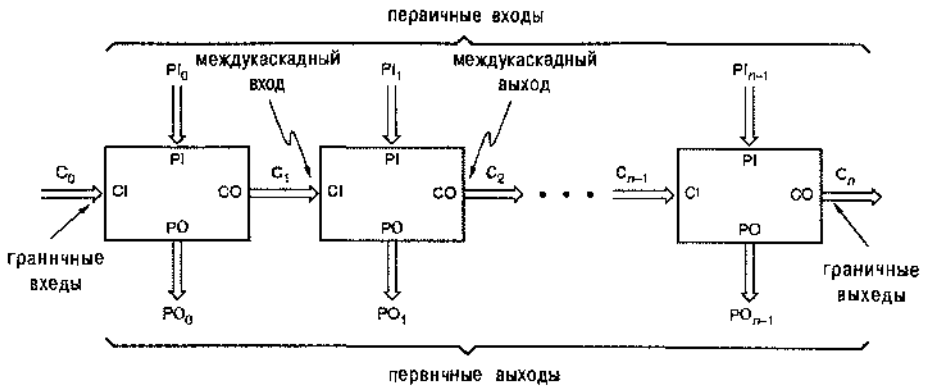


Рис. 5.79. Общая структура итерационной комбинационной схемы.

Итерационные схемы очень хорошо подходят для задач, которые можно решать применяя простой итерационный алгоритм:

1. Устанавливаем начальное значение C_0 и $i = 0$.
2. Используем C_i и PI_i для определения значений PO_i и C_{i+1} .
3. Увеличиваем значение i на 1.
4. Если $i < n$, то переходим к шагу 2.

В итерационной схеме цикл, состоящий из шагов 2–4, «распараллелен» путем выполнения шага 2 для каждого значения i отдельной комбинационной схемой.

Примерами итерационных схем могут служить компараторы, рассматриваемые в следующем разделе, а также сумматор со сквозным переносом, приведенный в разделе 5.10.2. 4-разрядный компаратор 74х85 и 4-разрядный сумматор 74х283 являются примерами СИС, которые можно использовать в качестве отдельных модулей в большой итерационной схеме. В параграфе 8.6 мы рассмотрим связь между итерационными схемами и соответствующими последовательностными схемами, которые реализуют описанный выше алгоритм по шагам, выполняемым в последовательные моменты времени.

5.9.3. Итерационная схема компаратора

Две n -разрядные величины X и Y можно сравнивать поочередно в каждом разряде, используя на каждом шаге единственный бит EQ_i для слежения за тем, что до данного шага включительно во всех парах биты были одинаковы:

1. Устанавливаем $EQ_0 = 1$ и $i = 0$.
2. Если $EQ_i = 1$ и значения X_i и Y_i одинаковы, то устанавливаем $EQ_{i+1} = 1$. В противном случае устанавливаем $EQ_{i+1} = 0$.
3. Увеличиваем значение i на 1.
4. Если $i < n$, то переходим к шагу 2.

На рис. 5.80 приведена соответствующая итерационная схема. Заметьте, что в этой схеме нет никаких первичных выходов; нас интересует только граничный выход. У других итерационных схем, например, у сумматора со сквозным переносом, который будет рассмотрен в разделе 5.10.2, имеются первичные выходы, и значения сигналов на этих выходах существенны.

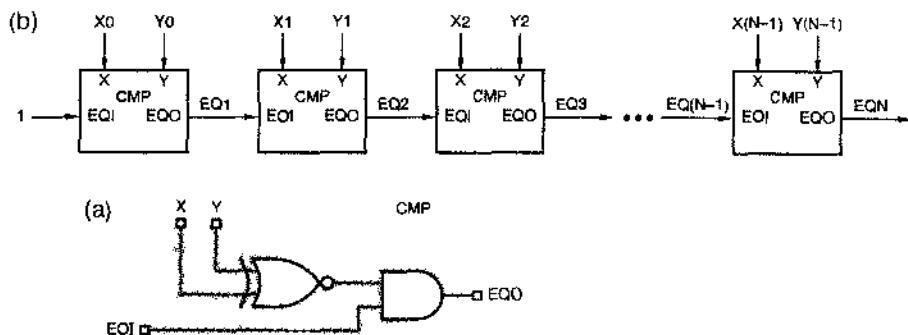


Рис. 5.80. Схема итерационного компаратора: (а) одноразрядный модуль, (б) полная схема

Имея возможность выбирать между итерационной схемой компаратора, рассмотренной в этом разделе, и одним из параллельных компараторов, приведенных ранее, вы, вероятно, отдадите предпочтение параллельному компаратору. Итерационный компаратор, возможно, позволит немного сэкономить в стоимости; но он очень медленный, потому что при последовательном включении модулей сигналам требуется время, чтобы «распространиться» от крайнего левого модуля до крайнего правого. Более перспективными с точки зрения использования в практических разработках являются итерационные схемы, в которых на каждом шаге обрабатывается большее число битов, для чего в качестве модулей применяются, например, 4-разрядные компараторы 74х85 или 4-разрядные сумматоры 74х283.

5.9.4. Стандартные компараторы в интегральном исполнении

Область применения компараторов настолько широка, что для серийного производства было разработано несколько СИС, являющихся компараторами. На рис. 5.81 дано условное обозначение 4-разрядного компаратора 74х85. У него есть выходы «больше» (AGTBOUT), «меньше» (ALTBOUT) и «равно» (AEQBOUT). У ИС '85 имеются также входы для каскадного включения (cascading inputs) (AGTBIN, ALTBIN, AEQBIN) для объединения нескольких таких ИС с целью создания компаратора с числом разрядов больше четырех. Сигналы на межкаскадных входах и выходах являются словами кода «1 из 3», так как при нормальной работе только на одном входе и на одном выходе должен присутствовать сигнал с активным уровнем.

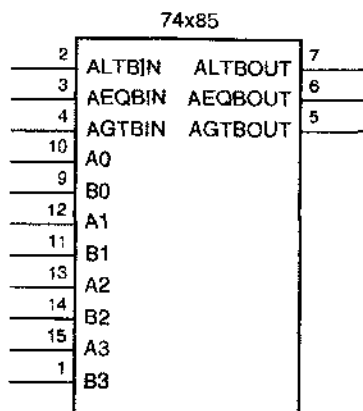


Рис. 5.81. Традиционное условное обозначение 4-разрядного компаратора 74x85

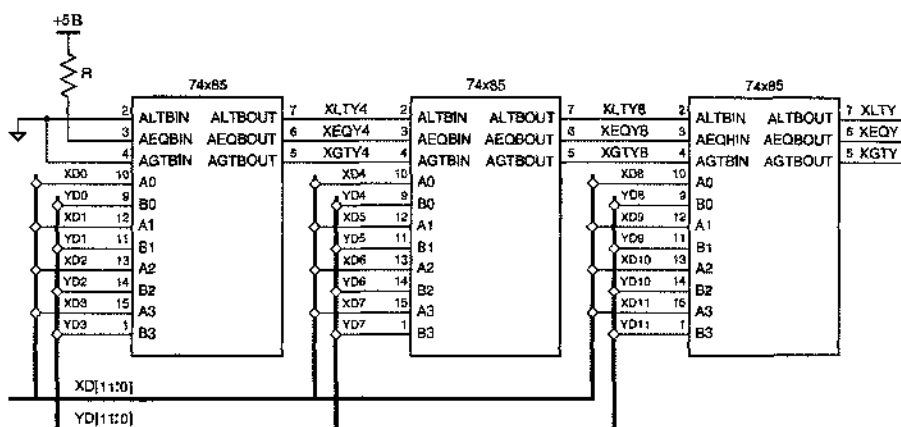


Рис. 5.82. 12-разрядный компаратор на основе ИС 74x85

Сигналы с выходов ИС '85, сравнивающих биты в младших разрядах, подаются на входы такой же ИС, сравнивающих биты в следующих разрядах, как показано на рис. 5.82 в случае 12-разрядного компаратора. Согласно определению, данному в разделе 5.9.2, эта схема является итерационной. Каждая ИС '85 вырабатывает на своих выходах сигналы, соответствующие примерно следующим выражениям псевдологики:

$$\text{AGTOUT} = (A > B) + (A=B) \cdot \text{AGTBIN}$$

$$\text{AEQOUT} = (A=B) \cdot \text{AEQBIN}$$

$$\text{ALTOUT} = (A < B) + (A=B) \cdot \text{ALTBIN}$$

Заключенные в скобки выражения не являются обычными логическими выражениями; правильнее сказать, что они означают арифметическое соотношение чисел, представленных сигналами на входах A3-A0 и B3-B0. Другими словами, сигнал на выходе AGTOUT появляется в двух случаях: когда $A > B$, либо когда $A = B$ и присутствует сигнал на входе AGTBIN (если на данном шаге биты равны, то

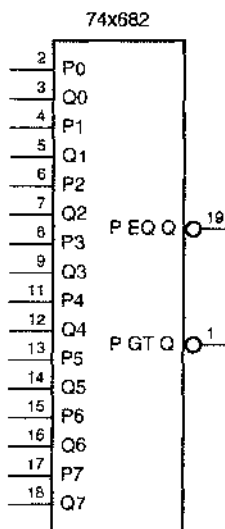
для получения правильного ответа необходимо учесть результат сравнения в младших разрядах). Мы снова встретимся с выражением такого вида при проектировании компаратора на языке ABEL в разделе 5.9.5. Арифметическое сравнение можно описать, применяя обычные логические выражения, например:

$$\begin{aligned}(A > B) = & A3 \cdot B3' + \\ & (A3 \oplus B3)' \cdot A2 \cdot B2' + \\ & (A3 \oplus B3)' \cdot (A2 \oplus B2)' \cdot A1 \cdot B1' + \\ & (A3 \oplus B3)' \cdot (A2 \oplus B2)' \cdot (A1 \oplus B1)' \cdot A0 \cdot B0'.\end{aligned}$$

Такие выражения нужно подставить в приведенные выше псевдологические равенства, чтобы получить настоящие логические равенства для сигналов на выходах компаратора.

В виде СИС выпускается несколько 8-разрядных компараторов. Самым простым из них является ИС 74х682, условное обозначение которого дано на рис. 5.83, а принципиальная схема показана на рис. 5.84. В верхней части схемы проверяется равенство двух 8-разрядных входных слов. Сигнал появляется на выходе каждой схемы ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ, если совпадают сигналы на его входах, а уровень сигнала на выходе PEQQ_L становится активным, если попарно равны сигналы во всех восьми разрядах чисел, подаваемых на входы. В нижней части схемы входные слова сравниваются арифметически и вырабатывается сигнал на выходе PGTO_L, если $P[7-0] > Q[7-0]$.

Рис. 5.83. Традиционное условное обозначение 8-разрядного компаратора 74х682



В отличие от ИС 74х85, у микросхемы 74х682 нет входов для каскадного включения и выхода «меньше». Однако любые требуемые условия, включая такие, как $\leq$ и $\geq$, можно получить в виде функции выходных сигналов PEQQ_L и PGTO_L, как показано на рис. 5.85.

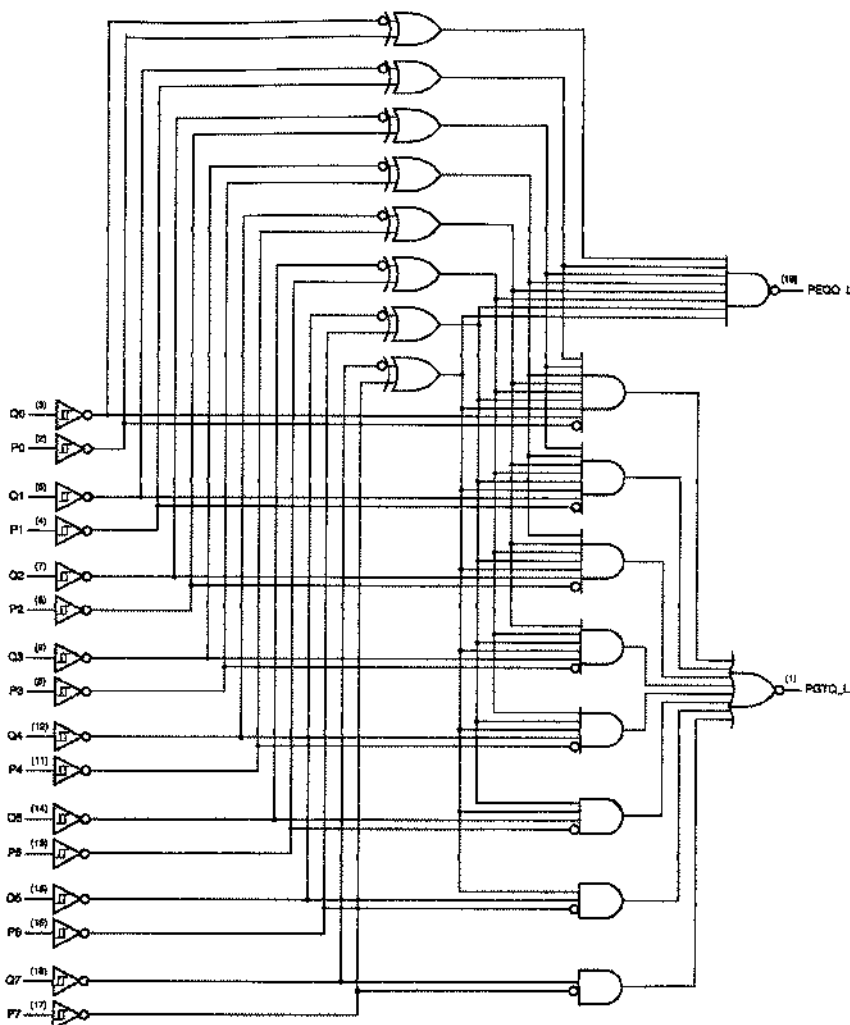
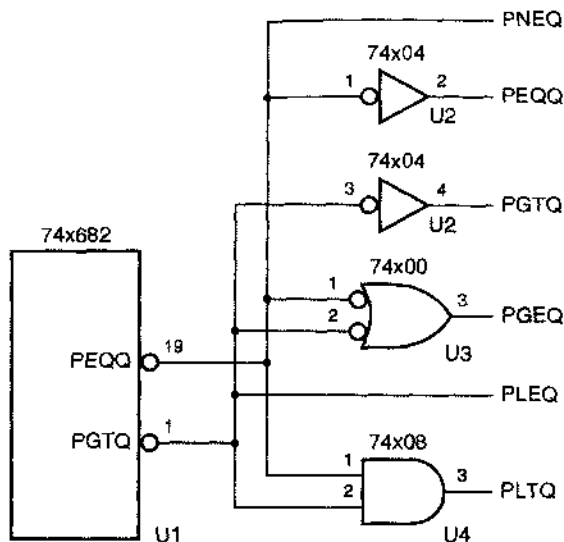


Рис. 5.84. Принципиальная схема 8-разрядного компаратора 74х682 с цоколевкой для стандартного корпуса DIP с 20 выводами

СРАВНЕНИЕ КОМПАРАТОРОВ

Отдельные 1-разрядные компараторы (вентили ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ) в ИС '682 используются иначе по сравнению с примерами раздела 5.9.4: сигналы на выходах этих вентилей имеют активный уровень при *равных* сигналах на входах и затем объединяются схемой И, в отличие от схем с активным уровнем выходных сигналов при *различных* сигналах на входах с последующим объединением по правилу ИЛИ. Работу компаратора можно рассматривать любым образом, пока мы не входим в противоречие.

Рис. 5.85. Формирование сигналов, выражающих арифметическое соотношение, из выходных сигналов ИС 74х682



5.9.5. Описание компараторов на языке ABEL и их реализация в ПЛУ

Используя язык ABEL, очень просто осуществить сравнение двух наборов с целью определения их равенства или неравенства с помощью операторов “=” или “!=” в выражении отношения. Единственным ограничением является то, что число элементов в сравниваемых наборах должно быть одинаковым. Таким образом, в ответ на заданное выражение отношения “A!=B”, где A и B – наборы, состоящие из n элементов каждый, компилятор сформирует логическое выражение

$$(A1 \neq B1) \# (A2 \neq B2) \# \dots \# (An \neq Bn).$$

Логическое выражение для отношения “A==B” является точной инверсией приведенного выше.

Согласно приведенному логическому выражению для сравнения в каждом бите требуется одна 2-входовая схема ИСКЛЮЧАЮЩЕЕ ИЛИ. Так как 2-входовую функцию ИСКЛЮЧАЮЩЕЕ ИЛИ можно представить в виде суммы двух термов-произведений, полное выражение можно реализовать с помощью ПЛУ в виде относительно небольшой суммы $2n$ термов-произведений:

$$(A1 \& !B1 \# !A1 \& B1) \# (A2 \& !B2 \# !A2 \& B2) \# \dots \# (An \& !Bn \# !An \& Bn).$$

Хотя в языке ABEL имеются операторы отношения «меньше» и «больше», результирующие логические выражения не так компактны и не так легко образуются. Рассмотрим, например, выражение отношения “A<B”, где {A...A1} и {Bn...B1} – наборы, состоящие из n элементов каждый. Чтобы получить соответствующее логическое выражение на языке ABEL, сначала составляются n соотношений вида:

$$Li = (!Ai \& (Bi \# Li-1)) \# (Ai \& Bi \& Li-1)$$

для $i = 1 \dots n$, где, по определению, $L_0 = 0$. По существу, это — итерационное определение функции «меньше», начиная с младшего разряда. Выражение для L_i позволяет говорить на i -м шаге о том, что A меньше B , если $A_i = 0$, а $B_i = 1$ или A уже было меньше B на предыдущем шаге, а также в том случае, когда A_i и B_i равны 1, но A было меньше B на предыдущем шаге.

Значение логического выражения для “ $A < B$ ” определяется просто равенством для L_n . Итак, после составления n соотношений, компаратор языка ABEL сворачивает их в одно выражение для L_n , включающее только элементы наборов A и B . Это осуществляется путем подстановки выражения L_{n-1} в правую часть равенства для L_n , затем в полученное выражение подставляется L_{n-2} и так далее, пока в качестве L_0 не будет подставлен 0. В конце концов это приводит к результирующему выражению вида минимальной суммы произведений.

Свертывание итерационной схемы до реализации в виде двухуровневой суммы произведений обычно приводит к экспоненциальному увеличению числа термов-произведений с ростом n . Если следовать этой модели, то для n -разрядного сравнения “ $<$ ” потребуется $2^n - 1$ термов-произведений. Таким образом, не прибегая к двухпроходной логике, в ПЛИУ можно реализовать только компараторы, рассчитанные на небольшое число разрядов.

Очевидно, что аналогичные результаты справедливы в отношении операции “ $>$ ”, а логические выражения для “ $>=$ ” и “ $<=$ ” еще хуже, поскольку являются инверсией выражений для “ $<$ ” и “ $>$ ”. Если используется ПЛИУ с управляемой инверсностью выходных сигналов, где доступны инверсия, то число термов-произведений то же самое; в противном случае минимальное число термов-произведений после инвертирования становится равным $2^n + 2^{n-1} - 1$.

5.9.6. Описание компараторов на языке VHDL

В языке VHDL есть операторы сравнения для всех встроенных типов. Операторы равенства (*equality*, $=$) и неравенства (*inequality*, $\neq$) применимы ко всем типам; для массивов и структурных типов операнды должны иметь одинаковые размеры и структуру, и их сравнение осуществляется покомпонентно. Во многих примерах в этой главе мы использовали оператор равенства для сравнения сигнала или сигнала с константой с константой. Когда мы сравниваем два сигнала или две переменные, механизм синтеза формирует выражения, аналогичные выражениям на языке ABEL, приведенным в предыдущем разделе.

Другие операторы сравнения $>$, $<$, $>=$ и $<=$ в языке VHDL применяются только к целочисленным типам, перечислимому типу (таким, как `STD_LOGIC`) и одномерным массивам перечислимых или целочисленных типов. Естественным порядком для целых чисел является их расположение от меньшего к большему, от минус бесконечности до плюс бесконечности, а в перечислимых типах используется тот порядок, в котором элементы массива были определены: от первого до последнего (если вы явно не изменяете порядок перебора с помощью команды, имеющейся в используемых вами средствах синтеза, то упорядочение будет таким, как вы его задали).

Сравнение массивов осуществляется итеративно, начиная с крайнего левого элемента в каждом массиве. Массивы всегда сравниваются слева направо незави-

симо от направления, в котором их индексы пробегают свой диапазон значений ("to" или "downto"). Порядок массивов определяется по старшинству в самой левой паре правых элементов. Если размеры массивов разные и все элементы короткого массива совпадают с соответствующими элементами длинного массива, то считается, что короткий массив меньше длинного.

В результате массивы равной длины типа `BIT_VECTOR` или `STD_LOGIC_VECTOR` воспринимаются встроенными операторами сравнения так, как будто они представляют собой целые числа без знака. Но если длина массивов разная, то операторы сравнения не дают того результата, который вы получили бы, дополнив более короткий массив нулями слева и сравнивая массивы арифметически; вскоре об этом будет сказано подробнее.

В табл. 5.50 приведена программа на языке VHDL, которая позволяет получить все возможные результаты сравнения двух 8-разрядных целых чисел без знака. Так как два входных вектора A и B имеют равные длины, программа дает правильный ответ.

Табл. 5.50. Поведенческая VHDL-программа сравнения 8-разрядных целых чисел без знака

```
library IEEE;
use IEEE.std_logic_1164.all;

entity vcompare is
  port (
    A, B: in STD_LOGIC_VECTOR (7 downto 0);
    EQ, NE, GT, GE, LT, LE: out STD_LOGIC
  );
end vcompare;

architecture vcompare_arch of vcompare is
begin
  process (A, B)
  begin
    EQ <= '0'; NE <= '0'; GT <= '0'; GE <= '0'; LT <= '0'; LE <= '0';
    if A = B then EQ <= '1'; end if;
    if A /= B then NE <= '1'; end if;
    if A > B then GT <= '1'; end if;
    if A >= B then GE <= '1'; end if;
    if A < B then LT <= '1'; end if;
    if A <= B then LE <= '1'; end if;
  end process;
end vcompare_arch
```

Для выполнения более гибких сравнений и арифметических действий стандартом IEEE 1076-3 предусмотрен типовый пакет `std_logic_arith`, в котором введены два новых важных типа и масса функций сравнения и арифметических функций для работы с ними. Двумя новыми типами являются `SIGNED` и `UNSIGNED`:

```
type SIGNED is array NATURAL range < > STD_LOGIC;
type UNSIGNED is array (NATURAL range < >) STD_LOGIC;
```

Отсюда видно, что оба типа определены как массивы элементов типа `STD_LOGIC` неопределенной длины и не отличаются от `STD_LOGIC_VECTOR`. Важно, что в пакете определены также новые функции сравнения, которые вызываются в тех случаях, когда один или оба операнда сравнения являются элементами одного из новых типов. Например, в пакете имеются восемь новых функций «меньше» со следующими комбинациями параметров:

```
function "<" (L: UNSIGNED; R: UNSIGNED) return BOOLEAN;
function "<" (L: SIGNED; R: SIGNED) return BOOLEAN;
function "<" (L: UNSIGNED; R: SIGNED) return BOOLEAN;
function "<" (L: SIGNED; R: UNSIGNED) return BOOLEAN;
function "<" (L: UNSIGNED; R: INTEGER) return BOOLEAN;
function "<" (L: INTEGER; R: UNSIGNED) return BOOLEAN;
function "<" (L: SIGNED; R: INTEGER) return BOOLEAN;
function "<" (L: INTEGER; R: SIGNED) return BOOLEAN;
```

Таким образом, оператор «<» можно использовать в любой комбинации операндов `SIGNED`, `UNSIGNED` и `INTEGER`; компилятор выбирает ту функцию, типы параметров у которой соответствуют фактическим операндам. Каждая из функций в пакете определена так, чтобы все делать «правильно», включая соответствующие расширения и преобразования, когда встречаются операнды разных размеров или типов. Аналогичные функции предусмотрены для пяти других операторов отношения: `=`, `/=`, `<=`, `>` и `>=`.

Используя пакет `IEEE_std_logic_arith`, можно писать программы удобно тому, как это сделано в табл. 5.51. В ней `A`, `B`, `C` и `D` являются 8-разрядными входными векторами трех различных типов. В сравнениях, включающих `A`, `B` и `C`, компилятор автоматически выбирает правильный вариант функций сравнения; например, для «`A < B`» он выбирает первую из приведенных выше функций «<», потому что оба операнда принадлежат типу `UNSIGNED`.

В сравнениях, включающих `D`, используются явные преобразования типов. Предполагается, что разработчик хочет, чтобы этот конкретный массив типа `STD_LOGIC_VECTOR` интерпретировался как элемент типа `UNSIGNED` в одном случае и как `SIGNED` в другом. Здесь важно понимать, что в пакете `std_logic_arith` не делается никаких предположений относительно того, как должны интерпретироваться массивы типа `STD_LOGIC_VECTOR`; указать необходимое преобразование должен пользователь.

В двух других пакетах `std_logic_signed` и `std_logic_unsigned`, приняты определенные предположения относительно массивов типа `STD_LOGIC_VECTOR`, и эти пакеты полезны в том случае, когда все элементы типа `STD_LOGIC_VECTOR` должны интерпретироваться одинаково. Каждый пакет содержит три варианта каждой из функций сравнения для того, чтобы при сравнении друг с другом или с целыми числами элементы типа `STD_LOGIC_VECTOR` интерпретировались как элементы типа `SIGNED` или элементы типа `UNSIGNED` соответственно.

Табл. 5.51. Поведенческая VHDL-программа сравнения 8-разрядных целых чисел различных типов

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity vcompa is
  port (
    A, B: in UNSIGNED (7 downto 0);
    C: in SIGNED (7 downto 0);
    D: in STD_LOGIC_VECTOR (7 downto 0);
    A_LT_B, B_GE_C, A_EQ_C, C_NEG, D_BIG, D_NEG: out STD_LOGIC
  );
end vcompa,

architecture vcompa_arch of vcompa is
begin
  process (A, B, C, D)
  begin
    A_LT_B <= '0'; B_GE_C <= '0'; A_EQ_C <= '0'; C_NEG <= '0'; D_BIG <= '0'; D_NEG <= '0';
    if A < B then A_LT_B <= '1'; end if;
    if B >= C then B_GE_C <= '1'; end if;
    if A = C then A_EQ_C <= '1'; end if;
    if C < 0 then C_NEG <= '1'; end if;
    if UNSIGNED(D) > 200 then D_BIG <= '1'; end if;
    if SIGNED(D) < 0 then D_NEG <= '1'; end if;
  end process;
end vcompa_arch,

```

Если в VHDL-программе имеется функция сравнения, то ее реализация в виде двухуровневой суммы произведений потребует столько же термов-произведений, сколько и в языке ABEL. Однако большинство VHDL-средств синтеза реализуют компаратор в виде итерационной схемы с гораздо меньшим числом вентилях, хотя и с большим числом уровней логики. Кроме того, лучшие программные средства синтеза обнаруживают возможность удаления целых схем компараторов. Например, в программе, приведенной в табл. 5.50, каждый из выходов NE, GE и LE можно реализовать путем инвертирования выходов EQ, LT и GT соответственно, используя для этого всего лишь по одному инвертору.

*5.10. Сумматоры, вычитающие устройства и АЛУ

В цифровых системах сложение является самым распространенным арифметическим действием. Сумматор (*adder*) объединяет два арифметических операнда по правилам сложения, описанным в главе 2. Как было показано в параграфе 2.6, одни и те же правила сложения справедливы для чисел без знака и для чисел, представленных в дополнительном двоичном коде; поэтому в обоих случаях используются одни и те же сумматоры. Сумматор может выполнять вычитание путем сложения уменьшаемого с дополнением к вычитаемому (инвертированного вычитаемого); но можно построить и *вычитающее устройство (subtractor)*, которое выполняет вычитание непосредственно. Ис средней степени интеграции, выполняющие сложение, вычитание и другие действия в зависимости от кода опе-

рации на управляющих входах, называются арифметическо-логическими устройствами (АЛУ); они описаны в разделе 5.10.6.

*5.10.1. Полусумматоры и полные сумматоры

Простейший сумматор, называемый *полусумматором* (*half adder*), складывает два 1-разрядных операнда X и Y , образуя 2-разрядную сумму. Сумма может принимать значения от 0 до 2, требуя для своего представления двух битов. Младший бит суммы можно назвать полусуммой HS , а старший бит – переносом CO (в старший разряд). Для величин HS и CO можно записать следующие выражения:

$$\begin{aligned} HS &= X \oplus Y \\ &= X \cdot Y' + X' \cdot Y \\ CO &= X \cdot Y. \end{aligned}$$

Чтобы сложить операнды с большим числом двоичных разрядов, необходимо обеспечить перенос между разрядами. Стандартный блок, применяемый для этой операции, называется *полным сумматором* (*full adder*). Помимо входов для битов слагаемых X и Y , у полного сумматора есть вход для бита переноса CIN . Сумма трех входных битов может принимать значения от 0 до 3; для ее представления все-таки достаточно двух выходных битов S и $COUT$, значения которых определяются следующими соотношениями:

$$\begin{aligned} S &= X \oplus Y \oplus CIN \\ &= X \cdot Y' \cdot CIN' + X' \cdot Y \cdot CIN' + X' \cdot Y' \cdot CIN + X \cdot Y \cdot CIN \\ COUT &= X \cdot Y + X \cdot CIN + Y \cdot CIN \end{aligned}$$

Здесь $S = 1$, если на нечетном числе входов присутствуют единицы, а $COUT = 1$, если единицы имеются на двух или большем числе входов. Эти соотношения представляют ту же самую операцию, которая определяется таблицей двоичного сложения (табл. 2.3).

Одна из возможных схем, реализующих соотношения, которыми описывается полный сумматор, приведена на рис. 5.86(а). Соответствующее условное обозначение дано на рис. 5.86(б). Иногда для более аккуратного изображения схем с последовательно включенными полными сумматорами их обозначают так, как показано на рис. 5.86(с); именно такое обозначение применено в следующем разделе.

*5.10.2. Сумматоры со сквозным переносом

Два n -разрядных двоичных слова можно сложить с помощью *сумматора со сквозным переносом* (*ripple adder*), состоящего из n последовательно включенных полных сумматоров, каждый из которых оперирует с одним битом. На рис. 5.87 показана схема 4-разрядного сумматора со сквозным переносом. На входе переноса младшего разряда (c_0) обычно устанавливается 0, а выход переноса каждого из полных сумматоров соединен со входом переноса полного сумматора в следующем разряде. Согласно определению, данному в разделе 5.9.2, сумматор со сквозным переносом является классическим примером итерационной схемы.

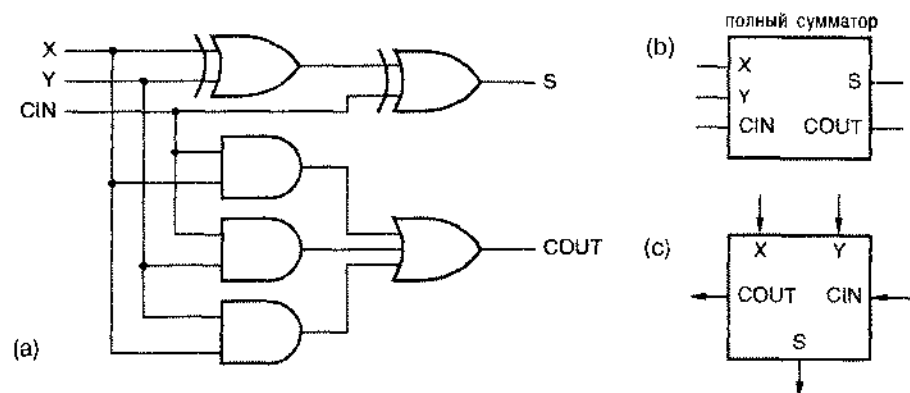


Рис. 5.86. Полный сумматор. (а) принципиальная схема на уровне вентилей, (б) условное обозначение, (с) другое условное обозначение, удобное для изображения последовательного включения

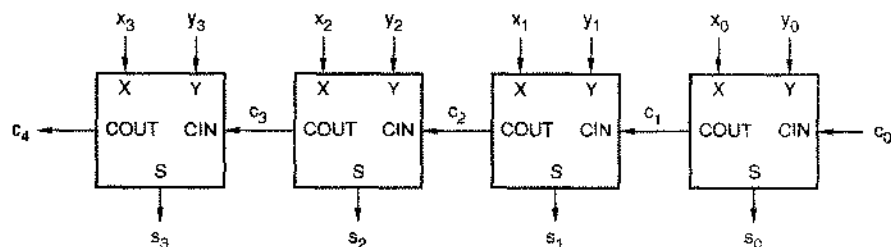


Рис. 5.87. 4-разрядный сумматор со сквозным переносом

Сумматор со сквозным переносом обладает малым быстродействием, так как в худшем случае сигнал переноса должен распространиться от младшего полного сумматора до старшего. Такая ситуация имеет место, например, если одно слагаемое равно $11 \dots 11$, а другое $-00 \dots 01$. Если все биты слагаемых подаются одновременно, то полная задержка в худшем случае равна

$$t_{\text{ADD}} = t_{\text{XYCOU}} + (n-2) \cdot t_{\text{CinCOU}} + t_{\text{CinS}},$$

где t_{XYCOU} — задержка от входов X или Y до выхода COU в сумматоре младшего разряда, t_{CinCOU} — задержка от входа CIN до выхода COU в сумматорах средних разрядов, а t_{CinS} — задержка от входа CIN до выхода S в сумматоре старшего разряда.

Более быстрый сумматор можно настроить, используя для формирования суммы на каждом выходе s_i двухуровневую логику. Это можно выполнить, записывая выражение для s_i через $x_0 \dots x_i, y_0 \dots y_i$ и c_0 , разложив множители по слагаемым или разложив слагаемые по сомножителям для преобразования выражения в сумму произведений или произведение сумм и применяя соответствующую схему И-ИЛИ или ИЛИ-И. К сожалению, выражения для сумм, начиная с s_2 , содержат очень много членов, и для их реализации требуется слишком много схем первого

уровня и слишком много входов у вентилях второго уровня по сравнению с тем, чем мы обычно располагаем. Например, даже при $c_0 = 0$ для образования s_2 требуется двухуровневая схема И–ИЛИ с четырнадцатью 4-входовыми схемами И, четырьмя 5-входовыми схемами И и 18-входовой схемой ИЛИ; с формированием сумм старших разрядов ситуация еще хуже. Тем не менее возможно, как мы увидим в разделе 5.10.4, обойтись более приемлемым количеством схем, строя сумматоры с совсем небольшим числом уровней задержки.

*5.10.3. Вычитающие устройства

Операция вычитания в двоичной системе, аналогичная двоичному сложению, также была определена в табл. 2.3. *Полное вычитающее устройство (full subtractor)* реализует алгоритм вычитания в двоичной системе для одного двоичного разряда при наличии на входах X , Y и BIN битов уменьшаемого, вычитаемого и заема; на выходах D и $BOUT$ вырабатываются биты разности и заема. Логические соотношения, соответствующие таблице вычитания в двоичной системе, можно записать следующим образом:

$$\begin{aligned} D &= X \oplus Y \oplus BIN \\ BOUT &= X' \cdot Y + X' \cdot BIN + Y \cdot BIN. \end{aligned}$$

Эти соотношения очень похожи на равенства для полного сумматора, и это не удивительно. В параграфе 2.6 было показано, что вычитание $X - Y$ в дополнительном коде можно выполнить с помощью операции сложения, а именно, производя сложение точного дополнения Y с X . Точное дополнение Y равно $\bar{Y} + 1$, где $\bar{Y}$ — поразрядное дополнение Y . Решая задачу 2.26, можно было убедиться в том, что двоичный сумматор можно использовать для нахождения разности $X - Y$ чисел без знака, производя сложение $X + \bar{Y} + 1$. Преобразуя логические выражения, приведенные выше, мы можем теперь еще раз подтвердить справедливость этого правила:

$$\begin{aligned} BOUT &= X' \cdot Y + X' \cdot BIN + Y \cdot BIN \\ BOUT' &= (X + Y') \cdot (X + BIN') \cdot (Y' + BIN') \quad (\text{обобщенная теорема Де Моргана}) \\ &= X \cdot Y' + X \cdot BIN' + Y' \cdot BIN' \quad (\text{разнесение множителей по слагаемым}) \\ D &= X \oplus Y \oplus BIN \\ &= X \oplus Y' \oplus BIN' \quad (\text{инвертирование сигналов на входах схемы ИСКЛЮЧАЮЩЕЕ ИЛИ}) \end{aligned}$$

В связи с последним преобразованием напомним, что при инвертировании сигналов на двух входах схемы ИСКЛЮЧАЮЩЕЕ ИЛИ реализуемая ею функция не изменяется.

Сравнение приведенных выше соотношений с равенствами для полного сумматора свидетельствует о том, что полное вычитающее устройство можно получить из полного сумматора, как показано на рис. 5.88. Только ради ясности изложения мы дали схеме полного сумматора на рис. 5.88(а) фиктивное название “74х999”. Как показано на рис. 5.88(с), функцию этой же самой физической схемы можно интерпретировать как вычитание, введя новое условное обозначение с низким

активным уровнем сигналов на входе заема, на выходе заема и на входе вычитаемого.

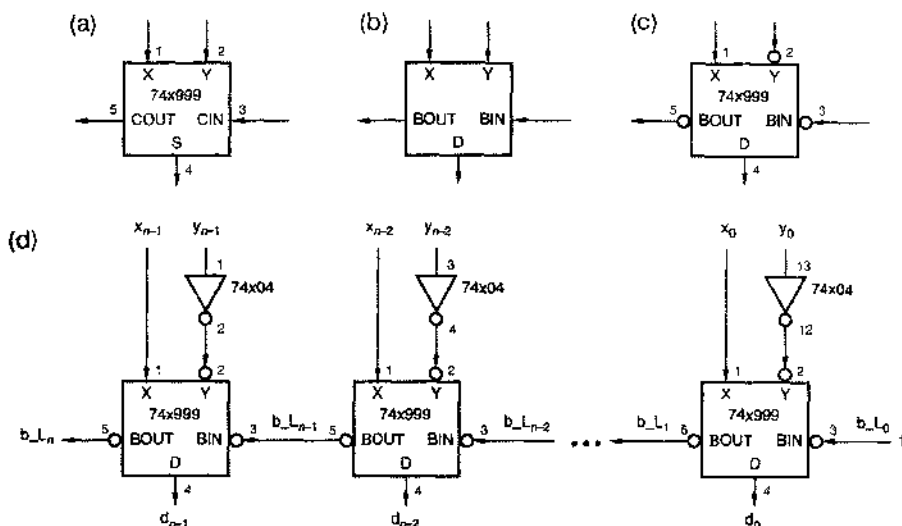


Рис. 5.88. Построение вычитающих устройств на основе сумматоров: (а) полиый сумматор; (б) полное вычитающее устройство; (с) интерпретация устройства, изображенного на рис. (а), как полиого вычитающего устройства; (д) вычитающее устройство со сквозным заемом

Таким образом, чтобы построить вычитающее устройство со сквозным переносом для двух n -разрядных операндов с высоким активным уровнем сигнала, можно воспользоваться n сумматорами 74x999 и n инверторами, как показано на рис. 5.88(д). Заметьте, что при вычитании сигнал на вход заема младшего разряда не должен поступать (отсутствие заема); при низком активном уровне входного сигнала это означает, что на реальный вывод должна быть подана логическая 1, то есть сигнал высокого уровня. Это прямо противоположно тому, что должно быть при суммировании, когда на тот же самый вход переиоса с высоким активным уровнем сигнала подается логический 0, то есть сигнал низкого уровня.

Возвращаясь к математике главы 2, можно показать, что этот вид преобразования справедлив для всех схем сумматоров и вычитающих устройств, а не только для сумматоров и вычитающих устройств со сквозным переносом. То есть любую n -разрядную схему сложения можно заставить работать как вычитающее устройство, беря дополнение вычитаемого и интерпретируя сигналы на входах и выходах переиоса как сигналы заема с противоположным активным уровнем. В остальной части этого параграфа мы будем говорить только о схемах сложения, имея в виду, что их легко применить и для вычитания.

*5.10.4. Сумматоры с ускоренным переносом

Логическое соотношение для суммы i -го разряда двоичного сумматора фактически можно записать очень просто:

$$s_i = x_i \oplus y_i \oplus c_i.$$

Значительные сложности появляются при попытке представить c_i через x_0-x_{i-1}, y_0-y_{i-1} и c_0 ; реальные неурядицы возникают в связи с ростом числа схем ИСКЛЮЧАЮЩЕЕ ИЛИ. Однако, если мы хотим предотвратить увеличение числа этих схем, то можно, по крайней мере, упростить логику формирования c_i , используя идеи *ускоренного переноса* (*carry lookahead*), рассматриваемые в этом разделе.

На рис. 5.89 продемонстрирована основная идея. В блоке, названном «Логическая схема ускоренного переноса», значение c_i вычисляется по правилам, предусматривающим небольшое, фиксированное число логических уровней при любом разумном значении i . Для схемы ускоренного переноса ключевыми являются следующие два определения:

- Говорят, что при заданной комбинации сигналов на входах x_i и y_i в i -м каскаде сумматора *генерируется сигнал переноса* (*carry generate*), если в этом каскаде вырабатывается 1 на выходе переноса ($c_{i+1} = 1$) независимо от значений входных сигналов x_0-x_{i-1}, y_0-y_{i-1} и c_0 .
- Говорят, что при заданной комбинации сигналов на входах x_i и y_i в i -м каскаде сумматора происходит *передача сигнала переноса* (*carry propagate*), если в этом каскаде вырабатывается 1 на выходе переноса ($c_{i+1} = 1$) в присутствии такой комбинации на входах x_0-x_{i-1}, y_0-y_{i-1} и c_0 , которая вызывает появление 1 на входе переноса данного каскада ($c_i = 1$).

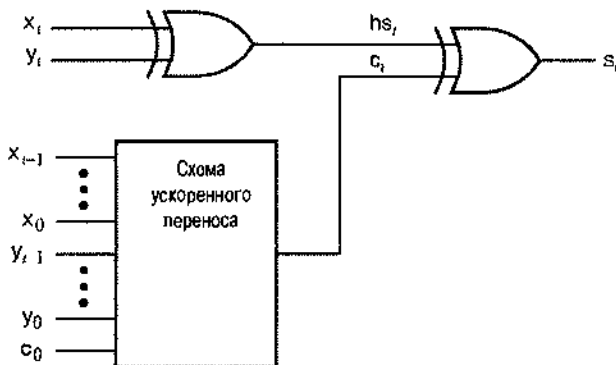


Рис. 5.89. Структура одного каскада сумматора с ускоренным переносом

В соответствии с этими определениями можно записать логические равенства для сигнала генерации переноса g_i и сигнала передачи переноса p_i в каждом каскаде сумматора с ускоренным переносом:

$$g_i = x_i \cdot y_i,$$

$$p_i = x_i \oplus y_i.$$

Другими словами, на выходе каскада безусловно генерируется перенос, если оба бита слагаемых равны 1, и передается перенос, если хотя бы один из битов слагаемых равен 1. Теперь сигнал на выходе переноса можно выразить через сигналы генерации и передачи переноса:

$$c_{i+1} = g_i + p_i \cdot c_i$$

Чтобы исключить сквозной перенос, мы для каждого каскада рекурсивно находим значения c_i , разности множителей по слагаемым, получаем выражения в виде двухуровневых функций И–ИЛИ. Используя эту методику, можно найти следующие выражения для сигналов переноса первых четырех каскадов сумматора:

$$\begin{aligned} c_1 &= g_0 + p_0 \cdot c_0 \\ c_2 &= g_1 + p_1 \cdot c_1 \\ &= g_1 + p_1 \cdot (g_0 + p_0 \cdot c_0) \\ &= g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0 \\ c_3 &= g_2 + p_2 \cdot c_2 \\ &= g_2 + p_2 \cdot (g_1 + p_1 \cdot g_0 + p_1 \cdot p_0 \cdot c_0) \\ &= g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0 \\ c_4 &= g_3 + p_3 \cdot c_3 \\ &= g_3 + p_3 \cdot (g_2 + p_2 \cdot g_1 + p_2 \cdot p_1 \cdot g_0 + p_2 \cdot p_1 \cdot p_0 \cdot c_0) \\ &= g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot g_0 + p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot c_0 \end{aligned}$$

Каждое приведенное выражение соответствует схеме, имеющей только три уровня задержки: один уровень связан с образованием сигналов генерации и передачи переноса, а два других – с образованием суммы произведений. В блоках «ускоренного переноса» каждого каскада (рис. 5.89) сумматора с ускоренным переносом (*carry lookahead adder*) используются трехуровневые выражения типа приведенных выше. Выходной сигнал суммы в каждом разряде формируется путем комбинации бита переноса с битами двух слагаемых данного разряда, как показано на рисунке. В следующем разделе мы рассмотрим некоторые серийно выпускаемые СИС, содержащие сумматоры и арифметическо-логические устройства с ускоренным переносом.

*5.10.5. Сумматоры, выполненные в виде ИС средней степени интеграции

В 4-разрядном двоичном сумматоре 74х283 применен метод ускоренного переноса, и сигналы суммы и переноса формируются логикой с небольшим числом уровней. На рис. 5.90 дано условное обозначение ИС 74х283. Более ранняя ИС 74х83 идентична схеме 74х283, за исключением нестандартного расположения выводов для подключения напряжения питания и земли.

Принципиальная схема сумматора 7283, приведенная на рис. 5.91, лишь немногим отличается от обычной схемы с ускоренным переносом, описанной в предыдущем разделе. Прежде всего, слагаемые вместо X и Y называются A и B; но это не самое главное. Второе отличие состоит в том, что этот сумматор вырабатывает сигналы генерации переноса (g_i') и передачи переноса (p_i') с низким активным уровнем, поскольку инвертирующие схемы обычно обладают большим быстродействием, чем неинвертирующие. Третье отличие является следствием следующего алгебраического преобразования полусуммы:

$$\begin{aligned} hs_i &= x_i \oplus y_i \\ &= x_i \cdot y_i' + x_i' \cdot y_i \end{aligned}$$

$$\begin{aligned}
 &= x_i \cdot y_i' + x_i' \cdot x_i + x_i' \cdot y_i + y_i' \cdot y_i' \\
 &= (x_i + y_i) \cdot (x_i' + y_i') \\
 &= (x_i + y_i) \cdot (x_i \cdot y_i)' \\
 &= p_i \cdot g_i'
 \end{aligned}$$

Таким образом, для формирования в каждом разряде значения полусуммы вместо схемы ИСКЛЮЧАЮЩЕЕ ИЛИ можно применять схему И с одним инвертированным входом.

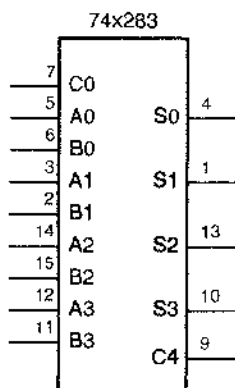


Рис. 5.90. Традиционное условное обозначение 4-разрядного двоичного сумматора 74x283

Наконец, в микросхеме '283 сигнал переноса вырабатывается с помощью структуры НЕ-ИЛИ-И (эквивалентной, согласно теореме Де Моргана, структуре И-ИЛИ-НЕ), которая вносит примерно такую же задержку, как один инвертирующий КМОП-или ТТЛ-вентиль. Этот факт требует некоторого объяснения, так как выражения для сигнала переноса, которые мы получили в предыдущем разделе, используются здесь в слегка измененном виде. В частности, в выражении для c_{i+1} слагаемое g_i заменено на $p_i \cdot g_i$. Это не влияет на значение сигнала переноса, так как p_i всегда равняется 1, когда $g_i = 1$. Но это позволяет следующим образом разложить выражение для c_{i+1} на множители:

$$\begin{aligned}
 c_{i+1} &= p_i \cdot g_i + p_i \cdot c_i \\
 &= p_i \cdot (g_i + c_i).
 \end{aligned}$$

Это приводит к следующим соотношениям для сигналов переноса, которые и реализуются в данной схеме:

$$\begin{aligned}
 c_1 &= p_0 \cdot (g_0 + c_0) \\
 c_2 &= p_1 \cdot (g_1 + c_1) \\
 &= p_1 \cdot (g_1 + p_0 \cdot (g_0 + c_0)) \\
 &= p_1 \cdot (g_1 + p_0) \cdot (g_1 + g_0 + c_0) \\
 c_3 &= p_2 \cdot (g_2 + c_2) \\
 &= p_2 \cdot (g_2 + p_1 \cdot (g_1 + p_0) \cdot (g_1 + g_0 + c_0)) \\
 &= p_2 \cdot (g_2 + p_1) \cdot (g_2 + g_1 + p_0) \cdot (g_2 + g_1 + g_0 + c_0) \\
 c_4 &= p_3 \cdot (g_3 + c_3) \\
 &= p_3 \cdot (g_3 + p_2 \cdot (g_2 + p_1) \cdot (g_2 + g_1 + p_0) \cdot (g_2 + g_1 + g_0 + c_0)) \\
 &= p_3 \cdot (g_3 + p_2) \cdot (g_3 + g_2 + p_1) \cdot (g_3 + g_2 + g_1 + p_0) \cdot (g_3 + g_2 + g_1 + g_0 + c_0).
 \end{aligned}$$

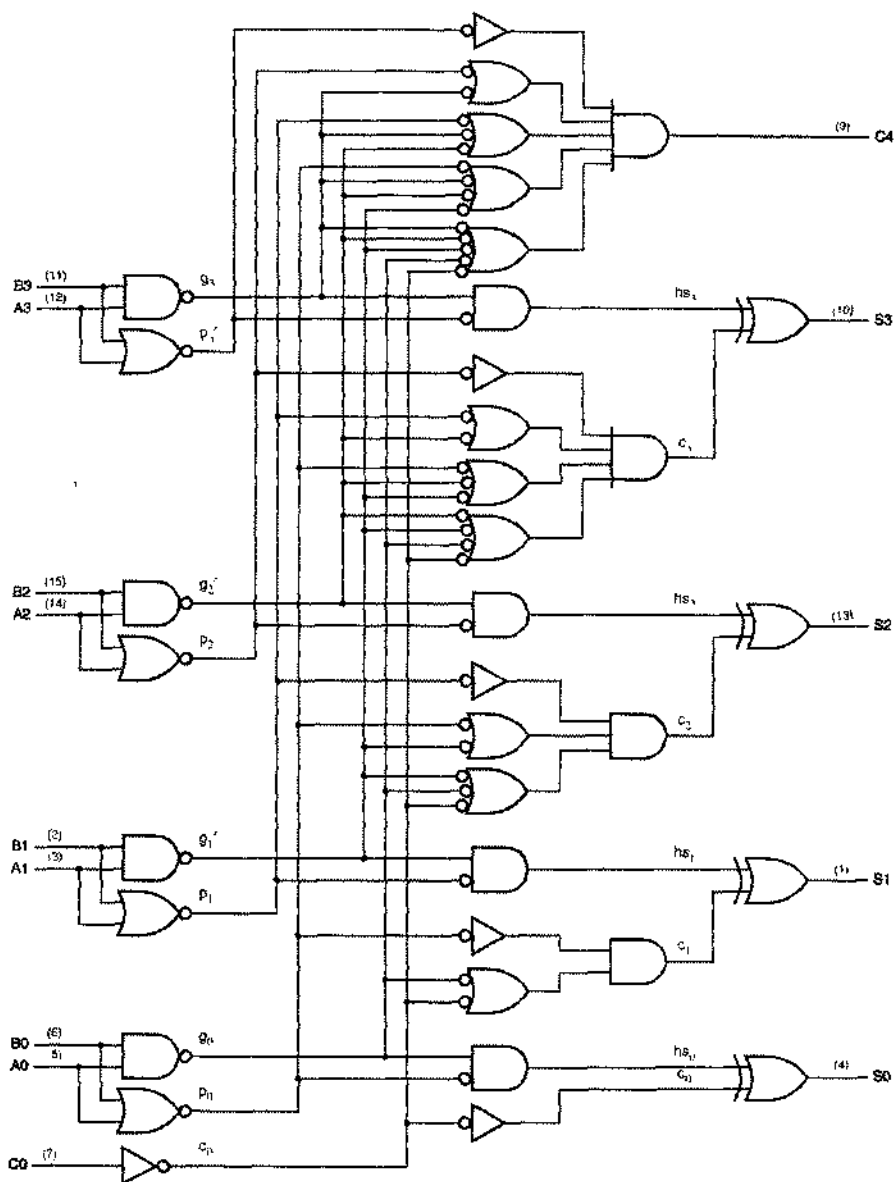


Рис. 5.91. Принципиальная схема 4-разрядного двоичного сумматора 74x283

Если вы уследили за выводом этих выражений и можете получить те же самые соотношения, глядя на принципиальную схему ИС '283, то поздравляю вас: вы уже в состоянии повышать быстродействие переключающих схем! Если нет, то вам возможно стоит повторить материал параграфов 4.1 и 4.2.

Задержка распространения от входа C0 до выхода C4 у микросхемы '283 примерно такая же, как у двух инверторов, то есть очень мала. В результате можно

достаточно просто собирать довольно быстрые сумматоры с групповым сквозным переносом (*group-ripple adder*) с числом разрядов больше четырех: для этого микросхемы 74х283 включаются последовательно путем соединения выходов переноса одних ИС с входами переноса других, как показано на рис. 5.92 для 16-разрядного сумматора. Полная задержка распространения от входа C0 до выхода C16 в этой схеме примерно такая же, как у восьми инверторов.

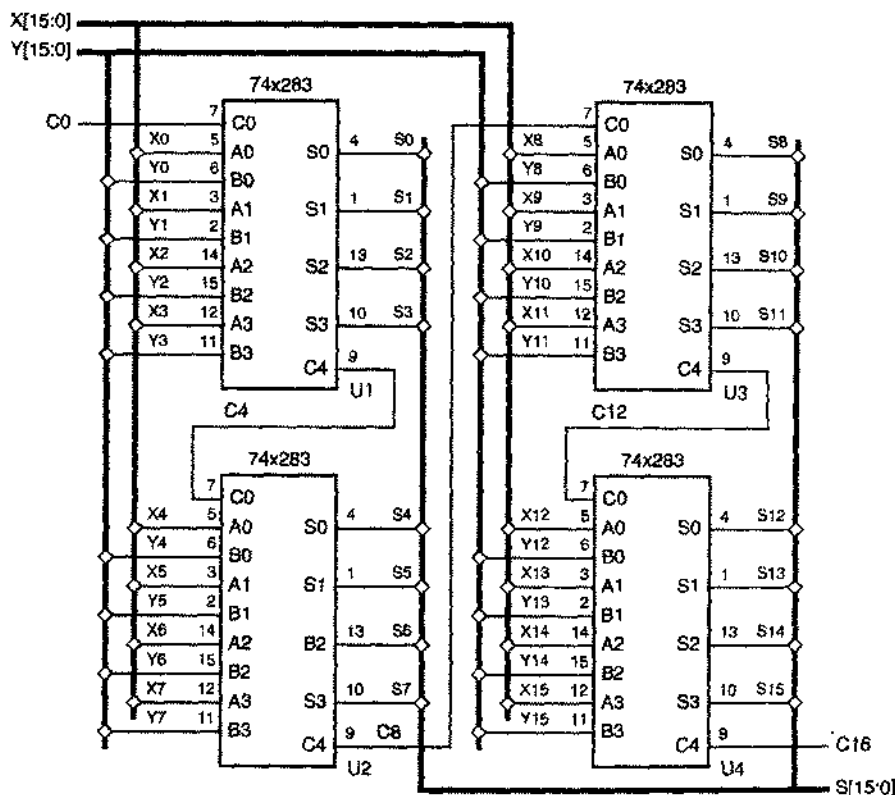


Рис. 5.92. 16-разрядный сумматор с групповым сквозным переносом

*5.10.5. Арифметическо-логические устройства, выполненные в виде ИС средней степени интеграции

Арифметическо-логическое устройство (АЛУ; *arithmetic and logic unit, ALU*) является комбинационной схемой, способной выполнять целый ряд различных арифметических и логических операций с парой b -разрядных операндов. Выполняемая операция определяется комбинацией сигналов на входах выбора функции. Типичные АЛУ, выполненные в виде ИС средней степени интеграции работают с 4-разрядными операндами и имеют от трех до пяти входов выбора функции, что позволяет им выполнять до 32 различных функций.

На рис. 5.93 представлено условное обозначение 4-разрядного АЛУ 74х181. Операция, выполняемая микросхемой 74х181, определяется сигналами на входах M и

S3–S0 согласно табл. 5.52. Заметьте, что идентификаторы A, B и F в таблице относятся к 4-разрядным словам A3–A0, B3–B0 и F3–F0, а символы $\cdot$ и $+$ к логическим операциям И и ИЛИ.

Сигналом на входе M в микросхеме '181 осуществляется выбор между арифметическими и логическими операциями. Если $M = 1$, то выполняются логические операции и значение сигнала на каждом выходе Fi является функцией только соответствующих входных данных Ai и Bi. Сигнал переноса между разрядами отсутствует, а сигнал на входе CIN игнорируется. Сигналами на входах S3–S0 выбирается конкретная логическая операция; можно выбрать любую из 16 различных комбинационных логических функций двух переменных.

Табл. 5.52. Функции, выполняемые 4-разрядным АЛУ 74х181

Входы				Функция	
S3	S2	S1	S0	$M = 0$ (арифметическая)	$M = 1$ (логическая)
0	0	0	0	$F = A \text{ minus } 1 \text{ plus } \text{Cin}$	$F = A'$
0	0	0	1	$F = A - B \text{ minus } 1 \text{ plus } \text{Cin}$	$F = A' + B'$
0	0	1	0	$F = A \cdot B' \text{ minus } 1 \text{ plus } \text{Cin}$	$F = A' + B$
0	0	1	1	$F = 1111 \text{ plus } \text{Cin}$	$F = 1111$
0	1	0	0	$F = A \text{ plus } (A + B') \text{ plus } \text{Cin}$	$F = A' - B'$
0	1	0	1	$F = A - B \text{ plus } (A + B') \text{ plus } \text{Cin}$	$F = B'$
0	1	1	0	$F = A \text{ minus } B \text{ minus } 1 \text{ plus } \text{Cin}$	$F = A \oplus B'$
0	1	1	1	$F = A + B' \text{ plus } \text{Cin}$	$F = A + B'$
1	0	0	0	$F = A \text{ plus } (A + B) \text{ plus } \text{Cin}$	$F = A' \cdot B$
1	0	0	1	$F = A \text{ plus } B \text{ plus } \text{Cin}$	$F = A \oplus B$
1	0	1	0	$F = A - B' \text{ plus } (A + B) \text{ plus } \text{Cin}$	$F = B$
1	0	1	1	$F = A + B \text{ plus } \text{Cin}$	$F = A + B$
1	1	0	0	$F = A \text{ plus } A \text{ plus } \text{Cin}$	$F = 0000$
1	1	0	1	$F = A - B \text{ plus } A \text{ plus } \text{Cin}$	$F = A - B'$
1	1	1	0	$F = A - B' \text{ plus } A \text{ plus } \text{Cin}$	$F = A - B$
1	1	1	1	$F = A \text{ plus } \text{Cin}$	$F = A$

Когда $M = 0$, выполняются арифметические операции, от разряда к разряду распространяется перенос, а CIN используется как вход переноса младшего разряда. Для операций с числом разрядов больше четырех, можно включить последовательно несколько АЛУ '181, подобно сумматору с групповым сквозным переносом на рис. 5.92. Выход переноса (COUT) каждого АЛУ в этом случае соединяется с входом переноса (Cin) следующего, более старшего каскада. На входы выбора функции (M, S3–S0) всех ИС '181, входящих в составное АЛУ, подаются одни и те же сигналы.

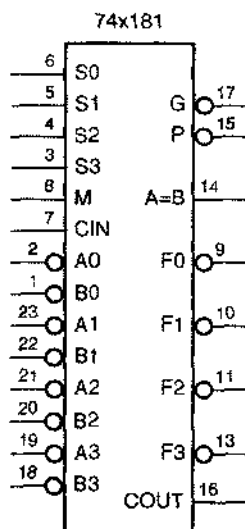


Рис. 5.93. Условное обозначение 4-разрядного АЛУ 74x181

Чтобы выполнить сложение в дополнительном коде, сигналами на входах $S3-S0$ выбирается операция "A plus B plus CIN". На время выполнения операций сложения на входе CIN младшего АЛУ обычно устанавливается 0. Для вычитания в дополнительном коде сигналами на входах $S3-S0$ выбирается операция "A minus B minus 1 plus CIN". В данном случае на входе CIN младшего АЛУ обычно устанавливается 1, так как при вычитании сигнал на этом входе воспринимается как инвертированное значение заема.

ИС '181 выполняет и другие арифметические действия; например, в некоторых приложениях (например, при уменьшении значения на 1) полезна операция "A minus 1 plus CIN". Данное АЛУ реализует также группу экзотических арифметических операций, типа " $A \cdot B$ " plus $(A + B)$ plus CIN", которые почти никогда не используются на практике, но предоставляются схемой бесилатио.

Обратите внимание, что входы операндов $A3_L-A0_L$, $B3_L-B0_L$ и выходы функций $F3_L-F0_L$ микросхемы '181 имеют низкий активный уровень. ИС '181 можно также применять при высоких активных уровнях сигналов на входах операндов и на выходах функций. Однако в этом случае должна быть составлена другая таблица функционирования. Если $M = 1$, то по-прежнему выполняются логические функции, но при заданной комбинации сигналов на входах $S3-S0$ реализуемая функция строго обратна приведенной в табл. 5.52. Когда $M = 0$, выполняются арифметические операции, но таблица реализуемых функций вновь другая. За большими подробностями следует обратиться к справочным данным микросхемы '181.

У двух других АЛУ, выпускаемых в виде ИС средней степени интеграции, 74x381 и 74x382 (рис. 5.94) код выбора операции более компактен: они выполняют только восемь разных, но полезных функций, перечисленных в табл. 5.53. Единственное различие между микросхемами '381 и '382 состоит в том, что у первой ИС имеются выходы ускоренного группового переноса (который мы рассмотрим ниже), в то время как другая ИС обладает выходами сквозного переноса и переноса.

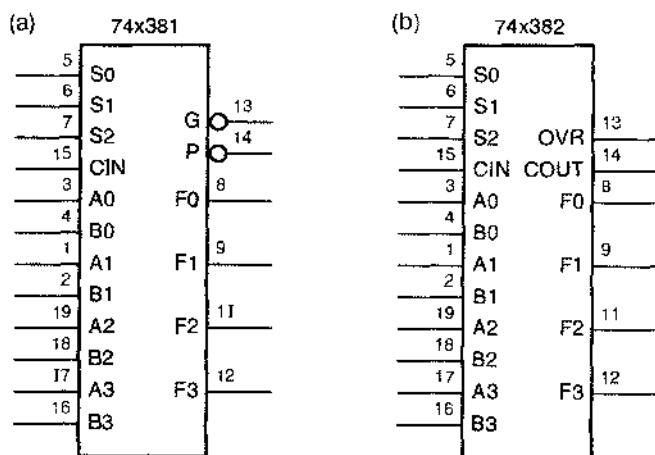


Рис. 5.94. Условные обозначения 4-разрядных АЛУ (а) 74x381, (б) 74x382

Табл. 5.53. Функции, выполняемые 4-разрядными АЛУ 74x381 и 74x382

Входы			Функция
S2	S1	S0	
0	0	0	$F = 0000$
0	0	1	$F = B \text{ minus } A \text{ minus } 1 \text{ plus CIN}$
0	1	0	$F = A \text{ minus } B \text{ minus } 1 \text{ plus CIN}$
0	1	1	$F = A \text{ plus } B \text{ plus CIN}$
1	0	0	$F = A \oplus B$
1	0	1	$F = A + B$
1	1	0	$F = A - B$
1	1	1	$F = 1111$

*5.10.7. Ускоренный групповой перенос

Микросхемы '181 и '381 снабжены выходами *ускоренного группового переноса* (*group-carry lookahead*), которые позволяют соединять последовательно несколько таких АЛУ без сквозного переноса между 4-разрядными группами. Внутри этих АЛУ, подобно микросхеме 74x283, применяется ускоренный перенос. Однако у них, кроме того, имеются выходы G_L и P_L , на которых появляются сигналы ускоренного переноса всей 4-разрядной группы. Сигнал на выходе G_L принимает активный уровень, если АЛУ генерирует перенос, то есть в том случае, когда сигнал на выходе переноса ($COUT = 1$) вырабатывается независимо от наличия или отсутствия сигнала на входе переноса ($CIN = 1$):

$$G_L = (g_3 + p_3 \cdot g_2 + p_3 \cdot p_2 \cdot g_1 + p_3 \cdot p_2 \cdot p_1 \cdot g_0)'.$$

На выходе P_L сигнал принимает активный уровень, если АЛУ передает перенос, то есть в случае, когда сигнал на выходе переноса вырабатывается только при наличии сигнала на входе переноса:

$$P_L = (p_3 \cdot p_2 \cdot p_1 \cdot p_0)'.$$

Когда АЛУ включаются последовательно одно за другим, для формирования сигнала переноса на входе каждого АЛУ достаточно всего лишь двухуровневой логики, чтобы надлежащим образом объединить сигналы, возникающие на выходах ускоренного группового переноса. Эту операцию выполняет *схема ускоренного переноса (lookahead carry circuit) 74x182*, показанная на рис. 5.95. У этой схемы имеются следующие входы: C_0 – вход переноса младшего АЛУ ("ALU 0"), G_0 – G_3 и P_0 – P_3 – входы для подключения выходов генерации и передачи переноса АЛУ с 0-го по 3-е. По сигналам на этих входах микросхема '182 вырабатывает на выходах C_1 – C_3 сигналы для подачи на входы переноса АЛУ с 1-го по 3-е. На рис. 5.96 показаны связи в 16-разрядном АЛУ, построенном на четырех микросхемах '381 и микросхеме '182.

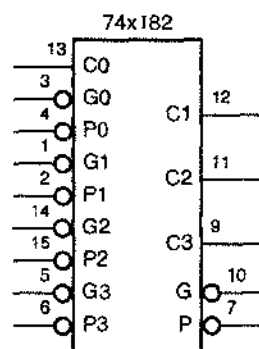


Рис. 5.95. Условное обозначение схемы ускоренного переноса 74x182

Соотношения, которыми определяются выходные сигналы микросхемы '182, получаются путем «разнесения слагаемого по сомножителям» в основном равенстве для ускоренного переноса из раздела 5.10.4:

$$\begin{aligned} c_{i+1} &= g_i + p_i \cdot c_i \\ &= (g_i + p_i) \cdot (g_i + c_i). \end{aligned}$$

В развернутом виде для трех первых значений i получаем следующие выражения для сигналов переноса:

$$\begin{aligned} C_1 &= (G_0 + P_0) \cdot (G_0 + C_0) \\ C_2 &= (G_1 + P_1) \cdot (G_1 + G_0 + P_0) \cdot (G_1 + G_0 + C_0) \\ C_3 &= (G_2 + P_2) \cdot (G_2 + G_1 + P_1) \cdot (G_2 + G_1 + G_0 + P_0) \cdot (G_2 + G_1 + G_0 + C_0). \end{aligned}$$

В микросхеме '182 реализация каждого из этих выражений сопровождается задержкой, вносимой только одноуровневой схемой НЕ-ИЛИ-И.

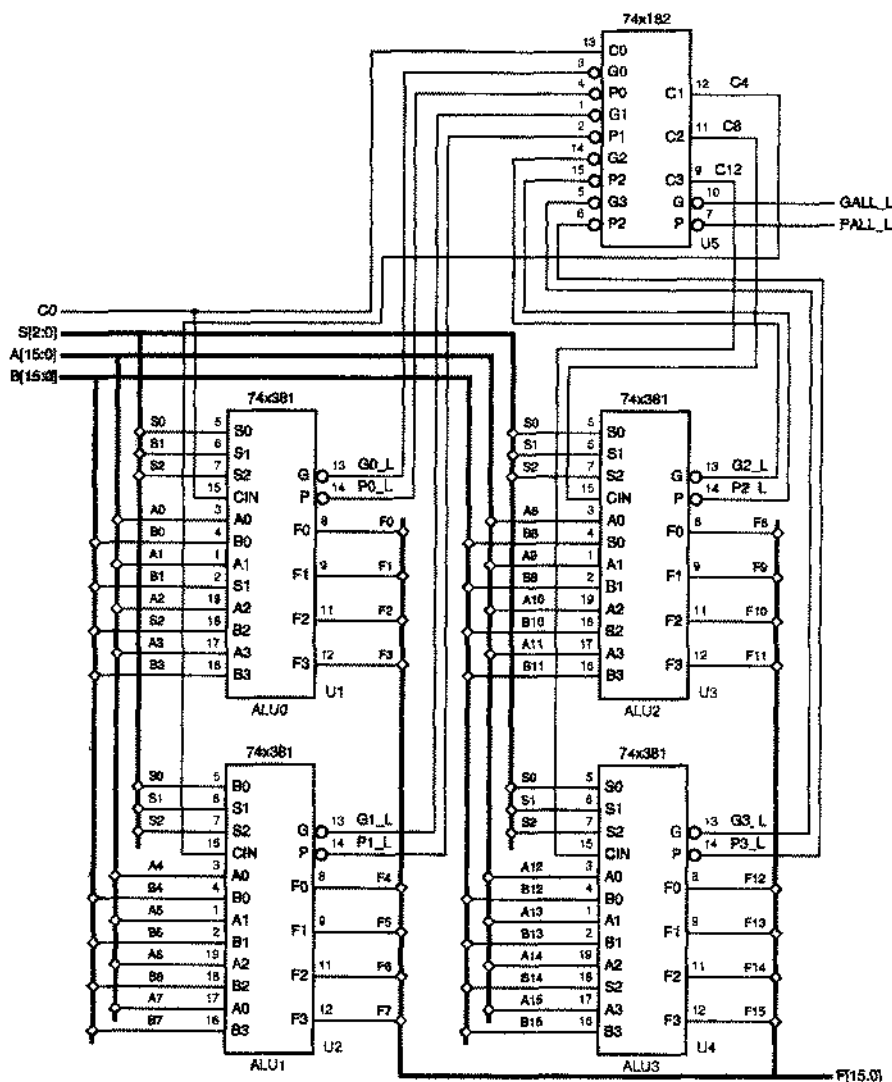


Рис. 5.95. 16-разрядное АЛУ с ускоренным групповым переносом

Когда последовательно включается больше четырех АЛУ, их можно разбить на «группы групп» (“supergroups”), каждая со своей собственной микросхемой '182. Например, 64-разрядный сумматор имел бы четыре группы групп, состоящие каждая из четырех АЛУ и одной микросхемы '182. Сигналы с выходов G_L и P_L каждой из микросхем '182 можно подать на входы микросхемы '182 следующего уровня, так как они показывают, генерирует или передает переносы данная группа групп:

$$G_L = ((G3 + P3) \cdot (G3 + G2 + P2) \cdot (G3 + G2 + G1 + P1) \cdot (G3 + G2 + G1 + G0))'$$

$$P_L = (P0 \cdot P1 \cdot P2 \cdot P3)'$$

*5.10.8. Описание сумматоров на языке ABEL и их реализация в ПЛУ

Язык ABEL поддерживает операторы сложения (+) и вычитания (−), которые могут применяться к наборам. Наборы интерпретируются как целые числа без знака; например, набор, состоящий из n битов, представляет собой целое число из интервала от 0 до $2^n - 1$. Вычитание выполняется путем взятия вычитаемого с обратным знаком и сложения. Для изменения знака числа берется его точное дополнение; то есть операнд поразрядно инвертируется, а затем добавляется 1.

В табл. 5.54 приведен пример сложения на языке ABEL. В определении набора SUM предусмотрено на единицу большее число разрядов, чем у слагаемых, для переноса из старшего разряда; в противном случае этот перенос нельзя будет учесть. При определении наборов для слагаемых, они расширены добавлением 0-го бита, чтобы соответствовать размеру набора SUM.

```

module add
title 'Adder Exercise'

" Input and output pins
A7..A0, B7..B0      pin;
SUM8..SUM0          pin istype 'com';

" Set definitions
A = [0, A7..A0];
B = [0, B7..B0];
SUM = [SUM8..SUM0];

equations
SUM = A + B;

end add

```

Табл. 5.54. Программа на языке ABEL для 8-разрядного сумматора

Хотя программа, описывающая сумматор, очень мала, ее трансляция требует большого времени и при этом возникает огромное число термов в минимальных двухуровневых суммах произведений. В то время как SUM0 содержит только два термина-произведения, выражения последующих SUM i состоят из $5 \cdot 2^i - 4$ термов; в частности, SUM7 состоит из 636 термов, а выражение для переноса из старшего разряда (SUM8) состоит из $2^8 - 1 = 255$ термов-произведений! Очевидно, что сумматоры с большим числом разрядов практически нельзя реализовать на основе двухуровневой логики.

О ПЕРЕНОСЕ

В нашем примере сумматора выражение для сигнала на выходе переноса (SUM8) имеет то же самое число термов-произведений (255), что и выражения для выходных сигналов «меньше» или «больше» у 8-разрядного компаратора. Это и не удивительно, коль скоро вы понимаете, что возникновение переноса при сложении $A + B$ функционально эквивалентно выполнению неравенства $A > \bar{B}$.

В связи с тем, что время от времени сумматоры и компараторы с большим числом разрядов по-прежнему необходимы в ПЛУ, в языке ABEL предусмотрена директива `@CARRY` (*@CARRY directive*), которая указывает компилятору на необходимость синтезировать сумматор с групповым сквозным переносом с n разрядами в группе. Например, если в программу, приведенную в табл. 5.54, включить оператор `"@CARRY 1;"`, то компилятор создаст восемь новых выходных сигналов переноса в двоичных разрядах с 0-го по 7-й. Эти внутренние переносы будут использованы в выражениях для SUM1–SUM8, то есть по существу будет создан 8-разрядный сумматор со сквозным переносом, имеющий задержку, в худшем случае равную времени восьми проходов сигнала через ПЛУ.

Если применен оператор `"@CARRY 2;"`, то компаратор сформирует два бита переноса одновременно, создавая четыре новых выходных сигнала переноса в разрядах 1, 3, 5 и 7. В этом случае максимальное число термов-произведений, необходимых для получения сигнала на любом из выходов, все еще остается приемлемым, всего лишь 7, и задержка в наихудшем случае равна времени только четырех проходов сигнала через ПЛУ. При трех битах в группе (`@CARRY 3;`) максимальное число термов-произведений увеличивается до 28, что становится неосуществимым.

Особенно часто встречается в языке ABEL и реализуется в ПЛУ прибавление или вычитание константы, равной 1. Эта операция используется в описании счетчиков, где говорится, что очередное состояние счетчика равно текущему состоянию плюс 1 для «суммирующего» счетчика и минус 1 для «вычитающего» счетчика. Выражение для i -го разряда счетчика можно очень просто сформулировать словами: «Инвертировать i -й бит, если счет разрешен и биты всех разрядов с номерами меньше i равны 1». Реализация этой операции требует только $i + 2$ термов-произведений для любого значения i , а в некоторых ПЛУ и ИС типа CPLD возможно еще большее сокращение их числа, вплоть до одного термина-произведения и одного вентиля ИСКЛЮЧАЮЩЕЕ ИЛИ.

*5.10.9. Описание сумматоров на языке VHDL

Хотя язык VHDL имеет встроенные операторы сложения (+) и вычитания (–), они работают только с целыми и действительными числами и физическими типами. В частности, они не работают с типом `BIT_VECTOR` и с типом `STD_LOGIC_VECTOR` стандарта IEEE. Для этих типов в стандартных пакетах определены специальные операторы.

Как было объяснено в разделе 5.9.6, в пакете IEEE_std_logic_arith определены два новых типа массивов — SIGNED и UNSIGNED — и набор функций сравнения для операндов типа INTEGER, SIGNED и UNSIGNED. В данном пакете определены операции сложения и вычитания для операндов тех же типов, а также для 1-разрядных операндов типа STD_LOGIC и STD_ULOGIC.

При большом числе перекрывающихся функций сложения и вычитания не столь очевидно, каким окажется тип результата сложения или вычитания. Если хотя бы один из операндов принадлежит типу SIGNED, то обычно результат будет типа SIGNED, в противном случае результат будет типа UNSIGNED. Но если результирующее значение присваивается сигналу или переменной типа STD_LOGIC_VECTOR, то результат типа SIGNED или UNSIGNED преобразуется к этому типу. Разрядность любого результата обычно равна разрядности самого длинного операнда. Однако, когда операнд типа UNSIGNED участвует в одной операции с операндом типа SIGNED или INTEGER, его разрядность увеличивается на 1 для размещения в нем знакового бита, равного 0, и только после этого устанавливается разрядность результата.

В табл. 5.55 приведена VHDL-программа сложения 8-разрядных операндов различного типа, иллюстрирующая эти правила. Первый результат S объявлен как 9-разрядное двоичное число в предположении, что разработчика интересует перенос, который может возникнуть при сложении 8-разрядных операндов A и B типа UNSIGNED. С помощью оператора конкатенации & операнды A и B расширяются так, чтобы функция сложения помещала бит переноса в старший разряд результата.

Табл. 5.55. VHDL-программа сложения и вычитания 8-разрядных целых чисел различных типов

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity vadd is
  port (
    A, B: in UNSIGNED (7 downto 0);
    C: in SIGNED (7 downto 0);
    D: in STD_LOGIC_VECTOR (7 downto 0);
    S: out UNSIGNED (8 downto 0);
    T: out SIGNED (8 downto 0);
    U: out SIGNED (7 downto 0);
    V: out STD_LOGIC_VECTOR (8 downto 0)
  );
end vadd;

architecture vadd_arch of vadd is
begin
  S <= ('0' & A) + ('0' & B);
  T <= A + C;
  U <= C + SIGNED(D);
  V <= C - UNSIGNED(D);
end vadd_arch;
```

Следующий результат T также является 9-разрядным, так как функция сложения расширяет операнд A типа `UNSIGNED` при его сложении с операндом C типа `SIGNED`. В третьей операции сложения 8-разрядный операнд D типа `STD_LOGIC_VECTOR` преобразуется в операнд типа `SIGNED` и складывается с операндом C , так что в результате получается 8-разрядное двоичное число U типа `SIGNED`. В последнем операторе величина D преобразуется в операнд типа `UNSIGNED`, автоматически расширяется на один разряд и вычитается из C , так что результат V оказывается 9-разрядным.

Так как сложение и вычитание являются довольно дорогими операциями в смысле числа требуемых вентилях, многие VHDL-средства синтеза будут пытаться многократно использовать блок сумматора всякий раз, когда это возможно. В табл. 5.56, например, приведена VHDL-программа, включающая два различных сложения. Вместо того, чтобы образовать два сумматора и с помощью мультиплексора выбрать выход одного из них, синтезирующая программа может создать только один сумматор и с помощью мультиплексоров переключать его входы, в результате чего схема в целом будет иметь меньшие размеры.

Табл. 5.56. VHDL-программа с многократным использованием сумматора

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity vaddshr is
  port (
    A, B, C, D: in SIGNED (7 downto 0);
    SEL: in STD_LOGIC;
    S: out SIGNED (7 downto 0)
  );
end vaddshr;

architecture vaddshr_arch of vaddshr is
begin
  S <= A + B when SEL = '1' else C + D;
end vaddshr_arch;
```

*5.11. Комбинационные умножители

*5.11.1. Структура комбинационных умножителей

В параграфе 2.8 в общих чертах был описан алгоритм перемножения n -разрядных двоичных чисел посредством n сдвигов и сложений. Хотя этот алгоритм реализует способ, которым мы умножаем десятичные числа вручную, в нем нет ничего принципиально «последовательного» или «зависящего от времени». Другими словами, при наличии n -разрядных входных слов X и Y можно составить таблицу истинности, которая представляет $2n$ -разрядное произведение $P = X \cdot Y$ в виде комбинационной функции X и Y . Комбинационный умножитель

(combinational multiplier) является логической схемой с такой таблицей истинности.

В большинстве случаев реализация комбинационного умножения основана на алгоритме сдвига и сложения, применяемого при умножении вручную. Рис. 5.97 служит иллюстрацией основной идеи перемножения двух 8-разрядных целых чисел без знака: множимого $X = x_7x_6x_5x_4x_3x_2x_1x_0$ и множителя $Y = y_7y_6y_5y_4y_3y_2y_1y_0$; такую схему называют умножителем 8×8 . Каждая строка, называемая компонентом произведения (product component), является сдвинутым множимым, умноженным на 0 или на 1 в зависимости от значения соответствующего разряда множителя. Каждый небольшой прямоугольник представляет собой один бит компонента произведения y_jx_i , получаемый в результате выполнения логической операции И над битом множителя y_j и битом множимого x_i . Произведение $P = p_{15}p_{14} \dots p_2p_1p_0$ получается путем сложения всех компонентов произведения и содержит 16 битов.

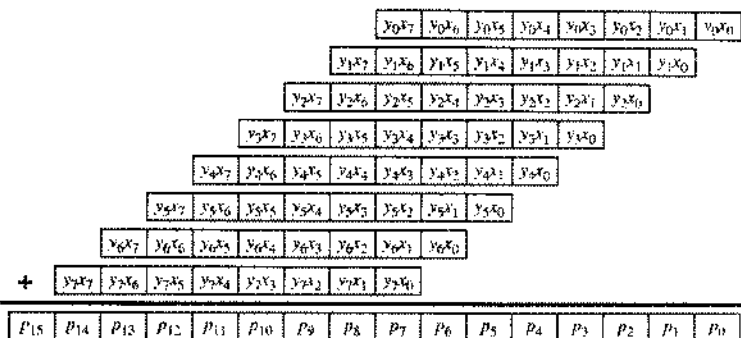


Рис. 5.97. Компоненты произведения в умножителе 8×8

На рис. 5.98 продемонстрирован один из способов сложения компонентов произведения. Здесь биты компонента произведения размещены в разрядку, а каждый прямоугольник со знаком “+” является полным сумматором, эквивалентным сумматору, приведенному на рис. 5.86(с). В каждой строке выходы сигналов переноса полных сумматоров соединены так, чтобы получался 8-разрядный сумматор со сквозным переисосом. Таким образом, первый сумматор со сквозным переисосом суммирует первые два компонента произведения, образуя первое частичное произведение согласно определению, данному в параграфе 2.8. Последующие сумматоры складывают предыдущее частичное произведение со следующим компонентом произведения.

Интересно рассмотреть задержку распространения в схеме, приведенной на рис. 5.98. Худшим является такой случай, когда биты на входах младшего сумматора (y_0x_1 и y_1x_0) влияют на значение старшего разряда произведения (p_{15}). Если ради простоты предположить, что задержки сигналов в полном сумматоре от любого входа до любого выхода одинаковы и равны, скажем, t_{pd} , то в худшем случае сигнал проходит через 20 сумматоров и задержка составляет $20t_{pd}$. Соответствующий результат можно получить и в том случае, когда задержки различны; см. задачу 5.83.

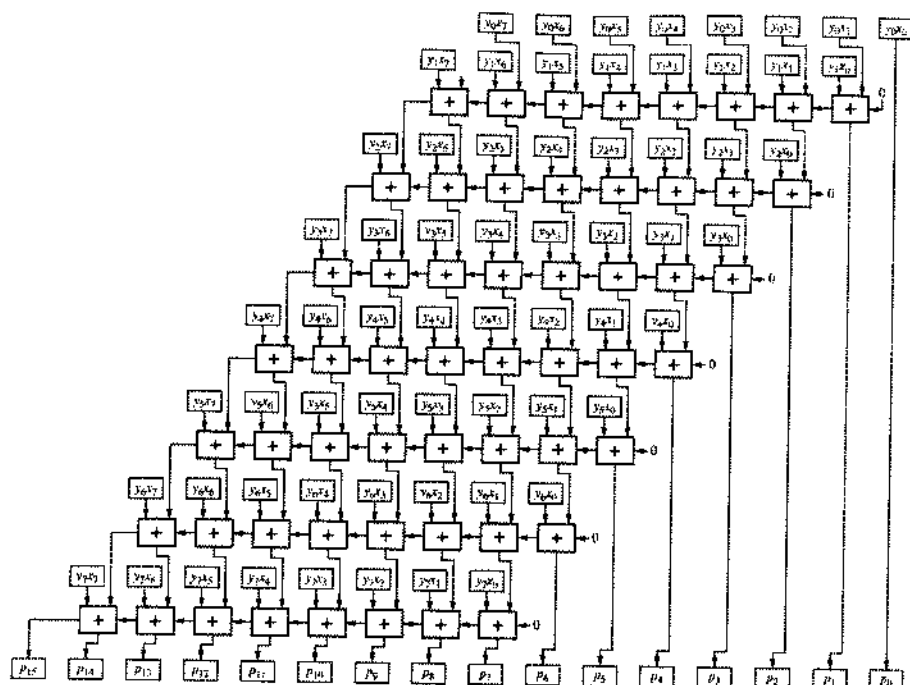


Рис. 5.98. Промежуточные соединения в комбинационном умножителе 8×8

В *последовательных умножителях (sequential multipliers)* используется единственный сумматор и регистр накопления частичных произведений. Сначала в регистр частичного произведения заносится первый компонент произведения, а затем – при перемножении двух n -разрядных чисел – выполняется $n - 1$ шагов: на каждом шаге очередной компонент произведения с помощью сумматора добавляется в регистр частичного произведения.

Для ускорения процедуры умножения в некоторых последовательных умножителях используется *сложение с сохранением переноса (carry-save addition)*. Идея состоит в разрыве цепочки переносов в сумматоре со сквозным переносом, чтобы уменьшить время, затрачиваемое на каждое сложение. Достигается это путем соединения выхода переноса i -го разряда на j -м шаге со входом переноса $(i + 1)$ -го разряда на *следующем*, $(j + 1)$ -м шаге. После того как добавлен последний компонент произведения, необходим еще один шаг, на котором переносы подключаются обычным образом и сумматору предоставляется возможность функционировать по принципу сквозного переноса от младшего разряда к старшему.

На рис. 5.99 показан комбинационный эквивалент умножителя 8×8 , в котором реализовано сложение с сохранением переноса. Обратите внимание, что выход переноса каждого полного сумматора в первых семи рядах соединен с одним из входов сумматора, расположенного *ниже*. Выходы переносов полных сумматоров в восьмом ряду соединены со входами переносов следующих сумматоров так,

что образуется обычный сумматор со сквозным переносом. Хотя этот комбинационный умножитель состоит из точно такого же числа логических схем, что и предыдущий (64 двухвходовых вентиля И и 56 полных сумматоров), задержка распространения в нем существенно меньше. В худшем случае задержка определяется временем прохождения сигнала только через 14 полных сумматоров. При использовании в последнем ряду сумматора с укороченным переносом задержка может быть еще меньше.

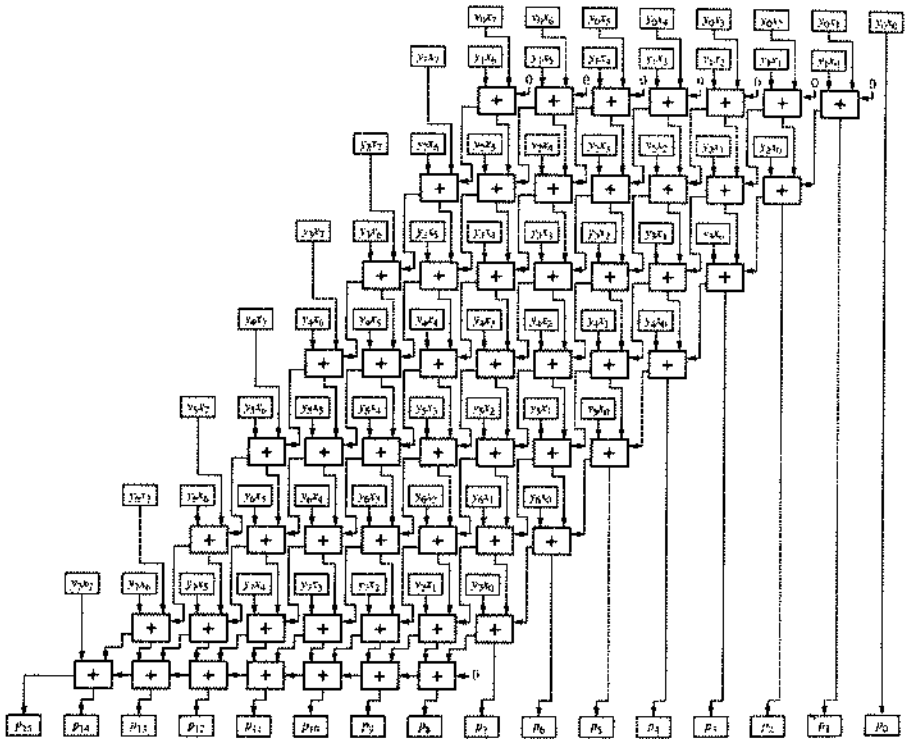


Рис. 5.99. Промежуточные соединения в комбинационном умножителе 8×8 с повышенным быстродействием

Регулярность структуры комбинационных умножителей делает их идеальными для реализации внутри СБИС и специализированных ИС. Важность быстрого умножения в микропроцессорах, цифровой видеотехнике и многих других приложениях привела к многочисленным исследованиям в этой области и разработке еще лучших структур и схем комбинационных умножителей (см. Обзор литературы).

*5.11.2. Описание процедуры умножения на языке ABEL и ее реализация в ПЛУ

В языке ABEL существует оператор умножения $*$, но его можно применять только к отдельным сигналам, числам или специальным константам, но не к наборам.

Таким образом, пользуясь языком ABEL, нельзя синтезировать схему умножителя единственным соотношением типа $P=X*Y$.

Однако языком ABEL можно воспользоваться для описания комбинационного умножителя, если разбить его на меньшие части. В табл. 5.57, например, приведена программа перемножения 4-разрядных чисел без знака в соответствии с той же общей структурой умножителя, какая была представлена на рис. 5.97. Сначала находятся четыре компонента произведения PC1, PC2, PC3 и PC4, которые затем складываются в разделе программы equations. Эта процедура не создает массивов полных сумматоров, как на рис. 5.98 или 5.99. Вместо этого компилятор языка ABEL послушно исполнит инструкцию сложения и составит минимальную сумму для каждого из восьми выходных битов произведения. Удивительно, но для формирования сигнала на выходе P4 в худшем случае требуется только 36 термов-произведений, что многовато, но безусловно реализуемо за два прохода через ПЛУ.

Табл. 5.57. Программа на языке ABEL для комбинационного умножителя 4x4

```

module mul4x4
title '4x4 Combinational Multiplier'

X3..X0, Y3..Y0 pin; " multiplicand, multiplier
P7..P0          pin istype 'com'; " product

P = [P7..P0];
PC1 = Y0 & [0, 0, 0, 0, X3, X2, X1, X0];
PC2 = Y1 & [0, 0, 0, X3, X2, X1, X0, 0];
PC3 = Y2 & [0, 0, X3, X2, X1, X0, 0, 0];
PC4 = Y3 & [0, X3, X2, X1, X0, 0, 0, 0];

equations
P = PC1 + PC2 + PC3 + PC4;

end mul4x4

```

*5.11.3. Описание процедуры умножения на языке VHDL

Язык VHDL достаточно богат, чтобы выражать умножение разными способами; лучший из них мы оставим напоследок.

В табл. 5.58 представлена приведенная программа на языке VHDL, которая воспроизводит структуру умножителя, приведенную на рис. 5.99. Для указанных на рисунке внутренних сигналов в программе определяется новый тип данных array8x8, который является двумерным массивом элементов типа STD_LOGIC (напомним, что STD_LOGIC_VECTOR — это одномерный массив элементов типа STD_LOGIC). Переменная PC объявлена как массив типа array8x8, предназначенный для хранения битов компонентов произведения, а переменные PCS и PCS представляют собой аналогичные массивы для хранения сумм и переносов основного массива полных сумматоров.

Табл. 5.58. Поведенческая VHDL-программа для комбинационного умножителя 8x8

```

library IEEE;
use IEEE.std_logic_1164.all;

entity vmul8x8p is
    port ( X: in STD_LOGIC_VECTOR (7 downto 0);
          Y: in STD_LOGIC_VECTOR (7 downto 0);
          P: out STD_LOGIC_VECTOR (15 downto 0) );
end vmul8x8p;

architecture vmul8x8p_arch of vmul8x8p is
    function MAJ (I1, I2, I3: STD_LOGIC) return STD_LOGIC is
        begin
            return ((I1 and I2) or (I1 and I3) or (I2 and I3));
        end MAJ;
    begin
        process (X, Y)
            type array8x8 is array (0 to 7) of STD_LOGIC_VECTOR (7 downto 0);
            variable PO: array8x8;      -- product component bits
            variable PCS: array8x8;     -- full-adder sum bits
            variable PCC: array8x8;     -- full-adder carry output bits
            variable RAS, RAC: STD_LOGIC_VECTOR (7 downto 0); -- ripple adder sum
                                                    -- and carry bits
            for i in 0 to 7 loop for j in 0 to 7 loop
                PC(i)(j) := Y(i) and X(j); -- compute product component bits
            end loop; end loop;
            for j in 0 to 7 loop
                PCS(0)(j) := PC(0)(j); -- initialize first-row "virtual"
                PCC(0)(j) := '0';      -- adders (not shown in figure)
            end loop;
            for i in 1 to 7 loop -- do all full adders except last row
                for j in 0 to 6 loop
                    PCS(i)(j) := PC(i)(j) xor PCS(i-1)(j+1) xor PCC(i-1)(j);
                    PCC(i)(j) := MAJ(PC(i)(j), PCS(i-1)(j+1), PCC(i-1)(j));
                    PCS(i)(7) := PC(i)(7); -- leftmost "virtual" adder sum output
                end loop;
            end loop;
            RAC(0) := '0';
            for i in 0 to 6 loop -- final ripple adder
                RAS(i) := PCS(7)(i+1) xor PCC(7)(i) xor RAC(i);
                RAC(i+1) := MAJ(PCS(7)(i+1), PCC(7)(i), RAC(i));
            end loop;
            for i in 0 to 7 loop
                P(i) <= PCS(1)(0); -- first 8 product bits from full-adder sums
            end loop;
            for i in 8 to 14 loop
                P(i) <= RAS(i-8); -- next 7 bits from ripple-adder sums
            end loop;
            P(15) <= RAC(7);      -- last bit from ripple-adder carry
        end process;
    end vmul8x8p_arch;

```

В одномерных массивах RAS и RAC хранятся суммы и переносы сумматора со сквозным переносом. На рис. 5.100 приведены обозначения и нумерация переменных. Целые переменные i и j используются в качестве индексов циклов по строкам и столбцам соответственно.

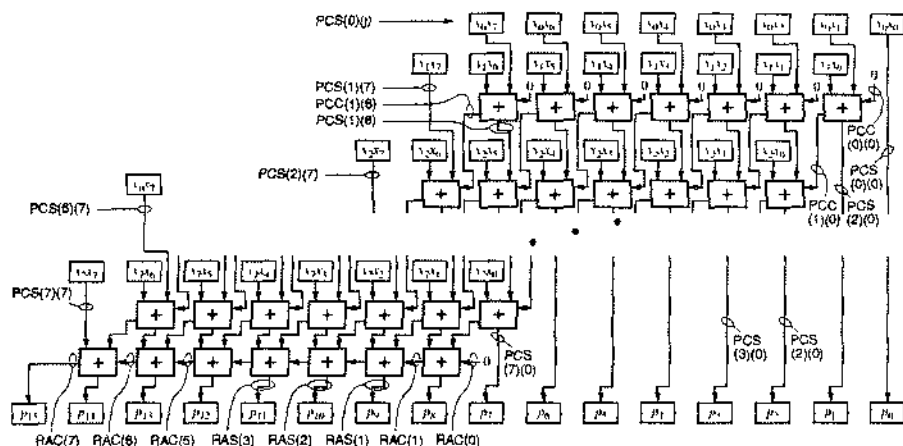


Рис. 5.100. Имена переменных в VHDL-программе для умножителя 8×8

В программе предпринята попытка показать логические вентили, которые использовались бы в реализации, точно соответствующей рис. 5.99, хотя синтезатор имеет право на основе этой новежденческой программы ездить совершенно другую структуру. Если вы хотите задать определенную структуру, то необходимо написать структурную VHDL-программу, о чем речь пойдет ниже.

В первом вложенном операторе `for` в программе выполняются 64 операции И, в результате которых получают биты компонентов произведения. Следующий оператор цикла `for` инициализирует граничные условия сверху умножителя, используя понятия 0-й строки «виртуальных» полных сумматоров, не показанных на рисунке, у которых выходы сумм равны первой строке битов в массиве PC, а выходы переносов равны 0. Третий вложенный цикл `for` соответствует основному массиву сумматоров, изображенных на рис. 5.99 во всех строках, кроме последней, которая реализуется четвертым циклом `for`. В последних двух циклах `for` сигналам на выходах умножителя присваиваются значения, возникающие на выходах сумматора со сквозным переносом.

Применяя структурный подход можно написать другую VHDL-программу, как показано в табл. 5.59. Этот подход позволяет разработчику полностью задавать структуру синтезируемой схемы, например, в том случае, когда желательнее реализовать ее в специализированной ИС. В программе предполагается, что архитектуры AND2, XOR3 и MAJ3 определены в другом месте, например, в библиотеке данной специализированной ИС.

Табл. 5.59. Структурная VHDL-архитектура для 8x8 комбинационного умножителя

```

architecture vmul8x8s_arch of vmul8x8s is
  component AND2
    port( I0, I1: in STD_LOGIC;
          O: out STD_LOGIC );
  end component;
  component XOR3
    port( I0, I1, I2: in STD_LOGIC;
          O: out STD_LOGIC );
  end component;
  component MAJ -- Majority function,  $O = I0 \cdot I1 + I0 \cdot I2 + I1 \cdot I2$ 
    port( I0, I1, I2: in STD_LOGIC;
          O: out STD_LOGIC );
  end component;

  type array8x8 is array (0 to 7) of STD_LOGIC_VECTOR (7 downto 0);
  signal PC: array8x8; -- product-component bits
  signal PCS: array8x8; -- full-adder sum bits
  signal PCC: array8x8; -- full-adder carry output bits
  signal RAS, RAC: STD_LOGIC_VECTOR (7 downto 0); -- sum, carry
begin
  g1: for i in 0 to 7 generate -- product-component bits
    g2: for j in 0 to 7 generate
      U1: AND2 port map (Y(i), X(j), PC(i)(j));
    end generate;
  end generate;
  g3: for j in 0 to 7 generate
    PCS(0)(j) <= PC(0)(j); -- initialize first-row "virtual" adders
    PCC(0)(j) <= '0';
  end generate;
  g4: for i in 1 to 7 generate -- do full adders except the last row
    g5: for j in 0 to 6 generate
      U2: XOR3 port map (PC(i)(j), PCS(i-1)(j+1), PCC(i-1)(j), PCS(i)(j));
      U3: MAJ port map (PC(i)(j), PCS(i-1)(j+1), PCC(i-1)(j), PCC(i)(j));
      PCS(i)(7) <= PC(i)(7); -- leftmost "virtual" adder sum output
    end generate;
  end generate;
  RAC(0) <= '0';
  g6: for i in 0 to 6 generate -- final ripple adder
    U7: XOR3 port map (PCS(7)(i+1), PCC(7)(i), RAC(i), RAS(i));
    U3: MAJ port map (PCS(7)(i+1), PCC(7)(i), RAC(i), RAC(i+1));
  end generate;
  g7: for i in 0 to 7 generate
    P(i) <= PCS(i)(0); -- get first 8 product bits from full-adder sums
  end generate;
  g8: for i in 8 to 14 generate
    P(i) <= RAS(i-8); -- get next 7 bits from ripple-adder sums
  end generate;
  P(15) <= RAC(7); -- get last bit from ripple-adder carry
end vmul8x8s_arch;

```

В этой программе демонстрируется эффективное использование оператора *generate* (*generate statement*) для создания массивов компонентов, используемых в умножителе. Оператор *generate* должен иметь метку; подобно оператору *for-loop*, им задается итеративная схема управления повторением включенных в него операторов. Операторами, содержащимися в конструкции *for-generate*, могут быть любые параллельные операторы вида *IF-THEN-ELSE* и структурные компоненты, в свою очередь предусматривающие выполнение циклов. Иногда операторы *generate* объединяются с операторами *IF-THEN-ELSE*, обеспечивая возможность своего рода условной компиляции.

СИГНАЛЫ И ПЕРЕМЕННЫЕ

В процессе, приведенном в табл. 5.58, используются переменные, а не сигналы, чтобы ускорить процесс моделирования. Работа с переменными происходит быстрее, потому что моделирующая программа следит за их значениями только тогда, когда процесс запущен. Поскольку значения присваиваются переменным последовательно, для их вычисления в пущном порядке процесс в табл. 5.58 должен быть написан очень аккуратно. Другими словами, переменную нельзя использовать раньше, чем ей будет присвоено значение.

С другой стороны, сигналы имеют те или иные значения постоянно. Когда сигнал в процессе изменяется, моделирующая программа заносит необходимость изменения его значения в будущем в свой список событий. Если сигнал появляется в процессе в правой части оператора присваивания, то этот сигнал должен быть включен также в список чувствительности процесса. Если сигнал изменяется, то процесс выполняется снова и продолжает повторяться до тех пор, пока все сигналы в его списке чувствительности не примут установившиеся значения.

Если желательно видеть промежуточные значения и отсчет времени в процессе моделирования, то в программе в табл. 5.58 можно было бы заменить все переменные (кроме *i* и *j*) на сигналы и включить их в список чувствительности. Полученная программа будет синтаксически правильна, если объявления *type* и *signal* помещены сразу после заголовка архитектуры и все операторы присваивания “:=” заменены на “<=”.

Моделирование будет выполняться намного медленнее после того, как вы выполните указанные выше замены. Как отмечалось в связи с рис. 5.99, в наихудшем случае при распространении сигнала по массиву сумматоров он проходит через 14 сумматоров. Таким же является число циклов моделирования, которые должны быть выполнены неизменяемой VHDL-программой, чтобы получить установившиеся значения сигналов, плюс еще один цикл, на котором моделирующая программа обнаруживала бы, что сигналы не изменяются.

От выбора между сигналами и переменными зависит скорость моделирования, но в большинстве случаев это не оказывает влияния на результаты синтеза в среде VHDL.

Ну, мы говорили, что лучше приберем на конец, и вот этот момент наступил. В библиотеке IEEE std_logic_arith, введенной нами в разделе 5.9.6, имеются функции умножения для классов SIGNED и UNSIGNED, и эти функции представлены оператором “*”. Таким образом, в программе, приведенной в табл. 5.60, умножение чисел без знака можно осуществить в одну строку простым оператором присваивания.

В библиотеке IEEE std_logic_arith функция умножения определена поведенчески, в соответствии с алгоритмом сдвига и сложения. Мы могли бы указать на эту возможность в самом начале данного раздела, но тогда вы не стали бы читать все остальное. Не так ли?

```
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity vmul8x8i is
  port (
    X: in UNSIGNED (7 downto 0);
    Y: in UNSIGNED (7 downto 0);
    P: out UNSIGNED (15 downto 0)
  );
end vmul8x8i;

architecture vmul8x8i_arch of vmul8x8i is
begin
  P <= X * Y;
end vmul8x8i_arch;
```

Табл. 5.60. Истинно поведенческая VHDL-программа для комбинационного умножителя 8×8

НА ПОРОГЕ МЕЧТЫ

В начале табл. 5.58 определена «мажоритарная функция» MAJ трех переменных, используемая затем при вычислении выходов переноса. Значение *мажоритарной функции* n переменных равно 1, если большинство переменных равно 1; в случае 3-входовой схемы, реализующей мажоритарную функцию, 1 появляется на выходе, когда на двух из трех входов присутствуют единицы. (Если n — четное число, то единицы должны быть на $n/2 + 1$ входах.)

Более тридцати лет назад заметный академический интерес вызывал более общий класс n -входовых *пороговых схем*, на выходе которых вырабатывается 1, если на k или большем числе входов присутствует 1. Помимо обеспечения полной занятости теоретиков в области логического проектирования, с помощью пороговых схем многие логические функции можно было бы реализовать, используя меньшее число элементов, чем при применении обычных схем И/ИЛИ. Например, для формирования переноса в сумматоре нужны три вентиля И и один вентиль ИЛИ, и всего лишь одна трехвходовая пороговая логическая схема.

К счастью или к несчастью, так и не появилась экономичная технология изготовления пороговых логических схем, и поэтому до сих пор они остаются академической диковиной.

СИНТЕЗ ПОВЕДЕНЧЕСКИХ ПРОЕКТОВ

Вы, вероятно, слышали, что трансляторы языков программирования высокого уровня, типа C, обычно создают программы в кодах лучше, чем люди, пишущие на ассемблере, даже при «вылизывании» вручную. Большинство разработчиков цифровых схем надеются, что компиляторы поведенческих VHDL-программ также однажды дадут лучшие результаты, нежели типичный проект, выполненный вручную независимо от того, является он схемным решением или структурным описанием на языке VHDL. Лучшие компиляторы не оставят проектировщиков без работы; они просто позволят им справиться с большими по масштабам проектами.

Пока мы не достигли этого уровня. Тем не менее наиболее совершенные программные средства синтеза позволяют осуществлять довольно тонкую оптимизацию обычно используемых поведенческих структур. Например, я вполне допускаю, что средства синтеза FPGA, которые я применял для тестирования VHDL-программ этого раздела, создают по программе в табл. 5.60 столь же быстрый умножитель, какой получится при любой более детальной проработке структуры!

Обзор литературы

Разработчикам цифровых схем, желающим улучшить свой стиль изложения, следует начать с классического труда Странка и Уайта *Элементы стиля* (William Strunk, Jr., and E. B. White. *Elements of Style*, third edition. Allyn & Bacon, 1979). Вероятно, самое дешевое и краткое, но очень полезное руководство по составлению технических описаний — это *Элементы технического письма* Блейка и Блая (Gary Blake and Robert W. Bly. *The Elements of Technical Writing*. Macmillan, 1993). Более энциклопедический характер имеет *Справочник по техническим описаниям* Брюсу, Алреда и Олиу (Brusaw, Alred and Oliu. *Handbook of Technical Writing*, fifth edition. St. Martin's Press, 1997).

Стандарт ANSI/IEEE условных обозначений логических схем установлен документом Std 91-1984, *IEEE Standard Graphic Symbols for Logic Functions*. Другим стандартом, представляющим интерес для разработчиков логических схем, является документ ANSI/IEEE 991-1986, *Logic Circuit Diagrams*. Эти два стандарта и десяток других, включая условные обозначения 10-дюймовых звонков и гнездовых сигнальных разъемов, можно найти в одном удобном пятифунтовом справочнике *Electrical and Electronics Graphic and Letter Symbols and Reference Designations Standards Collection*, издаваемом IEEE в 1996 году (www.ieee.org).

Реальные логические устройства описаны в информационных изданиях и справочниках, издаваемых производителями. Обычно каждые несколько лет выходят обновленные издания справочников, но в последнее время наметилась тенденция к сокращению или ликвидации бумажных изданий и замене их самой последней информацией в сети Интернет. Двумя крупнейшими компаниями-поставщиками, имеющими наиболее полные сайты, являются Texas Instruments (www.ti.com) и Motorola (www.mot.com).

Для определенного логического семейства, такого как 74ALS, все производители приводят одни и те же параметры, так что можно обойтись только одним справочником для каждого семейства. Некоторые параметры, особенно временные характеристики, у разных производителей могут слегка отличаться, и в тех случаях, когда временные соотношения на пределе, лучше проверить пару разных источников и принять во внимание худший вариант. Это *намного* проще, чем убеждать ваш производственный отдел покупать данный компонент исключительно у одного поставщика.

Первые устройства PAL были созданы в компании Monolithic Memories, Inc. (MMI) в 1978 году Беркнером (John Birkner) и Чуа (H. T. Chua). На свое изобретение они получили патент США под номером 4,124,899, а компания MMI вознаградила их покупкой новых марок автомобилей Порше и Мерседес соответственно! Учитывая ценность этой техники (устройств PAL, а не автомобилей), компания Advanced Micro Devices (AMD) в начале 80-х годов приобрела MMI и стала ведущим разработчиком и поставщиком новых ПЛУ и схем типа CPLD. В 1997 году компания AMD передала свои работы по ПЛУ дочерней фирме — компании Vantis Corporation, которая в 1999 году была продана давнему конкуренту — компании Lattice Semiconductor.

Производители ПЛУ предоставляют прекрасные возможности для обучения конструированию на основе ПЛУ. Компания Xilinx Corporation, которая начинала с производства схем FPGA, теперь выпускает также схемы типа CPLD и публикует исчерпывающие справочные данные в виде книги *Xilinx Data Book* (San Jose, CA 95124) и в Интернете (www.xilinx.com). Точно так же компания Lattice Semiconductor, разработавшая микросхемы GAL, издает подробный справочник *Lattice Data Book* (Hillsboro, OR 97124) и поддерживает сайт в Интернете (www.latticesemi.com).

Намного более подробное рассмотрение работы БИС и СБИС, включая ПЛУ, ПЗУ и ОЗУ, можно найти в книгах по электронике, например, в *Микроэлектронике* Миллмана и Грабеля (J. Millman and A. Grabel. *Microelectronics*, second edition. McGraw-Hill, 1987) и в *Проектировании СБИС* Диллинджера (Thomas E. Dillinger. *VLSI Engineering*. Prentice Hall, 1988).

Что касается технической стороны цифрового проектирования, то принципы цифрового проектирования рассматриваются в большинстве учебников, но лишь в немногих из них говорится о практических аспектах этой деятельности. Превосходной небольшой книгой, в которой внимание сосредоточено на практических вопросах, является *Хорошо темперированное цифровое проектирование* Сейденстикера (Robert B. Seidensticker. *The Well-Tempered Digital Design*. Addison-Wesley, 1986). В ней вы найдете массу полезных сведений, изложенных в доступной форме, от философии проектирования до технологии производства.

Упражнения

- 5.1. Приведите три примера комбинационных логических схем, для описания которых в виде таблицы истинности требуются *миллиарды* строк. Для каждой схемы опишите ее входы и выход(ы), и точно укажите, сколько строк содержит таблица истинности; таблицу истинности выписывать не надо. (Указание: Несколько таких схем можно найти в этой главе.)

- 5.2. Используя теорему Де Моргана, нарисуйте эквивалентное условное обозначение 8-входового вентиля И-НЕ 74х30.
- 5.3. Используя теорему Де Моргана, нарисуйте эквивалентное условное обозначение 3-входового вентиля ИЛИ-НЕ 74х27.
- 5.4. Что неправильно в пмепп сигнала "READY"?
- 5.5. Вас может раздражать необходимость следить за активными уровнями всех сигналов в логической схеме. А почему бы не применять только неинвертирующие схемы, ведь при этом все сигналы имеют высокий активный уровень?
- 5.6. Правильно или ложно высказывание: При логическом проектировании по принципу «инверсия к инверсии» выходы с кружком могут быть соединены только с входами с кружком.
- 5.7. Проектируемая цифровая система передачи имеет двенадцать идентичных сетевых портов. Какая структура схемы, вероятнее всего, наилучшим образом соответствует проекту?
- 5.8. Определите точную максимальную задержку распространения от входа IN до выхода OUT в схеме на рис. X5.8 как для перехода сигнала от низкого уровня к высокому уровню, так и для перехода от высокого уровня к низкому, используя значения временных параметров, приведенные в табл. 5.2. Повторите расчет, воспользовавшись единственным значением задержки в наихудшем случае для каждого вентиля. Сравните и объясните полученные результаты.

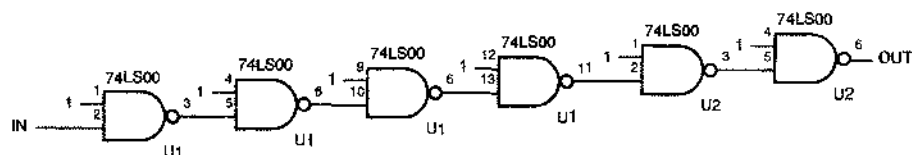


Рис. X5.8.

- 5.9. Повторите упражнение 5.8, заменив микросхемы 74LS00 на ИС 74НСТ00.
- 5.10. Повторите упражнение 5.8, заменив микросхемы 74LS00 на ИС 74LS08.
- 5.11. Повторите упражнение 5.8, заменив микросхемы 74LS00 на ИС 74АНСТ02 и считая, что на свободные входы подаются не логические единицы, а логические нули. Воспользуйтесь типичными, а не максимальными значениями временных параметров.
- 5.12. Оцените минимальную задержку распространения от входа IN до выхода OUT для схемы, показанной на рис. X5.12. Обоснуйте ваш ответ.

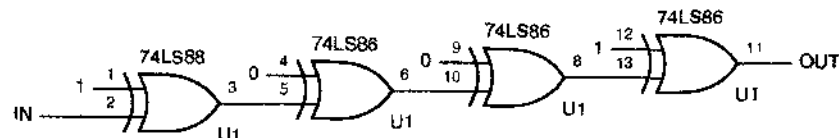


Рис. X5.12.

- 5.13. Определите точную максимальную задержку распространения от входа IN до выхода OUT для схемы на рис. X5.12 как для перехода сигнала от низкого уровня к высокому, так и для перехода от высокого уровня к низкому, используя значения временных параметров, приведенные в табл. 5.2. Повторите расчет, воспользовавшись единственным значением задержки в наихудшем случае для каждого вентиля. Сравните и объясните полученные результаты.
- 5.14. Повторите упражнение 5.13, заменив микросхемы 74LS86 на ИС 74HCT86.
- 5.15. Как вы считаете, какой дешифратор быстрее: с высоким активным уровнем сигналов на выходах или с низким активным уровнем сигналов на выходах?
- 5.16. Используя информацию о микросхемах серии 74LS из табл. 5.3, определите максимальную задержку распространения от любого входа до любого выхода в дешифраторе 5×32 , схема которого приведена на рис. 5.39. Можно воспользоваться методом анализа «в худшем случае».
- 5.17. Повторите упражнение 5.16, проведя детальный анализ для каждого направления перехода сигнала, и сравните свои результаты.
- 5.18. Нарисуйте символы, создаваемые семисегментным дешифратором 74x49 при десятичных входных комбинациях от 1010 до 1111.
- 5.19. Покажите, как реализовать следующие логические функции в виде схем с одним и двумя выходами соответственно, применяя один или большее число полных дешифраторов 74x138 или 74x139 и вентили И-НЕ. (Указание: Каждая реализация должна быть эквивалентна сумме мinterмов.)
- (a) $F = \sum_{x,y,z}(2, 4, 7)$ (b) $F = \prod_{a,b,c}(3, 4, 5, 6, 7)$
 (c) $F = \sum_{a,b,c,d}(2, 4, 6, 14)$ (d) $F = \sum_{w,x,y,z}(0, 1, 2, 3, 5, 7, 11, 13)$
 (e) $F = \sum_{w,x,y}(1, 3, 5, 6)$ (f) $F = \sum_{a,b,c}(0, 4, 6)$
 (g) $G = \sum_{w,x,y}(2, 3, 4, 7)$ (h) $G = \sum_{c,d,e}(1, 2)$.
- 5.20. Исходя из принципиальной схемы приоритетного шифратора 74x148, запишите логические выражения для сигналов на его выходах $A2_L$, $A1_L$ и $A0_L$. Чем они отличаются от «универсальных» выражений, приведенных в разделе 5.5.1?
- 5.21. Что сделано совершенно неправильно в схеме на рис. X5.21? Предложите изменение, которое ликвидирует указанную проблему.
- 5.22. Используя информацию о микросхемах серии 74LS из табл. 5.2 и 5.3, определите максимальную задержку распространения сигнала от любого входа до любого выхода в мультиплексоре 32×1 , схема которого приведена на рис. 5.66. Можно воспользоваться методом анализа «в худшем случае».
- 5.23. Повторите упражнение 5.22, используя микросхемы серии 74HCT.
- 5.24. Древовидную n -входовую схему проверки на четность можно построить на основе вентилей ИСКЛЮЧАЮЩЕЕ ИЛИ в духе схемы, приведенной на рис. 5.74(а). При каких условиях подобная n -входовая древовидная схема проверки на четность, построенная на вентилях ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ, выполняет точно ту же функцию?

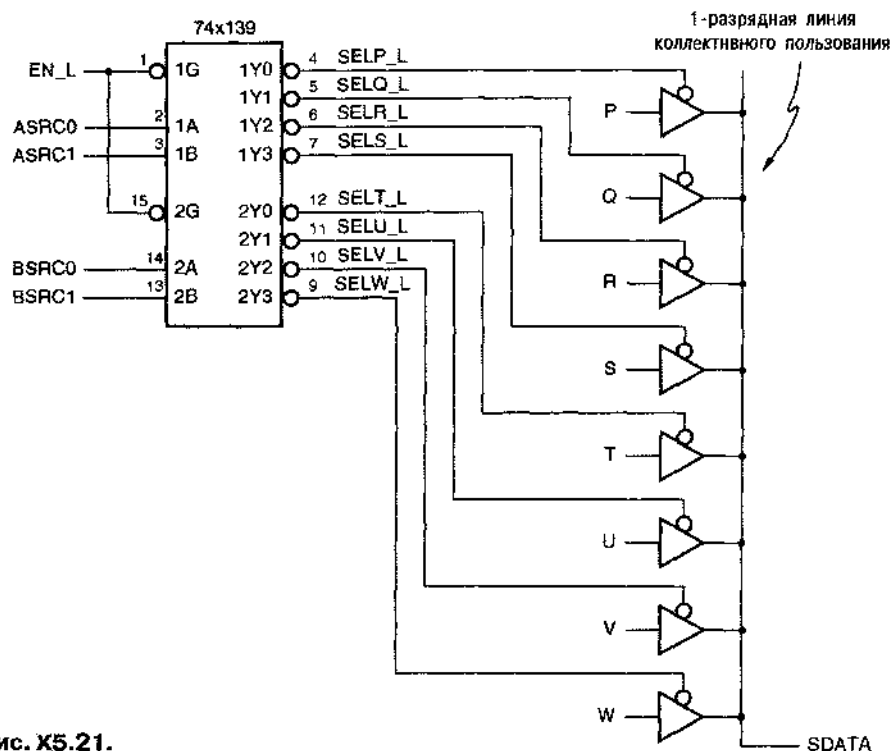


Рис. X5.21.

- 5.25.** Используя информацию о микросхемах серии 74LS из табл. 5.2 и 5.3, определите максимальную задержку распространения сигнала от шины DU до шины DC в приведенной на рис. 5.77 схеме, исправляющей ошибки. Можно воспользоваться методом анализа «в худшем случае».
- 5.26.** Повторите упражнение 5.25, используя микросхемы серии 74HCT.
- 5.27.** Воспользовавшись соотношениями, приведенными в разделе 5.9.4, запишите полное логическое выражение для выхода $ALTOUT$ микросхемы 74x85.
- 5.28.** Запишите алгебраическое выражение для третьего бита суммы S_2 двоичного сумматора, как функцию сигналов на входах x_0, x_1, x_2, y_0, y_1 и y_2 . Предположите, что $c_0 = 0$, и не пытайтесь «разнести множитель по слагаемым» или минимизировать это выражение.
- 5.29.** Исходя из принципиальной схемы ИС 74x682, запишите логическое выражение для сигнала на выходе $PGTQ_L$ в зависимости от сигналов на ее входах.
- 5.30.** Используя информацию о микросхемах серии 74LS из табл. 5.3, определите максимальную задержку распространения от любого входа до любого выхода в 16-разрядном сумматоре с групповым сквозным переносом, изображением на рис. 5.92. Можно воспользоваться методом анализа «в худшем случае».

Задачи

- 5.31. Возможно следующее определение логической схемы BUT (задача 4.50): « $Y_1 = 1$, если A_1 и B_1 равны 1, но при этом либо A_2 , либо B_2 равно 0; Y_2 определяется симметрично». Запишите таблицу истинности и найдите минимальные выражения вида «сумма произведений» для выходов схемы BUT. Нарисуйте принципиальную схему в виде структуры И-НЕ-И-НЕ, реализующей эти выражения, считая, что имеются только неинвертированные входные сигналы. Вы можете использовать микросхемы 74х00, '04, '10, '20 и '30.
- 5.32. Изобразите на уровне вентилей логическую схему BUT из задачи 5.31, в которой использовалось бы минимальное число транзисторов при ее реализации по КМОП-технологии. Можно воспользоваться схемами 74х00, '02, '04, '10, '20 и '30. Запишите выражения для сигналов на выходах (которые не обязательно должны быть двухуровневыми суммами произведений) и нарисуйте принципиальную схему.
- 5.33. Для каждой из схем в двух предыдущих задачах вычислите задержку от входа до выхода в наихудшем случае, используя значения задержек для микросхем серии 74НСТ из табл. 5.2. Сравните стоимость (число транзисторов), быстродействие и нагрузку со стороны входов этих двух схем. Какая из них лучше?
- 5.34. Осуществите батификацию (*butification*) функции $F = \sum_{wxyz} (3, 7, 11, 12, 13, 14)$. Другими словами, покажите, как реализовать функцию F с помощью одной логической схемы BUT из задачи 5.31 и одного 2-входового вентиля ИЛИ.
- 5.35. Предположим, что дешифратор 74LS138 включен так, что на все входы разрешения подан сигнал активного уровня, а $СВА = 101$. Используя информацию из табл. 5.3 и внутреннюю логическую структуру микросхемы '138, определите задержку распространения сигнала от входа до всех соответствующих выходов при каждом возможном изменении сигнала на одном из входов. (Указание: Всего имеется девять значений задержек, так как изменение сигналов на входах А, В или С влияет на значения сигналов на двух выходах, а изменение сигнала на любом из трех входов разрешения проявляется в значении сигнала на одном выходе.)
- 5.36. Предположим, что вас просят разработать новый компонент – десятичный дешифратор, оптимизированный для приложений, в которых ожидается появление только десятичных входных комбинаций. На сколько можно уменьшить стоимость такого дешифратора по сравнению с простым дешифратором 4×16 с шестью удаленными выходами? Запишите логические выражения для всех десяти выходов минимизированного дешифратора, предполагая, что активным является высокий уровень входных и выходных сигналов и отсутствуют входы разрешения.
- 5.37. Сколько потребовалось бы карт Карно для решения задачи 5.36 согласно формальной процедуре минимизации схем со многими выходами, описанной в разделе 4.3.8?

- 5.38. Иредноложим, что требуется нолный дешифратор 5×32 с одним входом разрешения и низким активным уровнем сигнала на этом входе, подобный изображенному на рис. 5.39. Если на входе EN1 поддерживается высокий уровень, то один из входов EN2_L или EN3_L, указанных на рисунке, можно было бы использовать в качестве входа разрешения, при условии, что другой из этих входов заземлен. Рассмотрите все за и против использования EN2_L и EN3_L в качестве входа разрешения.
- 5.39. Определите, соответствуют ли выходы a, b и c семисегментного дешифратора 74х49 минимальным выражениям вида «произведение сумм» для этих сегментов, предполагая, что десятичные входные комбинации являются «безразличными» и $B_L = 1$.
- 5.40. Видоизмените схему семисегментного дешифратора 74х49 так, чтобы цифры 6 и 9 имели вид, указанный на рис. X5.40. Изменятся ли в результате этого изображения, соответствующие десятичным входным комбинациям от 1010 до 1111?

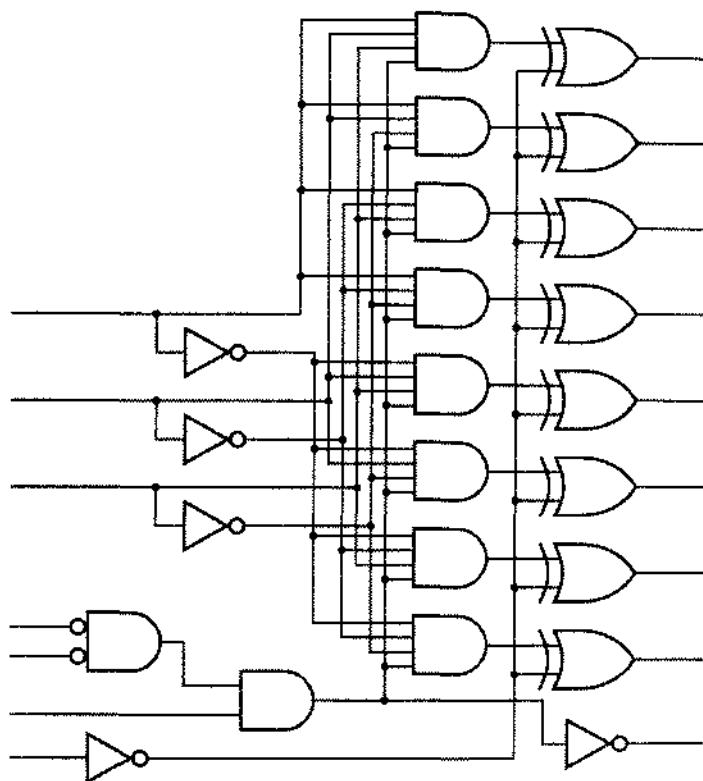


Рис. X5.40.

- 5.41. Взяв за основу программу на языке ABEL, приведенную в табл. 5.22, напишите программу для семисегментного дешифратора со следующими усовершенствованиями:
- Все выходные сигналы имеют низкий активный уровень.
 - Введены два новых входа ENH_{EX} и ERRDET, управляющих декодированием сегментных выходов.
 - Если ENH_{EX} = 0, то сигналы на выходах соответствуют сигналам микросхемы 74х49.
 - Если ENH_{EX} = 1, то цифры 6 и 9 изображаются так, как показано на рис. X5.40, а сигналы на выходах для цифр A–F зависят от значения ERRDET.
 - Если ENH_{EX} = 1 и ERRDET = 0, то входным комбинациям A–F соответствуют изображения букв от A до F, как в исходной программе.
 - Если ENH_{EX} = 1 и ERRDET = 1, то при подаче на входы комбинаций A–F изображения имеют вид буквы S.
- 5.42. Один известный разработчик логических схем решил оставить преподавание и попытать счастья продаж прав на использование схемы, показанной на рис. X5.42.
- Обозначьте входы и выходы схемы, дав соответствующие имена сигналам с указанием активного уровня.
 - Что делает эта схема? Будьте конкретны и объясните назначение всех входов и выходов.
 - Изобразите условное обозначение, которое могло бы быть предложено для этой схемы в справочнике.

- (d) Напишите для этой схемы программу на языке ABEL или наведенческую программу на языке VHDL.
- (e) С какими стандартными схемами конкурирует новая схема? Как вы думаете, нашла бы применение эта схема как ИС средней степени интеграции?

Рис. X5.42.

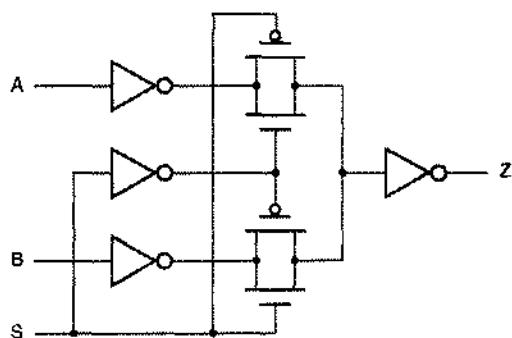


- 5.43. Выход FCT-буфера с тремя состояниями подключен к десяти входам вентиля семейства FCT и соединен резистором 4.7 кОм с шиной питания 5.0 В. Оцените, через какое время сигнал на входах вентилей будет воспринят ими как сигнал высокого уровня, если на выходе буфера происходит переключение с низкого уровня в состояние Hi-Z. Сформулируйте используемые вами предположения.
- 5.44. Выходы десяти FCT-буферов с тремя состояниями выведены на шину, к которой подключены десять входов вентилей семейства FCT; резистором 4.7 кОм шина соединена также с источником питания 5.0 В. Предполагая, что никакие другие устройства не выдают сигналы на шину, оцените, как долго сигнал на шине остается в пределах значений, отнесенных к тому или другому из логических уровней, когда активный выход переходит в состояние Hi-Z. Сформулируйте используемые вами предположения.

- 5.45. Составьте принципиальную схему шифратора 10×4 , на входы которого поступают сигналы кода «1 из 10», а выходные сигналы являются словами двоично-десятичного кода.
- 5.46. Нарисуйте логическую схему шифратора 16×4 , используя лишь четыре 8-входовых вентиля И-НЕ. Каковы активные уровни входных и выходных сигналов в вашей схеме?
- 5.47. Воспользовавшись микросхемой 74х148, нарисуйте принципиальную схему устройства, находящего сигнал с наибольшим приоритетом среди сигналов с высоким активным уровнем на восьми входах I_0 – I_7 , из которых вход I_7 имеет высший приоритет. Желательно, чтобы на адресных выходах A_2 – A_0 вырабатывались сигналы с высоким активным уровнем, указывающие номер входа с наивысшим приоритетом из числа тех, на которые поданы сигналы активного уровня. Если ни на один из входов сигнал активного уровня не подан, то сигналы на выходах A_2 – A_0 должны быть равны 111, а сигнал на выходе $IDLE$ должен иметь активный уровень. В дополнение к микросхеме 74х148 вы можете использовать отдельные вентили. Убедитесь, что все сигналы названы в соответствии с их активными уровнями.
- 5.48. Нарисуйте принципиальную схему устройства, находящего сигнал с наибольшим приоритетом среди сигналов с низким активным уровнем на восьми входах I_0_L – I_7_L , из которых I_0_L имеет высший приоритет. Желательно, чтобы на адресных выходах A_2 – A_0 вырабатывались сигналы с высоким активным уровнем, указывающие номер входа с наивысшим приоритетом из числа тех, на которые поданы сигналы активного уровня. Если хотя бы на одном из входов присутствует сигнал активного уровня, то сигнал на выходе $VALID$ должен иметь активный уровень. Убедитесь, что все сигналы названы в соответствии с их активными уровнями. Это устройство можно построить на одной ИС 74х148 без применения других вентилях.
- 5.49. Решая задачу 5.48 вы должны были убедиться в том, что не всегда можно быть последовательным в обозначении активных уровней, если только нет желания ввести другие условные обозначения для ИС средней степени интеграции, которые можно использовать различным образом. Предложите другое условное обозначение для микросхемы 74х148, которое позволило бы быть последовательным в задаче 5.48.
- 5.50. Постройте комбинационную схему с восемью входами запроса R_0_L – R_7_L с низким активным уровнем сигналов и восемью выходами A_2 – A_0 , $VALID$, B_2 – B_0 и $BVALID$. Входы R_0_L – R_7_L и выходы A_2 – A_0 и $VALID$ определены так, как это сделано в задаче 5.48. Выходы B_2 – B_0 и $BVALID$ должны указывать номер входа из числа тех, на которые поданы сигналы активного уровня, приоритет которого является следующим за наибольшим приоритетом. Постарайтесь построить эту схему, используя не более шести корпусов МИС и СИС, и уж во всяком случае не более 10.
- 5.51. Повторите задачу 5.50, написав программу на языке ABEL. Размещается ли вся схема в одной микросхеме GAL20V8?
- 5.52. Повторите задачу 5.50, написав программу на языке VHDL.

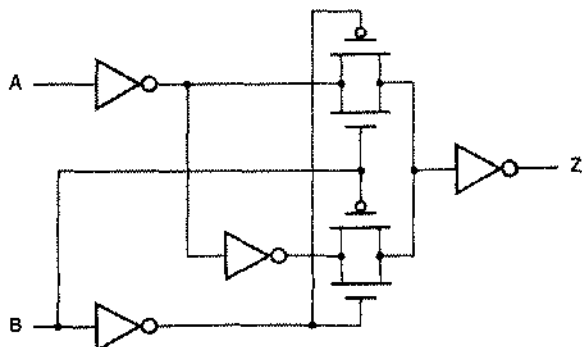
- 5.53. Введите иновый тип в VHDL-программе, согласующийся со стандартом IEEE 1164, который служил бы моделью выходов с открытым коллектором, причем такой, что соединение таких выходов вместе реализовало бы функцию «монтажное И». Вам следует смоделировать также нагрузочный резистор, соединяющий шину с источником питания, такой что в случае, когда нагрузочного резистора нет и никакое из подключаемых к шине устройств не находится в активном режиме, на шине вырабатывается сигнал, имеющий значение "unknown". Проверьте ваши определения путем моделирования схемы, приведенной на рис. X3.92 для всех комбинаций входных сигналов с резистором R1 и без него.
- 5.54. Постройте 3-входовой 5-разрядный мультиплексор, который размещался бы в корпусе ИС с 24 выводами. Запишите таблицу истинности и нарисуйте принципиальную схему и условное обозначение вашего мультиплексора.
- 5.55. Запишите таблицу истинности и нарисуйте схему, состоящую из логических вентилях и реализующую ту же логическую функцию, что и КМОП-схема, изображенная на рис. X5.55. (Схема содержит логические ключи, описанные в разделе 3.7.1.)

Рис. X5.55.



- 5.56. Какая логическая функция реализуется КМОП схемой, показанной на рис. X5.56?

Рис. X5.56.



5.57. Одни известный разработчик логических схем решил оставить преподавание и попытаться счастья продажей и прав на пользование схемой, показанной на рис. X5.57.

- Обозначьте входы и выходы схемы, дав соответствующие имена сигналам с указанием активного уровня.
- Что делает эта схема? Будьте конкретны и объясните назначение всех входов и выходов.
- Изобразите условное обозначение, которое могло бы быть предложено для этой схемы в справочнике.
- Напишите для этой схемы программу на языке ABEL или поведенческую программу на языке VHDL.
- С какими стандартными схемами конкурирует новая схема? Как вы думаете, нашла бы применение эта схема как ИС средней степени интеграции?

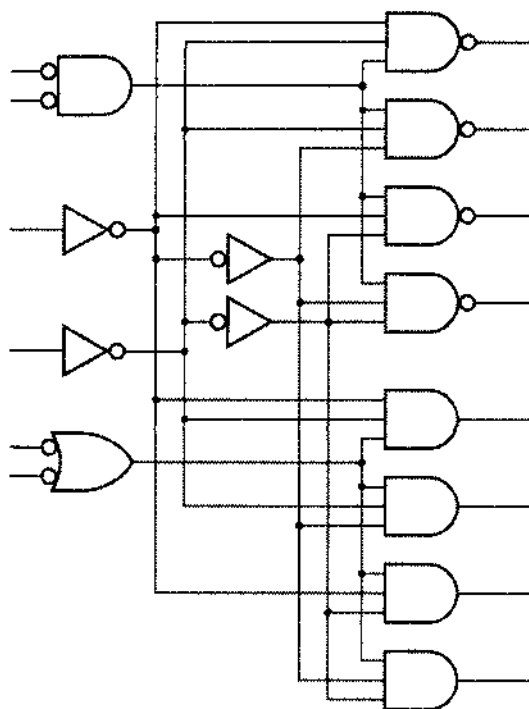


Рис. X5.57.

5.58. Напишите программу на языке VHDL для мультиплексора 74х157, функционирование которого описано в табл. 5.35.

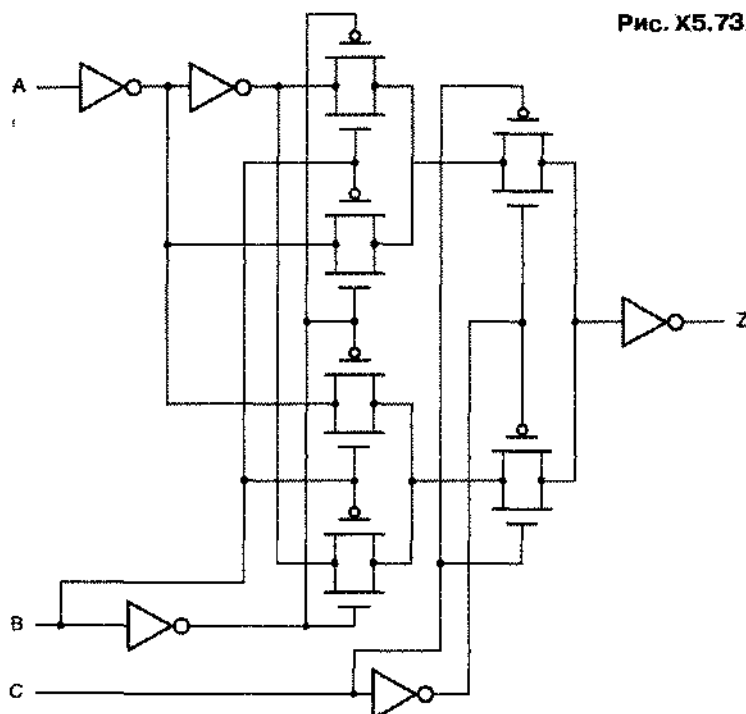
5.59. Напишите программу на языке VHDL для мультиплексора 74х153, функционирование которого описано в табл. 5.36.

5.60. Покажите, как реализовать 4-входовой 18-разрядный мультиплексор с функциональными возможностями, описанными в табл. 5.40, воспользовавшись 18-ю микросхемами 74х15.

- 5.61. Покажите, как реализовать 4-входовой 18-разрядный мультиплексор с функциональными возможностями, описанными в табл. 5.40, воспользовавшись 9-ю микросхемами 74x153 и «преобразователем кода» со входами S2–S0 и выходами C1, C0, таким, что $[C1, C0] = 00\text{--}11$, когда сигналами на входах S2–S0 выбираются A–D соответственно.
- 5.62. Создайте из отдельных вентилях 3-входовую комбинационную схему с 2-мя выходами, которая осуществляла бы преобразование кода, указанное в предыдущей задаче.
- 5.63. Добавьте вход управления третьим состоянием по выходу OE мультиплексору, описанному на языке VHDL в табл. 5.43. Ваше решение должно содержать только один процесс.
- 5.64. 16-разрядная *схема быстрого сдвига (barrel shifter)* является комбинационной логической схемой с 16 входами данных, 16 выходами данных и 4 входами управления. Выходное слово равно входному слову, циклически сдвинутому на некоторое число разрядов, определяемое сигналами на входах управления. Например, если входное слово имеет вид: ABCDEFGHIJKLMNOP (каждый символ представляет собой один бит), а сигналы на входах управления равны 0101 (5), то выходное слово равно FGHIJKLMNOPABCDE. Постройте 16-разрядное устройство быстрого сдвига, используя комбинационные микросхемы средней степени интеграции, рассмотренные в этой главе. Ваше устройство должно содержать не более 20 микросхем. Не рисуйте полную схему; составьте структурную схему, опишите ее в общих чертах и укажите общее число и типы необходимых ИС.
- 5.65. Напишите программу на языке ABEL для схемы быстрого сдвига из задачи 5.64.
- 5.66. Напишите программу на языке VHDL для схемы быстрого сдвига из задачи 5.64.
- 5.67. Разработчик, создавая схему, приведенную на рис. 5.76, случайно использовал микросхемы 74x00 вместо '08 и обнаружил, что схема осталась работоспособной, за исключением того, что изменился активный уровень сигнала ERROR. Как это оказалось возможным?
- 5.68. Схему проверки на нечетность с 2^n входами можно построить на $2^n - 1$ схемах ИСКЛЮЧАЮЩЕЕ ИЛИ. Опишите два разных варианта этой схемы, один из которых дает минимальную задержку распространения от входа до выхода в наихудшем случае, а другой – максимальную задержку. Для каждого варианта укажите наибольшее число схем ИСКЛЮЧАЮЩЕЕ ИЛИ, вносящих задержку, и опишите ситуацию, в которой каждый из вариантов мог бы быть предпочтительнее, чем другой.
- 5.69. Запишите 4-шаговый итерационный алгоритм, соответствующий итерационной схеме компаратора на рис. 5.80.
- 5.70. Напишите программу на языке VHDL для 16-разрядного итерационного компаратора, воспользовавшись схемой, приведенной на рис. 5.80. Используйте возможности оператора "generate" языка VHDL.

- 5.71. Используя микросхему 74х85, постройте такую древовидную структуру 16-разрядного компаратора, чтобы максимальная задержка при сравнении равнялась удвоенной задержке одной микросхемы 74х85.
- 5.72. Напишите программу на языке VHDL для устройства с функциональными возможностями микросхемы 74х85.
- 5.73. Запишите таблицу истинности и нарисуйте схему, состоящую из логических вентилей и реализующую ту же логическую функцию, что и КМОП-схема, приведенная на рис. X5.73.

Рис. X5.73.



- 5.74. Постройте компаратор, подобный компаратору 74х85, который мог бы быть применен при обратном порядке соединения каскадов, а именно: при сравнении, например, 12-разрядных чисел, выходы компаратора старших разрядов соединялись бы с входами для каскадного включения компаратора средних разрядов, а выходы компаратора средних разрядов соединялись бы с входами для каскадного включения компаратора младших разрядов. Не надо выполнять логическое проектирование в полном объеме и рисовать всю схему, достаточно составить таблицу истинности и описать словами взаимосвязи в 12-разрядном компараторе.
- 5.75. Постройте 24-разрядный компаратор из трех микросхем 74х682, используя при необходимости дополнительные вентили. Ваша схема должна сравнивать

- два 24-разрядных числа без знака P и Q и вырабатывать два выходных сигнала, принимающих единичные значения при $P = Q$ и $P > Q$ соответственно.
- 5.76. Используя информацию из табл. 5.3, определите максимальную задержку распространения сигнала от любого входа в шинах A или B до любого выхода в шине F в схеме на рис. 5.96 при ее работе в режиме 16-разрядного сумматора с ускоренным переносом. Можно воспользоваться методом анализа «в худшем случае».
- 5.77. Воспользовавшись принципиальной схемой сумматора 74x283, изображенной на рис. 5.91, напишите логическое выражение для сигнала на выходе S_2 в зависимости от сигналов на входах и докажите, что он действительно, как и утверждается, равен третьему биту суммы при двоичном сложении. Можно предположить, что $c_0 = 0$ (то есть игнорировать c_0).
- 5.78. Опираясь на справочные данные для схемы ускоренного переноса 74S182, определите, действительно ли сигналы на ее выходах соответствуют равенствам, приведенным в разделе 5.10.7.
- 5.79. Оцените число термов-произведений в минимальном выражении вида «сумма произведений» для сигнала на выходе c_{32} 32-разрядного двоичного сумматора. Ответ должен быть более конкретным, чем слова «миллиарды и миллиарды». Обоснуйте ваш ответ.
- 5.80. Нарисуйте принципиальную схему 64-разрядного АЛУ на шестнадцати микросхемах 74x181 с двухуровневой схемой ускоренного переноса на пяти микросхемах 74S182. У схем 181 необходимо показать только входы CIN и выходы P_L и G_L .
- 5.81. Опишите модель АЛУ 74x181 на языке VHDL.
- 5.82. Покажите, как реализовать каждую из следующих четырех функций, используя одну ИС малой степени интеграции и одну микросхему 74x138:
- $$F1 = X' \cdot Y' \cdot Z' + X \cdot Y \cdot Z \quad F2 = X' \cdot Y' \cdot Z + X \cdot Y \cdot Z'$$
- $$F3 = X' \cdot Y \cdot Z' + X \cdot Y' \cdot Z \quad F4 = X \cdot Y' \cdot Z' + X' \cdot Y \cdot Z$$
- 5.83. Определите задержку распространения сигналов в умножителе, изображенном на рис. 5.98, в наихудшем случае, считая, что задержка распространения сигнала от входа любого сумматора до его выхода суммы вдвое больше задержки до выхода переноса. Повторите задачу, принимая противоположное соотношение между задержками сигналов. Если бы вы проектировали ячейку сумматора с самого начала, для какого из путей вы предпочли бы меньшую задержку? Имеется ли оптимальное соотношение между значениями этих задержек?
- 5.84. Повторите предыдущую задачу для умножителя, представленного на рис. 5.99.
- 5.85. Постройте специализированный дешифратор, работающий согласно табл. Х5.85, используя ИС малой и средней степени интеграции. Минимизируйте число используемых корпусов ИС.

Табл. X5.85.

CS_L	A2	A1	A0	Активизируемый выход
1	x	x	x	ни один
0	0	0	x	BILL_L
0	0	x	0	MARY_L
0	0	1	x	JOAN_L
0	0	x	1	PAUL_L
0	1	0	x	ANNA_L
0	1	x	0	FRED_L
0	1	1	x	DAVE_L
0	1	x	1	KATE_L

- 5.86. Повторите задачу 5.85, написав программу на языке ABEL и имея в виду реализацию на одной микросхеме GAL16V8.
- 5.87. Повторите задачу 5.85, написав программу на языке VHDL.
- 5.88. Для кода Хэмминга, реализуемого схемой, изображенной на рис. 5.77, напишите VHDL-программу объекта, который был бы кодером Хэмминга с 4-разрядным входом данных и 7-разрядным кодовым словом на выходе.
- 5.89. Напишите программу на языке ABEL, имея в виду реализацию на одной микросхеме GAL16V8 специализированного мультиплексора с четырьмя 3-разрядными входными шинами P, Q, R, T и тремя входами для сигналов управления S2–S0, с помощью которых, согласно табл. X5.89, выбирают одну из шин так, что сигналы с этой шины поступают на 3-разрядную выходную шину Y.

Табл. X5.89.

S2	S1	S0	Выбираемый выход
0	0	0	P
0	0	1	P
0	1	0	P
0	1	1	Q
1	0	0	P
1	0	1	P
1	1	0	R
1	1	1	T

- 5.90. Постройте специализированный мультиплексор с четырьмя 4-разрядными входными шинами P, Q, R и T, в котором выбор одной из шин осуществляется согласно табл. X5.89, так что сигналы с выбранной шины поступают на 4-разрядную выходную шину Y. Воспользуйтесь двумя микросхемами 74x153 и преобразователем кода, который отображает восемь возможных комбинаций сигналов на входах S2–S0 в четыре кодовых слова выбора для микросхем 74x153. Выберите такой код, при котором размеры преобразователя кода и задержка распространения были бы минимальными.

- 5.91. Постройте специализированный мультиплексор с пятью 4-разрядными входными шинами A, B, C, D и E, в котором выбор одной из шин, осуществляется согласно табл. X5.91, так что сигналы с выбранной шины поступают на 4-разрядную выходную шину T. Можно использовать не больше трех ИС малой и средней степени интеграции.

Табл. X5.91.

S2	S1	S0	Выбираемый вход
0	0	0	A
0	0	1	B
0	1	0	A
0	1	1	C
1	0	0	A
1	0	1	D
1	1	0	A
1	1	1	E

- 5.92. Повторите задачу 5.91, написав программу на языке ABEL и имея в виду реализацию на одном или большем числе устройств PAL/GAL, описанных в этой главе. Минимизируйте число таких устройств и их размеры.
- 5.93. Постройте 3-разрядную схему совпадения с шестью входами SLOT [2–0] и GRANT [2–0] и одним выходом MATCH_L с низким активным уровнем сигнала. Когда схема устанавливается в систему, на входы SLOT подаются фиксированные значения, а сигналы на входах GRANT при нормальной работе системы изменяются циклически. Применяя только ИС малой и средней степени интеграции, упомянутые в табл. 5.2 и 5.3, постройте компаратор с минимальным возможным значением максимальной задержки распространения сигнала от входов GRANT [2–0] до выхода MATCH_L. (Замечание: Автору пришлось решать эту проблему «в реальной жизни» при проектировании 25-мегагерцовой системы, где существенным оказалось сокращение на 2 нс задержки прохождения сигнала по основному пути.)



ПРОЕКТИРОВАНИЕ ЦИФРОВЫХ УСТРОЙСТВ



Дж. Чэ́йкерли



БИБЛИОТЕКА СОВРЕМЕННОЙ ЭЛЕКТРОНИКИ

Джон Ф. Уэйкерли

Проектирование цифровых устройств

том II

Перевод с английского Е.В. Воронова, А.Л. Ларина

ПОСТМАРКЕТ
МОСКВА
2002

Дж. Ф. Уэйкерли

Проектирование цифровых устройств, том 2.

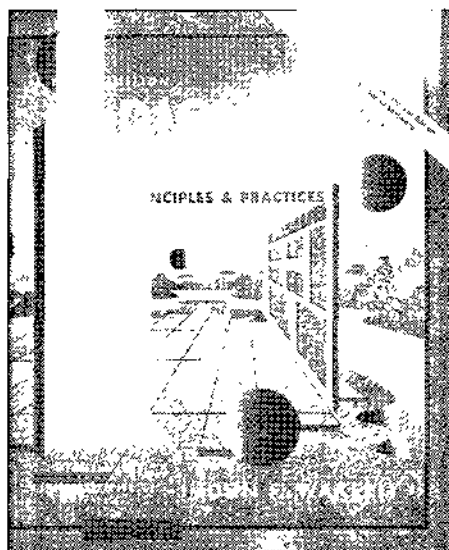
Москва:

Постмаркет, 2002. – 528 с.

Основополагающий учебник, в котором рассмотрены все направления современной цифровой электроники. Особое внимание уделено программируемым логическим интегральным схемам (ПЛИС).

На двух прилагаемых лазерных дисках размещено программное обеспечение фирмы Xilinx.

Предназначен для студентов, аспирантов и преподавателей ВУЗов, разработчиков аппаратуры.



© 2000, 1994, 1990 by Prentice Hall

© 2002, перевод на русский язык

ЗАО «Предприятие Постмаркет»

ISBN 5-901095-12-X

Содержание

Глава 6.

Примеры проектирования комбинационных схем	553
6.1. Примеры проектирования на основе стандартных блоков	554
6.1.1. Устройство быстрого сдвига	554
6.1.2. Простой шифратор для получения чисел с плавающей точкой	557
6.1.3. Двойной приоритетный шифратор	561
6.1.4. Расширение компараторов	562
6.1.5. Компаратор с управляемым режимом работы	564
6.2. Примеры проектирования схем с использованием языка ABEL и их реализация в ПЛУ	566
6.2.1. Устройство быстрого сдвига	566
6.2.2. Простой шифратор для получения чисел с плавающей точкой	569
6.2.3. Двойной приоритетный шифратор	571
6.2.4. Расширение компараторов	573
6.2.5. Компаратор с управляемым режимом работы	575
6.2.6. Счетчик числа единиц	578
6.2.7. Игра в крестики и нолики	579
6.3. Примеры проектирования с использованием языка VHDL	588
6.3.1. Устройство быстрого сдвига	588
6.3.2. Простой шифратор для получения чисел с плавающей точкой	596
6.3.3. Двойной приоритетный шифратор	600
6.3.4. Расширение компараторов	602
6.3.5. Компаратор с управляемым режимом работы	604
6.3.6. Счетчик числа единиц	606
6.3.7. Игра в крестики и нолики	609
Задачи	616

Глава 7.

Принципы проектирования последовательностных логических схем	619
7.1. Элементы с двумя устойчивыми состояниями	621
7.1.1. Цифровой подход	622
7.1.2. Аналоговый подход	622
7.1.3. Неустойчивое равновесие	624
7.2. Защелки и триггеры	625
7.2.1. SR-защелка	626
7.2.2. SR-защелка	629
7.2.3. SR-защелка с входом разрешения	630
7.2.4. D-защелка	631
7.2.5. D-триггер, переключающийся по фронту	632
7.2.6. Переключающийся по фронту D-триггер с входом разрешения	636
7.2.7. Тестируемый триггер	636
....	

7.2.8. Двухтактный SR-триггер	638
7.2.9. Двухтактный JK-триггер	639
7.2.10. JK-триггер, переключающийся по фронту	641
7.2.11. T-триггер	642
7.3. Анализ тактируемых синхронных конечных автоматов	644
7.3.1. Структура конечного автомата	644
7.3.2. Выходная логика	645
7.3.3. Характеристические уравнения	646
7.3.4. Анализ конечных автоматов с D-триггерами	647
7.3.5. Анализ конечных автоматов на JK-триггерах	656
7.4. Проектирование тактируемых синхронных конечных автоматов	658
7.4.1. Пример составления таблицы состояний	659
7.4.2. Минимизация числа состояний	664
7.4.3. Кодирование состояний	665
7.4.4. Синтез с использованием D-триггеров	669
7.4.5. Синтез с использованием JK-триггеров	673
7.4.6. Дальнейшие примеры проектирования на основе D-триггеров	677
7.5. Проектирование конечных автоматов с помощью диаграмм состояний	682
7.6. Синтез конечных автоматов на основе списка переходов	690
7.6.1. Уравнения переходов	690
7.6.2. Уравнения возбуждения	692
7.6.3. Варианты схем	692
7.6.4. Реализация конечного автомата	693
7.7. Другой пример проектирования конечного автомата	693
7.7.1. Игра на угадывание	693
7.7.2. Неиспользуемые состояния	696
7.7.3. Кодирование состояний выходными комбинациями	697
7.7.4. Кодирование «безразличных» состояний	699
7.8. Разбиение конечных автоматов на блоки	701
7.9. Последовательностные схемы с обратной связью	704
7.9.1. Анализ	704
7.9.2. Анализ схем с несколькими целями обратной связи	709
7.9.3. Гонки	711
7.9.4. Таблицы состояний и таблицы потока	713
7.9.5. Анализ работы D-триггера в КМОП-исполнении	716
7.10. Проектирование последовательностных схем с обратной связью	717
7.10.1. Зашелки	717
7.10.2. Составление таблицы потока для схемы классического образца	719
7.10.3. Минимизация таблицы потока	722
7.10.4. Кодирование состояний, гарантирующее отсутствие гонок	722
7.10.5. Уравнения возбуждения	726
7.10.6. Существенные источники опасности	727
7.10.7. Краткие выводы	730
7.11. Особенности проектирования последовательностных схем на языке ABEL	731

7.11.1. Регистровые выходы	731
7.11.2. Диаграммы состояний	733
7.11.3. Внешняя память состояния	739
7.11.4. Задание выходных сигналов автомата Мура	739
7.11.5. Задание сигналов на выходах типа Мили и на конвейерных выходах с помощью оператора WAIT	741
7.11.6. Проверочные векторы	744
7.12. Особенности проектирования последовательностных схем на языке VHDL	747
7.12.1. Последовательностные схемы с обратной связью	747
7.12.2. Тактируемые схемы	749
Обзор литературы	751
Упражнения	753
Задачи	757
Глава 8.	
Практическая разработка схем последовательностной логики	767
8.1. Стандарты документации на последовательностные схемы	767
8.1.1. Общие требования	767
8.1.2. Условные обозначения	768
8.1.3. Описание конечных автоматов	769
8.1.4. Временные диаграммы и временные параметры	770
8.2. Защелки и триггеры	775
8.2.1. Защелки и триггеры в ИС малой степени интеграции	775
8.2.2. Защита от дребезга при переключении	776
8.2.3. Простейшая схема защиты от дребезга	777
8.2.4. Шинный фиксатор уровня	779
8.2.5. Многоразрядные регистры и защелки	780
8.2.6. Описание регистров и защелок на языке ABEL и их реализация в ПЛУ	784
8.2.7. Описание регистров и защелок на языке VHDL	788
8.3. Последовательностные ПЛУ	792
8.3.1. Биполярные последовательностные ПЛУ	792
8.3.2. Последовательностные устройства типа GAL	796
8.3.3. Временные характеристики ПЛУ	801
8.4. Счетчики	804
8.4.1. Счетчики с последовательным переносом	805
8.4.2. Синхронные счетчики	806
8.4.3. Счетчики в ИС средней степени интеграции и их применение	807
8.4.4. Декодирование состояний двоичного счетчика	815
8.4.5. Описание счетчиков на языке ABEL и их реализация в ПЛУ	817
8.4.6. Описание счетчиков на языке VHDL	820
8.5. Регистры сдвига	825
8.5.1. Структура регистра сдвига	825
8.5.2. Регистры сдвига в ИС средней степени интеграции	827
8.5.3. Самое распространенное в мире применение регистров сдвига	832

8.5.4. Последовательно-параллельное преобразование	833
8.5.5. Счетчики на регистрах сдвига	838
8.5.6. Кольцевые счетчики	839
8.5.7. Счетчики Джонсона	842
8.5.8. Счетчики на регистрах сдвига с дуплексной обратной связью	845
8.5.9. Описание регистров сдвига на языке ABEL и их реализация в ПЛИУ	848
8.5.10. Описание регистров сдвига на языке VHDL	858
8.6. Итерационные и последовательностные схемы	863
8.7. Методология синхронного проектирования	866
8.7.1. Структура синхронной системы	866
8.7.2. Пример построения синхронной системы	870
8.8. Трудности синхронного проектирования	874
8.8.1. Разброс задержек тактового сигнала	874
8.8.2. Стробирование тактового сигнала	878
8.8.3. Асинхронные входы	880
8.9. Сбой в работе синхронизирующего устройства и метастабильность	883
8.9.1. Сбой в работе синхронизирующего устройства	883
8.9.2. Время выхода из метастабильности	885
8.9.3. Разработка надежного синхронизирующего устройства	885
8.9.4. Анализ времени пребывания в состоянии метастабильности	886
8.9.5. Более совершенные синхронизирующие устройства	888
8.9.6. Другие схемы синхронизирующих устройств	890
8.9.7. Триггеры с защитой от метастабильности	893
8.9.8. Синхронизация при высокоскоростной передаче данных	894
Обзор литературы	906
Упражнения	909
Задачи	912
Глава 9.	
Примеры проектирования последовательностных схем	921
9.1. Примеры проектирования на языке ABEL	922
9.1.1. Временные характеристики и компоновка конечных автоматов на основе ПЛИУ	922
9.1.2. Несколько простых автоматов	926
9.1.3. Задние огни автомобиля марки Ford Thunderbird	929
9.1.4. Игра на угадывание	931
9.1.5. Построим заново контроллер светофора!	935
9.2. Примеры проектирования на языке VHDL	940
9.2.1. Несколько простых автоматов	940
9.2.2. Задние огни автомобиля марки Ford Thunderbird	949
9.2.3. Игра на угадывание	951
9.2.4. Продолжение работы над контроллерами светофоров	953
Задачи	956

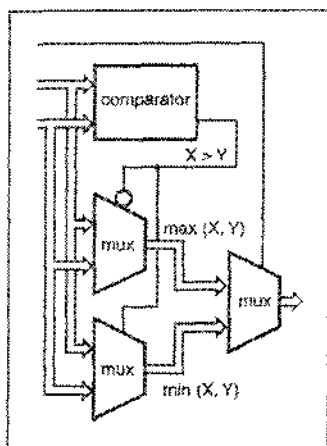
Глава 10.

Память и микросхемы типа CPLD и FPGA	959
10.1. Постоянные запоминающие устройства	960
10.1.1. Применение ПЗУ для реализации «произвольных» комбинационных логических функций	961
10.1.2. Внутренняя структура ПЗУ	965
10.1.3. Двумерное декодирование	967
10.1.4. Изготавливаемые серийно постоянные запоминающие устройства	970
10.1.5. Входы управления и временные параметры ПЗУ	974
10.1.6. Применения ПЗУ	977
10.2. Оперативные запоминающие устройства	983
10.3. Статические оперативные запоминающие устройства	984
10.3.1. Входы и выходы статического ОЗУ	984
10.3.2. Внутренняя структура статического ОЗУ	986
10.3.3. Временные параметры статического ОЗУ	988
10.3.4. Стандартные статические ОЗУ	990
10.3.5. Синхронные статические ОЗУ	992
10.4. Динамические оперативные запоминающие устройства	997
10.4.1. Структура динамического ОЗУ	997
10.4.2. Временные параметры динамического ОЗУ	1000
10.4.3. Синхронные динамические ОЗУ	1003
10.5. Интегральные схемы типа CPLD	1004
10.5.1. Семейство ИС XC9500 фирмы Xilinx	1006
10.5.2. Архитектура функционального блока	1008
10.5.3. Архитектура блока ввода/вывода	1012
10.5.4. Переключающая матрица	1013
10.6. Интегральные схемы типа FPGA	1016
10.6.1. Семейство ИС типа FPGA XC4000 фирмы Xilinx	1017
10.6.2. Перестраиваемый логический блок	1019
10.6.3. Блок ввода/вывода	1021
10.6.4. Программируемые соединения	1023
Обзор литературы	1027
Упражнения	1028
Задачи	1028

Глава 11.

Практические дополнения	1033
11.1. Средства автоматизированного проектирования	1033
11.1.1. Языки описания схем	1034
11.1.2. Ввод схемы	1034
11.1.3. Временные диаграммы и временные параметры	1037
11.1.4. Анализ схемы и моделирование	1037
11.1.5. Разработка печатной платы	1040

11.2. Проектирование, предусматривающее тестируемость	1041
11.2.1. Тестирование	1042
11.2.2. Тестер с игольчатыми контактами и внутрисхемное тестирование	1043
11.2.3. Методы сканирования	1047
11.3. Оценка надежности цифровой системы	1048
11.3.1. Интенсивность отказов	1050
11.3.2. Надежность и среднее время между отказами	1052
11.3.3. Надежность системы	1052
11.4. Длинные линии, отражения и согласованная нагрузка	1054
11.4.1. Основы теории длинных линий	1054
11.4.2. Передача логических сигналов по длинным линиям	1057
11.4.3. Согласованные нагрузки на концах линий передачи логических сигналов	1061
Обзор литературы	1063
Алфавитный указатель	1065



ПРИМЕРЫ ПРОЕКТИРОВАНИЯ КОМБИНАЦИОННЫХ СХЕМ

В предыдущих главах рассмотрены основные понятия, относящиеся к системам счисления, цифровым схемам и комбинационной логике, и описаны главные составные блоки комбинационных устройств: дешифраторы, мультиплексоры и т.п. Все это является необходимой основой. Но конечной целью при изучении цифровой электроники является способность решать реальные задачи, возникающие в процессе разработки цифровых систем. Как правило, для решения этих задач, помимо чтения учебника, требуется опыт. В этой главе мы пытаемся дать вам возможность продвинуться в этом направлении, разбирая примеры проектирования больших комбинационных схем.

Глава состоит из трех параграфов. Первый параграф содержит примеры проектирования путем использования стандартных комбинационных схем. Эти примеры рассматриваются в терминах функций, реализуемых посредством ИС средней степени интеграции. Но те же самые функции широко используются при разработке на основе специализированных ИС и микросхем типа FPGA. Основная мысль здесь состоит в том, чтобы показать, что требуемую комбинационную функцию часто можно реализовать, используя набор меньших по размерам стандартных блоков. Это важно по двум причинам: во-первых, иерархический подход обычно упрощает задачу проектирования в целом; во-вторых, меньшие стандартные блоки часто более эффективно и оптимально реализуются внутри ИС типа FPGA и в специализированных ИС по сравнению с тем, чего вы добились бы, описав требуемое устройство как единое целое и поручив программным средствам его синтезировать.

Во втором параграфе приведены примеры проектирования на языке ABEL. При этом предполагается, что реализация будет осуществляться на небольших ПЛУ типа 16V8 и 20V8. Помимо собственно использования языка ABEL, некоторые примеры служат иллюстрацией того, как лучше принимать решение о разбиении на части, когда проектируемое устройство целиком не помещается в одной интегральной схеме.

Третий параграф посвящен использованию языка VHDL, который лучше всего подходит для больших проектов, реализуемых в одной микросхеме типа CPLD или FPGA и в специализированной ИС. Обратите внимание, что в этих примерах не конкретизируется ИС, в которой должно быть реализовано разрабатываемое уст-

ройство. Несомненно, это — одно из достоинств проектирования на языках описания схем; большинство таких разработок, если не все, оказываются «переносимыми» и их можно реализовать на основе любой из множества технологий.

Единственным необходимым условием для изучения этой главы является знакомство с содержанием предшествующих глав. Параграфы этой главы в значительной степени независимы один от другого, поэтому вы можете не читать материал, относящийся к языку ABEL, если вас интересует только язык VHDL, и наоборот. Кроме того, остальная часть книги написана так, что вы можете прочесть эту главу сейчас или пропустить ее и вернуться к ней позже.

6.1. Примеры проектирования на основе стандартных блоков

6.1.1. Устройство быстрого сдвига

Устройство быстрого сдвига (barrel shifter) представляет собой комбинационную логическую схему с n входами данных, n выходами данных и несколькими управляющими входами, сигналы на которых задают сдвиг между данными на входе и данными на выходе. Устройство быстрого сдвига, являющееся частью микропроцессора, обычно характеризуется такими параметрами, как направление сдвига (влево или вправо), тип сдвига (циклический, арифметический или логический) и величина сдвига (обычно от 0 до $n-1$ разрядов, но иногда от 1 до n разрядов).

В этом разделе мы будем строить простое 16-разрядное устройство быстрого сдвига, осуществляющее только циклические сдвиги влево; величина сдвига пусть определяется сигналами, поступающими на 4-разрядный управляющий вход $S[3:0]$. Если, например, входное слово имеет вид: ABCDEFGHIJKLMNOP (где каждая буква представляет собой один бит), а сигнал управления равен 0101 (5), то выходное слово равно FGHIJKLMNOPABCDE.

На первый взгляд, решение этой задачи обманчиво просто. Каждый выходной бит можно получить с помощью 16-входового мультиплексора, на входы данных которого поступают соответствующие биты данных. Величина сдвига определяется сигналами на управляющих входах. Но, глядя на такую схему внимательнее, мы видим, что нужно идти на компромисс между быстродействием и размерами схемы.

Рассмотрим сначала варианты, в которых используются готовые мультиплексоры в виде ИС средней степени интеграции. Одноразрядный 16-входовой мультиплексор можно составить из двух ИС 74х151, используя сигнал на входе S_3 и его инверсию в качестве сигналов, поступающих на входы EN_L этих схем, и объединяя с помощью вентиля И-НЕ сигналы на выходах данных Y_L , так, как было показано на рис. 5.66 для 32-входового мультиплексора. Младшие разряды сигнала управления сдвигом S_2-S_0 подаются на одноименные входы выбора микросхем '151.

Задача будет решена, если мы 16 раз повторим этот 16-входовой мультиплексор и подключим входы данных согласно схеме на рис. 6.1. На верхние ИС '151 в каждой паре поступает сигнал разрешения S_3_L , а на нижние — сигнал разрешения S_3 ; остальные сигналы выбора подаются на все 32 микросхемы '151. На входы данных D_0-D_7 каждой из ИС '151 поступают сигналы DIN в том порядке, в каком они перечислены на рисунке слева направо.

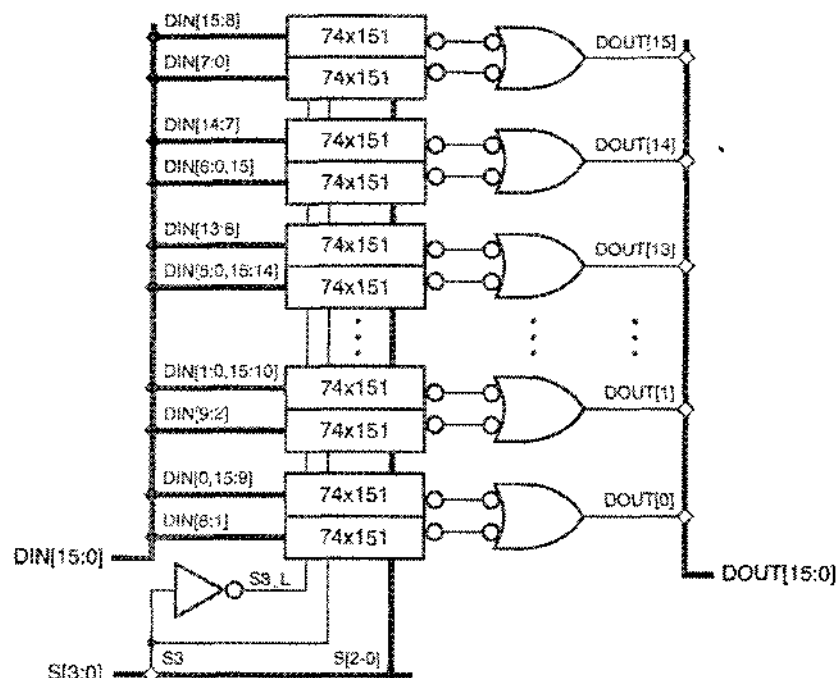


Рис. 6.1. Один из подходов к построению 16-разрядного устройства быстрого сдвига

В первой строке в табл. 6.1 приведены характеристики этого варианта. Для его реализации потребуется 36 микросхем средней и малой степени интеграции (точнее, чуть больше: 32 ИС 74x151, 4 ИС 74x00 и 1/6 ИС 74x04). Число микросхем можно уменьшить до 32, заменяя ИС 74x151 на ИС 74x251 и соединяя вместе их выходы Y с тремя состояниями; в результате получим то, что указано во второй строке таблицы. В каждом из этих вариантов оказывается очень сильно нагруженным источник управляющих сигналов: сигнал каждого разряда управляющего слова $S[2:0]$ необходимо подать на одноименный вход выбора всех 32 мультиплексоров. Входы данных также довольно сильно нагружают источники; сигнал каждого разряда данных должен быть подан на входы 16 мультиплексоров, соответствующих 16 возможным значениям

Табл. 6.1. Свойства четырех различных вариантов устройства быстрого сдвига

Используемый мультиплексор	Нагрузка по шине данных	Задержка данных	Нагрузка по шине управления	Число ИС
74x151	16	2	32	36
74x251	16	1	32	32
74x153	4	2	8	16
74x157	2	4	4	16

сдвига. Если предположить, что значительная нагрузка по шине управления и по шине данных не слишком сильно замедляет работу схемы, то вариант с микросхемами 74x251 дает наименьшую задержку по отношению к данным, поскольку сигнал в каждом разряде данных проходит лишь через один мультиплексор.

С другой стороны, можно создать устройство быстрого сдвига на 16 2-входовых 4-разрядных мультиплексорах 74x157; параметры такого устройства приведены в последней строке таблицы. Сначала данные проходят через 2-входовой 16-разрядный мультиплексор, образуемый первым набором из четырех микросхем 74x157 (рис. 6.2). В зависимости от значения управляющего сигнала S_0 входное слово оказывается сдвинутым влево на 0 разрядов или на 1 разряд. Выходы данных первых четырех мультиплексоров соединены с входами второго набора мультиплексоров, которые по сигналу S_1 сдвигают свое входное слово влево на 0 разрядов или на 2 разряда. Последовательно пропуская получающиеся слова через третий и четвертый наборы мультиплексоров, управляемые соответственно сигналами S_2 и S_3 , как показано на рис. 6.2, мы можем осуществить сдвиг на 4 и на 8 разрядов. Сигналы на входы 1A–4A и 1B–4B каждой ИС '157 подаются в том порядке, в каком они перечислены на схеме слева направо; то же самое относится к выходам 1Y–4Y каждой ИС.

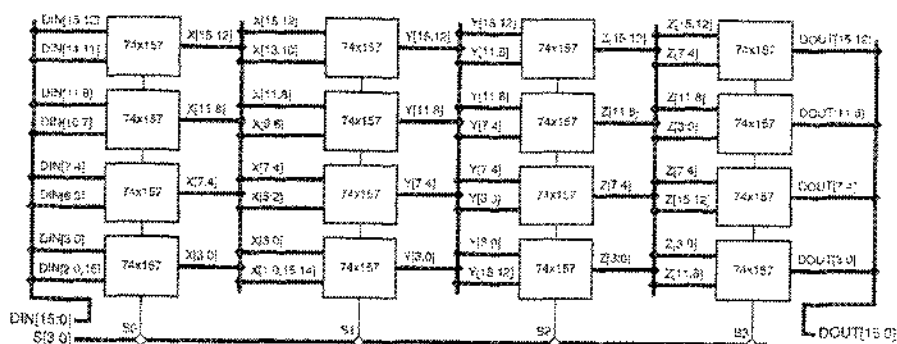


Рис. 6.2. Другой подход к построению 16-разрядного устройства быстрого сдвига

Для устройства на ИС '157 требуется вдвое меньшее число микросхем средней степени интеграции, и оно гораздо меньше нагружает источники управляющих сигналов и сигналов данных. Но такое решение дает наибольшую задержку в канале данных, так как сигнал каждого разряда данных должен пройти через четыре ИС 74x157.

Промежуточным между рассмотренными двумя подходами является вариант, основанный на использовании восьми 4-входовых 2-разрядных мультиплексоров 74x153 для реализации 4-входового 16-разрядного мультиплексирования. Два таких набора микросхем нужно включить последовательно один за другим, используя сигналы $S[3:2]$ для сдвига на 0, 4, 8 или 12 разрядов, а сигналы $S[1:0]$ — для сдвига на число разрядов от 0 до 3-х. Рабочие характеристики этого варианта представлены в третьей строке табл. 6.1. Если от вас не требуется достичь минимально возможной задержки данных, то последний вариант представляется наилучшим компромиссом.

Подобными соображениями можно руководствоваться и в том случае, когда устройство быстрого сдвига строится не на микросхемах средней степени интеграции, а из ячеек специализированной ИС, только вместо корпусов ИС средней и малой степени интеграции вам нужно будет подсчитывать площадь, занимаемую этими ячейками на поверхности кристалла.

Типичные библиотеки ячеек специализированных ИС содержат 1-разрядные мультиплексоры с числом входов от 2 до 8, обычно реализуемые на логических КМОП-ключах. Для построения мультиплексора больших размеров вам следует образовать необходимую комбинацию из ячеек меньших размеров. Помимо тех проблем, с которыми мы встретились при выборе подходящего варианта в примере построения устройства на ИС средней степени интеграции, в данном случае возникает еще одна трудность: задержки в КМОП-схемах сильно зависят от нагрузки. Поэтому, в зависимости от выбранного подхода, мы должны решить, где в цепях управляющих сигналов или в цепях сигналов данных – или в тех и в других – необходимо включить буферы, чтобы минимизировать задержки, обусловленные нагрузкой. Может оказаться, что схема, которая хорошо выглядит на бумаге до анализа этих задержек и введения буферов, в действительности будет иметь большую задержку или занимать большую площадь кристалла, чем схема, являющаяся реализацией другого варианта.

6.1.2. Простой шифратор для получения чисел с плавающей точкой

В предыдущем примере многократно повторялась одна и та же ИС – мультиплексор. То, что именно мультиплексор окажется подходящим блоком, было довольно очевидно с самого начала. Следующий пример показывает, что иногда бывает необходимо более пристально взглянуть на постановку задачи, чтобы увидеть решение на основе известных стандартных блоков.

Давайте рассмотрим такую задачу, решение которой с использованием ИС средней степени интеграции не совсем очевидно. Пусть нужно построить «шифратор, преобразующий числа с фиксированной точкой в числа с плавающей точкой». Целое двоичное число без знака B из интервала $0 \leq B < 2^{11}$ можно представить 11 битами в формате «с фиксированной точкой»: $B = b_{10}b_9 \dots b_1b_0$. Для представления чисел из того же диапазона с меньшей точностью в системе обозначений с плавающей точкой достаточно 7 битов: $F = M \cdot 2^E$, где M – 4-разрядная двоичная мантисса $m_3m_2m_1m_0$, а E – 3-разрядный двоичный показатель экспоненты $e_2e_1e_0$. Наименьшее целое число, представимое в этом формате, равно $0 \cdot 2^0$, а наибольшее число равно $(2^4 - 1) \cdot 2^7$.

Преобразование заданного 11-разрядного двоичного числа B с фиксированной точкой в 7-разрядное число с плавающей точкой можно выполнить «беря» из числа B четыре бита, начиная с самого старшего бита, равного 1. Например:

$$\begin{aligned} 11010110100 &= 1101 \cdot 2^7 + 0110100 \\ 00100101111 &= 1001 \cdot 2^5 + 01111 \\ 00000111110 &= 1111 \cdot 2^2 + 10 \\ 00000001011 &= 1011 \cdot 2^0 + 0 \\ 00000000010 &= 0010 \cdot 2^0 + 0. \end{aligned}$$

Последнее слагаемое в каждом равенстве справа представляет собой ошибку усечения в результате потери точности при преобразовании. Имея в виду такую процедуру преобразования, можно в следующем виде сформулировать требования, предъявляемые к устройству, преобразующему числа с фиксированной точкой в числа с плавающей точкой:

- комбинационная схема должна преобразовывать 11-разрядное двоичное целое число без знака B в 7-разрядное число с плавающей запятой M, E , где M и E являются 4-разрядным и 3-разрядным двоичными числами соответственно. Соотношение между числами имеет вид: $B = M \cdot 2^E + T$, где T — ошибка усечения, $0 \leq T < 2^E$.

Чтобы рационально построить схему, начиная с подобной постановки задачи, требуется творческий подход: технические требования не содержат никакой подсказки. Некоторые идеи могут появиться в результате более детального изучения того, как мы преобразовываем числа вручную. По существу, мы просматриваем каждое входное число слева направо, чтобы найти первый разряд, содержащий 1, и останавливаемся на разряде b_3 , если 1 не найдена. Мы выделяем четыре бита, начинающиеся с найденного разряда и используем их как мантиссу, а номером первого разряда определяется показатель экспоненты. Эти действия уже похожи на то, что выполняют стандартные блоки в виде ИС средней степени интеграции.

«Сканирование до первой 1» — это именно то, что делает универсальный приоритетный шифратор. На выходе приоритетного шифратора появляется число, говорящее нам о положении первой 1. Номером этого разряда определяется показатель экспоненты: когда первая единица находится в одном из разрядов $b_{10} \dots b_3$, это соответствует показателю экспоненты от 7 до 0; наличие первой единицы на позициях $b_2 \dots b_0$ или полное отсутствие единиц означает, что показатель экспоненты равен 0. Поэтому для нахождения первой 1 можно воспользоваться 8-входовым приоритетным шифратором, на входы которого с 17 (высший приоритет) по 10 подаются биты $b_{10} \dots b_3$. Сигналы, появляющиеся на выходах A2–A0 приоритетного шифратора, можно непосредственно использовать в качестве показателя экспоненты; если 1 не найдена, то A2–A0 = 000.

«Взятие четырех битов» напоминает процедуру «выбора» при мультиплексировании. 3-разрядный показатель экспоненты определяет, какие четыре бита числа B мы выбираем, поэтому биты показателя экспоненты можно использовать в качестве управляющих сигналов 8-входового 4-разрядного мультиплексора, на выход которого проходят нужные четыре бита числа B , образующие мантиссу M .

Шифратор на основе ИС средней степени интеграции, реализующий эти идеи, показан на рис. 6.3. В схеме осуществлена некоторая оптимизация:

- Так как у имеющегося приоритетного шифратора 74х148 активным является низкий уровень входных сигналов, предполагается, что входное число B поступает по шине $B_L[10:0]$ с низким активным уровнем сигналов. Если число B представлено сигналами с высоким активным уровнем, то можно воспользоваться восьмью инверторами для получения сигналов с низким активным уровнем.

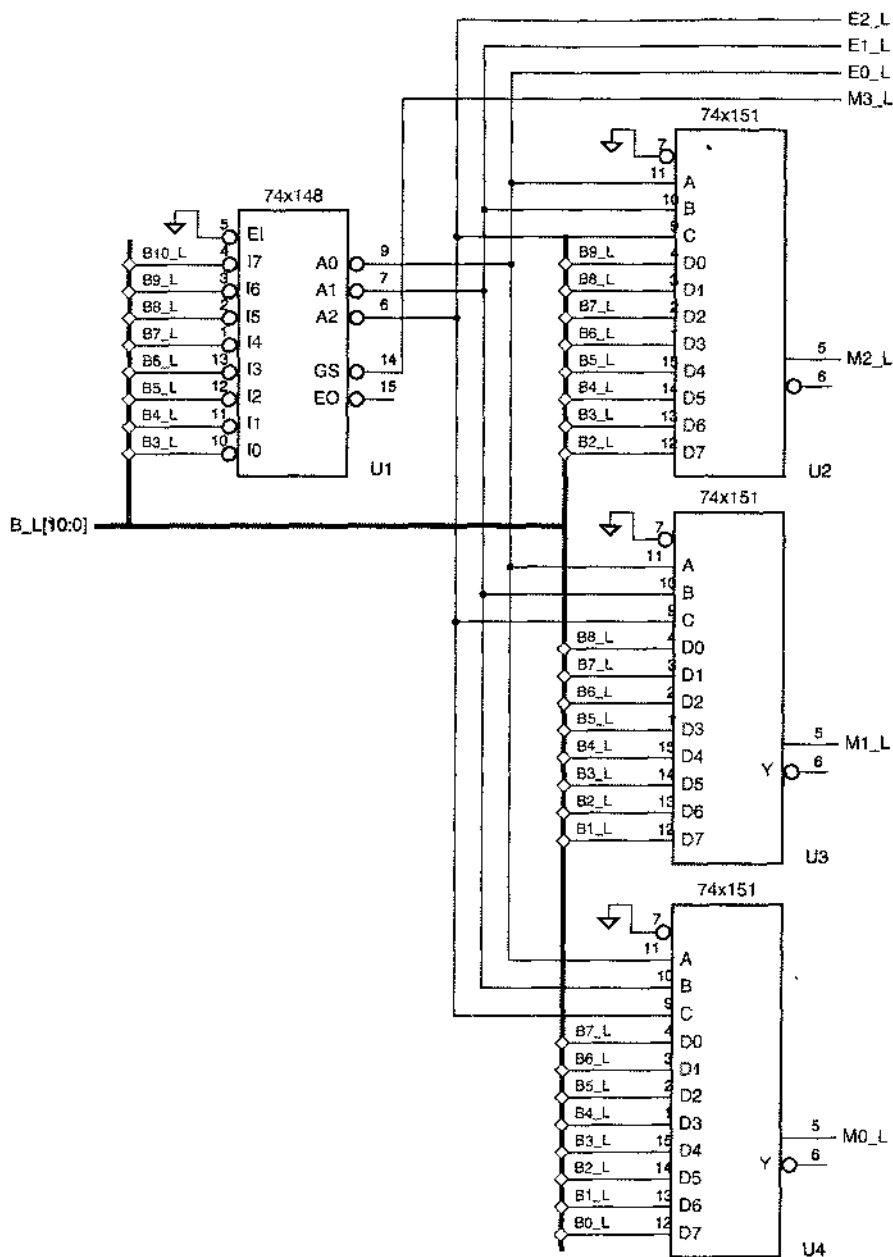


Рис. 5.3. Комбинационный цифратор, преобразующий числа с фиксированной точкой в числа с плавающей точкой

- Если вы внимательнее всмотритесь в процедуру преобразования, то поймете, что старший значащий бит мантиссы m_3 всегда равен 1, за исключением случая, когда 1 не пайдена. У ИС '148 есть выход GS_L, на котором сигнал возникает как раз в этом случае, что позволяет пам получить сигнал m_3 без мультиплексора.
- Выходные сигналы ИС '148 имеют низкий активный уровень, поэтому биты показателя экспоненты E0_L–E2_L также представлены сигналами с низким активным уровнем. Естественно, что для получения сигналов с высоким активным уровнем можно было бы воспользоваться тремя инверторами.
- Поскольку все сигналы имеют низкий активный уровень, биты мантиссы также представлены сигналами с низким активным уровнем, но на выходе EO_L ИС '148 и на выходах Y_L ИС '151 имеются также значения битов мантиссы, представленные сигналами с высоким активным уровнем.

Строго говоря, мультиплексоры изображены на рис. 6.3 не вполне корректно. Как показано на рис. 6.4, возможно другое условное обозначение ИС 74х151. Словами это можно выразить так: если входные данные мультиплексора имеют низкий активный уровень, то активный уровень данных на выходах противоположен тому, который указан в исходном условном обозначении. На рис. 6.3 следует предпочесть обозначение «активного низкого уровня данных», так как только в этом случае активные уровни сигналов на входах и выходах ИС '151 будут согласованы с названиями сигналов на этих выходах. Однако при передаче данных и при их хранении разработчики (и автор данной книги тоже) не всегда следуют этому правилу. Обычно бывает ясно из контекста, что при прохождении через мультиплексор и при хранении в многоразрядном регистре (рассматриваемом в разделе 8.2.5) активный уровень данных не изменяется.

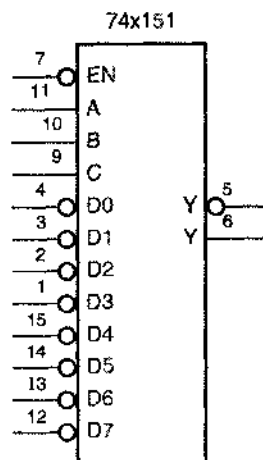


Рис. 6.4. Нетрадиционное условное обозначение 8-входового мультиплексора 74х151

6.1.3. Двойной приоритетный шифратор

Очень часто стандартные ИС средней степени интеграции не могут реализовать свои функции без помощи их «меньших братьев» – простых вентилях. В качестве следующего примера мы хотим построить приоритетный шифратор, который находит среди восьми сигналов запроса не только сигнал с наивысшим приоритетом, но также и сигнал со «вторым по старшинству приоритетом».

Предположим, например, что для входов запроса $R_L[0:7]$ активным является низкий уровень сигнала, причем входу R_L0 принадлежит высший приоритет. Пусть сигналы $A[2:0]$ и $AVALID$ указывают на запрос с наивысшим приоритетом. Уровень сигнала $AVALID$ активен только в том случае, когда присутствует хотя бы один запрос. Сигналы $B[2:0]$ и $BVALID$ пусть указывают на «второй по старшинству приоритета» запрос, причем уровень сигнала $BVALID$ становится активным только тогда, когда имеются, по крайней мере, два запроса.

Обнаружить запрос с наивысшим приоритетом довольно просто; достаточно воспользоваться ИС 74х148. Другая ИС '148 позволяет находить запрос со «вторым по старшинству приоритетом», но только при условии, что сначала мы «исключаем» запрос с высшим приоритетом на входе этой микросхемы. Это можно сделать с помощью дешифратора, выбирающего сигнал, который надо исключить, на основе сигналов $A[2:0]$ и $AVALID$, поступающих от первой ИС '148. Эти идеи воплощены в решении, приведенном на рис. 6.6. Активный уровень сигнала возникает не более чем на одном из восьми выходов дешифратора 74х138 – на том, который соответствует запросу с наивысшим приоритетом. Сигналы с выхода дешифратора поступают на входы вентилях И-НЕ, чтобы «исключить» запрос с наивысшим приоритетом.

Использованный прием позволяет получить на выходах ИС '148 сигналы с высоким активным уровнем, как это следует из рис. 6.5. Адресные выходы $A_L[2:0]$ можно переименовать так, чтобы они имели высокий активный уровень сигналов, если изменим также имя входа запроса, относящегося к каждой выходной комбинации. В частности, мы инвертируем биты номера запроса. В видоизмененном условном обозначении высший приоритет имеет сигнал запроса на входе 10.

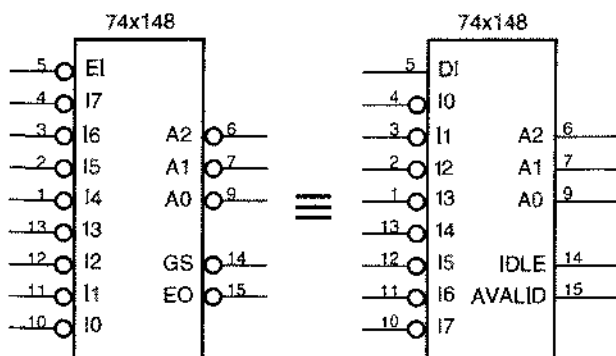


Рис. 6.5. Альтернативные условные обозначения 8-входового приоритетного шифратора 74х148

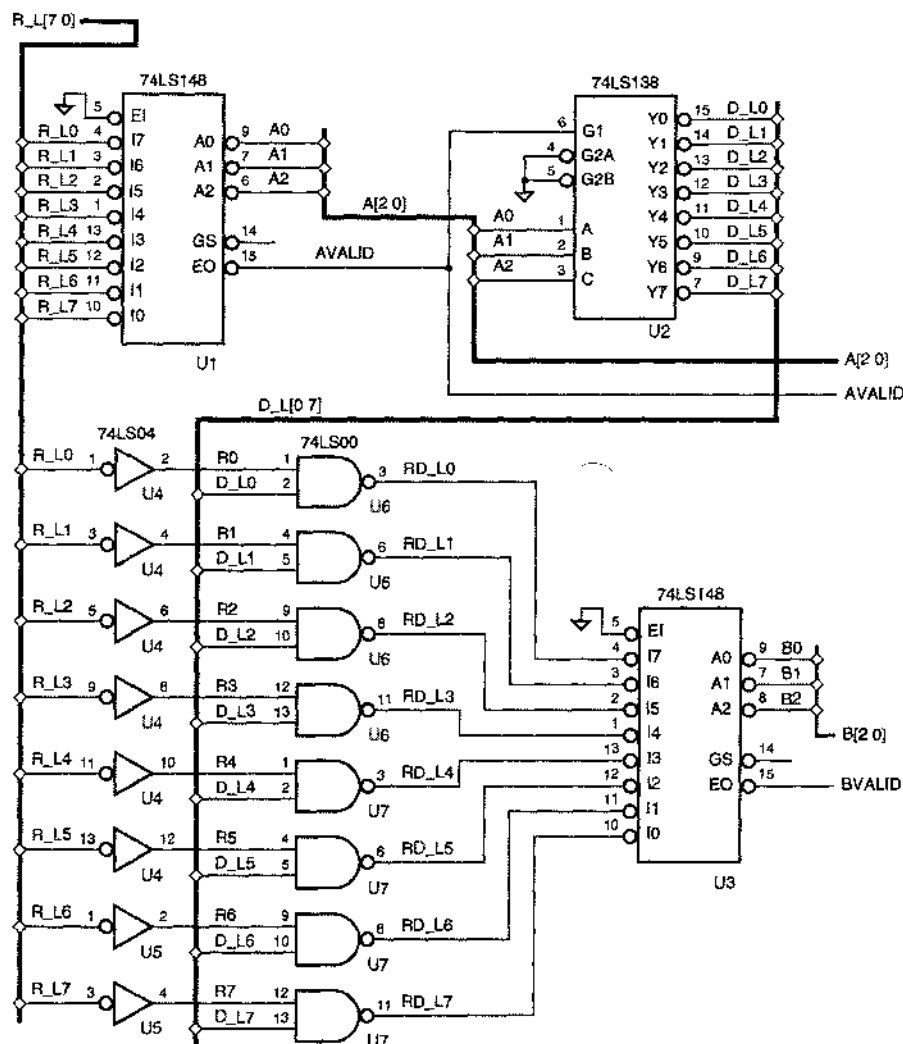


Рис. 6.6. Схема шифратора, обнаруживающего запросы с первым и вторым по старшинству приоритетами

6.1.4. Расширение компараторов

В разделе 5.9.4 мы показали, как можно строить большие компараторы путем каскадного включения 4-разрядных компараторов 74х85. Поскольку ИС 74х85 предусматривают их последовательное включение, на основе этих микросхем можно создавать сколь угодно большие компараторы. У 8-разрядного компаратора 74х682 вообще нет никаких входов и выходов для каскадного включения. Поэтому может показаться, на первый взгляд, что этой ИС нельзя воспользоваться для построения больших компараторов. Но это не так

Если вы задумаетесь о сущности сравнения многоразрядных слов, то станет ясно, что два многоразрядных операнда – скажем, по 32 бита (но четыре байта) в каждом – равны только в том случае, когда равны их соответствующие байты. Если при сравнении необходимо принимать решение вида «больше чем» или «меньше чем», то результат сравнения определяется по самым старшим из неравных байтов.

Реализацией этих идей служит схема (рис. 6.7), позволяющая обнаруживать равенство двух 24-разрядных операндов или выносить решение вида «больше чем» с помощью трех 8-разрядных компараторов 74х682. Сравнение 24-разрядных операндов осуществляется на основе результатов сравнения отдельных 8-разрядных слов; выходные сигналы формируются дополнительной комбинационной схемой согласно следующим равенствам:

$$PEQQ = EQ2 \cdot EQ1 \cdot EQ0$$

$$PGTQ = GT2 + EQ2 \cdot GT1 + EQ2 \cdot EQ1 \cdot GT0$$

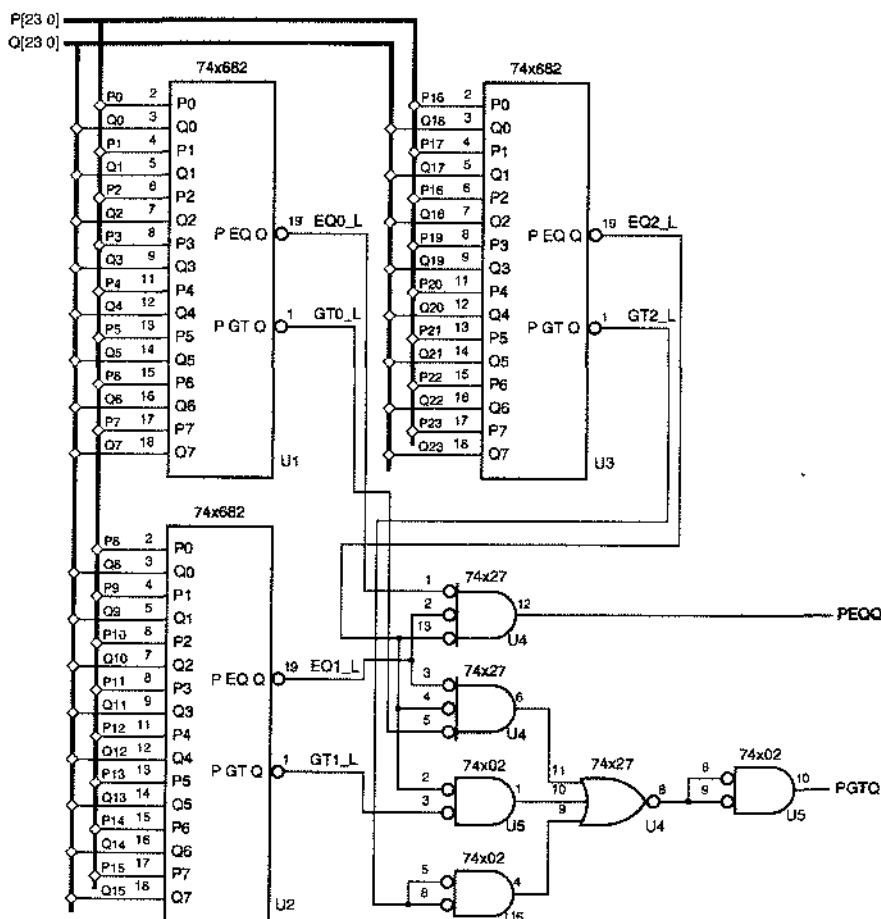


Рис. 6.7. Схема 24-разрядного компаратора

Такой «параллельный» подход к расширению компаратора в действительности дает большее быстроедействие, нежели последовательное включение ИС 74х85, потому что исключается задержка распространения сигналов в каждом каскаде по нути от входов до выходов, с помощью которых компараторы соединяются один за другим. Параллельный подход и двухуровневая логика И-ИЛИ для объединения результатов сравнения 8-разрядных слов позволяют создавать компараторы с очень большим числом входов, которое ограничено только коэффициентом объединения по входу схем в логике И-ИЛИ. Применяя для объединения дополнительные логические схемы, можно строить сколь угодно большие компараторы.

6.1.6. Компаратор с управляемым режимом работы

Довольно часто требования, предъявляемые к цифровой схеме делают очевидным решение поставленной задачи на основе ИС средней степени интеграции или других стандартных блоков. Рассмотрим, например, такую задачу:

- Построить комбинационную схему, на входы которой подаются два 8-разрядных целых двоичных числа без знака X и Y и сигнал управления MIN/MAX , а на выходе возникает 8-разрядное целое двоичное число без знака Z , такое что $Z = \min(X, Y)$, если $MIN/MAX = 1$, и $Z = \max(X, Y)$, если $MIN/MAX = 0$.

Нетрудно представить себе эту схему составленной из ИС средней степени интеграции. Очевидно, что для определения, какое из чисел X или Y больше, можно воспользоваться компаратором. Сигнал с выхода компаратора может управлять мультиплексорами, на выходах которых будут вырабатываться сигналы $\min(X, Y)$ и $\max(X, Y)$, а с помощью другого мультиплексора можно выбрать один из этих результатов в зависимости от значения сигнала MIN/MAX . Блок-схема устройства, в котором реализован этот подход, приведена на рис. 6.8(а).

НЕ СЛЕДУЙТЕ СЛЕПО ЗА РЕКЛАМОЙ!

Расточительность исходного варианта, представленного на рис. 6.8(а), возможно, была очевидна для вас с самого начала, но он демонстрирует важный принцип проектирования на основе стандартных блоков:

- Для обработки данных используйте стандартные блоки и ищите способ заставить одни и те же блоки в разное время выполнять различные функции или работать в различных режимах. По мере надобности создавайте схемы управления для выбора соответствующих функций, чтобы уменьшить общее число компонентов в устройстве.

Как впечатляюще показано на рис. 6.8(с), этот подход позволяет сэкономить много корпусов. При проектировании на основе интегральных микросхем *не следует* поддаваться рекламе: «У нас есть все, что вам нужно, и даже больше»!

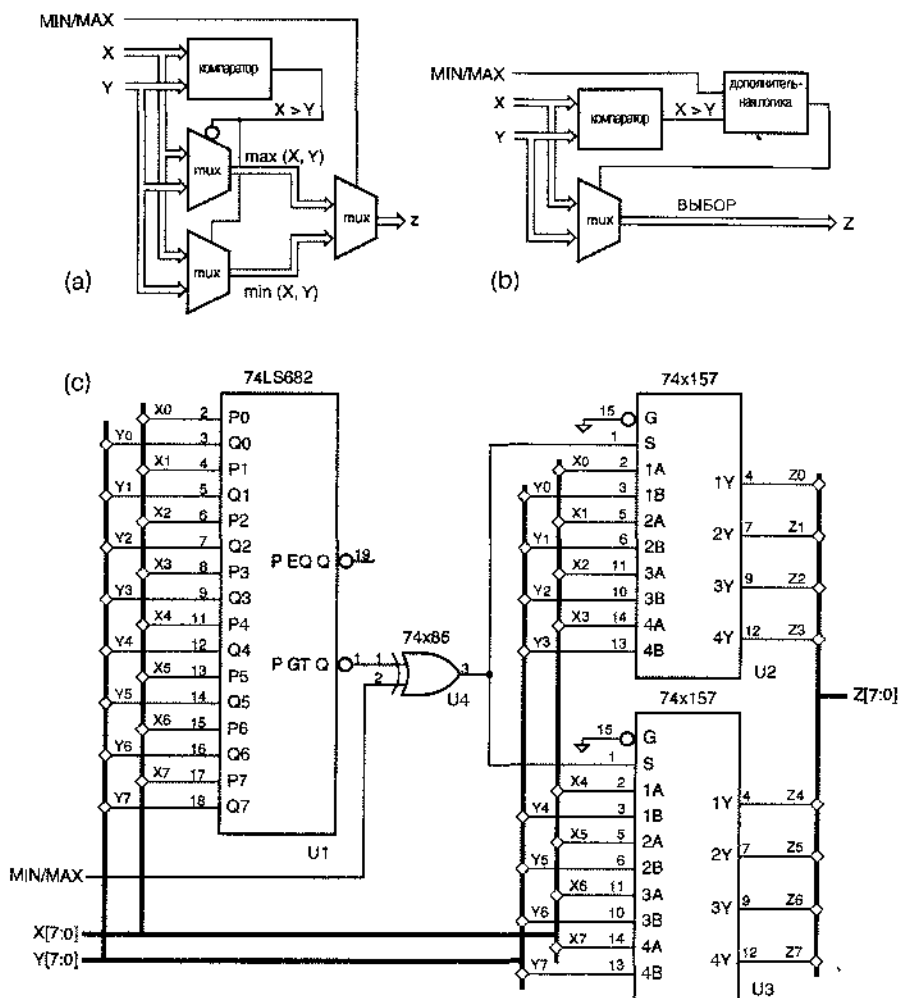


Рис. 6.8. Схема компаратора с управляемым режимом работы: (а) блок-схема первого приходящего на ум решения (mux – мультиплексор); (б) блок-схема более рационального решения с точки зрения стоимости; (с) принципиальная схема для варианта на рис. (б)

Наше первое решение достигает цели, но оно дороже, чем могло бы быть. Хотя схема содержит три двухвходовых мультиплексора, в конце концов нужно выбрать и пропустить на выход только одно из двух имеющихся входных слов X и Y. Поэтому задача состоит в том, чтобы настроить схему, в которой решение о выборе одного из двух входных слов осуществлялось бы единственным двухвходовым мультиплексором и некоторой дополнительной логикой. Иллюстрацией такого подхода служат схемы на рис. 6.8(б) и (с). «Дополнительная логика» оказывается совсем простой: это всего лишь единственный вентиль ИСКЛЮЧАЮЩЕ ИЛИ-НЕ.

6.2. Примеры проектирования схем с использованием языка ABEL и их реализация в ПЛУ

6.2.1. Устройство быстрого сдвига

Устройство быстрого сдвига, рассмотренное в разделе 6.1.1, является хорошим примером проектирования некоторой схемы, когда *не используются* ПЛУ. Однако для описания подобного устройства удобно воспользоваться языком ABEL, и мы увидим, почему ПЛУ не вполне подходит для реализации типичного устройства быстрого сдвига.

В табл. 6.2 приведены равенства для 16-разрядного устройства быстрого сдвига с теми же самыми функциональными возможностями, что и в примере в разделе 6.1.1. В этом устройстве осуществляются только циклические сдвиги влево, а величина сдвига определяется сигналами, поступающими на 4-разрядный управляющий вход S[3...0]. Язык ABEL позволяет легко описать в целом выполняемые схемой функции, не заботясь о том, как схема могла бы быть разбита на несколько отдельных микросхем. Кроме того, компилятор языка ABEL послушно находит минимальное выражение вида «сумма произведений» для каждого выходного бита. В данном случае каждому выходному сигналу необходимы 16 термов-произведений.

Табл. 6.2. Программа для 16-разрядного устройства быстрого сдвига на языке ABEL

```

module barrel16
title '16-bit Barrel Shifter'

" Inputs and Outputs
DIN15..DIN0, S3..S0           pin;
DOUT15..DOUT0                pin istype 'com';

S = [S3..S0];

equations

[DOUT15..DOUT0] = (S==0) & [DIN15..DIN0]
                # (S==1) & [DIN14..DIN0,DIN15]
                # (S==2) & [DIN13..DIN0,DIN15..DIN14]
                # (S==3) & [DIN12..DIN0,DIN15..DIN13]
                ...
                # (S==12) & [DIN3..DIN0,DIN15..DIN4]
                # (S==13) & [DIN2..DIN0,DIN15..DIN3]
                # (S==14) & [DIN1..DIN0,DIN15..DIN2]
                # (S==15) & [DIN0,DIN15..DIN1];

end barrel16

```

Разбиение 16-разрядного устройства быстрого сдвига на составные части для реализации в нескольких ПЛУ является трудной задачей по двум причинам. Во-первых, очевидно, что природа этой функции такова, что каждый выходной бит зависит от каждого входного бита. ПЛУ, которое вырабатывает, скажем, выход DOUT0, должно иметь доступ ко всем 16 входам DIN и ко всем четырем входам S. Поэтому явно нельзя использовать ИС GAL16V8, у нее только 16 входов.

ИС GAL20V8 подобна ИС GAL16V8, но имеет четыре дополнительных вывода, работающих только на вход. Если мы воспользуемся всеми 20 имеющимися входами, то останутся два вывода (эквивалентные верхнему и нижнему выводам на рис. 5.27), работающие только на выход. Таким образом, кажется возможным реализовать устройство быстрого сдвига на восьми микросхемах 20V8, вырабатывая по два выходных бита в одной микросхеме.

Но это не все. Вторая причина затруднений при попытке реализовать устройство быстрого сдвига в ПЛУ кроется в числе термов-произведений, приходящихся на один выход. Для построения устройства быстрого сдвига требуется 16 термов-произведений, а в ИС 20V8 их только 7. Мы зашли в тупик, любая реализация устройства быстрого сдвига на основе ИС 20V8 приводит к необходимости применять логику с несколькими проходами. В этой ситуации имеет смысл подумать о разделении, аналогичном тому, какое мы производили в разделе 6.1.1.

16-разрядное устройство быстрого сдвига нетрудно реализовать в более крупном программируемом устройстве, то есть в ИС типа CPLD или FPGA с достаточным количеством выводов «вход/выход». Однако представьте себе, какие сложности возникнут при проектировании 32-разрядного или 64-разрядного устройства быстрого сдвига. Ясно, что нам понадобится ИС с еще большим числом выводов «вход/выход», но и это еще не все. Из-за большого числа термов-произведений и множества связей (все входы соединяются со всеми выходами) нам по-прежнему придется «пошевелить мозгами».

Действительно, типичной программе компоновки в случае ИС типа CPLD или FPGA будет трудно реализовать большое устройство быстрого сдвига с малой задержкой, и она может вообще не справиться с этой задачей. При разбиении устройства быстрого сдвига на стандартные блоки в разделе 6.1.1 мы считали само собой разумеющейся возможность осуществления необходимых соединений! В ИС типа FPGA возможности образования внутренних связей ограничены, а у ИС типа CPLD эти ограничения еще больше. Следовательно, даже в том случае, когда вы пользуетесь современными средствами проектирования применительно к ИС типа FPGA и CPLD, вам все же придется «поработать головой», чтобы в какой-то степени разбить схему на части и таким образом помочь программным средствам справиться со своей работой.

Устройства быстрого сдвига могут быть даже более сложными, чем те, о которых мы говорили до сих пор. Только ради шутки в табл. 6.3 приведена программа для устройства быстрого сдвига, которое выполняет шесть различных видов сдвига. Для его реализации требуется даже большее число термов-произведений, до 40 на выход! Хотя вы никогда не стали бы реализовать такое устройство на основе ПЛУ, ИС типа CPLD или на небольшой ИС типа FPGA, минимизированные равенства, выдаваемые компилятором языка ABEL, все же полезны, потому что позволяют понять, к чему ведет тот или иной выбираемый вами вариант. Например, изменяя кодирование SLA и

SRA на [1, .X., 0] и [1, .X., 1], можно сократить общее число термов-произведений в устройстве с 624 до 608. Число термов-произведений можно еще уменьшить, изменяя — для некоторых сдвигов — представление числа, задающего величину сдвига (см. задачу 6.3). Эти приемы экономии, достигаемой подобными изменениями, можно перенести и на другие подходы к проектированию.

Табл. 6.3. Программа для многорежимного 16-разрядного устройства быстрого сдвига на языке ABEL

```

module barr16f
Title 'Multi-mode 16-bit Barrel Shifter'

" Inputs and Outputs
DIN15..DINO, S3..S0, C2..C0      pin;
DOUT15..DOUT0                  pin istype 'com';

S = [S3..S0]; C = [C2..C0]; " Shift amount and mode
L = DIN15; R = DINO;         " MSB and LSB

ROL = (C == [0,0,0]); " Rotate (circular shift) left
ROR = (C == [0,0,1]); " Rotate (circular shift) right
SLL = (C == [0,1,0]); " Shift logical left (shift in 0s)
SRL = (C == [0,1,1]); " Shift logical right (shift in 0s)
SLA = (C == [1,0,0]); " Shift left arithmetic (replicate LSB)
SRA = (C == [1,0,1]); " Shift right arithmetic (replicate MSB)

equations

[DOUT15..DOUT0] = ROL & (S==0) & [DIN15..DINO]
# ROL & (S==1) & [DIN14..DINO,DIN15]
# ROL & (S==2) & [DIN13..DINO,DIN15..DIN14]
...
# ROL & (S==15) & [DINO,DIN15..DIN1]
# ROR & (S==0) & [DIN15..DINO]
# ROR & (S==1) & [DINO,DIN15..DIN1]
...
# ROR & (S==14) & [DIN13..DINO,DIN15..DIN14]
# ROR & (S==15) & [DIN14..DINO,DIN15]
# SLL & (S==0) & [DIN15..DINO]
# SLL & (S==1) & [DIN14..DINO,0]
...
# SLL & (S==14) & [DIN1..DINO,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
# SLL & (S==15) & [DINO,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0]
# SRL & (S==0) & [DIN15..DINO]
# SRL & (S==1) & [0,DIN15..DIN1]
...
# SRL & (S==14) & [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,DIN15..DIN14]
# SRL & (S==15) & [0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,DIN15]
# SLA & (S==0) & [DIN15..DINO]
# SLA & (S==1) & [DIN14..DINO,R]
...
# SLA & (S==14) & [DIN1..DINO,R,R,R,R,R,R,R,R,R,R,R,R,R,R,R]
# SLA & (S==15) & [DINO,R,R,R,R,R,R,R,R,R,R,R,R,R,R,R]
# SRA & (S==0) & [DIN15..DINO]
# SRA & (S==1) & [L,DIN15..DIN1]
...
# SRA & (S==14) & [L,L,L,L,L,L,L,L,L,L,L,L,L,L,L,DIN15..DIN14]
# SRA & (S==15) & [L,L,L,L,L,L,L,L,L,L,L,L,L,L,L,DIN15];

end barr16f

```

6.2.2. Простой шифратор для получения чисел с плавающей точкой

В разделе 6.1.2 мы ввели простой формат числа с плавающей точкой и сформулировали задачу построения преобразователя числа с фиксированной точкой в число с плавающей точкой. Требования этой схемы ограничиваются 11 входами и 7 выходами, так что потенциально для замены четырех микросхем средней степени интеграции можно использовать единственное ПЛУ.

Программа на языке ABEL для преобразования чисел с фиксированной точкой в числа с плавающей точкой представлена в табл. 6.4. Оператором WHEN определяется показатель экспоненты E вполне естественным образом. Затем значение E используется для выбора соответствующих битов числа B в качестве мантиссы M. Несмотря на большую глубину вложений в операторе WHEN, в минимальной сумме для каждого бита показателя экспоненты E необходимы только четыре термина-произведения. Равенства, определяющие биты мантиссы M, также не слишком плохи: в каждом требуется только восемь термов-произведений. К сожалению,

```

module fpenc
title 'Fixed-point to Floating-point Encoder'
FPENC device 'P20L8';

" Input and output pins
B10..B0      pin 1..11;
E2..E0, M3..M0  pin 21..15 istype 'com';

" Constant expressions
B = [B10..B0];
E = [E2..E0];
M = [M3..M0];

equations

WHEN B < 16 THEN E = 0;
ELSE WHEN B < 32 THEN E = 1;
ELSE WHEN B < 64 THEN E = 2;
ELSE WHEN B < 128 THEN E = 3;
ELSE WHEN B < 256 THEN E = 4;
ELSE WHEN B < 512 THEN E = 5;
ELSE WHEN B < 1024 THEN E = 6;
ELSE E = 7;

M = (E==0) & [B3..B0]
# (E==1) & [B4..B1]
# (E==2) & [B5..B2]
# (E==3) & [B6..B3]
# (E==4) & [B7..B4]
# (E==5) & [B8..B5]
# (E==6) & [B9..B6]
# (E==7) & [B10..B7];

end fpenc

```

Табл. 6.4. Программа на языке ABEL для шифратора, преобразующего числа с фиксированной точкой в числа с плавающей точкой, на основе ПЛУ

ИС GAL20V8 позволяет получить только семь термов-произведений на выход. Однако в ИС GAL22V10 (см. рис. 8.22) можно реализовать большее число термов-произведений, так что при желании можно воспользоваться ею.

Недостаток проекта, представленного в табл. 6.4, состоит в том, что выходы [M3..M0] медленные: для получения сигналов на этих выходах используются сигналы с выходов [E2..E0], поэтому необходимы два прохода через ПЛУ. Схема могла бы обладать большим быстродействием, если только она влезает в выбранную ИС, в результате записи «термов выбора» ($E=0$ и др.) в виде промежуточных соотношений до раздела равенств, что позволило бы компилятору языка ABEL развернуть получающиеся равенства для битов мантиссы M и представить их одноуровневой логикой. К сожалению, язык ABEL не допускает наличия операторов WHEN вне раздела equations, так что нам придется «засучить рукава» и написать свои собственные логические выражения в промежуточных равенствах.

Видоизмененный вариант программы приведен в табл. 6.5. Выражения для S7–S0 являются взаимоисключающими И-термами; ими определяется значение показателя экспоненты от 7 до 0 в зависимости от расположения старшего единичного бита в числе с фиксированной точкой на входе. Показатель экспоненты [E2..E0] представляет собой двоичную запись значений термов выбора, а биты мантиссы [M3..M0] в каждом случае определяются с учетом значений термов выбора. Оказывается, тем не менее, что равенства для каждого бита мантиссы M содержат 8 термов-произведений, но этот вариант по крайней мере намного быстрее, так как используется только одноуровневая логика.

```

module fpenc
title 'Fixed-point to Floating-point Encoder'
FPENCE device 'P20L8';

" Input and output pins
B10..B0                                pin 1..11;
E2..E0, M3..M0                         pin 21..15 astype 'com';

" Intermediate equations
S7 = B10;
S6 = !B10 & B9;
S5 = !B10 & !B9 & B8;
S4 = !B10 & !B9 & !B8 & B7;
S3 = !B10 & !B9 & !B8 & !B7 & B6;
S2 = !B10 & !B9 & !B8 & !B7 & !B6 & B5;
S1 = !B10 & !B9 & !B8 & !B7 & !B6 & !B5 & B4;
S0 = !B10 & !B9 & !B8 & !B7 & !B6 & !B5 & !B4;

equations

E2 = S7 # S6 # S5 # S4;
E1 = S7 # S6 # S3 # S2;
E0 = S7 # S5 # S3 # S1;

[M3..M0] = S0 & [B3..B0] # S1 & [B4..B1] # S2 & [B5..B2]
          # S3 & [B6..B3] # S4 & [B7..B4] # S5 & [B8..B5]
          # S6 & [B9..B6] # S7 & [B10..B7];

end fpenc

```

Табл. 6.5. Другой вариант программы на языке ABEL для преобразователя чисел с фиксированной точкой в числа с плавающей точкой на основе ПЛУ

6.2.3. Двойной приоритетный шифратор

Этот пример демонстрирует проектирование на основе ПЛУ двойного приоритетного шифратора, который находит сигнал активного уровня с самым высоким приоритетом и сигнал со вторым по старшинству приоритетом во входном наборе из восьми сигналов запроса с высоким активным уровнем, названных [R0..R7], где сигнал R0 имеет высший приоритет. Выходы [A2..A0] и AVALID будем использовать для представления запроса с самым высоким приоритетом, причем на выходе AVALID сигнал возникает только тогда, когда существует запрос с наивысшим приоритетом. Точно так же [B2..B0] и BVALID представляют запрос со вторым по старшинству приоритетом.

Программа для приоритетного шифратора на языке ABEL представлена в табл. 6.6. Как обычно, вложенный оператор WHEN идеален для описания процедуры определения приоритетов. Чтобы найти активизированный вход со вторым по

```

title 'Dual Priority Encoder'
PRIORTWO device 'P16V8';

" Input and output pins
R7..R0                                pin 1..8;
AVALID, A2..A0, BVALID, B2..B0       pin 19..12 istype 'com';

" Set definitions
A = [A2..A0]; B = [B2..B0];

equations

WHEN R0==1 THEN A=0;
ELSE WHEN R1==1 THEN A=1;
ELSE WHEN R2==1 THEN A=2;
ELSE WHEN R3==1 THEN A=3;
ELSE WHEN R4==1 THEN A=4;
ELSE WHEN R5==1 THEN A=5;
ELSE WHEN R6==1 THEN A=6;
ELSE WHEN R7==1 THEN A=7;

AVALID = ((R7..R0) != 0);

WHEN (R0==1) & (A!=0) THEN B=0;
ELSE WHEN (R1==1) & (A!=1) THEN B=1;
ELSE WHEN (R2==1) & (A!=2) THEN B=2;
ELSE WHEN (R3==1) & (A!=3) THEN B=3;
ELSE WHEN (R4==1) & (A!=4) THEN B=4;
ELSE WHEN (R5==1) & (A!=5) THEN B=5;
ELSE WHEN (R6==1) & (A!=6) THEN B=6;
ELSE WHEN (R7==1) & (A!=7) THEN B=7;

BVALID = (R0==1) & (A!=0) # (R1==1) & (A!=1)
# (R2==1) & (A!=2) # (R3==1) & (A!=3)
# (R4==1) & (A!=4) # (R5==1) & (A!=5)
# (R6==1) & (A!=6) # (R7==1) & (A!=7);

end priortwo

```

Табл. 6.6.
Программа
на языке
ABEL для
двойного
приоритет-
ного шиф-
ратора

старшинству приоритетом, мы исключаем тот вход, номер которого соответствует номеру А активизированного входа с самым высоким приоритетом. Таким образом, для определения выхода В используется логика с двумя проходами. Легко написать равенство для выхода AVALID: AVALID равняется 1, если не на всех входах запроса сигнал равен 0. Чтобы найти значение BVALID, нужно логикой ИЛИ объединить все условия, при которых в операнде WHEN присваивается то или иное значение выходу В.

Даже при использовании логики с двумя проходами для получения выходных сигналов В требуется слишком много термов-произведений, чтобы разместить их в ИС 16V8; в табл. 6.7 приведены сведения о числе необходимых термов-произведений. Выходу В нужно слишком много термов, даже если иметь в виду реализацию в ИС 22V10, у которой имеется 16 термов для двух из ее выходов и от 8 до 14 термов — для других. Так что вам придется усердно потрудиться, чтобы поместить в ИС все, что нужно!

P-Terms	Fan-in	Fan-out	Type	Name
8/1	8	1	Pin	AVALID
4/5	8	1	Pin	A2
4/5	8	1	Pin	A1
4/5	8	1	Pin	A0
24/8	11	1	Pin	BVALID
24/17	11	1	Pin	B2
20/21	11	1	Pin	B1
18/22	11	1	Pin	B0

Табл. 6.7. Использование термов-произведений в двойном приоритетном шифраторе, реализуемом в ПЛУ

=====
 106/84 Best P-Term Total: 76
 Total Pins: 16
 Average P-Term/Output: 9

СУММЫ ПРОИЗВЕДЕНИЙ И ПРОИЗВЕДЕНИЯ СУММ (БЫСТРО ПОВТОРИТЕ ЭТО 5 РАЗ)

Возможно, вы помните из раздела 4.3.6, что минимальное выражение вида «произведение сумм» для исходной функции можно получить путем преобразований по теореме Де Моргана из минимального выражения вида «суммы произведений» для дополнения этой функции. Возможно, вы помните также, что число термов-произведений в минимальной сумме произведений может отличаться от числа термов-сумм в минимальном произведении сумм. В столбце «P-Terms» в табл. 6.7 приведено число термов в обеих минимальных формах (термы-произведения /термы-суммы). Если число термов в какой-либо из двух минимальных форм меньше или равно числу термов, допускаемых матрицей вентилях И-ИЛИ в ИС 22V10, то с помощью этой микросхемы функцию реализовать можно.

Так как же можно уменьшить число необходимых термов-произведений? Обратите внимание на одну важную вещь: R0 никогда не может быть входом со вторым по старшинству приоритетом, и поэтому В никогда не может равняться 0. Таким образом, случай R0==1 можно исключить из оператора WHEN. Это изменение приводит к сокращению числа термов для B2-B0 до 14, 17 и 15 соответственно. Если бы только знать, как исключить один терм из выражения для B1, то наш шифратор, по-видимому, можно было бы разместить в ИС 22V10.

Хорошо, давайте попробуем сделать кое-что еще. Если R0==1, то во втором предложении в операторе WHEN излишне использовать все, что мы знаем. Нам не нужна абсолютной общности, выражаемой условием A!=1; этот случай важен только тогда, когда R0=1. Поэтому давайте заменим первые две строки исходного оператора WHEN на

```
WHEN (R1==1) & (R0==1) THEN B=1;
```

Эта хитрая замена уменьшает минимальное число термов для B2-B0 до 12, 16 и 13 соответственно. Мы добились своего! А можно ли еще уменьшить число термов-произведений так, чтобы вписаться в ИС 16V8, сохранив те же самые функциональные возможности? Такое кажется невероятным, но мы оставим это читателю в качестве задачи 6.4!

6.2.4. Расширение компараторов

В разделе 5.9.5 было показано, что в ПЛУ легко реализовать устройство, определяющее равенство двух операндов, но сравнение по величине (принятие решения вида «больше чем» или «меньше чем») даже при небольшом числе разрядов плохо реализуется в ПЛУ из-за большого числа требуемых термов-произведений. Таким образом, большие компараторы лучше строить на основе компараторов, выполненных в виде отдельных микросхем средней степени интеграции, или на основе готовых узлов, имеющихся в библиотеках специализированных ИС или ИС типа FPGA. Однако ПЛУ, как мы сейчас покажем, хорошо подходят для реализации комбинационной логики, используемой при создании многоразрядных компараторов путем «параллельного расширения» компараторов меньших размеров.

В разделе 5.9.4 мы показали, как последовательно включаемые 4-разрядные компараторы 74x85 образуют компараторы с большим числом разрядов. Хотя для построения сколь угодно больших компараторов схема с последовательным включением небольших компараторов не требует никакой дополнительной логики, у этого варианта есть существенный недостаток, состоящий в том, что задержка растет линейно с увеличением числа каскадов.

С другой стороны, в разделе 6.1.4, было показано, что для выполнения 24-разрядного сравнения можно параллельно включить 8-разрядные компараторы 74x682 и применить комбинационную логику. Эту схему можно расширить на случай сравнения операндов любого размера.

В табл. 6.8 приведена программа на языке ABEL для ИС GAL22V10, посредством которой объединяются выходные сигналы «равно» (EQ) и «больше чем» (GT) восьми ИС 74x682 и вырабатываются сигналы для всех шести возможных соотношений (=, ≠, >, ≥, <, ≤) между двумя сравниваемыми 64-разрядными операндами.

Табл. 6.8. Программа на языке ABEL для объединения восьми ИС 74х682 в 64-разрядный компаратор

```

module compexp
title 'Expansion logic for 64-bit comparator'
COMPEXP device 'P22V10';

" Inputs from the individual comparators, active-low, 7 = MSByte
EQ_L7..EQ_L0, GT_L7..GT_L0                pin 1..11, 13..14, 21..23;

" Comparison outputs
PEQQ, PNEQ, PGTQ, PQEQ, PLTQ, PLEQ        pin 15..20 istype 'com';

" Active-level conversions
EQ7 = !EQ_L7; EQ6 = !EQ_L6; EQ5 = !EQ_L5; EQ4 = !EQ_L4;
EQ3 = !EQ_L3; EQ2 = !EQ_L2; EQ1 = !EQ_L1; EQ0 = !EQ_L0;
GT7 = !GT_L7; GT6 = !GT_L6; GT5 = !GT_L5; GT4 = !GT_L4;
GT3 = !GT_L3; GT2 = !GT_L2; GT1 = !GT_L1; GT0 = !GT_L0;

" Less-than terms
LT7 = !(EQ7 # GT7); LT6 = !(EQ6 # GT6); LT5 = !(EQ5 # GT5);
LT4 = !(EQ4 # GT4); LT3 = !(EQ3 # GT3); LT2 = !(EQ2 # GT2);
LT1 = !(EQ1 # GT1); LT0 = !(EQ0 # GT0);

equations

PEQQ = EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & EQ1 & EQ0;

PNEQ = !(EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & EQ1 & EQ0);

PGTQ = GT7 # EQ7 & GT6 # EQ7 & EQ6 & GT5
      # EQ7 & EQ6 & EQ5 & GT4 # EQ7 & EQ6 & EQ5 & EQ4 & GT3
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & GT2
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & GT1
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & EQ1 & GT0;

PLEQ = !(GT7 # EQ7 & GT6 # EQ7 & EQ6 & GT5
      # EQ7 & EQ6 & EQ5 & GT4 # EQ7 & EQ6 & EQ5 & EQ4 & GT3
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & GT2
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & GT1
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & EQ1 & GT0);

PLTQ = LT7 # EQ7 & LT6 # EQ7 & EQ6 & LT5
      # EQ7 & EQ6 & EQ5 & LT4 # EQ7 & EQ6 & EQ5 & EQ4 & LT3
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & LT2
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & LT1
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & EQ1 & LT0;

PQEQ = !(LT7 # EQ7 & LT6 # EQ7 & EQ6 & LT5
      # EQ7 & EQ6 & EQ5 & LT4 # EQ7 & EQ6 & EQ5 & EQ4 & LT3
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & LT2
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & LT1
      # EQ7 & EQ6 & EQ5 & EQ4 & EQ3 & EQ2 & EQ1 & LT0);

end compexp

```

ВЫ НА ПРАВИЛЬНОМ ПУТИ!

В первых ПЛУ типа PAL16L8 не было управления полярностью выходных сигналов. Разработчики, использовавшие эти ИС, были вынуждены выбирать конкретную полярность, то есть высокий или низкий активный уровень сигналов на различных выходах, чтобы сократить выражения до такой степени, при которой их можно было реализовать в данной ИС. Когда используются ИС 16V8, 20V8, 22V10 или какие-либо другие микросхемы из множества появившихся в последнее время в изобилии ИС типа CPLD, такого ограничения не существует. Если число термов в выражении *или в его дополнении* можно сократить до числа доступных термов-произведений, то активным можно сделать как высокий, так и низкий уровень выходного сигнала, соответствующим образом запрограммировав его полярность.

В этой программе для выходных сигналов PEQQ и PNEQ нужно по одному терму-произведению на каждый. Остальным восьми выходам необходимо по восемь термов-произведений на каждый. Как мы упоминали ранее, ИС 22V10 позволяет иметь от 8 до 16 термов-произведений на выход, так что проект реализуем.

6.2.5. Компаратор с управляемым режимом работы

В качестве следующего примера предположим, что речь идет о системе, которая в обычных условиях должна сравнивать два 32-разрядных слова, но иногда бывает нужно игнорировать один или два младших разряда входных слов. Режим работы определяется двумя битами выбора режима M1 и M0 согласно табл. 6.9.

M1	M0	Сравнение
0	0	32 разряда
0	1	31 разряд
1	0	30 разрядов
1	1	не используется

Табл. 6.9. Биты выбора режима компаратора

Как было отмечено ранее, сравнение, сложение и другие «итеративные» операции обычно плохо поддаются реализации на основе ПЛУ, потому что эквивалентное двухуровневое выражение вида «сумма произведений» содержит слишком много термов-произведений. В разделе 5.9.5 мы посчитали, сколько термов-произведений необходимо для n -разрядного компаратора. Согласно этим результатам мы, конечно, не можем создать 32-разрядный компаратор с управляемым режимом работы или даже 8-разрядную секцию такого компаратора в ПЛУ; 8-разрядный компаратор 74х682 является, пожалуй, наиболее эффективной возможностью реализовать 8-разрядное сравнение с помощью единственной микросхемы. Однако подход, основанный на использовании ПЛУ, вполне приемлем для реализации логики выбора режима и той части схемы сравнения, которая зависит от выбранного режима (учет двух младших разрядов).

На рис. 6.9 показана полная схема компаратора, построенная с использованием этой идеи, а в табл. 6.11 приведена программа на языке ABEL для ПЛУ MODECOMP (ИС 16V8), на основе которого реализуется «нерегулярная логика». Четыре ИС '682 используются для сравнения большей части разрядов, а в ИС 16V8 объединяются выходы всех ИС '682 и обрабатываются два младших разряда в зависимости от выбранного режима. Промежуточные выражения EQ30 и GT30 введены для сокращения программы в разделе equations.

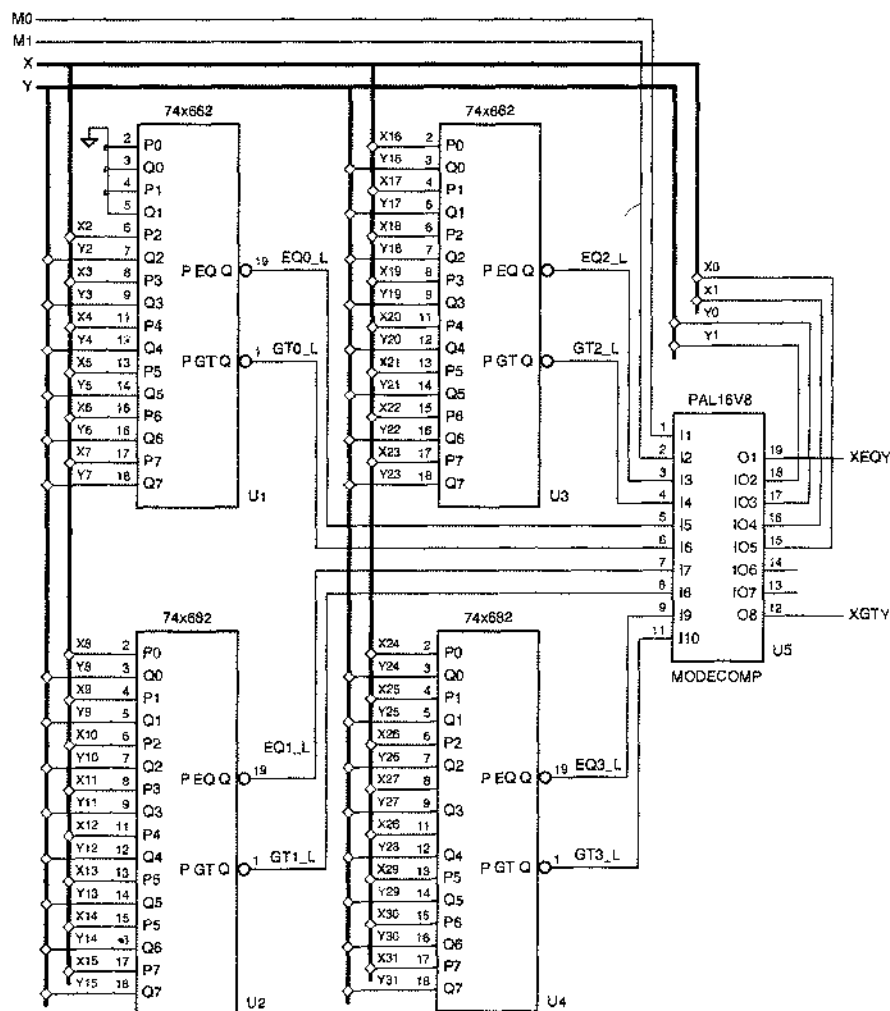


Рис. 6.9. 32-разрядный компаратор с управляемым режимом работы

Как видно из табл. 6.10, сигналы на выходах XEQY и XGTY представлены 7-к и 11-ю термами-произведениями соответственно. Таким образом, сигнал XGTY не укладывается в 7 термов-произведений, доступных для выходных сигналов ИС 16V8. Однако это еще один пример, где имеется некоторая свобода в выбо-

ре способа кодирования. Изменяя кодирование MODE30 на [1, .X.], можно уменьшить требуемое число термов-произведений для XGTY до 7/12 и таким образом поместить схему в ИС 16V8.

P-Terms	Fan-in	Fan-out	Type	Name
7/9	10	1	Pin	XEQY
11/13	14	1	Pin	XGTY
=====				
18/22	Best P-Term Total: 18			
	Total Pins: 16			
	Average P-Term/Output: 9			

Табл. 6.10. Использование термов-произведений в ПЛУ MODECOMP

Табл. 6.11. Программа на языке ABEL для объединения четырех ИС 74х682 в 32-разрядный компаратор

```

module modecomp
title 'Control PLD for Mode-Dependent Comparator'
MODECOMP device 'P16V8';

" Input and output pins
M0, M1, EQ2_L, GT2_L, EQ0_L, GT0_L      pin 1..6;
EQ1_L, GT1_L, EQ3_L, GT3_L, X0, X1, Y0, Y1  pin 7..9, 10, 15..18;
XEQY, XGTY                                pin 19, 12 istype 'com';

" Active-level conversions
EQ3 = !EQ3_L; EQ2 = !EQ2_L; EQ1 = !EQ1_L; EQ0 = !EQ0_L;
GT3 = !GT3_L; GT2 = !GT2_L; GT1 = !GT1_L; GT0 = !GT0_L;

" Mode definitions
MODE32 = ([M1,M0] == [0,0]); " 32-bit comparison
MODE31 = ([M1,M0] == [0,1]); " 31-bit comparison
MODE30 = ([M1,M0] == [1,0]); " 30-bit comparison
MODEXX = ([M1,M0] == [1,1]); " Unused

" Expressions for 30-bit equal and greater-than
EQ30 = EQ3 & EQ2 & EQ1 & EQ0;
GT30 = GT3 # (EQ3 & GT2) # (EQ3 & EQ2 & GT1) # (EQ3 & EQ2 & EQ1 & GT0);

equations

WHEN MODE32 THEN {
  XEQY = EQ30 & (X1==Y1) & (X0==Y0);
  XGTY = GT30 # (EQ30 & (X1>Y1)) # (EQ30 & (X1==Y1) & (X0>Y0));
}
ELSE WHEN MODE31 THEN {
  XEQY = EQ30 & (X1==Y1);
  XGTY = GT30 # (EQ30 & (X1>Y1));
}
ELSE WHEN MODE30 THEN {
  XEQY = EQ30;
  XGTY = GT30;
}

end modecomp

```

6.2.6. Счетчик числа единиц

Существует несколько важных алгоритмов, которые включают операцию подсчета числа «единичных» битов в операнде. Недавно некоторые микропроцессорные системы команд были расширены путем включения операции подсчета числа единиц в качестве одной из основных команд.

Подсчет числа единиц в операнде легко реализовать в виде итеративного процесса, в котором вы просматриваете слово с одного конца до другого и увеличиваете содержимое счетчика на единицу каждый раз, когда встречается 1. Однако внутри АЛУ микропроцессора эта операция должна выполняться быстрее. В идеале мы бы хотели, чтобы подсчет числа единиц происходил так же быстро, как любое другое арифметическое действие типа сложения двух чисел. Поэтому требуется комбинационная схема.

Предположим в этом примере, что нам требуется создать счетчик числа единиц в 32-разрядном слове как часть более крупной системы. Очевидно, что необходимое число входов и выходов не позволяет нам разместить счетчик в одном ПЛУ типа 22V10, но мы могли бы разбить устройство на отдельные компоненты для размещения в относительно небольшом числе ПЛУ.

На рис. 6.10 показано такое разбиение. С помощью двух первых идентичных ИС 22V10 ONESCNT1 осуществляется счет числа единиц в двух 15-разрядных отрезках 32-разрядного входного слова $D[31:0]$ с выдачей каждой из них результата в виде 4-разрядного числа. Следующая ИС 22V10 ONESCNT2 используется для сложения двух полученных 4-разрядных чисел и числа единиц в оставшихся 2-х разрядах входного слова.

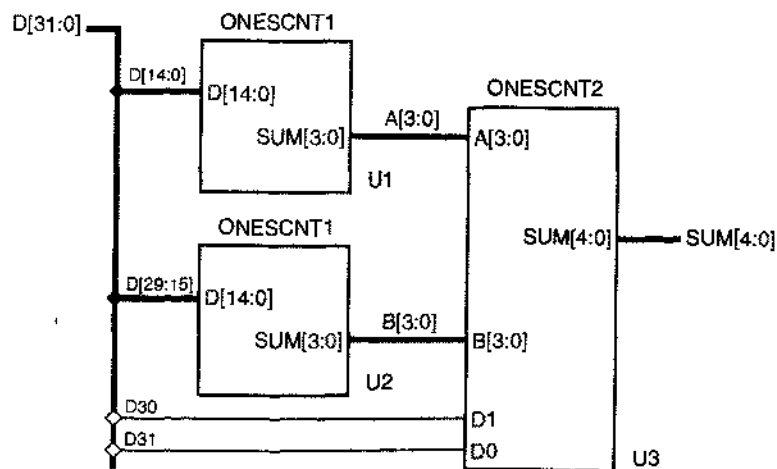


Рис. 6.10. Возможное разбиение схемы счетчика единиц на отдельные компоненты

Программа для ONESCNT1, приведенная в табл. 6.12, обманчиво проста. Операнд “@CARRY1” включен для ограничения цепочки переносов одним разрядом; как объяснялось в разделе 5.10.8, это уменьшает число термов-произведений за счет появления вспомогательных выходов и увеличения задержки.

Табл. 6.12. Программа на языке ABEL для подсчета числа разрядов, содержащих 1, в 15-разрядном слове

```

module onescnt1
title 'Count the ones in a 15-bit word'
ONESCNT1 device 'P22V10';

" Input and output pins
D14..D0      pin 1..11, 13..15, 23;
SUM3..SUM0    pin 17..20 istype 'com';

equations

@CARRY 1;
[SUM3..SUM0] = D0 + D1 + D2 + D3 + D4 + D5 + D6 + D7
              + D8 + D9 + D10 + D11 + D12 + D13 + D14;

end onescnt1

```

К сожалению, при компиляции этой программы мой компьютер из-за ограниченных возможностей процессора в течение часа не выдавал никаких результатов. Это дало мне время воспользоваться своей головой – хорошее упражнение для тех из нас, кто стал слишком зависимым от автоматизированных средств проектирования. Тогда я понял, что для выхода SUM0 могу записать логическую функцию вручную всего лишь за несколько секунд:

$$\text{SUM0} = D0 \oplus D1 \oplus D2 \oplus D3 \oplus D4 \oplus D5 \oplus D6 \oplus D7 \oplus \dots \oplus D13 \oplus D14.$$

Карта Карно для этой функции выглядит как шахматная доска, и минимальное выражение в виде суммы произведений содержит 2^{14} термов-произведений. Очевидно, что это невозможно реализовать за один или несколько проходов через ИС 22V10! Так или иначе, я прекратил процесс компиляции и перезагрузил Windows на тот случай, если компилятор что-то испортил.

Понятно, что для создания схемы, подсчитывающей число единиц, требуется разделение на небольшие блоки. Хотя мы могли бы и дальше продолжать пользоваться языком ABEL в расчете на реализацию в нескольких ПЛЮ, но интереснее выполнить структурное проектирование такого устройства, пользуясь средствами VHDL, как это и будет сделано в разделе 6.3.6, а вариант с языком ABEL и микросхемами ПЛЮ пусть останется в качестве задачи 6.6.

6.2.7. Игра в крестики и нолики

В этом примере мы построим комбинационную схему, которая выбирает очередной ход в игре в крестики и нолики. Прежде всего нужно решить, какой будет стратегия выбора очередного хода. Давайте попробуем подражать типичной стратегии человека, который принимает решения по шагам:

1. Ищем строку, столбец или диагональ, где имеются две наши метки (X или O в зависимости от того, на чьей мы стороне) и одна пустая клетка. Если такая ситуация обнаруживается, то помещаем свою метку в эту пустую клетку. Мы победили!

ПРАВИЛА ИГРЫ В КРЕСТИКИ И НОЛИКИ (НА СЛУЧАЙ, ЕСЛИ ВЫ НЕ ЗНАЕТЕ)

Игра в крестики и нолики ведется двумя игроками на поле, разбитом на клетки, по 3 клетки в каждой строке и в каждом столбце. Первоначально клетки пустые. Один игрок выбирает себе в качестве метки крестик X, а другой – нолик O. Игроки поочередно ставят свои метки в одной из пустых клеток; игрок X всегда начинает первым. Победителем считается тот, кто первым заполнит своими метками три клетки подряд в строке, в столбце или по диагонали. Хотя у игрока X, начинающего игру, есть небольшое преимущество, можно показать, что игра между двумя сообразительными игроками всегда заканчивается вничью: ни один игрок не поставит три метки подряд к моменту, когда поле заполнится.

- Ищем строку, столбец или диагональ, где имеются две метки противника и одна пустая клетка. Если такая ситуация обнаруживается, то помещаем свою метку в эту пустую клетку, чтобы блокировать возможный выигрыш противника.
- Выбираем клетку, руководствуясь своим опытом. Например, если центральная клетка пустая, то обычно стоит заполнить ее. В противном случае полезно заполнять угловые клетки. Умный игрок может заметить также и блокировать развитие конфигурации противником или выбрать хороший ход благодаря «предвидению».

	1	2	3	столбец
строка				
1	x11, y11	x12, y12	x13, y13	
2	x21, y21	x22, y22	x23, y23	
3	x31, y31	x32, y32	x33, y33	

Рис. 6.11. Поле для игры в крестики и нолики и имена сигналов в программе на языке ABEL.

Заранее предвидя нутяницу между символом O и нулем 0 в наших программах, мы, во избежание неприятностей, назовем второго игрока буквой Y. Далее, нужно подумать о том, как следует кодировать входные и выходные сигналы в разрабатываемой схеме. У каждого игрока существует только девять возможных ходов, поэтому для представления выходных сигналов достаточно четырех двоичных разрядов. Входными сигналами схемы служит текущее состояние игрового поля. Каждая из девяти клеток, может находиться в одном из трех состояний (пустая, заполненная меткой X, заполненная меткой Y).

Можно различными способами отобразить состояние одной клетки. Поскольку игра симметрична, выберем симметричное представление, которое поможет нам в дальнейшем:

КОМПАКТНОЕ КОДИРОВАНИЕ

Так как каждая клетка в этой игре может находиться только в одном из трех состояний, а не из четырех, общее число игровых конфигураций составляет $3^9 = 19683$. Это число меньше, чем 2^{15} , поэтому состояние поля можно кодировать всего 15-ю разрядами. Однако в этом случае схема, выбирающая очередной ход, была бы слишком сложной, если только функции такой схемы не поручить ПЗУ (см. задачу 10.26).

- 00 – клетка пуста;
- 10 – клетка занята меткой X;
- 01 – клетка занята меткой Y.

Таким образом, состояние поля размером 3×3 можно кодировать 18-ю битами. Будем обозначать клетки двумя числами, указывая номер строки и номер столбца, как это сделано на рис. 6.11, а в программе на языке ABEL будем использовать символы X_{ij} и Y_{ij} при наличии метки X или Y в клетке i, j . Кодирование выходных сигналов давайте рассмотрим позже.

Схема с 18 входами и 4 выходами, играющая в крестики и нолики, могла бы, в принципе, разместиться в одной ИС 22V10. Однако опыт показывает, что этого сделать нельзя. Поэтому функцию, реализуемую схемой в целом, необходимо разбить на части. Сделать это можно в соответствии с тем, как игрок принимает решение по шагам, и мысль эта выглядит неплохой идеей.

На самом деле, шаги 1 и 2 очень похожи: они отличаются только тем, что игроки меняются ролями. Именно поэтому может оказаться полезным симметричное кодирование. ПЛУ, которое находит две мои метки подряд и пустую клетку для победного хода (шаг 1), может найти также две метки моего противника подряд и пустую клетку для выполнения блокирующего хода (шаг 2). Все, что нам надо сделать, – это переставить коды меток X и Y. При выбранном нами способе кодирования, не требующем никакой логики, достаточно во всех клетках физически поменять местами сигналы X_{ij} и Y_{ij} . С учетом этого для выполнения шагов 1 и 2 можно воспользоваться двумя одинаковыми ПЛУ TWOINROW, как показано на рис. 6.12. Обратите внимание, что сигналы X_{11} – X_{33} поданы на верхние входы первого ПЛУ TWOINROW и на нижние входы второго ПЛУ.

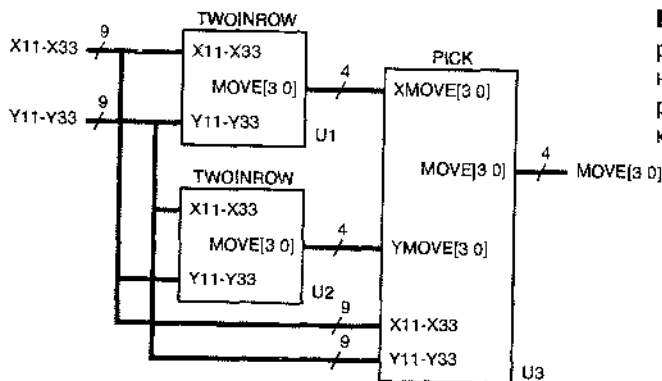


Рис. 6.12. Предварительное разбиение на блоки схемы для реализации игры в крестики и нолики

Сигналы с выходов двух ПЛУ TWOINROW пусть поступают на входы еще одного ПЛУ PICK. Если хотя бы одним из двух нервых ПЛУ найден ход, то блоком PICK он передается на выход; в противном случае этим устройством выполняется шаг 3. Похоже, что у блока PICK слишком много входов и выходов и его нельзя реализовать в ИС 22V10, но мы вернемся к этому позже.

В табл. 6.13 представлена программа для ПЛУ TWOINROW. В ней анализируется состояние игрового поля с точки зрения игрока X; другими словами, ищется ход, при котором метка X будет занимать три идущие подряд клетки. В этой программе активно используются промежуточные равенства для определения всех возможных ходов по строкам, столбцам и диагоналям. В выражении для G_{ij} объединяются все условия, при которых ход в клетку i, j (ее заполнение меткой X) будет правильным ходом. Наконец, в разделе equations с помощью оператора WHEN выбирается нужный ход.

Табл. 6.13. Программа на языке ABEL для нахождения двух меток подряд при игре в крестики и нолики

```

module twoinrow
Title 'Find Two Xs and an empty cell in a row, column, or diagonal'
TWOINROW device 'P22V10';

" Inputs and Outputs
X11, X12, X13, X21, X22, X23, X31, X32, X33 pin 1..9;
Y11, Y12, Y13, Y21, Y22, Y23, Y31, Y32, Y33 pin 10,11,13..15,20..23;
MOVE3..MOVE0                                     pin 16..19 istype 'com';

" MOVE output encodings
MOVE = [MOVE3..MOVE0];
MOVE11 = [1,0,0,0]; MOVE12 = [0,1,0,0]; MOVE13 = [0,0,1,0];
MOVE21 = [0,0,0,1]; MOVE22 = [1,1,0,0]; MOVE23 = [0,1,1,1];
MOVE31 = [1,0,1,1]; MOVE32 = [1,1,0,1]; MOVE33 = [1,1,1,0];
NONE = [0,0,0,0];

" Find moves in rows. Rxy ==> a move exists in cell xy
R11 = X12 & X13 & !X11 & !Y11;
R12 = X11 & X13 & !X12 & !Y12;
R13 = X11 & X12 & !X13 & !Y13;
R21 = X22 & X23 & !X21 & !Y21;
R22 = X21 & X23 & !X22 & !Y22;
R23 = X21 & X22 & !X23 & !Y23;
R31 = X32 & X33 & !X31 & !Y31;
R32 = X31 & X33 & !X32 & !Y32;
R33 = X31 & X32 & !X33 & !Y33;

" Find moves in columns. Cxy ==> a move exists in cell xy
C11 = X21 & X31 & !X11 & !Y11;
C12 = X22 & X32 & !X12 & !Y12;
C13 = X23 & X33 & !X13 & !Y13;
C21 = X11 & X31 & !X21 & !Y21;
C22 = X12 & X32 & !X22 & !Y22;
C23 = X13 & X33 & !X23 & !Y23;
C31 = X11 & X21 & !X31 & !Y31;

```

Табл. 6.13. Программа на языке ABEL для нахождения двух меток подряд при игре в крестики и нолики (продолжение)

```

C32 = X12 & X22 & !X32 & !Y32;
C33 = X13 & X23 & !X33 & !Y33;

" Find moves in diagonals. Dxy or Exy ==> a move exists in cell xy
D11 = X22 & X33 & !X11 & !Y11;
D22 = X11 & X33 & !X22 & !Y22;
D33 = X11 & X22 & !X33 & !Y33;
E13 = X22 & X31 & !X13 & !Y13;
E22 = X13 & X31 & !X22 & !Y22;
E31 = X13 & X22 & !X31 & !Y31;

" Combine moves for each cell. Gxy ==> a move exists in cell xy
G11 = R11 # C11 # D11;
G12 = R12 # C12;
G13 = R13 # C13 # E13;
G21 = R21 # C21;
G22 = R22 # C22 # D22 # E22;
G23 = R23 # C23;
G31 = R31 # C31 # E31;
G32 = R32 # C32;
G33 = R33 # C33 # D33;

equations
WHEN G22 THEN MOVE= MOVE22;
ELSE WHEN G11 THEN MOVE = MOVE11;
ELSE WHEN G13 THEN MOVE = MOVE13;
ELSE WHEN G31 THEN MOVE = MOVE31;
ELSE WHEN G33 THEN MOVE = MOVE33;
ELSE WHEN G12 THEN MOVE = MOVE12;
ELSE WHEN G21 THEN MOVE = MOVE21;
ELSE WHEN G23 THEN MOVE = MOVE23;
ELSE WHEN G32 THEN MOVE = MOVE32;
ELSE MOVE = NONE;

end twoinrow

```

Обратите внимание, что необходимо использовать вложенный оператор WHEN, а не девять параллельных операторов WHEN или присваиваний, поскольку выбрать нужно только один ход, даже если имеется несколько возможностей. Заметьте также, что первой проверяется центральная клетка G22, а потом – угловые клетки. Это делается в надежде, что, помещая наиболее частые ходы во вложенном операторе WHEN раньше, мы минимизируем число термов. Увы, нашему проекту все еще требуется масса термов-произведений, как следует из табл. 6.14.

Между прочим, мы до сих пор не объяснили, почему выбран тот способ кодирования выходных сигналов, который указан в программе для MOVE11, MOVE22 и т.д. Довольно очевидно, что изменение способа кодирования не позволит сократить число термов-произведений настолько, чтобы схема могла быть размещена в ИС 22V10. Однако, как сейчас будет показано, все же существует метод борьбы с кошмарным числом термов-произведений.

P-Terms	Fan-in	Fan-out	Type	Name	Табл. 6.14. Использование термов-произведений в ПЛУ TWOINROW
61/142	18	1	Pin	MOVE3	
107/129	18	1	Pin	MOVE2	
77/88	17	1	Pin	MOVE1	
133/87	18	1	Pin	MOVE0	
=====					
378/446	Best P-Term Total: 332				
	Total Pins: 22				
	Average P-Term/Output: 83				

Ясно, что необходимо разбить блок TWOINROW на две или большее число частей. Как и в любой задаче проектирования, при этом возможны несколько различных стратегий. Первая стратегия, которую я попробовал, состояла в использовании двух различных ПЛУ, из которых одно предназначалось для поиска ходов по всем строкам и по одной из диагоналей, а другое — для нахождения ходов по всем столбцам и остающейся диагонали. Это помогло, но не настолько, чтобы каждая половина помещалась в ИС 22V10.

Вторая стратегия состояла в попытке разделить задачу на части другим путем, когда первое ПЛУ находит все ходы в клетки 11, 12, 13, 21 и 22, а второе ПЛУ — все ходы в остающиеся клетки. Это сработало! Программа для первого ПЛУ, названного TWOINHAF, получается из табл. 6.13 простым переводением в комментарий четырех строк оператора WHEN, относящихся к клеткам 23, 31, 32 и 33.

Аналогично из программы для TWOINROW можно получить программу для второго ПЛУ, но подождите минуту. При изготовлении реальных цифровых систем всегда желательно минимизировать число различных типов применяемых блоков: это сокращает стоимость изделия и уменьшает его сложность. Точно так же в отношении программируемых блоков желательно минимизировать число различных используемых программ. Даже если блоки реализуются на одних и тех же ПЛУ, для каждой новой программы необходимо составлять новый набор тестовых векторов, а это стоит денег. Кроме того, изделие может оказаться настолько удачным, что можно будет сэкономить, заменив ПЛУ жестко запрограммированными ИС для каждой программы, а это снова подталкивает нас к минимизации числа различных программ.

Игра в крестики и нолики останется той же самой игрой, если мы повернем игровое поле на 90° или на 180°. Но если повернуть поле на 180°, то ПЛУ TWOINHAF сможет находить ходы в клетки 33, 32, 31, 23 и 22. Выбранный способ представления текущего состояния поля отдельной парой чисел для каждой клетки позволяет «повернуть поле» просто путем соответствующей перестановки этих пар чисел. Другими словами, достаточно поменять местами содержимое следующих клеток: 33 ↔ 11, 32 ↔ 12, 31 ↔ 13 и 23 ↔ 21.

При смене содержимого клеток программа TWOINHAF будет по-прежнему выдавать коды ходов в клетки в верхней половине поля. Чтобы не было ошибок, необходимо преобразовать эти результаты в коды, соответствующие клеткам в нижней половине поля. Хорошо бы такое преобразование потребовало

минимума логики. Это как раз то место, где пригодится выбранный нами способ кодирования ходов. Если вы внимательно посмотрите на коды MOVE в начале табл. 6.13, то вы увидите, что ход в клетку, занимающую данное положение в результате поворота поля на 180° , представлен кодом, который получается из кода для хода в клетку, занимающую то же самое положение до поворота поля, путем побитового дополнения и изменения порядка следования битов. Другими словами, нужное преобразование кода можно реализовать четырьмя инверторами и переключением соединений. Это можно делать «бесплатно» в ПЛУ, которое анализирует сигналы на выходах блоков TWOINHAF.

Вы, вероятно, никогда не думали, что игра в крестики и нолики может быть настолько запутанной. Но мы уже на полпути к решению задачи. Давайте продолжим наше проектирование с учетом разбиения на блоки, показанного рис. 6.13. Каждое ПЛУ TWOINHROW, появившееся при первоначальном разбиении на блоки, заменено теперь парой ПЛУ TWOINHAF. Нижним ПЛУ в каждой паре предшествуют блоки, помеченные буквой P, в каждом из которых переставляется содержимое клеток так, чтобы тем самым повернуть поле на 180° , как об этом говорилось выше. Кроме того, за каждым из этих ПЛУ следует блок, помеченный буквой T, в котором компенсируется поворот поля путем преобразования выходного кода; в действительности, эти блоки будут поглощены следующим за ними ПЛУ PICK1.

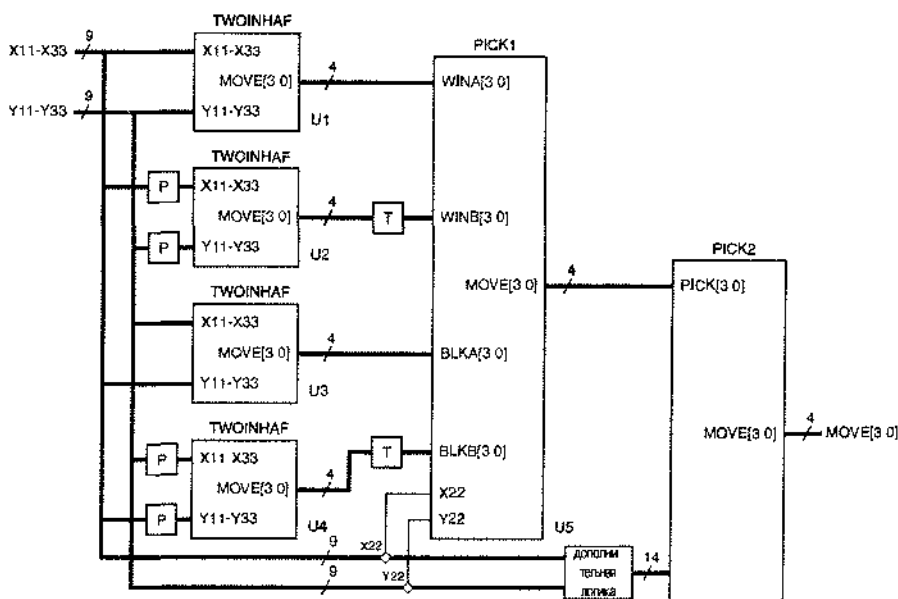


Рис. 6.13. Окончательное разбиение ехемы для игры в крестики и нолики на блоки, реализуемые в ПЛУ

Блоком PICK1 реализуется совсем простая функция. Как следует из табл. 6.15, она состоит просто в том, что делается победный или блокирующий ход в случае, когда они имеются. Так как у ПЛУ 22V10 есть два дополнительных вхо-

да, воспользуемся ими для ввода состояния центральной клетки. Тогда можно будет выполнить первую часть 3-го шага игрового алгоритма, выбирая центральную клетку, если нет ни победного, ни блокирующего хода. ПЛУ PICK1 нужно не более 9 термов-произведений на выход.

Табл. 6.15. Программа на языке ABEL для выбора хода по сигналам на четырех входах

```

module pick1
Title 'Pick One Move from Four Possible'
PICK1 device 'P22V10';

" Inputs from TWOINHAIF PLDs
WINA3..WINA0 pin 1..4; "Winning moves in cells 11,12,13,21,22
WINB3..WINB0 pin 5..8; "Winning moves in cells 11,12,13,21,22 of rotated grid
BLKA3..BLKA0 pin 9..11, 13; "Blocking moves in cells 11,12,13,21,22
BLKB3..BLKB0 pin 14..16, 21; "Blocking moves in cells 11,12,13,21,22 of rotated grid
" Inputs from grid
X22, Y22 pin 22..23; "Center cell; pick if no other moves
" Move outputs to PICK2 PLD
MOVE3..MOVE0 pin 17..20 istype 'com';

" Sets
WINA = {WINA3..WINA0}; WINB = {WINB3..WINB0};
BLKA = {BLKA3..BLKA0}; BLKB = {BLKB3..BLKB0};
MOVE = {MOVE3..MOVE0};

" Non-rotated move input and output encoding
MOVE11 = [1,0,0,0]; MOVE12 = [0,1,0,0]; MOVE13 = [0,0,1,0];
MOVE21 = [0,0,0,1]; MOVE22 = [1,1,0,0]; MOVE23 = [0,1,1,1];
MOVE31 = [1,0,1,1]; MOVE32 = [1,1,0,1]; MOVE33 = [1,1,1,0];
NONE = [0,0,0,0];

equations
WHEN WINA != NONE THEN MOVE = WINA;
ELSE WHEN WINB != NONE THEN MOVE = !{WINB0..WINB3}; " Map rotated coding
ELSE WHEN BLKA != NONE THEN MOVE = BLKA;
ELSE WHEN BLKB != NONE THEN MOVE = !{BLKB0..BLKB3}; " Map rotated coding
ELSE WHEN !X22 & !Y22 THEN MOVE = MOVE22; " Pick center cell if it's empty
ELSE MOVE = NONE;

end pick1

```

Последним блоком в схеме на рис. 6.13 является ПЛУ PICK2. Это ПЛУ должно обеспечить «использование опыта» на 3-м шаге игрового алгоритма, если PICK1 не находит хода.

С блоком PICK2 возникает небольшая проблема, состоящая в том, что у ИС 22V10 нет достаточного числа выводов для 4-разрядного входа со стороны блока PICK1, собственного 4-разрядного выхода и 18-разрядного входа, по которому в блок PICK2 могла бы поступать информация о состоянии игрового поля: у этой ИС только 22 вывода «вход/выход». На самом деле нам нет необходимости подключать X22 и Y22, так как они анализируются уже в блоке PICK1, но даже с учетом этого нам все еще не хватает двух выводов. Поэтому назначение «дополнительной логики» на рис. 6.13 заключается в преобразовании части информации, чтобы сэкономить два вывода. Метод, который мы применим здесь, состоит в том, чтобы объединить сигналы, соответствующие средним клеткам

12, 21, 23 и 32 по краям игрового поля, и вырабатывать четыре сигнала E12, E21, E23 и E32, принимающих единичное значение только в том случае, когда соответствующие клетки пусты. Это можно сделать с помощью четырех 2-входных схем ИЛИ-НЕ, оставляя незанятыми два входа или выхода у ИС 22V10.

Табл. 6.16. Программа на языке ABEL для выбора хода «на основании опыта»

```

module pick2
Title 'Pick a move using experience'
PICK2 device 'P22V10';

" Inputs from PICK1 PLD
PICK3..PICK0                                pin 1..4; " Move, if any, from PICK1 PLD
" Inputs from Tic-Tac-Toe grid corners
X11, Y11, X13, Y13, X31, Y31, X33, Y33      pin 5..11, 13;
" Combined inputs from external NOR gates; 1 ==> corresponding cell is empty
E12, E21, E23, E32                          pin 14..15, 22..23;
" Move output
MOVE3..MOVE0                                pin 17..20 istype 'com';

PICK = [PICK3..PICK0]; " Set definition
" Non-rotated move input and output encoding
MOVE = [MOVE3..MOVE0];
MOVE11 = [1,0,0,0]; MOVE12 = [0,1,0,0]; MOVE13 = [0,0,1,0];
MOVE21 = [0,0,0,1]; MOVE22 = [1,1,0,0]; MOVE23 = [0,1,1,1];
MOVE31 = [1,0,1,1]; MOVE32 = [1,1,0,1]; MOVE33 = [1,1,1,0];
NONE   = [0,0,0,0];

" Intermediate equations for empty corner cells
E11 = !X11 & !Y11; E13 = !X13 & !Y13; E31 = !X31 & !Y31; E33 = !X33 & !Y33;

equations

"Simplest approach -- pick corner if available, else side
WHEN PICK != NONE THEN MOVE = PICK;
ELSE WHEN E11 THEN MOVE = MOVE11;
ELSE WHEN E13 THEN MOVE = MOVE13;
ELSE WHEN E31 THEN MOVE = MOVE31;
ELSE WHEN E33 THEN MOVE = MOVE33;
ELSE WHEN E12 THEN MOVE = MOVE12;
ELSE WHEN E21 THEN MOVE = MOVE21;
ELSE WHEN E23 THEN MOVE = MOVE23;
ELSE WHEN E32 THEN MOVE = MOVE32;
ELSE MOVE = NONE;

end pick2

```

В табл. 6.16 приведена программа для ПЛУ PICK2, где предполагается, что в качестве «дополнительной логики» применены четыре схемы ИЛИ-НЕ. Для выбора хода в этой программе использован простейший эвристический алгоритм: если имеется пустая угловая клетка, то она и выбирается; в противном случае выбирается средняя клетка на краю поля. Эту программу надо бы немного улучшить, потому что иногда она проигрывает (см. задачу 6.8). К счастью, равенствам, возникающим в результате компиляции программы из табл. 6.16, требует только от 8 до 10 термов на выход, поэтому еще есть возможность сделать эту программу более умной (см. задачи 6.9 и 6.10).

6.3. Примеры проектирования с использованием языка VHDL

6.3.1. Устройство быстрого сдвига

В разделе 6.1.1 устройство быстрого сдвига было определено как комбинационная логическая схема с n входами данных, m выходами данных и набором входных управляющих сигналов, которые задают величину сдвига выходных данных относительно входных. Там же было показано, как построить простое устройство быстрого сдвига, выполняющее только циклические сдвиги влево, используя стандартные микросхемы средней степени интеграции. Затем в разделе 6.2.1 было показано, как на языке ABEL описывается устройство быстрого сдвига, обладающее большими возможностями, но там мы отметили также, что для реализации такого устройства ПЛИУ обычно не подходит. В этом разделе мы покажем, как можно воспользоваться языком VHDL для описания как поведения, так и структуры устройств быстрого сдвига при их реализации на основе специализированных ИС или ИС типа FPGA.

В табл. 6.17 представлена поведенческая программа на языке VHDL для 16-разрядного устройства быстрого сдвига, которое выполняет сдвиг для любой из шести возможных комбинаций типа сдвига и направления. Как мы уже видели ранее (см. в табл. 6.3), сдвиги бывают циклическими, логическими и арифметическими, и сдвиг может производиться, естественно, влево или вправо. Как видно из объявления объекта, 4-разрядным входным сигналом управления S задается величина сдвига, а 3-разрядным входным сигналом управления C определяется режим сдвига (тип и направление). Используя пакет `IEEE std_logic_arith`, мы определяем тип величины сдвига S как `UNSIGNED` для того, чтобы позже можно было воспользоваться функцией `CONV_INTEGER` из этого пакета.

Обратите внимание, что объявление объекта включает шесть определений постоянных, которыми устанавливается соответствие между режимами сдвига и значением C . Хотя мы не обсуждали это в разделе 4.7, но язык VHDL позволяет помещать константы, тип, сигнал и другие объявления в объявление объекта. Определение таких элементов в объявлении объекта имеет смысл только в том случае, когда они должны быть одними и теми же в любой архитектуре. В нашем случае за режимами сдвига закрепляются вполне определенные двоичные коды, поэтому здесь им самое место. Другим элементам предстоит появиться в определении архитектуры.

В части программы, относящейся к архитектуре, мы определяем шесть функций, по одной для каждого вида сдвига 16-разрядного элемента типа `STD_LOGIC_VECTOR`. В архитектуре определен также подтип `DATAWORD`, чтобы сэкономить на его объявлении в определениях функций.

Табл. 6.17. Поведенческое VHDL-описание устройства быстрого сдвига, выполняющего 6 видов сдвига

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity barrel16 is
  port (
    DIN: in STD_LOGIC_VECTOR (15 downto 0); -- Data inputs
    S: in UNSIGNED (3 downto 0); -- Shift amount, 0-15
    C: in STD_LOGIC_VECTOR (2 downto 0); -- Mode control
    DOUT: out STD_LOGIC_VECTOR (15 downto 0) -- Data bus output
  );
  constant Lrotate: STD_LOGIC_VECTOR := "000"; -- Define the coding of
  constant Rrotate: STD_LOGIC_VECTOR := "001"; -- the different shift modes
  constant Llogical: STD_LOGIC_VECTOR := "010";
  constant Rlogical: STD_LOGIC_VECTOR := "011";
  constant Larith: STD_LOGIC_VECTOR := "100";
  constant Rarith: STD_LOGIC_VECTOR := "101";
end barrel16;

architecture barrel16_behavioral of barrel16 is
  subtype DATAWORD is STD_LOGIC_VECTOR(15 downto 0);

  function Vrot (D: DATAWORD; S: UNSIGNED)
    return DATAWORD is
    variable N: INTEGER;
    variable TMPD: DATAWORD;
  begin
    N := CONV_INTEGER(S); TMPD := D;
    for i in 1 to N loop
      TMPD := TMPD(14 downto 0) & TMPD(15);
    end loop;
    return TMPD;
  end Vrot;

  ...

begin
  process(DIN, S, C)
  begin
    case C is
      when Lrotate => DOUT <= Vrot(DIN,S);
      when Rrotate => DOUT <= Vror(DIN,S);
      when Llogical => DOUT <= Vsl1(DIN,S);
      when Rlogical => DOUT <= Vsr1(DIN,S);
      when Larith => DOUT <= Vsla(DIN,S);
      when Rarith => DOUT <= Vsra(DIN,S);
      when others => null;
    end case;
  end process;
end barrel16_behavioral;

```

ВАШИ СОБСТВЕННЫЕ СДВИГИ

В действительности у языка VHDL-93 есть встроенные операторы сдвига `rol`, `ror`, `sl`, `srl`, `sla` и `sra` элементов типа `array`, соответствующие операциям сдвига, перечисленным в табл. 6.3. Так как этих операторов нет в языке VHDL-87, мы определили в табл. 6.17 свои собственные функции. На самом деле, в таблице приведена только одна из них (`Vrol`); определение остальных функций оставлено читателю в качестве задачи (задача 6.11).

В табл. 6.17 полностью приведена только первая функция (`Vrol`); остальные подобны ей, за исключением изменения в одной строке. Используемая в цикле `for` переменная `N` является результатом преобразования величины сдвига `S` в целое число. Кроме того, мы присваиваем значение входного вектора `D` локальной переменной `TPMD`, которая в цикле `for` сдвигается `N` раз. Тело цикла `for` образует единственный оператор присваивания. В нем берется 15-битовый отрезок слова данных `[TPMD(14 downto 0)]` и осуществляется конкатенация `[&]`; результат возвращается в `TPMD` вместе с битом `[TPMD(15)]`, который «выдвинулся» с левого края. Подобными действиями можно описать и другие типы сдвига. Заметьте, что функции сдвига нельзя было бы определить в другом, поведенческом описании объекта `barrel16`, например, в структурной архитектуре.

Часть «параллельных операторов» в этой архитектуре исчерпывается единственным процессом, список чувствительности которого составляют все входы объекта. Оператор `case` этого процесса присваивает результат выводу `DOUT`, вызывая соответствующую функцию в зависимости от значения сигнала `C` на входе выбора режима.

Процесс, приведенный в табл. 6.17, является хорошим поведенческим описанием устройства быстрого сдвига, но многие средства синтеза не смогут синтезировать схему по такому описанию. Проблема заключается в том, что большинству программных средств требуется, чтобы диапазон цикла `for` был статическим на момент компиляции, тогда как у цикла `for` в функции `Vrol` диапазон динамический: он зависит от значения входного сигнала `S` во время работы схемы.

Ну, действительно трудно представить себе, какую схему могла бы выдать программа синтеза даже в том случае, если бы она была способна обрабатывать циклы `for` с динамическим диапазоном. Это пример того, когда разработчику следует хотя бы немного порекомендовать средствами синтеза при выборе структуры схемы, если есть желание получить достаточно быстрый и эффективный результат.

На рис. 6.2 было показано, как может выглядеть 16-разрядное устройство быстрого сдвига, выполняющее циклические сдвиги влево, собранное из стандартных ИС средней степени интеграции. В этой схеме последовательно один за другим включены четыре 16-разрядных 2-входовых мультиплексора, осуществляющих сдвиг входных данных на 0 разрядов или на 1, 2, 4 и 8 разрядов в зависимости от значений сигналов `S0–S3` соответственно. Подобный характер поведения и структуру такого вида можно описать средствами языка VHDL так, как это сделано в программе, приведенной в табл. 6.18. Несмотря на то, что в программе используется процесс и она написана в «поведенческом» стиле, можно быть более или менее уверенным в том, что для каждого оператора “`if`” в большинстве случаев синтезатор создаст по 2-входовому мультиплексору, которые образуют последовательную цепочку, подобную приведенной на рис. 6.2.

Табл. 6.18. Программа на языке VHDL для 16-разрядного устройства быстрого сдвига, осуществляющего только циклический сдвиг влево

```

library IEEE;
use IEEE std_logic_1164 all;

entity rol16 is
  port (
    DIN  in STD_LOGIC_VECTOR(15 downto 0), -- Data inputs
    S    in STD_LOGIC_VECTOR(3 downto 0),   -- Shift amount, 0-15
    DOUT out STD_LOGIC_VECTOR(15 downto 0) -- Data bus output
  ),
end rol16;

architecture rol16_arch of rol16 is
begin
  process(DIN, S)
    variable X, Y, Z : STD_LOGIC_VECTOR(15 downto 0);
  begin
    if S(0)='1' then X := DIN(14 downto 0) & DIN(15); else X := DIN; end if;
    if S(1)='1' then Y := X(13 downto 0) & X(15 downto 14); else Y := X; end if;
    if S(2)='1' then Z := Y(11 downto 0) & Y(15 downto 12); else Z := Y; end if;
    if S(3)='1' then DOUT <= Z(7 downto 0) & Z(15 downto 8); else DOUT <= Z; end if;
  end process;
end rol16_arch;

```

Но в решаемой здесь задаче требуется, чтобы устройство быстрого сдвига могло выполнять сдвиг и влево, и вправо. В табл. 6.19 представлена исправленная предыдущая программа, способная выполнять циклические сдвиги в любом направлении. Направление сдвига задается дополнительным входным сигналом DIR: 0 — для сдвига влево, 1 — для сдвига вправо. В каждом из звеньев, образующих последовательную цепочку, характер сдвига определяется оператором `case`, выбирающим одну из четырех возможностей по значению сигнала DIR и того бита S, который управляет этим звеном. Обратите внимание, что мы ввели локальные 2-разрядные переменные CTRL_i для хранения пары значений DIR и S {i}; каждый оператор `case` управляется одной из этих переменных. У вас может появиться желание исключить эти переменные и просто управлять каждым оператором `case` с помощью конкатенации DIR & S {i}, но синтаксис языка VHDL не позволяет это сделать, потому что тип конкатенации был бы неизвестен.

Типичный синтезатор языка VHDL создаст 3- или 4-входовой мультиплексор для каждого оператора `case` в табл. 6.19. Для последнего оператора `case` хороший синтезатор сформирует только 2-входовой мультиплексор.

Итак, теперь у нас есть устройство быстрого сдвига, которое будет выполнять циклические сдвиги влево или вправо, но мы сделали еще не все: необходимо позаботиться о логических и арифметических сдвигах в обоих направлениях. На рис. 6.14 представлен наш план завершения проекта. Согласно этому плану наше устройство начинается с блока ROLR16, разработку которого мы только что завершили, а для управления направлением сдвига, в зависимости от сигнала S, используется дополнительная логика.

Теперь необходимо «скорректировать» некоторые из полученных битов, если выполняется логический или арифметический сдвиг. При логическом или арифметическом сдвиге на n разрядов влево мы должны присвоить правым $n - 1$ би-

там значение 0 или первоначальное значение бита, находившегося в крайнем правом разряде соответственно. Для логического или арифметического сдвига на n разрядов вправо мы должны присвоить левым $n-1$ битам значение 0 или первоначальное значение бита, находившегося в крайнем левом разряде соответственно.

Табл. 6.19. Программа на языке VHDL для 16-разрядного устройства быстрого сдвига, выполняющего циклические сдвиги влево и вправо

```
library IEEE;
use IEEE.std_logic_1164.all;

entity rolr16 is
  port (
    DIN:  in STD_LOGIC_VECTOR(15 downto 0); -- Data inputs
    S:    in STD_LOGIC_VECTOR(3 downto 0);  -- Shift amount, 0-15
    DIR:  in STD_LOGIC;                     -- Shift direction, 0=>L, 1=>R
    DOUT: out STD_LOGIC_VECTOR(15 downto 0) -- Data bus output
  );
end rolr16;

architecture rol16r_arch of rolr16 is
begin
  process(DIN, S, DIR)
    variable X, Y, Z: STD_LOGIC_VECTOR(15 downto 0);
    variable CTRL0, CTRL1, CTRL2, CTRL3: STD_LOGIC_VECTOR(1 downto 0);
  begin
    CTRL0 := S(0) & DIR; CTRL1 := S(1) & DIR; CTRL2 := S(2) & DIR; CTRL3 := S(3) & DIR;
    case CTRL0 is
      when "00" | "01" => X := DIN;
      when "10" => X := DIN(14 downto 0) & DIN(15);
      when "11" => X := DIN(0) & DIN(15 downto 1);
      when others => null; end case;
    case CTRL1 is
      when "00" | "01" => Y := X;
      when "10" => Y := X(13 downto 0) & X(15 downto 14);
      when "11" => Y := X(1 downto 0) & X(15 downto 2);
      when others => null; end case;
    case CTRL2 is
      when "00" | "01" => Z := Y;
      when "10" => Z := Y(11 downto 0) & Y(15 downto 12);
      when "11" => Z := Y(3 downto 0) & Y(15 downto 4);
      when others => null; end case;
    case CTRL3 is
      when "00" | "01" => DOUT <= Z;
      when "10" | "11" => DOUT <= Z(7 downto 0) & Z(15 downto 8);
      when others => null; end case;
  end process;
end rol16r_arch;
```

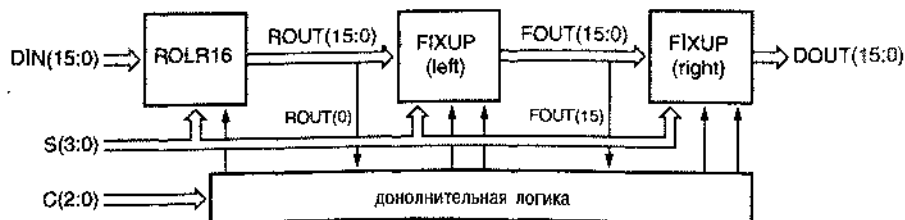


Рис. 6.14. Блок-схема устройства быстрого сдвига

Как показано на рис. 6.14, наша стратегия состоит в том, чтобы вслед за устройством ROLR16, осуществляющим циклический сдвиг, включить схему коррекции FIXUP(left), которая заполняет нужные младшие разряды при логическом или арифметическом сдвиге влево, и схему коррекции FIXUP(right), которая заполняет нужные старшие разряды при логическом или арифметическом сдвиге вправо.

В табл. 6.20 приведена поведенческая программа на языке VHDL для схемы коррекции при сдвиге влево. У нее имеются 16-разрядные входы и выходы данных DIN и DOUT соответственно. На входы управления поступают сигналы S, которыми задается величина сдвига, сигнал разрешения коррекции FEN и новая величина FDATA для вставки в корректируемые разряды выходных данных. Схема помещает значение корректирующего бита в каждый разряд данных на выходе DOUT(i), если i меньше S и коррекция разрешена; в противном случае на выходы схемы передаются значения битов на входе DIN(i) без изменений.

Табл. 6.20. Поведенческая программа на языке VHDL для схемы коррекции при сдвиге влево

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity fixup is
  port (
    DIN:  in STD_LOGIC_VECTOR(15 downto 0); -- Data inputs
    S:    in UNSIGNED(3 downto 0);           -- Shift amount, 0-15
    FEN:  in STD_LOGIC;                       -- Fixup enable
    FDATA: in STD_LOGIC;                     -- Fixup data
    DOUT: out STD_LOGIC_VECTOR(15 downto 0) -- Data bus output
  );
end fixup;

architecture fixup_arch of fixup is
begin
  process(DIN, S, FEN, FDATA)
  begin
    for i in 0 to 15 loop
      if (i < CONV_INTEGER(S)) and (FEN = '1') then DOUT(i) <= FDATA;
      else DOUT(i) <= DIN(i); end if;
    end loop;
  end process;
end fixup_arch;

```

Блок, соответствующий циклу `for` в табл. 6.20, синтезировать легко, но заранее нельзя быть уверенным, что его логика будет вполне подходящей. В частности, наличие операции "`<`", выполняемой на каждом проходе цикла, может заставить синтезатор включить универсальный компаратор, сравнивающий значения величин, несмотря на то, что один из операндов является константой, и поэтому сигнал на каждом выходе можно было бы получить с помощью небольшого числа вентилях. (Например, реализация логики "`7 < CONV_INTEGER(S)`" состоит всего лишь в подведении провода `S(3)`!) В рассуждении, вынесенном за пределы основного текста и озаглавленном «Последовательная структура схемы коррекции», говорится о структурном варианте этой функции.

ПОСЛЕДОВАТЕЛЬНАЯ СТРУКТУРА СХЕМЫ КОРРЕКЦИИ

Структурная архитектура схемы коррекции приведена в табл. 6.21. По существу, здесь определена итерационная схема, вырабатывающая 16-разрядный вектор FSEL с равным 1 значением FSEL(i), если i-й бит нуждается в коррекции. Процедура начинается с того, что биту FSEL(15) присваивается значение 0, поскольку в этом разряде никогда не требуется коррекция. Для остальных значений i величина FSEL(i) должна равняться 1, если значение S равно i+1, и в том случае, когда биту FSEL(i+1) уже присвоено единичное значение. Таким образом, присваивание значений битам FSEL оператором generate создает последовательную цепочку 2-входовых вентилей ИЛИ: один из входов каждого вентиля ИЛИ предназначен для подачи 1, когда S=i (что обнаруживается по результату декодирования 4-входовым вентилям И), а другой вход каждого следующего вентиля ИЛИ соединен с выходом предыдущего вентиля ИЛИ. Оператор присваивания DOUT(i) создает 16 двухвходовых мультиплексоров, каждый из которых выбирает бит входных данных DIN(i) или корректирующий бит (FDAT) в зависимости от значения FSEL(i).

Реализация схемы коррекции в виде последовательной цепочки оказывается компактной, но очень медленной по сравнению со схемой, в которой сигнал на каждом выходе FSEL вырабатывается двухуровневой логикой по выражениям вида «сумма произведений». Однако в данном случае большая задержка не имеет значения, поскольку схема коррекции располагается в конце пути прохождения данных. Если все же быстродействие существенно, то можно воспользоваться «бесплатным» приемом, который позволяет уменьшить задержку вдвое (см. задачу 6.12).

При сдвиге вправо коррекция начинается с противоположной стороны слова данных, поэтому может показаться, что необходима еще одна схема коррекции. Однако, как мы скоро увидим, для этого можно воспользоваться уже имеющейся схемой, если только изменить порядок следования входных и выходных битов на противоположный.

Табл. 6.21. Структурная VHDL-архитектура схемы коррекции при сдвиге влево

```
architecture fixup_struct of fixup is
  signal FSEL: STD_LOGIC_VECTOR(15 downto 0);      -- Fixup select
begin
  FSEL(15) <= '0'; DOUT(15) <= DIN(15);
  U1: for i in 14 downto 0 generate
    FSEL(i) <= '1' when CONV_INTEGER(S) = i+1 else FSEL(i+1);
    DOUT(i) <= FDAT when (FSEL(i) = '1' and FEN = '1') else DIN(i);
  end generate;
end fixup_struct;
```

В табл. 6.22 приведена объединенная структурная архитектура полного 16-разрядного устройства быстрого сдвига с 6-ю режимами работы, в котором реализован подход, изображенный на рис. 6.14. Объявление объекта barrel16

оставлено таким же, как и в табл. 6.17. В архитектуре объявлены два компонента: `rolr16` и `fixup`, для которых используются наши предыдущие определения объектов. Обращение к этим компонентам происходит в части программы, содержащей исполняемые операторы. Там же имеется несколько операторов присваивания, которые вырабатывают необходимые управляющие сигналы (то есть реализуют «дополнительную логику», указанную на рис. 6.14).

Табл. 6.22. Структурная VHDL-архитектура устройства быстрого сдвига с 6-ю режимами работы

```
architecture barrel16_struct of barrel16 is

  component rolr16 port (
    DIN: in STD_LOGIC_VECTOR(15 downto 0); -- Data inputs
    S:   in UNSIGNED(3 downto 0);          -- Shift amount, 0-15
    DIR: in STD_LOGIC;                      -- Shift direction, 0=>L, 1=>R
    DOUT: out STD_LOGIC_VECTOR(15 downto 0) -- Data bus output
  ); end component;

  component fixup port (
    DIN: in STD_LOGIC_VECTOR(15 downto 0); -- Data inputs
    S:   in UNSIGNED(3 downto 0);          -- Shift amount, 0-15
    FEN: in STD_LOGIC;                    -- Fixup enable
    FBAT: in STD_LOGIC;                   -- Fixup data
    DOUT: out STD_LOGIC_VECTOR(15 downto 0) -- Data bus output
  ); end component;

  signal DIR_RIGHT, FIX_RIGHT, FIX_RIGHT_DAT, FIX_LEFT, FIX_LEFT_DAT: STD_LOGIC;
  signal ROUT, FOUT, RFIXIN, RFIXOUT: STD_LOGIC_VECTOR(15 downto 0);

begin
  DIR_RIGHT <= '1' when C = Rrotate or C = Rlogical or C = Rarith else '0';
  FIX_LEFT <= '1' when DIR_RIGHT='0' and (C = Llogical or C = Larith) else '0';
  FIX_RIGHT <= '1' when DIR_RIGHT='1' and (C = Rlogical or C = Rarith) else '0';
  FIX_LEFT_DAT <= DIN(0) when C = Larith else '0';
  FIX_RIGHT_DAT <= DIN(15) when C = Rarith else '0';
  U1: rolr16 port map (DIN, S, DIR_RIGHT, ROUT);
  U2: fixup port map (ROUT, S, FIX_LEFT, FIX_LEFT_DAT, FOUT);
  U3: for i in 0 to 15 generate RFIXIN(i) <= FOUT(15-i); end generate;
  U4: fixup port map (RFIXIN, S, FIX_RIGHT, FIX_RIGHT_DAT, RFIXOUT);
  U5: for i in 0 to 15 generate DOUT(i) <= RFIXOUT(15-i); end generate;
end barrel16_struct;
```

Например, первый оператор присваивания устанавливает единичное значение сигнала `DIR_RIGHT`, когда значением `C` задается один из сдвигов вправо. При логических и арифметических сдвигах влево и вправо вырабатываются сигналы разрешения для схем коррекции `FIX_LEFT` и `FIX_RIGHT`. Значениям корректирующих битов присвоены имена `FIX_LEFT_DAT` и `FIX_RIGHT_DAT`.

Хотя все операторы в этой архитектуре выполняются одновременно, для удобства чтения они перечислены в табл. 6.22 в порядке фактического потока данных. Сначала вызывается компонент `rolr16` для выполнения основного циклического сдвига влево или вправо. Результат этого сдвига подается на вход первого компонента `fixup` (U2) для осуществления коррекции битов при логическом и арифметическом сдвигах влево. Затем следует оператор `generate` (U3), который изме-

СТИЛЬ УПРЯТЫВАНИЯ ИНФОРМАЦИИ

Зная, как кодируется управляющий сигнал *C*, вы, возможно, захотите написать первый оператор присваивания в табл. 6.22 в виде `DIR_RIGHT<=C(0)`, что гарантировало бы более эффективную реализацию схемы, которая вырабатывает этот управляющий сигнал: схема состояла бы всего лишь из одного соединения! Но при этом нарушился бы программистский стиль упрятывания информации, и это могло бы привести к появлению конструктивных недостатков.

Мы в явном виде записали коды сдвигов в объявлении объекта `barrel16` посредством определения констант. Архитектуре не нужно знать детали кодирования. Предположим, однако, что в нашей архитектуре произведена все же предложенная выше замена. Если бы кто-то другой (или мы сами!) захотел позднее изменить определения `constant` в объявлении объекта `barrel16`, задавая коды сдвигов иначе, то при новом способе кодирования уже нельзя было бы воспользоваться данной архитектурой! В задаче 6.13 требуется так изменить определения, чтобы объявление объекта позволяло уменьшить стоимость проекта, осуществив предложенную нами замену.

нует норядок следования битов данных для следующего обращения к компоненту `fixup (U4)`, производящему коррекцию при логическом и арифметическом сдвигах вправо. Наконец, другой оператор `generate (U5)` возвращает прежний порядок следования битов, измененный оператором `U3`. Заметьте, что исполнение `U3` и `U5` заключается в простом изменении порядка соединений.

Для исходного объекта `barrel16` можно написать много других архитектур. В задаче 6.14 мы предлагаем архитектуру, которая позволяет выполнять циклический сдвиг с помощью объекта `roll16`, использующего только 2-входные мультиплексоры, а не с помощью более дорогого объекта `rolr16`.

6.3.2. Простой шифратор для получения чисел с плавающей точкой

В разделе 6.1.2 мы определили простой формат числа с плавающей точкой и изложили задачу проектирования преобразователя числа с фиксированной точкой в число с плавающей точкой. Задача нахождения показателя экспоненты числа с плавающей точкой легко решается с помощью приоритетного шифратора, выполненного в виде ИС средней степени интеграции. При программировании на любом из языков описания схем решение той же самой задачи отображается в виде вложенных операторов `“if”`. В табл. 6.23 приведена новедеченская VHDL-программа шифратора для получения чисел с плавающей точкой. В пределах архитектуры `fpenc_arch` с помощью вложенного оператора `“if”` проверяется величина входной переменной `B` и устанавливаются соответствующие значения `M` и `E`. Обратите внимание, что в программе используется пакет `IEEE std_logic_arith`; это сделано для того, чтобы у нас были тип `UNSIGNED` и операции сравнения, которые сопровождают его, как было объяснено в разделе 5.9.6. Ради представления программы в компактном виде введен переменная `BU`, выражающая значение переменной `B` в формате типа `UNSIGNED`; в принципе, в каждом вложенном операторе `“if”` вместо `BU` можно написать `UNSIGNED (B)`.

Табл. 5.23. Поведенческая VHDL-программа для преобразования чисел с фиксированной точкой в числа с плавающей точкой

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity fpenc is
  port (
    B: in STD_LOGIC_VECTOR(10 downto 0); -- fixed-point number
    M: out STD_LOGIC_VECTOR(3 downto 0); -- floating-point mantissa
    E: out STD_LOGIC_VECTOR(2 downto 0) -- floating-point exponent
  );
end fpenc;

architecture fpenc_arch of fpenc is
begin
  process(B)
    variable BU: UNSIGNED(10 downto 0);
  begin
    BU := UNSIGNED(B);
    if BU < 16 then M <= B( 3 downto 0); E <= "000";
    elsif BU < 32 then M <= B( 4 downto 1); E <= "001";
    elsif BU < 64 then M <= B( 5 downto 2); E <= "010";
    elsif BU < 128 then M <= B( 6 downto 3); E <= "011";
    elsif BU < 256 then M <= B( 7 downto 4); E <= "100";
    elsif BU < 512 then M <= B( 8 downto 5); E <= "101";
    elsif BU < 1024 then M <= B( 9 downto 6); E <= "110";
    else M <= B(10 downto 7); E <= "111";
    end if;
  end process;
end fpenc_arch;

```

ПЕРЕМЕННАЯ "В" НЕ МОЕГО ТИПА

В табл. 6.23 мы использовали выражение `UNSIGNED(B)` для преобразования переменной `B`; массив типа `STD_LOGIC_VECTOR` преобразуется в массив типа `UNSIGNED`. Эта операция называется *явным преобразованием типов*. Язык VHDL позволяет преобразовывать тесно связанные между собой типы, записывая желаемый тип, за которым в круглых скобках следует преобразуемая величина. Два типа массивов считаются «тесно связанными», если у них один и тот же тип элементов, одна и та же размерность и одинаковые типы индексов (обычно `INTEGER`), а также те массивы, тип которых можно преобразовать. Элементы старого массива размещаются в новом массиве на соответствующих позициях в том же порядке слева направо.

Хотя программа, текст которой приведен в табл. 6.23, полностью синтезируема, некоторые программные средства синтеза могут оказаться не настолько толковыми, чтобы распознать, что во вложенных сравнениях на каждом уровне нужно проверять только один бит. Вместо этого такие программные средства могут для каждого уровня создать полный 11-разрядный компаратор. Такая логическая схема была бы намного больше и работала бы медленнее, чем то, что можно было бы сделать. Когда мы сталкиваемся с такой проблемой, всегда можно записать архитектуру немного иначе и в более явном виде, чтобы помочь программе выйти из затруднения, как это сделано в табл. 6.24

Табл. 6.24. Другой вариант VHDL-архитектуры для преобразования чисел с фиксированной точкой в числа с плавающей точкой

```

architecture fpence_arch of fpenc is
begin
  process(B)
  begin
    if B(10) = '1' then M <= B(10 downto 7); E <= "111";
    elsif B(9) = '1' then M <= B( 9 downto 6); E <= "110";
    elsif B(8) = '1' then M <= B( 8 downto 5); E <= "101";
    elsif B(7) = '1' then M <= B( 7 downto 4); E <= "100";
    elsif B(6) = '1' then M <= B( 6 downto 3); E <= "011";
    elsif B(5) = '1' then M <= B( 5 downto 2); E <= "010";
    elsif B(4) = '1' then M <= B( 4 downto 1); E <= "001";
    else
      M <= B( 3 downto 0); E <= "000";
    end if;
  end process;
end fpence_arch;

```

С другой стороны, для увеличения функциональных возможностей нашего устройства может появиться желание воспользоваться реальными компараторами и затратить даже большее число вентилях. В частности, наш вариант устройства при нахождении битов мантиисы выполняет усечение, а не округление. Более точный результат достигается при округлении, но это приводит к гораздо более сложному устройству. Во-первых, чтобы прибавить 1 к выбранным битам мантиисы при округлении вверх, необходим сумматор. Однако добавление 1, когда мантииса уже равна 1111, выталкивает нас в диапазон, представляемый следующим значением показателя экспоненты, так что надо быть готовым к этому случаю. Наконец, никогда нельзя выполнить округление вверх, если до округления мантииса и показатель экспоненты равны 1111 и 111, поскольку в нашем представлении числа с плавающей точкой отсутствует большее значение числа, до которого следует округлять.

Программа, приведенная в табл. 6.25, выполняет желаемое округление. Функция `round` берет биты из 5 определенных разрядов числа с фиксированной точкой и возвращает в качестве результата четыре старших разряда из них с добавленной к ним 1, если младший разряд равен 1. Таким образом, если считать, что непосредственно слева от младшего разряда находится двоичная точка, то

округление происходит в том случае, когда отбрасываемая часть мантиссы равна 1/2 или больше. В каждом предложении вложенного оператора "if" в процессе выполняется сравнение, чтобы при округлении вверх выбранной величины не происходило «переполнения», которое переводило бы результат в диапазон чисел, представляемых следующим значением показателя экспоненты. В противном случае преобразование и округление происходят в следующем предложении. Последним предложением гарантируется, что не произойдет округления вверх, когда мы находимся на краю диапазона чисел, представимых в формате с плавающей точкой.

Табл. 6.25. Поведенческая VHDL-архитектура для преобразования числа с фиксированной точкой в число с плавающей точкой с округлением

```

architecture fpencr_arch of fpenc is
function round (BSLICE: STD_LOGIC_VECTOR(4 downto 0))
    return STD_LOGIC_VECTOR is
    variable BSU: UNSIGNED(3 downto 0);
begin
    if BSLICE(0) = '0' then return BSLICE(4 downto 1);
    else null;
        BSU := UNSIGNED(BSLICE(4 downto 1)) + 1;
        return STD_LOGIC_VECTOR(BSU);
    end if;
end;
begin
process(B)
    variable BU: UNSIGNED(10 downto 0);
begin
    BU := UNSIGNED(B);
    if BU < 16 then M <= B( 3 downto 0); E <= "000";
    elsif BU < 32-1 then M <= round(B( 4 downto 0)); E <= "001";
    elsif BU < 64-2 then M <= round(B( 5 downto 1)); E <= "010";
    elsif BU < 128-4 then M <= round(B( 6 downto 2)); E <= "011";
    elsif BU < 256-8 then M <= round(B( 7 downto 3)); E <= "100";
    elsif BU < 512-16 then M <= round(B( 8 downto 4)); E <= "101";
    elsif BU < 1024-32 then M <= round(B( 9 downto 5)); E <= "110";
    elsif BU < 2048-64 then M <= round(B(10 downto 6)); E <= "111";
    else M <= "1111"; E <= "111";
    end if;
end process;
end fpencr_arch;

```

Еще раз: результаты синтеза, выполненного по этой поведенческой программе, не обязательно окажутся эффективными. Помимо многочисленных операторов сравнения, мы должны теперь побеспокоиться относительно большого числа 4-разрядных сумматоров, которые могут возникнуть при синтезе как следствие многократных обращений к функции округления round. Вопрос о том, как следует изменить архитектуру, чтобы по ней синтезировался только один сумматор, оставлен читателю в качестве задачи 6.15.

«ПОЕДАНИЕ ВЕНТИЛЕЙ»

Для операции округления не требуется 4-разрядный сумматор, необходима только схема увеличения числа на 1, так как одно из слагаемых — всегда 1. Некоторые VHDL-средства могут выдать в результате синтеза полиый сумматор, в то время как другие могут оказаться настолько сообразительными, что синтезируют схему увеличения числа на 1, состоящую из существенно меньшего количества вентилей.

В некоторых случаях это может не быть существенным. Самые развитые программные средства проектирования устройств на основе ИС типа FPGA и специализированных ИС содержат программы *поглощения вентилей*. Эти программы ищут вентили с постоянными сигналами на входах и либо исключают такие вентили целиком, либо уменьшают число входов у таких вентилей. Например, можно исключить вентиль И, на одном из входов которого постоянно присутствует 1, а вентиль И с постоянно присутствующим 0 на одном из его входов можно заменить постоянным сигналом, равным 0.

Программа поглощения вентилей прослеживает влияние постоянных значений входных сигналов настолько далеко в схеме, насколько это возможно. Следовательно, такая программа может преобразовать 4-разрядный сумматор с постоянной 1 на одном из входов в более экономичную 4-разрядную схему увеличения числа на 1.

6.3.3. Двойной приоритетный шифратор

В этом примере мы воспользуемся языком VHDL для поведенческого описания реализуемого в ПЛУ приоритетного шифратора, который находит сигналы с активным уровнем с самым высоким приоритетом и со вторым по старшинству приоритетом в наборе из восьми входных сигналов запроса с высоким активным уровнем [R0...R7], среди которых сигнал R0 имеет высший приоритет. Сигналы A (2 downto 0) и AVALID будем использовать для представления запроса с самым высоким приоритетом, причем сигнал AVALID пусть принимает единичное значение только тогда, когда запрос с высшим приоритетом присутствует. Точно так же сигналы B (2 downto 0) и BVALID пусть представляют запрос со вторым по старшинству приоритетом.

В табл. 6.26 представлена поведенческая VHDL-программа для приоритетного шифратора. Вместо вложенных операторов “if”, как в предыдущем примере, мы применяем здесь цикл “for”. Этот подход позволяет нам обработать запросы как наивысшего, так и второго по старшинству приоритетов в пределах одного и того же цикла, проходя весь путь от сигнала с высшим приоритетом до сигнала с наименьшим приоритетом. Помимо пакета std_logic_1164, в программе используется пакет IEEE std_logic_arith, из которого берется функция CONV_STD_LOGIC_VECTOR. Эта функция в явном виде была приведена в табл. 4.39.

Обратите внимание, что порты AVALID и BVALID объявлены в программе как выходные сигналы вида buffer, поскольку они читаются в пределах архи-

тектуры. Если бы вы поместили объявление сигналов AVALID и BVALID в определение объекта и указали бы, что они являются выходными сигналами вида out, то в архитектуре можно было бы использовать тот же самый подход, но при этом вам пришлось бы объявить внутри процесса локальные переменные, соответствующие сигналам AVALID и BVALID. Обратите также внимание на то, что мы включили AVALID и BVALID в список чувствительности процесса. Хотя, строго говоря, делать это не обязательно, однако такой шаг предотвратит появление предупреждений, которые в противном случае стал бы выдавать компилятор по поводу использования значений сигналов, которых нет в списке чувствительности.

Табл. 6.26. Поведенческая VHDL-программа для двойного приоритетного шифратора

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity Vprior2 is
    port (
        R: in STD_LOGIC_VECTOR (0 to 7);
        A, B: out STD_LOGIC_VECTOR (2 downto 0);
        AVALID, BVALID: buffer STD_LOGIC
    );
end Vprior2;

architecture Vprior2_arch of Vprior2 is
begin
    process(R, AVALID, BVALID)
    begin
        AVALID <= '0'; BVALID <= '0'; A <= "000"; B <= "000";
        for i in 0 to 7 loop
            if R(i) = '1' and AVALID = '0' then
                A <= CONV_STD_LOGIC_VECTOR(i,3); AVALID <= '1';
            elsif R(i) = '1' and BVALID = '0' then
                B <= CONV_STD_LOGIC_VECTOR(i,3); BVALID <= '1';
            end if;
        end loop;
    end process;
end Vprior2_arch;

```

Возможен также другой подход к созданию двойного приоритетного шифратора с вложенным оператором "if". Пример программы, реализующей этот подход, приведен в табл. 6.27. Такой подход приводит к более длинной программе с большим числом возможных сбоев, но, с другой стороны, этот вариант может дать лучший результат синтеза; единственный способ узнать возможности конкретной программы состоит в том, чтобы синтезировать схему и проанализировать результаты с точки зрения задержки и числа логических ячеек или вентилей.

Табл. 6.27. Другой вариант VHDL-архитектуры для двойного приоритетного шифратора

```

architecture Vprior2i_arch of Vprior2 is
begin
    process(R, A, AVALID, BVALID)
    begin
        if R(0) = '1' then A <= "000"; AVALID <= '1';
        elsif R(1) = '1' then A <= "001"; AVALID <= '1';
        elsif R(2) = '1' then A <= "010"; AVALID <= '1';
        elsif R(3) = '1' then A <= "011"; AVALID <= '1';
        elsif R(4) = '1' then A <= "100"; AVALID <= '1';
        elsif R(5) = '1' then A <= "101"; AVALID <= '1';
        elsif R(6) = '1' then A <= "110"; AVALID <= '1';
        elsif R(7) = '1' then A <= "111"; AVALID <= '1';
        else
            A <= "000"; AVALID <= '0';
        end if;

        if R(1) = '1' and A /= "001" then B <= "001"; BVALID <= '1';
        elsif R(2) = '1' and A /= "010" then B <= "010"; BVALID <= '1';
        elsif R(3) = '1' and A /= "011" then B <= "011"; BVALID <= '1';
        elsif R(4) = '1' and A /= "100" then B <= "100"; BVALID <= '1';
        elsif R(5) = '1' and A /= "101" then B <= "101"; BVALID <= '1';
        elsif R(6) = '1' and A /= "110" then B <= "110"; BVALID <= '1';
        elsif R(7) = '1' and A /= "111" then B <= "111"; BVALID <= '1';
        else
            B <= "000"; BVALID <= '0';
        end if;
    end process;
end Vprior2i_arch;

```

Вложенные операторы “if” и “for” могут приводить в процессе синтеза к появлению цепочек с большими задержками. Чтобы гарантировать получение быстрого двойного приоритетного шифратора, необходимо при проектировании следовать структурному или полуструктурному подходу. Можно, например, начать с описания модели быстрого 8-входового приоритетного шифратора в стиле потока данных, используя идеи, нашедшие свое отражение в принципиальной схеме ИС 74x148, приведенной на рис. 5.50, или в соответствующей программе на языке ABEL (табл. 5.24). Затем можно два таких шифратора поместить в одну структуру, где для нахождения входа со вторым по старшинству приоритетом исключается вход с высшим приоритетом, чтобы найти второй вход, как это было в схеме, изображенной на рис. 6.6.

6.3.4. Расширение компараторов

Каскадное включение компараторов является чем-то таким, что мы обычно не стали бы делать в поведенческой модели, написанной на языке VHDL, потому что этот язык и пакет IEEE std_logic_arith позволяют нам непосредственно определять компараторы любой желаемой длины. Однако, в действительности может потребоваться запись структурных или полуструктурных VHDL-программ, которые специальным образом включают меньшие компоненты компаратора для получения высокой эффективности.

В табл. 6.28 приведена простая поведенческая модель 64-разрядного компаратора с выходами «равно» и «больше чем». В этой программе используется пакет IEEE std_logic_unsigned, чьи встроенные функции сравнения автоматически воспринимают все сигналы типа STD_LOGIC_VECTOR как целые числа без знака. Хотя эта программа безусловно синтезируема, быстродействие и размеры результирующей схемы зависят от «интеллектуальных возможностей» технологий программирования, которыми вы пользуетесь.

Табл. 6.28. Поведенческая VHDL-программа для 64-разрядного компаратора

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity comp64 is
    port ( A, B: in STD_LOGIC_VECTOR (63 downto 0);
          EQ, GT: out STD_LOGIC );
end comp64;

architecture comp64_arch of comp64 is
begin
    EQ <= '1' when A = B else '0';
    GT <= '1' when A > B else '0';
end comp64_arch;

```

Альтернативой может служить последовательное включение таких, например, меньших компонентов, как 8-разрядные компараторы. В табл. 6.29 представлена поведенческая модель 8-разрядного компаратора. Развитые программные средства синтеза могут по этой программе создать очень быстрый компаратор, но даже при меньших возможностях программных средств можно быть уверенным, что в любом случае такой компаратор будет значительно более быстрым, чем 64-разрядный компаратор.

Табл. 6.29. VHDL-программа для 8-разрядного компаратора

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity comp8 is
    port ( A, B: in STD_LOGIC_VECTOR (7 downto 0);
          EQ, GT: out STD_LOGIC );
end comp8;

architecture comp8_arch of comp8 is
begin
    EQ <= '1' when A = B else '0';
    GT <= '1' when A > B else '0';
end comp8_arch;

```

Теперь мы можем написать структурную программу, которая предусматривает создание восьми таких 8-разрядных компараторов, выходные сигналы которых пропускаются через дополнительную логику, чтобы найти полный результат сравнения. Один из способов, каким это можно сделать, показан в табл. 6.30. Оператор `generate` создает не только отдельные 8-разрядные компараторы, но также и логику последовательного включения, с помощью которой информация, необходимая для выдачи окончательного результата, собирается путем последовательного учета сигналов на выходах отдельных каскадов, начиная с самого старшего и кончая самым младшим.

Табл. 6.30. Структурная VHDL-архитектура для 64-разрядного компаратора

```

architecture comp64s_arch of comp64 is
  component comp8
    port ( A, B: in STD_LOGIC_VECTOR (7 downto 0);
          EQ, GT: out STD_LOGIC);
  end component;
  signal EQ8, GT8: STD_LOGIC_VECTOR (7 downto 0); -- =, > for 8-bit slice
  signal SEQ, SGT: STD_LOGIC_VECTOR (8 downto 0); -- serial chain of slice results
begin
  SEQ(8) <= '1'; SGT(8) <= '0';
  U1: for i in 7 downto 0 generate
    U2: comp8 port map (A(7+i*8 downto i*8), B(7+i*8 downto i*8), EQ8(i), GT8(i));
    SEQ(i) <= SEQ(i+1) and EQ8(i);
    SGT(i) <= SGT(i+1) or (SEQ(i+1) and GT8(i));
  end generate;
  EQ <= SEQ(0); GT <= SGT(0);
end comp64s_arch;

```

Если по архитектуре, приведенной в табл. 6.28, относительно простые программные средства могут выдать в качестве результата синтеза схему медленного итерационного компаратора, то в случае архитектуры из табл. 6.30 результатом синтеза будет более быстрое устройство, поскольку в нем в более явной форме «извлекается» информация из каждого 8-разрядного звена, которая затем пропускается через более быструю комбинационную схему (состоящую всего лишь из 8-уровневой логики И–ИЛИ, а не из 64-уровневой). Более сильные программные средства могут «распараллелить» 8-разрядный компаратор, предложив более быструю неитерационную структуру наподобие ИС средней степени интеграции 74х682 (см. рис. 5.84), и преобразовать нашу итерационную логику объединения сигналов, состоящую от отдельных каскадов, в двухуровневую схему, реализующую выражения вида «сумма произведений», подобные тем, какие были приведены в программе на языке ABEL в табл. 6.8.

6.3.5. Компаратор с управляемым режимом работы

В качестве следующего примера давайте предположим, что имеется система, в которой нужно, как правило, сравнивать два 32-разрядных двоичных слова, но иногда во входных словах необходимо игнорировать значения одного или двух младших разрядов. Режим работы задается двумя битами M1 и M0, как указано в табл. 6.9.

Желаемое функционирование разрабатываемого устройства совсем легко обеспечить средствами языка VHDL, используя оператор `case` для выбора нужного

поведения, как это сделано в программе в табл. 6.31. Эта программа представляет собой вполне доброкачественное поведенческое описание, которое также полностью синтезируемо. Однако у такого описания имеется все же существенный недостаток: оно, вероятнее всего, приведет к созданию в процессе синтеза трех отдельных компараторов для обнаружения равенства или неравенства сравниваемых величин (представленных 32, 31 и 30 двоичными разрядами), по одному на каждый из случаев в операторе case. Отдельные компараторы могут при этом быть или не быть быстродействующими, как это обсуждалось в предыдущем разделе, но в данном примере мы не будем подробно разбирать этот вопрос.

Табл. 6.31. VHDL-программа с поведенческой архитектурой для 32-разрядного компаратора с управляемым режимом работы

```

library IEEE;
use IEEE.std_logic_164.all;
use IEEE.std_logic_unsigned.all;

entity Vmodecmp is
  port ( M: in STD_LOGIC_VECTOR (1 downto 0);    -- mode
        A, B: in STD_LOGIC_VECTOR (31 downto 0); -- unsigned integers
        EQ, GT: out STD_LOGIC );                -- comparison results
end Vmodecmp;

architecture Vmodecmp_arch of Vmodecmp is
begin
  process (M, A, B)
  begin
    case M is
      when "00" =>
        if A = B then EQ <= '1'; else EQ <= '0'; end if;
        if A > B then GT <= '1'; else GT <= '0'; end if;
      when "01" =>
        if A(31 downto 1) = B(31 downto 1) then EQ <= '1'; else EQ <= '0'; end if;
        if A(31 downto 1) > B(31 downto 1) then GT <= '1'; else GT <= '0'; end if;
      when "10" =>
        if A(31 downto 2) = B(31 downto 2) then EQ <= '1'; else EQ <= '0'; end if;
        if A(31 downto 2) > B(31 downto 2) then GT <= '1'; else GT <= '0'; end if;
      when others => EQ <= '0'; GT <= '0';
    end case;
  end process;
end Vmodecmp_arch;

```

Более эффективное решение заключается в выполнении только одного сравнения входных слов по 30 старшим битам и получении окончательного результата с помощью дополнительной логики, которая реализует зависящую от режима работы функцию и позволяет, по мере необходимости, учитывать значения младших разрядов. Этот подход продемонстрирован в табл. 6.32. Результат сравнения 30 старших битов представлен внутри процесса двумя переменными: EQ30 и GT30. Затем используется оператор case, аналогичный приведенному в предыдущей архитектуре, посредством которого получается окончательный результат в зависимости от режима работы. Если желательно, то 30-разрядный компаратор можно оптимизировать в отношении быстродействия методами, которые были рассмотрены в предыдущем разделе.

Табл. 6.32. Более эффективная архитектура для 32-разрядного компаратора с управляемым режимом работы

```

architecture Vmodecpe_arch of Vmodecpe is
begin
  process (M, A, B)
    variable EQ30, GT30: STD_LOGIC; -- 30-bit comparison results
  begin
    if A(31 downto 2) = B(31 downto 2) then EQ30 := '1'; else EQ30 := '0'; end if;
    if A(31 downto 2) > B(31 downto 2) then GT30 := '1'; else GT30 := '0'; end if;
    case M is
      when "00" =>
        if EQ30='1' and A(1 downto 0) = B(1 downto 0) then
          EQ <= '1'; else EQ <= '0'; end if;
        if GT30='1' or (EQ30='1' and A(1 downto 0) > B(1 downto 0)) then
          GT <= '1'; else GT <= '0'; end if;
        when "01" =>
          if EQ30='1' and A(1) = B(1) then EQ <= '1'; else EQ <= '0'; end if;
          if GT30='1' or (EQ30='1' and A(1) > B(1)) then
            GT <= '1'; else GT <= '0'; end if;
          when "10" => EQ <= EQ30; GT <= GT30;
          when others => EQ <= '0'; GT <= '0';
        end case;
    end process;
  end Vmodecpe_arch;

```

6.3.6. Счетчик числа единиц

Несколько важных алгоритмов предусматривают счет числа единичных битов в слове данных. Подсчет числа единиц недавно был включен в системы команд ряда микропроцессоров в качестве одной из основных операций. В этом примере предполагается, что нам надо построить комбинационную схему, считающую число единиц в 32-разрядном двоичном слове, которая могла бы быть частью арифметическо-логического устройства микропроцессора.

Подсчет числа единиц совсем нетрудно описать в поведенческой VHDL-программе, как это видно из табл. 6.33. Эта программа вполне синтезируема, но результатом синтеза может оказаться очень медленная и неэффективная реализация, состоящая из 32 последовательно включенных 5-разрядных сумматоров.

Чтобы синтезировать счетчик числа единиц с лучшими параметрами, необходимо придумать экономичную структуру и затем описать ее в виде архитектуры. Такой структурой является дерево сумматоров, показанное на рис. 6.15. Полный сумматор (FA) выполняет сложение трех входных битов, вырабатывая 2-разрядную сумму. Пары 2-разрядных чисел складываются с помощью 2-разрядных сумматоров ADDER2, у каждого из которых имеется вход переноса, позволяющий добавить к сумме значение еще одного бита. Полученные 3-разрядные суммы объединяются 3-разрядными сумматорами ADDER3, а последние пара 4-разрядных сумм складывается в 4-разрядном сумматоре ADDER4. С учетом сигналов, подаваемых на входы переноса, эта древовидная структура обеспечивает подсчет числа единиц в 31 разряде, эта древовидная структура обеспечивает подсчет числа единиц в 31 разряде. При наличии единицы в оставшемся входном разряде это обстоятельство учитывается с помощью отдельного 5-разрядного устройства INCR5 увеличения числа на единицу.

Табл. 6.33. Поведенческая VHDL-программа для счетчика числа единиц в 32-разрядном слове

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_unsigned.all;

entity Vcntis is
    port ( D: in STD_LOGIC_VECTOR (31 downto 0);
          SUM: out STD_LOGIC_VECTOR (4 downto 0) );
end Vcntis;

architecture Vcntis_arch of Vcntis is
begin
    process (D)
        variable S: STD_LOGIC_VECTOR(4 downto 0);
    begin
        S := "00000";
        for i in 0 to 31 loop
            if D(i) = '1' then S := S + "00001"; end if;
        end loop;
        SUM <= S;
    end process;
end Vcntis_arch;

```

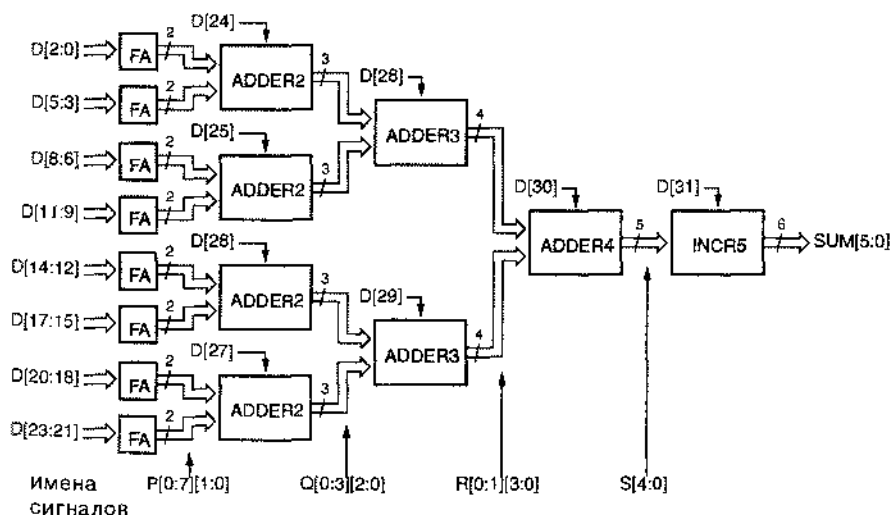


Рис. 6.15. Структура 32-разрядного счетчика числа единиц

Устройство, изображенное на рис. 6.15, отлично создается структурной VHDL-архитектурой, приведенной в табл. 6.34. Программа начинается с объявления всех компонентов, которые будут использованы в проекте, соответствующих блокам на рисунке.

Табл. 6.34. Структурная VHDL-архитектура для 32-разрядного счетчика числа единиц

```

architecture Vcnt1str_arch of Vcnt1str is
    component FA port ( A, B, CI: in  STD_LOGIC;
                      S, CO:  out STD_LOGIC );
    end component;

    component ADDER2 port ( A, B: in  STD_LOGIC_VECTOR(1 downto 0);
                          CI:  in  STD_LOGIC;
                          S:   out STD_LOGIC_VECTOR(2 downto 0) );
    end component;

    component ADDER3 port ( A, B: in  STD_LOGIC_VECTOR(2 downto 0);
                          CI:  in  STD_LOGIC;
                          S:   out STD_LOGIC_VECTOR(3 downto 0) );
    end component;

    component ADDER4 port ( A, B: in  STD_LOGIC_VECTOR(3 downto 0);
                          CI:  in  STD_LOGIC;
                          S:   out STD_LOGIC_VECTOR(4 downto 0) );
    end component;

    component INCR5 port ( A:  in  STD_LOGIC_VECTOR(4 downto 0);
                        CI: in  STD_LOGIC;
                        S:  out STD_LOGIC_VECTOR(5 downto 0) );
    end component;

    type Ptype is array (0 to 7) of STD_LOGIC_VECTOR(1 downto 0);
    type Qtype is array (0 to 3) of STD_LOGIC_VECTOR(2 downto 0);
    type Rtype is array (0 to 1) of STD_LOGIC_VECTOR(3 downto 0);
    signal P: Ptype; signal Q: Qtype; signal R: Rtype;
    signal S: STD_LOGIC_VECTOR(4 downto 0);

begin
    U1: for i in 0 to 7 generate
        U1C: FA port map (D(3*i), D(3*i+1), D(3*i+2), P(i)(0), P(i)(1));
    end generate;
    U2: for i in 0 to 3 generate
        U2C: ADDER2 port map (P(2*i), P(2*i+1), D(24+i), Q(i));
    end generate;
    U3: for i in 0 to 1 generate
        U3C: ADDER3 port map (Q(2*i), Q(2*i+1), D(28+i), R(i));
    end generate;
    U4: ADDER4 port map (R(0), R(1), D(30), S);
    U5: INCR5 port map (S, D(31), SUM);
end Vcnt1str_arch;

```

Под каждым столбцом сигналов в схеме на рис. 6.15 указано имя, присвоенное этой совокупности сигналов в программе. Каждый из сигналов P, Q и R — это массив, позволяющий представить все соединения в соответствующем столбце в виде одного вектора типа STD_LOGIC_VECTOR. Объявлению этих сигналов в программе предшествует определение соответствующих типов.

В этой программе создание однотипных сумматоров – восьми полных сумматоров FA, четырех сумматоров ADDER2 и двух сумматоров ADDER3 – успешно осуществляется операторами `generate`, а затем происходит обращение к компонентам ADDER4 и INCR5.

Определение компонентов счетчика числа единиц в виде отдельных объектов и архитектур, начиная с полного сумматора FA и кончая устройством INCR увеличения числа на 1, можно сделать в отдельных структурных или поведенческих программах. Например, в табл. 6.35 приведена структурная программа для полного сумматора FA. Описание остальных блоков оставлено в качестве задач 6.20–6.22.

Табл. 6.35. Структурная VHDL-программа для полного сумматора

```
library IEEE;
use IEEE.std_logic_1164.all;

entity FA is
    port ( A, B, CI: in  STD_LOGIC;
          S, CO:   out STD_LOGIC );
end FA;

architecture FA_arch of FA is
begin
    S <= A xor B xor CI;
    CO <= (A and B) or (A and CI) or (B and CI);
end FA_arch;
```

6.3.7. Игра в крестики и нолики

Наш последний пример состоит в проектировании комбинационной схемы, которая выбирает очередной ход игрока в игре в крестики и нолики. Те, кто не знаком с этой игрой, могут прочесть правила, приведенные в разделе 6.2.7. Здесь мы повторим нашу стратегию игры с целью достижения победы:

1. Ищем строку, столбец или диагональ, в которых имеется две наших метки (X или O в зависимости от того, за кого мы играем) и одна пустая клетка. Если такая комбинация существует, то помещаем свою метку в пустую клетку. Мы выиграли!
2. В противном случае ищем строку, столбец или диагональ, в которых имеется две метки противника и одна пустая клетка. Если такая комбинация обнаруживается, то помещаем свою метку в пустую клетку, чтобы заблокировать возможную победу противника.
3. Если не найдены две предыдущие комбинации, то выбираем клетку «на основании опыта». Например, если свободна центральная клетка, то обычно хорошим ходом является ее занятие. Другими хорошими ходами считается занятие угловых клеток. При выборе хода умные игроки могут также принять во внимание развитие конфигурации противником и заблокировать его, воспользовавшись «предвидением».

Чтобы избежать путаницы в наших программах между символами "О" и "o", присвоим второму игроку имя "Y". Теперь можно подумать о том, как кодировать входные и выходные сигналы схемы. Входные сигналы – это текущее состояние игрового поля. Всего в игровом поле девять клеток и каждая клетка находится в одном из трех возможных состояний (пустая, заполненная меткой X, заполненная меткой Y). Выходной сигнал – это ход, который надо сделать; предполагается, что очередной ход за игроком X. Игроку предстоит сделать только один из девяти возможных ходов, так что для представления выходного сигнала достаточно четырех битов.

Можно несколькими способами кодировать состояние одной клетки. Поскольку игра симметрична, выберем симметричное кодирование:

- 00 – клетка пуста;
- 10 – клетка занята меткой X;
- 01 – клетка занята меткой Y.

Такое представление поможет нам позднее.

Итак, состояние игрового поля размером 3×3 можно представить 18-ю битами. Поскольку язык VHDL поддерживает массивы, полезно определить тип массива `TTTgrid`, элементы которого соответствуют клеткам игрового поля. Так как мы повсюду в нашем проекте будем использовать этот тип, удобно включить его определение, наряду с несколькими другими, о которых будет сказано позднее, в VHDL-пакет, как показано в табл. 6.36.

Табл. 6.36. VHDL-пакет с определениями для устройства, играющего в крестики и нолики

```
library IEEE;
use IEEE.std_logic_1164.all;

package TTTdefs is

  type TTTgrid is array (1 to 3) of STD_LOGIC_VECTOR(1 to 3);
  subtype TTTmove is STD_LOGIC_VECTOR (3 downto 0);

  constant MOVE11: TTTmove := "1000";
  constant MOVE12: TTTmove := "0100";
  constant MOVE13: TTTmove := "0010";
  constant MOVE21: TTTmove := "0001";
  constant MOVE22: TTTmove := "1100";
  constant MOVE23: TTTmove := "0111";
  constant MOVE31: TTTmove := "1011";
  constant MOVE32: TTTmove := "1101";
  constant MOVE33: TTTmove := "1110";
  constant NONE:   TTTmove := "0000";

end TTTdefs;
```

Было бы естественно определить тип `TTTgrid` как двумерный массив элементов типа `STD_LOGIC`, но не все VHDL-средства поддерживают двумерные массивы. Поэтому введем массив 3-разрядных векторов типа `STD_LOGIC_VECTOR`, что является почти тем же самым. Для представления игрового поля игры в крестики и нолики воспользуемся двумя сигналами этого типа `X` и `Y`, где элемент переменной равен 1, когда у игрока с соответствующим именем стоит метка в данной клетке. На рис. 6.16 показано соответствие между именами сигналов и клетками на игровом поле.

	1	2	3	столбец
строка				
1	X(1)(1) Y(1)(1)	X(1)(2) Y(1)(2)	X(1)(3) Y(1)(3)	
2	X(2)(1) Y(2)(1)	X(2)(2) Y(2)(2)	X(2)(3) Y(2)(3)	
3	X(3)(1) Y(3)(1)	X(3)(2) Y(3)(2)	X(3)(3) Y(3)(3)	

Рис. 6.16. Поле для игры в крестики и нолики и имена сигналов в VHDL-программе

В пакете в табл. 6.36 определен также 4-разрядный тип `TTTmove` для представления ходов. У игрока есть девять возможных ходов, и нужно еще одно кодовое слово для случая, когда никакой ход не возможен. Выбор именно такого способа кодирования и его включение в пакет обусловлено только тем, что точно так же мы поступили в примере проектирования нашего устройства средствами языка ABEL в разделе 6.2.7. Благодаря тому, что способ кодирования определен в пакете, можно позднее изменить это определение без необходимости вносить изменения в использующие это определение объекты (см., например, задачу 6.23).

Имеет смысл не пытаться разработать устройство, выбирающее ход в игре в крестики и нолики, как единое целое, а попробовать разбить его на меньшие блоки. Действительно, представляется разумным разбиение устройства на блоки в соответствии с тремя шагами стратегии игры, указанными в начале этого раздела.

Отметим, что 1-й и 2-й шаги в нашей стратегии очень похожи: они отличаются только тем, что игроки меняются ролями. Объект, находящий победный ход для меня, может также находить ход, блокирующий выигрыш моего противника. С другой стороны, это означает, что объект, который находит победный ход для меня, может найти для меня и блокирующий ход, если поменять местами представление игрового поля с моей точки зрения и с точки зрения моего противника. Именно здесь дает выигрыш наше симметричное кодирование: игроки легко меняют местами простой перестановкой сигналов `X` и `Y`.

Имея это в виду, воспользуемся для выполнения 1-го и 2-го шагов двумя экземплярами одного и того же объекта `TwoInRow`, как показано на рис. 6.17. Обратите внимание, что сигнал `X` подан на верхний вход первого объекта `TwoInRow` и на нижний вход такого же второго объекта. Третий объект `PICK` выбирает победный ход, если он имеется на выходе объекта `U1`, в противном случае объект `PICK` выбирает блокирующий ход, если он имеется на выходе объекта `U2`; когда ни того, ни другого хода нет, в объекте `PICK` «используется опыт» для выбора хода на 3-м шаге.

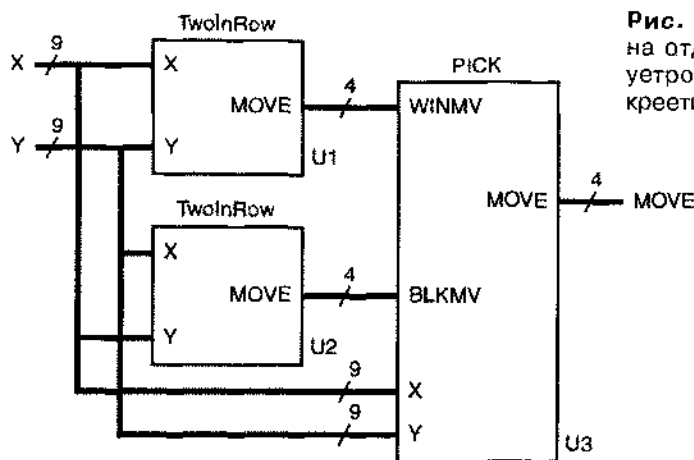


Рис. 6.17. Разбиение на отдельные объекты устройства для игры в крестики и нолики

Табл. 6.37 представляет собой структурную VHDL-программу для объекта верхнего уровня GETMOVE. Помимо пакета IEEE std_logic_1164 в ней используется наш пакет TTTdefs. Обратите внимание, что предложением “use” предписывается сохранение пакета TTTdefs в библиотеке “work”, которая создается автоматически для нашего проекта.

Табл. 6.37. Структурный VHDL-объект верхнего уровня для выбора хода в игре в крестики и нолики

```
library IEEE;
use IEEE.std_logic_1164.all;
use work.TTTdefs.all;

entity GETMOVE is
  port ( X, Y: in TTTgrid;
         MOVE: out TTTmove );
end GETMOVE;

architecture GETMOVE_arch of GETMOVE is

  component TwoInRow port ( X, Y: in TTTgrid;
                           MOVE: out STD_LOGIC_VECTOR(3 downto 0) );
  end component;

  component PICK port ( X, Y: in TTTgrid;
                       WINMV, BLKMOV: in STD_LOGIC_VECTOR(3 downto 0);
                       MOVE: out STD_LOGIC_VECTOR(3 downto 0) );
  end component;

  signal WIN, BLK: STD_LOGIC_VECTOR(3 downto 0);

begin
  U1: TwoInRow port map (X, Y, WIN);
  U2: TwoInRow port map (Y, X, BLK);
  U3: PICK port map (X, Y, WIN, BLK, MOVE);
end GETMOVE_arch;
```


В архитектуре в табл. 6.37 объявлены и используются только два компонента: TwoInRow и PICK, которые вскоре будут определены. От двух блоков TwoInRow к блоку PICK поступают только два внутренних сигнала WIN и BLK, приводящие к победному или блокирующему ходу, как и на рис. 6.17. Обработка этих сигналов производится всего лишь тремя операторами в исполняемой части архитектуры, соответствующими блокам, указанным на рисунке.

Теперь пришла очередь интересной работы: нужно создать отдельные объекты, изображенные на рис. 6.17. Начнем с объектов TwoInRow, так как они составляют две трети проекта. Согласно табл. 6.38, объявление такого объекта не представляет труда. Но в отношении его архитектуры, приведенной в табл. 6.39, есть целый ряд вопросов, которые следует обсудить.

```
library IEEE;
use IEEE.std_logic_1164 all;
use work.TTTdefs.all;

entity TwoInRow is
  port ( X, Y: in TTTgrid;
         MOVE: out TTTmove );
end TwoInRow;
```

Табл. 6.38. Объявление объекта TwoInRow

В архитектуре определены несколько функций, в каждой из которых решается, будет ли победным (с точки зрения игрока X) ход в конкретную клетку i, j . Победный ход существует, когда клетка i, j пуста и две другие клетки в той же строке, в том же столбце или на той же диагонали содержат метку X. Функции R и C выполняют поиск победного хода в клетки i, j по строкам и столбцам соответственно. Функции D и E осуществляют то же самое по двум диагоналям.

В единственном процессе архитектуры объявлены девять переменных G11–G33 типа BOOLEAN для указания возможности победного хода в каждую клетку. В начале процесса каждой из этих переменных присваивается значение TRUE, если посредством вызова нужных функций и объединения их результатов устанавливается, что ход в клетку i, j возможен.

Остальная часть процесса представляет собой оператор “if” с большой глубиной вложенных, который ищет победный ход во все возможные клетки. Хотя обычно это приводит к синтезу более медленной логики, тем не менее, вложенный оператор “if” предпочтительнее, нежели какая-либо разновидность оператора “case”, поскольку может быть несколько допустимых ходов. Если победного хода нет, то соответствующей переменной присваивается значение “NONE”.

ФУНКЦИИ ЧИСТЫЕ И НЕЧИСТЫЕ

Помимо индекса клетки i, j , в функциях R, C, D и E в табл. 6.39 передается состояние игрового поля в виде массивов X и Y. Это необходимо делать потому, что, по умолчанию, VHDL-функции являются *чистыми* (pure), а это означает, что сигналы и переменные, объявленные в структуре, порождающей функцию, непосредственно *не видимы* в пределах функции. Однако это ограничение можно ослабить путем явного объявления функций *нечистой*, помещая ключевое слово impure перед ключевым словом function в ее определении.

Табл. 6.39. Архитектура объекта TwoInRow

```

architecture TwoInRow_arch of TwoInRow is

function R(X, Y: TTTgrid; i, j: INTEGER) return BOOLEAN is
    variable result: BOOLEAN;
begin
    -- Find 2-in-row with empty cell i,j
    result := TRUE;
    for jj in 1 to 3 loop
        if jj = j then result := result and X(i)(jj)='0' and Y(i)(jj)='0';
        else result := result and X(i)(jj)='1'; end if;
    end loop;
    return result;
end R;

function C(X, Y: TTTgrid; i, j: INTEGER) return BOOLEAN is
    variable result: BOOLEAN;
begin
    -- Find 2-in-column with empty cell i,j
    result := TRUE;
    for ii in 1 to 3 loop
        if ii = i then result := result and X(ii)(j)='0' and Y(ii)(j)='0';
        else result := result and X(ii)(j)='1'; end if;
    end loop;
    return result;
end C;

function D(X, Y: TTTgrid; i, j: INTEGER) return BOOLEAN is
    variable result: BOOLEAN;
begin
    -- Find 2-in-diagonal with empty cell i,j.
    -- This is for 11, 22, 33 diagonal.
    result := TRUE;
    for ii in 1 to 3 loop
        if ii = i then result := result and X(ii)(ii)='0' and Y(ii)(ii)='0';
        else result := result and X(ii)(ii)='1'; end if;
    end loop;
    return result;
end D;

function E(X, Y: TTTgrid; i, j: INTEGER) return BOOLEAN is
    variable result: BOOLEAN;
begin
    -- Find 2-in-diagonal with empty cell i,j.
    -- This is for 13, 22, 31 diagonal.
    result := TRUE;
    for ii in 1 to 3 loop
        if ii = i then result := result and X(ii)(4-ii)='0' and Y(ii)(4-ii)='0';
        else result := result and X(ii)(4-ii)='1'; end if;
    end loop;
    return result;
end E;

begin
    process (X, Y)
        variable G11, G12, G13, G21, G22, G23, G31, G32, G33: BOOLEAN;
    begin
        G11 := R(X,Y,1,1) or C(X,Y,1,1) or D(X,Y,1,1);
        G12 := R(X,Y,1,2) or C(X,Y,1,2);
        G13 := R(X,Y,1,3) or C(X,Y,1,3) or E(X,Y,1,3);
        G21 := R(X,Y,2,1) or C(X,Y,2,1);
        G22 := R(X,Y,2,2) or C(X,Y,2,2) or D(X,Y,2,2) or E(X,Y,2,2);
        G23 := R(X,Y,2,3) or C(X,Y,2,3);
    end
end

```

Табл. 6.39. Архитектура объекта TwoInRow (продолжение)

```

G31 := R(X,Y,3,1) or C(X,Y,3,1) or E(X,Y,3,1);
G32 := R(X,Y,3,2) or C(X,Y,3,2);
G33 := R(X,Y,3,3) or C(X,Y,3,3) or D(X,Y,3,3);
if G11 then MOVE <= MOVE11;
elsif G12 then MOVE <= MOVE12;
elsif G13 then MOVE <= MOVE13;
elsif G21 then MOVE <= MOVE21;
elsif G22 then MOVE <= MOVE22;
elsif G23 then MOVE <= MOVE23;
elsif G31 then MOVE <= MOVE31;
elsif G32 then MOVE <= MOVE32;
elsif G33 then MOVE <= MOVE33;
else MOVE <= NONE;
end if;
end process;
end TwoInRow_arch;

```

Табл. 6.40. VHDL-программа для блока, который делает победный или блокирующий ходы в игре в крестики и нолики, либо «использует опыт» при выборе очередного хода, когда победного и блокирующего ходов нет

```

library IEEE;
use IEEE.std_logic_1164.all;
use work.TTTdefs.all;

entity PICK is
  port ( X, Y:          in TTTgrid;
         WINMV, BLKMV: in STD_LOGIC_VECTOR(3 downto 0);
         MOVE:          out STD_LOGIC_VECTOR(3 downto 0) );
end PICK;

architecture PICK_arch of PICK is
  function MT(X, Y: TTTgrid; i, j: INTEGER) return BOOLEAN is
  begin
    -- Determine if cell i,j is empty
    return X(i)(j)='0' and Y(i)(j)='0';
  end MT;
begin
  process (X, Y, WINMV, BLKMV)
  begin
    -- If available, pick:
    if WINMV /= NONE then MOVE <= WINMV; -- winning move
    elsif BLKMV /= NONE then MOVE <= BLKMV; -- else blocking move
    elsif MT(X,Y,2,2) then MOVE <= MOVE22; -- else center cell
    elsif MT(X,Y,1,1) then MOVE <= MOVE11; -- else corner cells
    elsif MT(X,Y,1,3) then MOVE <= MOVE13;
    elsif MT(X,Y,3,1) then MOVE <= MOVE31;
    elsif MT(X,Y,3,3) then MOVE <= MOVE33;
    elsif MT(X,Y,1,2) then MOVE <= MOVE12; -- else side cells
    elsif MT(X,Y,2,1) then MOVE <= MOVE21;
    elsif MT(X,Y,2,3) then MOVE <= MOVE23;
    elsif MT(X,Y,3,2) then MOVE <= MOVE32;
    else MOVE <= NONE; -- else grid is full
    end if;
  end process;
end PICK_arch;

```

В объекте PICK результаты работы двух объектов TwoInRow объединяются согласно программе, приведенной в табл. 6.40. Устанавливается приоритет победного хода по отношению к блокирующему ходу. Если таких ходов нет, то вызывается функция MT, которая для каждой клетки, начиная с центральной клетки и заканчивая клетками по бокам игрового поля, находит возможный ход. Этим завершается разработка устройства для игры в крестики и нолики.

Задачи

- 6.1. Объясните, как можно реализовать 16-разрядное устройство быстрого сдвига из раздела 6.1.1 на основе комбинации ИС 74х157 и 74х151. Сравните ваш вариант с другими возможными решениями с точки зрения задержки и количества ИС?
- 6.2. Покажите, как можно реализовать 16-разрядное устройство быстрого сдвига из раздела 6.1.1 с помощью восьми идентичных ИС GAL22V10.
- 6.3. Найдите способ кодирования величины сдвига ($S[3:0]$) и режима работы ($C[2:0]$) в устройстве быстрого сдвига из табл. 6.3, который приведет к сокращению общего числа термов-произведений, используемых в проекте.
- 6.4. Внесите изменения в программу двойного приоритетного шифратора (табл. 6.6), уменьшающие число требуемых термов-произведений. Определите, приведут ли ваши изменения к увеличению задержки схемы при ее реализации в ИС GAL22V10. Можете ли вы сократить число термов-произведений до такого значения, чтобы схема поместилась в ИС GAL16V8?
- 6.5. Вот вам возможность пошевелить мозгами подобно автору, когда ему пришлось выводить равенство для выходного сигнала SUM0 в табл. 6.12: потребуется ли для каждого из выходных сигналов SUM1–SUM3 большее или меньшее число термов-произведений, чем это оказалось необходимым для сигнала SUM0?
- 6.6. Завершите начатую в разделе 6.2.6 разработку схемы для подсчета числа единиц в слове на языке ABEL, имея в виду ее реализацию в ПЛУ. Используйте ИС 22V10 или меньшие ПЛУ и попытайтесь минимизировать общее число требуемых микросхем. Определите полную задержку вашего устройства в наихудшем случае в терминах числа задержек, вносимых отдельными ПЛУ, на пути сигнала от входа до выхода.
- 6.7. Найдите другой код для ходов в игре в крестики и нолики (табл. 6.13), который обладает теми же самыми свойствами в отношении поворота игрового поля, что и исходный код. Другими словами, должна быть возможной компенсация поворота поля на 180° с помощью только инверторов и переключения соединений. Определите, помещается ли схема, реализующая равенства для TWOINNAE при новом способе кодирования, в одной ИС 22V10.
- 6.8. Используя моделирующую программу, найдите последовательность ходов, при которой блок PICK2, реализованный в ПЛУ (табл. 6.16), проигрывает игру в крестики и нолики даже в том случае, когда игрок X ходит первым.

- 6.9. Измените программу в табл. 6.16 так, чтобы она имела больший шанс победить или, но крайней мере, не проиграть. Может ли все же ваша новая программа проиграть?
- 6.10. Измените блок «дополнительная логика» в схеме на рис. 6.13 и программу в табл. 6.16 так, чтобы дать программе больший шанс победить или, по крайней мере, не проиграть. Может ли все же ваша новая программа проиграть?
- 6.11. Запишите VHDL-функции *Vror*, *Vsll*, *Vsrl*, *Vsla* и *Vsra* для программы, приведенной в табл. 6.17, используя операции *ror*, *sll*, *srl*, *sla* и *sra*, определение которых дано в табл. 6.3.
- 6.12. В наилучшем случае задержка итерационной схемы коррекции *fixup* из табл. 6.20 от первого декодированного значения 1 (14) до появления сигнала *FSEL* (0) равна задержке, вносимой 15-ю вентилями ИЛИ. Найдите прием, который позволит сократить путь, приводящий к почти вдвое меньшей задержке, без затрат (или даже при сокращении затрат), выражаемых числом вентилях. Как можно продолжить применение этого приема для дальнейшего сокращения числа вентилях или числа входов у вентилях?
- 6.13. Перепишите объявление объекта *barrel16* в табл. 6.17 и определение архитектуры в табл. 6.22 так, чтобы единственный бит, определяющий направление сдвига, был в явном виде доступен архитектуре.
- 6.14. Перепишите определение архитектуры *barrel16* в табл. 6.22, используя структуру, показанную на рис. X6.14. Используйте имеющиеся объекты *ROL16* и *FIXUP*; вам остается придумать объект *MAGIC* и «дополнительную логику».

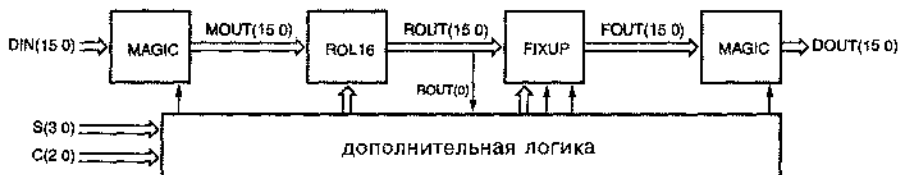


Рис. X6.14.

- 6.15. Напишите поведенческий или структурный варианты архитектуры *fpencr_arch* из табл. 6.25, в результате синтеза которых возник бы только один сумматор и не создавалось несколько 10-разрядных компараторов при реализации вложенного оператора “if”.
- 6.16. Повторите задачу 6.15, включая структурное определение экономичной схемы округления, которая выполняет функцию *round*. В вашей схеме должно быть значительно меньше вентилях, чем в 4-разрядном сумматоре.
- 6.17. Выполните заново разработку двойного приоритетного шифратора из раздела 6.3.3 на языке VHDL так, чтобы устройство обладало лучшими характеристиками в результате реализации идей, изложенных в последнем абзаце этого раздела.

- 6.18. Напишите структурную VHDL-архитектуру для 64-разрядного компаратора, которая была бы подобна архитектуре, приведенной в табл. 6.30, за исключением того, что результат сравнения в ней вырабатывается последовательно, начиная с самого младшего каскада и кончая самым старшим каскадом.
- 6.19. Какое существенное изменение произойдет в результате синтеза VHDL-программы из табл. 6.31, если мы заменим действия, предусмотренные альтернативой “when others” в операторе “case”, на “null”?
- 6.20. Напишите поведенческие VHDL-программы для компонентов ADDERx, фигурирующих в архитектуре из табл. 6.34.
- 6.21. Напишите структурные VHDL-программы для компонентов ADDERx, фигурирующих в архитектуре из табл. 6.34. Воспользуйтесь объявлением общности так, чтобы можно было обращаться к одному и тому же объекту из сумматоров ADDER2, ADDER3 и ADDER4, и укажите, какие изменения должны быть сделаны для этого в табл. 6.34.
- 6.22. Напишите структурную VHDL-программу для компонента INCR5 в табл. 6.34.
- 6.23. Используя имеющиеся VHDL-средства синтеза, синтезируйте устройство для игры в крестики и нолики из раздела 6.3.7, разместите его в подходящей ИС типа FPGA и посмотрите, сколько при этом расходуется внутренних ресурсов ИС. Затем попытайтесь сократить требуемый ресурс, задавая различные способы кодирования ходов в пакете TTTdefs.
- 6.24. Программа для игры в крестики и нолики, приведенная в разделе 6.3.7, в конце концов, проигрывает умному противнику, если состояние игрового поля оказывается таким, какое изображено на рис. X6.24. Используйте моделирующую VHDL-программу, которой вы располагаете, чтобы убедиться, что это так. Затем видоизмените объект PICK таким образом, чтобы побеждать в этой и в других подобных ситуациях, и проверьте правильность своего решения с помощью моделирующей программы.

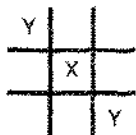
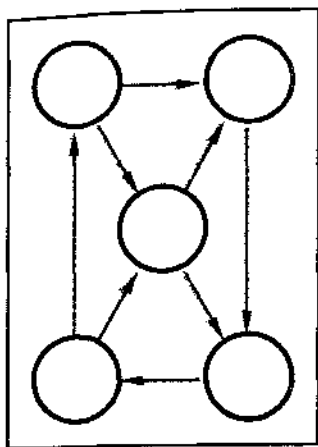


Рис. X6.24.



Глава 7

ПРИНЦИПЫ ПРОЕКТИРОВАНИЯ ПОСЛЕДОВАТЕЛЬНО- СТНЫХ ЛОГИЧЕСКИХ СХЕМ

Логические схемы подразделяются на два класса: «комбинационные» и «последовательностные». *Комбинационной* является такая логическая схема, сигналы на выходах которой зависят только от текущих значений входных сигналов. Барабанный переключатель каналов у старых телевизоров подобен комбинационной схеме: номер выбранного канала, служащий его «выходом», определяется только текущим положением ручки переключателя.

Последовательностная схема – это такая логическая схема, выходные сигналы которой определяются не только текущими значениями входных сигналов, но зависят также от последовательности значений входных сигналов в прошлом, причем возможно, что эта зависимость простирается сколь угодно далеко назад. Переключатель каналов телевизора или видеомаягнитофона, управляемый нажатием кнопок «вверх» и «вниз», является последовательностным устройством: выбор канала зависит от последовательности нажатий вверх/вниз, имевших место в прошлом, по крайней мере, с того момента, когда вы включили телевизор 10 часов назад, а, возможно, и с еще более давнего времени, когда вы впервые установили аппарат у себя дома.

Таким образом, неудобно, а часто невозможно, описать поведение последовательностной схемы таблицей, в которой выходные сигналы перечисляются как функции последовательности входных сигналов, принятой к настоящему моменту времени. Чтобы знать, где вы окажетесь в следующий момент, нужно знать, где вы находитесь сейчас. Нельзя определить, какой канал в данное время выберет переключатель каналов в вашем телевизоре, основываясь только на предшествующей последовательности нажатий управляющих кнопок, независимо от того, сколько предыдущих нажатий вы учтете – 10 или 1000. Нужно знать больше, необходима информация о текущем «состоянии» переключателя каналов. Лучшим, по-видимому, определением «состояния» из всего, что мне довелось увидеть, является определение, данное Хеллерманом (Herbert Hellerman) в его книге *Принципы построения компьютерных систем* (Digital Computer System Principles. McGraw-Hill, 1967):

Состояние (state) последовательностной схемы – это совокупность переменных состояния (state variables), чьи значения в любой фиксированный

момент времени содержат всю информацию о прошлом, необходимую для того, чтобы объяснить поведение схемы в будущем.

В примере с переключателем каналов текущий номер канала — это состояние переключателя в данный момент. Внутри телевизора это состояние может храниться в виде семи двоичных переменных состояния, представляющих десятичное число из интервала от 0 до 127. При заданном текущем состоянии (номер канала) мы всегда можем предсказать следующее состояние как функцию от входных воздействий (от нажатия кнопок переключения вверх или вниз). В этом примере один непосредственно наблюдаемый выходной сигнал последовательностной схемы — высвечиваемый номер канала — является выражением самого состояния в закодированном виде. Другие выходные сигналы (внутри телевизора) могут быть комбинационными функциями только состояния (например, выбор поддиапазона настройки приемника: VHF, UHF или кабельное телевидение), либо функциями состояния и входного воздействия (например, выключение телевизора, если текущее состояние переключателя каналов равно 0 и нажата клавиша «вниз»).

У переменных состояния может не быть прямого физического смысла. Как правило, существует много способов описания конкретной последовательностной схемы. Например, состояние переключателя каналов в телевизоре можно было бы представлять тремя двоично-десятичными цифрами или 12 двоичными разрядами, не используя многие из 4096 возможных комбинаций битов.

У цифровой схемы, как мы увидим дальше в этой главе, переменные состояния имеют двоичные значения, соответствующие определенным логическим сигналам в этой схеме. Схема с n двоичными переменными состояниями может находиться в одном из 2^n состояний. Как бы велико ни было число 2^n , оно всегда конечно и никогда не принимает бесконечного значения, поэтому последовательностные схемы называют *конечными автоматами* (*finite-state machines*).

В большинстве последовательностных схем изменение состояния происходит в моменты времени, задаваемые *тактовым сигналом* (*clock*) от независимого источника. На рис. 7.1 приведены временные диаграммы и терминология для типичных тактовых сигналов. Принято считать, что активным у тактового сигнала является высокий уровень, если состояние изменяется в момент, задаваемый нарастающим фронтом тактового сигнала, или тогда, когда тактовый сигнал имеет высокий уровень HIGH; в противном случае говорят, что активным у тактового сигнала является низкий уровень. *Периодом тактового сигнала* (*clock period*) называют отрезок времени между соседними переходами, совершаемыми сигналом в одном и том же направлении, а *частотой тактового сигнала* (*clock frequency*) — величину, обратную периоду. Первый перенос или импульс в пределах периода, а иногда и сам период называются *тактом системных часов* (*clock tick*). Выраженное в процентах относительное время, в течение которого тактовый сигнал имеет активный уровень, представляет собой *коэффициент заполнения* (*duty cycle*). Источником тактового сигнала в различных цифровых системах — от наручных часов до суперкомпьютеров — служит автономно работающий кварцевый генератор. Частота тактового сигнала колеблется в широком диапазоне — от 32,768 кГц (в наручных часах) до 500 МГц (в RISC-процессоре на основе КМОП-технологии с периодом 2 нс); в «типичной» системе, состоящей из ТТЛ- и КМОП-схем, частота тактового сигнала имеет значение от 5 до 150 МГц.

БЕСКОНЕЧНЫЕ АВТОМАТЫ

Группой математиков недавно были предложены конструкции, у которых число состояний не является конечным, и они все еще продолжают подсчет... Простите, это всего лишь шутка. *Существуют* математические модели автоматов с бесконечным числом состояний типа машин Тьюринга. Обычно они состоят из небольшого управляющего устройства, являющегося конечным автоматом, и вспомогательной памяти неограниченного объема типа бесконечной ленты.

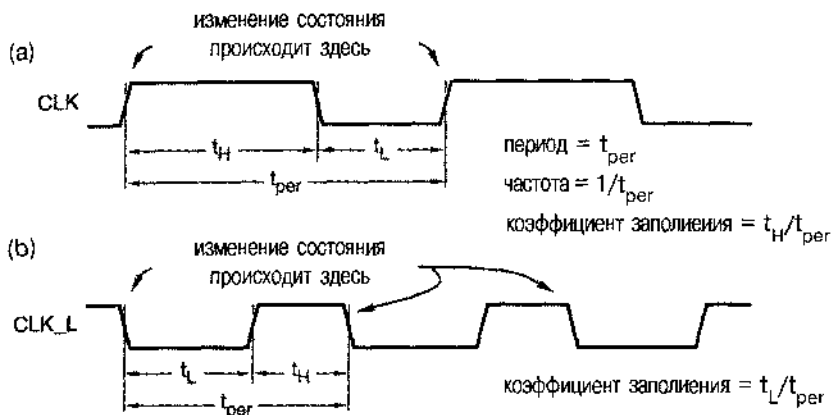


Рис. 7.1. Тактовые сигналы: (а) с высоким активным уровнем; (б) с низким активным уровнем

В этой главе мы рассмотрим последовательностные схемы двух типов, из которых состоит большинство практических цифровых устройств. *Последовательностная схема с обратной связью (feedback sequential circuit)*, построенная на обычных вентилях, благодаря наличию обратных связей обладает памятью; такого рода схемам являются стандартные узлы типа защелок и триггеров, используемые в качестве готовых блоков при проектировании на более высоком уровне. Эти узлы, в частности нереклюющиеся по фронту D-триггеры, используются в *тактируемых синхронных конечных автоматах (clocked synchronous state machines)* для создания устройств, в которых происходит опрос входных сигналов, а выходные сигналы изменяются в моменты времени, задаваемые управляющим тактовым сигналом. При разработке высокопроизводительных систем и СБИС иногда полезны рассматриваемые в продвинутых учебных курсах последовательностные схемы других типов: схемы с основным колебанием произвольного вида (*general fundamental mode circuit*), многоимпульсные и многофазные схемы.

7.1. Элементы с двумя устойчивыми состояниями

Простейшая последовательностная схема состоит из пары инверторов, охваченных петлей обратной связи, как показано на рис. 7.2. У этой схемы *нет* входов и есть два выхода: Q и Q_L.

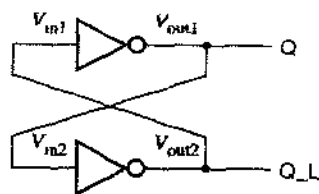


Рис. 7.2. Пара инверторов, образующих элемент с двумя устойчивыми состояниями

7.1.1. Цифровой подход

Приведенную на рис. 7.2 схему часто называют *бистабильной* (*bistable*), так как посредством формального цифрового анализа устанавливается, что у нее есть два устойчивых состояния. Прежде всего видим, что если на выходе Q имеет место высокий уровень, то, будучи подан на вход нижнего инвертора, он обеспечивает наличие низкого уровня на выходе этого инвертора, который, в свою очередь, удерживает сигнал на выходе верхнего инвертора на высоком уровне. Но если Q имеет низкий уровень, то нижний инвертор с низким уровнем на входе вырабатывает сигнал высокого уровня на своем выходе, который вынуждает верхний инвертор иметь низкий уровень на его выходе Q ; это другое устойчивое состояние. Состояние такой схемы можно было бы описать, воспользовавшись единственной переменной состояния — значением сигнала на выходе Q ; возможны два состояния: $Q = 0$ и $Q = 1$.

Этот элемент с двумя устойчивыми состояниями настолько прост, что у него нет входов, и поэтому нет возможности им управлять и изменять его состояние. Когда на схему подается напряжение питания, в ней случайно устанавливается то или другое состояние, в котором схема остается навсегда. Здесь эта схема нужна нам в качестве иллюстраций, но в разделах 8.2.3 и 8.2.4 мы приведем примеры, где она используется на самом деле.

7.1.2. Аналоговый подход

Мы узнаем об элементе с двумя устойчивыми состояниями заметно больше, если рассмотрим его работу с аналоговой точки зрения. Черной линией на рис. 7.3 изображена статическая (по постоянному току) передаточная характеристика T для одного инвертора; выходное напряжение является функцией входного напряжения: $V_{out} = T(V_{in})$. Для двух инверторов, соединенных в петлю обратной связи, как показано на рис. 7.2, имеем: $V_{in1} = V_{out2}$ и $V_{in2} = V_{out1}$; поэтому на одном и том же графике можно начертить передаточные характеристики для обоих инверторов, откладывая по осям координат соответствующие величины. Таким образом, черная кривая представляет собой передаточную характеристику верхнего на рис. 7.2 инвертора, а синяя кривая — передаточную характеристику нижнего инвертора.

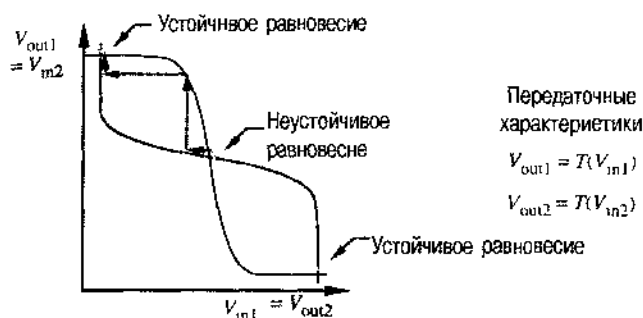


Рис. 7.3. Передаточные характеристики инверторов в петле обратной связи с двумя устойчивыми состояниями

Рассматривая только статическое поведение схемы с петлей обратной связи, но не динамические эффекты в ней, мы видим, что равновесие наступает в этой петле тогда, когда напряжения на входах и выходах обоих инверторов имеют постоянные значения, согласующиеся с требованиями, диктуемыми конфигурацией цепи, и с передаточными характеристиками инверторов на постоянному току. Другими словами, должны выполняться равенства:

$$\begin{aligned} V_{in1} &= V_{out2} \\ &= T(V_{in2}) \\ &= T(V_{out1}) \\ &= T(T(V_{in1})) \end{aligned}$$

и, аналогично,

$$V_{in2} = T(T(V_{in2})).$$

Воспользовавшись рисунком, можно найти точки равновесия графически — это точки, в которых пересекаются кривые, изображающие передаточные характеристики. Как ни странно, оказывается, что точек равновесия не две, а *три*. Две из них, отмеченные как «устойчивое равновесие» (*stable state*), соответствуют тем двум состояниям, которые мы обнаружили раньше при цифровом подходе: сигнал на входе Q имеет значение 0 (низкий уровень) или 1 (высокий уровень).

Третья точка, отмеченная как «неустойчивое равновесие» (*метастабильное состояние, metastable state*), соответствует случаю, когда напряжения V_{out1} и V_{out2} находятся примерно посередине между значениями напряжений, соответствующих логической 1 и логическому 0; в этой точке ни Q , ни $\bar{Q}$ не являются логическими сигналами в строгом смысле слова. Однако уравнения, описывающие поведение цепи, удовлетворяются и в этой точке; если бы нам удалось поставить схему в состояние, соответствующее данной точке, то, теоретически, схема могла бы оставаться в этом состоянии неограниченно долго.

7.1.3. Неустойчивое равновесие

Более детальный анализ того, что происходит в точке неустойчивого равновесия, позволяет воздать должное точности названия. Равновесие в этой точке не является по-настоящему устойчивым, поскольку случайный шум будет сталкивать схему из точки неустойчивого равновесия в сторону одного из устойчивых состояний. Объясним это подробнее.

Предположим, что рассматриваемая схема находится в точке неустойчивого равновесия, указанной на рис. 7.3. Предположим теперь, что из-за небольшого шума в цепи напряжение V_{in1} чуть-чуть уменьшается. Это совсем малое изменение вызовет определенное *увеличение* напряжения V_{out1} . Поскольку выходное напряжение верхнего инвертора V_{out1} является входным напряжением нижнего инвертора V_{in2} , мы можем перейти по первой горизонтальной стрелке из окрестности точки неустойчивого равновесия к другой точке на второй передаточной характеристике, согласно которой напряжение V_{out2} должно иметь теперь меньшее значение и, значит, меньшим должно стать равное ему напряжение V_{in1} . Вернемся снова к отправной точке наших рассуждений, за исключением того, что изменение напряжения V_{in1} теперь много больше, чем то, которое было вызвано шумом, и рабочая точка продолжает смещаться. Этот «регенеративный» процесс происходит до тех пор, пока не будет достигнуто устойчивое положение рабочей точки в верхнем левом углу на рис. 7.3. Если же мы предпримем «шумовой» анализ любой из устойчивых рабочих точек, то увидим, что обратная связь возвращает схему назад к устойчивой рабочей точке, а не в сторону от нее.

Неустойчивое равновесие рассматриваемой бистабильной схемы можно сравнить с поведением мяча, упавшего на вершину холма (рис. 7.4). Если мяч попадет на холм с нашей подачи, то, вероятнее всего, он немедленно скатится по ту или другую сторону от холма. Но если мяч аккуратно поместить точно на вершину, то он будет ненадежно располагаться там до тех пор, пока из-за какого-нибудь случайного воздействия (ветер, грызуны или землетрясение) не начнется его скатывание с холма. Подобно мячу на вершине холма, наша электронная схема может оставаться в состоянии неустойчивого равновесия неопределенное время до того момента, пока она недетерминированным образом не сместится в то или в другое устойчивое состояние.

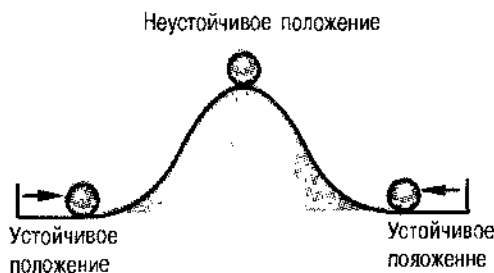


Рис. 7.4. Мяч на вершине холма в качестве иллюстрации неустойчивого равновесия

Даже у *простейшей* последовательной схемы имеется неустойчивое равновесие, так что можете быть уверены, что этим свойством обладают *все* последовательные схемы. Причем процессы, связанные с выходом из неустойчивого

равновесия, не являются чем-то таким, что происходит только при включении питания.

Возвращаясь к аналогии с мячом на холме, рассмотрим, что случится, если мы попытаемся ударом по мячу переместить его с одной стороны холма на другую. Если удар будет сильным (если он будет сделан Арнольдом Шварценеггером), то мяч перелетит через вершину и приземлится на другую сторону холма в месте, где его положение будет устойчивым. Если же удар будет слабым (если его произведет Mr. Rogers), то мяч скатится обратно к своему начальному положению. Но если удар по своей силе будет ни туда ни сюда (Charlie Brown), то мяч взлетит на вершину холма, покачается и в конце концов скатится по ту или по другую сторону.

Поведение мяча совершенно аналогично тому, что происходит с защелками и триггерами в условиях, когда они срабатывают от сигналов, имеющих предельно допустимые значения. Мы скоро встретимся, например, с SR-защелками, у которых импульс на входе S переводит схему из состояния 0 в состояние 1. Бывает задана минимальная длительность импульса на входе S. Если подать импульс такой или большей длительности, то защелка сразу же перейдет в состояние 1. При подаче очень короткого импульса схема останется в состоянии 0. Если длительность импульса чуть меньше минимального значения, то защелка может войти в метастабильное состояние. Поведение схемы, после того как она попала в метастабильное состояние, зависит от «формы ее холма». Защелки и триггеры, настроенные на элементах с большим усилением и быстродействием, выходят из неустойчивого равновесия, как правило, быстрее, чем схемы, собранные из более медленных элементов с меньшим усилением.

В следующем параграфе мы подробнее рассмотрим вопросы, связанные с неустойчивостью, применительно к защелкам и триггерам определенного типа, а в параграфе 8.9 эти вопросы будут обсуждены с позиций методологии синхронного проектирования и с разбором отказов из-за несовершенств в цепи синхронизации.

7.2. Защелки и триггеры

Защелки и триггеры являются основными составными элементами в большинстве последовательностных устройств. В типичных цифровых системах используются защелки и триггеры, находящиеся внутри функционально законченных узлов и блоков в стандартной интегральной схеме. В среде проектирования на основе специализированных ИС защелки и триггеры обычно бывают готовыми библиотечными элементами, которые предоставляет продавец самих интегральных микросхем. Однако внутри как обычных ИС, так и специализированных ИС, защелка или триггер представляют собой последовательностную схему с обратной связью, состоящую из отдельных логических вентилей и содержащую петли обратной связи. Мы займемся изучением этих конструкций из дискретных компонентов по двум причинам: во-первых, чтобы лучше понять поведение готовых элементов, а во-вторых, чтобы мы могли приобрести навыки построения защелок или триггеров, начиная с нуля, как это порой бывает необходимо в практике разработки цифровых устройств и как это часто требуется на экзаменах по цифровому проектированию.

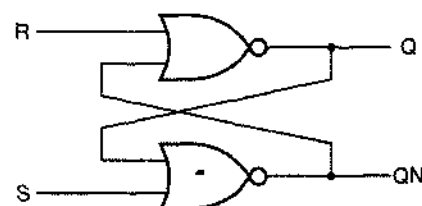
Все специалисты по цифровой электронике называют *триггером (flip-flop)* последовательностную схему, в которой значения входных сигналов принимаются во внимание и выходные сигналы изменяются только в моменты времени, задава-

емые тактовым сигналом. С другой стороны, большинство разработчиков цифровой техники используют название «защелка» (*latch*) для последовательностной схемы, которая чувствительна к сигналам на ее входах непрерывно в течение всего времени, и значения сигналов на выходах такой схемы могут изменяться в любой момент независимо от тактового сигнала. В этом учебнике мы будем следовать этому правилу. Однако в других учебниках и другими инженерами-электронщиками (неправильно) называется «триггером» устройство, которое мы называем «защелкой».

Поскольку функциональное поведение защелок и триггеров совершенно различно, разработчику в любом случае важно знать, к какому типу относится используемый им элемент; при этом все равно, откуда он узнает об этом: глядя на номер микросхемы (например, 74х374 или 74х373), или по той информации, какую можно почерпнуть из контекста. В следующих разделах мы рассмотрим защелки и триггеры наиболее распространенных типов.

7.2.1. SR-защелка

На рис. 7.5(а) показана *SR-защелка* (*set-reset latch*, *S-R latch*) на вентилях ИЛИ-НЕ. У этой схемы два входа S и R и два выхода Q и QN, где сигнал QN в нормальных условиях представляет собой инверсию сигнала Q. Сигнал QN иногда обозначают и так: $\bar{Q}$ или Q_L .



(а)

S	R	Q	QN
0	0	last Q	last QN
0	1	0	1
1	0	1	0
1	1	0	0

(b)

Рис. 7.5. SR-защелка: (а) принципиальная схема на вентилях ИЛИ-НЕ; (b) таблица, описывающая работу схемы (last – последнее значение)

Если оба входных сигнала S и R равны 0, то схема ведет себя аналогично элементу с двумя устойчивыми состояниями: благодаря наличию нетли обратной связи сохраняется одно из двух логических состояний — $Q = 0$ или $Q = 1$. Как указано в таблице на рис. 7.5(b), подавая сигналы на входы S и R можно заставить схему с петлей обратной связи переходить в желаемое состояние. Сигнал S *устанавливает* (*set*) или *задает* (*preset*) состояние, при котором выходной сигнал Q равен 1; сигнал R *сбрасывает* (*reset*) или *очищает* (*clear*) схему, в результате чего выходной сигнал Q становится равным 0. После того, как входной сигнал S или R переходит на неактивный уровень, защелка остается в том состоянии, в какое ее установил этот сигнал. На рис. 7.6 представлено функциональное поведение SR-защелки при воздействии на нее типичной последовательности входных сигналов. Цветными стрелками указана причинно-следственная связь, то есть показано, какие переходы во входных сигналах вызывают те или иные переходы в выходных сигналах.

ДОСТОИНСТВА И НЕДОСТАТКИ ОБОЗНАЧЕНИЙ Q И QN

В большинстве приложений SR-зашелок выходной сигнал QN (известный также под именем $\bar{Q}$) всегда является инверсией выходного сигнала Q. Однако обозначение $\bar{Q}$ не вполне корректно, поскольку существуют обстоятельства, при которых этот выходной сигнал не является инверсией сигнала Q. Когда оба входных сигнала S и R равны 1, как это происходит в нескольких местах на рис. 7.5(b), оба выходных сигнала вынужденно принимают значение 0. Как только любой из входных сигналов снимается, схема возвращается к работе в обычном режиме, и выходные сигналы становятся инверсными один по отношению к другому. Однако если входные сигналы снимаются одновременно, то состояние, в которое зашелка перейдет в следующий момент времени, непредсказуемо, и при этом могут возникнуть колебания, либо схема может войти в метастабильное состояние. Неустойчивость может наступать также в том случае, когда единичный импульс, подаваемый на входы S или R, слишком короткий.

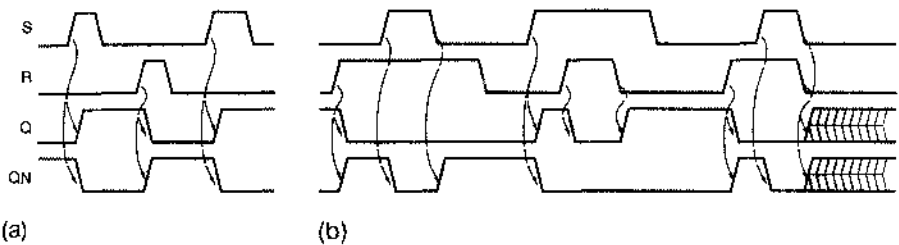


Рис. 7.6. Типичная работа SR-зашелки: (a) «нормальные» входные сигналы; (b) сигналы S и R имеют активный уровень одновременно

На рис. 7.7 приведены три условных обозначения одной и той же SR-зашелки. Эти обозначения различаются тем, как изображен инверсный выход. По исторически сложившейся традиции в первом из этих условных обозначений низкий активный уровень сигнала на этом выходе указывается в имени сигнала внутри прямоугольника, изображающего функциональный блок. Однако в логическом проектировании с использованием принципа «инверсия к инверсии» более предпочтительным является второе условное обозначение, согласно которому символ инверсии располагается снаружи функционального прямоугольника. Последнее условное обозначение, очевидно, является ошибочным.

На рис. 7.8 перечислены временные параметры SR-зашелки. *Задержка распространения (propagation delay)* – это время, которое уходит на то, чтобы переход во входном сигнале привел к переходу в сигнале на выходе. Для данной зашелки или для данного триггера может быть указано несколько значений задержки распространения, по одному на каждую пару вход-выход. Кроме того, значения задержки распространения могут быть различными в зависимости от того, в каком направлении изменяется выходной сигнал: от низкого уровня до

высокого или наоборот. У SR-защелки переход входного сигнала S с низкого уровня на высокий может вызвать переход выходного сигнала Q с низкого уровня на высокий; этому переходу соответствует задержка распространения $t_{pLH(SQ)}$ (случай 1 на рис. 7.8). Аналогично, переход входного сигнала R с низкого уровня на высокий может вызвать переход выходного сигнала Q с высокого уровня на низкий с задержкой распространения $t_{pHL(RQ)}$ (случай 2 на рис. 7.8). Не показанные на рисунке соответствующие переходы выходного сигнала QN можно было бы охарактеризовать задержками распространения $t_{pHL(SQN)}$ и $t_{pLH(RQN)}$.

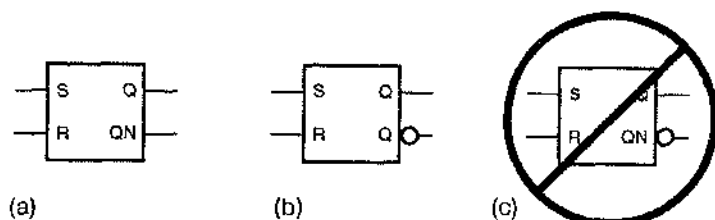


Рис. 7.7. Условное обозначение SR-защелки: (a) без символа инверсии; (b) предпочтительное обозначение при логическом проектировании с использованием принципа «инверсия к инверсии»; (c) неправильное обозначение, поскольку отрицание указано дважды

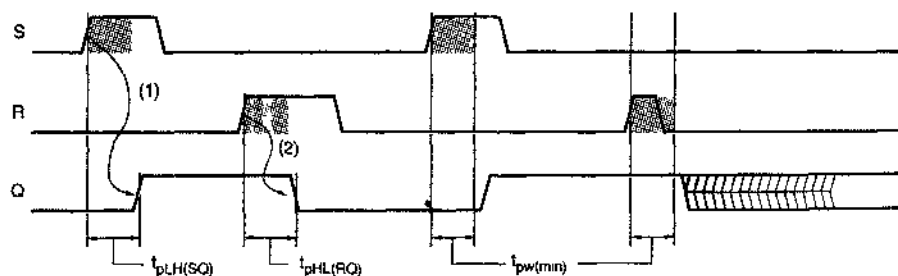


Рис. 7.8. Временные параметры SR-защелки

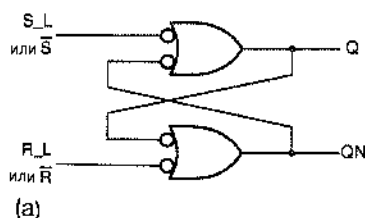
Для входных сигналов S и R обычно бывает задана *минимальная длительность импульса (minimum pulse width)*. Как показано на рис. 7.8, защелка может войти в метастабильное состояние и оставаться в нем в течение отрезка времени случайной длительности, если на входах S или R действует импульс, длительность которого меньше минимального значения $t_{pw(min)}$. Надежное непопадание защелки в метастабильное состояние возможно только в том случае, когда требования в отношении минимальной длительности импульса на входах S или R удовлетворены или превышены.

ЧТО ЗНАЧИТ «БЛИЗКО»?

Как уноминалось в предыдущем замечании, SR-защелка может войти в метастабильное состояние, если входные сигналы S и R переходят на неактивный уровень одновременно. В отношении серийно выпускаемых защелок часто, хотя и не всегда, понятие «одновременно» характеризуется вполне определенной величиной (это имеет место, например, если переходы сигналов S и R на неактивный уровень происходят в пределах отрезка времени длительностью не более 20 нс). Соответствующий параметр иногда называют *временем восстановления* (*recovery time*) t_{rec} . Это минимальный временной сдвиг между переходами S и R на неактивный уровень, при котором эти события считаются неодновременными. Параметр t_{rec} тесно связан с минимальной длительностью импульса. Обе характеристики являются мерой того, как долго петля обратной связи в защелке приходит в равновесие при изменении состояния.

7.2.2. $\overline{SR}$ -защелка

На рис. 7.9 представлена $\overline{SR}$ -защелка ($\overline{S}\text{-}\overline{R}$ latch; читается: S-бар-R-бар latch, S-с-чертой-R-с-чертой-защелка) на вентилях И-НЕ с низким активным уровнем сигналов установки и сброса. В логических семействах ТТЛ и КМОП $\overline{SR}$ -защелки используются гораздо чаще, чем SR-защелки, поскольку вентили И-НЕ предпочтительнее вентилей ИЛИ-НЕ.



S_L	R_L	Q	ON
0	0	1	1
0	1	1	0
1	0	0	1
1	1	last Q	last ON

(b)

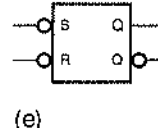


Рис. 7.9. $\overline{SR}$ -защелка: (a) принципиальная схема на вентиль И-НЕ; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение

Как видно из таблицы, описывающей работу схемы, принцип действия $\overline{SR}$ -защелки подобен механизму функционирования SR-защелки; правда, с двумя важными отличиями. Во-первых, у сигналов $\overline{S}$ и $\overline{R}$ активным является низкий уровень, так что при $\overline{S}=\overline{R}=1$ защелка помнит свое предыдущее состояние; на активный низкий уровень входных сигналов указывают символы инверсии на входах в условном обозначении. Во-вторых, при одновременном наличии сигналов активного уровня на входах $\overline{S}$ и $\overline{R}$ оба выходных сигнала защелки становятся

равными 1, а не 0, как это было в SR-защелке. За исключением этих отличий, $\overline{SR}$ -защелка работает точно так же, как SR-защелка, в том числе в отношении временных требований и метастабильности.

7.2.3. SR-защелка с входом разрешения

SR- и $\overline{SR}$ -защелки чувствительны к входным сигналам S и R в течение всего времени. Однако их легко видоизменить таким образом, чтобы схема была чувствительна к этим входным сигналам только тогда, когда подан сигнал на вход разрешения C. Такая SR-защелка с входом разрешения (*S-R latch with enable*) показана на рис. 7.10. Как видно из таблицы, описывающей работу схемы, при C, равном 1, данная схема ведет себя как SR-защелка, а при C, равном 0, она удерживается в прежнем состоянии. На рис. 7.11 приведены временные диаграммы, иллюстрирующие поведение этой схемы при типичном наборе входных воздействий. Если оба сигнала S и R равны 1 в момент, когда сигнал C переходит из 1 в 0, то схема ведет себя подобно SR-защелке при одновременном переходе сигналов S и R на неактивный уровень: следующее состояние непредсказуемо и выходная цепь может стать метастабильной.

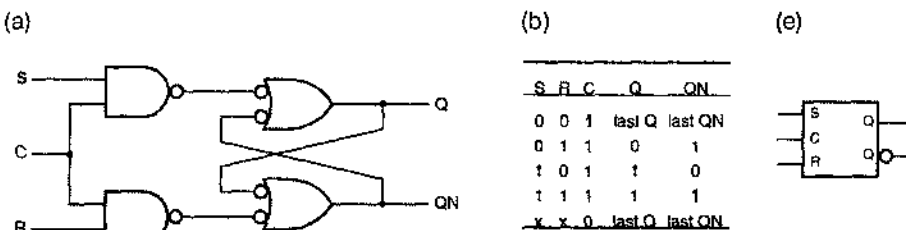


Рис. 7.10. SR-защелка с входом разрешения: (a) принципиальная схема на вентилях И-НЕ; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение



Рис. 7.11. Работа SR-защелки с входом разрешения в типичных условиях

7.2.4. D-защелка

SR-защелки полезны в разного рода управляющих устройствах, где часто возникают ситуации, когда в качестве реакции на выполнение того или иного условия нужно «выставить флаг», а при изменении условий его сбрасывать. В таких случаях управление входами установки и сброса осуществляется в определенной степени независимо. Однако часто бывают нужны защелки, чтобы просто запомнить биты информации, когда каждый бит поступает по отдельной сигнальной линии и его надо как-то сохранить. В задачах такого рода можно воспользоваться *D-защелками (D latch)*

D-защелка показана на рис. 7.12. Ее схема состоит из SR-защелки с входом разрешения и дополнительного инвертора, обеспечивающего формирование входных сигналов S и R из единственного входного сигнала D (data, данные). При этом устраняется присущая SR-защелке неприятность, связанная с одновременной подачей входных сигналов S и R активного уровня. На рис. (с) управляющий входной сигнал обозначен буквой C, но иногда его называют ENABLE, CLK или G и в некоторых схемах D-защелок у этого сигнала активным является низкий уровень.

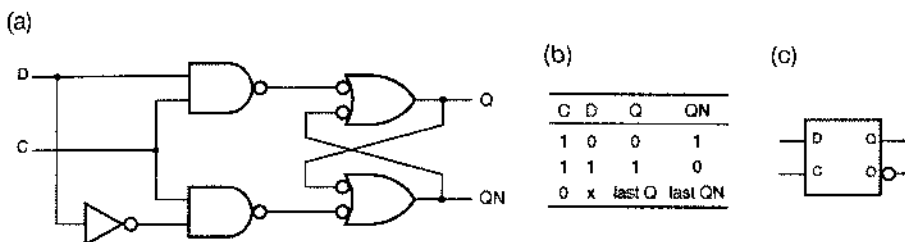


Рис. 7.12. D-защелка: (a) принципиальная схема на вентилях И-НЕ; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение

Пример функционального поведения D-защелки приведен на рис. 7.13. Когда подан входной сигнал C, выходной сигнал Q повторяет значения входного сигнала D. В этом случае говорят, что защелка «прозрачна» и путь от входа до выхода Q «открыт»; по этой причине данную схему часто называют *прозрачной защелкой (transparent latch)*. Когда сигнал C снимается, защелка запирается; выходной сигнал Q сохраняет свое последнее значение и больше не реагирует на изменения входного сигнала D, пока C остается на неактивном уровне.

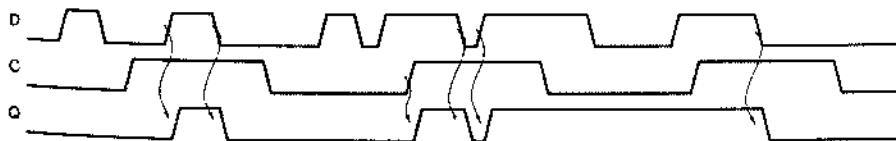


Рис. 7.13. Функциональное поведение D-защелки при различных входных сигналах

На рис. 7.14 подробнее показано, что происходит в D-защелке с течением времени. Четыре различных параметра задержки характеризуют прохождение сигналов от входов C и D до выхода Q. Например, к моментам переходов 1 и 4 защелка «заперта» и значение входного сигнала D противоположно значению выходного сигнала Q, поэтому когда C становится равным 1, защелка «отпирается» и выходной сигнал Q изменяет свое значение с задержками $t_{\text{rHL}}(\text{CQ})$ и $t_{\text{rLH}}(\text{CQ})$. В моменты переходов 2 и 3 входной сигнал C равен 1 и защелка открыта, так что выходной сигнал Q напрямую повторяет переходы во входном сигнале D, но с задержками $t_{\text{rHL}}(\text{DQ})$ и $t_{\text{rLH}}(\text{DQ})$. Четыре другие задержки, не указанные на рисунке, описывают задерживание, с которым происходят изменения выходного сигнала QN.

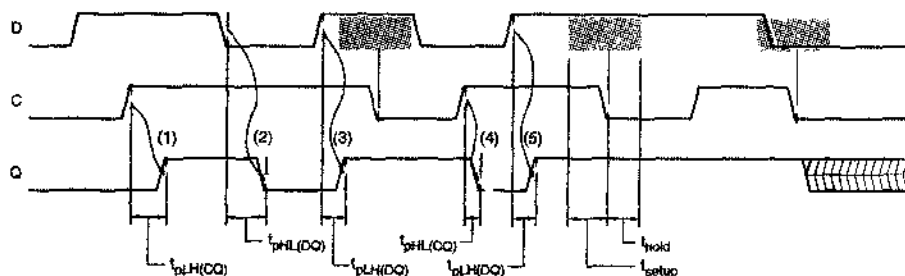


Рис. 7.14. Временные параметры D-защелки

Хотя с D-защелкой не возникает той проблемы, с какой мы сталкиваемся в случае SR-защелки, когда $S = R = 1$, затруднения, связанные с метастабильностью, все же остаются. Как показано на рис. 7.14, существует заштрихованный интервал времени в окрестности спадающего фронта в сигнале C, в пределах которого входной сигнал D не должен изменяться. Этот интервал начинается за время t_{setup} до спадающего фронта в сигнале C (до момента защелкивания); параметр t_{setup} называется *временем установления (setup time)*. Заканчивается рассматриваемый интервал спустя время t_{hold} ; величину t_{hold} называют *временем удержания (hold time)*. Если входной сигнал D изменяется в какой-то момент внутри интервала, состоящего из времени установления и времени удержания, то значение сигнала на выходе защелки непредсказуемо и состояние выходной цепи может оказаться метастабильным; этот случай изображен на рисунке вслед за последним защелкивающим перепадом.

7.2.5. D-триггер, переключающийся по фронту

Показанное на рис. 7.15 объединение пары D-защелок, называемое *D-триггером, переключающимся по положительному фронту (positive-edge-triggered D flip-flop)*, представляет собой схему, в которой опрос ее входа D и изменение ее выходных сигналов Q и QN происходит только в моменты времени, задаваемые нарастающим фронтом управляющего сигнала CLK. Первая защелка называется *ведущей (master)*; при значении CLK, равном 0, она открыта и ее выходной сигнал повторяет входной сигнал. Когда сигнал CLK становится равным 1, ведущая защелка запирается и ее выходной сигнал переносится во вторую защелку, называемую

ведомой (slave). Ведомая защелка открыта в течение всего времени, пока значение CLK остается равным 1, но изменение сигнала на ее выходе возможно только в самом начале этого интервала, так как ведущая защелка занерта и сигнал на ее выходе остается неизменным на протяжении всего этого отрезка времени.

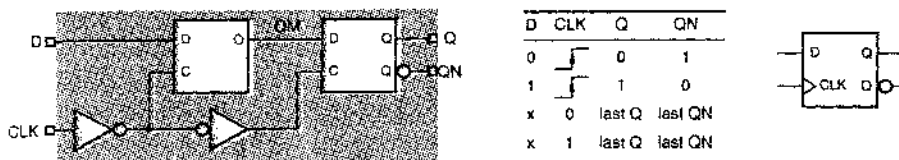


Рис. 7.15. D-триггер, переключающийся по положительному фронту: (a) принципиальная схема на D-защелках; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение

Треугольник на входе CLK у D-триггера указывает на срабатывание схемы по фронту и носит название *указателя динамического входа (dynamic-input indicator)*. Примеры функционального поведения триггера при нескольких переходах во входных сигналах приведены на рис. 7.16. Фигурирующий на этих временных диаграммах сигнал QM – это выходной сигнал ведущей защелки. Заметьте, что сигнал QM изменяется только при CLK, равном 0. Когда CLK становится равным 1, текущее значение QM переносится на выход Q, тогда как изменение сигнала QM невозможно до тех пор, пока CLK снова не станет равным 0.

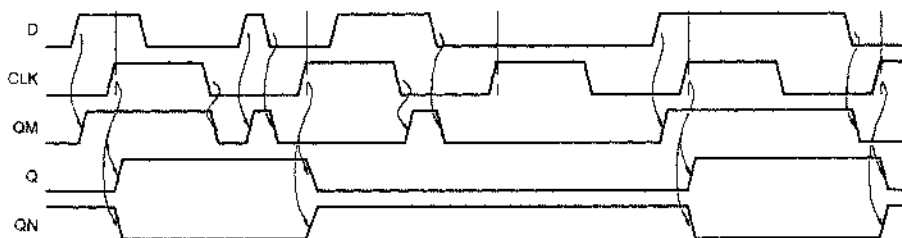


Рис. 7.16. Функциональное поведение D-триггера, переключающегося по положительному фронту

На рис. 7.17 временные зависимости сигналов в D-триггере представлены более подробно. Все задержки распространения измеряются относительно нарастающего фронта в сигнале CLK, поскольку только это событие вызывает изменение выходного сигнала. Задержки могут быть различными при переходе выходного сигнала с низкого уровня на высокий и в обратном направлении.

Так же, как и в случае D-защелки, у D-триггера есть интервал времени, состоящий из времени установления и времени удержания, в течение которого сигнал на входе D не должен изменяться. Этот интервал находится в окрестности переключающего фронта сигнала CLK и указан на рис. 7.17 цветной штриховкой. Если требования, предъявляемые временем установления и временем удержания, не удовлетворяются, то выход триггера, как правило, переходит в одно из устойчивых состояний

0 или 1, хотя предсказать, в какое именно, нельзя. Однако в некоторых случаях на выходе могут возникнуть колебания, либо выходная цепь может перейти в метастабильное состояние посередине между 0 и 1, как показано на рисунке после предпоследнего такта в управляющем сигнале. Если триггер попадает в метастабильное состояние, то со случайной задержкой он, в конце концов, сам вернется в одно из своих устойчивых состояний; подробнее это обстоятельство разбирается в параграфе 8.9. Можно принудительно перевести триггер в устойчивое состояние в момент действия следующего переключающего фронта в тактовом сигнале, по отношению к которому сигнал на входе D удовлетворяет требованию не изменяться в пределах времени установления и времени удержания; этот случай изображен на рис. 7.17, где он приходится на последний такт в управляющем сигнале.

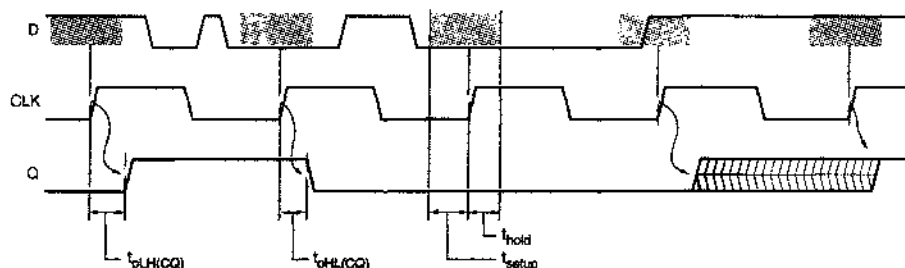


Рис. 7.17. Временные зависимости сигналов в D-триггере, переключающемся по положительному фронту

Чтобы получить *D-триггер, переключающийся по отрицательному фронту (negative-edge-triggered D flip-flop)*, достаточно инвертировать тактовый входной сигнал, так что все переключения будут происходить на спадающем фронте сигнала CLK_L; при срабатывании по спадающему фронту принято считать активным низкий уровень сигнала. Таблица, описывающая работу такого триггера, и его условное обозначение приведены на рис. 7.18.

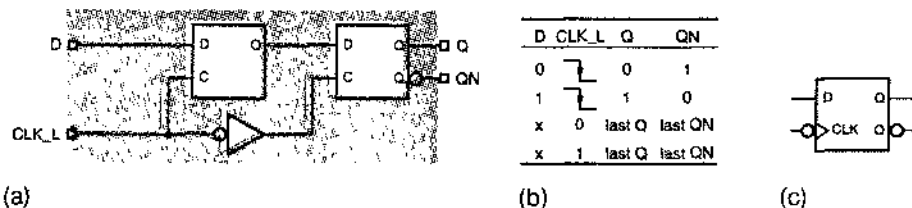


Рис. 7.18. D-триггер, переключающийся по отрицательному фронту: (a) принципиальная схема на D-зашелках; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение

У некоторых D-триггеров бывают *асинхронные входы (asynchronous inputs)*, воздействуя на которые можно переводить триггер в то или другое состояние независимо от сигналов на входах CLK и D. Обычно эти входы обозначаются PR (*preset*, установка в единичное состояние) и CLR (*clear*, сброс); по отношению к сигналам на этих входах D-триггер ведет себя подобно тому, как SR-зашелка реагирует на сигналы на входе установки в единичное состояние и входе сброса. Услов-

ное обозначение нереклюющего по фронту D-триггера с такими входами и его принципиальная схема на вентилях И-НЕ приведены на рис. 7.19. Хотя некоторые разработчики логических схем и используют асинхронные входы для реализации хитрых последовательностных функций, лучше всего сохранить их для целей тестирования и инициализации, то есть для установки последовательностной схемы в заданное начальное состояние; подробнее эти вопросы рассмотрены в параграфе 8.7 в связи с методологией синхронного проектирования.

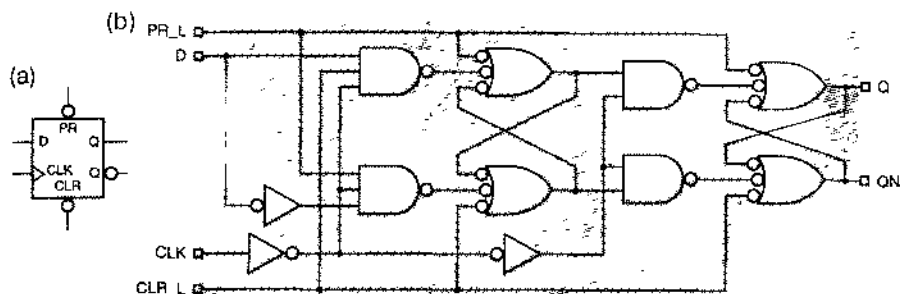


Рис. 7.19. D-триггер, переключающийся по положительному фронту, с входами уетановки и сброса: (а) условное обозначение; (б) принципиальная схема на вентилях И-НЕ

КАК УСТРОЕНЫ «НАСТОЯЩЕ» ТРИГГЕРЫ

Серийно производимые в семействах ТТЛ D-триггеры, нереклюющие по положительному фронту, устроены не по принципу «ведущий-ведомый», на основе которого действуют схемы, приведенные на рис. 7.15 и 7.19. Вместо этого в триггерах типа 74LS74 реализована схема на 6 вентилях (рис. 7.20), которая меньше по объему и быстрее. В параграфе 7.9 мы покажем, как формально проводится анализ переходов в каждом случае.

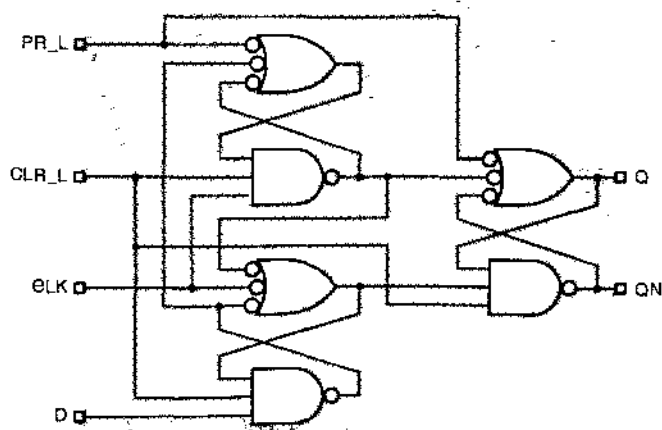


Рис. 7.20. Серийная схема D-триггера, переключающегося по положительному фронту, типа 74LS74

7.2.6. Переключающийся по фронту D-триггер с входом разрешения

Обычно бывает желательным, чтобы выполняемая D-триггерами функция состояла в удержании последнего запомненного значения, а не в загрузке нового значения на каждом такте управляющего сигнала. Это реализуется путем добавления *входа разрешения (enable input)*, обозначаемого как EN или CE (*clock enable, разрешение тактового сигнала*). Название «разрешение тактового сигнала» является описательным; оно вовсе не означает, что функция дополнительного входа состоит в пропуске или непропуске тактового сигнала. Напротив, как показано на рис. 7.21(а), с помощью 2-входового мультиплексора выбирается значение, подаваемое на внутренний D-вход триггера. Когда действует разрешающее значение сигнала EN, выбирается сигнал, поступающий на внешний D-вход; когда сигнал EN переходит на неактивный уровень, через мультиплексор проходит текущее значение сигнала на выходе триггера. В результате таблица, описывающая работу схемы, имеет вид, представленный на рис. (b). Условное обозначение такого триггера указано на рис. (c); у некоторых триггеров активным является низкий уровень сигнала на входе разрешения, что находит свое отражение в помещении символа инверсии на этом входе.

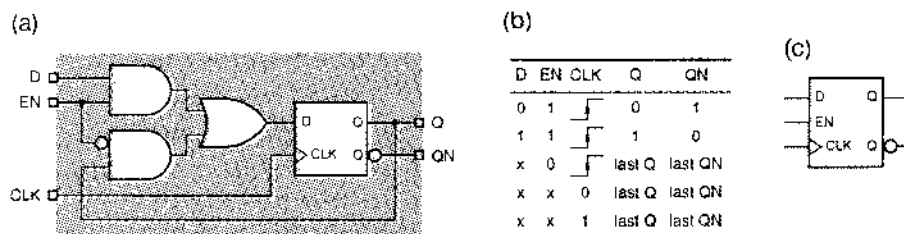


Рис. 7.21. Переключающийся по положительному фронту D-триггер с входом разрешения: (а) принципиальная схема; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение

7.2.7. Тестируемый триггер

При тестировании специализированных ИС важным свойством триггера является так называемая *возможность опроса (scan capability)*. Идея состоит в том, чтобы во время тестирования подавать на D-вход триггера данные от альтернативного источника. Когда все триггеры в данной специализированной ИС переведены в режим тестирования, в них – через альтернативные входы данных – «загружается» тестовая комбинация. После этого триггеры возвращаются в «нормальный» режим работы, и в этом режиме на все триггеры обычным образом подается тактовый сигнал. После одного или нескольких тактов, триггеры снова переводятся в режим тестирования, и результаты тестирования «считываются».

На рис. 7.22 показан типичный тестируемый триггер. Все, что есть в данной схеме, – это D-триггер с 2-входовым мультиплексором на входе D. Когда сигнал TE на *входе разрешения тестирования (test enable input)* имеет неактивное зна-

чение, схема ведет себя как обычный D-триггер. Когда же подан сигнал TE, разрешающий тестирование, триггер берет данные с входа TI (*test input, тестовый вход*), а не с входа D. Таблица, описывающая работу схемы, представлена на рис. (b), а условное обозначение данного устройства – на рис. (c).

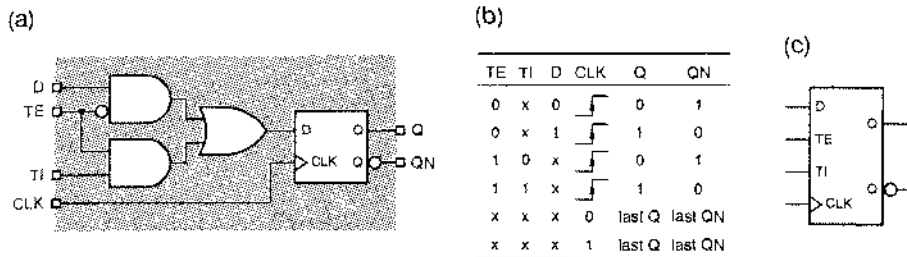


Рис. 7.22. Тестируемый D-триггер, переключающийся по положительному фронту: (a) принципиальная схема; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение

Дополнительные входы используются для соединения всех триггеров в специализированной ИС с целью тестирования в одну *цепочку сканирования (scan chain)*.

На рис. 7.23 приведен простой пример цепочки сканирования, состоящей из четырех триггеров. Входы TE всех триггеров объединяются вместе в один глобальный вход TE, тогда как выход Q каждого триггера соединяется с входом TI другого триггера, образуя последовательную цепочку (*daisy chain*). Соединения, относящиеся ко входам TI и TE, а также к выходу TO (*test output, тестовый выход*), предназначены исключительно для целей тестирования; другие соединения, относящиеся ко входам D и выходам Q, необходимые для того чтобы схема в целом могла делать что-то полезное, на рисунке не показаны.

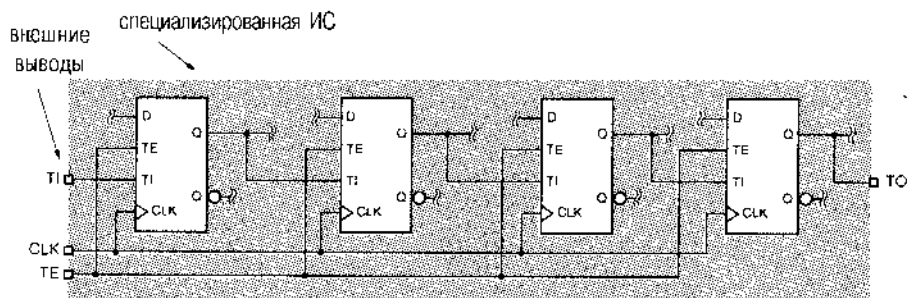


Рис. 7.23. Цепочка сканирования из четырех триггеров

Для того чтобы протестировать схему, в том числе и основную логику, разрешающий сигнал удерживается на глобальном входе TE в течение n периодов тактового сигнала, в то время как через глобальный вход TI в n триггеров посредством сдвига загружается n -разрядный тестовый вектор; на рис. 7.23 число n равно 4. Затем сигнал TE снимается и схеме предоставляется возможность функционировать в течение одного или нескольких следующих тактов. Новое состояние схемы,

представленное новыми значениями сигналов на выходах n триггеров, можно считать, наблюдая сигнал на выходе ТО в течение очередных n тактов при условии, что на входе ТЕ действует разрешающий сигнал. После того как результаты одного тестирования прочтены, можно загрузить другой проверочный вектор; такая возможность делает процесс тестирования более эффективным.

Существует столько различных типов тестируемых триггеров, сколько имеется различных способов реализации самих триггеров. Например, возможностью тестирования мог бы быть наделен D-триггер с входом разрешения, изображенный на рис. 7.21, путем замены его 2-входового внутреннего мультиплексора на 3-входовой. Тогда, в зависимости от значений сигналов EN и TE, в триггеры на каждом такте записывались бы сигналы с входов D или TI или его собственное текущее состояние. Возможность тестирования можно добавить также и в случае триггеров других типов, в частности JK-триггеров и T-триггеров, о которых речь пойдет позднее в этом параграфе.

*7.2.8. Двухтактный SR-триггер

Как указывалось ранее, SR-зашелки полезны в «управляющих устройствах», где возможны независимые условия установки и снятия контрольного бита. Если предполагается, что контрольный бит должен меняться только в определенные моменты времени, привязанные к тактовому сигналу, то нужен SR-триггер, подобный D-триггеру, у которого сигналы на выходах изменялись бы только на определенном перепаде тактового сигнала. В этом и в двух следующих разделах описаны триггеры, полезные при решении таких задач.

Если D-зашелки в D-триггере, переключающемся по отрицательному фронту [рис. 7.18(а)], заменить SR-зашелками, то получим *двухтактный SR-триггер* (*master/slave S-R flip-flop*; триггер, действующий по принципу ведущий-ведомый), показанный на рис. 7.24. Аналогично D-триггеру, выходные сигналы SR-триггера изменяются только по спадающему фронту в тактовом сигнале C. Однако новое значение выходного сигнала зависит теперь не от значений входных сигналов точно в тот момент времени, на который приходится спадающий фронт, а от значений входных сигналов на протяжении всего интервала времени, в течение которого сигнал C был равен 1 перед отрицательным перепадом. Как следует из временных диаграмм на рис. 7.25, короткий импульс S в любом месте в пределах указанного интервала времени может установить ведущую зашелку в единичное состояние; точно так же импульс на входе R может сбросить ее. Значение, переносимое на выход триггера в момент действия спадающего фронта в сигнале C, зависит от того, каким было последнее состояние ведущей зашелки, пока сигнал C оставался равным 1.

Согласно рис. 7.24(с), в условном обозначении двухтактного SR-триггера не используется указатель динамического входа, поскольку этот триггер не является истинно переключающимся по фронту. Он скорее похож на зашелку, в которой происходит повторение ее входного сигнала в течение интервала времени, на котором C равен 1, но изменения сигнала на выходе триггера отражают конечное зашелкнутое значение только тогда, когда C становится равным 0. На тот факт, что выходной сигнал не изменяется до тех пор, пока сигнал на входе разрешения C не перейдет на неактивный уровень, указывает *индикатор задержки*

вход R ведущей защелки только тогда, когда текущее значение Q, равно 1. Таким образом, при одновременном действии сигналов на входах J и K триггер переходит в состояние, противоположное тому, в котором он находится.

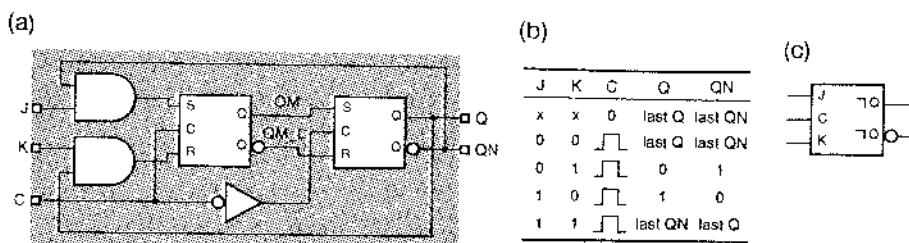


Рис. 7.26. Двухтактный JK-триггер: (a) схема на SR-защелках; (b) таблица, описывающая работу схемы (last – последнее значение); (c) условное обозначение

Временные диаграммы на рис. 7.27 иллюстрируют работу двухтактного JK-триггера при типичном наборе входных сигналов. Заметьте, что для изменения сигнала на выходе триггера в момент окончания переключающего импульса нет необходимости подавать входные сигналы J и K именно в этот момент времени. Действительно, коль скоро только один из входных сигналов проходит через вентиль на входы S и R ведущей защелки, сигнал на выходе триггера может стать равным 1 даже в том случае, когда к концу нереклюющего импульса действует только сигнал K, а сигнал J снят. Такое поведение, известное под названием *захвата единиц* (*1s catching*), продемонстрировано на рисунке: оно имеет место на предпоследнем переключающем импульсе. Аналогичное явление, называемое *захватом нулей* (*0s catching*), можно наблюдать на последнем нереклюющем импульсе. Из-за этого свойства двухтактного JK-триггера входные сигналы J и K должны оставаться неизменными в течение всего интервала времени, пока C равно 1.

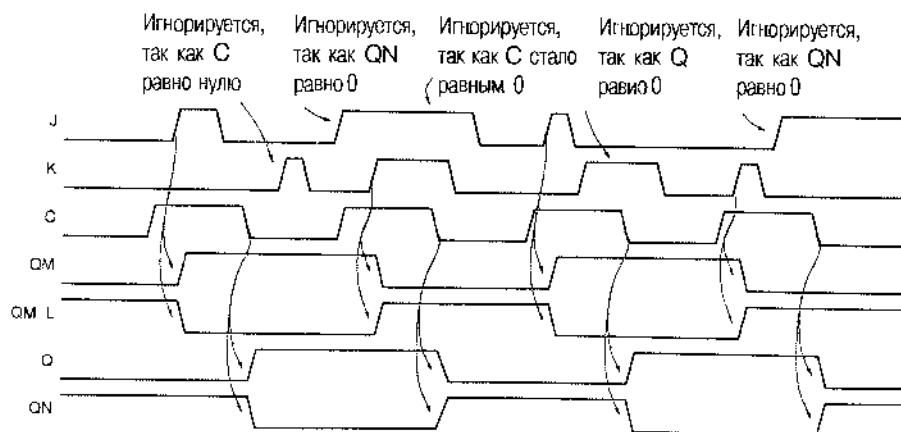


Рис. 7.27. Временные диаграммы, иллюстрирующие работу двухтактного JK-триггера

7.2.10. JK-триггер, переключающийся по фронту

Проблема захвата единиц и нулей решена в *JK-триггере, переключающемся по фронту* (*edge-triggered J-K flip-flop*), функциональный эквивалент которого изображен на рис. 7.28. Внутри него используется переключающийся по фронту D-триггер, благодаря чему входы JK-триггера опрашиваются на нарастающем фронте тактового сигнала и очередное значение выходного сигнала вырабатывается в соответствии с «характеристическим уравнением»: $Q^* = J \cdot Q' + K' \cdot Q$ (см. раздел 7.3.3).

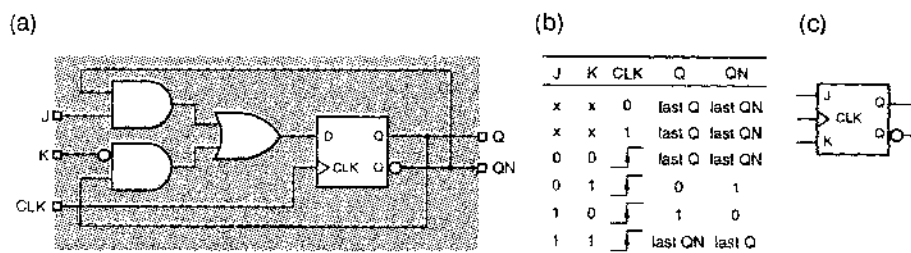


Рис. 7.28. JK-триггер, переключающийся по фронту: (а) функциональный эквивалент на переключающемся по фронту D-триггере; (б) таблица, описывающая работу схемы (last – последнее значение); (с) условное обозначение

Типичное поведение переключающегося по фронту JK-триггера представлено на рис. 7.29. Так же, как и в случае сигнала на входе D переключающегося по фронту D-триггера, для надлежащей работы JK-триггера его входные сигналы J и K в окрестности переключающегося фронта тактового сигнала должны удовлетворять требованиям, вытекающим из известных значений времени установления и времени удержания.

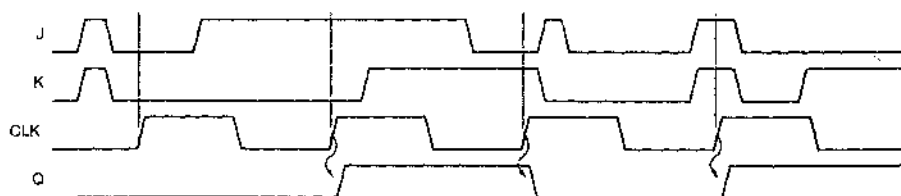


Рис. 7.29. Поведение JK-триггера, переключающегося по положительному фронту

Поскольку в JK-триггерах, переключающихся по фронту, исключены проблемы захвата единиц и нулей и проблемы, связанные с одновременным действием управляющих входных сигналов, эти триггеры по существу вытеснили переключающиеся по фронту триггеры более старых типов. Микросхемы 74x109 в семействе ТТЛ представляют собой JK-триггеры, переключающиеся по положительному фронту, с активным низким уровнем входного сигнала K (который называется K или K_L).

ЕЩЕ ОДИН «НАСТОЯЩИЙ» ТРИГГЕР

По своей внутренней структуре ИС 74х109 очень похожа на ИС 74LS74, схема которой была приведена на рис. 7.20. Как следует из сравнения схем на рис. 7.20 и 7.30, микросхема '109 получается из микросхемы '74 простой заменой нижнего левого вентиля, реализующего характеристическое уравнение $Q^* = D$, на структуру И-ИЛИ, реализующую характеристическое уравнение JK-триггера: $Q^* = J \cdot Q' + K_L \cdot Q$.

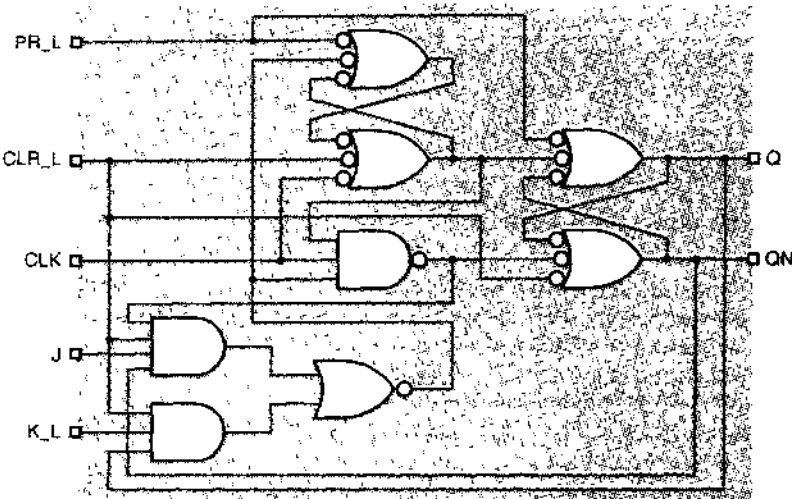


рис. 7.30. Внутреннее устройство переключающегося по положительному фронту JK-триггера 74LS109

Самым распространенным применением JK-триггеров являются тактируемые синхронные конечные автоматы. Как мы объясним в разделе 7.4.5, логика переходов JK-триггеров иногда проще, чем у D-триггеров. Однако в большинстве случаев конечные автоматы строятся на D-триггерах, поскольку методология проектирования при этом чуть проще и большинство проектируемых последовательностных устройств состоит из D-триггеров, а не из JK-триггеров. Поэтому в основном мы будем обращать внимание на D-триггеры.

7.2.11. Т-триггер

триггер (T flip-flop; T – toggle, переключать) изменяет свое состояние на каждом периоде тактового сигнала. На рис. 7.31 приведены условное обозначение и временные диаграммы для переключающегося по положительному фронту Т-триггера. Заметьте, что частота переключений сигнала на выходе триггера Q равна точно половине частоты переключений входного сигнала Т. На рис. 7.32 показано, как получить Т-триггер из D- или JK-триггера. Как мы увидим в параграфе 4, Т-триггеры чаще всего используются в счетчиках и делителях частоты.

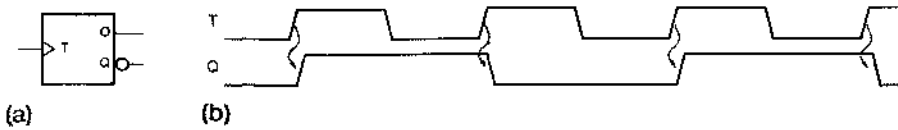


Рис. 7.31. Т-триггер, переключающийся по положительному фронту: (а) условное обозначение; (б) функциональное поведение

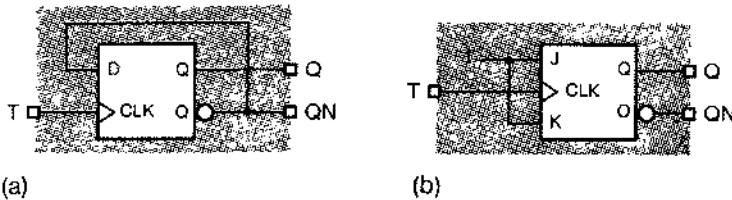


Рис. 7.32. Возможные схемы Т-триггеров: (а) на основе D-триггера; (б) на основе JK-триггера

Во многих приложениях Т-триггеров требуется, чтобы такой триггер переключался не на каждом такте. В таком случае можно воспользоваться *Т-триггером с входом разрешения* (*T flip-flop with enable*). Как показано на рис. 7.33, такой триггер изменяет свое состояние на переключающем фронте тактового сигнала только в том случае, когда на вход EN подан разрешающий сигнал. Подобно сигналам на входах D, J и K у других триггеров, переключающихся по фронту, сигнал на входе EN должен удовлетворять требованию оставаться неизменным в интервале, состоящем из времени установления и времени удержания, в окрестности переключающего фронта тактового сигнала. Схемы, приведенные на рис. 7.32, легко видоизменить таким образом, чтобы имелся вход разрешения EN, как это сделано на рис. 7.34.

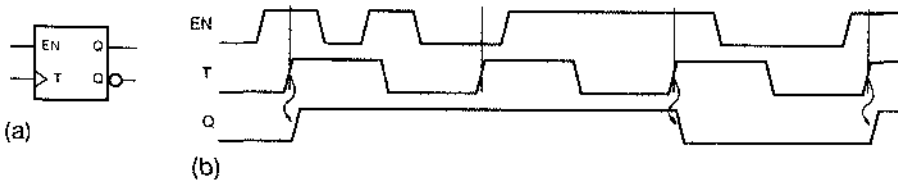


Рис. 7.33. Переключающийся по положительному фронту Т-триггер с входом разрешения: (а) условное обозначение; (б) функциональное поведение

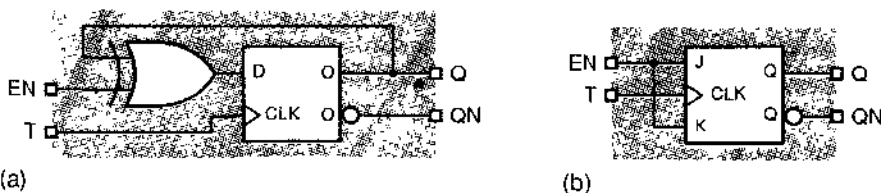


Рис. 7.34. Возможные схемы Т-триггеров с входом разрешения: (а) на основе D-триггера; (б) на основе JK-триггера

7.3. Анализ тактируемых синхронных конечных автоматов

Хотя основные составные элементы последовательностных схем — защелки и триггеры — сами по себе являются последовательностными схемами с обратной связью, которые можно подвергнуть формальному анализу, мы начнем с изучения работы *тактируемых синхронных конечных автоматов* (*clocked synchronous state machines*), поскольку их легче понять. «Конечный автомат» — это общее название последовательностных схем; слово «тактируемый» указывает на тот факт, что элементы памяти в конечном автомате (триггеры) имеют тактовый вход; слово «синхронный» означает, что все триггеры используют один и тот же тактовый сигнал. Состояние такого конечного автомата изменяется только в момент времени, когда в тактовом сигнале происходит нереклюющий переход или, как гонорят, на очередном «такте».

7.3.1. Структура конечного автомата

На рис. 7.35 приведена общая структура тактируемого синхронного конечного автомата. Память состояния (*state memory*) представляет собой набор из n триггеров, в которых хранится текущее состояние автомата; всего имеется 2^n различных состояний. Все триггеры подключены к общему источнику тактового сигнала, который позволяет им изменять состояние на каждом *такте* (*tick*) тактового сигнала. Что именно представляет собой такт, зависит от типа триггеров (нереклюющий фронт, переключающий импульс и т.д.). В этом параграфе мы рассмотрим переключающиеся по положительному фронту D- и JK-триггеры, для которых тактом служит нарастающий фронт тактового сигнала.

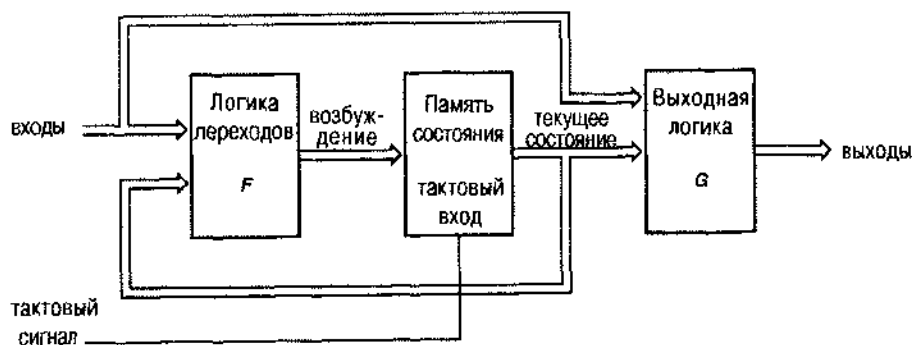


Рис. 7.35. Структура тактируемого синхронного конечного автомата (автомата Мили)

Следующее состояние конечного автомата, изображенного на рис. 7.35, определяется *логикой переходов F* (*next-state logic*) и является функцией текущего состояния и входного воздействия. Выходные сигналы определяются *выходной логикой G* (*output logic*) и также зависят от текущего состояния и входного воздействия. Оба блока *F* и *G* являются строго комбинационными схемами. Сказанное можно выразить в виде следующей записи:

Следующее состояние = F (текущее состояние, вход)

Выход = G (текущее состояние, вход).

Память состояния конечного автомата может быть построена на D-триггерах, непереклюкающихся по положительному фронту; в этом случае все события происходят в моменты времени, соответствующие нарастающим фронтам тактового сигнала. Возможно также использование в памяти состояния переклюкающихся по отрицательному фронту D-триггеров, D-защелок или JK-триггеров. Но ввиду того, что большинство конечных автоматов создаются сегодня на основе программируемых логических устройств с непереклюкающимися по положительному фронту D-триггерами, мы сконцентрируем свое внимание именно на этом случае.

7.3.2. Выходная логика

Представленная на рис. 7.35 последовательностная схема, выход которой зависит как от состояния, так и от входа, называется *автоматом Мили (Mealy machine)*. В некоторых приложениях выход зависит только от состояния:

Выход = G (текущее состояние).

Такая схема называется *автоматом Мура (Moore machine)*; ее общая структура приведена на рис. 7.36

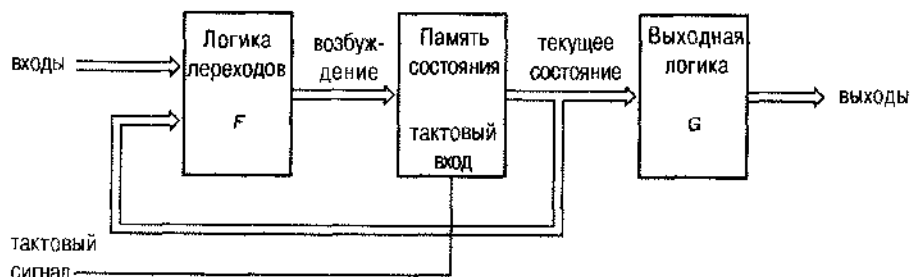


Рис. 7.36. Структура тактируемого синхронного конечного автомата (автомата Мура)

Очевидно, что единственное различие между этими двумя моделями конечных автоматов заключается в том, как вырабатываются выходные сигналы. На практике многие конечные автоматы должны быть отнесены к автоматам Мили, поскольку у них имеется один или большее число *выходов типа Мили*, то есть выходов, зависящих не только от состояния, но также и от входа. Однако многие из автоматов имеют также один или большее число *выходов типа Мура*, которые зависят только от состояния.

При проектировании быстродействующих схем часто бывает необходимо обеспечить возникновение новых значений выходных сигналов возможно раньше и гарантировать, что они не будут меняться в течение периода тактового сигнала. Один из способов достижения этого состоит в таком кодировании состояния, когда сами переменные состояния служат выходами. Мы называем такой способ *записью состояния в форме выходного кода (output-coded state assignment)*; в результате

получается автомат Мура, у которого выходная логика, указанная на рис. 7.36, отсутствует, то есть реализуется непосредственными соединениями

Другой подход заключается в построении такого конечного автомата, у которого выходы в течение данного периода тактового сигнала зависят от состояния и сигналов на входах в *предыдущем* периоде тактового сигнала. Мы говорим в таком случае о *конвейерном выходе (pipelined output)*, который можно реализовать, подключив еще один блок памяти (состоящий из триггеров) к автомату Мили со стороны его выходов, как это сделано на рис. 7.37.

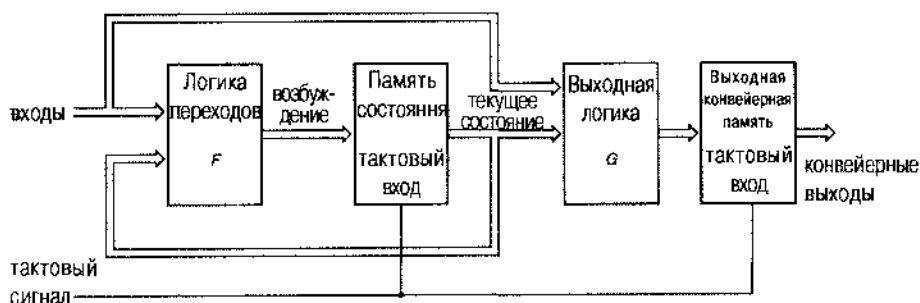


Рис. 7.37. Автомат Мили с конвейерным выходом

Путем соответствующих преобразований схемы или рисунка можно один конечный автомат отобразить на другой. Например, можно было бы триггеры, реализующие конвейерный выход в автомате Мили, объявить частью его памяти состояния и тем самым прийти к автомату Мура с записью состояния в форме выходного кода.

Строгое отношение конечного автомата к тому или иному классу не так важно. Важно другое – ваше представление о структуре выходных цепей и то, в какой мере достигаются цели всего вашего проекта, включая требования по временным параметрам и гибкости. Например, конвейерной организацией выхода удобно осуществить принуждение ко времени, но этим можно воспользоваться только в том случае, когда вы в состоянии вычислять очередное желаемое значение выходного сигнала заранее, за один период тактового сигнала до наступления соответствующего момента времени. В любом приложении можно использовать различные подходы к разным выходным сигналам. В разделе 7.1.5 мы увидим, в частности, что различную организацию выходных цепей можно задать с помощью соответствующих операторов языка ABEL.

7.3.3. Характеристические уравнения

Функциональное поведение защелки или триггера можно описать формально с помощью *характеристического уравнения (characteristic equation)*, посредством которого следующее состояние триггера задается как функция его текущего состояния и значений сигналов на его входах.

В табл. 7.1 перечислены характеристические уравнения для триггеров, рассмотренных в параграфе 7.2. Принято считать, что звездочка * (* suffix) в записи Q^* означает «следующее значение Q ». Заметьте, характеристическое уравнение

не содержит деталей того, как ведет себя устройство во времени (защелкивание или нереключение по фронту и т.д.); оно выражает собой функциональный отклик на управляющие сигналы. Как мы вскоре увидим, такое упрощенное описание полезно при анализе конечных автоматов.

Табл. 7.1. Характеристические уравнения защелок и триггеров

Тип схемы	Характеристическое уравнение
SR-защелка	$Q^* = S + R' \cdot Q$
D-защелка	$Q^* = D$
Переключающийся по фронту D-триггер	$Q^* = D$
D-триггер с входом разрешения	$Q^* = EN \cdot D + EN' \cdot Q$
Двухтактный SR-триггер	$Q^* = S + R' \cdot Q$
Двухтактный JK-триггер	$Q^* = J \cdot Q' + K' \cdot Q$
Переключающийся по фронту JK-триггер	$Q^* = J \cdot Q' + K' \cdot Q$
T-триггер	$Q^* = Q'$
T-триггер с входом разрешения	$Q^* = EN \cdot Q' + EN' \cdot Q$

7.3.4. Анализ конечных автоматов с D-триггерами

Рассмотрим формальное определение конечного автомата, данное выше:

Следующее состояние $= F$ (текущее состояние, вход)

Выход $= G$ (текущее состояние, вход).

Помня, что «состояние» включает в себе все, что нужно знать о прошлом данной схемы, о ее предыстории, мы видим, что согласно первому из приведенных равенств, все, что нам нужно узнать о ближайшем будущем, можно определить по тому, что известно сейчас, и по текущим значениям входного воздействия. Второе равенство говорит нам о том, что из той же самой информации можно извлечь текущие значения выходных сигналов. Цель анализа последовательностной схемы состоит в таком определении логических функций переходов и выхода, чтобы поведение схемы было предсказуемым.

Анализ тактируемых синхронных конечных автоматов выполняется в три основных шага:

1. Определяются функции переходов и выхода F и G .
2. Функции F и G используются для построения таблицы «состояние/выход» (*state/output table*), которой полностью задаются следующие состояния и выходы схемы при всех возможных комбинациях текущего состояния и текущих значений входных сигналов.
3. (Необязательный шаг.) Вычерчивается диаграмма состояний (*state diagram*), представляющая информацию, полученную на предыдущем шаге, в графической форме.

На рис. 7.38 показан простой конечный автомат с двумя D-триггерами, переключающимися по положительному фронту. Чтобы определить функцию пере-

ходов F , мы должны рассмотреть сначала поведение памяти состояния. На нарастающем фронте тактового сигнала в каждом D-триггере берется выборка сигнала на входе D и это значение передается на выход триггера Q; характеристическое уравнение D-триггера имеет вид $Q^* = D$. Поэтому для определения следующего значения Q (то есть величины Q^*) нам необходимо знать текущее значение входного сигнала D.

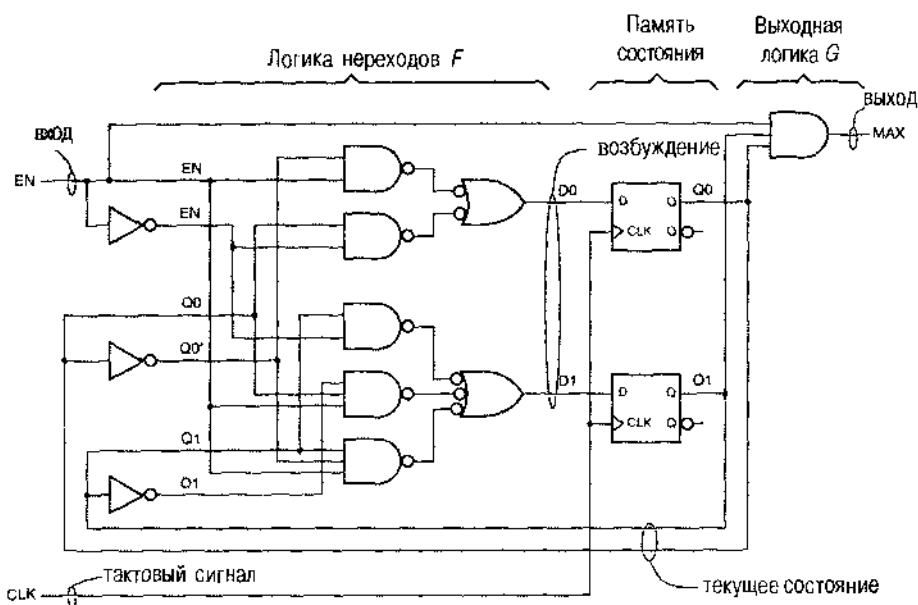


Рис. 7.38. Тактируемый синхронный конечный автомат с D-триггерами, переключающимися по положительному фронту

В схеме на рис. 7.38 имеется два D-триггера и сигналы на их выходах названы Q_0 и Q_1 . Эти два выходных сигнала являются переменными состояния; их значения — это текущее состояние автомата. Сигналы на D-входах соответствующих триггеров называются D_0 и D_1 . Эти сигналы играют роль *возбуждения* (*excitation*) для D-триггеров на каждом такте. Логические соотношения, выражающие сигналы возбуждения через текущее состояние и значение входного сигнала, носят название *уравнений возбуждения* (*excitation equations*), и их можно вывести из схемы автомата:

$$D_0 = Q_0 \cdot EN' + Q_0' \cdot EN$$

$$D_1 = Q_1 \cdot EN' + Q_1' \cdot Q_0 \cdot EN + Q_1 \cdot Q_0' \cdot EN.$$

Принято следующее значение переменной состояния, то есть ее значение после контрольного момента в тактовом сигнале, обозначать добавлением звездочки к имени переменной состояния, например, Q_0^* или Q_1^* . Воспользовавшись характеристическим уравнением D-триггера ($Q^* = D$), мы можем описать функцию переходов в нашем примере уравнениями для очередных значений переменных состояния:

$$Q0^* = D0$$

$$Q1^* = D1.$$

Подставляя D0 и D1 из уравнений возбуждения, получаем:

$$Q0^* = Q0 \cdot EN' + Q0' \cdot EN$$

$$Q1^* = Q1 \cdot EN' + Q1' \cdot Q0 \cdot EN + Q1 \cdot Q0' \cdot EN.$$

Эти соотношения, позволяющие выразить очередные значения переменных состояния в виде функций текущего состояния и выходного сигнала, носят название *уравнений переходов (transition equations)*.

Для каждой комбинации текущего состояния и значения входного сигнала уравнения переходов предсказывают следующее состояние. Каждое состояние описывается двумя битами (текущими значениями Q0 и Q1 в данный момент): (Q1 Q0) = 00, 01, 10 или 11 [Причина, по которой «произвольно», на первый взгляд, выбран порядок (Q1 Q0), а не (Q0 Q1), станет очевидной чуть ниже.] В каждом состоянии нашего автомата на его входе возможны только два значения сигнала: EN = 0 или EN = 1; таким образом, существует 8 комбинаций «состояние/вход». (В общем случае у автомата, состояние которого выражаются s битами, а входной сигнал — i битами, имеется 2^{s+i} комбинаций «состояние/вход».)

Табл. 7.2(а) представляет собой *таблицу переходов (transition table)*, которая составляется путем вычислений по уравнениям переходов для каждой возможной комбинации «состояние/вход». По традиции, состояния перечислены в таблице переходов в левом столбце сверху вниз, а значения входного сигнала — нверху таблицы слева направо, как показано в нашем примере.

Табл. 7.2. Таблица переходов, таблица состояний и таблица «состояние/выход» для конечного автомата, приведенного на рис. 7.38.

Q1 Q0	EN		S	EN		S	EN	
	0	1		0	1		0	1
00	00	01	A	A	B	A	A, 0	B, 0
01	01	10	B	B	C	B	B, 0	C, 0
10	10	11	C	C	D	C	C, 0	D, 0
11	11	00	D	D	A	D	D, 0	A, 1
	Q1* Q0*			S*			S*, MAX	

Назначение данного автомата очевидно из его таблицы переходов: это 2-рядный двоичный счетчик с входом разрешения EN. Если EN = 1, то с каждым периодом тактового сигнала результат счета увеличивается на 1, переходя к значению 00 по достижении максимальной величины 11.

При желании можно присваивать состояниям конечного автомата буквенные *имена состояний (state names)*. В простейшем случае это могло бы иметь такой вид: 00 = A, 01 = B, 10 = C и 11 = D. В результате подстановки имен состояний в табл. 7.2(а) взамен комбинаций Q1 и Q0 (а также Q1* и Q0*) получим *таблицу*

состояний [state table; табл. 7.2(b)]. Здесь “S” означает текущее состояние, а “S*” – следующее состояние автомата. Обычно таблицу состояний бывает легче воспринимать, чем таблицу переходов, поскольку применительно к сложным автоматам мы можем называть состояния именами, имеющими смысловую нагрузку. Однако в таблице состояний содержится меньше информации, чем в таблице переходов, так как в ней не указаны двоичные значения переменных состояния в каждом из состояний.

После того как таблица состояний записана, остается проанализировать только выходную логику. В рассматриваемом примере выходной сигнал – единственный, и он является функцией как текущего состояния, так и входного сигнала (это – автомат Мили). Таким образом, можно записать единственное уравнение выхода (output equation):

$$\text{MAX} = Q1 \cdot Q0 \cdot \text{EN}$$

Объединяя поведение выходного сигнала, предсказываемое этим уравнением, с информацией о переходах, можно составить таблицу «состояние/выход» (state/output table), представленную в качестве табл. 7.2(c).

Для автомата Мура таблица «состояние/выход» немного проще. Предположим, например, что в схеме на рис. 7.38 сигнал EN не подается на один из входов вентиля И, вырабатывающего сигнал MAX, и пусть в этом случае сигнал на выходе автомата Мура называется MAXS. Тогда величина MAXS является функцией только состояния и список значений MAXS в таблице «состояние/выход» будет иметь вид одного столбца независимо от входного сигнала. Этот случай приведен в табл. 7.3.

S	EN		MAXS
	0	1	
A	A	B	0
B	B	C	0
C	C	D	0
D	D	A	1
S*			

Табл. 7.3. Таблица «состояние/выход» для автомата Мура

Диаграмма состояний (state diagram) представляет информацию, содержащуюся в таблице «состояние/выход» графически. На ней каждому состоянию соответствует кружок [или узел (node)], а стрелками [или направленными дугами (directed arc)] указаны возможные переходы. В нашем примере диаграмма состояний имеет вид, показанный на рис. 7.39. Буквы внутри каждого кружка – это имя состояния. Каждая стрелка, выходящая из данного состояния, направлена к одному из следующих состояний в зависимости от входного воздействия; вблизи стрелок указано также значение выходного сигнала, вырабатываемое автоматом в данном состоянии при этом входном воздействии.

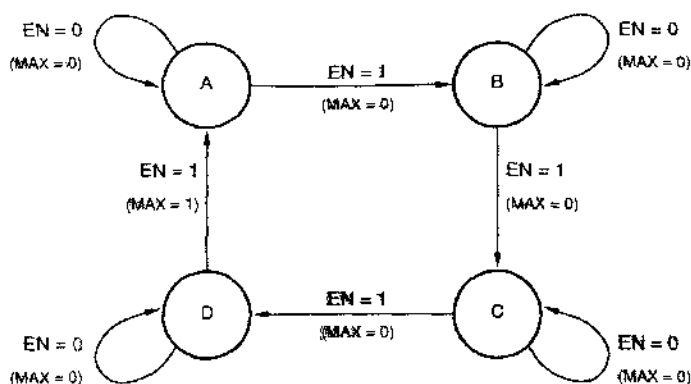


Рис. 7.39. Диаграмма состояний конечного автомата с таблицей «состояние/выход» 7.2(с)

Диаграмма состояний для автомата Мура несколько проще. В этом случае выходные сигналы можно указать внутри кружков, относящихся к отдельным состояниям, поскольку значения этих сигналов зависят только от состояния. Согласно этому правилу диаграмма состояний для автомата Мура имеет вид, приведенный на рис. 7.40.

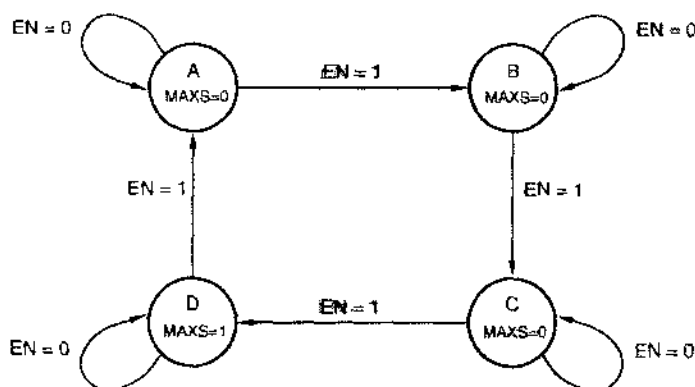


Рис. 7.40. Диаграмма состояний конечного автомата с таблицей «состояние/выход» 7.3

УТОЧНЕНИЕ

Способ указания выходных значений на диаграмме состояний автомата Мура может ввести в заблуждение. Следует помнить, что сигналы на выходе присутствуют в течение всего времени, пока автомат находится в данном состоянии с данным входным воздействием, а не возникают только в момент перехода к следующему состоянию.

СТРЕЛКИ, СТРЕЛКИ, ПОВСЮДУ СТРЕЛКИ

Поскольку в нашем примере всего один входной сигнал, возможны лишь два различных воздействия и только две стрелки выходят из каждого состояния. В автомате с n входными сигналами из каждого состояния выходило бы 2^n стрелок. Если n велико, то за всем этим невозможно уследить. Позднее на рис. 7.44 мы продемонстрируем правило, позволяющее не выводить из каждого состояния столько стрелок, сколько имеется различных входных комбинаций, а лишь по одной стрелке для каждого возможного следующего состояния.

Исходная принципиальная схема рассматриваемого нами конечного автомата была изображена на рис. 7.38, чтобы наилучшим образом соответствовать идее автомата Мили. Однако никто не заставляет нас именно так располагать логику переходов, память состояния и выходную логику. На рис. 7.41 приведена другая принципиальная схема того же самого конечного автомата. Чтобы проанализировать эту схему, разработчик (в данном случае — аналитик) все еще может извлечь всю необходимую информацию из самой схемы. Единственное отличие новой схемы от старой состоит в том, что мы воспользовались сигналами с инверсных выходов триггеров QN (которые в нормальных условиях являются дополнениями сигналов на выходах Q), сэкономив таким образом пару инверторов.

КАРТИНКИ, НАВОДЯЩИЕ НА РАЗМЫШЛЕНИЯ

Воспользовавшись таблицами переходов, состояний и значений выходных сигналов, можно нарисовать временные диаграммы и с их помощью представить поведение конечного автомата с любого желаемого начального состояния и при любой желаемой входной последовательности. На рис. 7.42, например, показано поведение автомата в нашем примере, начиная с состояния 00 (A), при указанном на рисунке изменении входного сигнала EN.

Обратите внимание на то, что следующее состояние зависит от значения EN только в момент нарастающего фронта тактового сигнала CLOCK; другими словами, счет производится счетчиком только тогда, когда $EN = 1$ в момент времени, соответствующий нарастающему фронту в сигнале CLOCK. С другой стороны, на значение MAX выходного сигнала автомата Мили сигнал EN оказывает влияние в течение всего времени, тогда как значение MAXS выходного сигнала автомата Мура, упоминаемого в основном тексте, зависит только от состояния, что и отражено на рисунке.

Временные диаграммы нарисованы так, чтобы показать, что смена значения в выходных сигналах MAX и MAXS происходит чуть позднее тех моментов времени, когда изменяются состояния и входной сигнал, следствием чего является смена значений сигнала на выходе; в этом находит свое отражение задержка выходных сигналов в комбинационных логических схемах. Естественно, что эти временные диаграммы носят иллюстративный характер; точное описание поведения схемы во времени обычно приводится в виде таблиц временных соотношений, примеры которых приведены в разделе 5.2.3.

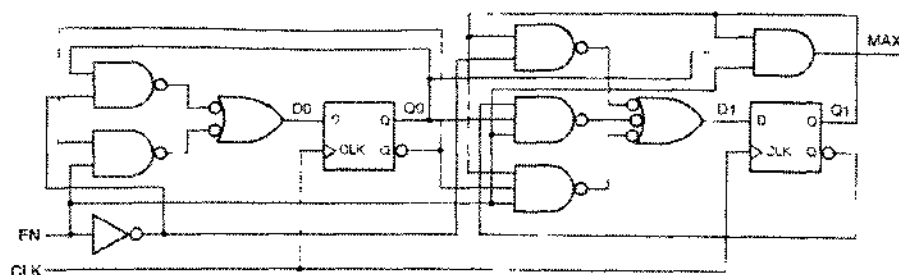


Рис. 7.41. Перерисованная иначе принципиальная схема тактируемого синхронного конечного автомата

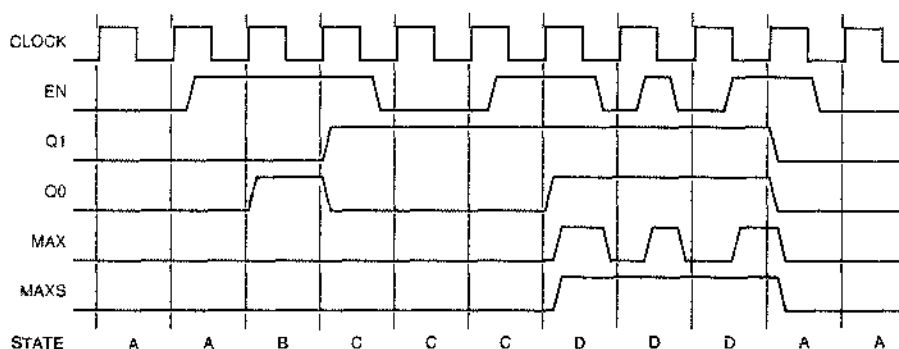


Рис. 7.42. Временные диаграммы для рассматриваемого в качестве примера конечного автомата

В заключение, перечислим подробно все шаги, выполняемые при анализе тактируемого синхронного конечного автомата:

1. Составляем *уравнения возбуждения* для управляющих входов триггеров.
2. Для получения *уравнений переходов* производим подстановку из уравнений возбуждения в характеристические уравнения триггеров.
3. По уравнениям переходов строим *таблицу переходов*.
4. Записываем *уравнения выхода*.
5. Значения выходных сигналов добавляем в таблицу переходов для каждого состояния (в случае автомата Мура) или в таблицу «состояние/вход» (в случае автомата Мили), в результате чего получаем *таблицу «переход/выход» (transition/output table)*.
6. Даем состояниям имена и замечаем ими комбинации переменных состояния в таблице «переход/выход»; таким образом, получаем таблицу «состояние/выход».
7. (Необязательный шаг.) Вычерчиваем диаграмму состояний, соответствующую таблице «состояние/выход».

Мы сейчас пройдем всю эту последовательность шагов, чтобы проанализировать другой тактируемый синхронный конечный автомат, изображенный на рис. 7.43. Глядя на принципиальную схему, записываем уравнения возбуждения:

$$D0 = Q1' \cdot X + Q0 \cdot X' + Q2$$

$$D1 = Q2' \cdot Q0 \cdot X + Q1 \cdot X' + Q2 \cdot Q1$$

$$D2 = Q2 \cdot Q0' + Q0' \cdot X' \cdot Y.$$

Подставляя $D0$, $D1$ и $D2$ в характеристическое уравнение D-триггеров, получим уравнения переходов:

$$Q0^* = Q1' \cdot X + Q0 \cdot X' + Q2$$

$$Q1^* = Q2' \cdot Q0 \cdot X + Q1 \cdot X' + Q2 \cdot Q1$$

$$Q2^* = Q2 \cdot Q0' + Q0' \cdot X' \cdot Y.$$

Строим по этим уравнениям таблицу переходов [табл. 7.4(а)]. Из принципиальной схемы находим следующие два уравнения для выходных сигналов:

$$Z1 = Q2 + Q1' + Q0'$$

$$Z2 = Q2 \cdot Q1 + Q2 \cdot Q0'.$$

Результирующие значения выходных сигналов перечислены в крайнем правом столбце табл. (а). Присваиваем состояниям имена от А до Н и приходим к таблице «состояние/выход» [табл. (b)].

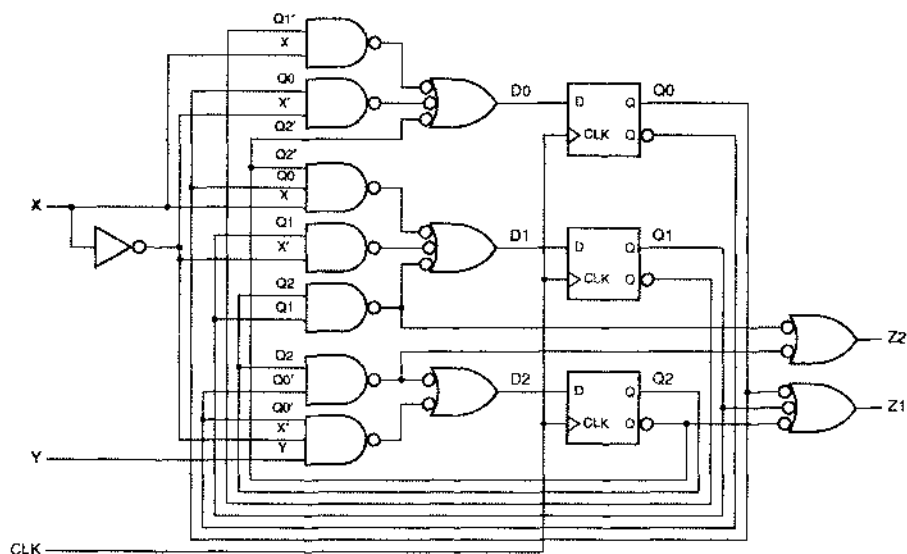


Рис. 7.43. Тактируемый синхронный конечный автомат с тремя триггерами и восьмью состояниями

Табл. 7.4. Таблицы «переход/выход» и «состояние/выход» для конечного автомата на рис. 7.43

(a)							(b)						
XY							XY						
Q2 Q1 Q0	00	01	10	11	Z1 Z2		S	00	01	10	11	Z1 Z2	
000	000	100	001	001	10		A	A	E	B	B	10	
001	001	001	011	011	10		B	B	B	D	D	10	
010	010	110	000	000	10		C	C	G	A	A	10	
011	011	011	010	010	00		D	D	D	C	C	00	
100	101	101	101	101	11		E	F	F	F	F	11	
101	001	001	001	001	10		F	B	B	B	B	10	
110	111	111	111	111	11		G	H	H	H	H	11	
111	011	011	011	011	11		H	D	D	D	D	11	
Q2* Q1* Q0*							S*						

Диаграмма состояний в данном примере имеет вид, показанный на рис. 7.44. Поскольку здесь мы имеем дело с автоматом Мура, значения сигналов указаны вместе с именами состояний. Каждая стрелка помечена *выражением перехода* (*transition expression*); переход по стрелке происходит при таких входных комбинациях, для которых это выражение перехода равно 1. Переходы, помеченные единицами, выполняются всегда.

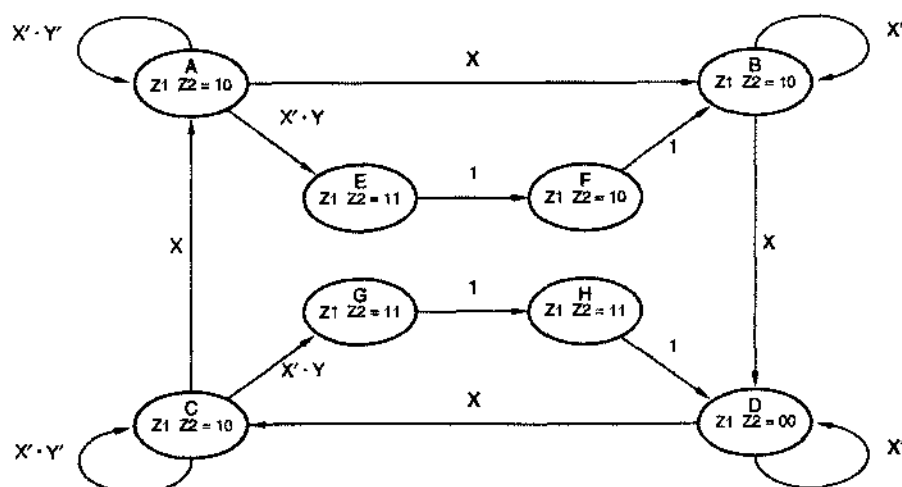


Рис. 7.44. Диаграмма состояний в соответствии с табл. 7.4

Выражения переходов у стрелок, выходящих из данного состояния, должны быть *взаимно исключающими* (*mutual exclusion*) и *исчерпывающими в совокупности* (*all inclusion*) согласно следующим правилам:

- Никакие два выражения перехода не могут равняться 1 при одной и той же входной комбинации, поскольку при фиксированной входной комбинации у автоматов не может быть двух следующих состояний.
- Для любой возможной входной комбинации какое-то выражение перехода должно равняться 1, так чтобы оказались предусмотренными все возможные следующие состояния.

Используя информацию, содержащуюся в таблице состояний, можно записать выражение перехода из любого текущего состояния в следующее состояние в виде суммы минтермов для тех входных комбинаций, которые вызывают этот переход. При желании выражение можно минимизировать, что позволяет представить информацию в более компактном виде. Выражения переходов более всего полезны при проектировании таких конечных автоматов, в отношении которых эти выражения можно вывести из словесного описания задачи, как мы увидим это в параграфе 7.5.

*7.3.5. Анализ конечных автоматов на JK-триггерах

Основная процедура анализа тактируемых синхронных конечных автоматов с JK-триггерами — та же, что и в предыдущем разделе. Единственное отличие состоит в том, что теперь у каждого триггера имеются два уравнения возбуждения: по одному для входов J и K. Чтобы получить уравнение переходов, необходимо подставить оба выражения для этих входных сигналов в характеристическое уравнение JK-триггера: $Q^* = J \cdot Q' + K' \cdot Q$.

На рис. 7.45 приведен пример конечного автомата с JK-триггерами. Глядя на принципиальную схему, можно записать следующие уравнения возбуждения:

$$\begin{aligned} J_0 &= X \cdot Y' \\ K_0 &= X \cdot Y' + Y \cdot Q_1 \\ J_1 &= Y \cdot Q_0 + Y \\ K_1 &= Y \cdot Q_0' + X \cdot Y' \cdot Q_0. \end{aligned}$$

Подставляя J_0 , K_0 , J_1 и K_1 в характеристические уравнения JK-триггеров, получим уравнения переходов:

$$\begin{aligned} Q_0^* &= J_0 \cdot Q_0' + K_0' \cdot Q_0 \\ &= X \cdot Y' \cdot Q_0' + (X \cdot Y' + Y \cdot Q_1)' \cdot Q_0 \\ &= X \cdot Y' \cdot Q_0' + X' \cdot Y' \cdot Q_0 + X' \cdot Q_1' \cdot Q_0 + Y \cdot Q_1' \cdot Q_0 \\ Q_1^* &= J_1 \cdot Q_1' + K_1' \cdot Q_1 \\ &= (X \cdot Q_0 + Y) \cdot Q_1' + (Y \cdot Q_0' + X \cdot Y' \cdot Q_0)' \cdot Q_1 \\ &= X \cdot Q_1' \cdot Q_0 + Y \cdot Q_1' + X' \cdot Y' \cdot Q_1 + Y' \cdot Q_1 \cdot Q_0' + X' \cdot Q_1 \cdot Q_0 + Y \cdot Q_1 \cdot Q_0. \end{aligned}$$

Табл. 7.5(а) представляет собой таблицу переходов, составленную по этим уравнениям. Из принципиальной схемы следует также, что уравнение выхода имеет вид:

$$Z = X \cdot Q_1 \cdot Q_0 + Y \cdot Q_1' \cdot Q_0'.$$

Результирующие значения выходного сигнала указаны в табл. (а) в каждом из столбцов вместе со следующим состоянием. Присваивая состояниям имена от А до D, получаем таблицу «состояние/выход» [табл. (б)]. Соответствующая диаграмма состояний с выражениями переходов показана на рис. 7.46.

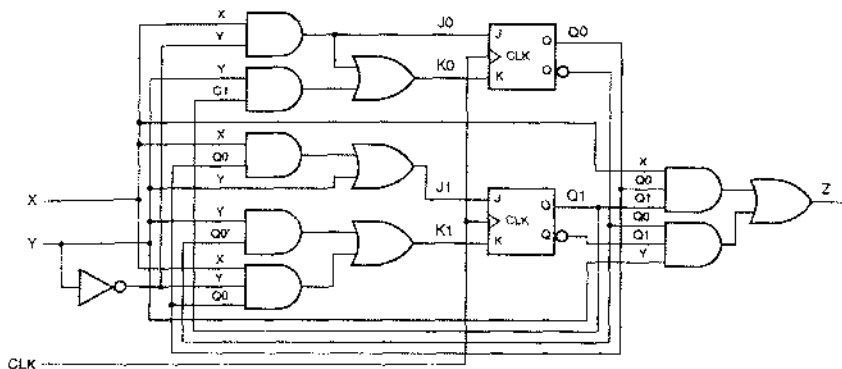


Рис. 7.45. Тактируемый синхронный конечный автомат с JK-триггерами

Табл. 7.5. Таблицы «переход/выход» и «состояние/выход» для конечного автомата на рис. 7.45

(а)		XY			
Q_1Q_0		00	01	10	11
00	00, 0	10, 1	01, 0	10, 1	
01	01, 0	11, 0	10, 0	11, 0	
10	10, 0	00, 0	11, 0	00, 0	
11	11, 0	10, 0	00, 1	10, 1	
		$Q_1^*Q_0^*, Z$			

(б)		XY			
S		00	01	10	11
A	A, 0	C, 1	B, 0	C, 1	
B	B, 0	D, 0	C, 0	D, 0	
C	C, 0	A, 0	D, 0	A, 0	
D	D, 0	C, 0	A, 1	C, 1	
		S^+, Z			

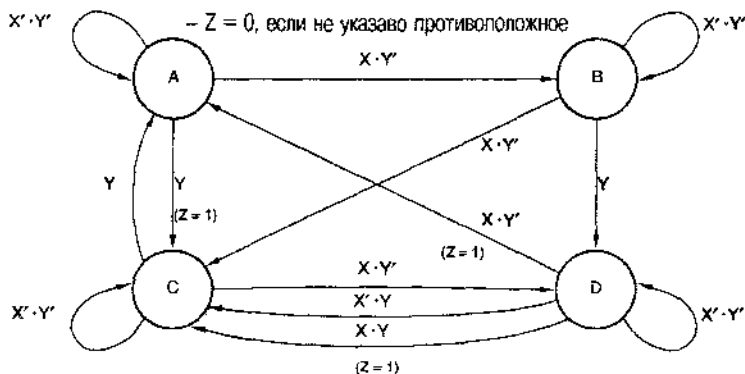


Рис. 7.46. Диаграмма состояний для конечного автомата, описываемого табл. 7.5.

7.4. Проектирование тактируемых синхронных конечных автоматов

Создание тактируемого синхронного конечного автомата начинается с его словесного описания или задания его технических характеристик, а затем выполняются те же шаги, что и при анализе, перечисленные в предыдущем параграфе, только в обратном порядке:

1. На основе словесного описания составляется таблица «состояние/выход»; для состояний используются мнемонические имена. (Можно также начинать с диаграммы состояний; этот метод рассмотрен в параграфе 7.5.)
2. (Необязательный шаг.) Минимизируется число состояний в таблице «состояние/выход» (*state minimization*).
3. Выбирается набор переменных состояния и комбинации переменных состояния ставятся в соответствие именам состояний (*state assignment*).
4. Чтобы получить таблицу «переход/выход», комбинации переменных состояния подставляются в таблицу «состояние/выход»; в результате каждой возможной комбинации «состояние/вход» теперь соответствует желаемая следующая комбинация переменных состояния и желаемый выход.
5. Выбирается тип триггеров для памяти состояния (например, D- или JK-триггеры). В большинстве случаев этот выбор бывает уже произведен в уме на начальной стадии проекта, но данный шаг — это ваша последняя возможность передумать.
6. Составляется *таблица возбуждения (excitation table)*, в которую записываются значения возбуждающих воздействий, необходимые для того, чтобы получить желаемое следующее состояние для каждой комбинации «состояние/вход».
7. Из таблицы возбуждения выводятся уравнения возбуждения.
8. Из таблицы «переход/выход» выводятся уравнения выхода.
9. Рисуются принципиальная схема, включающая элементы, хранящие переменные состояния, и реализующая требуемые уравнения возбуждения и выхода. (Впрочем, уравнения можно реализовать непосредственно в программируемом логическом устройстве.)

В данном параграфе мы рассмотрим каждый из этих шагов в процессе построения конечного автомата. Самым важным является шаг 1, так как именно здесь разработчик занимается собственно проектированием в творческом процессе перевода словесного (возможно, неоднозначного) описания конечного автомата на обычном языке в формальное табличное описание. Шаг 2 практически никогда опытными разработчиками не выполняется, а вот для шага 3 требуется большое мастерство.

После того как первые три шага пройдены, все оставшееся можно считать рутинной работой, требующей следования вполне определенным правилам синтеза. Самыми утомительными являются шаги 4 и 6–9, но их легко автоматизировать. Например, при разработке конечного автомата, который предстоит реализовать в программируемом логическом устройстве, рутинную часть работы можно поручить компилятору языка ABEL, как это будет сделано в разделе 7.11.2. Но все же важно понимать детали процедуры синтеза как для того, чтобы иметь возможность оценивать работу компилятора, так и для того, чтобы была возмож-

посты сообразить, что происходит в действительности, когда компилятор дает неожиданные результаты. Поэтому ниже в этом параграфе рассматриваются все девять шагов процедуры проектирования конечного автомата.

СОСТАВЛЕНИЕ ТАБЛИЦЫ СОСТОЯНИЙ – ЭТО СВОЕГО РОДА ПРОГРАММИРОВАНИЕ

Составление таблицы состояний (или, что эквивалентно, вычерчивание диаграммы состояний) – творческий процесс, многими своими чертами напоминающий написание программы для компьютера:

- Вы начинаете с достаточно точного описания входов и выходов, но с возможно неоднозначным описанием желаемого соотношения между ними, не заботясь о том, как действительно получить желаемые выходы при имеющихся входах.
- В процессе осуществления проекта вам, возможно, приходится уточнять, что именно и каким из многих способов будет реализовываться, иногда руководствуясь здравым смыслом, а иногда произвольно.
- Возможно, вам понадобится оговорить специальные случаи, о которых не было речи в первоначальном описании, и предусмотреть их обработку.
- Вероятно, при выполнении этой работы вам надо будет не упускать из виду несколько идей.
- Поскольку выполняемые действия не алгоритмизуемы, нет никакой гарантии, что вы составите таблицу состояний с конечным числом состояний или напишете программу, состоящую из конечного числа строк. Однако, если вы не работаете на правительство, вы должны попытаться сделать это.
- Когда, наконец, вы запустите конечный автомат или программу, он или она будут делать точно то, что вы им повелели, – ни больше и ни меньше.
- Нет гарантии, что ваш продукт заработает с первого раза; возможно, придется его отлаживать или даже весь процесс в целом повторить.

Хотя составление таблицы состояний – это, конечно, проблема, не надо бояться. Если вам приходилось сталкиваться с подобными задачами во время учебы, то вы, вероятно, уже написали несколько работающих программ; вот почему вы вполне можете стать также хорошим составителем таблиц состояний.

7.4.1. Пример составления таблицы состояний

Существует несколько различных способов описания таблицы состояний конечного автомата. Позднее мы увидим, как можно косвенно задавать таблицу состояний на языках ABEL и VHDL. Однако в этом параграфе мы будем иметь дело только с таблицами состояний, задаваемыми явно, в той же самой табличной форме, в какой мы пользовались ими при анализе в предыдущем параграфе.

Процесс составления таблицы состояний, а затем – в следующих разделах – вся процедура синтеза, будут представлены нами на примере следующей простой задачи:

Построить тактируемый синхронный конечный автомат с двумя входами A и B и одним выходом Z, таким что выходной сигнал Z равен 1, если

- входной сигнал А имел одно и то же значение на каждом из двух предыдущих тактов в тактовом сигнале *clock*
- входной сигнал В равен 1 с последнего момента времени, когда первое условие было выполнено.

В противном случае выходной сигнал должен равняться 0.

Не волнуйтесь, если на этом этапе смысл данного задания, мягко говоря, вам не вполне ясен. Ваша задача как разработчика состоит в преобразовании описания такого рода в таблицу состояний, которая должна быть совершенно недвусмысленной; даже если не получится то, что имелось в виду, все же будет, по крайней мере, основа для обсуждения и усовершенствования.

В качестве дополнительного «указания» или требования условия задачи о составлении таблицы состояний включают временные диаграммы, демонстрирующие ожидаемое поведение конечного автомата для одной или нескольких последовательностей входных сигналов. Такого рода временные диаграммы вряд ли однозначно зададут поведение автомата при всех возможных входных воздействиях, но, как и раньше, это служит хорошей отправной точкой и контрольным тестом, по которому можно будет проверить правильность работы созданного устройства. Примечательно к рассматриваемой нами задаче составления таблицы состояний временные диаграммы пусть имеют вид, указанный на рис. 7.47.

РЕАЛИЗАЦИЯ НАДЕЖНОЙ НАЧАЛЬНОЙ УСТАНОВКИ

Система будет работать надлежащим образом только в том случае, если конструкция конечного автомата гарантирует, что при включении он войдет в известное начальное состояние типа состояния INIT в разбираемом нами примере.

Обычно для этого аналоговой схемой генерируется сигнал начальной установки RESET. Такая схема, как правило, обнаруживает напряжение, близкое к полному напряжению питания (скажем, 4.5 В) и повторяет это напряжение на своем выходе в течение времени (например, порядка 100 мс), достаточного для того, чтобы все компоненты (в том числе генераторы) успели войти в установившийся режим, пока система еще остается «незапущенной». Такой аналоговой ИС, предназначенной для начального запуска, является микросхема TL7705 фирмы Texas Instruments; она имеет встроенный эталонный источник напряжения 4.5 В, используемый для обнаружения включения питания, а время, в течение которого система остается «незапущенной», задается внешними резистором и конденсатором.

Если конечный автомат строится на дискретных триггерах с асинхронными входами установки в состояние 1 и сброса, то автомат можно ввести в желаемое начальное состояние, подавая сигнал RESET на эти входы. Когда таких входов нет, или в том случае, когда запуск должен быть синхронным (как это бывает в системах с быстродействующими микропроцессорами), для сигнала RESET можно выделить отдельный вход конечного автомата, такой что при действии сигнала на этом входе происходит желаемая начальная установка всех элементов, ответственных за следующее состояние автомата.

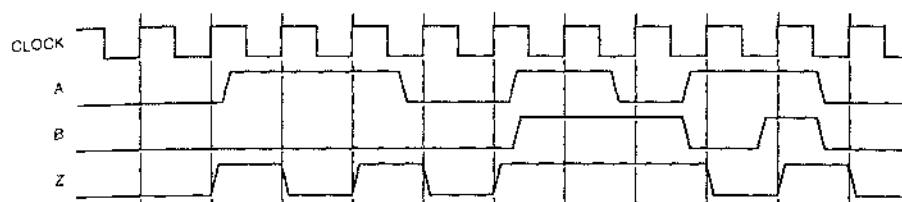


Рис. 7.47. Временные диаграммы для конечного автомата, рассматриваемого в качестве примера (CLOCK – тактовый сигнал)

Первый шаг при составлении таблицы состояний заключается в подготовке трафарета. Из словесного описания нам известно, что предмет нашего рассмотрения является автоматом Мура: сигнал на его выходе зависит только от текущего состояния, то есть от того, что происходило в течение предшествующих периодов тактового сигнала. Следовательно, мы должны предусмотреть по одному столбцу, в котором будут перечислены следующие состояния, для каждой возможной комбинации входных сигналов и один столбец для значений выходного сигнала, что и сделано на рис. 7.48(а).

Порядок, в котором перечисляются комбинации входных сигналов, на этом этапе существуетен, но мы выбираем его таким, как на карте Карно, чтобы позднее нам было проще вывести уравнения возбуждения. В случае автомата Мили нам надо было бы опустить столбец, относящийся к выходу, а значения выходного сигнала записывать вместе со следующими состояниями для каждой комбинации входных сигналов. Левый столбец предназначен для словесного комментария, который просто напомнил бы нам значение каждого состояния или связной с ним «истории».

В словесном описании ничего не говорится о том, что должно происходить в автомате с самого начала, так что здесь мы должны импровизировать. Предположим, что сразу после включения питания автомат входит в *начальное состояние* (initial state), которое мы назовем в этом примере INIT. Пишем имя начального состояния (INIT) в первой строке и оставляем достаточно места для других строк (состояний), которые понадобятся в дальнейшем. Мы можем также заполнить место для значения Z в состоянии INIT; здравый смысл подсказывает нам, что это значение следует взять равным 0, поскольку *никаких* входных сигналов у нас не было.

Теперь нам необходимо заполнить строку INIT информацией о следующих состояниях. Выходной сигнал Z не сможет стать равным 1, пока мы не увидим, по меньшей мере, двух значений сигнала на входе A, поэтому возможны только два состояния A0 и A1, которые будут «помнить» значение сигнала A на предыдущем такте [рис. 7.48 (b)]. В каждом из этих состояний сигнал Z равен 0, так как условия возникновения 1 на выходе все еще не удовлетворены. Точное значение состояния A0 таково: «На предыдущем такте сигнал A равнялся 0, за такт до этого сигнал A не равнялся нулю и сигнал B не оставался равным 1 со времени предыдущей пары одинаковых значений сигнала A». Аналогично определяется состояние A1.

(a) Значения		A B				Z
Начальное состояние	S	00	01	11	10	0
	INIT					0
S^*						

(b) Значения		A B				Z
Начальное состояние	S	00	01	11	10	0
На А было 0	INIT	A0	A0	A1	A1	0
На А было 1	A1					0
S^*						

(c) Значения		A B				Z
Начальное состояние	S	00	01	11	10	0
На А было 0	A0	OK	OK	A1	A1	0
На А было 1	A1					0
На А дважды было одно и то же	OK					1
S^*						

(d) Значения		A B				Z
Начальное состояние	S	00	01	11	10	0
На А было 0	A0	OK	OK	A1	A1	0
На А было 1	A1					0
На А дважды было одно и то же	OK					1
S^*						

Рис. 7.48. Процесс составления таблицы состояний

К этому моменту мы знаем, что у нашего конечного автомата есть, по крайней мере, три состояния, и мы приготовились заполнить еще две пустые строки. Хм! Это не очень хорошая тенденция! При заполнении данными о следующих состояниях *одной* строки (соответствующей состоянию INIT) нам понадобилось *два* новых состояния A0 и A1. Если продолжать в том же темпе, то к вечеру у нас будет 65 535 состояний! Впрочем, нам надо будет внимательно присматриваться к уже существующим состояниям, имеющим то же самое значение, всякий раз, когда нам, возможно, понадобится вводить новые состояния. Давайте посмотрим, как это делается.

Пусть автомат находится в состоянии A0; тогда известно, что на последнем такте сигнал A равнялся 0. Поэтому, если сигнал A будет равен 0 снова, то мы перейдем в новое состояние OK со значением выходного сигнала Z, равным 1 [рис. 7.48(c)]. Если же сигнал A будет равен 1, то у нас не будет двух одинаковых значений этого сигнала подряд, так что автомат перейдет в состояние A1, запомнив тем самым, что в последний раз была 1. Аналогично, из состояния A1 мы переходим в состояние OK, если получаем вторую 1 на входе A подряд, или в состояние A0, если придет 0 [см. рис. (d)].

Описание автомата говорит нам, что после того, как автомат попал в состояние OK, он может оставаться в нем, пока $B = 1$, независимо от значений сигнала на входе A [см. рис. 7.49(a)]. Если $B = 0$, то снова необходимо проверить, нет ли двух единиц или двух нулей подряд на входе A. Однако в этом случае мы сталкиваемся с небольшой проблемой. Текущее значение входного сигнала A может быть вторым следующим подряд тем же самым значением, а может и не быть; таким образом, мы можем опять остаться в состоянии OK или должны вернуться назад к состоянию A0 или A1. Мы определили состояние OK слишком широко: оно не «помнит» достаточно информации, чтобы сказать нам, куда идти дальше.

(a)		Значение					
Значение	S	A B				Z	
		00	01	11	10		
Начальное состояние	INIT	A0	A0	A1	A1	0	
На A был 0	A0	OK	OK	A1	A1	0	
На A была 1	A1	A0	A0	OK	OK	0	
На A дважды было одно и то же	OK	?	OK	OK	?	1	
S*							

(b)		Значение					
Значение	S	A B				Z	
		00	01	11	10		
Начальное состояние	INIT	A0	A0	A1	A1	0	
На A был 0	A0	OK0	OK0	A1	A1	0	
На A была 1	A1	A0	A0	OK1	OK1	0	
Для равных значений последнее A = 0	OK0					1	
Для равных значений последнее A = 1	OK1					1	
S*							

(c)		Значение					
Значение	S	A B				Z	
		00	01	11	10		
Начальное состояние	INIT	A0	A0	A1	A1	0	
На A был 0	A0	OK0	OK0	A1	A1	0	
На A была 1	A1	A0	A0	OK1	OK1	0	
Для равных значений последнее A = 0	OK0	OK0	OK0	OK1	A1	1	
Для равных значений последнее A = 1	OK1					1	
S*							

(d)		Значение					
Значение	S	A B				Z	
		00	01	11	10		
Начальное состояние	INIT	A0	A0	A1	A1	0	
На A был 0	A0	OK0	OK0	A1	A1	0	
На A была 1	A1	A0	A0	OK1	OK1	0	
Для равных значений последнее A = 0	OK0	OK0	OK0	OK1	A1	1	
Для равных значений последнее A = 1	OK1	A0	OK0	OK1	OK1	1	
S*							

Рис. 7.49. Продолжение процесса соетавления таблицы состояний

Эта проблема решается расщеплением состояния OK на рис. 7.49(b) на два состояния OK0 и OK1, которые «помнят» последнее значение сигнала на входе A. Для состояний A0 и A1 все следующие состояния можно выбрать из уже существующих, как это следует из рисунков 7.49(c) и (d). Если, например, автомат, находясь в состоянии OK0, получает 0 на входе A, то он может оставаться в состоянии OK0; мы не должны вводить новое состояние, чтобы «помнить» три нуля подряд, так как описание автомата не требует от нас обнаружения этого случая. Таким образом, мы достигли «замкнутости» таблицы состояний, которая описывает теперь автомат с *конечным* числом состояний. Ради спокойствия, в качестве проверки на рис. 7.50 повторены временные диаграммы, приведенные ранее на рис. 7.47, только на этот раз они сопровождаются перечислением состояний, через которые должен проходить автомат согласно пашей окончательной таблице состояний.

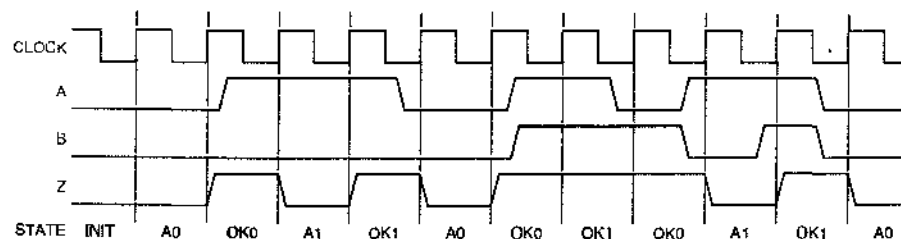


Рис. 7.50. Временные диаграммы и последовательность состояний для конечного автомата в рассматриваемом примере (CLOCK – тактовый сигнал, STATE – состояние)

7.4.2. Минимизация числа состояний

При нашем словесном описании таблица состояний на рис. 7.49(d) является «минимальной» в том смысле, что содержит наименьшее возможное число состояний. Но возможны и другие таблицы с большим числом состояний, как, например, на рис. 7.51, которые также будут работать. К таким таблицам можно применить формальную процедуру минимизации числа состояний.

(a)		A B					
Значение	S	00	01	11	10	Z	
Начальное состояние	INIT	A0	A0	A1	A1	0	
На A был 0	A0	OK00	OK00	A1	A1	0	
На A была 1	A1	A0	A0	OK11	OK11	0	
На A было 00	OK00	OK00	OK00	OKA1	A1	1	
На A было 11	OK11	A0	OKA0	OK11	OK11	1	
OK, на A был 0	OKA0	OK00	OK00	OKA1	A1	1	
OK, на A была 1	OKA1	A0	OKA0	OK11	OK11	1	
S*							

(b)		A B					
Значение	S	00	01	11	10	Z	
Начальное состояние	INIT	A0	A0	A1	A1	0	
На A был 0	A0	OK00	OK00	A1	A1	0	
На A была 1	A1	A0	A0	OK11	OK11	0	
На A было 00	OK00	OK00	OK00	A001	A1	1	
На A было 11	OK11	A0	A110	OK11	OK11	1	
На A было 001, B = 1	A001	A0	AE10	OK11	OK11	1	
На A было 110, B = 1	A110	OK00	OK00	AE01	A1	1	
На A было 0010, B = 1	AE10	OK00	OK00	AE01	A1	1	
На A было 0011, B = 1	AE01	A0	AE10	OK11	OK11	1	
S*							

Рис. 7.51. Неминимальные таблицы состояний, эквивалентные таблице на рис. 7.49(d)

Основная идея формальных процедур минимизации заключается в обнаружении *эквивалентных состояний* (*equivalent states*): два состояния эквивалентны, если их невозможно различить, наблюдая только текущие и следующие значения сигналов на выходе автомата (но не переменных состояния, характеризующих его внутренние состояния). Пару эквивалентных состояний можно заменить одним состоянием.

Два состояния S1 и S2 эквивалентны, если выполнены следующие два условия. Во-первых, находясь в состояниях S1 и S2 автомат должен вырабатывать одинаковые значения сигнала (или сигналов) на выходе; в случае автомата Мили это должно быть верно для всех входных комбинаций. Во-вторых, при каждой входной комбинации за состояниями S1 и S2 должно следовать либо одно и то же состояние, либо эквивалентные состояния.

НЕУЖЕЛИ ВСЕ ЭТО НУЖНО НА САМОМ ДЕЛЕ?

Детали формальных процедур минимизации числа состояний можно найти в литературе. Однако большинство разработчиков цифровых устройств редко пользуется этими процедурами. Более тщательное увязывание значения состояний с требованиями задачи позволяет опытным проектировщикам — в случае задач небольшого объема — составлять таблицы с минимальным или близким к минимальному числом состояний, не прибегая к формальной процедуре минимизации. Бывают также такие ситуации, когда *увеличение* числа состояний может упростить конструкцию или уменьшить ее стоимость; в таких случаях даже нет необходимости прибегать к помощи автоматизированной процедуры минимизации числа состояний. Разработчик может больше сделать для упрощения конечного автомата на этапе представления состояний в двоичной записи, о чем идет речь в разделе 7.4.3.

Применяя формальную процедуру минимизации числа состояний к таблице состояний, приведенной на рис. 7.51(а), мы видим, что состояния ОК00 и ОКА0 эквивалентны, поскольку в этих состояниях вырабатывается одно и то же значение выходного сигнала и их множества следующих состояний тождественны. Так как эти состояния эквивалентны, состояние ОК00 можно исключить, а там, где оно встречается в таблице, заменить его состоянием ОКА0; можно поступить и наоборот. Аналогично, эквивалентными являются состояния ОК11 и ОКА1.

Чтобы минимизировать таблицу состояний, приведенную на рис. 7.51(б), в рамках формальной процедуры придется прибегнуть к рассуждениям, несущим отчасти циклический характер. Находясь в состояниях ОК00, А110 и АЕ10, автомат вырабатывает один и тот же сигнал и в качестве следующих состояний имеет почти идентичные множества состояний, так что эти три состояния могли бы быть эквивалентными. Они строго эквивалентны только в том случае, если эквивалентны состояния А011 и АЕ01. Аналогично, состояния ОК11, А001 и АЕ01 эквивалентны только тогда, когда эквивалентны А110 и АЕ10. Другими словами, состояния, входящие в первый набор, эквивалентны, если эквивалентны состояния, входящие во второй набор, и наоборот. Так вот, давайте двигаться вперед, приняв, что они эквивалентны.

7.4.3. Кодирование состояний

Следующий шаг в процессе проектирования состоит в определении того, сколько требуется двоичных переменных для представления состояний в таблице состояний, и в распределении двоичных комбинаций между состояниями с теми или иными именами. Мы будем называть двоичную комбинацию, присвоенную конкретному состоянию, *кодом состояния (coded state)*. *Полное число состояний (total number of states)* автомата с n триггерами равно 2^n , так что число триггеров, необходимое для кодирования s состояний, равно $\lceil \log_2 s \rceil$, то есть наименьшему целому числу, большему, чем $\log_2 s$, или равному этой величине.

Ради удобства, в табл. 7.6 вновь воспроизведена таблица «состояние/выход» для автомата, рассматриваемого в качестве примера. У него пять состояний, так что ему требуется три триггера. Конечно, у трех триггеров полное число состояний равно восьми, поэтому в данном случае 3 из них являются *неиспользуемыми состояниями (unused states)*. Мы рассмотрим альтернативные варианты обращения с неиспользуемыми состояниями в конце этого раздела. А сейчас нам нужно разобраться с массой возможностей, возникающих при выборе способа кодирования состояний.

НАЧАЛЬНОЕ СОСТОЯНИЕ И СОСТОЯНИЕ НЕЗНАЯТОСТИ

Конечный автомат, рассматриваемый в этом разделе в качестве примера, попадает в начальное состояние только при запуске, тогда как многие автоматы проектируются таким образом, чтобы у них было «состояние незнания» ("idle" state), в которое автомат попадал бы как при запуске, так и во всех тех случаях, когда он ничего не должен делать.

S	A B				Z
	00	01	11	10	
INIT	A0	A0	A1	A1	0
A0	OK0	OK0	A1	A1	0
A1	A0	A0	OK1	OK1	0
OK0	OK0	OK0	OK1	A1	1
OK1	A0	OK0	OK1	OK1	1
S*					

Табл. 7.6. Таблица состояний и значений выходного сигнала в рассматриваемом примере

Простейший способ назначения s кодов состояний из 2^n возможных двоичных комбинаций заключается в использовании первых s целых двоичных чисел в порядке двоичного счета, как это сделано в левом столбце назначаемых кодов состояний в табл. 7.7. Однако простейший способ назначения не всегда приводит к простейшим уравнениям возбуждения и уравнениям выхода, и, следовательно, к наиболее простой принципиальной схеме. Действительно, назначение состояний часто существенно влияет на стоимость схемы и может быть тесно связано с другими факторами, такими как выбор элементов памяти (например, D-триггеры или JK-триггеры) и подход к реализации логики возбуждения и логики выхода (например, в виде «суммы произведений», «произведения сумм» или в каком-то частном виде применительно к данному проекту).

МАТЕМАТИКА ПРЕДОСТЕРЕГАЕТ

Число различных способов, которыми можно выбрать m кодов состояний из n возможных кодовых комбинаций, задается *биномиальным коэффициентом*

$\binom{n}{m}$, равным $\frac{n!}{m!(n-m)!}$. (Мы уже встречались с биномиальными коэффициентами в параграфе 2.10 в связи с кодированием десятичных чисел.) В

нашем примере имеется $\binom{8}{5}$ различных способов выбора пяти кодов состояний из восьми возможных, и при каждом таком выборе существует $5!$ способов распределения пяти состояний с именами по выбранным двоичным комбинациям. Таким образом, мы можем $\frac{8!}{5!3!} \cdot 5! = 6720$ способами устано-

вить соответствие между пятью состояниями нашего автомата и комбинациями из трех двоичных переменных состояния. Нам не хватит времени перебрать все способы.

Табл. 7.7. Возможные способы кодирования состояний в случае автомата, описываемого табл. 7.6

Имя состояния	Кодирование			
	Простейшее Q1–Q3	С разбиением Q1–Q3	Прямое Q1–Q5	Почти прямое Q1–Q4
INIT	000	000	00001	0000
A0	001	100	00010	0001
A1	010	101	00100	0010
OK0	011	110	01000	0100
OK1	100	111	10000	1000

Спрашивается: как же наилучшим образом выбрать кодирование состояний в данной задаче? В общем случае единственный способ найти *лучшее* кодирование состоит в том, чтобы перепробовать *все* варианты кодирования. Это, конечно, слишком большая работа даже для студента. Вместо этого большинство разработчиков цифровой техники полагаются на свой опыт и руководствуются несколькими практическими принципами при выборе разумного способа кодирования:

- Код начального состояния выбирается таким образом, чтобы автомат можно было легко установить в это состояние при запуске (в типичных схемах это 00...00 или 11...11).
- Минимизируется число перемешанных состояний, которые изменяются на каждом переходе.
- Максимизируется число переменных состояний, которые не изменяются в пределах группы связанных друг с другом состояний (то есть в пределах такой группы состояний, для которой большинство переходов не выводит за пределы этой группы).
- Используется симметрия в условиях задачи и соответствующая ей симметрия в таблице состояний. Другими словами, предполагается, что одно состояние или группа состояний означают почти то же самое, что и другое состояние или группа. После назначения кода состоянию первому из сопоставляемых по принципу симметрии состояний, подобное же назначение следует произвести и в отношении второго состояния, изменив лишь один бит.
- Если существуют неиспользуемые состояния (то есть если $s < 2^n$, $n = \lceil \log_2 s \rceil$), то выбираются «лучшие» из имеющихся комбинаций переменных состояний для достижения упомянутых выше целей. Из этого следует, что, как правило, не остаются на выборе в качестве кодов состояний s первых n -разрядных целых чисел.
- Множество переменных состояний разбивают на отдельные биты или поля таким образом, чтобы каждый бит или поле имели определенное значение в смысле воздействия входных сигналов или поведения автомата со стороны выхода.

- Рассматриваются варианты с использованием большего числа переменных состояния, нежели минимальное, с тем чтобы оказалось возможным кодирование с разбиением.

Некоторые из этих идей нашли свое отражение в кодировании «с разбиением», приведенном в табл. 7.7. Как и ранее, начальному состоянию присвоена комбинация 000, которую легко установить асинхронно (путем подачи сигнала RESET на входы CLR триггеров) или синхронно (путем пропускания сигнала RESET через вентили И на входы всех D-триггеров). Наилучший замечателен тем, что имеется только четыре состояния помимо состояния INIT, которое является совершенно «особым» состоянием: будучи однажды запущенным, автомат более никогда не попадет в это состояние. Поэтому можно выделить переменную состояния Q1 для указания, является данное состояние автомата состоянием INIT или нет, а переменные состояния Q2 и Q3 использовать для различения четырех других состояний, не являющихся состоянием INIT.

Присвоение двоичных комбинаций этим четырем состояниям выглядит в столбце табл. 7.7, озаглавленном «С разбиением», как осуществленное в порядке двоичного счета, но это простое совпадение. На самом деле разряды состояний Q2 и Q3 несут индивидуальную смысловую нагрузку, относящуюся к входам автомата и к его выходу. Бит Q3 указывает последнее значение сигнала на входе A, а по биту Q2 можно судить о том, выполняются ли в текущем состоянии условия, когда выходной сигнал должен равняться 1. Закрепляя за отдельными разрядами кода состояния такие значения, можно надеяться, что логика переходов и логика выхода окажутся проще, чем в том случае, когда комбинации Q2, Q3 будут «случайно» распределены между состояниями, не являющимися состоянием INIT. В следующих разделах мы продолжим разработку нашего конечного автомата на основе именно такого кодирования.

Другим полезным правилом назначения двоичных комбинаций, которое применимо к любым конечным автоматам, является *прямое кодирование* (*one-hot assignment*), также представленное в табл. 7.7. Число переменных состояния при таком способе представления больше минимального: предусматривается выделение по одному биту на каждое состояние. Помимо простоты, достоинством прямого кодирования является то, что оно приводит к самым коротким уравнениям возбуждения, так как в каждый триггер единица записывается только при переходе в одно состояние. Недостаток прямого кодирования, особенно для автоматов с большим числом состояний, очевиден: оно требует применения (много) большего числа триггеров, чем это минимально необходимо. Однако прямое кодирование является идеальным для таких автоматов с состояниями, выход которых должен указывать их текущее состояние в виде слов кода «1 из n ». В этом случае выходы триггеров, хранящих состояние в прямом коде, могут служить непосредственными выходами автомата и применения какой бы то ни было дополнительной комбинационной логики на выходе не требуется.

В последнем столбце табл. 7.7 приведено «почти прямое кодирование», в котором для начального состояния использована двоичная комбинация, не являющаяся одной из комбинаций прямого кодирования. В этом есть большой смысл по двум причинам: во-первых, легко в самом начале сбросить все элементы памяти в 0, а во-вторых, однажды запущенный, автомат никогда вновь не войдет в начальное

состояние. Реализация конечного автомата с таким кодированием состояний рассматривается в задачах 7.37 и 7.40.

Выше мы пообещали рассмотреть расстановку *неиспользуемых состояний*, когда число возможных состояний автомата при наличии n триггеров, равно 2^n , больше требуемого числа состояний автомата s . Имеют смысл два подхода, в зависимости от предъявляемых требований:

- *Минимальный риск.* Предполагается, что конечный автомат каким-то образом может попадать в неиспользуемые («запрещенные») состояния, например, из-за неисправностей аппаратуры, неожиданных входных воздействий или ошибок проектирования. Поэтому все неиспользуемые комбинации переменных состояния включаются в таблицу состояний и явным образом указываются переходы из них, так чтобы при любом входном воздействии автомат из неиспользуемого состояния попадал в «начальное состояние», «состояние незнания» или в какое-то другое «безопасное состояние». Иногда, по самой идеологии проектирования, это осуществляется автоматически, если начальное состояние закодировано комбинацией 00 ... 00.
- *Минимальная стоимость.* Предполагается, что автомат никогда не попадает в неиспользуемое состояние. Поэтому в таблице переходов и в таблице возбуждения неиспользуемые состояния можно пометить как «безразличные». В большинстве случаев это упрощает логику возбуждения. Правда, поведение автомата, если он все же попадает в неиспользуемое состояние, может оказаться при этом весьма причудливым.

Мы увидим, к чему приводят оба подхода, по мере разработки конечного автомата, выбранного нами в качестве примера.

7.4.4. Синтез с использованием D-триггеров

После того как состояниям автомата с различными именами присвоены коды состояний, оставшаяся часть разработки становится в значительной степени рутинным процессом. В разделе 7.11.2 и в самом деле будут рассмотрены программные средства, которые могут выполнить эту рутинную работу за вас. Однако в этом разделе мы пройдем по всем этапам вручную, для того чтобы впоследствии иметь возможность оценить эффективность действия программных средств.

Заменяя имена состояний в таблице состояний (возможно минимизированной) кодами состояний, мы получаем *таблицу переходов*. В таблице переходов для каждой комбинации кода состояния и входного воздействия указан код следующего состояния. Табл. 7.8 представляет собой таблицу переходов и значений выходного сигнала, которая получается для рассматриваемого конечного автомата с табл. 7.6 в качестве таблицы состояний в результате кодирования «с разбиением», указанного в табл. 7.7.

Следующий шаг заключается в составлении *таблицы возбуждения*; в этой таблице для каждой комбинации кода состояния и входного воздействия указываются значения сигналов, которые необходимо подать на входы триггеров, чтобы заставить автомат перейти в желаемое следующее состояние с соответствующим кодом. Структура и содержание этой таблицы зависят от типа используемых триггеров (D-, JK-, T-триггеры и т.д.). Обычно конкретный тип триггеров имеется

в виду с самого начала разработки устройства и уж *определенно* мы решили для себя этот вопрос, приступая к данному разделу, что нашло отражение в его заголовке. Действительно, в большинстве случаев конечные автоматы строятся сегодня на основе D-триггеров ввиду их наличия как в дискретном исполнении, так и внутри программируемых логических микросхем, а также по причине простоты их использования (по сравнению с JK-триггерами, как это станет ясным из следующего раздела).

			AB			
Q1	Q2	Q3	00	01	11	10
000			100	100	101	101
100			110	110	101	101
101			100	100	111	111
110			110	110	111	101
111			100	110	111	111
			Q1* Q2* Q3*			
			Z			
						0
						0
						0
						1
						1

Табл. 7.8. Таблицы переходов и значений выходного сигнала в задаче, решаемой в качестве примера

Из всех типов триггеров у D-триггеров самое простое характеристическое уравнение: $Q^* = D$. У каждого D-триггера в конечном автомате один D-вход, поэтому в таблице возбуждения должно быть указано значение сигнала, который необходимо подать на D-вход каждого триггера при всех возможных комбинациях (код состояния)/вход. В нашем примере таблица возбуждения имеет вид, указанный в табл. 7.9. Поскольку $D = Q^*$, таблица возбуждения тождественна таблице переходов, за исключением того, как называется то, что вносится в эту таблицу. Таким образом, при использовании D-триггеров фактически не нужно составлять отдельную таблицу возбуждения; достаточно назвать таблицу переходов таблицей «переход/возбуждение» (*transition/excitation table*).

			AB			
Q1	Q2	Q3	00	01	11	10
000			100	100	101	101
100			110	110	101	101
101			100	100	111	111
110			110	110	111	101
111			100	110	111	111
			D1 D2 D3			
			Z			
						0
						0
						0
						1
						1

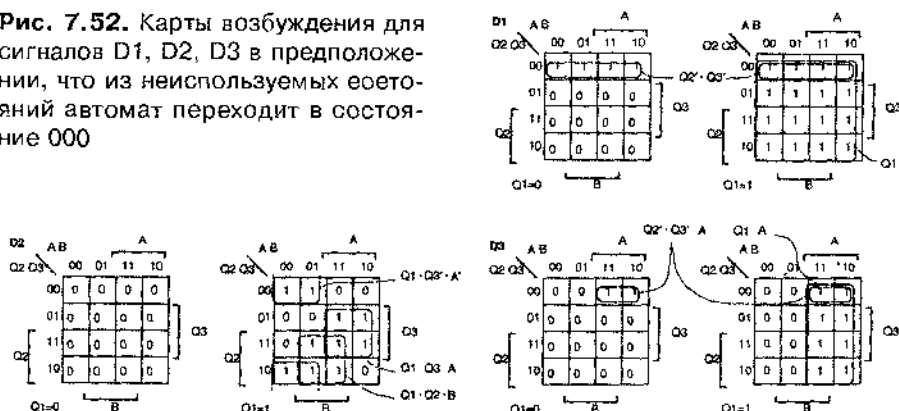
Табл. 7.9. Таблица возбуждения и значений выходного сигнала, получаемая из табл. 7.8 в случае использования D-триггеров

Таблица возбуждения похожа на таблицу истинности для трех комбинационных логических функций (D1, D2, D3) пяти переменных (A, B, Q1, Q2, Q3). В

соответствии с этим для реализации данных функций в виде схемы в нашем распоряжении любой из методов комбинационного проектирования. В частности, можно перенести информацию из таблицы возбуждения в карты Карно, которые мы можем называть теперь *картами возбуждения* (*excitation maps*), и для каждой функции найти минимальное выражение вида «сумма произведений» или «произведение сумм».

В случае нашего конечного автомата карты возбуждения имеют вид, показанный на рис. 7.52. Каждая из функций, например, $D1$, является функцией пяти переменных; поэтому для каждой из них мы должны воспользоваться *картой Карно с 5-ю переменными* (*5-variable Karnaugh map*). Карта с 5-ю переменными изображена в виде пары карт с 4-мя переменными, в которых одинаково расположенные клетки считаются соседними. Эти карты немного громоздки, но если вы хотите спроектировать вручную какую-нибудь не самый тривиальный конечный автомат, вам придется столкнуться с картами с 5-ю переменными или с еще худшим случаем. По крайней мере, мы предвидели это, перебирая комбинации входных сигналов в исходной таблице состояний в том порядке, в каком они встречаются в карте Карно, что облегчает перенос информации в карты на данном этапе. Заметьте, однако, что состояния *кодируются* не в том порядке, который принят в картах Карно; в частности, строки, соответствующие состояниям 110 и 111 в таблице возбуждения следуют в порядке, противоположном тому, в каком они расположены на карте.

Рис. 7.52. Карты возбуждения для сигналов $D1$, $D2$, $D3$ в предположении, что из неиспользуемых состояний автомат переходит в состояние 000



Именно на этом этапе переноса содержимого таблицы возбуждения в карты возбуждения обнаруживается, почему таблица возбуждения не является в точности таблицей истинности: в таблице возбуждения значения функций указаны *не для всех* комбинаций переменных. В частности, ничего не говорится о том, какими должны быть очередные состояния вслед за неиспользуемыми состояниями 001, 010 и 011. Здесь мы должны выбрать одну из стратегий обработки неиспользуемых состояний, о которых шла речь в предыдущем разделе: подход минимального риска или подход минимальной стоимости. На рис. 7.52 принята стратегия минимального риска: для каждого из неиспользуемых состояний и при любой комби-

наши входных сигналов в качестве следующего состояния взято начальное состояние 000, то есть состояние INIT. Результатом этого выбора являются три ряда цветных нулей в каждой карте Карно. Теперь, когда карты заполнены целиком, мы можем получить выражения вида «сумма произведений» для сигналов на входах триггеров:

$$D1 = Q1 + Q2' \cdot Q3'$$

$$D2 = Q1 \cdot Q3' \cdot A' + Q1 \cdot Q3 \cdot A + Q1 \cdot Q2 \cdot B$$

$$D3 = Q1 \cdot A + Q2' \cdot Q3' \cdot A.$$

Уравнение выхода легко выводится непосредственно из информации, содержащейся в табл. 7.9. В данном случае оно проще, чем уравнения возбуждения, так как выходной сигнал является функцией только состояния. Мы могли бы воспользоваться картой Карно, но легче найти выходную функцию минимального риска алгебраически, записав выходной сигнал Z как сумму двух кодов состояний (110 и 111), в которых этот сигнал равен 1:

$$\begin{aligned} Z &= Q1 \cdot Q2 \cdot Q3' + Q1 \cdot Q2 \cdot Q3 \\ &= Q1 \cdot Q2. \end{aligned}$$

Теперь мы, наконец, готовы реализовать наш проект конечного автомата. Если мы собираемся построить конечный автомат на дискретных триггерах и вентилях, то заключительный этап состоит в рисовании принципиальной схемы. С другой стороны, если мы используем программируемое логическое устройство, то нам необходимо только ввести уравнения возбуждения и выхода в компьютерный файл, которым задается, как именно будет запрограммировано устройство; пример таких действий рассматривается в разделе 7.11.1. Но если бы мы подумали заранее, то с самого начала сформулировали бы наши пожелания на языке описания конечных автоматов типа языка ABEL (см. раздел 7.11.2) и компьютер выполнил бы за нас всю работу, которую мы проделали в этом разделе!

РЕШЕНИЕ МИНИМАЛЬНОЙ СТОИМОСТИ

Если выбрать в нашем примере стратегию минимальной стоимости при выводе уравнений возбуждения, то в качестве элементов, выражающих состояния, следующие за неиспользуемыми состояниями, надо было бы указывать «безразличные состояния». Результатом такого выбора стали бы цветные символы d на рис. 7.53. Получаемые из этих карт уравнения возбуждения немного проще, чем то, что было у нас ранее:

$$D1 = 1$$

$$D2 = Q1 \cdot Q3' \cdot A' + Q3 \cdot A + Q2 \cdot B$$

$$D3 = A.$$

Значение выходного сигнала Z при неиспользуемых состояниях в этом случае также «безразлично», что приводит к еще более простой функции выхода: $Z = Q2$. Принципиальная схема автомата, построенного в соответствии со стратегией минимальной стоимости, приведена на рис. 7.54.

Рис. 7.53. Карты возбуждения для сигналов D1, D2 и D3 в предположении, что состояния, следующие за неиспользуемыми состояниями, «безразличны»

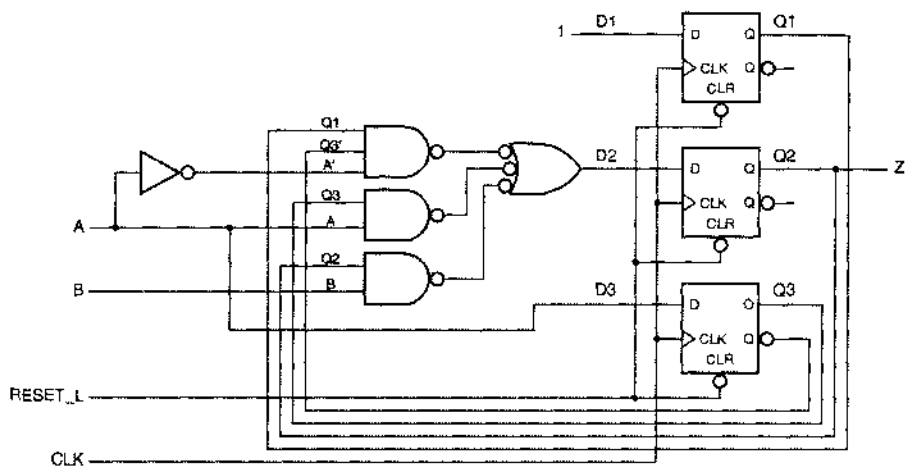
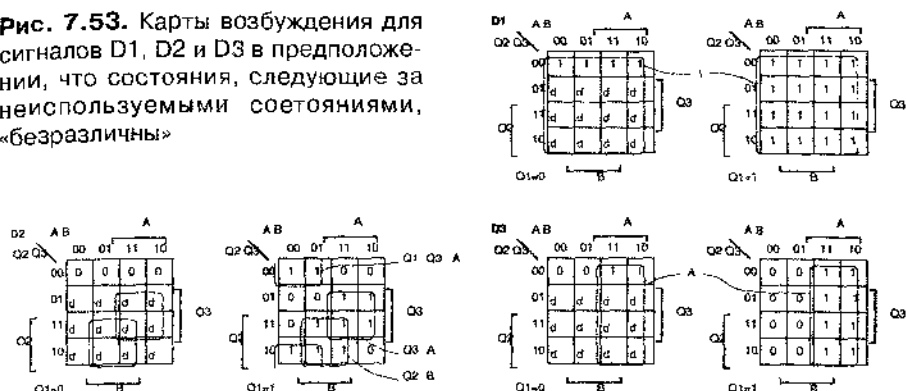


Рис. 7.54. Принципиальная схема автомата, описываемого картами, приведенными на рис. 7.53

*7.4.5. Синтез с использованием JK-триггеров

Одновременно JK-триггеры были популярным средством реализации конечных автоматов на основе дискретных ИС малой степени интеграции, поскольку JK-триггеры позволяли обеспечить большие функциональные возможности, нежели D-триггеры, в пересчете на один корпус одинаковых размеров. Под «большими функциональными возможностями» мы понимаем большее разнообразие комбинаций сигналов, подаваемых на входы J и K для управления JK-триггером по сравнению с тем, что можно делать с единственным входом D-триггера. Как следствие этого, логика возбуждения в конечном автомате на JK-триггерах может быть проще, чем при использовании D-триггеров, что приводит к уменьшению числа корпусов, когда логика возбуждения реализуется с помощью вентилях в ИС малой степени интеграции.

ТОЛЬКО ЧТОБЫ ПОУПРАЖНЯТЬСЯ

Во времена ИС малой степени интеграции минимизация логики возбуждения была важным делом, но с переходом на ПЛУ и специализированные ИС акцент сместился. Как вы можете догадаться, зная структуру И-ИЛИ комбинационных ПЛУ, необходимость наличия отдельных решеток И-ИЛИ для J- и K-входов JK-триггеров была бы очевидным недостатком последовательностных ПЛУ.

С точки зрения технологий, применяемых в специализированных ИС, JK-триггеры также нежелательны. Например, в случае БИС LCA500K фирмы Logic Corp., представляющих собой решетку КМОП-вентилей, макроячейка D-триггера FD1QP строится из 7 «вентильных элементов», а макроячейка JK-триггера FJK1QP состоит из 9 вентильных элементов, то есть занимает на 25% большую площадь кристалла. Поэтому проект оказывается более рентабельным, если отдать предпочтение D-триггерам и использовать дополнительную площадь кристалла для реализации более сложной логики возбуждения в тех случаях, конечно, когда это действительно нужно.

Мы все же рассмотрим в этом разделе синтез на основе JK-триггеров, но только для того, «чтобы поупражняться».

До этапа кодирования состояний включительно процедура разработки с использованием JK-триггеров, в основном, такая же, как и в случае с D-триггерами. Единственное различие состоит в том, что разработчик может остановиться на слегка отличающемся способе назначения состояниям двоичных комбинаций, имея в виду возможности, легко реализуемые на JK-триггерах (например, переключение в противоположное состояние при наличии единиц на входах J и K).

Значительное отличие возникает при составлении таблицы возбуждения ио таблице переходов. В случае D-триггеров эти две таблицы тождественны; характеристическое уравнение D-триггера ($Q^* = D$) позволяет в каждом элементе таблицы произвести замену $D = Q^*$. В случае JK-триггеров на каждом месте в таблице возбуждения вдвое больше двоичных разрядов, нежели в таблице переходов, так как у каждого триггера имеется два входа, на которые нужно подавать сигналы.

Характеристическое уравнение JK-триггера $Q^* = J \cdot Q' + K' \cdot Q$ нельзя преобразовать таким образом, чтобы получились два независимых уравнения для входов J и K. Вместо этого требуемые значения сигналов J и K находят как функции состояний Q и Q^* по *таблице использования JK-триггера (J-K application table)*, приведенной в табл. 7.10. Согласно первой строке, при нулевом текущем состоянии триггера все, что нужно для того, чтобы следующее значение Q также равнялось нулю, — это подать 0 на вход J; значение сигнала K не играет никакой роли. Аналогично, согласно третьей строке, при единичном текущем состоянии триггера следующее значение Q будет нулевым, если сигнал K равен 1 независимо от значения сигнала J. Каждый желаемый переход можно осуществить, подавая на входы J и K одну из двух различных комбинаций сигналов, поэтому в каждой из строк таблицы использования мы имеем «безразличные» значения.

Q	Q^*	J	K
0	0	0	d
0	1	1	d
1	0	d	1
1	1	d	0

Табл. 7.10. Таблица использования JK-триггеров

Чтобы составить таблицу возбуждения в случае применения JK-триггеров, разработчик должен посмотреть в таблицу переходов на значения каждого бита текущего состояния и желаемого следующего состояния и подставить соответствующую пару значений J и K из таблицы использования. Для таблицы переходов, приведенной в табл. 7.8, эти подстановки дадут таблицу возбуждения, представленную в табл. 7.11. Пусть, например, автомат находится в состоянии 100 и на входах действует комбинация сигналов 00; значение Q_1 равняется 1 и требуемое значение Q_1^* также равняется 1, поэтому для пары сигналов $J_1 K_1$ мы помещаем в таблицу возбуждения "d0". При той же комбинации «состояние/вход» Q_2 равно 0, требуемое значение Q_2^* равно 1, так что пара сигналов $J_2 K_2$ должна иметь значение "1d". Очевидно, что требуются определенное терпение и аккуратность при заполнении таблицы возбуждения (лучше всего оставить эту работу компьютеру).

Табл. 7.11. Таблица возбуждения и значений выходного сигнала для конечного автомата с табл. 7.8 в качестве таблицы переходов при использовании JK-триггеров

AB					
Q1 Q2 Q3	00	01	11	10	Z
000	1d, 0d, 0d	1d, 0d, 0d	1d, 0d, 1d	1d, 0d, 1d	0
100	d0, 1d, 0d	d0, 1d, 0d	d0, 0d, 1d	d0, 0d, 1d	0
101	d0, 0d, d1	d0, 0d, d1	d0, 1d, d0	d0, 1d, d0	0
110	d0, d0, 0d	d0, d0, 0d	d0, d0, 1d	d0, d1, 1d	1
111	d0, d1, d1	d0, d0, d1	d0, d0, d0	d0, d0, d0	1
J1 K1, J2 K2, J3 K3					

Как и в случае синтеза на основе D-триггеров в предыдущем разделе, таблица возбуждения является почти таблицей истинности для функций возбуждения. Эту информацию мы переносим на карты Карно, как показано на рис. 7.55.

В таблице возбуждения не оговорены состояния, следующие за неиспользуемыми состояниями, так что снова мы должны выбирать между подходами с позиций минимального риска и минимальной стоимости. Содержимое клеток в картах Карно, указанное на рис. 7.55 цветным шрифтом, – это результат выбора стратегии минимального риска.

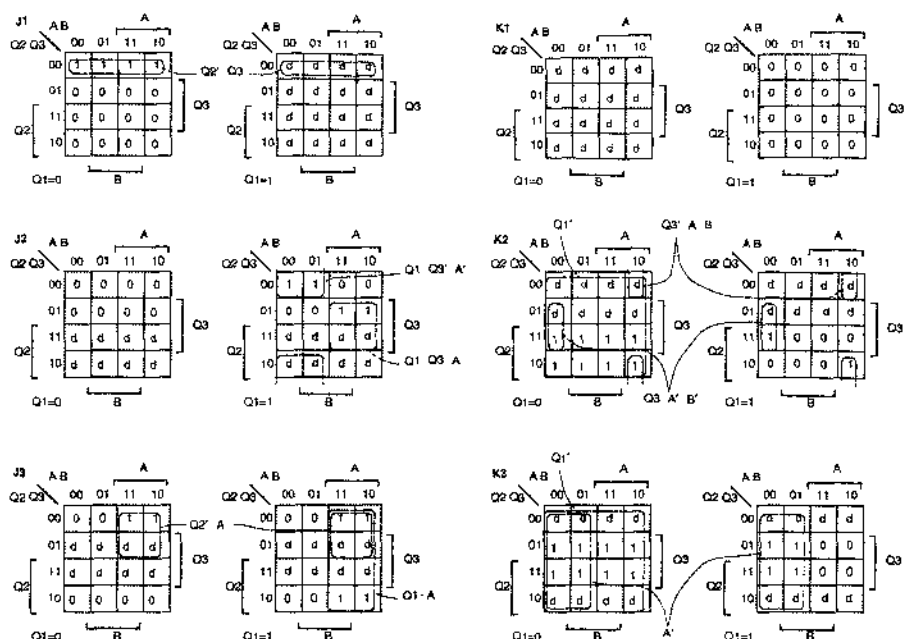


Рис. 7.55. Карты возбуждения для сигналов J1, K1, J2, K2, J3 и K3 в предположении, что из неиспользуемых состояний автомат переходит в состояние 000

Заметьте, что даже несмотря на наличие «безопасного» состояния 000, следующего за неиспользуемыми состояниями, мы все же не вписали нули в соответствующие клетки карт, как мы могли сделать это в случае D-триггеров. Вместо этого мы должны продолжить работу с таблицей использования, чтобы найти надлежащие комбинации сигналов J и K, необходимые для того, чтобы имело место равенство $Q^* = 0$ во всех ситуациях, относящихся к неиспользуемым состояниям; надо сказать еще раз, что это утомительный и сопровождающийся ошибками процесс.

На основании карт, представленных на рис. 7.55, можно вывести уравнения возбуждения с правыми частями вида «сумма произведений»:

$$\begin{aligned}
 J1 &= Q2' \cdot Q3' & K1 &= 0 \\
 J2 &= Q1 \cdot Q3' \cdot A' + Q1 \cdot Q3 \cdot A & K2 &= Q1' + Q3' \cdot A \cdot B' + Q3 \cdot A' \cdot B' \\
 J3 &= Q2' \cdot A + Q1 \cdot A & K3 &= Q1' + A'.
 \end{aligned}$$

Для реализации этих уравнений требуется на два вентиля больше, чем при использовании D-триггеров согласно стратегии минимального риска, так что мы ничего не сэкономили на применении JK-триггеров и менее всего в отношении времени, которое затрачено на проектирование.

РЕШЕНИЕ МИНИМАЛЬНОЙ СТОИМОСТИ

В нашем примере карты возбуждения при выборе стратегии минимальной стоимости составить немного легче, поскольку во всех местах в таблице переходов, где должны быть указаны состояния, следующие за неиспользуемыми, можно просто поставить символ безразличия "d". Уравнения возбуждения с правыми частями вида «сумма произведений», следующие из (не приведенных) карт минимальной стоимости имеют вид:

$$J1 = 1$$

$$K1 = 0$$

$$J2 = Q1 \cdot Q3' \cdot A' + Q3 \cdot A$$

$$K2 = Q3' \cdot A \cdot B' + Q3 \cdot A' \cdot B'$$

$$J3 = A$$

$$K3 = A'$$

Кодирование состояний при использовании JK-триггеров то же самое, что и в случае D-триггеров, так что уравнения выхода такие же как для минимального риска ($Z = Q1 \cdot Q2$), так и для минимальной стоимости ($Z = Q2$).

Принципиальная схема, реализующая уравнение минимальной стоимости, представлена на рис. 7.56. В этой схеме на два вентиля больше, чем в схеме минимальной стоимости на D-триггерах, приведенной на рис. 7.54, из чего следует, что JK-триггеры так ничего нам и не сэкономили.

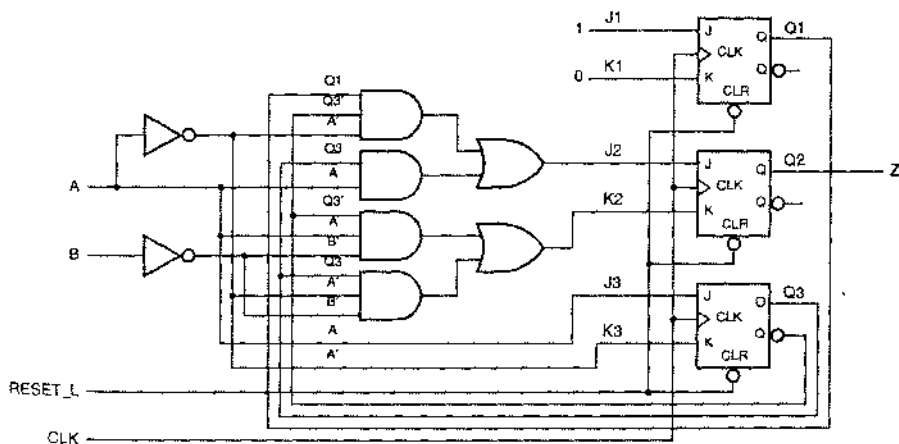


Рис. 7.56. Принципиальная схема рассматриваемого в качестве примера конечного автомата на JK-триггерах с логикой возбуждения минимальной стоимости

7.4.6. Дальнейшие примеры проектирования на основе D-триггеров

В заключение этого параграфа приведем еще два примера проектирования конечных автоматов на основе D-триггеров. Первый пример — это «автомат, считающий число единиц»:

Построить тактируемый синхронный конечный автомат с двумя входами X и Y и одним выходом Z . Выходной сигнал должен равняться 1, если число единиц, поступивших на входы X и Y с момента запуска, кратно 4; в противном случае сигнал на выходе должен равняться 0.

С первого взгляда может показаться, что этому автомату понадобится бесконечное число состояний, коль скоро подсчет числа единиц на входах продолжается неограниченно долго. Но выходной сигнал должен указывать число принятых единиц *по модулю 4*, поэтому будет достаточно четырех состояний. Назовем их S_0 – S_3 , где S_0 – начальное состояние, а полное число принятых единиц, когда автомат находится в состоянии S_i , равно i по модулю 4. В результате получаем таблицу состояний и значений выходного сигнала, приведенную в табл. 7.12.

Табл. 7.12. Таблица состояний и значений выходного сигнала для автомата, считающего единицы.

Значение	S	XY				Z
		00	01	11	10	
Принято 0 единиц (по модулю 4)	S_0	S_0	S_1	S_2	S_1	1
Принята 1 единица (по модулю 4)	S_1	S_1	S_2	S_3	S_2	0
Принято 2 единицы (по модулю 4)	S_2	S_2	S_3	S_0	S_3	0
Принято 3 единицы (по модулю 4)	S_3	S_3	S_0	S_1	S_0	0
S^*						

Для кодирования четырех состояний автомата, считающего единицы, можно воспользоваться двумя переменными состояния и при этом неиспользуемых состояний не будет. В таком случае имеется только 4! возможных способов присвоить состояниям с именами те или иные кодовые комбинации. И все же мы выберем из них только один и вполне определенный. Мы установим соответствие между состояниями с именами и кодами состояний в том порядке, какой принят на картах Карно (00, 01, 11, 10), по двум причинам: во-первых, при заданной таблице состояний это минимизирует число изменяющихся переменных состояния в большинстве переходов, потенциально упрощая тем самым уравнения возбуждения; во-вторых, это упростит механический перенос информации в карты возбуждения.

При выбранном способе кодирования состояний таблица «переход/возбуждение» имеет вид, указанный в табл. 7.13. Поскольку мы используем D-триггеры, таблицы переходов и возбуждения одинаковы. Соответствующие карты Карно для сигналов D1 и D2 показаны на рис. 7.57. Так как неиспользуемых состояний нет, вся нужная нам информация содержится в таблице возбуждения и не нужно выбирать между минимальным риском и минимальной стоимостью. Уравнения возбуждения можно записать, глядя на карты, а уравнение выхода непосредственно вытекает из таблицы «переход/возбуждение»:

$$\begin{aligned}
 D1 &= Q2 \cdot X' \cdot Y + Q1' \cdot X \cdot Y + Q1 \cdot X' \cdot Y' + Q2 \cdot X \cdot Y' \\
 D2 &= Q1' \cdot X' \cdot Y + Q1' \cdot X \cdot Y' + Q2 \cdot X' \cdot Y' + Q2' \cdot X \cdot Y \\
 Z &= Q1' \cdot Q2'
 \end{aligned}$$

По этим уравнениям можно составить принципиальную схему на D-триггерах с логикой возбуждения на структурах И-ИЛИ или И-НЕ-И-НЕ.

Q1 Q2	XY				Z
	00	01	11	10	
00	00	01	11	01	1
01	01	11	10	11	0
11	11	10	00	10	0
10	10	00	01	00	0

Q1 → Q2* or D1 D2

Табл. 7.13. Таблица «переход/возбуждение» и значения выходного сигнала для автомата, считающего единицы

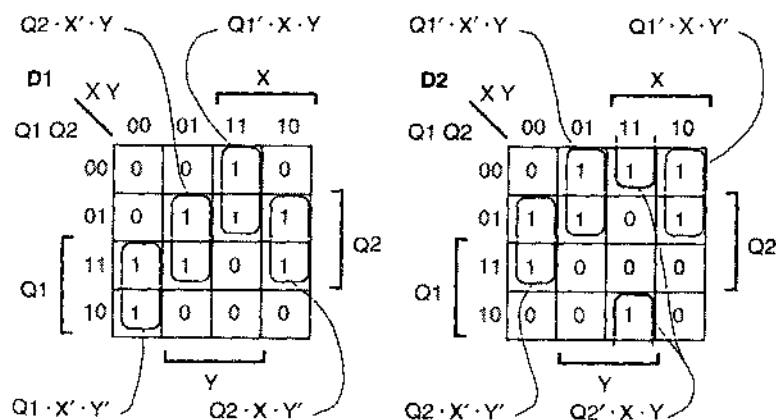


Рис. 7.57. Карты возбуждения для входных сигналов D1 и D2 в автомате, считающем единицы

Второй пример – конечный автомат, управляющий кодовым замком: автомат выдает сигнал «открыть», когда ему на вход поступает определенная двоичная комбинация:

Построить тактируемый синхронный конечный автомат с одним входом X и двумя выходами UNLK и HINT. Выходной сигнал UNLK должен равняться 1 в том и только в том случае, когда X равно 0, а в предшествующие семь тактов на вход X поступила последовательность 0110111. Выходной сигнал HINT должен равняться 1 тогда и только тогда, когда текущее значение X является правильным с точки зрения продвижения автомата в сторону «открытого» состояния (то есть состояния с UNLK = 1).

Из словесного описания очевидно, что речь идет об автомате Мили. Значение выходного сигнала UNLK зависит как от предыстории входных воздействий, так и от текущего значения X , и сигнал HINT зависит от состояния и от того, что действует на входе (в самом деле, если текущее значение входного сигнала X таково, что $HINT = 0$, то знающий секрет пользователь захочет изменить значение X до наступления очередного такта).

Таблица состояний и значений выходных сигналов для кодового замка представлена в табл. 7.14. Предполагается, что в начальном состоянии A не принято никакой части требуемой входной последовательности; автомат ждет прихода первого 0 последовательности. Поэтому он остается в состоянии A в течение всего времени, пока на вход поступают единицы; и переходит в состояние B , когда принят 0. Находясь в состоянии B , автомат ждет прихода 1. Если поступает именно она, то автомат переходит в состояние C ; если нет, то должен оставаться в состоянии B , поскольку только что принятый 0 может оказаться первым нулем требуемой последовательности. Пребывая в каждом следующем состоянии, автомат будет переходить дальше, если на вход поступает правильный сигнал, и возвращаться в состояния A или B в случае прихода неправильного сигнала. Исключением является состояние G ; если автомат находится в этом состоянии и на вход поступает неправильное значение сигнала (0), то предыдущие три входных сигнала все же могут оказаться первыми тремя битами требуемой последовательности, так что автомату надо возвращаться в состояние E , а не в состояние B . Попав в состояние H , когда принята требуемая последовательность, автомат выдает 1 на выходе UNLK, если X будет равен 0. В каждом состоянии вырабатывается единичное значение сигнала HINT, если значение X таково, что оно продвигает нас ближе к состоянию H .

Значение	S	X	
		0	1
"Счет не открыт"	A	B, 01	A, 00
Принято 0	B	B, 00	C, 01
Принято 01	C	B, 00	D, 01
Принято 011	D	E, 01	A, 00
Принято 0110	E	B, 00	F, 01
Принято 01101	F	B, 00	G, 01
Принято 011011	G	E, 00	H, 01
Принято 0110111	H	B, 11	A, 00
S*, UNLK HINT			

Табл. 7.14. Таблица состояний и значений выходных сигналов для автомата, управляющего кодовым замком

Восемь состояний кодового замка можно представить тремя переменными состояниями; неиспользуемых состояний при этом не будет. Соответствие между состояниями с именами и двоичными комбинациями можно выбрать $8!$ способами. Чтобы не усложнять себе жизнь, воспользуемся простейшим способом кодирования и назначим состояниям $A-H$ кодовые комбинации в порядке двоичного

счета, что приведет к таблице «переход/возбуждение», приведенной в табл. 7.15. Соответствующие карты Карно для сигналов D1, D2 и D3 показаны на рис. 7.58. Неносредственно из этих карт получаем уравнения возбуждения:

$$D1 = Q1 \cdot Q2' \cdot X + Q1' \cdot Q2 \cdot Q3 \cdot X' + Q1 \cdot Q2 \cdot Q3'$$

$$D2 = Q2' \cdot Q3 \cdot X + Q2 \cdot Q3' \cdot X$$

$$D3 = Q1 \cdot Q2' \cdot Q3' + Q1 \cdot Q3 \cdot X' + Q2' \cdot X' + Q3' \cdot Q1' \cdot X' + Q2 \cdot Q3' \cdot X.$$

Значения выходных сигналов переносим из таблицы «переход/возбуждение» в другой набор карт (рис. 7.59). Соответствующие уравнения выхода имеют вид:

$$UNLK = Q1 \cdot Q2 \cdot Q3 \cdot X'$$

$$HINT = Q1' \cdot Q2' \cdot Q3' \cdot X' + Q1 \cdot Q2' \cdot X + Q2' \cdot Q3 \cdot X + Q2 \cdot Q3 \cdot X' + Q2 \cdot Q3' \cdot X.$$

Заметьте, что в уравнениях возбуждения и в уравнениях выхода есть повторяющиеся термы, что дает возможность немного сэкономить на стоимости реализации логики И-ИЛИ. Если бы мы преодолели трудности формальной одновременной минимизации нескольких логических функций в отношении всех пяти функций возбуждения и выхода, то мы могли бы сэкономить еще два вентиля (см. задачу 7.55).

Q1 Q2 Q3	X	
	0	1
000	001, 01	000, 00
001	001, 00	010, 01
010	001, 00	011, 01
011	100, 01	000, 00
100	001, 00	101, 01
101	001, 00	110, 01
110	100, 00	111, 01
111	001, 11	000, 00

Q1*Q2*Q3*, UNLK HINT

Табл. 7.15. Таблица «переход/возбуждение» для автомата, управляющего кодовым замком

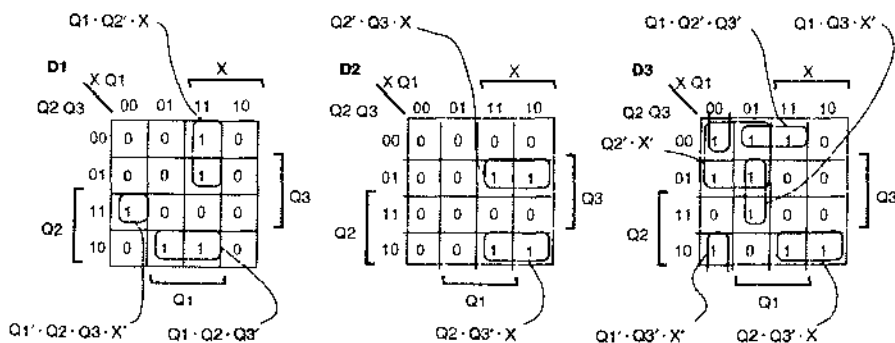


Рис. 7.58. Карты возбуждения для сигналов D1, D2 и D3 в автомате, управляющем кодовым замком

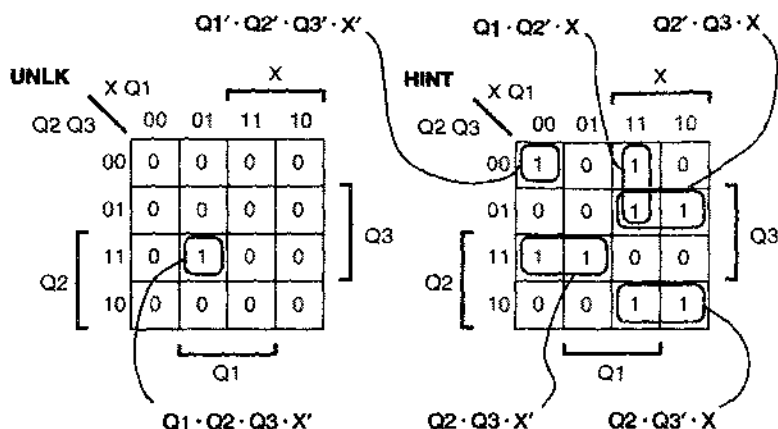


Рис. 7. 59. Карты Карно для выходных функций UNLK и HINT в автомате, управляющем кодовым замком

7.5. Проектирование конечных автоматов с помощью диаграмм состояний

Помимо планирования структуры цифровой системы в целом, построение конечных автоматов, возможно, является самой творческой задачей, какую приходится решать конструкторам цифровых устройств. Большинству из них нравится графический подход. Вам, наверное, приходилось решать многие задачи, рисуя эскиз. По этой причине при проектировании небольших конечных автоматов и устройств средних размеров часто пользуются диаграммами состояний. В этом параграфе мы приведем примеры разработки с использованием диаграмм состояний и опишем простую процедуру синтеза схем на их основе. Эта процедура лежит в основе метода, реализуемого средствами автоматизированного проектирования, которые могут создавать логическую структуру из графического представления или даже из текстового представления диаграмм состояний.

Составление диаграмм состояний во многом подобно составлению таблицы состояний, которое, в свою очередь, очень похоже на написание программы, как мы видели это в разделе 7.4.1. Однако имеется одно принципиальное различие между диаграммой состояний и таблицей состояний, которое делает составление диаграммы состояний проще, хотя при этом риск допустить ошибку увеличивается:

- Таблица состояний представляет собой исчерпывающий список следующих состояний для каждой комбинации состояние/вход. Никакая неоднозначность не возможна.
- В диаграмме состояний имеется множество стрелок, у которых надписаны выражения переходов. Даже при наличии многих входов с каждой стрелкой связано только одно выражение перехода. Однако при составлении диаграммы состояний нет гарантии, что выражениями переходов у стрелок, выходящих из данного состояния, все комбинации входных сигналов покрываются точно по одному разу.

В неоднозначной диаграмме состояний (*ambiguous state diagram*), то есть в диаграмме состояний, составленной ненадлежащим образом, для некоторых комбинаций входных сигналов следующее состояние может оказаться не заданным, что в общем случае нежелательно, тогда как для других комбинаций входных сигналов может быть указано несколько следующих состояний, что, очевидно, неправильно. Таким образом, составление диаграмм состояний необходимо осуществлять с большой осторожностью. Приведем несколько примеров.

Нашим первым примером будет конечный автомат, управляющий задними огнями в автомобиле марки Ford Thunderbird выпуска 1965 года (рис. 7.60). С каждой стороны имеется по три фонаря, и при поворотах они зажигаются последовательно, как показано на рис. 7.61, чтобы наглядно продемонстрировать направление поворота. У конечного автомата два входных сигнала LEFT и RIGHT, которые несут поступающую от водителя информацию о его намерении совершить левый или правый поворот. Имеется также вход аварийного мигания HAZ; при поступлении сигнала на этот вход задние огни должны работать в аварийном режиме: все шесть фонарей мигают одновременно, включаясь и выключаясь. Мы предположим также, что есть автономный источник тактового сигнала, частота которого равна желаемой частоте мигания.

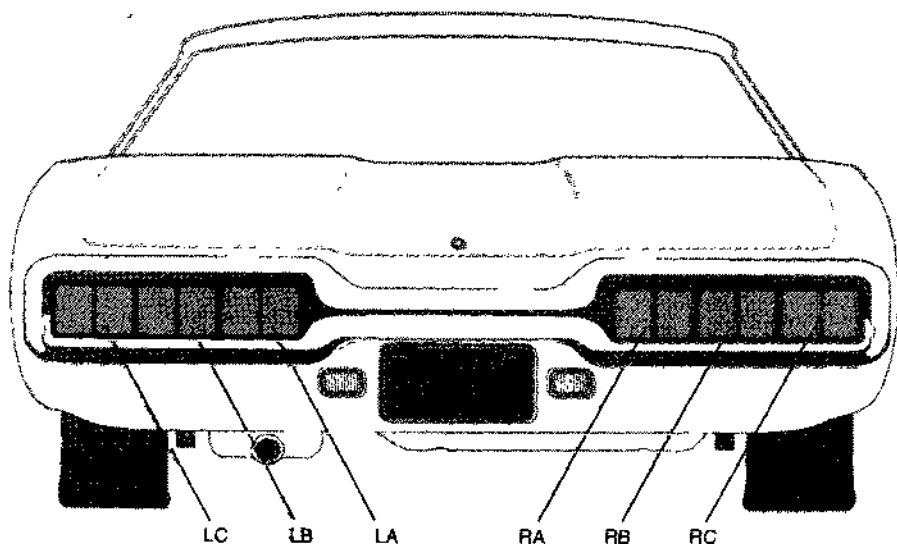


Рис. 7.60. Задние фонари автомобиля марки Ford Thunderbird

ЧЕЙ ЭТО ВИД СЗАДИ?

То, что изображено на рис. 7.60, на самом деле больше похоже на вид сзади у автомобиля марки Mercury Capri, который тоже имеет бегущие задние огни.

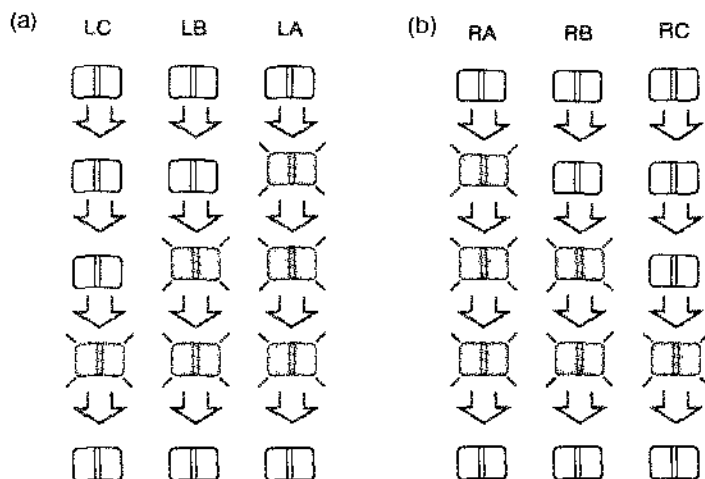


Рис. 7.61. Последовательность переключения задних огней у автомобиля марки Ford Thunderbird (а) при повороте налево, (б) при повороте направо

При перечисленных требованиях мы можем настроить тактируемый синхронный конечный автомат для управления задними огнями этого автомобиля. Мы создадим автомат Мура, так что только от состояния будет зависеть, какие огни окажутся зажженными, а какие — погашенными. При левом повороте автомат циклически должен проходить через четыре состояния, в каждом из которых правые огни не горят, а слева включены 0, 1, 2 или 3 фонаря. Аналогично, при правом повороте должны циклически повторяться четыре состояния с погашенными левыми огнями и с числом горящих фонарей справа 0, 1, 2 или 3. В аварийном режиме необходимы только два состояния, когда все огни включены и все огни выключены.

На рис. 7.62 показан первый набросок диаграммы состояний для такого автомата. Введено общее состояние IDLE, в котором все огни погашены. Когда включен левый поворот, автомат проходит три состояния с 1, 2 и 3 зажженными левыми огнями, а затем возвращается в состояние IDLE; то же происходит при правом повороте. В аварийном режиме автомат периодически переключается из состояния IDLE в состояние со всеми шестью зажженными огнями и обратно. Поскольку в данном случае выходов много, мы не стали указывать значения выходных сигналов на диаграмме состояний, а составили отдельную таблицу выхода. Глядя на эту таблицу, можно написать уравнения выхода, даже не прибегая к представлению состояний с именами в виде двоичных комбинаций, если допустить, что каждое имя состояния является логическим выражением, принимающим единичное значение только в этом состоянии:

$$LA = L1 + L2 + L3 + LR3$$

$$LB = L2 + L3 + LR3$$

$$LC = L3 + LR3$$

$$RA = R1 + R2 + R3 + RR3$$

$$RB = R2 + R3 + RR3$$

$$RC = R3 + RR3$$

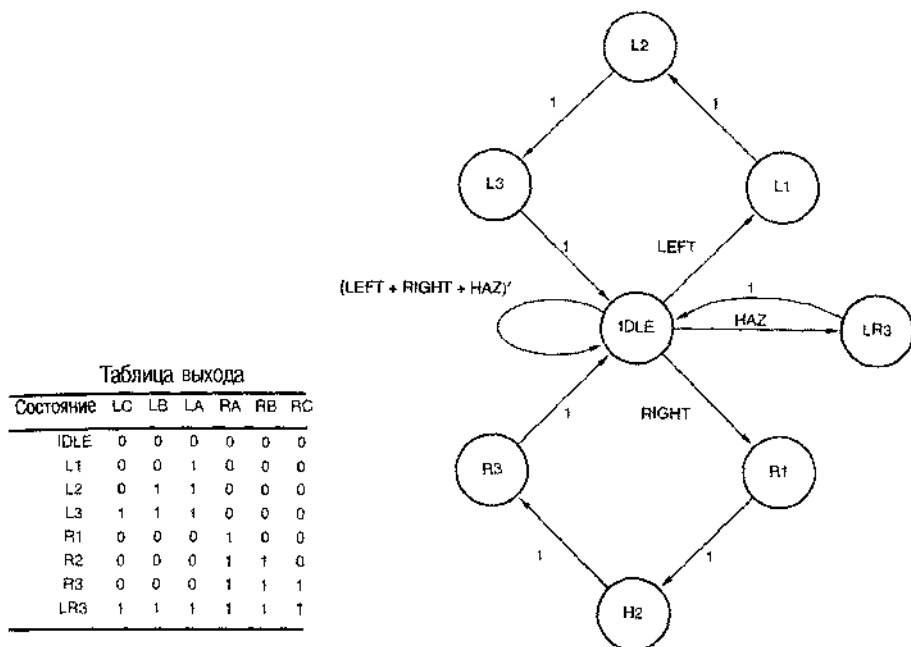


Рис. 7.62. Первоначальный вариант диаграммы состояний и таблица выхода для автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

В диаграмме состояний, представленной на рис. 7.62, остается нерешенной одна существенная проблема: в ней не предусмотрена обработка случаев, когда одновременно действует несколько входных сигналов. Например, что произойдет, если автомат находится в состоянии IDLE и поступают оба сигнала LEFT и HAZ? Согласно диаграмме состояний автомат переходит в два состояния L1 и LR3, что невозможно. На практике автомат перешел бы только в одно следующее состояние, которым могло бы быть L1, LR3 или вообще какое-то третье (возможно неиспользуемое) состояние, не имеющее никакого отношения к данной ситуации, в зависимости от того, как именно реализован конечный автомат (см., например, задачу 7.58).

Эта проблема преодолена в диаграмме состояний, показанной на рис. 7.63, где более высоким приоритетом наделяется входной сигнал HAZ. Кроме того, одновременное включение сигналов LEFT и RIGHT воспринимается как требование аварийного режима, поскольку водитель явно смущен и ему нужна помощь.

В новой диаграмме состояний нет неоднозначности, несколько выражаемая переходами, надписанные у стрелок, выходящих из каждого состояния, являются взаимно исключительными и исчерпывающими в совокупности. Другими словами, для каждого состояния нет двух выражений, которые равнялись бы 1 при одной и той же комбинации входных сигналов, и при каждой входной комбинации какое-то одно из выражений равно 1. В этом можно убедиться алгебраически, выполнив в отношении этой или любой другой диаграммы состояний следующие два шага:

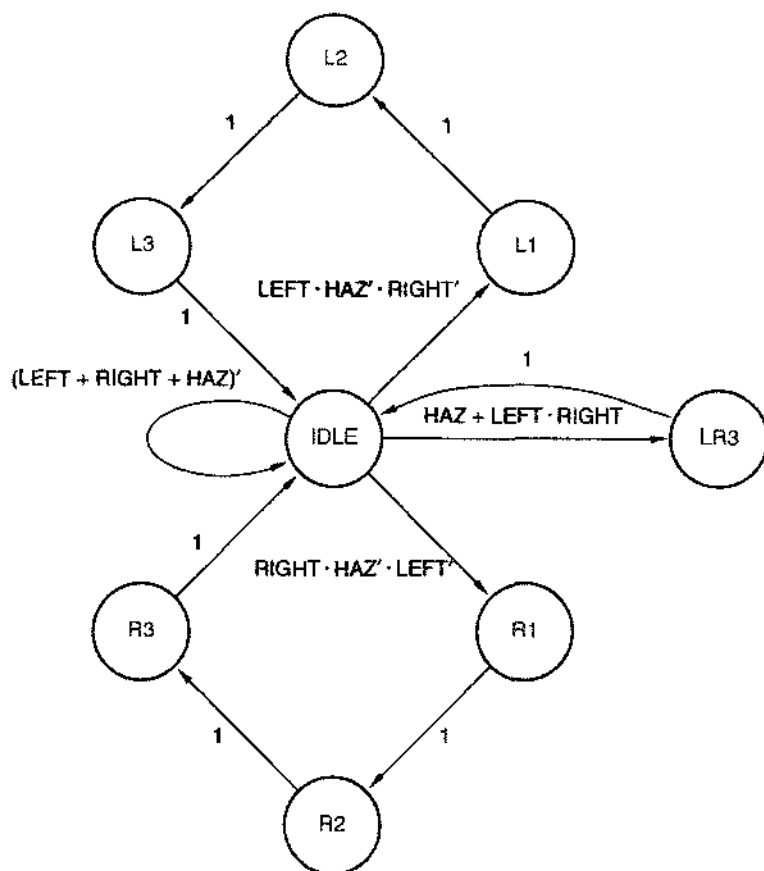


Рис. 7.63. Исправленная диаграмма состояний для автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

1. *Взаимное исключение (mutual exclusion)*. Для каждого состояния показать, что все возможные попарные логические произведения выражений переходов у стрелок, выходящих из данного состояния, равны 0. Если имеется n стрелок, то вычислить нужно $n(n-1)/2$ логических произведений.
2. *Исчерпывающее включение (all inclusion)*. Для каждого состояния показать, что логическая сумма выражений переходов у всех стрелок, выходящих из данного состояния, равна 1.

Когда имеется много переходов из каждого состояния, эти шаги, особенно первый из них, очень трудно выполнимы. Однако в типичном случае, даже при наличии у конечного автомата большого числа состояний и входов, число переходов из каждого состояния невелико, потому что большинство разработчиков вообще не в состоянии вообразить себе такие сложные автоматы. Именно в этом

проявляются относительные достоинства и недостатки таблиц состояний и диаграмм состояний. При составлении таблицы состояний указанные два аргумента не нужны, так как сама структура таблицы состояний гарантирует взаимное исключение переходов и полный охват всех возможностей. Но при большом числе входов у таблицы состояний *слишком много* столбцов.

Проверка диаграммы состояний на однозначность может быть трудной в принципе, но не очень страшной на практике в случае небольших диаграмм состояний. На рис. 7.63 из большинства состояний выходит единственная стрелка, у которой в качестве выражения перехода указана 1; в этом случае проверка тривиальна. На самом деле, требует проверки только состояние IDLE, из которого возможно четыре перехода. Это можно сделать на черновике, перечислив восемь комбинаций трех входных сигналов и убедившись, что каждая из них покрывается тем или иным выражением перехода. Каждая входная комбинация должна давать 1 точно в одном выражении. В качестве упражнения можно рассмотреть диаграммы состояний на рис. 7.44 и 7.46: обе они проверяются в уме.

Вернемся к автомату, управляющему задними огнями автомобиля марки Ford Thunderbird. Теперь, если есть желание, мы можем приступить к синтезу схемы по диаграмме состояний. Однако в случае, если мы хотим чуть изменить поведение автомата, то сейчас самое время это сделать до синтеза схемы. В частности, обратите внимание на следующее: если автомат начнет выполнять цикл правого или левого поворота, то, согласно диаграмме состояний на рис. 7.63, этот цикл будет выполнен до конца даже в том случае, если появится сигнал HAZ. Может быть это и хорошо с точки зрения эстетики рисунка, но для тех, кто находится в автомобиле, было бы безопаснее заставлять автомат переходить в аварийный режим возможно скорее. Видоизмененная диаграмма состояний, в которой сделано это уточнение, приведена на рис. 7.64.

Теперь мы окончательно готовы синтезировать схему автомата. В диаграмме имеется восемь состояний, поэтому нам необходимы, как минимум, три триггера, чтобы хранить коды состояний. Очевидно, что кодирование состояний можно выполнить многими способами (8! способами, если быть точным); мы воспользуемся тем из них, который указан в табл. 7.16, по следующим причинам:

1. Код 000 присваивается начальному состоянию (незанятости), поскольку большинство триггеров и регистров легко устанавливаются в 0.
2. Две переменные состояния Q1 и Q0 позволяют при выполнении левого поворота осуществлять «счет» в последовательности, задаваемой кодом Грея (IDLE $\rightarrow$ L1 $\rightarrow$ L2 $\rightarrow$ L3 $\rightarrow$ IDLE). Это сводит к минимуму число изменяющихся переменных состояния при переходе от одного состояния к другому, а это часто упрощает логику возбуждения.
3. В силу симметрии переменные Q1 и Q0 проходят ту же самую последовательность значений и при правом повороте. За различие между левым и правым поворотами отвечает переменная Q2.
4. Остающаяся комбинация переменных состояния используется для обозначения состояния LR3.

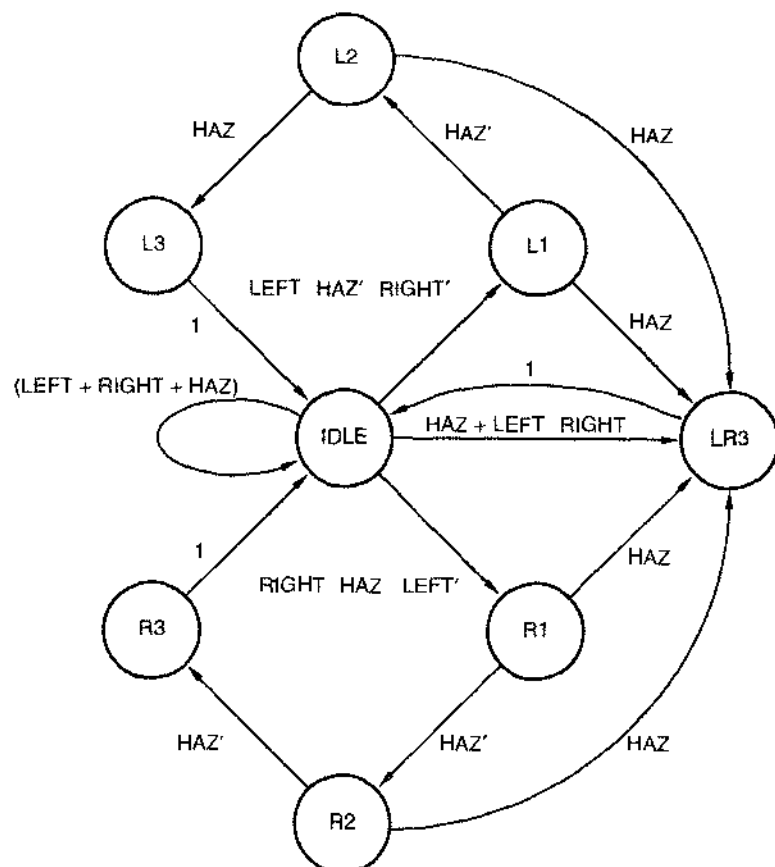


Рис. 7.54. Усовершенствованная диаграмма состояний для автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

Состояние	Q2	Q1	Q0
IDLE	0	0	0
L1	0	0	1
L2	0	1	1
L3	0	1	0
R1	1	0	1
R2	1	1	1
R3	1	1	0
LR3	1	0	0

Табл. 7.16. Кодирование состояний конечного автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

Следующий шаг состоит в написании своего рода таблицы переходов. Однако нам следует воспользоваться другим форматом, отличающимся от приведенного в разделе 7.4.4, поскольку в диаграмме состояний переходы задаются выражениями, а не исчерпывающим перечислением следующих состояний. Мы обратимся к записи переходов в новой форме, в виде *списка переходов (transition list)*, в котором одна строка соответствует каждому переходу, выражаемому стрелкой на диаграмме состояний.

Такой список переходов для диаграммы состояний на рис. 7.64 и способа кодирования, указанного в табл. 7.16, представлен в табл. 7.17. Каждая строка содержит текущее состояние, следующее состояние и выражение перехода для одной из стрелок на диаграмме состояний. Указаны оба названия текущего и следующего состояний: их имена и их коды. Имена состояний полезны с точки зрения привязки к рисунку, а коды используются при составлении уравнений переходов.

Табл. 7.17. Список переходов для конечного автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

S	Q2	Q1	Q0	Выражение перехода	S*	Q2*	Q1*	Q0*
IDLE	0	0	0	(LEFT + RIGHT + HAZ)	IDLE	0	0	0
IDLE	0	0	0	LEFT HAZ' RIGHT'	L1	0	0	1
IDLE	0	0	0	HAZ (LEFT RIGHT	LR3	1	0	0
IDLE	0	0	0	RIGHT HAZ' LEFT'	R1	1	0	1
L1	0	0	1	HAZ'	L2	0	1	1
L1	0	0	1	HAZ	LR3	1	0	0
L2	0	1	1	HAZ'	L3	0	1	0
L2	0	1	1	HAZ	LR3	1	0	0
L3	0	1	0	1	IDLE	0	0	0
R1	1	0	1	HAZ'	R2	1	1	1
R1	1	0	1	HAZ	LR3	1	0	0
R2	1	1	1	HAZ'	R3	1	1	0
R2	1	1	1	HAZ	LR3	1	0	0
R3	1	1	0	1	IDLE	0	0	0
LR3	1	0	0	1	IDLE	0	0	0

После того как список переходов составлен, оставшаяся часть синтеза представляет собой рутинную работу довольно большого объема. Процедурам син-

теза посвящен параграф 7.6. Хотя эти процедуры можно реализовать вручную, они обычно бывают включены в наборы программ системы автоматизированного проектирования (системы САД); таким образом, параграф 7.6 может помочь вам разобраться с тем, что делает (или в чем ошибается) ваш любимый программный продукт.

Мы уже столкнулись в этом параграфе с одним таким рутинным шагом — с поиском неоднозначности в диаграммах состояний. Хотя рассмотренную нами процедуру нетрудно автоматизировать, почти никакие программы систем САД этого не делают. Например, одна из программ «ввода диаграмм состояний» молча удаляет повторяющиеся переходы и в случае пропуска переходов указывает переход в состояние с кодовым именем “00...00”, не выдавая пользователю предупреждений. Таким образом, прибегая к помощи тех или иных средств проектирования, разработчик в большинстве случаев сам несет ответственность за неоднозначность в описании конечного автомата. Хорошим подспорьем в этом служат языки описания конечных автоматов, о которых идет речь в конце этой главы.

***7.6. Синтез конечных автоматов на основе списка переходов**

Построением диаграммы состояний автомата и выбором способа кодирования, по существу, исчерпывается творческая часть процесса проектирования. Остальную часть процедуры синтеза можно выполнить с помощью программ системы САД.

Как показано в предыдущем параграфе список переходов составляется по диаграмме состояний автомата с учетом выбранных кодов состояний. В этом параграфе мы продемонстрируем, как по списку переходов синтезируется сам конечный автомат. Подробно рассматривается ряд возможностей и нюансов, возникающих при проектировании конечного автомата на основе списка переходов. Хотя материал этого параграфа и полезен для синтеза автоматов вручную, его главное назначение — помочь вам понять логику работы и возможные капризы программ и языков, используемых в системах САД при конструировании конечных автоматов.

***7.6.1. Уравнения переходов**

Первый шаг при синтезе конечного автомата по списку переходов заключается в выводе системы уравнений переходов, которыми задаются следующие значения V^* всех переменных состояния как функции текущего состояния и входного воздействия. Список переходов можно считать своего рода гибридной таблицей истинности, в которой комбинации переменных состояния перечислены явно, а комбинации входных сигналов приведены в алгебраической форме. Двигаясь сверху вниз по столбцу V^* в списке переходов, мы получаем последовательность нулей и единиц, то есть значений V^* для различных комбинаций состояния/вход (при условии, конечно, что мы все записали в списке переходов правильно).

Выражение для следующего значения переменной состояния V^* в уравнении переходов может быть разновидностью гибридной канонической суммы:

$$V^* = \sum_{\text{по всем строкам списка переходов, в которых } V^* = 1} (\text{p-терм перехода}).$$

Другими словами, в уравнение переходов входит по одному «р-терму перехода» на каждую строку списка переходов, содержащую 1 в столбце V^* . *р-терм перехода (transition p-term)* данной строки – это произведение минтерма текущего состояния и выражения перехода.

Согласно списку переходов в табл. 7.17 уравнение переходов для переменной $Q2^*$ автомата, управляющего задними огнями автомобиля марки Ford Thunderbird, можно записать в виде суммы, состоящей из восьми р-термов:

$$\begin{aligned} Q2^* = & Q2' \cdot Q1' \cdot Q0' \cdot (HAZ + LEFT \cdot RIGHT) \\ & + Q2' \cdot Q1' \cdot Q0' \cdot (RIGHT \cdot HAZ' \cdot LEFT') \\ & + Q2' \cdot Q1' \cdot Q0 \cdot (HAZ) \\ & + Q2' \cdot Q1 \cdot Q0 \cdot (HAZ) \\ & + Q2 \cdot Q1' \cdot Q0 \cdot (HAZ') \\ & + Q2 \cdot Q1' \cdot Q0 \cdot (HAZ) \\ & + Q2 \cdot Q1 \cdot Q0 \cdot (HAZ') \\ & + Q2 \cdot Q1 \cdot Q0 \cdot (HAZ). \end{aligned}$$

Непосредственными алгебраическими преобразованиями это соотношение упрощается в результате объединения первых двух, следующих двух и последних четырех р-термов:

$$\begin{aligned} Q2^* = & Q2' \cdot Q1' \cdot Q0' \cdot (HAZ + RIGHT) \\ & + Q2' \cdot Q0 \cdot (HAZ) \\ & + Q2 \cdot Q0. \end{aligned}$$

Аналогично могут быть получены уравнения переходов для $Q1^*$ и $Q0^*$:

$$\begin{aligned} Q1^* = & Q2' \cdot Q1' \cdot Q0' \cdot (HAZ') \\ & + Q2' \cdot Q1 \cdot Q0 \cdot (HAZ') \\ & + Q2 \cdot Q1' \cdot Q0 \cdot (HAZ') \\ & + Q2 \cdot Q1 \cdot Q0 \cdot (HAZ') \\ & = Q0 \cdot HAZ' \\ Q0^* = & Q2' \cdot Q1' \cdot Q0' \cdot (LEFT \cdot HAZ' \cdot RIGHT') \\ & + Q2' \cdot Q1' \cdot Q0' \cdot (RIGHT \cdot HAZ' \cdot LEFT') \\ & + Q2' \cdot Q1' \cdot Q0 \cdot (HAZ') \\ & + Q2 \cdot Q1' \cdot Q0 \cdot (HAZ') \\ & = Q2' \cdot Q1' \cdot HAZ' (LEFT \oplus RIGHT) + Q1' \cdot Q0 \cdot HAZ'. \end{aligned}$$

За исключением переменной $Q1^*$, нет никаких гарантий, что найденные соотношения для переходов являются в том или ином смысле минимальными: действительно, выражения для $Q2^*$ и $Q0^*$ не имеют даже стандартного вида «суммы произведений» или «произведения сумм». Упрощения или неупрощенные уравнения служат лишь четко сформулированной отправной точкой, чтобы можно было выбрать какой-нибудь комбинационный метод синтеза логики возбуждения в конечном автомате: на основе структуры И-НЕ-И-НЕ, на ИС средней степени интеграции или как-то еще. При разработке на основе ПЛУ вы могли бы просто вставить эти уравнения в программу на языке ABEL и велеть компьютеру найти

минимальные выражения вида «сумма произведений» для реализации на решетке И–ИЛИ в ПЛУ.

*7.6.2. Уравнения возбуждения

Несмотря на то, что мы уже подошли к логике возбуждения, до сих пор нами выведены только *уравнения переходов*, но не *уравнения возбуждения*. Однако в случае, когда в качестве элементов памяти в наших конечных автоматах применяются D-триггеры, уравнения возбуждения являются тривиальным следствием уравнений переходов, поскольку характеристическое уравнение D-триггера имеет вид: $Q^* = D$. Поэтому из уравнения переходов для переменной состояния Q_i^*

$Q_i^* = \text{выражение}$

получаем следующее уравнение возбуждения для сигнала на входе соответствующего D-триггера:

$D_i = \text{выражение.}$

Рациональные уравнения возбуждения для триггеров других типов, особенно для JK-триггеров, выводятся не так легко (см. задачу 7.63). По этой причине в подавляющем большинстве случаев в конечных автоматах, создаваемых на дискретных компонентах, на основе ПЛУ или специализированных ИС, используются D-триггеры.

*7.6.3. Варианты схем

Существуют и другие способы получения уравнений переходов и уравнений возбуждения из списка переходов. Если столбец со следующими значениями какой-то одной переменной состояния содержит меньше нулей, чем единиц, то имеет смысл записать уравнение переходов для этой переменной в терминах нулей в этом столбце:

$$V^{*'} = \sum_{\text{по всем строкам списка переходов, в которых } V^* = 0} (\text{p-терм перехода}).$$

Другими словами, $V^{*'}$ равно 1 для всех p-термов, для которых V^* равно 0. Следовательно, уравнения переходов для $Q_2^{*'}$ можно записать в виде суммы, состоящей из семи p-термов:

$$\begin{aligned} Q_2^{*' } &= Q_2' \cdot Q_1' \cdot Q_0' \cdot ((\text{LEFT} + \text{RIGHT} + \text{HAZ})') \\ &+ Q_2' \cdot Q_1' \cdot Q_0' \cdot (\text{LEFT} \cdot \text{HAZ}' \cdot \text{RIGHT}') \\ &+ Q_2' \cdot Q_1' \cdot Q_0 \cdot (\text{HAZ}') \\ &+ Q_2' \cdot Q_1 \cdot Q_0 \cdot (\text{HAZ}') \\ &+ Q_2' \cdot Q_1 \cdot Q_0' \cdot (1) \\ &+ Q_2 \cdot Q_1 \cdot Q_0' \cdot (1) \\ &+ Q_2 \cdot Q_1' \cdot Q_0' \cdot (1) \\ &= Q_2' \cdot Q_1' \cdot Q_0' \cdot \text{HAZ}' \cdot \text{RIGHT}' + Q_2' \cdot Q_0 \cdot \text{HAZ}' + Q_1 \cdot Q_0' + Q_2 \cdot Q_0'. \end{aligned}$$

Чтобы прийти к уравнению для Q_2^* , достаточно просто взять дополнения обеих частей последнего, свернутого равенства.

Выражение для следующего значения переменной состояния V^* можно получить из нулей в списке переходов непосредственно, беря дополнение правой части общего выражения для V^* , тогда на основании теоремы Де Моргана приходим к разновидности гибридного канонического произведения:

$$V^* = \prod_{\text{по всем строкам списка переходов, в которых } V^* = 0} (s\text{-терм перехода}).$$

Здесь *s-терм перехода* (*transition s-term*) данной строки — это сумма макстерма текущего состояния и дополнения выражения перехода. Если выражение перехода является простым термом-произведением, то его дополнение есть сумма, и поэтому переменная V^* в уравнении переходов представляется в виде произведения сумм.

*7.6.4. Реализация конечного автомата

После того как уравнения возбуждения для конечного автомата получены, все, что нам остается, — это решить задачу построения комбинационной логики с несколькими выходами. Конечно, реализовать комбинационную логику, представленную уравнениями, можно многими способами, но простейший из них состоит в том, чтобы набрать соответствующую программу на языке ABEL или VHDL и воспользоваться компилятором для синтеза схемы в ПЛУ, в ИС типа FPGA или в специализированной ИС.

В комбинационных ПЛУ типа PAL16L8 и GAL16V8, рассмотренных нами в параграфе 5.3, можно реализовать уравнения возбуждения с числом входов, выходов и термов-произведений, не превосходящим определенного значения. Лучшие все же воспользоваться последовательностными ПЛУ, которые будут представлены в параграфе 8.3: в них на одном кристалле имеются и D-триггеры и комбинационная решетка И-ИЛИ. При одном и том же числе выводов, используемых в качестве входов и выходов, в последовательностных ПЛУ можно реализовать конечные автоматы большего объема, нежели в эквивалентных комбинационных ПЛУ, поскольку не нужно выводить сигналы возбуждения за пределы кристалла. В разделе 9.1.3 мы покажем, как реализуется автомат, управляющий задними огнями автомобиля марки Ford Thunderbird, в последовательностном ПЛУ.

*7.7. Другой пример проектирования конечного автомата

В этом параграфе приводится еще один пример построения конечного автомата по диаграмме состояний. На этом примере обсуждаются такие вопросы, как неиспользуемые состояния, кодирование состояний выходными комбинациями сигналов и назначение кодовых имен безразличным состояниям.

7.7.1. Игра на угадывание

Наш заключительный пример конечного автомата — «игра на угадывание»; этот забавный автомат можно построить в качестве лабораторного упражнения:

Построить тактируемый синхронный конечный автомат с четырьмя входами $G1-G4$, подключенными к кнопкам. У автомата четыре выхода $L1-L4$, к которым подключены лампочки или светодиоды, расположенные рядом с кнопками с теми же номерами. Имеется также выход ERR , к которому подключена красная лампочка. При нормальной работе на выходах $L1-L4$ индицируется комбинация «1 из 4». На каждом такте комбинация сдвигается на одну позицию; частота тактового сигнала равна 4 Гц.

Задача игрока состоит в том, чтобы вовремя нажать кнопку, соответствующую горящей лампочке. При нажатии i -ой кнопки вырабатывается единичный сигнал G_i . Если подан «ненравильный» сигнал, то возникает сигнал на выходе ERR и загорается красная лампочка; это происходит в том случае, когда автомат на очередном такте обнаруживает сигнал, номер которого не совпадает с номером лампочки, зажженной на предыдущем такте. Когда кнопка нажата, игра останавливается, и сигнал на выходе ERR сохраняет свое значение в течение одного или нескольких тактов, пока не будет снят удерживаемый вами сигнал G_i , и тогда игра возобновляется.

Ясно, что автомат должен иметь четыре состояния, по одному на каждую конфигурацию зажженных и погашенных лампочек, и, по меньшей мере, еще одно состояние, чтобы указывать, что игра остановлена. Возможная диаграмма состояний приведена на рис. 7.65. Автомат циклически проходит через состояния $S1-S4$, пока не подан ни один сигнал G_i , и переходит в состояние $STOP$, когда какая-либо кнопка нажата. В состоянии S_i вырабатывается единичный сигнал на выходе L_i .

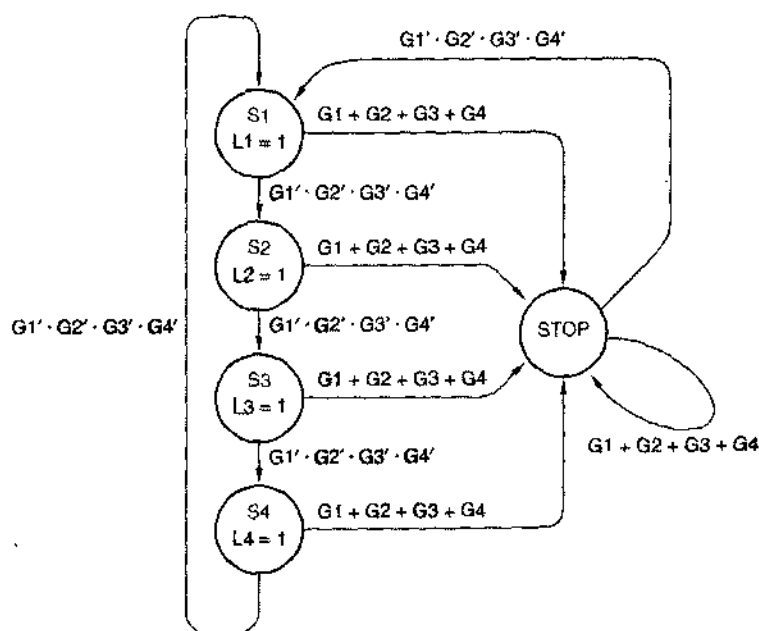


Рис. 7.65. Первая попытка нарисовать диаграмму состояний автомата для игры на угадывание

Чем плоха эта диаграмма состояний, так это тем, что она не обеспечивает «запоминания» в состоянии STOP, правильно вы угадали или нет, и, значит, нельзя узнать, каким должен быть выходной сигнал ERR. Эта проблема решается в диаграмме состояний на рис. 7.66 введением двух состояний «останова»: SOK и SERR. Если вы не угадали, то автомат переходит в состояние SERR, в котором вырабатывается единичный сигнал ERR; в противном случае, автомат переходит в состояние SOK. Хотя словесное описание автомата этого и не требует, в диаграмме состояний предусмотрен переход в состояние SERR также в том случае, когда игрок попытается обмануть автомат одновременным нажатием двух или большего числа кнопок, а также при попытке сменить нажатую кнопку, когда игра остановлена.

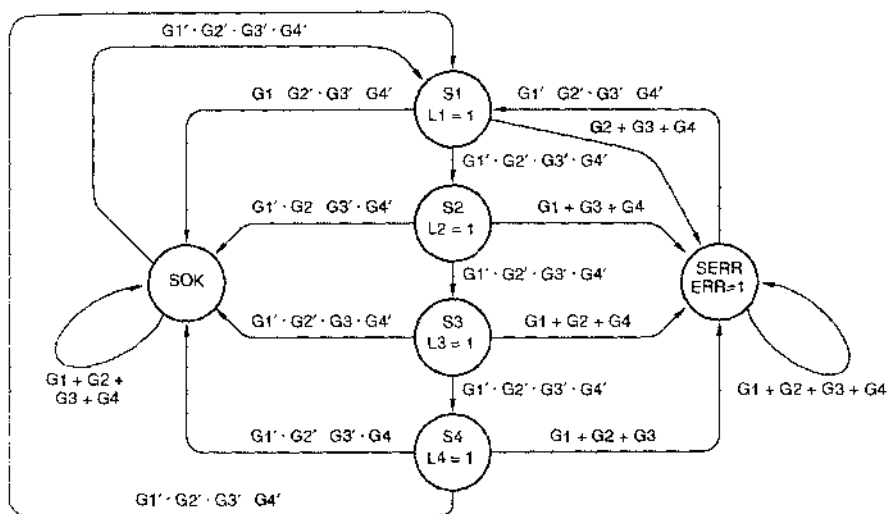


Рис. 7.66. Правильная диаграмма состояний автомата для игры на угадывание

Список переходов, соответствующий диаграмме состояний на рис. 7.66, представлен в табл. 7.18 в предположении, что состояния кодируются 3-разрядными двоичными словами кода Грея в том порядке, в каком они проходятся по циклу: S1–S4. Из этой таблицы получаются следующие уравнения переходов для Q1* и Q0*:

$$\begin{aligned}
 Q1^* &= Q2' \cdot Q1' \cdot Q0 \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &\quad + Q2' \cdot Q1 \cdot Q0 \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &= Q2' \cdot Q0 \cdot G1' \cdot G2' \cdot G3' \cdot G4' \\
 Q0^* &= Q2' \cdot Q1' \cdot Q0' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &\quad + Q2' \cdot Q1' \cdot Q0 \cdot (G2 + G3 + G4) \\
 &\quad + Q2' \cdot Q1' \cdot Q0 \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &\quad + Q2' \cdot Q1' \cdot Q0 \cdot (G1 + G3 + G4) \\
 &\quad + Q2' \cdot Q1 \cdot Q0 \cdot (G1 + G2 + G4) \\
 &\quad + Q2' \cdot Q1 \cdot Q0' \cdot (G1 + G2 + G3) \\
 &\quad + Q2 \cdot Q1' \cdot Q0 \cdot (G1 + G2 + G3 + G4).
 \end{aligned}$$

С помощью программы минимизации можно свести логическое выражение для Q0* к виду «сумма произведений» с 11 термами-произведениями. Выражение для Q2* лучше всего записать, перебирая нули в столбце Q2* в табл. 7.18:

$$\begin{aligned}
 Q2^{*'} &= Q2' \cdot Q1' \cdot Q0' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &+ Q2' \cdot Q1' \cdot Q0 \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &+ Q2' \cdot Q1 \cdot Q0 \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &+ Q2' \cdot Q1 \cdot Q0' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &+ Q2 \cdot Q1' \cdot Q0' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &+ Q2 \cdot Q1' \cdot Q0 \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
 &= (Q2' + Q1') \cdot (G1' \cdot G2' \cdot G3' \cdot G4').
 \end{aligned}$$

Табл. 7.18. Список переходов автомата для игры на угадывание

Текущее состояние				Выражение перехода	Следующее состояние				Выход				
S	Q2	Q1	Q0		S*	Q2*	Q1*	Q0*	L1	L2	L3	L4	ERR
S1	0	0	0	G1' G2' G3' G4'	S2	0	0	1	1	0	0	0	0
S1	0	0	0	G1 G2' G3' G4'	SOK	1	0	0	1	0	0	0	0
S1	0	0	0	G2 + G3 + G4	SERR	1	0	1	1	0	0	0	0
S2	0	0	1	G1' G2' G3' G4'	S3	0	1	1	0	1	0	0	0
S2	0	0	1	G1' G2 G3' G4'	SOK	1	0	0	0	1	0	0	0
S2	0	0	1	G1 + G3 + G4	SERR	1	0	1	0	1	0	0	0
S3	0	1	1	G1' G2' G3' G4'	S4	0	1	0	0	0	1	0	0
S3	0	1	1	G1' G2' G3 G4'	SOK	1	0	0	0	0	1	0	0
S3	0	1	1	G1 + G2 + G4	SERR	1	0	1	0	0	1	0	0
S4	0	1	0	G1' G2' G3' G4'	S1	0	0	0	0	0	0	1	0
S4	0	1	0	G1' G2' G3' G4	SOK	1	0	0	0	0	0	1	0
S4	0	1	0	G1 + G2 + G3	SERR	1	0	1	0	0	0	1	0
SOK	1	0	0	G1 + G2 + G3 + G4	SOK	1	0	0	0	0	0	0	0
SOK	1	0	0	G1' G2' G3' G4'	S1	0	0	0	0	0	0	0	0
SERR	1	0	1	G1 + G2 + G3 + G4	SERR	1	0	1	0	0	0	0	1
SERR	1	0	1	G1' G2' G3' G4'	S1	0	0	0	0	0	0	0	1

Последние пять столбцов в табл. 7.18 содержат значения выходных сигналов. Следовательно, уравнение выхода можно вывести в значительной степени так же, как выводятся уравнения переходов. Однако в данном примере мы имеем дело с автоматом Мура и поэтому в выходные сигналы выражения переходов не входят; для каждого текущего состояния необходимо рассматривать только одну строку в списке переходов. Уравнения выхода имеют вид:

$$L1 = Q2' \cdot Q1' \cdot Q0'$$

$$L3 = Q2' \cdot Q1 \cdot Q0$$

$$ERR = Q2 \cdot Q1' \cdot Q0.$$

$$L2 = Q2' \cdot Q1' \cdot Q0$$

$$L4 = Q2' \cdot Q1 \cdot Q0'$$

*7.7.2. Неиспользуемые состояния

В нашей диаграмме состояний для игры на угадывание шесть состояний, но у реального автомата с тремя триггерами восемь возможных состояний. Не упомянутые неиспользуемые состояния в списке переходов, мы тем самым считаем их «безразличными» в следующем ограничении смысле:

- При составлении уравнений для $Q1^*$ и $Q0^*$ мы образуем сумму р-термов переходов для тех комбинаций состояние/выход, которые явным образом отмечены единицами в соответствующих столбцах списка переходов. Хотя мы и не принимаем во внимание неиспользуемые состояния, в этой процедуре неявно считается, что им соответствуют нули в столбцах $Q1^*$ и $Q0^*$.
- И наоборот, если мы записываем уравнение для $Q2^*$ в виде суммы р-термов переходов для комбинаций состояние/выход, имеющих нули в соответствующем столбце списка переходов, то в отношении неиспользуемых состояний неявно считается, что у них в столбце $Q2^*$ стоят единицы.

Как следствие такого выбора, за всеми неиспользуемыми состояниями автомата для игры на угадывание следуют состояния с кодом 100 при любых входных комбинациях. Это не страшно: можно позволить автомату сбиться и войти в неиспользуемое состояние, поскольку кодовое имя 100 соответствует одному из нормальных состояний (SOK).

Если бы мы хотели обрабатывать неиспользуемые состояния как истинные «безразличные» состояния, то нам следовало бы позволить им переходить в любые следующие состояния при каких-то комбинациях входных сигналов. Это простой принцип, но его реализация на практике может оказаться затруднительной.

В конце раздела 7.4.4 мы показали, как можно обойтись с неиспользуемыми состояниями, которые полагаются «безразличными», при выводе уравнений переходов и уравнений возбуждения методом карт Карно. К сожалению, карты Карно слишком громоздки во всех случаях, кроме самых простейших задач. С большими задачами легко справляются имеющиеся в продаже программы логической минимизации, но многие из них не допускают наличия «безразличных» значений, либо требуют от разработчика введения для них специального кода. В программе для конечного автомата на языке ABEL совсем легко указывать состояния, следующие за безразличными состояниями, с помощью директивы @DCSET, как это объясняется в одном из замечаний в разделе 7.11.2. На языке VHDL это выглядит несколько неуклюже.

*7.7.3. Кодирование состояний выходными комбинациями

Давайте рассмотрим другую реализацию автомата для игры на угадывание. Сигналы на выходе автомата являются функциями только состояния; более того, в каждом из состояний с разными именами вырабатываются *различные* выходные комбинации. Поэтому можно воспользоваться выходными сигналами в качестве переменных состояния и присвоить каждому состоянию с именем кодовое обозначение в виде требуемой комбинации выходных сигналов. Такого рода *кодирование состояний выходными комбинациями* (*output-coded state assignment*) иногда приводит к более простым уравнениям возбуждения, чем в случае, когда для кодирования состояний используется минимальное число переменных состояния.

В табл. 7.19 приведен список переходов автомата для игры на угадывание, возникающий при кодировании состояний выходными комбинациями. В каждом уравнении переход/возбуждение совсем мало р-термов перехода, поскольку число единиц в столбцах следующих состояний в списке переходов невелико:

$$\begin{aligned}
L1^* &= L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
&\quad + L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
&\quad + L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
L2^* &= L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
L3^* &= L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
L4^* &= L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1' \cdot G2' \cdot G3' \cdot G4') \\
ERR^* &= L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G2 + G3 + G4) \\
&\quad + L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1 + G3 + G4) \\
&\quad + L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1 + G2 + G4) \\
&\quad + L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR' \cdot (G1 + G2 + G3) \\
&\quad + L1' \cdot L2' \cdot L3' \cdot L4' \cdot ERR \cdot (G1 + G2 + G3 + G4).
\end{aligned}$$

Уравнения выхода, естественно, отсутствуют. Самым плохим из приведенных является выражение для ERR^* ; ему требуется 16 термов независимо от того, в каком минимальном виде оно представлено: в виде суммы произведений или в виде произведения сумм.

Табл. 7.19. Список переходов автомата для игры на угадывание при использовании выходных сигналов в качестве переменных состояния

Текущее состояние						Выражение перехода	Следующее состояние					
S	L1	L2	L3	L4			S*	L1*	L2*	L3*	L4*	ERR*
S1	1	0	0	0	0	$G1' \cdot G2' \cdot G3' \cdot G4'$	S2	0	1	0	0	0
S1	1	0	0	0	0	$G1 \cdot G2' \cdot G3' \cdot G4'$	SOK	0	0	0	0	0
S1	1	0	0	0	0	$G2 + G3 + G4$	SERR	0	0	0	0	1
S2	0	1	0	0	0	$G1' \cdot G2' \cdot G3' \cdot G4'$	S3	0	0	1	0	0
S2	0	1	0	0	0	$G1' \cdot G2 \cdot G3' \cdot G4'$	SOK	0	0	0	0	0
S2	0	1	0	0	0	$G1 + G3 + G4$	SERR	0	0	0	0	1
S3	0	0	1	0	0	$G1' \cdot G2' \cdot G3' \cdot G4'$	S4	0	0	0	1	0
S3	0	0	1	0	0	$G1' \cdot G2' \cdot G3 \cdot G4'$	SOK	0	0	0	0	0
S3	0	0	1	0	0	$G1 + G2 + G4$	SERR	0	0	0	0	1
S4	0	0	0	1	0	$G1' \cdot G2' \cdot G3' \cdot G4'$	S1	1	0	0	0	0
S4	0	0	0	1	0	$G1' \cdot G2' \cdot G3' \cdot G4$	SOK	0	0	0	0	0
S4	0	0	0	1	0	$G1 + G2 + G3$	SERR	0	0	0	0	1
SOK	0	0	0	0	0	$G1 + G2 + G3 + G4$	SOK	0	0	0	0	0
SOK	0	0	0	0	0	$G1' \cdot G2' \cdot G3' \cdot G4'$	S1	1	0	0	0	0
SERR	0	0	0	0	1	$G1 + G2 + G3 + G4$	SERR	0	0	0	0	1
SERR	0	0	0	0	1	$G1' \cdot G2' \cdot G3' \cdot G4'$	S1	1	0	0	0	0

В совокупности выведенные здесь соотношения имеют примерно такую же сложность, как и уравнения переходов и уравнения выхода, получаемые из табл. 7.18. И хотя в данном примере кодирование состояний выходными комбинациями не приводит к более простому набору соотношений, все же при проектировании на основе ПЛУ можно сэкономить в стоимости, так как в целом оказываются необходимыми меньшее число макроячеек ПЛУ или выходов.

*7.7.4. Кодирование «безразличных» состояний

Из 32 возможных кодов состояний при пяти переменных в табл. 7.19 используются только шесть. Остальные состояния являются неиспользуемыми, и следующим за ними состоянием будет состояние 00000, если автомат построен по уравнениям, приведенным в предыдущем разделе. Аккуратное использование соображений «безразличия» при отнесении кодов к текущим состояниям позволяет иначе распределить неиспользуемые состояния, нежели это было сделано нами в предыдущем разделе.

В табл. 7.20 представлено одно из таких соответствий в автомате для игры на угадывание при кодировании состояний выходными комбинациями, о котором говорилось. В этом примере текущему состоянию с данным именем соответствует несколько возможных комбинаций переменных состояния (например: 10111 = S1, 00101 = S3). Однако *следующему состоянию* присваивается одна вполне определенная комбинация, как и в предыдущем разделе. В результате мы получаем список переходов, приведенный в табл. 7.21.

Табл. 7.20. Кодирование текущих состояний в автомате для игры на угадывание при безразличном отношении к некоторым комбинациям выходных сигналов

Состояние	L1	L2	L3	L4	ERR
S1	1	x	x	x	x
S2	0	1	x	x	x
S3	0	0	1	x	x
S4	0	0	0	1	x
SOK	0	0	0	0	0
SERR	0	0	0	0	1

При таком подходе каждое неиспользуемое текущее состояние подобно ближайшему «нормальному» состоянию; эту идею иллюстрирует рис. 7.67. Ничего плохого не произойдет с автоматом, если он нечаянно попадет в неиспользуемое состояние: он перейдет в «нормальное состояние». Вместе с тем, этот подход приводит к дальнейшему упрощению логики возбуждения и логики выхода благодаря введению символов безразличия в список переходов. При записи р-терма перехода, соответствующего данной строке, те нерелевантные состояния, значения которых в текущем состоянии отмечены как безразличные, опускаются; например,

$$\begin{aligned}
 \text{ERR}^* = & L1 \cdot (G2 + G3 + G4) \\
 & + L1' \cdot L2 \cdot (G1 + G3 + G4) \\
 & + L1' \cdot L2' \cdot L3 \cdot (G1 + G2 + G4) \\
 & + L1' \cdot L2' \cdot L3' \cdot L4 \cdot (G1 + G2 + G3) \\
 & + L1' \cdot L2' \cdot L3' \cdot L4' \cdot \text{ERR} \cdot (G1 + G2 + G3 + G4).
 \end{aligned}$$

Сравнивая это выражение с выражением, полученным для ERR* в предыдущем разделе, мы видим, что при представлении этого состояния в виде суммы произведений оно по-прежнему содержит 16 термов. Однако в виде минимального произведения

сумму требуется всего пять термов, благодаря чему при реализации автомата на основе ПЛУ оказывается более удобным формировать дополнение к ERR*.

Табл. 7.21. Список переходов автомата для игры на угадывание при безразличном отношении к некоторым комбинациям выходных сигналов

Текущее состояние						Выражение перехода	Следующее состояние					
S	L1	L2	L3	L4	ERR		S*	L1*	L2*	L3*	L4*	ERR*
S1	1	x	x	x	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	S2	0	1	0	0	0
S1	1	x	x	x	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	SOK	0	0	0	0	0
S1	1	x	x	x	x	$G2 + G3 + G4$	SEPR	0	0	0	0	1
S2	0	1	x	x	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	S3	0	0	1	0	0
S2	0	1	x	x	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	SOK	0	0	0	0	0
S2	0	1	x	x	x	$G1 - G3 + G4$	SEPR	0	0	0	0	1
S3	0	0	1	x	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	S4	0	0	0	1	0
S3	0	0	1	x	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	SOK	0	0	0	0	0
S3	0	0	1	x	x	$G1 - G2 + G4$	SEPR	0	0	0	0	1
S4	0	0	0	1	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	S1	1	0	0	0	0
S4	0	0	0	1	x	$G1' \cdot G2' \cdot G3' \cdot G4'$	SOK	0	0	0	0	0
S4	0	0	0	1	x	$G1 + G2 + G3$	SEPR	0	0	0	0	1
SOK	0	0	0	0	0	$G1 + G2 + G3 + G4$	SOK	0	0	0	0	0
SOK	0	0	0	0	0	$G1' \cdot G2' \cdot G3' \cdot G4'$	S1	1	0	0	0	0
SEPR	0	0	0	0	1	$G1 + G2 + G3 + G4$	SEPR	0	0	0	0	1
SEPR	0	0	0	0	1	$G1' \cdot G2' \cdot G3' \cdot G4'$	S1	1	0	0	0	0

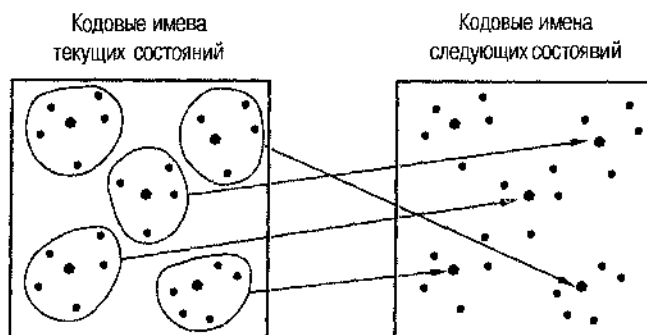


Рис. 7.67. Кодирование состояний при безразличном отношении к некоторым комбинациям выходных сигналов

*7.8. Разбиение конечных автоматов на блоки

Точно так же, как обстоит дело с большими процедурами и подпрограммами при программировании на языках высокого уровня, большие конечные автоматы трудно удерживать в сознании, проектировать и отлаживать. Поэтому, сталкиваясь с задачей, в которой речь идет о большом конечном автомате, разработчики цифровой техники часто ищут возможности решить проблему с помощью совокупности меньших по размерам конечных автоматов.

Существует развитая теория *разбиения конечных автоматов на блоки* (*state-machine decomposition*), позволяющая проанализировать любой заданный как единое целое конечный автомат на предмет возможности реализации его в виде совокупности меньших конечных автоматов. Однако теория разбиения на блоки не так уж полезна для тех, кто хотел бы вообще избежать проектирования больших конечных автоматов. Опытный разработчик скорее постарается представить требуемую конструкцию в виде упорядоченной иерархической структуры, в которой назначение и функции подавтоматов были бы очевидными, так что отпала бы необходимость составлять таблицу состояний для эквивалентного большого автомата в целом.

Простейший и чаще всего используемый тип разбиения на блоки представлен на рис. 7.68. *Главный автомат* (*main machine*) с основными входами и выходами исполняет управляющий алгоритм верхнего уровня. *Подавтоматы* (*submachines*) под контролем главного автомата выполняют шаги нижнего уровня и могут, в частности, обрабатывать сигналы на отдельных входах и выходах из числа основных.

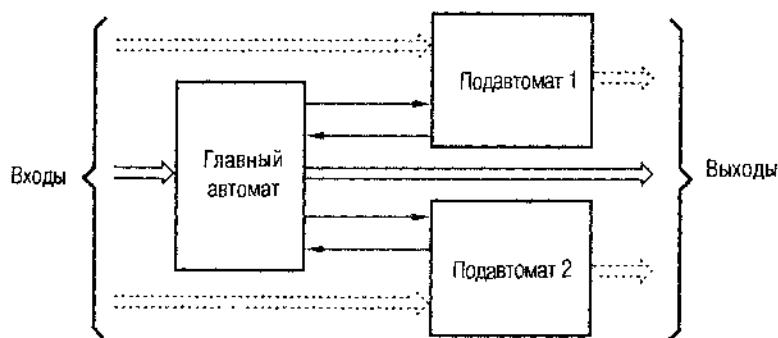


Рис. 7.68. Типичная иерархическая структура конечного автомата.

Роль подавтомата очень часто играет счетчик. Главный автомат запускает счетчик, когда нужно, чтобы автомат оставался в некотором частном состоянии в течение n периодов тактового сигнала; на n -м такте счетчик выдает сигнал DONE. Главный автомат спроектирован так, что он ждет, пребывая в одном и том же состоянии, пока не поступит сигнал DONE. Для этого главному автомату нужны дополнительные выход и вход (START и DONE), но зато экономится $n - 1$ состояние.

ВОТ УЖ СОВСЕМ ПЛОХАЯ ШУТКА

Заметьте, что заглавие этого параграфа не имеет ничего общего со «скрытыми триггерами», обнаруживаемыми в некоторых ПЛУ.

В качестве примера реализации этих идей осуществим разбиение на блоки конечного автомата для игры на угадывание из параграфа 7.7. Поскольку в исходной игре на угадывание цикл зажигания лампочек повторяется абсолютно регулярно, при частоте тактового сигнала 4 Гц легко научиться побеждать, попрактиковавшись с минутой. Чтобы сделать игру более интересной, можно удвоить или утроить частоту тактового сигнала, но позволить лампочкам оставаться в любом из состояний в течение случайного времени. Вот тогда игроку на самом деле надо будет угадывать, останется ли лампочка зажженной достаточно долго, чтобы нажать соответствующую кнопку.

Блок-схема усовершенствованного автомата для игры на угадывание показана на рис. 7.69. Главный автомат — это, в основном, тот же автомат, что и раньше, за исключением того, что он продвигается от одной конфигурации лампочек к следующей только при получении сигнала на входе разрешения EN, как это следует из диаграммы состояний, приведенной на рис. 7.70. Разрешающий входной сигнал поступает от регистра сдвига с линейной обратной связью (linear feedback shift register, LFSR), который является генератором псевдослучайной последовательности.

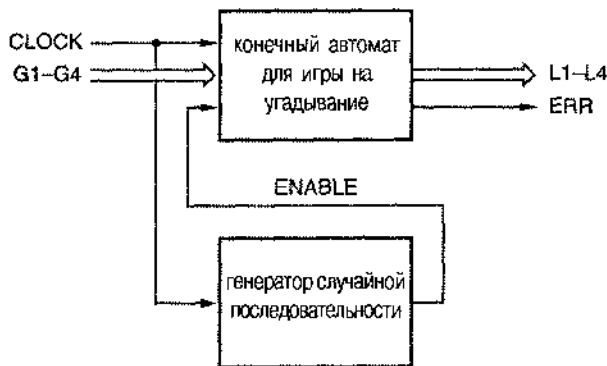


Рис. 7.69. Блок-схема автомата для игры на угадывание со случайной задержкой (CLOCK — тактовый сигнал, ENABLE — сигнал разрешения)

Другой явный кандидат на декомпозицию — это конечный автомат, реализующий умножение двоичных чисел по алгоритму сдвига и суммирования или деление двоичных чисел по алгоритму сдвига и вычитания. Чтобы выполнить операцию с n -разрядными числами по одному из этих алгоритмов, необходимы шаг инициализации, n шагов вычислений и, возможно, несколько шагов очистки. У главного автомата, действующего по такому алгоритму, должны быть состояния инициализации, основного вычисления и очистки, а в качестве подавтомата, задающего число выполняемых шагов при основном вычислении, можно применить счетчик с модулем счета, равным n .

ХИТРАЯ СХЕМА

Схема регистра сдвига с линейной обратной связью рассматривается в разделе 8.5.8. В схеме на рис. 7.69 сигнал с выхода младшего разряда n -разрядного регистра используется в качестве сигнала разрешения. Таким образом, длительность интервала времени, в течение которого горит данная лампочка, зависит от последовательности нулей и единиц, возникающих на выходе регистра.

В лучшем, с точки зрения игрока, случае в регистре хранится двоичное слово $10 \dots 00$, при этом лампочка будет гореть в течение $n-1$ тактов, поскольку именно столько времени нужно, чтобы единица сдвинулась из старшего разряда регистра в младший. В худшем случае регистр содержит $11 \dots 11$, и сдвиг в конфигурации горящих и потушенных лампочек будет происходить n тактов подряд. В других случаях время, в течение которого автомат будет оставаться в одном и том же состоянии, зависит от числа нулей в регистре сдвига, следующих один за другим, начиная с младшего разряда.

Наступаете того или иного из этих случаев можно предсказать, если только игрок помнит весь цикл работы регистра сдвига ($2^n - 1$ состояний) или умеет быстро считать в уме по правилам арифметики в полях Галуа. Очевидно, что веселее всего, когда n имеет большое значение ($n \geq 16$).

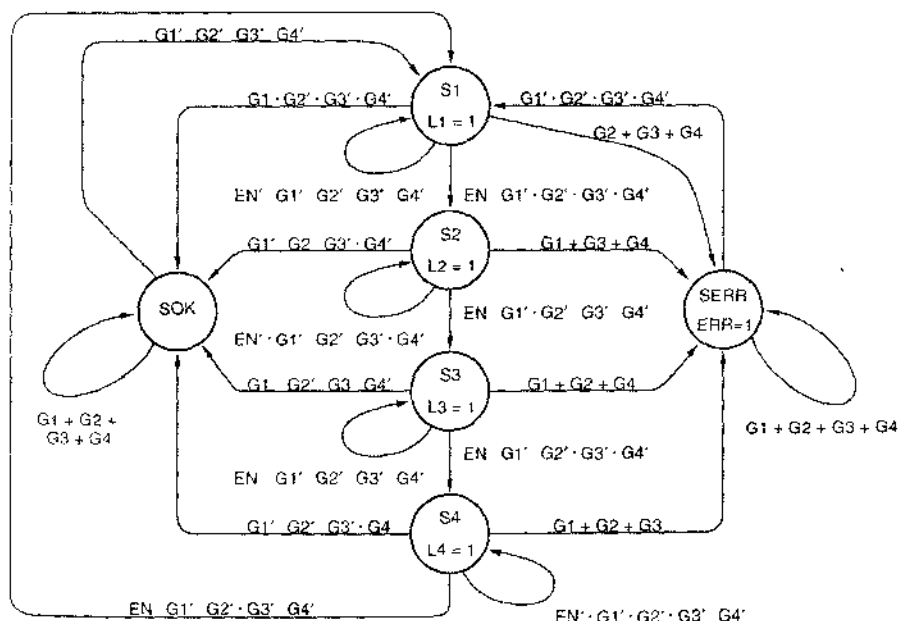


Рис. 7.70. Диаграмма состояний автомата для игры на угадывание с входом разрешения

*7.9. Последовательностные охемы с обратной связью

Простой элемент с двумя устойчивыми состояниями и все защелки и триггеры, рассмотренные нами в этой главе, являются последовательностными схемами с обратной связью. В каждой из них имеется, как минимум, одна петля обратной связи, обеспечивающая сколь угодно долгое хранение 0 или 1, не считая тех моментов, когда происходит переход из одного состояния в другое. Петли обратной связи являются элементами памяти; их поведение зависит как от текущих значений входных сигналов, так и от той величины, которая хранится в этой петле.

*7.9.1. Анализ

Последовательностные схемы с обратной связью являются самым распространенным примером *схем классического образца (fundamental-mode circuits)*. В таких схемах в нормальном режиме не допускается одновременное изменение сигналов на входах. При их анализе предполагается, что входные сигналы изменяются поодиночке, и интервал времени между соседними изменениями достаточно для того, чтобы схема успевала приходиться в установившееся новое внутреннее состояние. Эти схемы классического образца отличаются от тактируемых схем, у которых сигналы на нескольких входах могут изменяться практически в любое время, не влияя на состояние, и лишь в момент времени, задаваемый тактовым сигналом, значения всех входных сигналов приписываются во внимание и в соответствии с ними состояние схемы изменяется.

Подобно тактируемым синхронным конечным автоматам, структуру последовательностных схем с обратной связью можно представить в виде автоматов Мили и Мура, как показано на рис. 7.71. У схемы с n цепями обратной связи имеется n двоичных переменных состояний и 2^n состояний.

ВОЗДЕРЖИВАЙТЕСЬ ОТ ВВЕДЕНИЯ СОБСТВЕННОЙ ОБРАТНОЙ СВЯЗИ

Разработчик цифровой техники лишь изредка сталкивается с ситуацией, когда ему самому необходимо проанализировать или сконструировать последовательностную схему с обратной связью. В большинстве случаев такими схемами оказываются триггеры и защелки, используемые в качестве составных блоков при проектировании последовательностных схем большего размера. Их внутреннее устройство и технические характеристики бывают известны по справочным данным, которые сообщает производитель ИС.

Даже разработчик специализированной ИС обычно не собирает триггеры и защелки из отдельных вентилях, поскольку эти элементы имеются в «библиотеке» схем, реализующих стандартные функции по той технологии, которая применена в данной ИС. Но все же вам, наверно, будет любопытно узнать, как «на самом деле» функционируют триггеры и защелки; в этом параграфе рассказывается о том, как проводится анализ работы таких схем.

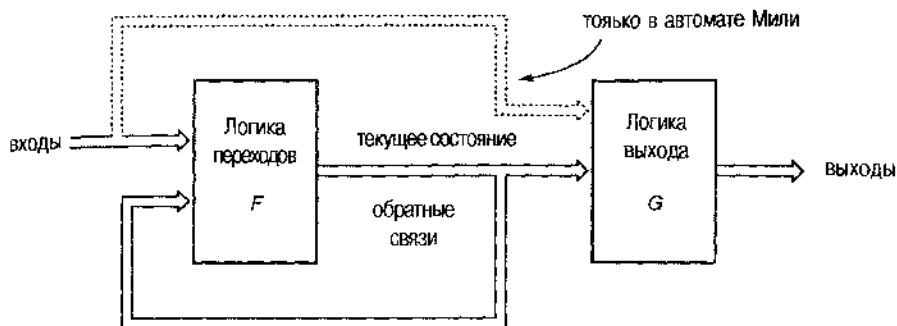


Рис. 7.71. Структура последовательностных схем с обратной связью в виде автоматов Милли и Мура

Анализируя работу последовательностной схемы с обратной связью, мы должны разорвать цепи обратной связи, показанные на рис. 7.71, для того чтобы иметь возможность предсказывать следующие значения, запоминаемые благодаря каждой из этих цепей, в виде функций от сигналов на входах схемы и текущих значений, сохраняемых во всех петлях обратной связи. На рис. 7.72 показано, как это сделать в случае D-защелки на вентилях И-НЕ с единственной петлей обратной связи. Мы мысленно разрываем обратную связь, вставляя в разрыв фиктивный буфер, как указано на рисунке. Сигнал Y на выходе буфера является единственной переменной состояния в этом примере.

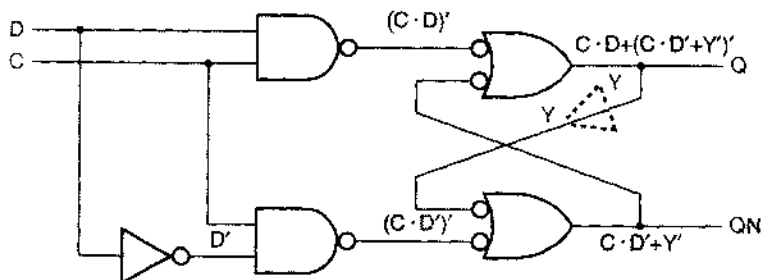


Рис. 7.72. Анализ обратной связи в D-защелке

Давайте предположим, что задержка распространения в фиктивном буфере равна 10 пс (с тем же успехом могло быть любое ненулевое число), а у всех других компонентов схемы задержка равна нулю. Если нам известны текущее состояние схемы Y и входные сигналы D и C , то мы можем предсказать значение, которое переменная Y будет иметь через 10 нс. Следующее значение Y обозначим Y^* ; оно является комбинационной функцией текущего состояния и сигналов на входах. Таким образом, глядя на принципиальную схему, мы можем написать следующее уравнение возбуждения (excitation equation) для Y^* :

$$\begin{aligned} Y^* &= (C \cdot D) + (C \cdot D' + Y)' \\ &= C \cdot D + C' \cdot Y + D \cdot Y. \end{aligned}$$

Теперь состояние петли обратной связи (и самой схемы) можно представить в виде функции текущего состояния и значений входных сигналов, перечислив следующие состояния в *таблице переходов* (*transition table*), приведенной на рис. 7.73. Каждый элемент в этой таблице является значением сигнала на выходе фиктивного буфера через 10 пс (или спустя любое другое время задержки, которое мы предположим) после того, как поступило соответствующее состояние и установились значения сигналов на входах.

Y	C D			
	00	01	11	10
0	0	0	1	0
1	1	1	1	0
Y*				

Рис. 7.73. Таблица переходов для D-защелки, приведенной на рис. 7.72

В таблице переходов на каждую возможную комбинацию переменных состояния приходится по одной строке, так что у схемы с n целями обратной связи таблица содержит 2^n строк. На каждую возможную комбинацию входных сигналов приходится по одному столбцу, поэтому у схемы с m входами таблица переходов содержит 2^m столбцов.

По определению, у схемы классического образца типа последовательностной схемы с обратной связью нет входа для подачи тактового сигнала, которым задавались бы моменты фиксации значений входных сигналов. Поэтому можно считать, что в такой схеме вычисление текущего состояния и учет значений входных сигналов происходит *непрерывно* (или, если хотите, каждые 10 пс). В результате каждого вычисления схема переходит в следующее состояние, указываемое таблицей переходов. Большую часть времени следующим оказывается то же состояние, каким является текущее состояние, в этом и состоит сущность работы по классическому образцу (*the essence of fundamental mode operation*). Дадим ряд определений, которые помогут нам изучить поведение таких схем подробнее.

Состояние в целом (*total state*) схемы классического образца — это некоторая частная комбинация *внутреннего состояния* (*internal state*), то есть совокупности значений, запомненных в копурах обратной связи, и *состояния входа* (*input state*), то есть совокупности значений сигналов на входах схемы. *Устойчивое состояние в целом* (*stable total state*) — это такая комбинация внутреннего состояния и состояния входа, что следующее внутреннее состояние, предсказываемое таблицей переходов, является точно таким же, как и текущее внутреннее состояние. Если следующее внутреннее состояние отличается от текущего внутреннего состояния, то комбинация текущего внутреннего состояния и состояния входа является *неустойчивым состоянием в целом* (*unstable total state*). Таблицу переходов D-защелки можно преобразовать в *таблицу состояний* (*state table*), представленную на рис. 7.74, назвав состояния S0 и S1 и обведя кружками устойчивые состояния в целом.

ТОЛЬКО ОДНА ОБРАТНАЯ СВЯЗЬ

Схема на рис. 7.72 изображена так, как если бы в ней имелись две петли обратной связи, но это не так. Действительно, когда петля обратной связи разрывается, как показано на рисунке, никаких обратных связей в схеме не остается, поскольку теперь каждый сигнал можно представить в виде комбинационной функции от других сигналов, но не от него самого.

S	C D			
	00	01	11	10
S0	S0	S0	S1	S0
S1	S1	S1	S1	S0
S ⁺				

Рис. 7.74. Таблица состояний для D-защелки, изображенной на рис. 7.72, с обведенными кружками устойчивыми состояниями в целом

Чтобы проанализировать работу схемы, нам необходимо определить также, как ведут себя выходные сигналы в зависимости от внутреннего состояния и сигналов, действующих на входах. В данном примере два выхода, поэтому выходные сигналы определяются двумя уравнениями выхода (*output equations*):

$$Q = C \cdot D + C' \cdot Y + D \cdot Y$$

$$QN = C \cdot D' + Y'$$

Обратите внимание на то, что Q и QN — это *выходы*, но не переменные состояния, хотя у схемы два выхода, сигналы на которых теоретически могут образовывать четыре комбинации значений, имеется только одна переменная состояния Y и, следовательно, только два состояния.

Значения выходных сигналов, предсказываемые уравнениями для Q и QN, можно включить в объединенную таблицу состояний и выхода, полностью описывающую работу схемы, как это сделано на рис. 7.75. Хотя обычно Q и QN дополняют друг друга, они могут все же иметь одинаковые значения (1) в течение короткого времени при переходе из состояния S0 в состояние S1, как это видно из записи в таблице в столбце для C D = 11.

S	C D			
	00	01	11	10
S0	S0, 01	S0, 01	S1, 11	S0, 01
S1	S1, 10	S1, 10	S1, 10	S0, 01
S*, Q QN				

Рис. 7.76. Таблица состояний и выхода для D-защелки

Теперь по таблице состояний и выхода мы можем предвидеть поведение схемы. Прежде всего отметим, что столбцы таблицы состояний расположены в том порядке, в каком входные сигналы перебираются в карте Карно, так что только один бит в совокупности входных сигналов меняется при последовательном переходе от столбца к столбцу. Такое расположение поможет нам при анализе, поскольку предполагается, что каждый раз меняется только один входной сигнал и что схема всегда успевает достичь устойчивого состояния в целом до того, как произойдет изменение другого входного сигнала.

В любой момент времени схема находится в каком-то внутреннем состоянии и на ее входах действуют какие-то сигналы; мы называем эту комбинацию состоянием схемы в целом. Давайте начнем с устойчивого состояния в целом "S0/00" (S = S0, CD = 00), как показано на рис. 7.76. Предположим, что сигнал D изменяется и становится равным 1. Состояние в целом смещается на одну позицию вправо и схема попадает в новое устойчивое состояние в целом. Теперь значение сигнала на входе D другое, но внутреннее состояние и выходные сигналы остаются теми же, что и раньше. Далее, пусть изменяется сигнал C и принимает единичное значение. Состояние в целом смещается еще на одну позицию вправо и оказывается неустойчивым состоянием S0/11. Запись в таблице указывает, что следующим внутренним состоянием схемы будет состояние S1, так что состояние в целом смещается на одну позицию вниз и становится равным S1/11. Смотрим, каким является следующее состояние, указываемое в этом месте таблицы, и убеждаемся, что схема снова достигла устойчивого состояния в целом. Подобным образом можно проследить поведение схемы при любой желаемой последовательности одиночных изменений входных сигналов.

S	C D			
	00	01	11	10
S0	(S0, 01)	(S0, 01)	S1, 11	(S0, 01)
S1	(S1, 10)	(S1, 10)	(S1, 10)	S0, 01

S*, Q QN

Рис. 7.76. Анализ поведения D-защелки при нескольких переходах

Теперь мы можем вернуться к вопросу об одновременном изменении входных сигналов. Несмотря на то, что на практике может происходить «почти одновременное» изменение входных сигналов, мы должны предполагать при анализе поведения последовательностных схем, что ничего не случается одновременно. В пользу невозможности одновременных событий говорит разброс самих задержек в компонентах схемы, которые зависят от напряжения питания, температуры и значений параметров, выдерживавшихся при изготовлении. Как следствие этого, наблюдаемое нами «одновременное» изменение n входных сигналов в действительности, с точки зрения работы схемы, происходит в каком-то определенном порядке, и при этом существует $n!$ различных вариантов.

Рассмотрим, например, поведение D-зашелки, изображенное на рис. 7.77. Предположим, что сначала схема находится в устойчивом состоянии в целом $S1/1$, и пусть оба сигнала C и D «одновременно» принимают нулевое значение. На самом деле схема ведет себя так, как если бы один из этих сигналов перешел в 0 первым. Предположим, что первым изменился сигнал C . Тогда последовательность из двух направленных влево стрелок в таблице показывает нам, что схема перейдет в устойчивое состояние в целом $S1/00$. Однако в случае, если первым изменяется сигнал D , другая последовательность стрелок показывает, что новым устойчивым состоянием в целом будет $S0/00$. Таким образом, конечное состояние схемы оказывается непредсказуемым; секрет заключается в том, что в действительности при одновременном переходе в 0 сигналов C и D петля обратной связи становится метастабильной. Интервал времени, в пределах которого действует представление об одновременности, определяется временем установления и временем удержания данной D-зашелки.

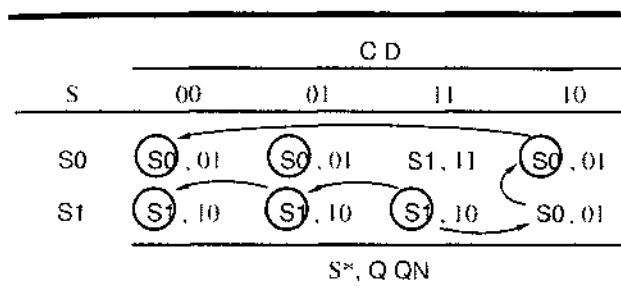


Рис. 7.77. Изменение нескольких сигналов на входах D-зашелки

Одновременное изменение входных сигналов не всегда вызывает непредсказуемое поведение. Но чтобы установить это, необходимо проанализировать все случаи изменения сигналов в любом возможном порядке; если все они приводят к одному и тому же результату, то сигналы на выходе схемы предсказуемы. Если, например, D-зашелка сначала находится в состоянии в целом $S0/00$ и сигналы C и D одновременно изменяются от 0 до 1, то конечным всегда будет состояние в целом $S1/11$.

7.9.2. Анализ схем с несколькими цепями обратной связи

В схеме с несколькими цепями обратной связи необходимо разорвать все петли, вставляя в каждую из них свой фиктивный буфер и вводя свою переменную состояния. Разрыв петель обратной связи в данной схеме можно осуществить многими способами; математики называют их *сечениями* (*cut set*). Как узнать, какое из них лучше? Ответ таков: хорошим является любое *минимальное сечение* (*minimal cut set*), то есть сечение с минимальным числом разрезов. Математики могут предложить алгоритм поиска минимального сечения, но если вы разрабатываете небольшую цифровую схему, то найти подходящие места в схеме можно невооруженным глазом.

Различные сечения схемы приводят к различным уравнениям возбуждения, таблицам переходов и таблицам состояние/выход. Однако существует взаимно однозначное соответствие между устойчивыми состояниями в целом, полученными при одном минимальном сечении данной схемы, и устойчивыми состояниями в целом, полученными при другом минимальном сечении той же самой схемы. Другими словами, таблицы состояние/выход, полученные для разных минимальных сечений, демонстрируют одно и то же поведение вход/выход, разве что различными будут имена и коды состояний.

Если вы воспользовались для анализа последовательностной схемы с обратными связями сечением с числом разрезов больше минимального, то результирующая таблица состояние/выход все же будет правильно описывать поведение схемы. Однако у нее будет в 2^m раз больше состояний, чем это необходимо, где m — число лишних разрезов. Для сведения этой большой таблицы к надлежащим размерам можно воспользоваться формальной процедурой минимизации числа состояний, но гораздо лучше сразу взять минимальное сечение.

Хорошим примером последовательностной схемы с несколькими обратными связями является серийно выпускаемый D-триггер TTL-семейств, переключающийся по положительному фронту, схема которого представлена на рис. 7.20. На рис. 7.78 эта схема воспроизведена в упрощенном виде в предположении, что на входах PR_L и CLR_L исходной схемы активные уровни сигналов никогда не возникают; на рис. 7.78 показаны также фиктивные буферы, вставленные в места разрыва трех петель обратной связи. Из-за наличия трех обратных связей здесь имеется восемь возможных состояний в отличие от схемы с двумя обратными связями и с минимальным числом состояний, равным четырем, которая приведена на рис. 7.19. Мы еще вернемся к этому загадочному отличию позднее.

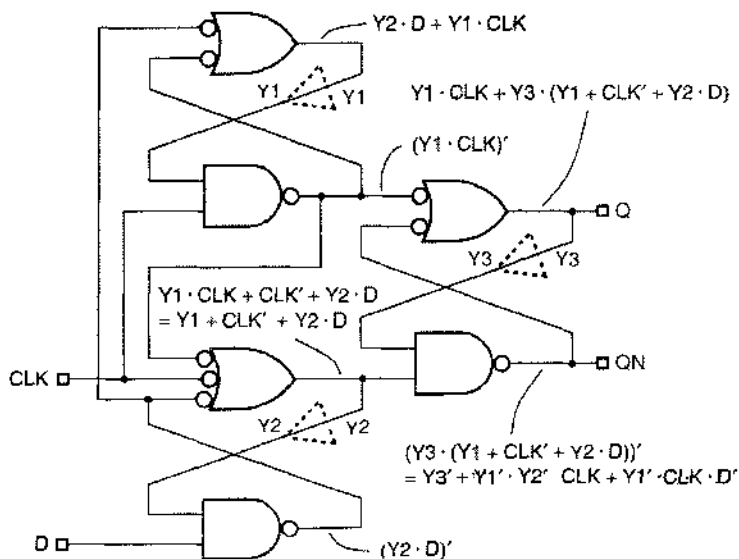


Рис. 7.78. Упрощенная схема D-триггера, переключающегося по положительному фронту

Из принципиальной схемы на рис. 7.78 можно вывести следующие уравнения возбуждения и уравнения выхода:

$$Y1^* = Y2 \cdot D + Y1 \cdot CLK$$

$$Y2^* = Y1 + CLK' + Y2 \cdot D$$

$$Y3^* = Y1 \cdot CLK + Y1 \cdot Y3 + Y3 \cdot CLK' + Y2 \cdot Y3 \cdot D$$

$$Q = Y1 \cdot CLK + Y1 \cdot Y3 + Y3 \cdot CLK' + Y2 \cdot Y3 \cdot D$$

$$QN = Y3' + Y1' \cdot Y2' \cdot CLK + Y1' \cdot CLK \cdot D'$$

Соответствующая таблица переходов приведена на рис. 7.79, где устойчивые состояния в целом обведены кружками. Но прежде чем продвинуться дальше, нам необходимо ввести понятие «гонок».

Y1	Y2	Y3	CLK D			
			00	01	11	10
000	010	010	000	000	000	000
001	011	011	000	000	000	000
010	010	110	110	110	000	000
011	011	111	111	111	000	000
100	010	010	111	111	111	111
101	011	011	111	111	111	111
110	010	110	111	111	111	111
111	011	111	111	111	111	111

Рис. 7.79. Таблица переходов для D-триггера, изображенного на рис. 7.78

*7.9.3. Гонки

Говорят, что в последовательностной схеме с обратной связью происходит *гонка* (состязание, *race*), когда в результате изменения одного входного сигнала меняется несколько переменных из числа тех, которыми определяется внутреннее состояние. На рис. 7.79, например, гонка имеет место, когда схема находится в устойчивом состоянии в целом 011/00 и сигнал CLK меняет свое значение с 0 на 1. Из таблицы видно, что следующим внутренним состоянием будет 000, то есть изменятся две переменные.

Как говорилось выше, логические сигналы никогда на практике не изменяются «одновременно». Поэтому переход от внутреннего состояния 011 к внутреннему состоянию 000 может происходить либо в последовательности 011 → 001 → 000, либо в последовательности 011 → 010 → 000. На рис. 7.80 показано, как рассматриваемая схема должна переходить из состояния в целом 011/00 в состояние в целом 000/10 при изменении сигнала CLK от 0 до 1. Как видим, но пути схема может на короткое время оказаться в состоянии в целом 001/10 или в состоянии в

целом 010/10. В этом нет ничего страшного, поскольку в обоих случаях следующим за временно посещаемыми состояниями является одно и то же внутреннее состояние 000; это означает, что в те моменты, когда схема пребывает во временных состояниях, логика возбуждения продолжает вести петли обратной связи в направлении к одному и тому же устойчивому состоянию в целом 000/10. Поскольку конечное состояние не зависит от порядка, в котором изменяются переменные состояния, такая гонка называется *некритической (noncritical race)*.

Y1 Y2 Y3	CLK D			
	00	01	11	10
000	010	010	000	000
001	011	011	000	000
010	010	110	110	000
011	011	111	111	000

Y1~ Y2~ Y3~

Рис. 7.80. Часть таблицы переходов D-триггера, демонстрирующая некритическую гонку

Представим себе теперь, что элемент таблицы переходов, относящийся к состоянию в целом 010/10, изменился и указывает в качестве следующего состояния 110, как показано на рис. 7.81; рассмотрим снова только что проанализированный случай. В результате перехода сигнала CLK к значению 1 схема, находившаяся в устойчивом состоянии в целом 011/00, может, в конце концов, оказаться во внутреннем состоянии 000 или во внутреннем состоянии 111 в зависимости от того в каком порядке и как быстро изменялись значения внутренних переменных. Такой случай называется *критической гонкой (critical race)*.

Y1 Y2 Y3	CLK D			
	00	01	11	10
000	010	010	000	000
001	011	011	000	000
010	010	110	110	110
011	011	111	111	000
100	010	010	111	111
101	011	011	111	111
110	010	110	111	111
111	011	111	111	111

Y1~ Y2~ Y3~

Рис. 7.81. Таблица переходов с критической гонкой

ОСТЕРЕГАЙТЕСЬ КРИТИЧЕСКИХ ГОНОК

Проектируя последовательностную схему с обратными связями, вы должны быть уверены, что таблица переходов не содержит критических гонок. В противном случае цепь будет функционировать непредсказуемым образом с совершенно неожиданными переходами в зависимости от таких факторов, как температура, напряжение питания и фаза луны.

***7.9.4. Таблицы состояний и таблицы потока**

Анализ реальной таблицы переходов D-триггера в нашем примере (рис. 7.79) показывает, что в ней нет критических гонок; в ней нет других гонок, кроме не критической гонки, разобранных нами выше. После того как этот факт установлен, нам больше не нужны переменные состояния. Вместо этого комбинации переменных состояния можно называть по именам и для каждой комбинации состояние/вход найти значения выходных сигналов, чтобы составить таблицу состояние/выход типа той, которая приведена на рис. 7.82.

S	CLK D			
	00	01	11	10
S0	S2 . 01	S2 . 01	(S0) . 01	(S0) . 01
S1	S3 . 10	S3 . 10	S0 . 10	S0 . 10
S2	(S2) . 01	S6 . 01	S6 . 01	S0 . 01
S3	(S3) . 10	S7 . 10	S7 . 10	S0 . 01
S4	S2 . 01	S2 . 01	S7 . 11	S7 . 11
S5	S3 . 10	S3 . 10	S7 . 10	S7 . 10
S6	S2 . 01	(S6) . 01	S7 . 11	S7 . 11
S7	S3 . 10	(S7) . 10	(S7) . 10	(S7) . 10
S* . Q QN				

Рис. 7.82. Таблица состояние/выход для D-триггера, изображенного на рис. 7.78

Таблица состояний показывает, что при однократном изменении какого-либо из входных сигналов схема совершает множественные «скачки», прежде чем достигнет нового устойчивого состояния в целом. Если, например, схема находится в состоянии S0/11 и входные сигналы принимают значения 01, то схема сначала переходит в состояние S2 и только затем попадает в устойчивое состояние в целом S6/01. В *таблице потока* (flow table) множественные скачки не принимаются во внимание, а сразу указывается конечное состояние каждой цепочки переходов. В таблице потока исключаются также строки, соответствующие неиспользуемым состояниям, то есть таким состояниям, которые не являются устойчивыми ни при

каких комбинациях входных сигналов. Кроме того, оставляются незаполненными те места, предназначенные для указания следующего состояния, которые, как состояние в целом, не могут быть достигнуты ни из какого устойчивого состояния в целом, как результат изменения лишь одного из сигналов. Составленная по этим правилам таблица потока для D-триггера в нашем примере приведена на рис. 7.83.

S	CLK D			
	00	01	11	10
S0	S2, 01	S6, 01	(S0), 01	(S0), 01
S2	(S2), 01	S6, 01	—, —	S0, 10
S3	(S3), 10	S7, 10	—, —	S0, 01
S6	S2, 01	(S6), 01	S7, 11	—, —
S7	S3, 10	(S7), 10	(S7), 10	(S7), 10
S*, Q QN				

Рис. 7.83. Таблица потока/выхода для D-триггера, изображенного на рис. 7.78

За поведением триггера, переключающегося по фронту, можно проследить по последовательности переходов от состояния к состоянию, как показано на рис. 7.84. Предположим, что вначале триггер находится в состоянии S0/10, то есть в нем записан 0 (поскольку $Q = 0$), сигнал CLK равен 1, сигнал D имеет нулевое значение. Пусть сигнал D изменится и становится равным 1; из таблицы потока следует, что схема сдвинется на одну позицию влево оставаясь в том же самом устойчивом состоянии в целом с теми же значениями сигналов на выходах. Сигнал D может сколько угодно раз менять свое значение с 0 на 1 и обратно, и схема при этом будет только скакать взад-вперед между этими двумя положениями в таблице. Однако в случае, когда сигнал CLK станет равным 0, схема перейдет во внутреннее состояние S2 или S6 в зависимости от значения сигнала D на этот момент времени; по выходные сигналы останутся неизменными. Мы снова можем изменять значение D, делая его равным 0 или 1, сколько нам вздумается, и схема при этом будет лишь перескакивать из состояния S2 в состояние S6 и обратно, а выходные сигналы меняться не будут.

Наконец, наступает решающий момент, когда сигнал CLK становится равным 1. В зависимости от того, в каком состоянии в этот момент находится схема, — в состоянии S2 или в состоянии S6, — она либо вернется в состояние S0 (и выходной сигнал Q останется равным 0), либо перейдет в состояние S7 (и сигнал Q станет равным 1). Аналогичное поведение схемы можно наблюдать, когда в момент действия нарастающего фронта тактового сигнала она находится в состояниях S3 или S7 и выходной сигнал Q изменяет свое значение с 1 на 0.

S	CLK D			
	00	01	11	10
S0	S2, 01	S6, 01	(S0), 01	(S0), 01
S2	(S2), 01	S6, 01	—, —	S0, 10
S3	(S3), 10	S7, 10	—, —	S0, 01
S6	S2, 01	(S6), 01	S7, 11	—, —
S7	S3, 10	(S7), 10	(S7), 10	(S7), 10

S*, Q QN

Рис. 7.84. Таблица поток/выход, демонстрирующая тот факт, что D-триггер переключается по фронту

На рис. 7.19 была приведена схема D-триггера, переключающегося по положительному фронту, с двумя петлями обратной связи и, следовательно, с четырьмя возможными состояниями, тогда как в схеме, которая только что была проанализирована, имеются три петли обратной связи и у нее возможны восемь состояний. Даже после исключения неиспользуемых состояний в таблице потока для схемы на рис. 7.78 остается пять состояний. Можно, однако, показать, воспользовавшись формальной процедурой минимизации числа состояний, что состояния S0 и S2 «совместимы», то есть их можно объединить в одно состояние SB, позволяющее учесть все переходы, относящиеся к обоим исходным состояниям (рис. 7.85). Таким образом, желаемый эффект, в действительности, мог бы быть достигнут с помощью схемы с четырьмя состояниями. Решая задачу 7.66, можно убедиться, что схема на рис. 7.19 ведет себя в соответствии с сокращенной таблицей потока.

S	CLK D			
	00	01	11	10
SB	(SB), 01	S6, 01	(SB), 01	(SB), 01
S3	(S3), 10	S7, 10	—, —	SB, 01
S6	SB, 01	(S6), 01	S7, 11	—, —
S7	S3, 10	(S7), 10	(S7), 10	(S7), 10

S*, Q QN

Рис. 7.85. Сокращенная таблица поток/выход для D-триггера, переключающегося по положительному фронту

***7.9.5. Анализ работы О-триггера в КМОП-исполнении**

В схемах КМОП-триггеров цепи обратной связи обычно содержат логические ключи. Например, на рис. 7.86 приведена схема переключающегося по положительному фронту D-триггера “FD1Q” в интегральных схемах серии LCA500K, представляющих собой решетки КМОП-вентилей. Как только найдены петли обратной связи, работу такого триггера можно проанализировать точно так же, как и работу схемы, собранной из чисто логических элементов. В схеме на рис. 7.86 имеются две петли обратной связи, каждая из которых содержит по паре ключей, управляемых сигналами CLK и CLK', и реализует функцию мультиплексора; уравнения, описывающие функционирование петель обратной связи, имеют вид:

$$Y1^* = CLK' \cdot D' + CLK \cdot Y1$$

$$Y2^* = CLK \cdot Y1' + CLK' \cdot Y2$$

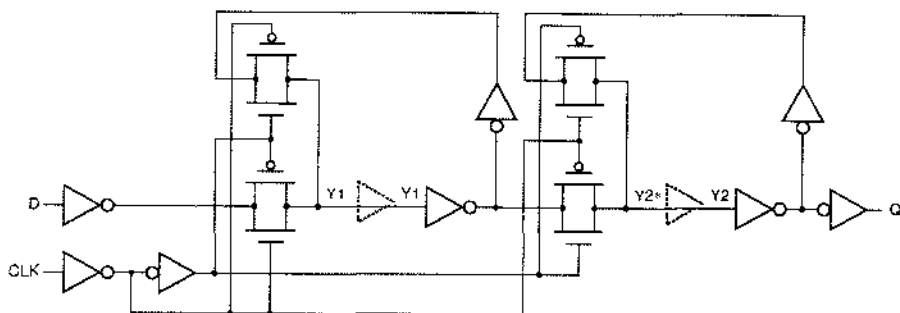


Рис. 7.86. К анализу работы переключающегося по положительному фронту D-триггера, выполненного по КМОП-технологии.

ПРОЕКТИРОВАНИЕ СХЕМ С ОБРАТНОЙ СВЯЗЬЮ

Рассмотренные нами в этом параграфе последовательностные схемы с обратной связью ведут себя вполне пристойно; в этом нет ничего удивительного, поскольку речь шла лишь о защелках и триггерах, успешно используемых на практике годами. Но если вы соберете «произвольную» комбинацию из вентилей и обратных связей, то вовсе не обязательно получится «разумно ведущая себя» последовательностная схема. В редких случаях может вообще получиться не последовательностная схема (см. задачу 7.67). Во многих случаях схема окажется неустойчивой при некоторых комбинациях входных сигналов, а иногда и при любых входных воздействиях (см. задачу 7.71). Таким образом, проектирование последовательностных схем с обратной связью продолжает оставаться чем-то вроде черной магии и осуществляется практически лишь небольшим числом знатоков в области цифровой электроники. Все же в следующем параграфе вводятся основные понятия, которые помогают сделать проектирование менее сложным.

За исключением того, что данные дважды инвертируются при прохождении от входа D до точки Y2* (один раз в уравнении для Y1* и второй раз в уравнении для Y2*), эти равенства сильно упрощают структуру D-триггера, состоящего из ведущей и ведомой защелок (рис. 7.15). Мы не станем здесь выполнять формальный анализ; он выплывает в задаче 7.73. Заметьте, однако, что в этой схеме две нетли обратной связи и поэтому в результирующих таблицах состояний и потока будет, как минимум, четыре состояния.

*7.10. Проектирование последовательностных схем с обратной связью

Иногда бывает полезно построить небольшую последовательностную схему типа специализированной защелки или ловушки импульсов; в этом параграфе будет показано, как это сделать. Может случиться также, что вы собираетесь стать специалистом по проектированию ИС и будете ответственным за разработку высококачественных защелок и триггеров, начиная с нуля. Этот параграф знакомит с основными идеями, которые вам при этом понадобятся. Но для того чтобы делать такую работу хорошо, необходимы значительно более серьезные усилия на стадии обучения, а также опыт и мастерство.

*7.10.1. Защелки

Хотя, как правило, проектирование последовательностных схем с обратной связью представляет собой трудную проблему, некоторые схемы строить довольно легко. Любая схема с одной петлей обратной связи является разновидностью SR- или D-защелки. Структура такой схемы имеет вид, указанный на рис. 7.87, а уравнение возбуждения всегда бывает следующей формы:

$$Q^* = (\text{выпуждающий член}) + (\text{удерживающий член}) \cdot Q.$$

Именно такими являются, например, уравнения возбуждения SR- и D-защелок:

$$Q^* = S + R' \cdot Q$$

$$Q^* = C \cdot D + C' \cdot Q.$$

Соответствующие схемы приведены на рис. 7.88(a) и (b).

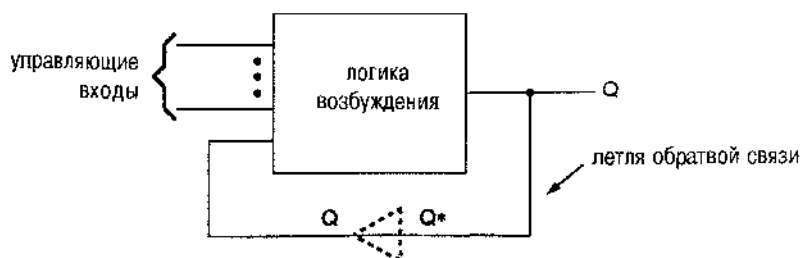


Рис. 7.87. Общая структура защелки

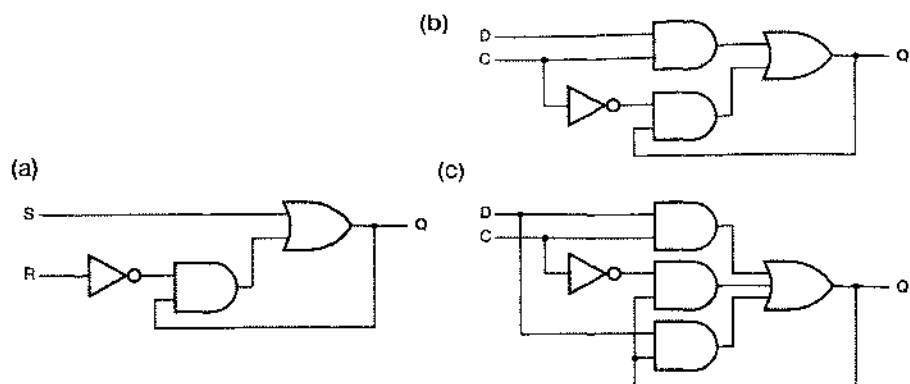


Рис. 7.88. Схемы защелок: (a) SR-защелка; (b) ненадежная D-защелка; (c) D-защелка без источников опасности

В общем случае логика возбуждения в последовательной схеме с обратной связью должна быть свободна от источников опасности (*hazard-free excitation logic*); мы продемонстрируем это на примере. На рис. 7.89(a) показана карта Карно для схемы возбуждения D-защелки, представленный на рис. 7.88(b). Из карты следует, что имеется единичный статический источник опасности, когда сигналы D и Q равны 1 и изменяется сигнал C. К сожалению, нетля обратной связи в этой защелке не может удержать хранимое ею значение, если из-за задержек при переходном процессе возникает короткий паразитный импульс. Рассмотрим, например, что случится, если D и Q равны 1 и C изменяется от 1 до 0; схема должна остаться защелкнутой со значением 1. Однако это будет не так, если только инвертор не является каким-то сверхбыстродействующим: на выходе верхнего вентиля И ноль появится раньше, чем единица появится на выходе нижнего вентиля И, поэтому сигнал на выходе вентиля ИЛИ станет ранним 0 и петля обратной связи сохранит нулевое значение.

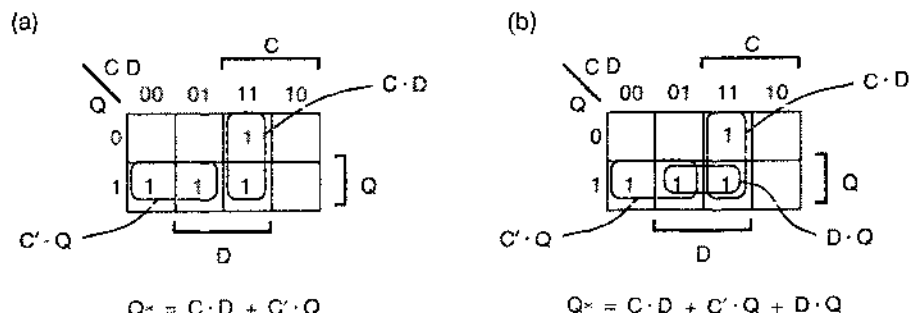


Рис. 7.89. Карты Карно для функций возбуждения D-защелок: (a) в первоначальной схеме с единичным статическим источником опасности; (b) в схеме, где источник опасности исключен

Методами, описанными в параграфе 4.5, источники опасности можно устранить. В случае D-защелки мы просто добавляем в уравнение возбуждения консенсусный терм:

$$Q^* = C \cdot D + C' \cdot Q + D \cdot Q.$$

Соответствующая исправленная схема D-защелки без источников опасности показана на рис. 7.88(с).

Представим себе теперь, что нам нужна специализированная "D"-защелка с тремя входами данных D1–D3, которая запоминала бы 1 только в том случае, когда D1–D3 = 010. Это словесное описание можно преобразовать в уравнение возбуждения, которое по форме подобно уравнению возбуждения простой D-защелки:

$$Q^* = C \cdot (D1' \cdot D2 \cdot D3') + C' \cdot Q.$$

Устраняя источники опасности, получим:

$$Q^* = C \cdot D1' \cdot D2 \cdot D3' + C' \cdot Q + D1' \cdot D2 \cdot D3' \cdot Q.$$

Это уравнение возбуждения без источников опасности можно реализовать на дискретных элементах или в ПЛУ, как будет показано в разделе 8.2.6.

БУРНЫЙ РОСТ ЧИСЛА ТЕРМОВ-ПРОИЗВЕДЕНИЙ

В некоторых случаях необходимость устранения источников опасности может приводить к быстрому росту числа термов-произведений при двухуровневой реализации логики возбуждения. Предположим, например, что нам нужна специализированная защелка с двумя управляющими входами C1 и C2 и тремя входами данных, как в приведенном выше примере. Защелка должна быть «открыта» только в том случае, когда на оба управляющих входа подана 1, и сохраняет единичное значение, если хотя бы один из сигналов на входах данных равен 1. Тогда минимальное уравнение возбуждения имеет вид:

$$\begin{aligned} Q^* &= C1 \cdot C2 \cdot (D1 + D2 + D3) + (C1 \cdot C2)' \cdot Q \\ &= C1 \cdot C2 \cdot D1 + C1 \cdot C2 \cdot D2 + C1 \cdot C2 \cdot D3 + C1' \cdot Q + C2' \cdot Q. \end{aligned}$$

Однако для устранения источников опасности в этом случае потребуется шесть консенсусных термов (см. задачу 7.76).

*7.10.2. Составление таблицы потока для схемы классического образца

Первое, что необходимо сделать при проектировании более сложных, чем защелка, последовательностных схем с обратной связью, это преобразовать словесное описание в таблицу потока. Имея таблицу потока, можно посредством рутинных действий (которые, впрочем, могут потребовать заметных усилий) получить саму схему.

При составлении таблицы потока для последовательностной схемы с обратной связью мы даем каждому состоянию смысловое название, вытекающее из постановки задачи, практически так же, как это делалось нами при проектировании тактируемых конечных автоматов. Однако при составлении таблицы потока

для последовательностной схемы с обратной связью легче оказаться сбитым с толку, поскольку не каждое состояние в целом устойчиво. Поэтому рекомендуемая процедура заключается в составлении *примитивной таблицы потока* (*primitive flow table*), то есть такой таблицы, у которой в каждой строке имеется только одно устойчивое состояние в целом. Если это условие выполнено, то можно показать, что выходной сигнал является функцией только состояния.

В примитивной таблице потока каждое состояние имеет более точный «смысл», нежели это могло бы быть в другом случае, а самой структурой таблицы подчеркивается основной принцип действия схем классического образца: каждый раз может меняться только один из входных сигналов, и интервал времени между изменениями достаточен для того, чтобы в схеме установилось новое устойчивое состояние. В примитивной таблице потока, как правило, есть избыточные состояния, но вслед за тем, как мы ее составили и убедились в ее правильности, к ней можно будет применить рутинную процедуру минимизации числа состояний.

Для демонстрации того, как составляется таблица потока, мы воспользуемся следующим примером, а именно схемой для «отлавливания импульсов»:

Построить последовательностную схему с обратной связью с двумя входами P (pulse, импульс) и R (reset, сброс) и одним выходом Z , сигнал на котором нормально равен 0. Единичный сигнал должен возникать на выходе, когда на входе P происходит переход от 0 к 1; сброс схемы должен осуществляться в тот момент, когда сигнал R становится равным 1. Типичные временные диаграммы приведены на рис. 7.90.

Примитивная таблица потока для такой схемы представлена на рис. 7.91. Сейчас мы пройдем по всему пути составления этой таблицы.

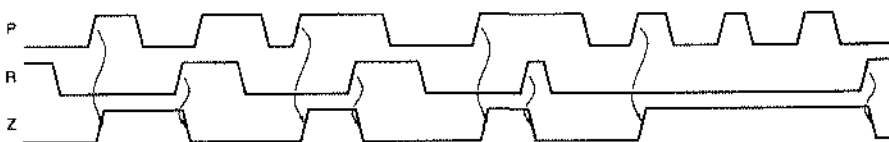


Рис. 7.90. Типичные временные диаграммы для ловушки импульсов

Предположим, что вначале ловушка импульсов не занята, а сигналы P и R равны 0; это состояние $IDLE$ с $Z = 0$. Находясь в этом состоянии, схема могла бы оставаться в нем с приходом сигнала сброса ($R = 1$), но поскольку мы хотим составить примитивную таблицу потока, мы вводим новое состояние $RES1$, чтобы в одной строке не было двух устойчивых состояний в целом. С другой стороны, если схема находится в состоянии $IDLE$ и приходит импульс ($P = 1$), мы определенно хотим перейти в другое состояние, которое мы назовем $PLS1$, поскольку импульс пойман и мы должны выработать на выходе единичный сигнал. Комбинацию входных сигналов 11, когда схема находится в состоянии $IDLE$, мы не рассматриваем в силу предположения о том, что наша схема является схемой классического образца и каждый раз может измениться только один входной сигнал; мы предполагаем, что схема всегда успевает перейти из состояния $IDLE$ в другое устойчивое состояние до того, как на входе возникнет комбинация 11.

Значение	S	P R				Z
		00	01	11	10	
Состояние бездействия, ожидание импульса	IDLE	IDLE	RES1	—	PLS1	0
Сброс, импульса нет	RES1	IDLE	RES1	RES2	—	0
Поступил импульс, на выходе 1	PLS1	PLS2	—	RES2	PLS1	1
Сброс, поступил импульс	RES2	—	RES1	RES2	PLSN	0
Импульс закончился, на выходе 1	PLS2	PLS2	RES1	—	PLS1	1
Импульс присутствует, но на выходе 0	PLSN	IDLE	—	RES2	PLSN	0
S*						

Рис. 7.91. Примитивная таблица потока для ловушки импульсов

Заполним теперь места в таблице, предназначенные для указания состояний, следующих за вновь введенным состоянием RES1. Когда сигнал сброса закончится, мы можем вернуться обратно в состояние IDLE. Если возникнет импульс, то мы должны оставаться в состоянии «сброса», так как, согласно временным диаграммам, переход сигнала P из 0 в 1 при R = 1 игнорируется. Снова, чтобы таблица потока была примитивной, мы должны ввести для этого случая новое состояние RES2.

Теперь, когда у нас в каждом столбце имеется по одному устойчивому состоянию в целом, стоит рассмотреть возможные переходы для уже имеющихся состояний, а не вводить все новые и новые состояния. В самом деле, если схема находится в устойчивом состоянии в целом PLS1/10 и сигнал R становится равным 1, то возможен переход в состояние RES2, в котором удовлетворяется требование равенства нулю выходного сигнала. С другой стороны, куда должна перейти схема, если сигнал P станет равным 0? В столбце 00 уже есть устойчивое состояние в целом IDLE, но было бы неправильно сказать, что схема должна перейти именно в это состояние. В состоянии PLS1 импульс был принят, а сигнала сброса не было видно, поэтому даже с окончанием импульса схема должна оставаться в таком состоянии, при котором все еще Z = 1. Следовательно, для этого случая мы должны ввести новое состояние PLS2.

Из состояния RES2 схема беспрепятственно может перейти в состояние RES1, когда импульс заканчивается. Однако в случае, когда заканчивается сигнал сброса, нам нужно быть осторожнее, как это следует из временных диаграмм. Коль скоро мы уже пропустили переход от 0 к 1 в сигнале P, мы не можем перейти в состояние PLS1, так как в этом случае выходной сигнал оставался бы равным 1. Вместо этого мы вводим новое состояние PLSN с 0 на выходе.

Мы можем, наконец, заполнить строки таблицы, относящиеся к состояниям PLS2 и PLSN, не вводя никаких новых состояний. Заметьте, что из состояния PLS2 схема может перескакивать в состояние PLS1 и обратно, поддерживая на выходе сигнал, равный 1, если приходит последовательность импульсов, а сигнал сброса не возникает.

*7.10.3. Минимизация таблицы потока

Как уже говорилось, в примитивной таблице потока обычно бывает больше состояний, чем это необходимо. Однако существует формальная процедура минимизации числа состояний в таблице потока, описываемая в литературе. Эта процедура часто оказывается очень сложной, если среди следующих состояний в таблице потока имеются безразличные состояния.

К счастью, в нашем примере таблица потока достаточно мала и проста, так что ее можно минимизировать непосредственно. В состояниях IDLE и RES1 вырабатывается одинаковый выходной сигнал, и у них одни и те же следующие состояния при таких комбинациях входных сигналов, когда следующие состояния указаны. Поэтому эти состояния совместимы и их можно заменить в сокращенной таблице потока одним состоянием IDLE. То же самое можно сказать о состояниях PLS1 и PLS2 (заменяемых на PLS) и о состояниях RES2 и RESN (заменяемых на RES). Результирующая сокращенная таблица потока всего с тремя состояниями представлена на рис. 7.92.

S	P R				Z
	00	01	11	10	
IDLE	IDLE	IDLE	RES	PLS	0
PLS	PLS	IDLE	RES	PLS	1
RES	IDLE	IDLE	RES	RES	0

S*

Рис. 7.92. Сокращенная таблица потока для ловушки импульсов

*7.10.4. Кодирование состояний, гарантирующее отсутствие гонок

Следующий, в определенной мере творческий (читай: «трудный») шаг в проектировании последовательностной схемы с обратной связью заключается в нахождении такого способа присвоения кодов состояниям, называемым в сокращенной таблице потока по именам, при котором не было бы гонок. Напомним, что, согласно сказанному в разделе 7.9.3, гонка возникает в том случае, когда в результате изменения одного входного сигнала изменяется несколько внутренних переменных состояния. В последовательностной схеме с обратной связью недопустимы никакие критические гоки; в противном случае схема может вести себя непредсказуемо. Как мы увидим, для исключения гонок часто оказывается необходимым *увеличить* число состояний схемы.

Возможность наличия гонок в таблице переходов схемы можно проанализировать с помощью *диаграммы смежности состояний (state adjacency diagram)* для ее таблицы потока. Диаграмма смежности представляет собой упрощенную

диаграмму состояний, где описаны пути, по которым состояния замыкаются сами на себя, и не указаны направления переходов (переход $A \rightarrow B$ изображается точно так же, как $B \rightarrow A$) и вызывающие их комбинации входных сигналов. На рис. 7.93 приведен пример таблицы потока для схемы классического образца, на рис. 7.94(a) – соответствующая диаграмма смежности.

S	X Y			
	00	01	11	10
A	(A)	B	(A)	B
B	(B)	(B)	D	(B)
C	(C)	A	A	(C)
D	(D)	B	(D)	C

S^*

Рис. 7.93. Пример таблицы потока как к задаче выбора кодов состояний

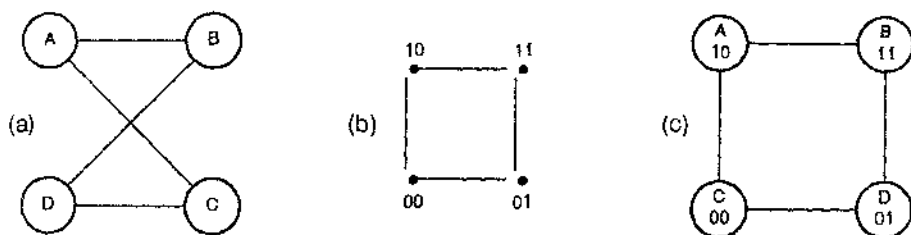


Рис. 7.94. Пример кодирования состояний: (a) диаграмма смежности; (b) 2-мерный куб; (c) один из восьми возможных способов кодирования состояний без гонок

Два состояния называют смежными (*adjacent states*), если они связаны дугой на диаграмме смежности состояний. Для того чтобы при переходах не было гонок, коды смежных состояний должны различаться только в одном разряде. Если два состояния A и B являются смежными, то неважно, имеется ли в исходной таблице потока переход от A к B или от B к A, или переходы могут происходить в обоих направлениях. Любой из этих переходов может содержать гонку, если состояния A и B различаются более чем одной переменной состояния. Вот почему нет необходимости в указании направления переходов на диаграмме смежности.

Задача нахождения свободного от гонок кодирования состояний посредством n переменных состояния эквивалентна отображению узлов и дуг диаграмм смежности на вершины и ребра n -мерного куба. На рис. 7.94 это сводится к отображению диаграммы смежности, изображенной на рис. (a), на 2-мерный куб, представленный на рис. (b). Вы можете визуальнo найти восемь способов осуществить такое отображение (четыре вращения умножить на два зеркальных соответствия), один из которых приводит к присвоению состояниям кодов, указанных на рис. (c).

На рис. 7.95(а) показана диаграмма смежности для нашей ловушки импульсов, построенная по сокращенной таблице потока, приведенной на рис. 7.92. Ясно, что не существует отображения этого «треугольника» состояний на 2-мерный куб. Все, что мы можем сделать, это вернуться назад и видоизменить исходную таблицу потока. В частности, в таблице потока бывают указаны состояния, которых схема должна достичь в каждом переходе *в конце концов*, но она не запрещает схеме пройти через другие состояния на этом пути. Как показано на рис. 7.96, можно ввести новое состояние RESA и заставить схему переходить из состояния PLS в состояние RES через состояние RESA. Новая диаграмма смежности для видоизмененной таблицы состояний изображена на рис. 7.95(б), и у нее уже много возможных способов кодирования, свободных от гонок. Таблица переходов, основанная на кодировании, указанном на рис. 7.95(с), приведена на рис. 7.98. Заметьте, что переход $PLS \rightarrow RESA \rightarrow RES$ будет происходить дольше, чем другие переходы в исходной таблице потока, так как для этого требуется два изменения внутреннего состояния, на которые уйдет удвоенное время распространения сигналов по петлям обратной связи.

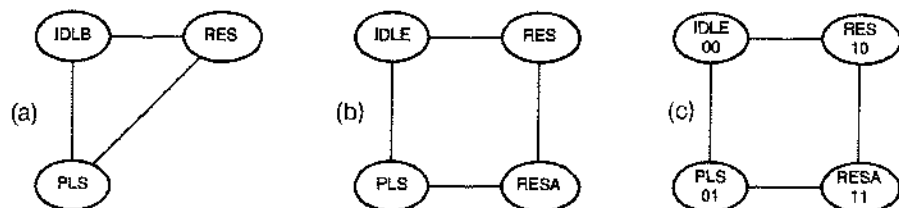


Рис. 7.95. Диаграмма смежности для ловушки импульсов: (а) согласно исходной таблице потока; (б) после добавления еще одного состояния; (с) один из восьми возможных способов кодирования состояний без гонок

S	P R				Z
	00	01	11	10	
IDLE	IDLE	IDLE	RES	PLS	0
PLS	PLS	IDLE	RESA	PLS	1
RESA	—	—	RES	—	—
RES	IDLE	IDLE	RES	RES	0
S*					

Рис. 7.96. Таблица состояний, допускающая кодирование без гонок, для ловушки импульсов

ПРИСВОЕНИЕ КОДОВ В ОБЩЕМ СЛУЧАЕ

Можно показать, что при наличии в таблице потока 2^n строк свободное от гонок кодирование возможно при использовании $2n - 1$ переменных состояний (см. Обзор литературы). Однако на практике встречается не много применений схем классического образца с числом состояний, превосходящим несколько единиц. Поэтому общий случай представляет, скорее всего, только академический интерес.

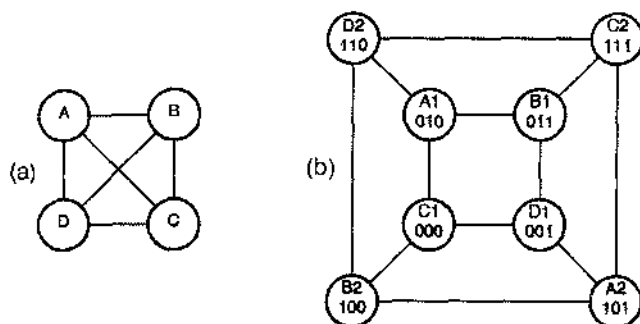


Рис. 7.97. Худший случай: (а) диаграмма смежности для 4-х состояний; (б) кодирование с использованием пар эквивалентных состояний

Y1 Y2	P R				Z
	00	01	11	10	
00	00	00	10	01	0
01	01	00	11	01	1
11	—	—	10	—	—
10	00	00	10	10	0
Y1* Y2*					

Рис. 7.98. Свободная от гонок таблица переходов для ловушки импульсов

Несмотря на то, что мы добавили в нашем примере одно состояние, мы обошлись все же двумя переменными состояниями. Однако для того, чтобы избежать гонок, иногда может понадобиться добавить одну или большее число переменных состояний. На рис. 7.97(а) показана худшая из возможных диаграмм смежности при наличии четырех состояний: каждое состояние является смежным с каждым другим. Ясно, что такую диаграмму смежности нельзя отобразить на 2-мерный куб. Существует, однако, свободное от гонок кодирование состояний путем сопоставления их с вершинами 3-мерного куба; такое кодирование представлено на рис. 7.97(б): каждому состоянию в исходной таблице потока поставлено в соот-

ветствие два эквивалентных состояния в итоговой таблице состояний. Оба состояния в наре, скажем А1 и А2, эквивалентны и обеспечивают один и тот же сигнал на выходе. Каждое состояние является смежным с одним из состояний во всех других парах, так что для каждого следующего состояния можно выбрать переход, при котором не будет возникать гонка.

*7.10.5. Уравнения возбуждения

Получив таблицу переходов схемы, свободную от гонок, мы можем теперь выполнять рутинную работу по выводу уравнений возбуждения для цепей обратной связи. На рис. 7.99 представлены карты Карно, составленные по таблице переходов для ловушки импульсов, приведенной на рис. 7.98. Обратите внимание на то, что следствием наличия в таблице безразличных следующих состояний и значений выходного сигнала стало содержание соответствующих клеток в картах, что упрощает логику возбуждения и логику выхода. Результирующие выражения для сигналов возбуждения и для выходного сигнала в форме «сумма произведений» имеют следующий вид:

$$Y1^* = P \cdot R + P \cdot Y1$$

$$Y2^* = Y2 \cdot R' + Y1' \cdot Y2 \cdot P + Y1' \cdot P \cdot R'$$

$$Z = Y2.$$

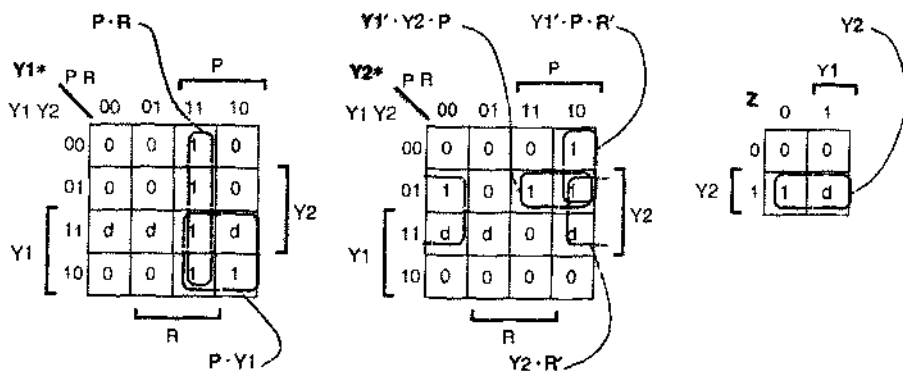


Рис. 7.99. Карты Карно для логики возбуждения и логики выхода ловушки импульсов

Вспомним теперь, что в логике возбуждения последовательностной схемы с обратной связью не должно быть источников опасности. Так уж вышло, что полученные нами выражения вида «сумма произведений» не только являются минимальными, но и не содержат источников опасности. Поэтому эти выражения могут служить основанием для построения ловушки импульсов, принципиальная схема которой представлена на рис. 7.100.

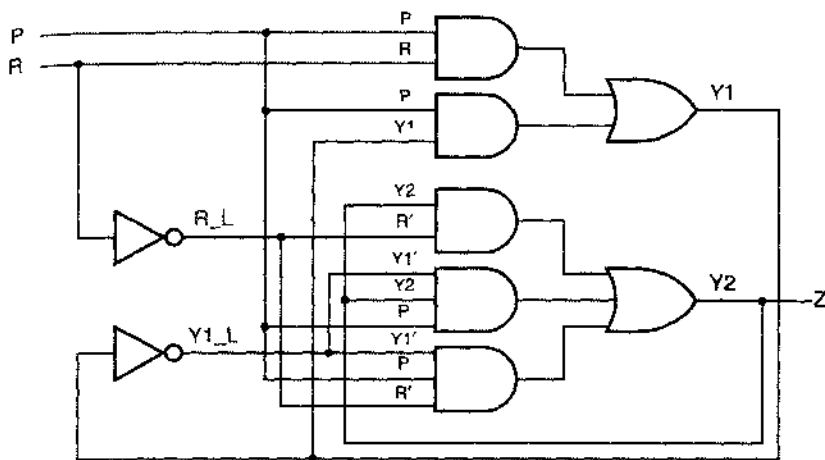


Рис. 7.100. Ловушка имнульсов

*7.10.6. Существенные источники опасности

После всех затраченных усилий вы можете подумать, что схема, к которой мы пришли, будет надежно работать всегда. К сожалению, для полной уверенности в этом оснований пока нет. В общем случае для надлежащей работы схемы классического образца должны удовлетворяться следующие пять требований:

1. Только один входной сигнал может изменяться каждый раз с некоторым ограничением на минимальное время между последовательными изменениями на входах.
2. Должен существовать максимум для задержки распространения при прохождении сигнала через логику возбуждения и по цепям обратной связи; этот максимум должен быть меньше времени между последовательными изменениями на входах.
3. Кодирование состояний и таблица переходов должны обеспечивать отсутствие критических гонок.
4. В логике возбуждения не должно быть источников опасности.
5. Минимальная задержка распространения сигналов по логике возбуждения и цепям обратной связи должна превосходить максимальный разброс времени прохождения сигналов через «входную логику».

Без первого требования было бы невозможно удовлетворить главную предпосылку работы схемы по классическому образцу, согласно которой у схемы должно быть время для того, чтобы между последовательными изменениями входных сигналов успевало установиться устойчивое состояние в целом. Второе

требование заключается в том, что логика возбуждения должна быть достаточно быстродействующей, чтобы могло осуществляться то, о чем только что было сказано. Выполнение третьего требования гарантирует, что надлежащие изменения состояний будут происходить, даже если у непей возбуждения для разных переменных состояния задержки будут различными. Четвертым требованием, когда оно выполнено, обеспечивается неизменность тех переменных состояния, которые при данном переходе должны сохранять свои значения.

Последнее, пятое требование относится к трудно уловимым временным погрешностям, которые могут проявляться в схемах классического образца даже в тех случаях, когда первые четыре требования удовлетворены. Речь идет о *существенном источнике опасности (essential hazard)*, то есть о возможности того, что схема перейдет в ошибочное следующее состояние в результате изменения одного входного сигнала; ошибка происходит в том случае, когда изменение входного сигнала не успевает достичь всех цепей возбуждения до того, как являющийся следствием этого изменения переход переменной состояния (или переходы нескольких переменных) вернется назад к входам цепей возбуждения. В мире, где обычно действует правило «чем быстрее, тем лучше», конструктору иногда приходится *притормозить* логику возбуждения, чтобы скрыть эти источники опасности.

Существенные источники опасности лучше всего объяснить на каком-либо примере, и таким примером вполне может служить наша ловушка импульсов. Предположим, что схема собирается на печатной плате или в кристалле, и мы (а вероятнее всего – используемая нами система CAD) непреднамеренно подключили входной сигнал Р к точке PD, указанной на рис. 7.101, по длинному и медленному пути. Допустим, что время прохождения сигнала по этому пути больше, чем задержка распространения в двухуровневой логике возбуждения И–ИЛИ.

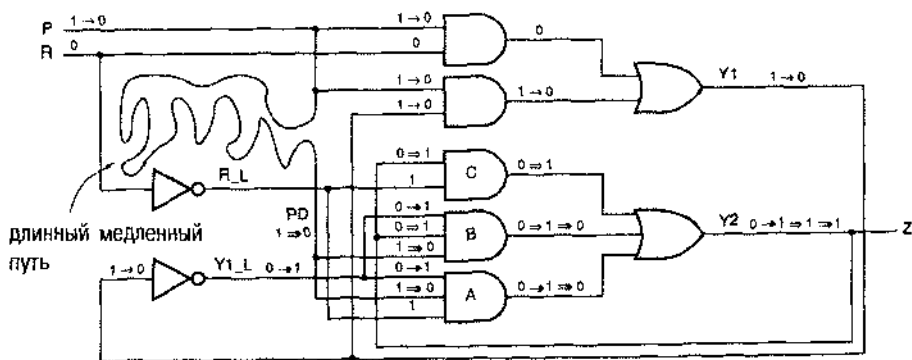


Рис. 7.101. Физические условия, при которых в ловушке импульсов проявляется существенный источник опасности

Рассмотрим теперь, что случится, если $PR = 10$, схема находится во внутреннем состоянии 10 и сигнал Р изменяется от 1 до 0. Согласно таблице переходов, воспроизведенной на рис. 7.102, схеме следует перейти во внутреннее состояние 00. Но давайте рассмотрим, что будет происходить в действительности, прослеживая прохождение сигналов по схеме, приведенной на рис. 7.101:

- (Изменения, указанные стрелкой " $\rightarrow$ ".) Первое, что случится после изменения сигнала Р, это переход сигнала Y1 со значения 1 на значение 0. Сейчас схема находится во внутреннем состоянии 00.
- (Изменения, указанные стрелкой " $\Rightarrow$ ".) Сигнал Y1_L изменяется с 0 на 1. Изменение Y1_L на входе вентиля А вызовет переход к 1 сигнала на его выходе, что, в свою очередь, приведет к возникновению 1 на выходе Y2. Вот так раз! Теперь схема находится во внутреннем состоянии 01.
- (Изменения, указанные стрелкой " $\Rightarrow$ ".) Изменение значения Y2 на входах вентилей В и С вызовет появление единиц на их выходах, подкрепляя наличие 1 на выходе Y2. В течение всего этого времени переход сигнала Р с 1 на 0 еще не проявил себя в точке PD.
- (Изменения, указанные стрелкой " $\Rightarrow$ ".) Наконец уровень сигнала в точке PD изменяется и вместо единичного становится нулевым, в результате чего сигналы на выходах вентилей А и В становятся равными 0. Однако на выходе вентилей С все еще удерживается 1, и поэтому схема остается в *неправильном состоянии* 01.

Y1 Y2	P R				Z
	00	01	11	10	
00	00	00	10	01	0
01	01	00	11	01	1
11	—	—	10	—	—
10	00	00	10	10	0
Y1* Y2*					

Рис. 7.102. Таблица переходов для лопушки импульсов, в которой обнаруживается существенный источник опасности

Избежать ошибочного поведения в принципе можно только одним способом: необходимо обеспечить поступление изменений в сигнале Р на входы всех цепей до того, как произойдут какие-либо изменения переменных состояния. Следовательно, неизбежное разлание моментов поступления входного сигнала, называемого *временным перекосом (timing skew)*, должно быть меньше задержки распространения по цепям возбуждения и обратной связи. В общем случае этому требованию, относящемуся к временным характеристикам, можно удовлетворить только путем осмотнительного проектирования на уровне электрических цепей.

В нашем примере этот источник опасности легко подавить даже не специалисту в области электроники, поскольку все, что для этого требуется, это чтобы прямой провод имел меньшую задержку распространения, чем структура И-ИЛИ, а это реализуется практически при любой технологии.

Однако во многих последовательностных схемах с обратной связью типа переключателя по фронту D-триггера какого-либо из ТТЛ-семейств (рис. 7.19) имеются существенные источники опасности при наличии инверторов на пути про-

хождения входного сигнала. В таких случаях необходимо, чтобы инверторы были гарантированно более быстродействующими, нежели логика возбуждения; это не так уж тривиально как при разработке печатной платы, так и при программировании ИС. Если бы, например, схема, представленная на рис. 7.101, физически строилась на элементах И-ИЛИ-НЕ, то задержка на пути от входов до точки Y1_L могла бы быть совсем малой и иметь значение порядка задержки при прохождении через один инвертор.

В большинстве схем классического образца существенные источники опасности можно обнаружить, но не во всех. Для этого есть простое правило, и оно является следствием более или менее принятого определения «существенного источника опасности»:

- Таблица потока схемы классического образца содержит источник существенной опасности в отношении устойчивого состояния в целом S и входного сигнала X, если в результате трех последовательных переходов в сигнале X схема, начиная с состояния S, достигает устойчивого состояния в целом, отличающегося от того устойчивого состояния в целом, которого схема достигает в результате одного перехода в сигнале X, начиная с состояния S.

По этому правилу существенный источник опасности в ловушке импульсов обнаруживается в результате прохода по стрелкам в таблице переходов (рис. 7.102), начиная с внутреннего состояния 10 при $PR = 10$.

Существенный источник опасности возможен только в такой схеме классического образца, у которой есть, как минимум, три состояния, так что у защелок нет существенных источников опасности. С другой стороны, у всех триггеров (то есть у схем, чувствительных к входным сигналам на фронте тактового сигнала) они имеются.

*7.10.7. Краткие выводы

В заключение перечислим вновь все шаги, через которые надо пройти при проектировании последовательностной схемы с обратной связью:

1. Из словесного описания схемы составить примитивную таблицу потока.
2. Минимизировать число состояний в таблице потока.
3. Найти свободное от гонок кодирование состояний, добавляя при необходимости вспомогательные состояния или расщепляя имеющиеся.
4. Составить таблицу переходов.

ЭТИ ИСТОЧНИКИ ОПАСНОСТИ ВООБЩЕ-ТО СУЩЕСТВЕННЫ

Существенные источники опасности называются «существенными», потому что они присущи таблице потока данной последовательностной функции и будут присутствовать в любой схемной реализации этой функции. Эти источники опасности можно скомпенсировать только регулировкой задержек в схеме. Сравните эти случаи со статическими источниками опасности в комбинационной логике, которые можно, в принципе, исключить, добавив консенсусные термы в логическое выражение.

ПОСЛЕДНИЙ ВОПРОС

Как скоро так трудно построить схему классического образца, которая работала бы нужным образом, не говоря уж о ее быстродействии и компактности, как могла кому-то прийти в голову идея серийного D-триггера из 6 вентилей с 8-ю состояниями (см. рис. 7.20)? Не спрашивайте меня, я не знаю!

5. Начертить карты возбуждения и найти не содержащую источников опасности реализацию уравнений возбуждения.
6. Проверить, нет ли существенных источников опасности. Видоизменить схему, если это необходимо, так, чтобы минимальные задержки в цепях возбуждения и обратной связи были больше максимальных задержек в инверторах и других элементах входной логики.
7. Нарисовать принципиальную схему.

Заметьте также, что во многих схемах вполне может быть нарушено основное предположение, касающееся схем классического образца и состоящее в том, что изменения входных сигналов происходят по отдельности. Например, входной сигнал D у нереклюющегося по положительному фронту D-триггера может измениться в тот же момент времени, когда сигнал CLK переходит из 1 в 0, и схема при этом все же будет работать правильно. Но можно ли сказать то же самое о моменте перехода сигнала CLK из 0 в 1? Чтобы гарантировать надлежащую работу схемы в подобных специальных случаях, необходимо проанализировать таблицу переходов и саму схему путем перебора всех возможных ситуаций.

7.11. Особенности проектирования последовательностных схем на языке ABEL

7.11.1. Регистровые выходы

В языке ABEL имеются средства для построения последовательностных схем. Как будет объяснено в параграфе 8.3, в большинстве случаев пользователь ПЛУ может сделать *выходы регистровыми (registered outputs)*, поместив вслед за логикой И-ИЛИ D-триггеры, как показано на рис. 7.103. Для того, чтобы один или большее число выходов были регистровыми, необходимо в объявлении выводов в программе на языке ABEL поместить предложение `1st type` с ключевым словом `reg` (а не `com`) для каждого регистрового выхода. Пример программы с тремя регистровыми и двумя комбинационными выходами приведен в табл. 7.22.

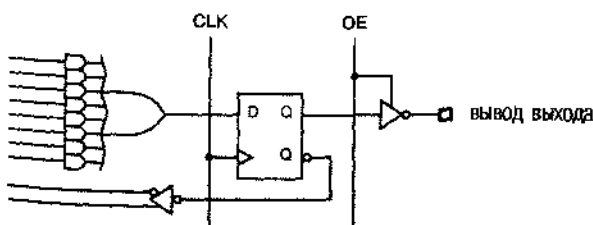


Рис. 7.103. Регистровый выход ПЛУ

Табл. 7.22. Программа на языке ABEL с регистровыми выходами

```

module CombLock
Title 'Combination-Lock State Machine'

" Input and Outputs
X, CLOCK      pin;
UNLK, HINT     pin istype 'com';
Q1, Q2, Q3     pin istype 'reg';

Q = [Q1..Q3];

Equations

Q.CLK = CLOCK; Q.OE = 1;

" State variables
Q1 := Q1 & !Q2 & X # !Q1 & Q2 & Q3 & !X # Q1 & Q2 & !Q3;
Q2 := !Q2 & Q3 & X # Q2 & !Q3 & X;
Q3 := Q1 & !Q2 & !Q3 # Q1 & Q3 & !X # !Q2 & !X
      # !Q1 & !Q3 & !X # Q2 & !Q3 & X;

" Mealy outputs
UNLK = Q1 & Q2 & Q3 & !X;
HINT = !Q1 & !Q2 & !Q3 & !X # Q1 & !Q2 & X # !Q2 & Q3 & X
      # Q2 & Q3 & !X # Q2 & !Q3 & X;

end CombLock

```

Как следует из рис. 7.103, у регистрового выхода имеются, по меньшей мере, два других связанных с ним атрибута. Буфер с тремя состояниями, помещенный перед выходным выводом, управляется сигналом разрешения выхода OE, а у самого триггера имеется вход тактового сигнала CLK. Из программы в табл. 7.22 видно, что сигналы, подаваемые на эти входы, определяются в разделе равенств. Каждому из этих сигналов присваивается имя соответствующего основного выходного сигнала, паразитируемое суффиксом-признаком .CLK или .OE. В некоторых ПЛУ у триггеров имеются дополнительные входы для подачи управляющих сигналов. Например, входные сигналы асинхронной установки в единичное состояние и сброса имеют суффиксы-признаки .AR и .AR, сигналы синхронной установки и сброса — суффиксы .SP и .SR. В ряде ПЛУ возможно использование не D-триггеров, а триггеров других типов; имена их входных сигналов обозначаются суффиксами типа .J и .K.

В разделе ABEL-программы equations логическое значение сигнала на регистровом выходе устанавливается *тактируемым оператором присваивания* (*clocked assignment operator*) :=. При комбинировании схемы в ПЛУ сигнал, задаваемый выражением в правой части, подается на вход D выходного триггера. Действуют те же самые правила, что и в случае комбинационного выхода в отношении управления

полнотой, генерирования множества включений, безразличных значений и т.д. В табл. 7.22 биты состояния Q1–Q3 являются сигналами на регистровых выходах, поэтому для них используется тактируемое присваивание “:=”, а в отношении сигналов UNLK и HINT на выходах автомата Милли, которые являются комбинационными функциями текущего состояния и входного сигнала, применяется нетактируемое присваивание “=”. Если автомат имеет конвейерный выход (см. рис. 7.37), то его выходным сигналам значения присваиваются тактируемым оператором.

В отношении регистровых выходов может быть использован синтаксис таблицы истинности в языке ABEL (см. табл. 4.16). Единственное отличие состоит в том, что оператор “->” между входом и выходом заменяется *тактируемым оператором таблицы истинности (clocked truth-table) “:=>”*.

Последовательностные схемы с обратной связью можно проектировать на языке ABEL и не используя специально предназначенные для этого средства. В разделе 8.2.6 мы покажем, например, как описываются защелки на языке ABEL.

7.11.2. Диаграммы состояний

Приведенный в предыдущем разделе пример конечного автомата представляет собой описание на языке ABEL кодового замка, синтезированного нами вручную в разделе 7.4.6. Однако в большинстве языков программирования ПЛУ имеется символика, позволяющая описывать и синтезировать конечные автоматы непосредственно, не прибегая к записи таблиц состояний, переходов или возбуждения и без вывода от руки уравнений возбуждения. Эта символика носит название *языка описания конечных автоматов (state-machine description language)*. В языке ABEL роль такой символики играет «диаграмма состояний», исходя из которой ABEL-компилятор выполняет всю работу по генерированию уравнений возбуждения, реализующих заданный автомат.

В языке ABEL ключевым словом `state_diagram` отмечается начало описания конечного автомата. Табл. 7.23 демонстрирует текстовую структуру «диаграммы состояний» на языке ABEL. Здесь *вектор состояния (state-vector)* представляет собой ABEL-набор, в котором перечислены переменные состояния данного автомата. Если в этом наборе n переменных, то автомат имеет 2^n возможных состояний, соответствующих 2^n различным способам присваивания постоянных значений этим n переменным. В ABEL-программе состояниям обычно даются символические имена; это позволяет легко перепробовать различные способы присваивания простым изменением определений констант.

ТАК ЛИ ВАЖЕН `istype`?

В старых схемах типа схем семейства PAL16Rx имеется фиксированная, заранее заданная смесь комбинационных и регистровых выходов и ее нельзя изменить. В таких схемах компилятор узнает о типе каждого выхода по номеру вывода, и в этом случае оператор `istype` не обязателен. Даже при наличии конфигурируемых выходов некоторые компиляторы в состоянии сделать правильное заключение о типе выхода из равенств. Но лучше все же сообщать эту информацию в предложении `istype` для страховки и придания проекту большей мобильности.

```

state_diagram state-vector
state state-value 1 : transition statement;
state state value 2 . transition statement;
...
state state value 2n : transition statement;

```

Табл. 7.23. Структура «диаграммы состояний» в языке ABEL.

Для каждой переменной состояния на основании информации, содержащейся в «диаграмме состояний», создается множество включений. Если переменная состояния является также в левой части равенства в разделе *equations*, то имеет место эффект накопления (см. текст в рамке в разделе 8.5.9, где объясняется, как этим можно воспользоваться). Ключевое слово *state* указывает, что для данного текущего состояния сейчас будут определены следующие состояния и текущие значения выходных сигналов; величина *state value* представляет собой константу, которой определяются значения переменных состояния в текущем состоянии. Оператор *transition statement* задает возможные следующие состояния для данного текущего состояния.

В языке ABEL чаще всего употребляются два оператора перехода: *operator GOTO* (*GOTO statement*), задающий следующее состояние безусловно, например, “GOTO INIT”, и *operator IF* (*IF statement*), посредством которого возможные следующие состояния представляются в виде зависимости от произвольных логических выражений. (Существует еще редко используемый оператор *CASE*, который мы не будем рассматривать.)

Синтаксис оператора *IF* указан в табл. 7.24. Здесь *TrueState* и *FalseState* – имена состояний, в которые переходит автомат, когда выражение *LogicExpression* истинно или ложно соответственно. Эти операторы могут быть вложенными: на месте *FalseState* может находиться другой оператор *IF*; оператор *IF* может находиться также на месте *TrueState*, если он заключен в фигурные скобки. Использование структуры *IF-THEN-ELSE* при нескольких возможных следующих состояниях исключает неоднозначности, которые случаются при вычерчивании диаграммы состояний от руки, когда условия переходов из данного состояния оказываются перекрывающимися (см. параграф 7.5).

```

IF LogicExpression THEN
    TrueState;
ELSE
    FalseState;

```

Табл. 7.24. Структура оператора *IF* в языке ABEL.

Мы продемонстрируем возможности «диаграмм состояний» языка ABEL на примере конечного автомата, который был сконструирован нами в разделе 7.4.1. Таблица состояний для этого автомата была приведена на рис. 7.49. То же самое на языке ABEL представлено в виде табл. 7.25. Следует отметить ряд отличительных особенностей этой программы:

Табл. 7.25. Пример представления диаграммы состояний на языке ABEL

```

module SMEX1
title 'PLD Version of Example State Machine'

" Input and output pins
CLOCK, RESET L, A, B           pin:
Q1..Q3                         pin istype 'reg';
Z                               pin istype 'com';

" Definitions
QSTATE = [Q1,Q2,Q3];           " State variables
INIT   = [ 0, 0, 0];
AO     = [ 0, 0, 1];
A1     = [ 0, 1, 0];
OK0    = [ 0, 1, 1];
OK1    = [ 1, 0, 0];
XTRA1  = [ 1, 0, 1];
XTRA2  = [ 1, 1, 0];
XTRA3  = [ 1, 1, 1];
RESET  = !RESET_L;

state_diagram QSTATE

state INIT: IF RESET THEN INIT
            ELSE IF (A==0) THEN AO
            ELSE A1;

state AO:   IF RESET THEN INIT
            ELSE IF (A==0) THEN OK0
            ELSE A1;

state A1:   IF RESET THEN INIT
            ELSE IF (A==0) THEN AO
            ELSE OK1;

state OK0:  IF RESET THEN INIT
            ELSE IF (B==1)&(A==0) THEN OK0
            ELSE IF (B==1)&(A==1) THEN OK1
            ELSE IF (A==0) THEN OK0
            ELSE IF (A==1) THEN A1;

state OK1:  IF RESET THEN INIT
            ELSE IF (B==1)&(A==0) THEN OK0
            ELSE IF (B==1)&(A==1) THEN OK1
            ELSE IF (A==0) THEN AO
            ELSE IF (A==1) THEN OK1;

state XTRA1: GOTO INIT;
state XTRA2: GOTO INIT;
state XTRA3: GOTO INIT;

equations
QSTATE.CLK = CLOCK; QSTATE.OE = 1;
Z = (QSTATE == OK0) # (QSTATE == OK1);

END SMEX1

```

ИСПОЛЬЗУЙТЕ ИМЕЮЩИЕСЯ ВОЗМОЖНОСТИ ИЛИ ПИШИТЕ ELSE

Структура IF-THEN-ELSE языка ABEL исключает неоднозначность переходов, которая может иметь место в диаграмме состояний. Однако предложение ELSE в операторе IF является необязательным. Если оно опущено, то следующее состояние для каких-то комбинаций входных сигналов может оказаться не заданным. Как правило, это происходит помимо воли проектировщика.

Тем не менее, если вы можете гарантировать, что не упоминаемые комбинации входных сигналов никогда не наступят, то можно сократить запись, касающуюся логики переходов. При наличии директивы @DCSET ABEL-компилятор трактует отсутствие указания на переход для не заданной комбинации состояние/вход как сообщение о переходе в «безразличное» состояние. Кроме того, он воспринимает все переходы из неиспользуемых состояний как переходы в «безразличное» состояние.

- Согласно определению, состояние QSTATE кодируется вектором состояний, состоящим из трех переменных.
- Определениями INIT-XTRA3 задаются кодовые имена отдельных состояний.
- Используются вложенные операторы IF-THEN-ELSE. В предложении с вложенными операторами IF-THEN-ELSE одно и то же следующее состояние может фигурировать во многих местах (например, состояния OK0 и OK1).
- Выражения вида $(B==1) * (A==0)$ использованы вместо эквивалентных им выражений вида $B * !A$ только потому, что первая из этих форм является чуть более наглядной.
- Первым оператором IF в каждом из состояний INIT-OK1 обеспечивается переход автомата в состояние INIT при поступлении сигнала RESET.
- Записи, относящиеся к состояниям XTRA1-XTRA3, гарантируют переход автомата в «безопасное» состояние, если он вдруг так или иначе попадет в неиспользуемое состояние.
- Единственным равенством в разделе программы «equations» определяется тип автомата по тому, от чего зависит выходной сигнал; в данном случае это автомат типа Мура.

В табл. 7.26 приведены результирующие уравнения возбуждения и выхода, составленные ABEL-компилятором (уравнения обратной полярности не показаны). Обратите внимание на имена переменных в правых частях равенств типа $Q1.FB$. Здесь признак-суффикс $.FB$ указывает, что это – сигнал обратной связи (“feedback”), поступающий в решетку И-ИЛИ с выхода триггера, но не с соответствующего вывода ПЛУ, с учетом выбираемого инвертирования в тех ПЛУ, где это возможно. На рис. 7.104 показано, что язык ABEL реально позволяет вам, путем использования различных признаков-суффиксов при имени сигнала, выбирать в правой части равенства одно из трех возможных значений сигнала:

- Q – фактический сигнал на выходе триггера до программируемой инверсии.
- FB – то значение сигнала, какое было бы на выходном выводе при наличии разрешения выхода.

.PIN — фактическое значение сигнала на выводе ПЛУ. Если буфер с тремя состояниями на выходе занерт, то этот сигнал задается другим устройством, либо данный выход остается в плавающем состоянии.

Очевидно, что значение .PIN не следует использовать в уравнении возбуждения конечного автомата, поскольку в этом случае не гарантируется, что оно всегда будет равно значению переменной состояния.

```

Q1 := (!Q2.FB & !Q3.FB & RESET_L
      # Q1.FB & RESET_L);
Q1.C = (CLOCK);
Q1.OE = (1);

Q2 := (Q1.FB & !Q3.FB & RESET_L & !A
      # Q1.FB & Q3.FB & RESET_L & A
      # Q1.FB & Q2.FB & RESET_L & B);
Q2.C = (CLOCK);
Q2.OE = (1);

Q3 := (!Q2.FB & !Q3.FB & RESET_L & A
      # Q1.FB & RESET_L & A);
Q3.C = (CLOCK);
Q3.OE = (1);

Z = (Q2 & Q1);

```

Табл. 7.26. Сокращенный перечень уравнений для ПЛУ SMEX1

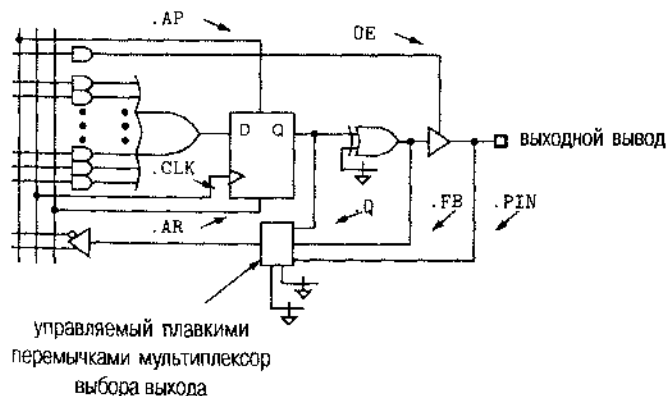


Рис. 7.104. Возможность выбора выходного сигнала в сложных ПЛУ

Несмотря на использование «языка высокого уровня», при написании программы на языке ABEL (табл. 7.25.) все же пришлось опираться на исходную, составленную вручную таблицу состояний (в данном случае это была таблица, представленная на рис. 7.49). В табл. 7.27 показан другой подход. Эта программа составляется непосредственно из словесного описания конечного автомата, воспроизводимого здесь еще раз:

Построить тактируемый синхронный конечный автомат с двумя входами А и В и одним выходом Z, таким что выходной сигнал Z равен 1, если

- входной сигнал А имел одно и то же значение на каждом из двух предыдущих тактов тактового сигнала *или*
- входной сигнал В равен 1 с последнего момента времени, когда первое условие было выполнено.

В противном случае выходной сигнал должен равняться 0.

Табл. 7.27. Более «естественная» программа на языке ABEL для конечного автомата из раздела 7.4.1

```

module SMEX2
title 'Alternate Version of Example State Machine'

" Input and output pins
CLOCK, RESET_L, A, B           pin;
LASTA, Q1, Q2                  pin istype 'reg';
Z                               pin istype 'com';

" Definitions
QSTATE = [Q1,Q2];               " State variables
INIT    = [ 0, 0];              " State encodings
LOOKING = [ 0, 1];
OK      = [ 1, 0];
XTRA    = [ 1, 1];
RESET   = !RESET_L;

state_diagram QSTATE

state INIT:   IF RESET THEN INIT ELSE LOOKING;

state LOOKING: IF RESET THEN INIT
               ELSE IF (A == LASTA) THEN OK
               ELSE LOOKING;

state OK:     IF RESET THEN INIT
               ELSE IF B THEN OK
               ELSE IF (A == LASTA) THEN OK
               ELSE LOOKING;

state XTRA:   GOTO INIT;

equations
LASTA.CLK = CLOCK; QSTATE.CLK = CLOCK; QSTATE.OE = 1;

LASTA := A;
Z = (QSTATE == OK);

END SMEX2

```

ОПЕРАНД-ФАНТОМ

Обычно в реальных ИС типа CPLD имеются только двухходовые мультиплексоры выбора выходного сигнала, так что выход `.FB`, указанный на рис. 7.104, бывает опущен. Если в уравнении возникает обращение к сигналу с признаком `.FB`, то компилятор использует соответствующий сигнал с признаком `.Q`, осуществляя, если это необходимо, его инвертирование.

Основная идея нового подхода заключается в отказе от состояния, ответственного за последнее значение сигнала `A`, и введении вместо этого отдельного триггера `LASTA` для хранения этой информации. Тогда должны быть определены только два состояния, помимо первоначального состояния `INIT: LOOKING` («ожидание пары одинаковых значений сигнала `A`») и `OK` («поступили два одинаковых значения сигнала `A`, либо сигнал `B` остается равным 1 с последнего момента поступления пары одинаковых значений сигнала `A`»). Сигнал на выходе `Z` — это результат простого комбинационного декодирования состояния `OK`.

***7.11.3. Внешняя память состояния**

Бывают случаи, когда информация о состоянии конечного автомата, построенного на основе ПЛУ, хранится во внешних по отношению к ПЛУ триггерах. В языке ABEL есть специальный вариант оператора `state_diagram` для такой ситуации:

```
state_diagram current-state-variables → next-state-variables
```

Здесь *current-state-variables* — ABEL-набор, в котором перечислены входные сигналы, представляющие в совокупности текущее состояние, а *next-state-variables* — набор соответствующих выходных сигналов, подаваемых на входы внешних D-триггеров, в которых хранится состояние автомата; например,

```
state_diagram [CURQ1, CURQ2] → [NEXTQ1, NEXTQ2]
```

***7.11.4. Задание выходных сигналов автомата Мура**

В рассматриваемом примере конечный автомат является автоматом Мура, так что его выходной сигнал `Z` зависит только от состояния, что и было нами реализовано с помощью соответствующих равенств в разделах `equations` программ, приведенных в табл. 7.25 и 7.27. Но можно было поступить и иначе; язык ABEL позволяет задавать выходные сигналы автомата Мура одновременно с определением самих состояний. Оператору перехода в определении состояния `state` может предшествовать одно или большее число необязательных равенств, как показано в табл. 7.28. Чтобы воспользоваться этой возможностью, например, в случае автомата, описываемого программой в табл. 7.27, следует исключить относящееся к сигналу `Z` равенство из раздела `equations` и переписать диаграмму состояний так, как это сделано в табл. 7.29.

```
state_diagram state-variables
```

```
state state-value 1 :
```

```
    optional equation;
```

```
    optional equation;
```

```
    ...
```

```
    transition statement;
```

```
state state-value 2 :
```

```
    optional equation;
```

```
    optional equation;
```

```
    ...
```

```
    transition statement;
```

```
...
state state-value 2n :
```

```
    optional equation;
```

```
    optional equation;
```

```
    ...
```

```
    transition statement;
```

Табл. 7.28. Структура диаграммы состояний в языке ABEL, в которой задаются значения выходных сигналов автомата Мура.

Табл. 7.29. Конечный автомат Мура со встроенными определениями выходных сигналов

```
state_diagram QSTATE
```

```
state INIT:    Z = 0;
```

```
               IF RESET THEN INIT ELSE LOOKING;
```

```
state LOOKING: Z = 0;
```

```
               IF RESET THEN INIT
```

```
               ELSE IF (A == LASTA) THEN OK
```

```
               ELSE LOOKING;
```

```
state OK:      Z = 1;
```

```
               IF RESET THEN INIT
```

```
               ELSE IF B THEN OK
```

```
               ELSE IF (A == LASTA) THEN OK
```

```
               ELSE LOOKING;
```

```
state XTRA:    Z = 0;
```

```
               GOTO INIT;
```

Если такая переменная, как Z, встречается в левой части нескольких соотношений, то так же, как это имеет место и в отношении других равенств в ABEL-программе, результирующее значение данной переменной получается путем объединения правых частей по правилам операции ИЛИ (подробнее это было объяснено в разделе 4.6.3). Заметьте, что все сказанное относилось к случаю, когда выход Z является комбинационным, не регистровым. Если бы выход Z был регистровым, то желаемые значения сигнала возникали на выходе лишь на следующем такте, то есть на один такт позднее, после того как автомат побывал в соответствующем состоянии.

*7.11.5. Задание сигналов на выходах типа Мили и на конвейерных выходах с помощью оператора WITH

Выходные сигналы конечных автоматов могут зависеть не только от состояния, но также и от входных сигналов. В разделе 7.3.2 мы назвали выходы таких автоматов выходами типа Мили или конвейерными выходами в зависимости от того, возникают ли соответствующие сигналы на выходе сразу же после изменения входных сигналов или только на очередном фронте тактового сигнала. Оператор *WITH* (*WITH statement*) языка ABEL позволяет задавать эти выходные сигналы одновременно со следующими состояниями, а не в разделе программы *equations*.

Синтаксис оператора *WITH* очень прост, как это видно из табл. 7.30. За именем любого следующего состояния, являющимся частью оператора перехода, может следовать ключевое слово *WITH* и заключенный в скобки список равенств, которые «исполняются» при данном переходе. Формально процедуру можно описать так: пусть «Е» – выражение возбуждения, истинное только при осуществлении данного перехода; тогда в каждом равенстве, содержащемся во взятом в скобки списке оператора *WITH*, результат объединения правой части с Е по правилам логической операции И присваивается сигналу, указанному в левой части. В равенствах можно использовать как неактивируемое, так и активируемое присваивание для получения сигнала на выходе Мили и на конвейерном выходе соответственно.

```
next-state WITH {
    equation;
    equation;
}
```

Табл. 7.30. Структура оператора *WITH* в языке ABEL

В разделе 7.4.6 мы рассмотрели в качестве примера конечного автомата с выходом Мили «кодовый замок» и пришли к табл. 7.14. Тот же самый конечный автомат представлен в виде программы на языке ABEL в табл. 7.31, где для выходов типа Мили использованы операторы *WITH*. Заметьте, что при этом определения состояний оканчиваются закрывающими скобками, а не точками с запятой, которыми обычно оканчиваются операторы переходов.

Согласно словесному описанию кодового замка, выходные сигналы *UNLK* и *HINT* не могут быть конвейерными, поскольку их значения зависят от текущего значения входного сигнала *X*. Если же неопределить состояния, положив, что *UNLK* – это полностью «незанятое» состояние, а *HINT* – само рекомендуемое следующее значение *X*, то можно построить новый автомат с конвейерными выходами, как это сделано в табл. 7.32. Важно не потерять из виду, что здесь значения сигналов *UNLK* и *HINT* другие, нежели в примере с выходом типа Мили, поскольку в данном случае эти сигналы вычисляются «с предвидением» на один такт вперед.

Табл. 7.31. Конечный автомат со встроенными определениями сигналов на выходах типа Мили

```

module SMEX4
title 'Combination-Lock State Machine'

" Input and output pins
CLOCK, X                                pin;
Q1..Q3                                pin istance 'reg';
UNLK, HINT                             pin istance 'com';

" Definitions
S      = [Q1,Q2,Q3];                    " State variables
ZIP    = [ 0, 0, 0];                    " State encodings
X0     = [ 0, 0, 1];
X01    = [ 0, 1, 0];
X011   = [ 0, 1, 1];
X0110  = [ 1, 0, 0];
X01101 = [ 1, 0, 1];
X011011 = [ 1, 1, 0];
X0110111 = [ 1, 1, 1];

state_diagram S

state ZIP:      IF X==0 THEN X0      WITH {UNLK = 0; HINT = 1}
                ELSE ZIP             WITH {UNLK = 0; HINT = 0}

state X0:       IF X==0 THEN X0      WITH {UNLK = 0; HINT = 0}
                ELSE X01             WITH {UNLK = 0; HINT = 1}

state X01:      IF X==0 THEN X0      WITH {UNLK = 0; HINT = 0}
                ELSE X011            WITH {UNLK = 0; HINT = 1}

state X011:     IF X==0 THEN X0110   WITH {UNLK = 0; HINT = 1}
                ELSE ZIP             WITH {UNLK = 0; HINT = 0}

state X0110:    IF X==0 THEN X0      WITH {UNLK = 0; HINT = 0}
                ELSE X01101          WITH {UNLK = 0; HINT = 1}

state X01101:   IF X==0 THEN X0      WITH {UNLK = 0; HINT = 0}
                ELSE X011011         WITH {UNLK = 0; HINT = 1}

state X011011:  IF X==0 THEN X0110   WITH {UNLK = 0; HINT = 0}
                ELSE X0110111        WITH {UNLK = 0; HINT = 1}

state X0110111: IF X==0 THEN X0      WITH {UNLK = 1; HINT = 1}
                ELSE ZIP             WITH {UNLK = 0; HINT = 0}

equations
S.CLK = CLOCK;

END SMEX4

```

Из-за этого «предвидения» автоматы с конвейерным выходом могут быть более трудными для понимания и проектирования, чем автоматы с выходом Мили. В приведенном примере нам пришлось изменить даже постановку задачи, чтобы приспособиться к новым требованиям. Достоинство конвейерного выхода связано с тем, что сигналы берутся непосредственно с регистровых выходов, и заключается в том, что они устанавливаются после изменения состояния быстрее.

чем в случае выходов типа Мура или типа Мили, когда обычно необходима дополнительная комбинационная логика; при этом экономится время, равное задержке прохождения сигнала через несколько вентилях. Возможно, что в примере с кодовым замком не так уж важно открыть валл замок или увидеть подсказку на несколько наносекунд раньше. Однако в быстродействующих устройствах исключение этой задержки может играть решающую роль.

Табл. 7.32. Конечный автомат со встроенными определениями сигналов на конвейерных выходах

```

module SMEX5
title 'Combination-Lock State Machine'

" Input and output pins
CLOCK, X
Q1..Q3, UNLK, HINT
pin;
pin istype 'reg';

" Definitions
S      = [Q1,Q2,Q3];
ZIP    = [ 0, 0, 0];
X0     = [ 0, 0, 1];
X01    = [ 0, 1, 0];
X011   = [ 0, 1, 1];
X0110  = [ 1, 0, 0];
X01101 = [ 1, 0, 1];
X011011 = [ 1, 1, 0];
X0110111 = [ 1, 1, 1];

" State variables
" State encodings

state_diagram S

state ZIP:      IF X==0 THEN X0      WITH {UNLK := 0; HINT := 1}
                ELSE ZIP             WITH {UNLK := 0; HINT := 0}

state X0:       IF X==0 THEN X0      WITH {UNLK := 0; HINT := 1}
                ELSE X01             WITH {UNLK := 0; HINT := 1}

state X01:      IF X==0 THEN X0      WITH {UNLK := 0; HINT := 1}
                ELSE X011            WITH {UNLK := 0; HINT := 0}

state X011:     IF X==0 THEN X0110   WITH {UNLK := 0; HINT := 1}
                ELSE ZIP             WITH {UNLK := 0; HINT := 0}

state X0110:    IF X==0 THEN X0      WITH {UNLK := 0; HINT := 1}
                ELSE X01101          WITH {UNLK := 0; HINT := 1}

state X01101:   IF X==0 THEN X0      WITH {UNLK := 0; HINT := 1}
                ELSE X011011         WITH {UNLK := 0; HINT := 1}

state X011011:  IF X==0 THEN X0110   WITH {UNLK := 0; HINT := 1}
                ELSE X0110111        WITH {UNLK := 1; HINT := 0}

state X0110111: IF X==0 THEN X0      WITH {UNLK := 0; HINT := 1}
                ELSE ZIP             WITH {UNLK := 0; HINT := 0}

equations
S.CLK = CLOCK; UNLK.CLK = CLOCK; HINT.CLK = CLOCK;
END SMEX5

```

7.11.6. Проверочные векторы

Полезность проверочных векторов и ограничения в отношении возможностей их использования при проектировании последовательностных схем на языке ABEL те же, что и в случае комбинационных схем (см. раздел 4.6.7). Одно важное добавление в синтаксис проверочных векторов состоит в использовании константы “.C.” для обозначения *фронта тактового сигнала* (clock edge) $0 \rightarrow 1 \rightarrow 0$. В табл. 7.33 приведена ABEL-программа с проверочными векторами для простого 8-разрядного регистра с входом разрешения тактового сигнала. С помощью набора векторов проверяется возможность загрузки различных значений входных сигналов и способность регистра их удерживать.

Табл. 7.33. Программа на языке ABEL с проверочными векторами для простого 8-разрядного регистра

```

module REG8EN
title '8-bit register with clock enable'

" Input and output pins
CLK, EN, D1..D8      pin;
Q1..Q8               pin istype 'reg';

" Sets
D = [D1..D8];
Q = [Q1..Q8];

equations

Q.CLK = CLK;

WHEN EN == 1 THEN Q := D ELSE Q := Q;

test_vectors ([CLK, EN, D ] -> [ Q ])
    [.C., 1, ^h00] -> [^h00]; " 0s in every bit
    [.C., 0, ^hFF] -> [^h00]; " Hold capability, EN=0
    [.C., 1, ^hFF] -> [^hFF]; " 1s in every bit
    [.C., 0, ^h00] -> [^hFF]; " Hold capability
    [.C., 1, ^h55] -> [^h55]; " Adjacent bits shorted
    [.C., 0, ^hAA] -> [^h55]; " Hold capability
    [.C., 1, ^hAA] -> [^hAA]; " Adjacent bits shorted
    [.C., 1, ^h55] -> [^h55]; " Load with quick setup
    [.C., 1, ^hAA] -> [^hAA]; " Again

END REG8EN

```

Типичный подход к тестированию конечных автоматов заключается в составлении таких векторов, которые заставят автомат не только побывать во всех состояниях, но и пройти по всем переходам из каждого состояния. Главное отличие и основная трудность по сравнению с проверочными векторами для

комбинационных схем состоят в том, что векторы должны сначала ставить автомат в желаемое состояние, перед тем как будет протестирован переход, а затем возвращать автомат назад и делать это столько раз, сколько необходимо для тестирования каждого перехода из данного состояния.

В табл. 7.34 приведены проверочные векторы для конечного автомата, описанного в табл. 7.27. Важно понимать, что, в отличие от комбинационных векторов, эти векторы должны подаваться на входы автомата точно в том порядке, в каком они написаны. Заметьте, что векторы пишутся так, чтобы не зависеть от кодов состояний. В результате они могут оставаться одними и теми же при изменении способа кодирования состояний.

Табл. 7.34. Проверочные векторы для конечного автомата из табл. 7.27

test_vectors									
([RESET_L, CLOCK, A, B] -> [QSTATE, LASTA, Z])									
[0	,	.C.	,	0	,	0]	->	[INIT	, 0, 0]; " Check -->INIT (RESET)
[0	,	.C.	,	1	,	0]	->	[INIT	, 1, 0]; " and LASTA flip-flop
[1	,	.C.	,	0	,	0]	->	[LOOKING,	0, 0]; " Come out of initialization
[0	,	.C.	,	0	,	0]	->	[INIT	, 0, 0]; " Check LOOKING-->INIT (RESET)
[1	,	.C.	,	0	,	0]	->	[LOOKING,	0, 0]; " Come out of initialization
[1	,	.C.	,	1	,	0]	->	[LOOKING,	1, 0]; " --> LOOKING since 0!=1
[1	,	.C.	,	1	,	0]	->	[OK	, 1, 1]; " --> OK since 1==1
[0	,	.C.	,	0	,	0]	->	[INIT	, 0, 0]; " Check OK-->INIT (RESET)
[1	,	.C.	,	0	,	0]	->	[LOOKING,	0, 0]; " Go back towards OK ...
[1	,	.C.	,	0	,	0]	->	[OK	, 0, 1]; " --> OK since 0==0
[1	,	.C.	,	1	,	1]	->	[OK	, 1, 1]; " --> OK since B, even though 1!=0
[1	,	.C.	,	1	,	0]	->	[OK	, 1, 1]; " --> OK since 1==1
[1	,	.C.	,	0	,	0]	->	[LOOKING,	0, 0]; " --> LOOKING since 0!=1

Мы встретимся с еще одной трудностью, если попытаемся составить проверочные векторы для кодового замка, описанного в табл. 7.31. При тестировании этого автомата главной проблемой оказывается отсутствие у него входа сброса. Его исходное состояние может быть различным при реализации на основе разных технологий и в разных ПЛУ: при включении питания все триггеры могут устанавливаться в единичные состояния или сбрасываться, либо случайным образом попадать в то или в другое состояние. В реальном воплощении рассматриваемому автомату не нужен вход сброса, но для целей тестирования его необходимо как-то заставлять входить в известное начальное состояние.

К счастью, у кодового замка есть *синхронизирующая последовательность* (*synchronizing sequence*), то есть фиксированная последовательность из одного или большего числа значений входного сигнала, которая всегда приводит автомат в определенное, известное состояние. В данном конкретном случае, независимо от того, в каком состоянии находился автомат сначала, при подаче на его вход значения $X = 1$ в течение четырех тактов на четвертом такте он всегда будет оказываться в состоянии ZIP. Это именно то, что делается первыми четырьмя векторами в табл. 7.35. До тех пор, пока автомат не достигнет известного состояния, мы указываем ему в правой части проверочных векторов в качестве следующего состояния «безразличное» состояние, благодаря чему ни моделирующая программа, ни тестер, реализованный в виде отдельного устройства, не реагирует на случайное состояние как на ошибку.

СИНХРОНИЗИРУЮЩИЕ ПОСЛЕДОВАТЕЛЬНОСТИ И ВХОДЫ СБРОСА

Нам повезло с кодовым замком: не у каждого конечного автомата есть синхронизирующие последовательности. Вот почему в большинстве случаев конечные автоматы проектируются с входом сброса, при наличии которого длина синхронизирующей последовательности становится равной единице.

Табл. 7.35. Проверочные векторы для кодового замка из табл. 7.31

```
test_vectors
([CLOCK, X] -> [ S      , UNLK, HINT])
[ .C. , 1] -> [.X.    , .X. , .X. ]; " Since no reset input, apply
[ .C. , 1] -> [.X.    , .X. , .X. ]; " a 'synchronizing sequence'
[ .C. , 1] -> [.X.    , .X. , .X. ]; " to reach a known starting
[ .C. , 1] -> [ZIP    , .X. , .X. ]; " state
[ 0 , 0] -> [ZIP    , 0 , 1 ]; " Test Mealy outputs for both
[ 0 , 1] -> [ZIP    , 0 , 0 ]; " values of X
[ .C. , 1] -> [ZIP    , .X. , .X. ]; " Test ZIP-->ZIP (X==1)
[ .C. , 0] -> [X0    , .X. , .X. ]; " and ZIP-->X0 (X==0)
[ 0 , 0] -> [X0    , 0 , 0 ]; " Test Mealy outputs for both
[ 0 , 1] -> [X0    , 0 , 1 ]; " values of X
[ .C. , 0] -> [X0    , .X. , .X. ]; " Test X0-->X0 (X==0)
[ .C. , 1] -> [X01   , .X. , .X. ]; " and X0-->X01 (X==1)
[ 0 , 0] -> [X01   , 0 , 0 ]; " Test Mealy outputs for both
[ 0 , 1] -> [X01   , 0 , 1 ]; " values of X
[ .C. , 0] -> [X0    , .X. , .X. ]; " Test X01-->X0 (X==0)
[ .C. , 1] -> [X01   , .X. , .X. ]; " Get back to X01
[ .C. , 1] -> [X011  , .X. , .X. ]; " Test X01-->X011 (X==1)
```

Приступив к проверке, мы встретимся еще кое с чем новым: с необходимостью тестирования выходов Мили. Как видно из пятого и шестого векторов, не в каждом проверочном векторе нам нужен тактовый сигнал для перехода. Вместо этого мы можем удерживать тактовый сигнал равным нулю — при этом автомат будет оставаться в состоянии, в которое он попал при последнем переходе, — и посмотреть, какими будут сигналы на выходах Мили при двух значениях входного сигнала X. Затем проверяется переход в следующее состояние.

При переходе в новое состояние мы указываем ожидаемое состояние, но значения выходных сигналов отмечаем как безразличные. В правильно занесенном проверочном векторе должны быть указаны значения выходных сигналов, которые возникают *после* перехода, определяемые *следующим* состоянием. Хотя в нашем случае эти значения можно было бы найти и включить в таблицу, задача уже достаточно сложна; поэтому, чтобы избавить вас от головной боли, мы всюду проверяем выходные сигналы следующими векторами с CLOCK = 0.

Составление проверочных векторов для конечного автомата вручную – трудоемкий процесс, и, независимо от вашей старательности, нет никакой гарантии, что будут протестированы все функции автомата и найдены все возможные недостатки схемной реализации. Например, проверочными векторами в табл. 7.34 не проверяется комбинация $(A \text{ LASTA}) = 10$ в состоянии (LOOKING) и комбинация $(A \text{ В LASTA}) = 100$ в состоянии ОК. Таким образом, составление полного набора проверочных векторов для обнаружения возможных недостатков лучше всего поручить программе, автоматически генерирующей проверочные векторы. В табл. 7.35, после составления векторов для первых нескольких состояний, мы бросили это дело, оставив для вас завершение данной процедуры в виде задачи 7.92. Все же полезно бывает проверить работоспособность конструкции, написав хотя бы несколько векторов для проверки того, как автомат выполняет свои самые главные функции; это позволит выявить и исправить очевидные ошибки на ранней стадии разработки. Более тонкие погрешности проекта лучше обнаруживать путем детального моделирования на системном уровне.

7.12. Особенности проектирования последовательностных схем на языке VHDL

Большая часть средств, предоставляемых в языке VHDL и используемых при проектировании последовательностных схем, уже была введена нами в параграфе 4.7, в частности, процессы, и мы пользовались этими средствами в параграфах главы 5, связанных с употреблением языка VHDL. В данном параграфе мы познакомим вас еще с несколькими возможностями и приведем простые примеры того, как ими воспользоваться. Примеры проектирования более сложных схем будут даны в относящихся к языку VHDL параграфах главы 8.

7.12.1. Последовательностные схемы с обратной связью

Фундамент для работы с последовательностными схемами с обратной связью средствами языка VHDL образуют VHDL-процессы и механизм сниска событий моделирующей программы. Напомним, что состояние последовательностной схемы с обратной связью может измениться под воздействием входных сигналов, и переход в новое состояние проявляется в изменениях, распространяющихся по нити обратной связи до тех пор, пока в петле не наступит стабилизация. При моделировании переход от одного состояния к другому сопровождается занесением изменений сигналов в список событий и составлением расписания, по которому процессы запускаются вновь с элементарным сдвигом по времени; при этом изменения сигналов продолжаются до тех пор, пока их список не будет исчерпан.

В табл. 7.36 приведена VHDL-программа для SR-защелки. Структура содержит два параллельных оператора присваивания, каждый из которых запускает процесс, как это было объяснено в разделе 4.7.9. Взаимодействие этих процессов реализует одиночную процедуру защелкивания в SR-защелке.

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vsrlatch is
  port (S, R: in STD_LOGIC;
        Q, QN: buffer STD_LOGIC);
end Vsrlatch;

architecture Vsrlatch_arch of Vsrlatch is
begin
  QN <= S nor Q;
  Q  <= R nor QN;
end Vsrlatch_arch;

```

Табл. 7.36. Поточковая VHDL-программа для SR-защелки

Моделирование в среде VHDL является достаточно точным, чтобы справиться со случаем, когда одновременно поданы оба сигнала S и R. Самый интересный результат моделирования получается тогда, когда сигналы S и R снимаются одновременно. В первом из замечаний в разделе 7.2.1, вынесенных за пределы основного текста, уже объяснялось, что в этой ситуации в реальной SR-защелке могут начаться колебания, либо она может войти в метастабильное состояние. При моделировании это приведет к потенциально бесконечному циклу, в котором каждое исполнение одного из операторов присваивания будет запускать очередное исполнение другого. После некоторого числа повторений хорошее средство моделирования «раскусит» проблему — число элементарных сдвигов по времени растет, а время в модели стоит на месте — и остановит процесс моделирования.

НЕ ВОСПОЛЬЗОВАТЬСЯ ЛИ НАМ СИГНАЛОМ 'U'?

Конечно, было бы замечательно, если бы в модели SR-защелки, представленной в табл. 7.36, при одновременном переходе сигналов S и R на неактивный уровень вырабатывался выходной сигнал 'U', но ведь *этого* нет. Однако язык VHDL является достаточно мощным, чтобы разработчик, имеющий опыт работы с VHDL, мог легко описать модель, обладающую таким свойством. В подобной модели надо было бы воспользоваться средствами языка VHDL, позволяющими имитировать течение времени (мы не рассматриваем эти средства), чтобы учесть «время восстановления» защелки (см. второе замечание в разделе 7.2.1, вынесенное за пределы основного текста) и вырабатывать сигнал на выходе 'U', если второе изменение во входных сигналах происходит слишком близко по времени. Таким способом можно смоделировать даже максимально допустимое разрешенное время пребывания в состоянии метастабильности.

Заметьте, что в случае, когда у схемы есть возможность попасть в метастабильное состояние, нет гарантии, что моделирующая программа обнаружит это, особенно в больших проектах. Лучший способ избежать каких бы то ни было проблем с метастабильностью при проектировании систем заключается в ясном задании и защите асинхронных входов в соответствии с тем, как это обсуждается в параграфе 8.9.

7.12.2. Тактируемые схемы

На практике большинство устройств, проектируемых и моделируемых в среде VHDL, представляют собой тактируемые синхронные системы, в которых используются переключающиеся по фронту триггеры. В дополнение к тому, что вам уже известно о возможностях языка VHDL, для описания переключающегося по фронту элемента нам понадобится еще средство, а именно – *признак event* (*event attribute*), который можно присоединить к имени сигнала, чтобы получить переменную типа *boolean*, принимающую значение *true*, если то или иное событие в сигнале запускает охватывающий процесс в текущем цикле моделирования, и значение *false* – в противном случае.

Используя признак *event*, можно смоделировать поведение переключающегося по положительному фронту D-триггера с асинхронным входом сброса так, как это сделано в табл. 7.37. Здесь асинхронный сигнал CLR на входе сброса преобладает над тактовым входным сигналом CLK и поэтому проверяется первым в предложении “if”. Только тогда, когда сигнал на входе CLR имеет неактивный уровень, вступает в действие то, что предусмотрено предложением “elsif”, и имеющиеся в нем операторы исполняются по положительному фронту сигнала CLK. Заметьте, что величина “CLK’event” истинна при любом изменении сигнала CLK, поэтому для переключения только по положительному перепаду в сигнале CLK предусмотрена проверка “CLK = '1’”. Существует много других способов задать процесс или составить оператор, отражающие чувствительность к перепаду сигнала; еще два способа описания D-триггера (без входа сброса) приведены в табл. 7.38.

Табл. 7.37. Поведенческое описание переключающегося по положительному фронту D-триггера на языке VHDL

```
library IEEE;
use IEEE.std_logic_1164.all;

entity VposDff is
    port (CLK, CLR, D: in STD_LOGIC;
          Q, QN: out STD_LOGIC);
end VposDff;

architecture VposDff_arch of VposDff is
begin
    process (CLK, CLR)
    begin
        if CLR='1' then Q <= '0'; QN <= '1';
        elsif CLK'event and CLK='1' then Q <= D; QN <= not D;
        end if;
    end process;
end VposDff_arch;
```

При тестировании тактируемой схемы вам понадобится еще одна вещь: нужно будет генерировать системный тактовый сигнал. Это совсем легко реализовать, организовав цикл внутри процесса, как это показано в табл. 7.39 для тактовой частоты 100 МГц с коэффициентом заполнения 60%.

«ВНУТРЕННОСТИ» СИНТЕЗА

Вам, наверное, интересно узнать, как программные средства синтеза реализуют в настоящем триггере описание чувствительности к фронту, приведенное в табл. 7.37 и 7.38. Большинство программных средств распознает только небольшое число predetermined способов описания поведения схемы, переключающейся по фронту, и отображает их в predetermined компоненты внутри программируемой ИС.

Программа синтеза фирмы Synopsys (Synopsys synthesis engine) распознает используемое нами в этой книге выражение "CLK'event and CLK='1'" с помощью программного продукта Foundation Series 1.5 фирмы Xilinx. Но язык VHDL предоставляет также и другие возможности для выражения того же самого функционального поведения, что и в табл. 7.38. Питер Ашенден (Peter Ashenden), автор *Справочника проектировщика по языку VHDL (The Designer's Guide to VHDL, Morgan Kaufmann, 1996)*, испытал способность различных средств синтеза воспринимать три приведенные здесь формы задания чувствительности к перепаду и еще одну, немного модифицированную. Только одно из программных средств смогло синтезировать по трем из четырех форм; большинство испытанных программ синтеза справляется только с двумя. Следовательно, вам нужно использовать тот метод, который предписан имеющимися у вас программными средствами.

```
process
    wait until CLK'event and CLK='1';
    Q <= D;
end process;

Q <= D when CLK'event and CLK='1' else Q;
```

Табл. 7.38. Два других способа описания переключающегося по положительному фронту D-триггера

Табл. 7.39. Тактовый процесс для тестирования

```
architecture TB_arch of TB is
    signal MCLK: STD_LOGIC;
    signal ... -- Declare other input and output signals

    process -- Clock generator
    begin
        MCLK <= 1, -- Start at 1 at time 0
        loop
            MCLK <= 0 after 6 ns;
            MCLK <= 1 after 4 ns;
        end loop;
    end process;

    process -- Generate the rest of the input stimuli, check outputs
    begin
        ..
    end;
```

Обзор литературы

Проблема метастабильности остается популярной уже в течение долгого времени. Еще греческие философы в глубокой древности писали о перешителности. О колебаниях и неуверенности поет и группа современных философов под пазванием Devo в заглавной песне их альбома *Свобода выбора*. Наконец, конгресс США все не может решить, как «сберечь» Обществениую Безонаспость.

Возможность опроса впервые была реализована несколько десятилетий назад на зашелках, не па триггерах, в разработках фирмы IBM в области интегральных микросхем. Замечательное рассмотрение примененного тогда приципа и других методов опроса содержится в книге Мак-Класки *Принципы логического проектирования* (Edward J. McCluskey. *Logic Design Principles*. Prentice Hall, 1986).

В большинстве случаев при разработке специализированных ИС, а также при проектировании устройств на основе СИС, ПЛИУ и ИС типа FPGA используются последовательностные схемы, рассмотренные в этой главе. Однако существуют и другие типы последовательностных схем, применявшиеся в ранних разработках на дискретных элементах (в том числе еще во времена логических схем на электронных лампах) и применяемые сегодня в современных заказных СБИС.

Тактируемые синхронные конечные автоматы представляют собой особый случай более широкого класса *импульсных схем* (схем, работающих в импульсном режиме; *pulse-mode circuit*). У таких схем имеется один или большее число *импульсных входов* (*pulse inputs*) и выполнены следующие условия: (а) только один импульс может поступить в каждый момент времени; (б) сигналы на неимпульсных входах остаются постоянными в момент действия импульса; (с) только импульсы могут вызвать изменение состояния; (d) один импульс вызывает самое большее однократное изменение состояния. В тактируемых синхронных конечных автоматах единственным импульсным входом является вход тактового сигнала и роль «импульса» играет переключающий фронт тактового сигнала. Однако можно построить схемы с несколькими импульсными входами и использовать другие элементы памяти, а не знакомые нам переключающиеся по фронту триггеры. Все эти возможности детально разбираются в упомянутой книге Мак-Класки.

Важным частным случаем импульсной схемы является *двухфазный автомат-защелка* (*two-phase latch machine*), рассматриваемый Мак-Класки и другими. Обоснование двухфазного тактирования в СБИС можно найти в книге Мида и Конвея *Введение в системы СБИС* (Carver Mead, Lynn Conway. *Introduction to VLSI Systems*. Addison-Wesley, 1980). В таких автоматах в принципе исключен существенный источник опасности, имеющий место в переключающихся по фронту триггерах, благодаря использованию двух защелок, для которых сигналами разрешения являются неперекрывающиеся тактовые сигналы.

Методы сокращения полностью и не полностью заданных таблиц состояний описаны в учебниках по логическому проектированию, предназначенных для более углубленного изучения предмета, в том числе – в указанной книге Мак-Класки. Рассмотрение этих методов и других «теоретических» аспектов проектирования последовательностных автоматов с математической точки зрения содержится в книге Кохави *Теория переключений и конечные автоматы* (Zvi Kohavi. *Switching and Finite Automata Theory*, second edition. McGraw-Hill, 1978).

Как мы увидели в этой главе, в неправильно составленной диаграмме состояний могут быть неоднозначности в отношении переходов к следующему состоянию. Структуры “IF-THEN-ELSE” в языках описания схем типа ABEL и VHDL позволяют исключить неоднозначности, но первоначально такие конструкции предназначались не для этого. За последние 25 лет получила распространение символика *алгоритмических автоматов* (*algorithmic state machine, ASM*), являющихся блок-схемным эквивалентом вложенных операторов IF-THEN-ELSE. Пионером в области блок-схем ASM (*ASM chart*) был Томас Осборн (Thomas E. Osborne) из Hewlett-Packard Laboratories, а затем эти идеи были развиты коллегой Осборна Клейром в книге *Проектирование логических систем с помощью конечных автоматов* (Christopher R. Clare. *Designing Logic Using State Machines*. McGraw-Hill, 1973). Впоследствии методы проектирования и синтеза, основанные на использовании блок-схем ASM, прочно утвердились во многих учебниках по цифровому проектированию, включая *Искусство цифрового проектирования* Проссера и Уинкеля (F.P. Prosser, D.E. Winkel. *Art of Digital Design*. Prentice-Hall, 1987) и *Цифровое проектирование* Мано (M. Morris Mano. *Digital Design*. Prentice-Hall, 1984), а также первые два издания книги, которую вы читаете.

Другой способ описания конечных автоматов, являющийся расширением «традиционного» представления диаграмм с мнемонической записью состояний [mnemonic documented state (MDS) diagrams], был развит Флетчером в книге *Инженерный подход к проектированию цифровых устройств* (William I. Fletcher. *An Engineering Approach to Digital Design*. Prentice-Hall, 1980). Сегодня в значительной мере на смену блок-схемам ASM и MDS-диаграммам пришли языки описания схем и их компиляторы.

Во многих системах автоматизированного проектирования имеются графические средства ввода диаграмм состояний. К сожалению, эти средства обычно поддерживают только традиционное представление диаграмм состояний, при котором проектировщику легко ошибиться и допустить неоднозначное описание переходов. Поэтому моя личная рекомендация вам – воздержаться от использования редакторов диаграмм состояний в пользу описания конечных автоматов на одном из языков описания схем.

Мы упомянули о важности синхронизирующих последовательностей в связи с проверочными векторами для конечных автоматов. На самом деле существует хорошо развитая теория и почти забытая практика применения синхронизирующих последовательностей и «поиска исходного состояния» (“homing experiments”); они описаны Хенпи в книге *Модели логических автоматов с конечным числом состояний* (Frederick C. Henne. *Finite-State Models for Logical Machines*. Wiley, 1968). Если на вашей книжной полке нет этой классики и если вы не знаете, как воспользоваться тем, что там написано, то, пожалуйста, не забывайте предусмотреть вход сброса в каждом создаваемом вами конечном автомате!

Упражнения

- 7.1. Приведите три примера метастабильности из повседневной жизни, помимо упомянутых в этой главе.
- 7.2. Для SR-защелки типа той, которая изображена на рис. 7.5, нарисуйте временную диаграмму выходного сигнала при входных воздействиях, указанных на рис. X7.2. Предположите, что длительность нарастающего и спадающего фронтов на входах и выходах равна пулю, что задержка сигнала при прохождении через вентиль ИЛИ-НЕ равна 10 нс и что каждое деление по оси времени на рис. X7.2 также равняется 10 нс.



Рис. X7.2.

- 7.3. Повторите упражнение 7.2 для входных сигналов, изображенных на рис. X7.3. Возможно, результат покажется вам невероятным, но такая ситуация может на самом деле возникнуть в реальном устройстве, если длительность переходов в сигналах сравнима с задержками их распространения.

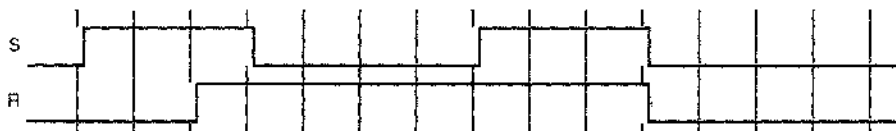
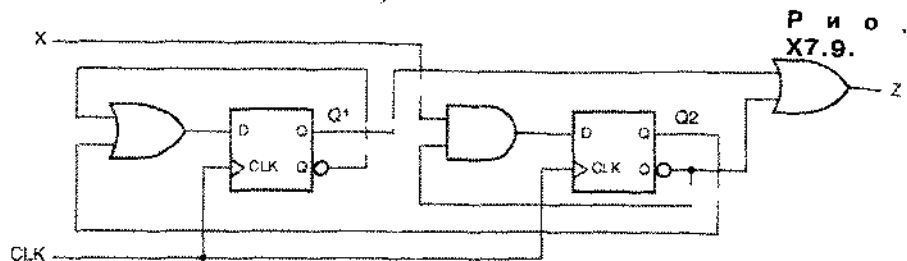


Рис. X7.3.

- 7.4. На рис. 7.34 показано, как из D-триггера и комбинационной логической схемы образовать T-триггер с входом разрешения. Покажите, как на основе T-триггера с входом разрешения и комбинационной логической схемы построить D-триггер.
- 7.5. Покажите, как построить JK-триггер, используя T-триггер с входом разрешения и комбинационную логическую схему.
- 7.6. Покажите, как настроить SR-защелку на одном переключающемся по положительному фронту D-триггере типа 74х74 без использования других компонентов.
- 7.7. Покажите, как настроить триггер, эквивалентный непереклюющемуся по положительному фронту JK-триггеру типа 74х109, на основе непереклюющегося по положительному фронту D-триггера типа 74х74, используя один или большее число вентилей из набора 74х00.
- 7.8. Покажите, как построить триггер, эквивалентный переключающемуся по положительному фронту D-триггеру типа 74х74, на переключающемся по положительному фронту JK-триггере типа 74х109 без использования других компонентов.

- 7.9. Проанализируйте тактируемый синхронный конечный автомат, изображенный на рис. X7.9. Запишите уравнения возбуждения, таблицу возбуждения/переход и таблицу состояние/выход (используя имена состояний A–D для комбинаций $Q_1 Q_2 = 00–11$).



- 7.10. Повторите упражнение 7.9, поменяв в принципиальной схеме местами вентили И и ИЛИ. Объясните, является ли новая таблица состояние/выход «двойственной» по отношению к такой же первоначальной таблице.
- 7.11. Нарисуйте диаграмму состояний для конечного автомата, описываемого таблицей 7.6.
- 7.12. Нарисуйте диаграмму состояний для конечного автомата, описываемого таблицей 7.12.
- 7.13. Нарисуйте диаграмму состояний для конечного автомата, описываемого таблицей 7.14.
- 7.14. Составьте таблицу состояние/выход, эквивалентную диаграмме состояний, изображенной на рис. X7.14. Заметьте, что диаграмма нарисована в предположении, что состояния изменяются только при таких комбинациях входных сигналов, которые указаны явно.

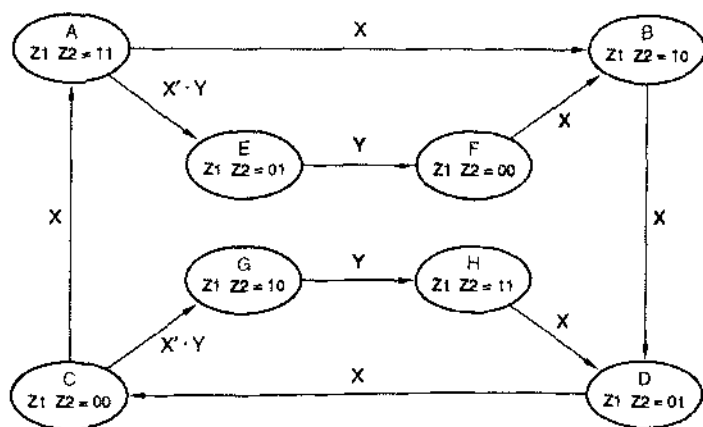
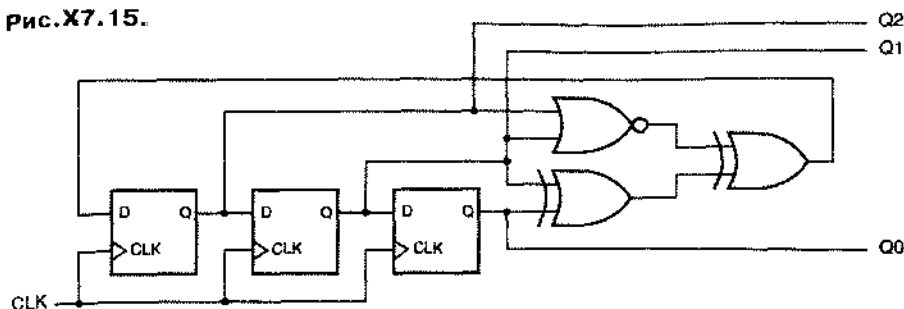


Рис. X7.14.

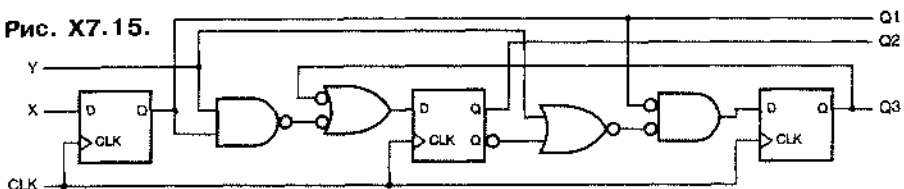
- 7.15. Проанализируйте тактируемый синхронный конечный автомат, изображенный на рис. X7.15. Запишите уравнения возбуждения, таблицу возбуждения/переход и таблицу состояние/выход (используя имена состояний A–H для комбинаций $Q_2 Q_1 Q_0 = 000–111$).

Рис. X7.15.



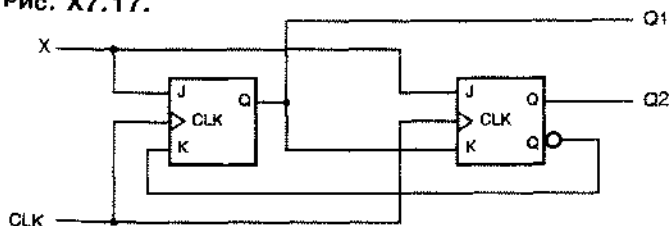
7.16. Проанализируйте тактируемый синхронный конечный автомат, изображенный на рис. X7.16. Запишите уравнения возбуждения, таблицу возбуждения/переход и таблицу состояние/выход (используя имена состояний A–H для комбинаций $Q1\ Q2\ Q3 = 000–111$).

Рис. X7.15.

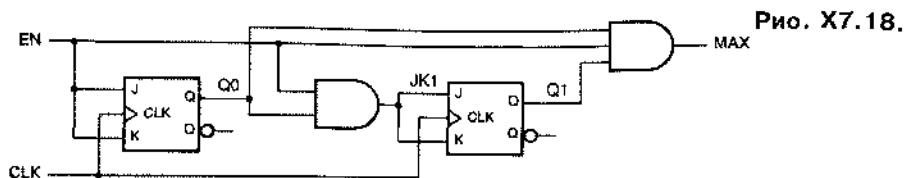


7.17. Проанализируйте тактируемый синхронный конечный автомат, изображенный на рис. X7.17. Запишите уравнения возбуждения, уравнения переходов, таблицу переходов и таблицу состояние/выход (используя имена состояний A–D для комбинаций $Q1\ Q2 = 00–11$). Нарисуйте диаграмму состояний, а также временные диаграммы для сигналов CLK, X, Q1 и Q2 на интервале времени в 10 периодов тактового сигнала в предположении, что вначале автомат находится в состоянии 00 и сигнал X в течение всего времени остается равным 1.

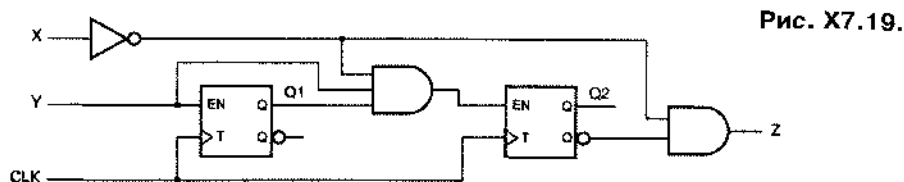
Рис. X7.17.



7.18. Проанализируйте тактируемый синхронный конечный автомат, изображенный на рис. X7.18. Запишите уравнения возбуждения, уравнения переходов, таблицу переходов и таблицу состояние/выход (используя имена состояний A–D для комбинаций $Q1\ Q0 = 00–11$). Нарисуйте диаграмму состояний, а также временные диаграммы для сигналов CLK, EN, Q1 и Q0 на интервале времени в 10 периодов тактового сигнала в предположении, что вначале автомат находится в состоянии 00 и сигнал EN в течение всего времени остается равным 1.

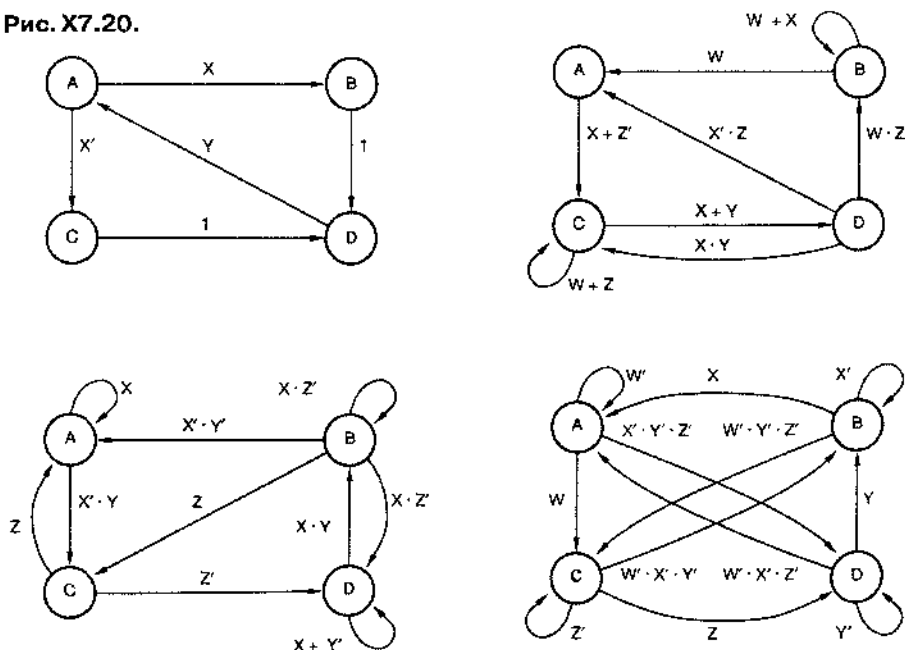


7.19. Проанализируйте тактируемый синхронный конечный автомат, изображенный на рис. X7.19. Запишите уравнения возбуждения, таблицу возбуждения/переход и таблицу состояние/выход (используя имена состояний A–D для комбинаций $Q_1 Q_2 = 00–11$).



7.20. Во всех диаграммах состояний на рис. X7.20 есть неопределенности. Перечислите их все для каждой из диаграмм состояний. (Указание: Воспользуйтесь картами Карно и найдите в них не покрытые или дважды покрытые комбинации входных сигналов.)

Рис. X7.20.



7.21. Синтезируйте схему по диаграмме состояний на рис. 7.64, используя шесть переменных для кодирования состояний, таких что выходные сигналы LA–LC и RA–RC равны значениям самих переменных состояния. Запишите список

переходов, выражения переходов для каждой переменной состояния в виде суммы р-термов и упрощенные уравнения переход/возбуждение в случае реализации на D-триггерах. Нарисуйте принципиальную схему на основе ИС малой и средней степени интеграции.

- 7.22. На основании списка переходов в табл. 7.18 найдите минимальное выражение вида «сумма произведений» для величины $Q2^*$ в предположении, что состояния, следующие за неиспользуемыми состояниями, являются безразличными.
- 7.23. Видоизмените диаграмму состояний на рис. 7.64 так, чтобы автомат переходил в аварийный режим сразу же, если во время поворота одновременно подаются сигналы LEFT и RIGHT. Занижите соответствующий список переходов.

Задачи

- 7.24. Рассматривая поведение петли обратной связи в D-защелке, объясните, как в ней возникает метастабильность, когда требования в отношении времени установления и времени удержания не удовлетворяются.
- 7.25. Чему равно минимальное время установления для переключающегося по импульсу триггера типа двухтактного JK- или SR-триггера? (Указание: Это время зависит от определенных характеристик тактового сигнала.)
- 7.26. Опишите ситуацию, отличную от состояния метастабильности, когда выходные сигналы Q и QN переключающегося по фронту D-триггера типа 74x74 могут сколь угодно долго не быть дополнениями друг друга.
- 7.27. Сравните схему, изображенную на рис. X7.27, с D-защелкой на рис. 7.12. Докажите, что эти схемы функционируют совершенно одинаково. Схема на рис. X7.27 применяется во многих серийных D-защелках. В каком смысле эта схема лучше?

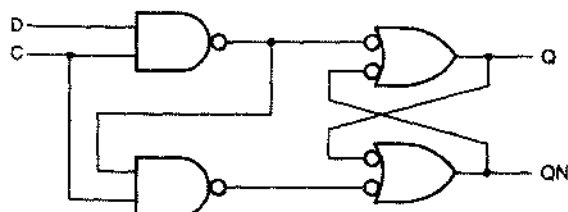


Рис. X7.27.

- 7.28. Предположим, что в тактируемом синхронном конечном автомате, изображенном на рис. 7.35, в качестве элементов памяти используются D-защелки с высоким активным уровнем входного сигнала C. Какие соотношения должны выполняться между перечисленными ниже временными параметрами, чтобы переходы из состояния в состояние происходили успешно:

- t_{Fmin}, t_{Fmax} — минимальная и максимальная задержки распространения по логике переходов;
- t_{CQmin}, t_{CQmax} — минимальная и максимальная задержки в D-защелке от тактового входа до выхода;
- t_{DQmin}, t_{DQmax} — минимальная и максимальная задержки в D-защелке от входа данных до выхода;

- $t_{\text{setup}}, t_{\text{hold}}$ – время установления и время удержания D-зашелки;
 t_H, t_L – длительность отрезков времени, в течение которых тактовый сигнал удерживается на высоком и на низком уровне.

- 7.29.** Постройте заново конечный автомат из упражнения 7.9, используя только три вентиля И-НЕ или ИЛИ-НЕ и не применяя инверторов.
- 7.30.** Нарисуйте диаграмму состояний для тактируемого синхронного конечного автомата с двумя входами INIT и X и одним выходом Z типа Мура. Когда подается сигнал INIT, выходной сигнал постоянен и равен 0. Когда сигнал INIT переходит на неактивный уровень, сигнал Z продолжает оставаться равным 0 до тех пор, пока на двух последовательных тактах сигнал X не будет оставаться равным 0 или 1 независимо от того, в каком порядке это случится. Когда это произойдет, сигнал Z станет равным 1 и будет оставаться равным 1 до тех пор, пока снова не будет подан сигнал INIT. Диаграмму состояний следует нарисовать аккуратно и она должна иметь планарный вид (линии не должны пересекаться). (Указание: Потребуется не более 10 состояний.)
- 7.31.** Повторите задачу 7.30, описав диаграмму состояний на языке ABEL.
- 7.32.** Постройте тактируемый синхронный конечный автомат для контроля четности в линии последовательной передачи данных. Кроме входа для тактового сигнала CLOCK у схемы должно быть два входа SYNC и DATA и один выход ERROR типа Мура. Составьте таблицу состояние/выход, предусматривающую только четыре состояния, указав смысловое значение каждого состояния. Выберите для состояний 2-разрядные коды, запишите уравнения переходов и возбуждения и нарисуйте принципиальную схему. Вы можете использовать в вашей схеме D-триггеры, JK-триггеры или по одному триггеру каждого из этих типов.
- 7.33.** Повторите задачу 7.32, выполнив проектирование на языке ABEL применительно к ПЛУ GAL16V8.
- 7.34.** Постройте тактируемый синхронный конечный автомат на D-триггерах с таблицей состояние/выход X7.34. Используйте две переменные состояния Q1 Q2, полагая, что состояния кодируются следующим образом: A = 00, B = 01, C = 11, D = 10.

X			
S	0	1	Z
A	B	D	0
B	C	B	0
C	B	A	1
D	B	C	0
S*			

Табл. X7.34.

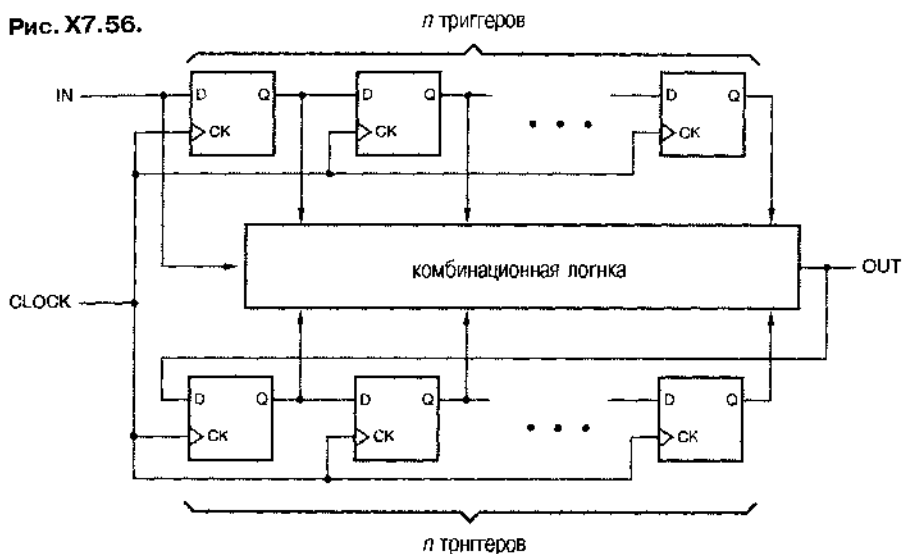
- 7.35.** Повторите задачу 7.34 на JK-триггерах.

- 7.36. Напишите новую таблицу переходов и выведите уравнения возбуждения и выхода минимальной стоимости для схемы с таблицей состояний 7.6, используя «простейшее» кодирование состояний из табл. 7.7 и имея в виду применение D-триггеров. Сравните стоимость ваших логики возбуждения и логики выхода (при их реализации на двухуровневой структуре И-ИЛИ) со стоимостью логики в схеме, представленной на рис. 7.54.
- 7.37. Повторите задачу 7.36, используя «почти прямое» кодирование состояний из табл. 7.7.
- 7.38. Предположим, что конечный автомат, представленный на рис. 7.54, должен быть построен на D-триггерах типа 74LS74. Какие сигналы следует подавать на входы триггера установки в единичное состояние и сброса?
- 7.39. Напишите новые таблицы переходов и возбуждения и выведите уравнения возбуждения и выхода минимальной стоимости для схемы с таблицей состояний 7.6, используя «простейшее» кодирование состояний из табл. 7.7 и имея в виду применение JK-триггеров. Сравните стоимость ваших логики возбуждения и логики выхода (при их реализации на двухуровневой структуре И-ИЛИ) со стоимостью логики в схеме, представленной на рис. 7.56.
- 7.40. Повторите задачу 7.39, используя «почти прямое» кодирование состояний из табл. 7.7.
- 7.41. Составьте таблицу использования, подобную табл. 7.10, для каждого из следующих триггеров: (а) SR-триггер; (б) T-триггер с входом разрешения; (с) D-триггер с входом разрешения. Применительно к одному из этих триггеров рассмотрите проблему единственности, с которой вы столкнетесь, попытавшись наиболее эффективным образом использовать безразличные значения сигналов.
- 7.42. Составьте новую таблицу возбуждения и выведите уравнения возбуждения и выхода минимальной стоимости для конечного автомата, заданного табл. 7.8, в случае применения T-триггеров с входом разрешения (рис. 7.33). Сравните стоимость ваших логики возбуждения и логики выхода (при их реализации на двухуровневой структуре И-ИЛИ) со стоимостью логики в схеме, приведенной на рис. 7.54.
- 7.43. Напишите полную таблицу состояний для всех 8 возможных состояний схемы, приведенной на рис. 7.54. Присвойте неиспользуемым состояниям имена U_1 , U_2 и U_3 (001, 010 и 011). Нарисуйте диаграмму состояний и объясните поведение схемы при попадании в неиспользуемые состояния.
- 7.44. Повторите задачу 7.43 для схемы на рис. 7.56.
- 7.45. Напишите таблицу переходов для неминимизированной таблицы состояний на рис. 7.51(а), которая получится в предположении, что состояния кодируются в порядке двоичного счета: $INI\bar{T}-OKA1 = 000-110$. Запишите соответствующие уравнения возбуждения для D-триггеров, разместив неиспользуемое состояние 111 так, чтобы стоимость была минимальной. Сравните по стоимости реализации ваши уравнения с уравнениями минимальной стоимости для минимизированной таблицы состояний, описанной в тексте.
16. Напишите таблицу использования для T-триггера с входом разрешения.

- 7.47.** Во многих приложениях во время начальной установки и сразу после нее па выходе конечного автомата вырабатываются ничего не значащие сигналы, и должно пройти некоторое время после снятия сигнала пачальной установки, прежде чем автомат пачнет вести себя нравильно. Если допустить это, то у автомата, задаваемого таблицей 7.6, состояние INIT можно исключить и для кодирования остающихся четырех состояний будет достаточно только двух переменных состояния. Постройте конечный автомат заново, используя эту идею. Нанишите новые таблицы состояний, переходов и возбуждения для D-триггеров, а также уравнения возбуждения и выхода минимальной стоимости. Сравните стоимость новой схемы со стоимостью схемы на рис. 7.54.
- 7.48.** Повторите задачу 7.47 в случае использования JK-триггеров и сравните результат по стоимости со схемой на рис. 7.56.
- 7.49.** Постройте заново описываемый таблицей 7.12 автомат для подсчета единиц при условии, что состояния кодируются в порядке двоичного счета ($S_0-S_3 = 00, 01, 10, 11$). Сравните по стоимости реализации результирующие выражения для сигналов возбуждения вида «сумма произведений» с выражениями, полученными в тексте.
- 7.50.** Повторите задачу 7.49 в случае использования JK-триггеров.
- 7.51.** Повторите задачу 7.49 в случае использования T-триггеров с входом разрешения.
- 7.52.** Постройте заново описываемый таблицей 7.12 автомат для подсчета единиц, описав диаграмму состояний на языке ABEL. Попытайтесь найти такое кодирование состояний, при котором полное число термов-произведений было бы минимальным, в предположении, что можно использовать любую полярность выходных сигналов. Сколько различных способов кодирования состояний вам при этом необходимо проверить?
- 7.53.** Постройте заново описываемый таблицей 7.14 кодовый замок при кодировании состояний в порядке, задаваемом кодом Грея ($A-H = 000, 001, 011, 010, 110, 111, 101, 100$). Сравните по стоимости реализации выражения для сигналов возбуждения вида «сумма произведений» с выражениями, полученными в тексте.
- 7.54.** Найдите способ кодирования состояний кодового замка, описываемого таблицей 7.14, 3-разрядными кодами, при котором стоимость реализации уравнений возбуждения была бы меньшей по сравнению с тем, что получено в тексте. (Указание: Используйте тот факт, что в требуемой последовательности биты в позициях 1–3 совпадают с битами в позициях 4–6.)
- 7.55.** Какие изменения произошли бы в уравнениях возбуждения и выхода для кодового замка из раздела 7.4.6 в результате выполнения формальной процедуры минимизации (раздел 4.3.8) в задаче с пятью функциями? При этом не нужно составлять 31 карту Карно для функций-произведений и проходить всю процедуру от начала до конца; постарайтесь, «разглядывая» карты возбуждения и выхода, приведенные в разделе 7.4.6, увидеть, на чем можно сэкономить.

- 7.56. Выходной сигнал автомата с конечной памятью (*finite-memory machine*) полностью определяется текущими значениями сигналов на его входах и его входными и выходными сигналами в течение последних n периодов тактового сигнала, где n – конечное, ограниченное натуральное число. Любой автомат, который может быть реализован так, как показано на рис. X7.56, является автоматом с конечной памятью. Заметьте, что автомат с конечным числом состояний не обязательно должен быть автоматом с конечной памятью; например, у счетчика по модулю n с входом разрешения и выходом “MAX” всего n состояний, но значение сигнала на его выходе может зависеть от значений сигнала на входе разрешения в каждом периоде тактового сигнала, начиная с момента инициализации. Покажите, как реализовать кодовый замок, задаваемый таблицей 7.14, в виде автомата с конечной памятью.

Рис. X7.56.



- 7.57. Синтезируйте схему по диаграмме состояний с неоднозначностью, представленной на рис. 7.62. Воспользуйтесь кодированием состояний из табл. 7.16. Напишите список переходов, выражение возбуждения для каждой переменной состояния в виде суммы р-термов и упрощенные уравнения переход/возбуждения при реализации из D-триггерах. Найдите, каким будет на самом деле состояние, следующее за состоянием IDLE, для каждой из следующих комбинаций входных сигналов (LEFT, RIGHT, HAZ): (1, 0, 1), (0, 1, 1), (1, 1, 0), (1, 1, 1). Прокомментируйте поведение автомата в этих случаях.
- 7.58. Предположим, что в некоторой содержащей неопределенность диаграмме состояний для состояния SA и комбинации входных сигналов I указали два следующих состояния SB и SC. Каким фактически будет следующее состояние SD при таком переходе, зависит от реализации конечного автомата. Объясните, каково соотношение между кодами состояний SB, SC и SD, если конечный автомат строится на D-триггерах и уравнения переход/возбуждения получаются по правилу: $V^* = \sum \text{p-термы с } V^* = 1$.

- 7.59. Повторите задачу 7.58 в предположении, что уравнения переход/возбуждение получаются по правилу: $V^* = \sum p\text{-термы}$ с $V^* = 0$.
- 7.60. Предположим, что в некоторой содержащей неопределенность диаграмме состояний для состояния SA и комбинации входных сигналов I не указано следующее состояние. Каким будет фактически следующее состояние SD при таком переходе, зависит от реализации конечного автомата. Предположим также, что конечный автомат строится на D-триггерах и уравнения переход/возбуждение получаются по правилу: $V^* = \sum p\text{-термы}$ с $V^* = 1$. Объясните, что представляет собой код состояния SD.
- 7.61. Повторите задачу 7.60 в предположении, что уравнения переход/возбуждение получаются по правилу: $V^* = \sum p\text{-термы}$ с $V^* = 0$.
- 7.62. Как вывести уравнения возбуждения для входных сигналов S и R при заданных уравнениях переходов в тактируемом синхронном конечном автомате, который должен быть построен на двухтактных SR-триггерах? (Указание: Покажите, что любое уравнение перехода вида $Q_i^* = \text{expr}$ можно записать в виде $Q_i^* = Q_i \cdot \text{expr1} + Q_i' \cdot \text{expr2}$, и посмотрите, к чему это приводит.)
- 7.63. Повторите задачу 7.62 для JK-триггеров. Как задать безразличные значения, когда они возможны, при проектировании автомата на JK-триггерах?
- 7.64. Нарисуйте принципиальную схему логики выхода в заданном таблице 7.18 автомате для игры на угадывание, воспользовавшись одним двойным дешифратором 2x4 типа 74x139. (Указание: Выходные сигналы должны иметь низкий активный уровень.)
- 7.65. Что означает надпись на персональном померном знаке на рис. 7.60? (Указание: Это старый номерной знак автора, запись OTTFSS в духе инженера-компьютерщика.)
- 7.66. Проанализируйте последовательностную схему с обратной связью, изображенную на рис. 7.19, предположив, что входные сигналы PR_L и CLR_L равны 1. Выведите уравнения возбуждения, составьте таблицу переходов и проверьте, нет ли в таблице переходов критических и некритических гонок. Назовите состояния какими-нибудь именами и нарисуйте таблицы состояние/выход и поток/выход. Покажите, что таблица потока эквивалентна таблице на рис. 7.85.
- 7.67. Нарисуйте принципиальную схему устройства с одной петлей обратной связи, не являющегося последовательностным устройством. Другими словами, сигнал на выходе схемы должен быть функцией только текущего значения сигнала на ее входе. Для проверки правильности вашего ответа, разорвите петлю обратной связи и проанализируйте схему, как если бы она была последовательностной схемой с обратной связью, чтобы убедиться, что при каждой входной комбинации выходной сигнал не зависит от «состояния» схемы.
- 7.68. Из вентиля NBUT можно сделать BUT-флип (BUT flop), показанный на рис. X7.68. [Вентиль NBUT (NBUT gate) -- это просто вентиль BUT с инверсными выходами; определение вентиля BUT см. в задаче 5.31.] На основании анализа BUT-флота как последовательностной схемы с обратной связью найдите уравнения возбуждения и нарисуйте таблицы переходов и потока. Голится эта схема для чего-нибудь или, собрав такую схему, вы совершите промах (flop)?

7.69. Повторите задачу 7.68 для BUT-флота, изображенного на рис. X7.69

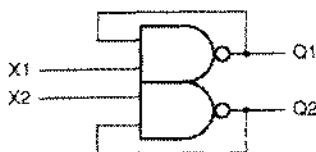


Рис. X7.68.

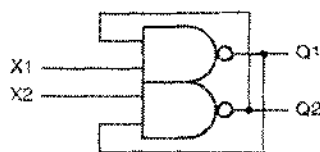


Рис. X7.69.

7.70. Умный студент построил нентиль BUT так, как показано на рис. X7.70. Однако эта схема не всегда работала правильно. Проанализируйте эту схему и объясните причину сбоев.

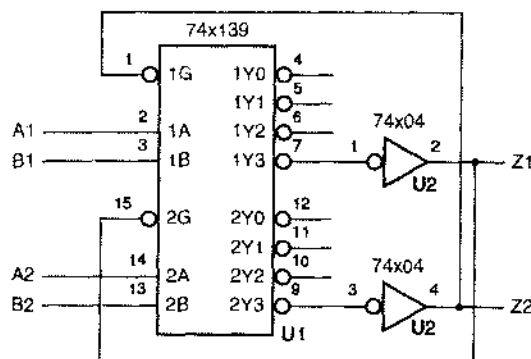


Рис. X7.70.

- 7.71. Проанализируйте последовательностную схему с обратной связью, изображенную на рис. X7.71. Разорвите цепи обратной связи, напишите уравнения возбуждения и составьте таблицу переходов и выхода, глядя на которую можно было бы увидеть устойчивые состояния в целом. Какое применение может быть у такой схемы?
- 7.72. Покажите, что 4-разрядный сумматор чисел в обратном коде с циклическим переисом является последовательностной схемой с обратной связью.
- 7.73. Выполните анализ переключающегося но положительному фронту D-триггера, изображенного на рис. 7.86, составив таблицы переход/выход, состояние/выход и поток/выход. Покажите, что функционирование этой схемы эквивалентно поведению D-триггера, приведенного на рис. 7.78.
- 7.74. В разделе 7.10.1 утверждается, что уравнение возбуждения любой последовательностной схемы с единственной обратной связью имеет вид:

$$Q^* = (\text{нынуждающийся член}) + (\text{удерживающий член}) \cdot Q.$$

Почему не бывает практических схем, у которых в уравнении возбуждения указанного вида Q было бы заменено на Q' ?

- 7.75. Промоделируйте работу зашелки, изображенной на рис. 7.88(b), в предположении, что выполнены условия, указанные в тексте, относящемся к этому рисунку. Сделайте это с помощью моделирующей программы в режиме единичных задержек, либо вручную при задержке в 1 нс на каждый

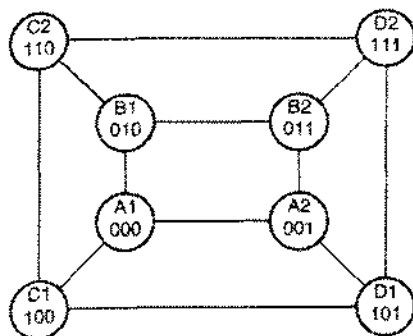


Рис. X7.79.

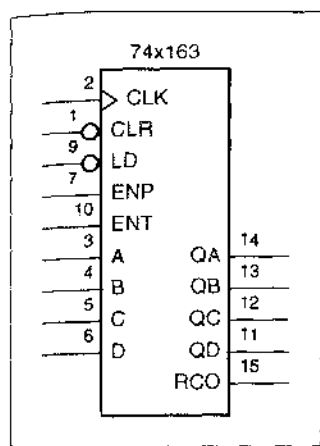
- 7.80. Составьте таблицу потока для ловушки импульсов, подобной схеме классического образца, описанной в разделе 7.10.2, с тем отличием, что новая схема должна обнаруживать в сигнале Р оба перехода из 0 в 1 и из 1 в 0.
- 7.81. Составьте таблицу потока для схемы классического образца, которая представляет собой D-триггер, переключающийся по двум фронтам. В таком триггере значения входных сигналов принимаются во внимание, а выходные сигналы изменяются в моменты времени, задаваемые обоими фронтами тактового сигнала.
- 7.82. Составьте таблицу потока для схемы классического образца с двумя входами EN и CLKIN и одним выходом CLKOUT, которая ведет себя следующим образом. Если сигнал EN имеет активный уровень на всем данном периоде тактового сигнала, то сигнал CLKOUT «включен» на всем следующем периоде тактового сигнала; другими словами, он должен быть тождественен сигналу CLKIN. (Здесь под периодом тактового сигнала понимается интервал времени между последовательными нарастающими фронтами сигнала CLKIN.) Если сигнал EN имеет неактивный уровень на всем данном периоде тактового сигнала, то сигнал CLKOUT должен быть «выключен» (и равен 1) в течение следующего периода тактового сигнала. Если в течение данного периода тактового сигнала в сигнале EN происходит переключение, то сигнал CLKOUT должен быть включен на следующем периоде тактового сигнала, если перед этим он был выключен, и – выключен, если до этого он был включен. После того как вы напишете таблицу потока классического образца, сократите ее, объединяя «совместимые» состояния там, где это возможно.
- 7.83. Постройте схему, удовлетворяющую условиям задачи 7.82, на переключающихся по фронту D-триггерах (74х74) или JK-триггерах (74х109) и микросхемах И-НЕ и ИЛИ-НЕ без обратных связей. Нарисуйте полную принципиальную схему и опишите словами, как именно в вашей схеме достигается желательное функционирование.
- 7.84. Имеется ли неустойчивость у одной или у обеих схем из двух предыдущих задач, и при каких условиях она проявляется?
- 7.85. Для таблицы потока X7.85 найдите способ кодирования состояний, позволяющий избежать всех критических гонок. Вы можете вводить дополнительные состояния по мере необходимости, используя при этом возможно меньшее

число переменных состояния. Пусть состоянию A соответствует комбинация из всех нулей. Начертите диаграмму смежности для исходной таблицы потока и составьте модифицированную таблицу потока и новую диаграмму смежности, которая могла бы служить доказательством того, что предлагаемый вам способ кодирования состояний удовлетворяет условиям задачи.

S	X Y			
	00	01	11	10
A	B	C	—	(A)
B	(B)	E	—	(B)
C	F	(C)	—	E
D	(D)	F	—	B
E	D	(E)	—	(E)
F	(F)	(F)	—	A

Табл. X7.85.

- 7.86. Триггер, у которого значение входного сигнала (или сигналов) принимается во внимание, а выходной сигнал (или сигналы) изменяется только на нарастающем фронте тактового сигнала CLK, рассматривается как схема классического образца. Докажите, что таблица потока для любого такого триггера содержит существенный источник опасности.
- 7.87. Укажите, где находится существенный источник опасности (или источники) в таблице потока переключающегося по положительному фронту D-триггера на рис. 7.85.
- 7.88. Найдите существенные источники опасности (если они есть) в таблице потока, полученной в задаче 7.81.
- 7.89. Найдите существенные источники опасности (если они есть) в таблице потока, полученной в задаче 7.82.
- 7.90. Придумайте словесный перевертыш — логическую головоломку, для которой можно найти правильный ответ одним из двух способов в зависимости от порядка слов. Как можно воспользоваться таким механизмом на политической арене?
- 7.91. Видоизмените программу на языке ABEL, приведенную в табл. 7.27, таким образом, чтобы кодирование состояний осуществлялось выходными сигналами и, тем самым, необходимое полное число выходов ПЛУ уменьшилось бы на единицу.
- 7.92. Закончите составление проверочных векторов, начатое в табл. 7.35, для кодового замка из табл. 7.31. Полный набор векторов должен тестировать все переходы из одного состояния в другое и значения выходных сигналов при всех возможных состояниях и значениях сигнала на входе.



ПРАКТИЧЕСКАЯ РАЗРАБОТКА СХЕМ ПОСЛЕДОВАТЕЛЬНО- СТНОЙ ЛОГИКИ

Цель этой главы состоит в том, чтобы познакомить вас с самыми распространенными и наиболее надежными методами проектирования последовательностных схем. Поэтому особый уклон будет сделан на *синхронные системы*, то есть на такие системы, в которых все триггеры переключаются одним и тем же общим тактовым сигналом. Хотя весь мир и не маршрутирует в такт с одним общим сигналом времени, в пределах одной цифровой системы или подсистемы это вполне осуществимо. Чуть позднее мы покажем, что при соединении цифровых систем или подсистем, у которых тактовые сигналы различны, можно выделить ограниченное число асинхронных сигналов и обрабатывать их особым способом.

Глава начинается с краткого обзора стандартов на документацию, которой сопровождаются последовательностные схемы. Сначала мы снова обратимся к самым главным составным блокам в последовательностных схемах — к защелкам и триггерам, а затем рассмотрим некоторые более гибкие узлы — последовательностные ПЛУ. Будет показано, как в ИС средней степени интеграции и в ПЛУ реализуются счетчики и регистры сдвига, и продемонстрировано их применение. Наконец, мы покажем, как эти элементы объединяются в синхронные системы и как при этом обрабатываются неизбежно асинхронные входные сигналы.

8.1. Стандарты документации на последовательностные схемы

8.1.1. Общие требования

Стандарты на документацию, относящиеся к названиям сигналов, условным обозначениям и компоновке схем, рассмотренные в главе 5, относятся к цифровой системе в целом, а потому — и к последовательностным схемам, в частности. Но все же имеет смысл специально выделить следующие правила, которыми необходимо руководствоваться в отношении «последовательностных» элементов систем:

- *Компоновка конечных автоматов.* Совокупность триггеров и комбинационной логики, образующих конечный автомат, следует размещать – в логическом формате – на одной странице таким образом, чтобы тот факт, что это – конечный автомат, был очевиден. (Вы не должны перелистывать страницы, чтобы найти обратную связь!)
- *Последовательно включенные элементы.* Счетчики и регистры, в том числе регистры сдвига, состоящие из нескольких ИС, следует изображать так, чтобы на принципиальной схеме отдельные звенья оказывались собранными вместе и их последовательное включение было очевидно.
- *Триггеры.* Обозначение отдельных элементов последовательностной схемы должно строго соответствовать принятому стандарту их изображения, чтобы тип каждого элемента, выполняемая им функция и реакция на воздействие тактового сигнала были ясны.
- *Описание конечных автоматов.* Конечные автоматы следует описывать таблицами состояний, диаграммами состояний, списками переходов или текстовыми файлами на том или ином языке описания конечных автоматов типа ABEL или VHDL.
- *Временные диаграммы.* Документация на последовательностную схему должна включать временные диаграммы, которые в общем виде показывают предполагаемое поведение схемы во времени.
- *Временные характеристики.* Последовательностная схема должна сопровождаться перечислением условий, накладываемых на значения временных параметров, при выполнении которых гарантируется надлежащее функционирование схемы (например, максимальная частота тактового сигнала), а также требований, предъявляемых к поступающим извне входным сигналам (например, значения времени установления и времени удержания по отношению к системному тактовому сигналу, минимальная длительность импульсов и др.).

8.1.2. Условные обозначения

В параграфе 7.2 были введены традиционные условные обозначения триггеров. Их всегда изображают в виде прямоугольников, и следуют тем же самым правилам, что и в отношении других элементов с подобным обозначением, а именно: входы располагают слева, выходы – справа, кружками отмечают низкий активный уровень и т.д. Кроме того, в отношении обозначения триггеров придерживаются также еще нескольких специальных правил:

- На тактовом входе триггеров, переключающихся по фронту, помещают указатель динамического входа.
- Выходы двухтактных триггеров помечают индикатором задержки срабатывания, если изменение выходного сигнала происходит в конце интервала времени, на котором тактовый сигнал имеет активный уровень.
- Асинхронные входы установки в единичное состояние и сброса могут быть указаны сверху и снизу условного обозначения триггера соответственно.

СТАНДАРТ IEEE НА УСЛОВНЫЕ ОБОЗНАЧЕНИЯ

Стандарт IEEE содержит богатый набор обозначений, обеспечивающий однозначное задание функции каждого сигнала. Справочник по условным обозначениям IEEE, в том числе по условным обозначениям рассматриваемых в этой главе последовательностных элементов, можно найти на сайте www.ddpp.com.

В условном обозначении более сложных последовательностных элементов, таких как счетчики и регистры сдвига, рассматриваемые в этой главе позднее, все входы, как правило, располагаются слева, включая входы установки и сброса, а все выходы – справа. Двухнаправленные выводы изображаются слева или справа – там, где это оказывается удобным.

Как и в случае отдельных триггеров, в условном обозначении более сложных последовательностных элементов используется указатель динамического входа, когда данный элемент не переключается по фронту тактового сигнала, ноступающего на этот вход. В «традиционном» условном обозначении имена входных и выходных сигналов содержат информацию об их назначении, хотя при этом иногда не удается избежать двусмысленности. Например, позднее в этой главе описаны два 4-разрядных счетчика 74x161 и 74x163, традиционные обозначения которых одинаковы, несмотря на то, что по отношению к тактовому сигналу они ведут себя совершенно по-разному.

8.1.3. Описание конечных автоматов

До сих пор мы имели дело с шестью способами представления конечных автоматов:

- Словесные описания.
- Таблицы состояний.
- Диаграммы состояний.
- Списки переходов.
- Программы на языке ABEL.
- VHDL-программы.

Вам может показаться изнурительным владение всеми этими способами представления конечных автоматов: слишком много надо учесть! Ну, не все из них так уж трудны для освоения, но все же здесь *есть* более топкая проблема.

Давайте рассмотрим похожую задачу из области программирования, когда для объяснения того, как работает программа, можно воспользоваться «псевдокодом» высокого уровня или сделать это с помощью блок-схемы. Псевдокод вполне подходит для того, чтобы выразить намерения автора программы, но при трансляции такой программы могут происходить ошибки и неправильное истолкование. В любом творческом процессе могут возникать несовместимости, когда существует несколько способов выражения своих представлений о желательных свойствах проекта.

Такого же рода несовместимости могут возникнуть и при создании конечного автомата. Разработчик логического устройства может описать желаемое поведение проектируемого автомата с помощью нарисованной от руки и правильной на 100% диаграммы состояний, но при переводе этой диаграммы в программу могут быть допущены ошибки: имеется масса возможностей все испортить, если вы должны выполнить ручную «рутинную работу» по преобразованию диаграммы состояний в таблицу состояний, список переходов, уравнения возбуждения и принципиальную схему.

Решение этой проблемы, как и при программировании, состоит в написании самодокументирующегося кода на языке высокого уровня. Ключевым является выбор такого представления, которое содержит достаточно выразительных средств для отображения намерений разработчика и в то же время может быть оттранслировано в физическую реализацию посредством выполнения автоматизированного, свободного от ошибок процесса. (Вам, наверное, доводилось слышать воли многих программистов «Ошибка в компиляторе!», когда их программы не шли с первого раза.)

Лучшим решением (по крайней мере, на сегодняшний день) является непосредственное описание конечного автомата в виде программы на одном из пригодных для этого языков высокого уровня типа ABEL и VHDL, то есть словесное описание без использования других возможных средств представления. Текст на языках типа ABEL и VHDL легко читается и может быть автоматически преобразован в логическую структуру внутри ПЛЮ, ИС типа FPGA или специализированной ИС. Некоторые автоматизированные средства проектирования допускают задание конечных автоматов и их синтез на основе диаграмм состояний или даже временных диаграмм, но они часто выдают неожиданные и двусмысленные результаты. Поэтому в оставшейся части этой книги мы будем пользоваться исключительно языками ABEL и VHDL.

8.1.4. Временные диаграммы и временные параметры

В главах 5 и 7 было много примеров временных диаграмм. Применительно к синхронным системам в большинстве случаев временные диаграммы показывают, как связаны между собой тактовый сигнал и различные входные, выходные и внутренние сигналы.

На рис. 8.1 приведены довольно типичные временные диаграммы, которыми задаются требования, предъявляемые к входным и выходным сигналам синхронной системы, а также определяются временные характеристики. На верхней диаграмме изображен системный тактовый сигнал и указаны его номинальные временные параметры. На остальных диаграммах приведены возможные интервалы задержек для различных сигналов.

В частности, на второй диаграмме показано, что изменение сигналов на выходах триггеров происходит в течение интервала времени t_{fpr} после нарастающего фронта в тактовом сигнале CLOCK. В течение данного отрезка времени значения этих сигналов не должны приниматься во внимание схемами, на входы которых они поданы. На этой временной диаграмме минимальное значение t_{fpr} равно нулю, но в полном пакете документации должна быть таблица с минимальным, типичным и максимальным значениями t_{fpr} , а также всех других временных параметров.

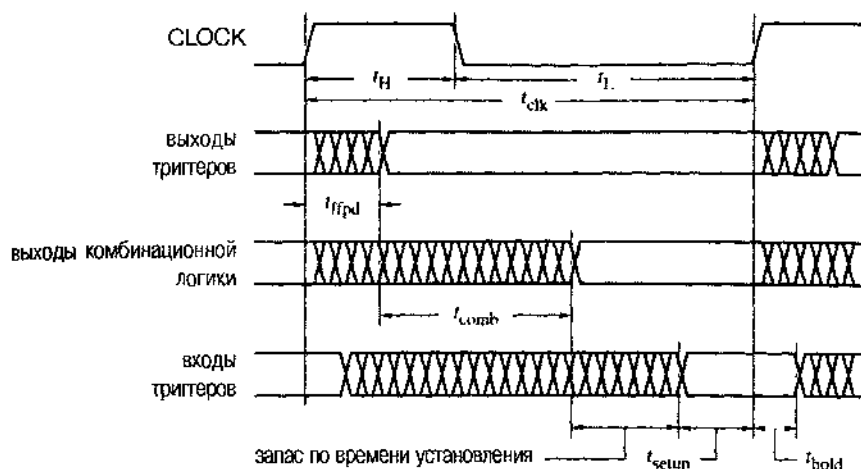


Рис. 8.1. Подробные временные диаграммы, на которых указаны задержки распространения, а также время установления и время удержания по отношению к тактовому сигналу CLOCK

На третьей из приведенных на рисунке временных диаграмм указано дополнительное время t_{comb} , требующееся для того, чтобы изменения сигналов на выходах триггеров прошли через элементы комбинационной логики типа тех, которые образуют логику возбуждения. Для установления сигналов на входах триггеров и других тактируемых устройств нужно отвести время t_{setup} , как это показано на четвертой временной диаграмме. Схема будет работать надлежащим образом только в том случае, когда выполняется неравенство: $t_{clk} - t_{ffpd} - t_{comb} > t_{setup}$.

Запас по времени (*timing margin*) показывает, насколько «плохими» могут быть «в худшем случае» отдельные компоненты схемы, так чтобы в целом это не приводило к ее отказу. У правильно спроектированной системы имеется положительный, ненулевой запас по времени, предусматривающий возможность возникновения неожиданных обстоятельств (компоненты с предельно допустимыми параметрами, пониженное напряжение питания, технические ошибки и т.д.), а также задержку тактового сигнала (см. раздел 8.8.1).

Величину $t_{clk} - t_{ffpd(max)} - t_{comb(max)} - t_{setup}$ называют *запасом по времени установления (setup-time margin)*; схема не будет работать, если эта величина отрицательна. Заметьте, что при расчете запаса по времени установления учитываются *максимальные* задержки распространения. Другим временным параметром задается требование к времени удержания t_{hold} : сумма *минимальных* значений t_{ffpd} и t_{comb} должна быть больше, чем t_{hold} ; величина $t_{ffpd(min)} + t_{comb(min)} - t_{hold}$ является *запасом по времени удержания (hold-time margin)*.

Временные диаграммы, приведенные на рис. 8.1, не отражают различия между временными характеристиками входных сигналов отдельных триггеров и сигналов в комбинационной логике, хотя такое различие в большинстве случаев имеет место. Например, сигнал Q с выхода одного триггера может быть напрямую подан на вход D другого триггера, так что значение t_{comb} для этого пути равно нулю, тогда как другой сигнал, возможно, должен пройти весь путь в 32-разрядном сумматоре

со сквозным переносом, прежде чем он попадет па вход триггера. Если при проектировании синхронной системы следовать надлежащей методике, то подобные временные расхождения не будут критическими, поскольку ни один из этих сигналов не вызывает изменения состояния схемы вплоть до очередного перепада в тактовом сигнале. Просто нужно найти задержку на самом длинном пути в пределах одного периода тактового сигнала, и по ней можно судить о работоспособности схемы. Но для того, чтобы пайти худший случай, вам придется проанализировать несколько различных путей.

Чаще встречаются, по-видимому, временные диаграммы другого рода, демонстрирующие только функциональное поведение, но не отражающие фактических значений задержек; пример таких временных диаграмм приведен на рис. 8.2. Здесь тактовый сигнал «идеален». Как следует изображать изменения сигналов, — вертикальными или наклонными линиями, — это дело вкуса, если только на тех или иных временных диаграммах не должны быть явно указаны время нарастания и время спада. На рис. 8.2 и в дальнейшем мы будем изображать переходы в тактовом сигнале вертикальными линиями, чтобы подчеркнуть, что, по предположению, тактовый сигнал является «идеальным» опорным сигналом.

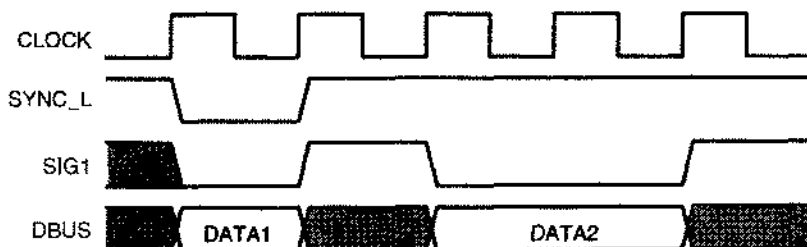


Рис. 8.2. Поведение синхронной схемы во времени

НЕТ В МНРЕ СОВЕРШЕНСТВА!

В действительности, идеальных тактовых сигналов не бывает. Одно из несовершенств, с которым сталкивается большинство разработчиков быстродействующих цифровых устройств, заключается в «фазовом сдвиге тактового сигнала». Как мы увидим в разделе 8.8.1, данный перепад в тактовом сигнале ноступает на входы различных схем в разные моменты времени из-за различия задержек при прохождении сигнала по проводам, из-за разных нагрузок и по другим причинам.

Другая неидеальность, паходящаяся немного в стороне от общей направленности этого учебника, — это «дрожание тактового сигнала». У тактового сигнала с частотой 10 МГц не каждый период равен точно 100 нс: длительность одного периода может равняться 100.05 нс, а другого — 99.95 нс. В такой медленной системе это не очень существенно, но в схеме с тактовой частотой 500 МГц то же самое дрожание в пределах 0.1 нс отнимает 5% у 2-наносекундного ресурса по времени. У некоторых источников тактового сигнала дрожание еще больше!

Табл. 8.1. Задержка распространения в наносекундах для некоторых КМОП- и TTL-триггеров, регистров и защелок (low – низкий уровень, high – высокий уровень) (продолжение)

Номер ИС	Параметр	74HCT		74AHCT		74FCT		74LS	
		Тип.	Макс.	Тип.	Макс.	Мин.	Макс.	Тип.	Макс.
175	t_{pd} CLK↑ до Q или $\bar{Q}$	33	41					21	30
	t_{pd} $\overline{CLR}$ ↓ до Q или $\bar{Q}$	35	44					23	35
	t_{su} D до CLK↑	20	25						20
	t_{hr} D от CLK↑	5	5						5
	t_{rec} CLK↑ от $\overline{CLR}$ ↑	5	5						25
	t_{wh} CLK low или high	20	25						20
	t_{wh} $\overline{CLR}$ low	20	25						20
	t_{pd} CLK↑ до Q	30	38	6.8	11	2	7.2	18	27
273	t_{pd} $\overline{CLR}$ ↓ до Q	32	40	8.5	12.6	2	7.2	18	27
	t_{su} D до CLK↑	12	15		5		2		20
	t_{hr} D от CLK↑	3	3		0		1.5		5
	t_{rec} CLK↑ от $\overline{CLR}$ ↑	10	13		2.5		2		25
	t_{wh} CLK low или high	20	25		6.5		4		20
	t_{wh} $\overline{CLR}$ low	12	15		6		5		20
	t_{pd} C↑ до Q	35	44	8.5	14.5	2	8.5	24	36
	t_{pd} D до Q	32	40	5.9	10.5	1.5	5.2	18	27
373	t_{su} D до C↓	10	13		1.5		2		0
	t_{hr} D от C↓	5	5		3.5		1.5		10
	t_{PHZ} $\overline{OE}$ до Q	35	44		12	1.5	6.5	12	20
	t_{PLZ} $\overline{OE}$ до Q	35	44		12	1.5	6.5	16	25
	t_{PZH} $\overline{OE}$ до Q	35	44		13.5	1.5	5.5	16	28
	t_{PZL} $\overline{OE}$ до Q	35	44		13.5	1.5	5.5	22	36
	t_{wh} C low или high	16	20		6.5		5		15
	t_{pd} CLK↑ до Q	33	41	6.4	11.5	2	6.5	22	34
374	t_{su} D до CLK↑	12	15		2.5		2		20
	t_{hr} D от CLK↑	5	5		2.5		1.5		0
	t_{PHZ} $\overline{OE}$ до Q	28	35		12	1.5	6.5		18
	t_{PLZ} $\overline{OE}$ до Q	28	35		12	1.5	6.5		24
	t_{PZH} $\overline{OE}$ до Q	30	38		12.5	1.5	5.5		28
	t_{PZL} $\overline{OE}$ до Q	30	38		12.5	1.5	5.5		36
	t_{wh} CLK low или high	16	20		6.5		5		15
	t_{pd} CLK↑ до Q	38	48			2	7.2	18	27
377	t_{su} D до CLK↑	12	15				2		20
	t_{hr} D от CLK↑	3	3				1.5		5
	t_{su} $\overline{EN}$ до CLK↑	12	15				2		25
	t_{hr} $\overline{EN}$ от CLK↑	5	5				1.5		5
	t_{wh} CLK low или high	20	25				6		20

Одни и те же временные параметры могут определяться различными производителями слегка по-разному, и их числовые значения могут не совпадать. Даже один и тот же производитель может использовать различные определения для разных семейств и для ИС с разными номерами в пределах одного семейства. Поэтому значения, приведенные в табл. 8.1, являются не более чем характерными; чтобы узнать точные значения соответствующих величин и их определения, вы должны обратиться к справочной информации данного производителя, относящейся к данной микросхеме.

8.2. Защелки и триггеры

8.2.1. Защелки и триггеры в ИС малой степени интеграции

Имеется несколько различных типов защелок и триггеров, доступных поодиночке в микросхемах малой степени интеграции. Эти устройства иногда используются при создании конечных автоматов и «пеструктурированных» последовательностных схем, которые не попадают ни в одну из категорий, рассматриваемых в этой главе позднее: регистры сдвига, счетчики и другие последовательностные ИС средней степени интеграции. В настоящее время отдельные защелки и триггеры практически не используются, так как их функции реализуются в ПЛУ и в ИС типа FPGA. Тем не менее, какое-то количество таких дискретных элементов все еще встречается во многих цифровых системах, поэтому важно познакомиться с ними.

На рис. 8.3 приведена цоколевка нескольких ИС малой степени интеграции с последовательностными элементами. Только в одной из микросхем, указанных на рисунке, — в ИС 74х375 — имеется четыре D-защелки, похожие по выполняемой ими функции на «классическую» D-защелку, рассмотренную в разделе 7.2.4. Из-за ограничения по числу выводов защелки объединены парно и имеют общий управляющий сигнал С в каждой паре.

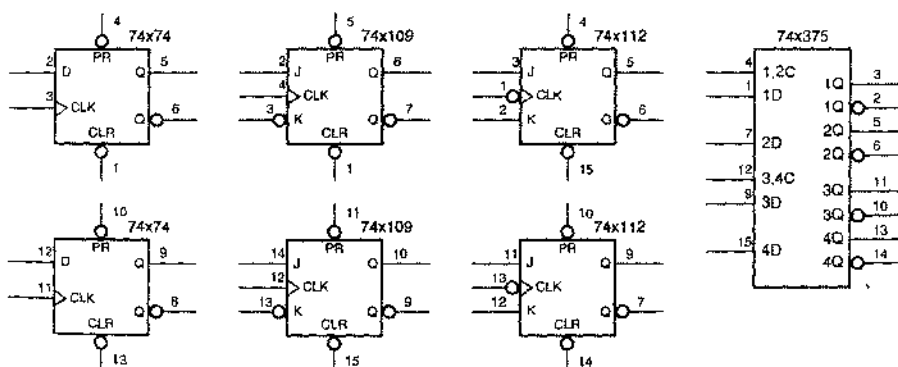


Рис. 8.3. Цоколевка ИС малой степени интеграции с защелками и триггерами

Самой важной среди микросхем, представленных на рис. 8.3, является ИС 74х74, содержащая два независимых D-триггера, переключающихся по положительному фронту, с входом установки в единичное состояние и входом сброса. В разделе 7.2.5 уже были рассмотрены выполняемые таким триггером функции, его временные диаграммы и внутренняя структура. Помимо использования триггеров из ИС 74х74 «от случая к случаю», их быстродействующие варианты, имеющиеся, например, в таких микросхемах, как 74F74 и 74ACT74, находят применение в устройствах синхронизации для асинхронных входных сигналов, о чем пойдет речь в параграфе 8.9.

ИС 74х109 содержит не переключающийся по положительному фронту JK-триггер с низким активным уровнем сигнала на входе К (этот сигнал называется $\bar{K}$ или K_L). Внутренняя структура такого триггера была рассмотрена в разделе 7.2.10. В ИС 74х112 имеется другой JK-триггер с низким активным уровнем тактового сигнала.

*8.2.2. Защита от дребезга при переключении

Распространенным применением защелок и простых элементов с двумя устойчивыми состояниями является защита от дребезга при переключении. Вы все хорошо знакомы с электрическими выключателями: вы пользуетесь ими на каждом шагу, зажигая свет, включая пылесос или другую бытовую технику. В цифровых системах с помощью переключателя происходит подключение к источнику постоянного логического сигнала, равного 0 или 1; такое подключение часто называют «подачей входного сигнала от пользователя». Но применительно к цифровым системам необходимо рассмотреть процедуру переключения в другом аспекте, представив себе ее развернутой во времени. Только в темпе сравнительно медленно действующего человека включение или выключение происходит как бы мгновенно. Быстродействующая цифровая логика различает в этой процедуре несколько фаз.

На рис. 8.4(а) показано, как можно воспользоваться однополюсным переключателем на одно направление для того, чтобы выработать одиполный логический входной сигнал. Резистор, соединяющий вход логического элемента с шиной питания, обеспечивает единичное значение входного сигнала, когда ключ разомкнут. Если кнопка, управляющая ключом, нажата, то вход логического элемента соединен с землей, в результате чего входной сигнал принимает нулевое значение.

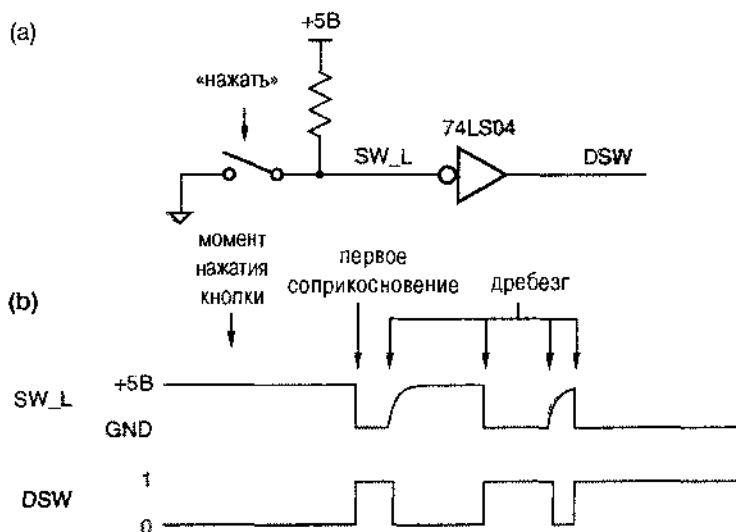


Рис. 8.4. Переключение входного сигнала без защиты от дребезга (GND – земля)

Как видно из рис. (b), проходит некоторое время после нажатия кнопки, прежде чем подвижный контакт коснется заземленного контакта. Коснувшись заземленного контакта в первый раз, подвижный контакт не остается в этом положении надолго, а совершает несколько отскоков и только после этого окончательно устанавливается в замкнутом состоянии. В результате при каждом однократном нажатии кнопки сигналы на входе SW_L и на выходе DSW несколько раз изменяют свои значения. Такое

явление называется *дребезгом контакта* (*contact bounce*). В типичном случае продолжительность дребезга составляет 10–20 мс, а это очень большое время с точки зрения скорости, с которой переключаются логические неинвертирующие элементы.

В зависимости от условий применения переключателей наличие дребезга может быть существенным или несущественным. Например, конфигурация некоторых компьютеров задается миниатюрными переключателями, которые называются *DIP-переключателями* (*DIP switches*), поскольку они имеют те же размеры, что и ИС в DIP-корпусе, то есть в корпусе с двухрядным расположением выводов. Поскольку, как правило, изменение конфигурации компьютера с помощью такого переключателя осуществляется лишь тогда, когда компьютер выключен, дребезг контактов не имеет значения. Дребезг контакта становится проблемой, когда путем замыкания или размыкания ключа подается сигнал о том или ином событии и ведется подсчет числа этих событий (например, когда считаются круги во время гонок). В этом случае нам необходима схема (или подходящая программа для микропроцессорной системы), обеспечивающая *защиту от дребезга* (*debounce*), в результате чего вырабатывалось бы только одиночное изменение сигнала (или одиночный импульс) на каждое внешнее воздействие.

*8.2.3. Простейшая схема защиты от дребезга

Защита от дребезга является хорошим примером применения простейшего последовательностного устройства — элемента с двумя устойчивыми состояниями, рассмотренного в параграфе 7.1. Им можно воспользоваться так, как показано на рис. 8.5. В этой схеме применяется однополюсный переключатель на два направления, устроенный таким образом, что при переключении подвижный контакт сначала размыкается с одним из неподвижных контактов и только потом замыкается на другой неподвижный контакт. Поэтому в процессе переключения подвижный контакт в течение некоторого времени пребывает в «плавающем» состоянии, где-то посередине между неподвижными контактами.

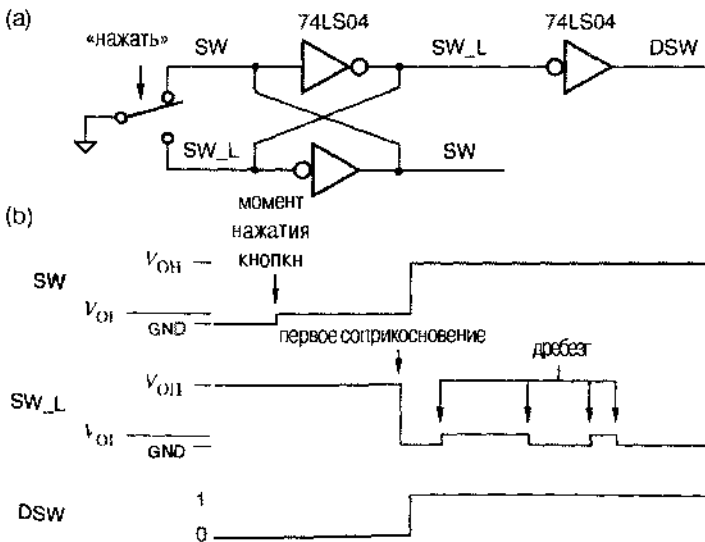


Рис. 8.5. Подавление дребезга при переключении входного сигнала с помощью элемента с двумя устойчивыми состояниями (GND — земля)

До нажатия кнопки на верхний неподвижный контакт подается 0 В, поэтому логическое значение сигнала SW равно 0, а на выходе верхнего инвертора удерживается сигнал с логическим значением 1, которым определяется потенциал SW_L нижнего неподвижного контакта. Когда кнопка нажимается и соединение между подвижным и верхним неподвижным контактом разрывается, петля обратной связи в элементе с двумя устойчивыми состояниями удерживает значение SW на уровне V_{OL} ($= 0.5$ В для ТТЛ-схем серии LS), что, по-прежнему, соответствует логическому 0.

Затем, когда подвижный контакт впервые касается нижнего неподвижного контакта, схема некоторое время ведет себя самым необычным образом. Верхний инвертор элемента с двумя устойчивыми состояниями пытается удерживать логическую 1 на сигнальной линии SW_L; при этом верхний транзистор в его выходной цепи открыт и поэтому точка SW_L через малое сопротивление соединена с шиной питания +5 В. А тут вдруг нижний неподвижный контакт механически соединяется с землей, заставляя потенциал SW_L стать равным 0.0 В. Ничего удивительного, что механическое воздействие побеждает.

Спустя короткое время (30 нс для ИС 74LS04) логический 0, механически удерживаемый на линии SW_L, проходит через два инвертора, образуя элемент с двумя устойчивыми состояниями, так что верхний инвертор отказывается от тщетных усилий по поддержанию на линии SW_L логической 1, а вместо нее выдает логический 0. Но теперь нет больше необходимости в замыкании выхода верхнего инвертора на землю: петля обратной связи в элементе с двумя устойчивыми состояниями удерживает логический 0 на линии SW_L даже в том случае, когда подвижный контакт отскакивает от нижнего неподвижного контакта. (Он отскакивает не очень далеко и не касается снова верхнего неподвижного контакта.)

Достоинство этой схемы по сравнению с другими вариантами защиты от дребезга заключается в малом расходе ресурса по числу микросхем (одна треть ИС 74LS04), в отсутствии резисторов, подтягивающих входы вентилей к шине питания, и в возможности получения входного сигнала любой полярности (с высоким или низким активным уровнем). Если все же необходимо избежать замыкания выходов вентилей на землю даже на короткое время, то подобную схему можно собрать на $\overline{SR}$ -защелке с резисторами, подтягивающими ее входы к шине питания, как показано на рис. 8.6.

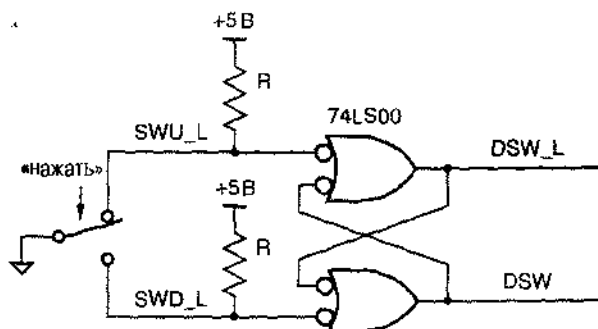


Рис. 8.6. Подавление дребезга при переключении входного сигнала с помощью $\overline{SR}$ -защелки

КОГДА ЛИШНИЕ ДОСТОИНСТВА НИ К ЧЕМУ

Хотя схема, приведенная на рис. 8.5, весьма привлекательна, ее не следует применять при использовании быстродействующих КМОП-клапанов типа 74АСТ04, которые способны отдавать большой ток со стороны выхода при высоком уровне сигнала на нем. Замыкание таких выходов на землю на короткое время не приводит к повреждению микросхем, но в результате этого в шине питания и в земляной шине возникает импульс помехи, который может привести к нежелательному срабатыванию каких-то других узлов в системе. Схема защиты от дребезга на инверторах функционирует вполне успешно, будучи собрана на более простых микросхемах типа 1СТ и LS-TTL.

***8.2.4. Шинный фиксатор уровня**

В разделе 3.7.3 и в параграфе 5.6 были рассмотрены схемы с тремя состояниями и объединение выходов таких схем в шину с тремя состояниями. В любой момент времени не более чем одна из схем выдает сигнал на шину; иногда ни одна из схем не является источником сигнала, и тогда шина оказывается в «плавающем» состоянии. Если к шине подключены входы быстродействующих КМОП-клапанов и шина оставлена в плавающем состоянии надолго (в случае схем с большим быстродействием — на время, превышающее один-два периода тактового сигнала), то могут происходить разные неприятности. В частности, из-за шумов, перекрестных помех и других эффектов на высокоимпедансной шине, пребывающей в плавающем состоянии, может возникнуть напряжение, близкое к порогу переключения КМОП-схем по входу, что, в свою очередь, приведет к протеканию чрезмерно больших токов по выходным цепям этих схем. По этой причине желательно соединять шины резисторами с источником питания, чтобы при их переходе в плавающее состояние на них быстро устанавливался логический сигнал высокого уровня. Так обычно и поступают.

Но резисторы, подтягивающие потенциал шины к напряжению питания, — не такое уж хорошее решение: они стоят денег и занимают ценное «жизненное пространство» на печатной плате. В случае сверхбыстродействующих схем трудной задачей является выбор сопротивления этих резисторов. Если сопротивление слишком велико и шина переходит в третье состояние с низкого уровня сигнала, то изменение потенциала шины от низкого уровня до высокого происходит медленно из-за большой постоянной времени RC , и напряжение на подключенных к шине входах слишком долго остается вблизи порога переключения. Если сопротивление резистора, подтягивающего шину к источнику питания, слишком мало, то схемы, которым предстоит выдавать на шину сигнал низкого уровня, должны будут допускать протекание в них со стороны выхода слишком большого тока.

Решение этой проблемы состоит в исключении резистора, подтягивающего шину к источнику питания, и замене его активным *шинным фиксатором уровня* (*bus holder circuit*), показанным на рис. 8.7. В этой схеме нет ничего нового: это — элемент с двумя устойчивыми состояниями с резистором R в одной из ветвей петли обратной связи. Вывод INOUT данной схемы соединяется с сигнальной линией

ей шины с тремя состояниями, уровень которой эта схема должна будет удерживать. Когда схема, выдававшая на шину сигнал низкого или высокого уровня, запирается и ее выход переходит в третье состояние, правый инвертор фиксатора уровня удерживает сигнальную линию в ее прежнем состоянии. При изменении сигнала, выдаваемого на линию, с низкого уровня на высокий или в обратном направлении схема, служащая источником этого сигнала, должна допускать протекание или втекание со стороны ее выхода небольшого дополнительного тока, протекающего по резистору R , чтобы преодолеть действие фиксатора уровня. Этот дополнительный ток течет лишь короткое время, которое необходимо элементу с двумя устойчивыми состояниями, чтобы переключиться из одного состояния в другое.

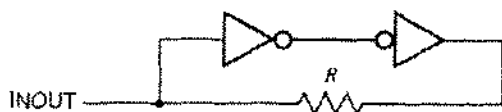


Рис. 8.7. Шинный фиксатор уровня

Сопротивление резистора R выбирается из соображений компромисса между малым током, затрачиваемым на преодоление действия фиксатора уровня (большое значение R), и хорошей помехоустойчивостью удержания шины в текущем состоянии (малое значение R). Приведем типичный пример: в случае 3.3-вольтовых КМОП-схем семейства LVC максимальное значение тока, который может быть затрачен на преодоление действия фиксатора уровня, равно 500 мкА; поэтому $R \approx 3.3/0.0005 = 6.6$ кОм.

Шинные фиксаторы уровня часто бывают встроены в такие ИС средней степени интеграции, как 8-разрядный шинный КМОП-буфер или приемопередатчик. Для шинных фиксаторов уровня не требуется дополнительных выводов, и они занимают очень мало места на поверхности кристалла, так что их можно иметь, по существу, бесплатно. Нет никакой проблемы в том, чтобы к одной и той же линии были подключены несколько (n) фиксаторов уровня при условии, что схемы, служащие источниками сигналов, выдаваемых на шину, допускают протекание по их выходным цепям в течение нескольких наносекунд (при переключении) n -кратного тока, затрачиваемого на преодоление действия всех фиксаторов уровня. Обратите внимание, что в случае, когда к шине подключены ТТЛ-входы, применение шинных фиксаторов уровня, как правило, не рационально (см. задачу 8.25).

8.2.5. Многоразрядные регистры и защелки

Регистром (register) называют совокупность из двух или большего числа D-триггеров с общим входом тактового сигнала. Регистры часто применяют для запоминания набора связанных между собой битов, например, для хранения байта данных в компьютере. Но тот или иной регистр можно использовать также для сохранения и не связанных между собой битов данных или управляющей информации; единственное ограничение состоит в том, что все биты запоминаются в один и тот же момент времени.

На рис. 8.8 приведены принципиальная схема и условное обозначение регистра в ИС средней степени интеграции 74x175. Эта микросхема содержит четыре

переключающихся по фронту D-триггера с общим тактовым входом и общим асинхронным входом сброса. На выводах этой ИС имеются выходные сигналы D-триггеров как с высоким, так и с низким активным уровнем.

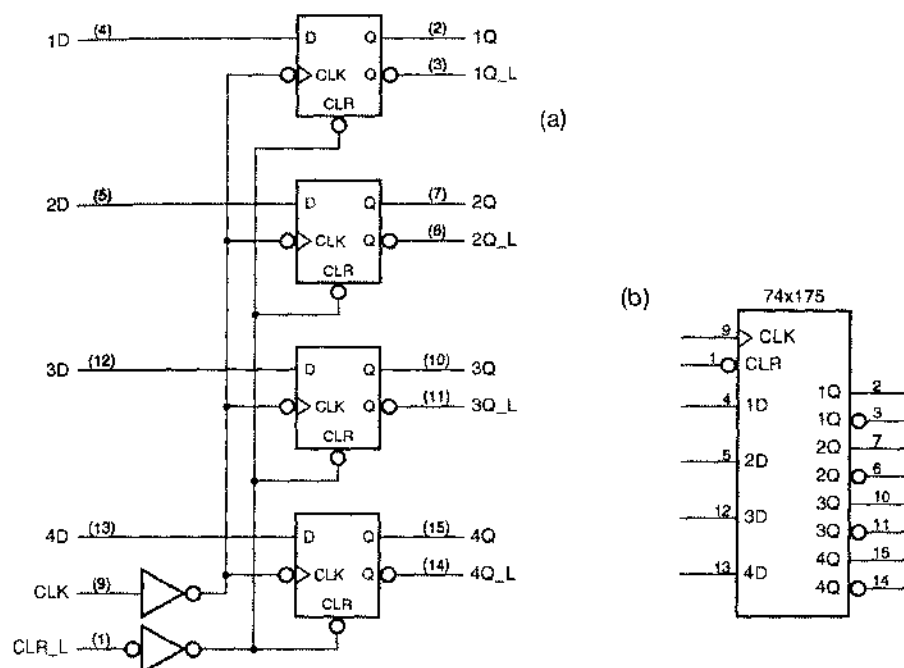


Рис. 8.8. 4-разрядный регистр 74x175: (а) принципиальная схема и цоколевка для стандартного DIP-корпуса с 16 выводами; (б) традиционное условное обозначение

Сами по себе триггеры в ИС '175 переключаются по отрицательному перепаду, что отмечено на рисунке кружками инверсии на их тактовых входах CLK. Однако в схеме имеется также инвертор, благодаря которому переключение триггеров происходит в момент положительного перепада в тактовом сигнале, подаваемом извне на вывод CLK данной микросхемы. Общий сигнал сброса с низким активным уровнем CLR_L подается на асинхронные входы сброса всех четырех триггеров. Оба сигнала CLK и CLR_L проходят через буферы и только после этого разводятся по четырем триггерам, так что по отношению к источникам этих сигналов регистр ведет себя как одиночная нагрузка по каждому входу. Это особенно важно в том случае, когда общий тактовый сигнал и сигнал сброса должны быть поданы на большое число таких регистров.

На рис. 8.9 показано условное обозначение 6-разрядного регистра 74x174. Внутренняя структура этой ИС подобна тому, как устроена микросхема 74x175, за

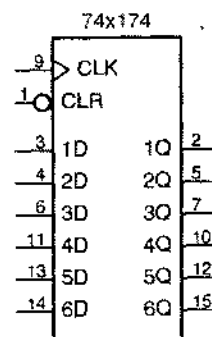


Рис. 8.9. Условное обозначение 6-разрядного регистра 74x174

исключением того, что добавлены два триггера, а выходы с низким активным уровнем исключены.

Во многих цифровых системах – в компьютерах, в узлах телекоммуникаций, в стереосистемах – обрабатываемая информация бывает представлена словами, состоящими из 8, 16 или 32 битов. Поэтому очень популярны ИС, которые на каждом такте имеют дело с 8 битами. Одной из таких ИС средней степени интеграции является 8-разрядный («октальный») регистр 74х374, состоящий из переключающихся по фронту D-триггеров.

Как видно из рис. 8.10, в ИС 74х374 имеется восемь D-триггеров, которые одновременно фиксируют значения сигналов на их входах в момент времени, задаваемый нарастающим фронтом общего тактового сигнала CLK; в этот же момент времени происходит изменение сигналов на выходах триггеров. Сигнал с выхода каждого триггера поступает на буфер с тремя состояниями, который, в свою очередь, вырабатывает выходной сигнал с высоким активным уровнем. Все буферы с тремя состояниями управляются общим входным сигналом OE_L с низким активным уровнем (сигналом разрешения выхода). Как и в других регистрах, рассмотренных выше, входы управляющих сигналов (CLK и OE_L) снабжены буферами, так что по отношению к источнику каждого из этих сигналов регистр представляет собой одиночную нагрузку.

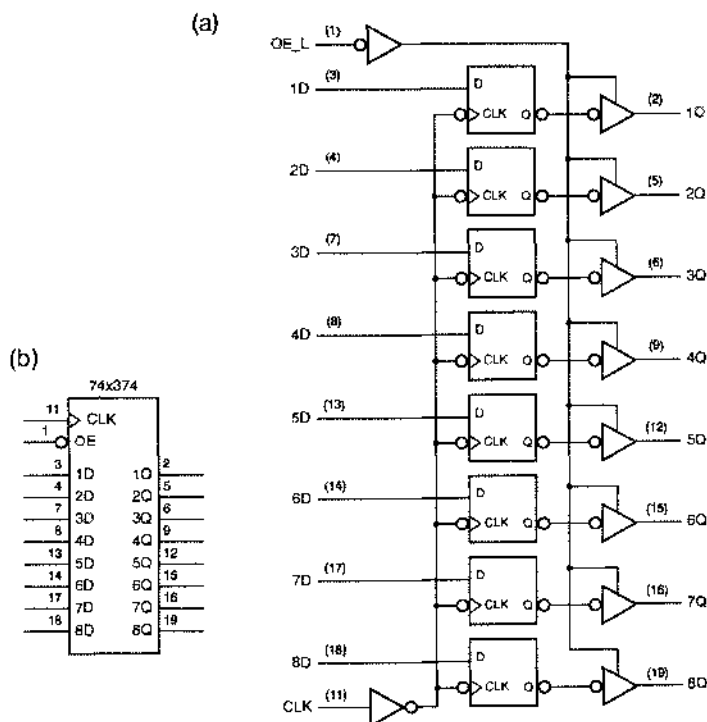


Рис. 8.10. 8-разрядный регистр 74х374: (а) принципиальная схема и цолевка для стандартного DIP-корпуса с 20 выводами; (б) традиционное условное обозначение

Разновидностью устройства, подобного ИС 74х374, является микросхема 74х373, условное обозначение которой приведено на рис. 8.11. ИС '373 состоит не из переключающихся по входу D-триггеров, а из D-защелок. Выходные сигналы защелок являются повторением сигналов, действующих на их входах, пока сигнал С имеет активный уровень; в момент, когда сигнал С переходит на неактивный уровень, происходит защелкивание, то есть запоминаются последние значения входных сигналов. На рис. 8.12 показан другой вариант 8-разрядного регистра – ИС 74х273. Выходы этой микросхемы не являются выходами с тремя состояниями, поэтому вход OE_L отсутствует, а вместо этого вывод 1 используется для подачи асинхронного сигнала сброса CLR_L.

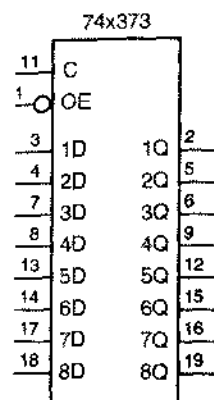


Рис. 8.11. Условное обозначение 8-разрядной защелки 74х373

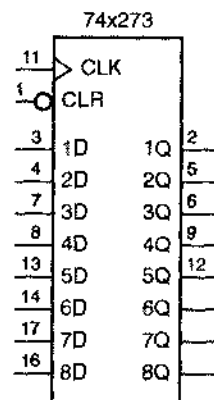


Рис. 8.12. Условное обозначение 8-разрядного регистра 74х273

ИС 74х377, условное обозначение которой дано на рис. 8.13(а), представляет собой переключающийся по фронту регистр, подобный регистру '374, но не имеющий выходов с тремя состояниями. Вместо этого вывод 1 используется для того, чтобы управлять тактовым входом путем подачи на него сигнала разрешения EN_L. Если сигнал EN_L имеет низкий уровень в момент, задаваемый нарастающим фронтом тактового сигнала, то в триггеры загружаются входные данные; в противном случае в триггерах сохраняются прежние значения, как это следует из схемы, приведенной на рис. (b).

Среди микросхем для поверхностного монтажа в корпусах с большим числом выводов имеются регистры, драйверы и приемопередатчики с еще большим числом разрядов. Самыми распространенными являются 16-разрядные устройства, но существуют также 18-разрядные схемы (с побайтовым контролем четности) и 32-разрядные схемы. Кроме того, микросхемы в корпусах с большим числом выводов могут обладать большими функциональными возможностями в отношении управления, такими как сброс, разрешение тактового сигнала, наличие нескольких входов разрешения выхода и даже выбор режима работы – в качестве защелки или в качестве регистра.

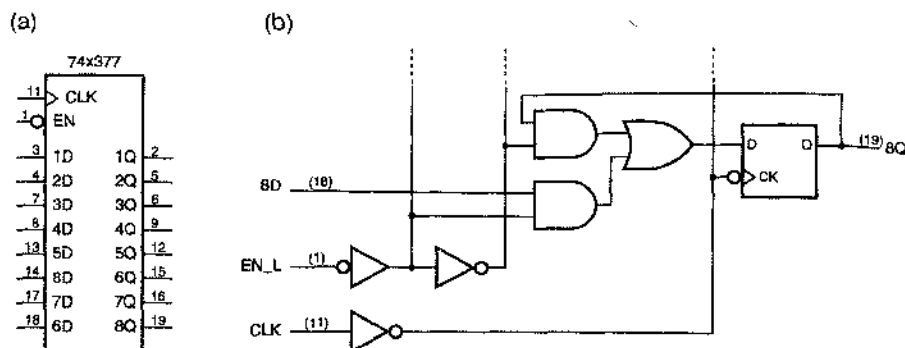


Рис. 8.13. 8-разрядный регистр 74x377 с управляемым тактовым входом. (a) условное обозначение; (b) схематическое изображение того, как ведет себя ехема в одном разряде

8.2.6. Описание регистра и защелок на языке ABEL и их реализация в ПЛУ

Как мы видели в параграфе 7.11, на языке ABEL очень легко задавать регистры. В табл. 7.33 была приведена в качестве примера программа на языке ABEL для 8-разрядного регистра с входом разрешения. Очевидно, что на языке ABEL можно обеспечить реализацию почти любых функций по отношению к сигналам на D-входах регистра по желанию заказчика; единственным ограничением является число входов и число термов-произведений в ПЛУ, в котором предстоит образовать этот регистр. Последовательностные ПЛУ мы рассмотрим в параграфе 8.3.

В большинстве последовательностных ПЛУ реализация каких-либо условий в отношении тактового сигнала (например, выбор полярности) и в отношении асинхронных входов (например, различные условия уставки в единичное состояние в разных разрядах) затруднена. Однако в языке ABEL имеется подходящий синтаксис, обеспечивающий выполнение желаемых условий в устройствах, допускающих требуемую конфигурацию, как это описано в разделе 7.11.1.

Лишь в небольшом числе ПЛУ есть встроенные защелки; значительно более распространенными и, в общем случае, более полезными являются переключающиеся по фронту регистры. Но, воспользовавшись комбинационной логикой и обратной связью, можно синтезировать также и защелку. Например, уравнение возбуждения для SR-защелки имеет вид:

$$Q^* = S + R' \cdot Q.$$

Таким образом, SR-защелку можно было бы образовать, используя один комбинационный выход согласно равенству на языке ABEL: $Q = S \# !R \& Q$. Более того, сигналы S и R, фигурирующие в этих соотношениях, можно заменить более сложными логическими функциями сигналов, действующих на входах ПЛУ, при единственном ограничении по максимально возможному числу термов-произведений (равному семи на каждый выход в ИС 16V8С и 16L8), которое может входить в окончательное уравнение возбуждения. Петлю обратной связи можно создать толь-

ко в том случае, когда сигнал Q выведен на двунаправленный внешний контакт (на выводы IO2–IO7 в ИС 16V8С и 16L8, но не на выводы Q1 и Q8). Кроме того, выход на этот вывод должен быть постоянно разрешен; в противном случае петля обратной связи будет разорвана, а состояние защелки потеряно.

Возможно, удобнее всего создавать на основе комбинационного ПЛУ D-защелку. Основное уравнение возбуждения для D-защелки выглядит так.

$$Q^* = C \cdot D + C' \cdot Q.$$

Однако, как мы видели в разделе 7.10.1, это уравнение содержит статический источник опасности, и соответствующая схема не запоминает данные надежно. Чтобы построить надежную D-защелку, необходимо включить в уравнение возбуждения консенсусный терм:

$$Q^* = C \cdot D + C' \cdot Q + D \cdot Q.$$

Переменную D в этом уравнении можно заменить более сложным выражением *expression*, но структура уравнения остается прежней:

$$Q^* = C \cdot \text{expression} + C' \cdot Q + \text{expression} \cdot Q.$$

В разделе 7.10.1 было показано, что возможно также более сложное выражение для переменной C. В любом случае очень важно, чтобы при реализации в ПЛУ был учтен консенсусный терм. При этом компилятор может оказать вам плохую услугу, так как на этапе минимизации он обнаружит, что этот терм избыточен, и удалит его.

Некоторые компиляторы языка ABEL позволяют предотвратить исключение консенсусных термов путем помещения ключевого слова *retain* в список свойств объявления *istype* для любого выходного сигнала, который не следует минимизировать (*свойство retain, retain property*). В случае, если ваш компилятор не допускает такой возможности, то все, что вы можете сделать, – это запретить минимизацию во всем проекте.

Наиболее распространенным, по-видимому, применением защелок, реализованных в ПЛУ, является одновременное декодирование и запоминание адресов при выборе ячейки памяти или одного из устройств ввода/вывода в микропроцессорной системе. На рис. 8.14 приведены временные диаграммы, относящиеся к выполнению этой функции в типичной системе. Микропроцессор выбирает устройство и место внутри этого устройства, выставляя адрес на своей адресной шине ABUS и выдавая сигнал AVALID в знак того, что адрес установлен на шине адреса. Спустя небольшое время он выдает сигнал чтения READ_L, и выбранное устройство откликается на него помещением данных на шину DBUS.

Обратите внимание, что адрес не остается на шине ABUS в течение всей процедуры. Согласно протоколу, по которому действует микропроцессор, он предполагает, что адрес запоминается по сигналу AVALID, играющему роль сигнала разрешения, а затем декодируется, как это показано на рис. 8.15. Дешифратор выбирает то или иное устройство по значениям битов в старших разрядах адреса (по 12 старшим разрядам в нашем примере), вырабатывая сигнал разрешения или сигнал «выбора кристалла». Младшие разряды адреса указывают определенное место (блок или ячейку) в выбранном устройстве.

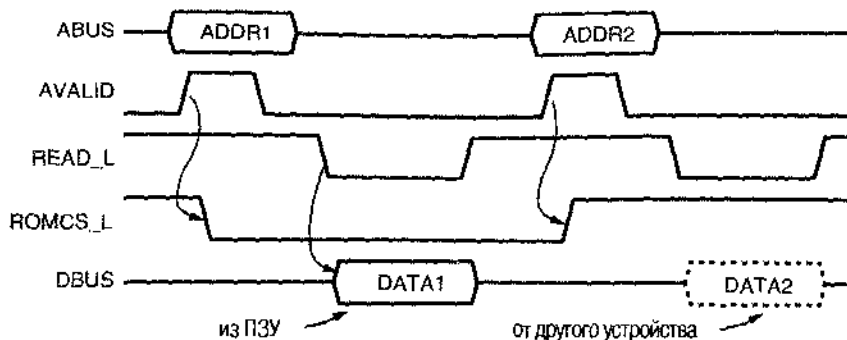


Рис. 8.14. Временные диаграммы для процедуры чтения в микропроцессорной системе

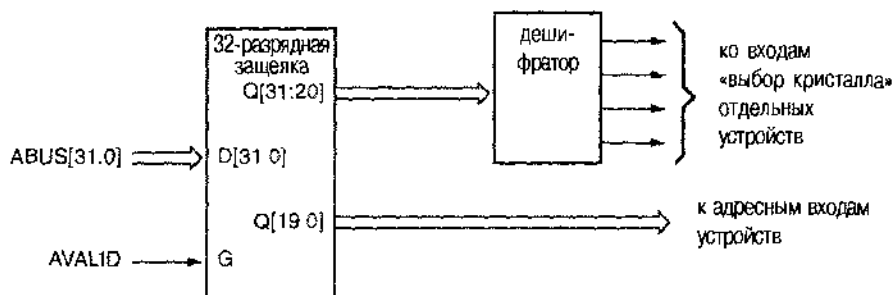


Рис. 8.15. Схема, запоминающая и декодирующая адрес, выставяемый микропроцессором

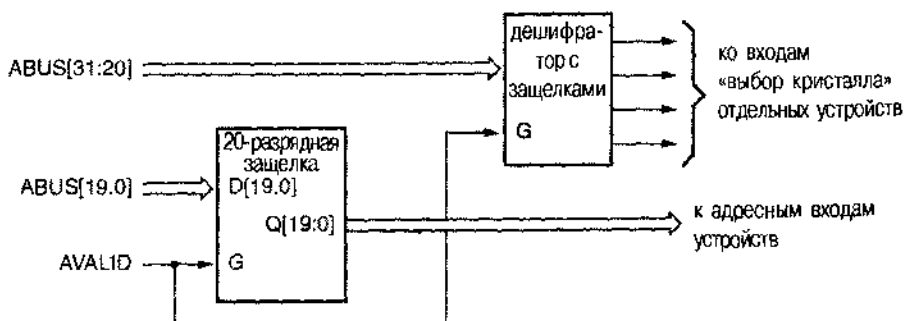


Рис. 8.16. Применение схемы, в которой фиксация и декодирование адреса совмещены

При реализации в ПЛУ можно совместить функции запоминания и декодирования старших битов в одном устройстве согласно блок-схеме, приведенной на рис. 8.16. «Дешифратор с защелкиванием» позволяет сократить необходимое число микросхем и выводов по сравнению со схемой на рис. 8.15 и быстрее выдать иолноценный сигнал на выходе выбора кристалла (см. упражнение 8.9).

ПОЧЕМУ ИМЕННО ЗАЩЕЛКА?

При рассмотрении указанного на рис. 8.14 протокола обмена сигналами по шинам микропроцессора возникает ряд вопросов:

- Почему адрес не удерживается на шине ABUS на протяжении всей процедуры? В реальной системе, функционирующей согласно этому протоколу, шины ABUS и DBUS бывают совмещены посредством мультиплексирования в одной шине с тремя состояниями, благодаря чему экономится число выводов и сигнальных линий.
- Почему сигнал AVALID не используется в качестве тактового сигнала для фиксации адреса в регистре, переключающемся по положительному фронту? Отсутствует достаточное время для установления; в реальной системе сигналы на адресной шине только-только устанавливаются к моменту, задаваемому нарастающим фронтом сигнала AVALID, или даже чуть-чуть позднее.
- Ну, ладно; тогда почему сигнал AVALID нельзя использовать в качестве тактового сигнала для регистра, переключающегося по отрицательному фронту? Так можно поступить, но сигналами с выходов защелки можно воспользоваться быстрее; установившиеся значения сигналов на шине ABUS проходят на выход защелки непосредственно, для этого не нужно ждать спадающего фронта тактового сигнала. Благодаря этому можно ослабить требования, предъявляемые к времени доступа элементов памяти и других устройств, на которые подаются сигналы с выходов защелки.

В табл. 8.2 представлена программа на языке ABEL для дешифратора с защелкиванием. Поскольку устройству доступны только старшие разряды адресной шины ABUS [31..20], результатом декодирования может быть лишь обращение к одному из участков памяти объемом 1 Мбайт или больше ($2^{20} = 1 \text{ М}$). Постоянному записывающему устройству (ПЗУ, ROM) отводится 1-мегабайтовый участок памяти с самыми старшими адресами 0xffff00000–0xfffffff, и этот участок выбирается сигналом ROMCS. Три 16-мегабайтовых блока оперативной памяти (ОЗУ, RAM) размещены на младших адресах, начиная с 0x00000000, 0x01000000 и 0x02000000 соответственно. Обратите внимание на то, что в определении адресного пространства ОЗУ используются безразличные значения, поскольку речь идет об обращении к участку памяти объемом больше 1 Мбайт. Возможны и другие способы таких определений (см. упрощение 8.1).

Равенства в табл. 8.2 для сигналов «выбор кристалла» составлены по образцу, приведенному в начале этого раздела. Каждое из выражений типа "ABUS==ROM", посредством которых выбирается устройство, порождает один терм-произведение, и каждое равенство дает три термина-произведения. Заметьте, что в объявлении выводов указано свойство `retain`, чтобы предотвратить исключение компилятором консенсусных термов в процессе оптимизации.

Видя, как легко образовать в комбинационном ПЛУТ SR- и D-защелки, вы, возможно, испытываете соблазн пойти дальше и попытаться создать переключающийся по фронту D-триггер. В принципе, это возможно, но будет дорого стоить.

Табл. 8.3. Структурная VHDL-программа для D-защелки, приведенной на рис. 7.12

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vdlatch is
  port (D, C: in STD_LOGIC;
        Q, QN: buffer STD_LOGIC);
end Vdlatch;

architecture Vdlatch_a of Vdlatch is
  signal DN, SN, RN: STD_LOGIC;
  component inv port (I: in STD_LOGIC; O: out STD_LOGIC); end component;
  component nand2b port (I0, I1: in STD_LOGIC; O: buffer STD_LOGIC); end component;
begin
  U1: inv port map (D,DN);
  U2: nand2b port map (D,C,SN);
  U3: nand2b port map (C,DN,RN);
  U4: nand2b port map (SN,QN,Q);
  U5: nand2b port map (Q,RN,QN);
end Vdlatch_a;

```

В табл. 8.4 помещена основанная на использовании процесса поведенческая VHDL-архитектура для D-защелки, в которой для описания поведения защелки понадобилась всего лишь одна строка текста. Обратите внимание: VHDL-компилятор из этого описания «делает вывод» о том, что речь идет о защелке; действительно, в этом описании ничего не говорится о том, что надо делать, если C не равно 1, и поэтому компилятор создает *предполагаемую защелку* (*inferred latch*), которая сохраняет значение Q до очередного возобновления процесса. VHDL-компилятор всегда создает защелку для сигнала, которому присваивается значение в операторе if или в операторе case, если перечислены не все возможные комбинации входных сигналов.

Табл. 8.4. Поведенческая VHDL-структура для D-защелки

```

architecture Vdlatch_b of Vdlatch is
begin
  process(C, D, Q)
  begin
    if (C='1') then Q <= D; end if;
    QN <= not Q;
  end process;
end Vdlatch_b;

```

ТИП buffer И ДРУГИЕ ВОЗМОЖНОСТИ

Заметьте, что в табл. 8.3 Q и QN определены как сигналы типа buffer, а не out, поскольку в определении архитектуры они используются как входы и как выходы. Но тогда мы должны ввести специальный 2-входовой веентиль NAND с выходным сигналом типа buffer, чтобы избежать несогласованности типов (out и buffer) при обращении к компонентам U4 и U5. Впрочем, можно было воспользоваться внутренними сигналами, чтобы обойти эту проблему, как это сделано в табл. 8.5. Как вы теперь уже знаете, в языке VHDL одни и те же вещи можно выразить многими различными способами.

Табл. 8.6. Альтернативный вариант структурной VHDL-программы для D-защелки, приведенной на рис. 7.12

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vdlatch is
  port (D, C: in STD_LOGIC;
        Q, QN: out STD_LOGIC );
end Vdlatch;

architecture Vdlatch_s2 of Vdlatch is
  signal DN, SN, RN, IQ, IQN: STD_LOGIC;
  component inv port (I: in STD_LOGIC; O: out STD_LOGIC ); end component;
  component nand2 port (I0, I1: in STD_LOGIC; O: out STD_LOGIC ); end component;
begin
  U1: inv port map (D,DN);
  U2: nand2 port map (D,C,SN);
  U3: nand2 port map (C,DN,RN);
  U4: nand2 port map (SN,IQN,IQ);
  U5: nand2 port map (IQ,RN,IQN);
  Q <= IQ; QN <= IQN;
end Vdlatch_s2;

```

Чтобы описать переключение триггеров по фронту, нам необходим один из предопределенных атрибутов сигнала в языке VHDL – *признак event* (*event attribute*). Если “SIG” – это имя сигнала, то конструкция “SIG ‘event” на любом элементарном шаге по времени выдает значение true, когда сигнал SIG изменяет свое значение, и false – в противном случае.

С помощью признака event можно смоделировать переключающийся по фронту триггер так, как это сделано в таблице 8.6. В операторе IF конструкция “CLK ‘event” выдает значение true на каждом перепаде тактового сигнала, но условие “CLK = ‘1” обеспечивает присвоение значения D сигналу Q только на *падающем* фронте. Заметьте, что список чувствительности процесса содержит только сигнал CLK; изменения сигнала D в какое-то другое время в этой функциональной модели не существуют.

Табл.8.6. Поведенческая VHDL-модель переключающегося по фронту D-триггера

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vdff is
  port (D, CLK: in STD_LOGIC;
        Q: out STD_LOGIC );
end Vdff;

architecture Vdff_b of Vdff is
begin
  process(CLK)
  begin
    if (CLK'event and CLK='1') then Q <= D; end if;
  end process;
end Vdff_b;

```

Модель D-триггера можно усовершенствовать, добавив асинхронные входы и инвертированный выход, как у каждого из триггеров в ИС 74х74; такое усовершенствование осуществлено в табл. 8.7.

Табл. 8.7. VHDL-модель D-триггера с установкой в единичное состояние и сбросом, подобного триггерам в ИС 74х74

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vdff74 is
  port (D, CLK, PR_L, CLR_L: in STD_LOGIC;
        Q, QN: out STD_LOGIC);
end Vdff74;

architecture Vdff74_b of Vdff74 is
  signal PR, CLR: STD_LOGIC;
begin
  process(CLR_L, CLR, PR_L, PR, CLK)
  begin
    PR <= not PR_L; CLR <= not CLR_L;
    if (CLR and PR) = '1' then Q <= '0'; QN <= '0';
    elsif CLR = '1' then Q <= '0'; QN <= '1';
    elsif PR = '1' then Q <= '1'; QN <= '0';
    elsif (CLK'event and CLK='1') then Q <= D; QN <= not D;
    end if;
  end process;
end Vdff74_b;

```

Табл. 8.8. VHDL-модель 16-разрядного регистра со многими функциональными возможностями

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vreg16 is
  port (CLK, CLKEN, OE_L, CLR_L: in STD_LOGIC;
        D: in STD_LOGIC_VECTOR(1 to 16); -- Input bus
        Q: out STD_ULOGIC_VECTOR (1 to 16)); -- Output bus (three-state)
end Vreg16;

architecture Vreg16 of Vreg16 is
  signal CLR, OE: STD_LOGIC; -- active-high versions of signals
  signal IQ: STD_LOGIC_VECTOR(1 to 16); -- internal Q signals
begin
  process(CLK, CLR_L, CLR, OE_L, OE, IQ)
  begin
    CLR <= not CLR_L; OE <= not OE_L;
    if (CLR = '1') then IQ <= (others => '0');
    elsif (CLK'event and CLK='1') then
      if (CLKEN='1') then IQ <= D; end if;
    end if;
    if OE = '1' then Q <= To_StdUlogicVector(IQ);
    else Q <= (others => 'Z'); end if;
  end process;
end Vreg16;

```

В этой более детальной функциональной модели обнаруживается, что выходные сигналы Q и QN не всегда являются дополнением один другого: это правило оказывается нарушенным, когда одновременно подаются сигналы сброса и установки в единичное состояние. Однако в этой модели не приняты во внимание такие временные характеристики, как задержка распространения, время установления и время удержания, поскольку этот материал выходит за рамки обсуждения возможностей языка VHDL в нашей книге.

Определяя входы данных и выходы как векторы, можно, конечно, смоделировать регистры с большим числом разрядов и включить дополнительные функции. В табл. 8.8, например, представлена модель 16-разрядного регистра с выходами с тремя состояниями, входом разрешения тактового сигнала, входом разрешения выхода и входом сброса. Вектор внутренних сигналов IQ используется для сохранения сигналов на выходах триггеров, а выходы с тремя состояниями и управление ими определяются так же, как это было сделано в разделе 5.6.4

8.3. Последовательностные ПЛУ

8.3.1. Биполярные последовательностные ПЛУ

Приведенная на рис. 8.17 ИС *PAL16R8* является представителем нового поколения последовательностных ПЛУ, изготовленных по биполярной (ТТЛ-) технологии. У этой микросхемы восемь основных входов, восемь выходов, общий тактовый сигнал и общий входной сигнал разрешения выхода; схема размещается в корпусе с 20 выводами.

Матрица И-ИЛИ в ИС *PAL16R8* — точно такая же, как у комбинационного ПЛУ *PAL16L8*. Но в ИС *PAL16R8* между матрицей И-ИЛИ и ее восемью выходами $O1-O8$ находятся переключающиеся по фронту D-триггеры. Ко всем триггерам подведен общий тактовый сигнал, подаваемый на вход CLK , и их состояние изменяется на нараста-

ОГРАНИЧЕНИЯ СИНТЕЗА

Теоретически первый оператор `elsif` в табл. 8.8 мог бы содержать все условия, которые должны выполняться для присвоения значения D сигналу IQ . Другими словами, он мог бы иметь вид: `“elsif (CLK 'event) and (CLK = '1') and (CLKEN = '1') then ...”`, а не быть вложенным оператором `if` для того, чтобы проверять значение $CLKEN$. Но сделано это так, как указано в программе, из весьма прагматических соображений.

При подготовке этой главы был использован VHDL-компилятор, способный синтезировать лишь подмножество языка VHDL; сегодня это справедливо для любого VHDL-компилятора. В частности, возможность употребления признака `“event”` для задания чувствительности к фронту ограничивалась только той формой, какая указана в данном примере и в ряде других примеров. Этим в процессе синтеза достигается отображение в переключающийся по фронту D-триггер. В нашем примере вложенный оператор `if`, проверяющий значение $CLKEN$, приводит к синтезу логики мультиплексирования на D-входах этих триггеров.

ющем фронте этого сигнала. Прежде чем попасть на внешний вывод ИС, выходной сигнал каждого триггера проходит через буфер с тремя состояниями; все буферы управляются общим сигналом разрешения выхода OE_L. Как и в комбинационной схеме PAL16L8, на внешних регистровых выходах ИС PAL16R8 вырабатывается сигнал, инверсный по отношению к тому, что ностуает с выхода матрицы И-ИЛИ.

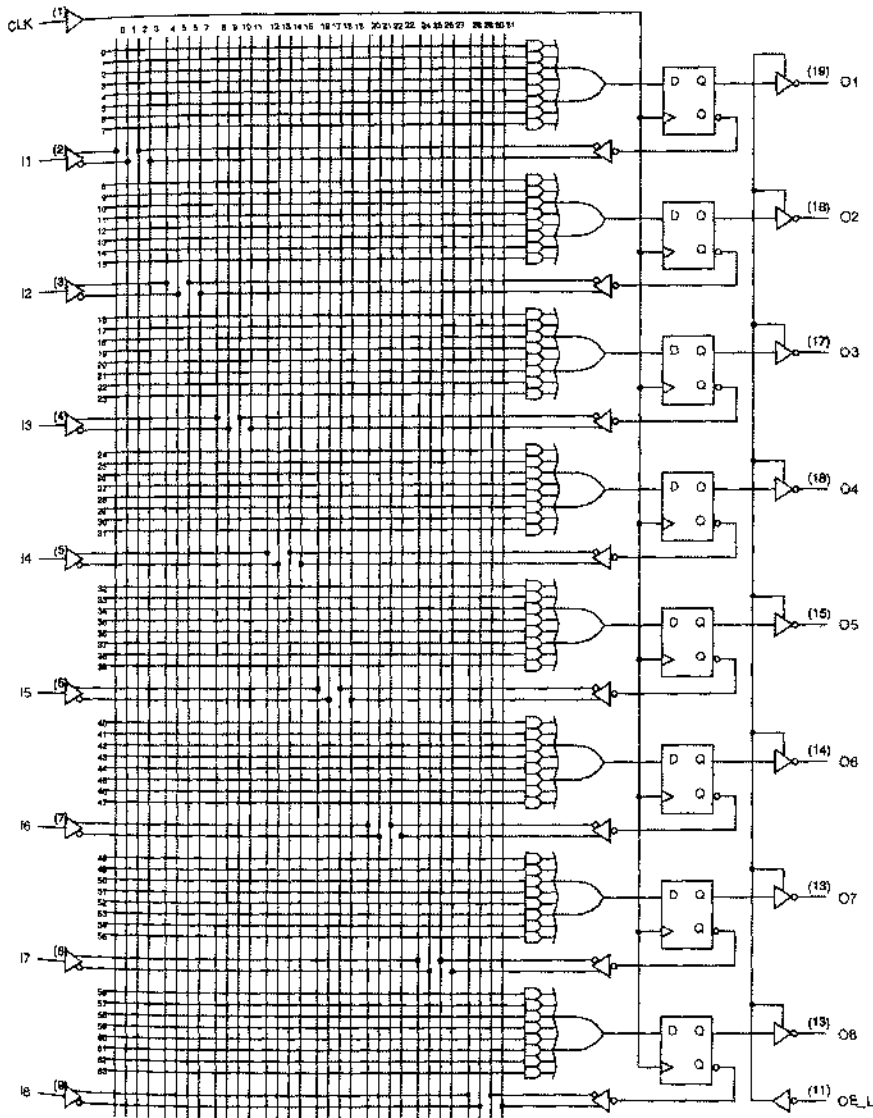


Рис. 8.17. Принципиальная схема ИС PAL16R8

Возможными входными сигналами для матрицы И-ИЛИ являются восемь основных входов I1-I8 и восемь выходов D-триггеров. Подавая сигналы с выходов

D-триггеров на матрицу И-ИЛИ, легко создавать регистры сдвига, счетчики и копейные автоматы общего вида. В отличие от комбинационных выходов ИС PAL16L8, выходные сигналы D-триггеров в схеме PAL16R8 доступны матрице И-ИЛИ независимо от того, разрешена выдача сигналов на выходы с тремя состояниями 01–08 или нет. Таким образом, внутренние триггеры могут переходить в следующее состояние, являющееся функцией текущего состояния, даже в том случае, когда выходы 01–08 занерты.

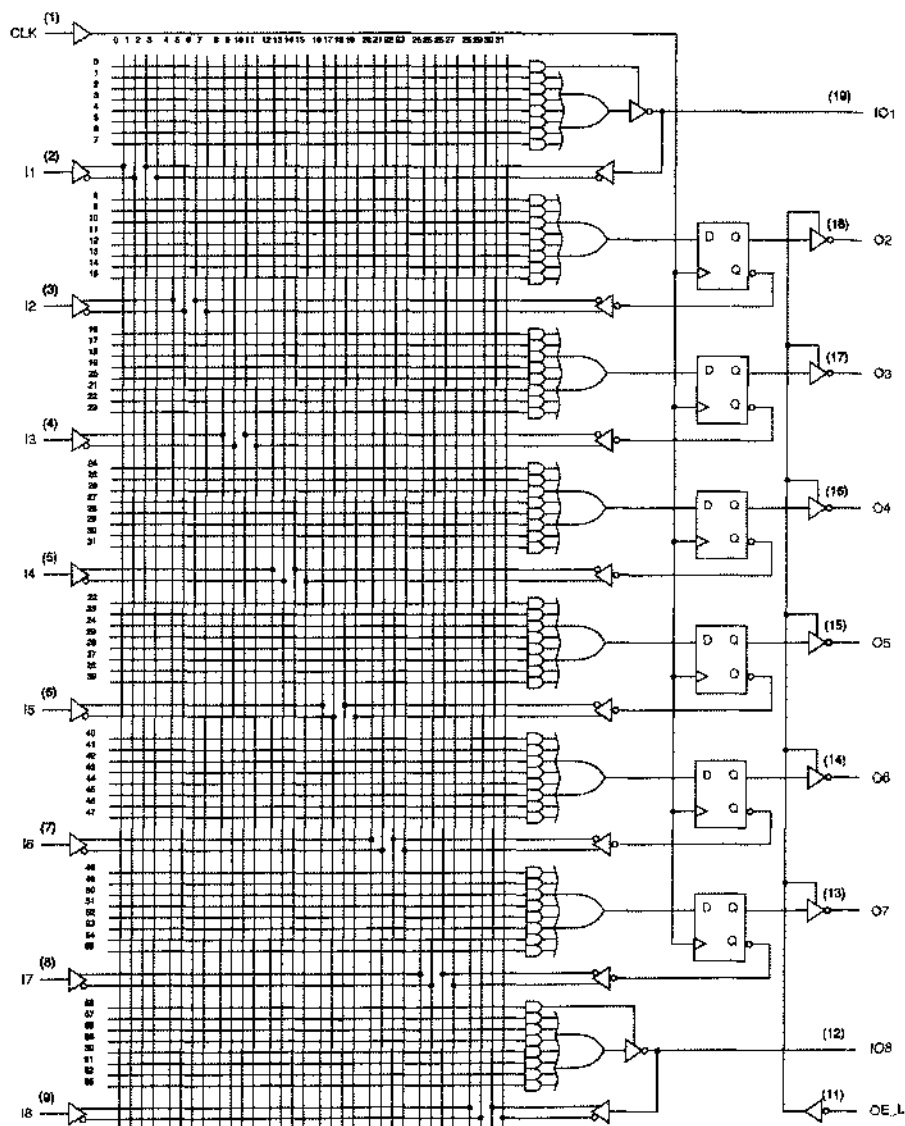


Рис. 8.18. Принципиальная схема ИС PAL16R8

Во многих приложениях бывают нужны в ПЛУ как комбинационные, так и последовательностные выходы. Производители ПЛУ отреагировали на эту потребность выпуском нескольких вариантов ИС типа PAL16R8, в которых у части выходных выводов D-триггеры опущены, а сами эти выходы наделены способностью быть входами и выходами подобно тому, как это сделано в отношении двунаправленных выводов в ИС PAL16L8. На рис. 8.18 в качестве примера представлена принципиальная схема ИС PAL16R6, у которой только шесть регистровых выходов. Два вывода — Ю1 и Ю8 — являются двунаправленными; они могут служить как входами, так и комбинационными выходами, и у них свои собственные сигналы управления третьим состоянием, аналогичные двунаправленным выводам ИС PAL16L8. Таким образом, возможными входными сигналами для матрицы И-ИЛИ являются сигналы на восьми внешних входах 11–18, выходные сигналы шести D-триггеров и сигналы на двух двунаправленных выводах Ю1 и Ю8.

В табл. 8.9 перечислены восемь стандартных биполярных ПЛУ с различным числом входов и выходов разного типа. Во всех микросхемах типа PAL16xx, указанных в этой таблице, одна и та же матрица И-ИЛИ, в которой имеется по восемь вентилях И на каждый выход с 16 возможными входными сигналами и их дополнениями у каждого вентиля И. Матрицы И-ИЛИ в ИС типа PAL20xx — такие же, как и в ИС типа PAL16xx, но с 20 возможными входными сигналами и их дополнениями. На рис. 8.19 указаны условные обозначения всех ПЛУ, упомянутых в табл. 8.9.

Табл. 8.9. Характеристики стандартных биполярных ПЛУ

Название микросхемы	Число выводов в корпусе	Число входов вентиля И	Число входов у матрицы И			
			Внешние входы	Двунаправленные комбинационные выходы	Регистровые выходы	Число комбинационных выходов
PAL16L8	20	16	10	6	0	2
PAL16R4	20	16	8	4	4	0
PAL16R6	20	16	8	2	6	0
PAL16R8	20	16	8	0	8	0
PAL20L8	24	20	14	6	0	2
PAL20R4	24	20	12	4	4	0
PAL20R6	24	20	12	2	6	0
PAL20R8	24	20	12	0	8	0

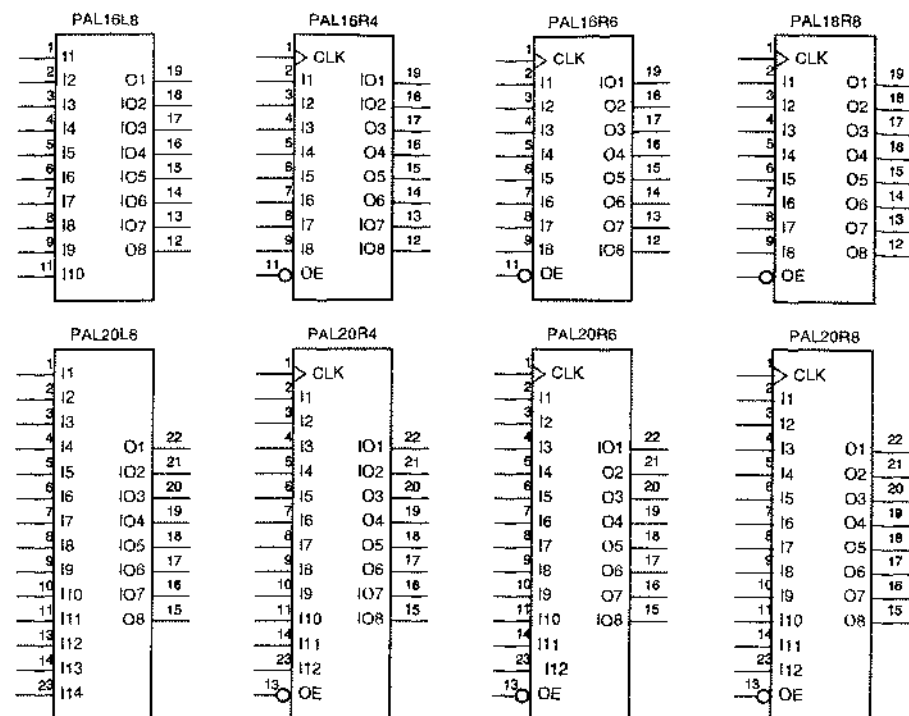


Рис. 8.19. Условные обозначения биополярных комбинационных и последовательностных ПЛУ

8.3.2. Последовательностные устройства типа GAL

В разделе 5.3.3 мы представили электрически стираемое ПЛУ GAL16V8. Два «управляющих архитектурой» программируемых соединения позволяют выбрать одну из трех основных конфигураций этого устройства. Одна из них – конфигурация 16V8C («C» – от слова «complex», «сложный») – была описана в разделе 5.3.3 и приведена там на рис. 5.27, по своей структуре она похожа на биополярное комбинационное ПЛУ PAL16L8. Конфигурация 16V8S («S» – от слова «simple», «простой») немного отличается от конфигурации 16V8C возможностями комбинационной логики (подробнее см. в отступлении от основного текста дальше в этом разделе).

Третья конфигурация 16V8R обеспечивает наличие триггеров на всех выходах или на некоторых из них. На рис. 8.20 показана структура этого устройства в том случае, когда триггеры имеются на всех выходах. Все триггеры управляются общим тактовым сигналом, подаваемым на вывод 1, как и в биополярных ПЛУ, рассмотренных в предыдущем разделе. Точно так же все выходные буферы управляются общим сигналом разрешения выхода, подаваемым на вывод 11.

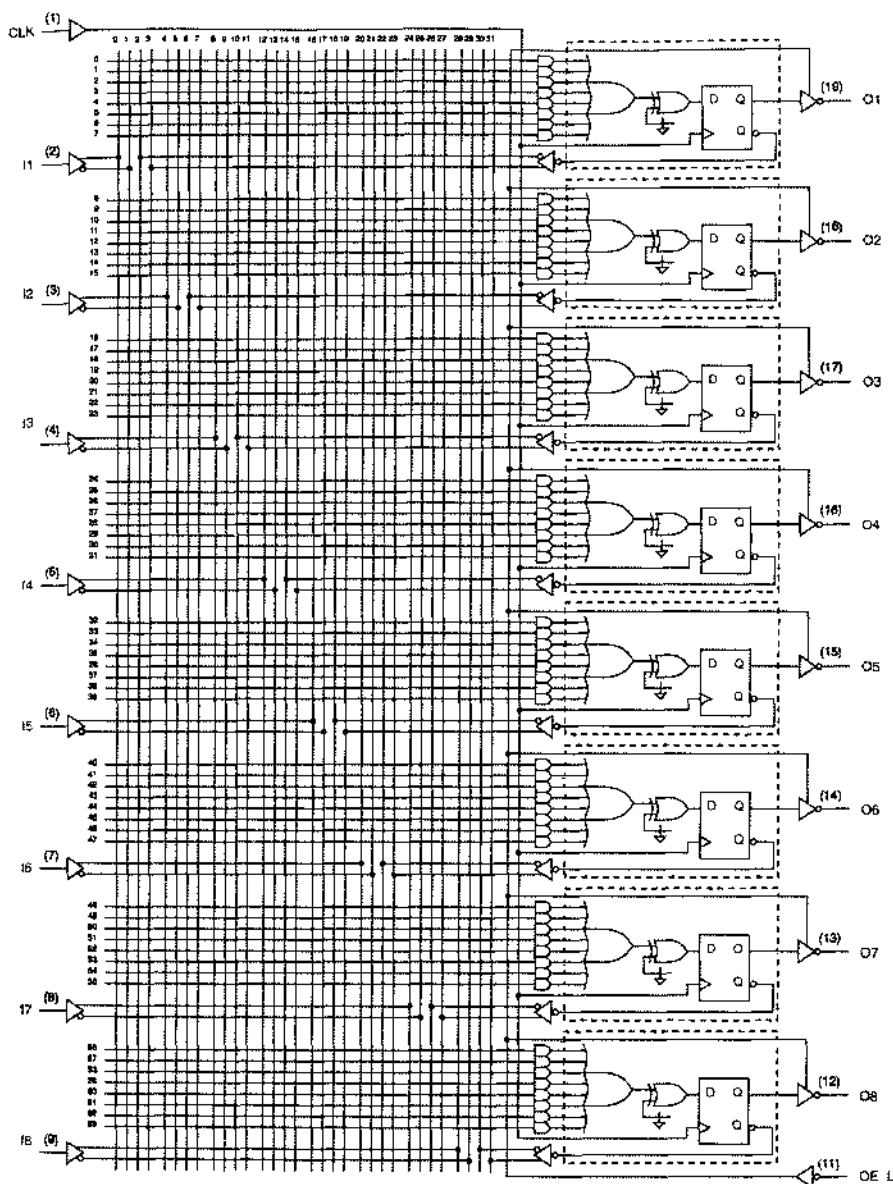


Рис. 8.20. Принципиальная схема ИС 16V8 в «регистровой» конфигурации

Схема внутри каждого блока, обведенного пунктирной линией на рис. 8.20, носит название *макроячейки выходной логики (output logic macrocell)*. ИС 16V8R значительно более гибкое устройство, нежели микросхема PAL16R8, поскольку можно по отдельности задавать конфигурацию каждой макроячейки так, чтобы

обойти триггер, то есть обеспечить наличие комбинационного выхода. На рис. 8.21 показаны две возможные конфигурации макроячейки в ИС 16V8R: (а) регистровая конфигурация и (b) комбинационная конфигурация. Следовательно, устройство можно запрограммировать так, чтобы любой набор выходов был регистровым или комбинационным вплоть до полного числа выходов, равного 8.

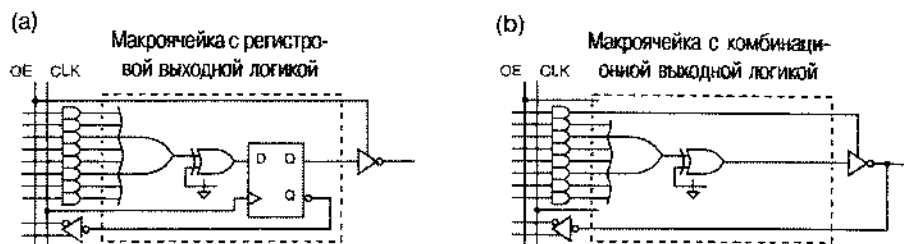


Рис. 8.21. Макроячейки выходной логики для ИС 16V8R: (а) регистровая конфигурация; (b) комбинационная конфигурация

ИС 20V8 похожа на микросхему 16V8, но поставляется в корпусе с 24 выводами; ее четыре дополнительных вывода служат только входами. У каждого вентиля И в ИС 20V8 имеется 20 входов, па что указывает число “20” в ее названии.

«ПРОСТАЯ» КОНФИГУРАЦИЯ 16V8S

«Простая» конфигурация 16V8S ИС GAL16V8 используется довольно редко, так как ее возможности являются, по большей части, подмножеством функций, выполняемых этой ИС в конфигурации 16V8C. В варианте 16V8S один из термов И на каждом выходе заменен программируемым соединением, позволяющим удерживать выходной буфер в открытом или в запертом состоянии. Другими словами, каждый выходной вывод можно запрограммировать таким образом, чтобы выдача сигнала на него была всегда разрешена или всегда запрещена (кроме выводов 15 и 16; выходы на эти выводы всегда разрешены). Сигналы, возникающие на всех выходных выводах (кроме выводов 15 и 16), могут быть входными сигналами матрицы И независимо от того, заперт буфер или открыт.

Единственное достоинство конфигурации 16V8S по сравнению с вариантом 16V8C заключается в том, что у вентиля ИЛИ на каждом выходе имеется по восемь входных термов И, а не по семь. Назначение архитектуры 16V8S состояло, главным образом, в том, чтобы эмулировать уже вышедшие из употребления биполярные ПЛУ, в которых допускалось наличие восьми термов-произведений на один выход и входами были выводы 12 и 19, которые в конфигурации 16V8C не являются входами. Надлежащим образом запрограммированная микросхема 16V8 в конфигурации “S” совместима по выводам с ИС PAL10H8, PAL12H6, PAL14114, PAL16H2, PAL10L8, PAL12L6, PAL14L4 и PAL16L2 и ею можно воспользоваться для замены перечисленных микросхем.

В корпусе с 24 выводами поставляется также ИС 22V10, общая структура которой приведена на рис. 8.22; но эта микросхема является более гибким устройством, нежели ИС 20V8. У ИС 22V10 нет такого «управления архитектурой», как у 16V8 и 20V8, однако с ее помощью можно не только реализовать любую функцию, выполняемую микросхемой 20V8, но достичь еще большего. ИС 22V10 допускает наличие большего числа термов-произведений, чем ИС 20V8, у схемы 22V10 на два входа общего назначения больше и большие возможности управления выходом. Главные отличия ИС 22V10 от ИС 20V8 состоят в следующем:

- Как и в архитектуре 20V8R, каждую макроячейку выходной логики можно запрограммировать так, чтобы она имела регистровую или комбинационную конфигурацию. Однако сами макроячейки в ИС 22V10 отличаются от макроячеек ИС 16V8 и ИС 20V8, как это видно из рис. 8.23.
- Один из термов-произведений управляет выходным буфером независимо от того, какая конфигурация выбрана для макроячейки: регистровая или комбинационная.
- Для любого выхода имеется, по меньшей мере, восемь термов-произведений, независимо от выбранной конфигурации макроячейки выходной логики. У «внутренних» выводов число термов-произведений даже больше; оно доходит до 16 для каждого из двух «самых внутренних» выводов. (Речь идет о средних выводах справа на рис. 8.22, где графическое изображение соответствует расположению выводов у DIP-корпуса с 24 выводами.)
- Тактовый сигнал, подаваемый на вывод 1, может играть роль комбинационного входного сигнала в любом терм-произведении.
- Глобальный асинхронный сигнал сброса генерируется в виде одного терм-произведения; с его помощью все внутренние триггеры сбрасываются в 0.
- Глобальный синхронный сигнал установки в единичное состояние генерируется в виде одного терм-произведения; этим сигналом осуществляется перевод всех внутренних триггеров в состояние 1, и происходит это на парастоящем фронте тактового сигнала.
- Как и в схемах 16V8 и 20V8, полярность выходных сигналов в ИС 22V10 программируется. Однако в случае регистровой конфигурации изменение полярности осуществляется на выходе D-триггера, а не на его входе. Это затрагивает некоторые детали программирования, когда необходимо изменить полярность, но не влияет на возможность реализации данной функции микросхемой 22V10 в целом. Различие, обусловленное тем, в каком месте происходит изменение полярности, становится явным при программировании ПЛУ на том или ином языке, например, на языке ABEL.

В 90-е годы ИС 16V8, 20V8 и 22V10 были наиболее популярными ПЛУ и их применение было самым рациональным с точки зрения стоимости (впрочем, см. замечание, вынесенное за пределы основного текста, в конце этого параграфа). На рис. 8.24 приведены условные обозначения этих трех микросхем. Большинство примеров в оставшейся части этой главы относится к проектированию соответствующих устройств на основе наименьшей из этих ИС, то есть на основе микросхемы 16V8.

МИКРОСХЕМЫ PAL И GAL

ИС типа GAL, в том числе GAL16V8 и GAL20V8, впервые были выпущены фирмой Lattice Semiconductor в середине 80-х годов. За этими схемами последовала совместимая с ними по выводам ИС PALCE16V8 фирмы Advanced Micro Devices ("С" в названии означает, что схема выполнена по КМОП-технологии, а "Е" – что она является стираемой). Несколько других производителей также выпускают совместимые устройства, но с другой маркировкой. В этой главе мы называем эти микросхемы их первоначальными именами 16V8, 20V8 и 22V10, не ставя себе целью представить в деталях всю номенклатуру различных производителей.

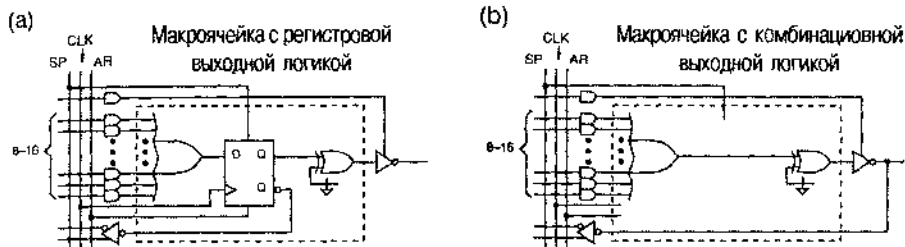


Рис. 8.23. Макроячейки выходной логики для ИС 22V10: (а) регистровая конфигурация; (б) комбинационная конфигурация (CLK – тактовый сигнал, SP – синхронная установка в 1, AR – асинхронный сброс)

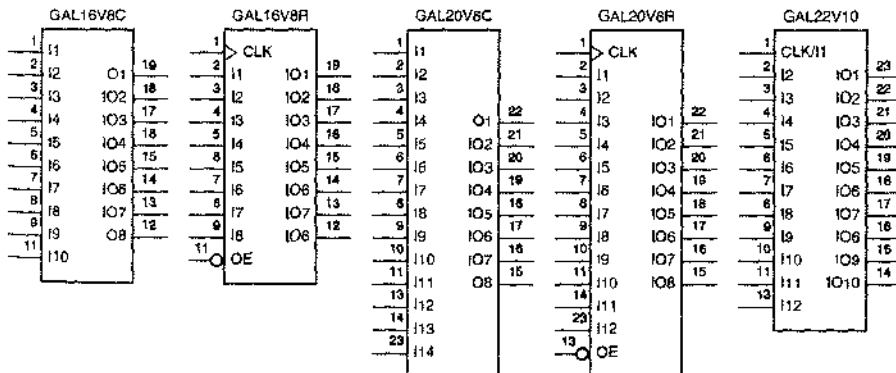


Рис. 8.24. Условные обозначения популярных микросхем типа GAL.

8.8.3. Временные характеристики ПЛУ

Комбинационные и последовательностные ПЛУ характеризуются несколькими временными параметрами. Самые важные из них указаны на рис. 8.25 и означают следующее:

t_{PD} – задержка распространения сигнала от вывода внешнего входа, от двуправленного вывода или от входа «со стороны обратной связи» до ком-

бинационного выхода; *вход со стороны обратной связи (feedback input)* — это внутренний вход матрицы И-ИЛИ, на который подается сигнал с регистравого выхода внутренней макроячейки; данный параметр относится к комбинационным выходам;

t_{CO} — задержка распространения тактового сигнала CLK, отсчитываемая от нарастающего фронта этого сигнала до внешнего выхода; этот параметр относится к регистровым выходам;

t_{CF} — задержка распространения от нарастающего фронта сигнала CLK до регистравого выхода макроячейки, соединенного с одним из входов матрицы И-ИЛИ со стороны обратной связи; когда значение t_{CF} задано, оно обычно меньше, чем t_{CO} ; однако некоторые производители не указывают значение t_{CF} ; в этом случае следует полагать, что $t_{CF} = t_{CO}$; этот параметр относится к регистровым выходам;

t_{SU} — время установления, в пределах которого входной сигнал должен удерживаться неизменным перед нарастающим фронтом тактового сигнала CLK; этот параметр относится к сигналам, поступающим на D-входы триггеров с внешних входов, с двенаправленных выводов и с входов со стороны обратной связи;

t_H — время удержания, в пределах которого входной сигнал должен оставаться неизменным после нарастающего фронта тактового сигнала CLK; этот параметр также относится к сигналам поступающим на D-входы триггеров;

f_{max} — этим параметром определяется тактирование в схеме ПЛУ: он представляет собой частоту, на которой данное ПЛУ может работать надежно; его величина обратна наименьшему периоду тактового сигнала; из указанных выше временных характеристик можно вывести два значения этого параметра в зависимости от того, какая обратная связь реализована в вашем устройстве — внешняя или внутренняя.

При *внешней обратной связи (external feedback)* сигнал с регистравого выхода данного ПЛУ подается на вход другого регистравого ПЛУ с похожими временными характеристиками; в этом случае для надлежащей работы необходимо, чтобы сумма t_{CO} первого ПЛУ и t_{SU} второго ПЛУ не превосходила период тактового сигнала.

При *внутренней обратной связи (internal feedback)* сигнал с регистравого выхода ПЛУ заводится снова на входы регистра в том же самом ПЛУ; в этом случае сумма t_{CF} и t_{SU} не должна превосходить период тактового сигнала.

Существует несколько градаций каждого из описанных ПЛУ, различающихся своим быстродействием. Характеристика быстродействия обычно указывается в виде приставки к названию микросхемы, например “16V8-10”; как правило, приставка выражает собой значение t_{PD} в наносекундах. В табл. 8.10 приведены временные характеристики нескольких распространенных биполярных ПЛУ и ПЛУ, выполненных по КМОП-технологии. Заметьте, что только один столбец — t_{PD} — относится к комбинационным выходам микросхемы, тогда как остальные четыре столбца относятся к регистровым выходам. Значения всех временных параметров указаны для наихудшего случая в пределах рабочего диапазона микросхем гражданского применения.

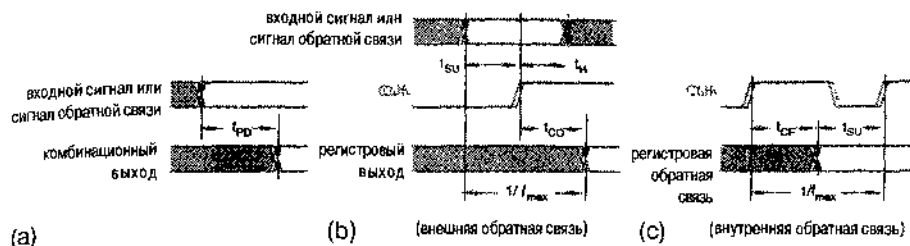


рис. 8.25. Временные характеристики ПЛУ (CLK – тактовый сигнал)

Табл. 8.10. Временные характеристики распространенных ПЛУ в наносекундах

Название ИС	Приставка	t_{PD}	t_{CO}	t_{CF}	t_{SU}	t_H
PAL16L8, PAL16Rx, PAL20L8, PAL20Rx	-5	5	4	—	4.5	0
PAL16L8, PAL16Rx, PAL20L8, PAL20Rx	-7	7.5	6.5	—	7	0
PAL16L8, PAL16Rx, PAL20L8, PAL20Rx	-10	10	8	—	10	0
PAL16L8, PAL16Rx, PAL20L8, PAL20Rx	B	15	12	—	15	0
PAL16L8, PAL16Rx, PAL20L8, PAL20Rx	B 2	25	15	—	25	0
PAL16L8, PAL16Rx, PAL20L8, PAL20Rx	A	25	15	—	25	0
PALCE16V8, PALCE20V8	-5	5	4	—	3	0
GAL16V8, GAL20V8	7	7.5	5	3	5	0
GAL16V8, GAL20V8	-10	10	7.5	6	7.5	0
GAL16V8, GAL20V8	-15	15	10	8	12	0
GAL16V8, GAL20V8	-25	25	12	10	15	0
PAICF22V10	-5	5	4	—	3	0
PAICF22V10	-7	7.5	4.5	—	4.5	0
GAL22V10	-10	10	7	2.5	7	0
GAL22V10	-15	15	8	2.5	10	0
GAL22V10	-25	25	15	13	15	0

При использовании последовательностных ПЛУ в критических по времени условиях важно помнить, что обычно у них время установления больше, чем у готовых переключающихся по фронту регистров, выполненных по той же технологии; это имеет место из-за задержки в матрице И-ИЛИ на каждом D-входе. С другой стороны в типичных условиях время удержания применительно к ПЛУ фактически оказывается отрицательным из-за той же самой задержки сигнала при прохождении через матрицу И-ИЛИ. Однако рассчитывать на отрицательное время удержания нельзя: в худшем случае этот параметр обычно равен нулю.

СКОЛЬКО ЭТО СТОИТ?

Теперь, когда вам известны возможности различных ПЛУ, вы можете спросить: «Почему бы всегда не применять лучшие из имеющихся ПЛУ?» Например: даже в том случае, когда наше устройство помещается в ИС с 20 выводами 16V8, почему бы нам не воспользоваться чуть большей ИС 20V8 с 24 выводами, чтобы иметь про запас лишние входы, которые могут пригодиться в случае каких-либо неприятностей? Или: коль скоро мы решим использовать ИС 20V8, то почему бы нам не взять немного лучшую ИС 22V10, которая поставляется в таком же корпусе с 24 выводами?

В реальном мире проектирования и конструирования ограничением является цена. Если бы нас не сдерживали соображения стоимости, то аргументы предыдущего абзаца можно было бы распространить *ad nauseam* на ИС типа CPLD и FPGA с еще большими возможностями.

С цифровыми микросхемами типа ПЛУ, CPLD и FPGA дело обстоит точно так же, как с автомобилями и топкими винами: цена не всегда пропорциональна их возможностям и полезности. В частности, чем ближе изделие — с точки зрения предоставляемых им возможностей — к последним достижениям в области технологии, тем большая часть стоимости, вероятнее всего, будет платой за новизну. Таким образом, при выборе изделия вам необходимо оценить, сколько будут стоить различные варианты. Например, дорогие ИС типа CPLD и FPGA с большой плотностью упаковки позволяют вам реализовать устройство в одном кристалле, внутренние функции в котором легко изменить при необходимости. С другой стороны, можно сэкономить на стоимости компонентов, выбрав нару или большее число ПЛУ или ИС типа CPLD и FPGA с меньшей плотностью упаковки, но при этом увеличится площадь, занимаемая микросхемами на печатной плате, и потребляемая ими мощность, а кроме того, внесение изменений в ваш проект впоследствии станет более затруднительным (так как при изготовлении платы соединения между микросхемами должны быть зафиксированы).

Не следует также упускать из виду, чторяду со стоимостью всегда необходимо принимать во внимание ясность проекта в целом и выгоды от создания устройства, которое будет пользоваться успехом (то есть принесет прибыль). Минимизация стоимости изделия обычно включает массу соображений экономического и технического плана, а также соображений здравого смысла, которые очень далеки от рутинных алгоритмических методов минимизации числа вентилей из главы 4.

8.4. Счетчики

В общем случае *счетчиком* (*counter*) называют любую тактируемую последовательностную схему, у которой диаграмма состояний представляет собой единственное кольцо (рис. 8.26). *Модулем счета* (*modulus*) называется число состояний в этом кольце. О счетчике с m состояниями говорят как о *счетчике с модулем счета m* (*modulo- m counter*); иногда его называют также *делителем частоты на m* (*divide-by- m counter*). У счетчика с модулем счета, не равным степени 2, есть лишние состояния, в которые он не попадает при нормальной работе.

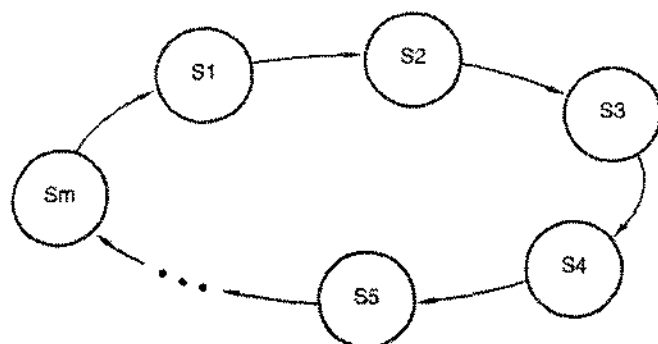


Рис. 8.26. Общий вид диаграммы состояний счетчика – единственное кольцо

По-видимому, самыми распространенными являются n -разрядные двоичные счетчики (n -bit binary counter). Такой счетчик состоит из n триггеров, и у него имеется 2^n состояний, через которые он проходит в последовательности 0, 1, 2, ..., $2^n - 1$, 0, 1, Кодом каждого состояния служит соответствующее n -разрядное двоичное число.

8.4.1. Счетчики с последовательным переносом

При любом значении n можно так построить n -разрядный двоичный счетчик, что в нем будут только n триггеров и не будет никаких других компонентов. Для $n = 4$ такой счетчик показан на рис. 8.27. Напомним, что состояние Т-триггера меняется (на противоположное) с каждым нарастающим фронтом сигнала на его тактовом входе. Таким образом, содержимое того или иного разряда в счетчике меняется на противоположное тогда и только тогда, когда значение бита в разряде, непосредственно предшествующем этому разряду, изменяется с 1 на 0. Это соответствует двоичному счету в прямом направлении: когда бит, хранящийся в данном разряде, изменяется с 1 на 0, возникает перенос в следующий по старшинству разряд. Такой счетчик называют *счетчиком с последовательным переносом (ripple counter)*, так как информация о переносе поочередно передается от младших разрядов к старшим, по одному биту за раз.

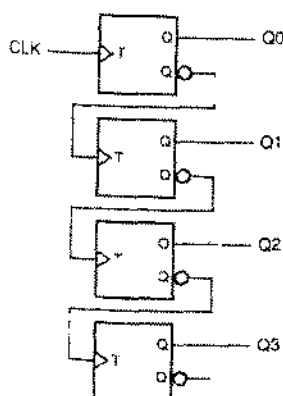


Рис. 8.27. 4-разрядный двоичный счетчик с последовательным переносом

8.4.2. Синхронные счетчики

Хотя для счетчика с последовательным переносом требуется меньше компонентов, чем для двоичного счетчика любого другого типа, за это приходится платить наименьшим быстродействием по сравнению с любыми другими счетчиками. В худшем случае, когда должен измениться бит в самом старшем разряде, выходной сигнал примет правильное значение только спустя время, равное $n \cdot t_{TQ}$, после нарастающего фронта тактового сигнала CLK, где t_{TQ} — задержка распространения в Т-триггере от входа до выхода.

В *синхронном счетчике (synchronous counter)* к тактовым входам всех триггеров подводится один и тот же общий тактовый сигнал CLK, так что изменения значений сигналов на выходах всех триггеров происходят в один и тот же момент времени с задержкой только на t_{TQ} наносекунд. Как видно из рис. 8.28, для этого нужно воспользоваться Т-триггерами со входом разрешения; сигнал на выходе триггера примет противоположное значение в момент, задаваемый нарастающим фронтом сигнала на его входе Т, только в том случае, если сигнал разрешения EN имеет активный уровень. Какие именно триггеры перейдут в состояние, противоположное предыдущему, на очередном нарастающем фронте сигнала на входе Т, определяется комбинационной логикой, включенной на входах разрешения EN.

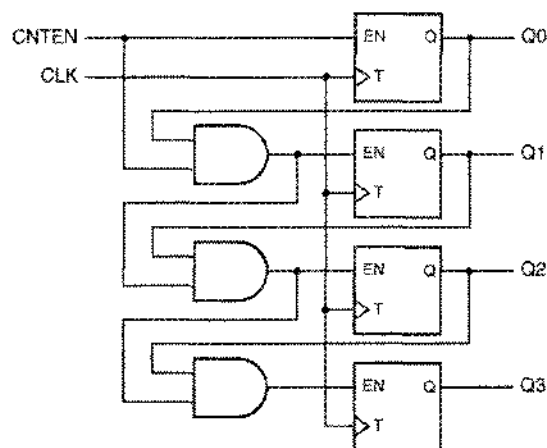


Рис. 8.28. 4-разрядный синхронный двоичный счетчик с последовательной логикой разрешения

На рис. 8.28 показано, что счетчик можно снабдить главным сигналом разрешения CNTEN. Любой из Т-триггеров может переключиться тогда и только тогда, когда сигнал CNTEN имеет единичное значение и равны 1 биты во всех разрядах, младше данного. Как и в случае двоичного счетчика с последовательным переносом, n -разрядный синхронный счетчик можно построить так, чтобы на один разряд приходилось фиксированное число логических схем; в данном случае в каждом разряде необходимы Т-триггер с входом разрешения и 2-входовой вентиль И.

Счетчик, изображенный на рис. 8.28, иногда называют *последовательным синхронным счетчиком* (*synchronous serial counter*), поскольку сигналы разрешения проходят через комбинационную логику последовательно от младшего разряда к старшему. Если период тактового сигнала слишком мал, то изменение в младшем разряде счетчика может не успеть дойти за это время до старшего разряда. Это затруднение преодолено в схеме на рис. 8.29, где сигнал разрешения на входе разрешения EN каждого триггера вырабатывается соответствующим вентилем И всего лишь с одним уровнем логики. В результате получается схема двоичного счетчика с самым высоким быстродействием, который носит название *параллельного синхронного счетчика* (*synchronous parallel counter*).

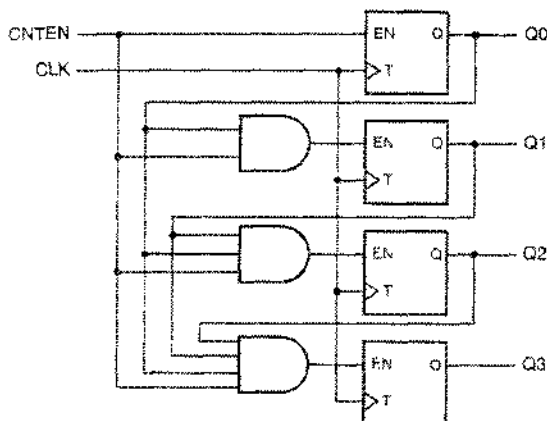


Рис. 8.29. 4-разрядный синхронный двоичный счетчик с параллельной логикой разрешения

8.4.3. Счетчики в ИС средней степени интеграции и их применение

Самым популярным счетчиком в ИС средней степени интеграции является 4-разрядный синхронный двоичный счетчик 74х163 с входами сброса и загрузки; сигналы на этих входах имеют низкий активный уровень. Традиционное условное обозначение такого счетчика приведено на рис. 8.30. Работа этого счетчика описывается таблицей состояний (табл. 8.11), а его принципиальная схема показана на рис. 8.31.

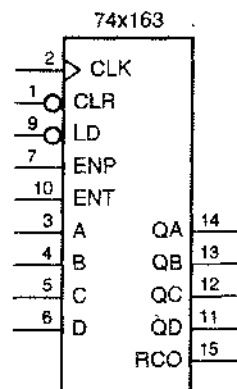


Рис. 8.30. Традиционное условное обозначение ИС 74х163

Табл. 8.11. Таблица состояний 4-разрядного двоичного счетчика 74х163

Входы				Текущее состояние				Следующее состояние			
CLR_L	LD_L	ENT	ENP	QD	QC	QB	QA	QD*	QC*	QB*	QA*
0	x	x	x	x	x	x	x	0	0	0	0
1	0	x	x	x	x	x	x	D	C	B	A
1	1	0	x	x	x	x	x	QD	QC	QB	QA
1	1	x	0	x	x	x	x	QD	QC	QB	QA
1	1	1	1	0	0	0	0	0	0	0	1
1	1	1	1	0	0	0	1	0	0	1	0
1	1	1	1	0	0	1	0	0	0	1	1
1	1	1	1	0	0	1	1	0	1	0	0
1	1	1	1	0	1	0	0	0	1	0	1
1	1	1	1	0	1	0	1	0	1	1	0
1	1	1	1	0	1	1	0	0	1	1	1
1	1	1	1	0	1	1	1	1	0	0	0
1	1	1	1	1	0	0	0	1	0	0	1
1	1	1	1	1	0	0	1	1	0	1	0
1	1	1	1	1	0	1	0	1	0	1	1
1	1	1	1	1	0	1	1	1	1	0	0
1	1	1	1	1	1	0	0	1	1	0	1
1	1	1	1	1	1	0	1	1	1	1	0
1	1	1	1	1	1	1	0	1	1	1	1
1	1	1	1	1	1	1	1	0	0	0	0

Внутри ИС '163 используются не Т-триггеры, а D-триггеры, чтобы упростить функции загрузки и сброса. На D-вход каждого триггера сигнал поступает с выхода 2-входового мультиплексора, состоящего из вентиля ИЛИ и двух вентилях И. Выходной сигнал мультиплексора равен 0, если подан входной сигнал CLR_L. В противном случае верхний из вентилях И пропускает входной сигнал данных (A, B, C или D) на выход, если подан сигнал LD_L. Если ни у одного из сигналов CLR_L и LD_L уровень не является активным, то нижний из вентилях И пропускает на выход мультиплексора выходной сигнал вентиля ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ.

Функция счета в ИС '163 выполняется с помощью вентилях ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ. На один из входов этого вентиля в каждом разряде поступает бит, хранящийся в этом разряде (QA, QB, QC или QD); на другой вход подана логическая 1, благодаря чему на выходе этого вентиля вырабатывается дополнение к биту, хранящемуся в данном разряде, но только в том случае, когда оба сигнала разрешения ENP и ENT имеют активный уровень и во всех разрядах счетчика.

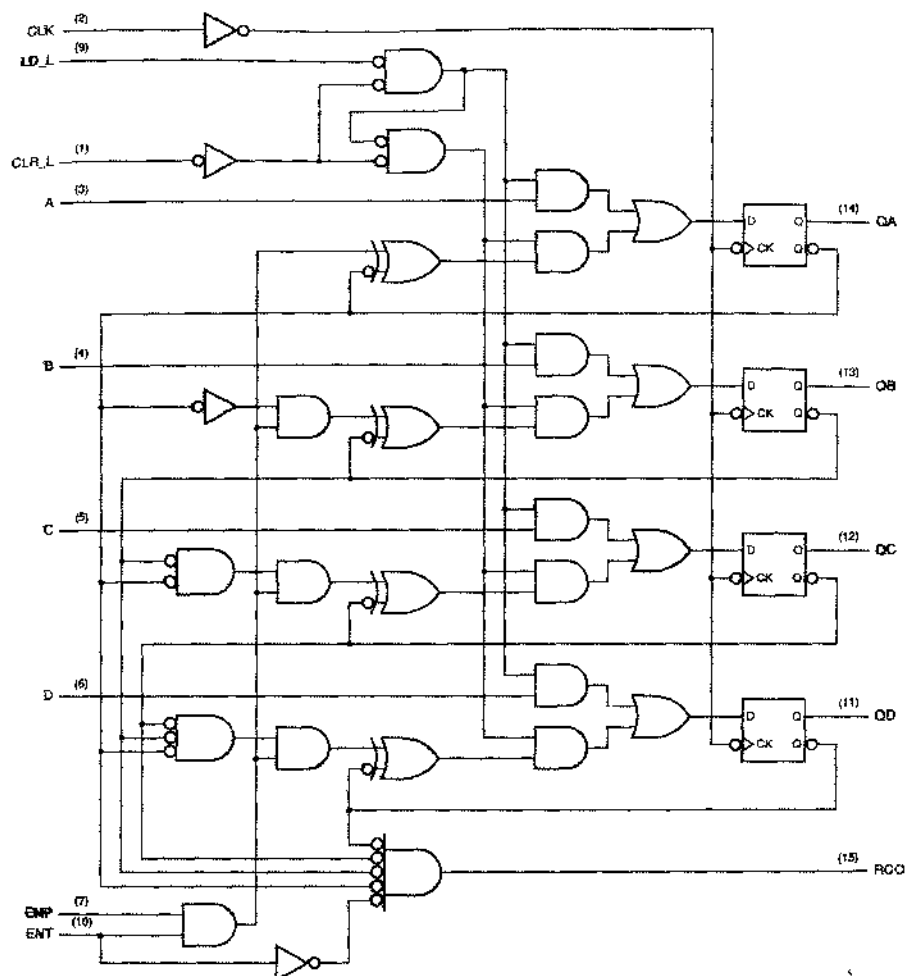


Рис. 8.31. Принципиальная схема синхронного 4-разрядного двоичного счетчика 74x163 с цоколевкой для стандартного DIP-корпуса с 16 выводами

младше данного, биты равны 1. Сигнал RCO («выход сквозного переноса») означает наличие перепоса из самого старшего разряда; он равен 1, когда равны 1 биты, хранящиеся во всех разрядах счетчика, и подан сигнал разрешения ENT.

Несмотря на наличие входа разрешения, счетчики, выполненные в виде ИС средней степени интеграции, часто *работают в непрерывном режиме (free-running counters)*, когда счет разрешен постоянно. На рис. 8.32 показано, какие соединения необходимо осуществить, чтобы счетчик '163 работал в таком режиме, а на рис. 8.33 приведены соответствующие этому режиму временные диаграммы. Обратите внимание: начиная с сигнала QA, частота переключений в каждом следующем сигнале вдвое меньше, чем в предыдущем. Таким образом, в режиме непрерывного счета ИС '163 может играть роль делителя частоты на 2, 4, 8 или 16; при этом ненужные старшие разряды игнорируются.

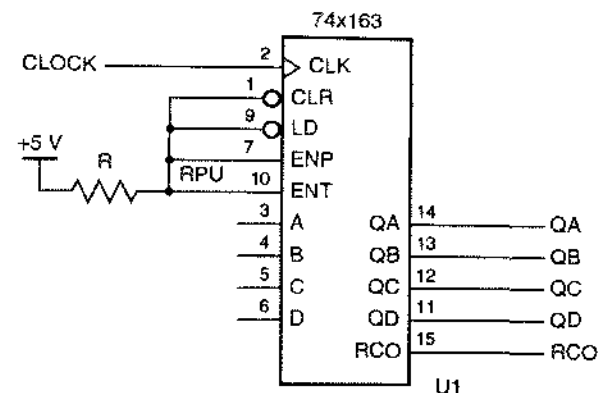


Рис. 8.32. Включение ИС 74x163 для работы в режиме непрерывного счета

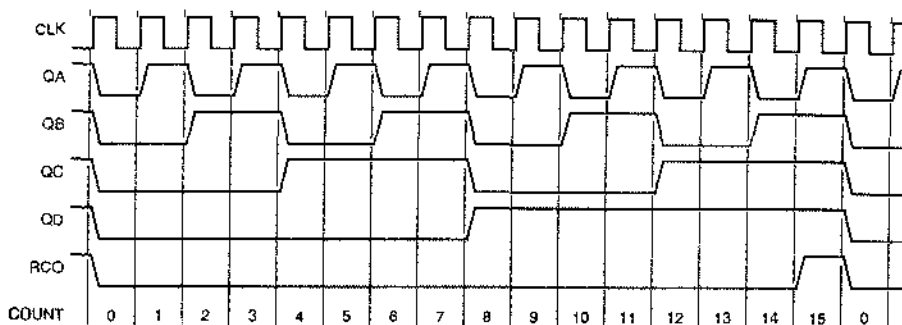


Рис. 8.33. Временные диаграммы тактового сигнала и сигналов на выходах отдельных разрядов в делителе частоты на 16

Заметьте, что счетчик '163 является полностью синхронным; это означает, что сигналы на его выходах изменяются только на нарастающем фронте сигнала CLK. Но в некоторых приложениях бывает необходимо осуществлять сброс асинхронно; это позволяет ИС 74x161. У микросхемы '161 такая же цоколевка, как и у ИС '163, но внутри нее вход CLR_L подключен к асинхронным входам сброса ее триггеров.

Другими вариантами счетчиков с точно такой же цоколевкой являются ИС 74x160 и 74x162; их функции в целом такие же, как и у микросхем '161 и '163, за исключением того, что в них последовательность счета изменена: за состоянием 9 следует состояние 0. Другими словами, эти ИС представляют собой счетчики по модулю 10; их иногда называют *декадными счетчиками* (*decade counters*). На рис. 8.34 приведены временные диаграммы для счетчиков '160 и '162 в режиме непрерывного счета. Хотя частота сигналов на выходах QD и QC равна одной десятой от частоты сигнала CLK, коэффициент заполнения у сигналов QD и QC не равен 50%, а сигнал QB, хотя и имеет частоту, в пять раз меньшую, чем частота тактового сигнала, его коэффициент заполнения не остается постоянным. Позднее в этом разделе мы покажем, как осуществляется деление частоты на 10 с 50%-ным коэффициентом заполнения у выходного сигнала

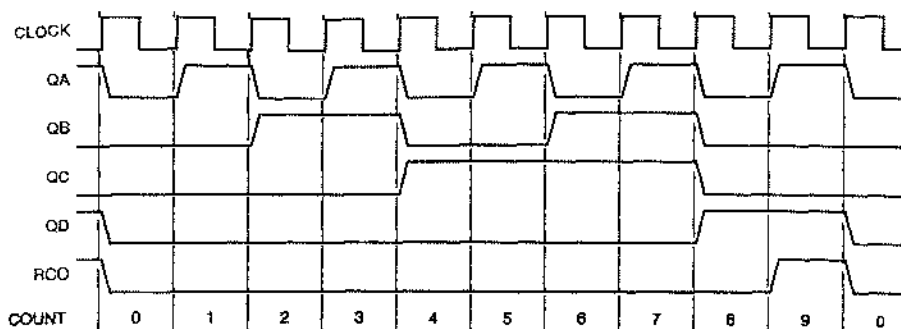


Рис. 8.34. Временные диаграммы тактового сигнала и сигналов на выходах отдельных разрядов делителя частоты на 10 в режиме непрерывного счета

Счетчик '163 сам по себе считает по модулю 16, но с помощью сигналов CLR_L и LD_L его можно заставить считать по меньшему модулю, чем 16, укоротив последовательность состояний. На рис. 8.35, например, показано использование ИС '163 в качестве счетчика по модулю 11. Когда счетчик находится в состоянии 15, на выходе RCO возникает единичный сигнал, который заставляет счетчик перейти в следующее состояние, равное 5; поэтому схема считает от 5 до 15 и снова начинает счет с 5, так что всего в цикле счета 11 состояний.

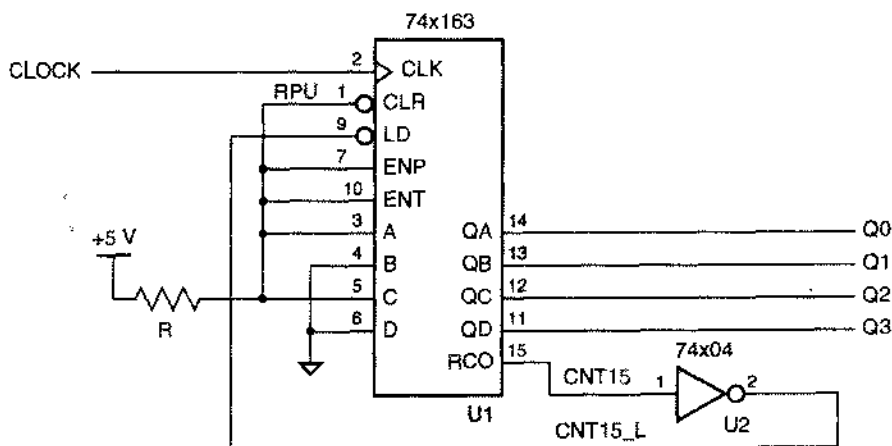


Рис. 8.35. Применение ИС 74x163 в качестве счетчика по модулю 11 с последовательностью счета 5, 6, ..., 15, 5, 6, ...

На рис. 8.36 демонстрируется другой подход к решению задачи о счете по модулю 11. В этой схеме для обнаружения состояния счетчика, равного 10, используется вентиль И-НЕ; сигнал, возникающий на выходе этого вентиля, заставляет счетчик перейти в состояние 0. Обратите внимание на то, что для обнаружения состояния 10 (в двоичной записи – 1010) нужен вентиль только с двумя входами. Хотя для обнаружения состояния $CNT10 = Q3 \cdot Q2' \cdot Q1 \cdot Q0'$ более

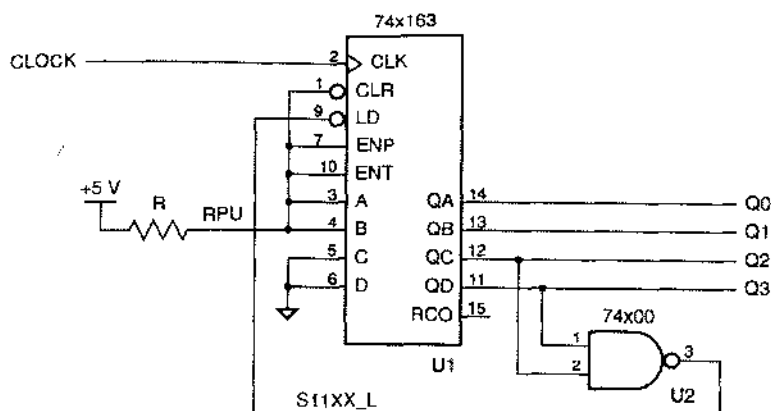


Рис. 8.37. Счетчик 74x163 в ехеме, осуществляющей десятичный счет в коде е избытком 3

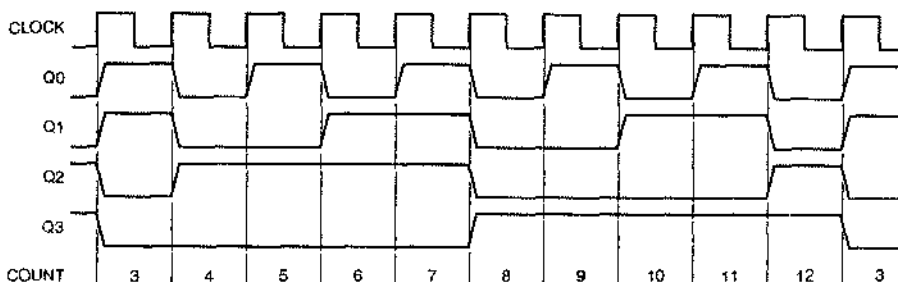


Рис. 8.38. Временные диаграммы для ехемы на основе ИС '163, осуществляющей десятичный счет в коде е избытком 3

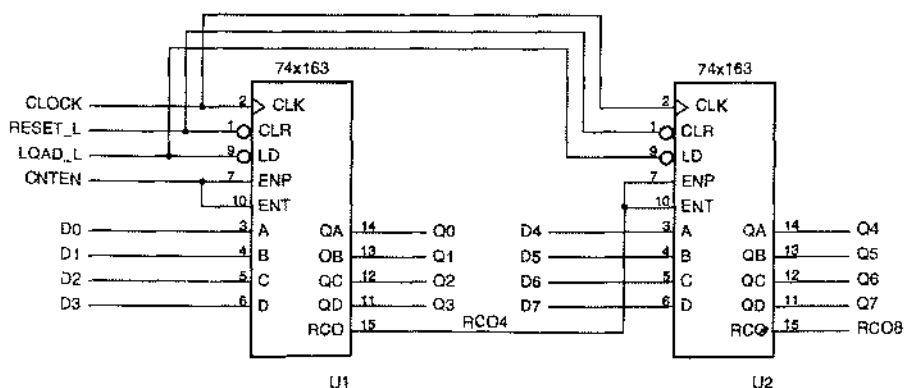


Рис. 8.39. Общая структура последовательного включения счетчиков 74x163

Даже опытные конструкторы не всегда учитывают разницу между входами разрешения ENP и ENT в счетчиках типа '163, поскольку счет возможен только тогда,

когда оба этих сигнала имеют активный уровень. Однако достаточно взглянуть на внутреннюю структуру ИС '163, представленную на рис. 8.31, чтобы увидеть вполне очевидное различие между этими сигналами: сигнал ENT проходит также на выход сквозного перепада. Во многих случаях это различие является существенным.

Рассмотрим, например, настроенный на основе ИС '163 счетчик по модулю 193, который считает от 63 до 255 (рис. 8.40). Выходной сигнал MAXCNT принимает единичное значение, когда счетчик оказывается в состоянии 255, и останавливает счет до тех пор, пока снова не появится сигнал загрузки GO_L. С приходом сигнала GO_L в счетчик загружается состояние 63 и он возобновляет счет до 255. (Заметьте, что схема чувствительна к сигналу GO_L только в том случае, когда счетчик находится в состоянии 255.) Для того чтобы счет был остановлен при достижении состояния 255, необходимо обеспечить наличие единичного сигнала на выходе MAXCNT, несмотря на то, что счет прерван. Поэтому на вход ENT младшего счетчика подан постоянный сигнал разрешения, выход RCO этого счетчика соединен с входом ENT старшего счетчика и состояние 255 обнаруживается по сигналу MAXCNT, хотя значение сигнала CNTEN равно нулю (сравните с сигналом RCO8 в схеме на рис. 8.39). Для разрешения счета сигнал CNTEN подают параллельно на входы ENP. Активным уровнем сигнала RELOAD_L на выходе вентиля И-НЕ счетчик возвращается в состояние 63 только в том случае, когда он находится в состоянии 255 и поступает сигнал GO_L.

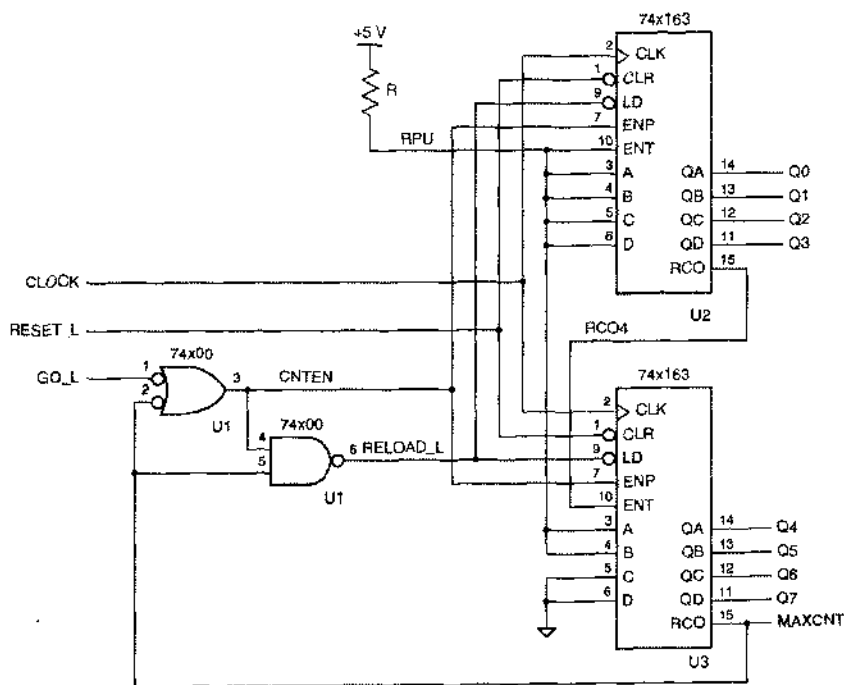


Рис. 8.40. Счетчик по модулю 193 на ИС 74x163 с последовательностью счета 63, 64, ..., 255, 63, 64, ... (На входы LD_L должен подаваться сигнал, являющийся результатом объединения по ИЛИ сигналов RESET_L и RELOAD_L. – Исправление автора.)

Другой счетчик, похожий по выполняемым функциям на ИС 74х163, – это микросхема 74х169; ее условное обозначение приведено на рис. 8.41. Одно из отличий счетчика '169 состоит в том, что выходной сигнал переноса и сигналы на входах разрешения имеют активный низкий уровень. Более существенно то, что ИС '169 является *реверсивным счетчиком* (*up/down counter*): он может считать в сторону увеличения или уменьшения содержащегося в нем двоичного числа в зависимости от значения входного сигнала UP/DN. Когда сигнал UP/DN равен 1, происходит счет в сторону увеличения; когда значение сигнала UP/DN равно 0, счет ведется в сторону уменьшения.

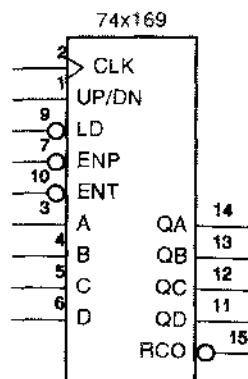


Рис. 8.41. Условное обозначение реверсивного счетчика 74х169

8.4.4. Декодирование состояний двоичного счетчика

Объединяя двоичный счетчик с дешифратором, можно вырабатывать кодовые слова кода «1 из m », содержащие по одному единичному сигналу на каждое состояние счетчика. Это бывает полезно в том случае, когда с помощью счетчика управляют набором устройств: сигналы разрешения ноступают на отдельные устройства, когда счетчик находится в соответствующем состоянии. При таком подходе каждый из выходных сигналов дешифратора служит сигналом разрешения для одного из устройств.

На рис. 8.42 показано, как можно связать между собой счетчик 74х163, считающий по модулю 8, и дешифратор 3х8 типа 74х138; в результате получается схема, вырабатывающая восемь сигналов, каждый из которых соответствует одному из состояний счетчика. На рис. 8.43 приведены типичные временные диаграммы для этой схемы. Сигнал на каждом выходе дешифратора имеет активный уровень в течение определенного периода тактового сигнала.

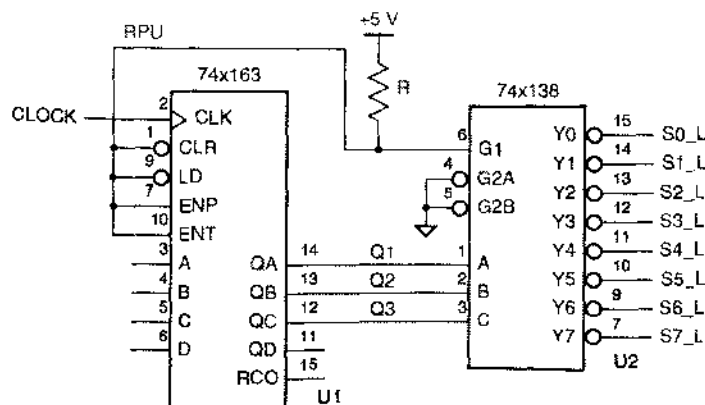


Рис. 8.42. Двоичный счетчик по модулю 8 с дешифратором

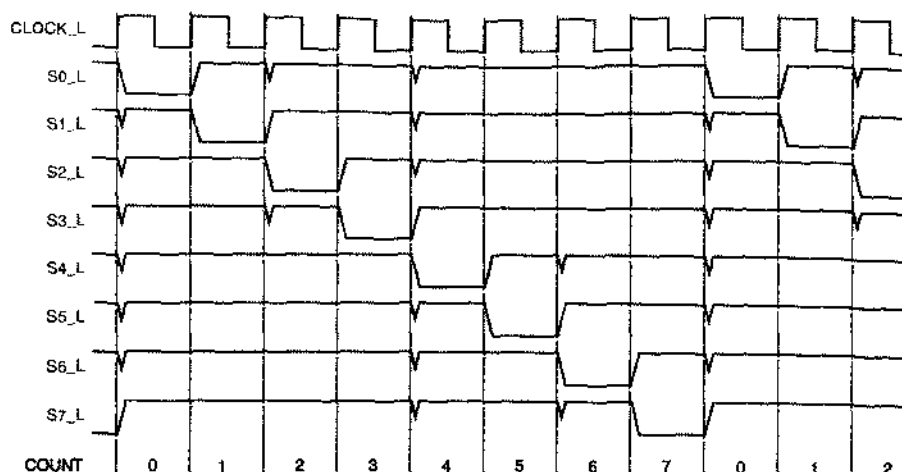


Рис. 8.43. Временные диаграммы для двоичного счетчика по модулю 8 с дешифратором, на которых видны паразитные импульсы на выходах дешифратора

Обратите внимание на то, что при изменении в счетчике содержимого двух или большего числа разрядов на выходах дешифратора могут возникать «паразитные импульсы», несмотря на отсутствие паразитных импульсов на выходах ИС '163 и статических источников опасности у дешифратора '138. В синхронном счетчике типа '163 выходные сигналы изменяются не точно в одно и то же время. Но более важным является то, что в дешифраторе типа '138 задержки прохождения сигналов по разным путям различны; например, задержка прохождения сигнала от входа В к выходу Y1_L меньше, чем задержка на пути от входа А к выходу Y1_L. Поэтому даже если значения входных сигналов, равные 011, изменяются одновременно и становятся равными 100, в сигнале на выходе Y1_L может появиться паразитный импульс. В данном примере мы видим, что паразитные импульсы могут возникать при *любой* реализации функции двоичного декодирования; в таком случае говорят о наличии *функционального источника опасности (function hazard)*.

В большинстве приложений выходные сигналы дешифратора, изображенные на рис. 8.43, используются в качестве сигналов, управляющих регистрами, счетчиками и другими устройствами, переключающимися по фронту (например, в качестве входных сигналов EN_L, LD_L и ENP_L, подаваемых на ИС 74x377, 74x163 и 74x169 соответственно). В этом случае указанные на рисунке паразитные импульсы на выходах дешифратора не страшны, поскольку они возникают строго *после* перепада в тактовом сигнале и кончаются задолго до очередного фронта тактового сигнала, когда выходные сигналы дешифратора принимаются во внимание другими устройствами, переключающимися по фронту. Однако паразитные импульсы могли бы вызвать затруднения при попытке использовать выходные сигналы дешифратора в качестве управляющих сигналов SR-защелки типа S_L или R_L. Точно так же подобные сигналы, в которых потенциально могут иметь место паразитные импульсы, ни в коем случае нельзя применять в качестве тактовых сигналов для переключающихся по фронту устройств.

Если нужно «очистить» сигналы от паразитных импульсов, указанных на рис. 8.43, то один из способов сделать это состоит в подаче выходных сигналов дешифратора '138 на другой регистр, в котором установившиеся значения этих сигналов фиксировались бы на следующем фронте тактового сигнала (рис. 8.44). Заметьте, что сигналам на выходах регистра присвоены другие имена, чтобы учесть задержку на один такт при прохождении через регистр. Но коль скоро вы готовы заплатить за решение проблемы путем применения 8-разрядного регистра, имейте в виду, что существует менее дорогое решение, состоящее в использовании 8-разрядного «кольцевого счетчика», на выходах которого непосредственно вырабатываются декодированные сигналы без паразитных импульсов, как это будет показано в разделе 8.5.6.

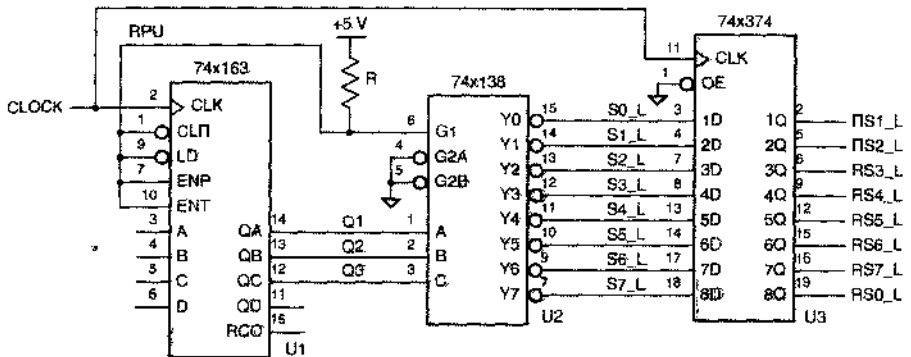


Рис. 8.44. Схема двоичного счетчика по модулю 8 с дешифратором без паразитных импульсов на выходах

8.4.5. Описание счетчиков на языке ABEL и их реализация в ПЛУ

Двоичные счетчики удобно создавать, описывая их на языке ABEL и реализуя в ПЛУ, по нескольким причинам:

- Большой конечный автомат часто можно разбить на два или большее число меньших по размерам конечных автоматов так, что один из них является двоичным счетчиком, задающим время, в течение которого другой блок будет оставаться в том или ином состоянии. Такой подход позволяет упростить как проект в целом, так и его схемное воплощение.
- Во многих приложениях бывают нужны счетчики с модулем счета более или менее кратным степени 2, к которым предъявляются определенные требования в отношении инициализации, а также обнаружения или пропуски тех или иных состояний. Например, счетчик в контроллере лифта может пропускать состояние 13. Вместо использования стандартного двоичного счетчика и применения дополнительной логики для удовлетворения предъявляемых требований разработчик может в программе на языке ABEL точно задать требуемые функции.

- У большинства обычных счетчиков в ИС средней степени интеграции только 4 разряда, в то время как в одном ПЛИУ с 24 выводами можно образовать двоичный счетчик с числом разрядов, достигающим до 10.

Самым популярным счетчиком в ИС средней степени интеграции является 4-разрядный двоичный счетчик 74х163, представленный на рис. 8.31. Даже поверхностного взгляда на схему этого счетчика достаточно, чтобы увидеть, что его логика возбуждения не так проста, особенно с точки зрения использования в ней вентилей ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ. А язык ABEL позволяет самым непосредственным образом описать поведение счетчика, к чему мы и приступаем.

Напомним, что в языке ABEL символ "+" употребляется для сложения целых чисел. Когда с помощью данного оператора «складываются» два набора, каждый из них интерпретируется как двоичное число; самый правый элемент набора соответствует младшему разряду числа. С учетом этого функцию ИС 74х163 можно задать так, как это сделано в программе на языке ABEL, приведенной в табл. 8.12. Когда счет разрешен, к текущему состоянию добавляется 1.

Табл. 8.12. Программа на языке ABEL для 4-разрядного двоичного счетчика типа 74х163

```

module Z74X163
title '4-bit Binary Counter'

" Input pins
CLK, LD_L, CLR_L, ENP, ENT      pin;
A, B, C, D                      pin;

" Output pins
QA, QB, QC, QD                 pin istype 'reg';
RCO                            pin istype 'com';

" Set definitions
INPUT = [D, C, B, A];
COUNT = [QD, QC, QB, QA];

LD = !LD_L; CLR = !CLR_L;      " Active-level conversions

equations

COUNT.CLK = CLK;

COUNT := !CLR & ( LD & INPUT
                # !LD & (ENT & ENP) & (COUNT + 1)
                # !LD & !(ENT & ENP) & COUNT);

RCO = (COUNT == [1,1,1,1]) & ENT;

end Z74X163

```

Табл. 8.13. Минимизированные равенства для 4-разрядного двоичного счетчика из табл. 8.12

```

QA := (CLR_L & LD_L & ENT & ENP & !QA
      # CLR_L & LD_L & !ENP & QA
      # CLR_L & LD_L & !ENT & QA
      # CLR_L & !LD_L & A);

QB := (CLR_L & LD_L & ENT & ENP & !QB & QA
      # CLR_L & LD_L & QB & !QA
      # CLR_L & LD_L & !ENP & QB
      # CLR_L & LD_L & !ENT & QB
      # CLR_L & !LD_L & B);

QC := (CLR_L & LD_L & ENT & ENP & !QC & QB & QA
      # CLR_L & LD_L & QC & !QA
      # CLR_L & LD_L & QC & !QB
      # CLR_L & LD_L & !ENP & QC
      # CLR_L & LD_L & !ENT & QC
      # CLR_L & !LD_L & C);

QD := (CLR_L & LD_L & ENT & ENP
      & !QD & QC & QB & QA
      # CLR_L & !LD_L & D
      # CLR_L & LD_L & QD & !QB
      # CLR_L & LD_L & QD & !QC
      # CLR_L & LD_L & !ENP & QD
      # CLR_L & LD_L & !ENT & QD
      # CLR_L & LD_L & QD & !QA);

RCD = (ENT & QD & QC & QB & QA);

```

В табл. 8.13 представлены минимизированные логические равенства, которые генерирует компилятор языка ABEL для 4-разрядного счетчика. Заметьте, что при переходе в сторону старших разрядов каждому следующему выходному сигналу требуется на один терм-произведение больше. В результате этого размер счетчиков, которые можно реализовать в ПЛУ 16V8 или даже 20V8 обычно ограничен пятью или шестью разрядами. В других устройствах, в том числе в ПЛУ X-серии и в некоторых ИС типа CPLD, имеется структура ИСКЛЮЧАЮЩЕЕ ИЛИ, позволяющая строить большие по размерам счетчики без увеличения требуемого числа термов-произведений.

Проектирование счетчика с заданной последовательностью состояний на языке ABEL много проще, нежели приспособление для этих целей обычного двоичного счетчика. Например, ABEL-программу из табл. 8.12 можно заставить считать в коде с избытком 3 (см. рис. 8.38), следующим образом изменив равенства:

```

COUNT := !CLR & ( LD & INPUT
      # !LD & (ENT & ENP) &
      ((COUNT==12) & 3) # ((COUNT!=12) & (COUNT + 1))
      # !LD & !(ENT & ENP) & COUNT);

RCD = (COUNT == 12) & ENT;

```

Чтобы получить счетчики с большим числом разрядов, можно включить несколько ПЛУ последовательно, предусмотрев в каждом каскаде выходной сигнал переноса, служащий индикатором того, что счетчик в этом каскаде собирается начать счет сначала. Существует два основных подхода к вырабатыванию сигнала переноса:

- *Комбинационный выходной сигнал переноса (combinational carry output)* указывает, что счет разрешен и в данный момент счетчик находится в своем последнем состоянии перед тем, как начать счет сначала. Для 5-разрядного счетчика при счете в прямом направлении имеем:

$COUT = CNTEN \ \& \ Q4 \ \& \ Q3 \ \& \ Q2 \ \& \ Q1 \ \& \ Q0;$

Поскольку правая часть равенства содержит сигнал $CNTEN$, такой подход допускает сквозной перенос в многокаскадных счетчиках, если выход $COUT$ в каждом каскаде соединен с входом $CNTEN$ в следующем каскаде.

- Правая часть равенства для сигнала переноса на регистровом выходе (*registered carry output*) указывает, что следующим состоянием счетчика будет последнее его состояние перед тем, как счет начнется сначала. Таким образом, на следующем такте счетчик входит в свое последнее состояние и сигнал переноса переходит на активный уровень. В случае 5-разрядного счетчика с входами загрузки и сброса имеем:

```
COUT := 'CLR & 'LD & CNTEN
      & Q4 & Q3 & Q2 & Q1 & 'Q0
      # 'CLR * 'LD * 'CNTEN
      & Q4 & Q3 & Q2 & Q1 & Q0
      # 'CLR & LD
      & D4 & D3 & D2 & D1 & D0;
```

Достоинство второго подхода заключается в том, что сигнал $COUT$ вырабатывается с меньшей задержкой, чем при комбинационном подходе. Но теперь требуются внешние вентили между каскадами, поскольку сигнал $CNTEN$ в каждом каскаде должен быть результатом объединения по И главного сигнала разрешения счета и выходных сигналов $COUT$ всех каскадов, младше данного. Необходимости помещения внешних вентилях можно избежать при наличии у старших каскадов счетчика нескольких входов разрешения.

8.4.6. Описание счетчиков на языке VHDL

Как и язык ABEL, VHDL позволяет совсем легко описывать счетчики. Наибольшее затруднение при этом может возникнуть только из-за строгих требований в языке VHDL к типам сигналов, которые должны быть определены правильно и последовательно.

В табл. 8.14 представлена VHDL-программа для двоичного счетчика типа 74х163. В программе используется библиотека `IEEE.std_logic_arith.all`, включающая тип `UNSIGNED`, как это было объяснено в разделе 5.9.6. Эта библиотека содержит определения операторов “+” и “-”, посредством которых выполняются сложение и вычитание без знака операндов типа `UNSIGNED`. В программе для счетчика входы и выходы счетчика объявлены как векторы типа `UNSIGNED`, а с помощью оператора “+” осуществляется требуемое инкрементирование содержимого счетчика.

Для хранения содержимого счетчика в программе определен внутренний сигнал `IQ`. Можно было бы использовать для этого сигнал `Q` непосредственно, но тогда мы должны были бы объявить его выходным сигналом типа `buffer`, а не `out`. Кроме того, мы могли бы определить тип портов `D` и `Q` как `STD_LOGIC_VECTOR`, но тогда нам пришлось бы выполнять преобразование типов в теле процесса (см. задачу 8.51).

Табл. 8.14. VHDL-программа для 4-разрядного двоичного счетчика типа 74x163

```

library IEEE;
use IEEE std_logic_1164 all;
use IEEE.std_logic_arith all;

entity V74x163 is
    port ( CLK, CLR_L, LD_L, ENP, ENT in STD_LOGIC,
          D in UNSIGNED (3 downto 0),
          Q out UNSIGNED (3 downto 0),
          RCO out STD_LOGIC ),
end V74x163;

architecture V74x163_arch of V74x163 is
    signal IQ UNSIGNED (3 downto 0);
begin
    process (CLK, ENT, IQ)
    begin
        if (CLK'event and CLK='1') then
            if CLR_L='0' then IQ <= (others => '0');
            elsif LD_L='0' then IQ <= D;
            elsif (ENT and ENP)='1' then IQ <= IQ + 1;
            end if;
        end if;
        if (IQ=15) and (ENT='1') then RCO <= '1',
        else RCO <= '0',
        end if;
        Q <= IQ;
    end process;
end V74x163_arch;

```

Воспользовавшись поведенческим описанием на языке VHDL, столь же легко, как и на языке ABEL, задать определенную последовательность состояний В табл. 8.15, например, счетчик типа 74x163 видоизменен таким образом, чтобы счет происходил согласно коду с избытком 3 (3, ..., 12, 3, ...).

К сожалению, некоторые VHDL-средства синтезируют счетчики не совсем удачно. В частности, они пытаются реализовать одиночный шаг в счете посредством двоичного сумматора, операндами которого служат содержимое счетчика и константа, равная 1. При таком подходе требуется много больше комбинационной логики, чем в счетчиках, изготовляемых в виде отдельных ИС, и этот подход оказывается особенно расточительным применительно к ИС типа CPLD и FPGA, содержащим Т-триггеры, вентили ИСКЛЮЧАЮЩЕЕ ИЛИ и другие структуры, специально оптимизированные для построения счетчиков. В этом случае полезной альтернативой является написание структурой VHDL-программы, ориентированной на имеющиеся в наличии ячейки в тех конкретных ИС типа CPLD и FPGA или в специализированных ИС, в которых предстоит реализовать проектируемое устройство.

Табл. 8.15. VHDL-архитектура для счета в порядке, задаваемом кодом с из-бытком 3

```

architecture V74xs3_arch of V74x163 is
  signal IQ: UNSIGNED (3 downto 0);
begin
  process (CLK, ENT, IQ)
  begin
    if CLK'event and CLK='1' then
      if CLR_L='0' then IQ <= (others => '0');
      elsif LD_L='0' then IQ <= D;
      elsif (ENT and ENP)='1' and (IQ=12) then IQ <= ('0','0','1','1');
      elsif (ENT and ENP)='1' then IQ <= IQ + 1;
      end if;
    end if;
    if (IQ=12) and (ENT='1') then RCO <= '1';
    else RCO <= '0';
    end if;
    Q <= IQ;
  end process;
end V74xs3_arch;

```

Одноразрядную ячейку для счетчика типа 74x163 можно построить, например, так, как показано на рис. 8.45. Эта схема рассчитана на последовательное распространение битов переноса, так что ею можно воспользоваться в любом каскаде произвольно большого счетчика; единственной ограничением будет коэффициент разветвления по выходу источников сигналов, являющихся общими для всех каскадов. Определения сигналов в одноразрядной ячейке таковы:

CLK	Тактовый сигнал, общий для всех каскадов.
LDNOCLE	Общий для всех каскадов сигнал, принимающий единичное значение, когда на вход счетчика LD сигнал подан, а сигнал CLR отсутствует.
NOCLEORLD	Общий для всех каскадов сигнал, принимающий единичное значение, когда отсутствуют сигналы на обоих входах счетчика CLR и LD.
CNTENP	Общий для всех каскадов сигнал, равный 1, если на вход счетчика ENP подан сигнал разрешения.
Di	Индивидуальный входной сигнал загрузки данных i -й ячейки.
CNTENi	Индивидуальный последовательный входной сигнал разрешения счета i -й ячейки.
CNTENi+1	Индивидуальный последовательный выходной сигнал разрешения счета i -й ячейки.
Qi	Индивидуальный выходной сигнал счетчика в i -м разряде.

В табл. 8.16 приведена VHDL-программа, соответствующая схеме одноразрядной ячейки, показанной на рис. 8.45. В этой программе предполагается, что D-триггер в виде компонента `Vdffqgn` уже определен; он подобен D-триггеру из

табл. 8.6 с добавлением инверсного выхода QN. При проектировании на основе специализированной ИС или ИС типа FPGA тип компонента, являющегося триггером, следует выбрать из библиотеки стандартных элементов производителя.

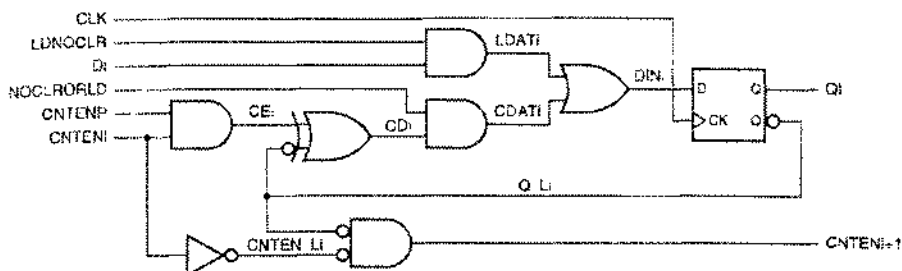


Рис. 8.45. Одноразрядная ячейка для синхронного счетчика с последовательным переносом типа 74х163

Табл. 8.16. VHDL-программа для ячейки, показанной на рис. 8.45

```
library IEEE;
use IEEE.std_logic_1164.all;

entity syncsercell is
    port( CLK, LDNOCLR, NOCLRORLD, CNTENP, D, CNTEN: in STD_LOGIC;
          CNTENO, Q: out STD_LOGIC );
end syncsercell;

architecture syncsercell_arch of syncsercell is
    component Vdffqn
        port( CLK, D: in STD_LOGIC;
              Q, QN: out STD_LOGIC );
    end component;
    signal LDAT, CDAT, DIN, Q_L: STD_LOGIC;
begin
    LDAT <= LDNOCLR and D;
    CDAT <= NOCLRORLD and ((CNTENP and CNTEN) xor not Q_L);
    DIN <= LDAT or CDAT;
    CNTENO <= (not Q_L) and CNTEN;
    U1: Vdffqn port map (CLK, DIN, Q, Q_L);
end syncsercell_arch;
```

В табл. 8.17 показано, как на основе рассмотренной одноразрядной ячейки строится 8-разрядный синхронный счетчик с последовательным переносом. Первыми двумя операторами присваивания в теле архитектуры синтезируются общие для всех разрядов сигналы LDNOCLR и NOCLRORLD. Следующие два оператора отражают граничные условия для последовательной цепочки разрешения счета. Наконец, оператор generate (см. раздел 5.11.3) реализует восемь 1-разрядных ячеек счетчика и связывает между собой звенья цепочки разрешения счета.

ВОПРОС СТИЛЯ

Программа, приведенная в табл. 8.16, написана в стиле, представляющем собой комбинацию потокового и структурного стилей в языке VHDL. Ее можно было бы написать полностью структурно, воспользовавшись, например, определениями компонентов вентилей производителя данной специализированной ИС, гарантируя тем самым, что результат синтеза будет точно соответствовать схеме на рис. 8.45. Однако большинство средств синтеза сами способны выбрать лучшую реализацию вентилей по простым сигнальным операторам присваивания, указанным в программе.

Табл. 8.17. VHDL-программа для 8-разрядного синхронного счетчика с последовательным переносом типа 74x163

```
library IEEE;
use IEEE.std_logic_1164.all;

entity V74x163s is
    port( CLK, CLR_L, LD_L, ENP, ENT: in STD_LOGIC,
          D in STD_LOGIC_VECTOR (7 downto 0);
          Q: out STD_LOGIC_VECTOR (7 downto 0);
          RCO: out STD_LOGIC );
end V74x163s;

architecture V74x163s_arch of V74x163s is
    component syncsercell
        port( CLK, LDNOCLR, NOCLRORLD, CNTENP, D, CNTEN: in STD_LOGIC;
              CNTENQ, Q: out STD_LOGIC );
    end component;
    signal LDNOCLR, NOCLRORLD: STD_LOGIC; -- common signals
    signal SCNTEN: STD_LOGIC_VECTOR (8 downto 0); -- serial count-enable inputs
begin
    LDNOCLR <= (not LD_L) and CLR_L; -- create common load and clear controls
    NOCLRORLD <= LD_L and CLR_L;
    SCNTEN(0) <= ENT; -- serial count-enable into the first stage
    RCO <= SCNTEN(8); -- RCO is equivalent to final count-enable output
    g1: for i in 0 to 7 generate -- generate the eight syncsercell stages
        U1: syncsercell port map ( CLK, LDNOCLR, NOCLRORLD, ENP, D(i), SCNTEN(i),
                                   SCNTEN(i+1), Q(i));
    end generate;
end V74x163s_arch;
```

Очевидно, что легко построить счетчик больших или меньших размеров простым изменением нескольких определений в этой программе. Удобно воспользоваться также оператором `generic` языка VHDL, чтобы задавать число разрядов в счетчике путем изменения всего лишь одной строки в тексте программы (см. задачу 8.53).

8.5. Регистры сдвига

8.5.1. Структура регистра сдвига

Регистр сдвига (shift register) – это n -разрядный регистр, содержимое которого можно сдвигать на один разряд на каждом такте. На рис. 8.46 показана структура регистра сдвига с *последовательным вводом (serial input)* и *последовательным выводом (serial output)*. Последовательный входной сигнал SERIN – это новый бит, который «вдвигается» с одного конца на данном такте. Этот бит появляется на последовательном выходе SEROUT спустя n тактов и теряется на следующем такте. Таким образом, n -разрядный регистр с последовательным вводом и последовательным выводом можно использовать для задержки сигнала на n тактов.

У *регистра сдвига с последовательным вводом и параллельным выводом (serial-in, parallel-out shift register)*, приведенного на рис. 8.47, имеются выходы для всех хранимых в нем битов, благодаря чему они доступны для других схем. Таким регистром можно воспользоваться для выполнения *преобразования последовательного кода в параллельный (serial-to-parallel conversion)*, как это будет объяснено в данном параграфе позднее.

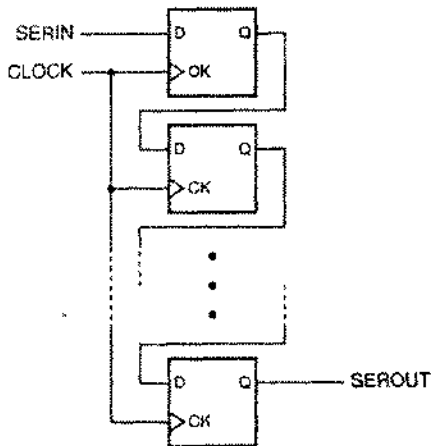


Рис. 8.46. Структура регистра сдвига с последовательным вводом и последовательным выводом

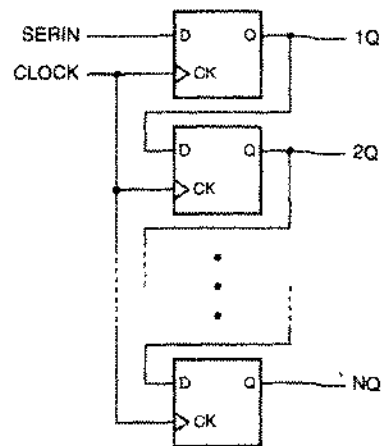


Рис. 8.47. Структура регистра сдвига с последовательным вводом и параллельным выводом

Можно поступить и наоборот, построив *регистр сдвига с параллельным вводом и последовательным выводом (parallel-in, serial-out shift register)*. На рис. 8.48 представлена общая структура такого устройства. В зависимости от значения сигнала на управляющем входе LOAD/SHIFT (этот сигнал можно было бы назвать также LOAD или SHIFT_L) на каждом такте происходит либо загрузка новых данных с входов 1D–ND, либо сдвиг уже имеющегося содержимого регистра. В схеме этого устройства на D-входе каждого триггера стоит 2-входовой мультиплексор, позволяющий выбирать тот или иной сигнал. С помощью регистра сдвига с параллельным вводом и последовательным выводом можно осуществить *преобразование параллельного кода в последовательный (parallel-to-serial conversion)*, о чем также пойдет речь позднее.

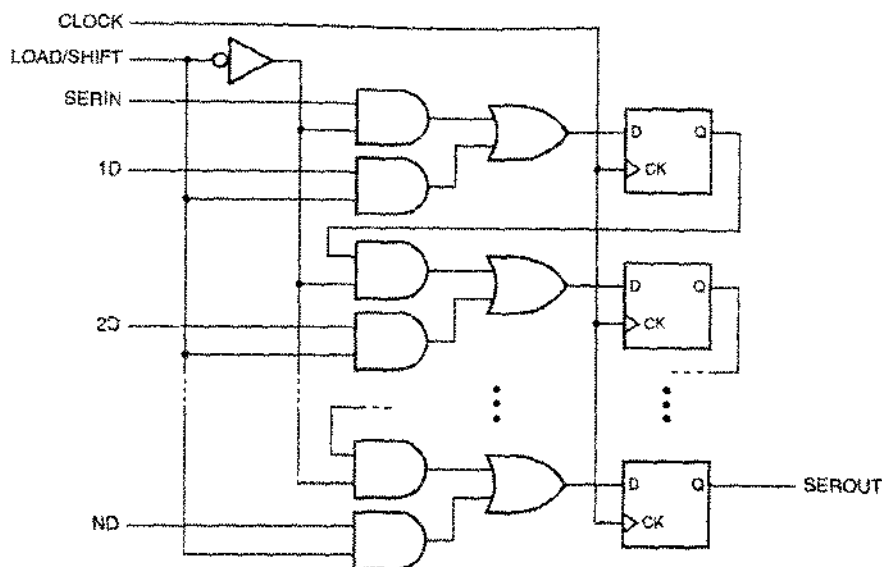


Рис. 8.48. Структура регистра сдвига с параллельным вводом и последовательным выводом

Если регистр сдвига с параллельным вводом снабдить выводами для всех сохраняемых в нем битов, то получится показанный на рис. 8.49 *регистр сдвига с параллельным вводом и параллельным выводом (parallel-in, parallel-out shift register)*. В общем случае такого устройства достаточно для любых применений из числа упомянутых выше.

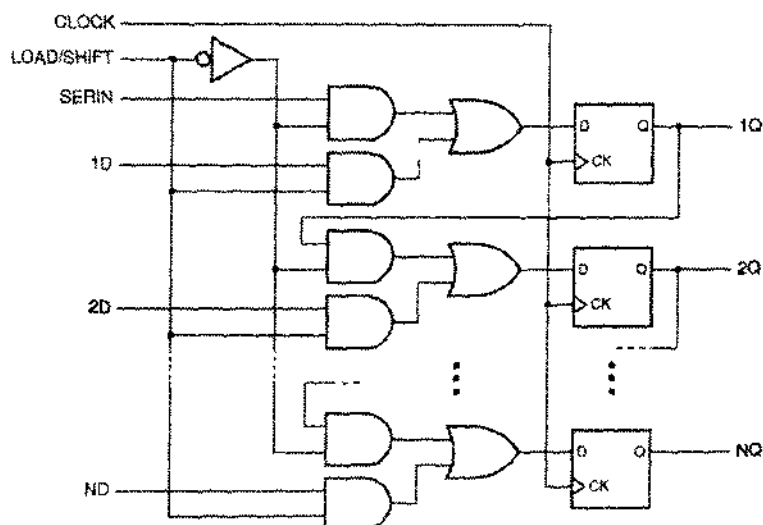


Рис. 8.49. Структура регистра сдвига с параллельным вводом и параллельным выводом

8.5.2. Регистры сдвига в ИС средней степени интеграции

На рис. 8.50 приведены условные обозначения трех популярных 8-разрядных регистров сдвига, выполненных в виде ИС средней степени интеграции. ИС 74x164 – это устройство с последовательным вводом и параллельным выводом, у которого имеется также асинхронный вход сброса CLR_L. У этого регистра два последовательных входа, объединяемые внутри ИС по правилу логического И. Другими словами, для записи единицы в первый разряд регистра необходимо, чтобы единичные значения были у обоих входных сигналов SERA и SERB.

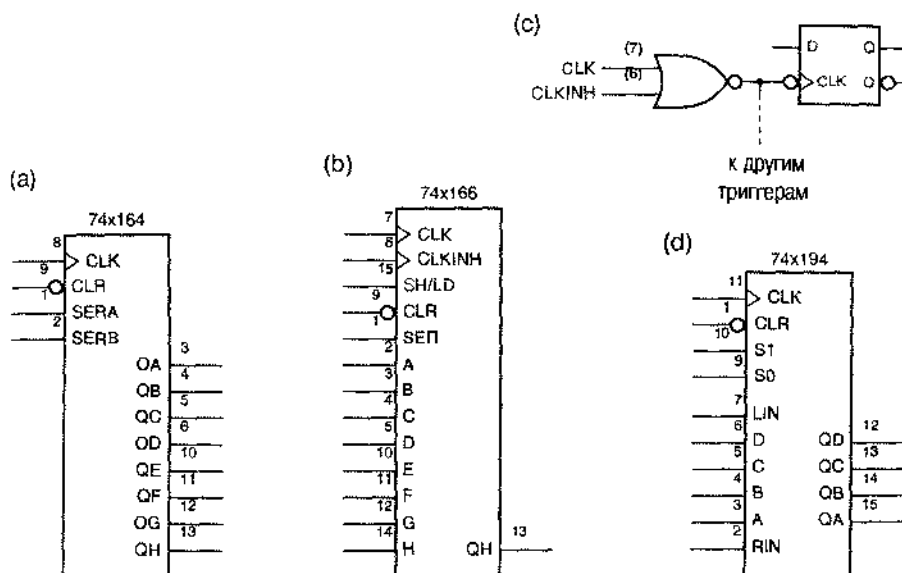


Рис. 8.50. Традиционные условные обозначения регистров сдвига, выполненных в виде ИС средней степени интеграции: (а) 8-разрядный регистр сдвига с последовательным вводом и параллельным выводом 74x164; (б) 8-разрядный регистр сдвига с параллельным вводом и последовательным выводом 74x166; (с) эквивалентная схема входной цепи для тактового сигнала в ИС 74x166; (д) универсальный регистр сдвига 74x194

У регистра сдвига с параллельным вводом и последовательным выводом 74x166 также есть асинхронный вход сброса. Сдвиг в этом устройстве происходит в том случае, когда входной сигнал SH/LD равен 1; в противном случае загружаются новые данные. В ИС '166 необычной является схема обработки тактового сигнала, который называют в этом случае «стробируемым тактовым сигналом» (см. также раздел 8.8.2): имеются два тактовых входа, подключенных к триггерам внутри ИС так, как показано на рис. 8.50(с). Создатели ИС '166 имели в виду, что на вход CLK будет поступать сигнал от источника тактового сигнала, работающего в непрерывном режиме, а на вход CLKINH будет подаваться сигнал запрета CLK, чтобы в пределах отрезка времени, пока действует сигнал CLKINH, не происходили ни сдвиг, ни загрузка. Это означает, что сигнал CLKINH должен быть активным низким.

стра сохранялось. Но для того, чтобы схема работала именно так, сигнал CLKINH должен изменяться только в те моменты времени, когда CLK равен 1; в противном случае на тактовых входах триггеров внутри ИС будут возникать нежелательные перепады сигнала. Значительно безопаснее реализовать функцию «удержания» с помощью устройств, к рассмотрению которых мы теперь переходим.

ИС 74x194 представляет собой 4-разрядный двунаправленный регистр сдвига с параллельным вводом и параллельным выводом. Его принципиальная схема приведена на рис. 8.51. Регистры сдвига, с которыми мы познакомимся до сих пор, называют *однонаправленными регистрами сдвига* (*unidirectional shift registers*), поскольку сдвиг в них может происходить только в одном направлении. ИС 74x194 является *двунаправленным регистром сдвига* (*bidirectional shift register*), так как его содержимое можно сдвигать в ту или другую сторону, в зависимости от значения управляющего входного сигнала. Про эти два направления говорят «сдвиг влево» и «сдвиг вправо», хотя графическое изображение принципиальной схемы и условное обозначение не обязательно соответствуют этим пространственным представлениям. Под *сдвигом влево* (*left*), применительно к ИС 74x194, понимают «сдвиг в направлении от QD к QA», а под *сдвигом вправо* (*right*) – «сдвиг в направлении от QA к QD». В нашем случае принципиальная схема и условное обозначение согласуются с этими названиями, если повернуть их на 90° по часовой стрелке.

Табл. 8.18 представляет собой функциональное описание ИС 74x194 в сжатом виде: в ней отсутствуют столбцы, соответствующие большинству входов (A–D, RIN, LIN) и текущему состоянию (QA–QD). Но поскольку каждое следующее состояние представлено в виде функции этих неявных переменных, этой таблицей полностью определено поведение ИС 74x194 для всех 2^{12} возможных комбинаций текущего состояния и значений входных сигналов без необходимости записи 4096 строк!

Табл. 8.18. Функциональное описание 4-разрядного универсального регистра сдвига 74x194

Функция	Входы		Следующее состояние			
	S1	S0	QA*	QB*	QC*	QD*
Хранение	0	0	QA	QB	QC	QD
Сдвиг вправо	0	1	RIN	QA	QB	QC
Сдвиг влево	1	0	QB	QC	QD	LIN
Загрузка	1	1	A	B	C	D

Заметьте, что «левый вход» LIN (left-in) ИС 74x194 принципиально размещается на микросхеме «справа», поскольку он служит для последовательного ввода *при сдвиге влево*. Аналогично, вход RIN используется для последовательного ввода *при сдвиге вправо*.

ИС 74x194 иногда называют *универсальным* регистром сдвига, так как его можно заставить вести себя как любой из регистров сдвига менее общего типа, которые мы рассматривали до сих пор (например, как однонаправленный регистр сдвига, как регистр сдвига с последовательным вводом и параллельным выводом или как регистр сдвига с параллельным вводом и последовательным выводом). По-

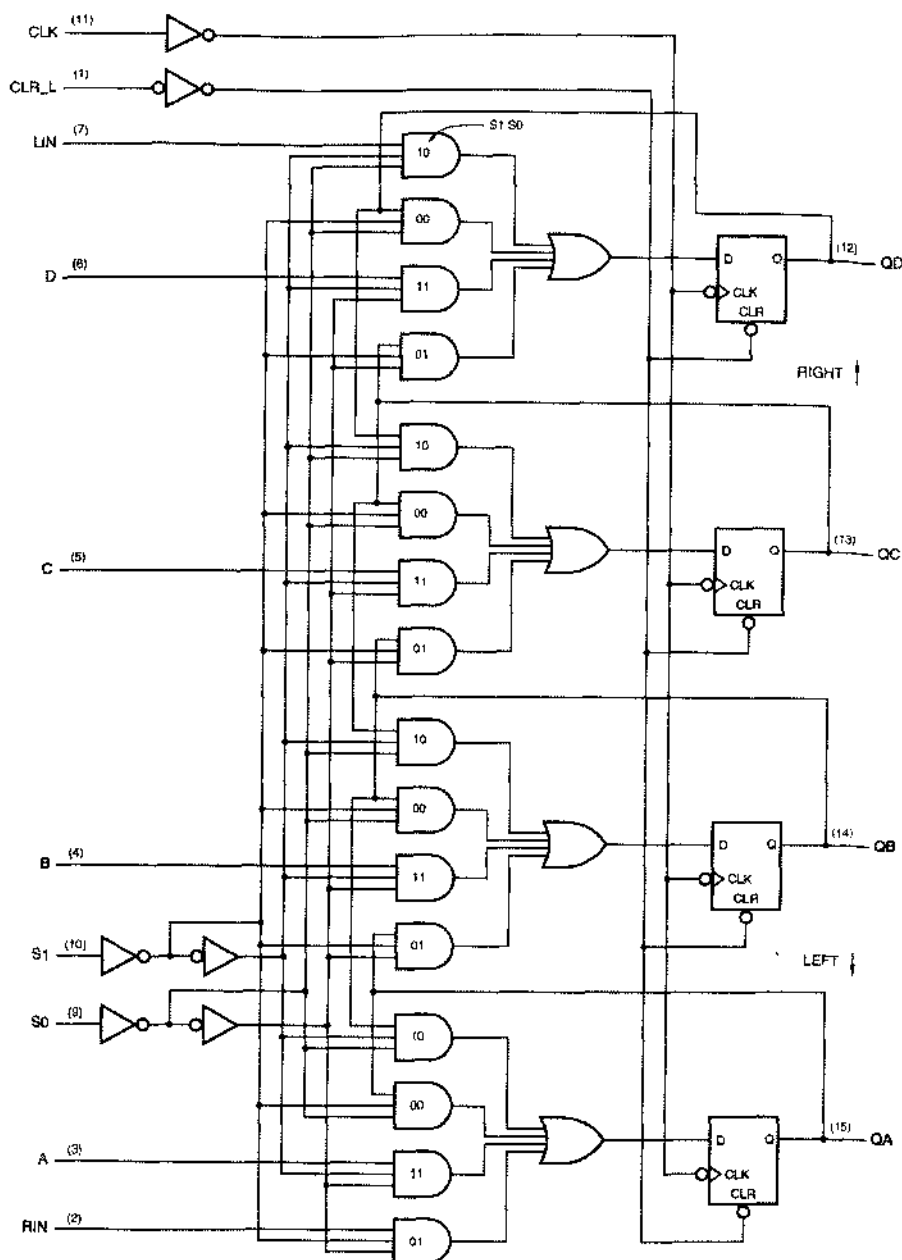


Рис. 8.51. Принципиальная схема 4-разрядного универсального регистра сдвига 74x194 с цоколевкой для стандартного DIP-корпуса с 16 выводами

этому во многих примерах в дальнейшем фигурирует ИС '194, включенная таким образом, что используется подмножество функций из числа тех, которые способна выполнять эта микросхема.

ИС 74х299 является 8-разрядным универсальным регистром сдвига, размещенным в корпусе с 20 выводами; ее условное обозначение и принципиальная схема приведены на рис. 8.52 и 8.53. Как видно из табл. 8.19, по выполняемым функциям и по функциональному описанию эта микросхема похожа на ИС '194. Чтобы сэкономить на числе выводов, в ИС '299 в качестве входов и выходов используются одни и те же сигнальные линии с тремя состояниями. При загрузке ($S1\ S0 = 11$) буферы с тремя состояниями заперты и записываемые данные поступают через выводы AQA–QHN. В других случаях при подаче сигналов на входы G1_L и G2_L запомненные биты выводятся на те же самые контакты. Содержимое крайнего левого и крайнего правого разрядов доступно другим схемам в течение всего времени на отдельных выводах QA и QH, которые служат только выходами.

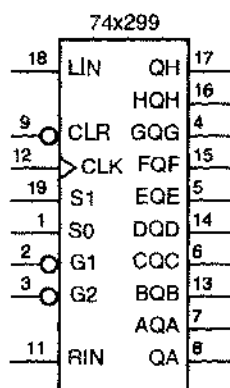


Рис. 8.52. Традиционное условное обозначение ИС 74х299

Табл. 8.19. Функциональное описание 8-разрядного универсального регистра сдвига 74х299

Функция	Входы		Следующее состояние							
	S1	S0	QA*	QB*	QC*	QD*	QE*	QF*	QG*	QH*
Хранение	0	0	QA	QB	QC	QD	QE	QF	QG	QH
Сдвиг вправо	0	1	RIN	QA	QB	QC	QD	QE	QF	QG
Сдвиг влево	1	0	QB	QC	QD	QE	QF	QG	QH	LIN
Загрузка	1	1	AQA	BQB	CQC	DQD	EQE	FQF	GQG	HQH

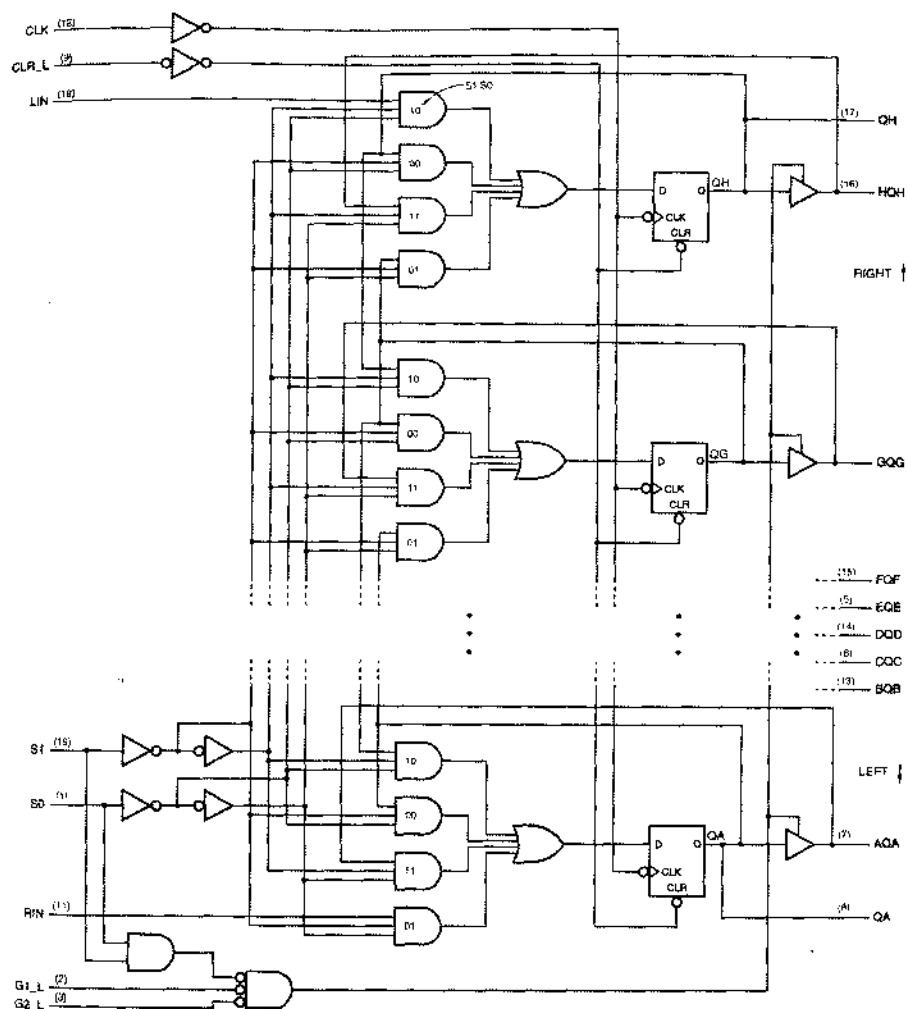


Рис. 8.53. Принципиальная схема 8-разрядного универсального регистра сдвига 74x299 с цоколевкой для стандартного DIP-корпуса с 20 выводами (RIGHT – вправо, LEFT – влево)

8.5.3. Самое распространенное в мире применение регистров сдвига

Чаще всего регистры сдвига применяются для того, чтобы преобразовать параллельные данные в последовательный формат при передаче и при записи, а также для обратного преобразования последовательных данных в параллельный формат для целей обработки и отображения (см. раздел 2.16.1). Самый распространенный пример преобразования последовательных данных, с которыми вы почти наверняка встречаетесь каждый день, – это *цифровая телефония* (*digital telephony*).

За прошедшие годы телефонные компании установили на своих *центральных станциях* (*central offices, COs*) оборудование с цифровой коммутацией. Большинство домашних телефонов связано с центральной станцией двухпроводными аналоговыми соединениями. Когда аналоговый речевой сигнал поступает на центральную станцию, из него – с помощью аналого-цифрового преобразователя – 8000 раз в секунду берутся выборки (каждые 125 микросекунд), в результате чего образуется последовательность байтов, – 8-разрядных двоичных слов, – выражающих собой знак и величину аналогового сигнала в момент взятия выборки. Затем повсюду в телефонной сети ваш голос передается в цифровом виде со скоростью 64 кбит/с по *последовательным каналам* (*serial channels*) до тех пор, пока он не преобразуется обратно в аналоговую форму цифро-аналоговым преобразователем на центральной станции адресата.

НУ, УЖ НЕ ЗНАЮ...

К концу 80-х годов получила развитие технология ISDN (Integrated Services Digital Network, цифровая сеть связи с комплексными услугами), целью которой было дотянуть до домашних телефонов последовательные цифровые каналы со скоростью передачи 144 кбит/с. Идея состояла в том, чтобы по одной паре проводов передавать два телефонных разговора со скоростью 64 кбит/с плюс сигналы управления, следующие со скоростью 16 кбит/с; в результате увеличивается пропускная способность уже проложенных телефонных линий.

В первом издании этой книги мы обратили внимание читателей на то, что из-за задержки с развертыванием сети ISDN среди специалистов появилась другая расшифровка аббревиатуры ISDN: "Imaginary Services Delivered Nowhere" («воображаемые услуги, не доставляемые нигде»). К середине 90-х годов сеть ISDN утвердилась, наконец, в США, но не столько для передачи речевых сигналов, сколько для обеспечения «скоростных» соединений Интернета.

К большому сожалению телефонных компаний внедрение сети ISDN затормозилось сначала распространением медленных 56-килобитовых аналоговых модемов, а затем – растущей доступностью высокоскоростных соединений (со скоростью передачи от 160 кбит/с до 2 Мбит/с и больше), в которых используются более новые кабель-модемная технология и технология DSL (Digital Subscriber Line, цифровая абонентская линия).

Полоса пропускания, необходимая для передачи одного оцифрованного речевого сигнала со скоростью 64 кбит/с, много меньше той, какую предоставляет одна цифровая сигнальная линия и коммутационное оборудование, построенное на цифровых ИС. Поэтому в большинстве случаев на цифровых телефонных станциях осуществляется *мультиплексирование (multiplexing)* нескольких 64-килобитовых каналов в одну линию передачи, на чем экономятся как провода, так и объем коммутационного оборудования, выражаемый числом интегральных схем. В следующем разделе мы покажем, как с помощью небольшого числа ИС средней степени интеграции можно обрабатывать сигналы, передаваемые по 32 каналам, причем функции, выполняемые этими микросхемами, легко реализовать в одной ИС типа CPLD. Это классический пример *пространственно-временного обмена (space/time tradeoff)* при цифровом проектировании: заставляя интегральные схемы работать быстрее, вы решаете задачу большего объема, используя меньшее число микросхем. В этом заключается главная причина, по которой телефонная сеть «стала цифровой».

8.5.4 Последовательно-параллельное преобразование

На рис. 8.54 приведен типичный пример последовательной передачи данных между двумя модулями (такое соединение может быть частью коммутационного оборудования на телефонной станции). Обычно передача в таком соединении от источника сигналов к месту назначения происходит по трем сигнальным линиям:

- *Тактовый сигнал* задает темп передачи, указывая интервалы времени, отводимые на передачу одного бита. Если система состоит всего лишь из двух модулей, то тактовый сигнал может генерироваться блоком управления, размещенным в передатчике сигнала, как показано на рисунке. В более крупных системах может быть один общий источник тактового сигнала, разводимого по модулям.
- *Последовательные данные*: сами по себе данные передаются по одной линии.
- *Синхронизация*. *Импульсом синхронизации (synchronization pulse* или *sync pulse)* указывается точка отсчета в формате данных, например, начало байта или слова в последовательном потоке данных. В некоторых системах этот сигнал бывает опущен, а синхронизация достигается передачей по линии данных последовательности специального вида.

ОБЩЕНАЦИОНАЛЬНОЕ ВРЕМЯ

Поверите вы мне или нет, но используемый в телефонной сети тактовый сигнал, частота которого с высокой точностью равна 8 кГц, генерируется в г. Сент-Луисе и распространяется повсеместно в США! Тактовый сигнал в конкретной части коммутационного оборудования местной телефонной станции обычно является производным от национального тактового сигнала. В частности, фигурирующий в примере этого параграфа тактовый сигнал с частотой 2.048 МГц мог бы быть получен с помощью схемы фазовой автоподстройки частоты путем умножения частоты национального тактового сигнала на 256.

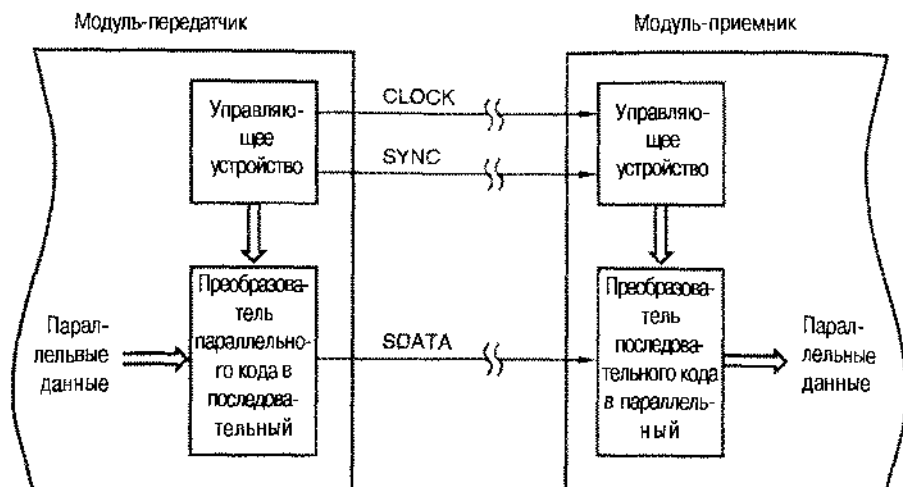


Рис. 8.54. Система с последовательной передачей данных между модулями

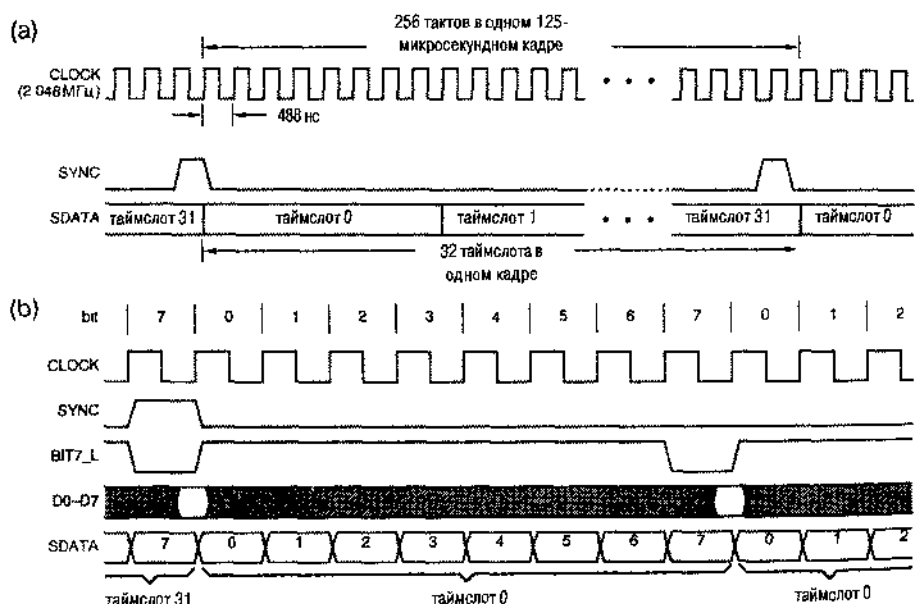


Рис. 8.55. Временные диаграммы преобразования параллельного кода в последовательный: (а) полный кадр; (б) один байт в начале каждого кадра

В ситуации, типичной для цифровой телефонии, временные параметры этих сигналов имеют значения, приведенные на рис. 8.55(а). Частота сигнала *CLOCK* равна 2.048 МГц, и это позволяет передавать 32×8000 8-битовых байтов в секунду. Импульсом сигнала *SYNC* длительностью в 1 бит определяется начало 125-микросекундного интервала, называемого *кадром* (*frame*). За это время по линии *SDATA* передается 256 битов, причем весь этот интервал разбит на 32

таймслота (timeslot), содержащих по 8 битов каждый. В каждом таймслоте передается в цифровом виде один речевой сигнал. Номера таймслотов и расположение битов внутри каждого из них отсчитываются от импульса сигнала SYNC.

На рис. 8.56 представлена схема, осуществляющая преобразование параллельных данных в последовательный формат, указанный на рис. 8.55(а), с учетом деталей, приведенных на рис. (б). Две ИС 74х163 образуют счетчик по модулю 256, работающий в непрерывном режиме; этим счетчиком задается кадр. Пять старших разрядов счетчика указывают номер таймслота, а три младших разряда – номер бита в пределах таймслота.

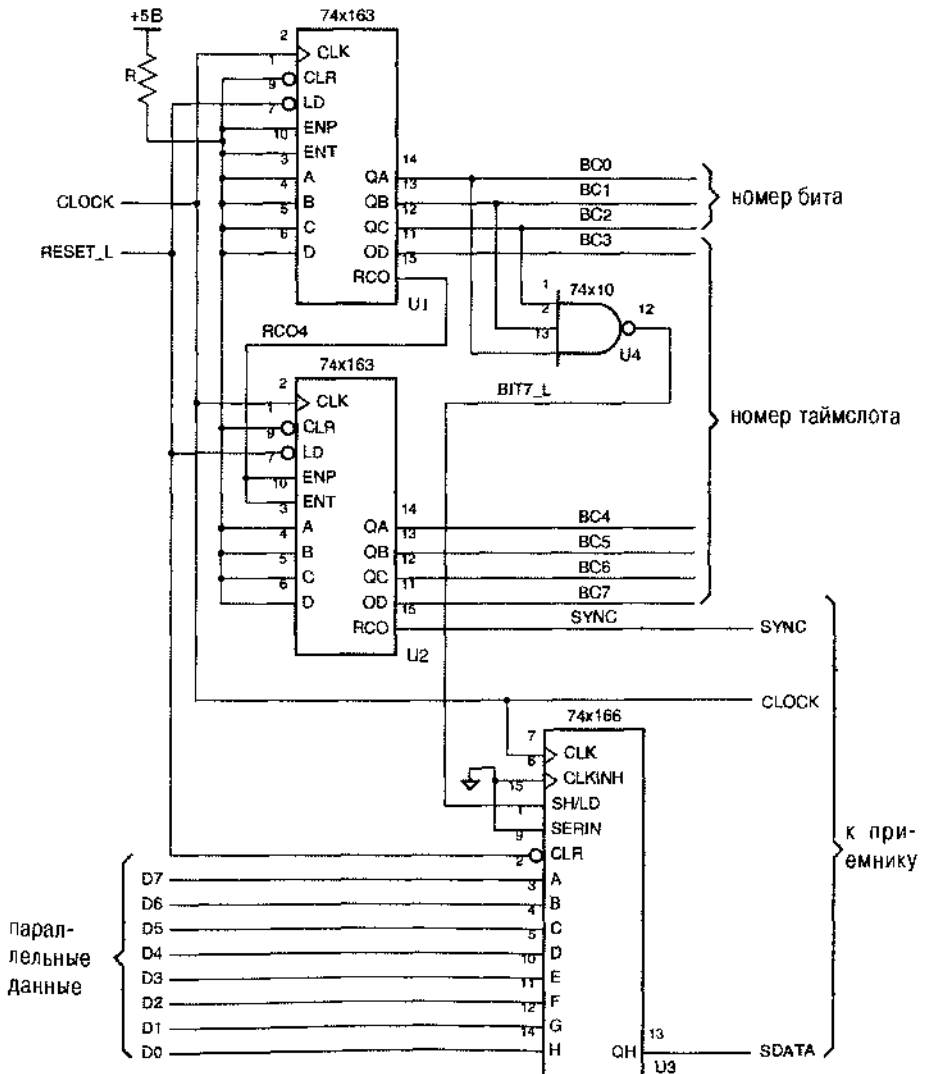


Рис.8.56. Преобразование параллельного кода в последовательный с помощью регистра сдвига с параллельным вводом

КАКОЙ БИТ ПЕРВЫЙ?

В действительности, в большинстве последовательных каналов передачи оцифрованного речевого сигнала первым передается 7-й бит, поскольку он первым появляется на выходе аналого-цифрового преобразователя, переводящего речевой сигнал в цифровую форму. Однако, ради простоты, мы указываем в наших примерах, что первым передается 0-й бит, так что в состоянии счетчика номер бита входит непосредственно.

Регистр сдвига с параллельным вводом 74x166 осуществляет преобразование параллельного кода в последовательный. 0-й бит параллельных данных (D0–D7) подается на вход ИС '166, ближайший к выходу SDATA, так что биты передаются последовательно в порядке 0, ..., 7. При передаче 7-го бита в каждом таймслоте вырабатывается сигнал BIT7_L, который приводит к загрузке ИС '166 данными D0–D7. Значения сигналов на входах D0–D7 несут существенны в течение всего времени, за исключением времени установления и времени удержания в окрестности того перепада в тактовом сигнале, на котором ИС '166 загружается; интервалы времени, в пределах которых значения сигналов на входах данных безразличны, на временных диаграммах заштрихованы. Из этого следует, что шиной, по которой поступают параллельные данные, в другое время можно пользоваться для решения каких-то других задач (см. задачу 8.54).

В модуле-приемнике преобразование последовательных данных обратно в параллельный формат может осуществляться схемой, приведенной на рис. 8.57. Счетчик по модулю 256, состоящий из двух ИС '163, позволяет восстановить номера таймслотов и битов. Поскольку сигнал SINC вырабатывается в то время, когда счетчик в модуле-передатчике находится в состоянии 255, и по этому сигналу выполняется загрузка в счетчик модуля-приемника нулевого содержимого, оба счетчика переходят в нулевое состояние по одному и тому же фронту тактового сигнала. Старшие биты счетчика (номер таймслота) никак не используются на рисунке, но они могут позволить другим схемам в модуле-приемнике идентифицировать байты, удерживаемые на шине параллельных данных (PD0–PD7) в пределах того или иного таймслота.

На рис. 8.58 приведены подробные временные диаграммы для схемы, осуществляющей преобразование последовательного кода в параллельный. Полностью принятый байт присутствует на параллельном выходе регистра сдвига 74x164 в течение периода тактового сигнала, следующего за приемом последнего (7-го) бита в байте. В нашем примере параллельные данные *дважды буферизованы* (*double-buffered data*): будучи полностью приняты, они переносятся в регистр 74x377, на выходах которого PD0–PD7 они доступны другим частям системы в течение восьми полных периодов тактового сигнала до окончания приема следующего байта. Сигнал разрешения BIT0_L обеспечивает загрузку ИС '377 в надлежащий момент времени. При наличии дополнительных регистров и декодирования можно было бы загружать байты из различных таймслотов в соответствующие регистры, в которых каждый байт удерживался бы на протяжении 125 мкс (см. задачу 8.57).

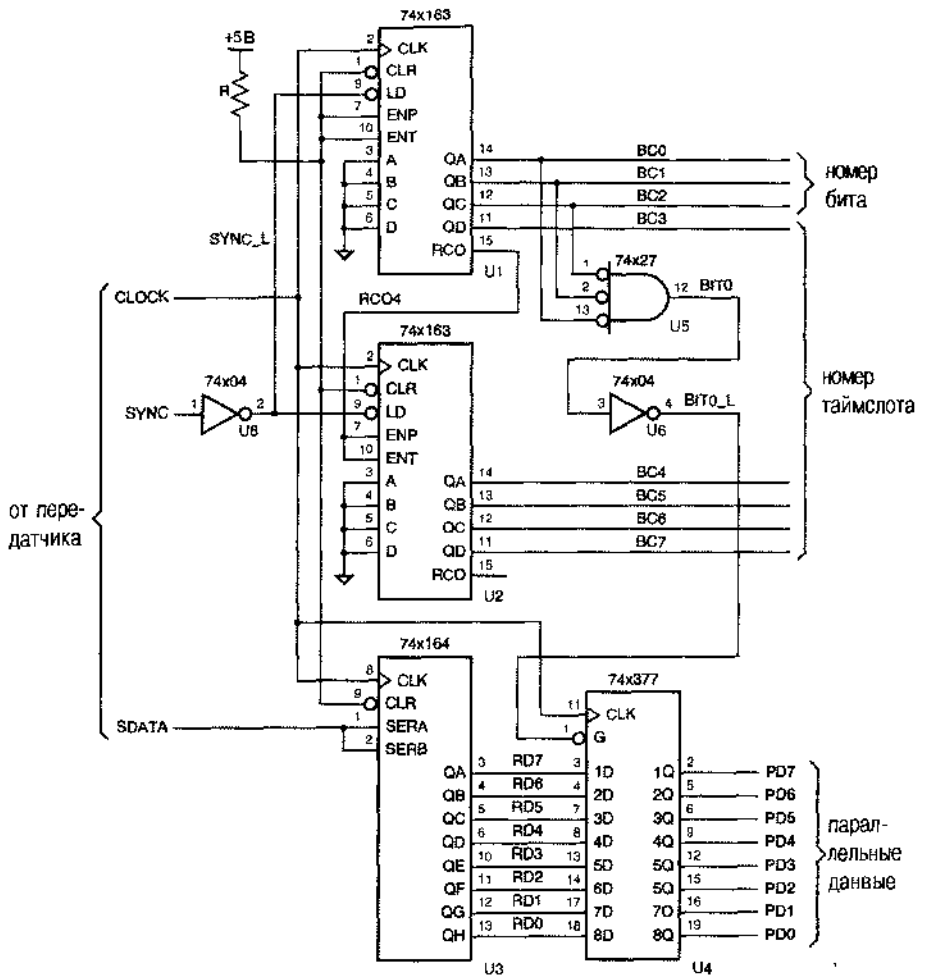


Рис. 8.57. Преобразование последовательного кода в параллельный с помощью регистра сдвига с параллельным выводом

Представленные в параллельном формате принятые данные легко запоминать и модифицировать в других цифровых схемах; в разделе 10.1.6 будут приведены соответствующие примеры. В цифровой телефонии принятые параллельные данные преобразуются обратно в аналоговое напряжение, которое фильтруется и отправляется в телефонную трубку или на громкоговоритель в течение 125 мкс, то есть до тех пор, пока не поступит следующее выборочное значение речевого сигнала.

ПРЯМОЙ И ОБРАТНЫЙ ПОРЯДОК СЛЕДОВАНИЯ

В истории развития цифровых систем был момент, когда обсуждение вопроса о том, в каком порядке нужно передавать биты и байты, стало носить характер религиозного спора. В своей знаменитой статье «Священные войны и призыв к миру» (“On Holy Wars and a Plea for Peace”, *Computer*, October 1981, pp. 48–54) Дэнни Козн (Danny Cohen) описал различия между соглашениями о порядке следования битов и байтов и указал на возможные (и проявившиеся в дальнейшем) отрицательные последствия этого различия.

Твердый стандарт так и не был установлен, и сегодня существуют популярные семейства компьютеров, в которых принят порядок нумерации и передачи байтов 32-разрядного слова, начиная с младшего байта (так называемые IBM-совместимые компьютеры) и начиная со старшего байта (компьютеры Apple Macintosh). Согласно терминологии Козна, в первом случае говорят о прямом порядке следования (“Little Endian”), а во втором – об обратном порядке следования (“Big Endian”), и по-прежнему продолжается дискуссия о том, какой из них предпочтительнее (about “endianness”), как если бы это что-нибудь значило.

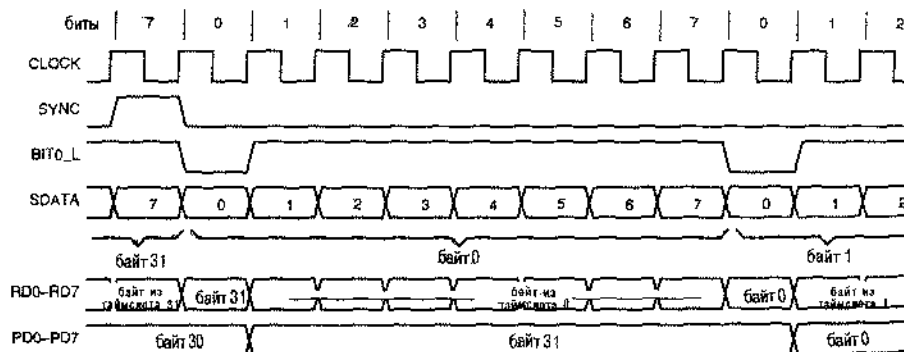


Рис. 8.58. Временные диаграммы для преобразования последовательного кода в параллельный

8.5.5. Счетчики на регистрах сдвига

Последовательно/параллельное преобразование представляет собой «обработку» данных, но регистры сдвига применяются также и в тех случаях, когда речь не идет о «данных». В результате объединения регистра сдвига с комбинационной логикой образуется конечный автомат, у которого диаграмма состояний является циклической. Такую схему называют *счетчиком на регистре сдвига* (*shift-register counter*). В отличие от двоичного счетчика последовательность состояний счетчика на регистре сдвига не образует ряд двоичных чисел, неребираемых в сторону увеличения или уменьшения, но такая схема все же полезна во многих приложениях, связанных с «управлением».

8.5.6. Кольцевые счетчики

В простейшем случае, используя n -разрядный регистр сдвига, можно получить счетчик с n состояниями, пазываемый *кольцевым счетчиком* (*ring counter*). На рис. 8.59 показана схема кольцевого счетчика. Универсальный регистр сдвига 74х194 включен так, что в нем обычно происходит сдвиг влево. Но если подан сигнал RESET, то в него загружается комбинация 0001 [см. функциональную таблицу ИС '194 (табл. 8.18)]. Если сигнал RESET снят, то на каждом такте происходит сдвиг содержимого ИС '194 влево. Последовательность состояний имеет вид: 0010, 0100, 1000, 0001, 0010, Следовательно, счетчик проходит через четыре различных состояния, прежде чем они начинают повторяться. На рис. 8.60 приведены соответствующие временные диаграммы. В общем случае n -разрядный кольцевой счетчик проходит в цикле через n состояний.

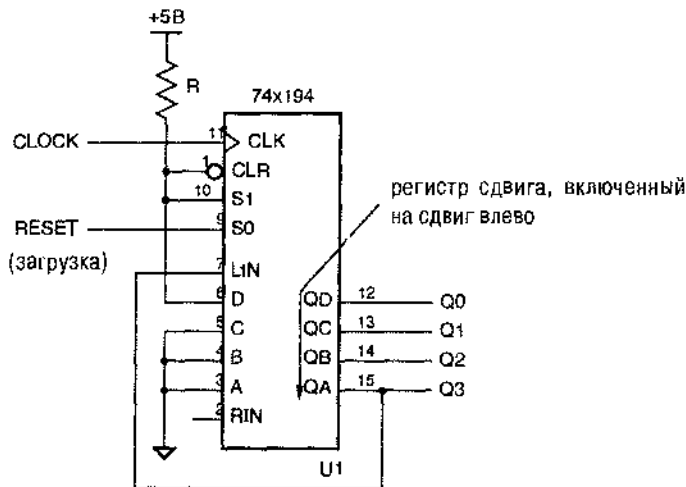


Рис. 8.59. Простейший 4-разрядный кольцевой счетчик с 4 состояниями, в котором циркулирует одна 1

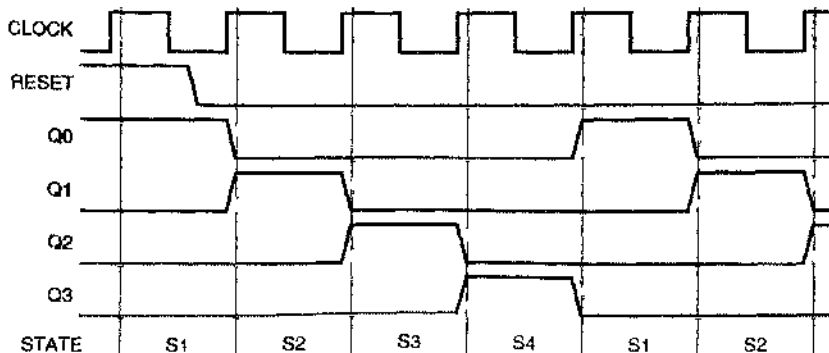


Рис. 8.60. Временные диаграммы для 4-разрядного кольцевого счетчика

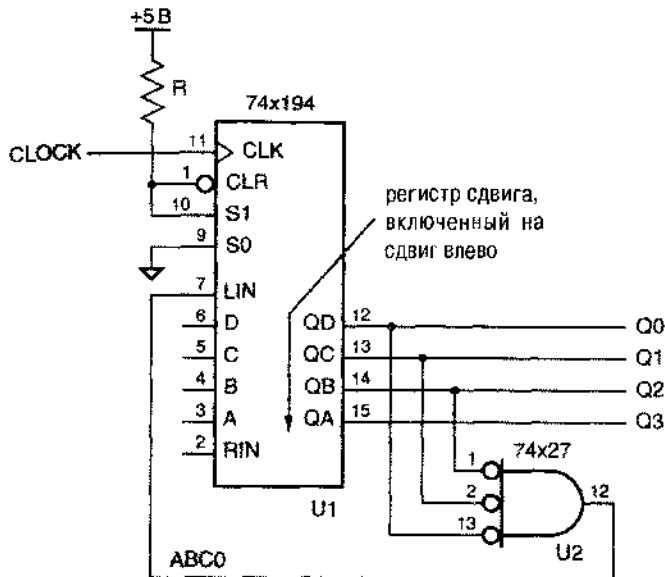


Рис. 8.62. Самокорректирующийся 4-разрядный кольцевой счетчик с 4 состояниями, в котором циркулирует одна 1

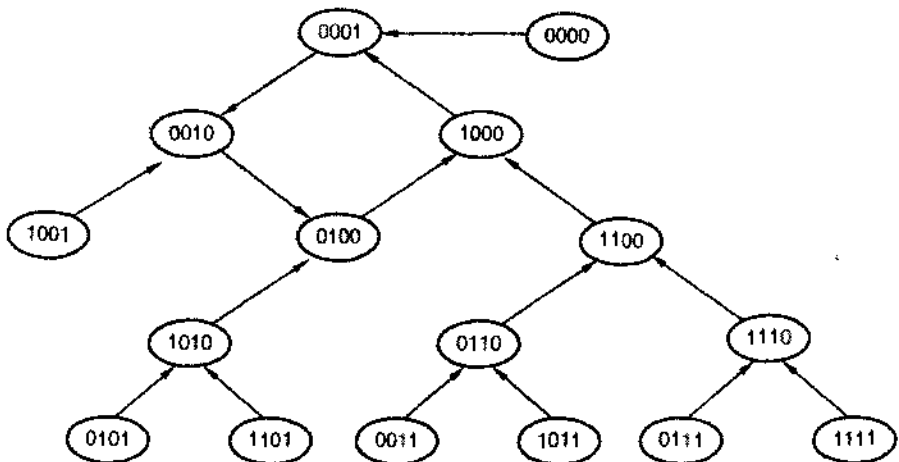


Рис. 8.63. Диаграмма состояний для самокорректирующегося кольцевого счетчика

В логических КМОП- и ТТЛ-семействах большое число входов чаще бывает у вентилях И-НЕ, а не у вентилях ИЛИ-НЕ, поэтому может оказаться более удобным построить самокорректирующийся кольцевой счетчик по схеме представленной на рис. 8.64. В каждом из состояний, образующих нормальный цикл такого счетчика, имеется только один 0.

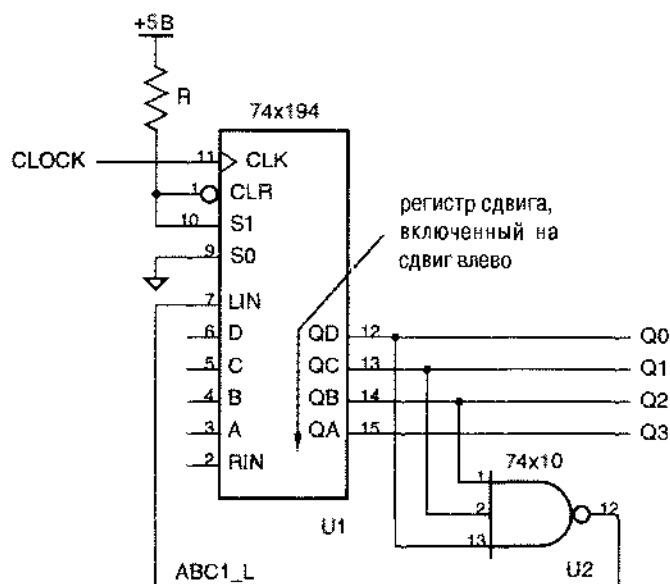


Рис. 8.64. Самокорректирующийся 4-разрядный кольцевой счетчик с 4 состояниями, в котором циркулирует один 0

Основное достоинство кольцевого счетчика с точки зрения его применения в задачах управления состоит в том, что его состояния, выражаемые совокупностью сигналов на выходах триггеров, являются словами кода «1 из n ». Это значит, что всегда только один из выходных сигналов триггеров имеет активный уровень. Кроме того, в выходных сигналах кольцевых счетчиков нет паразитных импульсов; сравните этот подход со случаем, когда двоичный счетчик дополняется дешифратором (см. рис. 8.42).

*8.5.7. Счетчики Джонсона

У n -разрядного регистра сдвига с инвертором в цепи обратной связи между последовательным выходом и последовательным входом имеется $2n$ состояний. Такая конструкция носит название *скрученного кольцевого счетчика* (*twisted-ring counter*), *счетчика Мебиуса* (*Moebius counter*) или *счетчика Джонсона* (*Johnson counter*). Основная схема счетчика Джонсона показана на рис. 8.65, а его временные диаграммы – на рис. 8.66. Нормальные состояния этого счетчика перечислены в табл. 8.20. Как видно из таблицы, при наличии обоих выходных сигналов всех триггеров каждое нормальное состояние счетчика можно обнаружить с помощью 2-входового вентиля И или И-НЕ. На выходах этих вентилей нет паразитных импульсов.

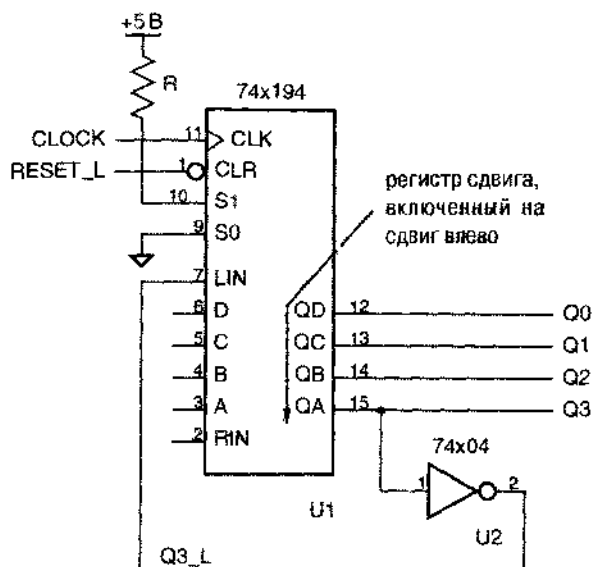


Рис. 8.65. Основная схема 4-разрядного счетчика Джонсона с 8 состояниями

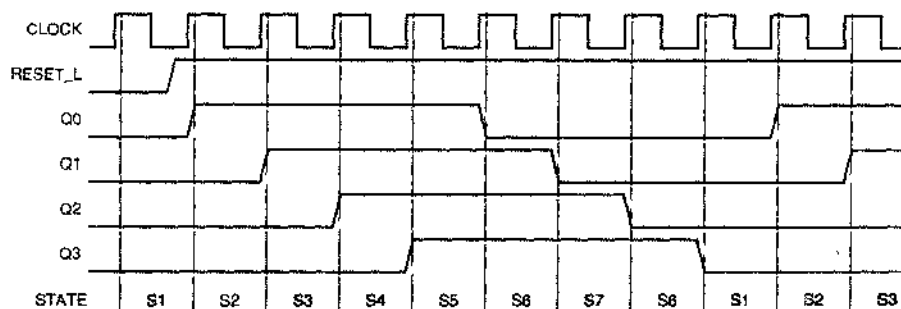


Рис. 8.66. Временные диаграммы для 4-разрядного счетчика Джонсона

Табл. 8.20. Состояния 4-разрядного счетчика Джонсона

Имя состояния	Q3	Q2	Q1	Q0	Декодирование
S1	0	0	0	0	$Q3' \cdot Q0'$
S2	0	0	0	1	$Q1' \cdot Q0$
S3	0	0	1	1	$Q2' \cdot Q1$
S4	0	1	1	1	$Q3' \cdot Q2$
S5	1	1	1	1	$Q3 \cdot Q0$
S6	1	1	1	0	$Q1 \cdot Q0'$
S7	1	1	0	0	$Q2 \cdot Q1'$
S8	1	0	0	0	$Q3 \cdot Q2'$

САМОКОРРЕКТИРУЮЩИЕСЯ СХЕМЫ ОБЕСПЕЧИВАЮТ ИСПРАВЛЕНИЕ САМИ ПО СЕБЕ!

Можно следующим образом доказать, что схема коррекции в самокорректирующемся счетчике Джонсона осуществляет исправление любого неправильного состояния. Неправильное состояние всегда можно представить в виде $x...x10x...x$, так как только нормальные состояния (00...00, 11...11, 01...1, 0...01...1 и 0...01) нельзя записать в таком виде. Поэтому не более чем через $n - 2$ такта регистр сдвига будет содержать комбинацию 10x...x. На следующем такте его содержимое станет равным 0x...x0, и, такт спустя, в него будет загружено нормальное состояние 00...01.

У n -разрядного счетчика Джонсона есть $2^n - 2n$ неправильных состояний, поэтому он также ненадежен, как и кольцевой счетчик. Но, как показано на рис. 8.67, можно построить самокорректирующийся счетчик Джонсона (*self-correcting Johnson counter*). В этой схеме происходит загрузка комбинации 0001 в качестве следующего состояния, если текущее состояние имеет вид 0xx0. По такому же принципу с помощью одного 2-входового вентиля ИЛИ-НЕ можно осуществлять коррекцию в счетчике Джонсона с любым числом разрядов. Схема коррекции должна загружать комбинацию 00...01 в качестве следующего состояния всякий раз, когда текущим оказывается состояние вида 0x...x0.

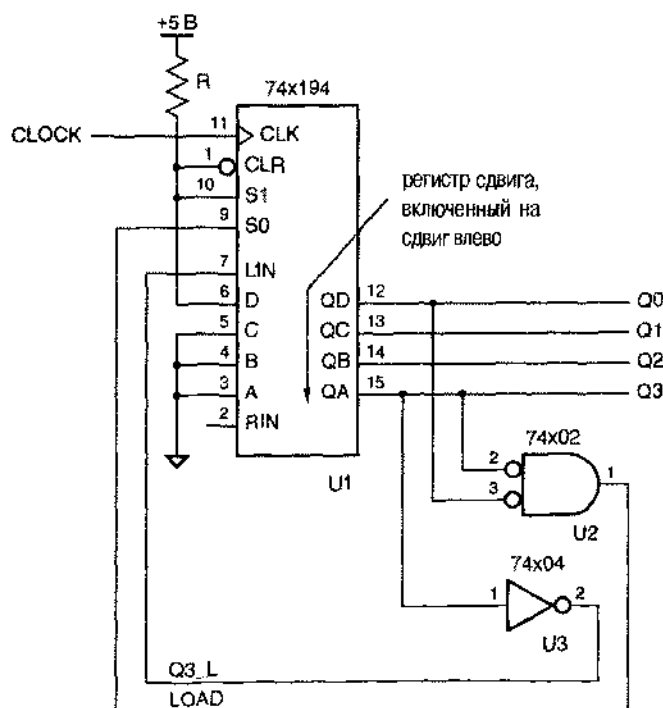


Рис. 8.67. Самокорректирующийся 4-разрядный счетчик Джонсона с 8 состояниями

*8.5.8. Счетчики на регистрах сдвига с линейной обратной связью

Число нормальных состояний у рассматривавшихся до сих пор счетчиков на n -разрядных регистрах сдвига было далеко от максимального возможного числа состояний, равного 2^n . Счетчик на основе n -разрядного регистра сдвига с линейной обратной связью (*linear feedback shift-register, LFSR*) имеет $2^n - 1$ состояний, то есть почти максимум. Такой счетчик часто называют *генератором последовательности максимальной длины* (*maximum-length sequence generator*).

LFSR-счетчики строятся на основе теории конечных полей (*finite fields*), развитой французским математиком Эваристом Галуа (1811–1832) незадолго до того, как он был убит на дуэли его политическим противником. В работе LFSR-счетчика реализуются операции над 2^n элементами в конечном поле.

На рис. 8.68 представлена структура n -разрядного LFSR-счетчика. На последовательный вход регистра сдвига поступает сумма по модулю 2 битов, содержащихся в определенном наборе разрядов регистра сдвига. Этой обратной связью определяется последовательность состояний, через которые проходит счетчик. Принято всегда нумеровать разряды так, как показано на рисунке, и считать, что сдвиг происходит в указанном направлении.

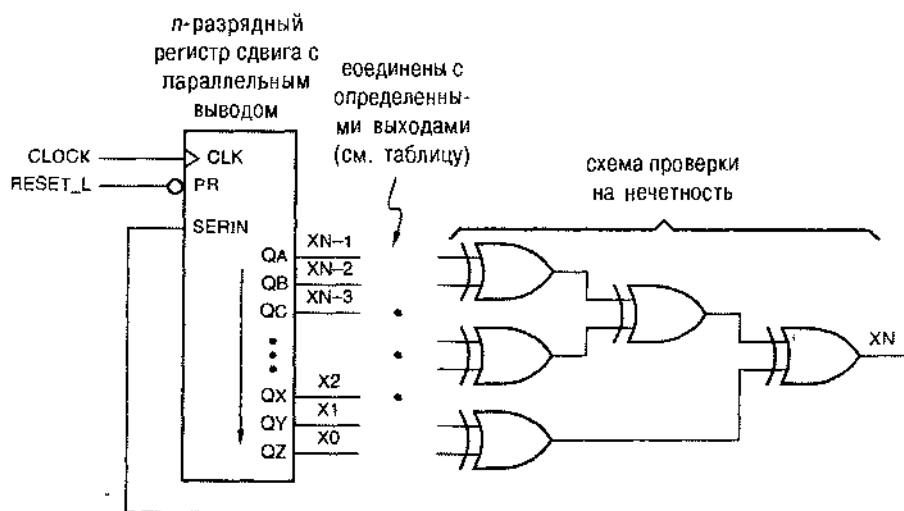


Рис. 8.68. Общая структура счетчика на основе регистра сдвига с линейной обратной связью

В табл. 8.21 для ряда значений n приведены уравнения, описывающие цепь обратной связи в тех случаях, когда результирующая последовательность оказывается последовательностью максимальной длины. Для каждого значения n больше 3-х существует много других уравнений обратной связи, обеспечивающих генерирование последовательностей максимальной длины, причем различным уравнениям соответствуют разные последовательности.

ДЕЙСТВИЯ В КОНЕЧНОМ ПОЛЕ

Конечное поле содержит конечное число элементов, и в нем определены две операции – сложение и умножение, – удовлетворяющие ряду требований. Примером конечного поля с P элементами, где P – простое число, может служить совокупность ненулевых чисел по модулю P . Операциями в этом поле являются сложение и умножение по модулю P .

Согласно теории, конечные поля обладают следующим свойством: если вы начнете с ненулевого элемента E и станете многократно умножать его на так называемый «примитивный» элемент α , то в течение $P - 2$ шагов вы будете получать все другие ненулевые элементы поля, прежде чем снова возникнет элемент E . Оказывается, что в поле с P элементами любое ненулевое число из интервала $2, \dots, P - 1$ является примитивным элементом. Вы можете убедиться в этом сами, взяв, например, $P = 7$ и $\alpha = 2$. Элементами поля при этом являются числа $0, 1, \dots, 6$, а операциями – сложение и умножение по модулю 7. (Здесь автор ошибается: не все элементы $2, \dots, P - 1$ являются примитивными; в частности, не является примитивным элемент 2. – Прим. перев.)

В предыдущем абзаце приведена центральная идея, на которой основывается теория генераторов последовательностей максимальной длины. Но для того, чтобы этой идеей можно было воспользоваться применительно к цифровым схемам, нам необходимо поле с 2^n элементами, где n – требуемое число разрядов. С одной стороны, нам повезло, так как Галуа доказал, что существуют конечные поля с P^n элементами при любом целом n , если только P – простое число, включая случай $P = 2$. Но, с другой стороны, приходится лишь сожалеть о том, что при $n > 1$ операции в полях с P^n элементами (в том числе с 2^n элементами) принципиально отличаются от обычных сложения и умножения целых чисел. Кроме того, труднее находить примитивные элементы.

Если вы, как и я, любите математику, то, должно быть, вас приводит в восторг теория конечных полей, на основе которой строятся генераторы последовательностей максимальной длины и другие LFSR-схемы (см. Обзор литературы). В противном случае, доверьтесь и следуйте рекомендациям этого параграфа по тому же принципу, по которому вы пользуетесь рецептами из «Книжки о вкусной и здоровой пище».

LFSR-счетчик со структурой, указанной на рис. 8.68, никогда не проходит в цикле через все возможные 2^n состояний. Независимо от конфигурации соединенный следующим состоянием за тем, при котором во всех разрядах находятся нули, является то же самое состояние с нулями во всех разрядах.

На рис. 8.69 показана принципиальная схема 3-разрядного LFSR-счетчика. Последовательность состояний этого счетчика приведена в левых трех столбцах табл. 8.22. Начиная с любого ненулевого состояния (в таблице – с состояния 100), счетчик проходит через семь состояний, прежде чем он возвращается в исходное состояние.

<i>n</i>	Уравнения обратной связи
2	$X2 = X1 \oplus X0$
3	$X3 = X1 \oplus X0$
4	$X4 = X1 \oplus X0$
5	$X5 = X2 \oplus X0$
6	$X6 = X1 \oplus X0$
7	$X7 = X3 \oplus X0$
8	$X8 = X4 \oplus X3 \oplus X2 \oplus X0$
12	$X12 = X6 \oplus X4 \oplus X1 \oplus X0$
16	$X16 = X5 \oplus X4 \oplus X3 \oplus X0$
20	$X20 = X3 \oplus X0$
24	$X24 = X7 \oplus X2 \oplus X1 \oplus X0$
28	$X28 = X3 \oplus X0$
32	$X32 = X22 \oplus X2 \oplus X1 \oplus X0$

Табл. 8.21. Уравнения обратной связи для счетчиков на основе регистров сдвига с линейной обратной связью

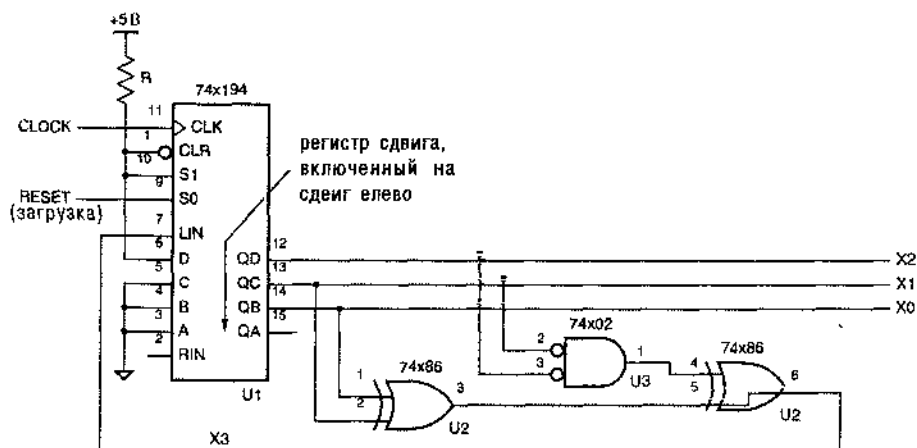


Рис. 8.69. 3-разрядный LFSR-счетчик; модификацией схемы, указанной синим цветом, достигается включение состояния со всеми нулями

Схему LFSR-счетчика можно видоизменить так, чтобы у него было 2^n состояний, включая состояние со всеми нулями; в схеме 3-разрядного счетчика, приведенной на рис. 8.69, синим цветом показано, как это сделать. В результате последовательность состояний будет такой, какая указана в правых трех столбцах табл. 8.22. То же самое можно сделать и в случае n -разрядного LFSR-счетчика: для этого необходимы вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ и вентиль ИЛИ-НЕ с $n-1$ входами, которые должны быть подключены к выходам регистра сдвига, за исключением выхода $X0$.

Табл. 8.22. Последовательность состояний 3-разрядного LFSR-счетчика, приведенного на рис. 8.69

Исходная последовательность			Модифицированная последовательность		
X2	X1	X0	X2	X1	X0
1	0	0	1	0	0
0	1	0	0	1	0
1	0	1	1	0	1
1	1	0	1	1	0
1	1	1	1	1	1
0	1	1	0	1	1
0	0	1	0	0	1
1	0	0	0	0	0
.	.	.	1	0	0
.	.	.	.	.	.

LFSR-счетчик переходит из одного состояния в другое не в порядке двоичного счета. Но во многих приложениях именно это свойство LFSR-счетчиков и является их достоинством. Основное применение LFSR-счетчиков состоит в генерировании тестовых входных сигналов для логических схем. В большинстве случаев «псевдослучайная» последовательность комбинаций, выдаваемых LFSR-счетчиком, предпочтительнее с точки зрения обнаружения ошибок, нежели последовательность комбинаций, перебираемых в порядке двоичного счета. Кроме того, LFSR-схемы используются для кодирования и декодирования применительно к кодам определенного вида, обнаруживающим и исправляющим ошибки, в том числе — к циклическим кодам, рассмотренным в разделе 2.15.4.

При передаче данных LFSR-счетчик часто является одним из узлов высокоскоростных модемов и сетевых интерфейсов, с помощью которых осуществляется «скремблирование» и «дескремблирование» данных. Достигается это путем пропуска выходного сигнала LFSR-схемы и потока данных пользователя через элемент ИСКЛЮЧАЮЩЕЕ ИЛИ. Даже в том случае, когда в потоке данных пользователя имеются длинные последовательности нулей и единиц, смешивание их с псевдослучайным выходным сигналом LFSR-схемы улучшает баланс передаваемого сигнала по постоянному току и создает достаточно большое число переходов, облегчающих извлечение приемником информации о тактовом сигнале.

8.5.9. Описание регистров сдвига на языке ABEL и их реализация в ПЛУ

На языке ABEL совсем легко описывать регистры сдвига общего назначения, а также эффективно размещать их в типичных последовательностных ПЛУ. На рис. 8.70, например, и в табл. 8.23 показано, как с помощью ИС 16V8 реализовать функции, подобные тем, которые выполняет универсальный регистр сдвига 74х194. Обратите внимание: один из выводов ИО (вывод 12 ИС 16V8) используется как вход.

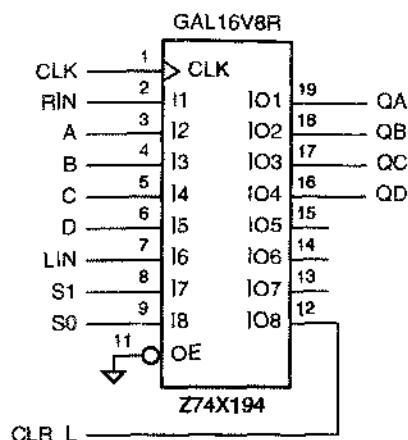


Рис. 8.70. Реализация в ПЛУ функций универсального регистра сдвига 74х194 с синхронным входом сброса

Табл. 8.23. Программа на языке ABEL для 4-разрядного универсального регистра сдвига

```

module Z74x194
title '4-bit Universal Shift Register'
Z74X194 device 'P16V8R';

" Input and output pins
CLK, RIN, A, B, C, D, LIN                pin 1, 2, 3, 4, 5, 6, 7;
S1, S0, CLR_L                            pin 8, 9, 12;
QA, QB, QC, QD                           pin 19, 18, 17, 16 istype 'reg';

" Active-level translation
CLR = !CLR_L;

" Set definitions
INPUT  = [ A, B, C, D ];
LEFTIN = [ QB, QC, QD, LIN ];
RIGHTIN = [ RIN, QA, QB, QC ];
OUT     = [ QA, QB, QC, QD ];

CTRL = [S1,S0];
HOLD  = (CTRL == [0,0]);
RIGHT = (CTRL == [0,1]);
LEFT  = (CTRL == [1,0]);
LOAD  = (CTRL == [1,1]);

equations
OUT.CLK = CLK;

OUT := 'CLR & (
        HOLD & OUT
        # RIGHT & RIGHTIN
        # LEFT & LEFTIN
        # LOAD & INPUT);

end Z74x194

```

Реализация в ИС 16V8 функций регистра '194 отличается от прототипа только тем, как действует входной сигнал CLR_L. У настоящей ИС '194 входной сигнал CLR_L является асинхронным, тогда как ИС 16V8 чувствительна к значению этого сигнала — так же, как и к значению всех других входных сигналов, — только на нарастающем фронте тактового сигнала CLK.

```

module shifty
title '8-bit shift register with decoded load'

" Inputs and Outputs
CLK, OP3..OP0                pin;
Q7..Q0                       pin istype 'reg';

" Definitions

Q = [Q7..Q0];
OP = [OP3..OP0];

HOLD = (OP == 0);
CLEAR = (OP == 1);
LEFT = (OP == 2);
RIGHT = (OP == 3);
NOP = (OP >= 4) & (OP < 8);
LOADQ0 = (OP == 8);
LOADQ1 = (OP == 9);
LOADQ2 = (OP == 10);
LOADQ3 = (OP == 11);
LOADQ4 = (OP == 12);
LOADQ5 = (OP == 13);
LOADQ6 = (OP == 14);
LOADQ7 = (OP == 15);

Equations

Q.CLK = CLK;

WHEN HOLD THEN Q := Q;
ELSE WHEN CLEAR THEN Q := 0;
ELSE WHEN LEFT THEN Q := [Q6..Q0, Q7];
ELSE WHEN RIGHT THEN Q := [Q0, Q7..Q1];
ELSE WHEN LOADQ0 THEN Q := 1;
ELSE WHEN LOADQ1 THEN Q := 2;
ELSE WHEN LOADQ2 THEN Q := 4;
ELSE WHEN LOADQ3 THEN Q := 8;
ELSE WHEN LOADQ4 THEN Q := 16;
ELSE WHEN LOADQ5 THEN Q := 32;
ELSE WHEN LOADQ6 THEN Q := 64;
ELSE WHEN LOADQ7 THEN Q := 128;
ELSE Q := Q;

end shifty

```

Табл. 8.24. Программа на языке ABEL для регистра сдвига со многими функциональными возможностями

Если вам, на самом деле, нужно осуществлять сброс асинхронно, то можно воспользоваться ИС 22V10, где имеется отдельная линия объединения входных сигналов по И для вырабатывания сигнала на входах сброса всех триггеров (см. задачу 8.69).

Гибкость языка ABEL позволяет создавать регистры сдвига с большими или отличными от уже рассмотренных функциональными возможностями. В табл. 8.24, например, описан 8-разрядный регистр сдвига с входом сброса, с возможностью загрузки одной 1 в любой разряд и с функциями сдвига влево, сдвига вправо и хранения. Операция, выполняемая на каждом такте, определяется 4-разрядным кодом операции OP[3:0]. Несмотря на большое разнообразие условий, выражаемых операторами "WHEN", оказывается возможным синтезировать схему, у которой на каждый выход приходится всего лишь пять термов-произведений.

С помощью языка ABEL легко создавать различного типа счетчики на регистрах сдвига, о которых говорилось в предыдущих разделах. В табл. 8.25, например, приведена программа для 8-разрядного кольцевого счетчика. Мы воспользовались здесь предоставляемой ПЛУ доп. возможностью и добавили еще две функции, которых не было в рассмотренном выше случае применения ИС средней степени интеграции: счет происходит только тогда, когда подан сигнал разрешения CNTEN и схема принудительно переводится в состояние S0 сигналом RESTART.

Табл. 8.25. Программа для 8-разрядного кольцевого счетчика

```

module Ring8
title '8-bit Ring Counter'

" Inputs and Outputs
MCLK, CNTEN, RESTART          pin;
S0..S7                        pin istype 'reg';

equations

[S0..S7].CLK = MCLK;

S0 := CNTEN & !S0 & !S1 & !S2 & !S3 & !S4 & !S5 & !S6 " Self-sync
    # !CNTEN & S0 " Hold
    # RESTART; " Start with one 1

[S1..S7] := !RESTART & ( !CNTEN & [S1..S7] " Shift
    # CNTEN & [S0..S6] ); " Hold

end Ring8

```

Кольцевые счетчики часто применяются в цифровых системах для генерирования многофазных тактовых сигналов и сигналов разрешения, потребность в которых велика и требования к которым в различных системах весьма разнообразны. Особым достоинством проектирования на языках описания схем является возможность легко изменять поведение счетчика посредством перепрограммирования.

На рис. 8.71 показана совокупность тактовых сигналов или сигналов разрешения; такие сигналы могут понадобиться цифровой системе, работу которой можно разбить на шесть различных фаз. Система остается в каждой фазе в течение двух тактов основного тактового сигнала MCLK: на протяжении этого интер-

вала времени вырабатывается соответствующий сигнал P_i_L с низким активным уровнем, указывающий на пребывание в данной фазе. Такого рода временные диаграммы можно получить с помощью кольцевого счетчика, добавив еще один триггер, для отсчета двух тактов в каждой фазе так, чтобы сдвиг происходил на втором такте в пределах каждой фазы.

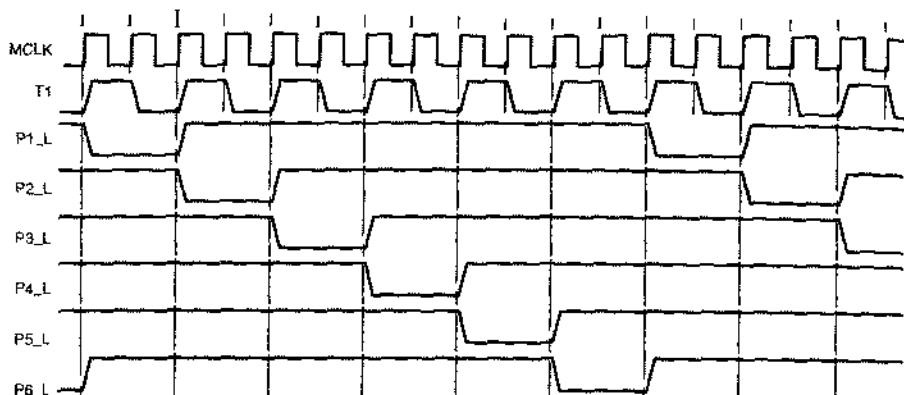


Рис. 8.71. Временные диаграммы совокупности управляющих сигналов с шестью фазами, которые могут понадобиться в той или иной цифровой системе

Табл. 8.26. Программа для генератора шестифазных колебаний

```

module TIMEGEN6
title 'Six-phase Master Timing Generator'

" Input and Output pins
MCLK, RESET, RUN, RESTART      pin;
T1, P1_L, P2_L, P3_L, P4_L, P5_L, P6_L  pin istype 'reg';

" State definitions
PHASES = [P1_L, P2_L, P3_L, P4_L, P5_L, P6_L];
NEXTPH = [P6_L, P1_L, P2_L, P3_L, P4_L, P5_L];
SRESET = [1, 1, 1, 1, 1, 1];
P1 =     [0, 1, 1, 1, 1, 1];

equations
T1.CLK = MCLK; PHASES.CLK = MCLK;

WHEN RESET THEN {T1 := 1; PHASES := SRESET;}
ELSE WHEN (PHASES==SRESET) & RESTART THEN {T1 := 1; PHASES := P1;}
ELSE WHEN RUN & T1 THEN {T1 := 0; PHASES := PHASES;}
ELSE WHEN RUN & !T1 THEN {T1 := 1; PHASES := NEXTPH;}
ELSE {T1 := T1; PHASES := PHASES;}

end TIMEGEN6

```


Генератор таких многофазных сигналов можно создать в ПЛИУ, предусмотрев наличие нескольких входов и выходов. Три управляющих входными сигналами задается следующее поведение устройства.

RESET	Когда подан этот сигнал, все выходные сигналы имеют неактивный уровень. После того, как сигнал RESET снимается, счетчик всегда переходит к первому такту фазы I.
RUN	Наличие сигнала на этом входе позволяет счетчику перейти ко второму такту в текущей фазе или к первому такту следующей фазы, в противном случае на данном такте продолжится пребывание в текущей фазе.
RESTART	Подача этого сигнала вызывает возврат счетчика к первому такту фазы I даже в том случае, когда действует сигнал RUN.

Табл. 8.26 представляет собой программу, посредством которой обеспечивается требуемое поведение. Заметьте, что для задания требуемой реакции на сигналы RESET, RESTART и RUN в любой фазе работы счетчика весьма эффективно использованы наборы.

Генератор, вырабатывающий точно такие же сигналы, какие указаны на рис. 8.71, можно представить в виде конечного автомата, как это сделано в табл. 8.27. Эта программа на языке ABEL длиннее предыдущей, но поведение синтезируемых по этим программам устройств одинаково, тогда как последнюю программу – с определенной точки зрения – легче понять. Все же реализация данной программы требует от 8 до 20 термов И на выход при необходимости только 3–5 термов-произведений на выход в исходном варианте кольцевого счетчика. Это хороший пример того, как можно повысить эффективность расходования схемных ресурсов и получить лучшие характеристики, приспособив к «требованиям заказчика» простую стандартную структуру, а не вымучивать из себя решение задачи «в лоб» путем ностроения конечного автомата.

Давайте рассмотрим теперь другой вариант многофазных колебаний, которые также могут понадобиться в той или иной системе. На рис. 8.72 приведены временные диаграммы колебаний, подобных рассмотренным выше, но отличающихся тем, что выходные сигналы F_1 – F_4 посредством которых отмечаются разные фазы, удерживаются на активном уровне только в пределах одного периода тактового сигнала в каждой из фаз. Это небольшое отличие может быть очень важным для проекта в целом.

В исходном варианте мы воспользовались 6-разрядным кольцевым счетчиком и одной дополнительной переменной состояния T1, с помощью которой отображались два состояния в пределах каждой фазы. В случае новых колебаний так поступить нельзя. В состояниях, которые имеют место между импульсами с низким активным уровнем (STATE = 0, 2, 4 и т.д. на рис. 8.72), все выходные сигналы принимают неактивное значение, так что по ним уже нельзя судить о том, в какое следующее состояние следует перейти. Необходимо что-то другое для отслеживания непочки состояний

Табл. 8.27. Альтернативный вариант программы для генератора шестифазных колебаний

```

module TIMEGN6A
title 'Six-phase Master Timing Generator'

" Input and Output pins
MCLK, RESET, RUN, RESTART      pin;
T1, P1_L, P2_L, P3_L, P4_L, P5_L, P6_L      pin istype 'reg';

" State definitions
TSTATE = [T1, P1_L, P2_L, P3_L, P4_L, P5_L, P6_L];
SRESET = [1, 1, 1, 1, 1, 1, 1];
P1F =    [1, 0, 1, 1, 1, 1, 1];
P1S =    [0, 0, 1, 1, 1, 1, 1];
P2F =    [1, 1, 0, 1, 1, 1, 1];
P2S =    [0, 1, 0, 1, 1, 1, 1];
P3F =    [1, 1, 1, 0, 1, 1, 1];
P3S =    [0, 1, 1, 0, 1, 1, 1];
P4F =    [1, 1, 1, 1, 0, 1, 1];
P4S =    [0, 1, 1, 1, 0, 1, 1];
P5F =    [1, 1, 1, 1, 1, 0, 1];
P5S =    [0, 1, 1, 1, 1, 0, 1];
P6F =    [1, 1, 1, 1, 1, 1, 0];
P6S =    [0, 1, 1, 1, 1, 1, 0];

equations
TSTATE.CLK = MCLK;
WHEN RESET THEN TSTATE := SRESET;

state_diagram TSTATE

state SRESET: IF RESET THEN SRESET ELSE P1F;

state P1F: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P1S ELSE P1F;

state P1S: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P2F ELSE P1S;

state P2F: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P2S ELSE P2F;

state P2S: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P3F ELSE P2S;

state P3F: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P3S ELSE P3F;

state P3S: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P4F ELSE P3S;

state P4F: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P4S ELSE P4F;

```

Табл. 8.27. Альтернативный вариант программы для генератора шестифазных колебаний (продолжение)

```

state P4S: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P5F ELSE P4S;

state P5F: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P5S ELSE P5F;

state P5S: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P6F ELSE P5S;

state P6F: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P6S ELSE P6F;

state P6S: IF RESET THEN SRESET ELSE IF RESTART THEN P1F
           ELSE IF RUN THEN P1F ELSE P6S;

end TIMEGN6A

```

НАДЕЖНЫЙ СБРОС

Заметьте, что в табл. 8.27 значение присваивается набору TSTATE в разделе equations, а результат используется в разделе state_diagram. Мы сейчас объясним, что это сделано с вполне определенной целью: обеспечить в программе переход в состояние SRESET из любого не определенного состояния.

Программа ABEL пополняет множество включений, относящееся к данному выходу, всякий раз, когда сигнал на этом выходе встречается в левой части равенства (см. раздел 4.6.3, где этот вопрос был рассмотрен применительно к комбинационным выходам). В случае регистровых выходов множество включений для каждого состояния в векторе состояний увеличивается на единицу при каждом упоминании "state" в разделе state_diagram. Всякая комбинация входных сигналов, которая вызывает появление 1 на выходе для каждой переменной состояния, добавляется в множество включений очередным предложением, начинающимся с ключевого слова state.

У конечного автомата, задаваемого программой в табл. 8.27, всего $2^7 = 128$ состояний, из которых явно определены только 13, и только для них указаны переходы в состояние SRESET. Но равенства, содержащие операторы WHEN, гарантируют переход автомата в состояние SRESET, когда бы ни возник сигнал RESET. Это справедливо независимо от определений state в разделе state_diagram. Когда сигнал RESET переходит на активный уровень, содержащее один единицы кодовое имя состояния SRESET, в действительности, объединяется по правилу ИЛИ со следующим состоянием, если только оно задается в разделе state_diagram. При таком подходе нельзя обеспечить надежный сброс, если кодовое имя состояния SRESET содержит, например, только нули.

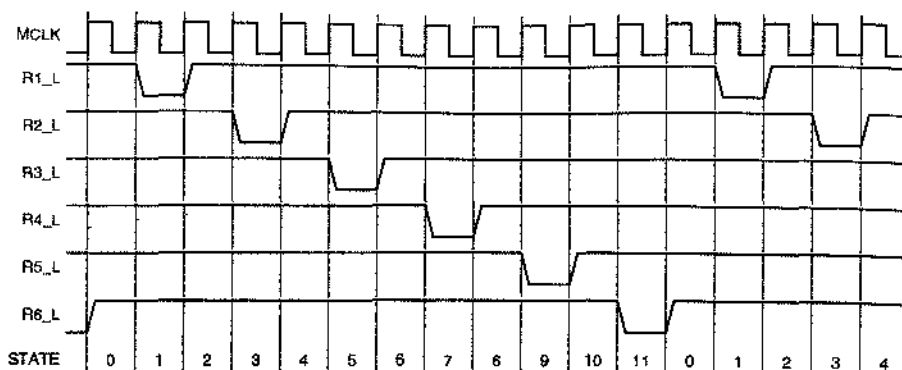


Рис. 8.72. Временные диаграммы для видоизмененных многофазных колебаний

Эту проблему можно решить многими способами. Одна из идей состоит в том, чтобы выбрать в качестве отправной точки исходный проект (табл. 8.26), используя фазовые сигналы P1_L, P2_L и т.д. только в качестве указателей на внутренние состояния. Тогда можно считать, что каждый фазовый сигнал R_i_L является комбинационным выходом автомата Мура, приписывающим активное значение, когда соответствующий сигнал P_i_L имеет активный уровень и схема находится на втором такте в пределах данной фазы. Для реализации этого первого подхода программу на языке ABEL нужно дополнить строками, приведенными в табл. 8.28.

Табл. 8.28. Добавления в программу, содержащуюся в табл. 8.26, для модифицированного генератора шестифазных колебаний

```

module TIMEG12K
...
R1_L, R2_L, R3_L, R4_L, R5_L, R6_L      pin istype 'com';
...
OUTPUTS = [R1_L, R2_L, R3_L, R4_L, R5_L, R6_L];

equations
...
!OUTPUTS = !PHASES & !T1;

end TIMEG12K

```

Этот первый подход легко реализовать и он дает прекрасные результаты, если мы собираемся использовать сигналы R_i_L только в качестве сигналов разрешения или других управляющих входных сигналов. Но это плохая идея в том случае, когда данным сигналам предстоит играть роль тактовых сигналов, поскольку, как мы сейчас объясним, у них могут быть паразитные импульсы. Сигналы R_i_L и T1 являются выходными сигналами триггеров, переключающихся одним и тем же основным тактовым сигналом MCLK. Хотя сигналы изменяются на выходах триггеров примерно в одно и то же время, это никогда не происходит точно

одновременно. Один из них может изменяться быстрее, а другой – медленнее; это обстоятельство называется *расхождением выходных сигналов по времени* (*output timing skew*). Предположим, например, что при переходе из состояния 1 в состояние 2 на рис. 8.71 низкий уровень появляется в сигнале P2_L раньше, чем уровень сигнала T1 становится высоким. В этом случае в выходном сигнале R2_L может возникнуть короткий паразитный импульс.

Чтобы обеспечить отсутствие паразитных импульсов, мы должны разработать схему, у которой каждый фазовый сигнал был бы регистровым выходным сигналом. Один из способов достичь этого заключается в применении 12-рядного кольцевого счетчика; для получения сигналов желаемого вида используются выходы только каждого второго триггера. Программа на языке ABEL, реализующая этот подход, приведена в табл. 8.29.

Табл. 8.29. Программа на языке ABEL для модифицированного генератора шестифазных колебаний

```

module TIMEG12
title 'Modified six-phase Master Timing Generator'

% Input and Output pins
MCLK, RESET, RUN, RESTART           pin;
P1_L, P2_L, P3_L, P4_L, P5_L, P6_L   pin istype 'reg';
P1A, P2A, P3A, P4A, P5A, P6A         pin istype 'reg';

% State definitions
PHASES = [P1A, P1_L, P2A, P2_L, P3A, P3_L, P4A, P4_L, P5A, P5_L, P6A, P6_L];
NEXTPH = [P6_L, P1A, P1_L, P2A, P2_L, P3A, P3_L, P4A, P4_L, P5A, P5_L, P6A];
SRESET = [1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1];
P1 =     [0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1];

equations

PHASES.CLK = MCLK;

WHEN RESET THEN PHASES := SRESET;
ELSE WHEN RESTART * (PHASES == SRESET) THEN PHASES = P1;
ELSE WHEN RUN THEN PHASES := NEXTPH;
ELSE PHASES := PHASES;

end TIMEG12

```

Еще один очевидный возможный путь – с учетом того, что в пределах цикла совокупность колебаний проходит через 12 состояний, – это построение двоичного счетчика по модулю 12 и дешифратора состояний такого счетчика. В табл. 8.30 представлена программа на языке ABEL, в которой использовал данный подход. Состояниям счетчика соответствуют значения переменной “STATE” на рис. 8.72. Поскольку фазовые выходные сигналы объявлены как регистровые, в них нет паразитных импульсов. Заметьте, что для компенсации задержки при декодировании на один такт предусмотрено декодирование на один период тактового сигнала раньше. Кроме того, при сбросе счетчик переводится в состояние 15, а

не в состояние 0, чтобы при осуществлении сброса не возникло активное значение сигнала P1_L.

Табл. 8.30. Программа для генератора шестифазных колебаний на основе счетчика

```

module TIMEG12A
title 'Counter-based six-phase master timing generator'

" Input and Output pins
MCLK, RESET, RUN, RESTART      pin;
P1_L, P2_L, P3_L, P4_L, P5_L, P6_L  pin istype 'reg';
CNT3..CNT0                       pin istype 'reg';

" Definitions
CNT = [CNT3..CNT0];
P_L = [P1_L, P2_L, P3_L, P4_L, P5_L, P6_L];

equations

CNT.CLK = MCLK;  P_L.CLK = MCLK;

WHEN RESET THEN CNT := 15
ELSE WHEN RESTART THEN CNT := 0
ELSE WHEN (RUN & (CNT < 11)) THEN CNT := CNT + 1
ELSE WHEN RUN THEN CNT := 0
ELSE CNT := CNT;

P1_L := !(CNT == 0);
P2_L := !(CNT == 2);
P3_L := !(CNT == 4);
P4_L := !(CNT == 6);
P5_L := !(CNT == 8);
P6_L := !(CNT == 10);

end TIMEG12A

```

8.5.10. Описание регистров сдвига на языке VHDL

На языке VHDL регистры сдвига можно задавать структурно и в поведенческом стиле; мы рассмотрим несколько поведенческих описаний и их применение. Табл. 8.31 представляет собой описание работы 8-разрядного регистра сдвига с расширенным набором функций. Помимо хранения, загрузки и сдвига, выполняемых ИС 74х194 и 74х299, в данном регистре осуществляются операции циклического и арифметического сдвига. При *циклическом сдвиге* (*circular shift*) бит, «теряющийся» в результате сдвига с одного конца регистра, поступает на другой его конец. При *арифметическом сдвиге* (*arithmetic shift*) значение бита, «заталкиваемого» в регистр, устанавливается из соображений умножения или деления на 2: при сдвиге влево на правый вход подается 0, а при сдвиге вправо повторяется крайний левый (знаковый) бит.

Табл. 8.31. Описание работы 8-разрядного регистра сдвига с расширенными функциями

Функция	Входы			Следующее состояние							
	S2	S1	S0	Q7+	Q6+	Q5+	Q4+	Q3+	Q2+	Q1+	Q0+
Хранение	0	0	0	Q7	Q6	Q5	Q4	Q3	Q2	Q1	Q0
Загрузка	0	0	1	D7	D6	D5	D4	D3	D2	D1	D0
Сдвиг вправо	0	1	0	RIN	Q7	Q6	Q5	Q4	Q3	Q2	Q1
Сдвиг влево	0	1	1	Q6	Q5	Q4	Q3	Q2	Q1	Q0	LIN
Циклический сдвиг вправо	1	0	0	Q0	Q7	Q6	Q5	Q4	Q3	Q2	Q1
Циклический сдвиг влево	1	0	1	Q8	Q5	Q4	Q3	Q2	Q1	Q0	Q7
Арифметический сдвиг вправо	1	1	0	Q7	Q7	Q6	Q5	Q4	Q3	Q2	Q1
Арифметический сдвиг влево	1	1	1	Q6	Q5	Q4	Q3	Q2	Q1	Q0	0

В табл. 8.32 приведена поведенческая VHDL-программа для регистра сдвига с расширенными функциями. Как и в предыдущих примерах, определяется процесс `n` для обеспечения желаемого переключения по фронту тактового сигнала используется признак `event`. Заслуживают внимание следующие особенности этой программы:

- Введен внутренний сигнал `IQ`, который в конце копцов становится выходным сигналом `Q`, по таким, что его могут читать и писать операторы процесса. Можно было бы поступить и иначе, определив тип выходного сигнала `Q` как "buffer".
- Вход `CLR` является асинхронным; поскольку этот сигнал входит в список чувствительности процесса, он проверяется всякий раз, когда претерпевает изменение. Операторам `IF` придана такая структура, что учет значения `CLR` предшествует анализу любого другого условия.
- Для определения операций, реализуемых регистром сдвига при восьми возможных значениях входных сигналов выбора `S (2 downto 0)`, применен оператор `CASE`.
- В операторе `CASE` необходимо предусмотреть случай "when others", чтобы предотвратить предупреждение компилятора о том, что примерно 2^{32} случаев остаются не принятыми во внимание.
- Оператор "null" указывает, что в некоторых случаях никаких действий производить не надо. Заметьте, что ничего не нужно делать в случае 0; бездействие зарезервировано в качестве сигнала удержания сохраняемых регистром данных до тех пор, пока не будет велено поступить иначе.
- В большинстве случаев для образования 8-разрядного массива из 7-разрядного подмножества `IQ` и еще одного бита применяется оператор конкатенации "&".
- Из-за строгих требований языка VHDL к согласованию типов в операторе `CASE` используется определенная в разделе 4.7.4 функция `CONV_INTEGER` для преобразования входного сигнала выбора `S` типа `STD_LOGIC_VECTOR` в целое число. Можно было бы сделать иначе, записав метку каждого случая как элемент типа `STD_LOGIC_VECTOR` [например: ('0', '1', '1'), а не целое число 3].

Табл. 8.32. VHDL-программа для 8-разрядного регистра сдвига с расширенными функциями

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vshftreg is
  port (
    CLK, CLR, RIN, LIN: in STD_LOGIC;
    S: in STD_LOGIC_VECTOR (2 downto 0); -- function select
    D: in STD_LOGIC_VECTOR (7 downto 0); -- data in
    Q: out STD_LOGIC_VECTOR (7 downto 0) -- data out
  );
end Vshftreg;

architecture Vshftreg_arch of Vshftreg is
  signal IQ: STD_LOGIC_VECTOR (7 downto 0);
begin
  process (CLK, CLR, IQ)
  begin
    if (CLR='1') then IQ <= (others=>'0'); -- Asynchronous clear
    elsif (CLK'event and CLK='1') then
      case CONV_INTEGER(S) is
        when 0 => null; -- Hold
        when 1 => IQ <= D; -- Load
        when 2 => IQ <= RIN & IQ(7 downto 1); -- Shift right
        when 3 => IQ <= IQ(6 downto 0) & LIN; -- Shift left
        when 4 => IQ <= IQ(0) & IQ(7 downto 1); -- Shift circular right
        when 5 => IQ <= IQ(6 downto 0) & IQ(7); -- Shift circular left
        when 6 => IQ <= IQ(7) & IQ(7 downto 1); -- Shift arithmetic right
        when 7 => IQ <= IQ(6 downto 0) & '0'; -- Shift arithmetic left
        when others => null;
      end case;
    end if;
    Q <= IQ;
  end process;
end Vshftreg_arch;

```

Одно из применений регистров сдвига – это кольцевые счетчики; примером такого применения служит рассмотренный в предыдущем разделе генератор щети-фазных колебаний, изображенных на рис. 8.71. В табл. 8.33 приведена VHDL-программа, обеспечивающая такое же поведение устройства. Как и в предыдущем VHDL-примере для чтения и записи используется внутренний сигнальный вектор *IP* с высоким активным уровнем, становящийся в конце концов выходным сигналом устройства, чтобы получить требуемый выходной сигнальный вектор с низким активным уровнем, удобно инвертировать этот внутренний сигнал в последнем операторе. Остальная часть программы не содержит никаких особенностей, но заметьте, что у вложенного оператора *IF* имеются три уровня.

Табл. 8.33. VHDL-программа для генератора шестифазных колебаний

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vtimegn6 is
  port (
    MCLK, RESET, RUN, RESTART: in STD_LOGIC; -- clock, control inputs
    P_L: out STD_LOGIC_VECTOR (1 to 6)      -- active-low phase outputs
  );
end Vtimegn6;

architecture Vtimegn6_arch of Vtimegn6 is
  signal IP: STD_LOGIC_VECTOR (1 to 6); -- internal active-high phase signals
  signal T1: STD_LOGIC;                -- first tick within phase
begin
  process (MCLK, IP)
  begin
    if (MCLK'event and MCLK='1') then
      if (RESET='1') then
        T1 <= '1'; IP <= ('0','0','0','0','0','0');
      elsif ((IP=('0','0','0','0','0','0')) or (RESTART='1')) then
        T1 <= '1'; IP <= ('1','0','0','0','0','0');
      elsif (RUN='1') then
        T1 <= not T1;
        if (T1='0') then IP <= IP(6) & IP(1 to 5); end if;
      end if;
    end if;
    P_L <= not IP;
  end process;
end Vtimegn6_arch;

```

Возможной модификацией рассмотренного приложения является устройство, выходные колебания которого удерживаются на активном уровне только во втором такте каждой фазы длительностью в два такта; эти колебания были показаны на рис. 8.72. Один из способов достичь этого заключается в создании 12-разрядного кольцевого счетчика и использовании выходов только каждого второго триггера. При реализации такого устройства VHDL-средствами в определении объекта фигурировали бы только шесть фазовых выходных сигналов P_L (1 to 6). Шесть дополнительных сигналов, названных NEXT P (1 to 6), объявлены в определении архитектуры и являются локальными. На рис. 8.73 показано соотношение между этими сигналами при выполнении операции сдвига, а в табл. 8.34 приведена соответствующая VHDL-программа.

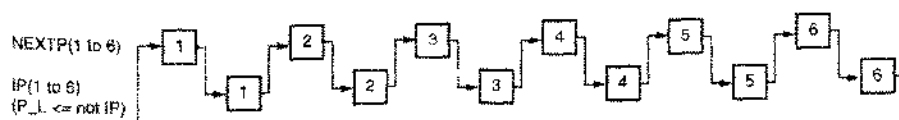


Рис. 8.73. Последовательность сдвигов в генераторе шестифазных колебаний на основе 12-разрядного кольцевого счетчика

Табл. 8.34. VHDL-программа для модифицированного генератора шестифазных колебаний

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vtimeg12 is
    port (
        MCLK, RESET, RUN, RESTART: in STD_LOGIC; -- clock, control inputs
        P_L: out STD_LOGIC_VECTOR (1 to 6)         -- active-low phase outputs
    );
end Vtimeg12;

architecture Vtimeg12_arch of Vtimeg12 is
    signal IP, NEXTP: STD_LOGIC_VECTOR (1 to 6); -- internal active-high phase signals
begin
    process (MCLK, IP, NEXTP)
        variable TEMP: STD_LOGIC_VECTOR (1 to 6); -- temporary for signal shift
        constant IDLE: STD_LOGIC_VECTOR (1 to 6) := ('0','0','0','0','0','0');
        constant FIRSTP: STD_LOGIC_VECTOR (1 to 6) := ('1','0','0','0','0','0');
    begin
        if (MCLK'event and MCLK='1') then
            if (RESET='1') then IP <= IDLE; NEXTP <= IDLE;
            elsif (RESTART='1') or (IP=IDLE and NEXTP=IDLE) then IP <= IDLE; NEXTP <= FIRSTP;
            elsif (RUN='1') then
                if (IP=IDLE) and (NEXTP=IDLE) then NEXTP <= FIRSTP;
                else TEMP := IP; IP <= NEXTP; NEXTP <= TEMP(6) & TEMP(1 to 5);
                end if;
            end if;
            end if;
            P_L <= not IP;
        end process;
    end Vtimeg12_arch;

```

Как и в предыдущей программе, в теле архитектуры объявлен 6-разрядный сигнал IP с высоким активным уровнем, который используется для чтения и записи и становится в конце концов выходным сигналом устройства P_L с низким активным уровнем. Добавочный 6-разрядный сигнал NEXTP хранит остающиеся 6 двоичных переменных состояния. Константы IDLE и FIRSTP делают программу более удобочитаемой.

Заметьте, что 6-разрядная переменная TEMP используется только как место временного хранения старого значения IP при осуществлении сдвига: в IP загружается содержимое NEXTP, а в NEXTP — сдвинутое старое значение IP. Поскольку операторы присваивания в процессе выполняются *последовательно*, мы не могли бы обойтись простой записью “IP <= NEXTP; NEXTP <= IP (6) & IP (1 to 5);”. Если бы мы так сделали, то в NEXTP попало бы *новое* значение IP, а не старое. Обратите также внимание на следующее: так как TEMP является локальной переменной, а не сигналом, ее значение вне процесса не видно. Кроме того, при очередном обращении к процессу никогда не используется значение TEMP, образовавшееся при предыдущем вызове процесса. Поэтому VHDL-компилятору не нужно синтезировать никаких триггеров для хранения значения TEMP.

*8.6. Итерационные и последовательностные схемы

С итерационными схемами мы познакомились в разделе 5.9.2. Функцию итерационной схемы, состоящей из n модулей, может выполнить последовательностная схема, в которой модуль имеется лишь в одном экземпляре, но ей требуется совершить n шагов (за n тактов), чтобы получить нужный результат. Это блестящий пример пространственно-временного обмена при цифровом проектировании.

Как видно из рис. 8.74, в последовательностной схеме для сохранения значений сигналов в межкаскадном соединении в конце каждого шага используются триггеры; именно их выходные сигналы служат входными сигналами межкаскадного соединения в начале следующего шага. Перед началом первого такта триггеры необходимо загрузить граничными значениями входных сигналов; по окончании n -го такта они содержат граничные значения выходных сигналов.

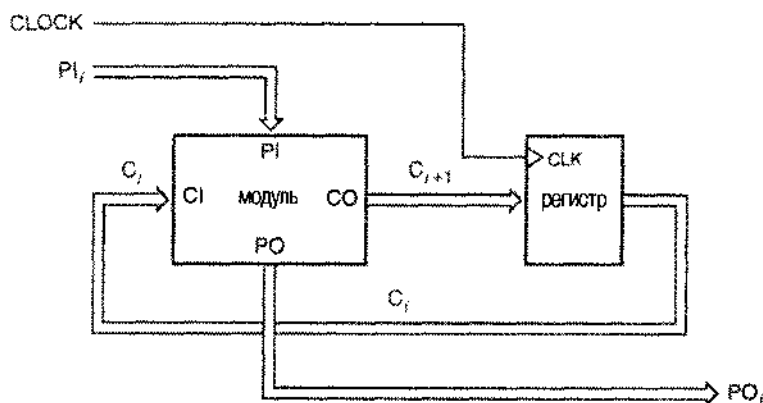


Рис. 8.74. Общая структура последовательностной схемы, эквивалентной итерационной схеме

Поскольку итерационная схема является комбинационной, все ее первичные и граничные входные сигналы можно подать одновременно, а все ее первичные и граничные выходные сигналы доступны спустя время, равное комбинационной задержке. В последовательностной схеме первичные входные сигналы должны поступать последовательно, по одному комплекту на каждом такте, и первичные выходные сигналы должны вырабатываться в соответствующие моменты времени. Поэтому входные сигналы часто формируют с помощью регистров сдвига с последовательным выводом, а выходные сигналы собирают в регистрах сдвига с последовательным вводом. По этой причине часто говорят, что в последовательностной схеме состояния итерационной схемы поочередно следуют одно за другим.

На рис. 8.75 в качестве примера приведена основная схема *последовательного компаратора (serial comparator)*. Затененный участок схемы тождественен одному модулю итерационного компаратора, изображенного на рис. 5.80. Более подробная схема показана на рис. 8.76 в предположении, что компаратор собирается на основе ИС малой степени интеграции. В схеме предусмотрен синхронный сброс: при подаче сигнала на этот вход междукаскадный триггер на следующем такте устанавливается в начальное единичное состояние. Это начальное значение соответствует граничному входному сигналу итерационного компаратора.

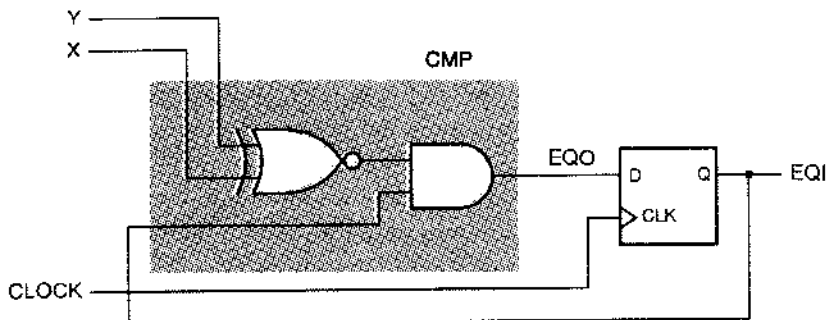


Рис. 8.75. Упрощенная схема последовательного компаратора

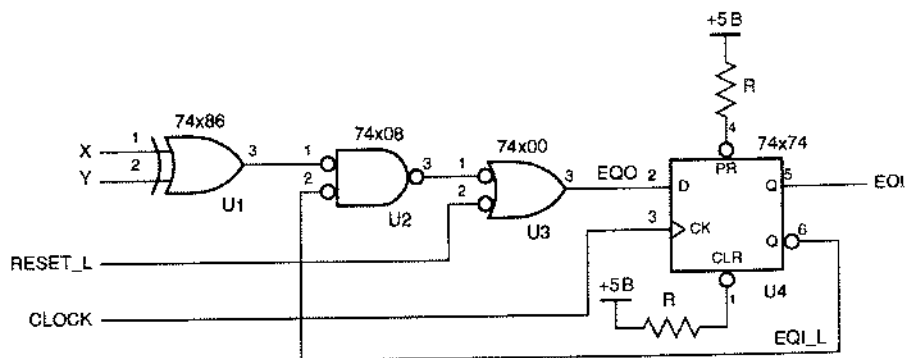


Рис. 8.76. Уточненная схема последовательного компаратора

С помощью последовательного компаратора n -разрядное сравнение осуществляется за $n + 1$ тактов. Сигнал RESET_L действует на первом такте, а на следующих n тактах этот сигнал имеет неактивный уровень, и на входы X и Y подаются биты данных. На выходе EQI вырабатывается результат сравнения, удерживаемый в течение периода, следующего за очередным перепадом в тактовом сигнале. На рис. 8.77 приведены временные диаграммы для последовательного сравнения в 4-х разрядах. Паразитные импульсы в сигнале EQO возникают в те моменты времени, когда изменяются сигналы на выходах комбинационных схем в качестве отклика на поступление новых значений сигналов на входы X и Y.

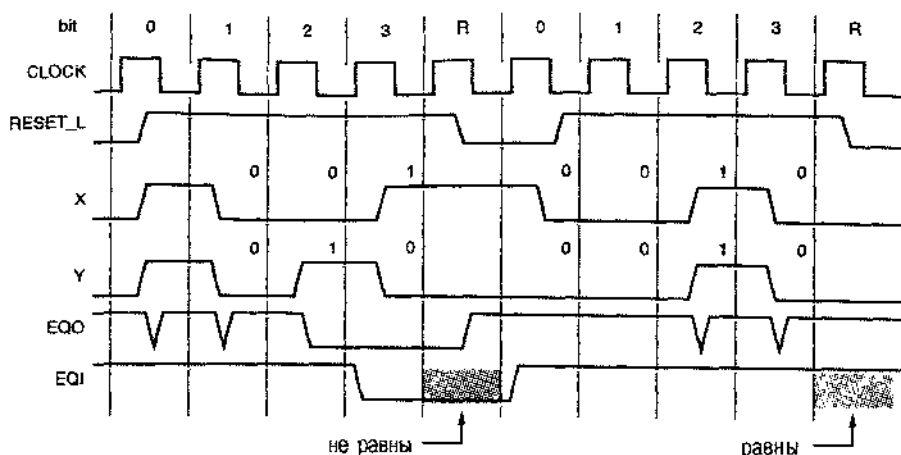


Рис. 8.77. Временные диаграммы для последовательного компаратора

На рис. 8.78 представлена схема *последовательного двоичного сумматора* (*serial binary adder*), состоящая из полного сумматора и D-триггера и позволяющая складывать двоичные числа любой длины. В триггере запоминается перенос между последовательными битами, возникающий при суммировании; при сбросе он устанавливается в 0. На входы A и B последовательно подаются биты складываемых чисел, начиная с младшего разряда, а на выходе S в том же порядке появляются биты суммы.

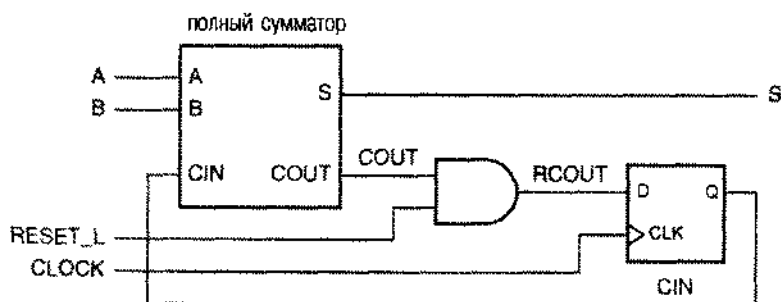


Рис. 8.78. Схема последовательного двоичного сумматора

На ранней стадии развития цифровой электроники логические схемы имели большие размеры и стоили дорого; поэтому во многих компьютерах и в калькуляторах для выполнения арифметических действий применялись последовательные сумматоры и последовательные эквиваленты других итерационных схем. Сегодня эти схемы почти не применяются для нужд арифметики, но служат поучительным напоминанием о возможности пространственно-временных обменов при разработке цифровых устройств.

8.7. Методология синхронного проектирования

В синхронной системе (*synchronous system*) все триггеры переключаются по одному и тому же общему сигналу, а входы установки в единичное состояние и сброса не используются за исключением начальной инициализации системы. Хотя весь мир и не марширует под одну и ту же музыку, в пределах цифровой системы или подсистемы мы вполне можем все делать в такт с общим тактовым сигналом. При состыковке цифровых систем или подсистем с различными тактовыми сигналами обычно бывает достаточно указать ограниченное число асинхронных сигналов, требующих специальной обработки; мы увидим это в разделе 8.8.3.

В синхронных системах гонки и источники опасности не страшны по двум причинам. Во-первых, схемами классического образца, в которых могли бы иметь место гонки и существенные источники опасности, являются только заранее отлаженные элементы, например, триггеры в дискретном исполнении или ячейки специализированной ИС, а в отношении таких элементов производитель гарантирует, что они будут работать хорошо. Во-вторых, статические, динамические и функциональные источники опасности, которыми могут обладать комбинационные схемы на управляющих входах триггеров, не оказывают никакого влияния на работу системы, так как существенны значения сигналов на управляющих входах только в такие моменты времени, когда паразитные импульсы уже не имеют возможности оказать какое-либо воздействие.

Помимо проработки функционального поведения конечного автомата, разработчику практической синхронной системы или подсистемы необходимо решить следующие три четко очерченные задачи, без чего нельзя гарантировать надежную работу системы:

1. Найти и минимизировать разброс задержек тактового сигнала при его прохождении по разным путям, о чем пойдет речь в разделе 8.8.1.
2. Обеспечить наличие у триггеров положительного запаса по времени установления и по времени удержания с учетом возможного разброса задержек тактового сигнала в соответствии с тем, что говорилось по этому поводу в разделе 8.1.4.
3. Синхронизировать воздействие асинхронных входных сигналов с тактовым сигналом, позаботившись о том, чтобы вероятность отказа у схем синхронизации была достаточно малой; эти вопросы обсуждаются в разделе 8.8.3 и в параграфе 8.9.

Прежде чем подробно разобрать эти три задачи, давайте рассмотрим общую модель синхронной системы и один конкретный пример.

8.7.1. Структура синхронной системы

В главе 7 примерами проектирования последовательностных схем были, главным образом, отдельные конечные автоматы с небольшим числом состояний. Но если число триггеров в последовательностной схеме заметно больше, то неудобно (а часто и невозможно) рассматривать схему как одно целое, поскольку число состояний такого конечного автомата слишком велико и нам с ним не справиться.

К счастью, в большинстве случаев цифровые системы или подсистемы можно разбить на две или большее число частей. Независимо от того, что именно обрабатывается в цифровой системе, – числа, оцифрованный речевой сигнал или поток импульсов в системе зажигания, – можно считать, что в определенной части цифровой системы происходит запоминание, перенаправление, объединение или преобразование «данных» в самом общем виде; эту часть мы будем называть *устройством обработки данных (data unit)*. Что касается другой части, которую мы будем называть *управляющим устройством (control unit)*, то она производит запуск или остановку устройства обработки данных, проверяет правильность его функционирования и, в зависимости от обстоятельств, решает, что надо делать дальше. Как правило, только управляющее устройство необходимо разрабатывать как конечный автомат. Об устройстве обработки данных и о его компонентах обычно можно думать на более высоком уровне абстракции, например:

- *Регистры.* Совокупность триггеров параллельно загружается многими битами «данных», которые затем могут быть использованы или извлечены все вместе.
- *Специализированные функции.* В этом случае речь может идти о многоразрядных счетчиках или регистрах сдвига, в которых по команде происходит приращение или сдвиг их содержимого.
- *Память,* предусматривающая запись и чтение. Совокупность отдельных защелок и триггеров, рассматриваемых как единое целое с точки зрения возможности записи и считывания.

Первые две темы уже обсуждались нами ранее в этой главе, а последний вопрос будет рассмотрен в главе 10.

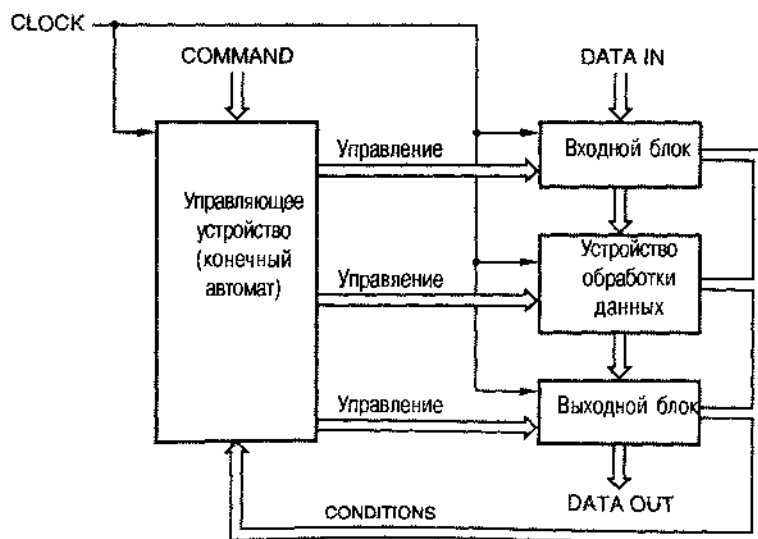


Рис. 8.79. Структура синхронной системы

На рис. 8.79 в самом общем виде представлена блок-схема системы, где, помимо управляющего устройства и устройства обработки данных, явным образом включены входной и выходной блоки; впрочем, выполняемые этими блоками функции легко включить в то, что делает устройство обработки данных. Управляющее устройство представляет собой конечный автомат с *командными входами* (*command inputs*) и *входами условий* (*condition inputs*); сигналами на командных входах определяется функция, которую должен выполнить автомат, а сигналы на входах условий вырабатываются устройством обработки данных. На командные входы сигналы поступают от другой подсистемы или от пользователя, и ими устанавливается режим работы управляющего конечного автомата (RUN/HALT, NORMAL/TURBO и т.д.). Сигналы на входах условий позволяют управляющему устройству изменить свое поведение, как того требуют обстоятельства, по результатам работы устройства обработки данных (ZERO_DETECTION, MEMORY_FULL и т.д.)

Ключевым моментом в структуре, приведенной на рис. 8.79, является то, что во всех блоках – в управляющем устройстве, в устройстве обработки данных, во входном блоке и выходном блоке – используется один и тот же общий тактовый сигнал. Рис. 8.80 служит иллюстрацией того, что происходит в управляющем устройстве и в устройстве обработки данных в пределах одного периода тактового сигнала:

1. Спустя небольшое время после начала очередного периода устанавливается состояние управляющего устройства и припадают установившиеся значения регистровых выходные сигналы устройства обработки данных.
2. Затем, спустя время, определяемое задержкой комбинационной логики, устанавливаются значения сигналов на выходах типа Мура управляющего конечного автомата. Для устройства обработки данных эти сигналы являются управляющими *входными сигналами*. Именно ими определяется, какие функции выполняет устройство обработки данных на оставшейся части периода тактового сигнала: например, выбор адреса в памяти, пути прохождения сигнала в мультиплексоре или реализация арифметического действия.
3. Ближе к концу периода тактового сигнала в устройстве обработки данных вырабатываются и становятся доступными управляющему устройству выходные сигналы условий типа обнаружения нуля или переполнения.
4. В конце периода тактового сигнала, к моменту времени, который непосредственно предшествует началу интервала, задаваемого временем установления, логикой переходов в управляющем конечном автомате определяется следующее состояние; оно зависит от текущего состояния, а также от сигналов на командных входах и входах условий. Примерно к этому же времени результаты вычислений в устройстве обработки данных оказываются готовыми к тому, чтобы быть загруженными в регистры этого устройства.
5. С приходом фронта тактового сигнала описанный цикл может повторяться.

Сигналы на управляющих входах устройства обработки данных, являющиеся выходными сигналами управляющего конечного автомата, могут быть сигналами типа Мура, Мили или сигналами на конвейерном выходе Мили; приведенные на рис. 8.80 временные диаграммы относятся к случаю выходов типа Мура. Сиг-

налы на выходах типа Мура или на конвейерных выходах типа Мили управляют работой устройства обработки данных в строгом соответствии с текущим состоянием и последними значениями входных сигналов, но это воздействие не зависит от сигналов условий, вырабатываемых устройством обработки данных *в этом периоде* тактового сигнала. Это повышает гибкость системы, но одновременно требует большего времени для правильной работы, поскольку много большими становятся задержки распространения сигнала, что приводит, как минимум, к увеличению требуемого периода тактового сигнала. Кроме того, выходы типа Мили не должны замыкаться в петли обратной связи; например, подача сигнала добавления 1 на вход сумматора при ненулевом сигнале на выходе сумматора вызовет осцилляции, если число на выходе сумматора равно -1 .

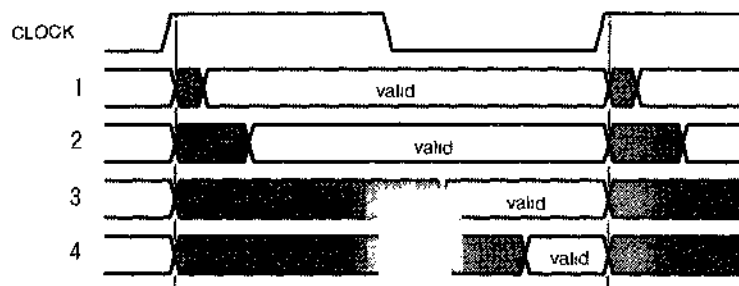


Рис. 8.80. События, происходящие в синхронной системе в пределах одного периода тактового сигнала

1. Состояние управляющего устройства и сигналы на регистровых выходах устройства обработки данных
 2. Сигналы на управляющих входах устройства обработки данных
 3. Сигналы условий, вырабатываемые устройством обработки данных
 4. Результирующие сигналы на входах устройства обработки данных и входные сигналы возбуждения и управляющем устройстве
- valid – установившиеся значения

КОНВЕЙЕРНЫЕ ВЫХОДЫ ТИПА МИЛИ

В разделе 7.3.2 говорилось, что у некоторых конечных автоматов бывают конвейерные выходы типа Мили. Если бы временные диаграммы, приведенные на рис. 8.80, относились к случаю конвейерных выходов типа Мили, то значения сигналов на этих выходах устанавливались бы раньше в пределах цикла в тот же самый момент, что и значения сигналов на выходах, отображающих состояние управляющего конечного автомата. Возникновение установившихся значений сигналов на этих выходах раньше, нежели в случае выходов типа Мура, когда сигналы должны пройти через комбинационную логику с соответствующей задержкой, позволяет, в принципе, всей системе в целом работать с тактовым сигналом большей частоты.

8.7.2. Пример построения синхронной системы

В этом разделе приводится типичный для практики пример, позволяющий рассмотреть ряд вопросов, которые возникают при проектировании синхронной системы.

Этим примером будет *схема умножения посредством сдвига и сложения* (*shift-and-add multiplier*), действующая согласно алгоритму, описанному в параграфе 2.8, когда операндами являются целые числа без знака. В устройстве обработки данных такой системы используются стандартные комбинационные и последовательностные элементы, а ее управляющее устройство описывается диаграммой состояний.

На рис. 8.81 указаны регистры и функции устройства обработки данных, необходимые для выполнения 8-разрядного умножения:

- MPY/LPROD** Регистр сдвига, в котором первоначально хранится множитель, а затем, по мере выполнения алгоритма, накапливаются младшие разряды произведения.
- HPROD** Первоначально очищенный регистр, в котором, по мере выполнения алгоритма, накапливаются старшие разряды произведения.
- MCND** Регистр, в котором на протяжении всей процедуры хранится множимое.
- F** Комбинационная функция, значение которой равно 9-разрядной сумме HPROD и MCND, если младший разряд в MPY/LPROD равен 1, и – HPROD (растянутому до 9 битов) в противном случае.

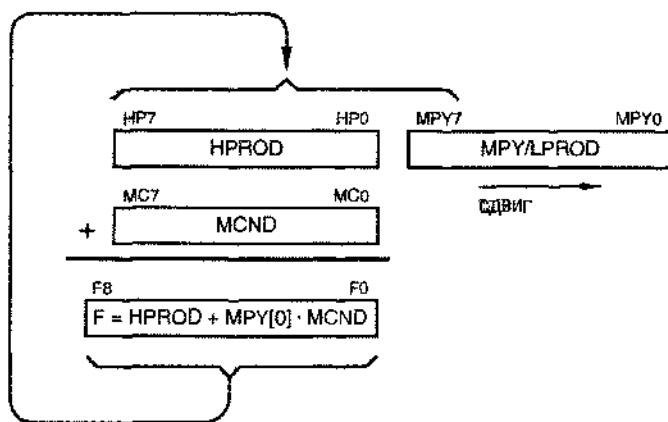


Рис. 8.81. Регистры и функции, используемые в алгоритме умножения посредством сдвига и сложения

С помощью регистра сдвига MPY/LPROD достигаются две цели: в процессе выполнения алгоритма в нем хранятся еще не принятые во внимание биты множителя (справа) и неизменяемые биты произведения (слева). На каждом шаге содержимое этого регистра сдвигается вправо на один разряд с потерей того бита множителя, который только что был принят во внимание, перемещением очередного бита множителя, которому предстоит быть принятым во внимание, в крайнее правое положение и загрузкой в крайнюю левую позицию еще одного

бита произведения, который уже останется неизменным при выполнении оставшейся части алгоритма.

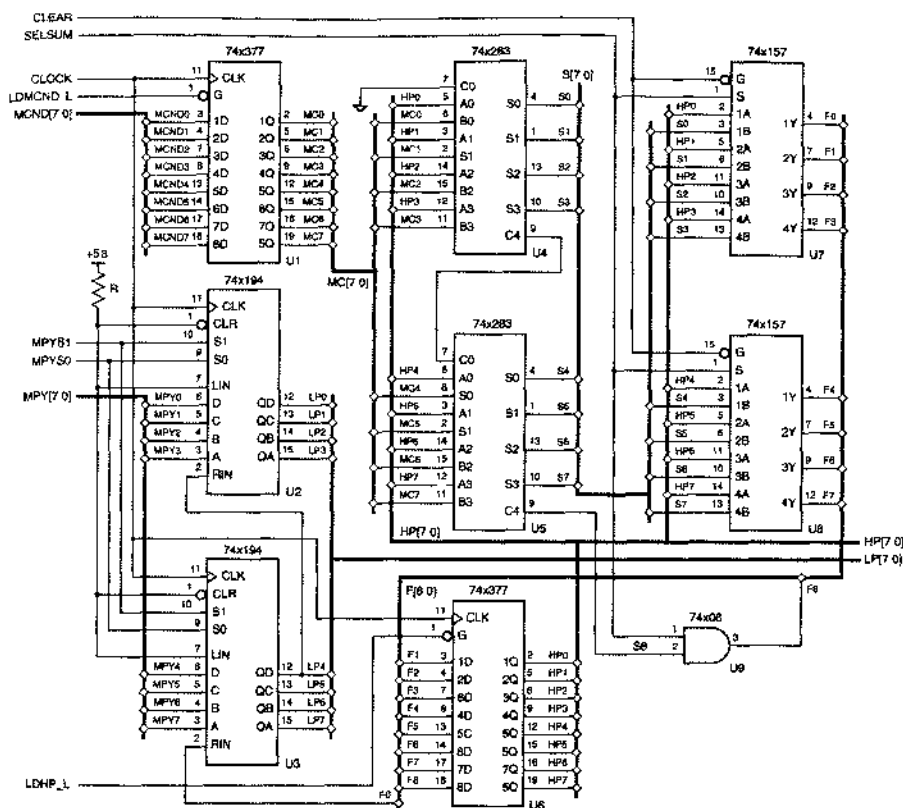


Рис. 8.82. Устройство обработки данных в 8-разрядном двоичном множителе, действующем по правилу сдвига и сложения

На рис. 8.82 приведена схема устройства обработки данных на ИС средней степени интеграции. Перед началом процедуры умножения в два регистра загружаются множитель МРУ [7:0] и множимое МСНД [7:0]. Когда процедура умножения завершается, произведение оказывается выведенным на шины НР[7:0] и ЛР[7:0]. Устройство обработки данных управляют следующие сигналы:

LDMCND_L
LDHP_L
MPYS[1:0]

Этим сигналом разрешается загрузка множимого в регистр U1. Этим сигналом разрешается загрузка НПРОД в регистр U6. Если значения этой пары сигналов равны 11, то разрешается загрузка на следующем такте МРУ/ЛПРОД в регистр, состоящий из ИС U2 и U3. На время выполнения процедуры умножения этим сигналам присваиваются значения 01, и этим разрешается регистру осуществлять сдвиг вправо; в другие моменты времени эти сигналы имеют значения 00, что предохраняет содержимое регистра от изменений.

- SELSUM** Когда этот сигнал имеет активный уровень, через мультиплексоры U7 и U8 проходят выходные сигналы сумматоров U4 и U5, представляющие собой сумму HPROD и множителя MC. Когда сигнал SELSUM имеет неактивный уровень, на выходы мультиплексоров проходит HPROD.
- CLEAR** При подаче этого сигнала на выходах мультиплексоров U7 и U8 устанавливаются нули.

Управляющее устройство умножителя показано на рис. 8.83; там же приведено устройство обработки данных. Управляющее устройство запускает устройство обработки данных и обеспечивает выполнение им процедуры умножения по шагам. Управляющее устройство разбито на счетчик (U10) и конечный автомат с диаграммой состояний, представленной на рис. 8.84.

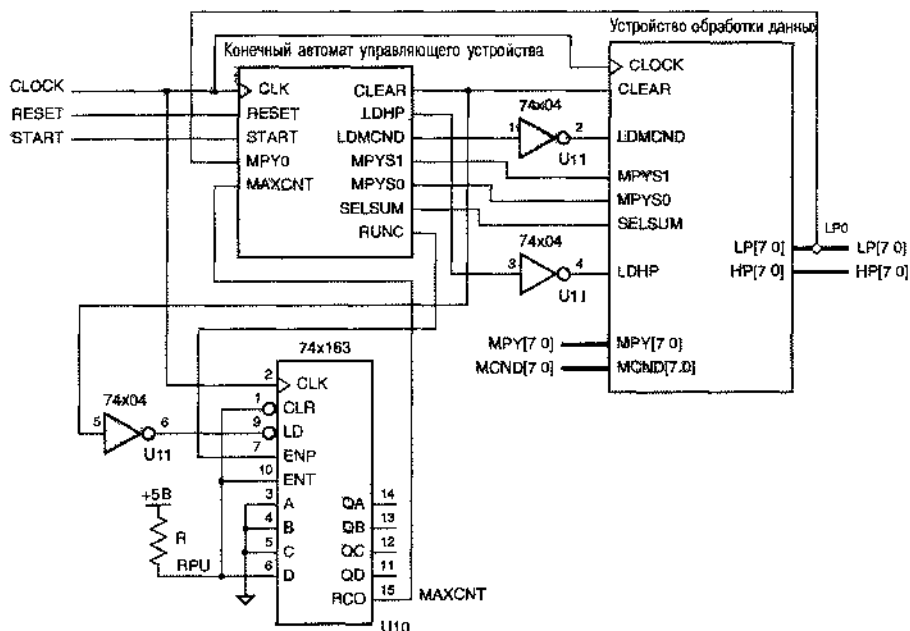


Рис. 8.83. Управляющее устройство для 8-разрядного двоичного умножителя, действующего по правилу сдвига и сложения

На диаграмме состояний указаны следующие входные и выходные сигналы:

- RESET** Входной сигнал сброса, возникающий при включении питания.
- START** Поступающий извне командный входной сигнал, по которому начинается процедура умножения.
- MPY0** Входной сигнал условия, поступающий от устройства обработки данных и представляющий собой очередной бит множителя, который следует принять во внимание.
- CLEAR** Выходной управляющий сигнал, по которому выходы мультиплексора обнуляются и запускается счетчик.

- LDMCND** Выходной управляющий сигнал, разрешающий загрузку регистра MCND.
- LDHP** Выходной управляющий сигнал, разрешающий загрузку регистра HPROD.
- RUNC** Выходной управляющий сигнал, по которому счет в счетчике разрешен.
- MPYS[1:0]** Выходные сигналы, управляющие сдвигом и загрузкой в регистре MPY/LPROD.
- SELSUM** Выходной управляющий сигнал, по которому происходит выбор между сдвинутым содержимым сумматора и сдвинутым содержимым регистра HPROD для загрузки обратно в регистр HPROD.

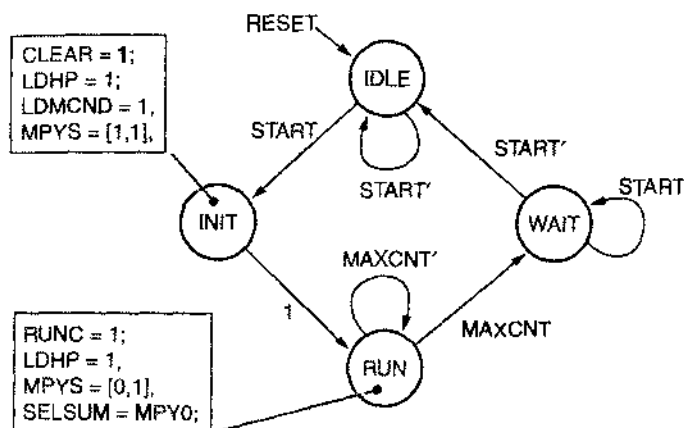


Рис. 8.84. Диаграмма состояний управляющего конечного автомата для двоичного умножителя, действующего по правилу сдвига и сложения

Диаграмму состояний можно преобразовать в соответствующий конечный автомат многими способами, начиная от рутинной процедуры (выполняемой вручную) и кончая применением автоматизированных средств синтеза, предоставив им необходимые описания на языках ABEL или VHDL. У этого конечного автомата почти все выходы являются выходами типа Мура; только выход SELSUM является выходом типа Мили. В рамках на диаграмме состояний перечислены выходные сигналы, имеющие активные значения в состояниях INIT и RUN; в остальное время эти сигналы имеют неактивный уровень. Автомат спроектирован так, чтобы при подаче сигнала RESET он из любого состояния переходил в состояние IDLE.

Процедура умножения начинается с состояния INIT, когда приходит сигнал START. В этом состоянии в счетчик заносится число 1000_2 , множитель и множимое загружаются в соответствующие регистры, а регистр HPROD очищается. Затем происходит переход в состояние RUN и счетчик начинает считать. В течение восьми тактов конечный автомат остается в состоянии RUN и за это время выполняется восемь шагов 8-разрядного алгоритма сдвига и сложения. На восьмом такте счетчик оказывается в состоянии 1111_2 , при этом возникает сиг-

нал MAXCNT и конечный автомат переходит в состояние WAIT. Автомат пребывает в этом состоянии до тех пор, пока не будет снят сигнал START, чтобы предотвратить возобновление процедуры умножения раньше, нежели сигнал START вновь примет активное значение.

Представляют интерес многие детали проектирования устройства обработки данных и управляющего устройства, но самое важное, что следует увидеть в данном примере, это тот факт, что все последовательностные элементы в схемах обоих устройств являются триггерами, переключающимися по фронту одного и того же общего тактового сигнала CLOCK. Таким образом, для рассматриваемой системы справедливы временные диаграммы, приведенные на рис. 8.80, и разработчику можно не беспокоиться по поводу гонок, источников опасности и асинхронных операций. Если только реализация конечного автомата не будет очень медленной, то максимальная частота тактового сигнала для системы в целом будет ограничена в основном задержками прохождения сигнала через устройство обработки данных.

8.8. Трудности синхронного проектирования

Хотя синхронный подход является наиболее прямым и самым надежным способом проектирования цифровых систем, ряд неприятных практических обстоятельств может затруднить продвижение по этому пути. В данном параграфе мы рассмотрим эти обстоятельства.

8.8.1. Разброс задержек тактового сигнала

Синхронная система на переключающихся по фронту триггерах работает правильно только в том случае, когда переключающий фронт тактового сигнала постукает на все триггеры в один и тот же момент времени. На рис. 8.85 показано, что может произойти в противном случае. В этом примере два триггера теоретически управляются одним и тем же тактовым сигналом, но триггер FF2 «видит» тактовый сигнал, который заметно задержан по отношению к тактовому сигналу, который «видит» триггер FF1. Это различие между моментами прихода тактового сигнала на различные устройства носит название *разброса задержек тактового сигнала (clock skew)*.

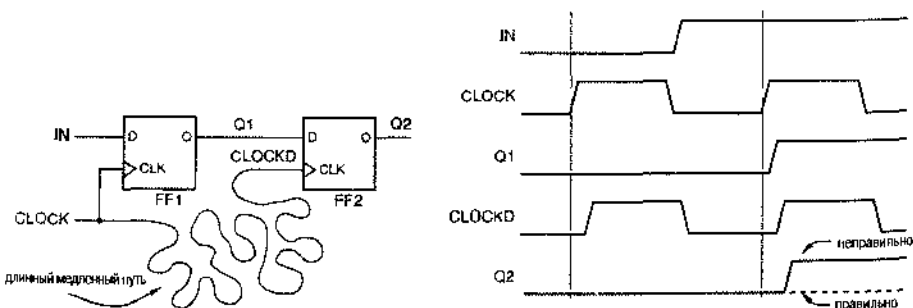


Рис. 8.85. Пример разброса задержек тактового сигнала

Мы назвали задержанный сигнал в схеме на рис. 8.85(а) “CLOCKD”. Если задержка распространения в триггере FF1 от входа CLOCK до выхода Q1 мала и

если физическое соединение выхода Q1 с триггером FF2 является коротким, то изменение значения Q1 по фронту сигнала CLOCK может и в самом деле достичь триггера FF2 *до того*, как на этот триггер поступит фронт сигнала CLOCKD. В этом случае триггер FF2 может перейти в неправильное следующее состояние, определяемое *следующим* состоянием триггера FF1, а не текущим его состоянием, как показано на рис. (b). Если изменение значения Q1 приходит на триггер FF2 лишь немного раньше относительно сигнала CLOCKD, то может оказаться нарушенным требование, касающееся времени удержания триггера FF2, и в этом случае триггер FF2 может стать метастабильным и выработать непредсказуемое значение сигнала на своем выходе.

Если рис. 8.85 напоминает вам существенный источник опасности, изображенный на рис. 7.10I, то вы не далеки от истины. Проблему разброса задержек тактового сигнала можно считать просто проявлением наличия существенных источников опасности в переключающихся по фронту устройствах.

Наличие или отсутствие проблемы, связанной с разбросом задержек тактового сигнала в данной системе можно описать количественно, определив t_{skew} как величину расхождения задержек тактового сигнала и воспользовавшись другими временными параметрами, указанными на рис. 8.1. Для правильной работы необходимо, чтобы выполнялось неравенство:

$$t_{\text{fipd(min)}} + t_{\text{comb(max)}} - t_{\text{hold}} - t_{\text{skew(max)}} > 0.$$

Другими словами, разброс задержек тактового сигнала вычитается из запаса по времени удержания, введенного в разделе 8.1.4.

Приведенный на рис. 8.85 пример, рассматриваемый сам по себе, может показаться несколько преувеличенным. В конце концов, почему бы это конструктор стал создавать короткое соединение для данных и длинное соединение для тактового сигнала, когда пути этих сигналов вполне могли бы проходить рядом? Существует несколько ситуаций, в которых это все же может случиться; одни из них являются результатом ошибок, тогда как другие неизбежны.

В большой системе коэффициент разветвления по выходу у источника тактового сигнала может быть недостаточным для подачи одного и того же сигнала на тактовые входы всех устройств; поэтому могут понадобиться две или большее число копий тактового сигнала. Метод буферизации, указанный на рис. 8.86(a), очевидно, вносит дополнительный разброс задержек тактового сигнала, поскольку сигналы CLOCK1 и CLOCK2 оказываются задержанными по отношению к сигналу CLOCK на время прохождения через буферы.

Рекомендуемый способ размножения сигналов приведен на рис. 8.86(b). Все тактовые сигналы проходят через идентичные буферы с примерно равными задержками. В идеальном случае все буферы должны быть элементами, находящимися в одной и той же интегральной схеме; тогда все они обладают близкими временными характеристиками, а также работают при одной и той же температуре и одном и том же напряжении питания. Некоторые производители выпускают специальные буферы, предназначенные как раз для таких приложений, и указывают разброс задержек отдельных буферов в одном корпусе ИС, который, в худшем случае, может составлять всего несколько десятых долей наносекунды.

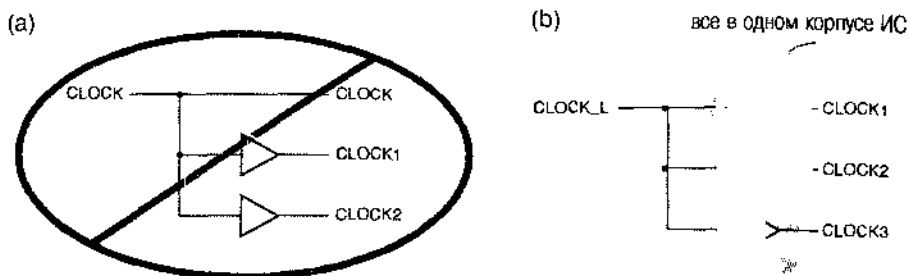


Рис. 8.86. Буферизация тактового сигнала. (а) внесение дополнительного разброса задержек; (б) находящийся под контролем разброс задержек

Однако способ буферизации, показанный на рис. 8.86, также может вносить дополнительный разброс задержек тактового сигнала, если нагрузка для одного из сигналов оказывается значительно большей, чем для других: переходы в тактовом сигнале с большей нагрузкой будут происходить позднее из-за увеличения задержки переключения выходных транзисторов, а также из-за того, что становятся большими время нарастания и время спада сигнала. Поэтому внимательный разработчик старается сбалансировать нагрузку различных тактовых сигналов, принимая во внимание как нагрузку по постоянному току (коэффициент разветвления по выходу), так и нагрузку по переменному току (емкость проводников и входные емкости).

При автоматизированной разводке соединений на печатных платах и внутри специализированных ИС с помощью соответствующих программных средств может возникнуть другая неприятная ситуация. На рис. 8.87 показана печатная плата или специализированная ИС с большим числом триггеров и более сложных элементов, которые переключаются общим тактовым сигналом CLOCK. Программные средства автоматизированной системы проектирования развели сигнал CLOCK таким образом, что он вьется серпантином между тактируемыми устройствами. Для других же сигналов – в каждом случае от выхода к небольшому числу входов – проложены более короткие пути. Еще хуже обстоит дело, когда внутри специализированной ИС распространение сигнала «по проводам» происходит медленнее, чем посредством соединений, выполненных по другой технологии (поликристаллические кремниевые соединения в КМОП-структуре). В результате фронт тактового сигнала CLOCK может поступать на триггер FF2 чуть позднее момента изменения данных на линии Q1.

Один из способов свести к минимуму проблему такого рода состоит в том, чтобы организовать распределение тактового сигнала CLOCK по древовидной структуре, как показано на рис. 8.88, с помощью самых быстрых соединений. Обычно такое «дерево для тактового сигнала» создается вручную или путем применения специализированных систем CAD. Но в большом проекте может случиться так, что нельзя гарантировать повсеместно поступление фронта тактового сигнала до того, как произойдет самое раннее изменение данных. Для обнаружения этого, как правило, применяются программы рисующие временные диаграммы; решением проблемы в общем случае может оказаться лишь включение дополнительной задержки (например, вставку пары инверторов) на том пути тактового сигнала, по которому он проходит слишком быстро.

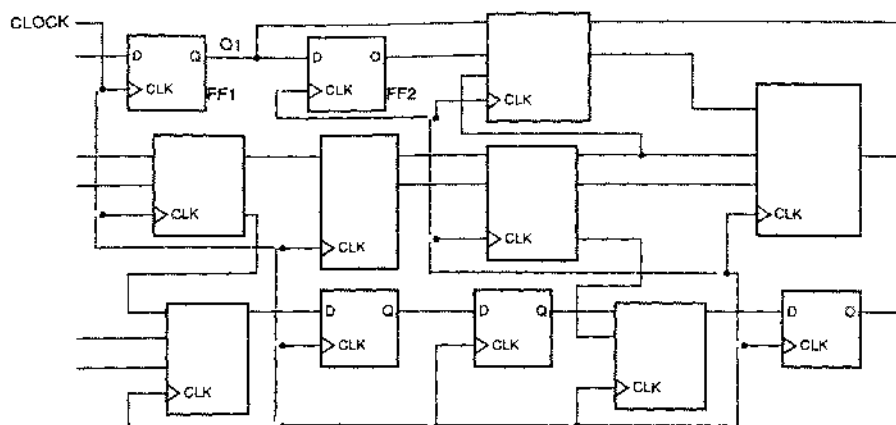


Рис. 8.87. Разводка тактового сигнала, которая может вызвать дополнительный разброс задержек на сложных печатных платах и внутри специализированных ИС

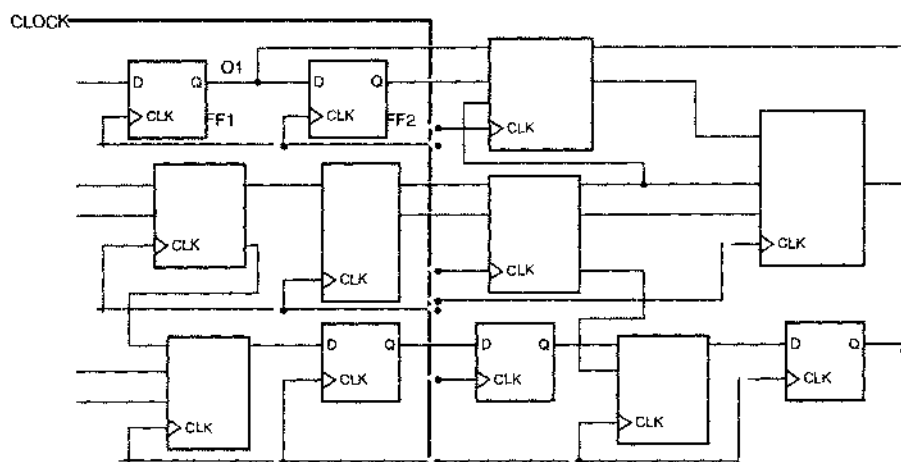


Рис. 8.88. Разводка тактового сигнала, минимизирующая разброс задержек

Хотя методология синхронного проектирования позволяет упростить алгоритм работы большой системы, мы видим все же, что в случае, когда в качестве запоминающих элементов используются нереклюющиеся по фронту триггеры, главной проблемой может оказаться разброс задержек тактового сигнала. Чтобы преодолеть это затруднение, во многих высокоскоростных системах и в СБИС применяется проектирование по принципу двухфазных защелок (*two-phase latch design*), с которым можно ознакомиться по литературе. При таком проектировании каждый нереклюющийся по фронту D-триггер разбивается на две составляющие его защелки, которыми управляют тактовые сигналы с неперекрывающимися рабочими фазами. Зазор между рабочими фазами поглощает разброс задержек тактового сигнала.

КАК ИЗБЕЖАТЬ ПОСЛЕДСТВИЙ РАЗБРОСА ЗАДЕРЖЕК

Различие в длине проводников и разная нагрузка являются очевидными причинами разброса задержек тактового сигнала. Но существует много других, менее явных источников расхождения. В частности, разброс задержек тактового сигнала может быть вызван *перекрестными помехами*, то есть наводками с одной сигнальной линии на другую. Перекрестные помехи неизбежны, если проводники проходят на печатной плате или внутри микросхемы параллельно на малом расстоянии друг от друга; при этом наводки возникают в моменты переходов сигналов с одного уровня на другой. Сигнал, соседний с тактовым, может изменяться в том же направлении или в противоположном; в зависимости от этого переход в тактовом сигнале может ускоряться или замедляться, в результате чего будет казаться, что переход происходит раньше или позже.

В большом устройстве на печатной плате или внутри специализированной ИС, как правило, нереально отследить и устранить все возможные источники разброса задержек тактового сигнала. Поэтому большинство производителей специализированных ИС требуют от проектировщиков, чтобы те предусматривали дополнительный запас по времени установления и времени удержания, эквивалентный задержке нескольких вентилей, в дополнение к тому минимуму, который следует из временных диаграмм, полученных путем моделирования. Только в этом случае все неизвестные факторы оказываются преодоленными.

8.8.2. Стробирование тактового сигнала

У большинства последовательностных ИС средней степени интеграции, упомянутых в этой главе, есть синхронный вход разрешения на выполнение данной ИС ее функций. Другими словами, сигналы на входах разрешения у таких ИС фиксируются в тот же момент, что и данные, на фронте тактового сигнала. Первым из рассмотренных нами примеров был регистр 74х377 с синхронным входом разрешения загрузки; другие подобные ИС – это счетчик 74х163 и регистр сдвига 74х194 с синхронными входами разрешения загрузки, счета и сдвига. Тем не менее, у многих ИС средней степени интеграции, а также у макроэлементов для ИС типа FPGA и у ячеек в специализированных ИС нет синхронных входов разрешения соответствующих действий; например, 8-разрядный регистр 74х374 имеет выходы с тремя состояниями, но у него нет входа разрешения загрузки.

Спрашивается: что может сделать разработчик, если требуется, чтобы у 8-разрядного регистра были и вход разрешения загрузки, и выходы с тремя состояниями? Одно из решений этой задачи состоит в использовании ИС 74х377 с входом разрешения загрузки и включении вслед за этой ИС буфера 74х241 с выходами с тремя состояниями. Однако при этом возрастут стоимость и задержка. Другой вариант заключается в применении большего по размерам и более дорогого изделия, а именно – ИС 74х823, которая обладает обоими требуемыми свойствами и у которой, кроме того, есть асинхронный вход сброса CLR_L. Но если разработчик не знает ничего лучше, кроме ИС '374, то более рискованным

решением является запираание сигнала на тактовом входе этой ИС на интервале времени, в течение которого не предполагается производить загрузку. Такое действие называют *стробированием тактового сигнала* (*gating the clock*).

На рис. 8.89 демонстрируется очевидный, но ошибочный способ стробирования тактового сигнала. Сигнал CLKEN подается для того, чтобы разрешить прохождение тактового сигнала CLOCK через вентиль И и выработать стробированный тактовый сигнал GCLK. В случае применения такого способа возникают две проблемы:

1. Если сигнал CLKEN является выходным сигналом конечного автомата или другим сигналом, который вырабатывается каким-либо регистром, то изменение сигнала CLKEN происходит чуть *позднее* того момента, когда сигнал CLOCK переходит на высокий уровень. Как показано на рис. 8.89(b), при этом в сигнале GCLK возникают паразитные импульсы, которые будут приводить к ошибочным переключениям регистров, на входы которых поступает этот сигнал.
2. Но даже если сигнал CLKEN каким-то образом вырабатывается раньше нарастающего фронта сигнала CLOCK (например, на выходе регистра, переключающегося по *спадающему* фронту сигнала CLOCK, что является особенно нежелательным), задержка в вентиле И приведет к увеличению разброса задержек тактового сигнала в системе, в результате чего могут возникнуть определенные проблемы в других местах.

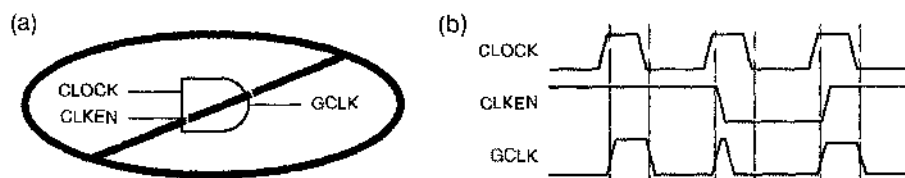


Рис. 8.89. Как не надо стробировать тактовый сигнал: (а) схема, решающая задачу «в лоб»; (б) временные диаграммы

Способ стробирования тактового сигнала, обеспечивающий минимальный разброс задержек, представлен на рис. 8.90. В приведенной схеме нестробированный тактовый сигнал и несколько стробированных тактовых сигналов вырабатываются от одного и того же главного тактового сигнала с низким активным уровнем. Чтобы минимизировать возможные различия в задержках, следует использовать вентили, находящиеся в одном и том же корпусе ИС. Сигнал CLKEN может меняться произвольно, пока сигнал CLOCK_L остается на низком уровне, а сигнал CLOCK имеет высокий уровень. Вот и прекрасно: сигнал CLKEN вырабатывается, как правило, конечным автоматом, у которого изменение сигналов на его выходах происходит строго после того, как сигнал CLOCK переходит на высокий уровень.

Подход, иллюстрируемый рис. 8.90, приемлем только в том случае, когда возникающий при этом разброс задержек является допустимым. Кроме того, заметьте, что сигнал CLKEN должен оставаться неизменным в течение всего интервала времени, на котором сигнал CLOCK_L имеет высокий уровень (а сигнал

CLOCK – низкий). Таким образом, запас по времени будет зависеть от коэффициента заполнения тактового сигнала особенно тогда, когда сигнал CLKEN оказывается значительно сдвинутым по отношению к переключающему фронту тактового сигнала из-за большой задержки в комбинационной логике (t_{comb}). В настоящей синхронной системе входной сигнал разрешения выполнения той или иной функции может изменяться почти в любой момент времени в пределах всего периода тактового сигнала вплоть до границы времени установления перед переключающим фронтом тактового сигнала; такого рода решение применительно к разрешению загрузки ИС 74х377 приведено на рис. 8.13.

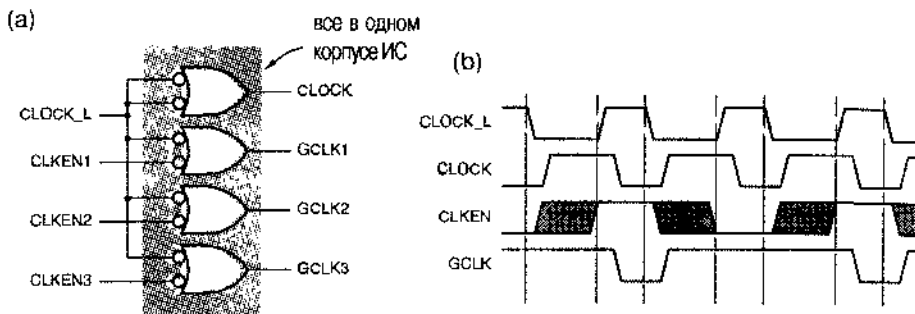


Рис. 8.90. Приемлемый способ стробирования тактового сигнала: (а) схема; (б) временные диаграммы

8.8.3. Асинхронные входы

Хотя теоретически можно построить полностью синхронный компьютер, вы вряд ли сможете им воспользоваться, если только не окажетесь способным нажимать на клавиши синхронно с 500-мегагерцным тактовым сигналом. Цифровой системе любого типа всегда приходится иметь дело с *асинхронными входными сигналами* (*asynchronous input signals*), которые не привязаны к тактовому сигналу системы.

Асинхронные входные сигналы часто бывают запросами на обслуживание (например, прерывания в компьютере) или признаками возникновения тех или иных условий (например, сигнал о том, что какой-то ресурс стал доступен). Обычно такие сигналы изменяются медленно по сравнению с тактовой частотой системы, и нет необходимости распознавать их на определенном такте. Если переход в таком сигнале будет пропущен на данном такте, его всегда можно обнаружить на следующем. Скорость переключений асинхронных сигналов может простирается от одного изменения в секунду (когда за клавиатурой сидит медлительный человек) до 100 МГц и более (запросы на доступ к совместно используемой памяти в 500-мегагерцной многопроцессорной системе).

Если не принимать во внимание проблему метастабильности, то нетрудно построить *синхронизирующее устройство* (*synchronizer*), то есть схему, в которой будет браться выборка асинхронного входного сигнала и вырабатываться выходной сигнал, удовлетворяющий требованиям синхронной системы по времени установления и времени удержания. Как видно из рис. 8.91, D-триггер берет выборку асинхронного входного сигнала на каждом такте системного тактового

сигнала и вырабатывает синхронный выходной сигнал, значение которого удерживается в течение следующего периода тактового сигнала.

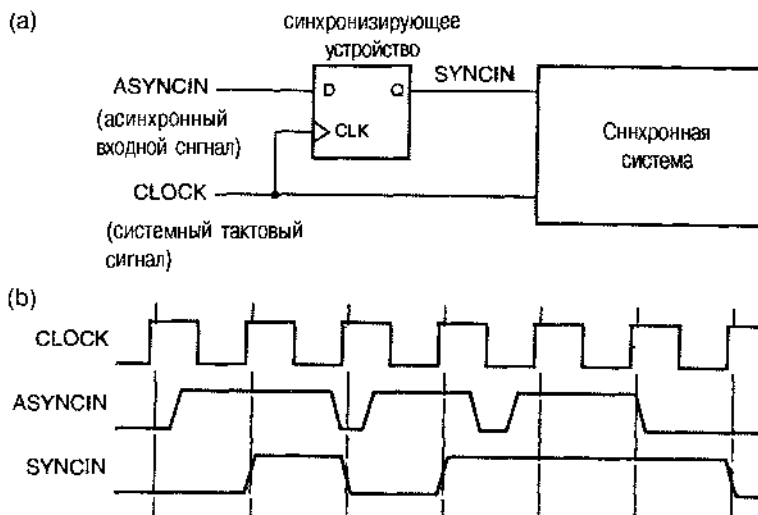


Рис. 8.91. Простое синхронизирующее устройство на одном D-триггере: (a) схема; (b) временные диаграммы

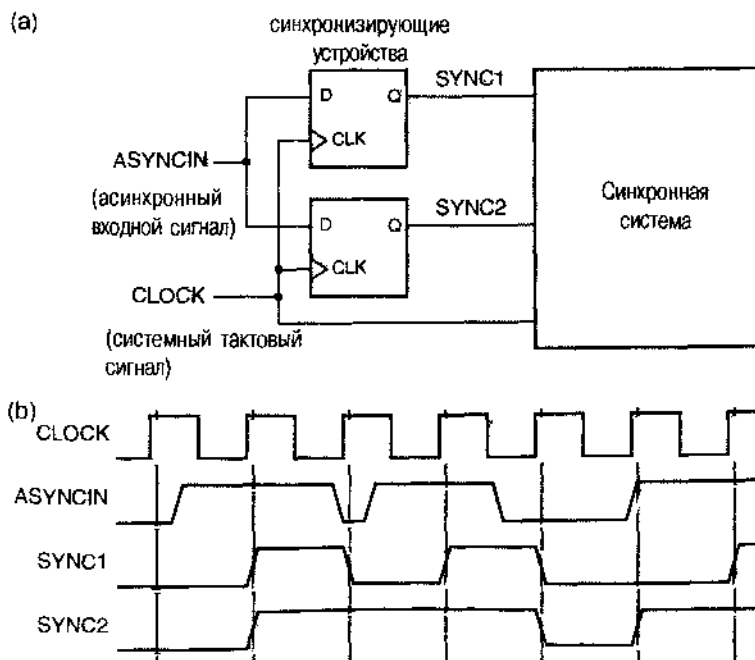


Рис. 8.92. Два синхронизирующих устройства для одного и того же асинхронного входного сигнала: (a) схема; (b) возможные временные диаграммы

Для асинхронных входных сигналов существенно, чтобы их принуждение к такому сигналу производилось в системе только *в одном месте*; на рис. 8.92 показано, что может произойти в противном случае. Из-за физических задержек на схеме два триггера не видят тактовый сигнал и входные сигналы точно в один и тот же момент времени. Поэтому в случае, когда асинхронный сигнал изменяется вблизи фронта тактового сигнала, существует небольшое временное окно, в пределах которого один триггер может воспринять и зафиксировать входной сигнал как 1, а другой — как 0. Противоположные значения сигналов на выходах этих триггеров могут стать причиной неправильной работы системы, одна часть которой будет вести себя так, как если бы входной сигнал был равен 1, а отклик другой части будет таким, как если бы этот сигнал был равен 0.

Если сигнал проходит через комбинационную логику, то наличие двух синхронизирующих устройств может не проявиться (рис. 8.93). Из-за различных задержек прохождения сигнала по разным путям в комбинационной логике вероятность появления на выходах триггеров несовместимых результатов только возрастает. Это распространенный случай, особенно тогда, когда асинхронные сигналы являются входными сигналами конечного автомата, поскольку две или более переменные состояния, вырабатываемые логикой возбуждения, могут зависеть от значения асинхронного входного сигнала. Правильный способ подачи асинхронного сигнала на вход конечного автомата показан на рис. 8.94. Логика возбуждения видит только один синхронизированный входной сигнал SYNCIN.

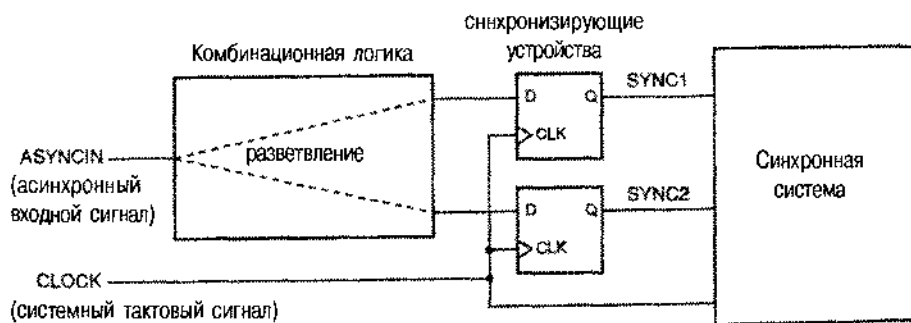


Рис. 8.93. Асинхронный входной сигнал, поступающий на входы двух синхронизирующих устройств после прохождения через комбинационную логику

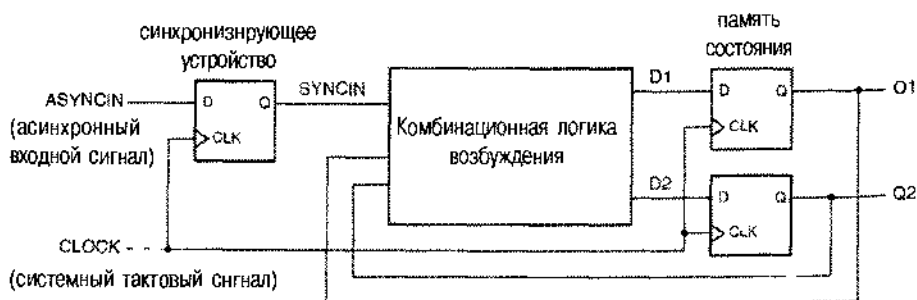


Рис. 8.94. Асинхронный сигнал на входе конечного автомата, пропущенный через единственное синхронизирующее устройство

СТОИТ ЛИ ВОЛНОВАТЬСЯ?

Вы, возможно, догадываетесь, что синхронизирующие устройства, изображенные на рис. 8.91 и 8.94, могут сработать неправильно. Это может произойти из-за нарушения требований, предъявляемых временем установления и временем удержания синхронизирующих триггеров, поскольку асинхронный входной сигнал может измениться в любой момент времени. «Стоит ли волноваться?», скажете вы. «Если сигнал на входе D изменяется вблизи фронта тактового сигнала, то триггер либо увидит это изменение на данном такте, либо пропустит его сейчас и обнаружит на следующем такте. В любом случае это меня вполне устраивает!» Проблема все же есть, и она связана с существованием третьей возможности, рассматриваемой в параграфе 8.9.

8.9. Сбой в работе синхронизирующего устройства и метастабильность

В параграфе 7.1 было показано, что в случае, когда требования в отношении времени установления и времени удержания триггера не удовлетворены, триггер может войти в третье, *метастабильное* состояние посередине между 0 и 1. Хуже всего то, что время пребывания в этом состоянии, то есть время до того момента, когда триггер «свалится» в одно из его законных состояний — в состояние 0 или в состояние 1, — теоретически неограниченно. Некоторые из вентилей и триггеров, на входы которых поступает метастабильный сигнал, могут интерпретировать его как 0, тогда как другие вентили и триггеры будут воспринимать его как 1, в результате чего возникнет того или иного рода несовместимость типа той, какая была указана на рис. 8.92. Впрочем, вентили и другие триггеры с метастабильным сигналом на входе, сами могут вырабатывать метастабильные сигналы на своих выходах (недз, в конце концов, эти схемы оказываются в линейной части их передаточной характеристики). К счастью, вероятность того, что сигнал на выходе триггера и дальше останется метастабильным, уменьшается со временем экспоненциально, хотя никогда и не становится равной нулю.

8.9.1. Сбой в работе синхронизирующего устройства

Говорят, что в работе *синхронизирующего устройства* произошел *сбой* (*synchronizer failure*), когда в системе используется выходной сигнал этого устройства, несмотря на то, что он остается метастабильным. Система может обезопасить себя от сбоев в синхронизирующем устройстве, если будет «достаточно долго» ждать, прежде чем воспользуется выходным сигналом этого устройства. Но что значит «достаточно долго»? Для этого необходимо, по крайней мере, чтобы среднее время между сбоями в синхронизирующем устройстве было на несколько порядков больше, чем планируемое разработчиком время использования системой выходного сигнала этого устройства.

Метастабильность — это нечто большее, чем академическая проблема. Многим конструкторам донелось стать создателями высокоскоростных цифровых си-

стем, которые страдали тем, что время от времени происходили сбои в их синхронизирующих устройствах (и которые, тем не менее, были доведены до серийного производства). Говорят, что связанные с метастабильностью проблемы первоначально были у целого ряда популярных ИС, в частности, у таких микросхем, как системный времязадающий контроллер AMD 9513, контроллер прерываний AMD 9519, последовательный интерфейс ввода/вывода Z-80 фирмы Zilog, однокристальный микрокомпьютер 8048 фирмы Intel и RISC-процессор AMD 29000. Вы, наверно, задаетесь вопросом: «И что, этих разработчиков еще не уволили?».

Существует два способа избавиться от пребывания триггера в метастабильном состоянии:

1. Принудительно перевести его в одно из его законных состояний с помощью сигналов, удовлетворяющих объявленным требованиям в отношении минимальной длительности импульса, времени установления и т.д.
2. Подождать «достаточно долго», пока триггер сам собой не выйдет из состояния метастабильности.

Неопытные проектировщики часто пытаются обойти метастабильность другим путем и, как правило, терпят неудачу. Одна из таких попыток представлена на рис. 8.95: коль скоро метастабильность является «аналоговой» проблемой, ее решение также должно быть «аналоговым», — так думает разработчик. Действительно, цепочки с триггерами Шмитта на входах и с конденсаторами могут быть использованы в обычных условиях для очистки сигналов от шума. Однако вместо исключения метастабильности, такая схема только усилит этот эффект: построенная из вполне «достойных» элементов, эта схема навсегда войдет в режим колебаний, как только одновременно будут переведены на неактивный уровень сигналы S_L и R_L. (Автор должен признаться, что больше 20 лет назад попытался это сделать!) В задачах 8.97 и 8.94 приведены примеры отважных, но неудачных попыток исключить метастабильность. Эти примеры позволяют вам почувствовать, что проблемы, возникающие в связи с синхронизирующими устройствами, могут быть очень тонкими, так что необходимо быть бдительным. Единственный способ сделать синхронизирующее устройство надежным состоит в том, чтобы ждать достаточно долго, пока выходной сигнал не перестанет быть метастабильным. На вопрос: «Как долго надо ждать, чтобы этого было достаточно?» мы ответим в этом параграфе позже.

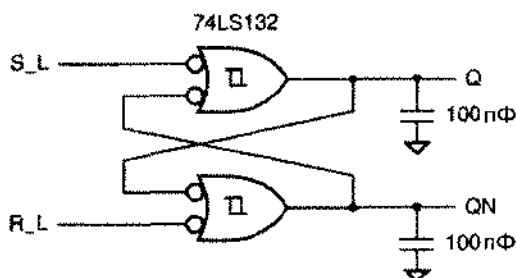


Рис. 8.95. Неудачная попытка построить $\overline{SR}$ -триггер, защищенный от метастабильности

8.9.2. Время выхода из метастабильности

Если требования D-триггера по времени установления и времени удержания удовлетворены, то триггер устанавливается в новое состояние в пределах интервала времени t_{pd} после того как прошел фронт тактового сигнала. Если эти требования нарушены, то выход триггера может быть метастабильным сколь угодно долго. Проектируя некоторую систему, мы пользуемся параметром t_r , носящим название *времени выхода из метастабильности (metastability resolution time)*, для обозначения максимального времени, в течение которого выходной сигнал может оставаться метастабильным без ущерба для работы синхронизирующего устройства (и системы).

Рассмотрим, например, конечный автомат, изображенный на рис. 8.94. В этом случае мы располагаем следующим временем выхода из метастабильности:

$$t_r = t_{clk} - t_{comb} - t_{setup},$$

где t_{clk} — период тактового сигнала, t_{comb} — задержка распространения сигнала по комбинационной логике и t_{setup} — время установления триггеров, используемых в памяти состояния.

8.9.3. Разработка надежного синхронизирующего устройства

Самое надежное синхронизирующее устройство — это такое устройство, которое успевает за отведенное время выйти из метастабильности. Но при проектировании цифровой системы мы редко можем позволить себе роскошь *понижить* тактовую частоту ради надежности. Обычно, напротив, от нас требуют *повысить* тактовую частоту, чтобы система обладала лучшими характеристиками. Поэтому чаще всего нам нужно, чтобы синхронизирующее устройство работало надежно при очень малых значениях периода тактового сигнала. Мы представим несколько таких схем и покажем, как можно оценить их надежность.

Как сказано выше, у конечного автомата с асинхронным входом, структура которого показана на рис. 8.94, время выхода из метастабильности равно $t_r = t_{clk} - t_{comb} - t_{setup}$. Чтобы сделать t_r возможно большим при заданном периоде тактового сигнала, нам следует минимизировать t_{comb} и t_{setup} . Значение t_{setup} определяется типом триггеров, используемых в памяти состояния; в общем случае, у более быстрого триггера время установления меньше. Минимальное значение t_{comb} равно 0 и достигается в синхронизирующем устройстве, приведенном на рис. 8.96; сейчас мы объясним, как работает эта схема.

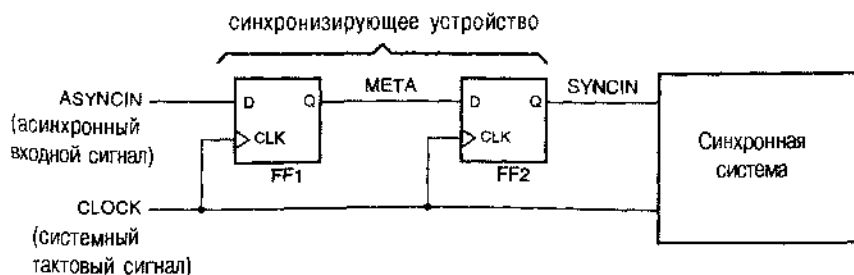


Рис. 8.98. Рекомендуемая схема синхронизирующего устройства

Сигналы на входе триггера FF1 асинхронны по отношению к тактовому сигналу и могут наступать с нарушением требований, предъявляемых временем установления и временем удержания. Если это происходит, то выходной сигнал МЕТА становится метастабильным и остается в этом состоянии произвольно долго. Предположим, однако, что максимальное время метастабильности, после того как прошел фронт тактового сигнала, равно t_1 . (В следующем разделе мы покажем, как найти вероятность того, что наше предположение верно.) Если скоро период тактового сигнала больше, чем t_1 плюс время установления триггера FF2, сигнал SYNCIN становится синхронной копией асинхронного входного сигнала на следующем такте тактового сигнала, никогда не оказываясь метастабильным. Далее сигнал SYNCIN можно использовать повсеместно в системе по мере необходимости.

8.9.4. Анализ времени пребывания в состоянии метастабильности

На рис. 8.97 приведены временные параметры, учитываемые при анализе времени пребывания в метастабильном состоянии. Обозначим указываемые производителем время установления и время удержания по обе стороны фронта в тактовом сигнале через t_s и t_h ; эти два интервала времени образуют *окно принятия решения* (decision window): на этом отрезке триггер берет выборку сигнала на входе данных и решает, нужно ему изменять выходной сигнал или нет. Если сигнал на входе D изменится за пределами этого окна, как показано на рис. (а), то производитель гарантирует нереклюшение триггера и его переход в одно из его законных логических состояний не позднее времени t_{pd} . Если входной сигнал D изменится в пределах окна принятия решения, как показано на рис. (b), то метастабильность может возникнуть и просуществовать до конца интервала времени t_1 .

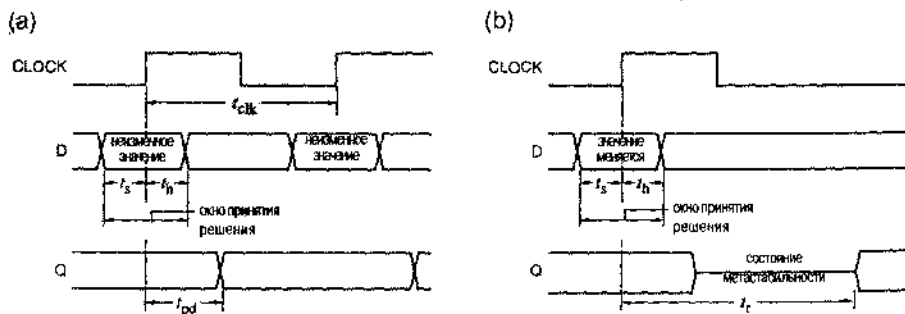


Рис. 8.97. Временные параметры, учитываемые при анализе метастабильности: (а) нормальная работа триггера; (b) метастабильное поведение

НЕБОЛЬШОЕ УТОЧНЕНИЕ

Рассматривая синхронизирующее устройство, изображенное на рис. 8.96, мы не допускали возможности возникновения метастабильности на выходе триггера FF2 даже на короткое время, поскольку предполагалось, что система спроектирована с нулевым запасом по времени. Но если система допускает небольшое увеличение задержки прохождения сигнала через триггер FF2, то значение MTBF, вычисляемое в разделе 8.9.4, будет темным лучше.

Теоретическое исследование показывает (а практический опыт подтверждает), что при изменении асинхронного входного сигнала в пределах окна принятия решения длительность пребывания выхода в метастабильном состоянии описывается экспоненциальной зависимостью:

$$\text{MTBF}(t_r) = \frac{\exp(t_r/\tau)}{T_0 \cdot f \cdot a}.$$

Здесь MTBF – *среднее время между отказами (Mean Time Between Failures)* синхронизирующего устройства, если считать, что отказ происходит в том случае, когда метастабильность выходит за пределы отрезка времени длительностью t_r после фронта тактового сигнала; $t_r \geq t_{\text{pd}}$. Значение MTBF зависит от частоты сигнала f на тактовом входе триггера; a – число изменений асинхронного входного сигнала, поступающего на вход данных триггера, в секунду; T_0 и τ – константы, зависящие от электрических характеристик триггера. В типичном случае – для ИС 74LS74 – $T_0 \approx 0.4$ с, $\tau \approx 1.5$ нс.

Предположим теперь, что создается микропроцессорная система с частотой тактового сигнала 10 МГц и со схемой, преобразующей асинхронный входной сигнал в синхронный, изображенной на рис. 8.96. Если сигнал ASYNCIN изменяется в пределах окна принятия решения триггера FF1, то выход META может оставаться в состоянии метастабильности в течение интервала времени t_r . Если сигнал META все еще остается метастабильным к началу окна принятия решения триггера FF2, то происходит сбой в работе синхронизирующего устройства, так как выход триггера FF2 может оказаться метастабильным; в этом случае поведение системы непредсказуемо.

Пусть D-триггерами в схеме на рис. 8.96 являются триггеры из ИС 74LS74. Время установления t_s для такого триггера равно 20 нс, тогда как период тактового сигнала в нашем примере микропроцессорной системы равен 100 нс; таким образом, допустимая продолжительность пребывания в метастабильном состоянии составляет 80 нс. Если асинхронный входной сигнал изменяется 100000 раз в секунду, то среднее время между сбоями синхронизирующего устройства равно

$$\text{MTBF}(80 \text{ нс}) = \frac{\exp(80/1.5)}{0.4 \cdot 10^7 \cdot 10^5} = 3.6 \cdot 10^{11} \text{ с}.$$

Это совсем неплохо: 100 столетий между сбоями! Правда, если бы нам удалось продать 10 000 таких систем, то одна из них давала бы сбой раз в году. Но рассмотрим все же более серьезную проблему.

Предположим, что, модернизируя нашу систему, мы используем кристалл процессора с тактовой частотой 16 МГц. Возможно, нам не понадобится заменить некоторые компоненты, чтобы система работала с большей скоростью, но триггеры в ИС 74LS74 вполне успешно переключаются с частотой 16 МГц. Или их тоже надо заменить? Коль скоро период тактового сигнала теперь равен 62.5 нс, новое значение MTBF для нашего синхронизирующего устройства равно

$$\text{MTBF}(42.5 \text{ нс}) = \frac{\exp(42.5/1.5)}{0.4 \cdot 1.6 \cdot 10^7 \cdot 10^5} = 3.1 \text{ с!}$$

ЧТО ПОНИМАТЬ ПОД a И f ?

Выход триггера может перейти в метастабильное состояние *только* в том случае, если сигнал на входе D изменяется в пределах окна принятия решения. Но в формулу для MTBF явным образом не входит число таких попаданий. Вместо этого в ней фигурирует общее число a изменений асинхронного входного сигнала в секунду и предполагается, что эти изменения распределены равномерно по периоду тактового сигнала. Поэтому доля изменений входного сигнала, действительно происходящих в пределах окна принятия решения, окажется «учтенной» в параметре f : с увеличением частоты тактового сигнала f все большее число изменений входного сигнала попадет в окно принятия решения.

Если в создаваемой системе изменения входного сигнала не распределены равномерно по периоду тактового сигнала, а группируются в окрестности окна принятия решения (это может происходить в том случае, когда изменения в синхронизируемом входном сигнале как-то привязаны к системному тактовому сигналу с фиксированным, но неизвестным сдвигом по фазе), то полезное правило оценки заключается в том, чтобы в качестве частоты брать величину, обратную длительности окна принятия решения (на основании сообщаемых производителем значений времени установления и времени удержания), умноженную для надежности на коэффициент порядка 10.

Единственное достоинство рассматриваемого синхронизирующего устройства состоит в том, что из-за его отвратительного поведения на частоте 16 МГц проблема, вероятнее всего, обнаружится на стадии лабораторных испытаний, а не после того, как изделие поступит в продажу! Не дай бог, чтобы значение MTBF было нолью года.

8.9.5. Более совершенные синхронизирующие устройства

Имеется несколько возможностей построения более надежных синхронизирующих устройств, чем в случае использования для этих целей ИС 74LS74, когда характеристики синхронизирующего устройства оказываются совсем плохими уже при средних значениях тактовой частоты. Простейшее решение — это применение триггеров, изготовленных по технологии, обеспечивающей большее быстродействие; в большинстве случаев это позволяет удовлетворить требованиям, предъявляемым к разрабатываемой системе. В настоящее время имеются триггеры со значительно большим быстродействием, как в отдельных микросхемах, так и внутри ПЛУ, ИС типа FPGA и в специализированных ИС.

В табл. 8.35 перечислены параметры, относящиеся к метастабильности, для нескольких распространенных логических семейств; эти сведения взяты, главным образом, из технических характеристик, публикуемых производителями. Числовые значения параметров в очень сильной степени зависят от схемных решений и технологии изготовления ИС. В отличие от гарантированных логических уровней сигналов и их временных параметров, числовые значения величин, характеризующих метастабильность, могут изменяться в очень широких пределах

для ИС одного и того же типа в зависимости от производителя, и поэтому нользоваться ими следует с большой осторожностью. Например, ИС 74F74 одного производителя могут давать приемлемые характеристики метастабильности, тогда как в случае другого производителя это условие оказывается невыполненным.

Табл. 8.35. Параметры, характеризующие метастабильность, для ряда рае-пространенных ИС

Источник информации	Тип ИС	τ (нс)	T_0 (с)	t_r (нс)
Chaney (1983)	74LS74	1.50	$4.0 \cdot 10^{-1}$	77.71
Chaney (1983)	74S74	1.70	$1.0 \cdot 10^{-6}$	66.14
Chaney (1983)	74S174	1.20	$5.0 \cdot 10^{-6}$	48.62
Chaney (1983)	74S374	0.91	$4.0 \cdot 10^{-4}$	40.86
Chaney (1983)	74F74	0.40	$2.0 \cdot 10^{-4}$	17.68
TI (1997)	74LSxx	1.35	$4.8 \cdot 10^{-3}$	63.97
TI (1997)	74Sxx	2.80	$1.3 \cdot 10^{-9}$	90.33
TI (1997)	74ALSxx	1.00	$8.7 \cdot 10^{-6}$	41.07
TI (1997)	74ASxx	0.25	$1.4 \cdot 10^3$	14.99
TI (1997)	74Fxx	0.11	$1.9 \cdot 10^8$	7.90
TI (1997)	74HCxx	1.82	$1.5 \cdot 10^{-6}$	71.55
Cypress (1997)	PALC16R8-25	0.52	$9.5 \cdot 10^{-12}$	14.22*
Cypress (1997)	PALC22V10B-20	0.26	$5.6 \cdot 10^{-11}$	7.57*
Cypress (1997)	PALCE22V10-7	0.19	$1.3 \cdot 10^{-13}$	4.38*
Xilinx (1997)	CPLD-серия 7300	0.29	$1.0 \cdot 10^{-15}$	5.27*
Xilinx (1997)	CPLD-серия 9500	0.17	$9.6 \cdot 10^{-18}$	2.30*

Заметьте, что у различных авторов и производителей параметры метастабильности могут быть определены по-разному. Например, Чейни (Chaney) и Texas Instruments (TI) отсчитывают время выхода из метастабильности от переключающего фронта тактового сигнала, то есть так же, как это делали мы в предыдущем разделе. В противоположность этому, Cypress и Xilinx под t_r понимают время, которое должно быть добавлено к нормальному времени задержки t_{pd} от тактового входа до выхода.

Критерий качества, указанный в последнем столбце таблицы, в какой-то степени произволен: это значение времени выхода из метастабильности t_r , необходимое для того, чтобы величина MTBF составляла 1000 лет при работе синхронизирующего устройства с частотой 25 МГц и при 100000 изменений асинхронного входного сигнала в секунду. Для схем фирм Cypress и Xilinx значения t_r помечены звездочкой в знак того, что имеется в виду их собственное определение этой величины согласно сказанному выше.

Как вы можете видеть, ИС 74ALS74 — одна из худших микросхем, перечисленных в таблице. Если в 16-мегагерцовой микропроцессорной системе, рассматривавшейся в качестве примера в предыдущем разделе, заменить FF1 на триггер из ИС 74ALS74, то получим:

$$MTBF(42.5\text{нс}) = \frac{\exp(42.5/100)}{8.7 \cdot 10^{-6} \cdot 1.6 \cdot 10^7 \cdot 10^5} = 2.06 \cdot 10^{11} \text{ с.}$$

Если вам годится синхронизирующее устройство со значением MTBF, равным 65 столетиям, для каждого из проданных изделий, то на этом можно остановиться. Но если на триггер из ИС 74ALS74 заменить также и FF2, то значение MTBF станет еще лучше, так как у схем 74ALS74 время установления меньше чем у ИС 74LS74, и составляет всего 10 нс. В результате использования триггера из ИС 74ALS74 на месте FF2 значение MTBF станет примерно в 20000 раз большим:

$$MTBF(52.5\text{нс}) = \frac{\exp(52.5/100)}{8.7 \cdot 10^{-6} \cdot 1.6 \cdot 10^7 \cdot 10^5} = 4.54 \cdot 10^{15} \text{ с.}$$

Даже если мы продадим миллион изделий с таким синхронизирующим устройством, у нас (или у наших клиентов) сбой будет происходить в синхронизирующем устройстве раз в 144 года. Сегодня это обеспечивает нам надежную работу!

На самом деле указанный запас надежности не так велик, как это может показаться. (Насколько большим кажется вам интервал длиной 144 года?) Большинство числовых значений в табл. 8.35 являются *средними*, и редко кто из производителей ИС указывает эти параметры в технических данных, не говоря уже о том, чтобы гарантировать их величину. Кроме того, вычисляемое значение MTBF крайне чувствительно к значению τ , которое, в свою очередь, может зависеть от температуры, напряжения и фазы луны. Поэтому данный триггер в реальной системе может работать много хуже (или много лучше), чем то, что предсказывает наша таблица.

Рассмотрим, например, что произойдет, если мы увеличим тактовую частоту в нашей 16-мегагерцовой системе до 20 МГц, то есть всего на 25%. Вы, естественно, можете подумать, что метастабильность ухудшится на 25%, или, быть может, на 250%, если считать с большим запасом. Но вы подсчитайте, и тогда увидите, что при использовании элементов ИС 74ALS74 на месте обоих триггеров FF1 и FF2 значение MTBF упадет до $3.7 \cdot 10^9$ с, то есть станет хуже более чем в миллион раз! Новое значение MTBF составляет примерно 429 лет, и это прекрасно, если иметь в виду одну систему; но если вы продадите миллион таких систем, то отказ в одной из них будет происходить раз в четыре часа. В результате, вместо того, чтобы в течение долгого времени оставаться незаменимым работником вашей фирмы, вы становитесь в ней козлом отпущения!

8.9.6. Другие схемы синхронизирующих устройств

Выше мы обещали рассказать о том, как строить более надежные синхронизирующие устройства. Первый способ уже указан: применяйте триггеры с большим быстродействием, в результате чего уменьшится значение τ в выражении для MTBF. Второй способ очевиден: нужно увеличить допустимое время t_f в выражении для MTBF.

Самое большое значение t_r в схеме на рис. 8.96 при заданной частоте системного тактового сигнала равно t_{clk} , и это достижимо в том случае, когда время установления триггера FF2 равно 0. Но значение t_r можно увеличить на порядок и сделать равным $n \cdot t_{clk}$, воспользовавшись схемой *многотактного синхронизирующего устройства* (*multiple-cycle synchronizer*), приведенной на рис. 8.98. Здесь частота системного тактового сигнала делится на n , полученное таким образом колебание играет роль тактового сигнала для синхронизирующего устройства и допустимое время метастабильности возрастает до $t_r = (n \cdot t_{clk}) - t_{setup}$. Обычно значения $n = 2$ или $n = 3$ обеспечивают подходящую надежность синхронизирующего устройства.

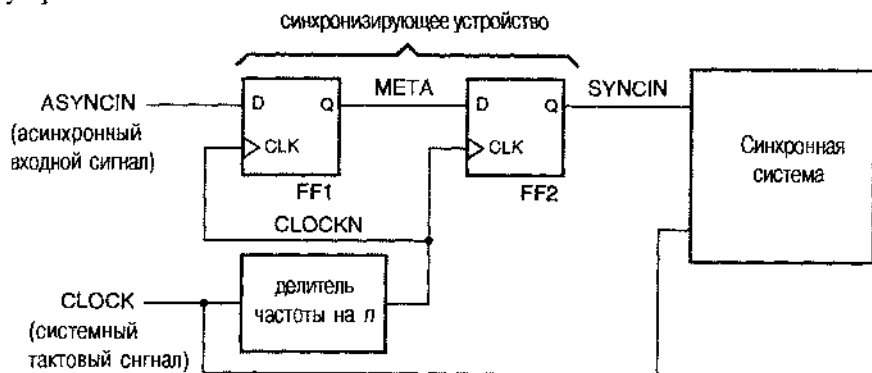


Рис. 8.98. Многотактное синхронизирующее устройство

Обратите внимание на то, что в этой схеме перепады в сигнале CLOCKN будут отставать от перепадов в сигнале CLOCK, поскольку CLOCKN является выходным сигналом счетчика, состоящего из триггеров, переключающихся по фронту сигнала CLOCK. Это означает, что сигнал SYNCIN, в свою очередь, будет задержан или затянут по отношению к другим сигналам в синхронной системе, которые вырабатываются триггерами, переключающимися по фронту сигнала CLOCK непосредственно. Если сигнал SYNCIN проходит в синхронной системе через дополнительную комбинационную логику, прежде чем достигает входов системных триггеров, то требования, предъявляемые временем установления этих триггеров, могут оказаться невыполнимыми. В этом случае можно воспользоваться схемным решением, представленным на рис. 8.99. Здесь сигнал SYNCIN еще раз тактируется триггером FF3 по сигналу CLOCK, в результате чего вырабатывается сигнал DSYNCIN, временные параметры которого будут такими же, как и у сигналов на выходах других триггеров синхронной системы. Задержка сигнала CLOCKN по отношению к сигналу CLOCK должна быть достаточно малой, чтобы удовлетворять требованию, предъявляемому временем установления триггера FF3.

Чем больше n в n -тактном синхронизирующем устройстве, тем дольше синхронной системе не нужно изменять асинхронного входного сигнала. Эта задержка является ценой, которую необходимо уплатить за надежную работу системы. В типичной микропроцессорной системе большая часть асинхронных входных сигналов извещает систему о внешних событиях (прерывания, требования прямого доступа в память и т.д.), так что не требуется распознать их очень быстро

с точки зрения задержки в синхронизирующем устройстве. Когда обращение к памяти критично по времени, опытные разработчики заставляют подсистему памяти работать от тактового сигнала процессора, если только это возможно. При этом надобность в синхронизирующем устройстве пропадает и система функционирует с наибольшим возможным быстродействием.

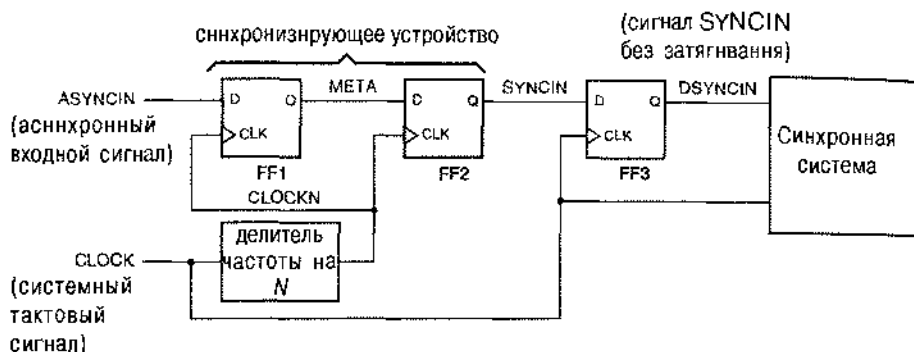


Рис. 8.99. Многотактное синхронизирующее устройство с компенсацией задерживания

На более высоких частотах возможность реализации многотактного синхронизирующего устройства по схеме, приведенной на рис. 8.98, ограничена разбросом задержек тактового сигнала. По этой причине некоторые проектировщики вместо деления частоты системного тактового сигнала на n применяют *последовательно включенные синхронизирующие устройства (cascaded synchronizers)*. При таком подходе используется цепочка из n триггеров (регистр сдвига), в которой все триггеры переключаются быстрым системным тактовым сигналом. Соответствующая схема показана на рис. 8.100

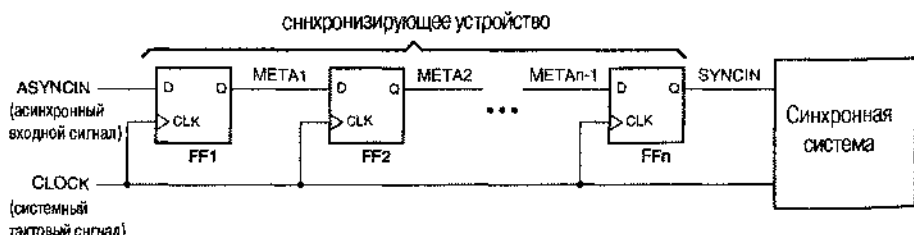


Рис. 8.100. Многокаскадное синхронизирующее устройство

Принцип действия многокаскадного синхронизирующего устройства основан на том, что с некоторой вероятностью выход из состояния метастабильности произойдет уже в первом триггере, а в случае неудачи — с равной вероятностью в каждом следующем из триггеров, включенных последовательно. Таким образом, вероятность отказа синхронизирующего устройства в целом оказывается порядка n -й степени вероятности отказа на данной частоте системного тактового сигнала синхронизирующего устройства с одним триггером. И хотя это отча-

сти верпо, нсе же величина МТВР для многокаскадного синхронизирующего устройства меньше, чем для многотактного синхронизирующего устройства с тем же временем задержки ($n \cdot t_{\text{clk}}$). В случае многокаскадного устройства время установления триггера t_{setup} необходимо вычесть n раз из времени t_p , тогда как в случае многотактного устройства значение t_{setup} вычитается только один раз.

Для построения синхронизирующего устройства можно воспользоваться внутренними триггерами ПЛУ; при этом оба триггера в схеме на рис. 8.96 находятся в одном ПЛУ. В большинстве приложений это очень удобно, так как исключается необходимость применения внешних триггеров, размещенных в отдельной ИС. Однако, как правило, значение МТВР для синхронизирующего устройства, образованного внутри ПЛУ, хуже, чем при использовании отдельных ИС, созданных на той же или подобной технологии. Это происходит потому, что на D-входе каждого триггера в ПЛУ имеется комбинационная логическая матрица, увеличивающая его время установления и тем самым уменьшающая время t_p , в течение которого должен произойти выход из состояния метастабильности, при заданном периоде t_{clk} системного тактового сигнала. Чтобы сделать значение t_p максимально возможным, не используя для этого специальных компонентов, в качестве FF2 на схеме на рис. 8.96 следует применить триггер из отдельной ИС с малым временем установления.

8.9.7. Триггеры с защитой от метастабильности

В конце 80-х годов фирма Texas Instruments и другие производители приступили к выпуску ИС малой и средней степени интеграции с триггерами, специально предназначенными для использования в синхронизирующих устройствах, встраиваемых в систему на уровне печатных плат. Микросхема 74AS4374 была, например, подобна ИС 74AS374, но с тем отличием, что отдельные триггеры заменены парами триггеров, включенных по схеме, представленной на рис. 8.101. Каждую пару триггеров можно было применить в качестве синхронизирующего устройства типа устройства, приведенного на рис. 8.96, так что с помощью одной ИС 74AS4374 оказалось возможным синхронизировать восемь асинхронных сигналов.

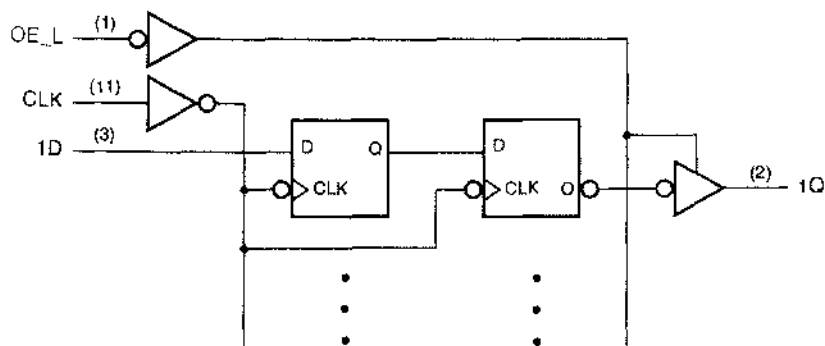


Рис. 8.101. Принципиальная схема одного из 8 двояных D-триггеров в ИС 74AS4374

Внутренняя структура ИС 74AS4374 была усовершенствована таким образом, чтобы уменьшить значения t и T_0 по сравнению с другими триггерами серии 74AS, но самым замечательным достижением было значительное сокращение времени установления t_{setup} . Поскольку вся схема синхронизирующего устройства, приведенного на рис. 8.96, в данном случае размещается в одном кристалле, между триггерами FF1 и FF2 нет входных и выходных буферов, и значение t_{setup} для триггера FF2 составляет всего 0.5 нс. У обычного триггера серии 74AS эта величина равняется 5 нс, поэтому — при $\tau = 0.40$ нс — переход на ИС 74AS4374 приводит к увеличению среднего времени между сбоями MTBF в $\exp(4.5/0.40) \approx 77000$ раз.

В последние годы по мере движения в сторону КМОП-технологий, обеспечивающих большее быстродействие и большую плотность упаковки, специализированные компоненты типа 74AS4374 почти полностью вышли из употребления. Как можно видеть из табл. 8.35, быстродействующие ПЛУ и ИС типа CPLD вполне конкурентоспособны по величине t с самыми быстродействующими устройствами, собранными на отдельных ИС, и в то же время предоставляют возможность объединить синхронизацию со многими другими функциями. Но все же подход, примененный в ИС 74AS4374 , заслуживает воспроизведения при проектировании на основе ИС типа FPGA и на основе специализированных ИС. Другими словами, на любой стадии осуществления контроля за компоновкой схемы синхронизирующего устройства следует располагать триггеры FF1 и FF2 как можно ближе один к другому и соединять их между собой сигнальными линиями с наибольшей доступной скоростью прохождения сигнала; это обеспечит максимизацию времени установления триггера FF2.

8.9.8. Синхронизация при высокоскоростной передаче данных

Широко распространенной проблемой, возникающей в компьютерных системах, является синхронизация переноса данных, поступающих по внешним линиям, с внутренним тактовым сигналом компьютера. Простым примером служит согласование между сетевой картой персонального компьютера и линией Ethernet со скоростью передачи 100 Мбит/с. Сетевая карта может быть вставлена в разъем линии PCI с тактовой частотой 33.33 МГц. Хотя скорость передачи в сети Ethernet приблизительно кратна частоте тактового сигнала и шине компьютера, сигнал, поступающий из линии Ethernet, был отправлен другим компьютером, а тактовые сигналы на передающем и приемном конце в любом случае не синхронизированы. Тем не менее, сетевая карта обязана надежно выдать данные на шину PCI.

Эта проблема схематически представлена на рис. 8.102. Последовательные данные RDATA, представленные в коде NRZ, принимаются по линии Ethernet со скоростью 100 Мбит/с. Цифровая схема ФАПЧ (Digital Phase-Locked Loop, DPLL) извлекает 100-мегагерцовый тактовый сигнал RCLK из потока данных, поступающих со скоростью 100 Мбит/с, и позволяет заталкивать данные по битов в 8-разрядный регистр сдвига. В то же самое время схема синхронизации по байтам ищет в принимаемом потоке данных последовательность битов специального вида, которой отмечаются границы между байтами. Обнаруживая одну из них, схема синхронизации по байтам выдает сигнал SYNC и поступает так и

каждом восьмом такте сигнала RCLK; таким образом, сигнал SYNC возникает всякий раз, когда регистр сдвига содержит выровненный по границам 8-битовый байт принимаемых данных. В остальной части системы тактирование осуществляется тактовым сигналом SCLK с частотой 33.33 МГц. Нам необходимо переносить каждый выровненный по границам байт RBYTE[7:0] в регистр SREG, находящийся в той части системы, которая работает с тактовым сигналом SCLK. Как это можно сделать?

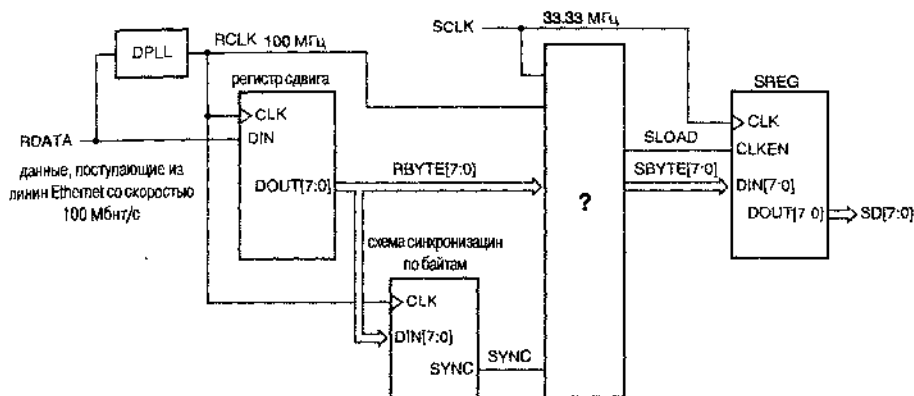


Рис. 8.102. Схематическое изображение проблемы синхронизации с сетью Ethernet

ПО ПОЛУБАЙТУ ЗА РАЗ

Здесь приводится сильно упрощенное описание процедуры приема сигналов из 100-мегабитной линии Ethernet, но этого достаточно для рассмотрения проблемы синхронизации. В действительности, скорость поступления принимаемых двоичных сигналов равна 125 Мбит/с, причем каждые 4 бита данных пользователя представлены в так называемом коде 4B5B, то есть символом, состоящим из 5 битов. Используется только 16 из 32 возможных слов кода 4B5B, и этим гарантируется, что, независимо от последовательности данных, передаваемой пользователем, число переходов в принимаемом потоке будет достаточным для извлечения из него тактового сигнала. Кроме того, код 4B5B включает специальные слова, которые передаются периодически и позволяют совсем просто осуществить синхронизацию по полубайтам (состоящим из 4 битов) и байтам.

В результате синхронизации по полубайтам типичный интерфейс 100-мегабитной сети Ethernet не видит несинхронизированного 100-мегагерцового потока битов. Вместо этого он имеет дело с несинхронизированным 25-мегагерцовым потоком полубайтов. Поэтому реальное синхронизирующее устройство для сети Ethernet в деталях отличается от рассматриваемого нами, но принцип действия тот же.

На рис. 8.103 приведены несколько временных диаграмм. Сразу видно, что сигнал выравнивания байтов по границам SYNC имеет активный уровень только в течение 10 нс в пределах байта. Нет никакой надежды, что удастся каждый раз привязывать этот сигнал к системному тактовому сигналу SCLK, период которого, равный 30 нс, много больше.

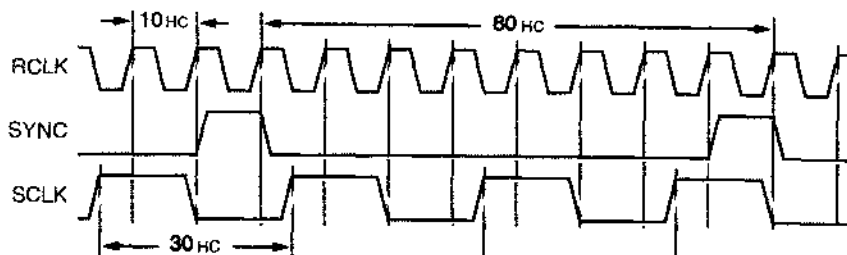


Рис. 8.103. Временные диаграммы сигналов в линии Ethernet и системный тактовый сигнал

Стратегия, которой следуют практически всегда в ситуации подобного рода, состоит в том, что сначала выровненные по границам данные заносят в регистр хранения HREG по тактовому сигналу RCLK из *принимаемого потока данных*. Это дает нам значительно больше времени, в данном случае – 80 нс, чтобы разобраться с принятым байтом. Таким образом, блок, помеченный вопросительным знаком “?” на рис. 8.102, можно заменить схемой, показанной на рис. 8.104, состоящей из регистра HREG и узла, названного “SCTRL”. Функция этого узла заключается в выработке сигнала SLOAD в течение точно одного периода системного тактового сигнала SCLK, равного 30 нс, так, чтобы сигналы на выходах регистра HREG на этом интервале оставались постоянными и тем самым было удовлетворено требование неизменности сигнала в течение времени установления и времени удержания регистра SREG, переключающегося по сигналу SCLK. Для остальной части интерфейса возникновение сигнала SLOAD означает, что «приняты новые данные» и что байт новых данных выдается на шину SBYTE[7:0] в течение очередного периода сигнала SCLK. На рис. 8.105 представлены возможные временные диаграммы для сигналов SLOAD и SBYTE в случае реализации такого подхода и с учетом временных диаграмм, приведенных ранее.

На рис. 8.106 показана схема, способная вырабатывать сигнал SLOAD с желательными свойствами. Основная идея состоит в использовании сигнала SYNC для установки SR-защелки в единичное состояние всякий раз, как новый байт становится доступным. Выходной сигнал этой защелки NEWBYTE опрашивается триггером FF1 по сигналу SCLK. Поскольку сигнал NEWBYTE не синхронизован с сигналом SCLK, триггер FF1 может оказаться в состоянии метастабильности, но его выходной сигнал безразличен для триггера FF2 до следующего переключающего фронта в тактовом сигнале, то есть на протяжении 30 нс. В предположении, что триггер FF1 является достаточно быстродействующим, мы имеем много времени для выхода из метастабильности. Сигнал на выходе триггера FF2 является требуемым сигналом SLOAD. Триггер FF1 позволяет удерживать единичное значение сигнала SLOAD только на интервале времени, равном периоду сигнала SCLK;

когда значение SLOAD уже равно 1, оно не может оставаться таким же с приходом очередного переключающего фронта тактового сигнала. Таким образом, у SR-защелки есть время быть сброшенной сигналом SLOAD и подготовиться к следующему байту.

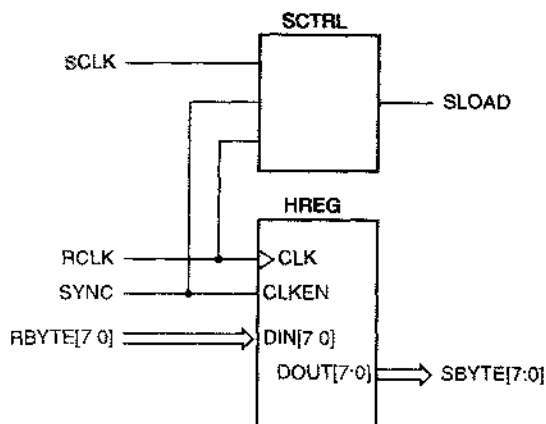


Рис. 8.104. Регистр хранения и управляющий узел

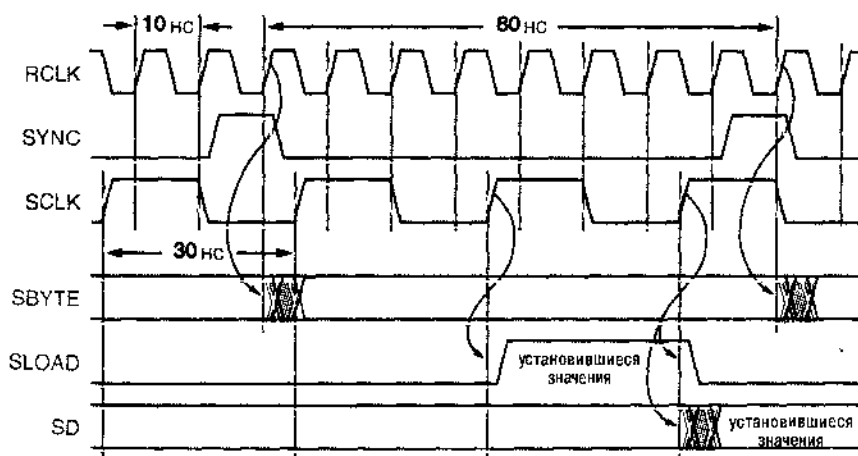


Рис. 8.105. Временные диаграммы сигналов в устройстве синхронизации, включая сигналы на шинах SBYTE и возможный вид сигнала SLOAD

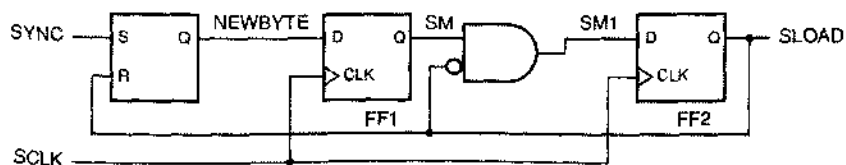


Рис. 8.105. Схема узла SCTRL, вырабатывающего сигнал SLOAD

На рис. 8.107 приведены временные диаграммы для узла SCTRL с «типичными» временными соотношениями между сигналами. Поскольку сигналы SCLK и RCLK асинхронны, сигнал SCLK может произвольно располагаться на оси времени по отношению к сигналам RCLK и SYNC. На рисунке показан случай, когда очередным нарастающим фронтом сигнала SCLK приходит спустя заметное время после возникновения сигнала NEWBYTE. Хотя на рисунке изображены окна, в которых сигналы SM и SM1 могли бы быть метастабильными в общем случае, однако в той ситуации, какая указана на рисунке, метастабильность, в действительности, не наступает. Позднее мы рассмотрим, что может произойти, если фронт сигнала SCLK совпадает по времени с изменением сигнала NEWBYTE.

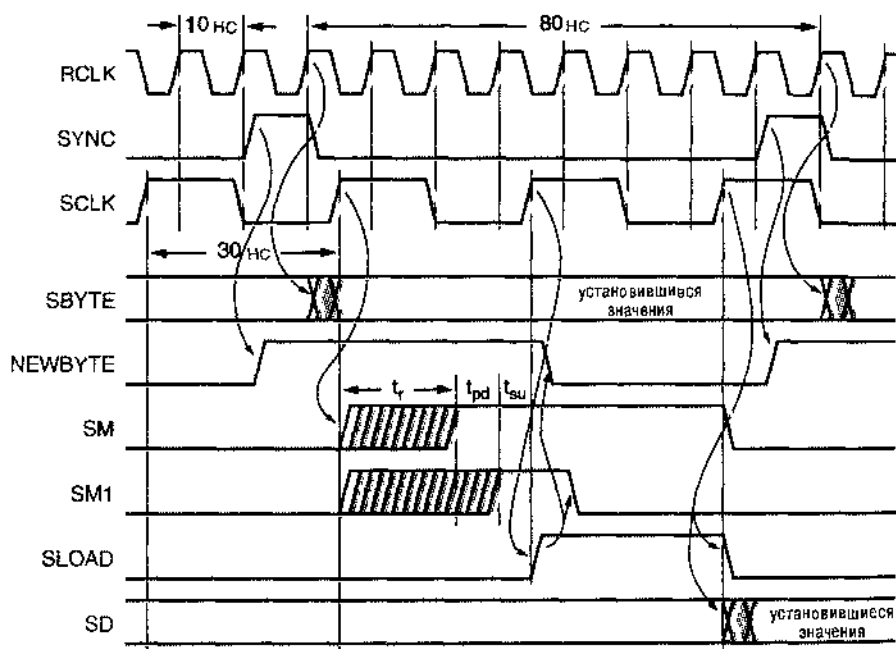


Рис. 8.107. Временные диаграммы для схемы SCTRL, приведенной на рис. 8.106

По поводу схемы на рис. 8.106 нужно сделать несколько замечаний. Во-первых, у сигнала SYNC не должно быть паразитных импульсов, т.к. он подан на вход S защелки, и он должен быть достаточной длительности, чтобы удовлетворить требованию защелки к минимальной ширине импульса. Поскольку защелка устанавливается в типичное состояние по нарастающему фронту сигнала SYNC, получается, что мы немного смещаемся: сигнал NEWBYTE может возникнуть чуть раньше того момента, когда новый байт действительно будет доступен на выходах регистра HREG. Но это не страшно: прежде чем произойдет загрузка регистра SREG, пройдет два периода сигнала SCLK после того, как триггер FF1 «увидит» сигнал NEWBYTE. На самом деле, можно было сказать еще больше, если имеется опережающая версия сигнала SYNC (см. задачу 8.95).

Пусть время установления D-триггера равно t_{su} , а задержка распространения для вентиля И — t_{pd} ; тогда для выхода из метастабильности мы располагаем временем t_s , равным периоду сигнала SCLK (30 нс) минус $(t_{su} + t_{pd})$, как показано на рис. 8.107. Из временных диаграмм также видно, почему нельзя воспользоваться непосредственно сигналом SM для сброса SR-зашелки. Возможная метастабильность сигнала SM могла бы полностью нарушить работу схемы. Например, половинное по отношению к высокому уровню значение сигнала SM может оказаться достаточным, чтобы сбросить зашелку, но затем сам сигнал SM может перейти обратно на низкий уровень, и тогда сигнал SLOAD не будет выработан и мы пропустим байт. Используя выходной сигнал синхронизирующего устройства (SLOAD) как для сброса зашелки, так и в качестве сигнала загрузки в части интерфейса, управляемой тактовым сигналом SCLK, мы гарантируем, что новый байт будет обнаружен и надлежащим образом воспринят в обеих частях интерфейса, работающих с тактовыми сигналами RCLK и SCLK.

Приведенные на рис. 8.107 временные диаграммы относятся к штатной ситуации, но нам необходимо проанализировать также, что произойдет при другом сдвиге по времени между сигналом SCLK и сигналами RCLK и SYNC, нежели тот, какой указан на рисунке. Вы, наверное, сами сможете убедиться в том, что все будет работать хорошо, как и в рассмотренном случае, если фронт сигнала SCLK придется на тот интервал времени, когда сигнал NEWBYTE только-только переходит на высокий уровень; просто в этом случае перенос данных заканчивается чуть раньше. Более интересен случай, когда сигнал SCLK приходит чуть позже, в результате чего сигнал NEWBYTE, переходящий на высокий уровень, оказывается пропущенным и обнаруживается на один период сигнала SCLK позднее. В этом случае временные диаграммы имеют вид, указанный на рис. 8.108.

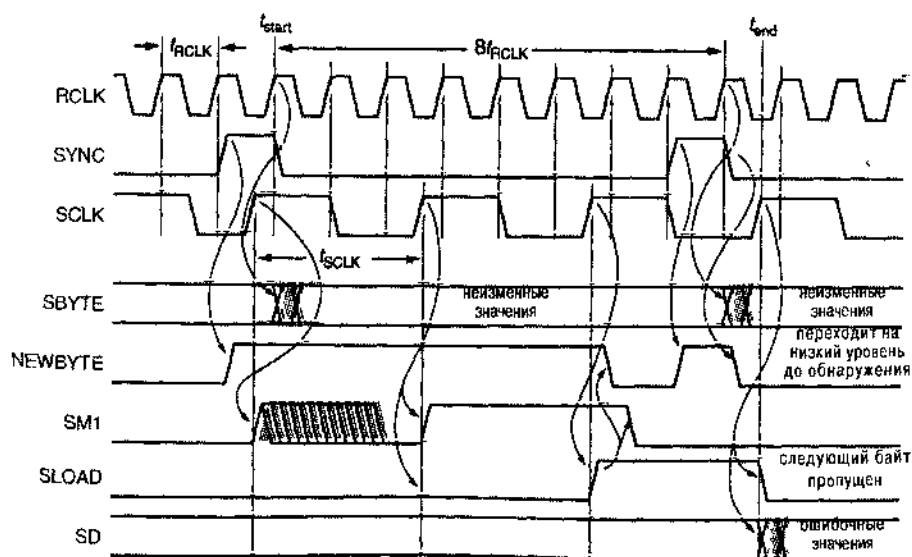


Рис. 8.108. Временные диаграммы для схемы SCTRL в случае максимальной задержки

На этих временных диаграммах изображен случай, в котором сигнал NEWBYTE переходит на высокий уровень примерно в то же время, когда поступает фронт сигнала SCLK, точнее – перед приходом фронта сигнала SCLK на расстоянии, меньшем времени t_{su} триггера FF1. Таким образом, триггер FF1 может «не увидеть», что сигнал NEWBYTE имеет высокий уровень, либо выход триггера может стать метастабильным и единичное значение сигнала NEWBYTE будет падежно зафиксировано спустя лишь один период сигнала SCLK. После этого только через два периода сигнала SCLK по фронту этого сигнала произойдет загрузка значений сигналов с шины SBYTE в регистр SREG.

При таком взаимном расположении сигналов по времени возникают неприятности, потому что к моменту загрузки значения сигналов на шине SBYTE уже изменились и выражают собой *следующий* принятый байт. Кроме того, сигнал SLOAD может иметь единичное значение в то время, когда приходит импульс SYNC, соответствующий следующему принятому байту, и чуть позднее. Но тогда на обоих входах зашелки S и R одновременно действуют сигналы активного уровня, и если они снимаются одновременно, то выход зашелки может войти в метастабильное состояние. Если же сигнал сброса R действует дольше, как это показано на временных диаграммах, то зашелка останется в нулевом состоянии, и следующий принятый байт вообще не будет обнаружен и не будет передан той части интерфейса, которая работает по тактовому сигналу SCLK.

Следовательно, нам необходимо более внимательно проанализировать временные диаграммы в случае максимальной задержки и решить, будет ли синхронизирующее устройство функционировать надлежащим образом. Начальной точкой в нашем анализе схемы SCTRL будет указанный на рис. 8.108 момент времени t_{start} , когда байт загружается в регистр HREG по фронту сигнала RCLK в конце импульса SYNC. Процедура заканчивается в момент времени t_{end} , когда значения сигналов с шины SBYTE загружаются в регистр SREG по фронту сигнала SCLK. Максимальная задержка между этими двумя моментами времени, которую мы назовем t_{maxd} , равна сумме следующих составляющих:

- t_{RCLK} Минус один период сигнала RCLK, расстояние между моментом t_{start} и предшествующим фронтом этого сигнала, на которому сигнал SYNC принял единичное значение. Это число отрицательно, поскольку сигнал SYNC возникает на один период тактового сигнала раньше того момента, когда фактически происходит загрузка в регистр HREG.
- t_{CQ} Максимальная задержка в триггере от входа CLK до выхода Q. Предполагается, что сигнал SYNC является выходным сигналом триггера, переключающегося по сигналу RCLK. Тогда величина t_{CQ} – это интервал времени между фронтом сигнала RCLK и моментом, когда возникает сигнал SYNC.
- t_{SQ} Максимальная задержка в SR-защелке на рис. 8.106 от входа S до выхода Q. Это задержка, с которой появляется сигнал NEWBYTE.
- t_{su} Время установления триггера FF1 на рис. 8.106. Необходимо, чтобы сигнал NEWBYTE принимал единичное значение к началу этого интервала времени или до него. Только тогда гарантируется обнаружение.

- t_{SCLK} Период сигнала SCLK. Поскольку сигналы RCLK и SCLK асинхронны, задержка между моментом возникновения сигнала NEWBYTE и моментом его фиксации в триггере FF1 на фронту сигнала SCLK может равняться одному периоду этого сигнала.
- t_{SCLK} После того, как сигнал NEWBYTE обнаружен триггером FF1, проходит еще один период сигнала SCLK, прежде чем возникнет сигнал SLOAD.
- t_{SCLK} Значения сигналов на шине SBYTE загружаются в регистр SREG на следующем такте после возникновения сигнала SLOAD.

Таким образом, $t_{\text{maxd}} = 3t_{\text{SCLK}} + t_{\text{CQ}} + t_{\text{SQ}} + t_{\text{su}} - t_{\text{RCLK}}$. Чтобы завершить анализ, необходимо определить еще несколько параметров:

- t_{h} Время удержания регистра SREG.
- $t_{\text{CQ(min)}}$ Минимальная задержка в регистре HREG от входа CLK к выходу Q; в качестве заниженной оценки этой величины принимается 0.
- t_{rec} Время восстановления SR-защелки, минимально допустимое время между переходами S и R на неактивный уровень (см. замечание, вынесенное за пределы основного текста, в конце раздела 7.2.1).

Чтобы значения сигналов на шине SBYTE были успешно загружены в регистр SREG, они должны оставаться неизменными, по крайней мере, до момента $t_{\text{end}} + t_{\text{h}}$. Момент времени, когда значения сигналов SBYTE изменяются, равен t_{start} плюс 8 периодов сигнала RCLK плюс $t_{\text{CQ(min)}}$. Следовательно, для надлежащей работы схемы необходимо, чтобы выполнялось неравенство:

$$t_{\text{end}} + t_{\text{h}} \leq t_{\text{start}} + 8t_{\text{RCLK}}.$$

В случае максимальной задержки мы подставляем в это неравенство $t_{\text{end}} = t_{\text{start}} + t_{\text{maxd}}$ и, вычитая t_{start} из обеих его частей, получаем:

$$t_{\text{maxd}} + t_{\text{h}} \leq 8t_{\text{RCLK}}.$$

Подставляя значение t_{maxd} и выполняя преобразования, получим следующее условие правильной работы схемы:

$$3t_{\text{SCLK}} + t_{\text{CQ}} + t_{\text{SQ}} + t_{\text{su}} + t_{\text{h}} \leq 9t_{\text{RCLK}}. \quad (8.1)$$

Это очень плохо. Даже если предположить, что задержки t_{CQ} , t_{SQ} , t_{su} и t_{h} совсем малы, все же $3t_{\text{SCLK}}$ (90 нс) плюс что-то заведомо больше, чем $9t_{\text{RCLK}}$ (тоже 90 нс). Значит, это устройство никогда не будет работать как следует при максимальной задержке.

Но даже если результат анализа времени задержки был бы хорошим, то и в этом случае нам все же следует рассмотреть требования, которым должна удовлетворять схема SCTRL, чтобы этот узел работал как надо. В частности, необходимо гарантировать, что мнимый SYNC, относящийся к следующему байту, не окончится раньше, чем через время t_{rec} после того, как будет снят сигнал SLOAD, относящийся к предыдущему байту. Таким образом, мы получаем еще одно условие правильной работы:

$$t_{\text{end}} + t_{\text{CQ}} + t_{\text{rec}} \leq t_{\text{start}} + 8t_{\text{RCLK}} + t_{\text{CQ(min)}}.$$

Производя подстановки и выполняя такие же преобразования, как и раньше, мы приходим к еще одному требованию, которое не удовлетворяется в нашем устройстве:

$$3t_{\text{SCLK}} + 2t_{\text{CQ}} + t_{\text{SQ}} + t_{\text{su}} + t_{\text{rec}} \leq 9t_{\text{RCLK}} \quad (8.2)$$

Можно несколькими способами внести изменения в устройство, чтобы удовлетворить требованиям, предъявляемым к нему в худшем случае. В начале нашего рассмотрения мы уже упоминали о «мошеничестве», когда сигнал SYNC вырабатывается за один период сигнала RCLK до того, как данные оказываются записанными в регистр HREG; действительно, сигнал SYNC можно было бы выставлять и еще раньше. Если так поступить, то это поможет нам удовлетворить требованиям, относящимся к случаю максимальной задержки за счет уменьшения величины, стоящей справа в полученных соотношениях. Если, например, сигнал SYNC возник бы на два периода сигнала RCLK раньше, то вместо “ $8t_{\text{RCLK}}$ ” в правой части неравенств можно было бы написать “ $6t_{\text{RCLK}}$ ”. Но «бесплатный сыр бывает только в мышеловке»: мы не можем выставлять сигнал SYNC сколь угодно рано. Нам необходимо рассмотреть также случай *минимальной задержки*, чтобы гарантировать фактическое наличие нового байта в регистре HREG, когда значения сигналов на шине SBYTE будут загружаться в регистр SREG. Минимальная задержка t_{mind} между моментами t_{start} и t_{end} складывается из следующих составляющих:

- nt_{RCLK} Минус n периодов сигнала RCLK, интервал времени в обратном направлении между моментом t_{start} и фронтом сигнала SYNC. В исходном варианте устройства $n = 1$.
- $t_{\text{CQ(min)}}$ Это минимальная задержка между фронтом сигнала RCLK и моментом установления сигнала SYNC; в качестве заниженной оценки принимается значение 0.
- t_{SQ} Это задержка, с которой вырабатывается сигнал NEWBYTE, также полагаемая равной 0.
- t_{h} Минус время удержания триггера FF1 на рис. 8.10б. Сигнал NEWBYTE может возникнуть к концу времени удержания и все же оказаться обнуленным.
- $0t_{\text{SCLK}}$ Равное нулю число периодов сигнала SCLK. Может случиться так, что «на наше счастье» фронт сигнала SCLK придется как раз на момент окончания времени удержания триггера FF1.
- t_{SCLK} Задержка на один период сигнала SCLK, с которой, как и ранее, возникает сигнал SLOAD.
- t_{SCLK} Задержка на один период сигнала SCLK, с которой, как и ранее, происходит загрузка значений сигналов на шине SBYTE в регистр SREG.

Другими словами, $t_{\text{mind}} = 2t_{\text{SCLK}} - t_{\text{h}} - nt_{\text{RCLK}}$.

В этом случае мы должны гарантировать, что новый байт достигнет выходов регистра HREG к моменту начала времени установления регистра SREG; следовательно, должно выполняться неравенство:

$$t_{\text{end}} + t_{\text{su}} \geq t_{\text{start}} + t_{\text{co}},$$

где t_{co} — максимальная задержка в регистре HREG от тактового входа до выхода. Подставляя $t_{end} = t_{start} + t_{mind}$ и вычитая t_{start} с обеих сторон, получаем:

$$t_{mind} - t_{su} \geq t_{co}.$$

Подставляя значения t_{mind} и производя преобразования, мы приходим к окончательному условию:

$$2t_{SCLK} - t_h - t_{su} - t_{co} \geq nt_{RCLK}. \quad (8.3)$$

Если, например, каждая из величин t_h , t_{su} и t_{co} равна 10 нс, то максимальное значение n равно 3; нельзя вырабатывать сигнал SYNC более чем на два периода тактового сигнала ранее его первоначального положения, указанного на рис. 8.108. В зависимости от других значений задержек этого может быть достаточно для решения проблемы в случае максимальной задержки, но может быть и не достаточно; для конкретного набора компонентов этот вопрос рассматривается в задаче 8.95.

Может случиться так, что сдвиг импульса SYNC в сторону более раннего времени недостаточен для компенсации задержек или его нельзя осуществить в той или иной системе. Существует другое решение проблемы, которое всегда можно осуществить. Оно состоит в увеличении времени между последовательными переносами данных из части схемы, работающей с одним тактовым сигналом, в часть схемы, работающую с другим тактовым сигналом. Это всегда возможно за счет переноса каждый раз большего числа битов. В интерфейсе сети Ethernet, например, мы могли бы собирать по 16 битов в той части устройства, где переключение происходит по сигналу RCLK, и переносить по 16 битов за раз в часть устройства с переключением по сигналу SCLK. В результате фигурировавшая ранее величина $8t_{RCLK}$ заменяется на $16t_{RCLK}$ и тем самым обеспечивается гораздо больший запас по времени для случая максимальной задержки. Перенося за один раз 16 битов в часть устройства, работающую по тактовому сигналу SCLK, мы можем затем разбить их на два 8-битовых отрезка, если нужно обрабатывать данные побайтно.

Характеристики устройства можно улучшить, видоизменив схему узла SCTRL. На рис. 8.109 показан вариант этой схемы, в котором сигнал SLOAD вырабатывается непосредственно триггером, на вход данных которого поступает сигнал NEWBYTE. При этом сигнал SLOAD появляется на один период сигнала SCLK раньше, чем в нашей исходной схеме SCTRL. Кроме того, раньше сбрасывается SR-защелка. Эта схема работает только в том случае, если оказываются выполненными следующие существенные предположения:

1. Для триггера FF1 приемлемым является уменьшенное время выхода из метастабильности, равное интервалу времени, в течение которого сигнал SCLK остается на высоком уровне. Метастабильность должна разрешиться до того, как сигнал SCLK перейдет на низкий уровень, так как в этот момент произойдет сброс SR-защелки, если сигнал SLOAD будет иметь высокий уровень.
2. Время установления регистра SREG по входу CLKEN (рис. 8.102) меньше или равно времени, в течение которого сигнал SCLK пребывает на низком уровне. Если справедливо предыдущее предположение, то сигнал SLOAD, поданный на вход CLKEN, может оставаться метастабильным до тех пор, пока сигнал SCLK не перейдет на низкий уровень.

3. Интервал времени, в пределах которого сигнал SCLK имеет низкий уровень, достаточно велик для того, чтобы был выработан импульс сброса в точке RNEW, удовлетворяющий требованию SR-защелки в отношении минимальной длительности импульса.

Заметьте, что при выполнении этих условий правильность работы схемы зависит от коэффициента заполнения сигнала SCLK. Если сигнал SCLK является относительно медленным и его коэффициент заполнения близок к 50%, то данная схема прекрасно работает. Но если частота сигнала SCLK слишком велика, либо его коэффициент заполнения очень мал, очень велик или непредсказуем, то необходимо воспользоваться первоначальной конструкцией.

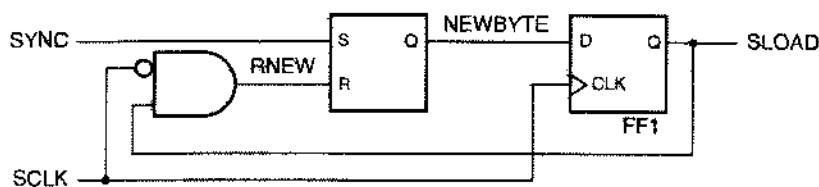


Рис. 8.109. Схема SCTRL, вырабатывающая сигнал SLOAD на половине периода тактового сигнала

Для правильной работы каждой из рассмотренных схем синхронизации требуется, чтобы частота тактового сигнала находилась в определенном диапазоне значений; у каждой схемы этот диапазон свой. Это необходимо учитывать при тестировании, когда тактовые сигналы обычно бывают более медленными, а также при модернизации, связанной с увеличением одной или обеих тактовых частот. В случае интерфейса сети Ethernet, например, не предполагается изменение стандартной частоты 100 Мбит/с, но частота тактового сигнала в шине PCI может быть повышена и стать равной не 33 МГц, а 66 МГц.

Проблемы, возникающие в связи с изменением частоты тактового сигнала, могут быть довольно тонкими. Чтобы получить представление о том, что может нарушиться, полезно посмотреть, как будет работать (или не работать!) синхронизирующее устройство, если одну из тактовых частот изменить в 10 раз или более.

Что произойдет, например, с временными диаграммами, представленными на рис. 8.107, если мы изменим частоту сигнала RCLK и сделаем ее равной не 100 МГц, а 10 МГц? На первый взгляд кажется, что все будет хорошо, поскольку теперь байт поступает раз в 800 нс и имеется много больше времени для переписывания его в часть схемы, работающую с тактовым сигналом SCLK. Верно: неравенства (8.1) и (8.2) в данном случае удовлетворяются с много большим запасом. Однако неравенство (8.3) более не выполняется, если только мы не уменьшим значение n до нуля! Это можно было бы исправить, вырабатывая сигнал SYNC на один такт позднее сигнала RCLK, нежели это показано на рис. 8.107.

Но даже при таком изменении некоторая проблема все же остается. На рис. 8.110 приведены новые временные диаграммы, в том числе для вырабатываемого позднее сигнала SYNC. Проблема заключается в том, что теперь длительность импульса SYNC равна 100 нс. Как и ранее, сигнал NEWBYTE (на выходе SR-защел-

ки в схеме на рис. 8.106) принимает активное значение по сигналу SYNC и сбрасывается сигналом SLOAD, но когда сигнал SLOAD заканчивается, сигнал SYNC все еще остается на активном уровне, как это видно из новых временных диаграмм. Следовательно, новый байт будет обнаружен и передан далее дважды!

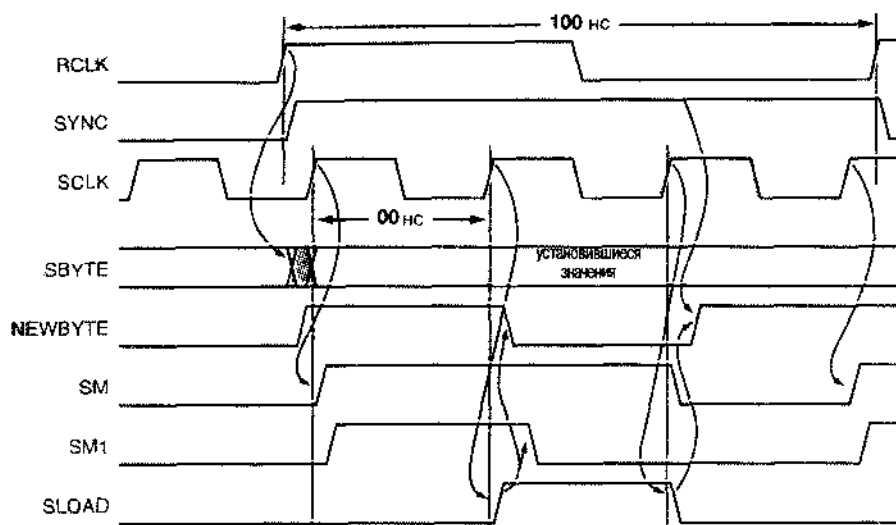


Рис. 8.110. Временные диаграммы для синхронизирующего устройства в случае медленного тактового сигнала (с частотой 10 МГц)

Решение этой проблемы состоит в том, чтобы реагировать только на нарастающий фронт сигнала SYNC, и тогда схема окажется нечувствительной к длительности импульса SYNC. В общем случае это делается путем замены SR-защелки переключателем, переключающимся по фронту D-триггером, как показано на рис. 8.111. По нарастающему фронту сигнала SYNC триггер устанавливается в единичное состояние, а сигналом SLOAD, как и ранее, осуществляется асинхронный сброс.

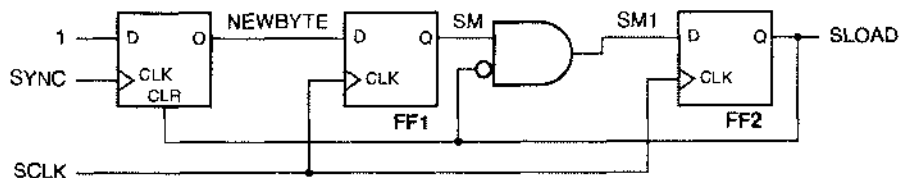


Рис. 8.111. Синхронизирующее устройство с обнаружением сигнала SYNC посредством переключения по его фронту

Приведенная на рис. 8.111 схема позволяет решить проблему, возникающую при слишком медленном сигнале RCLK, но при этом изменяются также выкладки, приводящие к соотношениям (8.1)–(8.3), результатом чего могут стать временные ограничения в каких-то других местах (см. задачу 8.96). Еще один недостаток последней

схемы состоит в том, что ее нельзя реализовать в типичном ПЛУ, у которого все триггеры переключаются одним и тем же тактовым сигналом; поэтому для обнаружения сигнала SYNC необходимо воспользоваться отдельным триггером.

Прочтя почти десять страниц, посвященные анализу всего лишь одного «простого» примера, вы, по-видимому, получили представление о том, как трудно правильно сконструировать синхронизирующее устройство. Вот несколько правил, которые могут вам помочь:

- Минимизируйте число подсистем, работающих с различными тактовыми сигналами.
- Четко определите границы между всеми тактовыми сигналами и в явном виде поместите на этих границах синхронизирующие устройства.
- Обеспечьте для каждого синхронизирующего устройства достаточное время выхода из метастабильности, чтобы сбои синхронизирующих устройств были редкими и происходили с много меньшей вероятностью, чем возникновение неисправности в других местах.
- Проанализируйте работу синхронизирующего устройства при различных возможных сдвигах сигналов во времени, в том числе при более быстрых и более медленных тактовых сигналах, которые могут подаваться на схему при моделировании или при модернизации системы.
- Осуществите моделирование работы системы в широком диапазоне возможных временных соотношений между сигналами.

Последнее правило может оказаться ловушкой для тех разработчиков, кто полагается на современные мощные и быстродействующие средства моделирования при поиске своих ошибок. Само по себе моделирование не может избавить от необходимости следования предыдущим четырем правилам. Если игнорировать эти правила, то можно столкнуться с проблемами, которые не обнаруживаются моделированием в типичных случаях, когда перебирается небольшое число вариантов соотношений между сигналами. Из всех цифровых схем синхронизирующие устройства являются такими конструкциями, для которых важнее всего быть «правильным по идее»!

Обзор литературы

По-видимому, первым источником подробных сведений о последовательных ИС средней степени интеграции стал *Справочник по применению TTL* под редакцией Алфке и Ларсена (*The TTL Applications Handbook*, edited by Peter Alfke and Ib Larsen. Fairchild Semiconductor, 1973). Эта очень полезная и содержательная книга была неоценимым подспорьем для автора и для многих других, чья деятельность на поприще цифрового проектирования начиналась в 70-е годы.

В книге Блейксли *Цифровое проектирование на основе стандартных МИС и БИС* (Thomas R. Blakeslee. *Digital Design with Standard MSI and LSI*, second edition. Wiley, 1979) упор был сделан на реализацию комбинационной и последовательной логики с помощью микросхем большей степени интеграции. Книга содержит блестящее изложение вопроса о пространственно-временном обмене, а ее автор был одним из первых, кто ввел представление о микропроцессоре как об «универсальной логической схеме».

Замечательным источником информации о практике проектирования цифровых устройств служат Web-сайты, позволяющие быстро ознакомиться с тем, что имеется на рынке ИС, от почти забытых производителей до тех, кого еще предстоит открыть. Например, исчерпывающее рассмотрение шинных фиксаторов уровня можно найти на сайте www.ti.com фирмы Texas Instruments в материале «Плавающие КМОП-входы и медленные сигналы на входах КМОП-схем» ("Implications of Slow or Floating CMOS Inputs", publ. SCBA004B, December 1997). Эти вопросы обсуждаются также в материале «Схемы с шинными фиксаторами уровня» ("Designing with Bushold", publ. AN-5006, April 1999) на сайте www.fairchildsemi.com фирмы Fairchild Semiconductor.

В Интернете можно найти также сообщения о всевозможных новых или модернизированных микросхемах средней и большей степени интеграции и их технические характеристики. Следуя лозунгу некоторых автомобильных компаний «Чем шире, тем лучше!», провозглашенному в 60-е годы и вновь появившемуся в конце 90-х годов, производители логических микросхем также падали выпуск регистров, формирователей и приемопередатчиков «с широкой шиной» в корпусах, предназначенных для поверхностного монтажа, с большим числом выводов; число разрядов у этих ИС может равняться 16, 18 или даже 32. Описание микросхем такого рода можно найти на Web-сайте фирмы Texas Instruments, осуществляя поиск по ключевому слову "widebus". Разнообразные сведения о технических характеристиках схем, об их применении и другую информацию вы найдете также на сайтах фирмы Motorola (www.mot.com), Fairchild Semiconductor (www.fairchildsemi.com) и Philips Semiconductor (www.philipslogic.com).

Развитие в мире цифровой электроники происходит настолько быстро, что у меня иногда возникает желание начать писать романы и повести; тогда мне не нужно было бы каждые несколько лет переписывать заново те разделы этой книги, где речь идет о практической разработке цифровых устройств. К счастью, значительное место в моей книге занимают теоретические вопросы, не претерпевающие изменений («принципы»), и данная глава не является исключением. Об источниках опасности в логических схемах известно, по крайней мере, с 50-х годов; функциональные источники опасности рассмотрены в книге Мак-Класки *Принципы логического проектирования* (Edward J. McCluskey. *Logic Design Principles*. Prentice Hall, 1986). Копечные поля были придуманы почти два столетия назад; с их применением в кодах, исправляющих ошибки, а также в теории регистров сдвига с линейной обратной связью можно ознакомиться по книгам, служащим введением в теорию кодирования, в частности, по книге Майкельсона и Левека *Методы контроля ошибок в цифровой связи* (A. M. Michelson and A. H. Levesque. *Error-Control Techniques for Digital Communication*. Wiley-Interscience, 1985). В течение ряда лет развивается математическая теория разбиения конечных автоматов; этой теме посвящена глава в классической книге Кохави *Теория переключений и конечные автоматы* (Zvi Kohavi. *Switching and Finite Automata Theory*. second edition. McGraw-Hill, 1978). Но давайте вернемся к более прозаическим вещам.

Как упоминалось в разделе 8.4.5, некоторые ПЛУ и ИС типа CPLD содержат структуры ИСКЛЮЧАЮЩЕЕ ИЛИ, позволяющие создавать многоразрядные счетчики, обходясь небольшим числом термов-произведений. Для этого требуется несколько более глубокий анализ уравнений возбуждения счетчика, приведен-

ный в параграфе 10.5 во втором издании данной книги. Этот материал можно найти также на Web-сайте www.ddpp.com.

Разброс задержек тактового сигнала и многофазное тактирование в самом общем виде рассмотрены в книге Мак-Класки *Принципы логического проектирования*. Поучительное рассмотрение этих вопросов применительно к схемам на основе СБИС можно найти в книге Мида и Конвея *Введение в системы СБИС* (Carver Mead and Lynn Conway, *Introduction to VLSI Systems*. Addison-Wesley, 1980). Авторы этой книги вводят также важное понятие *самосинхронизирующихся систем* (*self-timed systems*), в которых нет системного тактового сигнала и каждому логическому элементу позволено функционировать в своем темпе. Отдавая должное Миду и Конвею, заметим все же, что весь материал, относящийся к синхронизации, включая знаменитое обсуждение метастабильности, находится в главе 7, написанной Сейтцом (Charles L. Seitz).

Чейни (Thomas J. Chapey) потратил десятилетия на изучение проблемы метастабильности, и у него много публикаций на эту тему. В одной из его наиболее значительных работ «Экспериментальное исследование реакции триггера на переключение в предельных условиях» («Measured Flip-Flop Responses to Marginal Triggering». *IEEE Trans. Comput.*, Vol. C-32, No. 12, December 1983, pp. 1207-1209) приводятся некоторые из результатов, представленных в табл. 8.35.

Работа Клеемана и Кантони «О неизбежности метастабильности в цифровых системах» (Lindsay Kleeman and Antonio Cantoni, «On the Unavoidability of Metastable Behavior in Digital Systems». *IEEE Trans. Comput.*, Vol. C-36, No. 1, January 1987, pp. 109-112) рассчитана на читателей с математическими склонностями; ее название говорит само за себя. Те же авторы ставят вопрос: «Можно ли за счет избыточности и маскирования улучшить характеристики синхронизирующих устройств?» («Can Redundancy and Masking Improve the Performance of Synchronizers?» *IEEE Trans. Comput.*, Vol. C-35, No. 7, July 1986, pp. 643-646); в указываемой статье они дают ответ: «Нет», но отзыв рецензента заставил их изменить свою точку зрения на «Может быть». Скорее всего, они сами пребывали в состоянии метастабильности! В дальнейшем же авторы, по рецензенту не конкретизировали своей точки зрения, поэтому очевидный ответ на заданный вопрос: «Нет»! В любом случае, статьи Клеемана и Кантони содержат хороший перечень ссылок на основные работы по метастабильности.

Самый полный указатель на ранние исследования в области метастабильности (правда, без упоминания греческих философов и группы Devo) имеется в работе Болтона «Экскурсия по 35 годам изучения метастабильности» [Martin Bolton, «A Guided Tour of 35 Years of Metastability Research». *Proc. Wescon 1987*, Session 16, «Everything You Might Be Afraid to Know about Metastability» («Все, что вы, возможно, опасаетесь узнать о метастабильности»). *Wescon Session Records*, www.wescon.com, 8110 Airport Blvd., Los Angeles, CA 90045].

В последние годы, по мере постоянно возрастающей частоты системного тактового сигнала, многие производители ИС стали значительно добросовестнее измерять характеристики метастабильности выпускаемых ими изделий и сообщать результаты измерений, чаще всего – через Интернет. Подробное рассмотрение этих вопросов, включая схемы тестирования и измеряемые параметры, для десяти различных логических семейств имеется в работе Хазеллоффа «Метастабильное пове-

дение 5-вольтовых логических схем» (Eilhard Haseloff. "Metastable Response in 5-V Logic Circuits") на сайте фирмы Texas Instruments (www.ti.com, TI pub. SDYA006, 1997). Фирма Cypress Semiconductor в 1997 году опубликовала на своем сайте (www.cypress.com) специальный обзор, озаглавленный «Является ли метастабильным ваше ПЛУ?»; это отличный материал, содержащий немного истории (начиная с 1952 года!), анализ метастабильности аналоговых устройств, схемы тестирования и измерений и параметры метастабильности для выпускаемых этой фирмой ПЛУ. Другая недавняя публикация – «Обсуждение метастабильности» фирмы Xilinx ("Metastability Considerations", www.xilinx.com, publ. XAPP077, 1997), где приводятся измеренные значения параметров для ИС типа CPLD семейств XC7300 и XC9500. Особый интерес представляют остроумные схема и методология, позволяющие регистрировать наступление метастабильности внутри устройства даже в том случае, когда метастабильные сигналы нельзя наблюдать на внешних выводах.

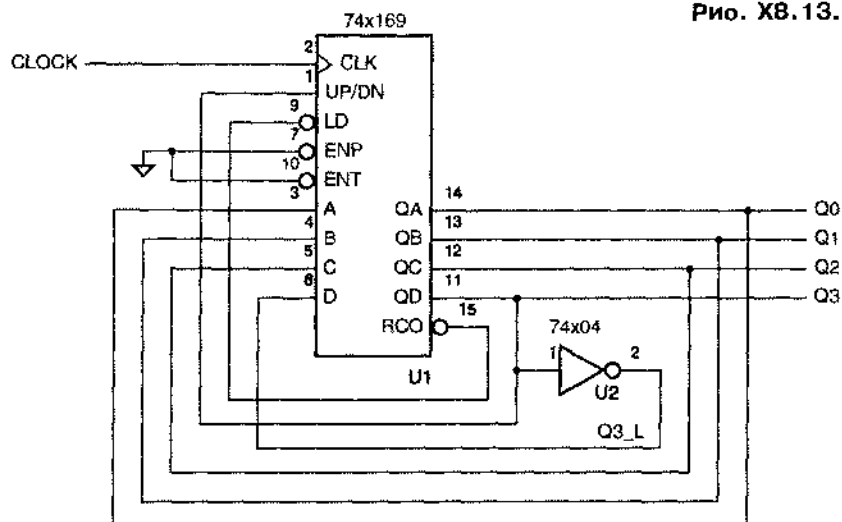
Сегодня в большинстве учебников по цифровому проектированию метастабильность рассматривается довольно подробно. К этому подталкивает существование метастабильности в реальных схемах, а также, возможно, состоящий неопределенности. В предыдущих изданиях учебника, который вы читаете, автор, начиная с 1990 года, вводит представление о метастабильности на самой ранней стадии рассмотрения последовательностных схем. Отличным введением в анализ метастабильных состояний с аналоговой точки зрения служит книга Джонсона и Грэхема *Проектирование быстродействующих цифровых устройств: Справочник черной магии* (Howard Johnson and Martin Graham. *High-Speed Digital Design: A Handbook of Black Magic*. Prentice Hall, 1993); там же объясняется, как наблюдать метастабильные состояния.

Упражнения

- 8.1. Предположим, что обращение ко второму блоку ОЗУ (RAMCS1) в табл. 8.2 происходит при выполнении условия $((ABUS \geq 0x010) \& (ABUS < 0x020))$. Объясните, даст ли это тот же самый результат, что и выражение $(ABUS == RAMBANK1)$ в исходном тексте программы.
- 8.2. Найдите число программируемых соединений в каждом ПЛУ, упомянутом в табл. 8.9.
- 8.3. Сколько программируемых соединений имеется в ИС 16V8 согласно сказанному в тексте? (У ИС гражданского применения есть дополнительные программируемые соединения для записи пользователем собственной информации и обеспечения безопасности.)
- 8.4. Сколько программируемых соединений имеется в ИС 22V10 согласно сказанному в тексте? (У ИС гражданского применения есть дополнительные программируемые соединения для записи пользователем собственной информации и обеспечения безопасности.)
- 8.5. Найдите значение f_{max} для всех ИС, перечисленных в табл. 8.10, при внешней обратной связи.
- 8.6. Найдите значение f_{max} для всех ИС типа GAL, перечисленных в табл. 8.10, при внутренней обратной связи.

- 8.7. Напишите программу на языке ABEL для устройства, реализуемого на ПЛУ 16V8, которое по своим функциональным возможностям было бы идентично ИС 74х374.
- 8.8. Напишите программу на языке ABEL для устройства, реализуемого на ПЛУ GAL 16V8 или GAL 20V8, которое по своим функциональным возможностям было бы идентично ИС 74х377.
- 8.9. Сравните между собой задержки распространения сигнала от входа AVALID до выхода выбора кристалла у двух схем декодирования, приведенных на рис. 8.15 и 8.16. Предположите, что в обоих случаях используются защелки 74FCT373 и 10-наносекундные ПЛУ 16V8С. Повторите упражнение для задержки от входа ABUS до выхода выбора кристалла.
- 8.10. Что произойдет, если в схеме на рис. 7.38 переключающиеся по фронту D-триггеры заменить D-защелками?
- 8.11. Перепишите приведенную в табл. 8.12 программу на языке ABEL так, чтобы устройство выполняло функции декадного счетчика 74х162.
- 8.12. Перепишите приведенную в табл. 8.12 программу на языке ABEL так, чтобы устройство выполняло функции реверсивного счетчика 74х169. Достаточно ли для этого возможностей ПЛУ 16V8?
- 8.13. В какой последовательности считает схема, изображенная на рис. X8.13.

Рис. X8.13.



- 8.14. Как будет вести себя счетчик, приведенный на рис. 8.36, если вместо ИС 74х163 включить микросхему 74х161?
- 8.15. Как будет вести себя счетчик, приведенный на рис. 8.36, если нижний вход элемента U2 соединить с QA, а не с QB?
- 8.16. На входы ENP, ENT, A и D счетчика 74х163 постоянно подан высокий уровень, а на входы B и C – низкий. На входах загрузки и сброса действуют

Задачи

- 8.22. Что должно быть сказано в справочнике по ТТЛ-схемам о кратковременном замыкании на землю выходов вентилях, применяемых в схеме защиты от дребезга, показанной на рис. 8.5?
- 8.23. Исследуйте работу схемы защиты от дребезга, показанной на рис. 8.5, если в ней используются инверторы из ИС 74НСТ04; повторите эту задачу для инверторов из ИС 74АС04.
- 8.24. Пусть от вас требуется построить схему, вырабатывающую логический входной сигнал без дребезга от однополюсного переключателя на *одно направление* [single-pole, single-throw (SPST) switch]. С какой проблемой вы при этом столкнетесь?
- 8.25. Объясните, почему шинный фиксатор уровня на КМОП-схемах не будет хорошо работать на шинах с тремя состояниями, к которым подключены ТТЛ-устройства. (Указание: Учтите входные характеристики ТТЛ-схем.)
- 8.26. Напишите единую VHDL-программу для устройства, в котором фиксация и декодирование адреса совмещены, согласно схеме на рис. 8.16 и по принципу, реализованному в табл. 8.2. Назовите сигналы на выходе схемы, запоминающей адрес, LA[19:0].
- 8.27. Постройте 4-разрядный счетчик со сквозным переносом на четырех D-триггерах без использования других компонентов.
- 8.28. Чему равняется максимальная задержка распространения от тактового входа до выхода в 4-разрядном счетчике со сквозным переносом из задачи 8.27, если счетчик построен на триггерах 74НСТ? Повторите эту задачу для триггеров 74АНСТ и 74LS74.
- 8.29. Постройте 4-разрядный счетчик, считающий *в обратном направлении*, на четырех D-триггерах без использования других компонентов.
- 8.30. Что ограничивает максимальную скорость счета в счетчике со сквозным переносом, если нет необходимости в том, чтобы результат счета можно было считывать в любой момент времени?
- 8.31. Для схемы счетчика из задачи 8.27 и с учетом ответа на вопрос, заданный в задаче 8.30, найдите максимальную скорость счета (частоту) для 4-разрядного счетчика со сквозным переносом на триггерах 74НСТ. Повторите эту задачу для триггеров 74АНСТ и 74LS74.
- 8.32. Напишите формулу для максимальной частоты тактового сигнала последовательного синхронного двоичного счетчика, схема которого приведена на рис. 8.28. Пусть в вашей формуле t_{TQ} означает задержку распространения от входа T до выхода Q в каждом T-триггере, t_{setup} – время установления на входе EN по отношению к нарастающему фронту сигнала T, а t_{AND} – задержку в вентиле И.
- 8.33. Повторите задачу 8.32 для параллельного синхронного двоичного счетчика, схема которого приведена на рис. 8.29, и сравните результаты этих двух задач.

- 8.34. Повторите задачу 8.32 для n -разрядного синхронного последовательного двоичного счетчика.
- 8.35. Повторите задачу 8.32 для n -разрядного синхронного параллельного двоичного счетчика. При каких значениях n ваша формула уже не будет справедлива?
- 8.36. Воспользовавшись 4-разрядным двоичным счетчиком 74х163, настройте счетчик по модулю 11 с последовательностью счета: 3, 4, 5, ..., 12, 13, 3, 4,
- 8.37. Найдите в справочнике принципиальную схему синхронного декадного счетчика 74х162 и составьте для него таблицу состояний в духе табл. 8.11, отразив в ней, в частности, поведение счетчика при попадании в нормально неиспользуемые состояния 10–15.
- 8.38. Постройте синхронный счетчик путем последовательного включения ИС 74х163 с параллельной логикой разрешения, как в схеме на рис. 8.29, так, чтобы максимальная скорость счета в нем была такой же, как у любой из этих микросхем, при числе разрядов, достигающем до 36 (девять ИС '163). Определите максимальную скорость счета, воспользовавшись сообщаемыми производителем значениями задержек в наихудшем случае для ИС '163 и для компонентов малой степени интеграции, применяемых в межкаскадных соединениях.
- 8.39. Постройте счетчик по модулю 129 на двух ИС 74х163 и одном инверторе.
- 8.40. Напишите программу на языке ABEL для 8-разрядного счетчика по модулю N с входом загрузки в расчете на реализацию в ПЛИС PAL 22V10; модуль счета N пусть задается в программе постоянной N .
- 8.41. Постройте тактируемую синхронную схему с четырьмя входами N_3, N_2, N_1 и N_0 ; пусть сигналами на этих входах представлено число N из интервала 0–15. У схемы имеется единственный выход Z , сигнал на котором принимает единичное значение точно на N тактах в пределах интервала длительностью 16 тактов (предполагается, что значение N остается постоянным на интервале наблюдения). (Указания: Воспользуйтесь комбинационной логикой и включите ИС 74х163 так, чтобы получился делитель частоты на 16, работающий в непрерывном режиме. Такты, на которых сигнал Z имеет единичное значение, должны быть возможно равномернее распределены в пределах интервала наблюдения; другими словами, – на каждом втором такте при $N = 8$, на каждом четвертом такте при $N = 4$ и т.д.)
- 8.42. Видоизмените схему из задачи 8.41 таким образом, чтобы на выходе Z вырабатывалось N переходов в интервале длительностью 16 тактов. Схему, которая у вас получится, называют *двоичным умножителем частоты* (*binary rate multiplier*); в свое время продавалась такая ТТЛ-схема средней степени интеграции 7497. (Указание: Стробите тактовый сигнал выходным сигналом схемы из предыдущей задачи.)
- 8.43. Повторите задачи 8.41 и 8.42 для случая, когда имеется 8-разрядный вход $N_7 \dots N_0$, написав программу на языке ABEL, предусматривающую реализацию каждой из схем в одном ПЛИС типа PAL22V10.

- 8.44. Повторите задачи 8.41 и 8.42 с 8-разрядным входом $N7 \dots N0$, описав каждую схему новежденческой VHDL-программой.
- 8.45. В последнюю минуту к разработчику (к автору!) обратились с просьбой дополнить возможности устройства еще одной функцией, когда на печатной плате оставалось место только для одной ИС средней степени интеграции с 16 выводами. На плате уже имелся 16-мегагерцный тактовый сигнал MCLK и свободный сигнал выбора SEL, поступающий от микропроцессора. От разработчика требовалось, чтобы вырабатывался новый тактовый сигнал UCLK с частотой 8 МГц или 4 МГц в зависимости от значения SEL. Хуже всего, что на плате не было свободных вентилях в ИС малой степени интеграции и требовалось, чтобы коэффициент заполнения у сигнала UCLK был равен 50% на обеих частотах. Разработчик придумал схему за пять минут. Теперь ваша очередь сделать то же самое. (Указание: К моменту решения этой задачи разработчик был убежден, что основным «строительным элементом» для «хитрых» последовательностных схем является ИС 76х163.)
- 8.46. Постройте счетчик по модулю 16 с последовательностью счета 7, 6, 5, 4, 3, 2, 1, 0, 8, 9, 10, 11, 12, 13, 14, 15, 7, ... на одной ИС 74х169 с использованием самое большее одного корпуса ИС малой степени интеграции.
- 8.47. Напишите программу на языке ABEL для 8-разрядного счетчика, который осуществляет счет в последовательности, подобной той, которая указана в задаче 8.46.
- 8.48. Постройте реверсивный двоичный счетчик для контроллера лифта в 20-этажном здании на одной ИС 16V8. У счетчика должны быть входы разрешения счета и управления направлением счета. Счет должен останавливаться на состоянии 1 при счете в обратном направлении и на состоянии 21 при счете в прямом направлении, а также пропускать состояние 13 в обоих режимах работы. Начертите принципиальную схему вашего устройства и напишите для него равенства на языке ABEL.
- 8.49. Повторите предыдущую задачу, воспользовавшись языком VHDL.
- 8.50. Напишите VHDL-программу для n -разрядного счетчика, который осуществляет счет в последовательности, подобной той, которая указана в задаче 8.46. Напишите программу таким образом, чтобы число разрядов в счетчике можно было изменять путем изменения значения единственной постоянной N .
- 8.51. Видоизмените VHDL-программу в табл. 8.14 так, чтобы порты D и Q были типа STD_LOGIC_VECTOR, включив функции преобразования там, где это необходимо.
- 8.52. Перепишите VHDL-программу из табл. 8.16 в структурном стиле так, чтобы описываемое ею устройство точно соответствовало схеме на рис. 8.45, включая имена сигналов, указанные на рисунке. Если в вашей VHDL-библиотеке нет следующих объектов: AND2, INV, NOR2, OR2, XNOR2 и Vdffqqn, то определите их и используйте по мере необходимости.
- 8.53. Видоизмените программу из табл. 8.17, воспользовавшись VHDL-оператором generic так, чтобы число разрядов счетчика можно было изменять посредством определения generic.

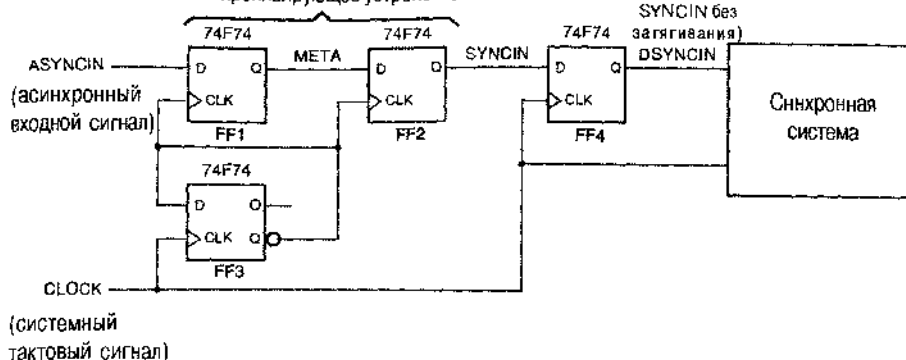
- 8.54.** Постройте схему параллельно-последовательного преобразования с восемью 32-канальными последовательными звеньями со скоростью передачи 2.048 Мбит/с и одной 8-разрядной параллельной шиной данных с частотой тактового сигнала 2.048 МГц, по которой передается 256 байтов в кадре. Формат кадра в каждом из последовательных звеньев должен иметь вид, указанный на рис. 8.55. У каждой последовательной линии передачи данных $SDATA_i$ пусть имеется ее собственный сигнал синхронизации $SYNC_i$, а импульсы этих сигналов разнесены таким образом, что импульс сигнала $SYNC_{(i+1)}$ сдвинут по отношению к импульсу сигнала $SYNC_i$ на один такт позднее.
- 8.55.** Рассмотрите временные соотношения между сигналами в параллельной шине и в последовательных звеньях и составьте таблицу или напишите формулу, показывающие, какие таймслоты параллельной шины передаются по тому или иному последовательному звену в пределах времени, отведенного на передачу соответствующего таймслота. Нарисуйте принципиальную схему устройства, построенного на ИС средней степени интеграции, упомянутых в этой главе; вы можете в сокращенном виде изобразить повторяющиеся элементы (например, регистры сдвига), указав для каждого из них только такие соединения, которые относятся индивидуально к данному элементу.
- 8.56.** Повторите задачу 8.54 в предположении, что все последовательные линии передачи данных должны привязывать свои данные к одному общему сигналу синхронизации $SYNC$. Сколько микросхем потребуется дополнительно в этом случае?
- 8.57.** Покажите, как следует нарастить схему последовательно-параллельного преобразования, приведенную на рис. 8.57, чтобы байт, принятый в каждом слоте, запоминался в своем собственном регистре на 125 мкс, то есть до тех пор, пока не будет принят следующий байт в этом таймслоте. Нарисуйте подробную схему со счетчиком и логикой декодирования для 32 таймслотов, а также регистры, в которых запоминаются параллельные данные, и соединения для таймслотов 31, 0 и 1. Начертите также временные диаграммы, — примерно так, как это сделано на рис. 8.58, — где были бы показаны сигналы декодирования и данные для слотов 31, 0 и 1.
- 8.58.** Представьте себе, что вам нужно построить последовательный компьютер, в котором данные передаются и обрабатываются по одному биту за раз. Первое, что вам надо будет решить, это — какой бит передавать и обрабатывать первым: младший или старший. Какой бы вы выбрали и почему?
- 8.59.** Постройте 8-разрядный самокорректирующийся кольцевой счетчик с состояниями 1111110, 1111101, ..., 0111111, не используя только два корпуса ИС малой или средней степени интеграции.
- 8.60.** Предложите две различные схемы 2-разрядного счетчика с 4 состояниями, для каждой из которых было бы достаточно одного корпуса ИС 74х74 (два переключающихся по фронту D-триггера) без использования других логических схем.

- 8.61. Постройте 4-разрядный счетчик Джонсона и схему декодирования всех восьми состояний, воспользовавшись только четырьмя триггерами и восемью вентилями. От вашего счетчика не требуется, чтобы он был самокорректирующимся.
- 8.62. Докажите, что в случае, когда n -разрядный регистр сдвига генерирует последовательность максимальной длины, ко входам схемы проверки на четность должно быть подключено четное число выходов регистра сдвига. (Заметьте, что это необходимое, но не достаточное условие. Кроме того, табл. 8.21 согласуется с тем, что вам предлагается доказать, но простая ссылка на эту таблицу не является доказательством!)
- 8.63. Докажите, что сигнал X_0 должен фигурировать в правой части любого уравнения обратной связи, при котором регистр сдвига с данной обратной связью генерирует последовательность максимальной длины. (Замечание: Предположите, что порядок следования битов в регистре сдвига и направление сдвига такие, как указано в тексте; другими словами, сдвиг в регистре производится вправо, в сторону разряда X_0 .)
- 8.64. Предположите, что n -разрядный регистр сдвига с линейной обратной связью построен по схеме, приведенной на рис. 8.68, в соответствии с табл. 8.21. Докажите, что если заменить схему проверки на четность схемой проверки на четность, то результирующая схема будет счетчиком, который проходит через $2^n - 1$ состояний, то есть через все возможные состояния за исключением $11 \dots 11$.
- 8.65. Найдите уравнение обратной связи для 3-разрядного LFSR-счетчика, при котором схема генерирует последовательность максимальной длины, отличное от указанного в табл. 8.21.
- 8.66. Пусть задан n -разрядный LFSR-счетчик, генерирующий последовательность максимальной длины (проходящий через $2^n - 1$ состояний). Докажите, что при подключении еще одного вентиля ИСКЛЮЧАЮЩЕЕ ИЛИ и ($n - 1$)-входного вентиля ИЛИ-НЕ так, как показано на рис. 8.69, получится схема с 2^n состояниями.
- 8.67. Докажите, что последовательность из 2^n состояний получится также и в том случае, если вентиль ИЛИ-НЕ, упомянутый выше, заменить вентилем И-НЕ, хотя это будет другая последовательность состояний.
- 8.68. Постройте итерационную схему проверки на четность для 16-разрядного слова данных с одним проверочным битом. Имеет ли значение порядок передачи битов?
- 8.69. Видоизмените программу из табл. 8.23 так, чтобы описываемый ею регистр сдвига имел асинхронный вход сброса при реализации на ПЛУ 22V10.
- 8.70. Напишите программу на языке ABEL для устройства, которое выполняло бы те же функции, что и регистр сдвига 74x299. Покажите, как разместить это устройство в одной ИС 22V10, или объясните, почему этого сделать нельзя.
- 8.71. Найдите число термов-произведений, которое требуется для каждого выходного сигнала в устройстве RING8 из табл. 8.25. Размещается это устройство в ПЛУ 16K8 или 16V8R?

- 8.72. В каких ситуациях программы на языке ABEL из табл. 8.26 и 8.27 будут приводить к синтезу устройств, работающих но-разному?
- 8.73. Видоизмените программу на языке ABEL из табл. 8.26 таким образом, чтобы длительность фаз всегда была равной по меньшей мере двум тактам, даже в том случае, когда сигнал RESTART поступает в начале фазы. Сигнал RESET пусть действует все же немедленно.
- 8.74. Повторите предыдущую задачу для программы из табл. 8.27.
- 8.75. Предположите, что генератор многофазных колебаний из табл. 8.26 применяется для управления динамической памятью, так что для чтения из памяти или записи в нее необходимо, чтобы последовательно выполнялись все шесть фаз. Если генератор колебаний будет сбрасываться во время операции чтения без прохождения через все шесть фаз, то содержимое памяти может портиться. Видоизмените равенства в табл. 8.26 таким образом, чтобы избежать этого.
- 8.76. Студенту предложено построить генератор колебаний, изображенных на рис. 8.72, взяв в качестве отправной точки программу на языке ABEL из табл. 8.27 и изменив кодирование каждого из состояний P1F, P2F, ..., P6F таким образом, чтобы на соответствующем фазовом выходе вырабатывалась 1, а не 0, и сигнал на фазовом выходе равнялся 0 только в пределах второго такта в каждой фазе. Правильен ли такой подход? Прокомментируйте результаты, которые выдает компилятор языка ABEL, когда вы пытаетесь сделать это.
- 8.77. Выходные сигналы устройств, описываемых программами в табл. 8.29 и 8.30, не совпадают при изменении сигналов RESTART и RUN. Объясните причину этого, а затем видоизмените программу из табл. 8.30 так, чтобы поведение описываемого ею устройства соответствовало тому, что делает устройство, описываемое программой из табл. 8.29.
- 8.78. Реализация кольцевого счетчика по программе из табл. 8.26 не дает самокорректирующейся схемы. Посмотрите, например, что случится, если выходным сигналам {P1_L, ..., P6_L} первоначально будет присвоено значение 0 и входной сигнал RUN примет единичное значение без подачи когда-либо сигналов на входы RESET и RESTART. При каких других начальных состояниях схема ведет себя подобным образом и самокоррекция не происходит? Видоизмените программу так, чтобы счетчик стал самокорректирующимся.
- 8.79. Повторите предыдущую задачу для счетчика, синтезируемого по VHDL-программе из табл. 8.33.
- 8.80. Постройте итерационную схему с одним входом V_i в каждом каскаде и с двумя граничными выходами X и Y , таким, что $X = 1$, если по меньшей мере два входных сигнала V_i равны 1, а выход $Y = 1$, если по меньшей мере на двух соседних входах сигналы V_i равны 1.
- 8.81. Постройте автомат для управления кодовым замком с таблицей состояний, приведенной в табл. 7.14, на одном счетчике 74х163 с комбинационной логикой на входах LD_L, CLR_L и A-D этой ИС. Пусть состояниям A-H соответствуют числа 0-7 в счетчике.

- 8.82.** Напишите программу на языке ABEL, соответствующую диаграмме состояний, показанной на рис. 8.84, для конечного автомата, управляющего умножителем.
- 8.83.** Напишите VHDL-программу, соответствующую диаграмме состояний, показанной на рис. 8.84, для конечного автомата, управляющего умножителем.
- 8.84.** Напишите VHDL-программу для устройства, которое имело бы такие же входы и выходы, что и блок обработки данных в умножителе, представленный на рис. 8.82, и выполняло бы те же функции.
- 8.85.** Напишите VHDL-программу, в которой были бы объединены программы из двух предыдущих задач и которая в целом описывала бы законченный 8-разрядный умножитель, действующий по принципу сдвига и сложения.
- 8.86.** В тексте говорится, что разработчику не нужно беспокоиться по поводу временных соотношений между сигналами в синхронной схеме, приведенной на рис. 8.83. На самом деле, есть одна небольшая причина для беспокойства. Посмотрите на временные параметры ИС 74х377 и догадайтесь, в чем тут дело.
- 8.87.** Найдите минимальный период тактового сигнала для схемы умножителя, действующего по принципу сдвига и сложения, на рис. 8.83 в предположении, что конечный автомат реализован на одной ИС GAL 16V8-20 и что все компоненты средней степени интеграции взяты из ТТЛ-семейства 74LS. Используйте информацию о временных параметрах в худшем случае из таблиц, имеющихся в этой книге. Для ИС '194 задержка t_{pd} от тактового входа до любого выхода равна 26 нс, а значение t_s равно 20 нс для последовательных и параллельных входов данных и 30 нс для входов, на которые подаются сигналы, управляющие режимом работы.
- 8.88.** Разработайте блок обработки данных и управляющий конечный автомат для перемножения 8-разрядных чисел, представленных в дополнительном коде, согласно алгоритму, описанному в параграфе 2.8.
- 8.89.** Разработайте блок обработки данных и управляющий конечный автомат для деления 8-разрядных чисел без знака согласно алгоритму сдвига и вычитания, описанному в параграфе 2.9.
- 8.90.** Предположите, что сигнал SYNCIN в упражнении 8.21 проходит в синхронной системе через комбинационную логическую схему и, в конце концов, поступает на D-входы триггеров 74ALS74, переключающихся тактовым сигналом CLOCK. Какова максимально допустимая задержка распространения сигнала в комбинационной логической схеме?
- 8.91.** В схеме на рис. X8.91 имеется триггер, устраняющий затягивание, так что синхронизированный сигнал появляется на выходе многотактного синхронизирующего устройства предельно близко к фронту сигнала CLOCK. Если не принимать во внимание метастабильность, то чему равна максимальная частота сигнала CLOCK? Считайте, что $t_{setup} = 5$ нс, а $t_{pd} = 7$ нс для ИС 74F74.
- 8.92.** Определите значение MTBF для синхронизирующего устройства из задачи 8.91, воспользовавшись найденной там максимальной частотой тактового сигнала и полагая, что частота переходов в асинхронном сигнале составляет 4 МГц.

Рис. X8.91. синхронизирующее устройство



- 8.93.** Определите значение МТБФ для синхронизирующего устройства, приведенного на рис. X8.91, предполагая, что частота переходов в асинхронном сигнале равна 4 МГц, а частота тактового сигнала – 40 МГц, что меньше максимальной частоты, найденной в задаче 8.91. В этих условиях «сбой синхронизирующего устройства» происходит только в том случае, когда сигнал DSYNIN оказывается метастабильным. Другими словами, можно допустить, чтобы сигнал SYNCIN был метастабильным в течение короткого времени, пока это не влияет на сигнал DSYNIN. В результате получается лучшее значение МТБФ.
- 8.94.** Посмотрите патент США № 4,999,528 «Триггер с защитой от метастабильности» и объясните, почему этот триггер не всегда работает так, как обещано. (Указания: Патенты можно найти на сайте www.patents.ibm.com. Для того чтобы понять, как может произойти сбой в этой схеме, достаточно информации, содержащейся в резюме данного патента.)
- 8.95.** В синхронизирующем устройстве, представленном на рис. 8.102, 8.104 и 8.106, можно уменьшить задержку переноса байта из части схемы, работающей с тактовым сигналом RCLK, в часть схемы, где нереклоуение происходит по тактовому сигналу SCLK, если запускать устройство опережающим импульсом SYNC. Представьте себе, что есть возможность вырабатывать этот импульс на любом бите принимаемого байта. Какой бы бит вы выбрали, чтобы минимизировать задержку? Пронерьте также, удовлетворяет ли ваше решение условиям правильной работы схемы в случае максимальной задержки. Считайте, что временные параметры у всех компонентов такие, как у логических схем семейства 74АНСТ, а SR-защелка построена на нарезе вентилей ИЛИ-НЕ с перекрестной связью. Проведите детальный временной анализ предлагаемого вами решения.
- 8.96.** В схеме управления синхронизирующего устройства, приведенной на рис. 8.106, вместо защелки иногда применяют переключающийся по фронту D-триггер, как показано на рис. 8.111. Выведите условия правильной работы этой схемы, эквивалентные неравенствам (8.1)–(8.3), для случаев максимальной и минимальной задержки и посмотрите, ослабевают или становятся более жесткими требования, предъявляемые к задержкам при таком подходе.

8.97. Известный проектировщик цифровых устройств изобрел схему, представленную на рис. X8.97(a), в которой, согласно предположению, метастабильность преодолевается за один период системного тактового сигнала. Узел M – это безынерционный аналоговый детектор напряжения, сигнал на выходе которого равен 1, если сигнал Q пребывает в метастабильном состоянии, и равен 0 в противном случае. Идея автора этой схемы состояла в следующем: если то обстоятельство, что сигнальная линия Q находится в метастабильном состоянии, обнаруживается тогда, когда сигнал CLOCK переходит на низкий уровень, то вентиль И-НЕ сбросит D-триггер, что, в свою очередь, исключит метастабильность выхода, вызовет появление 0 на выходе узла M, в результате чего сигнал на входе сброса D-триггера будет снят. Все компоненты в схеме достаточно быстрые для того, чтобы все описанное произошло задолго до того, как сигнал CLOCK перейдет на высокий уровень; ожидаемый вид сигналов приведен на рис. X8.97(b). К сожалению, это синхронизирующее устройство иногда все же дает сбой, и знаменитый разработчик теперь придумывает карманы для джинсов. Объясните подробно, как именно происходит сбой, нарисовав соответствующие временные диаграммы.

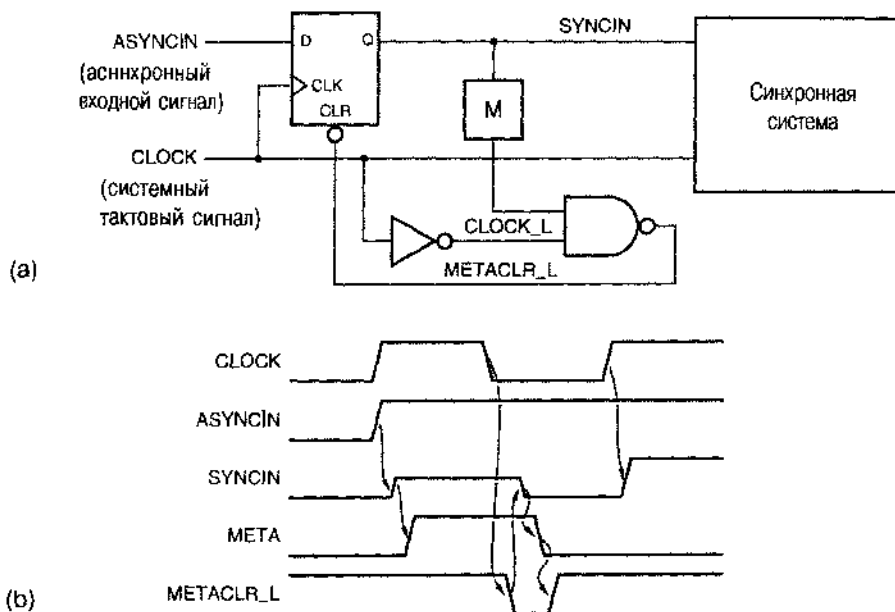
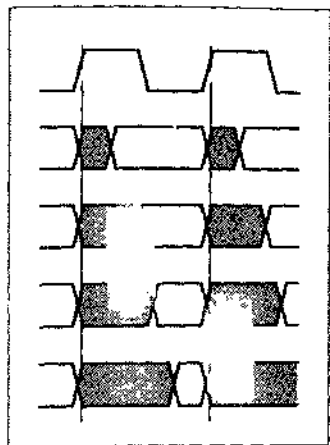


Рис. X8.97.



Глава

ПРИМЕРЫ ПРОЕКТИРОВАНИЯ ПОСЛЕДОВАТЕЛЬНО- СТНЫХ СХЕМ

Почти любая реальная цифровая система является последовательностной схемой: всегда имеется обратная связь, защелка или триггер, обеспечивающие зависимость поведения системы в данный момент от предыдущих входных воздействий. Но если бы мы стали разрабатывать или анализировать типичную цифровую систему как единую последовательностную схему, то все свелось бы к огромной таблице состояний. Например, компьютер с оперативной памятью объемом всего лишь 16 Мбайт является, строго говоря, последовательностной схемой с много большим числом состояний, чем $2^{128\,000\,000}$!

В предыдущих главах мы видели, что обычно цифровую систему можно разбить на блоки меньших размеров, выделяя, например, пути прохождения данных, регистры и управляющие устройства (см. раздел 8.7.1). Действительно, у типичной цифровой системы бывает несколько функциональных узлов с четко очерченными границами раздела и соединениями между ними (такое разбиение поддерживается средствами VHDL, см. раздел 4.7.2), а каждый функциональный узел содержит иерархию из нескольких слоев абстракции [поддерживаемую как средствами VHDL, так и схематическими редакторами, позволяющими графически изображать компоненты (см. раздел 5.1.6)]. Таким образом, всегда можно свести задачу к рассмотрению блоков значительно меньшего размера.

После всех этих разговоров я должен признать, что проектирование сложной иерархической цифровой системы выходит за рамки этого учебника. Но сердцем всякой системы или любой из ее подсистем является, как правило, конечный автомат или другая последовательностная схема, а это как раз то, что *мы можем* изучить здесь. Эта глава будет попыткой закрепить ваше понимание принципов проектирования последовательностных схем и конечных автоматов посредством разбора нескольких примеров.

На ранней стадии цифрового проектирования и даже на протяжении 80-х годов многие записывали таблицы состояний от руки и строили соответствующие схемы, используя методы синтеза, рассмотренные в параграфе 7.4. Однако сегодня едва ли кто-нибудь поступает таким образом. В настоящее время таблицы состоя-

ний, в большинстве случаев, задаются на том или ином языке описания схем: на языке ABEL, VHDL или Verilog. Затем компилятор этого языка выполняет работу, эквивалентную применению наших методов синтеза, и реализует требуемый автомат в ПЛУ, в ИС типа CPLD и FPGA, в специализированной ИС или в каком-то другом устройстве.

В этой главе мы рассмотрим примеры проектирования конечных автоматов и других последовательностных схем с помощью двух различных языков описания схем. В первом параграфе приводятся примеры на языке ABEL, имея в виду реализацию разрабатываемых устройств в небольших ПЛУ. Некоторые из этих примеров служат иллюстрацией того, как надо проводить разбиение системы на части, когда устройство в целом нельзя поместить в одной интегральной схеме. Во втором параграфе разбираются примеры на языке VHDL, которые можно реализовать почти в любой ИС, выполненной на той или иной технологии.

Как и в случае главы 6, чтение данной главы предполагает знакомство с материалом, содержащимся только в предыдущих главах. Два параграфа этой главы написаны так, чтобы быть практически независимыми один от другого, а остающаяся часть этой книги не требует чтения данной главы сейчас; вы можете вернуться к ней позднее.

9.1. Примеры проектирования на языке ABEL

В параграфе 7.4 мы построили несколько простых конечных автоматов путем перевода их словесных описаний в таблицы состояний, выбора способа кодирования состояний и последующего синтеза соответствующих схем. Один из этих примеров был повторен на языке ABEL в расчете на реализацию в ПЛУ (см. табл. 7.27 и 7.31). Такого рода проекты значительно легче осуществлять с применением языка ABEL и реализовать их в ПЛУ по двум причинам:

- Вам нет необходимости особенно беспокоиться по поводу сложности результирующих уравнений возбуждения, пока разрабатываемое устройство помещается в выбранный ПЛУ.
- Вы имеете возможность воспользоваться особенностями языка ABEL, чтобы представить проект в более ясной и понятной форме.

Прежде чем обратиться к дальнейшим примерам, давайте рассмотрим временные характеристики конечных автоматов, построенных на основе ПЛУ, и соображения, касающиеся их компоновки.

9.1.1. Временные характеристики и компоновка конечных автоматов на основе ПЛУ

На рис. 9.1 показано, как можно было бы воспользоваться программируемым логическим устройством общего вида с комбинационными и регистровыми выходами для построения конечного автомата. Временные параметры t_{CO} и t_{PD} были введены в разделе 8.3.3: t_{CO} — задержка от тактового входа до выхода, а t_{PD} — задержка прохождения сигнала через матрицу И-ИЛИ.

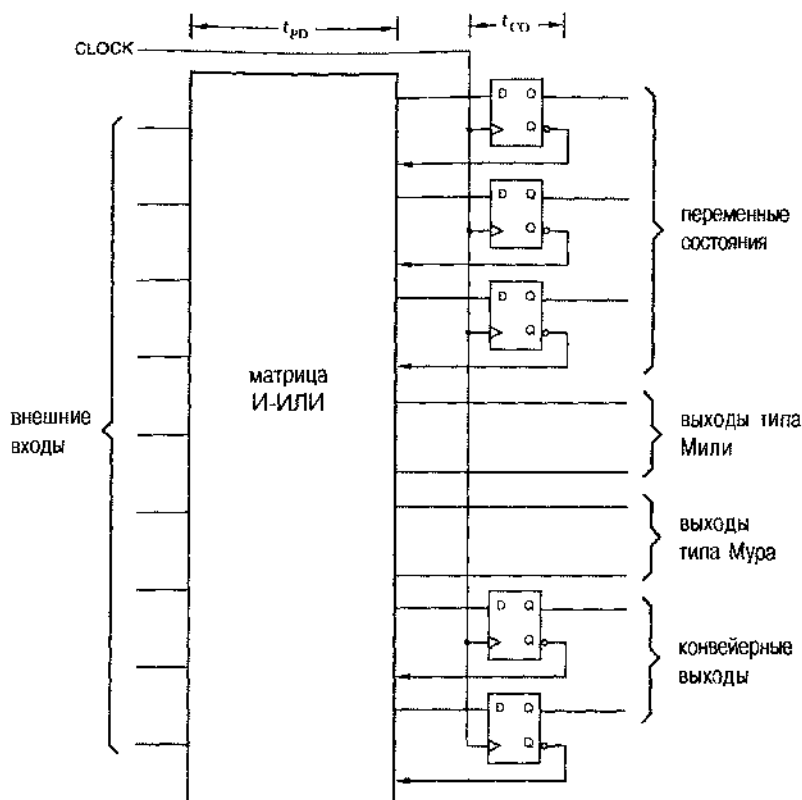


Рис.9.1. Структура и временные параметры ПЛУ, используемого в качестве конечного автомата

Переменные состояния, естественно, отнесены к регистровым выходам; их значения устанавливаются спустя время t_{CO} после нарастающего фронта сигнала CLOCK. Выходы типа Мили – это комбинационные выходы; сигналы на выходах типа Мили принимают установившиеся значения спустя время t_{PD} после любого изменения относящихся к ним входных сигналов. Сигналы на выходах типа Мили могут принимать новые значения также в том случае, когда изменяется состояние; момент, когда эти значения можно считать установившимися, наступает спустя время $t_{CO} + t_{PD}$ после нарастающего фронта сигнала CLOCK.

Сигнал на выходе типа Мура, по определению, является комбинационной логической функцией текущего состояния, поэтому он также устанавливается к моменту времени $t_{CO} + t_{PD}$ после прохождения переключающего фронта в сигнале CLOCK. Таким образом, при реализации конечного автомата в ПЛУ выходы Мура не имеют никаких преимуществ по сравнению с выходами Мили в отношении быстродействия. Для уменьшения задержки распространения мы ввели в разделе 7.3.2 конвейерные выходы, а в разделе 7.11.5 воспользовались ими. При реализации конечного автомата в ПЛУ сигналы на эти выходы поступают непосредственно с триггеров, их задержка, таким образом, составляет всего лишь t_{CO} по отношению к тактовому

сигналу. Но кроме меньшей задержки, конвейерные выходы гарантируют также отсутствие паразитных импульсов, что важно для целого ряда приложений.

Возможность построения конечного автомата на основе ПЛУ часто бывает ограничена числом выводов для входов и выходов у одного ПЛУ. Согласно модели, приведенной на рис. 9.1, на каждую переменную состояния, а также на каждый выход типа Мили и тина Мура требуется по одному выходному выводу ПЛУ. Например, для автомата, управляющего задними фонарями автомобиля марки Ford Thunderbird (см. параграф 7.5), требуется три регистровых выхода для переменных состояния и шесть комбинационных выходов для зажигания отдельных фонарей; это слишком много для большинства ПЛУ, которые мы рассматривали, за исключением ИС 22V10.

КАК ОВОЙТИСЬ БЕЗ ГОЛОВНОЙ ВОЛИ ПРИ РАЗБИЕНИИ

При размещении устройства в нескольких ПЛУ часто приходится идти путем проб и ошибок. Этого можно избежать, применяя современные программные средства проектирования цифровых систем на основе ПЛУ. Чтобы воспользоваться ими, разработчик вводит уравнения и описание проектируемого конечного автомата без конкретизации того, в каких микросхемах должно быть реализовано это устройство, и без назначения выводов. Так называемая *программа разбиения (partitioner)* пытается поместить устройство в возможно меньшем числе интегральных схем из указанного семейства, одновременно минимизируя число выводов, используемых для соединения этих ИС между собой. Процедура разбиения может быть полностью автоматизированной, либо осуществляться при частичном контроле со стороны пользователя или под его управлением.

При проектировании на ИС больших размеров типа CPLD и FPGA их внутренние архитектурные ограничения могут вызывать головную боль, если рядом нет специалиста по программным средствам. Разработчик исходит из необходимого числа входов и выходов, нужного объема комбинационной логики и требований, предъявляемых к триггерам, и ему кажется, что устройство помещается в одной ИС типа CPLD или FPGA. Но оказывается, что проект все же должен быть разбит на несколько ПЛУ или логических блоков внутри ИС больших размеров, функции каждого из которых ограничены.

Может случиться, например, что выходной сигнал, для формирования которого требуется большое число термов-произведений, необходимо отвести от других выходных сигналов, расположенных физически рядом. В свою очередь, от этого зависит, можно ли соседние выводы назначить темн или иными входами и выходами и сколько для них имеется термов-произведений. Указанные обстоятельства могут затронуть также возможности обмена сигналами между соседними блоками и повлиять на задержки этих сигналов в худшем случае. Все такие ограничения могут быть учтены *программой компоновки (fitter)*, в которой используется эвристический подход и находится наилучшее функциональное разбиение на блоки в пределах одной ИС типа CPLD или FPGA.

Чаще всего программа разбиения и программа компоновки работают в среде проектирования совместно и в интерактивном взаимодействии с разработчиком; в результате находится приемлемая реализация с использованием нескольких ПЛУ и ИС типа CPLD и FPGA.

С другой стороны, при записи состояний в форме выходного кода (см. раздел 7.3.2) требуется меньшее общее число выходов ПЛУ. Используя такое представление состояний, автомат, управляющий задними огнями автомобиля, можно разместить в одной ИС 16V8, как мы увидим это в разделе 9.1.3.

Аналогично любой переменной состояния или выходу типа Мура, переменная состояния, записанного в форме выходного кода, принимает установившееся значение спустя время t_{CO} после фронта тактового сигнала. Следовательно, представление состояния в форме выходного кода позволяет повысить быстродействие и обеспечивает большую эффективность компоновки. При управлении задними огнями автомобиля включение аварийной сигнализации на 10 нс раньше не существенно, однако в высококачественной цифровой системе лишние 10 нс задержки могут вызвать уменьшение максимальной частоты системного тактового сигнала со 100 МГц до 50 МГц.

При необходимости размещения системы в двух или большем числе ПЛУ лучше всего — с точки зрения простоты проекта и эффективности использования микросхем — осуществлять разбиение так, как показано на рис. 9.2. Единственное последовательностное ПЛУ применяется для того, чтобы обеспечить требуемые переходы из одного состояния в другое. С помощью комбинационных ПЛУ из переменных состояния и входных сигналов формируются значения сигналов на выходах типа Мили и типа Мура.

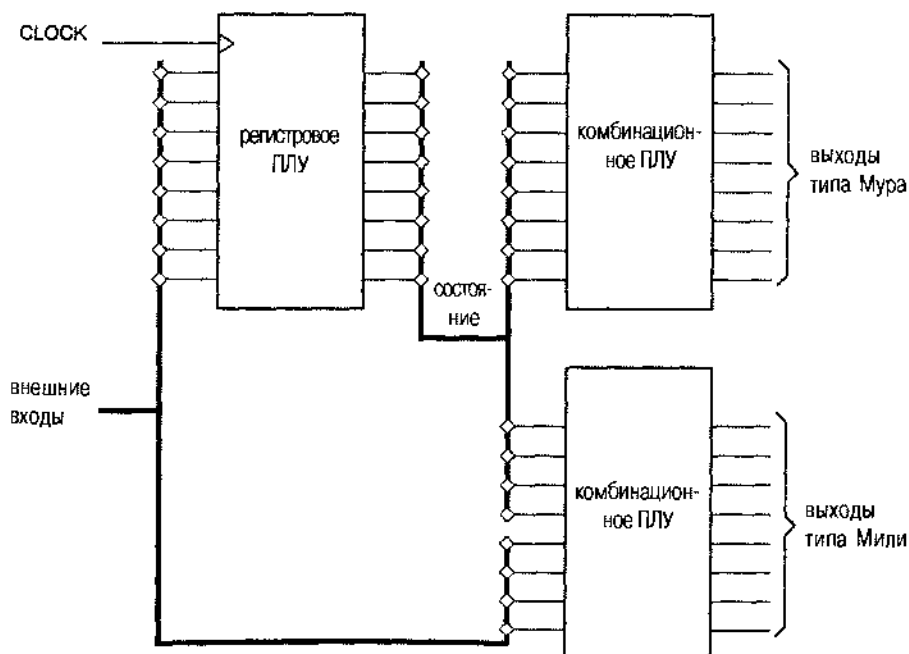


Рис. 9.2. Разбиение конечного автомата по трем ПЛУ

9.1.2. Несколько простых автоматов

В параграфе 7.4 мы построили несколько простых конечных автоматов традиционными методами. В табл. 7.25 проект, относящийся к первому из них, был представлен на языке ABEL в расчете на реализацию в ПЛУ, а затем в табл. 7.27 было показано, как лучше воспользоваться особенностями языка ABEL, чтобы поведение автомата сделать более понятным.

Наш второй пример – «автомат для подсчета числа единиц», к которому предъявляются следующие требования:

Должен быть настроен тактируемый синхронный конечный автомат с двумя входами X и Y и одним выходом Z . Сигнал на выходе должен равняться 1, если число единиц, поступивших на входы X и Y с момента запуска, кратно 4; в противном случае сигнал Z должен равняться 0.

Таблица состояний для этого автомата уже была нами составлена (см. табл. 7.12). Однако функции такого автомата значительно легче выразить средствами языка ABEL, как это видно из табл. 9.1. Заметьте, что в данном случае лучше воздержаться от использования синтаксиса «диаграмм состояний» языка ABEL. Функцию этого автомата естественнее выразить с помощью вложенного оператора WHEN и встроенной операции суммирования. Первым предложением WHEN автомат устанавливается в его начальное состояние и счет сбрасывается в 0, а в последующих предложениях счет увеличивается на 2, на 1 или на 0 при выполнении соответствующих условий, когда автомат не пребывает в сброшенном состоянии. Заметьте, что по правилам языка ABEL при сложении бит переноса «теряется», что эквивалентно выполнению сложения по модулю 4. Этот автомат легко умещается в ПЛУ GAL16V8. У него четыре состояния, поскольку он реализуется на двух триггерах.

```

module onesctsm
title 'Ones-counting State Machine'
ONESCTSM device 'P16V8R';

" Inputs and outputs
CLOCK, RESET, X, Y    pin 1, 2, 3, 4;
Z                      pin 13 istype 'com';
COUNT1..COUNT0     pin 14, 15 istype 'reg';

" Sets
COUNT = [COUNT1..COUNT0];

equations
COUNT.CLK = CLOCK;
WHEN RESET THEN COUNT := 0;
ELSE WHEN X & Y THEN COUNT := COUNT + 2;
ELSE WHEN X # Y THEN COUNT := COUNT + 1;
ELSE COUNT := COUNT;

Z = (COUNT == 0);

end onesctsm

```

Табл. 9.1. Программа на языке ABEL для конечного автомата, считающего число единиц

ОТКАЖЕМСЯ ОТ ДУРНЫХ ПРИВЫЧЕК!

В главе 7 мы начали проектирование автоматов с помощью таблиц состояний, не включая явно вход сброса. У этого были две причины: во-первых, с каждым дополнительным входом, в принципе, удваивается работа, которую необходимо выполнить для синтеза конечного автомата вручную, а во-вторых, с педагогической точки зрения, это давало бы небольшой эффект.

Но в реальных проектах практически всегда требуется вход сброса, сигнал на котором обеспечивает возврат автомата в известное состояние. Поэтому теперь, переходя к разработке конечных автоматов путем описания на том или ином языке и с помощью автоматизированных средств, мы должны отказаться от дурной привычки и обеспечивать наличие входа сброса в явном виде.

Другой пример из параграфа 7.4 – конечный автомат для кодового замка (в дальнейшем опущен выход HINT, имевшийся в исходном варианте):

Построить тактируемый синхронный конечный автомат с одним входом X и одним выходом UNLK. Сигнал на выходе UNLK должен равняться 1 тогда и только тогда, когда сигнал X равен 0 и последовательность его значений на предыдущих семи тактах имела вид: 0110111.

Нами была составлена таблица состояний для такого автомата (табл. 7.14) и написана эквивалентная программа на языке ABEL (табл. 7.31). Однако мы снова применим другой подход, который легче понять. Заметим, что в этом примере сигнал на выходе автомата в любой момент времени полностью определен значениями входного сигнала на последних восьми тактах. Следовательно, мы можем воспользоваться так называемым «принципом конечной памяти», когда явно отслеживаются семь предыдущих значений входного сигнала и выходной сигнал формируется как комбинационная функция этих значений.

Программа на языке ABEL в табл. 9.2 является реализацией принципа конечной памяти. В ней используются наборы, благодаря чему ее легко настраивать на другие кодовые комбинации. Заметьте, однако, что и в данном случае сформировать сигнал HINT было бы так же трудно, как и в исходной версии этого автомата (см. задачу 9.2).

ПРИНЦИП КОНЕЧНОЙ ПАМЯТИ

Возможно следующее обобщение идеи построения автомата с конечной памятью. Сигналы на выходах такого автомата полностью определяются текущими значениями входных сигналов и значениями выходных сигналов на последних n тактах, где n – конечное, ограниченное целое число. Автоматом с конечной памятью является любое устройство, которое можно реализовать так, как показано на рис. 9.3. Заметьте, что автомат с конечным числом состояний не обязательно представляет собой автомат с конечной памятью. Например, у автомата, считающего число единиц, всего четыре состояния, но он не является автоматом с конечной памятью: его выходной сигнал зависит от всех значений X и Y с момента запуска.

Табл. 9.2. Программа для конечного автомата с конечной памятью, управляющего кодовым замком

```

module combckf
title 'Combination-Lock State Machine'
"COMBLCKF device 'P16V8R';

" Input and output pins
CLOCK, RESET, X                pin 1, 2, 3;
X1..X7                          pin 12..18 istype 'reg';
UNLK                            pin 19;

" Sets
XHISTORY = [X7..X1];
SHIFTX  = [X6..X1, X];

equations

XHISTORY.CLK = CLOCK;
XHISTORY := 'RESET & SHIFTX;

UNLK = !RESET & (X == 0) & (XHISTORY == [0,1,1,0,1,1,1]);

END combckf

```

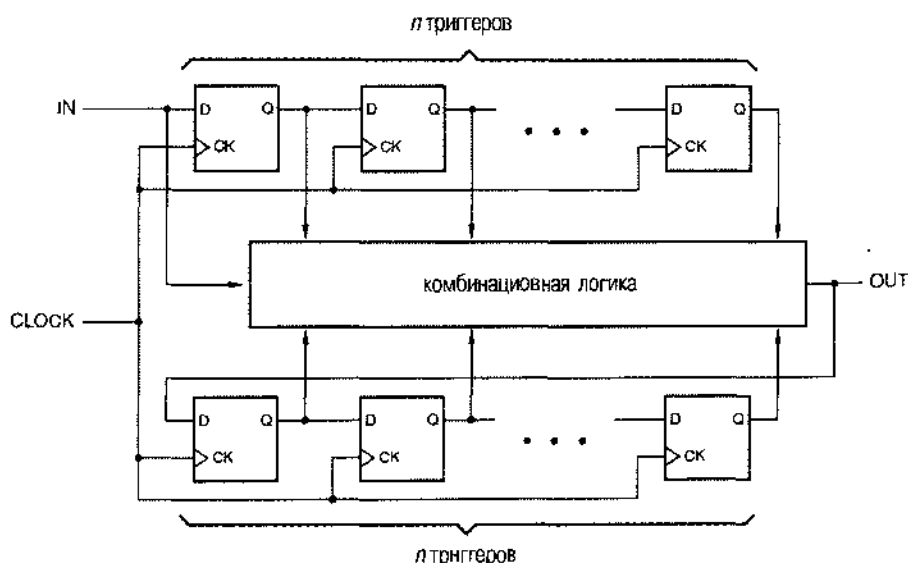


Рис. 9.3. Общая структура автомата с конечной памятью

9.1.3 Задние огни автомобиля марки Ford Thunderbird

В параграфе 7.5 был описан и построен конечный автомат для управления «задними огнями автомобиля марки Ford Thunderbird». В табл. 9.3 приведена эквивалентная «диаграмма состояний» такого автомата на языке ABEL. Существует тесная связь между этой программой и диаграммой состояний, показанной на рис. 7.64, при кодировании состояний согласно табл. 7.16. За исключением добавленного здесь входа RESET, программа выдает точно такие же приведенные уравнения, какими являются непосредственные уравнения переходов, полученные нами в параграфе 7.6 на основании списка переходов.

Табл. 9.3. Программа на языке ABEL для автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

```

module tbirdsd
title 'State Machine for T-Bird Tail Lights'
TBIRDSD device 'P16V8R';

" Input and output pins
CLOCK, LEFT, RIGHT, HAZ, RESET    pin 1, 2, 3, 4, 5;
Q0, Q1, Q2                        pin 14, 15, 16 istype 'reg';

" Definitions
QSTATE = [Q2,Q1,Q0];              " State variables
IDLE    = [ 0, 0, 0];             " States
L1      = [ 0, 0, 1];
L2      = [ 0, 1, 1];
L3      = [ 0, 1, 0];
R1      = [ 1, 0, 1];
R2      = [ 1, 1, 1];
R3      = [ 1, 1, 0];
LR3     = [ 1, 0, 0];

equations
QSTATE.CLK = CLOCK;

state_diagram QSTATE
state IDLE: IF RESET THEN IDLE
            ELSE IF (HAZ # LEFT & RIGHT) THEN LR3
            ELSE IF LEFT THEN L1 ELSE IF RIGHT THEN R1
            ELSE IDLE;
state L1:   IF RESET THEN IDLE ELSE IF HAZ THEN LR3 ELSE L2;
state L2:   IF RESET THEN IDLE ELSE IF HAZ THEN LR3 ELSE L3;
state L3:   GOTO IDLE;
state R1:   IF RESET THEN IDLE ELSE IF HAZ THEN LR3 ELSE R2;
state R2:   IF RESET THEN IDLE ELSE IF HAZ THEN LR3 ELSE R3;
state R3:   GOTO IDLE;
state LR3:  GOTO IDLE;

end tbirdsd

```

Программа в табл. 9.3 оперирует только переменными состояниями данного автомата. Для выходной логики необходимы шесть комбинационных выходов, но у ПЛУ 16V8, указанного в программе, имеется только пять свободных выходов. Для декодирования состояний можно было бы воспользоваться еще одним ПЛУ, осуществив разбиение, подобное тому, какое показано на рис. 9.2. Альтернативой этому является применение большего ПЛУ, например 22V10, у которого достаточно выходов, чтобы устройство можно было построить на одной ИС.

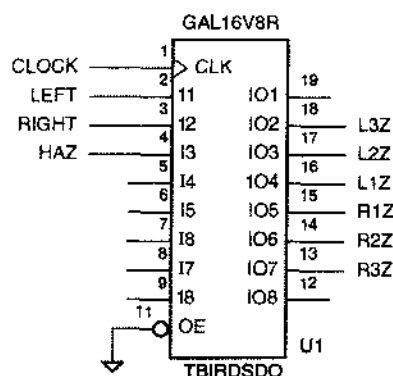


Рис. 9.4. Устройство управления задними огнями автомобиля марки Ford Thunderbird на одном ПЛУ

Табл. 9.4. Запись состояний в форме выходного кода для автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

```

module tbirdsdo
title 'Output-Coded T-Bird Tail Lights State Machine'
TBIRDSDO device 'P16V8R';

" Input and output pins
CLOCK, LEFT, RIGHT, HAZ, RESET    pin 1, 2, 3, 4, 5;
L3Z, L2Z, L1Z, R1Z, R2Z, R3Z      pin 18..13 istype 'reg';

" Definitions
QSTATE = [L3Z,L2Z,L1Z,R1Z,R2Z,R3Z]; " State variables
IDLE    = [ 0, 0, 0, 0, 0, 0]; " States
L3       = [ 1, 1, 1, 0, 0, 0];
L2       = [ 0, 1, 1, 0, 0, 0];
L1       = [ 0, 0, 1, 0, 0, 0];
R1       = [ 0, 0, 0, 1, 0, 0];
R2       = [ 0, 0, 0, 1, 1, 0];
R3       = [ 0, 0, 0, 1, 1, 1];
LR3      = [ 1, 1, 1, 1, 1, 1];

```

Однако возможно лучшее решение, основанное на том, что различным состояниям автомата соответствуют разные комбинации сигналов на его выходах. Это позволяет применить запись состояний в форме выходного кода. В этом случае потребуется только шесть регистровых выходов ПЛУ 16V8, как показано на рис.

9.4, и не понадобятся комбинационные выходы. Чтобы реализовать такой подход необходимо в предыдущей программе изменить только назначение выводов и определение состояний; в результате получим программу на языке ABEL для устройства с новым именем, приведенную в табл. 9.4. В каждом из шести результирующих уравнений возбуждения используются только четыре термина-произведения.

9.1.4. Игра на угадывание

В разделе 7.7.1 было дано следующее описание автомата для игры на угадывание:

Построить тактируемый синхронный автомат с четырьмя входами G1–G4, подключаемыми к кнопкам. У автомата четыре выхода L1–L4, к которым подключены лампочки или светодиоды, расположенные рядом с кнопками с теми же номерами. Имеется также выход ERR, к которому подключена красная лампочка. При нормальной работе на выходах L1–L4 индицируется комбинация «1 из 4». На каждом такте комбинация сдвигается на одну позицию; частота тактового сигнала равна 4 Гц.

Задача игрока состоит в том, чтобы вовремя нажать кнопку, соответствующую горящей лампочке. При нажатии *i*-ой кнопки вырабатывается единичный сигнал Gi. Если подан «неправильный» сигнал, то возникает сигнал на выходе ERR и загорается красная лампочка; это происходит в том случае, когда автомат на очередном такте обнаруживает сигнал, номер которого не совпадает с номером лампочки, зажженной на предыдущем такте. Когда кнопка нажата, игра останавливается, и сигнал на выходе ERR сохраняет свое значение в течение одного или нескольких тактов, пока не будет снят удерживаемый вами сигнал Gi, и тогда игра возобновляется.

Как мы видели в разделе 7.7.1, этому автомату необходимы шесть состояний: по одному состоянию на каждую зажженную лампочку и два состояния для тех случаев, когда игра остановлена после правильного или неправильного нажатия на кнопку. В табл. 9.5 приведена программа на языке ABEL для игры на угадывание. Добавлены два усовершенствования, обеспечивающие возможность тестирования и повышающие надежность автомата: введен вход RESET, позволяющий перевести автомат в известное начальное состояние, и предусмотрены два неиспользуемых состояния с указанными в явном виде переходами в начальное состояние.

Кодирование состояний в автомате, описываемом данной программой, такое же, как и в исходной версии в разделе 7.7.1. Следуя этому кодированию, компилятор языка ABEL выдает минимизированные равенства с числом термов-произведений, указанным в табл. 9.6. У выходного сигнала Q0 восемь термов-произведений, и это – предел того, что можно реализовать в ИС GAL16V8. Можно попытаться иначе кодировать состояния и написать программу так, чтобы сохранять значения отдельных термов (см. задачу 9.4).

Более продуктивной может оказаться попытка записывать состояние в форме выходного кода. Можно воспользоваться однобитовым представлением состояний и выходов (L1 . . . L4) в отношении каждой из лампочек и еще одним битом (ERR) для того, чтобы различать состояния SOK и SERR, когда все лампочки погашены. Это позволяет опустить в табл. 9.5 равенства для L1 . . . L4 и ERR. Новое кодирование представлено в табл. 9.7. При такой записи состояний для L1 нужны два термина-произведения, а для

L2..L4 – лишь по одному терму-произведению на каждый выходной сигнал. К сожалению, для сигнала на выходе ERR необходимы 16 термов-произведений.

Табл. 9.5. Программа на языке ABEL, описывающая автомат для игры на угадывание

```

module ggame
Title 'Guessing-Game State Machine'
GGAME device 'P16V8R';

" Inputs and outputs
CLOCK, RESET, G1..G4      pin 1, 2, 3..6;
L1..L4, ERR               pin 12..15, 19 istype 'com';
Q2..Q0                    pin 16..18 istype 'reg';

" Sets
G = [G1..G4];
L = [L1..L4];

" States
QSTATE = [Q2,Q1,Q0];
S1      = [ 0, 0, 0];
S2      = [ 0, 0, 1];
S3      = [ 0, 1, 1];
S4      = [ 0, 1, 0];
SOK      = [ 1, 0, 0];
SERR     = [ 1, 0, 1];
EXTRA1   = [ 1, 1, 0];
EXTRA2   = [ 1, 1, 1];

state_diagram QSTATE
state S1: IF RESET THEN SOK ELSE IF G2 # G3 # G4 THEN SERR
        ELSE IF G1 THEN SOK ELSE S2;
state S2: IF RESET THEN SOK ELSE IF G1 # G3 # G4 THEN SERR
        ELSE IF G2 THEN SOK ELSE S3;
state S3: IF RESET THEN SOK ELSE IF G1 # G2 # G4 THEN SERR
        ELSE IF G3 THEN SOK ELSE S4;
state S4: IF RESET THEN SOK ELSE IF G1 # G2 # G3 THEN SERR
        ELSE IF G4 THEN SOK ELSE S1;
state SOK: IF RESET THEN SOK
        ELSE IF G1 # G2 # G3 # G4 THEN SOK ELSE S1;
state SERR: IF RESET THEN SOK
        ELSE IF G1 # G2 # G3 # G4 THEN SERR ELSE S1;
state EXTRA1: GOTO SOK;
state EXTRA2: GOTO SOK;

equations
QSTATE.CLK = CLOCK;
L1 = (QSTATE == S1);
L2 = (QSTATE == S2);
L3 = (QSTATE == S3);
L4 = (QSTATE == S4);
ERR = (QSTATE == SERR);

end ggame

```


Табл. 9.6. Использование термов-произведений при реализации в ПЛУ конечного автомата для игры на угадывание

P-Terms	Fan-in	Fan-out	Type	Name
1/3	3	1	Pin	L1
1/3	3	1	Pin	L2
1/3	3	1	Pin	L3
1/3	3	1	Pin	L4
1/3	3	1	Pin	ERR
6/2	7	1	Pin	Q2.REG
1/7	7	1	Pin	Q1.REG
11/8	8	1	Pin	Q0.REG
=====				
23/32	Best P-Term Total: 16			
	Total Pins: 14			
	Average P-Term/Output: 2			

Табл. 9.7. Определения на языке ABEL применительно к автомату для игры на угадывание в случае записи состояний в форме выходного кода

```

module ggamecc
Title 'Guessing-Game State Machine'
"GGAMECC device 'P16V8R';

" Inputs and outputs
CLOCK, RESET, G1..G4           pin 1, 2, 3..6;
L1..L4, ERR                    pin 12..15, 18 istype 'reg';

" States
QSTATE = [L1,L2,L3,L4,ERR];
S1      = [ 1, 0, 0, 0, 0];
S2      = [ 0, 1, 0, 0, 0];
S3      = [ 0, 0, 1, 0, 0];
S4      = [ 0, 0, 0, 1, 0];
SQK     = [ 0, 0, 0, 0, 0];
SERR    = [ 0, 0, 0, 0, 1];

```

Применяя указанный конкретный способ записи состояний в форме выходного кода, мы не в полной мере используем достоинства такого подхода. Запись состояний в форме выходного кода является, по существу, «прямым кодированием», но для декодирования каждого состояния нужны все пять битов представления состояния согласно определению состояний в табл. 9.7. Возможен другой вариант кодирования с использованием «безразличных» значеий, как показано в табл. 9.8.

```

X = .X.;
QSTATE = [L1, L2, L3, L4, ERR];
S1      = [ 1, X, X, X, X];
S2      = [ X, 1, X, X, X];
S3      = [ X, X, 1, X, X];
S4      = [ X, X, X, 1, X];
SOK     = [ 0, 0, 0, 0, 0];
SERR    = [ X, X, X, X, 1];

```

Табл. 9.8. Занисъ состояний автомата для игры на угадывание в форме выходного кода с использованием «безразличных» значений

В новом варианте мы предполагаем, что состояния никогда не бывают такими, чтобы в комбинации битов, которой нредставлено состояние, были другие единицы, кроме указанных первоначально в табл. 9.7. Если, например, мы видим, что бит состояния L1 равен 1, то автомат должен находиться в состоянии S1 независимо от значений каких-либо других битов в коде состояния. Поэтому все другие биты, кроме L1, в онределении состояния S1 можно положить безразличными, как это сделано в табл. 9.8. В языке ABEL каждый бит, помеченный символом X, принимается равным 0 при кодировании следующего состояния, считается «безразличным» при декодировании текущего состояния. Следовательно, необходимо обратить *особое внимание* на то, чтобы декодируемые состояния были действительно взаимно исключающими; другими словами, нужно, чтобы никакое правильное следующее состояние не соответствовало двум или большему числу состояний, определенных по-разному. В противном случае сконпирированное устройство не будет вести себя ожидаемым образом.

КАК ДЕЙСТВУЕТ МЕХАНИЗМ БЕЗРАЗЛИЧНЫХ ЗНАЧЕНИЙ?

Чтобы разобраться в том, какую роль играют безразличные значения при кодировании состояний, необходимо понять, как компилятор языка ABEL создает равенства на основании диаграмм состояний. Применительно к данному состоянию S каждый оператор перехода (IF-THEN-ELSE или GOTO) пополняет множества включений соответствующих нерменных состояния согласно условию перехода. Условие перехода — это выражение, которое должно быть верно, чтобы переход состоялся, включая нребывание в состоянии S. Например, все условия, указанные для состояния S1 в табл. 9.5, неявно участвуют в логической операции И со значением выражения “QSTATE==S1”. Поскольку состояние S1 определено с использованием безразличных значений, эта проверка равенства породит только один литерал L1, а не терм-произведение, что в дальнейшем приведет к еще большим упрощениям.

Для каждого следующего состояния, уноминаемого в операторе перехода, происходит пополнение множеств включений только тех нерменных состояния, которые равны 1 в этом состоянии. Когда, например, в онераторе перехода в табл. 9.8 код состояния S1 встречается в качестве следующего состояния, пополняется только множество включений для L1. Этим объясняется, почему действительный код состояния S1 в качестве следующего состояния считается имеющим вид: 100000.

При записи состояний в форме выходного кода согласно табл. 9.8 приведенные равенства содержат три термина-произведения для L_1 , по одному термину-произведению на каждый из выходных сигналов $L_2, \dots, L_4$ и всего лишь семь термов-произведений для сигнала ERR . Таким образом, это было стоящее изменение. Необходимо помнить, однако, что новый автомат отличается от устройства, описываемого табл. 9.7. Посмотрим, что произойдет, если автомат попадет в одно из состояний, которые не определены. В исходном устройстве с полностью заданным описанием состояний в форме выходного кода, не указаны следующие состояния для $2^5 - 6 = 26$ состояний из числа не определенных, поэтому конечный автомат из любого такого состояния всегда будет переходить в состояние с именем 00000 (SOK). В новом автомате «не определенные» состояния, в действительности, не являются таковыми; например, состояние с именем 1111 фактически согласуется с пятью состояниями: S1–S4 и SERR. Поэтому следующим будет состояние, имя которого является результатом выполнения логической операции ИЛИ над именами следующих состояний, соответствующих согласующимся состояниям. (Чтобы понять, почему это происходит, см. вынесенное за пределы основного текста рассуждение о механизме действия безразличных значений.) Еще раз: нужно быть осторожным.

ЧЕГО СЛЕДУЕТ ОЖИДАТЬ ПРИ СБРОСЕ?

Из текста программы в табл. 9.5 следует, что появление сигнала на входе RESET заставит автомат перейти в состояние SOK. Однако в случае, когда у вас имеются не определенные кодовые имена состояний или имена состояний заданы частично, нельзя принимать на веру, что подача сигнала RESET будет приводить к тому же самому.

Имея в виду то, что говорится в предыдущем замечании, заключенном в рамку, следует помнить, что в программе на языке ABEL для конечного автомата операторы перехода наполняют множества включений, относящиеся к переменным состояния. Если какая-то конкретная комбинация битов для неиспользуемого состояния не согласуется ни с каким из состояний, для которых в программе имеются операторы перехода, то никакие множества включений не будут пополнены. Следовательно, единственным переходом из такого состояния будет переход в состояние с кодовым именем, состоящим из одних нулей.

Поэтому полезно кодировать начальное или «безопасное» состояние всеми нулями. Если это невозможно, но имя, состоящее из одних нулей, все же не используется, то можно явно задать переход из состояния, обозначаемого одними нулями, в желаемое безопасное состояние.

9.1.5. Построим заново контроллер светофора!

Наш последний пример – из мира автомобилей и дорожного движения. Контроллеры светофоров в Калифорнии тщательно сконструированы таким образом, чтобы *максимизировать* время простоя автомобилей на перекрестках; особенно это относится к замечательному городу Саннивейл. Здесь на незагруженном перекрестке имеются датчики движения и светофоры, показанные на рис. 9.5. (Если бы это было в Чикаго, то самое большее, чем был бы отмечен такой перекресток, – это

знак «уступи дорогу».) Светофорами управляет автомат с частотой тактового сигнала 1 Гц; у него имеются таймер и четыре входа:

- NSCAR** Сигнал принимает активное значение, если хотя бы один автомобиль находится в поле действия одного из датчиков в направлении движения «север-юг» по любую сторону перекрестка.
- EWCAR** Сигнал принимает активное значение, если хотя бы один автомобиль находится в поле действия одного из датчиков в направлении движения «восток-запад» по любую сторону перекрестка.
- TMLONG** Сигнал принимает активное значение, если с момента начала работы таймера прошло более пяти минут; он остается на активном уровне до тех пор, пока таймер не будет сброшен.
- TMSHORT** Сигнал принимает активное значение, если с момента начала работы таймера прошло более пяти секунд; он остается на активном уровне до тех пор, пока таймер не будет сброшен.

У этого конечного автомата семь выходных сигналов:

- NSRED, NSYELLOW, NSGREEN** Сигналы, зажигающие красный, желтый и зеленый свет в направлении «север-юг» соответственно.
- EWRED, EWYELLOW, EWGREEN** Сигналы, зажигающие красный, желтый и зеленый свет в направлении «восток-запад» соответственно.
- TMRESET** Когда этот сигнал принимает активное значение, таймер сбрасывается, а сигналы **TMSHORT** и **TMLONG** переходят на неактивный уровень. Таймер начинает отсчет времени, когда на неактивный уровень переходит сигнал **TMRESET**.

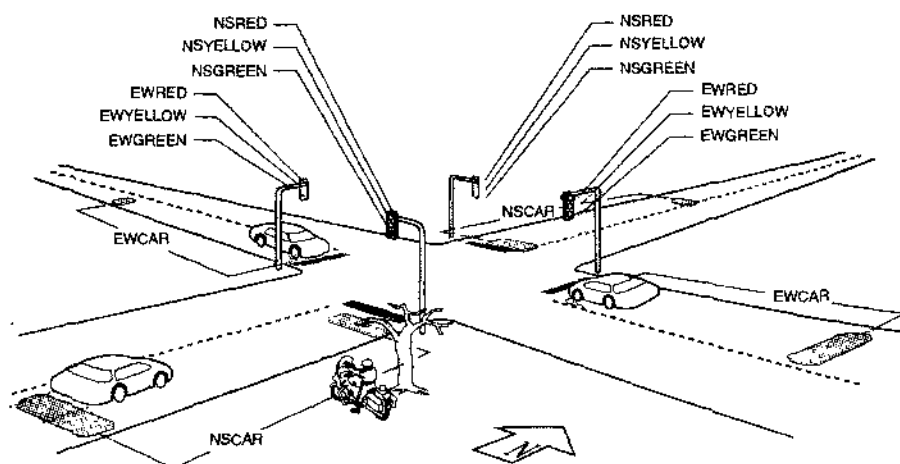


Рис. 9.5. Датчики дорожного движения и сигналы светофоров на перекрестке в г. Саннивейл, шт. Калифорния

Табл. 9.9. Программа для светофора в г. Саннивейл

```

module svalet1
title 'State Machine for Sunnyvale, CA, Traffic Lights'
SVALET1 device 'P16V8R';

" input and output pins
CLOCK, !OE                pin 1, 11;
NSCAR, EWCAR, TMSHORT, TMLONG  pin 2, 3, 8, 9;
Q0, Q1, Q2, TMRESET_L      pin 17, 18, 15, 14 istype 'reg';

" Definitions
LSTATE = [Q2,Q1,Q0];      " State variables
NSGO    = [ 0, 0, 0];     " States
NSWAIT  = [ 0, 0, 1];
NSWAIT2 = [ 0, 1, 1];
NSDELAY = [ 0, 1, 0];
EWGO    = [ 1, 1, 0];
EWWAIT  = [ 1, 1, 1];
EWWAIT2 = [ 1, 0, 1];
EWDELAY = [ 1, 0, 0];

state_diagram LSTATE
state NSGO:      " North-south green
  IF (!TMSHORT) THEN NSGO      " Minimum green is 5 seconds.
  ELSE IF (TMLONG) THEN NSWAIT " Maximum green is 5 minutes.
  ELSE IF (EWCAR & !NSCAR)     " If E-W car is waiting and no one
    THEN NSGO                 " is coming N-S, make E-W wait!
  ELSE IF (EWCAR & NSCAR)      " Cars coming in both directions?
    THEN NSWAIT              " Thrash!
  ELSE IF (!NSCAR)            " Nobody coming N-S and not timed out?
    THEN NSGO                 " Keep N-S green.
  ELSE NSWAIT;                " Else let E-W have it.

state NSWAIT: GOTO NSWAIT2;    " Yellow light is on for two ticks for safety.
state NSWAIT2: GOTO NSDELAY;   " (Drivers go 70 mph to catch this turkey green!)
state NSDELAY: GOTO EWGO;      " Red in both directions for added safety.

state EWGO: " East-west green; states defined analogous to N-S
  IF (!TMSHORT) THEN EWGO
  ELSE IF (TMLONG) THEN EWWAIT
  ELSE IF (NSCAR & !EWCAR) THEN EWGO
  ELSE IF (NSCAR & EWCAR) THEN EWWAIT
  ELSE IF (!EWCAR) THEN EWGO ELSE EWWAIT;

state EWWAIT: GOTO EWWAIT2;
state EWWAIT2: GOTO EWDELAY;
state EWDELAY: GOTO NSGO;

equations
LSTATE.CLK = CLOCK; TMRESET_L.CLK = CLOCK;
!TMRESET_L := (LSTATE == NSWAIT2) " Reset the timer when going into
+ (LSTATE == EWWAIT2); " state NSDELAY or state EWDELAY.
end svalet1

```

Программой на языке ABEL, приведенной в табл. 9.9, реализуется типичный, одобренный местными властями алгоритм управления светофорами. Этот алгоритм обеспечивает два часто наблюдаемых режима работы нашего «проворного» светофора. Ночью, при малой интенсивности движения, он удерживает автомобиль в состоянии ожидания до пяти минут, если только не появляется автомобиль на поперечной улице; в этом случае светофор переключается так, чтобы остановить движение в поперечном направлении и пропустить ожидающий автомобиль.

(Датчик «раннего оповещения» устанавливается достаточно далеко, чтобы сигналы светофора успели измениться до того, как приближающийся автомобиль достигнет перекрестка.) Днем, при напряженном движении всегда имеются автомобили, ожидающие проезда в обоих направлениях; тогда светофор переключается каждые пять секунд, чтобы минимизировать пропускную способность перекрестка и максимизировать время ожидания для всех, подталкивая тем самым возмущенную общественность к мысли о необходимости повышения налогов для решения этой проблемы.

Заслуживают внимания равенства для сигнала **TMRESET**. Этот выходной сигнал принимает активное значение во время состояний **NSDELAY** и **EWDELAY** («двойной красный свет»), когда происходит сброс таймера и его подготовка к следующему циклу с зажженным зеленым светом. Желательный выходной сигнал можно было бы сформировать на выводе, предназначенном для комбинационного выхода, обнаруживая данные два состояния; но вместо этого мы выбрали регистровый выход и предусмотрели обнаружение состояний, *предшествующих* этим двум состояниям.

В программе на языке **ABEL** в табл. 9.9 определены только нерменные состояния для контроллера светофора и один выход типа Мура. Из того, что остается в ИС **16V8**, нельзя образовать шесть других выходов типа Мура, необходимых для зажигания нужного света. Поэтому для формирования этих выходных сигналов применено отдельное комбинационное ПЛУ. Конструкция в целом представлена на рис. 9.6. Программа на языке **ABEL** для выходного ПЛУ приведена в табл. 9.10. Мы воспользовались имеющейся возможностью и добавили контроллеру вход **OVERRIDE**. Подавая сигнал на этот вход, полицейский может заблокировать работу контроллера и заставить светофор мигать красным светом (с частотой тактового сигнала **FLASHCLK**), и тогда у него появляется возможность вручную растаскивать пробки, возникающие благодаря этому удивительному изобретению.

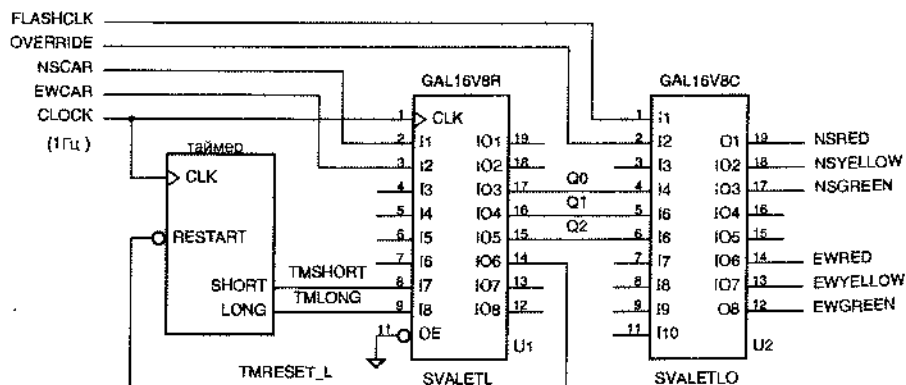


Рис. 9.6. Контроллер светофора в г. Саннивейл на двух ПЛУ

Табл. 9.10. Выходная логика контроллера светофора в г. Саннивейл

```

module svaletlo
title 'Output logic for Sunnyvale, CA, Traffic Lights'
"SVALETLO device 'P16V8C';

" Input pins
FLASHCLK, OVERRIDE, Q0, Q1, Q2  pin 1, 2, 4, 5, 6;

" Output pins
NSRED, NSYELLOW, NSGREEN        pin 19, 18, 17  istype 'com';
EWRED, EWYELLOW, EWGREEN        pin 14, 13, 12  istype 'com';

" Definitions (same as in state machine SVAETL)
...

equations

NSRED = !OVERRIDE & (LSTATE != NSGD) & (LSTATE != NSWAIT) & (LSTATE != NSWAIT2)
      # OVERRIDE & FLASHCLK;
NSYELLOW = !OVERRIDE & ((LSTATE == NSWAIT) # (LSTATE == NSWAIT2));
NSGREEN  = !OVERRIDE & (LSTATE == NSGD);

EWRED = !OVERRIDE & (LSTATE != EWGD) & (LSTATE != EWAIT) & (LSTATE != EWAIT2)
      # OVERRIDE & FLASHCLK;
EWYELLOW = !OVERRIDE & ((LSTATE == EWAIT) # (LSTATE == EWAIT2));
EWGREEN  = !OVERRIDE & (LSTATE == EWGD);

end svaletlo

```

Если перейти к записн состояний в форме выходного кода, то конечный автомат для управления светофором вместе с выходной логикой можно построить на одном ПЛУ (рис. 9.7). Как видно из табл. 9.11, для этого необходимо изменить только определения в исходной программе, приведенной в табл. 9.9. У этого ПЛУ нет входа OVERRIDE и соответствующего режима работы; эти вопросы вынесены в задачу 9.7.

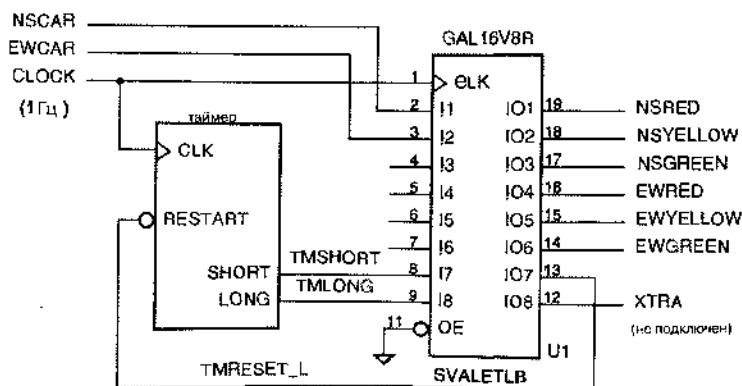


Рис. 9.7. Конечный автомат для управления светофором на одном ПЛУ с записью еостояний в форме выходного кода

Табл. 9.11. Определения для автомата, управляющего светофором в г. Сан-нивейл, с записью состояний в форме выходного кода

```

module svaletlb
title 'Output-Coded State Machine for Sunnyvale Traffic Lights'
"SVALETLB device 'P16V8R';

" Input and output pins
CLOCK, IOE                                pin 1, 11;
NSCAR, EWCAR, TMSHORT, TMLONG             pin 2, 3, 8, 9;
NSRED, NSYELLOW, NSGREEN                  pin 19, 18, 17 1stype 'reg';
EWRED, EWYELLOW, EWGREEN                  pin 16, 15, 14 1stype 'reg';
TMRESET_L, XTRA                           pin 13, 12 1stype 'reg';

" Definitions
LSTATE = [NSRED, NSYELLOW, NSGREEN, EWRED, EWYELLOW, EWGREEN, XTRA]; " State vars
NSGO    = [ 0,      0,      1,      1,      0,      0,      0]; " States
NSWAIT  = [ 0,      1,      0,      1,      0,      0,      0];
NSWAIT2 = [ 0,      1,      0,      1,      0,      0,      1];
NSDELAY = [ 1,      0,      0,      1,      0,      0,      0];
EWGO     = [ 1,      0,      0,      0,      0,      1,      0];
EWWAIT   = [ 1,      0,      0,      0,      1,      0,      0];
EWWAIT2  = [ 1,      0,      0,      0,      1,      0,      1];
EWDELAY  = [ 1,      0,      0,      1,      0,      0,      1];

```

9.2. Примеры проектирования на языке VHDL

В параграфе 7.12 мы уже объяснили, что основные правила языка VHDL, введенные нами еще в параграфе 4.7, в том числе, процессы – это почти все, что нужно для описания работы последовательностных схем. В отличие от языка ABEL в языке VHDL нет никаких специальных языковых средств для создания конечных автоматов. Вместо этого большинство программистов описывает конечные автоматы комбинацией имеющихся «обычных» средств, чаще всего с помощью перечислимых типов и операторов *case*. Мы воспользуемся этим методом в примерах данного параграфа.

9.2.1. Несколько простых автоматов

В разделе 7.4.1 мы проиллюстрировали процедуру создания автомата по таблице состояний на примере следующей простой задачи:

Построить тактируемый синхронный конечный автомат с двумя входами А и В и одним выходом Z, сигнал на котором равен 1, если

- сигнал на входе А имел одно и то же значение на двух последних тактах *или*
- сигнал на входе В оставался равным 1 с тех пор, когда в последний раз было выполнено первое условие.

В противном случае выходной сигнал должен равняться 0.

В среде проектирования на основе того или иного языка описания схем можно многими способами написать программу, удовлетворяющую сформулированным требованиям. Мы рассмотрим несколько таких возможностей.

Первый подход заключается в том, чтобы составить от руки таблицу состояний и значений выходного сигнала, а затем вручную преобразовать ее в программу. Поскольку в разделе 7.4.1 нами уже была составлена такая таблица состояний, то почему бы нам не воспользоваться ею? В табл. 9.12 эта таблица воспроизведена заново, а в табл. 9.13 приведена соответствующая VHDL-программа.

S	A B				Z
	00	01	11	10	
INIT	A0	A0	A1	A1	0
A0	OK0	OK0	A1	A1	0
A1	A0	A0	OK1	OK1	0
OK0	OK0	OK0	OK1	A1	1
OK1	A0	OK0	OK1	OK1	1
S*					

Табл. 9.12. Таблица состояний и значений выходного сигнала в рассматриваемом примере

Как обычно, объявление объекта в языке VHDL содержит только указания па входы и выходы; в нашем примере это — CLOCK, A, B и Z. Определением архитектуры задается внутреннее функционирование конечного автомата. Первое, что делается в архитектуре, — это образование неречислимого типа `Sreg_type`, значениями которого являются идентификаторы, соответствующие именам состояний. Затем объявляется сигнал `Sreg`, который будет применен для хранения текущего состояния автомата. С учетом последующего использования этого сигнала, он будет при синтезе отображен в переключающийся по фронту регистр.

В части архитектуры, содержащей операторы, имеется два параллельных оператора: процесс и оператор избирательного присваивания. Процесс чувствителен только к сигналу `CLOCK` и им осуществляются все переходы из одного состояния в другое, которые происходят на нарастающем фронте сигнала `CLOCK`. Оператором `if` в пределах процесса обнаруживается нарастающий фронт, а в операторе `case` для каждого состояния перебираются все возможные переходы.

В операторе `case` имеется шесть альтернатив, соответствующих пяти состояниям, определенным явно, и случаю, в котором собраны все другие состояния. Ради надежности, в случае `others` автомат отсылается назад в состояние `INIT`. В каждой из альтернатив используется вложенный оператор `if`, чтобы непосредственно перечислить все комбинации входных сигналов A и B. Однако, строго говоря, нет необходимости включать те комбинации, при которых автомат остается в текущем состоянии; сигнал `Sreg` сохраняет текущее состояние до тех пор, пока в каком-то из возобновлений процесса ему не будет присвоено новое значение.

Оператор избирательного присваивания в конце табл. 9.13 вырабатывает единственный выходной сигнал данного автомата — сигнал Z типа Мура, значение которого зависит от текущего состояния. Возможно, легче было определить также в пределах этого оператора выходные сигналы типа Мили. Другими словами, значе-

ние Z могло бы быть функцией не только текущего состояния, но и входных сигналов. Поскольку оператор "with" является параллельным оператором, любые изменения входных сигналов сразу же отражаются на значении выходного сигнала Z.

Табл. 9.13. VHDL-программа для рассматриваемого конечного автомата

```

library IEEE;
use IEEE.std_logic_1164.all;

entity smexamp is
  port ( CLOCK, A, B: in STD_LOGIC;
        Z: out STD_LOGIC );
end;

architecture smexamp_arch of smexamp is
  type Sreg_type is (INIT, AO, A1, OKO, OK1);
  signal Sreg: Sreg_type;
begin

  process (CLOCK) -- state-machine states and transitions
  begin
    if CLOCK'event and CLOCK = '1' then
      case Sreg is
        when INIT => if A='0' then Sreg <= AO;
                      elsif A='1' then Sreg <= A1; end if;
        when AO => if A='0' then Sreg <= OKO;
                    elsif A='1' then Sreg <= A1; end if;
        when A1 => if A='0' then Sreg <= AO;
                    elsif A='1' then Sreg <= OK1; end if;
        when OKO => if A='0' then Sreg <= OKO;
                     elsif A='1' and B='0' then Sreg <= A1;
                     elsif A='1' and B='1' then Sreg <= OK1; end if;
        when OK1 => if A='0' and B='0' then Sreg <= AO;
                     elsif A='0' and B='1' then Sreg <= OKO;
                     elsif A='1' then Sreg <= OK1; end if;
        when others => Sreg <= INIT;
      end case;
    end if;
  end process;

  with Sreg select -- output values based on state
    Z <= '0' when INIT | AO | A1,
        '1' when OKO | OK1,
        '0' when others;

end smexamp_arch;

```

На самом деле, нам следовало предусмотреть в табл. 9.13 вход сброса (см. заключенное рамку замечание в конце раздела 9.1.4). Сигнал RESET легко включить, видоизменив объявление объекта и добавив еще одно предположение в оператор "if" в определении архитектуры. Если сигнал RESET принимает активное значение, то автомат должен перейти в состояние INIT; в противном случае сле-

дует выполнять оператор `case`. В зависимости от того, когда проверяется сигнал `RESET`, — до проверки поступления фронта тактового сигнала или после, — реализуется асинхронный или синхронный сброс (см. задачу 9.10).

Так как же насчет кодирования состояний? Табл. 9.13 не содержит никакой информации о том, как должны присваиваться комбинации неремежных состояния состояниям с теми или иными именами. Ничего не известно даже о том, сколько необходимо двоичных переменных состояния.

Средства синтеза должны связывать с идентификаторами неречеислимого типа любые целые числа или двоичные комбинации, какие им только поправятся, по в типичном случае состояниям будут ноставлены в соответствие целые числа, начиная с 0, в том порядке, в каком перечислены их имена. Затем для представления этих чисел будет использовано наименьшее возможное число битов, равное $\lceil \log_2 s \rceil$ при наличии s состояний. Таким образом, синтез по программе из табл. 9.13 будет происходить с тем же самым «простейшим» кодированием состояний, которое было выбрано нами в исходном примере (см. табл. 7.7). Однако средствами языка VHDL можно заставить компилятор принять какой-то другой способ кодирования.

Один из способов навязать определенное кодирование состояний заключается в употреблении оператора `attribute`, как это сделано в табл. 9.14. Здесь `enum_encoding` — определяемый пользователем признак, значением которого является строка, указывающая, какое именно кодирование путем перечисления должно быть использовано средствами синтеза. Процессор языка VHDL игнорирует это значение, но передает имя признака и его значение средствам синтеза. Признак `enum_encoding` определен в большинстве средств синтеза и известен им, в том числе средствам синтеза фирмы Synopsys, Inc. Заметьте, что программой должен «использоваться» пакет `attributes` фирмы Synopsys; это необходимо для того, чтобы VHDL-компилятор распознал `enum_encoding` как законный определяемый пользователем признак. Между прочим, кодирование состояний, указанное в последней программе, эквивалентно «почти прямому» кодированию из табл. 7.7.

```
library IEEE;
use IEEE.std_logic_1164.all;
library SYNOPSYS;
use SYNOPSYS.attributes.all;
...
architecture smexampe_arch of smexamp is
type Sreg_type is (INIT, A0, A1, OK0, OK1);
attribute enum_encoding of Sreg_type: type is
    "0000 0001 0010 0100 1000";
signal Sreg: Sreg_type;
...
```

Табл. 9.14. Использование признака для того, чтобы заставить средства синтеза кодировать состояния по нрани-
лу неречеисления

Другой способ навязать то или иное кодирование состояний без обращения к внешним пакетам и без учета признаков при синтезе заключается в более явном задании регистра состояний с помощью обычных логических типов данных. Этот подход представлен в табл. 9.15. Здесь сигнал `Sreg` определен как 4-разрядный

элемент типа `STD_LOGIC_VECTOR` и введены константы, позволяющие повсюду в программе ссылаться на состояния по их именам. Никаких других изменений в программе не требуется.

Табл. 9.15. Использование обычной логики и констант для задания способа кодирования состояний

```
library IEEE;
use IEEE.std_logic_1164.all;
...
architecture smexamp_arch of smexamp is
  subtype Sreg_type is STD_LOGIC_VECTOR (1 to 4);
  constant INIT: Sreg_type := "0000";
  constant A0 : Sreg_type := "0001";
  constant A1 : Sreg_type := "0010";
  constant OK0 : Sreg_type := "0100";
  constant OK1 : Sreg_type := "1000";
  signal Sreg: Sreg_type;
  ...
end architecture;
```

Возвращаясь к нашей исходной VHDL-программе в табл. 9.13, отметим, что возможна еще одна интересная модификация. Исходной программой определяется обычный конечный автомат Мура со структурой, показанной на рис. 9.8(а). Что произойдет, если мы преобразуем оператор избирательного присваивания выходной логики в оператор `case` и переместим его в процесс, осуществляющий переходы из одного состояния в другое? Поступив так, мы создадим автомат, который в результате синтеза, вероятнее всего, будет иметь структуру, показанную на рис. 9.8(б). По существу, это автомат Мили с конвейерными выходами, и его поведение неотличимо от поведения исходного автомата, за исключением временных характеристик. Мы сократили задержку распространения от входа `CLOCK` до выхода `Z`, вырабатывая сигнал `Z` непосредственно на регистровом выходе, но одновременно с этим увеличилось требуемое время установления сигналов `A` и `B` по отношению к сигналу `CLOCK` из-за дополнительной задержки на прохождение сигнала через выходную логику ко входу `D` выходного регистра.

Все решения задачи построения рассматриваемого конечного автомата, о которых шла речь до сих пор, основывались на таблице состояний, которую мы первоначально составили вручную в разделе 7.4.1. Можно, однако, написать VHDL-программу непосредственно, без составления таблицы состояний вручную.

Основная идея упрощения следует из исходной формулировки задачи, приведенной в начале данного раздела, и состоит в исключении последнего значения входного сигнала `A` из определения состояний. Вместо этого предусматривается наличие отдельного регистра `LASTA` для отслеживания упомянутой величины. В этом случае необходимо определить только два состояния, помимо исходного состояния `INIT`: состояние `LOOKING` («смотреть дальше в ожидании совпадения») и состояние `OK` («имеет место совпадение двух последовательных значений `A` или сигнал `B` остается равным 1 с момента последнего совпадения»). VHDL-архитектура, реализующая этот подход приведена в табл. 9.16. В процессе, возбуждаемом сигналом

CLOCK, первый оператор присваивания создает регистр LASTA, а оператор case создает автомат с тремя состояниями. В конце программы выходной сигнал Z определяется как результат простого комбинационного обнаружения состояния ОК.

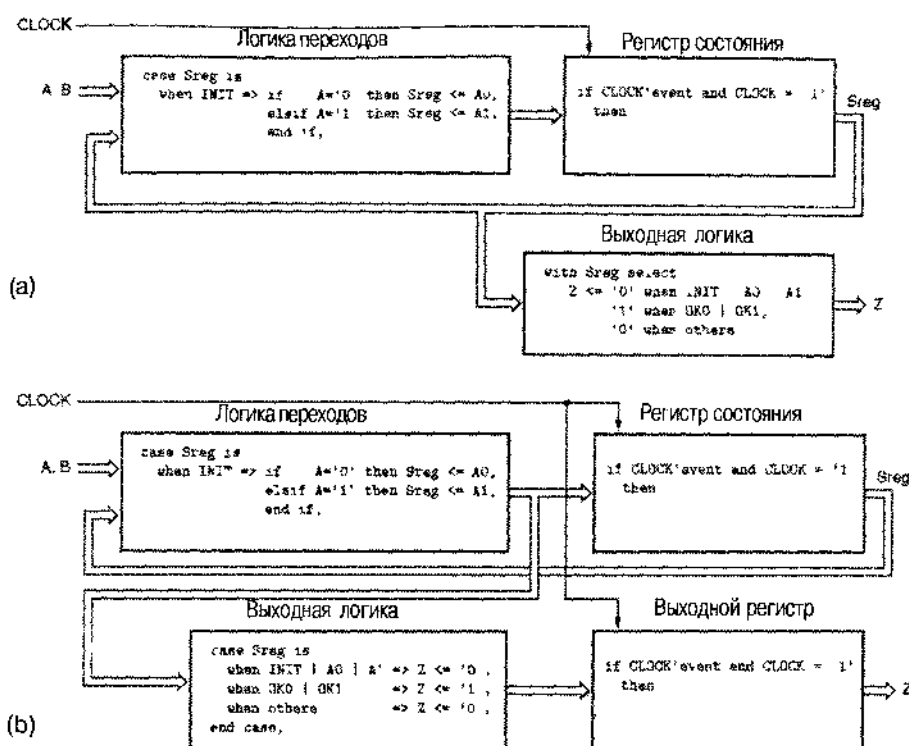


Рис. 9.8. Структура конечных автоматов, подразумеваемая VHDL-программами: (a) автомат Мура с комбинационной выходной логикой; (b) конвейерный автомат Мили с выходным регистром

КОВАРНОЕ ВРЕМЯ

При записи VHDL-архитектуры, соответствующей схеме на рис. 9.8(b), очень важно добавить сигнал Sreg в список чувствительности процесса. Действительно, значение Z определяется оператором case выходной логики как функция от значения Sreg. При первом исполнении процесса с приходом нарастающего фронта тактового сигнала значение Sreg повсюду соответствует старому состоянию автомата. Это имеет место потому, что Sreg является сигналом, а не переменной. Как было объяснено в разделе 4.7.9, сигналы, изменяющиеся внутри процесса, не принимают новых значений до тех пор, пока не будет сделан по крайней мере один элементарный сдвиг во времени после того, как процесс начал исполняться. Помещая Sreg в список чувствительности, мы гарантируем, что процесс будет повторен, так что конечное значение Z будет выработано с учетом нового значения Sreg.

Табл. 9.16. Упрощенный подход к построению рассматриваемого конечного автомата средствами VHDL

```

architecture smexampa_arch of smexamp is
type Sreg_type is (INIT, LOOKING, OK);
signal Sreg: Sreg_type;
signal lastA: STD_LOGIC;
begin
    process (CLOCK) -- state-machine states and transitions
    begin
        if CLOCK'event and CLOCK = '1' then
            lastA <= A;
            case Sreg is
                when INIT => Sreg <= LOOKING;
                when LOOKING => if A=lastA then Sreg <= OK;
                                else Sreg <= LOOKING;
                                end if;
                when OK => if B='1' then Sreg <= OK;
                            elsif A=lastA then Sreg <= OK;
                            else Sreg <= LOOKING;
                            end if;
                when others => Sreg <= INIT;
            end case;
        end if;
    end process;

    with Sreg select -- output values based on state
        Z <= '1' when OK,
              '0' when others;
end smexampa_arch;

```

Другим простым примером конечного автомата является «автомат, считающий число единиц». Задача формулируется так:

Построить тактируемый синхронный конечный автомат с двумя входами X и Y и одним выходом Z. Выходной сигнал должен равняться 1, если число единиц, поступивших на входы X и Y с момента запуска, кратно 4; в противном случае сигнал Z должен равняться 0.

В табл. 7.12 имеется составленная нами для этого автомата таблица состояний. Однако мы воспользуемся возможностями счета, предоставляемыми пакетом IEEE std_logic_arith, и напомним VHDL-программу для такого автомата непосредственно.

Как всегда существует много способов решить эту проблему. Наше решение представлено в табл. 9.17. Мы выбрали такой способ, который позволяет проиллюстрировать несколько различных особенностей языка. Внутри архитектуры объявлен подтип COUNTER для двухразрядных величин типа UNSIGNED. Подсчитываемое число единиц хранится в сигнале COUNT этого подтипа, а постоянная ZERO того же подтипа нужна для инициализации и проверки значения COUNT.

Табл. 9.17. VHDL-программа для автомата, считающего число единиц

```

library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.std_logic_arith.all;

entity Vonescnt is
  port ( CLOCK, RESET, X, Y: in STD_LOGIC;
        Z: out STD_LOGIC );
end;

architecture Vonescnt_arch of Vonescnt is
  subtype COUNTER is UNSIGNED (1 downto 0);
  signal COUNT: COUNTER;
  constant ZERO: COUNTER := "00";
begin
  process (CLOCK)
  begin
    if CLOCK'event and CLOCK = '1' then
      if RESET = '1' then COUNT <= ZERO;
      else COUNT <= COUNT + ('0', X) + ('0', X);
      end if;
    end if;
  end process;

  Z <= '1' when COUNT = ZERO else '0';
end Vonescnt_arch;

```

В процессе применен обычный метод обнаружения нарастающего фронта сигнала CLOCK. Предложением "if" выполняется синхронный сброс, а предложением else осуществляется простое добавление 0, 1 или 2 к содержимому COUNT, в зависимости от значений X и Y. Напомним, что выражение вида "('0', X)" – это массив-литерал; он образован здесь двумя элементами типа std_logic: нулем '0' и текущим значением X. Тип этого литерала совместим с типом UNSIGNED, поскольку число элементов у них и их тип одинаковы; поэтому их можно объединить операцией "+", определенной в пакете std_logic_arith. Параллельный сигнальный оператор присваивания, расположенный вне процесса, вырабатывает 1 на выходе типа Мура Z, когда значение COUNT равно 0.

С точки зрения синтеза оператор "if" и оператор присваивания значения сигналу COUNT не обязательно порождают компактную и быстродействующую схему. В случае простых средств синтеза это могут быть два 2-разрядных сумматора, соединенные последовательно. В табл. 9.18 показан другой подход, при котором «умные» средства оказываются способными синтезировать более компактную схему инкрементирования для каждого из двух сложений. В любом случае, представление в виде альтернатив оператора case позволяет двум сумматорам или схемам инкрементирования работать параллельно, а для переключения на один из выходов в соответствии с выбираемой альтернативой можно воспользоваться мультиплексором.

Табл. 9.18. Другой вариант процесса для автомата, считающего число единиц

```

process (CLOCK)
variable ONES: STD_LOGIC_VECTOR (1 to 2);
begin
  if CLOCK'event and CLOCK = '1' then
    ONES := (X, Y);
    if RESET = '1' then COUNT <= ZERO;
    else case ONES is
      when "01" | "10" => COUNT <= COUNT + "01";
      when "11"         => COUNT <= COUNT + "10";
      when others       => null;
    end case;
  end if;
end if;
end process;

```

Наш последний пример в этом разделе – конечный автомат, управляющий кодовым замком, из параграфа 7.4 (ниже выход HINT, имевшийся в исходном варианте, опущен):

Построить тактируемый синхронный конечный автомат с одним входом X и одним выходом UNLK. Сигнал на выходе UNLK должен принимать значение 1 тогда и только тогда, когда X равно 0 и последовательность значений входного сигнала X на семи предшествующих тактах имела вид: 0110111.

В табл. 7.14 приведена составленная нами таблица состояний. Но мы снова применим другой, более наглядный подход. Примем во внимание, что в данном случае сигнал на выходе автомата в любой момент времени полностью определяется значениями его входного сигнала на последних восьми тактах. Поэтому при проектировании этого автомата можно использовать так называемый принцип «конечной памяти» (см. помещенное в рамку замечание в конце раздела 9.1.2). В соответствии с этим принципом мы в явном виде отслеживаем семь последних значений входного сигнала и вырабатываем выходной сигнал как комбинационную функцию этих значений.

В табл. 9.19 приведена VHDL-программа, реализующая этот принцип. Архитектура содержит процесс, осуществляющий слежение за семью последними значениями X с помощью конструкции, являющейся, по существу, регистром сдвига, в котором бит на позиции с номером 7 представляет собой самое старое значение X. (Напомним, что оператор “&” в языке VHDL выполняет конкатенацию массивов.) Расположенный вне процесса параллельный сигнальный оператор присваивания вырабатывает 1 на выходе типа Мили UNLK, когда X равен 0, а семь предыдущих битов согласуются с ожидаемой комбинацией.

Табл. 9.19. VHDL-программа, реализующая принцип конечной памяти применительно к конечному автомату, управляющему кодовым замком

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vcomblock is
  port ( CLOCK, RESET, X: in STD_LOGIC;
        UNLK: out STD_LOGIC );
end;

architecture Vcomblock_arch of Vcomblock is
  signal XHISTORY: STD_LOGIC_VECTOR (7 downto 1);
  constant COMBINATION: STD_LOGIC_VECTOR (7 downto 1) := "0110111";
begin

  process (CLOCK)
  begin
    if CLOCK'event and CLOCK = '1' then
      if RESET = '1' then XHISTORY <= "0000000";
      else XHISTORY <= XHISTORY(6 downto 1) & X;
      end if;
    end if;
  end process;

  UNLK <= '1' when (XHISTORY=COMBINATION) and (X='0') else '0';

end Vcomblock_arch;

```

9.2.2. Задние огни автомобиля марки Ford Thunderbird

В параграфе 7.5 был описан и построен автомат для управления задними огнями автомобиля марки Ford Thunderbird. В табл. 9.20 представлена соответствующая VHDL-программа. Переходы автомата из одного состояния в другое происходят точно так же, как это изображено в виде диаграммы состояний на рис. 7.64. Применена запись состояний в форме выходного кода; это оказывается возможным, поскольку каждому из состояний автомата соответствует своя комбинация значений выходных сигналов, зажигающих задние фонари.

ВОЗМОЖНЫЕ УСОВЕРШЕНСТВОВАНИЯ

При описании конечного автомата на языке VHDL нет необходимости в явном присваивании следующего состояния, если оно остается тем же самым состоянием, в котором автомат уже находится. При исполнении процесса сигнал в языке VHDL сохраняет свое значение, если не выполняется присвоение ему нового значения. Поэтому заключительное предложение "else" в табл. 9.20 в состоянии IDLE можно было бы опустить, и это не повлияло бы на работу автомата.

Кроме того, можно повысить надежность конечного автомата, заменив оператор "null" в случае "when others" переходом в состояние IDLE.

Табл. 9.20. VHDL-программа для автомата, управляющего задними огнями автомобиля марки Ford Thunderbird

```
entity Vtbird is
    port ( CLOCK, RESET, LEFT, RIGHT, HAZ: in STD_LOGIC;
          LIGHTS: buffer STD_LOGIC_VECTOR (1 to 6) );
end;

architecture Vtbird_arch of Vtbird is
    constant IDLE: STD_LOGIC_VECTOR (1 to 6) := "000000";
    constant L3 : STD_LOGIC_VECTOR (1 to 6) := "111000";
    constant L2 : STD_LOGIC_VECTOR (1 to 6) := "110000";
    constant L1 : STD_LOGIC_VECTOR (1 to 6) := "100000";
    constant R1 : STD_LOGIC_VECTOR (1 to 6) := "000001";
    constant R2 : STD_LOGIC_VECTOR (1 to 6) := "000011";
    constant R3 : STD_LOGIC_VECTOR (1 to 6) := "000111";
    constant LR3 : STD_LOGIC_VECTOR (1 to 6) := "111111";
begin
    process (CLOCK)
    begin
        if CLOCK'event and CLOCK = '1' then
            if RESET = '1' then LIGHTS <= IDLE; else
                case LIGHTS is
                    when IDLE => if HAZ='1' or (LEFT='1' and RIGHT='1') then LIGHTS <= LR3;
                                elsif LEFT='1' then LIGHTS <= L1;
                                elsif RIGHT='1' then LIGHTS <= R1;
                                else LIGHTS <= IDLE;
                                end if;
                    when L1 => if HAZ='1' then LIGHTS <= LR3; else LIGHTS <= L2; end if;
                    when L2 => if HAZ='1' then LIGHTS <= LR3; else LIGHTS <= L3; end if;
                    when L3 => LIGHTS <= IDLE;
                    when R1 => if HAZ='1' then LIGHTS <= LR3; else LIGHTS <= R2; end if;
                    when R2 => if HAZ='1' then LIGHTS <= LR3; else LIGHTS <= R3; end if;
                    when R3 => LIGHTS <= IDLE;
                    when LR3 => LIGHTS <= IDLE;
                    when others => null;
                end case;
            end if;
        end process;
    end Vtbird_arch;
```

9.2.3. Игра на угадывание

Задача построения автомата для «игры на угадывание» была сформулирована следующим образом:

Построить тактируемый синхронный автомат с четырьмя входами G1–G4, подключенными к кнопкам. У автомата четыре выхода L1–L4, к которым подключены лампочки или светодиоды, расположенные рядом с кнопками с теми же номерами. Имеется также выход ERR, к которому подключена красная лампочка. При нормальной работе на выходах L1–L4 индицируется комбинация «1 из 4». На каждом такте комбинация сдвигается на одну позицию; частота тактового сигнала равна 4 Гц.

Задача игрока состоит в том, чтобы вовремя нажать кнопку, соответствующую горящей лампочке. При нажатии *i*-ой кнопки вырабатывается единичный сигнал Gi. Если подан «неправильный» сигнал, то возникает сигнал на выходе ERR и загорается красная лампочка, это происходит в том случае, когда автомат на очередном такте обнаруживает сигнал, номер которого не совпадает с номером лампочки, зажженной на предыдущем такте. Когда кнопка нажата, игра останавливается, и сигнал на выходе ERR сохраняет свое значение в течение одного или нескольких тактов, пока не будет снят удерживаемый вами сигнал Gi, и тогда игра возобновляется.

Как мы видели в разделе 7.7.1, у этого автомата шесть состояний: четыре состояния соответствуют зажженным лампочкам, а два – для тех случаев, когда игра остановлена после правильного или ошибочного нажатия кнопки. VHDL-программа для игры на угадывание приведена в табл. 9.21. В этом варианте устройство имеет также вход RESET: сигнал на этом входе заставляет устройство перейти в известное начальное состояние.

Эта программа является почти непосредственным переводом на язык VHDL диаграммы состояний, изображенной на рис. 7.66. Единственной, по-видимому, ее особенностью, которая заслуживает упоминания, является случай “SOK | SEER”. Поскольку переходы из этих двух состояний в следующие состояния совершенно одинаковы (нужно переходить в состояние S1 или оставаться в текущем состоянии), их можно обрабатывать как один и тот же случай. Однако эта хитрость, позволяющая уменьшить число строк в тексте программы, нежелательна, в частности, с точки зрения документации на этот конечный автомат и его отладки. Но автору эта хитрость позволила сократить размер программы до одной страницы в книге!

В программе, приведенной в табл. 9.21, кодирование состояний не задано; типичный синтезатор использует три бита для Sreg и кодирует шесть состояний в порядке следования двоичных комбинаций 000–101. Применительно к этому конечному автомату можно воспользоваться также записью состояний в форме выходного кода, то есть представить их с помощью уже имеющихся сигналов зажигания лампочек и сигнала ошибки. В языке VHDL нет удобного механизма для объединения выходных сигналов, определенных в данном объекте, в одно целое и представления ими состояний, но этого можно все-таки достичь так, как показано в табл. 9.22. Соответствие между входными сигналами и битами в новом 5-разрядном регистре Sreg указано в комментариях, а операторы присваивания значений выходным сигналам видоизменены таким образом, чтобы выбирать подходящий бит, а не обнаруживать состояние в целом.

Табл. 9.21. VHDL-программа, описывающая автомат для игры на угадывание

```

library IEEE;
use IEEE std_logic_1164.all;

entity Vggame is
  port ( CLOCK, RESET, G1, G2, G3, G4: in  STD_LOGIC;
        L1, L2, L3, L4, ERR:          out STD_LOGIC );
end;

architecture Vggame_arch of Vggame is
  type Sreg_type is (S1, S2, S3, S4, SOK, SERR);
  signal Sreg: Sreg_type;
begin

  process (CLOCK)
  begin
    if CLOCK'event and CLOCK = '1' then
      if RESET = '1' then Sreg <= SOK; else
        case Sreg is
          when S1 => if   G2='1' or G3='1' or G4='1' then Sreg <= SERR;
                     elsif G1='1'                      then Sreg <= SOK;
                     else                                Sreg <= S2;
                     end if;
          when S2 => if   G1='1' or G3='1' or G4='1' then Sreg <= SERR;
                     elsif G1='1'                      then Sreg <= SOK;
                     else                                Sreg <= S3;
                     end if;
          when S3 => if   G1='1' or G2='1' or G4='1' then Sreg <= SERR;
                     elsif G1='1'                      then Sreg <= SOK;
                     else                                Sreg <= S4;
                     end if;
          when S4 => if   G1='1' or G2='1' or G3='1' then Sreg <= SERR;
                     elsif G1='1'                      then Sreg <= SOK;
                     else                                Sreg <= S1;
                     end if;
          when SOK | SERR => if G1='0' and G2='0' and G3='0' and G4='0'
                           then Sreg <= S1; end if;
          when others => Sreg <= S1;
        end case;
      end if;
    end if;
  end process;

  L1 <= '1' when Sreg = S1  else '0';
  L2 <= '1' when Sreg = S2  else '0';
  L3 <= '1' when Sreg = S3  else '0';
  L4 <= '1' when Sreg = S4  else '0';
  ERR <= '1' when Sreg = SERR else '0';

end Vggame_arch;

```

Табл. 9.22. VHDL-архитектура для игры на угадывание с записью состояний в форме выходного кода

```

architecture Vggameoc_arch of Vggame is
  signal Sreg STD_LOGIC_VECTOR (1 to 5);
  -- bit positions of output coded assignment L1, L2, L3, L4, ERR
  constant S1 STD_LOGIC_VECTOR (1 to 5) = "10000",
  constant S2 STD_LOGIC_VECTOR (1 to 5) = "01000",
  constant S3 STD_LOGIC_VECTOR (1 to 5) = "00100",
  constant S4 STD_LOGIC_VECTOR (1 to 5) = "00010",
  constant SERR STD_LOGIC_VECTOR (1 to 5) = "00001",
  constant SOK STD_LOGIC_VECTOR (1 to 5) = "00000",
begin

  process (CLOCK)
    . (no change to process)
  end process;

  L1 <= Sreg(1);
  L2 <= Sreg(2);
  L3 <= Sreg(3);
  L4 <= Sreg(4);
  ERR <= Sreg(5);

end Vggameoc_arch;

```

9.2.4. Продолжение работы над контроллерами светофоров

Тем из вас, кто прочел пример из раздела 9.1.5, уже известны мои разглагольствования об ужасных контроллерах светофоров в г. Саннивейл, шт. Калифорния. Ситуация на самом деле выпадит такой, как если бы контроллеры были *специально* спроектированы так, чтобы сделать возможно большим время простоя автомобилей на перекрестках. В этом разделе мы разработаем контроллер для светофора, поведение которого будет подчеркнуто нацеливать работу светофоров в г. Саннивейл.

На незагруженном перекрестке имеются датчики движения и светофоры, показанные на рис. 9.5. (Если бы это было в Чикаго, то самое большее, чем был бы отмечен такой перекресток, — это знак «уступи дорогу».) Светофорами управляет автомат с частотой тактового сигнала 1 Гц; у него имеются таймер и четыре входа:

NSCAR	Сигнал принимает активное значение, если хотя бы один автомобиль находится в поле действия одного из датчиков в направлении движения «север-юг» по любую сторону перекрестка.
EWCAR	Сигнал принимает активное значение, если хотя бы один автомобиль находится в поле действия одного из датчиков в направлении движения «восток-запад» по любую сторону перекрестка.
TMLONG	Сигнал принимает активное значение, если с момента начала работы таймера прошло более пяти минут; он остается на активном уровне до тех пор, пока таймер не будет сброшен.

TMSHORT Сигнал принимает активное значение, если с момента начала работы таймера прошло более пяти секунд; он остается на активном уровне до тех пор, пока таймер не будет сброшен.

У этого конечного автомата семь выходных сигналов:

NSRED, NSYELLOW, NSGREEN Сигналы, зажигающие красный, желтый и зеленый свет в направлении «север-юг» соответственно.

EWRED, EWYELLOW, EWGREEN Сигналы, зажигающие красный, желтый и зеленый свет в направлении «восток-запад» соответственно.

TMRESET Когда этот сигнал принимает активное значение, таймер сбрасывается, а сигналы **TMSHORT** и **TMLONG** переходят на неактивный уровень. Таймер начинает отсчет времени, когда на неактивный уровень переходит сигнал **TMRESET**.

VHDL-программой, приведенной в табл. 9.23, реализуется типичный, одобренный местными властями алгоритм управления светофорами. Этот алгоритм обеспечивает два часто наблюдаемых режима работы нашего «проворного» светофора. Ночью, при малой интенсивности движения, он удерживает автомобиль в состоянии ожидания до пяти минут, если только не появляется автомобиль на поперечной уличной; в этом случае светофор переключается так, чтобы остановить движение в поперечном направлении и пропустить ожидающий автомобиль. (Датчик «раннего оповещения» установлен достаточно далеко, чтобы сигналы светофора успели измениться до того, как приближающийся автомобиль достигнет перекрестка.) Днем, при напряженном движении всегда имеются автомобили, ожидающие проезда в обоих направлениях; тогда светофор переключается каждые пять секунд, чтобы минимизировать пропускную способность перекрестка и максимизировать время ожидания для всех, подталкивая тем самым возмущенную общественность к мысли о необходимости новшества налогов для решения этой проблемы.

При написании этой программы мы воспользовались имевшейся возможностью и добавили контроллеру вход **OVERRIDE**. Подавая сигнал на этот вход, полицейский может заблокировать работу контроллера и заставить светофор мигать красным светом (с частотой тактового сигнала **FLASHCLK**), и тогда у него появляется возможность вручную растаскивать пробки, возникающие благодаря этому удивительному изобретению.

Как и в большинстве других наших примеров, в табл. 9.23 не указано какое-либо определенное кодирование состояний, и, подобно многим другим примерам, этот конечный автомат хорошо работает при записи состояний в форме выходного кода. Многие состояния можно отождествить с единственной комбинацией значений выходных сигналов, зажигающих огни светофора. Но существуют также три пары неразличимых состояний, если иметь в виду только световые сигналы: (**NSWAIT**, **NSWAIT2**), (**EWWAIT**, **EWWAIT2**) и (**NSDELAY**, **EWDELAY**). Это затруднение можно преодолеть, добавив еще одну переменную состояния «EXTRA» с различными значениями в состояниях, образующих каждую пару. Эта идея реализована в видоизмененной программе, приведенной в табл. 9.24.

Табл. 9.23. VHDL-программа для контроллера светофора в г. Саннивейл

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vsval is
  port ( CLOCK, RESET, NSCAR, EWCAR, TMSHORT, TMLONG: in  STD_LOGIC;
        OVERRIDE, FLASHCLK: in  STD_LOGIC;
        NSRED, NSYELLOW, NSGREEN: out STD_LOGIC;
        EWRED, EWYELLOW, EWGREEN, TMRESET: out STD_LOGIC );
end;

architecture Vsval_arch of Vsval is
  type Sreg_type is ( NSGO, NSWAIT, NSWAIT2, NSDELAY,
                     EWGO, EWAIT, EWAIT2, EWDELAY );
  signal Sreg: Sreg_type;
begin

  process (CLOCK)
  begin
    if CLOCK'event and CLOCK = '1' then
      if RESET = '1' then Sreg <= NSDELAY; else
        case Sreg is
          when NSGO =>
            if TMSHORT='0' then Sreg <= NSGO;
            elsif TMLONG='1' then Sreg <= NSWAIT;
            elsif EWCAR='1' and NSCAR='0' then Sreg <= NSGO;
            elsif EWCAR='1' and NSCAR='1' then Sreg <= NSWAIT;
            elsif EWCAR='0' and NSCAR='1' then Sreg <= NSWAIT;
            else Sreg <= NSGO;
            end if;
          when NSWAIT => Sreg <= NSWAIT2;
          when NSWAIT2 => Sreg <= NSDELAY;
          when NSDELAY => Sreg <= EWGO;
          when EWGO =>
            if TMSHORT='0' then Sreg <= EWGO;
            elsif TMLONG='1' then Sreg <= EWAIT;
            elsif NSCAR='1' and EWCAR='0' then Sreg <= EWGO;
            elsif NSCAR='1' and EWCAR='1' then Sreg <= EWAIT;
            elsif NSCAR='0' and EWCAR='1' then Sreg <= EWAIT;
            else Sreg <= EWGO;
            end if;
          when EWAIT => Sreg <= EWAIT2;
          when EWAIT2 => Sreg <= EWDELAY;
          when EWDELAY => Sreg <= NSGO;
          when others => Sreg <= NSDELAY;
        end case;
      end if;
    end if;
  end process;

  TMRESET <= '1' when Sreg=NSWAIT2 or Sreg=EWAIT2 else '0';
  NSRED <= FLASHCLK when OVERRIDE='1' else
    '1' when Sreg/=NSGO and Sreg/=NSWAIT and Sreg/=NSWAIT2 else '0';
  NSYELLOW <= '0' when OVERRIDE='1' else
    '1' when Sreg=NSWAIT or Sreg=NSWAIT2 else '0';
  NSGREEN <= '0' when OVERRIDE='1' else '1' when Sreg=NSGO else '0';
  EWRED <= FLASHCLK when OVERRIDE='1' else
    '1' when Sreg/=EWGO and Sreg/=EWAIT and Sreg/=EWAIT2 else '0';
  EWYELLOW <= '0' when OVERRIDE='1' else
    '1' when Sreg=EWAIT or Sreg=EWAIT2 else '0';
  EWGREEN <= '0' when OVERRIDE='1' else '1' when Sreg=EWGO else '0';

end Vsval_arch;

```

Табл. 9.24. Определения для автомата, управляющего светофором в г. Сан-нивейл, при записи состояний в форме выходного кода

```

library IEEE;
use IEEE.std_logic_1164.all;

entity Vsvalc is
  port ( CLOCK, RESET, NSCAR, EWCAR, TMSHORT, TMLONG: in  STD_LOGIC;
        OVERRIDE, FLASHCLK: in  STD_LOGIC;
        NSRED, NSYELLOW, NSGREEN: out STD_LOGIC;
        EWRED, EWYELLOW, EWGREEN, TMRESET: out STD_LOGIC );
end;

architecture Vsvalc_arch of Vsvalc is
  signal Sreg: STD_LOGIC_VECTOR (1 to 7);
  -- bit positions of output-coded assignment: (1) NSRED, (2) NSYELLOW, (3) NSGREEN,
  --                                           (4) EWRED, (5) EWYELLOW, (6) EWGREEN, (7) EXTRA
  constant NSGO: STD_LOGIC_VECTOR (1 to 7) := "0011000";
  constant NSWAIT: STD_LOGIC_VECTOR (1 to 7) := "0101000";
  constant NSWAIT2: STD_LOGIC_VECTOR (1 to 7) := "0101001";
  constant NSDELAY: STD_LOGIC_VECTOR (1 to 7) := "1001000";
  constant EWGO: STD_LOGIC_VECTOR (1 to 7) := "1000010";
  constant EWWAIT: STD_LOGIC_VECTOR (1 to 7) := "1000100";
  constant EWWAIT2: STD_LOGIC_VECTOR (1 to 7) := "1000101";
  constant EWDELAY: STD_LOGIC_VECTOR (1 to 7) := "1001001";

  begin

  process (CLOCK)
  ... (no change to process)
  end process;

  TMRESET <= '1' when Sreg=NSWAIT2 or Sreg=EWWAIT2 else '0';
  NSRED <= Sreg(1);
  NSYELLOW <= Sreg(2);
  NSGREEN <= Sreg(3);
  EWRED <= Sreg(4);
  EWYELLOW <= Sreg(5);
  EWGREEN <= Sreg(6);

end Vsvalc_arch;

```

Задачи

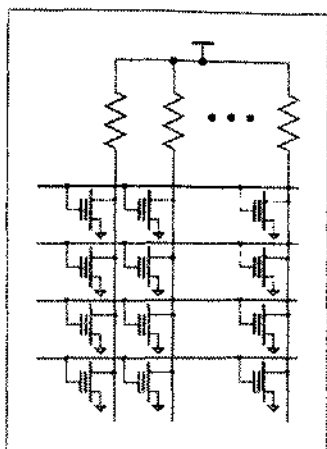
- 9.1. Напишите программу на языке ABEL для конечного автомата, описанного в задаче 7.30.
- 9.2. Видоизмените программу на языке ABEL, приведенную в табл. 9.2, добавив выход HINT согласно первоначальному описанию этого конечного автомата в параграфе 7.4.
- 9.3. Постройте заново автомат для управления задними огнями автомобиля марки Ford Thunderbird, у которого, в отличие от автомата из раздела 9.1.3, было бы предусмотрено включение стоп-сигнала и габаритных огней. Когда подаст входной сигнал BRAKE («тормоз»), все фонари должны загореться.

ся немедленно и оставаться включенными до тех пор, пока сигнал BRAKE не будет снят, независимо от каких-либо условий. При возникновении сигнала на входе PARK («парковка») каждый фонарь горит в половине яркости в течение всего времени, пока система не перейдет в другой режим. Это достигается подачей на фонарь сигнала DIMCLK с частотой 100 Гц и 50-процентным коэффициентом заполнения. Начертите принципиальную схему автомата на одном или двух ПЛУ, напишите программу на языке ABEL и представьте краткое описание того, как работает ваша система.

- 9.4. Найдите такое 3-разрядное кодирование состояний автомата для игры на угадывание из табл. 9.5, при котором максимальное число термов-произведений, приходящихся на один выход, сокращается до 7. Можно ли сделать еще лучше?
- 9.5. Работа автомата для игры на угадывание из раздела 9.1.4 полностью предсказуема; игроку нетрудно подстроиться под темп, с которым переключаются лампочки, и всегда нажимать на кнопку в нужный момент. Игра становится более забавной, если темп переключения сделать меняющимся.

Вспомогите программу на языке ABEL, приведенную в табл. 9.5, так, чтобы выход конечного автомата из состояний S1–S4 происходил только тогда, когда подан новый входной сигнал SEN. (Предполагается, что сигнал SEN поступает с выхода генератора псевдослучайного потока битов.) Правильное и ошибочное нажатие кнопки должно распознаваться независимо от того, подан сигнал SEN или нет. Посмотрите, помещается ли по-прежнему ваше устройство в одной ИС 16V8?
- 9.6. Напишите программу на языке ABEL для 8-разрядного регистра с линейной обратной связью в расчете на реализацию в одной ИС 16V8, которым можно было бы воспользоваться в качестве генератора псевдослучайного двоичного сигнала в условиях предыдущей задачи. Чему равен период двоичной последовательности, выраженный числом тактов? Каково максимальное число нулей, следующих подряд? А число единиц?
- 9.7. Добавьте конечному автомату, представленному на рис. 9.7, вход OVERRIDE, не выходя за пределы одной ИС 16V8. Когда на этот вход подан сигнал, красные фонари светофоров должны мигать, включаясь и выключаясь раз в секунду. Напишите полную программу на языке ABEL для вашего автомата.
- 9.8. Видоизмените программу из табл. 9.9 для автомата, управляющего светофором, заставив светофор переключаться так, как вам хотелось бы, чтобы это происходило в вашем собственном городе.
- 9.9. Напишите VHDL-программу для конечного автомата, описанного в задаче 7.30.
- 9.10. Покажите, как следует изменить программу в табл. 9.13, чтобы у устройства был асинхронный вход RESET, возникновение сигнала на котором заставляло бы конечный автомат переходить в состояние INIT. Повторите эту задачу для синхронного входа сброса так, чтобы конечный автомат переводился в состояние INIT в случае, когда сигнал RESET принимает активное значение по нарастающему фронту тактового сигнала.

- 9.11. Напишите и испытайте VHDL-программу для устройства, изображенного на рис. 9.8(b). Посмотрите с помощью имеющихся средств синтеза, отличается ли схема, показанная на рис. 9.8(b), от схемы, показанной на рис. 9.8(a), и в чем состоит это отличие.
- 9.12. Напишите VHDL-программу для конечного автомата, считающего число единиц, с таблицей состояний, приведенной в табл. 7.12.
- 9.13. Напишите VHDL-программу для конечного автомата, управляющего кодовым замком, с таблицей состояний, приведенной в табл. 7.14.
- 9.14. Видоизмените VHDL-программу из таблицы 9.19, добавив выход HINT согласно первоначальному описанию этого конечного автомата в параграфе 7.4.
- 9.15. Повторите задачу 9.3 на языке VHDL, предполагая, что ваше устройство будет реализовано в одной ИС типа CPLD или FPGA.
- 9.16. Видоизмените VHDL-программу для игры на угадывание из табл. 9.21 согласно идее, изложенной в задаче 9.5. Добавьте в программу еще один процесс для генератора псевдослучайной двоичной последовательности, о котором говорится в задаче 9.6.
- 9.17. Видоизмените режим работы управляющего светофором автомата, описываемого VHDL-программой из табл. 9.23, заставив светофор вести себя более разумно и переключаться так, как вам хотелось бы, чтобы это происходило в вашем собственном городе.



Глава 10

ПАМЯТЬ И МИКРОСХЕМЫ ТИПА CPLD И FPGA

Любая последовательностная схема обладает своего рода памятью, поскольку триггер или защелка хранят один бит информации. Однако мы оставляем использование слова «память» на случай, когда биты хранятся в структурированной форме, обычно в виде двумерного массива, в котором сразу доступна целая строка битов.

В этой главе рассматриваются несколько разных способов организации памяти и имеющиеся в продаже микросхемы памяти. Память может входить также в состав СБИС, где для выполнения необходимой функции она объединена с другими схемами.

Применения памяти многочисленны и разнообразны. Постоянное запоминающее устройство (ПЗУ) может применяться в центральном процессоре (ЦП) микропроцессорной системы для хранения информации об элементарных шагах, совершаемых при выполнении команд из набора команд этого ЦП. Быстрая «статическая память», расположенная рядом с ЦПУ, может служить в качестве кэша для хранения недавно использованных команд и данных. А микропроцессорные подсистемы основной памяти могут содержать сотни миллионов ячеек «динамической памяти», в которых хранятся операционные системы, программы и данные.

Применение памяти не ограничивается микропроцессорными системами и даже чисто цифровыми системами. В аппаратуре телефонных систем общего пользования, например, для выполнения некоторых преобразований оцифрованных речевых сигналов применяются ПЗУ, а быстрая «статическая память» используется в качестве «переключающего устройства» при маршрутизации цифровых сообщений, посредством которых осуществляется связь между абонентами. Многие портативные игровые звуковых компакт-дисков осуществляют «чтение вперед» и в течение нескольких секунд хранят звуковой фрагмент в «динамической памяти», благодаря чему звучание продолжается даже при тряске (для сохранения звукового фрагмента длительностью в 1 секунду требуется около 1,4 миллиона битов). Кроме этого существует много других примеров современной аудио- и видеоаппаратуры, в которой память применяется для временного хранения оцифрованных сигналов при их обработке в цифровых сигнальных процессорах.

Эта глава начинается с рассмотрения ПЗУ и их применений, а затем описываются два типа оперативных запоминающих устройств (ОЗУ). В необязательных для чтения разделах приводится внутренняя структура памяти различного типа.

Последние два параграфа этой главы посвящены микросхемам типа CPLD и FPGA. Эти устройства похожи на память тем, что представляют собой большие регулярные структуры, которые можно использовать самым различным образом. Благодаря возможности очень быстро разрабатывать на основе таких ИС заказные логические устройства, эти микросхемы становятся основными составными компонентами в современном цифровом проектировании.

10.1. Постоянные запоминающие устройства

Постоянное запоминающее устройство (ПЗУ; *read-only memory, ROM*) является комбинационной схемой с n входами и b выходами (рис. 10.1). Входы называются *адресными входами* (*address inputs*) и обычно обозначаются $A_0, A_1, \dots, A_{n-1}$. Выходы называются *выходами данных* (*data outputs*) и обозначаются, как правило, $D_0, D_1, \dots, D_{b-1}$.

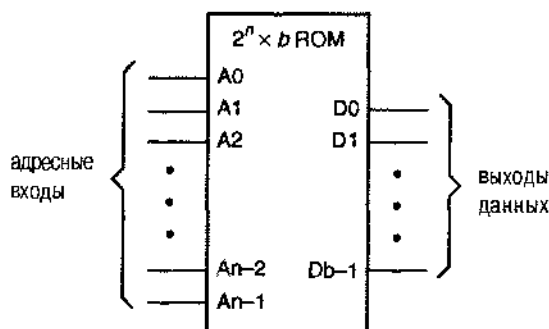


Рис. 10.1. Стандартная структура ПЗУ размером $2^n \times b$

ПЗУ «хранит» таблицу истинности комбинационной логической схемы с n входами и b выходами. В табл. 10.1 представлена таблица истинности комбинационной схемы с 3 входами и 4 выходами. Содержимое этой таблицы можно хранить в ПЗУ размером $2^3 \times 4 = (8 \times 4)$. Пренебрегая задержками распространения, можно сказать, что сигналы на выходах данных в любой момент времени равны битам той строки таблицы истинности, которая выбирается сигналами на адресных входах.

ЭТО НЕ ПАМЯТЬ!

На самом деле, в большинстве случаев ПЗУ не является памятью в прямом смысле этого слова, поскольку это комбинационная, а не последовательностная схема. Постоянные запоминающие устройства называют «памятью», потому что они функционируют как память.

Табл. 10.1. Таблица истинности для комбинационной логической схемы с 3 входами и 4 выходами

Входы			Выходы			
A2	A1	A0	D3	D2	D1	D0
0	0	0	1	1	1	0
0	0	1	1	1	0	1
0	1	0	1	0	1	1
0	1	1	0	1	1	1
1	0	0	0	0	0	1
1	0	1	0	0	1	0
1	1	0	0	1	0	0
1	1	1	1	0	0	0

Поскольку ПЗУ представляет собой комбинационную схему, вы вправе сказать, что оно вовсе не является настоящей памятью. Описывая работу ПЗУ, можно считать, что это устройство подобно любому другому комбинационному логическому элементу. Однако можно также считать, что в ПЗУ при изготовлении или программировании была «запомнена» определенная информация (о том, как это делается, мы расскажем подробнее в разделе 10.1.4).

Хотя мы полагаем, что ПЗУ – это один из типов памяти, важно отличать его от других интегральных схем памяти. ПЗУ является *энергонезависимой памятью* (*nonvolatile memory*), то есть ее содержимое сохраняется в отсутствие напряжения питания.

10.1.1. Применение ПЗУ для реализации «произвольных» комбинационных логических функций

Табл. 10.1, в действительности, представляет собой таблицу истинности для дешифратора 2х4 с управляемой полярностью выходных сигналов, то есть схемы, которую можно построить из дискретных вентилях, как показано на рис. 10.2. Итак, имеется два разных способа построения дешифратора: с помощью дискретных вентилях или на основе ПЗУ 8х4, в котором хранится таблица истинности (рис. 10.3).

Присвоение имен входов и выходов дешифратора входам и выходам ПЗУ на рис. 10.3 основано на том, как построена таблица истинности (табл. 10.1). Другими словами, такая физическая реализация дешифратора с помощью ПЗУ не является единственно возможной. Таким образом, для реализации той же самой логической функции можно записать строки или столбцы в таблице истинности в другом порядке и применить другое ПЗУ, просто присваивая имена сигналов дешифратора другим входам и выходам ПЗУ. Другой взгляд на эту проблему состоит в том, что можно переименовать отдельные адресные входы и выходы данных ПЗУ.

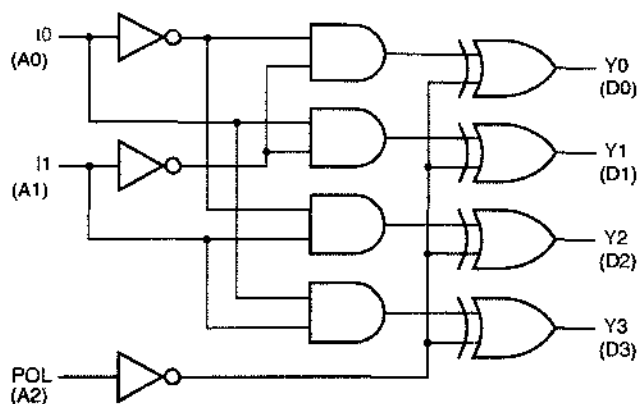


Рис. 10.2. Дешифратор 2×4 с управляемой полярностью выходных сигналов

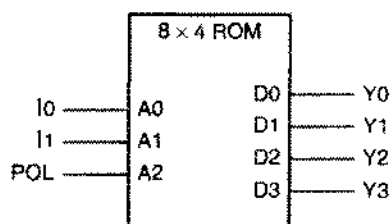


Рис. 10.3. Использование ПЗУ 8×4, в котором записана табл. 10.1, в качестве дешифратора 2×4

Например, переставив местами столбцы D0 и D3 в табл. 10.1, мы получим таблицу истинности другого ПЗУ. Однако для построения дешифратора 2×4 все же можно воспользоваться и новым ПЗУ, просто поменяв местами обозначения Y0 и Y3 по рис. 10.3. Аналогично, можно переставить местами строки данных в таблице истинности, как показано в табл. 10.2, и тогда мы получим еще одно ПЗУ, отличное от предыдущих, но и оно может быть использовано в качестве дешифратора 2×4 с другим распределением адресных входов: A0 = POL, A1 = I0, A2 = I1.

Табл. 10.2. Таблица истинности с переставленными строками данных

Входы			Выходы			
A2	A1	A0	D3	D2	D1	D0
0	0	0	1	1	1	0
0	0	1	0	0	0	1
0	1	0	1	1	0	1
0	1	1	0	0	1	0
1	0	0	1	0	1	1
1	0	1	0	1	0	0
1	1	0	0	1	1	1
1	1	1	1	0	0	0

При занесении в ПЗУ таблицы истинности биты входных и выходных сигналов читаются в таблице истинности справа налево, а комбинации битов, соответствующих адресным входам и выходам данных обычно перебираются в порядке возрастания. Комбинации битов в адресе и в данных можно считать целыми двоичными числами с «естественным» порядком следования. В типичном случае для записи таблицы истинности в ПЗУ при его изготовлении или программировании используется файл данных. При этом значения адреса и данных задаются в файле данных в виде шестнадцатеричных чисел. Например, табл. 10.2 можно описать в файле данных, сообщив, что по адресам с 0 по 7 в ПЗУ должны храниться значения E, 1, D, 2, B, 4, 7, 8.

Другим простым примером функции, которую можно реализовать с помощью ПЗУ, является умножение 4-разрядных двоичных целых чисел без знака. В разделе 5.11.2 была приведена программа на языке ABEL для такого умножителя и было показано, что требуемое число термов-произведений (36) слишком велико, чтобы выполнить эту операцию за один проход через матрицу вентилей И-ИЛИ обычного ПЛУ. В качестве альтернативы эту функцию можно реализовать за один проход, применяя ПЗУ $2^8 \times 8$ (256×8), включенное так, как показано на рис. 10.4.

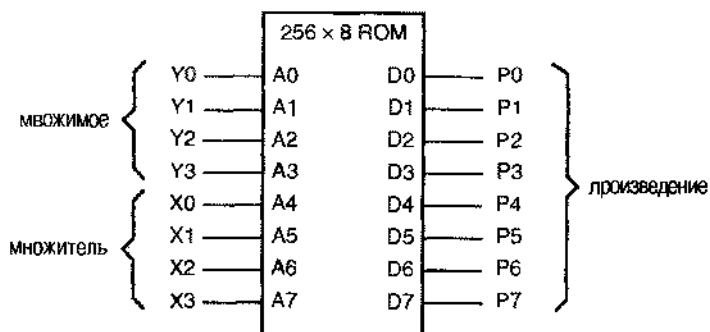


Рис. 10.4. Использование ПЗУ 256×8 для умножения 4-разрядных двоичных чисел без знака

Обычно содержимое ПЗУ задается файлом, содержащим по одному элементу для каждого адреса в ПЗУ. В табл. 10.3, например, в шестнадцатеричном коде представлено содержимое ПЗУ, на котором реализуется умножитель 4×4 . Каждая строка начинается с адреса ячейки в ПЗУ и содержит значения 8-разрядных данных, запоминаемых по 16 последующим адресам.

У разработок, основанных на применении ПЗУ, есть приятная особенность: обычно можно написать простую программу на языке высокого уровня для вычисления того, что должно быть запомнено в ПЗУ. Например, требуется лишь несколько минут, чтобы написать программу на языке C (табл. 10.4), с помощью которой формируется содержимое табл. 10.3.

Табл. 10.3. Текстовый файл в шестнадцатеричном коде, задающий содержимое ПЗУ, используемого в качестве умножителя 4x4

```

00: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
10: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
20: 00 02 04 06 08 0A 0C 0E 10 12 14 16 18 1A 1C 1E
30: 00 03 06 09 0C 0F 12 15 18 1B 1E 21 24 27 2A 2D
40: 00 04 08 0C 10 14 18 1C 20 24 28 2C 30 34 38 3C
50: 00 05 0A 0F 14 19 1E 23 28 2D 32 37 3C 41 46 4B
60: 00 06 0C 12 18 1E 24 2A 30 36 3C 42 48 4E 54 5A
70: 00 07 0E 15 1C 23 2A 31 38 3F 46 4D 54 5B 62 69
80: 00 08 10 18 20 28 30 38 40 48 50 58 60 68 70 78
90: 00 09 12 1B 24 2D 36 3F 48 51 5A 63 6C 75 7E 87
A0: 00 0A 14 1E 28 32 3C 46 50 5A 64 6E 78 82 8C 96
B0: 00 0B 16 21 2C 37 42 4D 58 63 6E 79 84 8F 9A A5
C0: 00 0C 18 24 30 3C 48 54 60 6C 78 84 90 9C A8 B4
D0: 00 0D 1A 27 34 41 4E 5B 68 75 82 8F 9C A9 B6 C3
E0: 00 0E 1C 2A 38 46 54 62 70 7E 8C 9A A8 B6 C4 D2
F0: 00 0F 1E 2D 3C 4B 5A 69 78 87 96 A5 B4 C3 D2 E1

```

Табл. 10.4. Программа, создающая текстовый файл, которым задается содержимое ПЗУ, используемого в качестве умножителя 4x4

```

#include <stdio.h>

/* Procedure to print d as a hex digit. */
void PrintHexDigit(int d)
{
    if (d<10) printf("%c", '0'+d);
    else printf("%c", 'A'+d-10);
}

/* Procedure to print i as two hex digits. */
void PrintHex2(int i)
{
    PrintHexDigit((i / 16) % 16);
    PrintHexDigit(i % 16);
}

void main()
{
    int x, y;

    for (x=0; x<=16; x++) {
        PrintHex2(x*16); printf(":");
        for (y=0; y<=15; y++) {
            printf(" ");
            PrintHex2(x*y);
        }
        printf("\n");
    }
}

```

*10.1.2. Внутренняя структура ПЗУ

Механизм «хранения» информации применяемый в ПЗУ различен для ПЗУ разных типов. В большинстве ПЗУ наличие или отсутствие диода или транзистора воспринимается как 0 или 1.

На рис. 10.5 приведена схема простейшего ПЗУ размером 8×4, которую вы можете собрать сами, применяя ИС средней степени интеграции в качестве дешифратора и небольшое число диодов. Сигналами на адресных входах активизируется один из выходов дешифратора. Каждый выход дешифратора называется *линией слова* (word line): сигналом на этой линии выбирается одна строка или одно слово таблицы, хранимой в ПЗУ. На рисунке показан случай, когда A2–A0 = 101, и активное значение имеет сигнал на выходе ROW5_L.

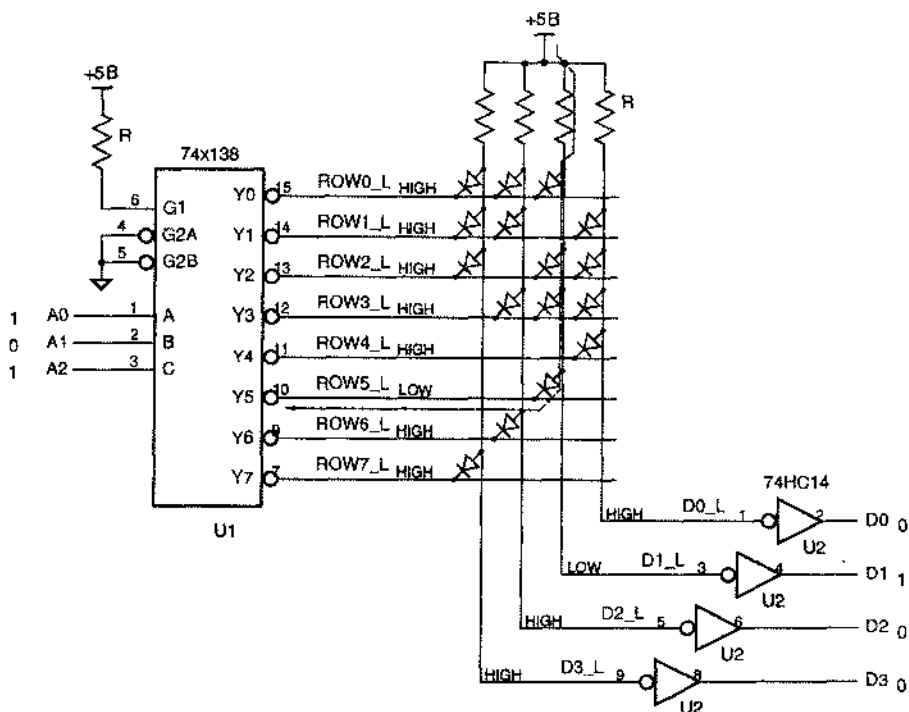


Рис. 10.5. Принципиальная схема простого диодного ПЗУ 8×4 (LOW – низкий уровень, HIGH – высокий уровень)

Каждая вертикальная линия на рис. 10.5 называется *линией бита* (bit line), так как она соответствует одному выходному биту ПЗУ. Появление низкого активного уровня LOW на линии слова приводит к тому, что низкий уровень устанавливается на тех линиях бита, которые через диод соединены с данной линией слова. В строке с номером 5 имеется только один диод и на соответствующей линии бита (D1_L) возникает низкий уровень. Сигналы на выходах ПЗУ D3–D0 образуются в результате прохождения сигналов с линий битов через инвертирующие буферы; в рассматриваемом случае – это 0010.

НЕКОТОРЫЕ ДЕТАЛИ

Для повышения помехоустойчивости схемы, изображенной на рис. 10.5, применяются КМОП-инверторы с определенным порогом срабатывания по входу, поскольку низкий уровень LOW на линиях битов оказывается недостаточно низким из-за падения напряжения на открытом диоде, равного 0.7 В. К счастью, у микросхем 74НС04 низкий уровень – это любое напряжение, меньшее 1.35 В. Конечно, кроме учебной лаборатории вы нигде не будете собирать эту схему: гораздо проще купить и запрограммировать готовую микросхему ПЗУ.

В изображенной на рис. 10.5 схеме ПЗУ каждому пересечению линии слова с линией битов соответствует один бит «памяти». Если на пересечении присутствует диод, то хранится 1, в противном случае хранится 0. Если бы вы стали собирать эту схему в лаборатории, то ее «программирование» заключалось бы во включении или не включении диодов в каждом пересечении. Хотя это может показаться примитивным, но владельцы мипиЭВМ PDP-11 фирмы DEC (приблизительно 1970 год) применяли подобный способ в «модуле ПЗУ пачальной загрузки» M792 32×16. Модуль поставлялся с 512 запаяными в местах пересечения диодами, и пользователь программировал его путем удаления диода там, где должен быть запоминен 0.

Показанное на рис. 10.5 включение диодов соответствует таблице истинности дешифратора 2×4, приведенной в табл. 10.1. Все это выглядит очень нерациональным: мы использовали дешифратор 3×8 и кучу диодов для создания ПЗУ, играющего роль дешифратора 2×4. Мы могли бы непосредственно воспользоваться частью дешифратора 3×8! Однако в дальнейшем мы покажем более эффективные структуры ПЗУ и приведем примеры более успешного проектирования.

СКРЫТЫЕ ПУТИ

В схеме ПЗУ, приведенной на рис. 10.5, в каждом месте, где хранится 1, необходимо использовать диоды, а не прямые соединения. На рис. 10.6 показано, что произойдет, если лишь несколько диодов, как, например, в строке 3, заменить непосредственными соединениями. Предположим, что сигналы на адресном входе равны 101; тогда активизируется выход ROW5_L и только на линии D1_L устанавливается низкий уровень, что дает на выходе 0010. Однако наличие непосредственных соединений позволяет току протекать по *скрытым путям (sneak paths)*, что приводит к появлению низкого уровня на линиях битов D2_L и D0_L и дает на выходе неправильный результат 0111. При наличии диодов скрытые пути разорваны занертыми диодами, что и позволяет получить на выходе правильный результат.

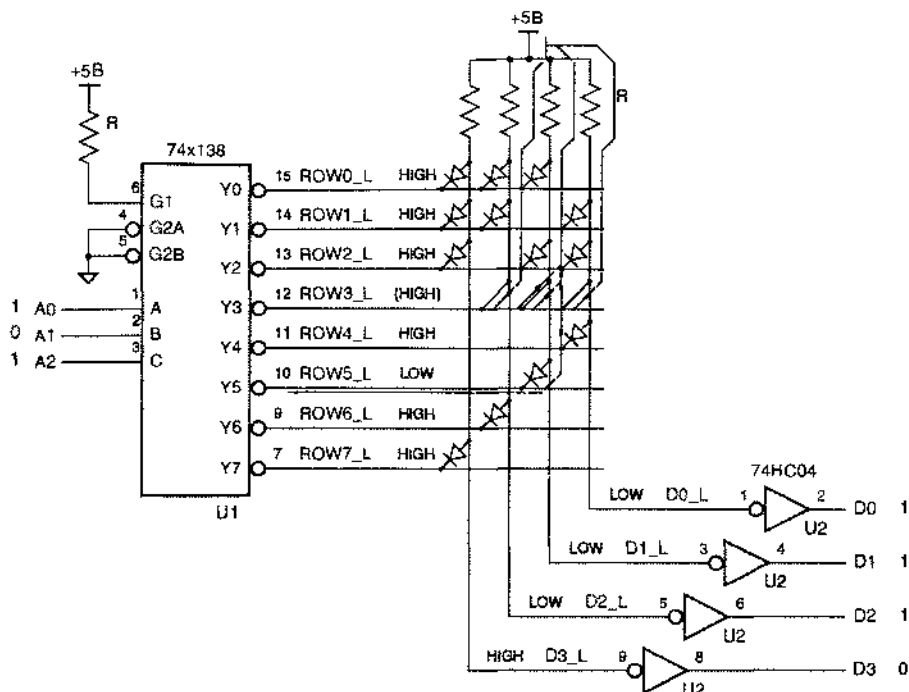


Рис. 10.6. Скрытые пути в ПЗУ (LOW – низкий уровень, HIGH – высокий уровень)

* 10.1.3. Двумерное декодирование

Предположим, что вы хотите построить ПЗУ 128x1, воспользовавшись структурой, описанной в предыдущем разделе. Возможно вы думаете, что для этого придется применить дешифратор 7x128. Попробуйте сначала взять 128 7-входовых вентилях И-НЕ и добавить 14 буферов и инверторов с нагрузочной способностью каждого, равной 64! Имеющиеся в продаже ПЗУ хранят миллион и более битов; поверьте мне, в них нет дешифраторов 20x1048576. Вместо этого для уменьшения дешифратора до размеров порядка квадратного корня из числа адресов применяется другой метод, называемый *двумерным декодированием* (*two-dimensional decoding*).

Основная идея двумерного декодирования состоит в компоновке ячеек ПЗУ в виде матрицы, по возможности приближающейся к квадратной. На рис. 10.7 показана, например, возможная внутренняя структура ПЗУ 128x1. Три старших разряда адреса A6–A4 используются для выбора строки. В каждой строке с начальным адресом (A6, A5, A4, 0, 0, 0, 0) хранится 16 битов. Когда на вход ПЗУ подан некоторый адрес, все 16 битов в выбранной строке «считываются» параллельно по линиям битов. 16-входовой мультиплексор выбирает требуемый бит данных по значениям младших разрядов адреса.

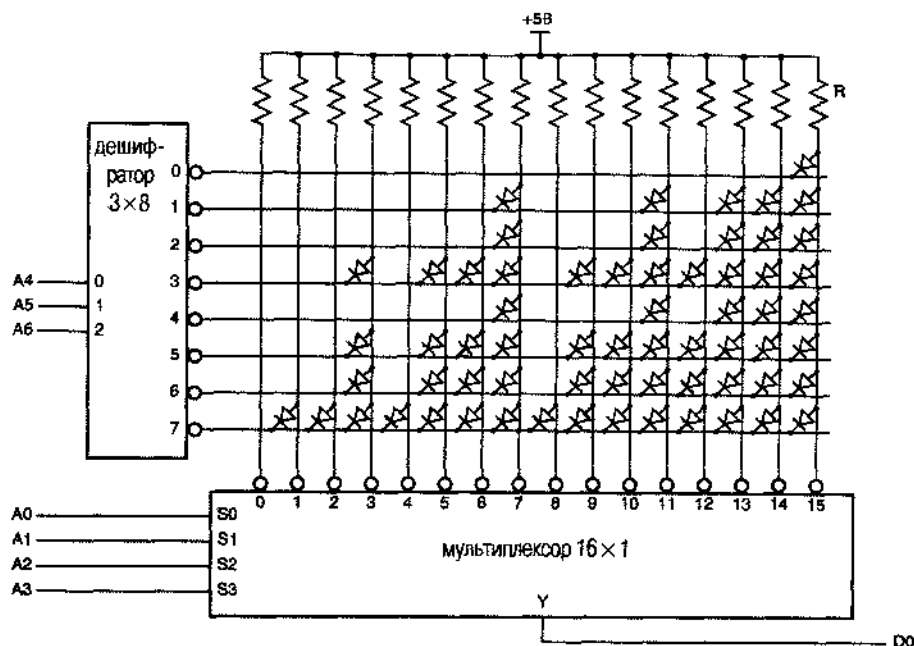


Рис. 10.7. Внутренняя структура ПЗУ 128x1, в котором применено двумерное декодирование

Между прочим, конфигурация диодов на рис. 10.7 выбрана не случайно. Это ПЗУ выполняет очень полезную логическую функцию с 7 переменными, для реализации которой потребовалось бы 35 4-входовых вентилях И для построения как минимум двухуровневой схемы И-ИЛИ (см. задачу 10.7). Вариант реализации этой функции с помощью ПЗУ позволяет реально сократить усилия при разработке устройства и уменьшить занимаемое этой схемой место на печатной плате.

Применение двумерного декодирования позволяет настроить ПЗУ 128x1, используя дешифратор 3x8 и 16-входовой мультиплексор (сложность которого сравнима со сложностью дешифратора 4x16). ПЗУ 1Mx1 можно построить, воспользовавшись дешифратором 10x1024 и 1024-входовым мультиплексором, что не легко, но значительно проще, чем одномерный вариант.

ТРАНЗИСТОРЫ В КАЧЕСТВЕ ЭЛЕМЕНТОВ ПЗУ

Фактически в ПЗУ, создаваемых на МОП-структурах, в каждом месте, где хранится бит, применяются не диоды, а транзисторы; на рис. 10.8 показана основная идея использования транзисторов. Дешифратор строк имеет выходы с высоким активным уровнем. Когда линия строки активизируется, открываются nМОП-транзисторы этой строки, что приводит к появлению на соответствующих линиях битов низкого уровня. Эту идею можно использовать при построении ПЗУ и на биполярных транзисторах.

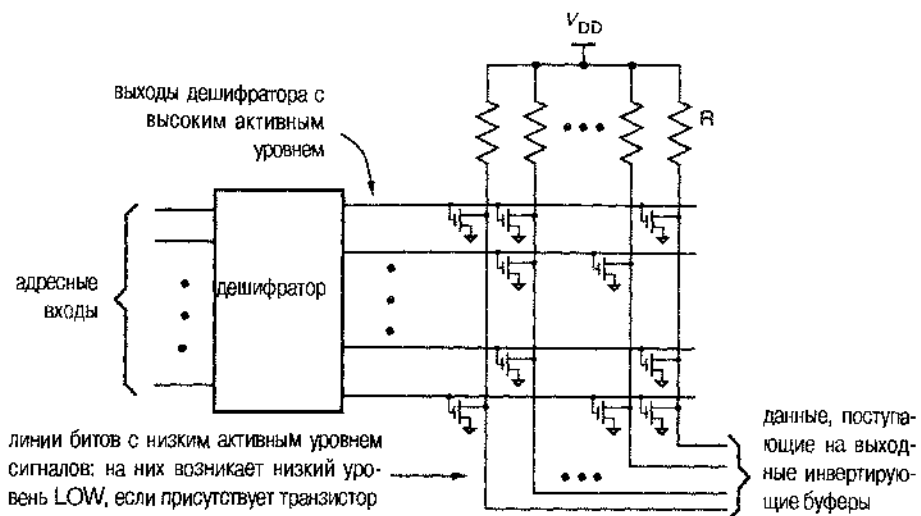


Рис. 10.8. Применение МОП-транзисторов в качестве элементов памяти в ПЗУ

Двумерное декодирование, помимо уменьшения сложности, обладает и другим достоинством: оно позволяет создать кристалл примерно квадратной формы, что важно при его изготовлении и размещении в корпусе. Микросхема, выполненная в виде матрицы $1\text{M} \times 1$, была бы *очень* длинной, выглядела бы как пленка, и ее нельзя было бы изготовить по экономическим соображениям.

В ПЗУ с несколькими выходами данных матрицы памяти, соответствующие каждому выходу, можно сделать более узкими для того, чтобы получить кристалл, близкий по форме к квадратному. На рис. 10.9 показана, например, возможная компоновка ПЗУ $32\text{K} \times 8$.

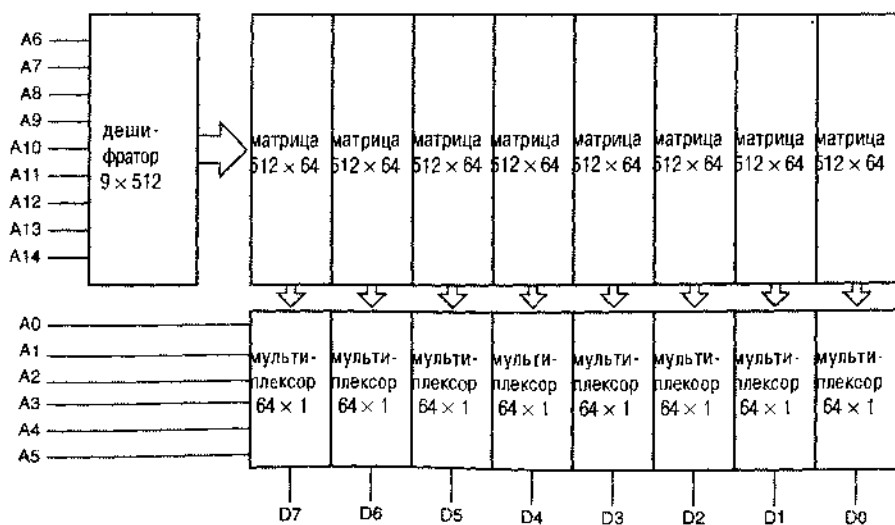


Рис. 10.9. Возможный вариант компоновки ПЗУ $32\text{K} \times 8$

10.1.4. Изготавливаемые серийно постоянные запоминающие устройства

В настоящее время ПЗУ на дискретных диодах можно увидеть, разве что посетив музей вычислительной техники. Современные ПЗУ изготавливаются в виде одной ИС; микросхема, хранящая четыре мегабита, может стоить менее 5 долларов. Для запоминания в ПЗУ информации применяются различные методы «программирования», сведенные в табл. 10.5 и рассматриваемые ниже.

Табл. 10.5. Типы изготавливаемых серийно ПЗУ

Тип	Технология	Цикл чтения	Цикл записи	Примечания
Масочное ПЗУ	пМОП, КМОП	10 – 200 нс	4 недели	Однократная запись; малая потребляемая мощность
Масочное ПЗУ	Бинолярная	< 100 нс	4 недели	Однократная запись; большая потребляемая мощность; низкая плотность
PROM	Биполярная	< 100 нс	10–50 мкс/байт	Однократная запись; большая потребляемая мощность; нет расходов на изготовление маски
EPROM	пМОП, КМОП	25 – 200 нс	10–50 мкс/байт	Многokrатное использование; малая потребляемая мощность; нет расходов на изготовление маски
EEPROM	пМОП	50 – 200 нс	10–50 мкс/байт	Ограничение числа записей: 10000–100000 записей в каждой ячейке

Большинство первоначальных интегральных схем ПЗУ были *программируемыми с помощью фотошаблонов ПЗУ (mask-programmable ROM)* или просто *масочные ПЗУ (mask ROM)*. Масочные ПЗУ программируются трафаретом соединений в одной из *масок (mask)*, применяемых в процессе изготовления ИС. Для программирования или записи информации в ПЗУ заказчик дает производителю листинг требуемого содержимого ПЗУ на флорпи-диске или другом носителе информации. Используя эту информацию, производитель создает, согласно техническим условиям заказчика, одну или большее число масок для изготовления ПЗУ с требуемой конфигурацией. *Расходы* производителей ПЗУ на *изготовление масок (mask charge)* доходят до нескольких тысяч долларов; это обусловлено индивидуальным характером изготовления «заказных» масок. Помимо значительных расходов на изготовление масок, для получения запрограммиро-

ванной микросхемы требуется срок порядка четырех недель; поэтому сегмашочные ПЗУ применяются, как правило, только при массовом производстве. Для мелкосерийного применения имеются экономически более эффективные возможности, рассматриваемые ниже.

Программируемые ПЗУ (programmable read-only memory, PROM) подны масочным ПЗУ, за исключением того, что пользователь может запомнить значения данных (то есть «запрограммировать ПЗУ») всего за несколько минут помощью *программатора PROM (PROM programmer)*. При изготовлении микросхем PROM все диоды или транзисторы «включены». Это соответствует то, что все биты имеют определенное значение, как правило, равное 1. С помощью программатора ПЗУ отдельным битам можно придать противоположные значения. В биполярных ПЗУ это делается путем пережигания тонких *плавких перемычек (fusible links)* внутри микросхемы PROM, соответствующих каждому биту. Перемычка пережигается путем ее выбора с помощью сигналов на линиях адреса и линиях данных этой микросхемы с последующей подачей высоковольтного импульса (10–30 В) на специальный вход устройства.

У первых биполярных PROM были проблемы с надежностью. Иногда значения запомненных битов изменялись из-за не полностью пережженных перемычек, которые «вырастают заново»; кроме того, иногда возникали нерегулярные сбои из-за застывших капель металла, хаотически перемещающихся внутри корпуса. Однако проблемы эти были преодолены и надежная технология плавких перемычек сегодня применяется не только в биполярных PROM, но и в биполярных PROM описанных в разделе 5.3.4.

Стираемые программируемые ПЗУ (erasable programmable read-only memory, EPROM) похожи на PROM, но их содержимое можно «стереть» путем облучения ультрафиолетовым светом, в результате чего все биты принимают значение, равное 1. Нет, свет не вызывает роста плавких перемычек! Просто в EPROM применяется другая технология, а именно: «МОП-транзисторы с плавающим затвором».

Как показано на рис. 10.10, в EPROM в месте хранения каждого бита имеет *МОП-транзистор с плавающим затвором (floating-gate MOS transistor)*. Для каждого транзистора есть два затвора. «Плавающий» затвор никуда не подключен и окружен изоляционным материалом с очень малой проводимостью. При программировании EPROM с помощью программатора на неплавающий затвор в месте хранения бита, значение которого должно быть равно 0, подается высокое напряжение. Это вызывает временный пробой слоя изоляции и приводит к накоплению отрицательного заряда в плавающем затворе. Затем высокое напряжение снимается, а отрицательный заряд остается. В дальнейшем при выполнении операции чтения отрицательный заряд не позволяет выбранному МОП-транзистору включаться.

Производители EPROM «гарантируют», что правильно запрограммированная ячейка удерживает 70% заряда в течение, по крайней мере, 10 лет, даже в том случае, когда микросхема хранится при температуре 125°C; поэтому EPROM определенно попадает в категорию «энергонезависимой памяти». Однако содержимое этой памяти можно стереть. Изоляционный материал вокруг плавающего затвора при облучении его ультрафиолетовым светом определенной длины волны становится немного проводящим. Таким образом, содержимое микросхемы EPROM можно стереть

облучая ее ультрафиолетовым светом в течение времени порядка 5–20 минут. Обычно кристалл EPROM располагается в корпусе с кварцевым окошком, через которое его можно засветить, чтобы стереть записанную в нем информацию.

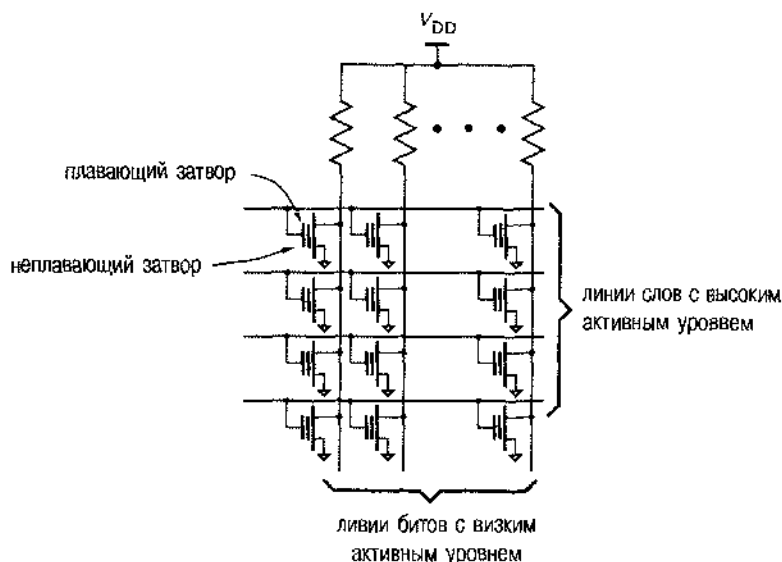


Рис. 10.10. Матрица памяти EPROM на МОП-транзисторах с плавающим затвором

Вероятно, самым распространенным применением EPROM является хранение программ в микропроцессорных системах. Часто EPROM применяют при разработке программ, когда в процессе отладки программа или другая информация, хранящаяся в EPROM, должны неоднократно изменяться. Однако ПЗУ и PROM, обычно дешевле, чем EPROM того же объема. Поэтому, как только программа отлажена, для снижения стоимости изделий можно воспользоваться ПЗУ или PROM. В действительности, сегодня большинство PROM представляют собой EPROM, помещенные в недорогие корпуса без кварцевых окошек; иногда их называют *однократно программируемыми ПЗУ* (*one-time programmable ROM, OTP ROM*).

Электрически стираемые программируемые ПЗУ (*electrically erasable programmable read-only memory, EEPROM*) подобны EPROM, за исключением того, что ранее записанные биты можно стереть электрически. Плавающие затворы транзисторов в EEPROM окружены гораздо более тонким изолирующим слоем, и имеющиеся на них заряды можно удалить, прикладывая к неплавающему затвору напряжение, полярность которого противоположна полярности напряжения, в результате подачи которого происходит накопление заряда. Большие EEPROM (емкостью 1 Мбит и больше) позволяют за одну операцию стирать информацию только блоками фиксированного размера; типичный размер блока 128–512 Кбит (16–64 Кбайт). Память такого типа обычно называется *флэш-памятью* (*flash memory, flash EPROM*), поскольку стирание данных группами происходит «мгновенно» (“in a flash”).

Как видно из табл. 10.5, программирование или «запись» ячейки EEPROM осуществляется гораздо дольше, чем чтение из нее, поэтому ИС EEPROM не годятся для использования в качестве оперативных запоминающих устройств, которые рассматриваются в этой главе позже. Кроме того, поскольку изолирующий слой очень тонок, он может разрушиться при многократном выполнении программирования. В результате EEPROM можно перепрограммировать ограниченное число раз, порядка 10000. Поэтому, как правило, EEPROM применяется для записи данных, которые должны сохраняться при отключении источника питания и изменяются не очень часто; пример таких данных – сведения о конфигурации компьютера по умолчанию.

На рис. 10.11 приведены условные обозначения популярных микросхем EPROM 2764, 27128, 27256 и 27512, часто используемых в микропроцессорных системах и в других устройствах с умеренным быстродействием. На рисунке указаны также выводы, на которые во время нормальной операции чтения подаются постоянные сигналы. На входы, помеченные символом V_{CC} , необходимо подать напряжение питания +5 В, на входах V_{IN} должен быть постоянный высокий уровень, а «N.C.» означает «не подключен». Для программирования и тестирования устройства применяются различные конфигурации сигналов на входах; на вывод VPP подается программирующее напряжение.

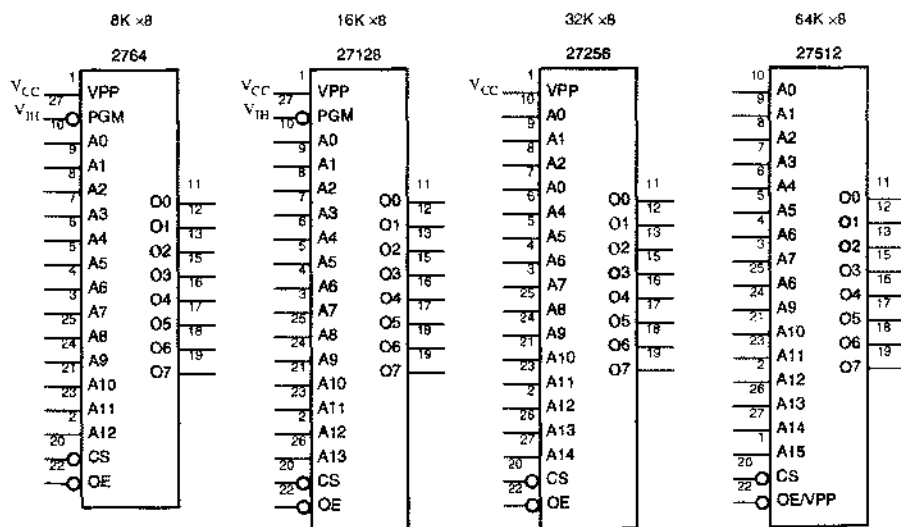


Рис. 10.11. Условные обозначения стандартных ИС EPROM в DIP-корпусе с 28 выводами

Имеются более крупные ПЗУ с большим числом битов, и в некоторых случаях число выходов данных увеличено до 16 или до 32. В 1999 году одним из наиболее впечатляющих достижений стала флэш-память AMD 29LV640 объемом 64 Мбита: по 32 килобайта в 256 секторах. Для специализированных применений, таких как загрузка информации в ИС типа FPGA при ее программировании, выпускаются ПЗУ гораздо меньшего объема с 3-разрядным последовательным интерфейсом.

Часто для применений, требующих энергонезависимого хранения большого количества данных, несколько микросхем флэш-памяти помещаются в один модуль размером с кредитную карту. Наиболее распространенным применением этих модулей является их использование в цифровых камерах, где для записи одного изображения с высокой степенью разрешения может потребоваться до 4 Мбайт памяти. В 1999 году самые большие флэш-карты продавались лидером в этой области, компанией SanDisk Corporation, и объем их памяти составлял 192 Мбайта (1536 Мбит).

10.1.5. Входы управления и временные параметры ПЗУ

Поскольку выходы ПЗУ часто должны подключаться к шине с тремя состояниями, сигналы в которую в разные моменты времени поступают от различных устройств, большинство серийных микросхем ПЗУ имеют выходы данных с тремя состояниями и вход разрешения выхода *OE (output-enable input)*; на этот вход необходимо подать активный уровень, для того чтобы на выходах появились сигналы.

Часто, особенно в тех случаях, когда ПЗУ применяются для хранения программ, к шине подключаются несколько микросхем ПЗУ, причем в каждый момент времени сигналы на шину выдает только одна из них. Чтобы упростить структуру таких систем, большинство микросхем ПЗУ снабжены входом выбора кристалла *CS (chip-select input)*. Для того чтобы вывести выходы ПЗУ из третьего состояния, необходимо подать сигнал активного уровня не только на вход OE, но также и на вход CS.

На рис. 10.12 показано, как можно было бы воспользоваться входами OE и CS при включении четырех микросхем ПЗУ 32К×8 в микропроцессорной системе с требуемым объемом постоянной памяти 128 Кбайт. В этом примере микропроцессор имеет 8-разрядную шину данных и 20-разрядную шину адреса, в результате чего адресное пространство составляет 1 Мбайт (2^{20} байтов). Предполагается, что отведенное под ПЗУ адресное пространство располагается на 128К верхних адресах. Чтобы обеспечить это, с помощью схемы И-НЕ вырабатывается сигнал $\overline{\text{HMEM_L}}$, который переходит на активный уровень при наличии на адресной шине адреса, соответствующего верхним 128К ячейкам ($A_{19}-A_{17} = 111$). С помощью дешифратора выбирается одна из четырех микросхем ПЗУ 32К×8. Выбранное ПЗУ выдает информацию на шину данных только в том случае, когда микропроцессор выполняет операцию чтения, вырабатывая при этом сигнал *READ*, поступающий на все входы OE.

Как мы уже говорили, вход CS является не более чем вторым входом разрешения выхода; сигнал на входе CS, объединенный логикой И с сигналом на входе OE, переводит выходы с тремя состояниями в активный режим. Однако во многих ПЗУ вход CS используется также как вход снижения потребляемой мощности (*power-down input*). Когда сигнал на входе CS имеет неактивный уровень, внутри ПЗУ отключается питание от дешифраторов, драйверов и мультиплексоров. В этом режиме ожидания (*standby mode*) типичное ПЗУ рассеивает менее 10% мощности, потребляемой в активном режиме (*active mode*) при активном уровне сигнала на входе CS. В схеме, приведенной на рис. 10.12, в ак-

тивном режиме одновременно может находиться не более одного ПЗУ, поэтому полная мощность, потребляемая всеми микросхемами ПЗУ, близка к мощности, потребляемой одной микросхемой, а не четырьмя.

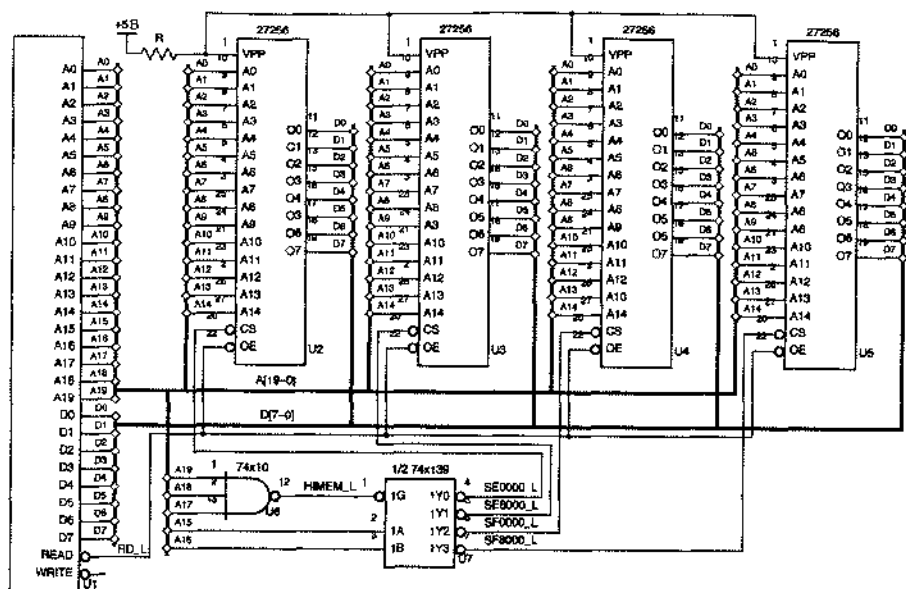


Рис. 10.12. Декодирование адреса и формирование сигнала выбора ПЗУ в микропроцессорной системе

На рис. 10.13 показано, как используются сигналы, поступающие на входы CS и OE, внутри типичного ПЗУ. На рис. 10.14 изображены временные характеристики типичного ПЗУ и указаны следующие временные параметры:

- t_{AA} *Время доступа по шине адреса (access time from address)*. Этот параметр определяет задержку между моментом установления стабильных значений сигналов на адресных входах ПЗУ и моментом установления достоверных сигналов на выходах данных.
- t_{ACS} *Время доступа по входу выбора кристалла (access time from chip select)*. Этот параметр характеризует задержку между моментом подачи сигнала на вход CS и моментом установления достоверных сигналов на выходах данных. Эта величина больше времени доступа по шине адреса, если схеме требуется время на переход из режима ожидания в активный режим. Когда сигнал на входе CS управляет только разрешением выхода, это время меньше.
- t_{OE} *Время разрешения выдачи данных (output-enable time)*. Значение этого параметра много меньше, чем время доступа. Время разрешения выдачи данных равно задержке между моментом времени, когда сигналы на обоих входах OE и CS становятся активными, и моментом, когда выходные каскады с тремя состояниями выходят из высокоомного состояния. В зависимости от того, насколько давно сигналы на адресных входах приняли устано-

вившиеся значения, сигналы на выходах данных к этому времени могут быть верными или не верными.

t_{OZ} *Время запрещения выдачи данных (output-disable time)*. Эта величина равна задержке между моментом установления неактивных значений сигналов на входах OE и CS и моментом перехода выходных каскадов с тремя состояниями в высокоомное состояние.

t_{OH} *Время удержания данных на выходе (output-hold time)*. Время удержания данных на выходе равно интервалу, в течение которого сигналы на выходах данных сохраняют свои значения после изменения адреса или после принятия сигналами на входах OE_L и CS_L неактивных значений.

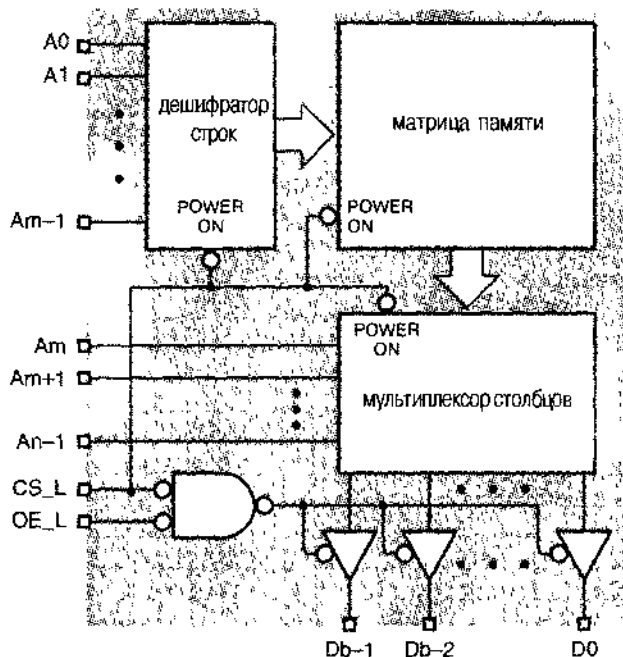


Рис. 10.13. Структура ПЗУ, иллюстрирующая использование сигналов, поступающих на входы управления (POWER ON – вход управления питанием)

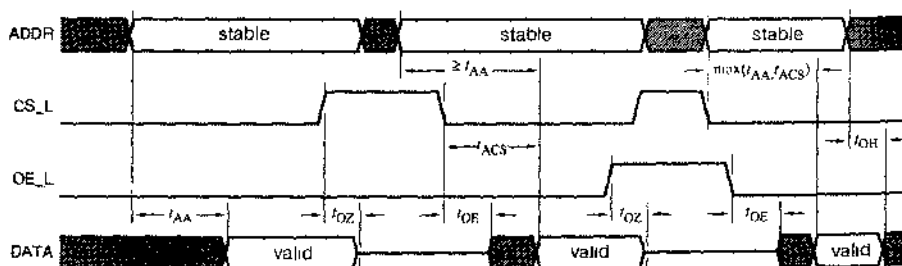


Рис. 10.14. Временные диаграммы, характеризующие работу ПЗУ (stable – установившиеся значения, valid – достоверные значения)

НЕ ВСЕ ВХОДЫ РАВНОЦЕННЫ

Рассматривая декодирование и мультиплексирование в схеме на рис. 10.13, можно ожидать, что время доступа для одних адресных входов меньше, чем для других. Если сигналы в тех или иных разрядах адреса появляются с задержкой по отношению к другим, то эту задержку можно компенсировать, подавая запаздывающие сигналы на более «быстрые» адресные входы ПЗУ. В конце концов, любой входной сигнал можно подать на любой адресный вход ПЗУ, если вы готовы соответствующим образом изменить содержимое ПЗУ.

Однако производители ПЗУ не указывают, какие из входов быстрее, даже тогда, когда таковые имеются. Фактические внутренние электрические характеристики большинства ПЗУ могут быть такими, что различия задержек очень малы и нет смысла беспокоиться по этому поводу.

Так же, как и в случае других компонентов, производитель указывает максимальные и, иногда, типичные значения всех временных параметров. Обычно для t_{OE} и t_{OH} указываются также минимальные значения. Минимальное значение t_{OH} часто принимают равным 0; это означает, что минимальная задержка комбинационной логики внутри ПЗУ равна нулю.

10.1.6. Применения ПЗУ

Как мы упоминали ранее, самым распространенным применением ПЗУ является хранение программ в микропроцессорных системах. Однако во многих случаях ПЗУ могут обеспечить дешевую реализацию сложных или «случайных» комбинационных функций. В этом разделе мы приведем в качестве примера пару схем на основе ПЗУ, которые используются в цифровых телефонных сетях.

Когда аналоговый речевой сигнал попадает в типичную цифровую телефонную систему, из него берутся выборки 8000 раз в секунду, которые преобразуются в последовательность 8-разрядных байтов, представляющих выборочные значения аналогового сигнала. В разделе 8.5.3 было показано, как можно преобразовывать цифровые выборки речевого сигнала из параллельного формата в последовательный и обратно, но процесс кодирования самих 8-разрядных байтов не был описан. Сейчас мы сделаем это, а затем покажем, как легко можно оперировать с этой закодированной информацией.

НЕВЕРОЯТНО БЫСТРЫЕ ПЗУ

Реальные задержки в ПЗУ, конечно, никогда не равны нулю, но они вполне могут быть меньше, чем это необходимо для удовлетворения требования нулевого времени удержания данных где-то в другом месте в вашем устройстве. Поэтому лучше предположить, что у ПЗУ $t_{OH} = 0$, если только вы не знаете, с чем имеете дело в действительности.

Простейшим 8-разрядным кодированием знака и амплитуды аналогового сигнала могло бы быть его представление в виде 8-разрядных целых чисел в дополнителном коде или в прямом коде со знаком; такой способ кодирования называется *линейным кодированием* (*linear encoding*). Однако 8-разрядное линейное кодирование имеет ограниченный *динамический диапазон* (*dynamic range*) – отношение диапазона представимых чисел к наименьшей представимой разности; в данном случае динамический диапазон был бы равен всего лишь $2^8 = 256$. Любители аудиотехники знают, что это соответствует динамическому диапазону $20 \log 256$, то есть около 48 дБ. Для сравнения напомним, что в звуковых компакт-дисках применяется 16-разрядное линейное кодирование, динамический диапазон которого теоретически равен $20 \log 2^{16}$ или около 96 дБ.

В Северной Америке в телефонной сети применяется не линейное кодирование, а 8-разрядное *кодирование с уплотнением* (*companded encoding*), называемое *μ-ИКМ* (импульсно-кодовая модуляция с *μ*-характеристикой; *μ-law PCM*). [В европейской телефонной сети применяется другой вариант 8-разрядного уплотнения, называемый *A-ИКМ* (*A-law PCM*).] На рис. 10.15 показан формат 8-разрядного кодового слова, являющегося разновидностью представления чисел с плавающей точкой, согласно которому слово состоит из полей знака (*S*), показателя экспоненты (*E*) и мантииссы (*M*). Аналоговая величина *V*, представленная байтом в этом формате, находится по формуле:

$$V = (1 - 2S) \cdot [(2^E) \cdot (2M + 33) - 33].$$



Рис. 10.15. Формат байта при *μ*-ИКМ

Аналоговые сигналы, представленные в этом формате, могут находиться в интервале от $-8159 \cdot k$ до $+8159 \cdot k$, где *k* – произвольный масштабный коэффициент. Весь интервал значений сигнала равен $2 \cdot 8159$, а наименьшая представляемая разность составляет всего лишь 2 (при *E* = 0), поэтому динамический диапазон равен $20 \log 8159$ или около 78 дБ, что заметно лучше, чем при 8-разрядном линейном кодировании.

ДЕНЬГИ НИ ЗА ЧТО...

Применяя *μ*-ИКМ, вы, конечно, ничего не получаете даром. При кодовом уплотнении разность между последовательными значениями при больших амплитудах сигнала больше, чем при малых амплитудах, поэтому при кодировании аналоговых сигналов большой амплитуды ошибки квантования больше (эти ошибки обусловлены различием между значением аналоговой выборки и ближайшим возможным представлением ее в виде кодового слова). Но ошибка квантования выражается в процентах от значения сигнала в выборке; поэтому она остается примерно постоянной в пределах всего диапазона представимых значений.

Давайте рассмотрим теперь некоторые применения ПЗУ, относящиеся к представлению речевого сигнала с помощью μ -ИКМ. Верите вы или нет, но во многих телефонных сетях для улучшения работы устройств ваш голос преднамеренно ослабляется на несколько децибел. Ослабление в аналоговых телефонных сетях осуществляется простой пассивной аналоговой схемой, но в цифровом мире дело обстоит иначе. Цифровой аттенюатор (*digital attenuator*) должен преобразовать один μ -ИКМ байт в другой ИКМ-байт, который представляет исходный аналоговый сигнал, умноженный на заданный коэффициент ослабления.

Один из вариантов построения цифрового аттенюатора показан на рис. 10.16. Исходный байт подается на вход μ -декодера, который преобразует байт в 14-разрядное целое число со знаком согласно приведенной ранее формуле. Эта 14-разрядная величина линейно связана с входным сигналом; она умножается на 14-разрядную двоичную дробь, соответствующую требуемому коэффициенту ослабления. Дробная часть произведения отбрасывается и результат снова кодируется с целью представления его в виде 8-разрядного μ -ИКМ байта. Каждый блок, изображенный на рисунке, можно, вероятно, реализовать на нескольких ИС средней степени интеграции или в ИС типа CPLD или FPGA.

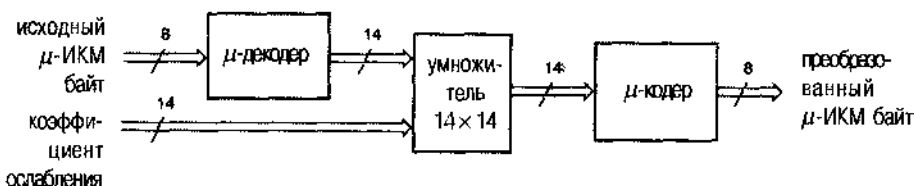


Рис. 10.16. Блок-схема цифрового аттенюатора

На рис. 10.17 приведена принципиальная схема цифрового аттенюатора, выполненного на основе всего лишь одной недорогой микросхемы ПЗУ 8K×8. Это ПЗУ позволяет реализовать один из 32 различных коэффициентов уменьшения числа, соответствующего входному μ -ИКМ байту: просто в ПЗУ хранятся 32 различные таблицы для разных значений коэффициента ослабления. Сигналами в старших разрядах адреса выбирается та или иная таблица, а младшими — ее содержимое. Содержимое каждой ячейки памяти является заранее вычисленным μ -ИКМ байтом, соответствующим заданному коэффициенту ослабления и исходному байту. В табл. 10.6 приведена программа на языке C, с помощью которой вычисляется содержимое ПЗУ. Детальное рассмотрение функций *UlawToLinear* и *LinearToUlaw* отнесено в одну из задач.

Другим применением ПЗУ в цифровой телефонии являются цифровые схемы для проведения конференций (селекторных совещаний). В аналоговых телефонных сетях совсем легко организовать соединение трех или большего числа участников. Просто подключите друг к другу аналоговые телефонные линии, и вы получите аналоговое суммирующее соединение, в котором каждый слышит любого другого. (Здесь изложена только идея, правильно реализовать ее совсем не так просто.) В цифровых сетях результатом непосредственного соединения выходов с цифровыми сигналами был бы, конечно, хаос. Вместо этого цифровые схемы для проведения конференций должны содержать цифровой *сумматор*, который формирует выходные выборки, соответствующие сумме входных выборок.

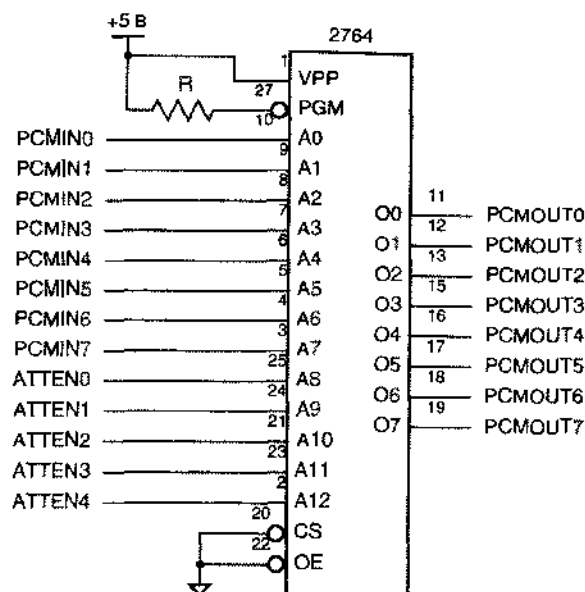


Рис. 10.17. Цифровой аттенюатор

Табл. 10.6. Программа формирования содержимого ПЗУ 8Кх8, выполняющей функции 32-позиционного аттенюатора для преобразования μ -ИКМ байтов

```
#include <stdio.h>
#include <math.h>

extern int UlawToLinear(int in);
extern int LinearToUlaw(int x);

void main()
{
    int i, j, position;
    int pcmIN, linearOUT, pcmOUT;
    double atten, attenDB, fpcmOUT;

    for (position=0; position<31; position++) { /* Make 32 256-byte tables. */
        printf("%i attenuation (dB): ", position); /* Get amount in dB from designer, */
        scanf("%f\n", &attenDB); /* negative for attenuation, positive for gain. */
        atten = exp(log(10)*attenDB/10); /* Convert to fraction. */
        for (i=0; i<=15; i++) { /* Construct output file in rows of 16. */
            printf("%04x:", position*256 + i*16);
            for (j=0; j<=15; j++) {
                pcmIN = i*16 + j;
                fpcmOUT = atten * UlawToLinear(pcmIN);
                if (fpcmOUT >= 0) linearOUT = floor(fpcmOUT + 0.5); /* Rounding */
                else linearOUT = ceil(fpcmOUT - 0.5);
                pcmOUT = LinearToUlaw(linearOUT);
                printf(" %2x", pcmOUT);
            }
            printf("\n");
        }
    }
}
```


... И КОЕ-ЧТО БЕСПЛАТНО

Цифровые аттенюаторы являются еще одним хорошим примером из множества эффективных с точки зрения снижения стоимости пространственно-временных обменов, которые оказываются возможными по мере того, как телефонные сети «становятся цифровыми». Одна выборка аналогового речевого сигнала в виде 8-разрядного μ -ИКМ байта вырабатывается каждые 125 мкс, а цифровой аттенюатор, реализованный на основе ПЗУ, может выдать правильный результат через несколько сот наносекунд или быстрее. Таким образом, в цифровой телефонной сети одна микросхема ПЗУ способна выполнять ослабление сотен цифровых речевых потоков, на что раньше требовались сотни аналоговых аттенюаторов.

Вы знаете, как складываются два 8-разрядных двоичных операнда, но двоичный сумматор не может оперировать непосредственно с μ -ИКМ байтами. Для сложения 8-разрядные μ -ИКМ байты должны быть преобразованы в 14-разрядный линейный формат, затем сложены и после этого снова закодированы. Так же, как и в отношении цифрового аттенюатора, схема которого приведена на рис. 10.16, при реализации этой функции на ИС средней степени интеграции может потребоваться большое число микросхем. С другой стороны, такое суммирование можно реализовать на одном ПЗУ 64К $\times$ 8, как показано на рис. 10.18. Два 8-разрядных μ -ИКМ операнда подаются на 16 адресных входов ПЗУ. Для каждой пары значений операндов в ПЗУ по соответствующему адресу хранятся вычисленные заранее суммы этих μ -ИКМ байтов. В табл. 10.7 приведена программа на языке С, которой можно воспользоваться для формирования содержимого ПЗУ.

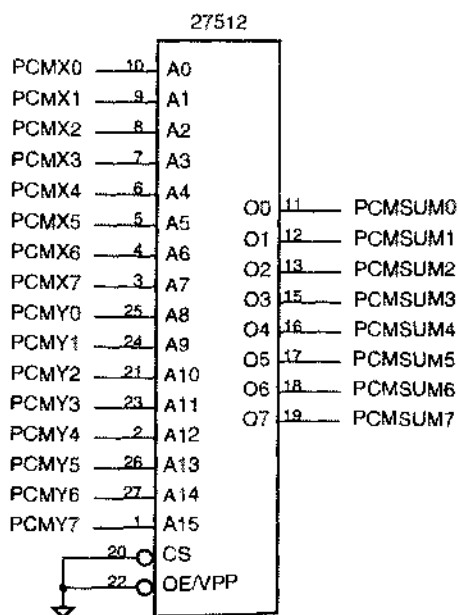


Рис. 10.18. Схема сумматора μ -ИКМ байтов

Табл. 10.7. Программа, формирующая содержимое ПЗУ 64Кх8, используемого в качестве сумматора μ -ИКМ байтов

```

#include <stdio.h>
#include <math.h>
#define MINLINEAR -8159
#define MAXLINEAR 8159

void main()
{
    int i, j, linearSum;
    int pcmINX, pcmINY;

    for (pcmINY=0; pcmINY<=255; pcmINY++) { /* For all Y samples... */
        for (i=0; i<=15; i++) { /* Construct output file in rows of 16. */
            printf("%04x:", pcmINY*256 + i*16);
            for (j=0; j<=15; j++) { /* For all X samples... */
                pcmINX = i*16 + j;
                linearSum = UlawToLinear(pcmINX) + UlawToLinear(pcmINY);
                /* The next two lines perform "clipping" on overflow. */
                if (linearSum < MINLINEAR) linearSum = MINLINEAR;
                if (linearSum > MAXLINEAR) linearSum = MAXLINEAR;
                printf(" %02x", LinearToUlaw(linearSum));
            }
            printf("\n");
        }
    }
}

```

Эти примеры иллюстрируют достоинства применения ПЗУ при построении сложных комбинационных схем. Обычно мы считаем схему «сложной», если знаем, что ее конструирование на уровне вентилей вызовет настоящую головную боль. Однако большинство «сложных» схем, подобных приведенным в качестве примеров в этом разделе, имеют довольно простые словесные описания. Обычно такое описание можно представить в виде компьютерной программы, которая «вычисляет», что должно быть на выходах для каждой возможной входной комбинации; поэтому схему можно построить, просто загружая в ПЗУ соответствующую ей таблицу истинности.

Кроме того, с точки зрения облегчения и ускорения проектирования, построение схем на основе ПЗУ имеет и другие важные достоинства:

- Умеренно сложные схемы на основе ПЗУ обычно оказываются более быстродействующими, чем схемы, построенные на большом числе МИС, СИС и ПЛУ, выполненных по сравнимой технологии; часто схемы на ПЗУ работают быстрее, чем в случае использования ИС типа FPGA или заказных БИС.
- Всегда нетрудно написать программу, формирующую содержимое ПЗУ, которая позволяет справиться с необычными или неопределенными ситуациями, тогда как при другой реализации потребуются дополнительные аппарат-

ные средства. Например, в программе сумматора, приведенной в табл. 10.7, легко преодолеваются случаи, когда сумма выходит за пределы диапазона представимых чисел. (См. также задачу 10.22.)

- Функция, реализуемая с помощью ПЗУ, легко модифицируется всего лишь путем изменения содержимого ПЗУ, обычно без изменения каких-либо внешних соединений. Например, аттенюатор и сумматор μ -ИКМ байтов, рассмотренные в этом разделе, можно приспособить для работы с 8-разрядными А-ИКМ словами в соответствии со стандартом кодового уплотнения речевого сигнала, принятым в Евроне.
- Цены на ПЗУ и другие логические устройства с регулярной структурой неизменно падают, делая их применение более экономичным, а плотность компонентов в них постоянно увеличивается, в результате чего расширяется область задач, которые можно решить с помощью одной микросхемы.

Однако схемы, построенные на основе ПЗУ, имеют также недостатки:

- При реализации простых и умеренно сложных функций схемы на основе ПЗУ могут оказаться более дорогими, рассеивать большую мощность или быть менее быстродействующими, чем схемы, построенные с применением нескольких МИС, СИС, ПЛИУ или небольшой ИС типа FPGA.
- Применение ПЗУ в схемах с числом входов более 20 невозможно из-за ограниченных размеров имеющихся ПЗУ. Например, нельзя построить на основе ПЗУ 16-разрядный сумматор: для этого потребовалась бы память емкостью в миллиарды бит.

10.2. Оперативные запоминающие устройства

Если к памяти можно обратиться в любой момент времени, чтобы запомнить в ней или извлечь из нее информацию, то ее называют *памятью с чтением и записью* (*read/write memory, RWM*). Большинство устройств памяти такого типа, применяемых сегодня в цифровых системах, являются *оперативной памятью* (*ОЗУ*) или *памятью с произвольным доступом* (*random-access memory, RAM*). Это означает, что каждый раз при чтении или записи можно выбрать любую ячейку памяти. С этой точки зрения ПЗУ (ROM) также является памятью с произвольным доступом, но название «ОЗУ» («RAM») обычно относится только к памяти с произвольным доступом, в которой возможны чтение и запись.

В *статическом ОЗУ* (*static RAM, SRAM*) слово, записанное однажды в какую-то ячейку, сохраняется в ней пока на микросхему подано напряжение питания, если только содержимое этой ячейки не изменяется в результате новой записи. В *динамическом ОЗУ* (*dynamic RAM, DRAM*) данные, сохраняемые в каждой ячейке, необходимо периодически обновлять путем их чтения и последующей повторной записи; в противном случае они будут потеряны. В этой главе мы рассмотрим оба типа памяти.

В большинстве ОЗУ хранящаяся в них информация теряется при отключении питания; другими словами ОЗУ является *энергозависимой памятью* (*volatile memory*). Но бывают также ОЗУ, называемые *энергонезависимой памятью* (*nonvolatile memory*), которые сохраняют записанную в них информацию даже

при отключении питания. Примерами энергонезависимых ОЗУ служат вышедшая из употребления память на магнитных сердечниках и современная статическая КМОП-память в сверхбольших корпусах, внутри которых имеются литиевые батареи с 10-летним сроком службы. Недавно было объявлено о выпуске энергонезависимого *ферроэлектрического ОЗУ (ferroelectric RAM)*; в этих устройствах в одной ИС объединены магнитные и электронные элементы, которые сохраняют свое состояние даже при отключенном питании точно так же, как в прежней памяти на магнитных сердечниках.

10.3. Статические оперативные запоминающие устройства

10.3.1. Входы и выходы статического ОЗУ

Как и в случае ПЗУ, у ОЗУ имеются адресные входы, входы управления и выходы данных; но кроме этого у ОЗУ есть еще и входы данных. На рис. 10.19 показаны входы и выходы простого статического ОЗУ, предназначенного для хранения $2^n \times b$ битов. Кроме тех же входов управления, что и у ПЗУ, имеется *вход разрешения записи WE (write-enable input)*. Входные данные записываются в выбранную ячейку памяти, когда сигнал на входе WE имеет активный уровень.

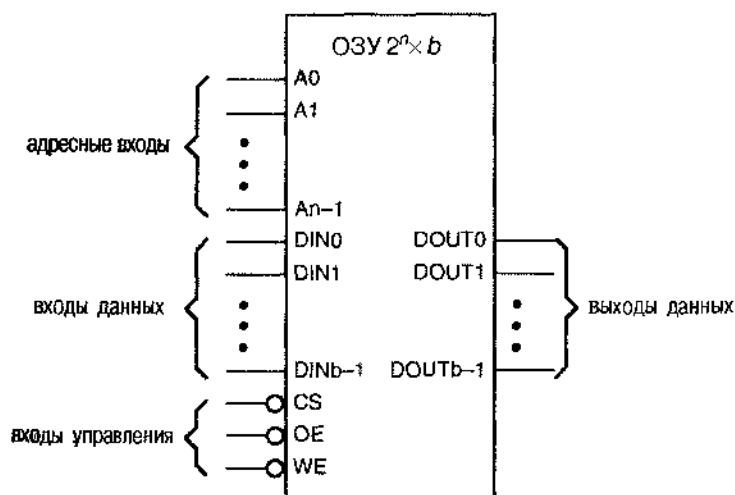


Рис. 10.19. Общая структура ОЗУ $2^n \times b$

Ячейки памяти статического ОЗУ ведут себя скорее как D-заселки, а не как переключающиеся по фронту D-триггеры. Это означает, что всякий раз, когда на вход WE подан сигнал активного уровня, заселка в выбранной ячейке памяти «открыта» (или «прозрачна»): входные данные поступают на заселку и появляются на ее выходе. Фактически запоминается то значение, которое присутствует на входе заселки в момент ее закрытия.

ПАМЯТЬ С ПОСЛЕДОВАТЕЛЬНЫМ ДОСТУПОМ

Памяти с произвольным доступом можно противопоставить *память с последовательным доступом (serial-access memory)*, у которой в каждый момент времени непосредственно доступна какая-то одна ячейка, а для доступа к другим ячейкам требуется выполнить дополнительные шаги.

В некоторых первых компьютерах применялись электромеханические устройства памяти с последовательным доступом, такие как линии задержки и вращающиеся барабаны. Программы и данные хранились на вращающемся носителе и в любой момент времени «головка чтения/записи» располагалась только над одной ячейкой. Для доступа к произвольной ячейке необходимо было ждать, когда в результате непрерывного вращения барабана требуемая ячейка окажется под головкой.

В 1970 году были созданы электронные эквиваленты вращающейся памяти с последовательным доступом, в том числе память, состоящая из элементов с зарядовой связью (ПЗС), и память на цилиндрических магнитных доменах (ЦМД). Оба эти устройства, грубо говоря, эквивалентны очень большому регистру сдвига с последовательным вводом и последовательным выводом, у которых выход соединен со входом. Точка соединения является логическим эквивалентом «головки чтения/записи» жесткого диска. Для чтения содержимого конкретной ячейки на регистр сдвига необходимо подавать тактовые импульсы до тех пор, пока нужный бит не появится на последовательном выходе, а для записи в данную ячейку необходимо в этот момент подать на последовательный вход новое желаемое значение.

В то время, когда создавались эти устройства, плотность размещения ячеек в них (число битов) была больше, чем у динамических ОЗУ; несмотря на это устройства памяти на ПЗС и цилиндрических магнитных доменах никогда не пользовались заметным успехом. Одной из причин этого было огромное неудобство последовательного доступа. Другой причиной послужило то, что они никогда не опережали динамические ОЗУ по достигаемой в них плотности ячеек более чем на пару лет.

Обычно у статического ОЗУ бывают только два режима доступа:

<i>Режим чтения</i>	На входы CS и OE поданы сигналы активного уровня, а на адресные входы поступают сигналы адреса. С выходов зашлюк выбранной ячейки памяти данные поступают на выходы данных DOUT.
<i>Режим записи</i>	На адресные входы подаются сигналы адреса, а на входы данных DIN — слово данных; затем на входы CS и WE поступают сигналы активного уровня. Открываются защелки выбранной ячейки памяти и в них запоминается входное слово данных.

При организации доступа к статическому ОЗУ требуется некоторая осторожность, поскольку в том случае, когда не удовлетворяются временные требования, предъявляемые микросхемой ОЗУ, при записи в выбранную ячейку возможно **непреднамеренное «затирание»** информации, хранящейся в одной или в несколь-

ких других ячейках. Для того чтобы показать, почему это происходит, в следующем разделе приведена детальная внутренняя структура статического ОЗУ, а затем рассматриваются реальные временные соотношения и их соответствие предъявляемым требованиям.

10.3.2. Внутренняя структура статического ОЗУ

Схема в каждом двоичном разряде статического ОЗУ (*ячейка статического ОЗУ*; *SRAM cell*) имеет вид, приведенный на рис. 10.20. Элементом, хранящим информацию в каждой ячейке, служит D-защелка. Когда на вход SEL_L подан сигнал активного уровня, сохраняемая в ячейке информация появляется на ее выходе, который соединен с соответствующей линией битов. Если сигнал активного уровня поступает на оба входа SEL_L и WR_L, то защелка открыта и в ней запоминается новый бит данных.

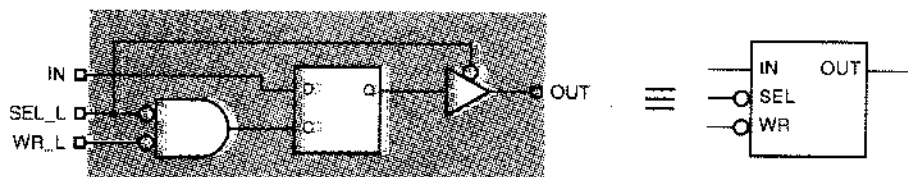


Рис. 10.20. Функциональная модель ячейки статического ОЗУ

На рис. 10.21 показано, как ячейки статического ОЗУ, объединенные в виде матрицы, вместе с дополнительной управляющей логикой образуют закопченное статическое ОЗУ емкостью 8×4 байтов. Как и в простом ПЗУ, с помощью дешифратора адресных линий в любой момент времени выбирается для доступа определенная строка статического ОЗУ.

Хотя на рис. 10.21 приведена до некоторой степени упрощенная модель внутренней структуры статического ОЗУ, она достаточно точно отражает основные моменты в работе этого устройства:

- При выполнении операций чтения выходные данные так же, как и в ПЗУ, являются комбинационными функциями сигналов на адресных входах. Изменение адреса в то время, когда разрешено появление выходных данных на шине, не наносит никакого вреда. Время доступа при выполнении операции чтения отсчитывается от момента, когда последний из сигналов на адресном входе принимает установившееся значение.
- При выполнении операций записи входные данные запоминаются в *защелках*. Это означает, что данные должны удовлетворять определенным требованиям во времени установления и времени удержания относительно *заднего перепада* в сигнале на входе разрешения защелки. Другими словами, сигнал данных на D-входе защелки не обязан оставаться неизменным в момент времени, когда сигнал WR_L внутри схемы переходит на активный уровень; сигнал данных должен оставаться неизменным лишь в течение некоторого времени, предшествующего тому моменту, когда сигнал WR_L переходит на неактивный уровень.

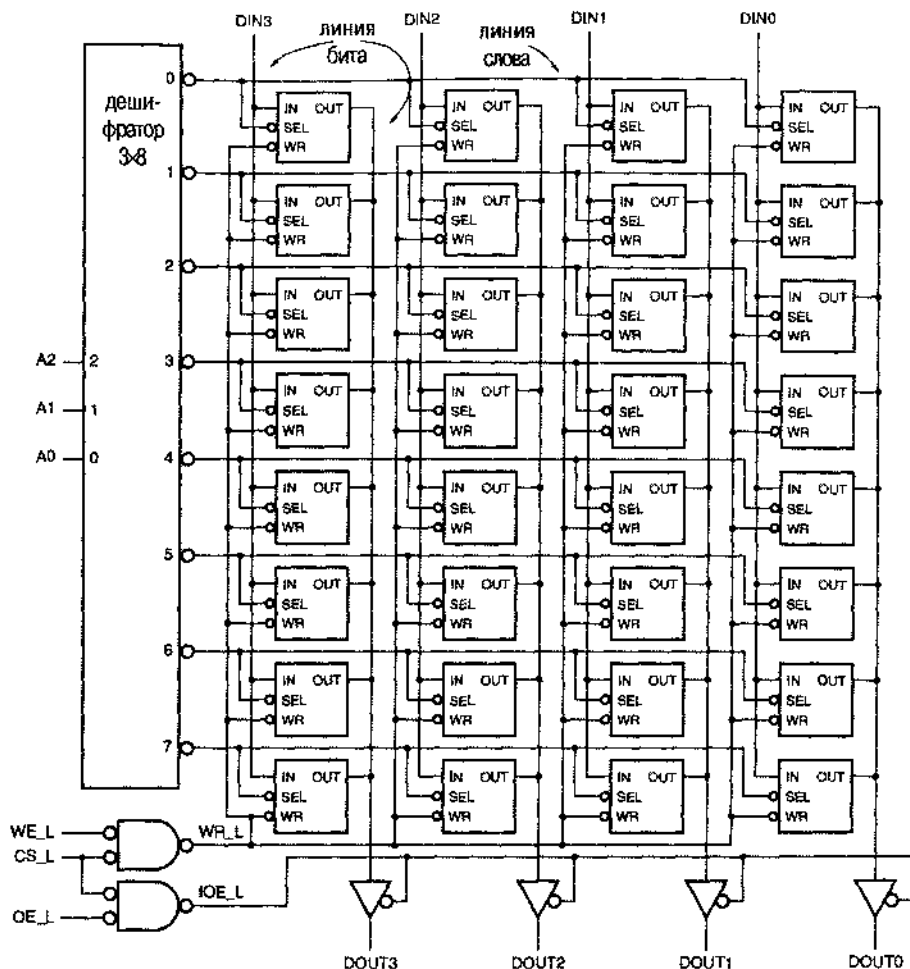


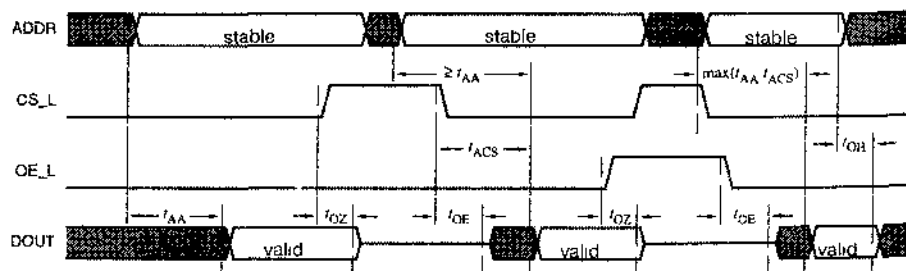
Рис. 10.21. Внутренняя структура статического ОЗУ 8x4.

- Во время операций записи сигналы на адресных входах *не должны* изменяться в течение определенного времени установления до перехода сигнала WR_L внутри схемы на активный уровень и в течение времени удержания после того, как сигнал WR_L перейдет на неактивный уровень. В противном случае данные могут оказаться «размазанными» по всему массиву ячеек из-за паразитных импульсов на линиях SEL_L , которые могут возникнуть при изменении сигналов на адресных входах дешифратора.
- Сигнал WR_L переходит на активный уровень внутри схемы только в том случае, когда активные значения имеют сигналы CS_L и WE_L . Поэтому цикл записи (*write cycle*) начинается с установления активного уровня сигналов CS_L и WE_L и заканчивается, когда любой из этих сигналов переходит на неактивный уровень. Время установления и время удержания адреса и данных определены относительно этих событий.

10.3.3. Временные параметры статического ОЗУ

На рис. 10.22 приведены временные диаграммы и определение временных параметров, которые обычно задаются для операции чтения из статического ОЗУ:

- t_{AA} *Время доступа по шине адреса (access time from address)*. Этим параметром определяется время, спустя которое выходные данные принимают установившееся значение после изменения адреса при условии, что сигналы OE и CS к этому времени уже имеют активный уровень или достаточно скоро должны стать такими. Когда разработчики говорят о 70-наносекундном статическом ОЗУ, обычно они имеют в виду этот параметр.
- t_{ACS} *Время доступа по входу выбора кристалла (access time from chip select)*. Этим параметром определяется время, спустя которое выходные данные принимают установившееся значение после перехода сигнала CS на активный уровень при условии, что сигналы на адресных входах и сигнал OE уже имеют активный уровень или достаточно скоро должны стать такими. Часто значение этого параметра совпадает с временем t_{AA} , но иногда его величина больше t_{AA} при работе статического ОЗУ в режиме пониженного потребления мощности и меньше t_{AA} , когда статическое ОЗУ не находится в этом режиме.
- t_{OE} *Время разрешения выхода (output-enable time)*. Этим параметром определяется время, через которое буферы с тремя состояниями на выходе выйдут из высокоомного состояния, после того как оба сигнала OE и CS перейдут на активный уровень. Этот параметр обычно меньше, чем величина t_{ACS} , поэтому внутри ОЗУ вызов данных возможен раньше, чем сигнал OE примет активное значение; во многих приложениях это свойство используется для достижения малых времен доступа, чтобы избежать «борьбы в шине».
- t_{OZ} *Время запрещения выхода (output-disable time)*. Этим параметром определяется время, необходимое для того, чтобы буферы с тремя состояниями на выходе перешли в высокоомное состояние, после того как сигналы OE_L или CS_L перейдут на неактивный уровень.
- t_{OH} *Время удержания сигнала на выходе (output-hold time)*. Этот параметр показывает, как долго выходные данные сохраняют установившееся значение после изменения адреса на входе.



Заметьте, что WE_L = HIGH

Рис. 10.22. Временные параметры для операции чтения из статического ОЗУ (stable – установившиеся значения; valid – достоверные данные)

Вы могли заметить, если обратили внимание, что временные диаграммы и временные параметры для режима чтения из статического ОЗУ идентичны с теми, которые были введены при рассмотрении режима чтения из ПЗУ в разделе 10.1.5. Это именно так: когда не производится запись в статическое ОЗУ, его можно использовать точно так же, как ПЗУ. Позже мы увидим, что совсем иначе обстоит дело с динамическими ОЗУ.

Временные параметры операции записи, указанные на рис. 10.23, определяются следующим образом:

- t_{AS} *Время установления адреса до начала записи (address setup time before write).* Сигналы на всех адресных входах должны оставаться постоянными в течение указанного времени перед тем, как оба сигнала CS и WE примут активное значение. В противном случае данные могут быть искажены и нельзя сказать, в каких именно ячейках это может произойти.
- t_{AH} *Время удержания адреса после окончания записи (address hold time after write).* Так же, как и в отношении параметра t_{AS} , сигналы на всех адресных входах должны поддерживаться неизменными в течение времени t_{AH} после того как хотя бы один из сигналов CS или WE перейдет на неактивный уровень.
- t_{CSW} *Время установления сигнала «выбор кристалла» до окончания записи (chip-select setup time before end of write).* Уровень сигнала CS должен оставаться активным в течение отрезка времени длительностью не менее t_{CSW} перед окончанием цикла записи.
- t_{WP} *Длительность импульса записи (write-pulse width).* Для надежного запоминания данных в выбранной ячейке сигнал WE должен иметь активный уровень в течение времени, равного, по крайней мере, t_{WP} .
- t_{DS} *Время установления данных до окончания записи (data setup time before end of write).* Сигналы на всех входах данных должны иметь постоянные значения в течение этого отрезка времени перед окончанием цикла записи. В противном случае данные могут оказаться незапомненными.
- t_{DH} *Время удержания данных после окончания записи (data hold time after end of write).* Аналогично параметру t_{DS} , сигналы на всех входах данных должны поддерживаться неизменными в течение этого интервала времени после окончания цикла записи.

Производителями статических ОЗУ определяются два типа цикла записи: *запись по сигналу WE (WE-controlled write)* и *запись по сигналу CS (CS-controlled write)*, как показано на рисунке. Единственное различие между этими циклами состоит в том, какой из сигналов WE или CS последним переходит на активный уровень и у какого из этих сигналов первым уровень становится неактивным при разрешении операции записи внутри статического ОЗУ.

Требования к временным параметрам при записи в статическое ОЗУ можно было бы несколько ослабить, если бы в ячейках вместо защелок применялись переключающиеся по фронту D-триггеры с объединенными тактовым входом и входом разрешения, на которые подавались бы сигналы SEL и WR. Однако так не поступают, потому что при этом по меньшей мере вдвое увеличилась бы пло-

щадь, занимаемая каждой ячейкой в кристалле, так как D-триггер состоит из двух защелок. Таким образом, разработчику логических устройств приходится согласовывать временные параметры статических ОЗУ на защелках с временными параметрами переключающихся фронту регистров и с временными требованиями конечных автоматов, используемых в системе.

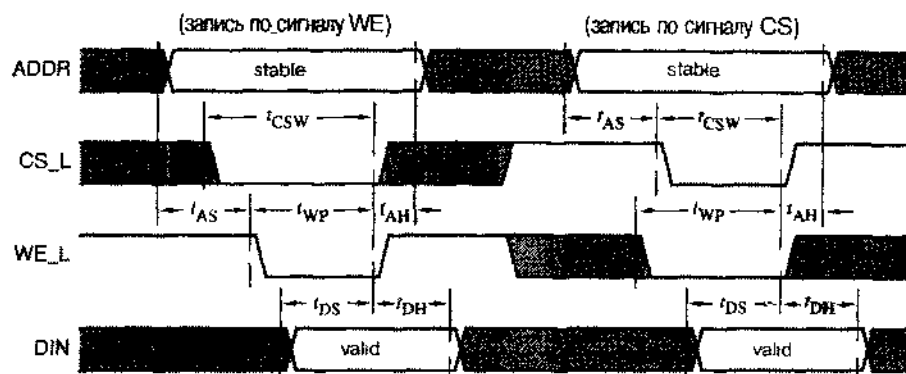


Рис. 10.23. Временные параметры для операции записи в статическое ОЗУ (stable – установленные значения; valid – достоверные данные)

Физические размеры матрицы в кристалле большого статического ОЗУ напрямую не связаны с емкостью памяти. Как и в ПЗУ, ячейки статического ОЗУ образуют почти квадратную матрицу, и при чтении внутри ОЗУ считывается полная строка. Например, структура микросхемы статического ОЗУ емкостью 32Кх8 может быть очень похожа на структуру ПЗУ 32Кх8, приведенную на рис. 10.9. Во время чтения требуемые данные проходят на выходную шину данных через мультиплексоры столбцов в соответствии с подмножеством значений адреса в определенных разрядах (в примере с ПЗУ такими разрядами являются А5–А0). Схема разрешения записи обеспечивает доступ при записи только к одному столбцу в каждом подмассиве, который определяется тем же самым подмножеством адресных битов.

10.3.4. Стандартные статические ОЗУ

В настоящее время выпускаются статические ОЗУ различной емкости и с разным быстродействием. В 1999 году самые большие обычные статические ОЗУ, выполненные по КМОП-технологии имели организацию 256Кх16 (256К 16-разрядных слов), 512Кх8 или 1Мх4 (4 Мбит) и время доступа 10 нс. Самые быстрые TTL/КМОП-совместимые статические ОЗУ, также выполненные по КМОП-технологии, содержали всего лишь 1Кх4 ячеек памяти (4 Кбита), но имели время доступа 2.7 нс!

Среди производителей статических ОЗУ нет согласия в отношении обозначения микросхем, хотя сами ИС часто имеют одинаковую цоколевку и взаимозаменяемы. На рис. 10.24 приведены обозначения и цоколевка статических ОЗУ фирмы Hitachi емкостью от 8Кх8 до 512Кх8. Эти микросхемы удобны для применения в лабораторном проектировании: им требуется напряжение питания 5 В и размещаются они в DIP-корпусах. Эти же статические ОЗУ имеются также и в более ком-

компактных корпусах, а в последнее время они обычно поставляются только в компактных корпусах и работают с более низкими напряжениями питания, равным 3.3 В или 2.5 В.

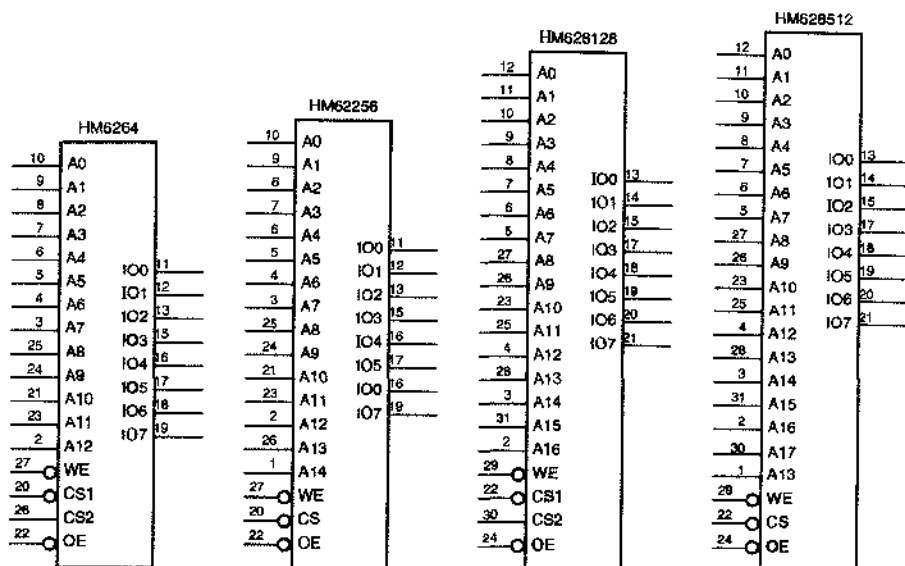


Рис. 10.24. Условные обозначения стандартных статических ОЗУ в DIP-корпусах с 28 и 32 выводами

Первые два статических ОЗУ, приведенные на рисунке, размещены в DIP-корпусах с 28 выводами и имеют такую же доколевку, как и программируемые ПЗУ с тем же объемом памяти (рис. 10.11). Микросхема *HM6264* емкостью 8Kx8 подобна EPROM 2764 за исключением того, что у нее имеется вход разрешения записи и два входа выбора кристалла, а не один; для того чтобы микросхема была переведена в активный режим, оба сигнала CS1 и CS2 должны перейти на активный уровень. Микросхема *HM62256* емкостью 32Kx8 подобна EPROM 27256 с добавлением входа разрешения записи. Обе микросхемы поставляются многими производителями, хотя часто имеют разные обозначения; время доступа у этих микросхем находится в диапазоне от 15 нс до 150 нс. Последние два статических ОЗУ, показанных на рисунке, выполнены в корпусах DIP с 32 выводами. Емкость ИС *HM628128* составляет 128Kx8, а ИС *HM628512* – 512Kx8.

Статические ОЗУ, приведенные на рис. 10.24, имеют двунаправленные шины данных: для чтения и для записи используются одни и те же выводы данных. Это требует незначительного изменения внутренней логики управления, как показано на рис. 10.25. Каждый буфер автоматически запирается (переходит в высокоомное состояние) всякий раз, когда сигнал WE_L принимает активное значение, даже в том случае, когда активный уровень имеет сигнал OE_L. Однако временные параметры и требования для операций чтения и записи практически тождественны тем, какие были приведены в предыдущем разделе.

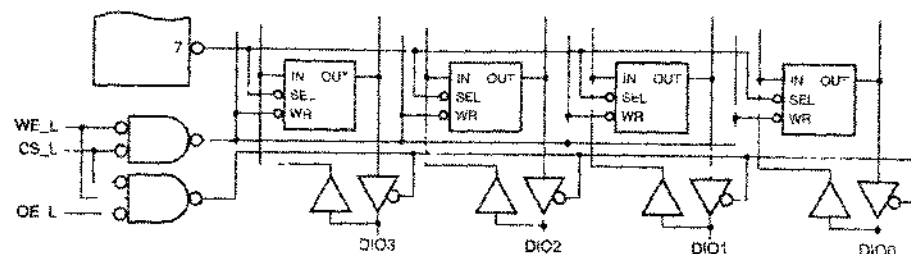


Рис. 10.25. Управление выходными буферами в статическом ОЗУ с двунаправленной шиной данных

Обычно статические ОЗУ используют для хранения данных в небольших микропроцессорных системах и часто они встречаются во «встроенных» цифровых системах (в телефонах, тостерах, стабилизаторах напряжения и т.д.). В универсальных вычислительных машинах чаще применяются рассматриваемые в параграфе 10.4 динамические ОЗУ (DRAM), поскольку у них выше плотность размещения ячеек памяти и меньше стоимость в расчете на один бит.

Очень быстрые статические ОЗУ часто используются в высокопроизводительных компьютерах в качестве кэш-памяти для сохранения многократно используемых команд и данных. Фактически, необходимость в высоком быстродействии кэш-памяти в персональном компьютере привела к разработке и широкому распространению сверхбыстрого, тактируемого интерфейса SRAM. Рассмотренные в этом разделе стандартные статические ОЗУ мы будем теперь называть *асинхронными статическими ОЗУ (asynchronous SRAM)*, чтобы отличать их от нового типа памяти, рассматриваемого ниже.

*10.3.5. Синхронные статические ОЗУ

Внутри новой разновидности статических ОЗУ, называемых *синхронными статическими ОЗУ (synchronous SRAM, SSRAM)*, по-прежнему применяются защелки, но имеется тактируемый интерфейс для сигналов управления, адресных сигналов и сигналов данных. Как показано на рис. 10.26, на пути адресных сигналов и сигналов управления находятся внутренние переключающиеся по фронту регистры AREG и CREG. В результате действие, задаваемое перед нарастающим фронтом тактового сигнала, выполняется внутри микросхемы на следующем такте. При записи в регистре INREG фиксируются входные данные. Если выходы микросхемы «конвейерные», то в ней имеется регистр OUTREG, обеспечивающий сохранение данных, выводимых при чтении; в случае «сквозных» выходов регистр OUTREG отсутствует.

Первыми появились *SSRAM с задержанной записью и сквозными выходами (late-write SSRAM with flow-through outputs)*. При выполнении операции чтения, показанной на рис. 10.27(а), выборка сигналов на входах управления и адресных входах производится на нарастающем фронте тактового сигнала, а внутренний регистр адреса AREG загружается только при условии, что сигнал ADS_L имеет активный уровень. В течение следующего такта происходит обращение к внутренней матрице статического ОЗУ, и считываемые данные поступают на выводы шины данных DIO устройства. Возможен также накатный режим работы такой памяти,

при котором данные читаются из следующих одна за другой ячеек. В этом режиме регистр AREG ведет себя как счетчик, так что нет необходимости в каждом цикле подавать новый адрес. (Управляющие сигналы, которые поддерживают пакетный режим, на рис. 10.26 и 10.27 не показаны.)

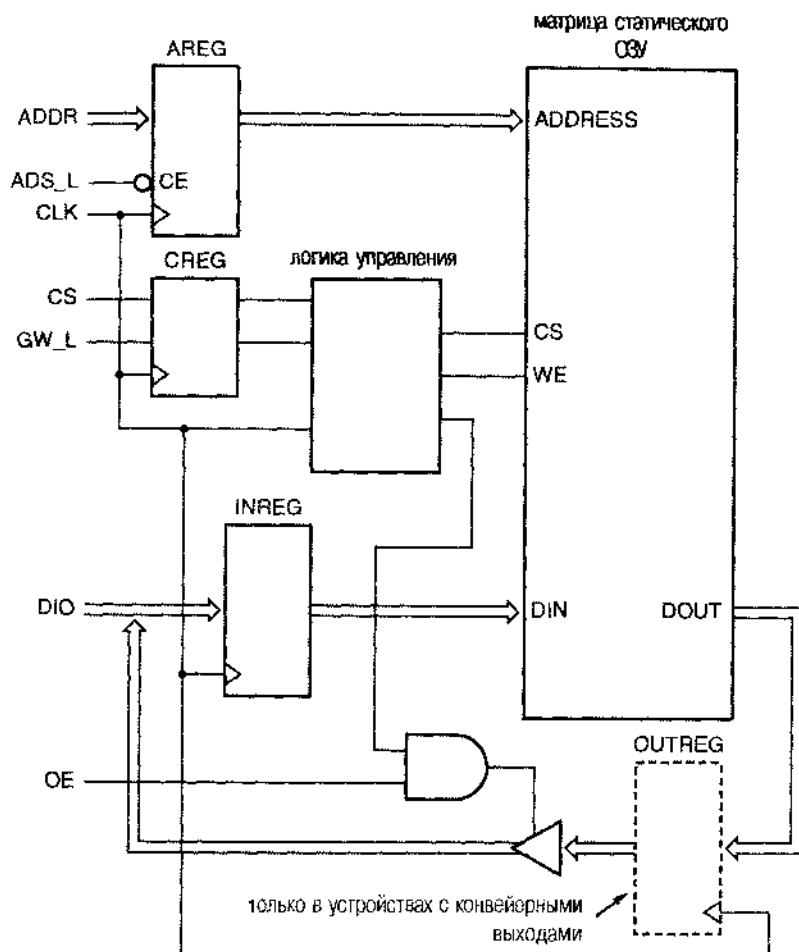
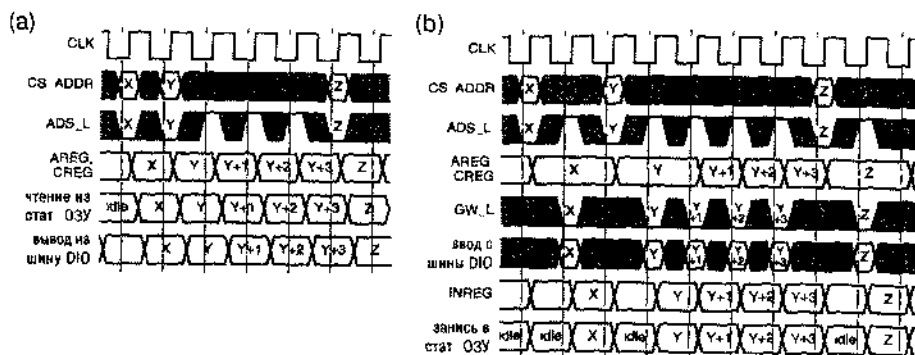


Рис. 10.26. Внутренняя структура синхронного статического ОЗУ

В цикле записи, изображенном на рис. 10.27(b), записываемые данные выбираются *спустя один такт после* загрузки регистра адреса и временно хранятся во внутреннем регистре INREG. Поэтому по крайней мере на один такт после загрузки адреса должна быть задержана подача сигнала ADS_L, чтобы при выполнении записи адрес в регистре AREG оставался неизменным. Запись происходит в течение такта, следующего за фронтом, по которому управляющий сигнал записи GW_L ("global write") переходит на активный уровень. Как и при чтении, возможен пакетный режим, когда запись по последовательным адресам осуществляется без подачи на вход новых значений.



Примечание: сигнал GW_L в течение всего времени имеет высокий уровень

Рис. 10.27. Временные диаграммы, поясняющие работу синхронного статического ОЗУ со сквозными выходами: (а) чтение; (б) запись (idle – ожидание)

Обратите внимание, что протокол с «задержанной записью» делает невозможным запись в соседних тактах по двум различным адресам в ячейки, не следующие одна за другой; матрица статического ОЗУ простаивает один такт между циклами записи (за исключением работы в накатном режиме). С точки зрения внутренних возможностей микросхемы простой статического ОЗУ не является необходимым. Однако протокол с задержанной записью был разработан именно таким, чтобы соответствовать протоколам шины микропроцессоров, в кэш-памяти которых используется синхронное статическое ОЗУ.

Память типа *SSRAM с задержанной записью и с конвейерными выходами (late-write SSRAM with pipelined outputs)* подобна памяти предыдущего типа, за исключением того, что между выходом матрицы статического ОЗУ и выходом микросхемы помещен регистр *OUTREG*, загружаемый при чтении. Как показано на рис. 10.28, выдача считываемых данных на выходы микросхемы задерживается до начала следующего такта, но это позволяет получать достоверные данные в течение почти всего периода тактового сигнала. Цикл записи выполняется так же, как и в случае памяти со сквозными выходами. Сравнение памяти со сквозными выходами и памяти с конвейерными выходами показывает, что последняя обеспечивает намного лучшее время установления для устройства, на входы которого поступают считываемые данные, и поэтому позволяет системе работать с более высокой тактовой частотой.

Как следует из рис. 10.26, у обычной памяти типа *SSRAM* одни и те же выходы используются в качестве входов и выходов данных. В пределах периода тактового сигнала выходы данных можно использовать либо для чтения, либо для записи, но не для того и другого одновременно. Внимательно рассмотрев, как используются шина данных и матрица статического ОЗУ в обоих вариантах памяти типа *SSRAM* с задержанной записью, вы увидите, что структура устройства не позволяет выполнить цикл чтения на следующем такте после цикла записи и наоборот (см. задачу 10.32). Таким образом, в случае применения памяти типа *SSRAM* с задержанной записью за смену режима приходится платить (*turn-around penalty*): внутренняя матрица статического ОЗУ должна простаивать в течение одного периода тактового сигнала, когда за циклом чтения следует цикл записи или наоборот.

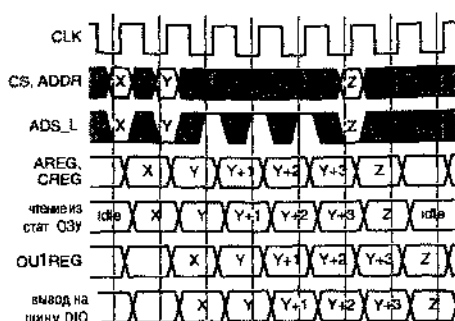


Рис. 10.28. Временные диаграммы, поясняющие выполнение операций чтения для памяти тип SSRAM с задержанной записью и конвейерными выходами (idle ожидание)

Необходимость платить за смену режима отсутствует, если у памяти тип *SSRAM* имеется *шина с нулевым временем смены режима [zero-bus-turn-around (ZBT) SSRAM, ZBT SSRAM]*. Временные диаграммы, поясняющие работу памяти типа *ZBT SSRAM* со сквозными выходами, показаны на рис. 10.29. Тип операции (чтение или запись) определяется управляющим сигналом $R/\overline{W}$, значение которого фиксируется по тому же фронту тактового сигнала, что и значения сигналов на адресных входах. Независимо от выполняемой операции шина *DIO* используется на следующем такте для передачи записываемых или считываемых данных. Какой-либо конфликт в отношении использования шины данных отсутствует при условии, что сигнал *OE* подается таким образом, чтобы избежать борьбы на шине между соседними циклами. Но если за циклом записи следует цикл чтения, то обеим операциям потребуется использовать матрицу статического ОЗУ в пределах одного и того же периода тактового сигнала. Чтобы избежать конфликта ресурсов, операция записи откладывается до следующего обращения к статическому ОЗУ, когда эта операция будет возможна. Такая возможность появляется в том случае, когда на линиях адреса и управления инициируется еще одна операция записи или не выполняется никакой операции.

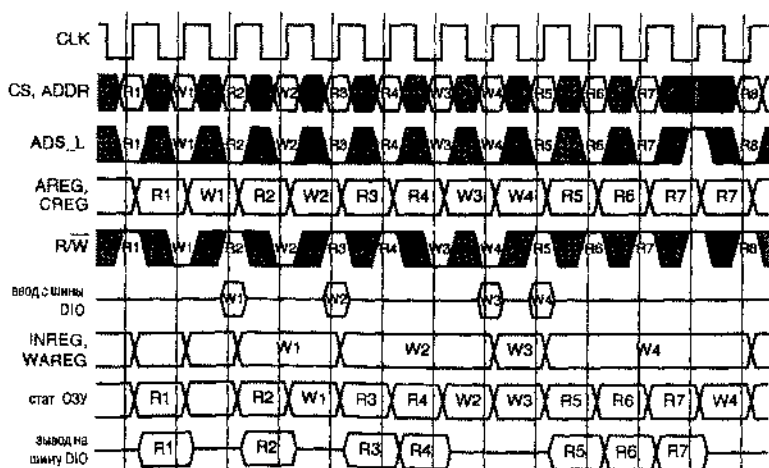


Рис. 10.29. Временные диаграммы, поясняющие работу памяти типа *ZBT SSRAM* со сквозными выходами

Хотя доступ к внутренней матрице статического ОЗУ в памяти типа ZBT SSRAM возможен на каждом такте, достигается это не даром. На время ожидания записи, адрес записи и связанная с ним информация должны быть сохранены в другом регистре WAREG, поскольку регистр AREG вновь используется другими операциями; это отражается на цене микросхемы, так как увеличивается площадь кристалла. Для некоторых приложений более существенным является то, что запись может оказаться задержанной на неопределенное время, если непосредственно за ней следует непрерывная серия циклов чтения. В таком аномальном режиме работы может потребоваться сложный контроллер, способный обнаруживать ситуации, когда в одном из циклов чтения происходит обращение по адресу, по которому только что производилась запись, так как хранящееся в матрице статического ОЗУ значение уже «устарело»!

На пути считываемых данных в памяти типа ZBT SSRAM с конвейерными выходами добавлен регистр OUTREG, а в остальном она подобна предыдущему устройству. В этом ОЗУ как при чтении, так и при записи, шина DIO используется в течение второго такта, следующего за фронтом тактового сигнала, по которому была инициирована данная операция. Как и в предыдущем устройстве, запись во внутреннюю матрицу статического ОЗУ задерживается до цикла, в котором запись будет возможна, посредством чего операции чтения предоставляется приоритет. Временные диаграммы, иллюстрирующие работу такой памяти, приведены на рис. 10.30. Как следует из этих диаграмм, необходимо наличие двух внутренних регистров WAREG для хранения адреса записи и двух внутренних регистров INREG для хранения данных, так как на время выполнения последовательных циклов чтения может быть задержано до двух циклов записи.

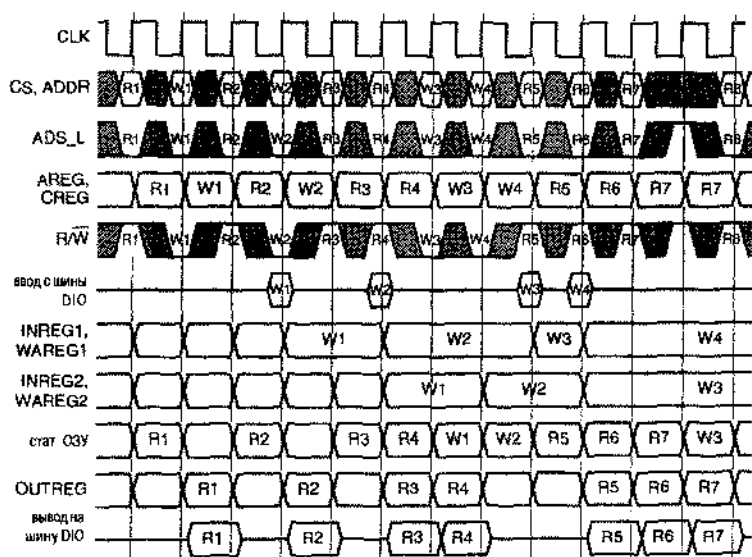


Рис. 10.30. Временные диаграммы, иллюстрирующие работу памяти типа ZBT SSRAM с конвейерными выходами

Протоколы синхронного доступа в статическое ОЗУ очень полезны в быстродействующих системах. Например, сигналы на адресные входы, на входы управления и на входы данных при записи можно подавать с соблюдением более или менее обычных требований по времени установления и времени удержания относительно системного тактового сигнала, а считываемые данные на выводах памяти с конвейерными выходами доступны в течение почти всего тактового цикла. Очень важно, что разработчик не должен беспокоиться о сложных схемах и о путях прохождения тактового сигнала; в противном случае это было бы необходимо для обеспечения надлежащей работы обычного статического ОЗУ на защелках.

Уже в 1999 году выпускались микросхемы памяти типа SSRAM с тактовой частотой до 166 МГц. Заметьте, что среди четырех рассмотренных нами разновидностей памяти типа SSRAM нет единственной «лучшей». Лучшей памятью типа SSRAM является та, которая наилучшим образом согласуется с протоколом шины и другими требованиями системы, в которой она используется.

10.4. Динамические оперативные запоминающие устройства

Основной ячейкой памяти в статическом ОЗУ является *D-защелка*, для которой требуются четыре вентиля в дискретном исполнении и от четырех до шести транзисторов в заказном статическом ОЗУ в виде БИС. Для того чтобы построить ОЗУ с более высокой плотностью (с большим числом двоичных ячеек в кристалле), разработчики микросхем памяти придумали ячейки, в которых на каждый бит приходится всего лишь по одному транзистору.

10.4.1. Структура динамического ОЗУ

Используя только один транзистор нельзя построить элемент с двумя устойчивыми состояниями. В ячейках памяти *динамического ОЗУ* (*dynamic RAM, DRAM*) информация сохраняется в виде напряжения на крошечном конденсаторе, доступ к которому осуществляется с помощью МОП-транзистора. На рис. 10.31 показана ячейка памяти динамического ОЗУ, в котором запоминается один бит и обращение к которой происходит при подаче на линию слова напряжения высокого уровня. Чтобы запомнить 1, на линию бита подается напряжение высокого уровня, которое через «открытый» транзистор поступает на конденсатор и заряжает его. Для сохранения 0 на линию бита подается напряжение низкого уровня, в результате чего конденсатор разряжается.

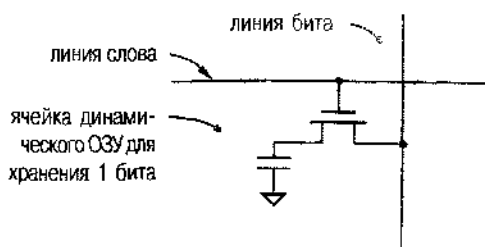


Рис. 10.31. Ячейка памяти в динамическом ОЗУ для хранения одного бита

Для чтения информации, хранящейся в ячейке динамического ОЗУ, на линии бита сначала устанавливается *напряжение предварительного уровня (precharge voltage)*, значение которого находится посередине между высоким и низким уровнями, а затем на линию слова подается напряжение высокого уровня. В зависимости от того, заряжен или разряжен конденсатор, напряжение на линии бита становится немного выше или немного ниже предварительного уровня. С помощью *усилителя считывания (sense amplifier)* это небольшое изменение напряжения обнаруживается и доводится до уровня логической 1 или логического 0 соответственно. Обратите внимание, что при чтении содержимого ячейки изменяется исходное напряжение на конденсаторе, поэтому после чтения хранившаяся в ячейке информация должна быть снова в нее записана.

Емкость конденсатора в ячейке динамического ОЗУ очень мала, но подключенный к конденсатору МОП-транзистор в запертом состоянии имеет очень большое сопротивление. Поэтому требуется относительно большое время (несколько миллисекунд) для того, чтобы конденсатор разрядился настолько, что имевшееся на нем напряжение высокого уровня упало до значения, соответствующего низкому уровню. В течение этого времени конденсатор хранит один бит информации.

Естественно, что при работе с компьютером вам было бы не до шуток, если бы приходилось каждые несколько миллисекунд его перезагружать из-за потери информации в его памяти (хотя именно так часто ведет себя операционная система Windows). Поэтому в системах памяти на основе динамических ОЗУ для обновления данных в каждой ячейке предусмотрены периодически повторяющиеся *циклы регенерации (refresh cycles)*. В первых динамических ОЗУ регенерация производилась каждые четыре миллисекунды. Цикл регенерации включает последовательно выполняемые чтение несколько ухудшенного содержимого каждой ячейки в D-защелку и повторную запись полноценного значения логического сигнала из защелки в ячейку. На рис. 10.32 показано напряжение на конденсаторе в ячейке памяти после записи и последующих циклов регенерации.

Первые динамические ОЗУ, появившиеся в начале 70-х годов, содержали только 1024 ячейки, а емкость современных динамических ОЗУ достигает 256 *мегабитов* и больше. Если бы требовалось обновлять все ячейки по очереди каждые четыре миллисекунды, то у вас возникли бы проблемы: время, отводимое на регенерацию содержимого одной ячейки, было бы гораздо меньше 1 нс, и не оставалось бы никакого времени для выполнения полезных операций чтения и записи. К счастью, как будет показано позже, динамические ОЗУ организованы в виде двухмерных матриц и за одну операцию регенерируется целая строка матрицы. У первых динамических ОЗУ было 256 строк, и требовалось 256 циклов регенерации каждые четыре миллисекунды, то есть цикл регенерации очередной строки должен был выполняться примерно через каждые 15.6 мкс. Современные матрицы памяти состоят из 4096 строк и их содержимое необходимо обновлять только один раз за 64 мс, так что по-прежнему цикл регенерации очередной строки должен производиться каждые 15.6 мкс. В типичном случае длительность цикла регенерации составляет менее 100 нс, так что динамическое ОЗУ доступно для полезных операций чтения и записи в течение более чем 99% времени.

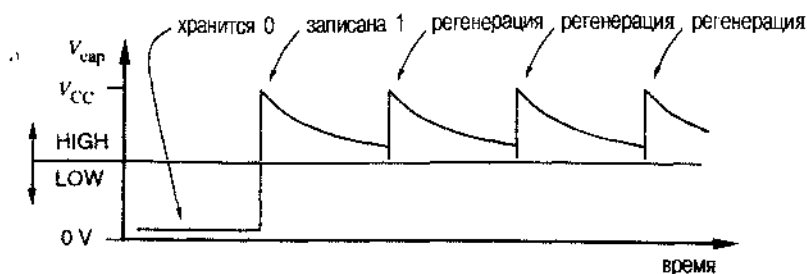


Рис. 10.32. Напряжение на конденсаторе в ячейке динамического ОЗУ после записи и выполнения циклов регенерации (HIGH – высокий уровень, LOW – низкий уровень)

На рис. 10.33 приведена внутренняя структура динамического ОЗУ 64Кх1. Емкость логической матрицы составляет 64Кх1 битов, но физически матрица представляет собой квадрат, состоящий из 256х256 ячеек. Несмотря на то, что память содержит 64К ячеек, микросхема имеет только восемь *мультиплексированных адресных входов (multiplexed address inputs)*. Полный 16-разрядный адрес поступает в микросхему за два шага по двум сигналам управления: *по строку адреса строки RAS_L (row address strobe)* и *по строку адреса столбца CAS_L (column address strobe)*. Благодаря мультиплексированию адресных входов удастся сократить число выводов, что важно для компактной реализации запоминающих устройств, и, кроме того, мультиплексирование совершенно естественно согласуется с двухступенчатыми методами доступа к динамическому ОЗУ, которые вскоре будут описаны.

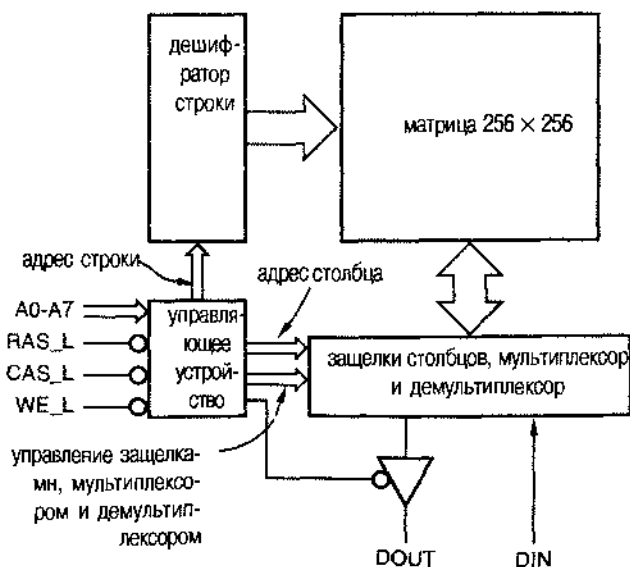


Рис. 10.33. Внутренняя структура динамического ОЗУ 64Кх1

Большинство динамических ОЗУ представляют собой большие матрицы, а часто состоят из нескольких матриц. Одно из достоинств применения нескольких матриц заключается в простоте решения электрических и физических проблем, которые возникали бы при проектировании матриц очень больших размеров. Но еще более важным является параллелизм, становящийся возможным при наличии нескольких матриц. Как мы увидим в следующем разделе, работа динамического ОЗУ намного сложнее, чем работа статического ОЗУ. Благодаря наличию в больших быстродействующих динамических ОЗУ нескольких матриц, современный контроллер динамического ОЗУ может выполнять параллельно несколько операций, например, завершать цикл записи в одной матрице, инициализируя цикл чтения в другой. В результате этого суммарный коэффициент использования памяти повышается.

10.4.2. Временные параметры динамического ОЗУ

Существует много различных временных сценариев работы динамических ОЗУ разного типа. В этом разделе мы рассмотрим наиболее общие циклы работы обычного динамического ОЗУ и укажем на их связь с внутренней структурой устройства. Самое замечательное свойство динамического ОЗУ состоит в том, что отсутствует тактовый сигнал. Операции в динамическом ОЗУ начинаются на спадающем фронте сигналов *RAS_L* и *CAS_L* и заканчиваются на нарастающем фронте этих сигналов. Чтобы реализовать это, необходимы серьезные временные ухищрения, но промышленность как-то умудряется преодолевать такого рода затруднения на протяжении последних 25 лет!

На рис. 10.34 приведены временные диаграммы *цикла регенерации только по стробу адреса строки (RAS-only refresh cycle)*. Этот цикл выполняется для обновления строки памяти фактически без чтения или записи каких-либо данных. Цикл начинается в тот момент, когда на мультиплексированных адресных входах (восемь битов в случае динамического ОЗУ 64К×1) присутствует адрес строки и сигнал *RAS_L* переходит на активный уровень. На спадающем фронте сигнала *RAS_L* во внутренний *регистр адреса строки (row-address register)* записывается адрес строки, и в *защелку строки (row latch)* на кристалле считывается выбранная строка матрицы памяти. Когда сигнал *RAS_L* переходит на неактивный уровень, содержимое строки из защелки строки перезаписывается в память. Для обновления содержимого всего динамического ОЗУ емкостью 64К×1 разработчик системы должен позаботиться о том, чтобы каждые четыре микросекунды выполнялись 256 таких циклов со всеми 256 возможными адресами строк. Для генерирования адреса строки можно применить внешний 8-разрядный счетчик, а для инициализации цикла регенерации каждые 15.6 мкс используется таймер.

Цикл чтения (read cycle), показанный на рис. 10.35, начинается аналогично циклу регенерации, при этом содержимое выбранной строки считывается в защелку строки. Затем на мультиплексированные адресные входы подается адрес столбца, который записывается во внутренний *регистр адреса столбца (column-address register)* по спадающему фронту сигнала *CAS_L*. Адрес столбца используется для выбора одного бита только что прочитанной строки, который появляется на выводе DOUT динамического ОЗУ. Пока сигнал *CAS_L* имеет активный уровень, выход DOUT с тремя состояниями, открыт. Тем временем, как только сигнал *RAS_L* переходит на неактивный уровень, содержимое всей строки переписывается обратно в матрицу.

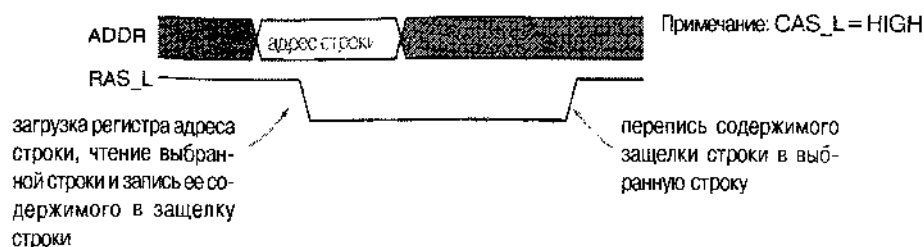


Рис. 10.34. Временные диаграммы цикла регенерации только по строку адреса строки (HIGH – высокий уровень)

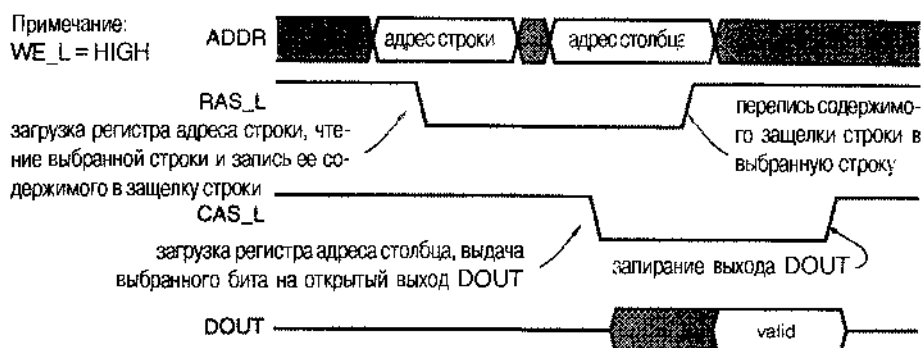


Рис. 10.35. Временные диаграммы цикла чтения из динамического ОЗУ (HIGH – высокий уровень, valid – установившееся значение)

ХИТРАЯ СИНХРОНИЗАЦИЯ

Вы, возможно, заметили, что интервалы установившихся значений сигналов адреса строки и адреса столбца на рис. 10.35 продлены вправо относительно спадающих фронтов сигналов RAS_L и CAS_L, но которым во внутренние перекрывающиеся по фронту регистры записываются адреса. Как правило, время установления сигналов для адресных входов динамического ОЗУ мало (часто 0 нс), но время удержания сигнала относительно велико (7-20 нс). Это может приводить к определенным затруднениям при конструировании другой, полностью синхронной системы, в которой используется единый тактовый сигнал и имеются триггеры с нулевым временем удержания. Возникающие трудности часто преодолеваются нежелательными способами, в том числе с помощью линий задержки с отводами путем переключения по другому фронту тактового сигнала. По этой причине многие разработчики считают проектирование систем с динамическими ОЗУ хитрой магией.

Цикл записи (*write cycle*), показанный на рис. 10.36, также начинается подобно циклу регенерации и чтения. Однако для того, чтобы выполнить цикл записи, сигнал *разрешения записи* *WE_L* (*write enable*) должен перейти на активный уровень прежде, чем будет установлен активный уровень сигнала *CAS_L*. Единственное, ради чего это делается, состоит в том, чтобы запереть выход *DOUT* на всю остальную часть цикла, несмотря на то, что впоследствии сигнал *CAS_L* переходит на активный уровень. Как только выбранная строка будет считана в защелку строки, бит, имеющийся на входе *DIN*, по сигналу *WE_L* записывается в ту ячейку в защелке строки, которая выбрана адресом столбца. Затем, когда содержимое строки переписывается в матрицу по нарастающему фронту сигнала *RAS_L*, в выбранном столбце этой строки присутствует новое значение.

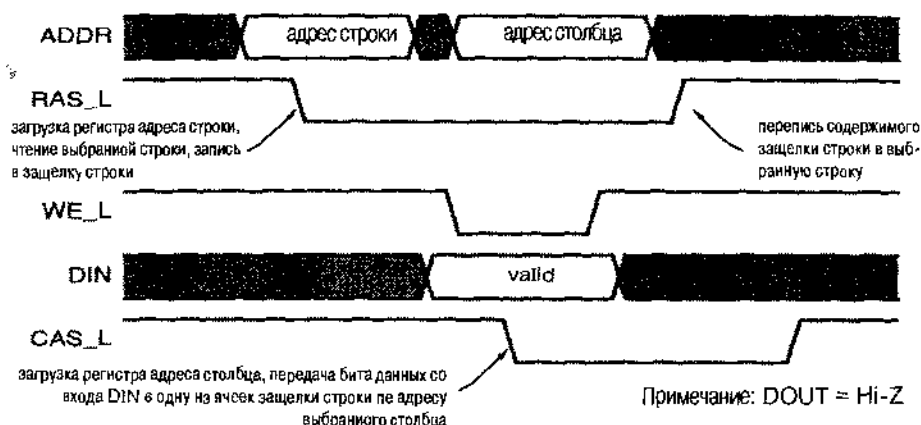


Рис. 10.36. Временные диаграммы цикла записи в динамическом ОЗУ (Hi-Z – третье состояние, *valid* – установившееся значение)

В типичных динамических ОЗУ возможны циклы и другого типа, не показанные на рисунке:

- *Цикл регенерации по строку адреса столбца, предшествующий циклу регенерации по строку адреса строки (CAS-before-RAS refresh cycle)*. В этом цикле осуществляется регенерация без подачи адреса строки от внешнего счетчика. Вместо этого используется внутренний счетчик адреса строки, имеющийся в самом динамическом ОЗУ. Если активный уровень сигнала *CAS_L* устанавливается раньше, чем активный уровень сигнала *RAS_L*, то в динамическом ОЗУ регенерируется строка, определяемая содержимым внутреннего счетчика, и затем оно увеличивается на единицу. Такая возможность упрощает разработку систем с динамической памятью: пропадает необходимость во внешнем счетчике регенерации и число мультиплексируемых источников, от которых поступают сигналы на адресные входы динамического ОЗУ, сокращается с трех (строка, столбец, регенерация) до двух.
- *Цикл «чтение-модификация-запись» (read-modify-write cycle)* начинается подобно обычному циклу чтения, при котором данные появляются на выходе *DOUT*, когда сигнал *CAS_L* переходит на активный уровень. Однако затем,

для того чтобы в то же самое место записать новые данные, может быть установлен активный уровень сигнала WE_L .

- *Цикл постраничного чтения (page-mode read cycle)* позволяет прочесть целую строку («страницу») данных без повторения полного цикла RAS-CAS. Когда в защелке строки уже хранится содержимое целой строки, для выполнения этого цикла просто требуется многократное повторение сигнала CAS_L в виде импульсов низкого уровня, в то время как сигнал RAS_L постоянно остается на активном уровне. На каждом спадающем фронте сигнала CAS_L формируется новый адрес столбца и на выходе DOUT появляется новый бит. Данный цикл обеспечивает намного более быстрый доступ к памяти при последовательном чтении из соседних ячеек, то есть из ячеек со следующими друг за другом адресами; такой доступ к памяти часто осуществляется в микропроцессорных системах при выборке команд и при заполнении кэш-памяти.
- *Цикл постраничной записи (page-mode write cycle)* аналогичен циклу постраничного чтения: он позволяет записать несколько битов строки, по одному сигналу RAS_L при многократном повторении сигнала CAS_L .

Одно время в некоторых динамических ОЗУ использовались два других режима многократного доступа: *режим статической выборки по столбцам (static-column mode)* и *режим выборки по слога́м (полубайтовый режим выборки; nibble mode)*. Их перестали применять в связи с преобладанием на рынке так называемых *динамических ОЗУ с увеличенным временем доступности данных [extended-data-out (EDO) DRAM]*. В этих устройствах во время циклов чтения состояние разрешения выхода уже не определяется сигналом CAS_L ; вместо этого используется отдельный вход управления выходом OE_L . Это существенно для быстрых циклов постраничного чтения, поскольку установившиеся значения данных удерживаются на выходе в течение большего интервала времени. Если раньше выходы переводились в третье состояние между импульсами CAS_L , то теперь (управляемые сигналом OE_L) они непрерывно остаются открытыми, и поэтому при постраничном чтении значения выходных сигналов неизменны от начала одного импульса CAS_L до начала следующего.

10.4.3. Синхронные динамические ОЗУ

Протокол доступа по фронту сигналов RAS/CAS обычных динамических ОЗУ не только сложен; трудно заставить эту память работать быстро и укладываться во временные графики при связи с остальными блоками системы. В результате в начале 90-х годов появились *синхронные динамические ОЗУ (synchronous DRAM, SDRAM)*, в которых применен более традиционный синхронный интерфейс, а к концу 90-х годов эти устройства стали доминирующими на рынке памяти для персональных компьютеров.

В синхронных динамических ОЗУ сохранен мультиплексный принцип адресации обычных динамических ОЗУ: адрес строки и адрес столбца подаются за два шага. Однако значения сигналов управления в синхронном динамическом ОЗУ, так же как и значения сигналов на адресных входах, фиксируются только на нарастающем фронте общего тактового сигнала CLK с частотой до 133 МГц. Кроме того, в синхронных динамических ОЗУ введен сигнал разрешения тактового сигнала CME_L .

если он имеет неактивный уровень, то другие сигналы управления и сигналы адреса игнорируются. При записи значения данных принимаются во внимание в момент прохождения фронта тактового сигнала, и при чтении данные поступают на выход по фронту тактового сигнала.

Точно так же, как и у обычного динамического ОЗУ, для реализации той или иной операции внутри синхронной памяти требуется выполнить определенное число шагов, а это занимает несколько тактов внешнего тактового сигнала. Внутри синхронного устройства памяти имеется несколько банков динамического ОЗУ, как правило, четыре, в которых могут осуществляться одновременно несколько операций.

В каждом периоде тактового сигнала сигналы управления RAS_L, CAS_L и WE_L интерпретируются синхронным динамическим ОЗУ как «командное слово», а не как отдельные управляющие воздействия. В то же время старшие адресные биты воспринимаются ОЗУ как «выбор банка»: они указывают, к какому банку относится команда. Например, «умный» контроллер синхронного динамического ОЗУ может использовать четыре тактовых импульса, для того чтобы инициализировать операции чтения в четырех различных банках, а затем вернуться к первому из них и считать готовые результаты, затрачивая по одному такту на каждый банк.

Внутренняя синхронизация в синхронном динамическом ОЗУ осуществляется внешним тактовым сигналом, подаваемым на вход CLK. Как правило, сигнал RAS, поступающий на внутреннюю матрицу, переходит на активный уровень немедленно после прохождения фронта синхросигнала, следующего за подачей команды чтения или команды записи. Для выполнения требований внутренней синхронизации, внутренний сигнал CAS в микросхеме вырабатывается позднее. На сколько тактов позже — зависит от частоты тактового сигнала CLK и быстродействия самой микросхемы памяти. Чтобы удовлетворить различным требованиям, интервал времени между сигналами RAS и CAS, называемый CAS-задержкой, делается программируемым. Величина этой задержки и несколько других важных рабочих параметров необходимо загружать в синхронное динамическое ОЗУ при инициализации. Загрузка довольно проста: устройство памяти распознает команду «загрузка параметров», когда одновременно переходят на активный уровень управляющие сигналы RAS_L, CAS_L и WE_L, а сами параметры поступают на адресные линии.

10.5. Интегральные схемы типа CPLD

С момента своего появления несколько лет назад программируемые логические устройства (ПЛУ) типа 16V8 и 22V10 стали основным, очень гибким инструментом цифрового проектирования. По мере развития технологии ИС, естественно, возрастал интерес к созданию ПЛУ с более развитой архитектурой, что позволило бы воспользоваться достоинствами повышенной плотности упаковки в кристалле. Вопрос заключается в следующем: почему производители не пошли по пути увеличения существующих устройств?

Плотность упаковки в динамических ОЗУ, например, увеличилась за последние 10 лет в 64 раза. Почему производители ПЛУ не смогли перейти от ИС 16V8 к выпуску более крупных ИС «128V64»? Такое устройство имело бы 64 входных

вывода, 64 I/O-вывода и некоторое количество термов-произведений (скажем, 8) со 128-переменными для каждой из 128 своих логических макроячеек. В таком устройстве можно было бы объединить функции, реализуемые большим числом ИС 16V8, и предложить потрясающую эффективность и гибкость функционального использования любых входных и выходных сигналов.

Было ли это возможно? Да, микросхема 128V64 была бы очень гибким компонентом, но она не обладала бы очень хорошими характеристиками. В отличие от ИС 16V8, у которой на каждый элемент И приходится 32 входа (16 сигналов и 16 их дополнений), в устройстве 128V64 у каждого элемента И было бы 256 входов. Из-за влияния емкостей, токов утечки и по другим причинам такая большая структура монтажного И была бы, по крайней мере, в восемь раз медленнее, чем матрица И в ИС 16V8.

С точки зрения производителя ситуация выглядит еще хуже: в микросхеме 128V64 экономически очень неэффективно использовалась бы площадь кристалла. Потребовался бы кристалл с площадью примерно в 64 раза больше, чем для ИС 16V8, при возможном числе входов и выходов всего лишь в восемь раз большем, чем у ИС 16V8. Другими словами, для n -кратного увеличения логики (в терминах числа входов, выходов и элементов И) микросхеме 128V64 понадобился бы кристалл с площадью, в n^2 раз большей. С точки зрения эффективного использования площади кристалла умный разработчик выиграл бы, разбив желаемую функцию так, чтобы реализовать ее на восьми ИС 16V8, если только такое разбиение возможно.

Эта идея была использована в сложных программируемых логических устройствах (*complex programmable logic device, CPLD*). Как показано на рис. 10.37, ИС типа CPLD является всего лишь совокупностью отдельных ПЛУ на одном кристалле с программируемой структурой взаимных связей, которая позволяет отдельным ПЛУ в пределах кристалла подключаться друг к другу так же, как это мог бы сделать талантливый разработчик с отдельными ПЛУ вне кристалла. Здесь площадь кристалла, необходимая для реализации увеличенной в n раз логики, равна площади только n одиночных ПЛУ плюс площадь, занимаемая программируемой структурой взаимных связей.

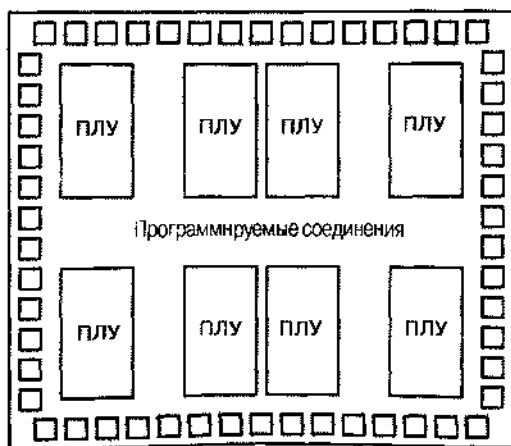


Рис. 10.37. Общая архитектура ИС типа CPLD

□ – блок ввода/вывода

Разные производители перепробовали много различных вариантов общей архитектуры, показанной на рисунке. Эти варианты отличаются блоками ПЛУ (состоящими из матрицы И и макроячеек), блоками ввода/вывода и структурой программируемых соединений. В данном параграфе мы рассмотрим каждую из этих составляющих, воспользовавшись в качестве примера архитектурой ИС типа CPLD серии 9500 фирмы Xilinx.

10.5.1. Семейство ИС XC9500 фирмы Xilinx

Микросхемы XC9500 фирмы Xilinx представляют собой семейство ИС типа CPLD однопакетной архитектуры, но с различным числом внешних I/O-выводов и с разным числом внутренних ПЛУ, которые фирма Xilinx называет *функциональными блоками (functional blocks, FBs)*. Как мы увидим позже, у каждого внутреннего ПЛУ 36 входов, он содержит 18 макроячеек и имеет 18 выходов; такое ПЛУ можно было бы назвать "36V18". Как следует из табл. 10.8, маркировка микросхем, определяется числом имеющихся в них макроячеек. Самый маленький представитель семейства содержит 2 функциональных блока с 36 макроячейками, а самый большой – 16 функциональных блоков с 288 макроячейками.

Табл. 10.8. Функциональные блоки и внешние I/O-выводы микросхем типа CPLD серии 9500 фирмы Xilinx

	Номер микросхемы					
	XC9536	XC9672	XC95108	XC96144	XC95216	XC96288
Число функциональных блоков и число макроячеек:	2/36	4/72	6/108	8/144	12/216	16/288
Тип корпуса	Число I/O-выводов					
VQFP с 44 выводами	34					
PLCC с 44 выводами	34	34				
CSP с 48 выводами	34					
PLCC		69	69			
с 84 выводами						
TQFP		72	81	81		
с 100 выводами						
PQFP		72	81	81		
с 100 выводами						
PQFP			108	133	133	
с 160 выводами						
HQFP					166	168
с 208 выводами						
BGA					166	192
с 352 выводами						

Другой важной особенностью этого семейства и большинства других семейств ИС типа CPLD является то, что одна и та же микросхема, скажем XC95108, выпускается в нескольких различных корпусах. Это существенно не только с точки зрения удовлетворения требований, предъявляемых различными технологиями производства, но также и для обеспечения оптимального выбора и возможности сэкономить на числе внешних I/O-выводов. В большинстве случаев не требуется, чтобы все внутренние сигналы микросхемы или подсистемы были доступны остальной частью системы и использовались ею.

Так, ИС XC95108 содержит 108 внутренних макроячеек, но при ее размещении в корпусе типа PLCC с 84 выводами наружу могут быть выведены выходы самое большее 69 макроячеек. На самом деле, как правило, большинство из 69 I/O-выводов используются как входы, поэтому вполне будет доступно еще меньшее число выходов. И это правильно: остальные выходы макроячеек вполне можно использовать внутри, так как к ним можно подключиться внутри через структуру программируемых соединений. Макроячейки, выходы которых доступны только внутри, иногда называют *скрытыми макроячейками* (*buried macrocells*).

Еще одним важным обстоятельством является то, что в одной строке и табл. 10.8 перечислены несколько микросхем. Оказывается, что в одинаковых корпусах любого типа, кроме двух, могут быть размещены, по крайней мере, два различных устройства с совместимыми выводами. Это значительно облегчает жизнь при изменении проекта в последнюю минуту. Предположим, например, что при проектировании вы выбрали ИС XC9572 в корпусе PLCC с 84 выводами. Возможно вы считаете, что 69 I/O-выводов, имеющихся у этой микросхемы, вполне достаточно. Вы хотели бы воспользоваться ИС XC9572 из-за ее низкой стоимости. Но если в вашем начальном проекте используются 68 из 72 макроячеек имеющихся внутри данной ИС, то это должно вызвать у вас определенную тревогу (со мной было бы именно так!). Глядя в табл. 10.8, можно быть спокойным, зная, что если обнаружатся ошибки или изменятся технические требования к проекту и потребуется более сложная внутренняя структура, то всегда можно перейти к ИС XC95108 в том же самом корпусе и воспользоваться еще 36 макроячейками.

На рис. 10.38 приведена блок-схема внутренней архитектуры типичной ИС типа CPLD из семейства XC9500. Ниже объясняется, что каждый внешний I/O-вывод можно использовать в качестве входа, выхода или двунаправленного вывода в соответствии с тем, как запрограммировано устройство. Выводы, расположенные в нижней части рисунка, можно использовать также для тех или иных специальных целей. На любой из трех выводов GSK можно подавать «общие тактовые сигналы»; как мы увидим ниже, каждую макроячейку можно запрограммировать так, чтобы на нее поступал тактовый сигнал с выбранного входа. Один вывод GSR можно использовать для подачи сигнала «общая установка/сброс»; снова, каждую макроячейку можно запрограммировать так, чтобы с помощью этого сигнала производилась асинхронная предварительная установка или сброс. Наконец, на любой из двух или из четырех выводов GTS (в зависимости от типа устройства) можно подавать сигнал, осуществляющий «общее управление третьим состоянием»; в каждой макроячейке можно выбрать один из этих сигналов для отключения или подключения соответствующего выхода, когда выход макроячейки подключен к внешнему I/O-выводу.

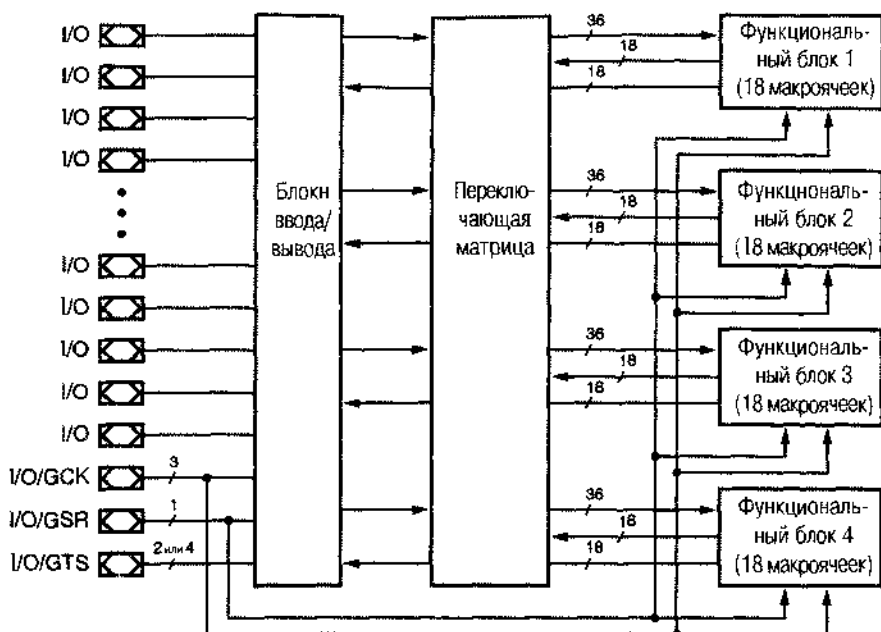


Рис. 10.38. Архитектура ИС типа CPLD семейства 9500 фирмы Xilinx

На рисунке показаны только четыре функциональных блока, по архитектура семейства XC9500 допускает наличие в ИС XC95288 16 функциональных блоков. Независимо от особенностей микросхемы, входящей в состав этого семейства, на входы каждого функционального блока путем программирования переключающей матрицы подаются 36 сигналов. На входы переключающей матрицы поступают сигналы с 18 выходов макроячеек от каждого функционального блока и внешние входные сигналы с I/O-выводов. Более подробно о том, как осуществляется коммутация в переключающей матрице, говорится в разделе 10.5.4.

Кроме того, у каждого функционального блока есть 18 выходов, сигналы на которых проходят «мимо» переключающей матрицы, как показано на рис. 10.38, и поступают на блоки ввода/вывода. Это просто сигналы разрешения выхода для выходных каскадов блока ввода/вывода; эти сигналы действуют в том случае, когда выход макроячейки данного функционального блока подключен к внешнему I/O-выводу.

10.5.2. Архитектура функционального блока

Архитектура функционального блока семейства XC9500 приведена на рис. 10.39. В программируемой матрице И имеется только 90 термов-произведений. По сравнению с такими ПЛЮ как 16V8 и 22V10 у ИС типа XC9500 и у большинства других ИС типа CPLD на одну макроячейку приходится меньшее число И-термов: у ИС XC9500 их всего лишь 5, в то время как у микросхемы 16V8 – 8, а у микросхемы 22V10 – от 8 до 16. Однако все не так плохо благодаря возможности *распределения термов-произведений (product-term allocation)*. У микросхем серии

XC9500, как и у других ИС типа CPLD, имеются *распределители термов-произведений (product-term allocators)*, поэтому термы-произведения, не востребованные в одной макроячейке, можно использовать в других, соседних макроячейках того же функционального блока.

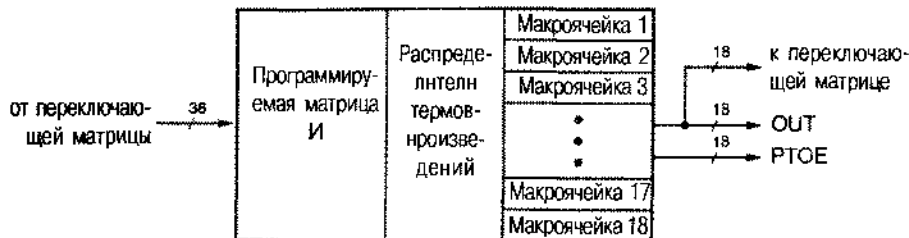


Рис. 10.39. Архитектура функционального блока

На рис. 10.40 представлена принципиальная схема распределителя термов-произведений и макроячейки ИС серии XC9500. На этом рисунке прямоугольниками с именами S1–S8 обозначены программируемые элементы, посредством которых сигналы, действующие на их входах, направляются на один из имеющихся у них выходов. Трапециевидные символы, обозначенные M1–M5, представляют собой программируемые мультиплексоры, которые подключают один из двух или четырех имеющихся у них входов к своему выходу.

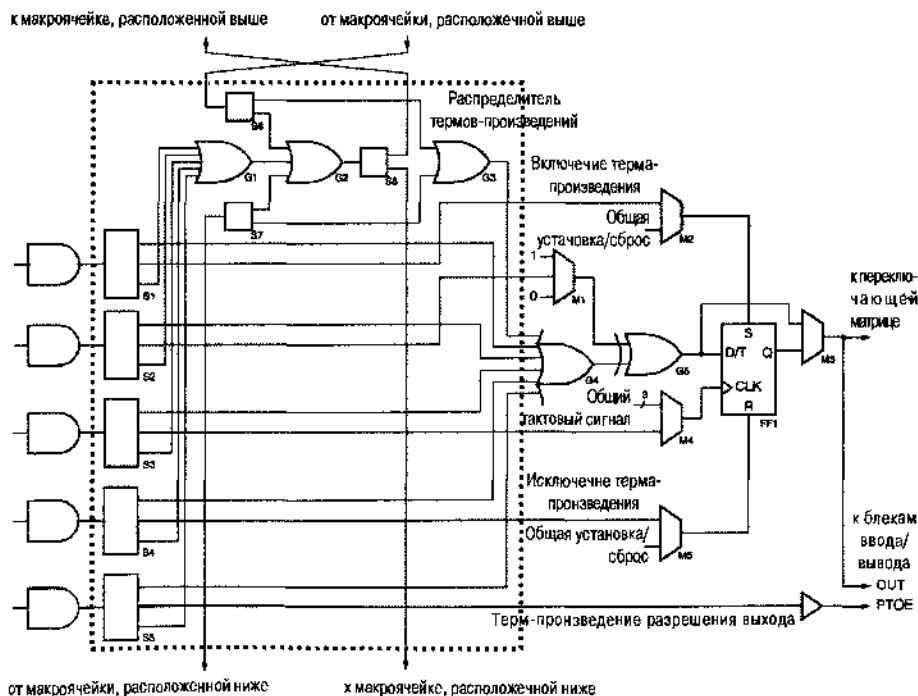


Рис. 10.40. Распределитель термов-произведений и макроячейка ИС серии XC9500

В левой части рисунка изображены 5 вентилей И, относящихся к данной макроячейке. Выходы каждого из них соединены с входами элементов, осуществляющих направление сигнала по тому или иному пути. С верхних выходов этих элементов терм-произведения поступают на вентиль G4 — главный вентиль ИЛИ данной макроячейки. Из сказанного можно заключить, что данной макроячейке доступны всего лишь пять термов-произведений. Однако верхний, ннестой вход вентиль G4 соединен с выходом другого вентиль ИЛИ (G3), на который поступают терм-произведения от макроячеек, расположенных выше и ниже данной макроячейки.

Любые не используемые в данной макроячейке терм-произведения можно с помощью направляющих узлов S1–S5 подать на входы объединяющего вентиль ИЛИ (G1), сигнал с выхода которого через элемент S8, в конце концов, может быть отправлен в макроячейку, расположенную выше, или в макроячейку, расположенную ниже. Перед направлением в другую макроячейку эти терм-произведения можно с помощью элементов S6, S7 и G2 объединить с термами-произведениями макроячеек, расположенных выше или ниже данной макроячейки. Таким образом, возможно «гирляндное подключение» термов-произведений через следующие одна за другой макроячейки для образования суммы, состоящей из большего числа произведений. В принципе, можно объединить и направить в одну макроячейку все 90 термов-произведений, имеющихс в данном функциональном блоке, хотя при этом 17 из 18 макроячеек этого функционального блока останутся вообще без термов-произведений.

За передачу термов-произведений по гирляндной цепочке соединений приходится платить не только тем, что другие макроячейки лишаются своих термов-произведений. При каждой «пересылке» терма-произведения вносится небольшая дополнительная задержка, которую можно минимизировать аккуратным размещением макроячеек, испытывающих недостаток термов-произведений, так чтобы они оказались соседними с макроячейками, в которых используется мало термов-произведений. Например, в макроячейке можно использовать 13 термов-произведений с задержкой, вносимой только одной дополнительной пересылкой, при условии, что эта макроячейка расположена между двумя макроячейками, в которых задействовано лишь по одному терму-произведению.

ДВИЖЕНИЕ В ОДНУ СТОРОНУ

При программировании данной макроячейки ИС типа XC9500 обычно не отправляют терм-произведения по гирляндной цепочке соединений назад, то есть в том направлении, откуда они поступают. Например, если терм-произведение попадает на вход элемента S6 сверху, то мы можем использовать его на месте, направляя сигнал с входа S6 к вентилю G3, или передать его на вентиль G2. В последнем случае элемент S8 должен направить сигнал с выхода G2 к макроячейке, расположенной ниже; снова направлять терм-произведение в верхнюю макроячейку нет никакого смысла. Если бы элементы S6 и S8 данной макроячейки отправляли терм-произведение вверх, а элементы S7 и S8 верхней макроячейки направляли бы его вниз, то у нас возникло бы нежелательное заикливание.

Третий вариант, когда сигнал появляется на среднем выходе какого либо из элементов S1–S5, служит для использования терма-произведения в качестве «специальной функции». Специальные функции – это подача сигнала на тактовый вход триггера, установка его в единичное состояние и сброс, а также управление вентилем ИСКЛЮЧАЮЩЕЕ ИЛИ и разрешение выхода. Обычно большинство этих специальных функций не используется.

Сердцевину макроячейки образует вентиль ИЛИ G4, на выходе которого возникает сумма всех выбранных термов-произведений, и вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ G5, на один из входов которого подается эта сумма произведений. Сигнал на другом входе вентиля G5 может быть равен 0 или 1, а также может быть термом-произведением в зависимости от того, что выбрано мультиплексором M1. При установке на этом входе вентиля G5 единицы, сумма произведений, поступающая с выхода вентиля G4, инвертируется, поэтому макроячейку можно сконфигурировать так, чтобы получить минимизированные логические выражения любой полярности. Подача на этот вход терма-произведения полезна при построении счетчиков. Если терм-произведение принимает значение 1 в том случае, когда биты младших разрядов счетчика равны 1 и счет разрешен, а сигнал на выходе вентиля G4 выражает собой текущее значение бита в данном разряде счетчика, то бит в данном разряде счетчика инвертируется, как и должно происходить в счетчике.

Триггер макроячейки FF1 можно запрограммировать для работы в качестве D-триггера или в качестве T-триггера с входом разрешения счета; последний вариант полезен при реализации счетчиков того или иного типа. С помощью мультиплексора M4 выбирается сигнал, подаваемый на тактовый вход триггера; этим сигналом может быть один из четырех входных сигналов мультиплексора: один из трех общих тактовых сигналов на входах ИС или терм-произведение. На выбор последнего сигнала в синхронных проектах наложен запрет, за исключением тщательно проработанных синхронизирующих устройств типа схемы, приведенной на рис. 8.111.

У триггера есть также входы асинхронной установки в единичное состояние и сброса. Выбор сигналов, подаваемых на эти входы, осуществляется мультиплексорами M2 и M5. В большинстве случаев входы установки и сброса бывают соединены с общим входом установка/сброс данной ИС и используются только при начальном запуске системы. Однако на эти входы можно также получить доступ к SR-защелке, у которой вход CLK не используется. Кроме того, тактовым входом CLK и входами S или R можно воспользоваться в синхронизирующих устройствах так, как это сделано в схеме на рис. 8.111, но при этом нужно быть очень внимательным.

В качестве выходного сигнала макроячейки OUT с помощью еще одного мультиплексора M3 выбирается сигнал с выхода триггера или сигнал, поступающий на его вход данных. Выходной сигнал OUT поступает на переключающую матрицу, где он может быть использован любой другой макроячейкой. Он может быть отправлен также к блокам ввода/вывода вместе с термом-произведением, выбранным элементом S5, который при необходимости, можно использовать как сигнал разрешения выхода PTOE.

10.5.3. Архитектура блока ввода/вывода

Структура блока ввода/вывода (*I/O block, IOB*) в ИС семейства XC9500 показана на рис. 10.41. Имеются семь вариантов выбора сигнала разрешения выхода для выходного буфера с тремя состояниями. Буфер может быть всегда открытым, всегда запертым, его состояние может определяться термом-произведением PTOE, поступающим от соответствующей макроячейки, или любым из четырех сигналов общего разрешения выхода. Сигналы общего разрешения выхода могут иметь как высокий активный уровень, так и низкий активный уровень, в зависимости от сигналов на внешних выводах GTS.

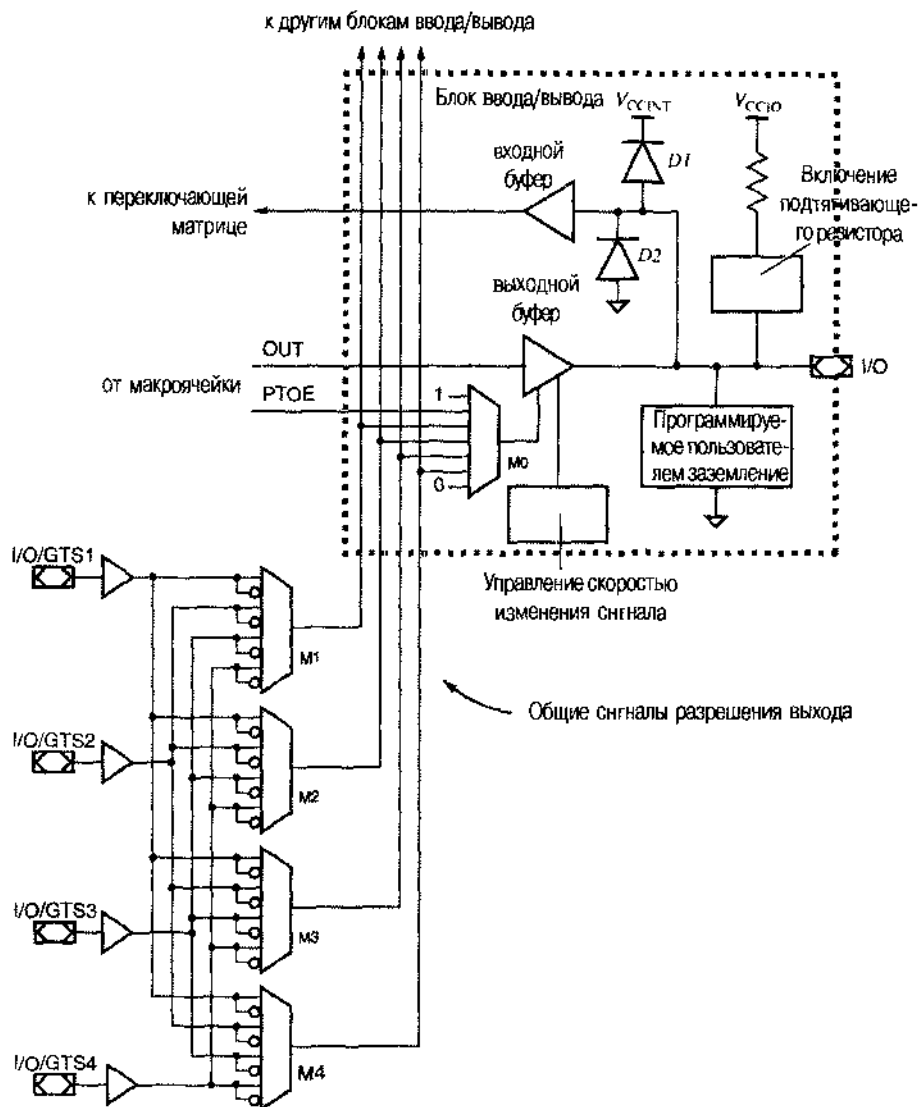


Рис. 10.41. Блок ввода/вывода микросхем серии XC9500

Блок ввода/вывода микросхем семейства XC9500 является хорошим примером важной тенденции в архитектуре блоков ввода/вывода микросхем типа CPLD и FPGA: в нем, кроме управления «логическими» действиями вроде разрешения выхода, имеется возможность изменять многие «аналоговые» параметры. В блоке ввода/вывода можно изменять значения трех аналоговых параметров:

- *Управление скоростью изменения выходного напряжения.* Для того чтобы получать быстродействующее или медленно работающее устройство, можно задавать время нарастания и спада выходных сигналов. Установка наибольшего быстродействия обеспечивает наименьшую возможную задержку распространения, в то время как задание режима медленной работы устройства позволяет уменьшить «звон» в линии передачи и шумы в системе за счет небольшой дополнительной задержки.
- *Включение резистора нагрузки между выходом и шиной питания.* Когда резистор нагрузки включен, он предотвращает появление на выходном выходе плавающего напряжения при подаче на микросхему напряжения питания. Это полезно в том случае, когда выходные сигналы поступают на входы разрешения других логических устройств с низким активным уровнем, в отношении которых не предполагается, что в момент включения питания на них будет подав сигнал, имеющий активное значение.
- *Образование программируемых пользователем выводов земли.* Эта возможность фактически позволяет перераспределять I/O-выводы так, чтобы те или иные выводы были выводами земли, а вовсе не сигнальными выводами. Это оказывается полезным в быстродействующих устройствах с высокой скоростью изменения сигналов. Необходимость в дополнительных выводах земли возникает в тех случаях, когда имеют место большие броски тока, возникающие при одновременном переключении сигналов на нескольких выходах.

Кроме этих особенностей, ИС семейства XC9500 совместимы с внешними устройствами с напряжением питания 5 В и 3.3 В. Выходной буфер и внутренняя логика работают от источника питания с напряжением V_{CCINT} , равным 5 вольтам. В зависимости от напряжения питания внешних устройств, в выходном каскаде используется напряжение питания V_{CCIO} , равное 5 В или 3.3 В. Обратите внимание, что включение резистора между выходом и шиной питания подтягивает напряжение на выходе до напряжения питания блока I/O, то есть до напряжения V_{CCIO} . Диоды $D1$ и $D2$ необходимы для фиксации выходного напряжения на уровне не выше значения V_{CCINT} и не ниже уровня земли. За эти границы величина выходного напряжения могла бы выходить из-за «звона» в линии передачи.

10.5.4. Переключающая матрица

Теоретически программируемая структура соединений внутри ИС типа CPLD должна допускать соединение любого внутреннего выхода ПЛУ любого внешнего входа с любым внутренним входом ПЛУ. Аналогично, должно быть возможным подключение любого выхода любого из внутренних ПЛУ к любому внешнему выводу. Но если присмотреться внимательнее, то становится ясно, что мы возвращаемся к той же самой «проблеме n^2 », которая возникала бы при попытке создания ИС 128V64.

Рис. 10.42 служит иллюстрацией требований, предъявляемых к переключающей матрице, на примере ИС XC95108 – типичного представителя семейства XC9500 фирмы Xilinx. У этой микросхемы имеются 108 выходов внутренних макроячеек и 108 внешних входных выводов, так что полное число сигналов, которые должны быть поданы на переключающую матрицу в качестве входных, составляет 216. Так как в ИС XC95108 имеется 6 функциональных блоков с 36 входами каждый, то переключающая матрица должна иметь в случае наибольшей полноты 216 мультиплексоров с 216 входами каждый, так чтобы сигнал с выхода каждого мультиплексора поступал на один из входов матрицы и одного из функциональных блоков.

Переключающая матрица, показанная на рисунке, может быть выполнена в микросхеме в виде прямоугольной структуры, у которой входы расположены по столбцам, а выходы – по строкам, с проходным транзистором (или логическим ключом) в каждой точке пересечения, позволяющим соединить данный вход с соответствующим выходом. Приведенный пример показывает, что это все же слишком большая структура: 216 строк и 216 столбцов!

При современной технологии создания ИС с высокой плотностью размещения компонентов проблема состоит не столько в размерах кристалла, сколько в быстродействии устройства. Большое число транзисторов, включенных в каждой строке и в каждом столбце, обуславливает наличие значительной емкости, которая приводит к уменьшению быстродействия. Поэтому производители ИС типа CPLD ищут способы уменьшить размер переключающей матрицы.

Глядя на рис. 10.42, можно увидеть, например, что не для каждого входа переключающей матрицы необходимо предусмотреть возможность его подключения к каждому выходу. Нужно лишь, чтобы каждый вход мог быть подключен к *какому-либо* входу каждого функционального блока, поскольку любой вход функционального блока может быть соединен с любым вентилем И матрицы и данного функционального блока.

Но снова задача совсем не так проста, как мы только что ее сформулировали. Предположим, что сигнал на выходе переключающей матрицы для i -го входа рассматриваемого функционального блока ($0 < i < 35$) вырабатывается простым 8-входовым мультиплексором, входы которого являются входами переключающей матрицы с номерами от $8i$ до $8i + 7$. С помощью такой переключающей матрицы нельзя осуществить многие из вариантов межсхемных соединений. Например, подключение входов переключающей матрицы с номерами от 0 до 35 к тому же самому функциональному блоку было бы невозможно. Как только вход 0 переключающей матрицы соединяется с входом 0 одного из функциональных блоков, входами переключающей матрицы с номерами от 1 до 7 уже нельзя воспользоваться.

Таким образом, требования, предъявляемые к возможности осуществления с помощью переключающей матрицы тех или иных соединений, следует сформулировать более широко: для каждого функционального блока должно быть возможным подключение любой комбинации входов переключающей матрицы к какой-либо комбинации входов данного функционального блока.

Переключающая матрица типичной ИС типа CPLD представляет собой компромисс между минимальной схемой мультиплексора и полной, неблокирующей структурой коммутационной матрицы. Если возможности переключающей мат-

рицы меньше, чем у неблокирующей матрицы, то проблема распределения соединений вход-выход в ней становится нетривиальной. При проектировании различных устройств на основе ИС типа CPLD каждый раз необходимо найти соответствующий набор связей, реализуемых перекрывающейся матрицей; это делается с помощью «программы компоновки», которой производитель ИС типа CPLD сопровождает свою продукцию.

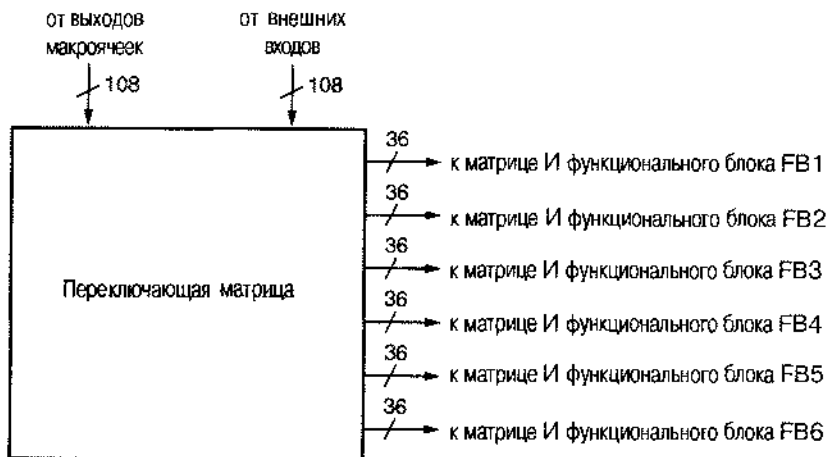


Рис. 10.42. Требования, предъявляемые к переключательной матрице ИС XC95108

Нахождение всех возможных комбинаций входов и выходов при разреженной переключательной матрице является одной из тех *NP*-полных задач, о которых вы, по-видимому, слышали в информатике. На практике это означает, что применительно к некоторым проектам, программе компоновки, вероятно, придется работать намного дольше, чем вам хотелось бы, чтобы узнать существует ли решение. Если в переключательной матрице слишком мало точек пересечения, как в нашем примере с очень маленькими мультиплексорами, то даже лучшая из программ компоновки при «сколь угодно долгой» работе в целом ряде случаев не сможет осуществить полный перебор всех возможных соединений.

Таким образом, структура переключательной матрицы ИС типа CPLD – это компромисс между характеристиками микросхемы (быстродействие, площадь кристалла, стоимость) и возможностями программы компоновки. Программа компоновки обычно не только устанавливает соединения в переключательной матрице, но также производит назначение входов и выходов функциональных блоков и макроячеек и их привязку к внешним выводам микросхемы, а также задает «внутреннюю логку» функциональных блоков и макроячеек. Эти назначения, в свою очередь, оказывают влияние на реализацию соединений внутри переключательной матрицы и на распределение термов-произведений. Решение этих проблем является «секретным ноу-хау» производителей ИС типа CPLD и разработчиков программного обеспечения и, как правило, ими не раскрывается.

ПРИВЯЗКА ВЫВОДОВ

Другой важной проблемой при разработке ИС типа CPLD и программного обеспечения является *привязка выводов (pin locking)*. В большинстве приложений ИС типа CPLD считается нормальным разрешить программе компоновки выбирать любые возможные выводы в качестве внешних входов и выходов данного устройства. Но когда проект закончен и изготовлена печатная плата, разработчик может пожелать «зафиксировать» назначенные выводы, так чтобы они оставались теми же самыми при небольших (или даже при больших!) изменениях, связанных с исправлением ошибок в проекте. Это приводит к экономии времени и стоимости и позволяет преодолеть препятствия, возникающие при переработке или доработке и переделке печатной платы.

Желаемая «привязка» выводов обычно указывается в файле, который читается программой компоновки. У первых ИС типа CPLD и FPGA привязка выводов до выполнения даже небольших изменений не гарантировала успеха: программа компоновки «поднимала руки» и жаловалась, что слишком много ограничений. Если вы разблокируете назначение выводов, то программа компоновки, возможно, найдет новое распределение, которое будет работать, но оно может оказаться совершенно не похожим на исходное.

Эти проблемы вовсе не обязательно возникали по вине программы компоновки; просто у ИС типа CPLD и FPGA не было достаточного количества внутренних связей, чтобы выдерживать постоянные изменения проекта при условии привязки выводов. Производители извлекли урок из опыта работы с этими первыми устройствами и усовершенствовали их внутреннюю архитектуру, приспособив ее к частым изменениям в процессе разработки проекта. В результате некоторые микросхемы теперь имеют, например, «выходную переключательную матрицу», которая гарантирует возможность соединения любого входа или выхода макроячейки, находящейся внутри данной ИС, с любым внешним I/O-выводом.

10.6. Интегральные схемы типа FPGA

Программируемая в условиях эксплуатации матрица вентилей (field-programmable gate array, FPGA) в какой-то мере подобна CPLD, вывернутой изнутри наружу. Как показано на рис. 10.43, на кристалле расположено большое число программируемых логических блоков, каждый из которых меньше, чем ПЛУ. Они распределены по всему кристаллу среди программируемых соединений, а вся матрица окружена программируемыми блоками ввода/вывода. Программируемый логический блок ИС типа FPGA обладает меньшими возможностями, чем типичное ПЛУ, но одна микросхема типа FPGA содержит гораздо больше логических блоков, чем ИС типа CPLD при том же самом размере кристалла.

Микросхемы типа FPGA были изобретены фирмой Xilinx, Inc., и в этом параграфе для иллюстрации архитектуры ИС типа FPGA мы воспользуемся одним из популярных семейств этой фирмы — семейством XC4000E.

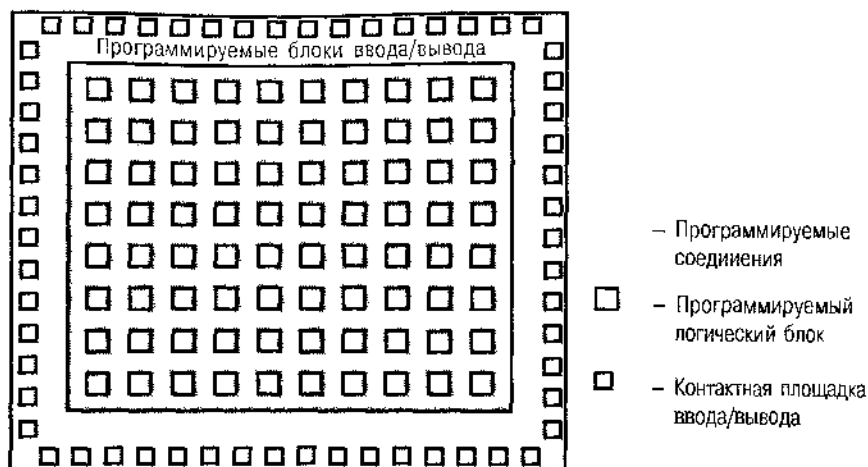


Рис. 10.43. Общая архитектура кристалла ИС типа FPGA

10.6.1. Семейство ИС типа FPGA XC4000 фирмы Xilinx

Программируемые логические блоки в ИС типа FPGA семейства XC4000E фирмы Xilinx пазваны *перестраиваемыми логическими блоками* (*configurable logic blocks, CLBs*). Наименьшая микросхема XC4003E содержит матрицу логических блоков размером 10×10 , а в самой большой ИС XC4025E – 1024 логических блока в виде матрицы размером 32×32 . Фирмой Xilinx на основе семейства XC4000E созданы также расширенные семейства XC4000EX и XC4000XL, у которых имеются дополнительные возможности и которые обладают не рассматриваемыми здесь свойствами. Самая большая ИС в расширенных семействах XC4085XL содержат 3136 логических блоков. В табл. 10.9 приведены о данные о выпускавшихся в 1999 году фирмой Xilinx микросхемах семейства XC4000.

Подобно семейству CPLD, семейство XC4000 включает набор микросхем разных размеров с разным числом I/O-выводов. В столбце таблицы «Макс. число доступных входов/выходов» указано максимальное число доступных пользователю блоков ввода/вывода, имеющихся в микросхеме. Однако ИС серии XC4000 размещают в различных корпусах и в случае корпуса меньших размеров не все имеющиеся входы/выходы выведены на внешние контакты. Как и в случае с ИС типа CPLD, пользователь микросхем типа FPGA имеет возможность перенести свой проект, выполненный на основе небольшой ИС, в микросхему большего объема, размещенную в том же самом корпусе, и наоборот.

В столбце «Число триггеров», как мы увидим позже, учтены все триггеры в устройстве, по два в каждом логическом блоке и по два в каждом блоке ввода/вывода. В типичном проекте используется только часть имеющихся триггеров, но полное их число является показателем возможностей, на которые ориентируется разработчик при грубой оценке размеров требуемой ИС типа FPGA. Данные столбца «Максимальное число битов в ОЗУ» являются другим показателем качества. Как мы увидим, каждый логический блок может либо выполнять логические операции, либо представлять собой небольшое статическое ОЗУ, хранящее до 32 битов.

Табл. 10.9. Микросхемы типа FPGA серии XC4000 фирмы Xilinx

Название микросхемы	Размер матрицы логических блоков	Число логических блоков	Макс. число доступных входов/выходов	Число триггеров	Макс. число битов ОЗУ (без логики)	Макс. число вентилях (без ОЗУ)	Типичное число вентилях (в логике и в ОЗУ)
XC4002XL	8×8	64	64	256	2 048	1 600	1 000–3 000
XC4003E	10×10	100	80	360	3 200	3 000	2 000–5 000
XC4005E/XL	14×14	196	112	616	6 272	5 000	3 000–9 000
XC4006E	16×16	256	128	768	8 192	6 000	4 000–12 000
XC4008E	18×18	324	144	936	10 368	8 000	7 000–15 000
XC4010E/XL	20×20	400	160	1 120	12 800	10 000	7 000–20 000
XC4013E/XL	24×24	576	192	1 536	18 432	13 000	10 000–30 000
XC4020E/XL	28×28	784	224	2 016	25 088	20 000	13 000–40 000
XC4025E	32×32	1 024	256	2 560	32 768	25 000	15 000–45 000
XC4028FX/XL	32×32	1 024	256	2 560	32 768	28 000	18 000–50 000
XC4036EX/XL	36×36	1 296	288	3 168	41 472	36 000	22 000–65 000
XC4044XL	40×40	1 600	320	3 840	51 200	44 000	27 000–80 000
XC4052XL	44×44	1 936	352	4 576	61 952	52 000	33 000–100 000
XC4062XL	48×48	2 304	384	5 376	73 728	62 000	40 000–130 000
XC4085XL	56×56	3 136	448	7 168	100 352	85 000	55 000–180 000

У параметра «Максимальное число вентилях» нет четкого определения. Согласно таблице каждый логический блок в ИС XC4002XL может выполнять функцию, реализуемую примерно 25 вентилями в дискретном исполнении. Микросхема XC4003E в этом отношении даже лучше: в ней каждый логический блок соответствует 30 вентилям. Так ли это? И еще: что такое «вентиль»? Считать ли схему ИСКЛЮЧАЮЩЕЕ ИЛИ или схему И-НЕ с большим числом входов одним вентилям? Вы можете решить это для себя позже, после того, как больше узнаете об архитектуре логического блока. Тогда же вы сможете решить, кем был составлен этот столбец, — разработчиками или людьми, занимающимися маркетингом.

Последний столбец таблицы, скорее всего, написан специалистами по маркетингу, так как верхняя граница каждого «типичного» диапазона для числа вентилях превышает максимальное значение в предыдущем столбце! На самом деле *имеется* приемлемое объяснение этого кажущегося противоречия. Данные этого столбца предполагают, что 20–30% логических блоков используется в качестве статических ОЗУ емкостью 32 бита каждое, а не в качестве логики. Для образования ячейки статического ОЗУ необходимо, как минимум, четыре вентиля, когда роль ячейки ОЗУ играет D-защелка (см. схему на рис. X7.27), и поэтому можно считать, что каждый логический блок, используемый в качестве статического ОЗУ, эквивалентен 128 вентилям. Таким образом, величина в последнем столбце равна числу вентилях, реализующих логические функции, *плюс* число эквивалентных логических схем, на которых построены статические ОЗУ.

10.6.2. Перестраиваемый логический блок

Так как ИС типа FPGA может содержать громадное число логических блоков, важно прежде всего разобраться в них! На рис. 10.44 показана внутренняя структура логического блока в микросхемах серии XC4000.

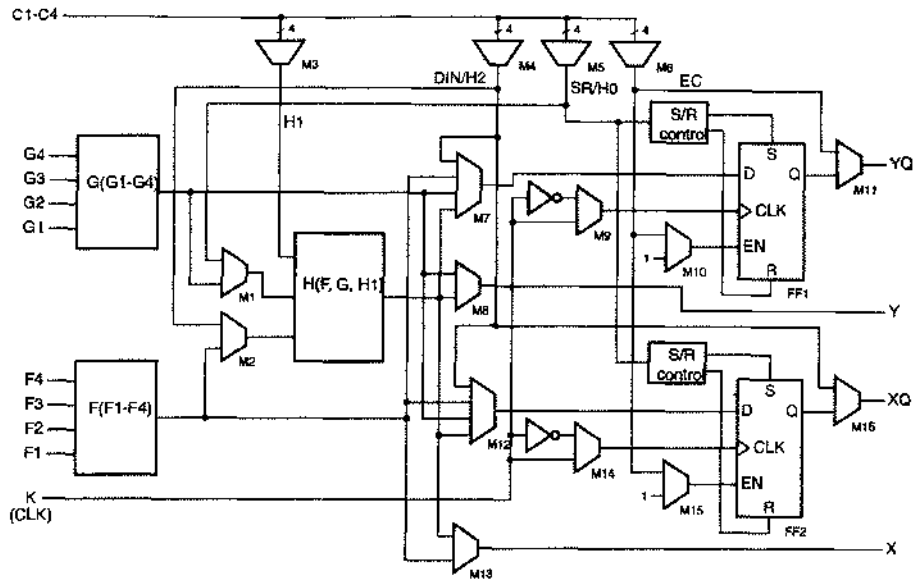


Рис. 10.44. Перестраиваемый логический блок микросхем семейства XC4000

Наиболее важными программируемыми элементами логического блока являются схемы F, G и H, вырабатывающие значения логических функций. С помощью элементов F и G можно реализовать любую комбинационную логическую функцию четырех переменных, а элемент H позволяет сформировать значение любой комбинационной логической функции трех переменных.

Как работают схемы F, G и H? Сколько вентилях вы затратили бы на построение «универсальной» логической схемы, реализующей функцию 4 переменных? Подумайте об этом, а мы возвратимся к этому ноже.

Как и при рассмотрении структуры ИС типа CPLD, трапециевидные символы на рис. 10.44 представляют собой программируемые мультиплексоры. Обратите внимание, что сигналы с выходов схем F и G, а также сигналы, поступающие на дополнительные входы логического блока можно подать через мультиплексоры M1–M3 на входы схемы H, поэтому можно реализовать логические функции с числом переменных больше четырех. Ниже приведен перечень функций, которые можно реализовать с помощью схем F, G и H в одном логическом блоке:

- Любая функция с числом переменных не более четырех, плюс любая другая функция с числом переменных не более четырех, которые не связаны с переменными первой функции, плюс любая третья функция с числом независимых переменных не более трех.

- Любая одна функция пяти переменных (см. задачу 10.34).
- Любая функция четырех переменных, плюс некоторые другие функции шести переменных, не зависящих от переменных первой функции.
- Некоторые функции с числом переменных до девяти, включая проверку на четность и проверку равенства двух 4-разрядных двончных слов; в последнем случае возможно последовательное включение схем проверки (см. задачи 10.35 и 10.36).

При соответствующем программировании мультиплексоров M7–M8 и M12–M13 сигналы с выходов схем, вырабатывающих значения функций, могут быть выведены на выходы X и Y логического блока или запомнены в переключающихся по фронту D-триггерах FF1 и FF2. Триггеры могут срабатывать по нарастающему или по спадающему фронту общего тактового сигнала на входе K, в зависимости от выбора, сделанного с помощью мультиплексоров M9 и M14. С помощью мультиплексоров M10 и M15 в триггерах может быть также использован вход разрешения тактового сигнала EC. Сигнал EC и три других внутренних сигнала выбираются мультиплексорами M3–M6, изображенными в верхней части рис. 10.44, из четырех различных сигналов, поступающих на входы C1–C4.

На выходы XQ и YQ выводятся сигналы с выходов триггеров данного логического блока. Если в каком-то логическом блоке триггеры не используются, то с помощью мультиплексоров M11 или M16 осуществляется «сквозное» прохождение на выходы XQ или YQ входных сигналов, выбранных мультиплексорами M4 и M6.

При программировании блока узлы, названные “S/R control”, задают начальное состояние каждого триггера: 1 или 0. Этими узлами устанавливается также, на какой сигнал реагируют триггеры: на общий сигнал установки/сброса (не показанный на рисунке) или на сигнал SR данного логического блока, выбираемый мультиплексором M5.

Ничего себе, как много возможностей для программирования! Естественно, что задание структуры логического блока в микросхемах серии XC4000 – независимо от того, сколько логических блоков они содержат: 3136 или только 64, – не выполняется вручную. Производитель обеспечивает пользователя программой компоновки, которая распределяет, конфигурирует и соединяет логические блоки так, чтобы схема соответствовала описанию проекта на более высоком уровне, то есть описанию на языках ABEL, VHDL или Verilog, либо в виде принципиальной схемы.

Давайте вернемся к нашему вопросу: как настроить универсальную схему, реализующую логические функции 4 переменных? Если вы решаете эту задачу на уровне вентилей, то она оказывается очень сложной, но если посмотреть на нее с другой точки зрения, то ее решение значительно облегчается. Любая функция 4 переменных может быть описана таблицей истинности, состоящей из 16 строк. Предположим, что мы храним таблицу истинности в 1-разрядной памяти на 16 слов. Подавая на адресные входы памяти четыре входных бита, мы получаем на выходе значение функции для этой комбинации значений переменных.

Именно такой подход был принят разработчиками ИС типа FPGA в фирме Xilinx. Схемы F и G, вырабатывающие значения логических функций, фактически являются всего лишь очень компактными и быстрыми статическими ОЗУ

16×1, а схема Н представляет собой статическое ОЗУ 8×1. Когда логический блок используется для выполнения логических операций, при формировании его структуры в статическое ОЗУ из внешнего ПЗУ загружаются таблицы истинности логических функций F, G и H. Программирование мультиплексоров, указанных на рис. 10.44, также осуществляется путем загрузки ячеек памяти, представляющих собой отдельные D-защелки, управляющие мультиплексорами; информация в них тоже заносится при конфигурировании ИС. Такого рода программирование выполняется для всех логических блоков, входящих в состав ИС типа FPGA.

Помимо удобства программирования, применение памяти для хранения таблиц истинности имеет другое важное достоинство. Любой логический блок в микросхеме серии XC4000 при запуске можно сконфигурировать так, чтобы использовать его в качестве памяти, а не логики. Возможны несколько разных режимов формирования структуры логического блока:

- *Два статических ОЗУ 16×1.* Схемы F и G используются в качестве статических ОЗУ с независимыми адресными входами и входами записываемых данных. Однако вход разрешения записи у них общий.
- *Одно статическое ОЗУ 32×1.* Одни и те же четыре адресных бита подаются на входы схем F и G, а пятый адресный бит, поступающий на схему H и на схему разрешения записи, позволяет выбирать между верхней и нижней половинами памяти F и G.
- *Асинхронный или синхронный режим работы.* Структура упомянутых выше статических ОЗУ может быть сформирована так, чтобы при записи происходила «нормальная» асинхронная фиксация данных, либо запоминание данных осуществлялось по заданному фронту тактового сигнала K.
- *Одно статическое ОЗУ 16×1 с двумя портами.* Возможно независимое выполнение чтения и записи для двух разных ячеек одного и того же статического ОЗУ по двум наборам сигналов на адресных входах. В данном режиме поддерживаются только синхронные операции записи.

В этих режимах сигналы на функциональных входах F1–F4 и G1–G4 играют роль адреса, а на входы H0–H2 логического блока подаются данные и сигнал разрешения записи; сигналы данных, появляющиеся на выходах F и G, можно запомнить в триггерах FF1 и FF2 или вывести на выходы X и Y данного логического блока.

10.6.3. Блок ввода/вывода

Структура блока ввода/вывода (*I/O block, IOB*) в ИС семейства XC4000 показана на рис. 10.45. I/O-вывод можно использовать в качестве входа или выхода, либо в качестве того и другого.

У блока ввода/вывода в микросхемах семейства XC4000 больше средств «логического» управления, чем у его ближайшего «родственника» в ИС типа CPLD семейства XC9500. В частности, на пути входного и выходного сигналов имеются переключающиеся по фронту D-триггеры, возможность записи в которые определяется мультиплексорами M5–M7. Размещение входного и выходного триггеров «рядом» с I/O-выводами является особенно полезным свойством ИС типа

FPGA. Относительно большие задержки при прохождении сигналов от выходов внутренних триггеров логических блоков до блоков ввода/вывода могут затруднить стыковку данной ИС со стороны ее выходов с внешними синхронными системами, если частота тактового сигнала очень высока. Большие задержки от I/O-выводов до входов триггеров в логических блоках могут затруднить сопряжение данной ИС со стороны ее входов с внешней системой с точки зрения удовлетворения требованиям по времени установления и времени удержания, если внешние входные сигналы поступают непосредственно на тактовые входы триггеров внутри логических блоков, а не фиксируются сначала триггерами в блоках ввода/вывода. Конечно, применение триггеров в блоках ввода/вывода возможно только в том случае, когда технические требования к внешнему интерфейсу ИС типа FPGA допускают «конвейерный режим» работы по входам и выходам.

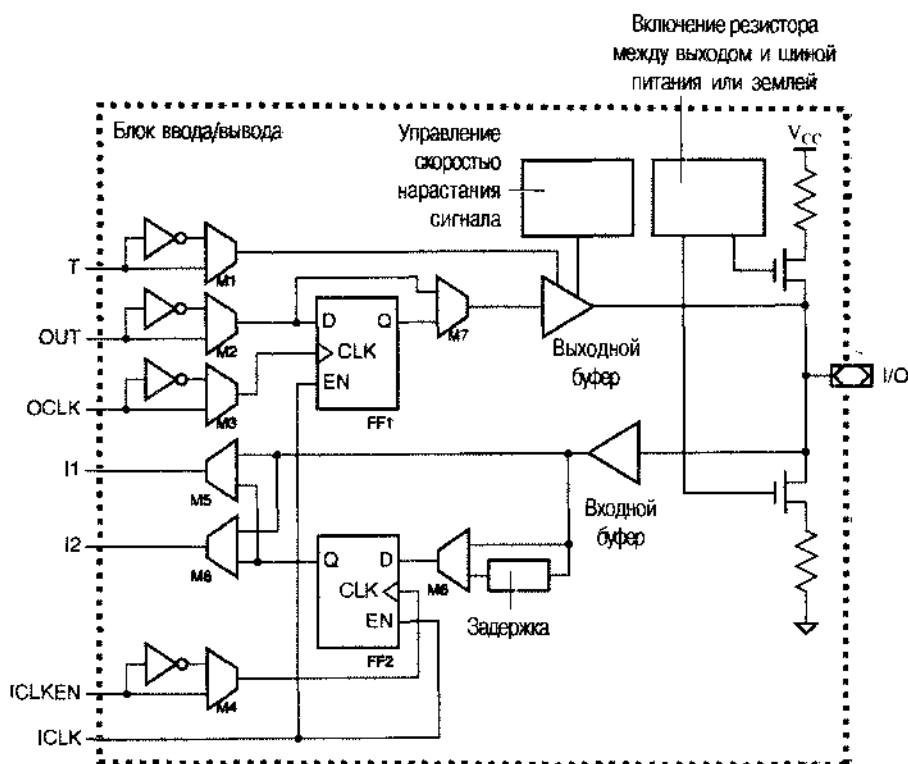


Рис. 10.45. Блок ввода/вывода в ИС семейства XC4000

Чтобы обеспечить конвейерный режим работы по входу в блоке ввода/вывода ИС серии XC4000 фактически делается еще один шаг в этом направлении: с помощью мультиплексора M8 на пути к D-входу входного триггера FF2 может быть введена задержка. Действие этого элемента заключается в задержке сигнала на D-входе относительно копий системных тактовых сигналов внутри данной ИС, гарантирующей, что время удержания по входу относительно внешнего тактового сигнала будет равно нулю. Этот режим достигается, конечно, за счет увеличения времени установления.

Другой возможностью логического управления в блоке ввода/вывода является выбор – с помощью мультиплексоров M1–M4 – полярности четырех входных сигналов, поступающих из матрицы логических блоков по структуре программируемых соединения. Этими входными сигналами являются: выходной бит OUT, сигнал разрешения T, открывающий выход с тремя состояниями, выходной тактовый сигнал OCLK и сигнал разрешения ICLKEN по тактовому входу.

Подобно блокам ввода/вывода в микросхемах серии XC9500, в блоках ввода/вывода микросхем серии XC4000 возможно управление аналоговыми параметрами выходного сигнала. Можно запрограммировать скорость изменения сигнала, вырабатываемого выходным буфером, а между I/O-выводом и шиной питания или землей можно включить резистор.

10.6.4. Программируемые соединения

Итак, лучшее мы оставили напоследок. Архитектура программируемых соединений в микросхемах серии XC4000 – прекрасный пример структуры, обеспечивающей богатые возможности образования необходимых связей и занимающей небольшую площадь на поверхности кремниевого кристалла.

Как показано на рис. 10.43, каждый логический блок в ИС типа FPGA окружен структурой соединений, которая в действительности является всего лишь совокупностью «проводов» с возможностью подключения к ним посредством соответствующего программирования. На рис. 10.46 структура соединений в ИС серии XC4000 изображена немного подробнее. Сигнальные линии действительно не являются «собственностью» какого-либо одного логического блока, а матрица логических блоков внутри ИС представляет собой мозаику, составленную точно из таких структур, какая показана на рисунке. Например, 100-кратное повторение этого рисунка дает матрицу логических блоков микросхемы XC4003 размером 10×10.

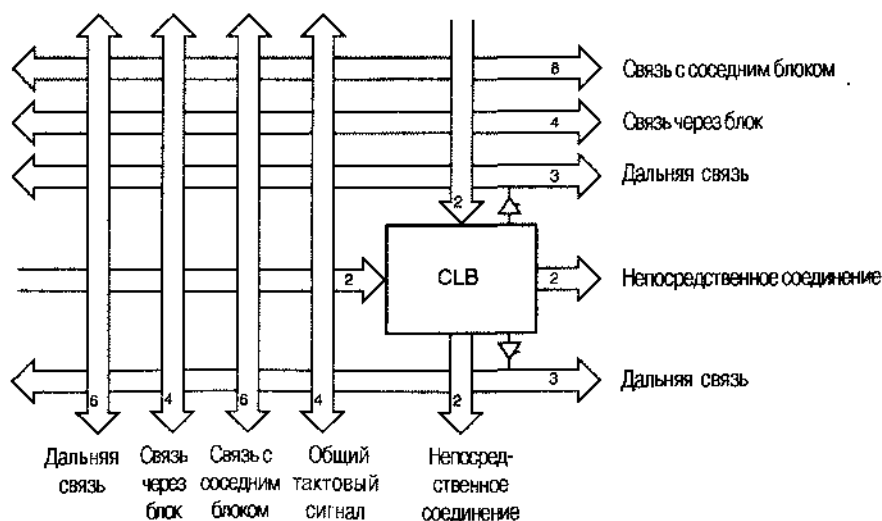


Рис. 10.46. Общая структура соединений в микросхемах серии XC4000

Цифрами внутри стрелок указано число сигнальных линий в каждой шине. Таким образом, мы видим, что у логического блока есть две выходные шины, идущие к логическим блокам, расположенным непосредственно под данным блоком и справа от него. Кроме того, каждый логический блок соединяется с тремя шинами, изображенными над ним, с одной шиной, находящейся под ним и с четырьмя шинами, расположенными слева от него. Сигналы по этим шинам могут передаваться в любом направлении.

Четыре сигнальные линии в шине, названной «Общий тактовый сигнал», позволяют логическому блоку наилучшим образом использовать тактовые входные сигналы, обеспечивая малую задержку и минимальный разброс задержек. Две шины «Связь с соседним блоком» предназначены для гибкой связи между расположенными рядом блоками помимо шин «Непосредственное соединение» с небольшим числом сигнальных линий и однонаправленной передачей сигналов.

По шине «Связь с соседним блоком» данный логический блок можно соединить не только с соседним, но и с другим блоком, но для этого потребуется более одной пересылки; при этом каждый раз сигналы должны пройти через программируемый переключатель, в результате чего задержка увеличивается. По шинам «Связь через блок» сигналы проходят мимо двух логических блоков, прежде чем попадают на переключатель, что позволяет получить меньшие задержки при более длинных связях. Для действительно длинных связей используются шины «Дальняя связь», в которых сигналы вообще не проходят через какие-либо программируемые переключатели; вместо этого сигналы вырабатываются буферами с тремя состояниями, помещенными рядом с логическим блоком, и проходят вдоль всей строки матрицы, образованной логическими блоками, или вдоль всего столбца.

Более детально логический блок и линии связи показаны на рис. 10.47. Маленькими квадратиками обозначены программируемые соединения: горизонтальная линия либо соединена с вертикальной линией, либо не соединена, в зависимости от значения бита (который опять хранится в защелке), указанного при программировании этого ключа. По краям матрицы, образованной логическими блоками, имеются дополнительные, специализированные программируемые соединения, обеспечивающие подключение к блокам ввода/вывода.

Заштрихованная область на рисунке, выделенная цветом, называется *матрицей программируемых переключателей (programmable switch matrix, PSM)*. Более подробно эта матрица показана на рис. 10.48. Каждый ромбик на рис. (а) представляет собой *элементарный программируемый переключатель (programmable switch element, PSE)*, с помощью которого любую линию можно соединить с любой другой, как показано на рис. (b). На рис. (b) приведены 6 возможных попарных соединений четырех линий, и для каждого из них в элементарном программируемом переключателе имеется логический ключ. Могут быть задействованы несколько логических ключей, при одном из них или все; это снова определяется конфигурацией битов, которые хранятся в защелках. Таким образом, как показано на рис. (c), возможно много различных вариантов связи.

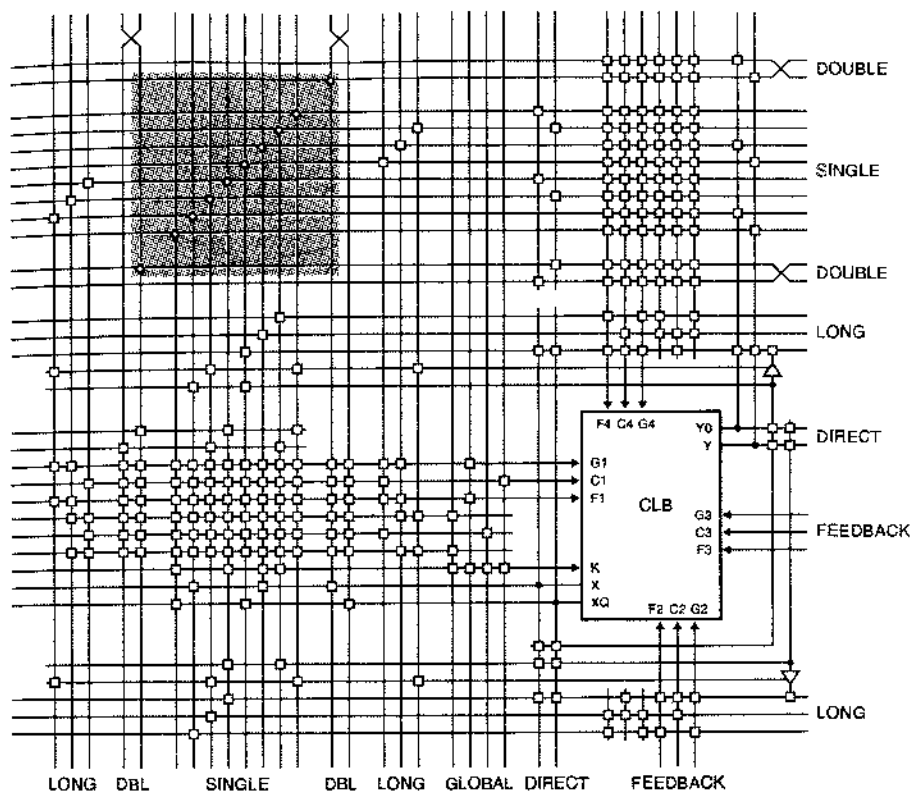


Рис. 10.47. Логический блок микросхем серии XC4000 и окружающая его структура соединений (LONG – дальняя связь; DBL, DOUBLE – связь через блок; SINGLE – связь с соседним блоком; GLOBAL – общий тактовый сигнал; DIRECT – непосредственное соединение; FEEDBACK – обратная связь)

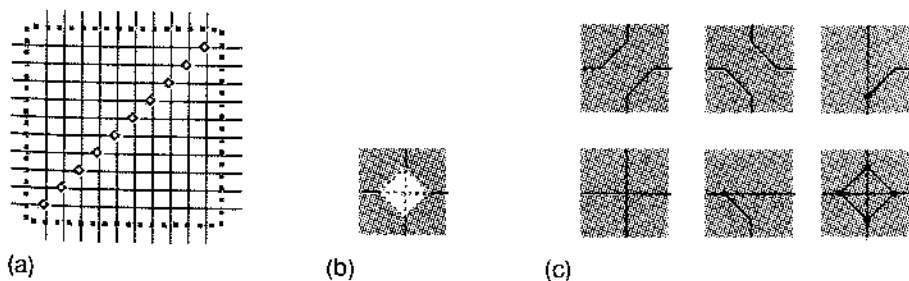


Рис. 10.48. Программируемые соединения в ИС серии XC4000: (а) матрица программируемых переключателей (PSM); (б) элементарный программируемый переключатель (PSE); (с) несколько возможных соединений

Матрица программируемых переключателей является существенным компонентом структуры соединений, представленной на рис. 10.47, обеспечивающей

связь между блоками. Установление соединений или их размыкание в элементарном программируемом переключателе позволяет продлевать и разрывать проводящие сегменты в шинах «Связь с соседним блоком» и «Связь через блок». Еще более важно, что матрица программируемых переключателей позволяет «повернуть сигналы на 90°» путем соединения горизонтального и вертикального проводников. Без этого нельзя было бы соединить данный логический блок с другими блоками, находящимися в другой строке или в другом столбце матрицы, образуемой логическими блоками.

Хотя матрица программируемых переключателей является необходимым компонентом, но за его использование приходится платить: при каждом прохождении сигналов через такую матрицу вносится небольшая задержка. Поэтому хорошая программа компоновки для ИС типа FPGA ищет не только какие-либо возможные размещения логических блоков и какую-то комбинацию соединений, которые будут работать. Программа «размещения и трассировки» затрачивает много времени, пытаясь оптимизировать характеристики устройства путем нахождения такого размещения, которое позволило бы сделать соединения короткими, и только после этого осуществляет реализацию самих соединений.

Подобно ИС типа CPLD, о достоинствах микросхем типа FPGA судят по гибкости их структур и устойчивости результатов, даваемых программой компоновки, при небольших изменениях в проекте. Самое большое разочарование наступает тогда, когда обнаруживается, что при небольших изменениях в большом проекте уже не выполняются требуемые временные соотношения. Как следствие этого, производители ИС типа FPGA научились обеспечивать наличие в архитектуре этих микросхем «дополнительных» ресурсов, чтобы помочь пользователям гарантированно получать стабильные результаты.

ХОРОШАЯ ПРАКТИКА

Размещение и трассировка являются понятной задачей, поскольку она является главной составляющей в действиях «внутреннего плана» при проектировании любой заказной микросхемы. Таким образом, одни и те же программы и одни и те же поставщики программ оказываются вовлеченными в решение задачи размещения и трассировки как в случае ИС типа FPGA, так и в случае специализированных ИС. Таким образом, можно считать, что любой опыт, приобретенный при работе с ИС типа FPGA, служит хорошей практикой проектирования специализированных ИС!

Обзор литературы

Производители ПЗУ и ОЗУ издают справочники, содержащие характеристики выпускаемых ими микросхем, но гораздо больше справочных данных и указаний по применению отдельных микросхем они публикуют на своих Web-сайтах. Информацию о выпускавшихся программируемых ПЗУ с ультрафиолетовым стиранием можно найти на сайте www.fairchildsemi.com фирмы Fairchild Semiconductor. На сайте www.amd.com фирмы Advanced Micro Devices имеется полная информация о более поздних программируемых ПЗУ со стиранием и о микросхемах флэш-памяти.

Среди сайтов, на которых перечислены все виды статических и динамических ОЗУ, как старые, так и новые, можно указать сайт semiconductor.hitachi.com фирмы Hitachi Semiconductor и сайт www.necel.com фирмы NEC Electronics. Самые последние и самые большие синхронные статические ОЗУ предлагают такие фирмы, как Integrated Device Technology (www.idt.com), Motorola Semiconductor (www.mot.com) и Micron Technology (www.micron.com).

В отраслевых публикациях типа *Электронные системы* (*Electronic Systems*, www.estd.com) и *Электронное проектирование* (*Electronic Design*) время от времени появляются материалы учебного характера и информация о новинках, где описываются возможности, предоставляемые самыми последними микросхемами памяти, и их применение; просто заходите на Web-сайт и начинайте с поиска "SDRAM". Там вы найдете архитектуры динамических ОЗУ, в том числе память типа Rambus DRAM (RDRAM), расширенное синхронное динамическое ОЗУ (ESDRAM), память типа SyncLink DRAM (SLDRAM) и синхронное динамическое ОЗУ с удвоенной скоростью обмена данными (DDR-DRAM).

Кроме рассмотренных в этой главе типов памяти, широкое применение нашли и другие «специализированные» запоминающие устройства. Наиболее распространенными, вероятно, являются *микросхемы памяти*, работающие по принципу «первым вошел – первым вышел» [*first-in, first-out (FIFO) memories*]; обычно их используют для передачи сигналов от одного узла обработки данных к другому или от устройств с одной тактовой частотой к устройствам с другой тактовой частотой. Web-сайты компаний IDT, Motorola и Texas Instruments (www.ti.com) являются хорошими источниками информации о схемах FIFO. Хотя может показаться, что с помощью FIFO магически решаются проблемы метастабильности, возникающие при переходе от блока с одной тактовой частотой к блоку с другой тактовой частотой, на самом деле это не так. Посмотрите, например, работу Джексона *Память FIFO: Решения, уменьшающие метастабильность в схемах FIFO* (Tom Jackson. *FIFO Memories: Solution to Reduce FIFO Metastability*. Texas Instruments publ. SCAA011A, March 1996).

Другим типом специализированной памяти является память с двумя портами, у которой имеются два независимых набора адресных входов, входов/выходов данных и шин управления и которая может одновременно выполнять независимые операции по обоим портам. Лидером по производству таких устройств является фирма Integrated Device Technology; кроме справочных дан-

ных, их Web-сайт содержит прекрасный набор рекомендаций по применению памяти с двумя портами.

Несколько производителей одновременно предложили различные архитектуры микросхем типа CPLD и FPGA. В то время как лидером в области разработки и производства ИС типа FPGA была и остается компания Xilinx (www.xilinx.com), пав разработкой ИС типа CPLD работают очень многие. Первые два семейства ИС типа CPLD были представлены в начале 90-х годов фирмами Altera Corporation (www.altera.com) и Advanced Micro Devices. Линия по производству ИС типа CPLD фирмы AMD теперь принадлежит ее бывшему конкуренту – компании Lattice Semiconductor (www.latticesemi.com). Другой важный игрок на рынке ИС типа CPLD – фирма Cypress Semiconductor (www.cypress.com). На Web-сайтах всех этих компаний имеются исчерпывающие справочные данные.

Упражнения

- 10.1. Определите емкость ПЗУ, необходимую для реализации той комбинационной логической функции, вычисление которой осуществляется схемами, изображенными на каждом из следующих рисунков: 4.39(b), 5.39, 5.77, 6.1 и 6.6.
- 10.2. Определите емкость ПЗУ, необходимую для реализации той комбинационной логической функции, вычисление которой осуществляется каждой из следующих интегральных схем средней степени интеграции: 74х49, 74х139, 74х153, 74х257, 74х381, 74х682.
- 10.3. Нарисуйте условное обозначение и определите емкость ПЗУ, реализующего комбинационную логическую функцию, которая вычисляется ИС 74х381 или 74х382 при различных значениях сигнала на входе MODE.
- 10.4. Нарисуйте условное обозначение и определите емкость ПЗУ, реализующего комбинационное умножение двух 8-разрядных чисел.
- 10.5. Покажите, как образовать статическое ОЗУ 2Мх8, используя в качестве исходных компонентов статические ОЗУ NM628512 и комбинационную схему средней степени интеграции.

Задачи

- 10.6. При рассмотрении скрытых путей в ПЗУ (рис. 10.6) мы утверждали, что при $A_2-A_0 = 101$ сигналы на линиях D2_L и D0_L переходят на низкий уровень из-за наличия прямых соединений. В действительности, это утверждение неверно, если только ИС 74х138 не заменена декодером, имеющим выходы с открытым коллектором. Объясните почему.
- 10.7. Опишите логическую функцию семи переменных, реализованную на основе ПЗУ 128х1 в соответствии с рис. 10.7. Один из способов описать логическую функцию состоит в том, чтобы, начиная со структуры ПЗУ, составить соответствующую таблицу истинности и записать каноничес-

- кую сумму. Но каноническая сумма содержит 64 терма-произведения с 7 переменными. Поэтому, возможно, вы захотите поискать простое, но точное словесное описание функции.
- 10.8. Для логической схемы с двумя выходами, представленной на рис. 4.39(б), сравните необходимое число диодов и транзисторов в матрицах И-ИЛИ и И с числом тех же элементов при реализации той же логической функции в массиве памяти на основе ПЛИМ, ПЛУ и ПЗУ.
 - 10.9. Покажите, как удвоить число уровней ослабления в цифровом аттенуаторе, схема которого приведена на рис. 10.17, не увеличивая емкость ПЗУ.
 - 10.10. Напишите на языке C функции `UlawToLinear` и `LinearToUlaw`, используемые в программе в табл. 10.6. В `LinearToUlaw` вам следует выбирать такой μ -ИКМ байт, теоретическое значение которого ближе всего к значению входного сигнала, представленному в линейном коде. (Указание: Наиболее эффективный подход заключается в том, чтобы при инициализации программы построить таблицу, состоящую из 256 элементов, которая в дальнейшем используется обеими функциями.) Обе функции должны выдавать сообщение о выходе значений входного сигнала за пределы допустимого диапазона значений.
 - 10.11. Измените представленную в табл. 10.6 программу на языке C так, чтобы ограничение производилось в том случае, когда заданный разработчиком коэффициент преобразования больше 1 и результат умножения входной величины на коэффициент преобразования выходит за пределы диапазона допустимых значений.
 - 10.12. Напишите программу на языке C для формирования такого содержимого ПЗУ 256×8 , с помощью которого 8-разрядный двоичный код преобразуется в 8-разрядный код Грея. (Указание: Ваша программа должна реализовать второй метод построения кода Грея, описанный в параграфе 2.11.)
 - 10.13. Напишите программу на языке C для формирования такого содержимого ПЗУ 256×8 , с помощью которого 8-разрядный код Грея преобразуется в 8-разрядный двоичный код. (Указание: Воспользуйтесь результатами задачи 10.12. Не важно, если ваша программа будет медленной.)
 - 10.14. Для последовательной передачи символов ASCII по каналу, в котором требуется, чтобы средний уровень сигнала равнялся нулю, была разработана некая система связи, в которой данные представляются кодом «5 из 10». Каждый 7-разрядный входной символ ASCII передается в виде 10-разрядного слова с пятью нулями и пятью единицами. Напишите программу на языке C для формирования такого содержимого ПЗУ 128×10 , с помощью которого символы ASCII преобразуются в кодовые слова.
 - 10.15. На приемном конце системы, описанной в задаче 10.14, каждое 10-разрядное кодовое слово должно быть снова преобразовано в 7-разрядный символ ASCII. Напишите программу на языке C для формирования такого со-

держимого ПЗУ $1K \times 8$, с помощью которого кодовые слова преобразуются в символы ASCII. Необходимо предусмотреть дополнительный бит па выходе — «флаг ошибки» — па тот случай, если принято не кодовое слово.

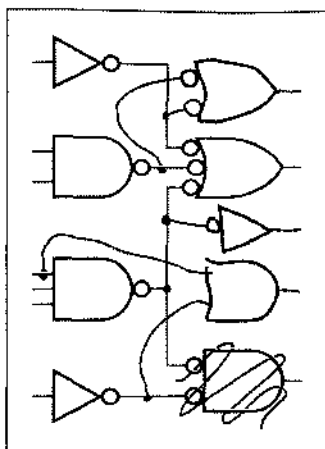
- 10.16.** Сколько битов должно храниться в ПЗУ, с помощью которого можно было бы выполнить суммирование/вычитание 16-разрядных чисел и имелся вход управления режимом работы, вход перепоса, выход переноса и выход переполнения при представлении чисел в двоичном дополнительном коде? Ответ должен быть более конкретный, чем «миллиарды и миллиарды», и обоснован.
- 10.17.** Повторите задачу 10.16 в предположении, что можно воспользоваться двумя ПЗУ, в результате чего задержка при сложении или вычитании 16-разрядных чисел возрастает вдвое по сравнению с задержкой, которая имеет место при использовании одного ПЗУ. Примите, что ПЗУ идентичны по емкости и одинаково запрограммированы. Попытайтесь минимизировать полную емкость ПЗУ и в общих чертах изобразите получающуюся в результате схему сложения/вычитания. Возможно ли дальнейшее уменьшение требуемой емкости ПЗУ, если допускается, что два используемых ПЗУ могут быть различными?
- 10.18.** Покажите, как воспользоваться микросхемой 2764 для реализации ПЗУ с организацией $64K \times 1$, применяя дополнительные микросхемы малой и средней степени интеграции. Каково время доступа для такого ПЗУ?
- 10.19.** Покажите, как воспользоваться микросхемой 2764 для реализации ПЗУ с организацией $2K \times 32$, применяя дополнительные микросхемы малой и средней степени интеграции. Вы можете предположить, что имеется независимый тактовый сигнал, период которого немного больше времени доступа микросхем 2764. Каково время доступа для полученного ПЗУ с организацией $2K \times 32$?
- 10.20.** Покажите, как образовать сумматор μ -ИКМ байтов, схема которого приведена на рис. 10.18, из ПЗУ $32K \times 8$ и двух вентилях ИСКЛЮЧАЮЩЕЕ ИЛИ. Напишите программу на языке C для формирования содержимого ПЗУ.
- 10.21.** Определите емкость ПЗУ, которая необходима для построения преобразователя чисел с фиксированной точкой в числа с плавающей точкой (рис. 6.3). Начертите принципиальную схему, используя одно из имеющихся в продаже ПЗУ.
- 10.22.** Напишите программу на языке C для формирования содержимого ПЗУ, используемого в задаче 10.21. В отличие от первоначального решения на основе схем средней степени интеграции, ваша программа должна выполнять округление; то есть для каждого числа с фиксированной точкой она должна вырабатывать ближайшее возможное число с плавающей точкой.
- 10.23.** Начертите полную принципиальную схему комбинационного умножителя на основе ПЗУ, который выполняет умножение двух 8-разрядных целых чисел без знака или целых чисел, представленных в двоичном дополн-

тельном коде. Выбор между режимом работы с числами без знака и режимом работы с числами со знаком должен осуществляться по отдельному входу SIGNED. Вы можете воспользоваться любым из приведенных на рис. 10.11 ПЗУ и, кроме того, отдельными вентилями.

- 10.24. Напишите программу на языке C для формирования содержимого ПЗУ в задаче 10.23 и проведите ее испытание.
- 10.25. Напишите программу на языке C для формирования содержимого ПЗУ 256Кх4, с помощью которого определялся бы следующий ход в игре в крестики-нолики, используя входное и выходное кодирование из раздела 6.2.7. Ваша программа должна быть достаточно «разумной», чтобы выбирать победный ход всякий раз, когда это возможно.
- 10.26. Повторите задачу 10.25, воспользовавшись ПЗУ 32Кх4. Чтобы реализовать это, состояние игрового поля необходимо кодировать лишь 15 битами. Объясните свой алгоритм кодирования и напишите на языке C функции для преобразования номера ячейки при вашем способе кодирования в номер ячейки при кодировании, указанном в разделе 6.2.7, и наоборот.
- 10.27. Для каждого из временных параметров, введенных в разделе 10.3.3, определите, действительно ли у статического ОЗУ 2Мх8, построенного вами в упражнении 10.5, их значения те же, что и у ИС HM628512. Если они различны, то укажите новые значения. Для учета задержек, вносимых микросхемами средней степени интеграции, воспользуйтесь значениями для наихудшего случая из столбца табл. 5.3 для ИС 74FCT.
- 10.28. Используя в качестве стандартных блоков статическое ОЗУ HM6264 8Кх8, несколько ИС средней степени интеграции и ПЛУ, постройте синхронное статическое ОЗУ 8Кх8 с задержанной записью и сквозными выходами.
- 10.29. Используя в качестве стандартных блоков статическое ОЗУ HM6264 8Кх8, несколько ИС средней степени интеграции и ПЛУ, постройте память типа SSRAM, имеющую шину с нулевым временем смены режима, то есть память типа ZBT SSRAM, емкостью 8Кх8 с конвейерными выходами.
- 10.30. Определите соответствующие временные параметры для синхронного статического ОЗУ с задержанной записью и сквозными выходами.
- 10.31. Вычислите значения тех же временных параметров, что и в задаче 10.30, для схемы, построенной в задаче 10.28.
- 10.32. Подобно тому, как это сделано на рис. 10.27, нарисуйте временные диаграммы для синхронного статического ОЗУ с задержанной записью и сквозными выходами для последовательности чередующихся циклов чтения и записи вида R-R-W-W-R-W-R-W. Разместите отдельные циклы как можно ближе друг к другу, но учтите конфликты на уровне ресурсов, не позволяющие выполнять циклы вплотную. Каков средний ко-

эффект использования матрицы статического ОЗУ, когда синхронное статическое ОЗУ работает с непрерывным потоком запросов вида R-W-R-W-R-W?

- 10.33. Повторите предыдущую задачу для синхронного статического ОЗУ с задержанной записью и конвейерными выходами.
- 10.34. Воспользовавшись одной из теорем, приведенных в параграфе 4.1, докажите, что с помощью логических блоков ИС серии XC4000 фирмы Xilinx можно реализовать любую логическую функцию пяти переменных.
- 10.35. Покажите, как воспользоваться схемами, вырабатывающими значения логических функций в логических блоках ИС серии XC4000, для реализации 9-разрядной схемы проверки на четность.
- 10.36. Покажите, как использовать формирователь логических функций в логических блоках ИС серии XC4000 для реализации устройства, проверяющего равенство двух 4-разрядных операндов. Убедитесь, что последовательное включение построенных вами устройств, позволяет проверять равенство 4*n*-разрядных операндов, используя всего лишь *n* логических блоков.



Глава

ПРАКТИЧЕСКИЕ ДОПОЛНЕНИЯ

В эту главу включен ряд «практических» вопросов, которые могут представлять определенный интерес для тех, кто собирается стать разработчиком цифровых устройств или уже работает в этой области. Мы лишь поверхностно коснемся каждой из этих тем, поэтому за подробностями нужно обратиться к литературе, специально посвященной затронутым вопросам.

11.1. Средства автоматизированного проектирования

*Если бы аппаратные средства не были такими «тяжелыми»,
их не называли бы «железом»
("If it wasn't hard, they wouldn't call it hardware")*

Многие разработчики цифровых устройств с двадцатилетним опытом считают это утверждение бесспорным. Тем не менее, все чаще цифровое проектирование выполняется с использованием программных средств, благодаря которым решение задачи упрощается.

Термины *автоматизированное проектирование* (computer-aided design, CAD) и *автоматизированное конструирование* (computer-aided engineering, CAE) применяются по отношению к совокупности программ, которые поддерживают разработку схем, систем и многого другого. Термин "CAD" является более общим и используется не только в области электроники, но также в архитектуре, например, и в машиностроении. В электронике понятие "CAD" часто относится к средствам *проектирования устройств на физическом уровне*, таким как программы компоновки ИС и разводки печатных плат. Термин "CAE" чаще используется по отношению к средствам проектирования на *конструктивном уровне* типа редакторов схем, программ моделирования схем и компиляторов ПЛИУ. Однако многие из работающих в области электроники (включая авторов) склоняются к тому, что эти термины являются синонимами. В этом параграфе мы рассмотрим некоторые средства CAD/CAE, используемые при разработке цифровых устройств.

ЯВЛЯЕТСЯ ЛИ ТЕПЕРЬ РАЗРАБОТКА АППАРАТНЫХ СРЕДСТВ ПРОСТЫМ ДЕЛОМ?

Поскольку все чаще аппаратные средства разрабатывают и отлаживают с помощью программных средств, действительно ли их создание становится проще? Согласно опыту автора – не обязательно.

Если раньше разработчикам приходилось тратить время на борьбу с паяльниками и пружинными зажимами, то теперь их время уходит на борьбу с ошибками в программах, работающих в программной среде, содержащей ошибки.

11.1.1. Языки описания схем

В предыдущие десятилетия большинство проектов логических устройств выполнялось графически в виде блок-схем и принципиальных схем. Однако широкое распространение в 90-е годы программируемых логических устройств, технологии создания сверхбольших специализированных ИС, языков описания схем и синтеза радикально изменило методы разработки больших цифровых устройств.

Традиционное использование языков высокого уровня, таких как C, C++ и Java, при создании программного обеспечения, настолько повысило уровень абстракции, что теперь программисты способны создавать более крупные и более сложные системы, правда с некоторой потерей эффективности по сравнению с программными продуктами, написанными на ассемблере и отлаженными вручную. Ситуация с проектированием аппаратных средств аналогична. Схема, описанная с помощью средств синтеза в языках VHDL или Verilog, не может быть такой же маленькой по размерам и столь же быстродействующей, как схема разработанная и вылизанная вручную опытным инженером, но в хороших руках программные средства позволяют создавать гораздо более крупные системы. Без применения программных средств нельзя обойтись, когда речь идет о том, чтобы воспользоваться наличием миллионов вентилях в наиболее совершенных интегральных схемах типа CPLD и FPGA или в специализированных ИС.

Некоторое время в ходу были языки описания схем, не поддерживаемые средствами синтеза, но с появлением таких средств использование этих языков уменьшилось. Наиболее известны языки *межрегистровых пересылок* (*register-transfer languages*), которыми в течение десятилетий пользовались для описания работы синхронных систем. В таком языке система обозначений потокового управления поведением конечного автомата объединяется со средствами описания работы схем на многоразрядных регистрах. Языки межрегистровых пересылок были особенно полезны при машинном проектировании, когда отдельные команды машинного языка представлялись в виде последовательности таких элементарных шагов, как загрузка, хранение, объединение и проверка содержимого регистров.

11.1.2. Ввод схемы

Если не считать домашних заготовок, то ваш первый шаг при проектировании схемы состоит, как правило, в том, чтобы убедить кого-то, что предложенный

вами подход правильный. Это значит, что вам надо подготовить блок-схему и слайды для презентации и обсудить ваши идеи с менеджерами и коллегами с целью предварительной *оценки проекта (design review)*. После того, как ваш проект одобрен, можно, не рискуя, приступить к «функциональному наполнению», то есть рисовать схему.

Раньше, как правило, схемы рисовались вручную, но теперь при подготовке все больше используют *схемные редакторы (schematic editors)* – программы системы CAD, запускаемые на автоматизированном рабочем месте (АРМ) разработчика. Процесс создания принципиальной схемы на компьютере часто называют *вводом схемы (schematic capture)*. Этот термин используется потому, что схемный редактор фиксирует больше информации, чем просто рисунок. Основная информация, содержащаяся в схеме, представлена в виде «надписей» на ней, позволяющих позже, по мере необходимости, извлекать нужную в процессе проектирования информацию автоматически.

Чтобы упростить ввод информации и ее извлечение, в схемном редакторе для каждого типа информации обычно предусмотрены *поля (fields)*. Информация может быть либо скрыта, либо автоматически выводиться рядом с элементом схемы, к которому она относится. Вот некоторые типичные поля и их назначение:

- *Тип компонента (component type)*. Задавая тип компонента (например, резистор, конденсатор, ИС 74FCT374 или ИС GAL16V8), разработчик может вызывать символическое изображение этого элемента из *библиотеки компонентов (component library)*. У некоторых компонентов есть несколько вариантов изображения, из которых можно выбирать (например, эквивалентные представления вентилей согласно теореме Де Моргана). Тип компонента используется в документации и при моделировании.
- *Параметр компонента (component value)*. Для большинства аналоговых компонентов в этом поле должен быть задан параметр (например: 2.7 кОм). В полях «номинальная мощность» и «допуск» могут быть приведены дополнительные сведения (например: 1/4 Вт, 1%). Для цифрового компонента в таком поле может быть указано его быстродействие (например: –10 для 10-наносекундного ПЛУ).

ЧТО ТАКОЕ «АРМ»?

В настоящее время «автоматизированное рабочее место» на основе персонального компьютера может стоить порядка 500 долларов, но при использовании параллельных процессоров его стоимость может возрасти до 50000 долларов. «Образцом» автоматизированного рабочего места с точки зрения его стоимости (то есть суммы, которую типичная компания может потратить на одного инженера) и эффективности (из числа лучших) является рабочая станция фирмы Sun. В 2001 году за 5000 долларов можно было получить любую модель такой рабочей станции.

- *Шифр компонента, разрешенного к применению (approved-part number)*. Используя информацию о типе компонента и о его параметрах, программа CAD может автоматически выбрать шифр компонента из представленного компанией «списка компонентов, разрешенных к применению» (например: 126-10117-0272 для металлопленочного резистора с параметрами 2.7 кОм, 1/4 Вт, 1%) или выдать предупреждение, если такого компонента нет. [Работая в большой компании вы, безусловно, можете воспользоваться только таким компонентом из каталога, который прошел проверку на соответствие техническим условиям (parts qualification).]
- *Позиционное обозначение (reference designator)*. Этой алфавитно-цифровой меткой указывается вид компонента и его порядковый номер в схеме.
- *Расположение компонента (component location)*. В этом поле можно указать положение компонента на печатной плате в той или иной системе координат, благодаря чему упрощается задача поиска компонента в процессе отладки. Конечно, на начальной стадии проектирования окончательное положение компонента обычно не известно. Однако программное обеспечение системы CAD, способное осуществлять обратную аннотацию (back annotation), может вставить эту информацию в схему, когда разводка печатной платы закончена.
- *Номера выводов (pin numbers)*. Номерами выводов определяется назначение выводов каждого компонента. Обычно они заранее введены в условное обозначение компонента, который вызывается из библиотеки компонентов. Однако для микросхем малой степени интеграции и для других компонентов с несколькими идентичными элементами в одном корпусе номера выводов (и даже их позиционное обозначение) не могут быть определены до тех пор, пока не закончена разводка печатной платы. Это еще один случай, когда удобна обратная аннотация.
- *Тип соединения (wire type)*. Как правило, при рисовании схем используются соединения только двух типов: «обычное» и «шина», где шина представляет собой совокупность обычных соединений, которые объединены с целью компактного изображения. Однако в некоторых системах могут понадобиться другие типы соединений, например, сверхширокие шины на печатной плате, по которым передаются аналоговые сигналы и по которым течет большой ток. Такого рода информация сообщается программе разводки печатных плат.
- *Имя сигнала (signal name)*. Пользователь может дать имя каждому сигналу, и это следует сделать. Наличие у сигнала имени особенно полезно при моделировании и отладке, но оно не оказывает влияния на разводку печатной платы.
- *Флажок связи (connection flag)*. Этот элемент рисунка указывает на соединение одной сигнальной линии с другой на той же странице или на другой. Программа может проверить, все ли исходящие соединения согласуются с входящими. При иерархической организации схемы программа может использовать флажки связи для условного представления целой страницы в виде одного «блока» с соответствующими именами сигналов.

Когда принципиальная схема «введена» в систему CAD, вы можете ее распечатать, но кроме этого у вас появляется масса других возможностей. Для изготовления устройства можно создать, по крайней мере, два документа:

- *Список компонентов (parts list)*. В нем содержится перечень компонентов и их позиционное обозначение.
- *Список соединений (net list)*. *Соединение (net)* представляет собой набор выводов, объединенных в один электрический узел или одним сигналом. Список соединений содержит все такие связи, обычно в алфавитном порядке по именам сигналов, в соответствии с принципиальной схемой. Для каждого имени сигнала в этом списке перечисляются выводы компонентов (позиционное обозначение компонента и номер вывода), к которым относится этот сигнал.

Кроме того, список соединений можно сортировать по позиционным обозначениям компонентов и по номерам их выводов с указанием имени сигнала на каждом выводе; иногда такой список называют *списком выводов (pin list)*.

Список компонентов и список соединений являются основными входными данными для процедуры разводки печатной платы. Разработчик может использовать список компонентов также для оценки стоимости схемы и надежности ее работы, а отдел снабжения, очевидно, может воспользоваться им для заказа компонентов.

11.1.3. Временные диаграммы и временные параметры

Блок-схемы и принципиальные схемы не единственные графические документы, используемые при разработке цифровой системы. Существенную часть почти любого пакета документации составляют временные диаграммы.

Большие системы разбиваются на подсистемы меньших размеров, соединенные одна с другой четко определенными интерфейсами. Определение интерфейса обычно содержит не только имена сигналов и выполняемые ими функции, но также и параметры ожидаемого поведения системы во времени. Отправной точкой обычно служат технические требования, накладываемые на максимальное, а иногда и на минимальное значение частоты тактового сигнала. Входы подсистемы предъявляют свои требования в отношении необходимого времени установления и времени удержания сигналов относительно фронта тактового сигнала. Выходные сигналы подсистемы характеризуются своими минимальными и максимальными задержками относительно фронта тактового сигнала.

Программы типа TimingDesigner (www.chronology.com) могут автоматизировать утомительную задачу вычерчивания сложных временных диаграмм и определения временных характеристик, когда изменение одного из временных параметров может повлиять на значения многих других.

11.1.4. Анализ схемы и моделирование

В библиотеке компонентов сложной системы CAD имеются не только условные обозначения каждого компонента. Библиотека ИС может содержать *модель компонента (component model)*, описывающую логическое и электрическое функционирование интегральной схемы. Языки описания схем типа VHDL и Verilog первоначально предназначались исключительно для моделирования компонентов и собранных из них систем. Такое моделирование позволяет находить логические и временные ошибки.

В модели ИС указывается, как минимум, является данный вывод входом или выходом. При наличии только этой информации программа проверки правильности схемы (*design-rule checker*) может обнаружить некоторые из наиболее распространенных «глупых ошибок» в проекте типа замкнутых между собой выходов и плавающих входов. Если модель содержит параметры, характеризующие нагружающее действие каждого входа и нагрузочную способность каждого выхода, то программа проверки может также определить, не превышен ли где-либо в схеме коэффициент разветвления по выходу.

Следующим шагом является проверка временных соотношений (*timing verification*). Даже в отсутствие детальной модели поведения логической ИС, в библиотеке компонентов может быть указана величина задержки в наихудшем случае для каждого пути от входа до выхода, а также время установления и время удержания для синхронных устройств. Используя эту информацию, верификатор временных соотношений (*timing verifier*) находит в схеме пути с наихудшими задержками, благодаря чему разработчик может определить, укладываются ли задержки в заданные временные границы.

Наконец, библиотека может содержать детальную модель каждого логического компонента; в этом случае моделирование (*simulation*) позволяет предсказать поведение схемы в целом при любой заданной последовательности входных сигналов. Разработчик задает входную последовательность, а моделирующая программа (*simulator*) определяет, как схема будет реагировать на эту последовательность. Результат работы моделирующей программы обычно отображается графически в виде временных диаграмм, которые разработчик мог бы видеть на экране осциллографа или логического анализатора, если бы те же самые сигналы были поданы на входы реальной схемы. В таком режиме можно отладить всю схему без «макетирования», собрать ее на печатной плате, и она заработает с первой попытки.

В моделирующей программе для разных типов компонентов используются различные модели. В программах типа SPICE, предназначенных для моделирования аналоговых устройств, используются математические модели, в которых резистор описывается всего лишь одним параметром, а для описания биполярного транзистора требуется от одного до нескольких десятков параметров. На основе этих моделей и в соответствии со структурой схемы моделирующая программа составляет и решает уравнения, описывающие поведение схемы.

При цифровом моделировании обычно используется одна из двух следующих моделей каждого логического элемента:

- *Поведенческая модель (behavioral model)*. Согласно этой модели программа «понимает» основную функцию, выполняемую логическим элементом. Например, вентиль И понимается как устройство, вырабатывающее на выходе логическую 1 с задержкой t_{PLH} после того, как логические единицы устанавливаются на всех его входах, и вырабатывающее на выходе логический 0 с задержкой t_{PHL} после того, как логический 0 подан на любой из его входов.
- *Структурная модель (structural model)*. Более крупные компоненты можно моделировать, объединяя в группы элементы меньших размеров, описываемые поведенческими моделями.

Мы говорили, что язык VHDL поддерживает оба типа моделей. Например, в табл. 5.15 приведена поведенческая модель дешифратора 3×8 типа 74х138. Одним предложением языка VHDL (оператором “after” или с помощью других механизмов, не рассматриваемых в этой книге) мы можем задать задержку от входа до выхода. В табл. 5.14, в качестве альтернативы, приведена структурная модель дешифратора 2×4 в виде набора вентилях, соединенных согласно принципиальной схеме на рис. 5.32. Если имеется информация о временных параметрах отдельных схем в такой структуре (inv и and3), то можно точнее предсказать временные параметры структурной модели больших размеров, поскольку есть возможность проследить реальный путь, прохождения каждого входного сигнала при переходе с низкого уровня на высокий или в обратном направлении.

Обе упомянутые модели называются *программными моделями (software models)*, поскольку моделирование работы микросхемы выполняется исключительно программными средствами. Для некоторых микросхем, в том числе для микропроцессоров и других БИС, требуются огромные программные модели. Во многих случаях изготовители микросхем не предоставляют таких моделей (считается, что они являются частной собственностью изготовителя), и пользователю пришлось бы потратить месяцы или годы для их создания. Чтобы обойти эту проблему, разработчиками современных систем САД был введен третий тип модели:

- *Физическая модель (physical model)*, которую иногда называют *аппаратной моделью (hardware model)*. Реальный, работающий экземпляр микросхемы подключается к компьютеру, который осуществляет моделирование. Сигналы, вырабатываемые моделирующей программой, подаются на входы реальной интегральной схемы, наблюдаются выходные сигналы, которые затем используются в качестве входных сигналов при моделировании работы других компонентов.

Входной информацией, вводимой разработчиком цифровой аппаратуры при логическом моделировании, являются принципиальная схема и последовательность входных векторов, подаваемых на схему в процессе моделирования. Как правило, первое, что видит разработчик, — “ххххх”; это означает, что моделирующая программа не может выяснить, какой будет ответ. Происходит это обычно потому, что в устройстве имеются последовательностные схемы (триггеры и защелки), которые не установлены в определенное начальное состояние. Результат моделирования имеет физический смысл только в том случае, когда все схемы явным образом установлены в известное состояние, даже если для правильной работы реальной системы начальная установка не требуется. Например, нельзя смоделировать приведенные на рис. 8.62 и 8.64 самокорректирующиеся кольцевые счетчики, потому что у моделирующей программы нет никакой возможности узнать их начальные состояния.

Когда моделируемая схема установлена в известное начальное состояние, возможности разработчика проверить ее работу ограничены только быстродействием моделирующей программы. Моделирование схем, описываемых комплексными числами, может быть очень медленным по сравнению с моделированием, в котором используются только действительные числа. В зависимости от уровня детализации, время моделирования может превосходить длительность моделируе-

мого процесса в 10^3 – 10^8 раз. Самым быстрым является строго функциональное моделирование, когда предполагается, что все компоненты имеют нулевую задержку. При гораздо более реальном моделировании вычисляются и учитываются задержки для всех путей прохождения сигналов в наихудшем случае, но такое моделирование является самым медленным.

11.1.5. Разработка печатной платы

Разработка печатной платы может быть выполнена с помощью программы или поручена специалисту по разработке печатных плат, либо путем комбинации того и другого. Тем, кто занимается разработкой печатных плат, обычно нравится решать головоломки. Их работа состоит в размещении всех компонентов схемы на плате заданного размера и последующем соединении их между собой согласно принципиальной схеме. Как правило, требуется, чтобы размер печатной платы и число слоев в ней были по возможности меньшими.

С помощью программы разработку печатной платы можно осуществить автоматически, но результаты не всегда получаются хорошими. Как специалисту по печатным платам, так и разработчику схемы, уже на первом этапе размещения компонентов приходится принимать во внимание массу практических соображений:

- *Механические ограничения (mechanical constraints)*. Некоторые разъемы, индикаторы и другие компоненты, вероятно, придется поместить на печатной плате в определенных местах, заданных техническими условиями.
- *Критические пути прохождения сигналов (critical signal paths)*. Автоматическая программа размещения может расположить компоненты так, чтобы минимизировать среднюю длину соединений, но разработчик обычно руководствуется интуитивным чутьем в отношении того, какие части схемы и какие конкретно сигналы наиболее критичны в отношении длины соединений. Разработчик схемы обычно предоставляет специалисту по печатным платам *общую топологическую структуру (floorplan)*, то есть предлагает размещение групп компонентов, в котором отражены идеи разработчика схемы относительно того, какие компоненты необходимо разместить близко друг к другу.
- *Температурные проблемы (thermal concerns)*. Когда плата установлена в систему, некоторые ее участки могут охлаждаться лучше, чем другие, и, естественно, какие-то компоненты при работе нагреваются сильнее других. Разработчик должен гарантировать, что температура компонентов не выйдет за пределы рабочего диапазона температур.

Температура в любом месте правильно разработанной печатной платы при надлежащей системе охлаждения обычно не превышает температуру окружающей среды более чем на 10°C , но в случае микросхемы, потребляющей значительную мощность (примером может служить микропроцессор) и помещенной там, где нет воздушного потока, ее температура может превышать температуру окружающей среды на 30°C и больше. Если температура окружающего воздуха была 40°C , то температура микропроцессора составит 70°C , что является предельным значением для большинства микросхем гражданского применения.

- *Радиочастотные излучения (radio-frequency emissions)*. Побочным эффектом при работе электронного оборудования является радиочастотное излучение. Правительственными органами США и других стран определяются максимально допустимые уровни излучения; в США допустимые уровни указаны в Разделе 15 иорм, установленных *Федеральной комиссией связи (FCC Part 15)*.

Уровни излучения схемы обычно зависят не только от характеристик компонентов и конфигурации схемы, но и от того, как разведена печатная платы. Поэтому при разводке платы особое внимание необходимо уделять размещению компонентов, заземлению, изоляции и развязке, с тем чтобы минимизировать нежелательные (и недопустимые по закону!) радиочастотные излучения.

- *Глупые ошибки (stupid mistakes)*. Вероятно, наиболее важным шагом при проектировании печатной платы, особенно при использовании новых или незнакомых компонентов, является проверка наличия глупых ошибок. Почти каждый разработчик цифровых устройств, даже преуспевающий, может рассказать вам много историй о разводке печатных плат, когда использовалось зеркальное представление выводов нового компонента, монтажные отверстия и разъемы оказывались не там, где надо, имелся в виду не тот корпус ИС, был предусмотрен штыревой разъем вместо гнездового или наоборот и не учитывались внутренние слои разводки, причем обнаруживалось все это, когда плата уже была изготовлена. Помните: «Компьютеры не делают ошибок, их делают люди!»

После размещения компонентов необходимо соединить их между собой; эта операция называется *разводкой (routing)*. Эта фаза разработки печатной платы легче всего автоматизируется; приличная программа автоматической разводки может очень быстро развести печатную плату умеренного размера. Однако разработчикам, вероятно, придется обратить особое внимание на критические пути прохождения сигналов и на такие вопросы, как включение оконечных нагрузок в линиях передачи сигналов и размещение контрольных точек. Указанные вопросы обсуждаются далее в этой главе.

11.2. Проектирование, предусматривающее тестируемость

Когда вы покупаете новое устройство, то ожидаете, что оно будет правильно работать «прямо из коробки». Однако даже при наличии современного автоматизированного оборудования производитель не может гарантировать, что каждое произведенное им изделие будет безупречным. Некоторые устройства не работают потому, что содержат отдельные дефектные компоненты, либо потому, что были неправильно или небрежно собраны, либо потому, что были повреждены при транспортировке. Для сохранения репутации большинство производителей предпочитает выявлять эти проблемы у себя, а не отправлять заказчикам неисправные изделия. Эта цель достигается с помощью *тестирования (testing)*.

Основным видом испытаний является *тест «годен — не годен» (go/no-go test)*, выходом которого является всего лишь один бит информации: система работает правильно на все 100% или нет? Если ответ положительный, то систему можно отправлять. Если нет, то следующее действие зависит от размера системы. Под-

ходящей реакцией в случае цифровых часов было бы выбросить их. Если обнаруживается, что неисправных часов много, то целесообразнее исправить то, что, вероятнее всего, является слабым звеном в процессе сборки или недостатком того или иного компонента, а не пытаться спасти отдельные экземпляры.

С другой стороны, вам не захочется выбрасывать компьютер стоимостью 10000 долларов, который не загружается. Вместо этого для обнаружения конкретной неисправной подсистемы можно провести более детальное *диагностическое тестирование* (*diagnostic test*). В зависимости от стоимости, неисправную подсистему можно восстановить или заменить. Типичной цифровой подсистемой является печатная плата, которая вместе с компонентами стоит от 50 до 1000 долларов. Решение о ремонте или замене представляет собой экономический компромисс между стоимостью собранной печатной платы и предполагаемым временем, необходимым для обнаружения и ремонта неисправного узла.

Таким образом, возникает необходимость в *проектировании, предусматривающем возможность тестирования* (*design for testability, DFT*). «DFT» является общим термином, применяемым к методам проектирования, обеспечивающим возможность более полного и менее дорогостоящего тестирования. Значительный выигрыш можно получить в том случае, если проектирование системы или подсистемы выполнено так, что отказы легко выявить и локализовать:

- Более правдоподобным становится результат тестирования по принципу «годен – не годен». Чем меньше отправленных устройств будет иметь скрытые дефекты, тем меньшее число клиентов будет огорчено, что дает в результате очевидные экономические и психологические выгоды.
- Диагностические тесты выполняются быстрее и дают более точные результаты. Они уменьшают стоимость выявления подсистемы, которая не проходит тест «годен – не годен», позволяя произвести больше изделий за меньшую цену.
- Оба теста – «годен – не годен» и диагностический – требуют меньших затрат времени на проведение тестирования.
- Хотя экономия во времени тестирования может потребовать дополнительных усилий при проектировании по принципу DFT, увеличение полий стоимости разработки изделия почти всегда компенсируется меньшей стоимостью производства.

11.2.1. Тестирование

Тестирование цифровых схем осуществляется с помощью *проверочных векторов* (*test vectors*), представляющих собой комбинации входных сигналов и ожидаемые комбинации выходных сигналов. Схема «проходит проверку», если выходные сигналы соответствуют ожидаемым сигналам. В худшем случае для тестирования комбинационной схемы с n входами требуется 2^n проверочных векторов. Но если мы кое-что знаем о том, как реализована схема, и делаем некоторые предположения относительно типа возможных отказов, то число векторов, необходимых для полной проверки схемы, можно значительно сократить. Наиболее общее предположение состоит в том, что отказы носят характер *одиночных заливаний* (*single stuck-at faults*), то есть их можно смоделировать в виде «залипания» на уровне логического 0 или логической 1 одного входного или вы-

ходного сигнала. Согласно этому предположению, 8-входовой вентиль И-НЕ можно полностью проверить лишь девятью векторами: 11111111, 01111111, 10111111, ..., 11111110, тогда как в общем случае для его тестирования могло бы потребоваться 256 проверочных векторов.

Для отдельных логических элементов легко составить проверочные векторы, если предполагать наличие одиочисных неисправностей. Однако проблема состоит в том, что на практике для тестирования логических элементов, скрытых глубоко в схеме, проверочные векторы *подаются на входы схемы*, а результаты *наблюдаются на ее выходах*. Предположим, например, что мы хотим проверить 8-входовой вентиль И-НЕ, между входами которого и внешними входами схемы имеется десяток комбинационных и последовательностных логических элементов. Совсем не очевидно, какой входной вектор или последовательность входных векторов необходимо подать на внешние входы, чтобы получить проверочный вектор 11111111 на входах вентиль И-НЕ. Кроме того, не ясно, что еще может потребоваться для передачи выходного сигнала вентиль И-НЕ к внешнему выходу схемы.

В таких сложных случаях изощренные программы генерирования тестов (*test-generation programs*) пытаются создать полный набор тестов (*complete test set*) для данной схемы, то есть последовательность тестовых конфигураций, которые полностью проверяют каждый логический элемент в схеме. Однако часто требуемый объем вычислений оказывается при этом столь огромным, что полный набор тестов просто невозможно получить.

Генерирование тестовых конфигураций пытаются упростить путем обеспечения в схеме большей «управляемости» и «наблюдаемости» отдельных логических элементов. В схеме с хорошей *управляемостью* (*controllability*) легко создать любые желаемые значения сигналов во внутренних точках схемы, подавая на внешние входы комбинацию входных сигналов, соответствующую тому или иному проверочному вектору. Говоря о хорошей *наблюдаемости* (*observability*), имеют в виду, что любой внутренний сигнал можно легко передать на внешний выход для сравнения с ожидаемым значением при подаче соответствующей комбинации сигналов на внешние входы. Наиболее общий метод улучшения управляемости и наблюдаемости состоит во введении *контрольных точек* (*test points*) и дополнительных внешних входов и выходов, которые используются в процессе тестирования.

11.2.2. Тестер с игольчатыми контактами и внутрисхемное тестирование

В цифровой схеме, собранной на одной печатной плате, «максимальная» наблюдаемость достигается, когда в качестве контрольных точек используются все выводы всех ИС. Для этой цели применяется специальное *тестирующее приспособление* (*test fixture*), в котором имеются подпружиненные *игольчатые контакты* (*nails*) на месте каждого вывода ИС в соответствии с разводкой печатной платы. Печатная плата помещается на это *ложе из игольчатых контактов* (*bed of nails*), а контакты соединены с *автоматическим тестером* (*automatic tester*), который может наблюдать сигнал на каждом выводе согласно тестовой программе.

Двигаясь еще на один шаг дальше, — применяя *внутрисхемное тестирование* (*in-circuit testing*), — мы получаем «предельно возможную» управляемость.

Этот метод позволяет не только наблюдать сигналы на игольчатых контактах, но, также подключать каждый игольчатый контакт к имеющемуся в тестере источнику с очень малым выходным сопротивлением. Благодаря этому можно *подменять* (*override*) любой сигнал, вырабатываемый в схеме, [*принудительно задавать* (*overdrive*) его значение] и, тем самым, непосредственно генерировать любой желаемый проверочный вектор в виде внутренних сигналов печатной платы. Принудительное задание сигнала на выходе вентиля, вырабатывающего противоположное значение сигнала, вызывает протекание избыточного тока как в тестере, так и в преодолеваемом вентиле, но неприятностей удастся избежать благодаря тому, что тестер выдает сигналы в течение коротких интервалов времени (миллисекунды).

Чтобы проверить 8-входовый вентиль И-НЕ, внутрисхемный тестер должен сформировать только девять тестовых векторов, упомянутых выше, игнорируя значения сигналов, которые пытаются подать на эти восемь входов другие вентили внутри схемы. Выходной сигнал вентиля И-НЕ, естественно, можно непосредственно наблюдать на его выходе. При внутрисхемном тестировании каждый логический элемент можно проверить независимо от других.

Хотя внутрисхемное тестирование значительно расширяет управляемость и наблюдаемость схемы, собранной на печатной плате, разработчики логических схем для большей эффективности по-прежнему должны следовать определенным принципам DFT. Часть из них перечислена ниже.

- *Инициализация.* Необходимо предусмотреть возможность устанавливать в известное начальное состояние все последовательностные элементы схемы.

СОВМЕЩЕНИЕ ИГОЛОК С ВЫВОДАМИ МИКРОСХЕМ

Возрастающее применение микросхем в корпусах для поверхностного монтажа с малыми расстояниями между выводами привело к тому, что тестирование с помощью игольчатых контактов значительно усложнилось по сравнению с тестированием схем в DIP-корпусах, монтируемых на печатной плате в сквозные отверстия. Поскольку компоненты могут устанавливаться с обеих сторон печатной платы, для тестирования может понадобиться специальное приспособление, называемое *грейфером* (*clam shell*), для подключения игольчатых контактов с обеих сторон печатной платы.

Более того, выводы и расстояния между ними у многих микросхем, предназначенных для поверхностного монтажа, настолько малы (0.625 мм и меньше), что может оказаться невозможным точно попасть измерительным щупом на соответствующий вывод. В этих случаях разработчик печатной платы может предусмотреть специальные контактные площадки в виде дополнительных участков, покрытых медью, имеющих размеры, достаточные для подключения измерительных щупов (например, диаметром 1.25 мм). Отдельную контактную площадку нужно обеспечить для каждого сигнала, который не подведен где-нибудь на печатной плате к выводу компонента больших размеров (например, к выводу компонента, вставляемого в сквозное отверстие диаметром 1.55 мм).

Поскольку при внутрисхемном тестировании имеется возможность подать на входы регистров и триггеров сигналы установки в единичное состояние и сброса, можно подумать, что никакой проблемы не существует. Однако, на рис. 11.1(а) показан классический случай, когда схема (счетчик, считающий в коде Грея) не может быть инициализирована, так как состояние триггера не предсказуемо, когда сигналы на входах PR и CLR одновременно переходят на неактивный уровень. На рис. 11.1(б) показано, как правильно решается проблема установки в единичное состояние и сброса

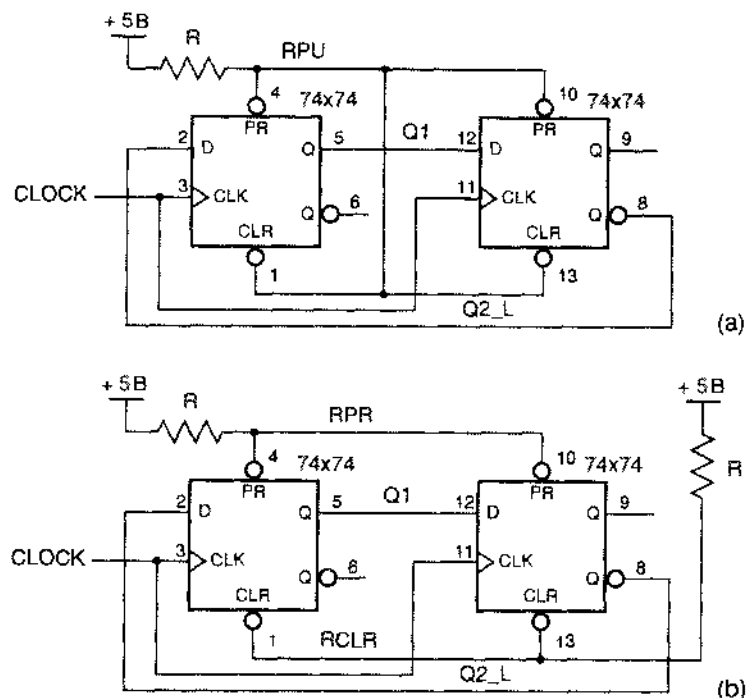


Рис. 11.1. Триггеры с резисторами, подключенными между шиной питания и неиспользуемыми входами: (а) не тестируемая схема, (б) тестируемая схема

- *Генерирование тактового сигнала.* Тестер должен иметь возможность вырабатывать свой собственный тактовый сигнал, не задевая тактовых сигналов, имеющих на плате.

Обычно по нескольким причинам требуется, чтобы тестер подменял тактовый сигнал, имеющийся на плате: скорость, с которой могут подаваться проверочные векторы, ограничена; необходимо дополнительное время, чтобы принудительно задаваемые сигналы приняли установившиеся значения; а иногда необходимо остановить тактовый сигнал. Однако принудительное задание тактового сигнала совершенно недопустимо. В принудительно задаваемом сигнале может быть «звон», и такой сигнал может несколько раз перейти с низкого уровня на высокий и обратно перед тем, как примет установившееся значение, определяемое тестером.

Такие переходы в тактовом сигнале могут приводить к нежелательным изменениям состояния.

На рис. 11.2 показана рекомендуемая схема источника тактового сигнала. Чтобы ввести собственный тактовый сигнал, тестер устанавливает на входе CLKEN низкий уровень и подает свой тактовый сигнал на вход TESTCLK_L. Поскольку никакие выходы вентилях тестером не преодолеваются, мы получаем чистый сигнал CLOCK. Вообще, из-за недопустимости появления паразитных выбросов в сигнале, который используется в качестве сигнала на тактовом входе или на каком-либо асинхронном входе, тестер не должен принудительно задавать такой сигнал; с этими сигналами следует обращаться так, как показано на рис. 11.2. Это еще одна причина, по которой желательно, чтобы в синхронной системе был один тактовый сигнал.

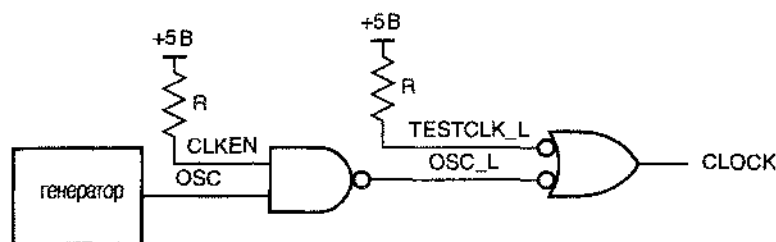


Рис. 11.2. Схема источника тактового сигнала, позволяющая тестеру аккуратно подменить системный тактовый сигнал

- *Заземленные входы.* В общем случае непосредственное заземление не следует использовать в качестве источника логического 0.

Внутрисхемный тестер может принудительно задавать значения *большинства* сигналов, но он не может изменить значение сигнала на заземленном входе. Поэтому входы, на которые должен быть постоянно подан логический 0 во время нормальной работы схемы, нужно заземлять через резистор, что позволит тестеру устанавливать на этих входах логическую 1, если это потребуется при тестировании. Посмотрите, например, что произошло бы при формировании с помощью ПЛУ GAL16V8 требуемой совокупности тактовых сигналов из сигнала задающего генератора, как мы делали это в программе на языке ABEL (табл. 8.26). Если не учитывать потребности тестирования, то мы могли бы непосредственно соединить с землей вывод 11 ИС 16V8, низкий уровень сигнала на котором является глобальным разрешением выхода. Однако этот вывод следует заземлить через резистор, чтобы тестер мог переводить выходы ПЛУ в третье состояние и сам задавать тактовые сигналы P1_L–P6_L. Хотя теоретически тестер мог бы принудительно устанавливать значения этих сигналов, делать этого не следует, если они используются в качестве тактовых сигналов.

- *Шинные формирователи.* Для того чтобы тестер мог подавать сигналы на шину и при этом не преодолевать значения сигналов, вырабатываемых другими источниками, нужно отключать эти источники.

Это означает, что должна существовать возможность переводить все выходы схем, подключаемые к шине, в третье состояние, чтобы тестер выдавал сигналы на «плавающую» шину. При этом уменьшается электрическая нагрузка как на тестер, так и на многоразрядные источники сигналов (например, на ИС 74х244), которые в противном случае могут перегреваться и выходить из строя при одновременном принудительном задании на их выходы значений, противоположных тем, какие вырабатываются этими микросхемами.

11.2.3. Методы сканирования

Внутрисхемное тестирование хорошо работает до определенных пределов. Оно не годится для заказных СБИС и специализированных ИС, потому что внутренние сигналы в них просто недоступны. Даже в схемах, собранных на печатных платах, методы плотной упаковки типа поверхностного монтажа значительно затрудняют создание на плате контрольных точек для каждого сигнала. В результате, для обеспечения управляемости и наблюдаемости во все большем числе проектов используют «методы сканирования».

Метод сканирования (scan method) предполагает, что для задания внутренних сигналов в схеме и для наблюдения за ними используется лишь небольшое число контрольных точек. *Метод сканирования по заданному пути (scan-path method)* основан на предположении, что любая цифровая схема представляет собой комбинацию триггеров или других запоминающих элементов, объединенных комбинационной логикой. С помощью этого метода осуществляется управление и наблюдение за состоянием запоминающих элементов. Метод сканирования по заданному пути предполагает наличие двух режимов работы: режима нормальной работы и *режима сканирования (scan mode)*, при котором все запоминающие элементы входят в состав гигантского регистра сдвига. В режиме сканирования состояние n запоминающих элементов схемы может быть прочитано за n сдвигов (*наблюдаемость*). За то же время все элементы можно установить в новое состояние (*управляемость*).

На рис. 11.3 показана схема, в которой применен метод сканирования по заданному пути. В этой схеме каждый запоминающий элемент является сканируемым триггером (см. раздел 7.2.7), информацию в который можно загрузить от одного из двух источников. Сигналом, поданным на вход разрешения тестирования TE , выбирается источник данных: обычные данные (поступающие на вход D) или проверочные данные (поступающие на вход T). Для образования пути сканирования, показанного синим цветом, вход T каждого триггера соединен с выходом предыдущего триггера, в результате чего возникает последовательная цепочка. Удерживая в течение 11 периодов тактового сигнала активный уровень на входе $ENSCAN$, тестер может «увидеть» текущие состояния триггеров и загрузить новые состояния. Инженеру, занимающемуся тестированием, остается лишь написать тестовые последовательности для отдельных комбинационных логических блоков, которые становятся, таким образом, полностью управляемыми и наблюдаемыми со стороны внешних входов и выходов посредством сканирования по заданному пути.

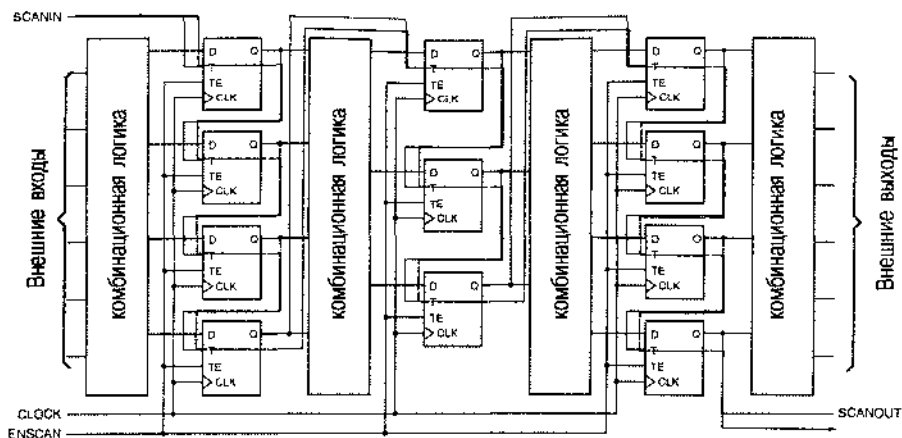


Рис. 11.3. Схема, содержащая путь сканирования, показанный синим цветом

Проектирование с использованием пути сканирования чаще всего применяется при разработке заказных СБИС и специализированных ИС из-за невозможности обеспечить большое число обычных контрольных точек. Однако для необходимых в этом случае триггеров с двумя входами требуется кристалл большей площади. Например, в серии БИС LCA500K фирмы Logic Corp., выполненной в виде матриц КМОП-вентилей, D-триггеры в макроячейке FD1QP состоят из 7 «вентильных ячеек», в то время как сканируемый D-триггер в макроячейке FD1SQP состоит из 5 вентильных ячеек, занимая, таким образом, почти на 30% большую площадь кремниевой пластины. Однако результирующее увеличение площади кристалла значительно меньше, так как триггеры составляют только часть микросхемы, а большие «регулярные» структуры памяти (например, ОЗУ) можно проверить другими способами. В любом случае повышение уровня тестируемости может реально *уменьшить* стоимость готовой микросхемы, даже если принять во внимание стоимость тестирования. При разработке больших специализированных ИС с развитой и сложной системой управления, образование пути сканирования следует считать необходимым требованием.

11.3. Оценка надежности цифровой системы

Надежность (reliability) цифровой системы качественно определяется как вероятность того, что она работает правильно, когда вам надо. Специалисты по сбыту и продавцы любят говорить, что устройства, которые они продают, имеют «высокую надежность», имея в виду, что системы «с большой вероятностью работоспособны». Однако здравомыслящие заказчики задают вопросы, требующие более конкретных ответов, например: «Если я покупаю 100 устройств, то сколько из них выйдет из строя через год?». Чтобы ответить на эти вопросы, разработчики цифровых устройств часто проводят расчет надежности системы, которую они проектируют; в любом случае они должны знать факторы, оказывающие влияние на надежность.

Количественно надежность выражается математической функцией времени:

$R(t)$ = Вероятность того, что система продолжает правильно работать в момент времени t .

Надежность представляет собой вещественное число от 0 до 1: в любой момент времени $0 \leq R(t) \leq 1$. Мы предполагаем, что $R(t)$ является монотонно убывающей функцией, и после возникновения отказа он сохраняется; мы не принимаем во внимание возможность восстановления. Типичный вид функции надежности приведен на рис. 11.4.

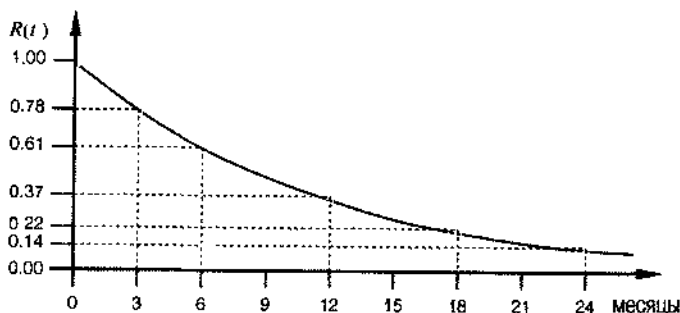


Рис. 11.4. Типичный вид функции надежности системы

Понятие надежности предполагает, что вы знаете математическое определение вероятности. Если это не так, то проще всего надежность и соответствующую вероятность выразить в терминах, используемых при проведении экспериментов. Предположим, что мы должны построить и использовать N идентичных экземпляров рассматриваемого устройства. Пусть $W_N(t)$ означает число устройств, которые продолжают работать в момент времени t . Тогда

$$R(t) = \lim_{N \rightarrow \infty} W_N(t) / N.$$

Другими словами, если мы построим *много* устройств, то $R(t)$ — это доля устройств, которые остаются работоспособными к моменту времени t . Когда мы говорим о надежности одного устройства, мы просто используем имеющийся опыт работы с большой совокупностью устройств в качестве оценки наших шансов в отношении данного устройства.

Если бы единственным способом нахождения $R(t)$ было проведение эксперимента, то это обошлось бы очень дорого: пришлось бы изготовить и испытать N экземпляров одного и того же устройства. Хуже того, для любого t мы не знали бы значение $R(t)$ до тех пор, пока реально не прошел интервал времени длительностью t . Таким образом, чтобы ответить на поставленный заказчиком вопрос, мы должны были бы взять большое число устройств и ждать в течение года; к тому времени наш потенциальный заказчик купил бы что-нибудь другое.

Вместо этого надежность системы можно оценить с помощью простой математической модели, используя информацию о надежности отдельных компонентов. Надежность давно выпускаемых компонентов (например, надежность КМОП-

микросхем 74FCT) может быть известна по результатам фактически проведенных экспериментов и опубликована, в то время как надежность новых компонентов (например, надежность микропроцессора *Sehium*) можно оценить, экстраполируя опыт работы с подобными устройствами. В любом случае надежность компонента обычно задается одним числом — «интенсивностью отказов», о чем говорится ниже.

11.2.1. Интенсивность отказов

Интенсивность отказов (failure rate) — это число отказов компонента или системы в единицу времени. В математических формулах интенсивность отказов обычно обозначают греческой буквой λ . Так как отказы в электронном оборудовании происходят редко, интенсивность отказов измеряется или оценивается путем исследования большого числа идентичных экземпляров данного компонента или устройства. Если, например, мы наблюдаем за работой 10000 микропроцессоров в течение 1000 часов и за это время восемь из них вышли из строя, то можно сказать, что интенсивность отказов равна

$$\lambda = \frac{8 \text{ отказов}}{10^4 \text{ микросхем} \cdot 10^3 \text{ часов}} = 8 \cdot 10^{-7} \text{ отказов в час на одну микросхему.}$$

Таким образом, интенсивность отказов в расчете на одну микросхему составляет $8 \cdot 10^{-7}$ отказов/час.

В действительности, процесс оценки надежности совокупности микросхем не так прост, как только что было описано; для получения более полной информации следует обратиться к специальной литературе. Однако, как мы покажем позже в этом параграфе, можно непосредственно воспользоваться полученной любыми способами интенсивностью отказов отдельных компонентов для предсказания общей надежности системы.

Поскольку у типичных электронных компонентов интенсивность отказов очень мала, принято указывать ее числом единиц в том или ином временном масштабе: процент отказов за 10^3 часов, число отказов за 10^6 или за 10^9 часов. Последняя единица называется *FIT*:

$$1 \text{ FIT} = 1 \text{ отказ} / (10^9 \text{ часов}).$$

Можно сказать, что в предыдущем примере с микропроцессором $\lambda = 800 \text{ FIT}$.

Для типичного электронного компонента интенсивность отказов является функцией времени. Как показано на рис. 11.5, типичный компонент имеет высокую интенсивность отказов в течение начального срока службы, когда проявляется большинство производственных дефектов; отказы в течение этого периода называются *отказами в начальный период эксплуатации (infant mortality)*. В связи с большой вероятностью отказов в начальный период эксплуатации при производстве высококачественного оборудования проводится его *испытание на отказ (burn-in)*, состоящее в том, что перед отправкой оборудования заказчику в течение некоторого времени наблюдают за его работой — от 8 часов до 8 суток. При испытании на отказ большинство сбоев, которые могут произойти в начальный период, происходит на заводе, а не у заказчика. По-видимому, даже без ис-

черпывающей проверки на отказ в начальный период работы, гарантия сроком 90 дней, оговариваемая многими производителями электронного оборудования, фактически охватывает большинство отказов, которые происходят в течение нескольких первых лет эксплуатации (когда отказ происходит на 91-ый день, это ужасно!). Ситуация здесь принципиально отличается от той, какая имеет место в случае с автомобилем или узлом иного механического устройства, когда в результате износа интенсивность отказов со временем увеличивается.

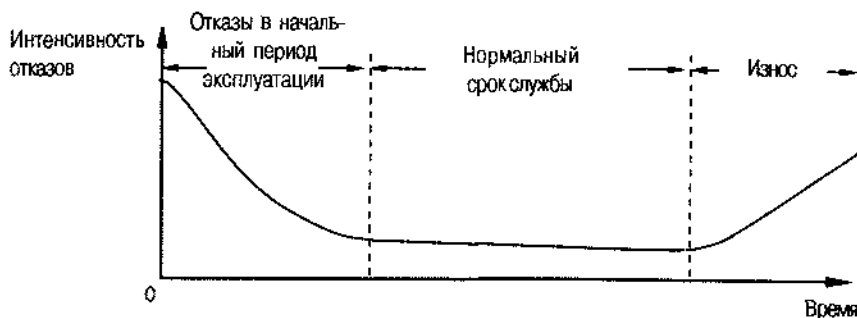


Рис. 11.5. U-образная кривая интенсивности отказов электронного компонента

Если электронный компонент успешно прошел испытание на отказ, то можно ожидать, что интенсивность отказов будет оставаться практически постоянной. На более позднем этапе работы может сказываться *износ* (wear-out) компонента, что приводит к увеличению интенсивности отказов. Прежде, нередко, после нескольких тысяч часов работы происходило ухудшение свойств электронных ламп из-за старения катода, вызванного тепловыми нагрузками. Теперь в большинстве случаев электронное оборудование устаревает прежде, чем начинают происходить отказы полупроводниковых компонентов. Например, несмотря на то, что широкое применение стираемых программируемых ПЗУ началось более 25 лет назад и для большинства из них гарантировалось сохранение данных в течение только 10 лет, мы не видели массовых отказов оборудования из-за потери информации. (Вы знаете когда-нибудь, у кого есть персональный компьютер или видеомаягнитофон 10-летней давности?)

Таким образом, на практике не принимаются во внимание отказы электронных компонентов в начальный период работы и по причине износа в конце срока службы, и надежность рассчитывается при условии, что интенсивность отказов электронного оборудования остается постоянной в течение нормального срока службы. Это предположение означает, что отказ одинаково вероятен в любой момент времени в течение срока службы компонента, и это позволяет нам, как будет показано позже в этом параграфе, использовать упрощенную математическую модель для предсказания надежности системы.

Существуют и другие факторы, оказывающие влияние на интенсивность отказов компонента, такие как температура, влажность, ударные воздействия, вибрация и периодическое включение и выключение питания. Для ИС наиболее существенным из перечисленных факторов является температурный фактор. Многие механизмы отказа ИС связаны с химическими реакциями, происходящими в кристалле полупроводника из-за разного рода загрязнений, которые ускоряются при

более высоких температурах. Аналогично, на надежность влияют электрические перегрузки транзисторов, в результате которых они слишком сильно нагреваются и в конечном счете выходят из строя, причем происходит это тем чаще, чем выше температура, при которой работает устройство. Теория и практика наглядно подтверждают следующее широко применяемое эмпирическое правило:

- Интенсивность отказов интегральных схем примерно удваивается при повышении их температуры на каждые 10°C .

В большей или меньшей степени это правило справедливо и для большинства других электронных компонентов.

Заметьте, что температура, фигурирующая в приведенном правиле, — это *внутренняя* температура ИС, а не температура окружающего воздуха. В системе без принудительного воздушного охлаждения внутренняя температура компонента, потребляющего большую мощность, может быть на $40\text{--}50^{\circ}\text{C}$ выше температуры окружающей среды. Удачно расположенный вентилятор позволяет понизить эту разность до $10\text{--}20^{\circ}\text{C}$, в результате чего интенсивность отказов компонентов может упасть в 10 раз.

11.3.2. Надежность и среднее время между отказами

Можно показать, что для компонентов с постоянной интенсивностью отказов λ надежность является показательной функцией времени:

$$R(t) = e^{-\lambda t}.$$

Кривая надежности, изображенная на рис. 11.4, представляет собой именно такую функцию; приведенный на этом рисунке график соответствует значению λ , равному 1 отказ/год.

Другим критерием надежности компонента или системы служит *среднее время между отказами* (*mean time between failures, MTBF*), то есть среднее время, спустя которое компонент выходит из строя. Для компонентов с постоянной интенсивностью отказов λ среднее время между отказами просто равно величине, обратной λ :

$$\text{MTBF} = 1/\lambda.$$

11.3.3. Надежность системы

Допустим, что мы построили систему из m компонентов с интенсивностью отказов $\lambda_1, \lambda_2, \dots, \lambda_m$. Предположим, что для правильной работы системы *все* компоненты должны быть исправны. Согласно элементарной теории вероятностей, надежность системы в этом случае падает по формуле

$$\begin{aligned} R_{\text{sys}}(t) &= R_1(t) \cdot R_2(t) \cdot \dots \cdot R_m(t) \\ &= e^{-\lambda_1 t} \cdot e^{-\lambda_2 t} \cdot \dots \cdot e^{-\lambda_m t} \\ &= e^{-(\lambda_1 + \lambda_2 + \dots + \lambda_m)t} \\ &= e^{-\lambda_{\text{sys}} t}, \end{aligned}$$

где

$$\lambda_{\text{sys}} = \lambda_1 + \lambda_2 + \dots + \lambda_m.$$

Таким образом, надежность системы также является показательной функцией, в которой интенсивность отказов системы λ_{sys} равна сумме интенсивностей отказов отдельных компонентов.

Предположение о постоянстве интенсивности отказов сильно упрощает определение надежности системы: чтобы найти интенсивность отказов системы, просто складываются интенсивности отказов отдельных компонентов. Интенсивность отказов отдельных компонентов можно узнать у производителя или из справочников по надежности, а также воспользовавшись стандартами компании, в которой вы работаете.

В качестве примера в табл. 11.1 приведены некоторые данные стандарта одной компании. Так как интенсивность отказов служит лишь оценкой, в этой компании решили упростить жизнь разработчика путем указания только основных категорий компонентов, не приводя «точных» значений интенсивности отказов для каждого компонента в отдельности. В других компаниях используют более подробные таблицы, а в некоторых системах САД поддерживается база данных с интенсивностями отказов компонентов, что позволяет автоматически рассчитать суммарную интенсивность отказов схемы по перечню элементов, входящих в состав схемы.

Табл. 11.1. Типичные значения интенсивности отказов компонентов при температуре 55°C

<i>Компонент</i>	<i>Интенсивность отказов (FIT)</i>
ИС малой степени интеграции	90
ИС средней степени интеграции	160
Большая интегральная схема	250
Микропроцессор (СБИС)	500
Резистор	10
Развязывающий конденсатор	15
Разъем (в расчете на один контакт)	10
Печатная плата	1000

Предположим, что нам необходимо собрать на одной плате систему, состоящую из микропроцессора, выполненного в виде СБИС, 16 микросхем памяти и других больших интегральных схем, 2 ИС малой степени интеграции, 4 ИС средней степени интеграции, 10 резисторов, 24 развязывающих конденсаторов и разъемов с 50 контактами. Используя данные из табл. 11.1, находим суммарную интенсивность отказов:

$$\begin{aligned} \lambda_{\text{sys}} &= (1000 + 16 \cdot 250 + 2 \cdot 90 + 4 \cdot 160 + 10 \cdot 10 + 24 \cdot 15 + 50 \cdot 10 + 500) \text{ FIT} \\ &= 7280 \text{ отказов} / 10^9 \text{ часов.} \end{aligned}$$

Среднее время между отказами одноилатной системы равно $1/\lambda$ или приблизительно 15 с лишним лет. Да-да, небольшие устройства действительно могут быть такими надежными. Конечно, если мы учтем интенсивность отказов типичного источника питания, превышающую 10000 FIT, то среднее время между отказами сократится более чем вдвое.

11.4. Длинные линии, отражения и согласованная нагрузка

Ничто не происходит мгновенно, особенно в тех случаях, когда это касается цифровых схем. В частности, необходимо учесть тот факт, что задержка электрических сигналов, распространяющихся *по проводам* «со скоростью света», составляет порядка 4.5–6 нс на метр (точное значение задержки зависит от характеристик сигнальной линии). Когда задержки в линиях соизмеримы с временем перехода сигналов с низкого уровня на высокий и в обратном направлении, мы не должны считать, что соединения выполнены идеальными проводниками с нулевой задержкой, а рассматривать их как «длинные линии», чем они в действительности и являются.

В длинной линии наблюдаются такие изменения сигналов, которые нельзя предвидеть, анализируя работу схемы по «постоянному току». Наиболее значительные изменения происходят в течение интервалов времени длительностью примерно $2T$ после того, как изменяется сигнал на выходе источника, где T – задержка распространения сигнала от выхода источника до дальнего конца линии, по которой передается этот сигнал.

Если длительность переходов в сигнале и задержка его распространения в схеме намного больше, чем $2T$, то процессы, происходящие в длинной линии, практически не влияют на логику работы этого устройства. Таким образом, свойства длинной линии учитываются, как правило, только тогда, когда длина соединений превышает 60 см в случае ИС семейства 74LS, 30 см – для ИС семейств 74AS, 74AC и 74ACT и 10 см – для ИС семейства 74FCT и ЭСЛ-микросхем. Естественно, что, в зависимости от конкретной задачи, иногда можно избежать неприятностей при наличии и более длинных связей, но бывают ситуации, когда теория длинных линий должна применяться и при более коротких соединениях. Даже при разработке внутренней структуры СБИС с большим быстродействием необходимо учитывать явления, которые могут возникать в длинных линиях.

11.4.1. Основы теории длинных линий

Простейшая *длинная линия* (*transmission line*) представляет собой два параллельно идущих проводника. Рассмотрим пару проводников бесконечной длины, изображенную на рис. 11.6(а). Если к этой паре проводников мгновенно подключить источник напряжения, то потечет ток, образуя волну напряжения, распространяющуюся вдоль линии. Отношение напряжения к току V_{out}/I_{out} зависит от физических характеристик проводников и называется *волновым сопротивлением* (*characteristic impedance*) линии Z_0 .

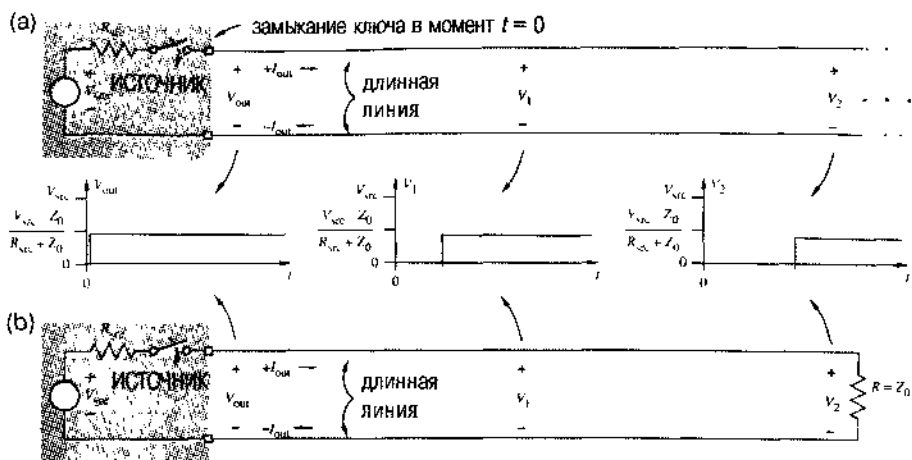


Рис. 11.6. Длинные линии: (а) линия бесконечной длины; (б) линия конечной длины, нагруженная на конце сопротивлением, равным волновому

Рассматривая последовательно включенные R_{src} и Z_0 как делитель напряжения, найдем величину V_{out} :

$$V_{out} = V_{src} \cdot \frac{Z_0}{R_{src} + Z_0}.$$

Конечно, у нас нет никаких бесконечно длинных проводников. Однако предположим, что имеется пара проводников длиной 1.5 метра и на дальнем конце этой линии включен резистор, сопротивление которого равно Z_0 . Если на эту пару проводов мгновенно подать напряжение, как показано на рис. 11.6(б), то потечет тот же самый ток, что и в случае с линией бесконечной длины. Таким образом, когда линия нагружена сопротивлением, равным волновому, нам не нужно учитывать какие-либо еще эффекты в длинной линии.

Иная ситуация наблюдается в том случае, когда длинная линия конечной длины не нагружена сопротивлением, равным волновому. Предельный случай имеет место, когда дальний конец замкнут накоротко, как показано на рис. 11.7(а). Ради простоты предположим, что в этом примере $R_{src} = Z_0$. В первый момент источник «видит» волновое сопротивление линии, и скачок напряжения величиной $V_{src}/2$ благополучно распространяется вдоль линии. Но когда в момент времени T волна достигает дальнего конца, она «натывается» на короткое замыкание. Чтобы удовлетворялись требования законов Кирхгофа, вдоль линии в обратном направлении пачкается распространяться волна напряжения *противоположной полярности*, гася исходную волну. На дальнем конце происходит *отражение (reflection)* исходной волны, и спустя время $2T$ источник сигнала «видит» короткое замыкание.

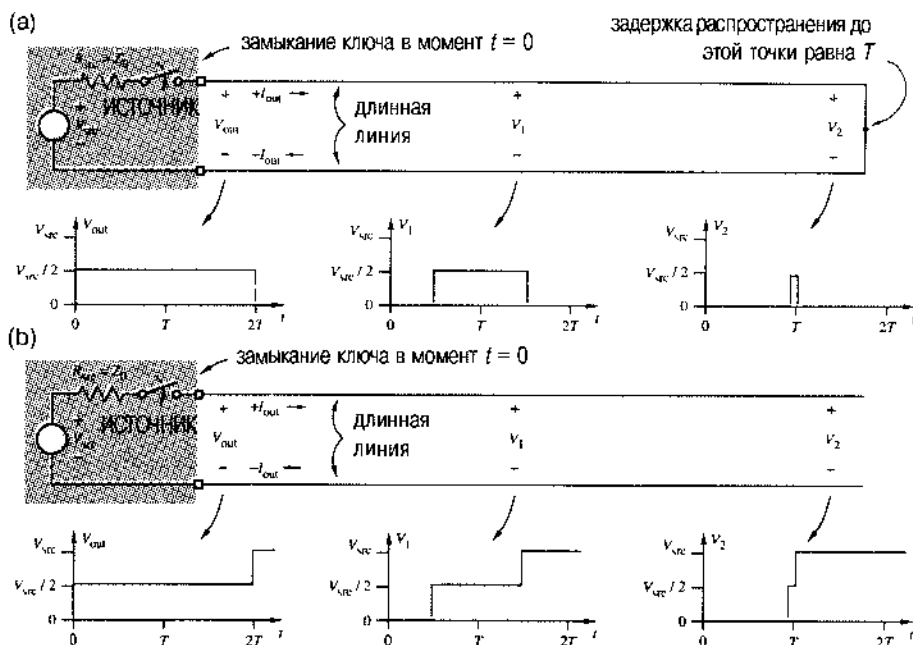


Рис. 11.7. Длинные линии: (а) замкнутая накоротко на дальнем конце; (б) разомкнутая на дальнем конце

Другой крайний случай имеет место при разомкнутом дальнем конце, как показано на рис. 11.7(б). Процесс начинается так же, как и прежде. Но в момент времени, когда начальная волна напряжения достигает дальнего конца, току течь некуда, и поэтому возникает волна напряжения *той же полярности*, которая распространяется к началу линии, добавляясь к исходному напряжению. К моменту времени $2T$, когда отраженная волна достигает источника, повсюду вдоль линии установится напряжение, равное V_{src} , и дальнейших изменений происходить не будет.

В общем случае амплитуда волны, отраженной от конца длинной линии, определяется коэффициентом отражения ρ (reflection coefficient). Величина ρ зависит от значения Z_0 и от сопротивления нагрузки Z_{term} (termination impedance), подключенной на конце линии:

$$\rho = \frac{Z_{\text{term}} - Z_0}{Z_{\text{term}} + Z_0}.$$

Когда волна напряжения с амплитудой V_{wave} достигает конца длинной линии, происходит ее отражение с амплитудой $\rho \cdot V_{\text{wave}}$. Заметьте, что нами рассмотрены три простых случая:

$Z_{\text{term}} = Z_0$ Длинная линия нагружена волновым сопротивлением, коэффициент отражения равен 0.

- $Z_{\text{term}} = 0$ Коэффициент отражения короткозамкнутой линии равен -1 ; возникает отраженный сигнал, равный по величине пришедшему, но противоположной полярности.
- $Z_{\text{term}} = \infty$ Коэффициент отражения разомкнутой линии равен $+1$; возникает отраженный сигнал, равный по величине пришедшему, той же полярности.

Если выходное сопротивление источника R_{src} не равно Z_0 , то на ближнем конце линии тоже возникают отражения и происходит это так же, как и на дальнем конце. На каждом из концов линии имеется свой коэффициент отражения ρ . Для линий справедлив принцип суперпозиции (*principle of superposition*), состоящий в том, что напряжение в любой момент времени в любой точке линии равно сумме начального напряжения в этой точке и напряжений всех волн, прошедших через эту точку к данному моменту времени. Говорят, что длинная линия согласована (*matched transmission line*), если она нагружена волновым сопротивлением.

На рис. 11.8 показаны сигналы в длинной линии, которая не согласована с обоих концов. (Здесь считается, что источник напряжения V_{src} является идеальным, то есть его внутреннее сопротивление равно 0 Ом .) Отражения происходят от обоих концов линии с все меньшими и меньшими амплитудами отраженных волн, распространяющихся в прямом и в обратном направлении. Напряжение по всей длине линии асимптотически стремится к величине $0.9V_{\text{src}}$, которую дает закон Ома при анализе схемы по постоянному току.

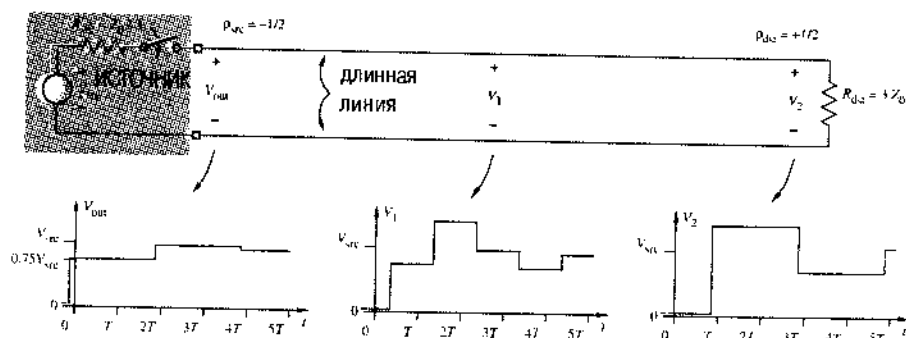


Рис. 11.8. Длинная линия, не согласованная с обоих концов

11.4.2. Передача логических сигналов по длинным линиям

Итак, какое влияние оказывают эффекты в длинных линиях на передачу логических сигналов? Давайте рассмотрим случай, когда сигнал с выхода КМОП-схемы семейства 74НС поступает на вход другой КМОП-схемы, которая включена на конце длинной линии, образованной сигнальным проводом и землей, как показано на рис. 11.9. Сопротивление «открытого» p - или n -канального транзистора на выходе микросхем семейства НСТ равно примерно $100\text{--}200 \text{ Ом}$; для простоты примем

его равным 150 Ом как при высоком, так и при низком уровне выходного сигнала. Волновое сопротивление типичного соединения в виде дорожки (*trace*) на печатной плате относительно земли находится в пределах 100–150 Ом; давайте для удобства примем его равным 150 Ом, чтобы коэффициент отражения на том конце линии, к которому подключен источник сигнала, был равен 0. Входное сопротивление КМОП-схемы в типичном случае больше 1 МОм, поэтому коэффициент отражения на приемном конце линии можно принять равным +1.

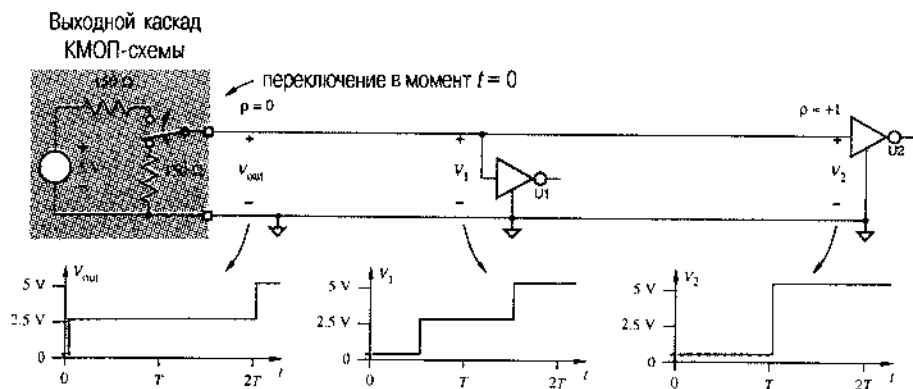


Рис. 11.9. Отражения в линии, по которой передается логический сигнал, изменяющийся от низкого уровня до высокого

На рисунке представлен случай перехода сигнала с низкого уровня на высокий. Когда уровень сигнала на выходе КМОП-схемы изменяется с низкого на высокий, источник напряжения 5 В оказывается нагруженным последовательно включенными выходным сопротивлением КМОП-схемы, равным 150 Ом, и волновым сопротивлением линии $Z_0 = 150$ Ом, поэтому вдоль линии начинает распространяться волна напряжения с амплитудой 2.5 В. Спустя время T эта волна достигает входа вентиля U2 на дальнем конце линии и отражается. По прошествии времени $2T$ отраженная волна достигает конца, к которому подключен выход логического элемента, и поглощается без отражения, поскольку на этом конце $\rho = 0$.

Все работает прекрасно в том, что касается вентиля U2 на дальнем конце линии, который «видит» мгновенное изменение напряжения от 0 В до 5 В через время T после того, как произошло переключение в источнике сигнала. Однако посмотрите на форму сигнала, поступающего на вход другого вентиля U1, расположенного на полпути между источником сигнала и вентилем U2. Как видно из рисунка, на входе вентиля U1 в течение времени T присутствует сигнал с напряжением только 2.5 В. На входе логического элемента расположенного еще ближе к источнику сигнала, такое напряжение продержится даже дольше. В этом и состоит проблема, поскольку напряжение 2.5 В является как раз пороговым входным напряжением для 5-вольтовых КМОП-схем. Если такое входное напряжение присутствует на входе вентиля U1 достаточно долго, то на его выходе могут возникнуть колебания или установится выходное напряжение, не соответствующее ни одному из логических уровней.

Любое соединение, по которому передается логический сигнал, независимо от его физической протяженности, является длинной линией. Однако наше рассмотрение в значительной степени идеализировано. На практике эффекты, возникающие в длинных линиях не вызывают никаких проблем, если время T меньше длительности переходов в логических сигналах и задержек, вносимых логическими элементами. Когда выполнены эти условия, колебания, вызванные отражениями, заканчиваются, как правило, прежде, чем приемники, подключенные к шине, успевают их заметить.

Некоторые неприятности возникают в том случае, когда с высокого уровня на низкий переходит сигнал на выходе быстродействующего элемента. Например, при низком уровне сигнала выходной каскад КМОП-вентиля семейства FCT представляет собой резистор с сопротивлением 10 Ом, включенный между выходом и землей (рис. 11.10). Поэтому коэффициент отражения на передающем конце равен примерно -0.88 . В первый момент, когда выходной сигнал переходит на низкий уровень, система выглядит как делитель напряжения, состоящий из резистора с сопротивлением 10 Ом, включенного последовательно с волновым сопротивлением линии, равным 150 Ом. Учитывая, что начальное напряжение на линии равнялось 5 В, новое напряжение на входе линии должно стать равным

$$V_{\text{out}} = \frac{10 \text{ Ом}}{150 \text{ Ом} + 10 \text{ Ом}} \cdot 5 \text{ В} = 0.31 \text{ В}.$$

Следовательно, вдоль линии начнет распространяться волна напряжения с амплитудой $(0.31 - 5.0) = -4.69 \text{ В}$. Когда спустя время T эта волна достигает приемного конца линии, происходит отражение с сохранением знака и с той же амплитудой отраженной волны (так как $\rho = 1$). Таким образом, напряжение на приемном конце линии теперь равно $V_2 = 5.0 - 4.69 - 4.69 = -4.38 \text{ В}$, то есть напряжение становится отрицательным! Это явление называется *отрицательным выбросом (undershoot)*.

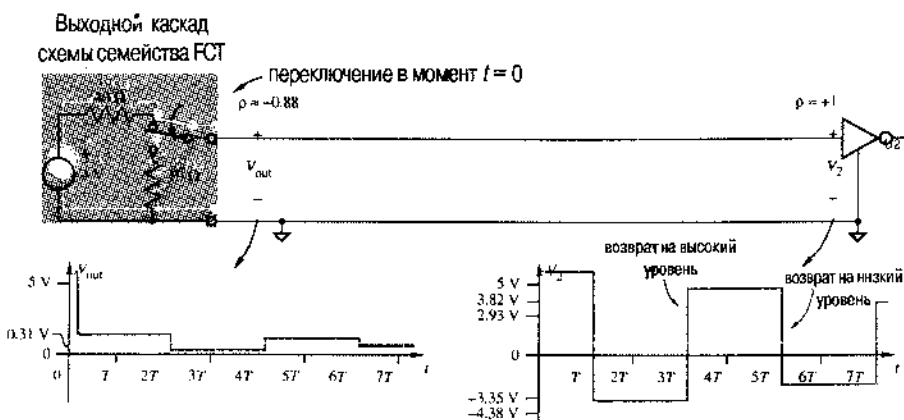


Рис. 11.10. Отражения в линии при изменении передаваемого логического сигнала с высокого уровня на низкий

Когда в момент времени $2T$ отраженная волна напряжения с амплитудой -4.69 В возвращается к передающему концу линии, происходит еще одно отражение. На сей раз, поскольку значение ρ отрицательно, отраженная волна положительна, а ее амплитуда равна $-0.88 \cdot (-4.69) = +4.10$ В. Теперь напряжение на выходе источника сигнала изменяется на величину, равную сумме амплитуд двух волн: волны, пришедшей со стороны дальнего конца линии, и новой отраженной волны, уходящей в сторону дальнего конца линии; поэтому $V_{\text{out}} = 0.31 - 4.69 + 4.10 = -0.28$ В. В этом месте не возникает никаких проблем. Но когда в момент времени $3T$ отраженная волна напряжения с амплитудой $+4.10$ В достигает приемного конца линии, происходит очередное отражение, в результате которого возникает волна той же полярности, и напряжение на входе вентиля U2 становится равным $V_2 = -4.38 + 4.10 + 4.10 = 3.82$ В, то есть мы снова возвращаемся к положительному напряжению!

Согласно рисунку, отражения продолжают и в дальнейшем, и напряжения на передающем и приемном концах линии асимптотически стремятся к 0 В. Это значение можно было бы предсказать, анализируя поведение схемы по постоянному току. Такой колебательный характер изменения напряжения называется *звоном (ringing)*.

Большая амплитуда звона на приемном конце линии может вызвать осложнения, так как в течение интервала времени длительностью $27T$ напряжение V_2 не попадает в диапазон значений, меньших, чем величина порога для сигнала низкого уровня (0.8 В). Таким образом, фактическая задержка передачи сигнала в этой схеме оказывается во много раз больше задержки распространения сигнала по линии. Хуже того, если данный сигнал является тактовым сигналом, то в результате звона появятся дополнительные фронты, приводящие к ошибочному переключению триггеров на приемном конце линии.

Еще раз, рассмотренные нами эффекты, возникающие в длинной линии при передаче логических сигналов, не вызывают никаких проблем в том случае, когда время T намного меньше длительности переходов в логических сигналах и задержек, вносимых логическими элементами. Кроме того, во входных цепях ТТЛ-схем и многих КМОП-схем имеются *демпфирующие диоды (clamping diodes)*. При нормальной работе эти диоды, включенные между каждым из входов и землей, смещены в обратном направлении (см. рис. 3.75). Благодаря наличию этих диодов входное сопротивление вентиля при отрицательных напряжениях становится очень малым, в результате чего отрицательный выброс, возникающий в момент времени T , оказывается не таким большим по величине и его значение равняется примерно 1 В. Поэтому уменьшается амплитуда отраженной волны, распространяющейся назад к передающему концу линии, что в свою очередь дает меньший выброс в момент времени $3T$, не превосходящий 1 В. Кроме того, на входах некоторых схем имеются диоды, подключенные к источнику питания V_{CC} , что ограничивает выбросы напряжения, возникающие тогда, когда сигналы на выходах вентиля с малым выходным сопротивлением переходят с низкого уровня на высокий.

11.4.3. Согласованные нагрузки на концах линий передачи логических сигналов

Отражения могут быть устранены при включении в конце длинной линии нагрузки с сопротивлением, равным волновому. Обычно применяются два метода.

Как видно из рис. 11.11(а), *оконечная нагрузка (Thevenin termination)* состоит из пары резисторов, включенных в конце линии. Согласно теореме Тевенина, эту оконечную нагрузку можно представить в виде эквивалентной схемы, показанной на рис. 11.11(б). При выборе сопротивлений резисторов учитывается несколько факторов:

1. Эквивалентное сопротивление R_{Thev} должно равняться волновому сопротивлению линии Z_0 или быть близким к нему. Эквивалентное сопротивление равно сопротивлению параллельно соединенных резисторов $R1$ и $R2$, то есть $(R1 \cdot R2)/(R1 + R2)$.
2. Эквивалентное напряжение V_{Thev} можно выбрать так, чтобы оптимально удовлетворить требованиям, которые предъявляются к току, вытекающему в выходной каскад вентиля, и к току вытекающему из него. В данном случае $V_{\text{Thev}} = V_{\text{CC}} \cdot R2/(R1 + R2)$.
 - Если значения вытекающего и втекающего токов при высоком и низком уровнях сигнала на выходе вентиля, являющегося источником сигнала, передаваемого по линии, равны (как это имеет место в стандартном КМОП-вентиле, не совместимом с ТТЛ-схемами), то значение V_{Thev} можно выбрать посередине между V_{OL} и V_{OH} .
 - С другой стороны, если выходной каскад вентиля таков, что втекающий ток больше вытекающего тока (что имеет место в ТТЛ-схемах и в ТТЛ-совместимых КМОП-схемах), то значение V_{Thev} выбирается большим по величине, и тогда, в соответствии с предъявляемыми требованиями, значение вытекающего тока при высоком уровне выходного сигнала будет меньше, а значение втекающего тока при низком уровне выходного сигнала – больше.
3. Для шин с тремя состояниями эквивалентное напряжение V_{Thev} можно выбрать так, чтобы оно строго соответствовало тому или иному логическому уровню, когда на шину не подается сигнал ни одним из подключенных к ней источников сигналов. В этом случае особенно важно выбрать значение V_{Thev} далеким от порога переключения вентиля, чтобы они не потребляли чрезмерный ток и не переходили в режим колебаний.
4. В конечном счете, сопротивления резисторов выбираются из существующих номинальных значений (например, 150, 220, 270, 330, 390, 470 Ом).

«Стандартная» оконечная нагрузка в устройствах с ТТЛ-схемами выглядит следующим образом: $R1 = 220 \text{ Ом}$, $R2 = 330 \text{ Ом}$, что дает $R_{\text{Thev}} = 132 \text{ Ом}$ и $V_{\text{Thev}} = 3.0 \text{ В}$. При низком уровне выходного сигнала источник сигнала должен допускать втекание тока, равного $(3.0 \text{ В})/(132 \text{ Ом}) = 22.7 \text{ мА}$, а высокий уровень выходного сигнала надежно поддерживается даже в том случае, когда вытекающий ток источника сигнала равен 0.

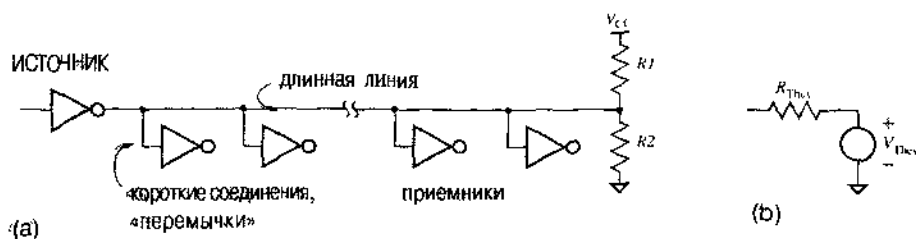


Рис. 11.11. Оконечная нагрузка: (а) схема; (б) эквивалентная схема оконечной нагрузки

При согласованной нагрузке в конце длинной линии отражений не будет или они будут небольшими. Нижний резистор оконечной нагрузки — это та часть нагрузки, которая всегда потребляет мощность по постоянному току, и от источника сигнала требуются относительно большие токи. *Согласование со стороны источника (source termination)* [иногда называемое *последовательным согласованием (series termination)*] решает эти проблемы.

Как показано на рис. 11.12, между длинной линией и источником сигнала, физически рядом с ним, последовательно включается резистор с сопротивлением ($Z_0 - R_d$), где R_d — характерное значение выходного сопротивления источника сигнала. Форма сигналов в линии при таком согласовании приведена на рис. 11.13. При переходе напряжения на выходе вентиля от значения V_{CC} к 0 (или наоборот) источник сигнала нагружен сопротивлением $2Z_0$, получающимся в результате сложения сопротивления добавочного резистора и сопротивления самой линии. Из всего перепада напряжения одна половина падает на выходном сопротивлении вентиля и добавочном резисторе, а другая приложена к входу линии. Таким образом, спустя время T , равное времени распространения сигнала по линии, вентиль U2 на дальнем конце линии видит изменение напряжения $V_{CC}/2$. В связи с тем, что коэффициент отражения равен 1, этот перепад напряжения тотчас же отражается назад к источнику сигнала, что приводит к повышению напряжения на входе U2 до величины V_{CC} . Отраженная волна достигает источника сигнала и поглощается в согласованной нагрузке с сопротивлением Z_0 , в результате чего новые отражения в линии не возникают.

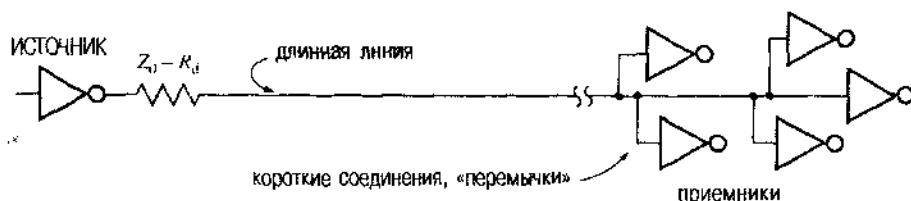


Рис. 11.12. Согласование со стороны источника сигнала

Согласование со стороны источника сигнала хорошо работает в том случае, когда выходные сопротивления источника сигнала при низком и высоком уровнях на его выходе можно считать практически равными (как, например, у КМОП-схем). В типичных схемах при волновых сопротивлениях линий порядка 50–100 Ом применяют добавочные резисторы с сопротивлениями 15–40 Ом.

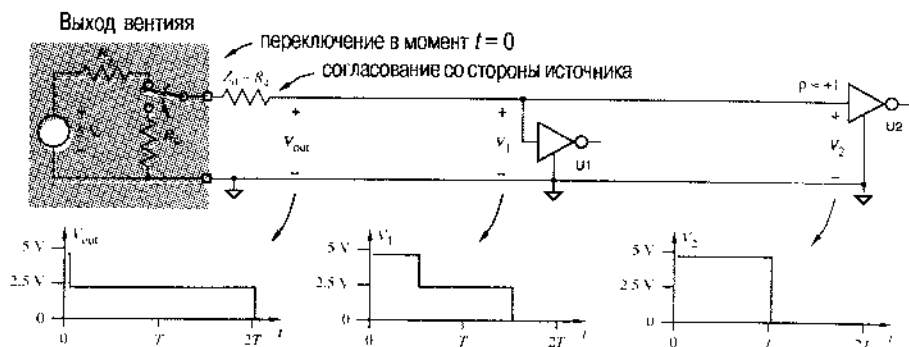


Рис. 11.13. Отражения в линии, согласованной со стороны источника, при переходе сигнала с высокого уровня на низкий

Согласование со стороны источника сигнала нежелательно в том случае, когда какие-либо вентили, такие, например, как вентиль U_1 на рис. 11.13, размещаются где-то посередине длинной линии. При изменении сигнала начальный перепад на входах «средних» вентилях, составляет только $V_{CC}/2$; полный перепад возникает только тогда, когда подойдет отражение от дальнего конца. Плохо, если такое состояние длится очень долго, поскольку напряжение $V_{CC}/2$ может оказаться близким к порогу переключения вентиля. Поэтому согласование со стороны источника обычно применяется только тогда, когда печатная плата физически выполнена примерно так, как показано на рис. 11.12. Соединение представляет собой относительно длинную линию, по которой сигнал передается к одной или к нескольким нагрузкам, расположенным близко друг от друга.

Обзор литературы

Пакет программ CAD – всего лишь инструмент, и вы не найдете там никаких подробных справочных данных о том, как им пользоваться. Однако журнал *IEEE Design & Test of Computers (D&T)* является прекрасным источником постоянно обновляющейся информации об инструментальных средствах CAD. Лучшим источником информации по конкретной системе CAD, естественно, является документация поставщика. В том случае, когда какая-то конкретная система используется *очень широко*, может появиться специальное учебное пособие; хорошим примером является книга Туиненги *SPICE: Руководство по моделированию и анализу схем с использованием Pspice* (Paul W. Tuinenga. *SPICE: A Guide to Circuit Simulation and Analysis Using Pspice*[®]. Prentice Hall, 1988).

Языки межрегистровых пересылок рассмотрены в большинстве книг по машинному проектированию, в том числе в книге Хэйеса *Архитектура и организация вычислительных систем* (John P. Hayes. *Computer Architecture and Organization*, second edition. McGraw-Hill, 1988). В феврале 1985 года специальный выпуск журнала *IEEE Computer* был посвящен языкам описания аппаратуры, а специальный выпуск журнала *D&T*, посвященный языку VHDL, увидел свет в апреле 1986 года.

Авторитетное рассмотрение принципов проектирования логических устройств с обеспечением их тестируемости содержится в книге Мак-Класки *Принципы логического проектирования* (Edward J. McCluskey. *Logic Design Principles*. Prentice Hall, 1986). Детальное рассмотрение вопроса с точки зрения инженеров-разработчиков и инженеров, проводящих тестирование, имеется в книге Кортнера *Техника тестирования цифровых устройств* (J. Max Cortner. *Digital Test Engineering*. Wiley, 1987). Напомним еще раз, что прекрасным источником информации является журнал *D&T*. В специальном августовском выпуске 1989 года, посвященном внутрисхемным методам самоконтроля, имеется статья Паркера *Роль периферийного сканирования в тестировании плат* (Kenneth P. Parker. *The Impact of Boundary Scan on Board Test*); в этой статье с практических позиций рассмотрены перспективы методов тестирования, приобретающих все большее значение при разработке плат.

Необходимый математический фундамент и общие методы анализа надежности систем можно найти в небольшом справочнике Клати *Надежность электронного оборудования* (J. C. Cluley. *Electronic Equipment Reliability*. Halsted Press/Wiley, 1974). Чтобы иметь представление о том, как в действительности выходят из строя интегральные схемы и как можно прогнозировать интенсивность отказов в пределах срока жизни (вашей или интегральных схем), вам следует прочесть книгу Амерасекеры и Кэмпбелла *Механизмы отказов в полупроводниковых приборах* (E. A. Amerasekera, D. S. Campbell. *Failure Mechanisms in Semiconductor Devices*. Wiley, 1987).

Превосходное, с точки зрения разработчика цифровых устройств, введение в теорию и практику длинных линий можно найти в книге Блэйксли *Цифровое проектирование на основе стандартных МИС и БИС* (Thomas R. Blakeslee. *Digital Design with Standard MSI and LSI*, second edition. Wiley, 1979).

Настоятельно рекомендуемым чтением и авторитетной трактовкой аналоговых аспектов цифрового проектирования, о которых я люблю без конца повторять, является книга Джоисона и Грэхэма *Проектирование быстродействующих цифровых устройств* (Howard W. Johnson, Martin Graham. *High-Speed Digital Design*. Prentice Hall, 1993). Хотя в подзаголовке книги значится *Справочник черной магии* (*A Handbook of Black Magic*), авторы проводят читателя по тернистому пути исключения таинственности в аналоговом поведении цифровых устройств, с которым должны научиться справляться все разработчики быстродействующих цифровых систем.

Предметный указатель

0-set 357
 0s catching 640
 0x prefix 53
 1-bit parity code 86
 1-out-of-10 code 77
 1-out-of-n code 81
 1-set 357
 10's complement 60
 16V8R 796
 16V8S 796
 16V8C 796
 1s catching 640
 20V8 798
 22V10 799
 2421 code 76
 27128 973
 27256 973
 27512 973
 2764 973
 4000-series CMOS 169
 5-variable Karnaugh map 366, 671
 74-series TTL 203
 74ALS 204
 74AS 204
 74AS4374 893
 74F 204
 74F74 775
 74L 203
 74LS 204
 74S 204
 74x125 452
 74x126 452
 74x138 420
 74x139 417
 74x148 442
 74x151 465
 74x153 468
 74x157 467
 74x181 509
 74x182 513
 74x240 453
 74x241 453
 74x251 468
 74x253 469
 74x257 469
 74x280 481
 74x283 506
 74x381 511
 74x382 511
 74x49 438
 74x540 453
 74x541 452
 74x83 506
 74x85 492
 74x86 481

74ACT74 775
 74H 203
 74x109 641, 775
 74x112 775
 74x160 810
 74x161 810
 74x162 810
 74x163 807
 74x164 827
 74x166 827
 74x169 815
 74x174 781
 74x175 780
 74x194 828
 74x273 783
 74x299 830
 74x373 783
 74x374 782
 74x375 775
 74x377 783
 74x74 775
 8421 code 76
 8B10B code 83
 9's complement 63

A

A-law PCM 978
 ABEL compiler 298
 ABEL language processor 298
 absolute maximum ratings 206
 AC fanout 141
 AC load 146
 access time from address 975, 988
 access time from chip select 975, 988
 active high 376
 active level 376
 active low 376
 active mode 974
 active pull-up 160
 active region 186
 active-level naming convention 376
 actual parameters 329
 adder 500
 adding out 243
 address hold time after write 989
 address input 960
 address setup time before write 989
 adjacent states 723
 advanced low-power Schottky TTL 204
 advanced Schottky TTL 204
 after 351
 algebraic operator 240
 algorithmic state machine 752
 all inclusion 656, 686
 alternate mark inversion 100

ALU 509
 American Standard Code for Information Interchange 79
 AMI 100
 amplifier 186
 analog 23
 AND 300
 AND gate 27, 110
 AND operation 241
 AND plane 407
 AND-OR circuit 263
 AND-OR-INVERT 123
 anode 180
 AOI gate 123
 application-specific IC (ASIC) 39
 approved-part number 1036
 architecture 319
 architecture definition 321, 322
 architecture-name 322
 arguments 329
 arithmetic and logic unit 509
 arithmetic shift 858
 array 327
 array index 327
 array literal 328
 array slice 329
 array types 327
 ASCII 79
 ASIC design 47
 ASIC 39
 ASM 752
 assert 376
 asymmetric output drive 170
 asynchronous input signal 880
 asynchronous input 634
 asynchronous SRAM 992
 attribute 332
 attribute suffix .OE 457
 automatic tester 1043
 axiom 239

B

β 187
 back unnotation 1036
 "back-end" 318
 balanced code 99
 barrel shifter 539
 base 50, 185
 basis step 244
 BCD 74
 BCD addition 75
 BCD decoder 416
 bed of nails 1043
 behavioral description 344
 behavioral design 344
 behavioral model 1038
 bias 64
 bidirectional bus 454

bidirectional shift register 828
 bit of materials 371
 binary addition 56
 binary decoder 414
 binary digit 50, 108
 binary encoder 440
 binary operator 243
 binary point 50
 binary rate multiplier 913
 binary subtraction 57
 binary-coded decimal 74
 binary-to-hexadecimal conversion 52
 binary-to-octal conversion 52
 bipolar junction transistor 113, 185
 bipolar logic family 113
 Bipolar Return-to-Zero 100
 biquinary code 76
 bistable 622
 bit cell 96
 bit line 965
 bit rate 96
 bit time 96
 bits 50
 BJT 113, 185
 block diagram 370, 372
 board-level design 46
 BOM 371
 boolean 324
 borrow 69
 boundary inputs 490
 boundary outputs 490
 BPRZ 100
 bps 96
 branching method 276
 bubble-to-bubble logic design 379
 buffer amplifier 108
 buried macrocell 1007
 burn-in 1050
 bus 372
 bus holder circuit 779
 bus transceiver 453
 BUT 363
 BUT flop 762
 BUT gate 363
 butification 540

C

CAD 30, 1033
 CAF 1033
 canonical product 253
 canonical sum 253
 capacitive load 146
 carry 68
 carry generate 505
 carry lookahead 505
 carry lookahead adder 506
 carry propagate 505
 carry-save addition 520

- CAS-before-RAS refresh cycle 1002
- cascaded synchronizers 892
- cascading input 492
- cascading inputs and outputs 490
- case statement 348
- cathode 180
- causality 390
- CE 636
- ceiling function 79
- central office 832
- character 324
- characteristic equation 646
- characteristic impedance 1054
- check bits 87
- checksum 95
- checksum code 95
- chip-select input 974
- chip-select setup 989
- circuit description 371
- circuit specification 370
- circular shift 858
- clam shell 1044
- clamp diode 192, 212
- clamping diodes 1060
- CLB 1017
- cleat 626, 634
- clock 620
- clock edge 744
- clock enable 636
- clock frequency 620
- clock period 620
- clock skew 874
- clock tick 620
- clocked assignment operator 732
- clocked synchronous state machine 621, 644
- clocked truth-table operator 733
- CLR 634
- CML 215
- chip-select (CS) input 974
- CMOS 114
- CMOS load 175
- CMOS logic 116
- CO 832
- code 74
- code word 74
- coded state 665
- collector 185
- column address strobe 999
- column-address register 1000
- com 300
- combinational circuit 27, 110
- combinational multiplier 519
- combining theorem 244
- commutative input 868
- comments 300, 321
- common-emitter configuration 186
- common-mode signal 217
- companded encoding 978
- comparator 490
- comparing numbers 58
- combinational carry output 819
- complement 240
- complement number system 60
- complement of a logic expression 246
- complementary MOS circuits 114
- complete set 363
- complete sum 274
- complete test set 1043
- complex PLD 37, 411
- complex programmable logic device 1005
- component declaration 337
- component library 1035
- component location 1036
- component model 1037
- component statement 336
- component type 1035
- component value 1035
- computer-aided design 30
- computer-aided design 1033
- computer-aided engineering 1033
- computing the radix complement 61
- concatenation operator 329
- concurrent signal-assignment statement 341
- concurrent statement 336
- condition input 868
- conditional signal-assignment statement 341
- configurable logic block 1017
- connection flag 1036
- consensus 244, 294
- consensus theorem 244
- constant declaration 326
- constant 326
- constraints 318
- contact bounce 777
- control unit 867
- controllability 1043
- core logic 210
- counter 804
- cover 244
- covering theorem 244
- cover 273
- CPLD 37, 411, 1005
- CRC code 92
- critical race 712
- critical signal paths 1040
- CS 974
- CS-controlled write 989
- cube representation 284
- current flow 135
- current spikes 142
- current-mode logic 215
- custom LSI 39
- cut off 186
- cut set 709
- CV^2 /power 154
- cyclic-redundancy-check code 92

D

D latch 631
 d-set 279
 data hold time 989
 data output 960
 data setup time 989
 data sheet 126
 data unit 867
 dataflow description 341
 dataflow design 341
 dc 310
 DC balance 99
 DC fanout 140
 DC load 131
 DC noise margin 131, 195
 dead time 451
 deasserted 376
 debounce 777
 decade counter 810
 decimal decoder 416
 decimal-to-radix- r conversion 54
 decision window 886
 decode 414
 decoder 87, 413
 decoding 87
 decoupling capacitors 143
 delay 390
 delta-delay 353
 DeMorgan's theorem 245
 demultiplexer 472
 design flow 315
 design for testability 1042
 design review 1035
 design-rule checker 1038
 device 300
 DFT 1042
 diagnostic test 1042
 die 34
 differential amplifier 216
 differential inputs 217
 differential outputs 217
 digital 23
 digital attenuator 979
 digital logic 108
 digital phase-locked loop 98
 digital telephony 832
 diminished radix-complement system 63
 diode action 180
 diode AND gate 192
 diode breakdown 181
 diode logic 179
 diode-drop 182
 DIP package 34
 DIP switch 777
 diaphase code 100
 directed arc 650
 distance 84
 distinguished 1-cell 274, 282

divide-by- m counter 804
 division overflow 73
 don't-care bit 84, 279
 don't-care unlocked assignment operator 310
 double-buffered data 836
 downto 326
 DPLL 98
 drain 115
 DRAM 983, 997
 dual in-line-pin package 34
 dual of a logic expression 248
 duty cycle 620
 dynamic hazard 296
 dynamic power dissipation 153
 dynamic RAM 983, 997
 dynamic range 978
 dynamic-input indicator 633

E

ECL 215
 ECL 100K family 222
 ECL 10H family 219
 ECL 10K 219
 eclipse 275
 edge-triggered J-K flip-flop 641
 EDO DRAM 1003
 EEPROM 972
 electrical characteristics 206
 electrically erasable PLD 411
 electrically erasable programmable read-only memory 972
 electrostatic discharge 143
 element-type 328
 else 341, 347
 elsif 347
 emitter 185
 emitter-coupled logic 215
 EN 636
 enable input 414, 636
 encoder 440
 end 301
 end-around carry 69
 engineering design margins 125
 entity 319
 entity declaration 321
 entity-name 322
 enumerated type 324
 EPLD 409
 EPROM 971
 equality 497
 equation block 304
 equations 300
 equivalence 479
 equivalent load circuit 146
 equivalent states 664
 erasable programmable logic device 409
 erasable programmable read-only memory 971
 error 84

error correction 87
 error model 85
 error-correcting code 88
 error-correcting decoder 91
 error-detecting code 85
 ESD 143
 essential hazard 728
 essential prime implicant 274, 282
 even-parity circuit 481
 even-parity code 86
 event attribute 749, 790
 event list 353
 excess- 2^{m-1} system 64
 excess-3 code 76
 excess- B representation 64
 excitation 648
 excitation equation 648, 706
 excitation maps 671
 excitation table 658
 Exclusive NOR 479
 Exclusive OR gate 362
 Exclusive OR 479
 exit statement 349
 expression 240
 extended-data-out DRAM 1003
 external feedback 802

F

f 887
 failure 84
 failure rate 1050
 fall time 145
 false 324
 fan-in 120
 fanout 140, 196
 fast CMOS 177
 fast TTL 204
 FB 1006
 FCC Part 15 1041
 FCT 176
 FCT-T 177
 feedback input 802
 feedback sequential circuit 621
 ferroelectric RAM 984
 field-programmable gate array 33, 38, 411, 1016
 field 1035
 FIFO 1027
 fighting 165, 450
 filtering capacitors 143
 fine line 40
 finite field 845
 finite induction 244
 finite-memory machine 761
 finite-state machines 620
 first-in, first-out memory 1027
 fitter 318, 924
 fitting 318
 fixed-OR element 403

flash EPROM 972
 flash memory 972
 flip-flop 28, 625
 floating input 142
 floating state 157
 floating-gate MOS transistor 410, 971
 floorplan 1040
 flow table 713
 FOE 403
 for loop 349
 formal parameters 329
 forward resistance 182
 forward-biased diode 181
 FPGA 33, 38, 411, 1016
 frame 834
 free-running counter 809
 "front-end" 316
 full adder 501
 full subtractor 503
 function 329
 function declaration 336
 function definition 329
 function hazard 816
 function table 419
 functional block 1006
 functional verification 318
 fundamental-mode circuit 705
 fuse 398
 fusible link 408, 971

G

gain 187
 GAL16V8 405
 GAL16V8C 405
 GAL20V8 407
 gate 27, 115
 gate array 40
 gating the clock 879
 generalized DeMorgan's theorem 246
 generalized Shannon expansion theorems 362
 generate statement 338, 526
 generic array logic device 405
 generic constant 339
 generic declaration 339
 generic map 339
 glitch 292
 go/no-go test 1041
 GOTO statement 734
 Gray code 78
 group-carry lookahead 512
 group-ripple adder 509

H

half adder 501
 Hamming code 88
 Hamming distance 84
 hardware description language 26
 hardware model 1039

hazard 292
 hazard-free excitation logic 718
 HC 169
 HCT 169
 HDL 26
 helper output 428
 hexadecimal addition 58
 hexadecimal number system 51
 Hi-Z state 157
 high-impedance state 157
 high-order 50, 51
 high-speed CMOS 169
 high-speed TTL 203
 HIGH-state fanout 140, 197
 HIGH-state unit load 197
 HM62256 991
 HM6264 991
 HM628128 991
 HM628512 991
 hold time 632
 hold-time margin 771
 hysteresis 156, 452

I

I/O block 1012, 1021
 I/O pin 404
 IC 32, 113
 IC type 386
 identifier 298, 321
 IEEE Standard Graphic Symbols for Logic Functions 374
 if statement 347, 734
 imply 273
 in-circuit testing 1043
 in-system programmability 412
 includes 273
 independent error model 85
 induction step 245
 inequality 497
 infant mortality 1050
 inferred latch 789
 information bit 86
 initial state 661
 input-list 304, 312
 input state 706
 inputs 397
 instantiate 336
 integer 324
 integrated circuit 32, 113
 intermediate equation 301
 internal feedback 802
 internal state 706
 inversion bubble 110, 374
 inverted 1-out-of- n code 82
 inverter 28, 110
 IOB 1012, 1021
 irredundant sum 364
 istype 300

iterative circuit 490

J

J-K application table 674
 Johnson counter 842
 JTAG port 412
 juxtaposition 241

K

Karnaugh map 267
 keyword 321

L

label 336
 large-scale integration 35
 latch 626
 latch-up 143
 late-write SSRAM with flow-through outputs 992
 late-write SSRAM with pipelined outputs 994
 leakage current 116, 181
 least significant bit 51
 least significant digit 50
 left 828
 level shifter 214
 level translator 214
 LFSR counter 845
 library 333
 library clause 334
 line code 96
 linear encoding 978
 linear feedback shift-register 845
 literal 252
 logic diagram 370
 logic family 113
 logic value 108
 logical addition 241
 logical multiplication 240
 lookahead carry circuit 513
 loop statement 349
 low-order 50, 51
 low-power Schottky TTL 204
 low-power TTL 203
 LOW-state fanout 140, 197
 LOW-state unit load 197
 low-voltage CMOS 210
 low-voltage TTL 210
 LSB 51
 LSI 35

M

μ -law PCM 978
 m -out-of- n code 82
 m -product function 282
 m -subcube 84
 magnitude comparator 490
 main machine 701
 Manchester code 100
 mask 970

mask charge 970
 mask ROM 970
 mask-programmable ROM 970
 master 632
 master/slave J-K flip-flop 639
 master/slave S-R flip-flop 638
 matched transmission line 1057
 maximum delay 393
 maximum-length sequence generator 845
 maxterm 252
 maxterm i 253
 maxterm list 253
 MCM 41
 Mealy machine 645
 mean time between failures 887, 1052
 mechanical constraints 1040
 medium-scale integration 35
 metal-oxide semiconductor field-effect transistor
 ii3
 metastability resolution time 885
 metastable state 623
 metatheorem 247
 minimal cut set 709
 minimal product 277
 minimal sum 272
 minimize 266
 minimum delay 393
 minimum distance 86
 minimum pulse width 628
 minterm 252
 minterm i 253
 minterm list 253
 minterm number 253
 minuend 58
 module 298
 modulo- m counter 804
 modulus 804
 Moebius counter 842
 Moore machine 645
 Moore's law 41
 MOS circuits 114
 MOS transistor 113
 MOSFET 113
 most significant bit 51
 most significant digit 50
 ms 26
 MSB 51
 MSI 35
 MTBF 1052
 multichip module 41
 multiple error 85
 multiple-cycle synchronizer 891
 multiple-output prime implicant 282
 multiple-valued logic 357
 multiplexed address inputs 999
 multiplexer 464
 multiplexing 833
 multiplication dot 240

multiplying out 243
 mutual exclusion 656, 686
 mux 464

N

n -bit binary counter 805
 n -channel MOS transistor 115
 n -cube 83
 nail 1043
 NAND gate 111
 NAND-NAND circuit 263
 NBUT gate 762
 negate 376
 negative logic 108
 negative-edge-triggered D flip-flop 634
 nested expansion formula 54
 net 1037
 net list 318, 1037
 next statement 350
 next-state logic 644
 nibble 53
 nibble mode 1003
 NMOS transistor 115
 node 650
 noise margin 29
 non-return-to-zero 96
 non-return-to-zero invert-on-1s 98
 noncode word 85
 noncritical race 712
 nonrecurring engineering cost 39
 nonvolatile memory 961, 983
 NOR gate 111
 NOR-NOR circuit 264
 normal term 252
 NOT 300
 NOT gate 28, 110
 NOT operation 240
 npn transistor 185
 NRE cost 39
 NRZ 96
 NRZI 98
 ns 26
 null statement 332

O

OAI gate 124
 observability 1043
 octal number system 51
 odd-parity circuit 481
 odd-parity code 86
 OFF 186
 off-set 253
 ON 187
 on-set 253
 one-hot assignment 668
 one-time programmable ROM 972
 one-to-one mapping 413
 ones' complement 63

ones'-complement addition 69
 ones'-complement checksum code 95
 ones'-complement subtraction 70
 open-collector output 203
 open-drain bus 164
 open-drain output 160
 operator overloading 330
 OR 300
 OR gate 28, 110
 OR operation 241
 OR plane 407
 OR-AND circuit 264
 OR-AND-INVERT 124
 other declarations 300
 others 328, 342
 OTP ROM 972
 output enable 126
 output equation 650, 707
 output logic 644
 output logic macrocell 797
 output polarity 405
 output stage 192
 output timing skew 857
 output-coded state assignment 645, 697
 output-disable time 976, 988
 output-enable gate 403
 output-enable input 974
 output-enable time 975, 988
 output-hold time 976, 988
 output-list 304, 312
 outputs 397
 overall fanout 140, 197
 overdrive 1044
 overflow 66
 overflow rule 67
 override 1044

P

p-channel MOS transistor 115
P-set 357
 package 334
 packed-BCD representation 75
 pad ring 210
 page-mode read cycle 1003
 page-mode write cycle 1003
 PAL16L8 401
 PAL16R6 795
 PAL16R8 792
 PAL20L8 404
 PALCE16V8 407
 PALCE20V8 407
 PAL device 37, 401
 parallel data 96
 parallel-in, parallel-out shift register 826
 parallel-in, serial-out shift register 825
 parallel-to-serial conversion 825
 parity bit 86
 parity-check matrix 88

partial product 71
 partitioner 924
 parts list 1037
 parts qualification 1036
 passive pull-up 160
 PCB 40
 PCB traces 40
 PECL 222
 permanent failure 84
 phase splitter 192
 physical model 1039
 pin declaration 300
 pin diagram 34
 pin list 1037
 pin locking 1016
 pin number 388, 1036
 pinout 34
 pipelined output 646
 PLA 397
 place and route 318
 PLA 37
 PLD 37
 PLD programmer 411
 PMOS transistor 115
pnp transistor 186
 port 322
 port declaration 322
 port map 337
 positional number system 49
 positive ECL 222
 positive logic 108
 positive-edge-triggered D flip-flop 632
 positive-logic convention 239
 postponed-output indicator 639
 postulate 239
 power-dissipation capacitance 153
 power-down input 974
 power-supply rail 131
 PFI 634
 precedence 241
 precharge voltage 998
 predefined types 324
 preset 626, 634
 primary inputs and outputs 490
 prime 240
 prime implicant 273
 prime-implicant table 289
 primitive flow table 720
 principle of superposition 1057
 printed-circuit board 40
 printed-wiring board 40
 priority 442
 priority encoder 442
 procedure 333
 process 344
 process statement 344
 product component 519
 product term 252, 397

product-of-sums expression 252
 product-term allocation 1008
 product-term allocator 1009
 programmable array logic device 37, 401
 programmable logic array 37, 397
 programmable logic device 37
 programmable read-only memory 971
 programmable switch element 1024
 programmable switch matrix 1024
 PROM 971
 PROM programmer 411, 971
 propagation delay 151, 627
 PSE 1024
 PSM 1024
 pull-up resistor 160
 pull-up-resistor calculation 165
 pulse input 751
 pulse-mode circuit 751
 pulse-triggered flip-flop 639
 pure 613
 push-pull output 192
 PWB 40

Q

quiescent power dissipation 153
 Quine-McCluskey algorithm 283

R

race 711
 radio-frequency emission 1041
 radix 50
 radix point 50
 radix-complement system 60
 RAM 983
 random-access memory 983
 range 307, 332
 range-type 327
 RAS-only refresh cycle 1000
 RC time constant 148
 read cycle 1000
 read-modify-write cycle 1002
 read-only memory 960
 read/write memory 983
 realization 261
 realize 261
 recommended operating conditions 206
 recovery time 629
 rectangular sets of 1s 273
 redundant prime implicant 290
 reference designator 388, 1036
 reflected code 78
 reflection 1055
 reflection coefficient 1056
 refresh cycle 998
 reg 731
 register 780
 register-transfer language 1034
 registered carry output 820

registered output 731
 relation 309
 relational operators 309, 342
 reliability 1048
 reserved words 321
 reset 626
 resistive load 131
 resolution function 461
 result 329
 return 330
 return-to-zero 99
 reverse-biased diode 181
 right 828
 ring counter 839
 ringing 1060
 ripple adder 501
 ripple counter 805
 rise time 145
 ROM 960
 routing 1041
 row address strobe 999
 row latch 1000
 row-address register 1000
 running disparity 99
 running process 345
 RWM 983
 RZ 99

S

S-Flatch 626
 S-R latch with enable 630
 S-set 357
 saturated 187
 saturation region 187
 saturation resistance 187
 scan capability 636
 scan chain 637
 scan method 1047
 scan mode 1047
 scan-path method 1047
 schematic capture 1035
 schematic diagram 370
 schematic editor 1035
 Schmitt-trigger input 156
 Schottky diode 190
 Schottky transistor 190
 Schottky TTL 204
 Schottky-clamped transistor 190
 SDRAM 1003
 secondary essential prime implicant 276
 security fuse 413
 selected signal-assignment statement 342
 self-complementing code 76
 self-correcting counter 840
 self-correcting Johnson counter 844
 self-correcting ring counter 840
 self-dual logic function 364
 self-timed systems 908

- semiconductor diode 113, 180
- semicustom IC 39
- sense amplifier 998
- sensitivity list 345
- sequential circuit 28, 110
- sequential multiplier 520
- sequential signal-assignment statement 345
- serial binary adder 865
- serial channel 832
- serial data 96
- serial input 825
- serial output 825
- serial-access memory 985
- serial-in, parallel-out shift register 825
- serial-to-parallel conversion 826
- series termination 1062
- set 307, 626
- set-reset latch 626
- setup time 632
- setup-time margin 771
- seven-segment decoder 438
- seven-segment display 436
- shift register 825
- shift-and-add multiplication 71
- shift-and-add multiplier 870
- shift-and-subtract division 73
- shift-register counter 838
- sign bit 59
- sign extension 62
- signal declaration 323
- signal flags 384
- signal name 1036
- signal path 151
- signed division 73
- signed multiplication 72
- signed numbers 68
- signed-magnitude adder 59
- signed-magnitude subtractor 60
- simulation 1038
- simulation cycle 353
- simulation time 352
- simulator 1038
- single error 85
- single stuck-at fault model 313
- single stuck-at fault 1042
- single-ended input 218
- sinking current 135, 194
- slave 633
- small-scale integration 34
- SMT 40
- sneak path 966
- software model 1039
- source 115
- source termination 1062
- sourcing current 135, 195
- space/time trade-off 833
- specification 29
- speed-power product 173
- S-R latch 626
- $\bar{S}$ - $\bar{R}$ latch 626
- SRAM 983
- SRAM cell 986
- SSI 34
- SSRAM 992
- stable state 623
- stable total state 706
- standard cells 39
- standard-cell design 39
- standby mode 974
- state 28
- state 619
- state adjacency diagram 722
- state assignment 658
- state diagram 647, 650
- state memory 644
- state minimization 658
- state name 649
- state table 110, 650, 706
- state variable 619
- state-machine decomposition 701
- state-machine description language 733
- state-vector 733
- state/output table 647, 650
- static power dissipation 153
- static RAM 983
- static-0 hazard 293
- static-1 hazard 293
- static-column mode 1003
- std_logic 325
- std_logic_vector 329
- steady-state behavior 292
- storage time 190
- stray capacitance 146
- string 299, 328
- structural description 337
- structural design 338
- structural model 1038
- structured logic device description 371
- stupid mistakes 1041
- submachines 701
- subtractor 500
- subtrahend 60
- subtype 325, 460
- sum term 252
- sum-of-products expression 252
- surface-mount technology 40
- suspended process 345
- switching algebra 239
- switching characteristics 206
- symmetric output drive 170
- sync pulse 833
- synchronization pulse 833
- synchronization signal 97
- synchronizer 880
- synchronizer failure 883
- synchronizing sequence 745

synchronous counter 806
 synchronous DRAM 1003
 synchronous parallel counter 807
 synchronous serial counter 807
 synchronous SRAM 992
 synchronous system 866
 syndrome 92
 synthesis 318

T

t 887
 T flip-flop 642
 T flip-flop with enable 643
 t_{AA} 975, 988
 t_{ACS} 975, 988
 t_{AH} 989
 t_{AS} 989
 t_{clk} 885
 t_{comb} 885
 t_{CSW} 989
 t_{DH} 989
 t_{DS} 989
 t_{OE} 975, 988
 t_{OH} 976, 988
 t_{OZ} 976, 988
 t_{WP} 989
 temporary failure 84
 termination impedance 1056
 test benches 31, 317
 test enable input 636
 test fixture 1043
 test input 637
 test output 637
 test points 1043
 test vectors 301, 1042
 test-generation program 1043
 test_vectors 301, 312
 testing 1041
 thermal concerns 1040
 three-state buffer 158, 449
 three-state bus 158
 three-state driver 449
 three-state enable 449
 three-state output 158
 Thévenin equivalent 133
 Thévenin resistance 133
 Thévenin termination 1061
 Thévenin voltage 133
 TI 637
 tick 644
 timeslot 835
 timing diagram 371, 390
 timing margin 771
 timing skew 729
 timing table 391
 timing verification 318, 1038
 timing verifier 1038
 title statement 298

T 887
 T_0 637
 t_0 326
 total number of states 665
 total state 706
 totem-pole output 192
 t_{PHL} 151
 t_{PLH} 151
 t_{PLH} 885
 t 885
 trace 1058
 transient behavior 292
 transistor simulation 187
 transistor-transistor logic 113
 transition equation 649
 transition expression 655
 transition frequency 153
 transition list 689
 transition p-term 691
 transition s-term 693
 transition-sensitive media 98
 transition table 649, 706
 transition time 145
 transition/excitation table 670
 transition/output table 653
 transmission gate 155
 transmission line 1054
 transparent latch 631
 tri-state output 158
 true 324
 truth_table 110, 250, 304
 truth_table 304
 t_{setup} 885
 TTL 113
 TTL compatible 169, 170, 177
 TTL compatible with TTL V_{OH} 177
 TTL load 175
 turn-around penalty 995
 twisted-ring counter 842
 two-dimensional code 93
 two-dimensional decoding 967
 two-pass logic 429
 two-phase latch design 877
 two-phase latch machine 751
 two's complement 62
 two's-complement addition 64
 two's-complement multiplication 72
 two's-complement subtraction 67
 type 323
 typical delay 393

U

unlocked assignment operator 301
 unlocked truth-table operator 304
 unconstrained array type 329
 undershoot 1059
 unidirectional error 96
 unidirectional shift register 828
 unresolved type 460

unsigned binary multiplication 71
 unsigned division 73
 unsigned numbers 68
 unstable total state 706
 unused states 665
 up/down counter 815
 use clause 334
 user-defined type 324

V

variable 323, 345
 variable definition 323
 variable-assignment statement 345
 vee 241
 verification 317
 very large-scale integration 37
 very high-speed CMOS 170
 VHC 170
 VHCT 170
 VHDL 314
 VHDL compiler 317
 VHDL simulator 317
 VHDL synthesis tools 315
 VHDL text editor 317
 VHDL-87 315
 VHDL-93 315
 VLSI 37
 volatile memory 983

W

wafer 34
 wait statement 352
 WE-controlled write 989
 wear-out 1051
 weight 49
 weight of MSB 62
 weighted code 76
 when 341
 WHEN statement 303
 while loop 350
 wired AND 165
 wired logic 164
 wire type 1036
 WITH statement 745
 word line 965
 worst-case delay 397
 write cycle 987, 1002
 write enable 1002
 write-enable (WE) input 984
 write-pulse width 989

X

XNOR 300, 479
 XOR 300, 362, 479

Z

Z₀ 1054
 ZBT SSRAM 995

zener diode 182
 zero-bias-turnaround SSRAM 995
 zero-code suppression 100

A

автомат с конечной памятью 761
 автоматизированное конструирование 1033
 автоматизированное проектирование 30, 1033
 автоматический тестер 1043
 автомат Милла 645
 автомат Мура 645
 адресные входы 960
 аксиомы 239
 активная область 186
 активная омическая нагрузка 131
 активное подтягивание 160
 активный режим 974
 активный уровень 376
 активный сигнал высокого уровня 376
 активный сигнал низкого уровня 376
 алгебраический оператор 240
 алгебра переключений 239
 алгоритмический автомат 752
 алгоритм Куинна-Мак-Класки 283
 АЛУ 509
 аналоговые устройства 23
 анод 180
 аппаратная модель 1039
 аргумент 329
 арифметическо-логическое устройство 509
 арифметический сдвиг 858
 архитектура 319
 асимметричный выходной каскад 170
 асинхронные входы 634
 асинхронные входные сигналы 880
 асинхронные статические ОЗУ 992
 атрибут 332

Б

база 185
 байт 53
 балансный код 99
 батификация 533
 безразличные комбинации 279
 библиотека 333
 библиотеки компонентов 1035
 биполярный плоскостной транзистор 113
 биполярный транзистор 185
 бистабильная схема 622
 бит 50
 бит с безразличным значением 84
 бит/с 96
 битовая ячейка 96

блок равенств 304
 блок-схема 370, 372
 блок ввода/вывода 1012, 1021
 борьба 165, 450
 броски тока 142
 буфер с тремя состояниями 158, 449
 буферные усилители 108
 быстродействующие КМОП-схемы 169
 быстродействующие ТТЛ-схемы 203, 204

В

ввод схемы 1035
 ведомая защелка 633
 ведущая защелка 632
 вектор состояния 733
 векторы проверок 301
 вентили 27
 решетки вентиля, программируемые в процессе эксплуатации 33
 вентили разрешения выходного сигнала 403
 вентиль BUT 363
 вентиль NBUT 762
 вентиль И 27
 вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ 362, 479
 вентиль ИСКЛЮЧАЮЩЕЕ ИЛИ-НЕ 479
 вентиль ЭКВИВАЛЕНТНОСТЬ 479
 вентиляжные матрицы 40
 верификатор временных соотношений 1038
 верификация 317
 вес 49
 вес старшего бита 62
 взаимно-однозначное соответствие 413
 взаимное исключение 686
 взвешенный код 76
 «включение» 273
 «внешний план» 316
 внешние входы и выходы 490
 внешняя обратная связь 802
 внутреннее состояние 706
 «внутренний план» 318
 внутренняя обратная связь 802
 внутрисхемное тестирование 1043
 возбуждение 648
 возможность опроса 636
 волновое сопротивление 1054
 восьмеричная система счисления 51
 временные неисправности 84
 время в модели 352
 время восстановления 629
 время выхода из метастабильности 885
 время доступа по входу выбора кристалла 975, 988
 время доступа по адине адреса 975, 988
 время запрещения выдачи данных 976
 время запрещения выхода 988
 время нарастания 145
 время переходного процесса 145
 время разрешения выдачи данных 975
 время разрешения выхода 988
 время снада 145
 время удержания 190, 632
 время удержания адреса после окончания записи 989
 время удержания данных на выходе 976
 время удержания данных после окончания записи 989
 время удержания сигнала на выходе 988
 время установления 632
 время установления адреса до начала записи 989
 время установления данных до окончания записи 989
 время установления сигнала «выбор кристалла» до окончания записи 989
 временная диаграмма 371, 390
 временной перекося 729
 вспомогательный выход 428
 протекающий ток 135
 вход выбора кристалла CS 974
 вход для каскадного включения 492
 вход разрешения 449, 636
 вход разрешения выхода OE 974
 вход разрешения записи WE 984
 вход разрешения тестирования 636
 вход снижения потребляемой мощности 974
 вход со стороны обратной связи 802
 вход условий 868
 входной сигнал разрешения 414
 выводы
 И/О-выводы 404
 выполнение процесса 345
 выражение 240
 перехода 655
 взаимно исключающие 656
 исчерпывающие в совокупности 656
 вырезка из массива 329
 высокая степень интеграции 35
 высокоомное состояние 157
 протекающий ток 135
 выход данных 960
 выход регистровый 731
 выход с открытым коллектором 203
 выход с тремя состояниями 158
 выходная логика 644
 выходной каскад 192
 вычисление точного дополнения 61
 вычисление сопротивления резистора R , соединяющего выход схемы с шиной питания 165
 вычитаемое 60
 вычитание в дополнительном коде 67
 вычитание чисел в обратном коде 70
 вычитающее устройство 60, 500

Г

генератор последовательности максимальной длины 845

генерирование сигнала перебора 505
гистерезис 156, 452
главный автомат 701
глупые ошибки 1041
глюк 292
гонка 711
графичные входы 490
граничные выходы 490
грейфер 1044

Д

дважды буферизованные данные 836
двоичная точка 50
двоичная цифра 108
двоично-десятичные числа 74
двоично-десятичный дешифратор 416
двоично-пятеричный код 76
двоичное вычитание 57
двоичный дополнительный код 62
двоичные числа 50
двоичные счетчики
 n-разрядные 805
 преобразование двоичных чисел в
 восмеричные 52
двоичный шифратор 440
двоичный умножитель частоты 913
двойственное логическое выражение 248
двумерное декодирование 967
двумерный код 93
двунаправленная линия 454
двунаправленный регистр сдвига 828
двуполярный код с возвратом к нулю 100
двухтактный JK-триггер 639
двухтактный SR-триггер 638
двухтактный выходной каскад 192
двухфазный автомат-защелка 751
двухфазный код 100
двучленные операторы 243
действительные параметры 329
дескадные счетчики 810
декодер 87
декодер, исправляющий ошибки 91
декодирование 87, 414
деление чисел без знака 73
деление чисел со знаком 73
деление путем сдвига и вычитания 73
делитель частоты на *m* 804
демпфирующие диоды 1060
демультиплексор 472
десятичный дешифратор 416
дешифратор 413
диагностическое тестирование 1042
диаграмма состояний 650, 647
диаграмма смежности состояний 722
диапазон 307
динамический диапазон 978
динамический источник опасности 296
динамические ОЗУ 983, 997

динамические ОЗУ с увеличенным временем
 доступности 1003
динамическая рассеиваемая мощность 153
диод, смещенный в обратном направлении 181
диод Шоттки 190
диодная логика 179
диодная схема И 192
директива @CARRY 516
дифференциальный усилитель 216
дифференциальные входы 217
дифференциальные выходы 217
липная линия 1054
 согласованная длинная линия 1057
длительность импульса записи 989
длительность бита 96
дополнение 240
дополнение логического выражения 246
дополнения в десятичной системе 60
дополнительный код 64
дорожки на печатной плате 40, 1058
драйвер с тремя состояниями 449
дребезг контакта 777
другие объявления 300

Е

едиичная нагрузка при высоком уровне 197
едиичная нагрузка при низком уровне 197
едиичный статический источник опасности
 293
единовременные затраты на проектирование
 39
емкостная нагрузка 146
емкость, определяющая рассеиваемую
 мощность 153

З

задержка 390
задержка в наихудшем случае 396
задержка распространения 151, 392, 627
засм 69
заказная БИС 39
закон Мура 41
запас по времени 771
запас по времени удержания 771
запас по времени установления 771
запас помехоустойчивости 29
запас помехоустойчивости по постоянному
 току 131, 195
запись по сигналу CS 989
запись по сигналу WE 989
занятое состояние в форме выходного кода 645
зарезервированные слова 321
затвор 115
захват единиц 640
захват нулей 640
зашелка 626
 D-зашелка 631
 SR-зашелка 626

SR-зашелка 629
 SR-зашелка с входом разрешения 630
 строки 1000
 защелкивание 143
 защита конфигурации соединений 413
 защита от дребезга 777
 звездочка * 646
 звон 1060
 знак дизъюнкции 241
 знак умножения — точкой 240
 знаковос расширение 62
 знаковый бит 59
 значения указанного типа 327

И

иглычатые контакты 1043
 идентификаторы 298, 321
 иерархический принцип 384
 избирательное присваивание сигналу его значения 342
 избыточные простые импликанты 290
 износ 1051
 ИКМ
 А-ИКМ 978
 μ -ИКМ 978
 имена состояний 649
 импликанта схемы со многими выходами 282
 импликация 273
 импульсные входы 751
 импульсные схемы 751
 импульсы синхронизации 833
 имя архитектуры 322
 имя объекта 322
 имя сигнала 376, 1036
 инвертор 28, 110
 индекс массива 327
 индикатор задержки срабатывания 638
 интегральная схема 32, 113
 интенсивность отказов 1050
 интерфейсный блок 210
 информационные биты 86
 ИС 32, 113
 исправление ошибки 87
 испытание на отказ 1050
 испытательные стенды 317
 исток 115
 источник опасности 292
 исчерпывающее включение 686
 итерационная схема 490

К

кадр 834
 каноническая сумма 253
 каноническое произведение 253
 карта возбуждения 671
 карта Карно 267
 карта Карно с 5 переменными 366, 671
 катод 180

ключевые слова 321
 КМОП-буфер с тремя состояниями 158
 КМОП-логика 116
 КМОП-нагрузка 175
 КМОП-схема 114
 КМОП-схема с повышенным быстродействием 170
 КМОП-схемы серии 4000 169
 код 74
 код 2421 76
 код 8421 76
 код 8В10В 83
 код "1 из 10" 77
 код "1 из n " 81
 код "m из n" 82
 код без возврата к нулю 96
 код без возврата к нулю с инверсией 98
 код Грея 78
 код, обнаруживающий ошибки 85
 код с возвратом к нулю 99
 код с избытком 3 76
 код с контрольной суммой 95
 код с контрольной суммой, образуемой путем сложения в обратном коде 95
 код с проверкой на четность 86
 код с проверкой на четность 86
 код с одним контрольным битом 86
 код состояния 665
 код Хэмминга 88
 кодирование с чередованием полярности 100
 кодирование состояний выходными комбинациями 697
 кодовое уплотнение 978
 кодовые слова 74
 коллектор 185
 кольцевые счетчики 839
 командные входы 868
 комбинационная схема 27, 110
 комбинационный выходной сигнал переноса 819
 комбинационный умножитель 518
 комментарии 300, 321
 коммутатор 488
 коммутатор значений 490
 коммутатор языка ABEL 298
 коммутатор языка VHDL 317
 комплементарные МОП-схемы 114
 компонент произведения 519
 компоновка 318
 конвейерный выход 646
 конечные автоматы 620
 конечные поля 845
 консенсус 244, 294
 константы 326
 контрольная сумма 95
 контрольные точки 1043
 контрольный бит 86
 корпус с двухрядным расположением выводов 34

корректирующий код 88
 коэффициент заполнения 620
 коэффициент разветвления по выходу 140, 196
 для переменного тока 141
 для постоянного тока 140
 при низком уровне 140, 197
 при высоком уровне 140, 197
 коэффициент объединения по входу 120
 коэффициент отражения ρ 1056
 коэффициент усиления тока 187
 кристалл 34
 критическая гонка 712
 критические пути прохождения сигналов 1040
 кружок инверсии 374
 куб

л-мерный 83

Л

линейное кодирование 978
 линия бита 965
 линия слова 965
 литерал 252
 логика возбуждения 718
 логика возбуждения, свободная от источников опасности 718
 логика переходов F 644
 логика с двумя проходами 429
 логическая схема 370
 логическая схема И 27
 логическая схема ИЛИ 28
 логическая схема НЕ 28
 логическая функция, двойственная по отношению к самой себе 364
 логические схемы на переключателях тока 215
 логический ключ 155
 логическое выражение 376
 логическое значение 108
 логическое проектирование по принципу «инверсия к инверсии» 379
 логическое равенство 376
 логическое сложение 241
 логическое умножение 240
 логическое ядро 210
 ложе из-под язычков контактов 1043

М

макроячейки выходной логики 797
 максимальная задержка 393
 макстерм 252
 макстерм τ 253
 малая степень интеграции 34
 маломощные TTL-схемы 203
 с транзисторами Шоттки 204
 манчестерский код 100
 маска 970
 масочный ПЗУ 970
 массив 327
 массивы-литералы 328

матрица И 407
 матрица ИЛИ 407
 матрица программируемых переключателей 1024
 междускадные входы и выходы 490
 мертвое время 451
 метастабильное состояние 623
 метатеорема 247
 метод ветвления 276
 метод сканирования 1047
 по заданному пути 1047
 механические ограничения 1040
 микросекунда 26
 микросхемы памяти, работающие по принципу «первым вошел – первым вышел» 1027
 минимальная длительность импульса 628
 минимальная задержка 393
 минимальная сумма 272
 минимальное произведение 277
 минимальное расстояние 86
 минимальное сечение 709
 минимизация 266
 минимизация числа состояний в таблице «состояние/выход» 658
 минтерм 252
 минтерм τ 253
 мкс 26
 младший бит 51
 младший разряд 50
 многозначная логика 357
 многократные ошибки 85
 многокристальные модули 41
 многотактное синхронизирующее устройство 891
 множество
 d-множество 279
 выключений 253
 полное 363
 включений 253
 модели ошибок 85
 моделирование 1038
 моделирующая программа 1038
 моделирующая программа VHDL 317
 моделирование поведения транзистора 187
 модель компонента 1037
 модель одиночной неисправности типа запитания 313
 модуль счета 804
 монтажная логика 164
 «монтажное И» 165
 МОП-транзистор 113
 МОП-транзистор с каналом n -типа 115
 МОП-транзистор с каналом p -типа 115
 МОП-транзисторы с плавающим затвором 410, 971
 мощность
 рассеиваемая в режиме покоя 153
 CV^2f -мощность 154

мультиплексирование 833
 мультиплексированные адресные входы 999
 мультиплексор 464

Н

наблюдаемость 1043
 набор 307
 нагрузка по переменному току 146
 нагрузка по постоянному току 131
 нагрузочная способность 140
 надежность 1048
 наносекунда 26
 направленные дуги 650
 направление источника в эквивалентной схеме 133
 напряжение предварительного уровня 998
 напряжение Тесенина 133
 настраиваемые константы 339
 насыщенный транзистор 187
 начальное состояние 661
 неиспользуемые состояния 665
 неисправность 84
 нескритическая гонка 712
 неприводимая сумма 364
 неравенства 497
 неразрешенный тип 460
 несимметричный вход 218
 неактивируемый оператор таблицы истинности 304
 неустойчивое равновесие 623
 неустойчивое состояние в целом 706
 низковольтные уровни КМОП-схем 210
 низковольтные уровни схем, совместимых с ТТЛ 210
 номер минтерма 253
 номера выводов 1036
 нормальный терм 252
 нс 26
 нулевой статистический источник опасности 293

О

область насыщения 187
 обобщенная теорема Де Моргана 246
 обобщенная теорема расапливания Шеннопа 362
 обратная аннотация 1036
 обратный код 63
 обратный код "1 из 1" 82
 общая топологическая структура 1040
 общая шина 164
 объект 319
 объявление выводов 300
 объявление компонента 337
 объявление констант 326
 объявление объекта 321
 объявление общности 339
 объявление переменных 323
 объявление портов 322
 объявление сигнала 323

объявление функции 336
 ограничения 318
 ограниченная индукция 244
 однократная ошибка 85
 однократные запитания 1042
 однократно программируемые ПЗУ 972
 односторонняя ошибка 96
 односторонние регистры сдвига 828
 ОЗУ 983
 окно принятия решения 886
 окончательная нагрузка 1061
 оперативная память 983
 оператор case 348
 оператор component 336
 оператор exit 349
 оператор generate 338, 526
 оператор goto 734
 оператор if 347, 734
 оператор loop 349
 оператор next 349
 оператор null 332
 оператор wait 352
 оператор when 303
 оператор with 741
 оператор конкатенации 329
 оператор инициализируемого присваивания = 301
 оператор инициализируемого присваивания не инициализируемых во внимание значений 310
 оператор отношений 309, 342
 оператор присваивания значения переменной 345
 оператор процесса 344
 операция И 241
 операция ИЛИ 241
 операция НЕ 240
 описание схемы 371
 определение архитектуры 321, 322
 определение функции 329
 определяемые пользователем типы 324
 основание 50
 основной шаг 244
 особая клетка, содержащая 1 274, 282
 отказы в начальный период эксплуатации 1050
 открытый сток на выходе 160
 отношение 309
 отражение 1055
 отрицательная логика 108
 отрицательный выброс 1059
 отсечка 186
 оценка проекта 1035
 ошибки 84
 ошибки независимые 85

П

падение напряжения на открытом диоде 182
 пакет 334
 память с последовательным доступом 985
 память состояния 644

- память с произвольным доступом 983
- память с чтением и записью 983
- память типа SRAM 992
 - с задержанной записью и конвейерными выходами 994
 - с задержанной записью и сквозными выходами 992
 - с асинхронной с нулевым временем смены режима 995
- паразитная емкость 146
- паразитный импульс 292
- параллельный оператор 336
- параллельный сигнальный оператор присваивания 341
- параллельный синхронный счетчик 807
- параллельный формат 96
- параметр компониста 1035
- пассивное подтягивание 160
- перегрузка операторов 330
- передача сигнала переноса 505
- переключаемые перемычки 408
- переключатели
 - DIP-переключатели 777
- перекрытие 275
- переменная 323, 345
- переменные состояния 619
- перемножение путем сдвига и сложения 71
- перенос 68
- переполнение 66
- переполнение при делении 73
- перепрограммируемые вентильные матрицы 411
- перестраиваемые логические блоки 1017
- переход
 - p-n* переход 180
- переходный процесс 292
- перечислимые типы 324
- период тактового сигнала 620
- печатная плата 40
- ПЗУ 960
- плавающее состояние 157
- плавающий вход 142
- плавающие перемычки 398, 971
- плата за смену режима 994
- ПЛИМ 37, 397
- ПЛИУ 37
- поведение схемы в установившемся режиме 292
- поведенческая модель 1038
- поведенческий проект 344
- поведенческое описание 344
- поглощение 244
- подавление нулевого кода 100
- подавтоматы 701
- подкуб
 - m*-мерный 84
- подмена сигнала 1044
- подтипы 325, 460
- подтягивающий резистор 160
- позиционная система счисления 49
- позиционное обозначение 1036
- «покрытые» 0187 273
- полупроводниковый транзистор со структурой «металл-окисел-полупроводник» 113
- полная сумма 274
- полное ввчиташее устройство 503
- полное число состояний 665
- полный дешифратор 414
- полный коэффициент разветвления по выходам 140
- полный набор тестов 1043
- полный сумматор 501
- положительная логика 108, 239
- полубайтовый режим выборки 1003
- полубайт 53
- полузаказные ИС 39
- полупроводниковая пластина 34
- полупроводниковый диод 113, 180
- полусумматор 501
- поля 1035
- полярность выходного сигнала 405
- поразрядное дополнение до девяти 63
- порты 322
- порты JTAG 412
- порядковый номер 388
- последовательно включенные синхронизирующие устройства 892
- последовательное согласование 1062
- последовательностная схема 28, 110
- последовательностная схема с обратной связью 621
- последовательные умножители 520
- последовательные форматы 96
- последовательный ввод 825
- последовательный вывод 825
- последовательный двоичный сумматор 865
- последовательный канал 832
- последовательный сигнальный оператор присваивания 345
- последовательный синхронный счетчик 807
- постоянное запоминающее устройство 960
- постоянная 84
- постоянная времени 148
- постулаты 239
- постулаты Хаттингтона 355
- потоковое описание 341
- потоковое проектирование 341
- правило обнаружения переполнения 67
- предельные значения 206
- предельные технические характеристики 125
- предложение use 334
- предложение library 334
- предопределенные типы 324
- предполагаемая защелка 789
- представление в виде куба 284
- представление с избытком *B* 64

представление числа в виде итерационных
 вложений 54
 преобразование арифметических чисел в
 аналитические 52
 преобразование десятичного числа D к
 представлению в системе счисления с
 основанием r 54
 преобразование параллельного кода в
 последовательный 825
 преобразование последовательного кода в
 параллельный 826
 преобразователь уровня 214
 префикс "0x" 53
 привязка выводов 1016
 признак event 749, 790
 признак рассогласования 99
 примитивная таблица потока 720
 принудительное задание 1044
 принцип суперпозиции 1057
 принципиальная схема 370
 приоритет 241, 441
 приоритетный арифметик 442
 приостановка процесса 345
 присутствие сигнала 376
 причинная обусловленность связи 390
 пробой диода 181
 проверка временных соотношений 318, 1038
 проверка на соответствие техническим
 условиям 1036
 проверочная матрица 88
 проверочные биты 87
 проверочные векторы 1042
 программа генерирования тестов 1043
 программа компоновки 318, 924
 программа проверки правильности схемы 1038
 программа разбиения 924
 программатор PROM 971
 программаторы ПЗУ 411
 программаторы ПЛУ 411
 программирование ПЗУ с помощью
 фотошаблонов 970
 программируемая в условиях эксплуатации
 матрица 1016
 программируемость в системе 412
 программируемые логические матрицы 37, 397
 программируемые логические устройства 37
 программируемые матричные логические
 устройства PAL 401
 программируемые ПЗУ 971
 программируемые логические устройства
 решетчатой структуры 37
 программные средства тестирования 31, 317
 программные модели 1039
 программные средства синтеза на основе VHDL
 315
 проектирование на уровне печатной платы 46
 проектирование по принципу двухфазных
 защелок 877

проектирование, предусматривающее
 возможность тестирования 1042
 проектирование с использованием стандартных
 элементов 39
 прозрачная защелка 631
 произведение
 m -произведение 282
 быстрое действие на мощность 173
 двух двоичных чисел без знака 71
 сумм 252
 промежуточное равенство 301
 простая импликация 273
 пространственно-временной обмен 833
 процедура 333
 процесс 344
 процессор языка ABEL 298
 прямое кодирование 668
 прямоугольные наборы единиц 271
 путь прохождения сигнала 151

Р

работа в непрерывном режиме 809
 равенства 300, 497
 радиочастотные излучения 1041
 разбиение конечных автоматов на блоки 701
 разброс задержек тактового сигнала 874
 разводка 1041
 развязывающие конденсаторы 143
 размещение и разводка 318
 «разнесение множителя по слагаемым» 243
 «разнесение слагаемого по сомножителям» 243
 «разрешение выхода» 126
 разрешение записи WE_L 1002
 разрешение тактового сигнала 636
 расположение компонента 1036
 распределение термов-произведений 1008
 распределители термов-произведений 1009
 расстояние 84
 расстояние Хэмминга 84
 расходы на изготовление масок 970
 расхождение выходных сигналов по времени 857
 реализации 261
 реверсивный счетчик 815
 регистр 780
 регистр адреса столбца 1000
 регистр адреса строки 1000
 регистр сдвига 825
 регистр сдвига с обратной связью 845
 регистр сдвига с параллельным вводом и
 параллельным выводом 825
 режим выборки по словам 1003
 режим ожидания 974
 режим сканирования 1047
 режим статической выборки по столбцам 1003
 результирующий коэффициент разветвления по
 выходу 197
 рекомендуемые условия работы 206
 рефлексный код i реф 78

решетки вентиля, программируемые в процессе эксплуатации 38

С

- самоодинающийся код 76
- самокорректирующийся кольцевой счетчик 840
- самокорректирующийся счетчик 840
- самокорректирующийся счетчик Джонсона 844
- самосинхронизирующаяся система 908
- сбалансированный по постоянному току поток данных 99
- СБИС 37
- сброс 626, 634
- сверхбольшая интегральная схема 37
- сдвиг влево 828
- сдвиг вправо 828
- семейство логических схем 113
- семейство логических схем на биполярных транзисторах 113
- семисегментный дешифратор 438
- семисегментный индикатор 436
- сечения 709
- сигнал переноса на регистравом выходе 820
- сигнал не подан или отсутствует 376
- сигнал синхронизации 97
- сигнальный код 96
- символ инверсии 110
- симметричный выходной каскад 170
- синдром 92
- синтез 318
- синфазный сигнал 217
- синхронизирующая последовательность 745
- синхронизирующее устройство 880
- синхронная система 866
- синхронный счетчик 806
- синхронные динамические ОЗУ 1003
- синхронные статические ОЗУ 992
- система представления чисел в форме дополнения 60
- система с избытком 2^{m-1} 64
- система с поразрядным дополнением 63
- скорость передачи, измеряемая числом битов в секунду 96
- скрученный кольцевой счетчик 842
- скрытые макроячейки 1007
- скрытый путь 966
- сложение двоично-десятичных чисел 75
- сложение двоичных чисел 56
- сложения двоичных чисел в обратном коде 69
- сложение с сохранением переноса 520
- сложение шестнадцатеричных чисел 58
- «сложные» ПЛУ 411
- сложные программируемые логические устройства 1005
- смежные состояния 723
- смещение 64
- совместимые с ТТЛ быстродействующие КМОП-схемы 169
- совместимые с ТТЛ КМОП-схемы с повышенным быстродействием 170
- совместимые с ТТЛ сверхбыстродействующие КМОП-схемы 176, 177
- совокупности
- Р- и S- 357
- согласование со стороны источника 1062
- согласованная длинная линия 1057
- согласование обимпах сигналов с активным уровнем 376
- соединение 1037
- создание устройств на основе специализированных ИС 47
- сопротивление в эквивалентной схеме 133
- сопротивление нагрузки 1056
- сопротивление насыщения 187
- сопротивление открытого диода 182
- сопротивление Тевенина 133
- составные ПЛУ 37
- состояние 28, 619
- состояние в целом 706
- состояние входа 706
- состояние Hi-Z 157
- специализированные ИС 39
- список входных сигналов 304
- список выводов 1037
- список выходных сигналов 304
- список компонентов 371, 1037
- список макетов 253
- список минтермов 253
- список событий 353
- список соединений 318, 1037
- список переходов 689
- список чувствительности 345
- справочные данные 126
- сравнение чисел 58
- средняя чувствительность к перепадам 98
- среднее время между отказами 887, 1052
- средняя степень интеграции 35
- стабилитрон 182
- стандарт IEEE на графическое условное изображение логических функций 374
- стандартные элементы 39
- старший бит 51
- старший разряд 50
- статическая рассеиваемая мощность 153
- статическое ОЗУ 983
- стираемое программируемое логическое устройство 409
- стираемые программируемые ИЗУ 971
- сток 115
- стробирование тактового сигнала 879
- стробирующее устройство 155
- строб адреса столбца CAS_L 999
- строб адреса строки RAS_L 999
- строка 299, 328
- строка, не являющаяся кодовым словом 85
- структурная модель 337, 1038

структурное описание 337
 структурное описание логического устройства 371
 сумма произведений 252
 сумматор 500
 сумматор с ускоренным переносом 506
 сумматор со сквозным переносом 501
 сумматоры с групповым сквозным переносом 509
 суффикс $_L$ 376
 суффикс атрибута $_OE$ 457
 существенная простая импликанта 274, 282
 второго порядка 276
 существенный источник опасности 728
 схема классического образца 704
 схема быстрого сдвига 539
 схема И 110
 схема И-ИЛИ 263
 схема И-ИЛИ-НЕ 123
 схема И-НЕ 111
 схема И-НЕ-И-НЕ 263
 схема ИЛИ 110
 схема ИЛИ-И 264
 схема ИЛИ-И-НЕ 124
 схема ИЛИ-НЕ 111
 схема ИЛИ-НЕ-ИЛИ-НЕ 264
 схема НЕ 110
 схема проверки на четность 481
 схема проверки на четность 481
 схема расположения выводов 34
 схема с общим эмиттером 186
 схема с тремя состояниями 449
 схема сдвига уровня 214
 схема сложения чисел в прямом коде со знаком 59
 схема умножения посредством сдвига и сложения 870
 схема ускоренного переноса 513
 схемные редакторы 1035
 счетчик 804
 счетчик Джонсона 842
 счетчик Мебиуса 842
 счетчик с модулем счета m 804
 счетчик на регистре сдвига 838
 счетчик с исследовательным переносом 805

Т

таблица возбуждения 658
 таблица временных параметров 391
 таблица истинности 110, 250, 304
 таблица использования JK-триггера 674
 таблица переходов 649, 706
 таблица «переход/возбуждение» 670
 таблица «переход/выход» 653
 таблица потока 713
 таблица простых импликант 289
 таблица состояний 110, 649, 706
 таблица «состояние/выход» 647, 650
 тайм-слот 835

такт 644
 такт системных часов 620
 тактируемый оператор ирисаивания 732
 тактируемый оператор таблицы истинности 733
 тактируемый синхронный конечный автомат 621, 644
 тактовый сигнал 620
 текстовый редактор VHDL 317
 температурные проблемы 1040
 теорема Де Моргана 245
 теорема объединения 244
 теорема согласованности 244
 теорема поглощения 244
 терм
 p -терм перехода 691
 s -терм перехода 693
 терм-произведение 252
 терм-сумма 252
 тест «годен — не годен» 1041
 тестирование 1041
 тестирующее приспособление 1043
 тестовый вход 637
 тестовый выход 637
 технические требования 370
 технические условия 29
 технология поверхностного монтажа 40
 тип 323
 тип ИС 386
 тип компонента 1035
 тип массива без ограничений 329
 тип соединения 1036
 тип элементов 328
 типичная задержка 393
 типы массивов 327
 ток втекающий 194
 ток вытекающий 195
 ток утечки 116, 181
 толстые линии 40
 точки, отделяющие целую часть числа от дробной 50
 точное дополнение 60
 транзистор
 n - p - n -транзистор 185
 n МОП-транзистор 115
 p - n - p -транзистор 186
 p МОП-транзистор 115
 закрыт 186
 открыт 187
 с фиксирующими диодами Шоттки 190
 Шоттки 190
 транзисторно-транзисторная логика 113
 триггеры 28, 625
 JK-триггер, переключающийся по фронту 641
 D-триггер, переключающийся по отрицательному фронту 634
 D-триггер, переключающийся по положительному фронту 632

T-триггер 642

T-триггер с входом разрешения 643

двухтактный JK-триггер 639

двухтактный SR-триггер 638

со срабатыванием по импульсу 639

Шмитта на входе 156

ТТЛ 113

ТТЛ-нагрузка 175

ТТЛ-схемы с транзисторами Шоттки 204

У

узел 650

указатель динамического входа 633

улучшенные маломощные ТТЛ-схемы с транзисторами Шоттки 204

улучшенные ТТЛ-схемы с транзисторами Шоттки 204

уменьшаемое 58

умножение чисел в дополнительном коде 72

умножение чисел со знаком 72

универсальные матричные логические устройства GAL 405

унакованные двоично-десятичные числа 75

управляемость 1043

управляющее устройство 867

уравнение возбуждения 648, 706

уравнение выхода 650, 707

уравнение переходов 649

уровни LVCMOS 210

уровни LVTTL 210

усилитель 186

усилитель считывания 998

ускоренный перенос 505

ускоренный групповой перенос 512

установка 626

установка в единичное состояние 634

устойчивое равновесие 623

устойчивое состояние в целом 706

устройство обработки данных 867

Ф

фазорасширитель 192

ФАПЧ 98

ферроэлектрическое ОЗУ 984

физическая модель 1039

фиксирующие диоды 192, 212

филирующие конденсаторы 143

флажки 384

флажок связи 1036

флэш-память 972

формальные параметры 329

фронт тактового сигнала 744

функциональная верификация 318

функциональная таблица 419

функциональные блоки 1006

функциональный источник опасности 816

функция 329

функция разрешения 461

Х

характеристики переключения 206

характеристическое уравнение 646

ход выполнения проекта 315

Ц

центральная станция 832

цепочка сканирования 637

цикл for 349

цикл while 350

цикл записи 987, 1002

цикл моделирования 353

цикл постраничного чтения 1003

цикл постраничной записи 1003

цикл регенерации 998

цикл регенерации по стробу адреса столбца, предшествующий циклу регенерации по стробу адреса строки 1002

цикл «чтение-модификация-запись» 1002

цикл чтения 1000

цикл регенерации только по стробу адреса строки 1000

циклические коды 92

циклический перенос 69

циклический сдвиг 858

цифровая логика 108

цифровая схема фазовой автоподстройки частоты 98

цифровая телефония 832

цифровой аттенуатор 979

цифровые схемы и системы 23

цоколевка 34, 388

Ч

частичное произведение 71

частота переключений 153

частота тактового сигнала 620

числа без знака 68

числа со знаком 68

число входов 397

число выходов 397

число термов-произведений 397

чистые VHDL-функции 613

Ш

шаг по индукции 244

шестнадцатеричная система счисления 51

шина 372

шина питания 131

шина с нулевым временем смены режима 995

шина с тремя состояниями 158

шинный приемопередатчик 453

шинный фиксатор уровня 779

аппаратный компонент, разрешенный к применению 1036

шифратор 440

«штрих» 240

Э

- эквивалентная схема 133
- эквивалентная схема нагрузки 146
- эквивалентные состояния 664
- электрически стираемые программируемые
ПЗУ 972
- электрически стираемые программируемые
логические устройства 411
- электрические характеристики 206
- электрический ток 135
- электростатический разряд 143
- элемент с фиксированной матрицей ИЛИ 403
- элементарный программируемый
переключатель 1024
- элементарный сдвиг по времени 353
- эмиттер 185
- эмиттерно-связанная логика 215
- энергозависимая память 983
- энергонезависимая память 961, 983
- ЭСЛ 215
- ЭСЛ-семейство 10К 219
- ЭСЛ-семейство 100К 222
- ЭСЛ-семейство 10Н 219
- ЭСЛ-схема с положительным напряжением
питания 222

Я

- язык описания конечных автоматов 733
- язык описания схем 26
- языки межрегистровых пересылок 1034
- ячейка статического ОЗУ 986

Вышли в издательстве "Постмаркет" :

- ✓ - М.Х. Джонс
"Электроника - практический курс." 1999; объем 528 стр.; пер. №7
- ✓ - К.Б. Клаасен "Основы измерений. Электронные методы и приборы в измерительной технике" 2000; объем 352 стр.; пер. №7
- ✓ - И.М. Готтлиб "Источники питания. Инверторы, конверторы, линейные и импульсные стабилизаторы" 2002; объем 544 стр.; Пер.№7
- ✓ - Т.Е. Фабер "Гидроаэродинамика" 2001; объем 560 стр.; Пер.№7
- ✓ - Под редакцией А.Боуместера, А.Экерта и А.Целлингера
"Физика квантовой информации." 2002; объем 376 стр.; Пер.№7.

Заявки на книги присылайте по адресу
107140 Москва, Краснопрудный пер., д 7
Телефоны
рассылка по заявкам (095) 2076091
оптовая торговля (095) 2076085
факс (095) 264-43-47
e-mail **postmarket @ mtu-net.ru**

В заявке обязательно указывайте
свой почтовый адрес!

С полными оглавлениями всех уже вышедших,
а также готовящихся к печати наших книг
Вы можете ознакомиться на сайте
[http:// www.top-kniga.ru/](http://www.top-kniga.ru/)
(в разделе "наши поставщики")

Оформление переплета - С.В. Алексеев
Компьютерная верстка - М.В. Алексеева
Ответственный за выпуск - А.С. Товбис

лицензия ЛР №090215

Подписано в печать 30.08.02
Формат 70Х100/16 Печать офсетная Гарнитура Таймс
Печ. л. 33 Тираж 2000 экз. Заказ 2334
Оригинал-макет подготовлен ЗАО "Предприятие Постмаркет"

Отпечатано в ФГУП издательство "Известия" Управление делами Президента РФ
101999, ГСП-9, г. Москва, К-6, Пушкинская пл., д. 5

ISBN 5-901095-12-X



9 785901 095126